

Yinshi

Generated from source

Contents

1	Introduction	6
2	Request and Runtime Flow	7
3	Backend Application Boundary	8
3.1	main.py	8
3.2	config.py	25
3.3	models.py	38
3.4	model_catalog.py	55
3.5	exceptions.py	61
3.6	rate_limit.py	63
3.7	utils/paths.py	64
4	Database Layer and Tenancy	65
4.1	db.py	65
4.2	tenant.py	101
5	Authentication and Session Resolution	116
5.1	backup.py	116
5.2	auth.py	134
5.3	live_auth_sessions.py	142
5.4	api/auth_routes.py	145
5.5	api/deps.py	171
6	Repository and Workspace Lifecycle	173
6.1	api/repos.py	174
6.2	api/workspaces.py	185
6.3	services/git.py	188
6.4	services/git_runtime.py	200
6.5	services/workspace.py	202
6.6	services/workspace_runtime_paths.py	212
7	Agent Streaming and Run Coordination	214
7.1	api/stream.py	214

7.2	services/sidecar.py	242
7.3	services/run_coordinator.py	256
8	Container Isolation	257
8.1	services/container.py	257
8.2	services/sidecar_runtime.py	281
9	Workspace Files and Terminals	296
9.1	api/workspace_files.py	296
9.2	services/workspace_files.py	301
9.3	api/terminals.py	313
10	Encryption & Key Management	323
10.1	services/crypto.py	323
10.2	services/control_encryption.py	328
10.3	services/keys.py	330
11	Provider Connections and Pi Configuration	336
11.1	api/settings.py	336
11.2	services/provider_connections.py	345
11.3	services/pi_config.py	354
11.4	services/user_settings.py	378
11.5	services/pi_releases.py	382
11.6	api/catalog.py	387
12	GitHub App Integration	390
12.1	api/github.py	390
12.2	services/github_app.py	391
13	Accounts, Runners, and Observability	402
13.1	services/accounts.py	402
13.2	api/runners.py	410
13.3	services/runners.py	416
13.4	runner_agent.py	438
13.5	api/datadog_proxy.py	460
14	Desktop Runtime Bridge	464
14.1	yinshi/api/desktop_devices.py	464
14.2	yinshi/desktop_bootstrap.py	465
14.3	yinshi/desktop_runtime.py	469
14.4	yinshi/services/desktop_auth.py	473
14.5	yinshi/services/desktop_devices.py	481
14.6	yinshi/services/desktop_terminal.py	484
14.7	yinshi/services/desktop_tokens.py	486
15	Managed Runtime Control Plane	490
15.1	yinshi/api/managed_runtime.py	490

15.2	yinshi/managed_sprite_cleanup.py	497
15.3	yinshi/services/managed_artifacts.py	499
15.4	yinshi/services/managed_guest_installer.py	502
15.5	yinshi/services/managed_hosted_runtime.py	508
15.6	yinshi/services/managed_operation_failures.py	509
15.7	yinshi/services/managed_runners.py	510
15.8	yinshi/services/managed_runtime_manager.py	527
15.9	yinshi/services/managed_sprite_reconciliation.py	545
15.10	yinshi/services/managed_sprite_registry.py	550
15.11	yinshi/services/sprite_task_lease.py	553
15.12	yinshi/services/sprites.py	556
16	Managed Backup and Recovery	584
16.1	yinshi/api/managed_recovery_drills.py	584
16.2	yinshi/managed_backup_guest.py	585
16.3	yinshi/managed_operations_check.py	604
16.4	yinshi/managed_recovery_drill_controller.py	605
16.5	yinshi/managed_recovery_live.py	608
16.6	yinshi/managed_recovery_staging.py	609
16.7	yinshi/managed_source_loss_recovery.py	616
16.8	yinshi/services/managed_backup_crypto.py	620
16.9	yinshi/services/managed_backup_manager.py	625
16.10	yinshi/services/managed_backup_store.py	654
16.11	yinshi/services/managed_backups.py	669
16.12	yinshi/services/managed_operational_failures.py	693
16.13	yinshi/services/managed_operational_status.py	695
17	Encrypted Runner Relay	699
17.1	yinshi/api/prompt_runs.py	700
17.2	yinshi/api/runner_relay.py	704
17.3	yinshi/api/runtime_uploads.py	707
17.4	yinshi/api/sessions.py	711
17.5	yinshi/api/terminal_channels.py	715
17.6	yinshi/runner_worker.py	722
17.7	yinshi/services/encrypted_uploads.py	730
17.8	yinshi/services/prompt_journal.py	735
17.9	yinshi/services/relay_process_lock.py	747
17.10	yinshi/services/repository_lifecycle.py	749
17.11	yinshi/services/runner_agent_relay.py	757
17.12	yinshi/services/runner_capabilities.py	763
17.13	yinshi/services/runner_data_key.py	771
17.14	yinshi/services/runner_noise.py	776
17.15	yinshi/services/runner_noise_session.py	782
17.16	yinshi/services/runner_relay.py	788
17.17	yinshi/services/runner_rpc.py	801
17.18	yinshi/services/runner_rpc_transport.py	814

17.19yinshi/services/terminal_journal.py	814
17.20yinshi/worker_auth.py	826
17.21yinshi/worker_runtime.py	829
18 Desktop Application	833
18.1 accountLease.ts	833
18.2 accountSession.ts	837
18.3 appController.ts	840
18.4 authCallbackListener.ts	843
18.5 automaticUpdates.ts	847
18.6 credentialStore.ts	849
18.7 desktopApi.ts	855
18.8 diskEncryption.ts	856
18.9 helperBootstrap.ts	857
18.10helperProtocol.ts	858
18.11helperSupervisor.ts	860
18.12hostedAccessSession.ts	863
18.13hostedApiGateway.ts	865
18.14hostedAuth.ts	874
18.15localRepositoryImport.ts	882
18.16localRuntime.ts	885
18.17main.ts	888
18.18preload.ts	903
18.19runtimeLaunchConfig.ts	904
18.20runtimeSecrets.ts	908
18.21secureJsonStore.ts	910
18.22security.ts	914
18.23shellPolicy.ts	915
18.24sidecarSupervisor.ts	917
18.25signInRenderer.ts	921
19 Frontend Shell and Routing	923
19.1 main.tsx	923
19.2 rum.ts	924
19.3 App.tsx	925
19.4 index.css	926
19.5 components/Layout.tsx	931
19.6 components/RequireAuth.tsx	932
19.7 components/ChunkErrorBoundary.tsx	933
20 Frontend Pages	936
20.1 pages/Landing.tsx	936
20.2 pages/Login.tsx	942
20.3 pages/EmptyState.tsx	944
20.4 pages/Session.tsx	944
20.5 pages/Settings.tsx	959

21 Frontend API and State Hooks	983
21.1 api/client.ts	983
21.2 api/piCommandsCache.ts	996
21.3 hooks/useAuth.tsx	998
21.4 hooks/useTheme.tsx	1000
21.5 hooks/useCatalog.ts	1001
21.6 hooks/usePiCommands.ts	1002
21.7 hooks/usePiConfig.ts	1004
21.8 hooks/usePiReleaseNotes.ts	1010
21.9 hooks/useAgentStream.ts	1011
21.10 hooks/useMobileDrawer.ts	1017
21.11 models/sessionModels.ts	1017
21.12 utils/repo.ts	1021
21.13 utils/turnEvents.ts	1021
22 Frontend Chat and Workspace UI	1028
22.1 components/ChatView.tsx	1028
22.2 components/AssistantTurn.tsx	1037
22.3 components/MessageBubble.tsx	1041
22.4 components/ThinkingBlock.tsx	1043
22.5 components/ToolCallBlock.tsx	1044
22.6 components/StreamingDots.tsx	1053
22.7 components/SlashCommandMenu.tsx	1054
22.8 components/Sidebar.tsx	1056
22.9 components/WorkspaceInspector.tsx	1083
23 Frontend Settings Components	1100
23.1 components/CloudRunnerSection.tsx	1100
23.2 components/PiConfigSection.tsx	1118
23.3 components/PiReleaseNotesSection.tsx	1124
24 Encrypted Browser Runtime	1128
24.1 crypto/noiseIrk.ts	1129
24.2 runner/encryptedRunnerClient.ts	1135
24.3 runner/repositories.ts	1150
24.4 runner/runnerRpcTransport.ts	1152
24.5 runtime/encryptedUpload.ts	1155
24.6 runtime/promptStream.ts	1160
24.7 runtime/resolveRuntime.ts	1166
24.8 runtime/runtimeRef.ts	1172
24.9 runtime/runtimeTransport.ts	1175
24.10 runtime/terminalChannel.ts	1186
24.11 runtime/useRuntimeResource.ts	1191
25 Sidecar Runtime	1193
25.1 constants.js	1193

25.2	index.js	1193
25.3	sidecar.js	1194
25.4	git_auth.js	1244
25.5	git_askpass_client.js	1255

1 Introduction

Yinshi turns a Git repository into a browser-accessible workspace for coding agents. A user signs in, binds a GitHub installation or chooses an approved local repository, creates an isolated worktree, and talks to the `pi` coding agent. The same workbench displays streamed messages, changed files, and an interactive terminal.

Hosted and desktop execution share one product contract. The FastAPI control plane owns identity, tenant records, encrypted configuration, managed-runtime state, and policy. Hosted work runs inside one Fly Sprite per user. Desktop work runs through a supervised local helper. The React frontend selects the runtime, then uses one transport contract for repository, prompt, file, and terminal operations. The Node.js sidecar owns direct `pi` SDK integration inside the selected runtime.

graph TD

```

Browser[Browser workbench] --> Control[FastAPI control plane]
Browser -->|Noise-encrypted RPC| Relay[runner relay]
Control --> ControlDB[(encrypted control SQLite)]
Control --> Provider[Fly Sprites API]
Provider --> Sprite[per-user Sprite]
Sprite --> Runner[Python runner worker]
Runner --> Sidecar[Node.js sidecar]
Sidecar --> Pi[pi coding agent SDK]
Browser --> Desktop[Electron desktop gateway]
Desktop --> Helper[supervised local helper]
Helper --> Sidecar

```

The browser never receives host shell access, provider credentials, or GitHub App key material. Hosted RPC payloads are encrypted between the browser and runner. Control-plane routes enforce tenant ownership before they issue runtime capabilities. Managed cleanup requires durable ownership records instead of provider-name inference.

This literate program captures production backend, frontend, desktop, and sidecar source files byte-for-byte. Tests, lock files, generated build artifacts, and deployment assets remain outside the tangle. Generated publications retain the established Pandoc typography and color rules.

2 Request and Runtime Flow

A browser request first resolves identity and runtime mode. Control-plane operations stay on FastAPI. Workspace operations use the selected runtime transport. Hosted mode sends encrypted RPC envelopes through the relay. Desktop mode sends the same logical request through the local gateway.

sequenceDiagram

```
participant User
participant Browser
participant Control as FastAPI control plane
participant Relay as runner relay
participant Runner as Sprite runner
participant Sidecar
participant Pi as pi SDK

User->>Browser: submit prompt
Browser->>Control: resolve tenant and runtime
Browser->>Relay: encrypted prompt RPC
Relay->>Runner: opaque relay frame
Runner->>Sidecar: run prompt in worktree
Sidecar->>Pi: stream agent events
Pi-->>Sidecar: assistant and tool events
Sidecar-->>Runner: normalized messages
Runner-->>Browser: encrypted event stream
```

Repository state follows one ownership chain. GitHub authorization remains in the control plane. A short-lived repository credential reaches only the authorized Git process. Each workspace uses a separate worktree. Runtime journals make prompt and terminal reconnection deterministic.

graph LR

```
Install[GitHub installation] --> Credential[scoped Git credential]
Credential --> Checkout[tenant repository]
Checkout --> Worktree[isolated worktree]
Worktree --> Prompt[agent prompt]
Worktree --> Files[file operations]
Worktree --> Terminal[terminal channel]
Prompt --> Journal[prompt journal]
Terminal --> TJournal[terminal journal]
```

Managed persistence runs independently from request execution. Backup work stops runtime services in a fixed order and creates an encrypted archive. It verifies the remote object version before publication and restart. Recovery uses the archived data-protection key and rejects ambiguous ownership or incomplete inventory.

The document follows the control path first. It then covers persistence, runtime

selection, encrypted relay transport, desktop supervision, browser behavior, and sidecar execution.

3 Backend Application Boundary

The backend boundary starts with environment-derived settings, FastAPI startup, shared schemas, and small primitives used by every route. These files make request handling predictable before tenant-specific work begins.

3.1 main.py

The application entry point wires middleware, lifespan startup, route registration, and health checks before Uvicorn serves requests. The root chunk below tangles back to `backend/src/yinshi/main.py`.

```
"""FastAPI application entry point."""

import asyncio
import logging
from collections.abc import AsyncIterator, Awaitable, Callable
from contextlib import asynccontextmanager
from datetime import UTC, datetime, timedelta
from pathlib import Path
from typing import Any, Literal, cast

import httpx
from fastapi import APIRouter, FastAPI, Request, Response
from fastapi.middleware.cors import CORSMiddleware
from slowapi import _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded
from starlette.exceptions import HTTPException as StarletteHTTPException
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.middleware.sessions import SessionMiddleware
from starlette.middleware.trustedhost import TrustedHostMiddleware
from starlette.responses import JSONResponse, RedirectResponse
from starlette.staticfiles import StaticFiles
from starlette.types import ASGIApp, Message, Receive, Scope, Send

from yinshi import db as database_module
from yinshi.api import (
    auth_routes,
    catalog,
    datadog_proxy,
    desktop_devices,
    github,
    managed_recovery_drills,
```



```

        managed_runtime,
        prompt_runs,
        repos,
        runner_relay,
        runners,
        runtime_uploads,
        sessions,
        settings,
        stream,
        terminal_channels,
        terminals,
        workspace_files,
        workspaces,
    )
    from yinshi.auth import AuthMiddleware, setup_oauth
    from yinshi.config import Settings, get_settings, https_required
    from yinshi.db import init_control_db, init_db
    from yinshi.managed_recovery_drill_controller import ManagedRecoveryDrillController
    from yinshi.managed_recovery_live import ManagedRecoveryLiveRunner
    from yinshi.managed_recovery_staging import StagingManagedRecoveryBoundary
    from yinshi.rate_limit import limiter
    from yinshi.services import managed_operational_failures
    from yinshi.services.encrypted_uploads import EncryptedUploadManager
    from yinshi.services.managed_backup_manager import ManagedBackupManager
    from yinshi.services.managed_backup_store import create_managed_backup_store
    from yinshi.services.managed_backups import (
        complete_managed_backup_restore,
        start_managed_backup_deletion,
        start_managed_backup_restore,
    )
    from yinshi.services.managed_guest_installer import (
        ManagedGuestInstaller as ConcreteManagedGuestInstaller,
    )
    from yinshi.services.managed_hosted_runtime import HostedManagedRuntime
    from yinshi.services.managed_operational_status import collect_managed_operational_status
    from yinshi.services.managed_runtime_manager import ManagedRuntimeManager
    from yinshi.services.managed_sprite_reconciliation import ManagedSpriteReconciler
    from yinshi.services.prompt_journal import PromptJournal
    from yinshi.services.relay_process_lock import RelayProcessLock
    from yinshi.services.runner_relay import runner_relay_broker
    from yinshi.services.sprites import SpritesClient
    from yinshi.services.terminal_journal import TerminalJournal
    from yinshi.tenant import TenantContext
    from yinshi.worker_auth import (
        WorkerPrincipal,
        WorkerPrincipalMiddleware,

```

```

        prepare_worker_principal_storage,
    )

    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    )
    logger = logging.getLogger(__name__)

    AppMode = Literal["desktop", "hosted", "worker"]

    _MAX_BOOTSTRAP_SCRIPT_BYTES = 1024 * 1024
    _MANAGED_REGION = "global"

    class RequestBodyLimitMiddleware:
        """Reject declared or streamed HTTP bodies above a process-wide limit."""

        def __init__(self, app: ASGIApp, *, body_bytes_max: int) -> None:
            if body_bytes_max <= 0:
                raise ValueError("body_bytes_max must be positive")
            self._app = app
            self._body_bytes_max = body_bytes_max

        async def __call__(self, scope: Scope, receive: Receive, send: Send) -> None:
            if scope["type"] != "http":
                await self._app(scope, receive, send)
                return

            headers = {key.lower(): value for key, value in scope.get("headers", [])}
            content_length = headers.get(b"content-length")
            if content_length is not None:
                try:
                    declared_bytes = int(content_length)
                except ValueError:
                    await Response(status_code=400)(scope, receive, send)
                    return
                if declared_bytes > self._body_bytes_max:
                    await Response(status_code=413)(scope, receive, send)
                    return

            received_bytes = 0

            async def limited_receive() -> Message:
                nonlocal received_bytes
                message = await receive()

```

```

        if message["type"] == "http.request":
            received_bytes += len(message.get("body", b""))
            if received_bytes > self._body_bytes_max:
                raise _RequestBodyTooLarge
            return message

    try:
        await self._app(scope, limited_receive, send)
    except _RequestBodyTooLarge:
        await Response(status_code=413)(scope, receive, send)

class _RequestBodyTooLarge(Exception):
    """Signal an ASGI receive stream that crossed its configured body limit."""

class DesktopTenantMiddleware:
    """Inject one profile-local identity behind the desktop bootstrap boundary."""

    def __init__(self, application: ASGIApp, *, tenant: TenantContext) -> None:
        if not callable(application):
            raise TypeError("desktop tenant application must be callable")
        if not isinstance(tenant, TenantContext) or not tenant.user_id:
            raise ValueError("desktop tenant must have an identity")
        self._application = application
        self._tenant = tenant

    async def __call__(self, scope: Scope, receive: Receive, send: Send) -> None:
        if scope["type"] in {"http", "websocket"}:
            state = scope.setdefault("state", {})
            state["tenant"] = self._tenant
            state["user_email"] = self._tenant.email
            await self._application(scope, receive, send)

_API_CONTENT_SECURITY_POLICY = "default-src 'none'; frame-ancestors 'none'; base-uri 'none'"
_DESKTOP_CONTENT_SECURITY_POLICY = (
    "default-src 'self'; "
    "script-src 'self' 'wasm-unsafe-eval'; "
    "style-src 'self' 'unsafe-inline'; "
    "img-src 'self' data:; "
    "font-src 'self' data:; "
    "connect-src 'self' https://yinshi.io wss://yinshi.io; "
    "object-src 'none'; "
    "base-uri 'none'; "
    "frame-ancestors 'none'; "

```

```

        "form-action 'self'"
    )

class SecurityHeadersMiddleware(BaseHTTPMiddleware):
    """Attach browser hardening headers to every HTTP response."""

    def __init__(
        self,
        app: ASGIApp,
        *,
        content_security_policy: str = _API_CONTENT_SECURITY_POLICY,
    ) -> None:
        super().__init__(app)
        if not isinstance(content_security_policy, str):
            raise TypeError("content_security_policy must be a string")
        if not content_security_policy.strip():
            raise ValueError("content_security_policy must not be empty")
        self._content_security_policy = content_security_policy

    async def dispatch(
        self,
        request: Request,
        call_next: Callable[[Request], Awaitable[Response]],
    ) -> Response:
        response = await call_next(request)
        response.headers.setdefault(
            "Content-Security-Policy",
            self._content_security_policy,
        )
        response.headers.setdefault("Referrer-Policy", "no-referrer")
        response.headers.setdefault("X-Content-Type-Options", "nosniff")
        response.headers.setdefault("X-Frame-Options", "DENY")
        response.headers.setdefault(
            "Permissions-Policy",
            "camera=(), geolocation=(), microphone=()",
        )
        return response

class DesktopStaticFiles(StaticFiles):
    """Serve packaged assets and limit SPA fallback to known UI routes."""

    async def get_response(self, path: str, scope: Scope) -> Response:
        try:
            return await super().get_response(path, scope)

```

```

except StarletteHTTPException as error:
    is_spa_route = path == "app" or path.startswith("app/")
    if error.status_code != 404 or not is_spa_route:
        raise
    return await super().get_response("index.html", scope)

class TransportSecurityMiddleware(BaseHTTPMiddleware):
    """Enforce HTTPS and HSTS when production transport hardening is enabled."""

    def __init__(
        self,
        app: ASGIApp,
        *,
        require_https: bool,
        hsts_enabled: bool,
        trusted_proxy_ips: set[str] | None = None,
    ) -> None:
        """Configure transport security behavior from validated settings."""
        super().__init__(app)
        if not isinstance(require_https, bool):
            raise TypeError("require_https must be a boolean")
        if not isinstance(hsts_enabled, bool):
            raise TypeError("hsts_enabled must be a boolean")
        if trusted_proxy_ips is not None and not isinstance(trusted_proxy_ips, set):
            raise TypeError("trusted_proxy_ips must be a set or None")
        self._require_https = require_https
        self._hsts_enabled = hsts_enabled
        self._trusted_proxy_ips = trusted_proxy_ips or set()

    async def dispatch(
        self,
        request: Request,
        call_next: Callable[[Request], Awaitable[Response]],
    ) -> Response:
        """Redirect plaintext requests and attach HSTS to HTTPS responses."""
        request_scheme = request.url.scheme.lower()
        client_host = request.client.host.lower() if request.client is not None else ""
        forwarded_proto = request.headers.get("x-forwarded-proto")
        if forwarded_proto and client_host in self._trusted_proxy_ips:
            request_scheme = forwarded_proto.split(", ", maxsplit=1)[0].strip().lower()
        if self._require_https:
            if request_scheme != "https":
                https_url = request.url.replace(scheme="https")
                return RedirectResponse(str(https_url), status_code=307)
        response = await call_next(request)

```

```

        if self._hsts_enabled:
            if request_scheme == "https":
                response.headers.setdefault(
                    "Strict-Transport-Security",
                    "max-age=31536000; includeSubDomains",
                )
        return response

def _create_provider_http_client(app_settings: Settings) -> httpx.AsyncClient:
    """Create the dedicated Fly provider HTTP client."""
    return httpx.AsyncClient(
        base_url=app_settings.sprites_api_url,
        follow_redirects=False,
    )

def _create_artifact_http_client() -> httpx.AsyncClient:
    """Create the dedicated immutable artifact HTTP client."""
    return httpx.AsyncClient(follow_redirects=False)

def _read_bounded_bootstrap_script(path: Path) -> bytes:
    """Read one validated bootstrap script without unbounded allocation."""
    with path.open("rb") as script_file:
        bootstrap_script = script_file.read(_MAX_BOOTSTRAP_SCRIPT_BYTES + 1)
    if len(bootstrap_script) > _MAX_BOOTSTRAP_SCRIPT_BYTES:
        raise RuntimeError("Managed bootstrap script exceeds the 1 MiB limit")
    return bootstrap_script

async def _initialize_managed_runtime(
    app_settings: Settings,
) -> HostedManagedRuntime:
    """Build managed Fly services and close resources after partial failures."""
    bootstrap_script = await asyncio.to_thread(
        _read_bounded_bootstrap_script,
        Path(app_settings.sprites_bootstrap_script_path),
    )
    provider_http_client: httpx.AsyncClient | None = None
    artifact_http_client: httpx.AsyncClient | None = None
    manager: ManagedRuntimeManager | None = None
    try:
        provider_http_client = _create_provider_http_client(app_settings)
        artifact_http_client = _create_artifact_http_client()
        api_token = app_settings.sprites_api_token

```

```

name_key = app_settings.sprites_name_key
if api_token is None or name_key is None:
    raise RuntimeError("Validated managed runtime secrets are unavailable")
provider = SpritesClient(
    api_token=api_token.get_secret_value(),
    http_client=provider_http_client,
)
guest_installer = ConcreteManagedGuestInstaller(
    client=provider,
    bootstrap_script=bootstrap_script,
    bootstrap_timeout_seconds=app_settings.sprites_operation_stale_seconds,
    storage_encryption_confirmed=app_settings.sprites_storage_encryption_confirmed,
    clock=asyncio.get_running_loop().time,
    sleep=asyncio.sleep,
)
manager = ManagedRuntimeManager(
    provider=provider,
    guest_installer=guest_installer,
    http_client=artifact_http_client,
    name_prefix=app_settings.sprites_name_prefix,
    name_key=name_key.get_secret_value(),
    artifact_url=app_settings.sprites_artifact_url,
    artifact_sha256=app_settings.sprites_artifact_sha256,
    artifact_version=app_settings.sprites_artifact_sha256,
    allowed_domains=tuple(app_settings.sprites_allowed_domains.split(",")),
    region=_MANAGED_REGION,
    control_url=app_settings.sprites_public_control_url,
    readiness_timeout_seconds=app_settings.sprites_wake_timeout_seconds,
    is_runner_connected=runner_relay_broker.is_runner_connected,
)
return HostedManagedRuntime(
    runtime_manager=manager,
    backup_provider=provider,
    inventory_provider=provider,
    provider_http_client=provider_http_client,
)
except BaseException:
    try:
        if manager is not None:
            await manager.aclose()
        elif artifact_http_client is not None:
            await artifact_http_client.aclose()
    finally:
        if provider_http_client is not None:
            await provider_http_client.aclose()
    raise

```

```

async def _attempt_shutdown_cleanup(
    cleanup: Callable[[], Awaitable[None]],
    first_error: BaseException | None,
) -> BaseException | None:
    """Run one shutdown cleanup and retain the first non-cancellation failure."""
    try:
        await cleanup()
    except asyncio.CancelledError:
        return first_error
    except BaseException as error:
        if first_error is None:
            return error
    return first_error

@asynccontextmanager
async def lifespan(app: FastAPI) -> AsyncIterator[None]:
    """Application startup and shutdown."""
    app_settings = get_settings()
    logger.info("Starting %s", app_settings.app_name)
    if not (app.state.mode == "hosted" and app_settings.managed_runtime_provider == "fly_springs"):
        init_db()
        init_control_db()
        setup_oauth()

    # Per-user container isolation
    reaper_task: asyncio.Task[None] | None = None
    if app_settings.container_enabled:
        from yinshi.services.container import ContainerManager

        mgr = ContainerManager(settings=app_settings)
        await mgr.initialize()
        app.state.container_manager = mgr
        reaper_task = asyncio.create_task(mgr.run_reaper())
        logger.info("Container isolation enabled (image=%s)", app_settings.container_image)
    else:
        app.state.container_manager = None

    relay_process_lock: RelayProcessLock | None = None
    managed_runtime_manager: ManagedRuntimeManager | None = None
    managed_backup_manager: ManagedBackupManager | None = None
    managed_sprite_reconciler_task: asyncio.Task[None] | None = None
    provider_http_client: httpx.AsyncClient | None = None
    try:

```



```

if app.state.mode == "hosted":
    relay_process_lock = RelayProcessLock(
        Path(app_settings.control_db_path).parent / "relay-process.lock"
    )
    relay_process_lock.acquire()
if app.state.mode == "hosted" and app_settings.managed_runtime_provider == "fly_sprite":
    managed_backup_store = create_managed_backup_store(app_settings)
    preflight_alert = (
        managed_operational_failures.ManagedPersistentAlertClass.STORAGE_PREFLIGHT_FAILED
    )
    try:
        await managed_backup_store.preflight()
    except Exception:
        managed_operational_failures.record_managed_operational_failure(preflight_alert)
        logger.exception("managed_storage_preflight_failed")
        raise
    managed_operational_failures.clear_managed_operational_failure(preflight_alert)
    managed_runtime = await initialize_managed_runtime(app_settings)
    managed_runtime_manager = managed_runtime.runtime_manager
    provider_http_client = managed_runtime.provider_http_client
    await managed_runtime_manager.reconcile_startup()
    sprites_name_key = app_settings.sprites_name_key
    if sprites_name_key is None:
        raise RuntimeError("Managed Sprite naming is unavailable")
    restore_name_prefix = f"{app_settings.sprites_name_prefix}-restore"[:30].rstrip()
    managed_sprite_reconciler = ManagedSpriteReconciler(
        provider=managed_runtime.inventory_provider,
        name_prefix=app_settings.sprites_name_prefix,
        restore_name_prefix=restore_name_prefix,
        restore_name_key=sprites_name_key.get_secret_value(),
        grace=timedelta(seconds=app_settings.sprites_reconcile_grace_seconds),
    )
    await managed_sprite_reconciler.reconcile_classified(raise_on_failure=True)
    backup_encryption_key = app_settings.backup_encryption_key
    sprites_name_key = app_settings.sprites_name_key
    if backup_encryption_key is None or sprites_name_key is None:
        raise RuntimeError("Managed backup encryption is unavailable")
    managed_backup_manager = ManagedBackupManager(
        provider=managed_runtime.backup_provider,
        store=managed_backup_store,
        relay=runner_relay_broker,
        runtime_service=managed_runtime_manager,
        restore_name_prefix=restore_name_prefix,
        restore_name_key=sprites_name_key.get_secret_value(),
        complete_restore=complete_managed_backup_restore,
        start_restore=start_managed_backup_restore,

```

```

        start_deletion=start_managed_backup_deletion,
        wrapping_key=bytes.fromhex(backup_encryption_key.get_secret_value()),
        object_prefix=app_settings.managed_backup_prefix,
        retention_days=app_settings.managed_backup_retention_days,
        staging_root=Path(app_settings.control_db_path).parent / "managed-backup-sta
    )
    await managed_backup_manager.start()
    managed_sprite_reconciler_task = asyncio.create_task(
        managed_sprite_reconciler.run(
            interval_seconds=app_settings.sprites_reconcile_interval_seconds
        )
    )
    app.state.managed_backup_store = managed_backup_store
    app.state.managed_backup_manager = managed_backup_manager
    app.state.managed_runtime_manager = managed_runtime_manager
    if app_settings.managed_recovery_drill_enabled:
        recovery_boundary = StagingManagedRecoveryBoundary(
            runtime_manager=managed_runtime_manager,
            backup_manager=managed_backup_manager,
            provider=managed_runtime.backup_provider,
            store=managed_backup_store,
        )
        await recovery_boundary.recover_retained_cleanup()
        recovery_runner = ManagedRecoveryLiveRunner(boundary=recovery_boundary)
        app.state.managed_recovery_drill_controller = ManagedRecoveryDrillController(
            run_drill=recovery_runner.run
        )

    yield
finally:
    cleanup_error: BaseException | None = None
    recovery_controller = getattr(app.state, "managed_recovery_drill_controller", None)
    recovery_close = getattr(recovery_controller, "aclose", None)
    if callable(recovery_close):
        cleanup_error = await _attempt_shutdown_cleanup(
            recovery_close,
            cleanup_error,
        )
    prompt_journal = getattr(app.state, "prompt_journal", None)
    if isinstance(prompt_journal, PromptJournal):
        cleanup_error = await _attempt_shutdown_cleanup(
            prompt_journal.close,
            cleanup_error,
        )
    terminal_journal = getattr(app.state, "terminal_journal", None)
    if isinstance(terminal_journal, TerminalJournal):

```

```

        cleanup_error = await _attempt_shutdown_cleanup(
            terminal_journal.close_all,
            cleanup_error,
        )
    if reaper_task is not None:

        async def stop_reaper() -> None:
            reaper_task.cancel()
            await asyncio.gather(reaper_task, return_exceptions=True)

        cleanup_error = await _attempt_shutdown_cleanup(stop_reaper, cleanup_error)
    container_manager = getattr(app.state, "container_manager", None)
    if container_manager is not None:
        cleanup_error = await _attempt_shutdown_cleanup(
            container_manager.destroy_all,
            cleanup_error,
        )
    if managed_sprite_reconciler_task is not None:

        async def stop_managed_sprite_reconciler() -> None:
            managed_sprite_reconciler_task.cancel()
            await asyncio.gather(managed_sprite_reconciler_task, return_exceptions=True)

        cleanup_error = await _attempt_shutdown_cleanup(
            stop_managed_sprite_reconciler,
            cleanup_error,
        )
    if managed_backup_manager is not None:
        cleanup_error = await _attempt_shutdown_cleanup(
            managed_backup_manager.aclose,
            cleanup_error,
        )
    if managed_runtime_manager is not None:
        cleanup_error = await _attempt_shutdown_cleanup(
            managed_runtime_manager.aclose,
            cleanup_error,
        )
    if provider_http_client is not None:
        cleanup_error = await _attempt_shutdown_cleanup(
            provider_http_client.aclose,
            cleanup_error,
        )
    if relay_process_lock is not None:

        async def release_relay_process_lock() -> None:
            relay_process_lock.release()

```

```

        cleanup_error = await _attempt_shutdown_cleanup(
            release_relay_process_lock,
            cleanup_error,
        )
    if cleanup_error is not None:
        raise cleanup_error
logger.info("Shutdown complete")

def _configure_middleware(
    application: FastAPI,
    app_settings: Settings,
    *,
    mode: AppMode,
    worker_principal: WorkerPrincipal | None,
) -> None:
    """Attach the shared hosted middleware stack to one application instance."""
    if not isinstance(application, FastAPI):
        raise TypeError("application must be a FastAPI instance")
    if not isinstance(app_settings, Settings):
        raise TypeError("app_settings must be Settings")

    if mode == "worker":
        if worker_principal is None:
            raise ValueError("worker mode requires a WorkerPrincipal")
        application.add_middleware(
            RequestBodyLimitMiddleware,
            body_bytes_max=app_settings.request_body_max_bytes,
        )
        application.add_middleware(
            SecurityHeadersMiddleware,
            content_security_policy=_API_CONTENT_SECURITY_POLICY,
        )
        application.add_middleware(
            TrustedHostMiddleware,
            allowed_hosts=app_settings.trusted_host_list,
        )
        application.add_middleware(
            WorkerPrincipalMiddleware,
            principal=worker_principal,
        )
    return

cors_origins = [app_settings.frontend_url]
if app_settings.debug and "http://localhost:5173" not in cors_origins:

```

```

cors_origins.append("http://localhost:5173")

# Middleware order: last registered = outermost = runs first.
# CORS must remain outermost so preflight responses carry its headers.
require_https = https_required(app_settings)
application.add_middleware(
    SessionMiddleware,
    secret_key=app_settings.secret_key,
    https_only=require_https,
    same_site="lax",
)
application.add_middleware(AuthMiddleware)
if mode == "desktop":
    desktop_tenant = TenantContext(
        user_id="desktop-local",
        email="desktop-local@yinshi.invalid",
        data_dir=str(Path(app_settings.user_data_dir).resolve()),
        db_path=str(Path(app_settings.db_path).resolve()),
    )
    application.add_middleware(DesktopTenantMiddleware, tenant=desktop_tenant)
application.add_middleware(
    TransportSecurityMiddleware,
    require_https=require_https,
    hsts_enabled=app_settings.hsts_enabled and not app_settings.debug,
    trusted_proxy_ips=app_settings.trusted_proxy_ip_set,
)
application.add_middleware(
    RequestBodyLimitMiddleware,
    body_bytes_max=app_settings.request_body_max_bytes,
)
content_security_policy = _API_CONTENT_SECURITY_POLICY
if mode == "desktop":
    content_security_policy = _DESKTOP_CONTENT_SECURITY_POLICY
application.add_middleware(
    SecurityHeadersMiddleware,
    content_security_policy=content_security_policy,
)
application.add_middleware(
    TrustedHostMiddleware,
    allowed_hosts=app_settings.trusted_host_list,
)
application.add_middleware(
    CORSMiddleware,
    allow_origins=cors_origins,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PATCH", "PUT", "DELETE", "OPTIONS"],

```

```

        allow_headers=["Authorization", "Content-Type", "X-Requested-With"],
    )

def _include_routes(
    application: FastAPI,
    app_settings: Settings,
    *,
    mode: AppMode,
) -> None:
    """Attach only the route groups allowed for one application mode."""
    if not isinstance(application, FastAPI):
        raise TypeError("application must be a FastAPI instance")
    if not isinstance(app_settings, Settings):
        raise TypeError("app_settings must be Settings")
    if mode not in ("desktop", "hosted", "worker"):
        raise ValueError(f"Unsupported application mode: {mode}")

    shared_execution_routers: tuple[APIRouter, ...] = (
        auth_routes.provider_router,
        settings.router,
    )
    local_execution_routers: tuple[APIRouter, ...] = (
        catalog.router,
        repos.router,
        workspaces.router,
        workspace_files.router,
        terminals.router,
        terminal_channels.router,
        sessions.router,
        prompt_runs.router,
        runtime_uploads.router,
        stream.router,
    )
    execution_routers: tuple[APIRouter, ...] = (
        *shared_execution_routers,
        *local_execution_routers,
    )
    control_routers: tuple[APIRouter, ...] = (
        auth_routes.router,
        datadog_proxy.router,
        desktop_devices.router,
        github.router,
        runner_relay.router,
        runners.router,
        managed_runtime.router,

```

```

)
selected_routers = list(execution_routers)
if mode == "hosted":
    hosted_execution_routers = execution_routers
    if app_settings.managed_runtime_provider == "fly_sprites":
        hosted_execution_routers = ()
    selected_routers = [*control_routers, *hosted_execution_routers]
if (
    mode == "hosted"
    and app_settings.managed_runtime_provider == "fly_sprites"
    and getattr(app_settings, "managed_recovery_drill_enabled", False)
    and getattr(app_settings, "deployment_environment", "local") == "staging"
    and bool(getattr(app_settings, "managed_recovery_operator_token_hash", ""))
):
    selected_routers.append(managed_recovery_drills.router)
for router in selected_routers:
    application.include_router(router)

async def health() -> dict[str, str]:
    """Return process liveness without exposing runtime details."""
    return {"status": "ok"}

def managed_health() -> JSONResponse:
    """Return sanitized managed operational health for external monitoring."""
    try:
        settings = get_settings()
        with database_module.get_control_db() as database:
            status = collect_managed_operational_status(
                database,
                now=datetime.now(UTC),
                backup_stale_seconds=86_400,
                operation_stuck_seconds=settings.sprites_operation_stale_seconds,
            )
    except Exception:
        logger.error("managed_operational_health_failed")
        return JSONResponse(status_code=503, content={"status": "critical"})
    if status.alerts:
        return JSONResponse(status_code=503, content={"status": "critical"})
    return JSONResponse(status_code=200, content={"status": "ok"})

def create_app(
    *,
    mode: AppMode = "hosted",

```

```

desktop_asset_dir: str | Path | None = None,
worker_principal: WorkerPrincipal | None = None,
) -> FastAPI:
    """Build one independently configured Yinshi application mode."""
    if mode not in ("desktop", "hosted", "worker"):
        raise ValueError(f"Unsupported application mode: {mode}")
    if desktop_asset_dir is not None and mode != "desktop":
        raise ValueError("desktop_asset_dir is only valid in desktop mode")
    if mode == "worker" and worker_principal is None:
        raise ValueError("worker mode requires a WorkerPrincipal")
    if mode != "worker" and worker_principal is not None:
        raise ValueError("worker_principal is only valid in worker mode")
    if worker_principal is not None:
        prepare_worker_principal_storage(worker_principal)
    asset_directory: Path | None = None
    if desktop_asset_dir is not None:
        asset_directory = Path(desktop_asset_dir).resolve()
        if not asset_directory.is_dir() or not (asset_directory / "index.html").is_file():
            raise ValueError("desktop_asset_dir must contain index.html")

    app_settings = get_settings()
    if not isinstance(app_settings, Settings):
        raise TypeError("get_settings must return Settings")

    application = FastAPI(
        title="Yinshi",
        lifespan=lifespan,
        docs_url="/docs" if app_settings.debug else None,
        openapi_url="/openapi.json" if app_settings.debug else None,
    )
    application.state.limiter = limiter
    application.state.mode = mode
    application.state.encrypted_upload_manager = EncryptedUploadManager()
    application.state.managed_backup_manager = None
    application.state.managed_backup_store = None
    application.state.managed_runtime_manager = None
    application.state.managed_recovery_drill_controller = None
    application.state.managed_recovery_operator_token_hash = (
        app_settings.managed_recovery_operator_token_hash
    )
    application.state.sprites_public_launch_enabled = (
        mode == "hosted" and app_settings.sprites_public_launch_enabled
    )
    application.state.prompt_journal = PromptJournal()
    application.state.terminal_journal = TerminalJournal()
    application.add_exception_handler(

```



```

        RateLimitExceeded,
        cast(Any, _rate_limit_exceeded_handler),
    )
    _configure_middleware(
        application,
        app_settings,
        mode=mode,
        worker_principal=worker_principal,
    )
    _include_routes(application, app_settings, mode=mode)
    application.add_api_route("/health", health, methods=["GET"])
    application.add_api_route("/health/managed", managed_health, methods=["GET"])
    if asset_directory is not None:
        application.mount(
            "/",
            DesktopStaticFiles(directory=str(asset_directory), html=True),
            name="desktop-ui",
        )
    return application

app = create_app()
app_settings = get_settings()

if __name__ == "__main__":
    import uvicorn

    uvicorn.run(app, host=app_settings.host, port=app_settings.port)

```

3.2 config.py

Settings normalize environment variables into one explicit configuration object and fail fast when production safety requirements are missing. The root chunk below tangles back to backend/src/yinshi/config.py.

```

"""Application configuration via environment variables."""

from __future__ import annotations

import ipaddress
import json
import re
import secrets
from functools import lru_cache
from pathlib import Path

```

```

from urllib.parse import urlsplit

from pydantic import SecretStr
from pydantic_settings import BaseSettings

_SECURITY_MODE_VALUES = {"auto", "disabled", "enabled", "required"}
_MANAGED_RUNTIME_PROVIDER_VALUES = {"disabled", "fly_sprites"}
_MANAGED_BACKUP_PROVIDER_VALUES = {"aws_s3", "digitalocean_spaces"}
_DNS_LABEL_PATTERN = re.compile(r"[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\Z")
_SPRITE_PREFIX_PATTERN = re.compile(r"[a-z0-9](?:[a-z0-9-]{0,28}[a-z0-9])?\Z")
_SHA256_PATTERN = re.compile(r"[0-9a-f]{64}\Z")

def _generate_secret() -> str:
    return secrets.token_hex(32)

def _decode_hex_secret(value: str, name: str) -> bytes:
    """Decode a hex secret and reject values too weak for AES-256 use."""
    if not isinstance(value, str):
        raise TypeError(f"{name} must be a string")
    normalized_value = value.strip()
    if not normalized_value:
        return b""
    try:
        decoded_value = bytes.fromhex(normalized_value)
    except ValueError as exc:
        raise RuntimeError(f"{name} must be a valid hex string: {exc}") from exc
    if len(decoded_value) < 32:
        raise RuntimeError(f"{name} must be at least 32 bytes (64 hex characters)")
    return decoded_value

def _require_https_url(
    value: str,
    name: str,
    *,
    reject_routing_metadata: bool = False,
) -> None:
    """Require a safe absolute HTTPS URL."""
    parsed_url = urlsplit(value)
    try:
        port = parsed_url.port
    except ValueError as exc:
        raise RuntimeError(f"{name} must be a complete HTTPS URL") from exc
    invalid_routing_metadata = reject_routing_metadata and (

```

```

        "?" in value or "#" in value or port not in {None, 443}
    )
    if (
        parsed_url.scheme != "https"
        or not parsed_url.hostname
        or parsed_url.username is not None
        or parsed_url.password is not None
        or any(character.isspace() for character in value)
        or invalid_routing_metadata
    ):
        raise RuntimeError(f"{name} must be a complete HTTPS URL")

def _validate_allowed_domains(value: str) -> None:
    """Require unique lowercase public DNS names with optional leading wildcards."""
    if not value:
        return
    if any(character.isspace() for character in value):
        raise RuntimeError("SPRITES_ALLOWED_DOMAINS must not contain whitespace")
    entries = value.split(",")
    if len(entries) != len(set(entries)):
        raise RuntimeError("SPRITES_ALLOWED_DOMAINS must not contain duplicates")
    for entry in entries:
        if entry == "*":
            raise RuntimeError("SPRITES_ALLOWED_DOMAINS must not contain a global wildcard")
        domain = entry[2:] if entry.startswith("*.") else entry
        if "*" in domain:
            raise RuntimeError("SPRITES_ALLOWED_DOMAINS contains an invalid wildcard")
        labels = domain.split(".")
        if (
            len(domain) > 253
            or len(labels) < 2
            or any(_DNS_LABEL_PATTERN.fullmatch(label) is None for label in labels)
            or labels[-1].isdigit()
        ):
            raise RuntimeError("SPRITES_ALLOWED_DOMAINS must contain public DNS names")
        try:
            ipaddress.ip_address(domain)
        except ValueError:
            continue
        raise RuntimeError("SPRITES_ALLOWED_DOMAINS must not contain IP literals")

def _control_domain_is_allowed(control_hostname: str, allowed_domains: str) -> bool:
    """Return whether an allowed-domain entry covers the public control host."""
    for entry in allowed_domains.split(","):

```

```

        if entry == control_hostname:
            return True
        if entry.startswith("*.") and control_hostname.endswith(f".{entry[2:]}"):
            return True
    return False

def _normalize_mode(value: str, name: str) -> str:
    """Normalize a security mode value and reject ambiguous configuration."""
    if not isinstance(value, str):
        raise TypeError(f"{name} must be a string")
    normalized_value = value.strip().lower()
    if normalized_value not in _SECURITY_MODE_VALUES:
        allowed_values = ", ".join(sorted(_SECURITY_MODE_VALUES))
        raise RuntimeError(f"{name} must be one of: {allowed_values}")
    return normalized_value

class Settings(BaseSettings):
    """Application settings loaded from .env."""

    app_name: str = "Yinshi"
    debug: bool = False

    # Database (legacy single-DB mode)
    db_path: str = "yinshi.db"

    # Multi-tenant databases
    control_db_path: str = "/var/lib/yinshi/control.db"
    user_data_dir: str = "/var/lib/yinshi/users"

    # Legacy pepper for wrapping per-user DEKs (hex string, 32+ bytes).
    # New deployments should use KEY_ENCRYPTION_KEY so wrapped DEKs carry a key id.
    encryption_pepper: str = ""
    key_encryption_key: str = ""
    key_encryption_key_id: str = "local-v1"
    key_encryption_keys_previous: SecretStr | None = None

    # Middle-ground data protection controls. "auto" enables the control in
    # authenticated non-debug deployments while keeping local tests explicit.
    tenant_db_encryption: str = "auto"
    control_field_encryption: str = "auto"
    user_data_encryption: str = "disabled"

    # Google OAuth
    google_client_id: str = ""

```

```

google_client_secret: str = ""
google_redirect_uri: str = "http://localhost:8000/auth/callback/google"

# GitHub OAuth
github_client_id: str = ""
github_client_secret: str = ""
github_redirect_uri: str = "http://localhost:8000/auth/callback/github"
github_app_id: str = ""
github_app_private_key_path: str = ""
github_app_slug: str = ""
github_app_client_id: str = ""
github_app_client_secret: SecretStr | None = None
github_app_user_callback_url: str = ""

# Session secret for cookies -- generated randomly if not set
secret_key: str = ""

# Explicit flag to disable auth (empty google_client_id alone is not enough)
disable_auth: bool = False

# Sidecar
sidecar_socket_path: str = "/tmp/yinshi-sidecar.sock"

# Pi package update and release-note metadata
pi_package_name: str = "@earendil-works/pi-coding-agent"
pi_release_repository: str = "earendil-works/pi"
# Encrypted database backups
backup_dir: str = "/var/lib/yinshi/backups"
backup_encryption_key: SecretStr | None = None

# CORS
frontend_url: str = "http://localhost:5173"

# Server
host: str = "127.0.0.1"
port: int = 8000

# Production transport controls. "auto" requires HTTPS in authenticated
# non-debug deployments and trusts the edge proxy to provide TLS.
require_https: str = "auto"
hsts_enabled: bool = True
trusted_proxy_ips: str = "127.0.0.1,::1"
trusted_hosts: str = "localhost,127.0.0.1,[::1]"
request_body_max_bytes: int = 10 * 1024 * 1024

# Allowed base directory for local repo imports (empty = reject all local imports)

```

```

allowed_repo_base: str = ""

# Per-user container isolation
container_enabled: bool = True
container_image: str = "yinshi-sidecar:latest"
container_idle_timeout_s: int = 300
container_memory_limit: str = "256m"
container_cpu_quota: int = 50000
container_pids_limit: int = 256
container_max_count: int = 10
container_socket_base: str = "/var/run/yinshi"
container_mount_mode: str = "narrow"

# Browser terminal runtime controls
terminal_keepalive_s: int = 7200
terminal_scrollback_lines: int = 1000

# Managed Fly Sprites runtime
managed_runtime_provider: str = "disabled"
sprites_public_launch_enabled: bool = False
sprites_storage_encryption_confirmed: bool = False
sprites_api_token: SecretStr | None = None
sprites_api_url: str = "https://api.sprites.dev/v1"
sprites_name_prefix: str = "yinshi"
sprites_name_key: SecretStr | None = None
sprites_artifact_url: str = ""
sprites_artifact_sha256: str = ""
sprites_allowed_domains: str = ""
sprites_public_control_url: str = ""
sprites_bootstrap_script_path: str = ""
sprites_wake_timeout_seconds: int = 30
sprites_operation_stale_seconds: int = 1800
sprites_reconcile_interval_seconds: int = 900
sprites_reconcile_grace_seconds: int = 3600

# Staging-only destructive recovery control
deployment_environment: str = "local"
managed_recovery_drill_enabled: bool = False
managed_recovery_operator_token_hash: str = ""

# Independent encrypted managed guest backups
managed_backup_provider: str = "aws_s3"
managed_backup_bucket: str = ""
managed_backup_endpoint_url: str = ""
managed_backup_region: str = ""
managed_backup_access_key_id: SecretStr | None = None

```

```

managed_backup_secret_access_key: SecretStr | None = None
managed_backup_prefix: str = "yinshi-managed-v1"
managed_backup_part_bytes: int = 16 * 1024 * 1024
managed_backup_retention_days: int = 30

model_config = {
    "env_file": ".env",
    "env_file_encoding": "utf-8",
    "case_sensitive": False,
    # The shared dotenv also contains sidecar provider credentials. Ignoring
    # unknown names prevents Pydantic from echoing their values on startup.
    "extra": "ignore",
}

@property
def encryption_pepper_bytes(self) -> bytes:
    """Return the legacy encryption pepper as bytes."""
    return _decode_hex_secret(self.encryption_pepper, "ENCRYPTION_PEPPE")

@property
def key_encryption_key_bytes(self) -> bytes:
    """Return the current server-managed KEK bytes."""
    return _decode_hex_secret(self.key_encryption_key, "KEY_ENCRYPTION_KEY")

@property
def key_encryption_keyring_previous(self) -> dict[str, bytes]:
    """Return explicitly configured previous KEKs keyed by their stable IDs."""
    if self.key_encryption_keys_previous is None:
        return {}
    raw_keyring = self.key_encryption_keys_previous.get_secret_value().strip()
    if not raw_keyring:
        return {}
    try:
        payload = json.loads(raw_keyring)
    except json.JSONDecodeError as exc:
        raise RuntimeError("KEY_ENCRYPTION_KEYS_PREVIOUS must be a JSON object") from exc
    if not isinstance(payload, dict):
        raise RuntimeError("KEY_ENCRYPTION_KEYS_PREVIOUS must be a JSON object")
    keyring: dict[str, bytes] = {}
    for raw_key_id, raw_key in payload.items():
        if not isinstance(raw_key_id, str) or not raw_key_id.strip():
            raise RuntimeError("Previous key IDs must be non-empty strings")
        if not isinstance(raw_key, str):
            raise RuntimeError("Previous KEK values must be hexadecimal strings")
        key_id = raw_key_id.strip()
        if key_id == self.key_encryption_key_id.strip():

```

```

        raise RuntimeError("Previous KEK IDs must differ from KEY_ENCRYPTION_KEY_ID")
    keyring[key_id] = _decode_hex_secret(raw_key, "previous KEK")
    return keyring

@property
def active_key_encryption_key_bytes(self) -> bytes:
    """Return the strongest configured key source for envelope encryption."""
    key_encryption_key_bytes = self.key_encryption_key_bytes
    if key_encryption_key_bytes:
        return key_encryption_key_bytes
    return self.encryption_pepper_bytes

@property
def tenant_db_encryption_mode(self) -> str:
    """Return the normalized tenant database encryption mode."""
    return _normalize_mode(self.tenant_db_encryption, "TENANT_DB_ENCRYPTION")

@property
def control_field_encryption_mode(self) -> str:
    """Return the normalized control-plane field encryption mode."""
    return _normalize_mode(self.control_field_encryption, "CONTROL_FIELD_ENCRYPTION")

@property
def user_data_encryption_mode(self) -> str:
    """Return the normalized filesystem encryption enforcement mode."""
    return _normalize_mode(self.user_data_encryption, "USER_DATA_ENCRYPTION")

@property
def require_https_mode(self) -> str:
    """Return the normalized HTTPS enforcement mode."""
    return _normalize_mode(self.require_https, "REQUIRE_HTTPS")

@property
def trusted_host_list(self) -> list[str]:
    """Return explicit hosts plus the configured frontend hostname."""
    configured_hosts = [
        host.strip().lower() for host in self.trusted_hosts.split(",") if host.strip()
    ]
    frontend_host = urlsplit(self.frontend_url).hostname
    if frontend_host:
        configured_hosts.append(frontend_host.lower())
    return list(dict.fromkeys(configured_hosts))

@property
def trusted_proxy_ip_set(self) -> set[str]:
    """Return normalized proxy addresses trusted to set forwarding headers."""

```



```

        return {
            address.strip().lower()
            for address in self.trusted_proxy_ips.split(",")
            if address.strip()
        }

def auth_is_enabled(settings: Settings) -> bool:
    """Return whether the operator explicitly enabled authentication."""
    if not isinstance(settings, Settings):
        raise TypeError("settings must be a Settings instance")
    return not settings.disable_auth

def _auth_is_enabled(settings: Settings) -> bool:
    """Backward-compatible wrapper for older internal tests and scripts."""
    return auth_is_enabled(settings)

def _mode_enabled(settings: Settings, mode: str) -> bool:
    """Resolve auto/enabled/required security modes against runtime posture."""
    if mode == "disabled":
        return False
    if mode == "enabled":
        return True
    if mode == "required":
        return True
    assert mode == "auto", "mode must be normalized before resolution"
    return auth_is_enabled(settings) and not settings.debug

def tenant_db_encryption_required(settings: Settings) -> bool:
    """Return whether tenant SQLite databases must use SQLCIPHER."""
    mode = settings.tenant_db_encryption_mode
    if mode == "enabled":
        return False
    return _mode_enabled(settings, mode)

def tenant_db_encryption_enabled(settings: Settings) -> bool:
    """Return whether tenant SQLite databases should use SQLCIPHER when possible."""
    return _mode_enabled(settings, settings.tenant_db_encryption_mode)

def control_field_encryption_enabled(settings: Settings) -> bool:
    """Return whether sensitive control-plane fields should be encrypted."""

```

```

    return _mode_enabled(settings, settings.control_field_encryption_mode)

def user_data_encryption_required(settings: Settings) -> bool:
    """Return whether user data directories must live on encrypted storage."""
    return _mode_enabled(settings, settings.user_data_encryption_mode)

def https_required(settings: Settings) -> bool:
    """Return whether HTTP requests must be upgraded or rejected in production."""
    mode = settings.require_https_mode
    if mode == "enabled":
        return True
    return _mode_enabled(settings, mode)

def _validate_settings(settings: Settings) -> None:
    """Reject invalid security-critical configuration."""
    if settings.sprites_public_launch_enabled and not settings.sprites_storage_encryption_confirmed:
        raise RuntimeError(
            "SPRITES_PUBLIC_LAUNCH_ENABLED requires " "SPRITES_STORAGE_ENCRYPTION_CONFIRMED"
        )
    if (
        settings.sprites_public_launch_enabled
        and settings.managed_runtime_provider != "fly_sprites"
    ):
        raise RuntimeError(
            "SPRITES_PUBLIC_LAUNCH_ENABLED requires MANAGED_RUNTIME_PROVIDER=fly_sprites"
        )
    if settings.managed_runtime_provider not in _MANAGED_RUNTIME_PROVIDER_VALUES:
        allowed_values = ", ".join(sorted(_MANAGED_RUNTIME_PROVIDER_VALUES))
        raise RuntimeError(f"MANAGED_RUNTIME_PROVIDER must be one of: {allowed_values}")
    if settings.managed_backup_provider not in _MANAGED_BACKUP_PROVIDER_VALUES:
        allowed_values = ", ".join(sorted(_MANAGED_BACKUP_PROVIDER_VALUES))
        raise RuntimeError(f"MANAGED_BACKUP_PROVIDER must be one of: {allowed_values}")
    if settings.deployment_environment not in {"local", "staging", "production"}:
        raise RuntimeError("DEPLOYMENT_ENVIRONMENT must be one of: local, production, staging")
    if settings.managed_recovery_drill_enabled:
        if settings.deployment_environment != "staging":
            raise RuntimeError(
                "MANAGED_RECOVERY_DRILL_ENABLED requires DEPLOYMENT_ENVIRONMENT=staging"
            )
    if _SHA256_PATTERN.fullmatch(settings.managed_recovery_operator_token_hash) is None:
        raise RuntimeError(
            "MANAGED_RECOVERY_OPERATOR_TOKEN_HASH must be a lowercase SHA-256 digest"
        )

```

```

if settings.managed_backup_provider == "digitalocean_spaces":
    expected_endpoint = f"https://{settings.managed_backup_region}.digitaloceanspaces.com"
    if settings.managed_backup_endpoint_url != expected_endpoint:
        raise RuntimeError(
            "MANAGED_BACKUP_ENDPOINT_URL must be the DigitalOcean Spaces regional endpoint"
        )
if settings.managed_runtime_provider == "fly_sprites":
    _require_https_url(
        settings.sprites_api_url,
        "SPRITES_API_URL",
        reject_routing_metadata=True,
    )
    _require_https_url(settings.sprites_artifact_url, "SPRITES_ARTIFACT_URL")
    _require_https_url(
        settings.sprites_public_control_url,
        "SPRITES_PUBLIC_CONTROL_URL",
        reject_routing_metadata=True,
    )
    _validate_allowed_domains(settings.sprites_allowed_domains)
    if _SPRITE_PREFIX_PATTERN.fullmatch(settings.sprites_name_prefix) is None:
        raise RuntimeError(
            "SPRITES_NAME_PREFIX must be a lowercase DNS label of 1 to 30 characters"
        )
    if _SHA256_PATTERN.fullmatch(settings.sprites_artifact_sha256) is None:
        raise RuntimeError(
            "SPRITES_ARTIFACT_SHA256 must be 64 lowercase hexadecimal characters"
        )
    if not 5 <= settings.sprites_wake_timeout_seconds <= 120:
        raise RuntimeError("SPRITES_WAKE_TIMEOUT_SECONDS must be between 5 and 120")
    if not 600 <= settings.sprites_operation_stale_seconds <= 86400:
        raise RuntimeError("SPRITES_OPERATION_STALE_SECONDS must be between 600 and 86400")
    if not 60 <= settings.sprites_reconcile_interval_seconds <= 86400:
        raise RuntimeError("SPRITES_RECONCILE_INTERVAL_SECONDS must be between 60 and 86400")
    if not 300 <= settings.sprites_reconcile_grace_seconds <= 604800:
        raise RuntimeError("SPRITES_RECONCILE_GRACE_SECONDS must be between 300 and 604800")
    if (
        not isinstance(settings.sprites_api_token, SecretStr)
        or not settings.sprites_api_token.get_secret_value().strip()
    ):
        raise RuntimeError("SPRITES_API_TOKEN is required for Fly Sprites")
    if not isinstance(settings.sprites_name_key, SecretStr):
        raise RuntimeError("SPRITES_NAME_KEY is required for Fly Sprites")
    sprites_name_key = settings.sprites_name_key.get_secret_value()
    if not sprites_name_key.strip():
        raise RuntimeError("SPRITES_NAME_KEY is required for Fly Sprites")
    if not settings.sprites_artifact_url:

```

```

        raise RuntimeError("SPRITES_ARTIFACT_URL is required for Fly Sprites")
if not settings.sprites_artifact_sha256:
    raise RuntimeError("SPRITES_ARTIFACT_SHA256 is required for Fly Sprites")
if not settings.sprites_public_control_url:
    raise RuntimeError("SPRITES_PUBLIC_CONTROL_URL is required for Fly Sprites")
if settings.container_enabled:
    raise RuntimeError("Fly Sprites mode requires CONTAINER_ENABLED=false")
if not auth_is_enabled(settings):
    raise RuntimeError("Fly Sprites mode requires AUTH_ENABLED=true")
if not https_required(settings):
    raise RuntimeError("Fly Sprites mode requires HTTPS enforcement through REQUIRE")
if settings.control_field_encryption_mode != "required":
    raise RuntimeError("Fly Sprites mode requires CONTROL_FIELD_ENCRYPTION=required")
if not settings.managed_backup_bucket:
    raise RuntimeError("MANAGED_BACKUP_BUCKET is required for Fly Sprites")
_require_https_url(
    settings.managed_backup_endpoint_url,
    "MANAGED_BACKUP_ENDPOINT_URL",
    reject_routing_metadata=True,
)
if not settings.managed_backup_region.strip():
    raise RuntimeError("MANAGED_BACKUP_REGION is required for Fly Sprites")
if (
    not settings.managed_backup_prefix
    or len(settings.managed_backup_prefix) > 128
    or any(
        character not in "abcdefghijklmnopqrstuvwxyz0123456789-/"
        for character in settings.managed_backup_prefix
    )
    or ".." in settings.managed_backup_prefix.split("/")
):
    raise RuntimeError("MANAGED_BACKUP_PREFIX is invalid")
if not 5 * 1024 * 1024 <= settings.managed_backup_part_bytes <= 5 * 1024**3:
    raise RuntimeError("MANAGED_BACKUP_PART_BYTES is outside S3 multipart limits")
if not 1 <= settings.managed_backup_retention_days <= 3650:
    raise RuntimeError("MANAGED_BACKUP_RETENTION_DAYS must be between 1 and 3650")
access_key = settings.managed_backup_access_key_id
secret_key = settings.managed_backup_secret_access_key
if (access_key is None) != (secret_key is None):
    raise RuntimeError("MANAGED_BACKUP credentials must be configured together")
if isinstance(access_key, SecretStr) and not access_key.get_secret_value().strip():
    raise RuntimeError("MANAGED_BACKUP credentials must not be blank")
if isinstance(secret_key, SecretStr) and not secret_key.get_secret_value().strip():
    raise RuntimeError("MANAGED_BACKUP credentials must not be blank")
if not isinstance(settings.backup_encryption_key, SecretStr):
    raise RuntimeError("BACKUP_ENCRYPTION_KEY is required for Fly Sprites")

```

```

backup_encryption_key = settings.backup_encryption_key.get_secret_value()
if _SHA256_PATTERN.fullmatch(backup_encryption_key) is None:
    raise RuntimeError("BACKUP_ENCRYPTION_KEY must be 64 lowercase hexadecimal characters")
control_hostname = urlsplit(settings.sprites_public_control_url).hostname
if not control_hostname or not _control_domain_is_allowed(
    control_hostname.lower(), settings.sprites_allowed_domains
):
    raise RuntimeError("SPRITES_ALLOWED_DOMAINS must cover SPRITES_PUBLIC_CONTROL_URL")
if len(settings.sprites_name_key.encode("utf-8")) < 32:
    raise RuntimeError("SPRITES_NAME_KEY must contain at least 32 UTF-8 bytes")
bootstrap_script_path = Path(settings.sprites_bootstrap_script_path)
if (
    not settings.sprites_bootstrap_script_path.strip()
    or not bootstrap_script_path.is_absolute()
    or not bootstrap_script_path.is_file()
):
    raise RuntimeError(
        "SPRITES_BOOTSTRAP_SCRIPT_PATH must be an absolute regular-file path"
    )

authentication_enabled = auth_is_enabled(settings)
if not authentication_enabled:
    normalized_host = settings.host.strip().lower()
    if normalized_host not in {"127.0.0.1", "::1", "localhost"}:
        raise RuntimeError("No-auth mode must bind to a loopback host")
    if settings.container_enabled:
        raise RuntimeError("No-auth mode requires CONTAINER_ENABLED=false")
if authentication_enabled:
    google_configured = bool(settings.google_client_id and settings.google_client_secret)
    github_configured = bool(settings.github_client_id and settings.github_client_secret)
    if not google_configured and not github_configured:
        raise RuntimeError(
            "At least one complete OAuth provider configuration is required when "
            "authentication is enabled"
        )
if authentication_enabled and not settings.secret_key:
    raise RuntimeError("SECRET_KEY must be set when authentication is enabled")
if authentication_enabled and len(settings.secret_key.encode("utf-8")) < 32:
    raise RuntimeError("SECRET_KEY must contain at least 32 bytes")
if authentication_enabled and len(settings.secret_key) < 8:
    raise RuntimeError("SECRET_KEY must contain at least 8 distinct characters")

settings.encryption_pepper_bytes
settings.key_encryption_key_bytes
settings.key_encryption_keyring_previous

```

```

if authentication_enabled:
    if not settings.debug:
        if not settings.active_key_encryption_key_bytes:
            raise RuntimeError(
                "KEY_ENCRYPTION_KEY or ENCRYPTION_PEPPER must be set when "
                "authentication is enabled outside debug mode"
            )

settings.tenant_db_encryption_mode
settings.control_field_encryption_mode
settings.user_data_encryption_mode
settings.require_https_mode

normalized_key_id = settings.key_encryption_key_id.strip()
if settings.key_encryption_key_bytes and not normalized_key_id:
    raise RuntimeError("KEY_ENCRYPTION_KEY_ID must not be empty when KEY_ENCRYPTION_KEY")
settings.key_encryption_key_id = normalized_key_id or "local-v1"

if settings.container_mount_mode not in {"narrow", "tenant-data"}:
    raise RuntimeError("CONTAINER_MOUNT_MODE must be either narrow or tenant-data")
if settings.terminal_keepalive_s < 300:
    raise RuntimeError("TERMINAL_KEEPLIVE_S must be at least 300 seconds")
if settings.terminal_scrollback_lines < 100:
    raise RuntimeError("TERMINAL_SCROLLBACK_LINES must be at least 100")
if settings.request_body_max_bytes < 1024:
    raise RuntimeError("REQUEST_BODY_MAX_BYTES must be at least 1024")
if not settings.trusted_host_list:
    raise RuntimeError("TRUSTED_HOSTS must configure at least one host")
if "*" in settings.trusted_host_list:
    raise RuntimeError("TRUSTED_HOSTS must not trust every host")
settings.trusted_proxy_ip_set

@lru_cache()
def get_settings() -> Settings:
    """Get cached settings instance."""
    settings = Settings()
    _validate_settings(settings)
    if not settings.secret_key:
        settings.secret_key = _generate_secret()
    return settings

```

3.3 models.py

Pydantic models define the HTTP contract shared by the backend and Type-Script client. The root chunk below tangles back to `backend/src/yinshi/models.py`.

```

"""Pydantic models for API request/response schemas."""

from datetime import datetime
from typing import Any, Literal
from urllib.parse import urlsplit

from pydantic import BaseModel, Field, ValidationInfo, field_validator

from yinshi.model_catalog import DEFAULT_SESSION_MODEL, get_provider_metadata, normalize_model

PI_CONFIG_CATEGORY_ORDER = (
    "skills",
    "extensions",
    "prompts",
    "agents",
    "themes",
    "settings",
    "models",
    "sessions",
    "instructions",
)
PI_CONFIG_CATEGORIES = frozenset(PI_CONFIG_CATEGORY_ORDER)

def _strip_required_text(value: str, message: str) -> str:
    """Trim a required string field and raise the caller's validation message."""
    normalized_value = value.strip()
    if not normalized_value:
        raise ValueError(message)
    return normalized_value

def _strip_optional_text(value: str | None, message: str) -> str | None:
    """Trim an optional string field and reject explicit blank values."""
    if value is None:
        return None
    return _strip_required_text(value, message)

class DesktopAuthorizationRequestIn(BaseModel):
    """Start a PKCE-bound desktop account authorization request."""

    redirect_uri: str = Field(..., min_length=1, max_length=2048)
    code_challenge: str = Field(
        ...,
        min_length=43,

```

```

        max_length=43,
        pattern=r"^[A-Za-z0-9_-]+$",
    )
    state: str = Field(
        ...,
        min_length=16,
        max_length=128,
        pattern=r"^[A-Za-z0-9._~-]+$",
    )

    @field_validator("redirect_uri")
    @classmethod
    def validate_loopback_redirect(cls, value: str) -> str:
        """Accept only the desktop helper's exact numbered IPv4 callback."""
        if not isinstance(value, str):
            raise TypeError("redirect_uri must be a string")
        try:
            parsed = urlsplit(value)
            port = parsed.port
        except ValueError as error:
            raise ValueError("redirect_uri must contain a valid port") from error
        if parsed.scheme != "http" or parsed.hostname != "127.0.0.1" or port is None:
            raise ValueError("redirect_uri must use a numbered IPv4 loopback address")
        if parsed.path != "/auth/desktop/callback":
            raise ValueError("redirect_uri must use the desktop callback path")
        if parsed.username or parsed.password or parsed.query or parsed.fragment:
            raise ValueError("redirect_uri must not contain credentials, query, or fragment")
        return value

class DesktopAuthorizationRequestOut(BaseModel):
    """Opaque hosted authorization request returned to a desktop client."""

    request_id: str
    authorize_url: str
    expires_at: datetime

class DesktopRefreshRequestIn(BaseModel):
    """Rotate one current desktop refresh credential."""

    refresh_token: str = Field(..., min_length=32, max_length=256)

class DesktopTokenRequestIn(BaseModel):
    """Exchange one browser-approved code using its original PKCE verifier."""

```



```

authorization_code: str = Field(..., min_length=32, max_length=128)
code_verifier: str = Field(
    ...,
    min_length=43,
    max_length=128,
    pattern=r"^[A-Za-z0-9._~-]+$",
)
device_name: str = Field(..., min_length=1, max_length=120)

class DesktopTokenUserOut(BaseModel):
    """Account identity attached to newly issued desktop credentials."""

    id: str
    email: str

class DesktopTokenOut(BaseModel):
    """Initial credentials and verification material for one desktop device."""

    token_type: Literal["Bearer"] = "Bearer"
    access_token: str
    access_token_expires_at: int
    refresh_token: str
    refresh_token_expires_at: int
    account_lease: str
    account_lease_expires_at: int
    device_id: str
    signing_public_key: str
    user: DesktopTokenUserOut

class RepoCreate(BaseModel):
    """Request to import a repository."""

    name: str = Field(..., max_length=255)
    remote_url: str | None = Field(None, max_length=2048)
    local_path: str | None = Field(None, max_length=4096)
    custom_prompt: str | None = Field(None, max_length=10_000)
    agents_md: str | None = Field(None, max_length=50_000)

class RepoOut(BaseModel):
    """Repository response."""

```

```

    id: str
    created_at: datetime
    updated_at: datetime
    name: str
    remote_url: str | None = None
    root_path: str
    custom_prompt: str | None = None
    agents_md: str | None = None

class RepoUpdate(BaseModel):
    """Request to update a repository."""

    name: str | None = Field(None, max_length=255)
    custom_prompt: str | None = Field(None, max_length=10_000)
    agents_md: str | None = Field(None, max_length=50_000)

class WorkspaceCreate(BaseModel):
    """Request to create a worktree workspace."""

    name: str | None = Field(None, max_length=255)

class WorkspaceOut(BaseModel):
    """Workspace response."""

    id: str
    created_at: datetime
    updated_at: datetime
    repo_id: str
    name: str
    branch: str
    path: str
    state: str = "ready"

class WorkspaceUpdate(BaseModel):
    """Request to update a workspace."""

    state: str | None = Field(None, pattern=r"^(ready|archived)$")

class SessionCreate(BaseModel):
    """Request to create an agent session."""

```

```

model: str = Field(DEFAULT_SESSION_MODEL, max_length=100)

@field_validator("model")
@classmethod
def validate_model(cls, value: str) -> str:
    """Normalize session model values to canonical refs."""
    return normalize_model_ref(value)

class SessionUpdate(BaseModel):
    """Request to update a session."""

    model: str | None = Field(None, max_length=100)

    @field_validator("model")
    @classmethod
    def validate_model(cls, value: str | None) -> str | None:
        """Normalize optional session model values to canonical refs.

        Rejects explicit null so that PATCH cannot write NULL into the
        database, which would break SessionOut deserialization.
        """
        if value is None:
            raise ValueError("model cannot be set to null")
        return normalize_model_ref(value)

class SessionOut(BaseModel):
    """Session response."""

    id: str
    created_at: datetime
    updated_at: datetime
    workspace_id: str
    status: str = "idle"
    model: str = DEFAULT_SESSION_MODEL
    pi_context_version: int = 0

class MessageOut(BaseModel):
    """Message response."""

    id: str
    created_at: datetime
    session_id: str
    role: str

```

```

    content: str | None = None
    full_message: str | None = None
    turn_id: str | None = None
    turn_status: str | None = None

class WSPrompt(BaseModel):
    """WebSocket message from client to send a prompt."""

    type: str = "prompt"
    prompt: str = Field(..., max_length=100_000)
    model: str | None = Field(None, max_length=100)

    @field_validator("model")
    @classmethod
    def validate_model(cls, value: str | None) -> str | None:
        """Normalize optional prompt model values to canonical refs."""
        if value is None:
            return None
        return normalize_model_ref(value)

class WSCancel(BaseModel):
    """WebSocket message from client to cancel."""

    type: str = "cancel"

# --- Multi-tenant models ---

ByocRunnerStorageProfile = Literal[
    "aws_ebs_s3_files",
    "archil_shared_files",
    "archil_all_posix",
]
RunnerStorageProfile = Literal[
    "aws_ebs_s3_files",
    "archil_shared_files",
    "archil_all_posix",
    "fly_sprites_posix",
]

class UserOut(BaseModel):
    """User account response (from control plane)."""

```

```

    id: str
    email: str
    display_name: str | None = None
    avatar_url: str | None = None
    status: str = "active"
    tier: str = "free"

class CloudRunnerCreate(BaseModel):
    """Request a one-time cloud runner registration token."""

    name: str = Field("AWS runner", min_length=1, max_length=120)
    cloud_provider: Literal["aws"] = "aws"
    region: str = Field("us-east-1", min_length=1, max_length=64)
    storage_profile: ByocRunnerStorageProfile = "aws_ebs_s3_files"

    @field_validator("name", "region")
    @classmethod
    def validate_non_blank_text(cls, value: str) -> str:
        """Reject blank runner setup fields after trimming whitespace."""
        return _strip_required_text(value, "Runner setup fields must not be blank")

class CloudRunnerOut(BaseModel):
    """Safe cloud runner status returned to the frontend."""

    id: str
    created_at: datetime
    updated_at: datetime
    name: str
    cloud_provider: str
    region: str
    status: Literal["pending", "online", "offline", "revoked"]
    registered_at: datetime | None = None
    last_heartbeat_at: datetime | None = None
    runner_version: str | None = None
    capabilities: dict[str, Any] = Field(default_factory=dict)
    data_dir: str | None = None
    noise_public_key: str | None = None
    noise_key_fingerprint: str | None = None
    noise_key_confirmed: bool = False

class CloudRunnerRegistrationOut(BaseModel):
    """Cloud runner registration response with the one-time token."""

```

```

runner: CloudRunnerOut
registration_token: str
registration_token_expires_at: datetime
control_url: str
environment: dict[str, str]

class RunnerRegisterIn(BaseModel):
    """Registration payload submitted by a freshly bootstrapped runner."""

    registration_token: str = Field(..., min_length=32, max_length=512)
    runner_version: str = Field(..., min_length=1, max_length=120)
    capabilities: dict[str, Any] = Field(default_factory=dict)
    data_dir: str = Field(..., min_length=1, max_length=4096)
    sqlite_dir: str | None = Field(None, min_length=1, max_length=4096)
    shared_files_dir: str | None = Field(None, min_length=1, max_length=4096)
    storage_profile: RunnerStorageProfile = "aws_ebs_s3_files"
    noise_public_key: str | None = Field(
        None, min_length=43, max_length=43, pattern=r"^[A-Za-z0-9_-]+$"
    )

    @field_validator("registration_token", "runner_version", "data_dir")
    @classmethod
    def validate_runner_registration_text(cls, value: str) -> str:
        """Reject blank runner registration strings."""
        return _strip_required_text(value, "Runner registration values must not be blank")

    @field_validator("sqlite_dir", "shared_files_dir")
    @classmethod
    def validate_runner_registration_path(cls, value: str | None) -> str | None:
        """Reject blank optional runner storage paths."""
        return _strip_optional_text(value, "Runner storage paths must not be blank")

class RunnerRegisterOut(BaseModel):
    """Registration response containing the runner's bearer token once."""

    runner_id: str
    runner_token: str
    capability_signing_public_key: str
    status: Literal["online"] = "online"

class RunnerNoiseKeyConfirmationIn(BaseModel):
    """Exact runner Noise identity a user has visually confirmed."""

```

```

noise_public_key: str = Field(
    min_length=43,
    max_length=43,
    pattern=r"^[A-Za-z0-9_-]+$",
)

class RunnerCapabilityCreateIn(BaseModel):
    """Requested least-privilege authority for one encrypted runner session."""

    initiator_public_key: str = Field(
        min_length=43,
        max_length=43,
        pattern=r"^[A-Za-z0-9_-]+$",
    )
    scopes: list[str] = Field(min_length=1, max_length=14)
    max_session_bytes: int = Field(ge=65_536, le=1_073_741_824)

    @field_validator("scopes")
    @classmethod
    def validate_scopes(cls, value: list[str]) -> list[str]:
        """Reject blank or repeated capability scopes before service validation."""
        normalized = [scope.strip() for scope in value]
        if any(not scope for scope in normalized):
            raise ValueError("scopes must not contain blank values")
        if len(set(normalized)) != len(normalized):
            raise ValueError("scopes must not contain duplicates")
        return normalized

class RunnerCapabilityOut(BaseModel):
    """Signed dispatch grant and pinned responder identity for one session."""

    capability: str
    transfer_id: str
    runner_id: str
    runner_public_key: str
    protocol: str
    issued_at: int
    expires_at: int
    max_frame_bytes: int
    max_session_bytes: int
    relay_url: str

class RunnerHeartbeatIn(BaseModel):

```

```

"""Heartbeat payload submitted by an already registered runner."""

runner_version: str = Field(..., min_length=1, max_length=120)
capabilities: dict[str, Any] = Field(default_factory=dict)
data_dir: str = Field(..., min_length=1, max_length=4096)
sqlite_dir: str | None = Field(None, min_length=1, max_length=4096)
shared_files_dir: str | None = Field(None, min_length=1, max_length=4096)
storage_profile: RunnerStorageProfile = "aws_ebs_s3_files"

@field_validator("runner_version", "data_dir")
@classmethod
def validate_runner_heartbeat_text(cls, value: str) -> str:
    """Reject blank runner heartbeat strings."""
    return _strip_required_text(value, "Runner heartbeat values must not be blank")

@field_validator("sqlite_dir", "shared_files_dir")
@classmethod
def validate_runner_heartbeat_path(cls, value: str | None) -> str | None:
    """Reject blank optional runner storage paths."""
    return _strip_optional_text(value, "Runner storage paths must not be blank")

class RunnerHeartbeatOut(BaseModel):
    """Heartbeat acknowledgement returned to the runner."""

    runner_id: str
    capability_signing_public_key: str
    status: Literal["online"] = "online"

class ProviderSetupFieldOut(BaseModel):
    """Describe one provider setup field to the frontend."""

    key: str
    label: str
    required: bool
    secret: bool = False

class ProviderDescriptorOut(BaseModel):
    """One provider in the catalog response."""

    id: str
    label: str
    auth_strategies: list[str]
    setup_fields: list[ProviderSetupFieldOut]

```



```

docs_url: str
connected: bool
model_count: int

class ModelDescriptorOut(BaseModel):
    """One model in the catalog response."""

    ref: str
    provider: str
    id: str
    label: str
    api: str
    reasoning: bool
    thinking_levels: list[str] = Field(default_factory=list)
    inputs: list[str]
    context_window: int
    max_tokens: int

class ProviderCatalogOut(BaseModel):
    """Catalog response for providers and models."""

    default_model: str
    providers: list[ProviderDescriptorOut]
    models: list[ModelDescriptorOut]

class ProviderAuthStartOut(BaseModel):
    """Response returned when a provider OAuth flow starts."""

    flow_id: str
    provider: str
    auth_url: str
    instructions: str | None = None
    manual_input_required: bool = False
    manual_input_prompt: str | None = None
    manual_input_submitted: bool = False

class ProviderAuthStatusOut(BaseModel):
    """Status payload for a provider OAuth flow."""

    status: str
    provider: str
    flow_id: str

```

```

instructions: str | None = None
progress: list[str] = Field(default_factory=list)
manual_input_required: bool = False
manual_input_prompt: str | None = None
manual_input_submitted: bool = False
error: str | None = None

class ProviderAuthInputIn(BaseModel):
    """Manual OAuth input submitted from the browser UI."""

    flow_id: str = Field(..., min_length=1, max_length=255)
    authorization_input: str = Field(..., min_length=1, max_length=8192)

    @field_validator("flow_id")
    @classmethod
    def validate_flow_id(cls, value: str) -> str:
        """Reject blank OAuth flow identifiers."""
        normalized_value = value.strip()
        if not normalized_value:
            raise ValueError("flow_id must not be empty")
        return normalized_value

    @field_validator("authorization_input")
    @classmethod
    def validate_authorization_input(cls, value: str) -> str:
        """Reject blank pasted OAuth callback input."""
        normalized_value = value.strip()
        if not normalized_value:
            raise ValueError("authorization_input must not be empty")
        return normalized_value

class ProviderConnectionCreate(BaseModel):
    """Request to create a provider connection."""

    provider: str = Field(..., min_length=1, max_length=100)
    auth_strategy: str = Field(..., min_length=1, max_length=100)
    secret: str | dict[str, Any]
    label: str = Field("", max_length=255)
    config: dict[str, Any] = Field(default_factory=dict)

    @field_validator("provider")
    @classmethod
    def validate_provider(cls, value: str) -> str:
        """Reject blank providers."""

```

```

        normalized_value = value.strip()
        if not normalized_value:
            raise ValueError("Provider must not be empty")
        return normalized_value

@field_validator("auth_strategy")
@classmethod
def validate_auth_strategy(cls, value: str, info: ValidationInfo) -> str:
    """Require one of the provider's supported auth strategies."""
    normalized_value = value.strip()
    if not normalized_value:
        raise ValueError("Auth strategy must not be empty")
    provider = info.data.get("provider")
    if isinstance(provider, str):
        metadata = get_provider_metadata(provider)
        if normalized_value not in metadata.auth_strategies:
            raise ValueError(f"{provider} does not support auth strategy {normalized_value}")
    return normalized_value

@field_validator("secret")
@classmethod
def validate_secret(
    cls,
    value: str | dict[str, Any],
    info: ValidationInfo,
) -> str | dict[str, Any]:
    """Match secret shape to the requested auth strategy."""
    auth_strategy = info.data.get("auth_strategy")
    if auth_strategy == "api_key":
        if not isinstance(value, str):
            raise TypeError("API key connections require a string secret")
        normalized_value = value.strip()
        if not normalized_value:
            raise ValueError("API key secret must not be empty")
        return normalized_value
    if auth_strategy == "api_key_with_config":
        if isinstance(value, str):
            normalized_value = value.strip()
            if not normalized_value:
                raise ValueError("API key secret must not be empty")
            return normalized_value
        if not isinstance(value, dict):
            raise TypeError("API key + config connections require a string or object secret")
        if not value:
            raise ValueError("API key + config secret must not be empty")
        return value

```

```

        if auth_strategy == "oauth":
            if not isinstance(value, dict):
                raise TypeError("OAuth connections require an object secret")
            if not value:
                raise ValueError("OAuth secret must not be empty")
            return value
        return value

class ProviderConnectionOut(BaseModel):
    """Provider connection response without the secret payload."""

    id: str
    created_at: datetime
    updated_at: datetime
    provider: str
    auth_strategy: str
    label: str = ""
    config: dict[str, Any] = Field(default_factory=dict)
    status: str
    last_used_at: datetime | None = None
    expires_at: datetime | None = None

class ApiKeyCreate(BaseModel):
    """Compatibility wrapper for legacy API-key endpoints."""

    provider: str = Field(..., min_length=1, max_length=100)
    key: str = Field(..., min_length=1, max_length=500)
    label: str = Field("", max_length=255)

class ApiKeyOut(BaseModel):
    """Compatibility wrapper for legacy API-key responses."""

    id: str
    created_at: datetime
    provider: str
    label: str = ""
    last_used_at: datetime | None = None

class PiConfigImport(BaseModel):
    """Import a Pi config from a GitHub repository."""

    repo_url: str = Field(..., max_length=2048)

```

```

@field_validator("repo_url")
@classmethod
def validate_repo_url(cls, value: str) -> str:
    """Reject blank repository URLs."""
    normalized_value = value.strip()
    if not normalized_value:
        raise ValueError("Repository URL must not be empty")
    return normalized_value

class PiConfigCategoryUpdate(BaseModel):
    """Toggle the enabled Pi resource categories."""

    enabled_categories: list[str]

    @field_validator("enabled_categories")
    @classmethod
    def validate_enabled_categories(cls, value: list[str]) -> list[str]:
        """Require unique, known category names."""
        seen_categories: set[str] = set()
        normalized_categories: list[str] = []
        for category in value:
            normalized_category = category.strip()
            if normalized_category not in PI_CONFIG_CATEGORIES:
                raise ValueError(f"Unsupported category: {category}")
            if normalized_category in seen_categories:
                raise ValueError(f"Duplicate category: {category}")
            seen_categories.add(normalized_category)
            normalized_categories.append(normalized_category)
        return normalized_categories

class PiConfigOut(BaseModel):
    """Pi config status response."""

    id: str
    created_at: datetime
    updated_at: datetime
    source_type: str
    source_label: str
    last_synced_at: datetime | None = None
    status: str
    error_message: str | None = None
    available_categories: list[str]
    enabled_categories: list[str]

```

```

class PiCommand(BaseModel):
    """One slash command exposed from the user's imported Pi config.

    The ``kind`` discriminator preserves the source of the command so the UI
    can group or style entries, while keeping the wire format flat. Fields
    are the minimum needed to render + invoke the command; host filesystem
    paths and rarely-used metadata are intentionally omitted.
    """

    kind: Literal["skill", "prompt", "extension"]
    name: str
    description: str = ""
    command_name: str


class PiConfigCommandsOut(BaseModel):
    """Slash commands resolved from the imported Pi config.

    See sidecar/src/sidecar.js listResources for the producer side. The
    wire contract is a single flat list; the ``kind`` field on each entry
    distinguishes skills, prompts, and extension-registered commands.
    """

    commands: list[PiCommand] = Field(default_factory=list)


class PiPackageReleaseOut(BaseModel):
    """One upstream pi release note entry."""

    tag_name: str
    version: str
    name: str
    published_at: str | None = None
    html_url: str
    body_markdown: str


class PiReleaseNotesOut(BaseModel):
    """Runtime pi version plus recent upstream release notes."""

    package_name: str
    installed_version: str | None = None
    latest_version: str | None = None
    node_version: str | None = None

```

```

release_notes_url: str
update_policy: str
runtime_error: str | None = None
release_error: str | None = None
releases: list[PiPackageReleaseOut] = Field(default_factory=list)

```

3.4 model_catalog.py

The model catalog normalizes provider metadata so UI labels, thinking levels, and sidebar requests use the same vocabulary. The root chunk below tangles back to `backend/src/yinshi/model_catalog.py`.

```

"""Shared model and provider catalog helpers."""

from __future__ import annotations

from dataclasses import dataclass

DEFAULT_SESSION_MODEL = "minimax/MiniMax-M2.7"

LEGACY_MODEL_ALIASES = {
    "haiku": "anthropic/claude-haiku-4-5-20251001",
    "minimax": DEFAULT_SESSION_MODEL,
    "minimax-m2.5-highspeed": "minimax/MiniMax-M2.5-highspeed",
    "minimax-m2.7": DEFAULT_SESSION_MODEL,
    "minimax-m2.7-highspeed": "minimax/MiniMax-M2.7-highspeed",
    "opus": "anthropic/claude-opus-4-20250514",
    "sonnet": "anthropic/claude-sonnet-4-20250514",
}

@dataclass(frozen=True, slots=True)
class ProviderSetupField:
    """Describe one provider-specific setup field."""

    key: str
    label: str
    required: bool
    secret: bool = False

@dataclass(frozen=True, slots=True)
class ProviderMetadata:
    """Metadata that Yinshi adds on top of the pi provider registry."""

    id: str

```

```

label: str
auth_strategies: tuple[str, ...]
setup_fields: tuple[ProviderSetupField, ...]
docs_url: str
supported: bool = True

def _titleize_provider(provider_id: str) -> str:
    """Convert a provider identifier into a readable label."""
    if not isinstance(provider_id, str):
        raise TypeError("provider_id must be a string")
    normalized_provider_id = provider_id.strip()
    if not normalized_provider_id:
        raise ValueError("provider_id must not be empty")
    pieces = normalized_provider_id.replace("-", " ").split()
    return " ".join(piece.upper() if len(piece) <= 3 else piece.capitalize() for piece in pieces)

_COMMON_DOCS_URL = "https://www.npmjs.com/package/@earendil-works/pi-ai"

PROVIDER_METADATA_BY_ID: dict[str, ProviderMetadata] = {
    "anthropic": ProviderMetadata(
        id="anthropic",
        label="Anthropic",
        auth_strategies=("api_key", "oauth"),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
    ),
    "azure-openai-responses": ProviderMetadata(
        id="azure-openai-responses",
        label="Azure OpenAI",
        auth_strategies=("api_key_with_config",),
        setup_fields=(
            ProviderSetupField("baseUrl", "Base URL", False),
            ProviderSetupField("resourceName", "Resource Name", False),
            ProviderSetupField("azureDeploymentName", "Deployment Name", False),
            ProviderSetupField("apiVersion", "API Version", False),
        ),
        docs_url=_COMMON_DOCS_URL,
    ),
    "github-copilot": ProviderMetadata(
        id="github-copilot",
        label="GitHub Copilot",
        auth_strategies=("oauth",),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
    ),
}

```



```

),
"google-vertex": ProviderMetadata(
    id="google-vertex",
    label="Google Vertex AI",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"google": ProviderMetadata(
    id="google",
    label="Google",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"google-antigravity": ProviderMetadata(
    id="google-antigravity",
    label="Google Antigravity",
    auth_strategies=("oauth",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"google-gemini-cli": ProviderMetadata(
    id="google-gemini-cli",
    label="Google Gemini CLI",
    auth_strategies=("oauth",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"minimax": ProviderMetadata(
    id="minimax",
    label="MiniMax",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"minimax-cn": ProviderMetadata(
    id="minimax-cn",
    label="MiniMax China",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"mistral": ProviderMetadata(
    id="mistral",
    label="Mistral",

```

```

        auth_strategies=("api_key"),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
    ),
    "openai": ProviderMetadata(
        id="openai",
        label="OpenAI",
        auth_strategies=("api_key"),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
    ),
    "openai-codex": ProviderMetadata(
        id="openai-codex",
        label="OpenAI Codex",
        auth_strategies=("oauth"),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
    ),
    "openrouter": ProviderMetadata(
        id="openrouter",
        label="OpenRouter",
        auth_strategies=("api_key"),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
    ),
    "opencode": ProviderMetadata(
        id="opencode",
        label="OpenCode Zen",
        auth_strategies=("api_key"),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
    ),
    "opencode-go": ProviderMetadata(
        id="opencode-go",
        label="OpenCode Go",
        auth_strategies=("api_key"),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
    ),
    "vercel-ai-gateway": ProviderMetadata(
        id="vercel-ai-gateway",
        label="Vercel AI Gateway",
        auth_strategies=("api_key"),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
    ),

```

```

"groq": ProviderMetadata(
    id="groq",
    label="Groq",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"cerebras": ProviderMetadata(
    id="cerebras",
    label="Cerebras",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"xai": ProviderMetadata(
    id="xai",
    label="xAI",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"zai": ProviderMetadata(
    id="zai",
    label="ZAI",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"huggingface": ProviderMetadata(
    id="huggingface",
    label="Hugging Face",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
"kimi-coding": ProviderMetadata(
    id="kimi-coding",
    label="Kimi Coding",
    auth_strategies=("api_key",),
    setup_fields=(),
    docs_url=_COMMON_DOCS_URL,
),
# Bedrock requires AWS SDK credential resolution rather than the auth.json-style
# API key path that Yinshi currently supports per session.
"amazon-bedrock": ProviderMetadata(
    id="amazon-bedrock",

```

```

        label="Amazon Bedrock",
        auth_strategies=(),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
        supported=False,
    ),
}

```

```

def get_provider_metadata(provider_id: str) -> ProviderMetadata:
    """Return metadata for a provider id, defaulting to unsupported."""
    if not isinstance(provider_id, str):
        raise TypeError("provider_id must be a string")
    normalized_provider_id = provider_id.strip()
    if not normalized_provider_id:
        raise ValueError("provider_id must not be empty")
    metadata = PROVIDER_METADATA_BY_ID.get(normalized_provider_id)
    if metadata is not None:
        return metadata
    return ProviderMetadata(
        id=normalized_provider_id,
        label=_titleize_provider(normalized_provider_id),
        auth_strategies=(),
        setup_fields=(),
        docs_url=_COMMON_DOCS_URL,
        supported=False,
    )

```

```

def normalize_model_ref(model: str | None) -> str:
    """Normalize stored or user-provided model values into canonical refs."""
    if model is None:
        return DEFAULT_SESSION_MODEL
    if not isinstance(model, str):
        raise TypeError("model must be a string or None")
    normalized_model = model.strip()
    if not normalized_model:
        raise ValueError("model must not be empty")
    canonical_model = LEGACY_MODEL_ALIASES.get(normalized_model.lower())
    if canonical_model is not None:
        return canonical_model
    return normalized_model

```

3.5 exceptions.py

Domain exceptions give routes and services typed failure modes instead of leaking low-level library errors. The root chunk below tangles back to `backend/src/yinshi/exceptions.py`.

```
"""Custom exception hierarchy for Yinshi."""

class YinshiError(Exception):
    """Base exception for all Yinshi errors."""

class RepoNotFoundError(YinshiError):
    """Raised when a repository is not found."""

class WorkspaceNotFoundError(YinshiError):
    """Raised when a workspace is not found."""

class SessionNotFoundError(YinshiError):
    """Raised when a session is not found."""

class GitError(YinshiError):
    """Raised when a git operation fails."""

class PiConfigError(YinshiError):
    """Raised when Pi config management cannot complete."""

class PiConfigNotFoundError(PiConfigError):
    """Raised when a Pi config record or directory is not available."""

class GitHubAppError(YinshiError):
    """Raised when the GitHub App integration cannot complete."""

class GitHubAccessError(GitHubAppError):
    """Raised when GitHub access cannot be granted for a repository."""

    def __init__(
        self,
```

```

        message: str,
        *,
        code: str,
        connect_url: str | None = None,
        manage_url: str | None = None,
    ) -> None:
        assert code, "code must not be empty"
        super().__init__(message)
        self.code = code
        self.connect_url = connect_url
        self.manage_url = manage_url


class GitHubConnectRequiredError(GitHubAccessError):
    """Raised when a user must connect GitHub before importing a repo."""

    def __init__(self, message: str, *, connect_url: str | None = None) -> None:
        super().__init__(
            message,
            code="github_connect_required",
            connect_url=connect_url,
        )


class GitHubAccessNotGrantedError(GitHubAccessError):
    """Raised when an installation exists but cannot access the repo."""

    def __init__(
        self,
        message: str,
        *,
        connect_url: str | None = None,
        manage_url: str | None = None,
    ) -> None:
        super().__init__(
            message,
            code="github_access_not_granted",
            connect_url=connect_url,
            manage_url=manage_url,
        )


class GitHubInstallationUnusableError(GitHubAccessError):
    """Raised when a connected installation is no longer usable."""

    def __init__(self, message: str, *, manage_url: str | None = None) -> None:

```

```

        super().__init__(
            message,
            code="github_installation_unusable",
            manage_url=manage_url,
        )

class SidecarError(YinshiError):
    """Raised when sidecar communication fails."""

class SidecarNotConnectedError(SidecarError):
    """Raised when the sidecar is not connected."""

class KeyNotFoundError(YinshiError):
    """Raised when no API key is available for a provider."""

class CreditExhaustedError(YinshiError):
    """Raised when a legacy platform-credit path runs out of allowance."""

class EncryptionNotConfiguredError(YinshiError):
    """Raised when encryption pepper is not configured."""

class ContainerStartError(YinshiError):
    """Raised when a per-user sidecar container fails to start."""

class ContainerNotReadyError(YinshiError):
    """Raised when a container's sidecar socket is not ready in time."""

class RunnerRegistrationError(YinshiError):
    """Raised when a cloud runner registration token cannot be used."""

class RunnerAuthenticationError(YinshiError):
    """Raised when a cloud runner bearer token is missing or invalid."""

```

3.6 rate_limit.py

Rate limiting uses tenant-aware keys so sensitive routes can be throttled without mixing users together. The root chunk below tangles back to

```

backend/src/yinshi/rate_limit.py.

"""Shared rate-limiting configuration for FastAPI routes."""

from __future__ import annotations

from fastapi import Request
from slowapi import Limiter
from slowapi.util import get_remote_address

def route_rate_limit_key(request: Request) -> str:
    """Return the tenant user id when available, otherwise the client IP."""
    tenant = getattr(request.state, "tenant", None)
    if tenant is not None:
        user_id = getattr(tenant, "user_id", None)
        if isinstance(user_id, str):
            normalized_user_id = user_id.strip()
            if normalized_user_id:
                return normalized_user_id

    client_address = get_remote_address(request)
    if isinstance(client_address, str):
        normalized_client_address = client_address.strip()
        if normalized_client_address:
            return normalized_client_address
    return "unknown-client"

limiter = Limiter(key_func=route_rate_limit_key)

```

3.7 utils/paths.py

Path utilities provide a small, audited primitive for checking containment after path resolution. The root chunk below tangles back to backend/src/yinshi/utils/paths.py.

```

"""Shared path utilities for security checks."""

import os

def is_path_inside(path: str, base: str) -> bool:
    """Check if *path* is inside *base* after resolving symlinks.

Returns True when the resolved *path* equals *base* or is a descendant
of it. The check uses ``os.path.realpath`` to resolve symlinks so

```



```

    that ``../`` traversal tricks are neutralised.
    """
    resolved = os.path.realpath(path)
    resolved_base = os.path.realpath(base)
    return resolved == resolved_base or resolved.startswith(resolved_base + os.sep)

```

4 Database Layer and Tenancy

Yinshi stores control data separately from user data. The control database tracks users, sessions, and runner records that must be found before a tenant database is opened. Tenant databases hold repositories, workspaces, messages, imported config, and provider secrets for one user.

4.1 db.py

The database module opens SQLite connections and applies control-plane and tenant schema migrations. The root chunk below tangles back to `backend/src/yinshi/db.py`.

```

"""SQLite database connection and schema management."""

import importlib
import logging
import os
import sqlite3
import stat
from collections.abc import Iterator
from contextlib import contextmanager
from pathlib import Path
from types import ModuleType
from typing import cast

from yinshi.config import (
    control_field_encryption_enabled,
    get_settings,
    tenant_db_encryption_enabled,
    tenant_db_encryption_required,
)
from yinshi.model_catalog import DEFAULT_SESSION_MODEL

logger = logging.getLogger(__name__)

_SCHEMA_VERSION = 5
_SQLCIPHER_MODULE_NAMES = ("sqlcipher3.dbapi2", "pysqlcipher3.dbapi2")
_PLAINTEXT_ROLLBACK_SUFFIX = ".plaintext.rollback"

```

```

class DatabaseEncryptionError(RuntimeError):
    """Raised when a required application database cannot be encrypted or unlocked."""

SCHEMA_SQL = f"""
PRAGMA journal_mode = WAL;

CREATE TABLE IF NOT EXISTS repos (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    name TEXT NOT NULL,
    remote_url TEXT,
    root_path TEXT NOT NULL,
    custom_prompt TEXT,
    agents_md TEXT,
    owner_email TEXT,
    installation_id INTEGER
);

CREATE TABLE IF NOT EXISTS workspaces (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    repo_id TEXT NOT NULL REFERENCES repos(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    branch TEXT NOT NULL,
    path TEXT NOT NULL,
    state TEXT DEFAULT 'ready' NOT NULL
);

CREATE TABLE IF NOT EXISTS sessions (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    workspace_id TEXT NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    status TEXT DEFAULT 'idle' NOT NULL,
    model TEXT DEFAULT '{DEFAULT_SESSION_MODEL}',
    pi_context_version INTEGER DEFAULT 1 NOT NULL
);

CREATE TABLE IF NOT EXISTS messages (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    session_id TEXT NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,

```

```

        role TEXT NOT NULL,
        content TEXT,
        full_message TEXT,
        turn_id TEXT,
        turn_status TEXT
    );

CREATE TABLE IF NOT EXISTS prompt_runs (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16))))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    session_id TEXT NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    idempotency_key TEXT NOT NULL,
    status TEXT DEFAULT 'starting' NOT NULL CHECK (
        status IN ('starting', 'running', 'stopping', 'completed', 'failed', 'cancelled', '')
    ),
    UNIQUE(session_id, idempotency_key)
);

CREATE TABLE IF NOT EXISTS prompt_events (
    run_id TEXT NOT NULL REFERENCES prompt_runs(id) ON DELETE CASCADE,
    sequence INTEGER NOT NULL CHECK (sequence >= 0),
    event_json TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    PRIMARY KEY(run_id, sequence)
);

CREATE UNIQUE INDEX IF NOT EXISTS idx_prompt_runs_active_session
    ON prompt_runs(session_id)
    WHERE status IN ('starting', 'running', 'stopping');
CREATE INDEX IF NOT EXISTS idx_prompt_events_run
    ON prompt_events(run_id, sequence);
CREATE INDEX IF NOT EXISTS idx_messages_session ON messages(session_id, created_at);
CREATE INDEX IF NOT EXISTS idx_messages_turn_id ON messages(turn_id);
CREATE INDEX IF NOT EXISTS idx_sessions_workspace ON sessions(workspace_id);
CREATE INDEX IF NOT EXISTS idx_workspaces_repo ON workspaces(repo_id);

CREATE TRIGGER IF NOT EXISTS update_repos_updated_at AFTER UPDATE ON repos
BEGIN UPDATE repos SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;

CREATE TRIGGER IF NOT EXISTS update_workspaces_updated_at AFTER UPDATE ON workspaces
BEGIN UPDATE workspaces SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;

CREATE TRIGGER IF NOT EXISTS update_sessions_updated_at AFTER UPDATE ON sessions
BEGIN UPDATE sessions SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;
"""

```

```

def _open_connection(db_path: str, *, check_same_thread: bool = True) -> sqlite3.Connection:
    """Open a SQLite connection with standard settings."""
    conn = sqlite3.connect(db_path, check_same_thread=check_same_thread)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    conn.execute("PRAGMA busy_timeout = 5000")
    return conn

def _load_sqlcipher_module() -> ModuleType:
    """Load a compatible SQLCipher DB-API driver or raise a sanitized error."""
    for module_name in _SQLCIPHER_MODULE_NAMES:
        try:
            module = importlib.import_module(module_name)
        except ImportError:
            continue
        if hasattr(module, "connect") and hasattr(module, "Row"):
            return module
    raise DatabaseEncryptionError("SQLCipher is unavailable for required database encryption")

def _application_database_key(*, context: str) -> bytes:
    """Derive one database key from the configured Keychain-injected pepper."""
    if context not in {"control", "local"}:
        raise ValueError("database key context is invalid")
    from yinshi.services.crypto import derive_subkey

    settings = get_settings()
    return derive_subkey(
        settings.encryption_pepper_bytes,
        purpose="application-sqlcipher",
        context=context,
    )

def _sqlcipher_error_type(sqlcipher_module: ModuleType) -> type[Exception]:
    """Return a safe exception class exported by one SQLCipher driver."""
    database_error = getattr(sqlcipher_module, "DatabaseError", sqlite3.DatabaseError)
    if not isinstance(database_error, type) or not issubclass(database_error, Exception):
        return sqlite3.DatabaseError
    return database_error

def _open_keyed_connection(

```

```

    db_path: str,
    *,
    sqlcipher_module: ModuleType,
    database_key: bytes,
) -> sqlite3.Connection:
    """Open and immediately validate one SQLCipher database with an exact key."""
    if len(database_key) != 32:
        raise ValueError("database_key must contain 32 bytes")
    Path(db_path).parent.mkdir(parents=True, exist_ok=True)
    connection = cast(sqlite3.Connection, sqlcipher_module.connect(db_path))
    connection.row_factory = getattr(sqlcipher_module, "Row")
    connection.execute(f"PRAGMA key = '{database_key.hex()}'") # noqa: S608
    connection.execute("PRAGMA cipher_memory_security = ON")
    connection.execute("PRAGMA foreign_keys = ON")
    connection.execute("PRAGMA busy_timeout = 5000")
    try:
        cipher_version = connection.execute("PRAGMA cipher_version").fetchone()
        connection.execute("SELECT count(*) FROM sqlite_master").fetchone()
    except (sqlite3.DatabaseError, _sqlcipher_error_type(sqlcipher_module)) as error:
        connection.close()
        raise DatabaseEncryptionError("Application database could not be unlocked") from error
    if cipher_version is None or not cipher_version[0]:
        connection.close()
        raise DatabaseEncryptionError("SQLCipher driver did not report a cipher version")
    os.chmod(db_path, 0o600)
    return connection

def _plaintext_database_readable(db_path: str) -> bool:
    """Return whether an existing application database is plaintext SQLite."""
    if not os.path.isfile(db_path):
        return False
    try:
        connection = _open_connection(db_path)
        try:
            connection.execute("SELECT count(*) FROM sqlite_master").fetchone()
            return True
        finally:
            connection.close()
    except sqlite3.DatabaseError:
        return False

def _validate_encrypted_database(
    db_path: str,
    *,

```

```

        sqlcipher_module: ModuleType,
        database_key: bytes,
    ) -> None:
        """Require an encrypted database to open and pass SQLCipher integrity checks."""
        connection = _open_keyed_connection(
            db_path,
            sqlcipher_module=sqlcipher_module,
            database_key=database_key,
        )
        try:
            integrity = connection.execute("PRAGMA integrity_check").fetchone()
            if integrity is None or str(integrity[0]).lower() != "ok":
                raise DatabaseEncryptionError(
                    "Encrypted application database failed integrity validation"
                )
        finally:
            connection.close()

def _fsync_file(path: str) -> None:
    """Flush one regular file through its filesystem."""
    flags = os.O_RDONLY | getattr(os, "O_CLOEXEC", 0)
    descriptor = os.open(path, flags)
    try:
        os.fsync(descriptor)
    finally:
        os.close(descriptor)

def _fsync_parent_directory(path: str) -> None:
    """Flush directory entries for one filesystem path."""
    parent_path = os.path.dirname(os.path.abspath(path))
    flags = os.O_RDONLY | getattr(os, "O_CLOEXEC", 0)
    flags |= getattr(os, "O_DIRECTORY", 0)
    descriptor = os.open(parent_path, flags)
    try:
        os.fsync(descriptor)
    finally:
        os.close(descriptor)

def _create_private_rollback_copy(source_path: str, rollback_path: str) -> None:
    """Copy a plaintext database into a new owner-only rollback file."""
    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL
    flags |= getattr(os, "O_CLOEXEC", 0)
    flags |= getattr(os, "O_NOFOLLOW", 0)

```

```

descriptor = os.open(rollback_path, flags, 0o600)
try:
    with open(source_path, "rb") as source, os.fdopen(descriptor, "wb") as rollback:
        descriptor = -1
        while chunk := source.read(1024 * 1024):
            rollback.write(chunk)
            rollback.flush()
        os.fchmod(rollback.fileno(), 0o600)
except Exception:
    if descriptor >= 0:
        os.close(descriptor)
    if os.path.exists(rollback_path):
        os.unlink(rollback_path)
    raise

def _remove_sqlite_sidecars(db_path: str) -> None:
    """Remove checkpointed WAL files and durably record their removal."""
    removed = False
    for suffix in ("-wal", "-shm"):
        sidecar_path = f"{db_path}{suffix}"
        if os.path.exists(sidecar_path):
            os.unlink(sidecar_path)
            removed = True
    if removed:
        _fsync_parent_directory(db_path)

def _application_rollback_is_trusted(descriptor: int, rollback_path: str) -> bool:
    """Return whether an open rollback still names one private owner-controlled file."""
    opened_stat = os.fstat(descriptor)
    try:
        path_stat = os.lstat(rollback_path)
    except FileNotFoundError:
        return False
    return (
        stat.S_ISREG(opened_stat.st_mode)
        and stat.S_ISREG(path_stat.st_mode)
        and not stat.S_ISLNK(path_stat.st_mode)
        and (opened_stat.st_dev, opened_stat.st_ino) == (path_stat.st_dev, path_stat.st_ino)
        and opened_stat.st_uid == os.geteuid()
        and opened_stat.st_nlink == 1
        and opened_stat.st_mode & 0o077 == 0
    )

```

```

def _open_trusted_application_rollback(rollback_path: str) -> int | None:
    """Open and validate an application rollback without following links."""
    flags = os.O_RDONLY | getattr(os, "O_CLOEXEC", 0)
    flags |= getattr(os, "O_NOFOLLOW", 0)
    flags |= getattr(os, "O_NONBLOCK", 0)
    try:
        descriptor = os.open(rollback_path, flags)
    except FileNotFoundError:
        return None
    except OSError as exc:
        raise DatabaseEncryptionError(
            "Application database migration rollback must be a trusted regular file"
        ) from exc
    if _application_rollback_is_trusted(descriptor, rollback_path):
        return descriptor
    os.close(descriptor)
    raise DatabaseEncryptionError(
        "Application database migration rollback must be a trusted regular file"
    )

def _recover_plaintext_migration_rollback(db_path: str) -> None:
    """Restore a durable rollback when an interrupted replacement lost its primary."""
    rollback_path = f"{db_path}_{PLAINTEXT_ROLLBACK_SUFFIX}"
    if os.path.lexists(db_path):
        return
    descriptor = _open_trusted_application_rollback(rollback_path)
    if descriptor is None:
        return
    try:
        os.fsync(descriptor)
        if os.path.lexists(db_path):
            return
        if not _application_rollback_is_trusted(descriptor, rollback_path):
            raise DatabaseEncryptionError(
                "Application database migration rollback must be a trusted regular file"
            )
        os.replace(rollback_path, db_path)
        os.chmod(db_path, 0o600)
        _fsync_file(db_path)
        _fsync_parent_directory(db_path)
    finally:
        os.close(descriptor)
    logger.warning("Recovered an interrupted application database migration")

```



```

def _remove_validated_migration_rollback(db_path: str) -> None:
    """Remove rollback only after the validated primary is durable."""
    rollback_path = f"{db_path}_{PLAINTEXT_ROLLBACK_SUFFIX}"
    if not os.path.exists(rollback_path):
        return
    _fsync_file(db_path)
    _fsync_parent_directory(db_path)
    os.unlink(rollback_path)
    _fsync_parent_directory(db_path)

def _migrate_plaintext_application_database(
    db_path: str,
    *,
    sqlcipher_module: ModuleType,
    database_key: bytes,
) -> None:
    """Export plaintext SQLite into SQLCipher and atomically replace the original."""
    temporary_path = f"{db_path}.encrypted.tmp"
    rollback_path = f"{db_path}_{PLAINTEXT_ROLLBACK_SUFFIX}"
    for path in (temporary_path, f"{temporary_path}-wal", f"{temporary_path}-shm"):
        if os.path.exists(path):
            os.unlink(path)

    if os.path.exists(rollback_path):
        _fsync_file(db_path)
        _fsync_parent_directory(db_path)
        os.unlink(rollback_path)
        _fsync_parent_directory(db_path)

    source = cast(sqlite3.Connection, sqlcipher_module.connect(db_path))
    try:
        checkpoint = source.execute("PRAGMA wal_checkpoint(TRUNCATE)").fetchone()
        if (
            checkpoint is None
            or len(checkpoint) < 3
            or int(checkpoint[0]) != 0
            or int(checkpoint[1]) != int(checkpoint[2])
        ):
            raise DatabaseEncryptionError("Application database WAL checkpoint did not complete")
        integrity = source.execute("PRAGMA integrity_check").fetchone()
        if integrity is None or str(integrity[0]).lower() != "ok":
            raise DatabaseEncryptionError(
                "Plaintext application database failed integrity validation"
            )
        source.execute(

```

```

        "ATTACH DATABASE ? AS encrypted KEY ?",
        (temporary_path, f"x'{database_key.hex()}'"),
    )
    try:
        source.execute("SELECT sqlcipher_export('encrypted')").fetchone()
    finally:
        source.execute("DETACH DATABASE encrypted")
except (sqlite3.DatabaseError, _sqlcipher_error_type(sqlcipher_module)) as error:
    raise DatabaseEncryptionError("Application database encryption migration failed")
finally:
    source.close()

_remove_sqlite_sidecars(db_path)
try:
    _validate_encrypted_database(
        temporary_path,
        sqlcipher_module=sqlcipher_module,
        database_key=database_key,
    )
    os.chmod(temporary_path, 0o600)
    _fsync_file(temporary_path)
    _create_private_rollback_copy(db_path, rollback_path)
    _fsync_file(rollback_path)
    _fsync_parent_directory(db_path)
    os.replace(temporary_path, db_path)
    _fsync_parent_directory(db_path)
    _validate_encrypted_database(
        db_path,
        sqlcipher_module=sqlcipher_module,
        database_key=database_key,
    )
    os.chmod(db_path, 0o600)
    _fsync_file(db_path)
    _fsync_parent_directory(db_path)
except (DatabaseEncryptionError, OSError):
    if os.path.exists(rollback_path):
        os.replace(rollback_path, db_path)
        os.chmod(db_path, 0o600)
        _fsync_file(db_path)
        _fsync_parent_directory(db_path)
    if os.path.exists(temporary_path):
        os.unlink(temporary_path)
        _fsync_parent_directory(db_path)
    raise
else:
    os.unlink(rollback_path)

```

```

        _fsync_parent_directory(db_path)
    _remove_sqlite_sidecars(db_path)
    logger.info("Migrated an application database to SQLCipher")

def _open_application_connection(db_path: str, *, context: str) -> sqlite3.Connection:
    """Open one application database under configured encryption policy."""
    settings = get_settings()
    _recover_plaintext_migration_rollback(db_path)
    if not tenant_db_encryption_enabled(settings):
        return _open_connection(db_path)
    try:
        sqlcipher_module = _load_sqlcipher_module()
    except DatabaseEncryptionError:
        if tenant_db_encryption_required(settings):
            raise
        logger.warning("SQLCipher unavailable; opening application database without encryption")
        return _open_connection(db_path)

    database_key = _application_database_key(context=context)
    if os.path.exists(db_path):
        try:
            connection = _open_keyed_connection(
                db_path,
                sqlcipher_module=sqlcipher_module,
                database_key=database_key,
            )
        except DatabaseEncryptionError:
            if not _plaintext_database_readable(db_path):
                raise
            _migrate_plaintext_application_database(
                db_path,
                sqlcipher_module=sqlcipher_module,
                database_key=database_key,
            )
        else:
            _remove_validated_migration_rollback(db_path)
            return connection
    return _open_keyed_connection(
        db_path,
        sqlcipher_module=sqlcipher_module,
        database_key=database_key,
    )

@contextmanager

```

```

def get_db() -> Iterator[sqlite3.Connection]:
    """Get a SQLite connection as a context manager."""
    settings = get_settings()
    conn = _open_application_connection(settings.db_path, context="local")
    try:
        yield conn
    finally:
        conn.close()

def _migrate(conn: sqlite3.Connection) -> None:
    """Apply versioned schema migrations."""
    conn.execute("CREATE TABLE IF NOT EXISTS schema_version (version INTEGER NOT NULL)")
    row = conn.execute("SELECT version FROM schema_version").fetchone()
    current = row[0] if row else 0

    if current < 1:
        columns = [r[1] for r in conn.execute("PRAGMA table_info(repos)").fetchall()]
        if "owner_email" not in columns:
            logger.info("Migration v1: adding owner_email column to repos")
            conn.execute("ALTER TABLE repos ADD COLUMN owner_email TEXT")

    if current < 2:
        columns = [r[1] for r in conn.execute("PRAGMA table_info(repos)").fetchall()]
        if "installation_id" not in columns:
            logger.info("Migration v2: adding installation_id column to repos")
            conn.execute("ALTER TABLE repos ADD COLUMN installation_id INTEGER")

    if current < 3:
        columns = [r[1] for r in conn.execute("PRAGMA table_info(messages)").fetchall()]
        if "turn_status" not in columns:
            logger.info("Migration v3: adding turn_status column to messages")
            conn.execute("ALTER TABLE messages ADD COLUMN turn_status TEXT")

    if current < 4:
        columns = [r[1] for r in conn.execute("PRAGMA table_info(repos)").fetchall()]
        if "agents_md" not in columns:
            logger.info("Migration v4: adding agents_md column to repos")
            conn.execute("ALTER TABLE repos ADD COLUMN agents_md TEXT")

    if current < 5:
        columns = [r[1] for r in conn.execute("PRAGMA table_info(sessions)").fetchall()]
        if "pi_context_version" not in columns:
            logger.info("Migration v5: adding pi_context_version column to sessions")
            conn.execute(
                "ALTER TABLE sessions ADD COLUMN pi_context_version INTEGER DEFAULT 0 NOT NULL")

```

```

    )

    if current != _SCHEMA_VERSION:
        conn.execute("DELETE FROM schema_version")
        conn.execute("INSERT INTO schema_version (version) VALUES (?)", (_SCHEMA_VERSION,))
        conn.commit()

def init_db() -> None:
    """Initialize the database schema."""
    logger.info("Initializing application database")
    try:
        with get_db() as conn:
            conn.executescript(SCHEMA_SQL)
            _migrate(conn)
    except sqlite3.Error:
        logger.error("Failed to initialize application database")
        raise
    logger.info("Database initialized")

# --- Control plane database (multi-tenant) ---

CONTROL_SCHEMA_SQL = """
PRAGMA journal_mode = WAL;

CREATE TABLE IF NOT EXISTS users (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    email TEXT NOT NULL UNIQUE,
    display_name TEXT,
    avatar_url TEXT,
    status TEXT DEFAULT 'active' NOT NULL,
    tier TEXT DEFAULT 'free' NOT NULL,
    disk_quota_mb INTEGER DEFAULT 5000,
    disk_used_mb INTEGER DEFAULT 0,
    encrypted_dek BLOB,
    credit_used_cents INTEGER DEFAULT 0,
    credit_limit_cents INTEGER DEFAULT 500,
    last_login_at TIMESTAMP,
    deletion_requested_at TIMESTAMP,
    deletion_scheduled_for TIMESTAMP
);

CREATE TABLE IF NOT EXISTS oauth_identities (

```

```

        id TEXT PRIMARY KEY DEFAULT (lower(hex(randblob(16)))),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
        user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
        provider TEXT NOT NULL,
        provider_user_id TEXT NOT NULL,
        provider_email TEXT NOT NULL,
        provider_data TEXT,
        UNIQUE(provider, provider_user_id)
    );

CREATE TABLE IF NOT EXISTS api_keys (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    provider TEXT NOT NULL,
    encrypted_key BLOB NOT NULL,
    label TEXT DEFAULT '',
    last_used_at TIMESTAMP
);

CREATE TABLE IF NOT EXISTS provider_connections (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    provider TEXT NOT NULL,
    auth_strategy TEXT NOT NULL,
    encrypted_secret BLOB NOT NULL,
    label TEXT DEFAULT '',
    config_json TEXT DEFAULT '{}' NOT NULL,
    status TEXT DEFAULT 'connected' NOT NULL,
    last_used_at TIMESTAMP,
    expires_at TIMESTAMP
);

CREATE TABLE IF NOT EXISTS auth_sessions (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    revoked_at TIMESTAMP
);

CREATE TABLE IF NOT EXISTS desktop_authorization_requests (
    request_id_hash TEXT PRIMARY KEY,
    created_at INTEGER NOT NULL,
    expires_at INTEGER NOT NULL,

```

```

        redirect_uri TEXT NOT NULL,
        code_challenge TEXT NOT NULL,
        state TEXT NOT NULL,
        user_id TEXT REFERENCES users(id) ON DELETE CASCADE,
        authorization_code_hash TEXT,
        approved_at INTEGER,
        consumed_at INTEGER
    );

CREATE INDEX IF NOT EXISTS idx_desktop_authorization_requests_expiry
ON desktop_authorization_requests(expires_at);

CREATE TABLE IF NOT EXISTS desktop_devices (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    created_at INTEGER NOT NULL,
    refresh_token_hash TEXT NOT NULL UNIQUE,
    refresh_token_expires_at INTEGER NOT NULL,
    revoked_at INTEGER,
    last_seen_at INTEGER
);

CREATE INDEX IF NOT EXISTS idx_desktop_devices_user ON desktop_devices(user_id);

CREATE TABLE IF NOT EXISTS desktop_used_refresh_tokens (
    token_hash TEXT PRIMARY KEY,
    device_id TEXT NOT NULL REFERENCES desktop_devices(id) ON DELETE CASCADE,
    rotated_at INTEGER NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_desktop_used_refresh_tokens_device
ON desktop_used_refresh_tokens(device_id);

CREATE TABLE IF NOT EXISTS usage_log (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    session_id TEXT NOT NULL,
    provider TEXT NOT NULL,
    model TEXT NOT NULL,
    input_tokens INTEGER DEFAULT 0,
    output_tokens INTEGER DEFAULT 0,
    cache_read_tokens INTEGER DEFAULT 0,
    cache_write_tokens INTEGER DEFAULT 0,
    cost_cents REAL DEFAULT 0,

```

```

        key_source TEXT NOT NULL
    );

CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);
CREATE INDEX IF NOT EXISTS idx_oauth_user ON oauth_identities(user_id);
CREATE INDEX IF NOT EXISTS idx_api_keys_user ON api_keys(user_id);
CREATE INDEX IF NOT EXISTS idx_provider_connections_user ON provider_connections(user_id);
CREATE INDEX IF NOT EXISTS idx_auth_sessions_user ON auth_sessions(user_id);
CREATE INDEX IF NOT EXISTS idx_usage_user ON usage_log(user_id);
CREATE INDEX IF NOT EXISTS idx_usage_session ON usage_log(session_id);

CREATE TABLE IF NOT EXISTS github_installations (
    id INTEGER PRIMARY KEY,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    installation_id INTEGER NOT NULL,
    account_login TEXT NOT NULL,
    account_type TEXT NOT NULL,
    html_url TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT (datetime('now')),
    UNIQUE(user_id, installation_id)
);

CREATE INDEX IF NOT EXISTS idx_github_installations_user ON github_installations(user_id);

CREATE TABLE IF NOT EXISTS github_install_flows (
    state_digest TEXT PRIMARY KEY,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    installation_id INTEGER,
    expires_at INTEGER NOT NULL,
    created_at TEXT NOT NULL DEFAULT (datetime('now'))
);

CREATE INDEX IF NOT EXISTS idx_github_install_flows_user ON github_install_flows(user_id);
CREATE INDEX IF NOT EXISTS idx_github_install_flows_expiry ON github_install_flows(expires_at);

CREATE TABLE IF NOT EXISTS pi_configs (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    user_id TEXT NOT NULL UNIQUE REFERENCES users(id) ON DELETE CASCADE,
    source_type TEXT NOT NULL,
    source_label TEXT NOT NULL,
    repo_url TEXT,
    available_categories TEXT DEFAULT '[]' NOT NULL,
    enabled_categories TEXT DEFAULT '[]' NOT NULL,
    last_synced_at TIMESTAMP,

```



```

        status TEXT DEFAULT 'ready' NOT NULL,
        error_message TEXT
    );

CREATE INDEX IF NOT EXISTS idx_pi_configs_user ON pi_configs(user_id);

CREATE TABLE IF NOT EXISTS user_settings (
    user_id TEXT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    pi_settings_json TEXT DEFAULT '{}' NOT NULL,
    pi_settings_enabled INTEGER DEFAULT 0 NOT NULL
);

CREATE TABLE IF NOT EXISTS user_runners (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16))))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    kind TEXT DEFAULT 'byoc' NOT NULL
        CHECK (kind IN ('byoc', 'managed', 'managed_restore', 'managed_retired')),
    name TEXT NOT NULL,
    cloud_provider TEXT NOT NULL,
    region TEXT NOT NULL,
    status TEXT DEFAULT 'pending' NOT NULL,
    registration_token_hash TEXT,
    registration_token_expires_at TEXT,
    runner_token_hash TEXT,
    registered_at TEXT,
    last_heartbeat_at TEXT,
    runner_version TEXT,
    capabilities_json TEXT DEFAULT '{}' NOT NULL,
    data_dir TEXT,
    revoked_at TEXT,
    noise_public_key TEXT,
    noise_public_key_confirmed_at TEXT,
    restore_job_id TEXT,
    UNIQUE(user_id, kind)
);

CREATE INDEX IF NOT EXISTS idx_user_runners_user ON user_runners(user_id);
CREATE INDEX IF NOT EXISTS idx_user_runners_registration_token
ON user_runners(registration_token_hash);
CREATE INDEX IF NOT EXISTS idx_user_runners_runner_token ON user_runners(runner_token_hash);

CREATE TABLE IF NOT EXISTS runner_transfer_grants (

```

```

transfer_id TEXT PRIMARY KEY,
user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
runner_id TEXT NOT NULL REFERENCES user_runners(id) ON DELETE CASCADE,
capability_hash TEXT UNIQUE NOT NULL,
expires_at INTEGER NOT NULL,
max_session_bytes INTEGER NOT NULL,
claimed_at INTEGER,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_runner_transfer_grants_runner
ON runner_transfer_grants(runner_id, expires_at);

CREATE TABLE IF NOT EXISTS managed_runtimes (
    user_id TEXT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    runner_id TEXT NOT NULL UNIQUE REFERENCES user_runners(id) ON DELETE CASCADE,
    provider_name TEXT NOT NULL CHECK (provider_name = 'fly_sprites'),
    sprite_external_id TEXT NOT NULL UNIQUE,
    lifecycle_status TEXT NOT NULL CHECK (
        lifecycle_status IN ('provisioning', 'ready', 'failed', 'deleting')
    ),
    generation INTEGER DEFAULT 1 NOT NULL CHECK (generation > 0),
    artifact_version TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    last_error TEXT CHECK (last_error IS NULL OR length(last_error) <= 1000)
);

CREATE TABLE IF NOT EXISTS managed_sprite_identities (
    sprite_name TEXT PRIMARY KEY,
    provider_name TEXT NOT NULL CHECK (provider_name = 'fly_sprites'),
    identity_kind TEXT NOT NULL CHECK (identity_kind IN ('runtime', 'restore_candidate')),
    user_id TEXT NOT NULL,
    job_id TEXT,
    lifecycle_status TEXT NOT NULL CHECK (
        lifecycle_status IN ('creating', 'active', 'retired', 'deleting')
    ),
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,
    CHECK (
        (identity_kind = 'runtime' AND job_id IS NULL)
        OR (identity_kind = 'restore_candidate' AND job_id IS NOT NULL)
    )
);

CREATE TABLE IF NOT EXISTS managed_operational_failures (

```

```

        alert_class TEXT PRIMARY KEY CHECK (
            alert_class IN (
                'managed_sprite_reconciliation_failed',
                'managed_storage_preflight_failed'
            )
        ),
        first_seen_at TEXT NOT NULL,
        last_seen_at TEXT NOT NULL
    );

CREATE TABLE IF NOT EXISTS managed_backup_archives (
    id TEXT PRIMARY KEY,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    runtime_generation INTEGER NOT NULL CHECK (runtime_generation > 0),
    status TEXT NOT NULL CHECK (
        status IN ('creating', 'uploaded', 'ready', 'failed', 'deleting', 'deleted')
    ),
    object_key TEXT NOT NULL UNIQUE,
    object_version TEXT,
    size_bytes INTEGER CHECK (size_bytes IS NULL OR size_bytes > 0),
    sha256 TEXT CHECK (sha256 IS NULL OR length(sha256) = 64),
    wrapped_key BLOB NOT NULL,
    key_id TEXT NOT NULL,
    owner_digest TEXT NOT NULL CHECK (length(owner_digest) = 64),
    created_at TEXT NOT NULL,
    completed_at TEXT,
    last_error TEXT CHECK (last_error IS NULL OR length(last_error) <= 1000)
);

CREATE INDEX IF NOT EXISTS idx_managed_backup_archives_user_created
ON managed_backup_archives(user_id, created_at DESC);

CREATE TABLE IF NOT EXISTS managed_backup_operations (
    user_id TEXT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    job_id TEXT NOT NULL UNIQUE,
    archive_id TEXT NOT NULL REFERENCES managed_backup_archives(id) ON DELETE CASCADE,
    operation TEXT NOT NULL CHECK (operation IN ('create', 'restore', 'delete')),
    status TEXT NOT NULL CHECK (status IN ('running', 'failed')),
    runtime_generation INTEGER NOT NULL CHECK (runtime_generation > 0),
    phase TEXT NOT NULL DEFAULT 'claimed',
    lease_owner TEXT,
    lease_token TEXT,
    lease_expires_at TEXT,
    attempt_count INTEGER NOT NULL DEFAULT 0,
    next_attempt_at TEXT,
    source_runner_id TEXT,

```

```

        source_sprite_id TEXT,
        source_lost INTEGER NOT NULL DEFAULT 0 CHECK (source_lost IN (0, 1)),
        candidate_runner_id TEXT,
        candidate_sprite_id TEXT,
        activation_generation INTEGER,
        started_at TEXT NOT NULL,
        updated_at TEXT NOT NULL,
        last_error TEXT CHECK (last_error IS NULL OR length(last_error) <= 1000),
        failure_class TEXT CHECK (
            failure_class IS NULL OR failure_class IN ('restore_failed', 'deletion_failed')
        )
    );

CREATE TABLE IF NOT EXISTS managed_runtime_activation_guards (
    user_id TEXT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    job_id TEXT NOT NULL UNIQUE,
    lease_token TEXT NOT NULL
);

CREATE TRIGGER IF NOT EXISTS validate_managed_runtime_runner_insert
BEFORE INSERT ON managed_runtimes
WHEN NOT EXISTS (
    SELECT 1 FROM user_runners
    WHERE id = NEW.runner_id AND user_id = NEW.user_id AND kind = 'managed'
)
BEGIN SELECT RAISE(ABORT, 'managed runtime must reference matching managed runner'); END;

CREATE TRIGGER IF NOT EXISTS validate_managed_runtime_runner_update
BEFORE UPDATE OF user_id, runner_id ON managed_runtimes
WHEN NOT EXISTS (
    SELECT 1 FROM user_runners
    WHERE id = NEW.runner_id AND user_id = NEW.user_id AND kind = 'managed'
)
AND NOT EXISTS (
    SELECT 1 FROM managed_runtime_activation_guards WHERE user_id = NEW.user_id
)
BEGIN SELECT RAISE(ABORT, 'managed runtime must reference matching managed runner'); END;

CREATE TRIGGER IF NOT EXISTS protect_linked_managed_runner_update
BEFORE UPDATE OF user_id, kind ON user_runners
WHEN EXISTS (SELECT 1 FROM managed_runtimes WHERE runner_id = OLD.id)
AND (NEW.user_id != OLD.user_id OR NEW.kind != OLD.kind)
AND NOT EXISTS (
    SELECT 1 FROM managed_runtime_activation_guards WHERE user_id = OLD.user_id
)
BEGIN SELECT RAISE(ABORT, 'cannot change linked managed runtime runner'); END;

```

```

CREATE TRIGGER IF NOT EXISTS update_users_updated_at AFTER UPDATE ON users
BEGIN UPDATE users SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;

CREATE TRIGGER IF NOT EXISTS update_pi_configs_updated_at AFTER UPDATE ON pi_configs
BEGIN UPDATE pi_configs SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;

CREATE TRIGGER IF NOT EXISTS update_user_settings_updated_at AFTER UPDATE ON user_settings
BEGIN UPDATE user_settings SET updated_at = CURRENT_TIMESTAMP WHERE user_id = NEW.user_id; END;

CREATE TRIGGER IF NOT EXISTS update_user_runners_updated_at AFTER UPDATE ON user_runners
BEGIN UPDATE user_runners SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;

CREATE TRIGGER IF NOT EXISTS update_managed_runtimes_updated_at AFTER UPDATE ON managed_runtimes
BEGIN UPDATE managed_runtimes SET updated_at = CURRENT_TIMESTAMP WHERE user_id = NEW.user_id; END;

CREATE TRIGGER IF NOT EXISTS update_provider_connections_updated_at AFTER UPDATE ON provider_connections
BEGIN UPDATE provider_connections SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;
"""

@contextmanager
def get_control_db() -> Iterator[sqlite3.Connection]:
    """Get a connection to the control plane database."""
    settings = get_settings()
    conn = _open_application_connection(settings.control_db_path, context="control")
    try:
        yield conn
    finally:
        conn.close()

def _migrate_control(conn: sqlite3.Connection) -> None:
    """Apply control DB schema migrations for existing databases."""
    columns = [r[1] for r in conn.execute("PRAGMA table_info(users)").fetchall()]
    if "credit_used_cents" not in columns:
        logger.info("Control migration: adding credit tracking columns to users")
        conn.execute("ALTER TABLE users ADD COLUMN credit_used_cents INTEGER DEFAULT 0")
        conn.execute("ALTER TABLE users ADD COLUMN credit_limit_cents INTEGER DEFAULT 500")
        conn.commit()

    pi_config_columns = [row[1] for row in conn.execute("PRAGMA table_info(pi_configs)").fetchall()]
    if pi_config_columns and "available_categories" not in pi_config_columns:
        logger.info("Control migration: adding available_categories column to pi_configs")
        conn.execute(
            "ALTER TABLE pi_configs ADD COLUMN available_categories TEXT DEFAULT '[]' NOT NULL"
        )

```

```

    )
    conn.commit()

provider_connection_columns = [
    row[1] for row in conn.execute("PRAGMA table_info(provider_connections)").fetchall()
]
if not provider_connection_columns:
    logger.info("Control migration: creating provider_connections table")
    conn.executescript("""
        CREATE TABLE IF NOT EXISTS provider_connections (
            id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
            updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
            user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
            provider TEXT NOT NULL,
            auth_strategy TEXT NOT NULL,
            encrypted_secret BLOB NOT NULL,
            label TEXT DEFAULT '',
            config_json TEXT DEFAULT '{}' NOT NULL,
            status TEXT DEFAULT 'connected' NOT NULL,
            last_used_at TIMESTAMP,
            expires_at TIMESTAMP
        );
        CREATE INDEX IF NOT EXISTS idx_provider_connections_user
        ON provider_connections(user_id);
        CREATE TRIGGER IF NOT EXISTS update_provider_connections_updated_at
        AFTER UPDATE ON provider_connections
        BEGIN
            UPDATE provider_connections SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
        END;
    """)
    conn.commit()

migrated_row = conn.execute(
    "SELECT COUNT(*) FROM provider_connections WHERE auth_strategy = 'api_key'"
).fetchone()
api_key_count = conn.execute("SELECT COUNT(*) FROM api_keys").fetchone()
assert migrated_row is not None, "provider connection count must be queryable"
assert api_key_count is not None, "api key count must be queryable"
if api_key_count[0] > 0 and migrated_row[0] < api_key_count[0]:
    logger.info("Control migration: backfilling api_keys into provider_connections")
    conn.executescript("""
        INSERT INTO provider_connections
        (id, created_at, updated_at, user_id, provider, auth_strategy,
         encrypted_secret, label, config_json, status, last_used_at, expires_at)
        SELECT id, created_at, created_at, user_id, provider, 'api_key',
    """)

```

```

        encrypted_key, label, '{}', 'connected', last_used_at, NULL
    FROM api_keys
    WHERE id NOT IN (SELECT id FROM provider_connections)
    """)
conn.commit()

desktop_device_columns = {row[1] for row in conn.execute("PRAGMA table_info(desktop_devices)")}
desktop_device_migrations = {
    "revoked_at": "ALTER TABLE desktop_devices ADD COLUMN revoked_at INTEGER",
    "last_seen_at": "ALTER TABLE desktop_devices ADD COLUMN last_seen_at INTEGER",
}
for column_name, statement in desktop_device_migrations.items():
    if column_name not in desktop_device_columns:
        conn.execute(statement)
conn.commit()

desktop_request_columns = {
    row[1] for row in conn.execute("PRAGMA table_info(desktop_authorization_requests)")
}
desktop_request_migrations = {
    "user_id": "ALTER TABLE desktop_authorization_requests ADD COLUMN user_id TEXT REFERENCES users(id)",
    "authorization_code_hash": "ALTER TABLE desktop_authorization_requests ADD COLUMN authorization_code_hash TEXT",
    "approved_at": "ALTER TABLE desktop_authorization_requests ADD COLUMN approved_at INTEGER",
    "consumed_at": "ALTER TABLE desktop_authorization_requests ADD COLUMN consumed_at INTEGER",
}
for column_name, statement in desktop_request_migrations.items():
    if column_name not in desktop_request_columns:
        conn.execute(statement)
conn.execute("""
CREATE UNIQUE INDEX IF NOT EXISTS idx_desktop_authorization_code_hash
ON desktop_authorization_requests(authorization_code_hash)
""")
conn.commit()

runner_columns = [row[1] for row in conn.execute("PRAGMA table_info(user_runners)")]
if not runner_columns:
    logger.info("Control migration: creating user_runners table")
    conn.executescript("""
CREATE TABLE IF NOT EXISTS user_runners (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    kind TEXT DEFAULT 'byoc' NOT NULL CHECK (kind IN ('byoc', 'managed')),
    name TEXT NOT NULL,
    cloud_provider TEXT NOT NULL,

```

```

        region TEXT NOT NULL,
        status TEXT DEFAULT 'pending' NOT NULL,
        registration_token_hash TEXT,
        registration_token_expires_at TEXT,
        runner_token_hash TEXT,
        registered_at TEXT,
        last_heartbeat_at TEXT,
        runner_version TEXT,
        capabilities_json TEXT DEFAULT '{}' NOT NULL,
        data_dir TEXT,
        revoked_at TEXT,
        noise_public_key TEXT,
        noise_public_key_confirmed_at TEXT,
        restore_job_id TEXT,
        UNIQUE(user_id, kind)
    );
    CREATE INDEX IF NOT EXISTS idx_user_runners_user ON user_runners(user_id);
    CREATE INDEX IF NOT EXISTS idx_user_runners_registration_token
    ON user_runners(registration_token_hash);
    CREATE INDEX IF NOT EXISTS idx_user_runners_runner_token
    ON user_runners(runner_token_hash);
    CREATE TRIGGER IF NOT EXISTS update_user_runners_updated_at
    AFTER UPDATE ON user_runners
    BEGIN
        UPDATE user_runners SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END;
    """)
conn.commit()

runner_columns = [row[1] for row in conn.execute("PRAGMA table_info(user_runners)")]
if runner_columns and "noise_public_key" not in runner_columns:
    logger.info("Control migration: adding Noise identity to user_runners")
    conn.execute("ALTER TABLE user_runners ADD COLUMN noise_public_key TEXT")
    conn.commit()
    runner_columns.append("noise_public_key")
if runner_columns and "noise_public_key_confirmed_at" not in runner_columns:
    logger.info("Control migration: adding Noise key confirmation to user_runners")
    conn.execute("ALTER TABLE user_runners ADD COLUMN noise_public_key_confirmed_at TEXT")
    conn.commit()
if runner_columns and "restore_job_id" not in runner_columns:
    logger.info("Control migration: adding restore job binding to user_runners")
    conn.execute("ALTER TABLE user_runners ADD COLUMN restore_job_id TEXT")
    conn.commit()

_migrate_runner_kinds(conn)
_migrate_managed_restore_runner_kind(conn)

```



```

conn.execute("""
    CREATE TABLE IF NOT EXISTS runner_transfer_grants (
        transfer_id TEXT PRIMARY KEY,
        user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
        runner_id TEXT NOT NULL REFERENCES user_runners(id) ON DELETE CASCADE,
        capability_hash TEXT UNIQUE NOT NULL,
        expires_at INTEGER NOT NULL,
        max_session_bytes INTEGER NOT NULL,
        claimed_at INTEGER,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
    )
    """)
conn.execute(
    "CREATE INDEX IF NOT EXISTS idx_runner_transfer_grants_runner "
    "ON runner_transfer_grants(runner_id, expires_at)"
)
conn.executescript("""
    CREATE TABLE IF NOT EXISTS managed_runtimes (
        user_id TEXT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
        runner_id TEXT NOT NULL UNIQUE REFERENCES user_runners(id) ON DELETE CASCADE,
        provider_name TEXT NOT NULL CHECK (provider_name = 'fly_sprites'),
        sprite_external_id TEXT NOT NULL UNIQUE,
        lifecycle_status TEXT NOT NULL CHECK (
            lifecycle_status IN ('provisioning', 'ready', 'failed', 'deleting')
        ),
        generation INTEGER DEFAULT 1 NOT NULL CHECK (generation > 0),
        artifact_version TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
        last_error TEXT CHECK (last_error IS NULL OR length(last_error) <= 1000)
    );
    CREATE TABLE IF NOT EXISTS managed_backup_archives (
        id TEXT PRIMARY KEY,
        user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
        runtime_generation INTEGER NOT NULL CHECK (runtime_generation > 0),
        status TEXT NOT NULL CHECK (
            status IN ('creating', 'uploaded', 'ready', 'failed', 'deleting')
        ),
        object_key TEXT NOT NULL UNIQUE,
        object_version TEXT,
        size_bytes INTEGER CHECK (size_bytes IS NULL OR size_bytes > 0),
        sha256 TEXT CHECK (sha256 IS NULL OR length(sha256) = 64),
        wrapped_key BLOB NOT NULL,
        key_id TEXT NOT NULL,
        owner_digest TEXT NOT NULL CHECK (length(owner_digest) = 64),

```

```

        created_at TEXT NOT NULL,
        completed_at TEXT,
        last_error TEXT CHECK (last_error IS NULL OR length(last_error) <= 1000)
    );
CREATE INDEX IF NOT EXISTS idx_managed_backup_archives_user_created
ON managed_backup_archives(user_id, created_at DESC);
CREATE TABLE IF NOT EXISTS managed_backup_operations (
    user_id TEXT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
    job_id TEXT NOT NULL UNIQUE,
    archive_id TEXT NOT NULL REFERENCES managed_backup_archives(id) ON DELETE CASCADE,
    operation TEXT NOT NULL CHECK (operation IN ('create', 'restore', 'delete')),
    status TEXT NOT NULL CHECK (status IN ('running', 'failed')),
    runtime_generation INTEGER NOT NULL CHECK (runtime_generation > 0),
    phase TEXT NOT NULL DEFAULT 'claimed',
    lease_owner TEXT,
    lease_token TEXT,
    lease_expires_at TEXT,
    attempt_count INTEGER NOT NULL DEFAULT 0,
    next_attempt_at TEXT,
    source_runner_id TEXT,
    source_sprite_id TEXT,
    source_lost INTEGER NOT NULL DEFAULT 0 CHECK (source_lost IN (0, 1)),
    candidate_runner_id TEXT,
    candidate_sprite_id TEXT,
    activation_generation INTEGER,
    started_at TEXT NOT NULL,
    updated_at TEXT NOT NULL,
    last_error TEXT CHECK (last_error IS NULL OR length(last_error) <= 1000),
    failure_class TEXT CHECK (
        failure_class IS NULL OR failure_class IN ('restore_failed', 'deletion_failed')
    )
);
CREATE TRIGGER IF NOT EXISTS validate_managed_runtime_runner_insert
BEFORE INSERT ON managed_runtimes
WHEN NOT EXISTS (
    SELECT 1 FROM user_runners
    WHERE id = NEW.runner_id AND user_id = NEW.user_id AND kind = 'managed'
)
BEGIN
    SELECT RAISE(ABORT, 'managed runtime must reference matching managed runner');
END;
CREATE TRIGGER IF NOT EXISTS validate_managed_runtime_runner_update
BEFORE UPDATE OF user_id, runner_id ON managed_runtimes
WHEN NOT EXISTS (
    SELECT 1 FROM user_runners
    WHERE id = NEW.runner_id AND user_id = NEW.user_id AND kind = 'managed'
)

```

```

    )
    BEGIN
        SELECT RAISE(ABORT, 'managed runtime must reference matching managed runner');
    END;
    CREATE TRIGGER IF NOT EXISTS protect_linked_managed_runner_update
    BEFORE UPDATE OF user_id, kind ON user_runners
    WHEN EXISTS (SELECT 1 FROM managed_runtimes WHERE runner_id = OLD.id)
    AND (NEW.user_id != OLD.user_id OR NEW.kind != OLD.kind)
    BEGIN
        SELECT RAISE(ABORT, 'cannot change linked managed runtime runner');
    END;
    CREATE TRIGGER IF NOT EXISTS update_managed_runtimes_updated_at
    AFTER UPDATE ON managed_runtimes
    BEGIN
        UPDATE managed_runtimes SET updated_at = CURRENT_TIMESTAMP
        WHERE user_id = NEW.user_id;
    END;
    """)
    _migrate_managed_runtime_activation_guards(conn)
    _migrate_managed_backup_archive_statuses(conn)
    _migrate_managed_backup_operation_columns(conn)
    _migrate_managed_sprite_identity_ownership(conn)
    _backfill_managed_sprite_identities(conn)
    conn.commit()

    _migrate_encrypted_control_fields(conn)

def _migrate_managed_runtime_activation_guards(conn: sqlite3.Connection) -> None:
    """Install the durable guard and replacement-aware integrity triggers."""
    conn.execute("""CREATE TABLE IF NOT EXISTS managed_runtime_activation_guards (
        user_id TEXT PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
        job_id TEXT NOT NULL UNIQUE,
        lease_token TEXT NOT NULL
    )""")
    conn.execute("DROP TRIGGER IF EXISTS validate_managed_runtime_runner_update")
    conn.execute("DROP TRIGGER IF EXISTS protect_linked_managed_runner_update")
    conn.executescript("""CREATE TRIGGER validate_managed_runtime_runner_update
    BEFORE UPDATE OF user_id, runner_id ON managed_runtimes
    WHEN NOT EXISTS (
        SELECT 1 FROM user_runners
        WHERE id = NEW.runner_id AND user_id = NEW.user_id AND kind = 'managed'
    )
    AND NOT EXISTS (
        SELECT 1 FROM managed_runtime_activation_guards WHERE user_id = NEW.user_id
    )
    """)

```

```

        BEGIN
            SELECT RAISE(ABORT, 'managed runtime must reference matching managed runner')
        END;
        CREATE TRIGGER protect_linked_managed_runner_update
        BEFORE UPDATE OF user_id, kind ON user_runners
        WHEN EXISTS (SELECT 1 FROM managed_runtimes WHERE runner_id = OLD.id)
            AND (NEW.user_id != OLD.user_id OR NEW.kind != OLD.kind)
            AND NOT EXISTS (
                SELECT 1 FROM managed_runtime_activation_guards WHERE user_id = OLD.user_id
            )
        BEGIN
            SELECT RAISE(ABORT, 'cannot change linked managed runtime runner');
        END;""")
    conn.commit()

```

```

def _migrate_managed_backup_archive_statuses(conn: sqlite3.Connection) -> None:
    """Expand existing archive status checks to include deleted tombstones."""
    schema_row = conn.execute(
        "SELECT sql FROM sqlite_master WHERE type = 'table' " "AND name = 'managed_backup_ar"
    ).fetchone()
    if schema_row is None or "'deleted'" in str(schema_row["sql"]):
        return
    conn.commit()
    conn.execute("PRAGMA foreign_keys = OFF")
    conn.execute("PRAGMA legacy_alter_table = ON")
    try:
        conn.execute("ALTER TABLE managed_backup_archives RENAME TO managed_backup_archives_")
        conn.execute("""CREATE TABLE managed_backup_archives (
            id TEXT PRIMARY KEY,
            user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
            runtime_generation INTEGER NOT NULL CHECK (runtime_generation > 0),
            status TEXT NOT NULL CHECK (
                status IN (
                    'creating', 'uploaded', 'ready', 'failed', 'deleting', 'deleted'
                )
            ),
            object_key TEXT NOT NULL UNIQUE,
            object_version TEXT,
            size_bytes INTEGER CHECK (size_bytes IS NULL OR size_bytes > 0),
            sha256 TEXT CHECK (sha256 IS NULL OR length(sha256) = 64),
            wrapped_key BLOB NOT NULL,
            key_id TEXT NOT NULL,
            owner_digest TEXT NOT NULL CHECK (length(owner_digest) = 64),
            created_at TEXT NOT NULL,
            completed_at TEXT,

```

```

        last_error TEXT CHECK (last_error IS NULL OR length(last_error) <= 1000)
    )""")
conn.execute("""INSERT INTO managed_backup_archives
    SELECT * FROM managed_backup_archives_old""")
conn.execute("DROP TABLE managed_backup_archives_old")
conn.execute(
    "CREATE INDEX idx_managed_backup_archives_user_created "
    "ON managed_backup_archives(user_id, created_at DESC)"
)
violation = conn.execute("PRAGMA foreign_key_check").fetchone()
if violation is not None:
    raise sqlite3.IntegrityError("Managed backup archive migration left an invalid l
conn.commit()
except sqlite3.Error:
    conn.rollback()
    raise
finally:
    conn.execute("PRAGMA legacy_alter_table = OFF")
    conn.execute("PRAGMA foreign_keys = ON")

def _migrate_managed_backup_operation_columns(conn: sqlite3.Connection) -> None:
    """Add resumable maintenance fields to existing managed backup tables."""
    columns = {row[1] for row in conn.execute("PRAGMA table_info(managed_backup_operations)")}
    if not columns:
        return
    migrations = {
        "phase": "ALTER TABLE managed_backup_operations ADD COLUMN phase TEXT NOT NULL DEFAULT",
        "lease_owner": "ALTER TABLE managed_backup_operations ADD COLUMN lease_owner TEXT",
        "lease_token": "ALTER TABLE managed_backup_operations ADD COLUMN lease_token TEXT",
        "lease_expires_at": (
            "ALTER TABLE managed_backup_operations ADD COLUMN lease_expires_at TEXT"
        ),
        "attempt_count": (
            "ALTER TABLE managed_backup_operations "
            "ADD COLUMN attempt_count INTEGER NOT NULL DEFAULT 0"
        ),
        "next_attempt_at": (
            "ALTER TABLE managed_backup_operations ADD COLUMN next_attempt_at TEXT"
        ),
        "source_runner_id": (
            "ALTER TABLE managed_backup_operations ADD COLUMN source_runner_id TEXT"
        ),
        "source_sprite_id": (
            "ALTER TABLE managed_backup_operations ADD COLUMN source_sprite_id TEXT"
        ),
    },

```

```

        "source_lost": (
            "ALTER TABLE managed_backup_operations "
            "ADD COLUMN source_lost INTEGER NOT NULL DEFAULT 0 CHECK (source_lost IN (0, 1))",
        ),
        "candidate_runner_id": (
            "ALTER TABLE managed_backup_operations ADD COLUMN candidate_runner_id TEXT",
        ),
        "candidate_sprite_id": (
            "ALTER TABLE managed_backup_operations ADD COLUMN candidate_sprite_id TEXT",
        ),
        "activation_generation": (
            "ALTER TABLE managed_backup_operations ADD COLUMN activation_generation INTEGER",
        ),
        "failure_class": "ALTER TABLE managed_backup_operations ADD COLUMN failure_class TEXT",
    }
    for column_name, statement in migrations.items():
        if column_name not in columns:
            conn.execute(statement)
    conn.execute("""
        UPDATE managed_backup_operations
        SET source_runner_id = (
            SELECT runtime.runner_id
            FROM managed_runtimes AS runtime
            WHERE runtime.user_id = managed_backup_operations.user_id
              AND runtime.generation = managed_backup_operations.runtime_generation
        ),
        source_sprite_id = (
            SELECT runtime.sprite_external_id
            FROM managed_runtimes AS runtime
            WHERE runtime.user_id = managed_backup_operations.user_id
              AND runtime.generation = managed_backup_operations.runtime_generation
        )
        WHERE status = 'running'
          AND source_runner_id IS NULL
          AND source_sprite_id IS NULL
          AND EXISTS (
            SELECT 1
            FROM managed_runtimes AS runtime
            WHERE runtime.user_id = managed_backup_operations.user_id
              AND runtime.generation = managed_backup_operations.runtime_generation
          )
    """)
    conn.commit()

```

```

def _migrate_managed_sprite_identity_ownership(conn: sqlite3.Connection) -> None:

```

```

        """Keep provider ownership after the related user row is removed."""
foreign_keys = conn.execute("PRAGMA foreign_key_list(managed_sprite_identities)").fetchall()
if not foreign_keys:
    return
conn.commit()
conn.execute("PRAGMA foreign_keys = OFF")
conn.execute("PRAGMA legacy_alter_table = ON")
try:
    conn.execute(
        "ALTER TABLE managed_sprite_identities " "RENAME TO managed_sprite_identities_old"
    )
    conn.execute("""CREATE TABLE managed_sprite_identities (
        sprite_name TEXT PRIMARY KEY,
        provider_name TEXT NOT NULL CHECK (provider_name = 'fly_sprites'),
        identity_kind TEXT NOT NULL CHECK (
            identity_kind IN ('runtime', 'restore_candidate')
        ),
        user_id TEXT NOT NULL,
        job_id TEXT,
        lifecycle_status TEXT NOT NULL CHECK (
            lifecycle_status IN ('creating', 'active', 'retired', 'deleting')
        ),
        created_at TEXT NOT NULL,
        updated_at TEXT NOT NULL,
        CHECK (
            (identity_kind = 'runtime' AND job_id IS NULL)
            OR (identity_kind = 'restore_candidate' AND job_id IS NOT NULL)
        )
    )""")
    conn.execute("""INSERT INTO managed_sprite_identities
        SELECT * FROM managed_sprite_identities_old""")
    conn.execute("DROP TABLE managed_sprite_identities_old")
    violation = conn.execute("PRAGMA foreign_key_check").fetchone()
    if violation is not None:
        raise sqlite3.IntegrityError("Managed Sprite identity migration left an invalid state")
    conn.commit()
except sqlite3.Error:
    conn.rollback()
    raise
finally:
    conn.execute("PRAGMA legacy_alter_table = OFF")
    conn.execute("PRAGMA foreign_keys = ON")

def _backfill_managed_sprite_identities(conn: sqlite3.Connection) -> None:
    """Register only provider identities already bound to durable control state."""

```

```

conn.execute("""INSERT OR IGNORE INTO managed_sprite_identities
(sprite_name, provider_name, identity_kind, user_id, job_id,
lifecycle_status, created_at, updated_at)
SELECT sprite_external_id, provider_name, 'runtime', user_id, NULL,
CASE WHEN lifecycle_status = 'ready' THEN 'active' ELSE 'creating' END,
created_at, updated_at
FROM managed_runtimes WHERE provider_name = 'fly_sprites'""")
conn.execute("""INSERT OR IGNORE INTO managed_sprite_identities
(sprite_name, provider_name, identity_kind, user_id, job_id,
lifecycle_status, created_at, updated_at)
SELECT candidate_sprite_id, 'fly_sprites', 'restore_candidate', user_id, job_id,
'creating', started_at, updated_at
FROM managed_backup_operations
WHERE operation = 'restore' AND status = 'running'
AND candidate_sprite_id IS NOT NULL""")

def _migrate_runner_kinds(conn: sqlite3.Connection) -> None:
    """Rebuild the runner table so one BYOC and one managed runner can coexist."""
    runner_columns = {row[1] for row in conn.execute("PRAGMA table_info(user_runners)")}
    if not runner_columns or "kind" in runner_columns:
        return

    logger.info("Control migration: adding runner kinds to user_runners")
    conn.commit()
    conn.execute("PRAGMA foreign_keys = OFF")
    try:
        conn.execute("BEGIN IMMEDIATE")
        conn.execute("DROP TRIGGER IF EXISTS update_user_runners_updated_at")
        conn.execute("DROP TRIGGER IF EXISTS validate_managed_runtime_runner_insert")
        conn.execute("DROP TRIGGER IF EXISTS validate_managed_runtime_runner_update")
        conn.execute("DROP TRIGGER IF EXISTS protect_linked_managed_runner_update")
        conn.execute("""
            CREATE TABLE user_runners_with_kinds (
                id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
                updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
                user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
                kind TEXT DEFAULT 'byoc' NOT NULL CHECK (kind IN ('byoc', 'managed')),
                name TEXT NOT NULL,
                cloud_provider TEXT NOT NULL,
                region TEXT NOT NULL,
                status TEXT DEFAULT 'pending' NOT NULL,
                registration_token_hash TEXT,
                registration_token_expires_at TEXT,
                runner_token_hash TEXT,
            )
        """)
    
```



```

        registered_at TEXT,
        last_heartbeat_at TEXT,
        runner_version TEXT,
        capabilities_json TEXT DEFAULT '{}' NOT NULL,
        data_dir TEXT,
        revoked_at TEXT,
        noise_public_key TEXT,
        noise_public_key_confirmed_at TEXT,
        restore_job_id TEXT,
        UNIQUE(user_id, kind)
    )
    """)
conn.execute("""
    INSERT INTO user_runners_with_kinds (
        id, created_at, updated_at, user_id, kind, name, cloud_provider,
        region, status, registration_token_hash,
        registration_token_expires_at, runner_token_hash, registered_at,
        last_heartbeat_at, runner_version, capabilities_json, data_dir,
        revoked_at, noise_public_key, noise_public_key_confirmed_at,
        restore_job_id
    )
    SELECT
        id, created_at, updated_at, user_id, 'byoc', name, cloud_provider,
        region, status, registration_token_hash,
        registration_token_expires_at, runner_token_hash, registered_at,
        last_heartbeat_at, runner_version, capabilities_json, data_dir,
        revoked_at, noise_public_key, noise_public_key_confirmed_at,
        restore_job_id
    FROM user_runners
    """)
conn.execute("DROP TABLE user_runners")
conn.execute("ALTER TABLE user_runners_with_kinds RENAME TO user_runners")
conn.execute("CREATE INDEX idx_user_runners_user ON user_runners(user_id)")
conn.execute(
    "CREATE INDEX idx_user_runners_registration_token "
    "ON user_runners(registration_token_hash)"
)
conn.execute(
    "CREATE INDEX idx_user_runners_runner_token ON user_runners(runner_token_hash)"
)
conn.execute("""
    CREATE TRIGGER update_user_runners_updated_at
    AFTER UPDATE ON user_runners
    BEGIN
        UPDATE user_runners SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
    END
""")

```

```

        """
    foreign_key_issue = conn.execute("PRAGMA foreign_key_check").fetchone()
    if foreign_key_issue is not None:
        raise sqlite3.IntegrityError("Runner kind migration left an invalid foreign key")
    conn.commit()
except sqlite3.Error:
    conn.rollback()
    raise
finally:
    conn.execute("PRAGMA foreign_keys = ON")

def _migrate_managed_restore_runner_kind(conn: sqlite3.Connection) -> None:
    """Expand existing runner kinds while preserving rows and foreign keys."""
    schema_row = conn.execute(
        "SELECT sql FROM sqlite_master WHERE type = 'table' AND name = 'user_runners'"
    ).fetchone()
    if schema_row is None or "managed_retired" in str(schema_row["sql"]):
        return
    conn.commit()
    conn.execute("PRAGMA foreign_keys = OFF")
    try:
        conn.execute("BEGIN IMMEDIATE")
        conn.execute("DROP TRIGGER IF EXISTS update_user_runners_updated_at")
        conn.execute("""CREATE TABLE user_runners_expanded (
            id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
            updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
            user_id TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
            kind TEXT DEFAULT 'byoc' NOT NULL CHECK (
                kind IN ('byoc', 'managed', 'managed_restore', 'managed_retired')
            ),
            name TEXT NOT NULL, cloud_provider TEXT NOT NULL, region TEXT NOT NULL,
            status TEXT DEFAULT 'pending' NOT NULL, registration_token_hash TEXT,
            registration_token_expires_at TEXT, runner_token_hash TEXT,
            registered_at TEXT, last_heartbeat_at TEXT, runner_version TEXT,
            capabilities_json TEXT DEFAULT '{}' NOT NULL, data_dir TEXT,
            revoked_at TEXT, noise_public_key TEXT,
            noise_public_key_confirmed_at TEXT, restore_job_id TEXT,
            UNIQUE(user_id, kind)
        )""")
        conn.execute("""INSERT INTO user_runners_expanded
            SELECT * FROM user_runners""")
        conn.execute("DROP TABLE user_runners")
        conn.execute("ALTER TABLE user_runners_expanded RENAME TO user_runners")
        conn.execute("CREATE INDEX idx_user_runners_user ON user_runners(user_id)")
    
```

```

        conn.execute(
            "CREATE INDEX idx_user_runners_registration_token "
            "ON user_runners(registration_token_hash)"
        )
        conn.execute(
            "CREATE INDEX idx_user_runners_runner_token ON user_runners(runner_token_hash)"
        )
        conn.execute("""CREATE TRIGGER update_user_runners_updated_at
            AFTER UPDATE ON user_runners BEGIN
                UPDATE user_runners SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id;
            END""")
        if conn.execute("PRAGMA foreign_key_check").fetchone() is not None:
            raise sqlite3.IntegrityError(
                "Managed restore runner migration left an invalid foreign key"
            )
        conn.commit()
    except sqlite3.Error:
        conn.rollback()
        raise
    finally:
        conn.execute("PRAGMA foreign_keys = ON")

def _migrate_encrypted_control_fields(conn: sqlite3.Connection) -> None:
    """Encrypt sensitive control-plane text fields when field encryption is active."""
    settings = get_settings()
    if not control_field_encryption_enabled(settings):
        return

    from yinshi.services.control_encryption import encrypt_control_text
    from yinshi.services.crypto import is_encrypted_text

    user_settings_rows = conn.execute(
        "SELECT user_id, pi_settings_json FROM user_settings"
    ).fetchall()
    for row in user_settings_rows:
        stored_value = row["pi_settings_json"]
        if isinstance(stored_value, str) and not is_encrypted_text(stored_value):
            conn.execute(
                "UPDATE user_settings SET pi_settings_json = ? WHERE user_id = ?",
                (
                    encrypt_control_text(
                        "user_settings.pi_settings_json",
                        row["user_id"],
                        stored_value,
                    ),
                ),
            )

```

```

        row["user_id"],
    ),
)

pi_config_rows = conn.execute(
    "SELECT id, user_id, source_label, repo_url, error_message FROM pi_configs"
).fetchall()
for row in pi_config_rows:
    updates: list[str] = []
    values: list[object] = []
    for field_name in ("source_label", "repo_url", "error_message"):
        stored_value = row[field_name]
        if stored_value is None:
            continue
        if isinstance(stored_value, str) and not is_encrypted_text(stored_value):
            updates.append(f"{field_name} = ?")
            values.append(
                encrypt_control_text(
                    f"pi_configs.{field_name}",
                    row["user_id"],
                    stored_value,
                )
            )
    if updates:
        values.append(row["id"])
        conn.execute(
            f"UPDATE pi_configs SET {' '.join(updates)} WHERE id = ?", # noqa: S608
            values,
        )
conn.commit()

def init_control_db() -> None:
    """Initialize the control plane database schema."""
    settings = get_settings()
    Path(settings.control_db_path).parent.mkdir(parents=True, exist_ok=True)
    logger.info("Initializing control database")
    try:
        with get_control_db() as conn:
            conn.executescript(CONTROL_SCHEMA_SQL)
            _migrate_control(conn)
    except sqlite3.Error:
        logger.error("Failed to initialize control database")
        raise
    logger.info("Control database initialized")

```

4.2 tenant.py

Tenant management maps a signed-in user to a private data directory and an optional SQLCipher-backed SQLite database. The root chunk below tangles back to backend/src/yinshi/tenant.py.

```
"""Multi-tenant context and per-user database management."""

from __future__ import annotations

import errno
import fcntl
import importlib
import logging
import os
import sqlite3
import stat
import threading
from collections.abc import Iterator
from contextlib import contextmanager
from dataclasses import dataclass
from pathlib import Path
from types import ModuleType
from typing import Final, cast

from yinshi.config import (
    get_settings,
    tenant_db_encryption_enabled,
    tenant_db_encryption_required,
    user_data_encryption_required,
)
from yinshi.db import _open_connection
from yinshi.model_catalog import DEFAULT_SESSION_MODEL
from yinshi.services.crypto import derive_subkey

logger = logging.getLogger(__name__)

_SQLCIPHER_MODULE_NAMES: Final[tuple[str, ...]] = (
    "sqlcipher3.dbapi2",
    "pysqlcipher3.dbapi2",
)

_STORAGE_ENCRYPTION_MARKER: Final[str] = ".yinshi-encrypted-storage"
_SCHEMA_OBJECT_TYPES: Final[tuple[str, ...]] = ("index", "table", "trigger", "view")
_MIGRATION_LOCK_SUFFIX: Final[str] = ".migration.lock"
_PLAINTEXT_ROLLBACK_SUFFIX: Final[str] = ".plaintext.rollback"
_MIGRATION_THREAD_LOCKS: Final[dict[str, threading.Lock]] = {}
```

```

_MIGRATION_THREAD_LOCKS_GUARD: Final[threading.Lock] = threading.Lock()

@dataclass
class TenantContext:
    """Per-request tenant context resolved from authentication."""

    user_id: str
    email: str
    data_dir: str
    db_path: str

def user_data_dir(base_dir: str, user_id: str) -> str:
    """Compute the data directory for a user, using a 2-char prefix."""
    prefix = user_id[:2]
    return os.path.join(base_dir, prefix, user_id)

def validate_user_path(tenant: TenantContext, path: str) -> None:
    """Validate that a path is within the tenant's data directory.

    Raises ValueError if the path is outside the tenant's data_dir.
    """
    resolved = os.path.realpath(path)
    data_dir = os.path.realpath(tenant.data_dir)
    if not resolved.startswith(data_dir + os.sep) and resolved != data_dir:
        raise ValueError(f"Path {path} is outside tenant data directory")

# User DB schema -- identical to main schema but WITHOUT owner_email
USER_SCHEMA_SQL = f"""
PRAGMA journal_mode = WAL;

CREATE TABLE IF NOT EXISTS repos (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    name TEXT NOT NULL,
    remote_url TEXT,
    root_path TEXT NOT NULL,
    custom_prompt TEXT,
    agents_md TEXT,
    installation_id INTEGER
);

```

```

CREATE TABLE IF NOT EXISTS workspaces (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    repo_id TEXT NOT NULL REFERENCES repos(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    branch TEXT NOT NULL,
    path TEXT NOT NULL,
    state TEXT DEFAULT 'ready' NOT NULL
);

CREATE TABLE IF NOT EXISTS sessions (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    workspace_id TEXT NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    status TEXT DEFAULT 'idle' NOT NULL,
    model TEXT DEFAULT '{DEFAULT_SESSION_MODEL}',
    pi_context_version INTEGER DEFAULT 1 NOT NULL
);

CREATE TABLE IF NOT EXISTS messages (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    session_id TEXT NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    role TEXT NOT NULL,
    content TEXT,
    full_message TEXT,
    turn_id TEXT,
    turn_status TEXT
);

CREATE TABLE IF NOT EXISTS prompt_runs (
    id TEXT PRIMARY KEY DEFAULT (lower(hex(randomblob(16)))),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    session_id TEXT NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    idempotency_key TEXT NOT NULL,
    status TEXT DEFAULT 'starting' NOT NULL CHECK (
        status IN ('starting', 'running', 'stopping', 'completed', 'failed', 'cancelled', 'finished')
    ),
    UNIQUE(session_id, idempotency_key)
);

CREATE TABLE IF NOT EXISTS prompt_events (
    run_id TEXT NOT NULL REFERENCES prompt_runs(id) ON DELETE CASCADE,

```

```

        sequence INTEGER NOT NULL CHECK (sequence >= 0),
        event_json TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
        PRIMARY KEY(run_id, sequence)
    );

CREATE UNIQUE INDEX IF NOT EXISTS idx_prompt_runs_active_session
    ON prompt_runs(session_id)
    WHERE status IN ('starting', 'running', 'stopping');
CREATE INDEX IF NOT EXISTS idx_prompt_events_run
    ON prompt_events(run_id, sequence);
CREATE INDEX IF NOT EXISTS idx_messages_session ON messages(session_id, created_at);
CREATE INDEX IF NOT EXISTS idx_messages_turn_id ON messages(turn_id);
CREATE INDEX IF NOT EXISTS idx_sessions_workspace ON sessions(workspace_id);
CREATE INDEX IF NOT EXISTS idx_workspaces_repo ON workspaces(repo_id);

CREATE TRIGGER IF NOT EXISTS update_repos_updated_at AFTER UPDATE ON repos
BEGIN UPDATE repos SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;

CREATE TRIGGER IF NOT EXISTS update_workspaces_updated_at AFTER UPDATE ON workspaces
BEGIN UPDATE workspaces SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;

CREATE TRIGGER IF NOT EXISTS update_sessions_updated_at AFTER UPDATE ON sessions
BEGIN UPDATE sessions SET updated_at = CURRENT_TIMESTAMP WHERE id = NEW.id; END;
"""

def _migrate_user_db(conn: sqlite3.Connection) -> None:
    """Apply forward-only schema fixes for existing per-user databases."""
    repo_columns = [row[1] for row in conn.execute("PRAGMA table_info(repos)").fetchall()]
    if "installation_id" not in repo_columns:
        conn.execute("ALTER TABLE repos ADD COLUMN installation_id INTEGER")

    message_columns = [row[1] for row in conn.execute("PRAGMA table_info(messages)").fetchall()]
    if "turn_status" not in message_columns:
        conn.execute("ALTER TABLE messages ADD COLUMN turn_status TEXT")

    # agents_md column for repo-level AGENTS.md override
    repo_columns = [row[1] for row in conn.execute("PRAGMA table_info(repos)").fetchall()]
    if "agents_md" not in repo_columns:
        conn.execute("ALTER TABLE repos ADD COLUMN agents_md TEXT")

    session_columns = [row[1] for row in conn.execute("PRAGMA table_info(sessions)").fetchall()]
    if "pi_context_version" not in session_columns:
        conn.execute(
            "ALTER TABLE sessions ADD COLUMN pi_context_version INTEGER DEFAULT 0 NOT NULL"
        )

```



```

    )

    conn.commit()

def _ensure_user_db_schema(conn: sqlite3.Connection) -> None:
    """Create missing tables and apply migrations for a per-user database."""
    conn.executescript(USER_SCHEMA_SQL)
    _migrate_user_db(conn)

def _load_sqlcipher_module() -> ModuleType:
    """Load an installed SQLCipher DB-API module or raise a clear error."""
    import_errors: list[str] = []
    for module_name in _SQLCIPHER_MODULE_NAMES:
        try:
            module = importlib.import_module(module_name)
        except ImportError as exc:
            import_errors.append(f"{module_name}: {exc}")
            continue
        if not hasattr(module, "connect"):
            import_errors.append(f"{module_name}: missing connect")
            continue
        if not hasattr(module, "Row"):
            import_errors.append(f"{module_name}: missing Row")
            continue
        return module
    joined_errors = "; ".join(import_errors) or "no SQLCipher module candidates configured"
    raise RuntimeError(
        "TENANT_DB_ENCRYPTION requires sqlcipher3 or pysqlcipher3. "
        f"Import failures: {joined_errors}"
    )

def _tenant_database_key(tenant: TenantContext) -> bytes:
    """Derive the SQLCipher key for one tenant database from the user's DEK."""
    if tenant is None:
        raise ValueError("tenant is required when tenant DB encryption is enabled")
    from yinshi.services.keys import get_user_dek

    user_dek = get_user_dek(tenant.user_id)
    return derive_subkey(user_dek, purpose="tenant-sqlcipher", context=tenant.user_id)

def _open_sqlcipher_connection(db_path: str, sqlcipher_key: bytes) -> sqlite3.Connection:
    """Open a SQLCipher-backed SQLite connection and validate the key immediately."""

```

```

if not isinstance(db_path, str):
    raise TypeError("db_path must be a string")
if not db_path.strip():
    raise ValueError("db_path must not be empty")
if not isinstance(sqlcipher_key, bytes):
    raise TypeError("sqlcipher_key must be bytes")
if len(sqlcipher_key) != 32:
    raise ValueError("sqlcipher_key must be exactly 32 bytes")

Path(db_path).parent.mkdir(parents=True, exist_ok=True)
sqlcipher_module = _load_sqlcipher_module()
conn = cast(sqlite3.Connection, sqlcipher_module.connect(db_path))
conn.row_factory = getattr(sqlcipher_module, "Row")
# The key is derived binary material, converted to hex locally, and never
# includes user-controlled SQL. SQLCipher requires PRAGMA key syntax.
conn.execute(f"PRAGMA key = \"x'{sqlcipher_key.hex()}'\"") # noqa: S608
conn.execute("PRAGMA foreign_keys = ON")
conn.execute("PRAGMA busy_timeout = 5000")
sqlcipher_database_error = getattr(sqlcipher_module, "DatabaseError", sqlite3.DatabaseError)
if not isinstance(sqlcipher_database_error, type):
    sqlcipher_database_error = sqlite3.DatabaseError
elif not issubclass(sqlcipher_database_error, Exception):
    sqlcipher_database_error = sqlite3.DatabaseError
try:
    conn.execute("SELECT count(*) FROM sqlite_master").fetchone()
except (sqlite3.DatabaseError, sqlcipher_database_error) as exc:
    conn.close()
    raise RuntimeError("Tenant database could not be opened with the configured key") from exc
return conn

def _plaintext_database_readable(db_path: str) -> bool:
    """Return whether a database can be opened by plaintext stdlib SQLite."""
    if not os.path.exists(db_path):
        return False
    try:
        conn = _open_connection(db_path)
        try:
            conn.execute("SELECT count(*) FROM sqlite_master").fetchone()
            return True
        finally:
            conn.close()
    except sqlite3.DatabaseError:
        return False

```

```

def _copy_plaintext_user_database(source_path: str, target_path: str, sqlcipher_key: bytes)
    """Export a complete plaintext tenant DB into a SQLCipher database."""
    sqlcipher_module = _load_sqlcipher_module()
    source = cast(sqlite3.Connection, sqlcipher_module.connect(source_path))
    attached = False
    try:
        checkpoint = source.execute("PRAGMA wal_checkpoint(TRUNCATE)").fetchone()
        if (
            checkpoint is None
            or len(checkpoint) < 3
            or int(checkpoint[0]) != 0
            or int(checkpoint[1]) != int(checkpoint[2])
        ):
            raise RuntimeError("Tenant database WAL checkpoint did not complete")
        integrity_row = source.execute("PRAGMA integrity_check").fetchone()
        if integrity_row is None or str(integrity_row[0]).lower() != "ok":
            raise RuntimeError("Plaintext tenant database failed integrity validation")
        source.execute(
            "ATTACH DATABASE ? AS encrypted KEY ?",
            (target_path, f"x'{sqlcipher_key.hex()}'"),
        )
        attached = True
        source.execute("SELECT sqlcipher_export('encrypted')").fetchone()
    finally:
        try:
            if attached:
                source.execute("DETACH DATABASE encrypted")
        finally:
            source.close()

def _database_schema_objects(
    connection: sqlite3.Connection,
) -> list[tuple[str, str, str, str | None]]:
    """Return all application-defined SQLite schema objects in stable order."""
    placeholders = ", ".join("?" for _ in _SCHEMA_OBJECT_TYPES)
    rows = connection.execute(
        f"""SELECT type, name, tbl_name, sql FROM sqlite_master
            WHERE type IN ({placeholders}) AND name NOT LIKE 'sqlite_%'
            ORDER BY type, name""", # noqa: S608
        _SCHEMA_OBJECT_TYPES,
    ).fetchall()
    return [
        (str(row[0]), str(row[1]), str(row[2]), None if row[3] is None else str(row[3]))
        for row in rows
    ]

```

```

def _quote_sqlite_identifier(identifier: str) -> str:
    """Quote one SQLite identifier obtained from trusted schema metadata."""
    return ''' + identifier.replace("'", ''') + '''

def _database_table_rows(
    connection: sqlite3.Connection,
    table_name: str,
) -> Iterator[tuple[object, ...]]:
    """Yield rows from one table in primary-key or rowid order."""
    quoted_table = _quote_sqlite_identifier(table_name)
    columns = connection.execute(f"PRAGMA table_info({quoted_table})").fetchall() # noqa: S608
    primary_key_columns = sorted(
        ((int(row[5]), str(row[1])) for row in columns if int(row[5]) > 0),
        key=lambda item: item[0],
    )
    if primary_key_columns:
        ordering = ", ".join(
            _quote_sqlite_identifier(column_name) for _, column_name in primary_key_columns
        )
    else:
        ordering = "rowid"
    rows = connection.execute(f"SELECT * FROM {quoted_table} ORDER BY {ordering}") # noqa: S608
    for row in rows:
        yield tuple(row)

def _database_table_row_count(connection: sqlite3.Connection, table_name: str) -> int:
    """Return row count for one table."""
    quoted_table = _quote_sqlite_identifier(table_name)
    row = connection.execute(f"SELECT count(*) FROM {quoted_table}").fetchone() # noqa: S608
    if row is None:
        raise RuntimeError("Tenant database row count query failed")
    return int(row[0])

def _validate_export_matches_source(
    source_path: str,
    target_path: str,
    sqlcipher_key: bytes,
) -> None:
    """Require exported schema, row counts, values, and ordering to match source."""
    source = _open_connection(source_path)
    target = _open_sqlcipher_connection(target_path, sqlcipher_key)

```

```

try:
    source_schema = _database_schema_objects(source)
    if _database_schema_objects(target) != source_schema:
        raise RuntimeError("Encrypted tenant database export does not match source schema")
    table_names = [name for object_type, name, _, _ in source_schema if object_type == 'table']
    for table_name in table_names:
        source_count = _database_table_row_count(source, table_name)
        if _database_table_row_count(target, table_name) != source_count:
            raise RuntimeError("Encrypted tenant database export does not match source data")
        source_rows = _database_table_rows(source, table_name)
        target_rows = _database_table_rows(target, table_name)
        for source_row, target_row in zip(source_rows, target_rows, strict=True):
            if target_row != source_row:
                raise RuntimeError(
                    "Encrypted tenant database export does not match source data"
                )
    finally:
        source.close()
        target.close()

def _validate_encrypted_user_database(db_path: str, sqlcipher_key: bytes) -> None:
    """Require an encrypted database to open and pass SQLCipher integrity checks."""
    if not db_path:
        raise ValueError("db_path must not be empty")
    if len(sqlcipher_key) != 32:
        raise ValueError("sqlcipher_key must contain 32 bytes")
    connection = _open_sqlcipher_connection(db_path, sqlcipher_key)
    try:
        integrity_row = connection.execute("PRAGMA integrity_check").fetchone()
        if integrity_row is None or str(integrity_row[0]).lower() != "ok":
            raise RuntimeError("Encrypted tenant database failed integrity validation")
        connection.execute("SELECT count(*) FROM sqlite_master").fetchone()
    finally:
        connection.close()

def _migration_thread_lock(db_path: str) -> threading.Lock:
    """Return the process-local lock for one tenant database path."""
    canonical_path = os.path.realpath(os.path.abspath(db_path))
    with _MIGRATION_THREAD_LOCKS_GUARD:
        return _MIGRATION_THREAD_LOCKS.setdefault(canonical_path, threading.Lock())

@contextmanager
def _tenant_migration_lock(db_path: str) -> Iterator[None]:

```

```

"""Hold an owner-only advisory lock for tenant database migration work."""
if not db_path:
    raise ValueError("db_path must not be empty")
lock_path = f"{db_path}_{MIGRATION_LOCK_SUFFIX}"
open_flags = os.O_CREAT | os.O_RDWR
open_flags |= getattr(os, "O_CLOEXEC", 0)
open_flags |= getattr(os, "O_NOFOLLOW", 0)
thread_lock = _migration_thread_lock(db_path)

with thread_lock:
    try:
        lock_fd = os.open(lock_path, open_flags, 0o600)
    except OSError as exc:
        if exc.errno == errno.ELOOP:
            raise RuntimeError(
                "Tenant database migration lock path must not be a symlink"
            ) from exc
        raise RuntimeError("Tenant database migration lock could not be opened") from exc
    try:
        lock_stat = os.fstat(lock_fd)
        path_stat = os.lstat(lock_path)
        if stat.S_ISLNK(path_stat.st_mode):
            raise RuntimeError("Tenant database migration lock path must not be a symlink")
        if not stat.S_ISREG(lock_stat.st_mode) or not stat.S_ISREG(path_stat.st_mode):
            raise RuntimeError("Tenant database migration lock must be a regular file")
        if (lock_stat.st_dev, lock_stat.st_ino) != (
            path_stat.st_dev,
            path_stat.st_ino,
        ):
            raise RuntimeError("Tenant database migration lock path changed during open")
        if lock_stat.st_uid != os.geteuid():
            raise RuntimeError("Tenant database migration lock must be owned by this user")
        os.fchmod(lock_fd, 0o600)
        fcntl.flock(lock_fd, fcntl.LOCK_EX)
        locked_path_stat = os.lstat(lock_path)
        if stat.S_ISLNK(locked_path_stat.st_mode) or (
            lock_stat.st_dev,
            lock_stat.st_ino,
        ) != (locked_path_stat.st_dev, locked_path_stat.st_ino):
            raise RuntimeError("Tenant database migration lock path changed while waiting")
        yield
    finally:
        try:
            fcntl.flock(lock_fd, fcntl.LOCK_UN)
        finally:
            os.close(lock_fd)

```

```

def _fsync_file(path: str) -> None:
    """Flush one regular file through its filesystem."""
    flags = os.O_RDONLY | getattr(os, "O_CLOEXEC", 0)
    descriptor = os.open(path, flags)
    try:
        os.fsync(descriptor)
    finally:
        os.close(descriptor)

def _fsync_parent_directory(path: str) -> None:
    """Flush directory entries for one filesystem path."""
    parent_path = os.path.dirname(os.path.abspath(path))
    flags = os.O_RDONLY | getattr(os, "O_CLOEXEC", 0)
    flags |= getattr(os, "O_DIRECTORY", 0)
    descriptor = os.open(parent_path, flags)
    try:
        os.fsync(descriptor)
    finally:
        os.close(descriptor)

def _create_private_rollback_copy(source_path: str, rollback_path: str) -> None:
    """Copy a plaintext database into a new owner-only rollback file."""
    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL
    flags |= getattr(os, "O_CLOEXEC", 0)
    flags |= getattr(os, "O_NOFOLLOW", 0)
    descriptor = os.open(rollback_path, flags, 0o600)
    try:
        with open(source_path, "rb") as source, os.fdopen(descriptor, "wb") as rollback:
            descriptor = -1
            while chunk := source.read(1024 * 1024):
                rollback.write(chunk)
            rollback.flush()
            os.fchmod(rollback.fileno(), 0o600)
    except Exception:
        if descriptor >= 0:
            os.close(descriptor)
        if os.path.exists(rollback_path):
            os.unlink(rollback_path)
        raise

def _remove_sqlite_sidecars(db_path: str) -> None:

```

```

"""Remove checkpointed WAL files and durably record their removal."""
removed = False
for suffix in ("-wal", "-shm"):
    sidecar_path = f"{db_path}{suffix}"
    if os.path.exists(sidecar_path):
        os.unlink(sidecar_path)
        removed = True
if removed:
    _fsync_parent_directory(db_path)

def _tenant_rollback_is_trusted(descriptor: int, rollback_path: str) -> bool:
    """Return whether an open rollback still names one private owner-controlled file."""
    opened_stat = os.fstat(descriptor)
    try:
        path_stat = os.lstat(rollback_path)
    except FileNotFoundError:
        return False
    return (
        stat.S_ISREG(opened_stat.st_mode)
        and stat.S_ISREG(path_stat.st_mode)
        and not stat.S_ISLNK(path_stat.st_mode)
        and (opened_stat.st_dev, opened_stat.st_ino) == (path_stat.st_dev, path_stat.st_ino)
        and opened_stat.st_uid == os.geteuid()
        and opened_stat.st_nlink == 1
        and opened_stat.st_mode & 0o077 == 0
    )

def _open_trusted_tenant_rollback(rollback_path: str) -> int | None:
    """Open and validate a tenant rollback without following links."""
    flags = os.O_RDONLY | getattr(os, "O_CLOEXEC", 0)
    flags |= getattr(os, "O_NOFOLLOW", 0)
    flags |= getattr(os, "O_NONBLOCK", 0)
    try:
        descriptor = os.open(rollback_path, flags)
    except FileNotFoundError:
        return None
    except OSError as exc:
        raise RuntimeError(
            "Tenant database migration rollback must be a trusted regular file"
        ) from exc
    if _tenant_rollback_is_trusted(descriptor, rollback_path):
        return descriptor
    os.close(descriptor)
    raise RuntimeError("Tenant database migration rollback must be a trusted regular file")

```



```

def _recover_plaintext_migration_rollback(db_path: str) -> None:
    """Restore a durable rollback when an interrupted replacement lost its primary."""
    rollback_path = f"{db_path}_{PLAINTEXT_ROLLBACK_SUFFIX}"
    if os.path.lexists(db_path):
        return
    descriptor = _open_trusted_tenant_rollback(rollback_path)
    if descriptor is None:
        return
    try:
        os.fsync(descriptor)
        if os.path.lexists(db_path):
            return
        if not _tenant_rollback_is_trusted(descriptor, rollback_path):
            raise RuntimeError("Tenant database migration rollback must be a trusted regular file")
        os.replace(rollback_path, db_path)
        os.chmod(db_path, 0o600)
        _fsync_file(db_path)
        _fsync_parent_directory(db_path)
    finally:
        os.close(descriptor)
    logger.warning("Recovered an interrupted tenant database migration")

def _remove_validated_migration_rollback(db_path: str) -> None:
    """Remove rollback only after the validated primary is durable."""
    rollback_path = f"{db_path}_{PLAINTEXT_ROLLBACK_SUFFIX}"
    if not os.path.exists(rollback_path):
        return
    _fsync_file(db_path)
    _fsync_parent_directory(db_path)
    os.unlink(rollback_path)
    _fsync_parent_directory(db_path)

def _migrate_plaintext_user_database(db_path: str, sqlcipher_key: bytes) -> None:
    """Replace a plaintext tenant DB without retaining the original copy."""
    if not _plaintext_database_readable(db_path):
        return
    rollback_path = f"{db_path}_{PLAINTEXT_ROLLBACK_SUFFIX}"
    temp_path = f"{db_path}.encrypted.tmp"
    temporary_paths = (temp_path, f"{temp_path}-wal", f"{temp_path}-shm")
    for stale_path in temporary_paths:
        if os.path.exists(stale_path):
            os.unlink(stale_path)

```

```

if os.path.exists(rollback_path):
    _fsync_file(db_path)
    _fsync_parent_directory(db_path)
    os.unlink(rollback_path)
    _fsync_parent_directory(db_path)
try:
    _copy_plaintext_user_database(db_path, temp_path, sqlcipher_key)
    _remove_sqlite_sidecars(db_path)
    _validate_encrypted_user_database(temp_path, sqlcipher_key)
    _validate_export_matches_source(db_path, temp_path, sqlcipher_key)
    os.chmod(temp_path, 0o600)
    _fsync_file(temp_path)
    _create_private_rollback_copy(db_path, rollback_path)
    _fsync_file(rollback_path)
    _fsync_parent_directory(db_path)
    os.replace(temp_path, db_path)
    _fsync_parent_directory(db_path)
    _validate_encrypted_user_database(db_path, sqlcipher_key)
    os.chmod(db_path, 0o600)
    _fsync_file(db_path)
    _fsync_parent_directory(db_path)
except Exception:
    if os.path.exists(rollback_path):
        os.replace(rollback_path, db_path)
        os.chmod(db_path, 0o600)
        _fsync_file(db_path)
        _fsync_parent_directory(db_path)
    for temporary_path in temporary_paths:
        if os.path.exists(temporary_path):
            os.unlink(temporary_path)
    raise
else:
    os.unlink(rollback_path)
    _fsync_parent_directory(db_path)
_remove_sqlite_sidecars(db_path)
logger.info("Migrated plaintext tenant database to encrypted storage")

def _remove_plaintext_migration_backups(db_path: str) -> None:
    """Remove legacy plaintext backups after the encrypted primary is validated."""
    if not db_path:
        raise ValueError("db_path must not be empty")
    database_path = Path(db_path)
    pattern = f"{database_path.name}.plaintext.*.bak"
    for backup_path in database_path.parent.glob(pattern):
        backup_path.unlink()

```

```

        logger.info("Removed legacy plaintext tenant database backup")

def _encrypted_storage_marker_exists(data_dir: str) -> bool:
    """Return whether operations marked a user directory as encrypted storage."""
    current_path = Path(data_dir).resolve()
    for candidate in (current_path, *current_path.parents):
        marker_path = candidate / _STORAGE_ENCRYPTION_MARKER
        if marker_path.is_file():
            return True
    return False

def _ensure_user_data_encryption_marker(tenant: TenantContext) -> None:
    """Fail closed when configured encrypted user storage is absent."""
    settings = get_settings()
    if not user_data_encryption_required(settings):
        return
    if _encrypted_storage_marker_exists(tenant.data_dir):
        return
    raise RuntimeError(
        "USER_DATA_ENCRYPTION is required, but no .yinshi-encrypted-storage marker "
        "was found. Mount an fscrypt, LUKS, or encrypted volume first."
    )

def _open_user_connection(
    db_path: str,
    tenant: TenantContext | None,
) -> sqlite3.Connection:
    """Open a tenant database using SQLCipher when policy enables it."""
    settings = get_settings()
    if tenant is not None:
        _ensure_user_data_encryption_marker(tenant)
    encryption_enabled = tenant_db_encryption_enabled(settings)
    encryption_required = tenant_db_encryption_required(settings)
    with _tenant_migration_lock(db_path):
        _recover_plaintext_migration_rollback(db_path)
        if not encryption_enabled:
            return _open_connection(db_path)
        if tenant is None:
            raise ValueError("tenant is required when tenant DB encryption is enabled")
        try:
            _load_sqlcipher_module()
        except RuntimeError:
            if encryption_required:

```

```

        raise
    logger.warning("SQLCipher unavailable; opening tenant database without encryption")
    return _open_connection(db_path)

sqlcipher_key = _tenant_database_key(tenant)
if os.path.exists(db_path):
    _migrate_plaintext_user_database(db_path, sqlcipher_key)
connection = _open_sqlcipher_connection(db_path, sqlcipher_key)
_remove_validated_migration_rollback(db_path)
_remove_plaintext_migration_backups(db_path)
return connection

def init_user_db(db_path: str, tenant: TenantContext | None = None) -> None:
    """Initialize a per-user SQLite database with the user schema."""
    conn = _open_user_connection(db_path, tenant)
    try:
        _ensure_user_db_schema(conn)
        if os.path.exists(db_path):
            os.chmod(db_path, 0o600)
    finally:
        conn.close()

@contextmanager
def get_user_db(tenant: TenantContext) -> Iterator[sqlite3.Connection]:
    """Get a SQLite connection to a user's database."""
    conn = _open_user_connection(tenant.db_path, tenant)
    try:
        _ensure_user_db_schema(conn)
        yield conn
    finally:
        conn.close()

```

5 Authentication and Session Resolution

Every protected request needs the same three answers: who is calling, which tenant owns the data, and whether the requested object belongs to that tenant. Middleware and dependencies answer those questions once so route handlers can stay direct.

5.1 backup.py

Backup tooling copies each SQLite database through its live connection and verifies each copy. It encrypts the validated archive before it leaves private

staging storage.

```
"""Encrypted, SQLCipher-aware backup creation and archive decryption."""
```

```
from __future__ import annotations

import argparse
import json
import os
import shutil
import sqlite3
import stat
import tarfile
import tempfile
from dataclasses import dataclass
from datetime import UTC, datetime
from pathlib import Path, PurePosixPath
from typing import BinaryIO

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.ciphers.base import CipherContext

from yinshi.config import get_settings, tenant_db_encryption_enabled
from yinshi.db import get_control_db
from yinshi.services.accounts import make_tenant
from yinshi.services.crypto import (
    derive_subkey,
    is_wrapped_dek_envelope,
    unwrap_dek,
    unwrap_dek_with_keks,
)
from yinshi.tenant import (
    TenantContext,
    _open_sqlcipher_connection,
    _tenant_database_key,
    _validate_encrypted_user_database,
    get_user_db,
)

_ARCHIVE_MAGIC = b"YINSHI-BACKUP-V1\n"
_BACKUP_CHUNK_BYTES = 1024 * 1024
_GCM_NONCE_BYTES = 12
_GCM_TAG_BYTES = 16
_MAX_ENCRYPTED_ARCHIVE_BYTES = 100 * 1024 * 1024 * 1024
_MAX_ARCHIVE_MEMBERS = 100_000
_MAX_ARCHIVE_MEMBER_BYTES = 32 * 1024 * 1024 * 1024
```

```

_MAX_ARCHIVE_TOTAL_BYTES = 100 * 1024 * 1024 * 1024
_MAX_MANIFEST_BYTES = 64 * 1024
_MANIFEST_KEYS = {"created_at", "format", "tenant_database_count"}
_MANAGED_FLY_BACKUP_ERROR = "Local backup commands are unavailable in managed Fly mode"

@dataclass(frozen=True)
class _RestoreArchive:
    """Validated archive metadata needed for restore."""

    tenant_members: dict[str, tarfile.TarInfo]
    control_member: tarfile.TarInfo

def _backup_key_from_settings() -> bytes:
    """Decode the separately managed 256-bit backup key."""
    settings = get_settings()
    if settings.backup_encryption_key is None:
        raise RuntimeError("BACKUP_ENCRYPTION_KEY must be configured for backups")
    encoded_key = settings.backup_encryption_key.get_secret_value().strip()
    try:
        key = bytes.fromhex(encoded_key)
    except ValueError as exc:
        raise RuntimeError("BACKUP_ENCRYPTION_KEY must be 64 hexadecimal characters") from exc
    if len(key) != 32:
        raise RuntimeError("BACKUP_ENCRYPTION_KEY must decode to exactly 32 bytes")
    return key

def _write_encrypted_chunks(
    source: BinaryIO,
    target: BinaryIO,
    encryptor: CipherContext,
) -> None:
    """Encrypt a source stream into a target stream with bounded memory."""
    if source.closed or target.closed:
        raise ValueError("source and target must be open")
    while True:
        chunk = source.read(_BACKUP_CHUNK_BYTES)
        if not chunk:
            break
        target.write(encryptor.update(chunk))
    target.write(encryptor.finalize())

def _encrypt_archive(source_path: Path, target_path: Path, key: bytes) -> None:

```

```

"""Encrypt one tar archive with AES-256-GCM and an authenticated header."""
if len(key) != 32:
    raise ValueError("key must contain exactly 32 bytes")
if not source_path.is_file():
    raise FileNotFoundError(source_path)
if target_path.exists():
    raise FileExistsError(target_path)

nonce = os.urandom(_GCM_NONCE_BYTES)
encryptor = Cipher(algorithms.AES(key), modes.GCM(nonce)).encryptor()
encryptor.authenticate_additional_data(_ARCHIVE_MAGIC)
file_descriptor = os.open(target_path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
try:
    with (
        source_path.open("rb") as source,
        os.fdopen(file_descriptor, "wb", closefd=False) as target,
    ):
        target.write(_ARCHIVE_MAGIC)
        target.write(nonce)
        _write_encrypted_chunks(source, target, encryptor)
        target.write(encryptor.tag)
        target.flush()
        os.fsync(target.fileno())
except Exception:
    target_path.unlink(missing_ok=True)
    raise
finally:
    os.close(file_descriptor)
os.chmod(target_path, 0o600)

def decrypt_backup_archive(source_path: Path, target_path: Path, key: bytes) -> None:
    """Decrypt and authenticate one backup into a tar archive."""
    if len(key) != 32:
        raise ValueError("key must contain exactly 32 bytes")
    if not source_path.is_file():
        raise FileNotFoundError(source_path)
    if target_path.exists():
        raise FileExistsError(target_path)

    source_size = source_path.stat().st_size
    if source_size > _MAX_ENCRYPTED_ARCHIVE_BYTES:
        raise ValueError("encrypted backup exceeds the restore size limit")
    minimum_size = len(_ARCHIVE_MAGIC) + _GCM_NONCE_BYTES + _GCM_TAG_BYTES
    if source_size <= minimum_size:
        raise ValueError("encrypted backup is truncated")

```

```

with source_path.open("rb") as source:
    magic = source.read(len(_ARCHIVE_MAGIC))
    if magic != _ARCHIVE_MAGIC:
        raise ValueError("encrypted backup header is invalid")
    nonce = source.read(_GCM_NONCE_BYTES)
    source.seek(-_GCM_TAG_BYTES, os.SEEK_END)
    tag = source.read(_GCM_TAG_BYTES)
    ciphertext_bytes = source_size - minimum_size
    source.seek(len(_ARCHIVE_MAGIC) + _GCM_NONCE_BYTES)

    decryptor = Cipher(algorithms.AES(key), modes.GCM(nonce, tag)).decryptor()
    decryptor.authenticate_additional_data(_ARCHIVE_MAGIC)
    file_descriptor = os.open(target_path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    try:
        with os.fdopen(file_descriptor, "wb", closefd=False) as target:
            remaining = ciphertext_bytes
            while remaining > 0:
                chunk = source.read(min(_BACKUP_CHUNK_BYTES, remaining))
                if not chunk:
                    raise ValueError("encrypted backup is truncated")
                remaining -= len(chunk)
                target.write(decryptor.update(chunk))
            target.write(decryptor.finalize())
            target.flush()
            os.fsync(target.fileno())
    except Exception:
        target_path.unlink(missing_ok=True)
        raise
    finally:
        os.close(file_descriptor)
os.chmod(target_path, 0o600)

def _backup_sqlite_connection(source: sqlite3.Connection, target_path: Path) -> None:
    """Write and validate one plaintext SQLite snapshot."""
    if target_path.exists():
        raise FileExistsError(target_path)
    target_path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    cipher_row = source.execute("PRAGMA cipher_version").fetchone()
    if cipher_row is not None and cipher_row[0]:
        attached = False
        try:
            source.execute(
                "ATTACH DATABASE ? AS backup_snapshot KEY ''",
                (str(target_path),),
            )

```



```

    )
    attached = True
    source.execute("SELECT sqlcipher_export('backup_snapshot').fetchone()
except Exception:
    target_path.unlink(missing_ok=True)
    raise
finally:
    if attached:
        source.execute("DETACH DATABASE backup_snapshot")
else:
    target = sqlite3.connect(target_path)
    try:
        source.backup(target)
        target.commit()
    finally:
        target.close()

target = sqlite3.connect(target_path)
try:
    integrity_row = target.execute("PRAGMA integrity_check").fetchone()
    if integrity_row is None or str(integrity_row[0]).lower() != "ok":
        raise RuntimeError("SQLite backup failed integrity validation")
finally:
    target.close()
for suffix in ("-wal", "-shm"):
    Path(f"{target_path}{suffix}").unlink(missing_ok=True)
os.chmod(target_path, 0o600)

def _backup_tenant_database(tenant: TenantContext, target_path: Path) -> None:
    """Snapshot one tenant database under its configured encryption policy."""
    settings = get_settings()
    target_path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    with get_user_db(tenant) as source:
        if tenant_db_encryption_enabled(settings):
            key = _tenant_database_key(tenant)
            target = _open_sqlcipher_connection(str(target_path), key)
            try:
                source.backup(target)
                target.commit()
                integrity_row = target.execute("PRAGMA integrity_check").fetchone()
                if integrity_row is None or str(integrity_row[0]).lower() != "ok":
                    raise RuntimeError("Encrypted tenant backup failed integrity validation")
            finally:
                target.close()
        for suffix in ("-wal", "-shm"):

```

```

        Path(f"{target_path}-{suffix}").unlink(missing_ok=True)
        os.chmod(target_path, 0o600)
    else:
        _backup_sqlite_connection(source, target_path)

def _purge_stale_staging(backup_directory: Path) -> None:
    """Remove abandoned private staging directories from interrupted backups."""
    for candidate in backup_directory.glob(".staging-*"):
        if candidate.is_symlink():
            candidate.unlink()
        elif candidate.is_dir():
            shutil.rmtree(candidate)
    for candidate in backup_directory.glob(".archive-*.tar.gz"):
        if candidate.is_file() and not candidate.is_symlink():
            candidate.unlink()

def create_backup() -> Path:
    """Create an encrypted control and tenant database backup archive."""
    settings = get_settings()
    if settings.managed_runtime_provider == "fly_sprites":
        raise RuntimeError(_MANAGED_FLY_BACKUP_ERROR)
    backup_key = _backup_key_from_settings()
    backup_directory = Path(settings.backup_dir).resolve()
    backup_directory.mkdir(mode=0o700, parents=True, exist_ok=True)
    os.chmod(backup_directory, 0o700)
    _purge_stale_staging(backup_directory)

    timestamp = datetime.now(UTC).strftime("%Y%m%dT%H%M%S%fZ")
    archive_path = backup_directory / f"yinshi-{timestamp}.tar.gz.enc"
    temporary_tar = backup_directory / f".archive-{timestamp}.tar.gz"
    previous_umask = os.umask(0o077)
    try:
        with tempfile.TemporaryDirectory(prefix=".staging-", dir=backup_directory) as staging_dir:
            staging_directory = Path(staging_dir)
            os.chmod(staging_directory, 0o700)
            with get_control_db() as control_database:
                control_database.execute("BEGIN IMMEDIATE")
            try:
                user_rows = control_database.execute(
                    "SELECT id, email FROM users ORDER BY id"
                ).fetchall()
            finally:
                with get_control_db() as snapshot_database:
                    _backup_sqlite_connection(
                        snapshot_database,

```

```

        staging_directory / "control.db",
    )

    tenant_count = 0
    for user_row in user_rows:
        tenant = make_tenant(str(user_row["id"]), str(user_row["email"]))
        if not Path(tenant.db_path).is_file():
            continue
        target_path = (
            staging_directory
            / "users"
            / tenant.user_id[:2]
            / tenant.user_id
            / "yinshi.db"
        )
        _backup_tenant_database(tenant, target_path)
        tenant_count += 1
    finally:
        control_database.rollback()

    manifest = {
        "created_at": datetime.now(UTC).isoformat(),
        "format": "yinshi-backup-v1",
        "tenant_database_count": tenant_count,
    }
    manifest_path = staging_directory / "manifest.json"
    manifest_path.write_text(
        json.dumps(manifest, indent=2, sort_keys=True) + "\n",
        encoding="utf-8",
    )
    os.chmod(manifest_path, 0o600)

    with tarfile.open(temporary_tar, mode="w:gz") as archive:
        for child in sorted(staging_directory.iterdir(), key=lambda path: path.name):
            archive.add(child, arcname=child.name, recursive=True)
    os.chmod(temporary_tar, 0o600)
    _encrypt_archive(temporary_tar, archive_path, backup_key)
    finally:
        temporary_tar.unlink(missing_ok=True)
        os.umask(previous_umask)

    return archive_path

def _read_member(archive: tarfile.TarFile, member: tarfile.TarInfo, limit: int) -> bytes:
    """Read one regular archive member under a strict byte limit."""

```

```

    if member.size > limit:
        raise ValueError(f"archive member exceeds size limit: {member.name}")
    source = archive.extractfile(member)
    if source is None:
        raise ValueError(f"archive member cannot be read: {member.name}")
    payload = source.read(limit + 1)
    if len(payload) != member.size or len(payload) > limit:
        raise ValueError(f"archive member has an invalid size: {member.name}")
    return payload

def _validate_manifest(archive: tarfile.TarFile, member: tarfile.TarInfo) -> int:
    """Require the exact version-one manifest object and field types."""
    try:
        manifest = json.loads(_read_member(archive, member, _MAX_MANIFEST_BYTES))
    except (UnicodeDecodeError, json.JSONDecodeError) as exc:
        raise ValueError("backup manifest is not valid UTF-8 JSON") from exc
    if not isinstance(manifest, dict) or set(manifest) != _MANIFEST_KEYS:
        raise ValueError("backup manifest must contain the exact v1 fields")
    if manifest["format"] != "yinshi-backup-v1":
        raise ValueError("backup manifest format is unsupported")
    created_at = manifest["created_at"]
    if not isinstance(created_at, str):
        raise ValueError("backup manifest created_at must be a timestamp")
    try:
        parsed_created_at = datetime.fromisoformat(created_at)
    except ValueError as exc:
        raise ValueError("backup manifest created_at must be an ISO timestamp") from exc
    if parsed_created_at.tzinfo is None:
        raise ValueError("backup manifest created_at must include a timezone")
    tenant_count = manifest["tenant_database_count"]
    if type(tenant_count) is not int or tenant_count < 0:
        raise ValueError("backup manifest tenant_database_count must be nonnegative")
    return tenant_count

def _tenant_id_from_member(name: str) -> str | None:
    """Return the tenant ID when a member follows the version-one database path."""
    parts = PurePosixPath(name).parts
    if len(parts) != 4 or parts[0] != "users" or parts[3] != "yinshi.db":
        return None
    prefix, user_id = parts[1], parts[2]
    if not user_id or prefix != user_id[:2]:
        return None
    return user_id

```

```

def _inspect_archive_members(tar_path: Path) -> _RestoreArchive:
    """Reject unsafe members and require the exact version-one archive layout."""
    members: dict[str, tarfile.TarInfo] = {}
    total_size = 0
    with tarfile.open(tar_path, mode="r:gz") as archive:
        for index, member in enumerate(archive, start=1):
            if index > _MAX_ARCHIVE_MEMBERS:
                raise ValueError("backup archive contains too many members")
            name = member.name
            parts = name.split("/")
            if (
                not name
                or name.startswith("/")
                or "\\\" in name
                or any(part in {"", ".", ".."} for part in parts)
            ):
                raise ValueError(f"backup archive contains an unsafe path: {name}")
            if name in members:
                raise ValueError(f"backup archive contains a duplicate path: {name}")
            if not member.isfile() and not member.isdir():
                raise ValueError(f"backup archive contains an unsafe member: {name}")
            if member.size < 0 or member.size > _MAX_ARCHIVE_MEMBER_BYTES:
                raise ValueError(f"backup archive member exceeds size limit: {name}")
            total_size += member.size
            if total_size > _MAX_ARCHIVE_TOTAL_BYTES:
                raise ValueError("backup archive exceeds the expanded size limit")
            members[name] = member

    file_members = {name: member for name, member in members.items() if member.isfile()}
    if "manifest.json" not in file_members or "control.db" not in file_members:
        raise ValueError("backup archive is missing required v1 files")
    tenant_members: dict[str, tarfile.TarInfo] = {}
    for name, member in file_members.items():
        if name in {"manifest.json", "control.db"}:
            continue
        user_id = _tenant_id_from_member(name)
        if user_id is None:
            raise ValueError(f"backup archive contains an unexpected file: {name}")
        if user_id in tenant_members:
            raise ValueError(f"backup archive contains duplicate tenant data: {user_id}")
        tenant_members[user_id] = member

    allowed_directories = {"users"}
    for member in tenant_members.values():
        path = PurePosixPath(member.name)

```

```

        allowed_directories.add(str(path.parent))
        allowed_directories.add(str(path.parent.parent))
    directory_names = {name for name, member in members.items() if member.isdir()}
    if not directory_names.issubset(allowed_directories):
        raise ValueError("backup archive contains unexpected directories")

    tenant_count = _validate_manifest(archive, file_members["manifest.json"])
    if tenant_count != len(tenant_members):
        raise ValueError("backup manifest tenant count does not match archive")
    return _RestoreArchive(
        tenant_members=tenant_members,
        control_member=file_members["control.db"],
    )

def _copy_archive_member(
    archive: tarfile.TarFile,
    member: tarfile.TarInfo,
    target: Path,
) -> None:
    """Copy one archive member with bounded memory and private permissions."""
    target.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    source = archive.extractfile(member)
    if source is None:
        raise ValueError(f"archive member cannot be read: {member.name}")
    descriptor = os.open(target, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    try:
        with os.fdopen(descriptor, "wb", closefd=False) as output:
            remaining = member.size
            while remaining:
                chunk = source.read(min(_BACKUP_CHUNK_BYTES, remaining))
                if not chunk:
                    raise ValueError(f"archive member is truncated: {member.name}")
                output.write(chunk)
                remaining -= len(chunk)
            output.flush()
            os.fsync(output.fileno())
    finally:
        os.close(descriptor)

def _validate_control_database(path: Path) -> None:
    """Require restored control database integrity before installation."""
    with path.open("rb") as database_file:
        if database_file.read(16) != b"SQLite format 3\x00":
            raise ValueError("control database is invalid")

```

```

try:
    database = sqlite3.connect(path)
    try:
        rows = database.execute("PRAGMA integrity_check").fetchall()
        if len(rows) != 1 or str(rows[0][0]).lower() != "ok":
            raise ValueError("control database failed SQLite integrity validation")
    finally:
        database.close()
except sqlite3.DatabaseError as exc:
    raise ValueError("control database is invalid") from exc

def _validate_plain_tenant_database(path: Path) -> None:
    """Require one staged plaintext tenant database to pass integrity checks."""
    with path.open("rb") as database_file:
        if database_file.read(16) != b"SQLite format 3\x00":
            raise ValueError("tenant database is invalid")
    try:
        database = sqlite3.connect(path)
        try:
            rows = database.execute("PRAGMA integrity_check").fetchall()
            if len(rows) != 1 or str(rows[0][0]).lower() != "ok":
                raise ValueError("tenant database failed SQLite integrity validation")
        finally:
            database.close()
    except sqlite3.DatabaseError as exc:
        raise ValueError("tenant database is invalid") from exc

def _staged_tenant_database_key(control_path: Path, user_id: str) -> bytes:
    """Derive one tenant database key from staged control data only."""
    database = sqlite3.connect(control_path)
    try:
        row = database.execute(
            "SELECT encrypted_dek FROM users WHERE id = ?",
            (user_id,),
        ).fetchone()
    finally:
        database.close()
    if row is None:
        raise ValueError("staged control database is missing the tenant user")
    wrapped_dek = row[0]
    if not isinstance(wrapped_dek, bytes) or not wrapped_dek:
        raise ValueError("staged control database is missing the tenant encryption key")

settings = get_settings()

```

```

try:
    if is_wrapped_dek_envelope(wrapped_dek):
        keyring = settings.key_encryption_keyring_previous
        current_kek = settings.key_encryption_key_bytes
        if current_kek:
            keyring[settings.key_encryption_key_id] = current_kek
            user_dek = unwrap_dek_with_keks(wrapped_dek, user_id, keyring)
        else:
            pepper = settings.encryption_pepper_bytes
            if not pepper:
                raise ValueError("legacy encryption pepper is unavailable")
            user_dek = unwrap_dek(wrapped_dek, user_id, pepper)
    except Exception:
        raise ValueError("staged tenant encryption key could not be unwrapped") from None
    return derive_subkey(
        user_dek,
        purpose="tenant-sqlcipher",
        context=user_id,
    )

def _validate_staged_tenant_database(
    path: Path,
    user_id: str,
    control_path: Path,
) -> None:
    """Validate one staged tenant under the configured encryption policy."""
    if not tenant_db_encryption_enabled(get_settings()):
        _validate_plain_tenant_database(path)
        return
    sqlcipher_key = _staged_tenant_database_key(control_path, user_id)
    try:
        _validate_encrypted_user_database(str(path), sqlcipher_key)
    except Exception:
        raise ValueError("encrypted tenant database is invalid") from None

def _assert_no_symlink_components(path: Path) -> None:
    """Reject every existing symlink component without resolving it."""
    absolute_path = Path(os.path.abspath(path))
    current = Path(absolute_path.anchor)
    for component in absolute_path.parts[1:]:
        current /= component
        try:
            component_stat = os.lstat(current)
        except FileNotFoundError:

```



```

        continue
    if stat.S_ISLNK(component_stat.st_mode):
        raise ValueError(f"restore destination contains a symlink: {current}")

def _prepare_destination_parent(path: Path) -> None:
    """Create missing destination parents without following existing symlinks."""
    absolute_path = Path(os.path.abspath(path))
    current = Path(absolute_path.anchor)
    for component in absolute_path.parts[1:]:
        current /= component
        try:
            component_stat = os.lstat(current)
        except FileNotFoundError:
            current.mkdir(mode=0o700)
            component_stat = os.lstat(current)
        if stat.S_ISLNK(component_stat.st_mode):
            raise ValueError(f"restore destination contains a symlink: {current}")
        if not stat.S_ISDIR(component_stat.st_mode):
            raise NotADirectoryError(current)

def _sync_directory(path: Path) -> None:
    """Persist directory entry changes before rollback data is removed."""
    descriptor = os.open(path, os.O_RDONLY | getattr(os, "O_DIRECTORY", 0))
    try:
        os.fsync(descriptor)
    finally:
        os.close(descriptor)

def _copy_private_file(source: Path, target: Path) -> None:
    """Copy one rollback file with bounded memory and private permissions."""
    target.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    descriptor = os.open(target, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    try:
        with (
            source.open("rb") as input_file,
            os.fdopen(descriptor, "wb", closefd=False) as output_file,
        ):
            shutil.copyfileobj(input_file, output_file, _BACKUP_CHUNK_BYTES)
            output_file.flush()
            os.fsync(output_file.fileno())
    finally:
        os.close(descriptor)

```

```

def _restore_private_file(source: Path, target: Path) -> None:
    """Restore one file without consuming its private recovery copy."""
    descriptor, temporary_name = tempfile.mkstemp(
        prefix=".yinshi-restore-recover-",
        dir=target.parent,
    )
    os.close(descriptor)
    temporary_path = Path(temporary_name)
    temporary_path.unlink()
    try:
        _copy_private_file(source, temporary_path)
        os.replace(temporary_path, target)
    finally:
        temporary_path.unlink(missing_ok=True)

def _existing_tenant_database_paths(users_root: Path) -> set[Path]:
    """Find tenant database files without traversing symlinked directories."""
    if not users_root.exists():
        return set()
    if not users_root.is_dir():
        raise NotADirectoryError(users_root)

    database_paths: set[Path] = set()
    for prefix_directory in users_root.iterdir():
        prefix_stat = os.lstat(prefix_directory)
        if stat.S_ISLNK(prefix_stat.st_mode):
            raise ValueError(f"restore destination contains a symlink: {prefix_directory}")
        if not stat.S_ISDIR(prefix_stat.st_mode):
            continue
        for tenant_directory in prefix_directory.iterdir():
            tenant_stat = os.lstat(tenant_directory)
            if stat.S_ISLNK(tenant_stat.st_mode):
                raise ValueError(f"restore destination contains a symlink: {tenant_directory}")
            if not stat.S_ISDIR(tenant_stat.st_mode):
                continue
            database_path = tenant_directory / "yinshi.db"
            try:
                database_stat = os.lstat(database_path)
            except FileNotFoundError:
                continue
            if stat.S_ISLNK(database_stat.st_mode):
                raise ValueError(f"restore destination contains a symlink: {database_path}")
            if not stat.S_ISREG(database_stat.st_mode):
                raise ValueError(f"restore destination is not a regular file: {database_path}")

```

```

        database_paths.add(database_path)
    return database_paths

def _install_staged_databases(
    installations: list[tuple[Path, Path]],
    removals: set[Path] | None = None,
) -> None:
    """Replace staged databases while retaining durable rollback copies."""
    removal_targets = set() if removals is None else set(removals)
    installation_targets = {target for _, target in installations}
    if removal_targets & installation_targets:
        raise ValueError("restore targets cannot be installed and removed")
    for target in installation_targets | removal_targets:
        _assert_no_symlink_components(target)
        _prepare_destination_parent(target.parent)
        _assert_no_symlink_components(target)
    sidecars = {
        Path(f"{target}{suffix}")
        for target in installation_targets | removal_targets
        for suffix in ("-wal", "-shm")
    }
    for sidecar in sidecars:
        _assert_no_symlink_components(sidecar)
    existing = {path for path in installation_targets | removal_targets | sidecars if path.exists()}
    rollback_roots: dict[Path, Path] = {}
    rollback_paths: dict[Path, Path] = {}
    changed: set[Path] = set()
    remove_rollback_roots = True
    try:
        for index, path in enumerate(sorted(existing, key=str)):
            root = rollback_roots.get(path.parent)
            if root is None:
                root = Path(tempfile.mkdtemp(prefix=".yinshi-restore-rollback-", dir=path.parent))
                os.chmod(root, 0o700)
                rollback_roots[path.parent] = root
            rollback = root / str(index)
            _copy_private_file(path, rollback)
            rollback_paths[path] = rollback

        for root in rollback_roots.values():
            _sync_directory(root)
            _sync_directory(root.parent)

    try:
        for sidecar in sidecars & existing:

```

```

        sidecar.unlink()
        changed.add(sidecar)
    for target in removal_targets & existing:
        target.unlink()
        changed.add(target)
    for stage, target in installations:
        os.replace(stage, target)
        changed.add(target)
        os.chmod(target, 0o600)
    for directory in {path.parent for path in changed}:
        _sync_directory(directory)
except Exception as installation_error:
    rollback_errors: list[Exception] = []
    for target in sorted(changed, key=str, reverse=True):
        rollback_candidate = rollback_paths.get(target)
        try:
            if rollback_candidate is None:
                target.unlink(missing_ok=True)
            else:
                _restore_private_file(rollback_candidate, target)
        except Exception as rollback_error:
            rollback_errors.append(rollback_error)
    for path, rollback in rollback_paths.items():
        if path in changed or path.exists() or not rollback.exists():
            continue
        try:
            _restore_private_file(rollback, path)
        except Exception as rollback_error:
            rollback_errors.append(rollback_error)
    for directory in {path.parent for path in changed}:
        try:
            _sync_directory(directory)
        except Exception as rollback_error:
            rollback_errors.append(rollback_error)
    if rollback_errors:
        remove_rollback_roots = False
        raise rollback_errors[0] from installation_error
    raise
finally:
    if remove_rollback_roots:
        for root in rollback_roots.values():
            shutil.rmtree(root)

```

```

def restore_backup(archive_path: Path, *, confirm_replace: bool = False) -> None:
    """Authenticate an encrypted backup before inspecting restore destinations."""

```

```

settings = get_settings()
if settings.managed_runtime_provider == "fly_sprites":
    raise RuntimeError(_MANAGED_FLY_BACKUP_ERROR)
source_path = Path(archive_path).resolve(strict=True)
backup_key = _backup_key_from_settings()
with tempfile.TemporaryDirectory(prefix="yinshi-restore-decrypt-") as decrypt_name:
    tar_path = Path(decrypt_name) / "archive.tar.gz"
    decrypt_backup_archive(source_path, tar_path, backup_key)
    restore_archive = _inspect_archive_members(tar_path)
    control_target = Path(os.path.abspath(settings.control_db_path))
    users_target = Path(os.path.abspath(settings.user_data_dir))
    _assert_no_symlink_components(control_target)
    _assert_no_symlink_components(users_target)
    if control_target.exists() and not confirm_replace:
        raise FileExistsError("restore replacement requires explicit confirmation")
    _prepare_destination_parent(control_target.parent)
    _prepare_destination_parent(users_target.parent)
    _assert_no_symlink_components(control_target)
    _assert_no_symlink_components(users_target)
    with (
        tempfile.TemporaryDirectory(
            prefix=".yinshi-restore-stage-", dir=control_target.parent
        ) as control_stage_name,
        tempfile.TemporaryDirectory(
            prefix=".yinshi-restore-stage-", dir=users_target.parent
        ) as tenant_stage_name,
        tarfile.open(tar_path, mode="r:gz") as archive,
    ):
        control_stage = Path(control_stage_name) / "control.db"
        tenant_stage = Path(tenant_stage_name)
        _copy_archive_member(archive, restore_archive.control_member, control_stage)
        _validate_control_database(control_stage)
        installations: list[tuple[Path, Path]] = []
        staged_tenants: list[tuple[str, Path]] = []
        for user_id, member in restore_archive.tenant_members.items():
            staged = tenant_stage / user_id[:2] / user_id / "yinshi.db"
            target = users_target / user_id[:2] / user_id / "yinshi.db"
            _copy_archive_member(archive, member, staged)
            installations.append((staged, target))
            staged_tenants.append((user_id, staged))
        for user_id, staged in staged_tenants:
            _validate_staged_tenant_database(staged, user_id, control_stage)
        installations.append((control_stage, control_target))
        archived_tenant_targets = {
            target for _, target in installations if target != control_target
        }

```

```

        removed_tenant_targets = (
            _existing_tenant_database_paths(users_target) - archived_tenant_targets
        )
        _install_staged_databases(installations, removed_tenant_targets)

def _main() -> int:
    """Run the backup command-line interface."""
    parser = argparse.ArgumentParser(
        description="Create or restore an encrypted Yinshi database backup"
    )
    commands = parser.add_subparsers(dest="command")
    commands.add_parser("create", help="create an encrypted backup")
    restore_parser = commands.add_parser(
        "restore",
        help="restore an encrypted backup while application writers are stopped",
    )
    restore_parser.add_argument("archive", type=Path)
    restore_parser.add_argument(
        "--confirm-replace",
        "--confirm",
        action="store_true",
        dest="confirm_replace",
        help="confirm replacement of configured databases",
    )
    arguments = parser.parse_args()
    if arguments.command == "restore":
        restore_backup(
            arguments.archive,
            confirm_replace=arguments.confirm_replace,
        )
        return 0
    archive_path = create_backup()
    print(archive_path)
    return 0

if __name__ == "__main__":
    raise SystemExit(_main())

```

5.2 auth.py

Authentication middleware verifies signed session cookies, resolves tenant context, and attaches identity to each request. The root chunk below tangles back to backend/src/yinshi/auth.py.

```

"""OAuth authentication and session middleware (Google + GitHub)."""

import logging
import sqlite3
import uuid
from typing import cast

from authlib.integrations.starlette_client import OAuth
from fastapi import Request, Response
from itsdangerous import BadSignature, BadTimeSignature, URLSafeTimedSerializer
from starlette.middleware.base import BaseHTTPMiddleware, RequestResponseEndpoint

from yinshi.config import get_settings
from yinshi.db import get_control_db
from yinshi.services.accounts import make_tenant
from yinshi.services.desktop_tokens import VerifiedDesktopAccess, verify_desktop_access_token
from yinshi.services.live_auth_sessions import (
    signal_auth_session_revoked,
    signal_user_sessions_revoked,
)
from yinshi.tenant import TenantContext

logger = logging.getLogger(__name__)

oauth = OAuth()

SESSION_MAX_AGE = 86400 * 30 # 30 days

def setup_oauth() -> None:
    """Register OAuth providers (Google and GitHub)."""
    settings = get_settings()
    if settings.google_client_id:
        oauth.register(
            name="google",
            client_id=settings.google_client_id,
            client_secret=settings.google_client_secret,
            server_metadata_url="https://accounts.google.com/.well-known/openid-configuration",
            client_kwargs={"scope": "openid email profile"},
        )
    else:
        logger.warning("Google OAuth not configured")

    if settings.github_client_id:
        oauth.register(
            name="github",

```

```

        client_id=settings.github_client_id,
        client_secret=settings.github_client_secret,
        authorize_url="https://github.com/login/oauth/authorize",
        access_token_url="https://github.com/login/oauth/access_token",
        api_base_url="https://api.github.com/",
        client_kwargs={"scope": "user:email"},
    )
else:
    logger.warning("GitHub OAuth not configured")

if not settings.google_client_id and not settings.github_client_id:
    logger.warning("No OAuth provider configured -- auth disabled")

def _session_serializer() -> URLSafeTimedSerializer:
    """Build the serializer used for auth session cookies."""
    settings = get_settings()
    return URLSafeTimedSerializer(settings.secret_key)

def _normalize_user_id(user_id: str) -> str:
    """Return a validated user id string."""
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    normalized_user_id = user_id.strip()
    if not normalized_user_id:
        raise ValueError("user_id must not be empty")
    return normalized_user_id

def _normalize_auth_session_id(auth_session_id: str) -> str:
    """Return a validated auth session id string."""
    if not isinstance(auth_session_id, str):
        raise TypeError("auth_session_id must be a string")
    normalized_auth_session_id = auth_session_id.strip()
    if not normalized_auth_session_id:
        raise ValueError("auth_session_id must not be empty")
    return normalized_auth_session_id

def _create_auth_session(user_id: str) -> str:
    """Insert and return a new auth session identifier."""
    normalized_user_id = _normalize_user_id(user_id)
    auth_session_id = uuid.uuid4().hex
    with get_control_db() as db:
        db.execute(

```



```

        "INSERT INTO auth_sessions (id, user_id) VALUES (?, ?)",
        (auth_session_id, normalized_user_id),
    )
    db.commit()
    return auth_session_id

def _serialize_session_token(user_id: str, auth_session_id: str) -> str:
    """Serialize the session payload into a signed token."""
    normalized_user_id = _normalize_user_id(user_id)
    normalized_auth_session_id = _normalize_auth_session_id(auth_session_id)
    serializer = _session_serializer()
    return serializer.dumps(
        {
            "user_id": normalized_user_id,
            "auth_session_id": normalized_auth_session_id,
        },
        salt="yinshi-session",
    )

def get_session_identity(token: str) -> tuple[str, str] | None:
    """Verify and decode a session token into user and auth session ids."""
    if not isinstance(token, str):
        return None
    normalized_token = token.strip()
    if not normalized_token:
        return None

    serializer = _session_serializer()
    try:
        payload = serializer.loads(
            normalized_token,
            salt="yinshi-session",
            max_age=SESSION_MAX_AGE,
        )
    except (BadSignature, BadTimeSignature):
        return None

    if not isinstance(payload, dict):
        return None

    user_id = payload.get("user_id")
    auth_session_id = payload.get("auth_session_id")
    if not isinstance(user_id, str):
        return None

```

```

if not isinstance(auth_session_id, str):
    return None

normalized_user_id = user_id.strip()
normalized_auth_session_id = auth_session_id.strip()
if not normalized_user_id:
    return None
if not normalized_auth_session_id:
    return None

session_row = _load_auth_session_row(normalized_user_id, normalized_auth_session_id)
if session_row is None:
    return None
if session_row["revoked_at"] is not None:
    return None
if session_row["user_status"] != "active":
    return None
return normalized_user_id, normalized_auth_session_id

def _load_auth_session_row(user_id: str, auth_session_id: str) -> sqlite3.Row | None:
    """Return the auth session row for a signed cookie payload."""
    normalized_user_id = _normalize_user_id(user_id)
    normalized_auth_session_id = _normalize_auth_session_id(auth_session_id)
    with get_control_db() as db:
        row = db.execute(
            "SELECT a.id, a.revoked_at, u.status AS user_status "
            "FROM auth_sessions a JOIN users u ON u.id = a.user_id "
            "WHERE a.id = ? AND a.user_id = ?",
            (normalized_auth_session_id, normalized_user_id),
        ).fetchone()
    return cast(sqlite3.Row | None, row)

def revoke_auth_session(user_id: str, auth_session_id: str) -> None:
    """Revoke one auth session that belongs to one user."""
    normalized_user_id = _normalize_user_id(user_id)
    normalized_auth_session_id = _normalize_auth_session_id(auth_session_id)
    with get_control_db() as db:
        db.execute(
            """
            UPDATE auth_sessions
            SET revoked_at = CURRENT_TIMESTAMP
            WHERE id = ? AND user_id = ? AND revoked_at IS NULL
            """,
            (normalized_auth_session_id, normalized_user_id),

```

```

        )
        db.commit()
        signal_auth_session_revoked(normalized_auth_session_id)

def revoke_auth_sessions(user_id: str) -> None:
    """Revoke every auth session that belongs to a user."""
    normalized_user_id = _normalize_user_id(user_id)
    with get_control_db() as db:
        db.execute(
            """
            UPDATE auth_sessions
            SET revoked_at = CURRENT_TIMESTAMP
            WHERE user_id = ? AND revoked_at IS NULL
            """,
            (normalized_user_id,),
        )
        db.commit()
        signal_user_sessions_revoked(normalized_user_id)

def create_session_token(user_id: str) -> str:
    """Create a signed session token encoding user and auth session ids."""
    normalized_user_id = _normalize_user_id(user_id)
    auth_session_id = _create_auth_session(normalized_user_id)
    return _serialize_session_token(normalized_user_id, auth_session_id)

def verify_session_token(token: str) -> str | None:
    """Verify and decode a session token. Returns user_id or None."""
    session_identity = get_session_identity(token)
    if session_identity is None:
        return None
    return session_identity[0]

def resolve_tenant_from_session_token(token: str) -> TenantContext | None:
    """Return the tenant identified by a signed browser session token."""
    if not isinstance(token, str):
        raise TypeError("token must be a string")
    normalized_token = token.strip()
    if not normalized_token:
        raise ValueError("token must not be empty")
    user_id = verify_session_token(normalized_token)
    if user_id is None:
        return None

```

```

        return _resolve_tenant_from_user_id(user_id)

def auth_disabled() -> bool:
    """Return whether authentication was explicitly disabled."""
    settings = get_settings()
    return settings.disable_auth

def resolve_desktop_principal(
    token: str,
) -> tuple[TenantContext, VerifiedDesktopAccess] | None:
    """Resolve active account and device authority from one access token."""
    if not isinstance(token, str) or not token:
        return None
    identity = verify_desktop_access_token(token)
    if identity is None:
        return None
    with get_control_db() as db:
        row = db.execute(
            """
            SELECT u.id, u.email, u.status, d.revoked_at
            FROM desktop_devices d
            JOIN users u ON u.id = d.user_id
            WHERE d.id = ? AND d.user_id = ?
            """,
            (identity.device_id, identity.user_id),
        ).fetchone()
    if row is None or row["status"] != "active" or row["revoked_at"] is not None:
        return None
    return make_tenant(row["id"], row["email"]), identity

def resolve_tenant_from_desktop_token(token: str) -> TenantContext | None:
    """Resolve an active desktop device and account from one access token."""
    principal = resolve_desktop_principal(token)
    return None if principal is None else principal[0]

def _resolve_tenant_from_user_id(user_id: str) -> TenantContext | None:
    """Resolve TenantContext from a user_id in the control DB."""
    with get_control_db() as db:
        row = db.execute("SELECT id, email FROM users WHERE id = ?", (user_id,)).fetchone()
    if not row:
        return None
    return make_tenant(row["id"], row["email"])

```

```

class AuthMiddleware(BaseHTTPMiddleware):
    """Middleware that checks for valid session cookie on protected routes."""

    # `~/rum/` is the Datadog browser-intake proxy; the SDK cannot send auth
    # cookies or the `X-Requested-With` CSRF header, so keep it public.
    OPEN_PREFIXES = (
        "/auth/",
        "/health",
        "/internal/managed-recovery/",
        "/runner/",
        "/static/",
        "/rum/",
    )
    PROTECTED_AUTH_PREFIX = "/auth/providers/"

    @classmethod
    def _is_open_path(cls, path: str) -> bool:
        if not isinstance(path, str) or not path.startswith("/"):
            raise ValueError("authentication path must be absolute")
        if path.startswith(cls.PROTECTED_AUTH_PREFIX):
            return False
        return any(path.startswith(prefix) for prefix in cls.OPEN_PREFIXES)

    async def dispatch(
        self,
        request: Request,
        call_next: RequestResponseEndpoint,
    ) -> Response:
        path = request.url.path

        # Allow CORS preflight requests to pass through to CORSMiddleware.
        if request.method == "OPTIONS":
            return await call_next(request)

        # Skip auth if explicitly disabled
        if auth_disabled():
            return await call_next(request)

        # Allow open paths
        if self._is_open_path(path):
            return await call_next(request)

        authorization = request.headers.get("Authorization")
        tenant: TenantContext | None = None

```

```

if authorization is not None:
    scheme, separator, bearer_token = authorization.partition(" ")
    if scheme.lower() != "bearer" or separator != " " or not bearer_token:
        return Response(status_code=401, content="Invalid bearer token")
    desktop_principal = resolve_desktop_principal(bearer_token)
    if desktop_principal is None:
        return Response(status_code=401, content="Invalid bearer token")
    tenant, desktop_identity = desktop_principal
    request.state.desktop_device_id = desktop_identity.device_id
else:
    token = request.cookies.get("yinshi_session")
    if not token:
        return Response(status_code=401, content="Not authenticated")

    user_id = verify_session_token(token)
    if not user_id:
        return Response(status_code=401, content="Invalid session")
    tenant = _resolve_tenant_from_user_id(user_id)
    if tenant is None:
        return Response(status_code=401, content="User not found")

request.state.user_email = tenant.email
request.state.tenant = tenant

# CSRF protection for mutating methods
if request.method in ("POST", "PATCH", "PUT", "DELETE"):
    if request.headers.get("X-Requested-With") != "XMLHttpRequest":
        return Response(status_code=403, content="CSRF validation failed")

return await call_next(request)

```

5.3 live_auth_sessions.py

Long-lived SSE and terminal connections register one local revocation signal. Durable database checks remain authoritative across processes, while the in-process registry lets logout wake connections immediately within the serving process.

```

"""In-process revocation signals for long-lived authenticated connections."""

from __future__ import annotations

import asyncio
import threading
from dataclasses import dataclass

```

```

@dataclass(frozen=True, slots=True)
class LiveAuthSessionRegistration:
    """One event-loop-bound connection waiting for auth-session revocation."""

    user_id: str
    auth_session_id: str
    event: asyncio.Event
    loop: asyncio.AbstractEventLoop

    def close(self) -> None:
        """Remove this registration after its connection closes."""
        _remove_registration(self)

_registry_lock = threading.Lock()
_registrations_by_session: dict[str, dict[int, LiveAuthSessionRegistration]] = {}
_registrations_by_user: dict[str, dict[int, LiveAuthSessionRegistration]] = {}

def _normalize_identifier(value: str, name: str) -> str:
    """Return one non-empty registry identifier."""
    if not isinstance(value, str):
        raise TypeError(f"{name} must be a string")
    normalized_value = value.strip()
    if not normalized_value:
        raise ValueError(f"{name} must not be empty")
    return normalized_value

def register_live_auth_session(
    user_id: str,
    auth_session_id: str,
) -> LiveAuthSessionRegistration:
    """Register one long-lived connection on the current event loop."""
    normalized_user_id = _normalize_identifier(user_id, "user_id")
    normalized_auth_session_id = _normalize_identifier(auth_session_id, "auth_session_id")
    loop = asyncio.get_running_loop()
    registration = LiveAuthSessionRegistration(
        user_id=normalized_user_id,
        auth_session_id=normalized_auth_session_id,
        event=asyncio.Event(),
        loop=loop,
    )
    registration_key = id(registration)
    with _registry_lock:

```

```

        _registrations_by_session.setdefault(normalized_auth_session_id, {})[
            registration_key
        ] = registration
        _registrations_by_user.setdefault(normalized_user_id, {})[registration_key] = registration
    return registration

def register_live_desktop_device(
    user_id: str,
    device_id: str,
) -> LiveAuthSessionRegistration:
    """Register one connection bound to a desktop device."""
    normalized_device_id = _normalize_identifier(device_id, "device_id")
    return register_live_auth_session(
        user_id=user_id,
        auth_session_id=f"desktop-device:{normalized_device_id}",
    )

def _remove_registration(registration: LiveAuthSessionRegistration) -> None:
    """Remove one registration from both indexes without assuming it exists."""
    if not isinstance(registration, LiveAuthSessionRegistration):
        raise TypeError("registration must be a LiveAuthSessionRegistration")
    registration_key = id(registration)
    with _registry_lock:
        session_registrations = _registrations_by_session.get(registration.auth_session_id)
        if session_registrations is not None:
            session_registrations.pop(registration_key, None)
            if not session_registrations:
                _registrations_by_session.pop(registration.auth_session_id, None)
        user_registrations = _registrations_by_user.get(registration.user_id)
        if user_registrations is not None:
            user_registrations.pop(registration_key, None)
            if not user_registrations:
                _registrations_by_user.pop(registration.user_id, None)

def _signal_registrations(registrations: tuple[LiveAuthSessionRegistration, ...]) -> None:
    """Signal registrations safely from event-loop or worker threads."""
    for registration in registrations:
        try:
            registration.loop.call_soon_threadsafe(registration.event.set)
        except RuntimeError:
            _remove_registration(registration)

```



```

def signal_auth_session_revoked(auth_session_id: str) -> None:
    """Wake every local connection created by one auth session."""
    normalized_auth_session_id = _normalize_identifier(auth_session_id, "auth_session_id")
    with _registry_lock:
        registrations = tuple(
            _registrations_by_session.get(normalized_auth_session_id, {}).values()
        )
    _signal_registrations(registrations)

def signal_desktop_device_revoked(device_id: str) -> None:
    """Wake every local connection created by one desktop device."""
    normalized_device_id = _normalize_identifier(device_id, "device_id")
    signal_auth_session_revoked(f"desktop-device:{normalized_device_id}")

def signal_user_sessions_revoked(user_id: str) -> None:
    """Wake every local connection owned by one user."""
    normalized_user_id = _normalize_identifier(user_id, "user_id")
    with _registry_lock:
        registrations = tuple(_registrations_by_user.get(normalized_user_id, {}).values())
    _signal_registrations(registrations)

```

5.4 api/auth_routes.py

Auth routes handle Google login, GitHub login, GitHub App installation state, logout, and provider-auth handoffs. The root chunk below tangles back to backend/src/yinshi/api/auth_routes.py.

```

"""OAuth login/callback endpoints for Google and GitHub."""

import hashlib
import logging
import secrets
import sqlite3
import time
from typing import Any, cast
from urllib.parse import urlencode

import httpx
from authlib.integrations.starlette_client import OAuthError
from fastapi import APIRouter, HTTPException, Request
from fastapi.responses import JSONResponse, RedirectResponse, Response
from itsdangerous import BadSignature, BadTimeSignature, URLSafeTimedSerializer

from yinshi.auth import (

```

```

        SESSION_MAX_AGE,
        _resolve_tenant_from_user_id,
        create_session_token,
        get_session_identity,
        oauth,
        resolve_tenant_from_desktop_token,
        revoke_auth_session,
        revoke_auth_sessions,
        verify_session_token,
    )
from yinshi.config import get_settings
from yinshi.db import get_control_db
from yinshi.exceptions import (
    ContainerNotReadyError,
    ContainerStartError,
    GitError,
    GitHubAppError,
    GitHubInstallationUnusableError,
    SidecarError,
)
from yinshi.models import (
    DesktopAuthorizationRequestIn,
    DesktopAuthorizationRequestOut,
    DesktopRefreshRequestIn,
    DesktopTokenOut,
    DesktopTokenRequestIn,
    ProviderAuthInputIn,
    ProviderAuthStartOut,
    ProviderAuthStatusOut,
)
from yinshi.rate_limit import limiter
from yinshi.services.accounts import resolve_or_create_user
from yinshi.services.desktop_auth import (
    DesktopAccountUnavailableError,
    DesktopAuthorizationExpiredError,
    DesktopAuthorizationNotFoundError,
    DesktopAuthorizationUsedError,
    DesktopCodeInvalidError,
    DesktopPkceMismatchError,
    DesktopRefreshInvalidError,
    DesktopTokenExchange,
    approve_desktop_authorization_request,
)
from yinshi.services.desktop_auth import (
    create_desktop_authorization_request as store_desktop_request,
)

```

```

from yinshi.services.desktop_auth import (
    exchange_desktop_authorization_code as exchange_desktop_code,
)
from yinshi.services.desktop_auth import (
    rotate_desktop_refresh_token,
)
from yinshi.services.github_app import get_installation_details
from yinshi.services.provider_connections import create_provider_connection
from yinshi.services.sidecar import create_sidecar_connection
from yinshi.services.sidecar_runtime import (
    protect_tenant_container,
    release_tenant_container,
    resolve_tenant_sidecar_context,
    tenant_container_activity,
    touch_tenant_container,
)
from yinshi.services.workspace import relink_github_repos_for_tenant
from yinshi.tenant import TenantContext, get_user_db

logger = logging.getLogger(__name__)
router = APIRouter(prefix="/auth", tags=["auth"])
provider_router = APIRouter(prefix="/auth", tags=["provider-auth"])
_GITHUB_INSTALL_STATE_MAX_AGE_S = 600
_PROVIDER_OAUTH_CONTAINER_LEASE_TIMEOUT_S = 1800

def _set_session_cookie(response: RedirectResponse, user_id: str) -> None:
    """Set the yinshi_session cookie on a response."""
    settings = get_settings()
    session_token = create_session_token(user_id)
    response.set_cookie(
        key="yinshi_session",
        value=session_token,
        httponly=True,
        secure=not settings.debug,
        max_age=SESSION_MAX_AGE,
        samesite="lax",
        path="/",
    )

def _clear_session_cookie(response: Response) -> None:
    """Remove the current session cookie from a response."""
    response.delete_cookie("yinshi_session", path="/")

```

```

def _github_install_state_serializer() -> URLSafeTimedSerializer:
    """Build a serializer dedicated to GitHub install state tokens."""
    settings = get_settings()
    return URLSafeTimedSerializer(settings.secret_key)

def _github_install_flow_digest(flow_token: str) -> str:
    """Hash an unguessable install-flow token for control-database storage."""
    if not flow_token:
        raise ValueError("flow_token must not be empty")
    return hashlib.sha256(flow_token.encode("utf-8")).hexdigest()

def _create_github_install_state(user_id: str) -> str:
    """Create a signed, database-backed GitHub install state token."""
    if not user_id:
        raise ValueError("user_id must not be empty")
    flow_token = secrets.token_urlsafe(32)
    state_digest = _github_install_flow_digest(flow_token)
    expires_at = int(time.time()) + _GITHUB_INSTALL_STATE_MAX_AGE_S
    with get_control_db() as db:
        db.execute("DELETE FROM github_install_flows WHERE expires_at < ?", (int(time.time()),))
        db.execute(
            "INSERT INTO github_install_flows (state_digest, user_id, expires_at) "
            "VALUES (?, ?, ?)",
            (state_digest, user_id, expires_at),
        )
        db.commit()
    serializer = _github_install_state_serializer()
    return serializer.dumps(
        {"user_id": user_id, "flow_token": flow_token},
        salt="github-install-state",
    )

def _create_github_install_verification_state(
    user_id: str,
    installation_id: int,
    flow_token: str,
) -> str:
    """Sign a user, installation, and one-time flow token for authorization."""
    if not user_id:
        raise ValueError("user_id must not be empty")
    if installation_id <= 0:
        raise ValueError("installation_id must be positive")
    if not flow_token:

```

```

        raise ValueError("flow_token must not be empty")
serializer = _github_install_state_serializer()
return serializer.dumps(
    {
        "user_id": user_id,
        "installation_id": installation_id,
        "flow_token": flow_token,
    },
    salt="github-install-verification-state",
)

def _verify_github_install_verification_state(state: str) -> tuple[str, int, str] | None:
    """Return the signed user, installation, and flow digest when valid."""
    if not state:
        raise ValueError("state must not be empty")
    serializer = _github_install_state_serializer()
    try:
        payload = serializer.loads(
            state,
            salt="github-install-verification-state",
            max_age=_GITHUB_INSTALL_STATE_MAX_AGE_S,
        )
    except (BadSignature, BadTimeSignature):
        return None
    if not isinstance(payload, dict):
        return None
    user_id = payload.get("user_id")
    installation_id = payload.get("installation_id")
    flow_token = payload.get("flow_token")
    if not isinstance(user_id, str) or not user_id:
        return None
    if not isinstance(installation_id, int) or installation_id <= 0:
        return None
    if not isinstance(flow_token, str) or not flow_token:
        return None
    return user_id, installation_id, _github_install_flow_digest(flow_token)

def _verify_github_install_state(state: str) -> tuple[str, str] | None:
    """Verify initial state and return its user id and raw one-time token."""
    if not state:
        raise ValueError("state must not be empty")
    serializer = _github_install_state_serializer()
    try:
        payload = serializer.loads(

```

```

        state,
        salt="github-install-state",
        max_age=_GITHUB_INSTALL_STATE_MAX_AGE_S,
    )
except (BadSignature, BadTimeSignature):
    return None
if not isinstance(payload, dict):
    return None
user_id = payload.get("user_id")
flow_token = payload.get("flow_token")
if not isinstance(user_id, str) or not user_id:
    return None
if not isinstance(flow_token, str) or not flow_token:
    return None
return user_id, flow_token

def _current_session_identity(request: Request) -> tuple[str, str] | None:
    """Return the authenticated user and auth session ids from the session cookie."""
    token = request.cookies.get("yinshi_session")
    if not token:
        return None
    return get_session_identity(token)

def _current_user_id(request: Request) -> str | None:
    """Return the authenticated cookie, bearer, worker, or desktop identity."""
    session_identity = _current_session_identity(request)
    if session_identity is not None:
        return session_identity[0]
    authorization = request.headers.get("Authorization")
    if authorization is not None:
        scheme, separator, token = authorization.partition(" ")
        if scheme.lower() != "bearer" or separator != " " or not token:
            return None
        tenant = resolve_tenant_from_desktop_token(token)
        if tenant is not None:
            return tenant.user_id
    tenant = getattr(request.state, "tenant", None)
    if isinstance(tenant, TenantContext) and tenant.user_id:
        return tenant.user_id
    return None

async def _refresh_connected_github_repos(
    user_id: str,

```

```

        account_login: str,
    ) -> None:
        """Backfill repo installation links after a GitHub App connect completes."""
        if not user_id:
            raise ValueError("user_id must not be empty")
        if not account_login:
            raise ValueError("account_login must not be empty")

        tenant = _resolve_tenant_from_user_id(user_id)
        if tenant is None:
            return

        try:
            with get_user_db(tenant) as db:
                await relink_github_repos_for_tenant(
                    db,
                    tenant,
                    account_login,
                )
        except (GitError, GitHubAppError):
            logger.warning("Failed to refresh GitHub repo links")

async def _resolve_provider_sidecar_socket(
    request: Request,
    user_id: str,
) -> tuple[TenantContext | None, str | None]:
    """Resolve the tenant record plus socket path for provider-auth sidecar calls."""
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    normalized_user_id = user_id.strip()
    if not normalized_user_id:
        raise ValueError("user_id must not be empty")

    request_tenant = getattr(request.state, "tenant", None)
    tenant: TenantContext | None
    if isinstance(request_tenant, TenantContext) and request_tenant.user_id == normalized_user_id:
        tenant = request_tenant
    else:
        tenant = _resolve_tenant_from_user_id(normalized_user_id)
    try:
        tenant_sidecar_context = await resolve_tenant_sidecar_context(request, tenant)
    except (ContainerStartError, ContainerNotReadyError) as error:
        raise HTTPException(
            status_code=503,
            detail="Agent environment temporarily unavailable",
        )

```

```

        ) from error
    return tenant, tenant_sidecar_context.socket_path

def _normalize_provider_flow_status(status: object) -> str:
    """Require a non-empty string OAuth flow status."""
    if not isinstance(status, str):
        raise HTTPException(status_code=500, detail="OAuth flow status is invalid")
    normalized_status = status.strip()
    if not normalized_status:
        raise HTTPException(status_code=500, detail="OAuth flow status is invalid")
    return normalized_status

def _provider_auth_lease_key(flow_id: str) -> str:
    """Build the container lease key used to protect one OAuth flow."""
    if not isinstance(flow_id, str):
        raise TypeError("flow_id must be a string")
    normalized_flow_id = flow_id.strip()
    if not normalized_flow_id:
        raise ValueError("flow_id must not be empty")
    return f"oauth:{normalized_flow_id}"

def _provider_auth_sidecar_http_error(error: Exception) -> HTTPException:
    """Translate provider-auth sidecar errors into stable HTTP responses."""
    message = str(error)
    if "OAuth flow not found" in message:
        return HTTPException(status_code=404, detail="OAuth flow not found")
    if "OAuth provider is not available" in message:
        detail = message.removeprefix("OAuth start failed: ")
        return HTTPException(status_code=400, detail=detail)
    return HTTPException(
        status_code=503,
        detail="Agent environment temporarily unavailable",
    )

def _build_provider_auth_start_payload(flow: dict[str, Any]) -> dict[str, Any]:
    """Validate and serialize one started provider OAuth flow."""
    if not isinstance(flow, dict):
        raise TypeError("flow must be a dictionary")
    flow_id = flow.get("flow_id")
    provider = flow.get("provider")
    auth_url = flow.get("auth_url")
    instructions = flow.get("instructions")

```



```

manual_input_prompt = flow.get("manual_input_prompt")
if not isinstance(flow_id, str) or not flow_id:
    raise HTTPException(status_code=500, detail="OAuth flow id is invalid")
if not isinstance(provider, str) or not provider:
    raise HTTPException(status_code=500, detail="OAuth flow provider is invalid")
if not isinstance(auth_url, str) or not auth_url:
    raise HTTPException(status_code=500, detail="OAuth flow auth URL is invalid")
if instructions is not None and not isinstance(instructions, str):
    raise HTTPException(status_code=500, detail="OAuth flow instructions are invalid")
if manual_input_prompt is not None and not isinstance(manual_input_prompt, str):
    raise HTTPException(status_code=500, detail="OAuth flow manual input prompt is invalid")
return {
    "flow_id": flow_id,
    "provider": provider,
    "auth_url": auth_url,
    "instructions": instructions,
    "manual_input_required": bool(flow.get("manual_input_required", False)),
    "manual_input_prompt": manual_input_prompt,
    "manual_input_submitted": bool(flow.get("manual_input_submitted", False)),
}

def _build_provider_auth_status_payload(
    flow: dict[str, Any],
    *,
    status: str,
    error: str | None = None,
) -> dict[str, Any]:
    """Validate and serialize one provider OAuth status payload."""
    if not isinstance(flow, dict):
        raise TypeError("flow must be a dictionary")
    normalized_status = _normalize_provider_flow_status(status)
    flow_id = flow.get("flow_id")
    provider = flow.get("provider")
    instructions = flow.get("instructions")
    manual_input_prompt = flow.get("manual_input_prompt")
    progress = flow.get("progress", [])
    if not isinstance(flow_id, str) or not flow_id:
        raise HTTPException(status_code=500, detail="OAuth flow id is invalid")
    if not isinstance(provider, str) or not provider:
        raise HTTPException(status_code=500, detail="OAuth flow provider is invalid")
    if instructions is not None and not isinstance(instructions, str):
        raise HTTPException(status_code=500, detail="OAuth flow instructions are invalid")
    if manual_input_prompt is not None and not isinstance(manual_input_prompt, str):
        raise HTTPException(status_code=500, detail="OAuth flow manual input prompt is invalid")
    if not isinstance(progress, list):

```

```

        raise HTTPException(status_code=500, detail="OAuth flow progress is invalid")
normalized_progress: list[str] = []
for progress_entry in progress:
    if not isinstance(progress_entry, str):
        raise HTTPException(status_code=500, detail="OAuth flow progress is invalid")
    normalized_progress.append(progress_entry)
if error is not None and not isinstance(error, str):
    raise HTTPException(status_code=500, detail="OAuth flow error is invalid")
return {
    "status": normalized_status,
    "provider": provider,
    "flow_id": flow_id,
    "instructions": instructions,
    "progress": normalized_progress,
    "manual_input_required": bool(flow.get("manual_input_required", False)),
    "manual_input_prompt": manual_input_prompt,
    "manual_input_submitted": bool(flow.get("manual_input_submitted", False)),
    "error": error,
}

def _require_provider_match(provider: str, flow: dict[str, Any]) -> None:
    """Reject OAuth flows that do not belong to the requested provider."""
    if not isinstance(provider, str):
        raise TypeError("provider must be a string")
    normalized_provider = provider.strip()
    if not normalized_provider:
        raise ValueError("provider must not be empty")
    flow_provider = flow.get("provider")
    if not isinstance(flow_provider, str) or not flow_provider:
        raise HTTPException(status_code=500, detail="OAuth flow provider is invalid")
    if flow_provider != normalized_provider:
        raise HTTPException(status_code=400, detail="Provider mismatch")

async def _persist_completed_provider_auth(
    user_id: str,
    provider: str,
    flow: dict[str, Any],
    sidecar: Any,
) -> None:
    """Persist completed provider credentials and clear the finished flow."""
    flow_id = flow.get("flow_id")
    if not isinstance(flow_id, str) or not flow_id:
        raise HTTPException(status_code=500, detail="OAuth flow id is invalid")
    credentials = flow.get("credentials")

```

```

if not isinstance(credentials, dict) or not credentials:
    raise HTTPException(status_code=500, detail="OAuth flow did not return credentials")
create_provider_connection(
    user_id,
    provider,
    "oauth",
    credentials,
    label="",
)
try:
    await sidecar.clear_oauth_flow(flow_id)
except SidecarError as error:
    if "OAuth flow not found" not in str(error):
        raise _provider_auth_sidecar_http_error(error) from error

@router.get("/desktop/authorize/{request_id}")
async def authorize_desktop_request(
    request_id: str,
    request: Request,
) -> RedirectResponse:
    """Return a PKCE authorization code to one validated desktop callback."""
    user_id = _current_user_id(request)
    if user_id is None:
        raise HTTPException(status_code=401, detail="Sign in before authorizing the desktop")
    try:
        approved_request = approve_desktop_authorization_request(
            request_id=request_id,
            user_id=user_id,
        )
    except DesktopAuthorizationNotFoundError:
        raise HTTPException(
            status_code=404,
            detail="Desktop authorization request not found",
        ) from None
    except DesktopAuthorizationExpiredError:
        raise HTTPException(
            status_code=410,
            detail="Desktop authorization request expired",
        ) from None
    except DesktopAuthorizationUsedError:
        raise HTTPException(
            status_code=409,
            detail="Desktop authorization request was used",
        ) from None
    return RedirectResponse(url=approved_request.callback_url, status_code=307)

```

```

def _desktop_token_response(exchange: DesktopTokenExchange) -> dict[str, object]:
    """Serialize one service-issued desktop credential set for the API."""
    if not isinstance(exchange, DesktopTokenExchange):
        raise TypeError("exchange must be DesktopTokenExchange")
    if not exchange.access_token or not exchange.refresh_token or not exchange.account_lease:
        raise ValueError("desktop credential set must be complete")
    return {
        "token_type": "Bearer",
        "access_token": exchange.access_token,
        "access_token_expires_at": exchange.access_token_expires_at,
        "refresh_token": exchange.refresh_token,
        "refresh_token_expires_at": exchange.refresh_token_expires_at,
        "account_lease": exchange.account_lease,
        "account_lease_expires_at": exchange.account_lease_expires_at,
        "device_id": exchange.device_id,
        "signing_public_key": exchange.signing_public_key,
        "user": {"id": exchange.user_id, "email": exchange.user_email},
    }

@router.post("/desktop/refresh", response_model=DesktopTokenOut)
@limiter.limit("60/minute")
async def refresh_desktop_credentials(
    request: Request,
    body: DesktopRefreshRequestIn,
) -> dict[str, object]:
    """Rotate one current desktop refresh credential."""
    del request
    try:
        exchange = rotate_desktop_refresh_token(refresh_token=body.refresh_token)
    except DesktopRefreshInvalidError:
        raise HTTPException(
            status_code=401,
            detail="Desktop refresh credential is invalid",
        ) from None
    except sqlite3.Error:
        raise HTTPException(
            status_code=503,
            detail="Desktop credential storage is temporarily unavailable",
        ) from None
    return _desktop_token_response(exchange)

@router.post("/desktop/token", response_model=DesktopTokenOut)

```

```

@limiter.limit("20/minute")
async def exchange_desktop_authorization_code(
    request: Request,
    body: DesktopTokenRequestIn,
) -> dict[str, object]:
    """Consume a PKCE code and issue the first desktop device credentials."""
    del request
    try:
        exchange = exchange_desktop_code(
            authorization_code=body.authorization_code,
            code_verifier=body.code_verifier,
            device_name=body.device_name,
        )
    except (DesktopCodeInvalidError, DesktopPkceMismatchError):
        raise HTTPException(
            status_code=400,
            detail="Desktop authorization code or PKCE verifier is invalid",
        ) from None
    except DesktopAuthorizationUsedError:
        raise HTTPException(
            status_code=409,
            detail="Desktop authorization code was used",
        ) from None
    except DesktopAuthorizationExpiredError:
        raise HTTPException(
            status_code=410,
            detail="Desktop authorization code expired",
        ) from None
    except DesktopAccountUnavailableError:
        raise HTTPException(
            status_code=403,
            detail="Desktop account is unavailable",
        ) from None
    except sqlite3.Error:
        raise HTTPException(
            status_code=503,
            detail="Desktop credential storage is temporarily unavailable",
        ) from None
    return _desktop_token_response(exchange)

@router.post(
    "/desktop/requests",
    response_model=DesktopAuthorizationRequestOut,
    status_code=201,
)

```

```

@limiter.limit("20/minute")
async def create_desktop_authorization_request(
    request: Request,
    body: DesktopAuthorizationRequestIn,
) -> dict[str, object]:
    """Issue one short-lived opaque desktop authorization request."""
    del request
    stored_request = store_desktop_request(
        redirect_uri=body.redirect_uri,
        code_challenge=body.code_challenge,
        state=body.state,
        frontend_url=get_settings().frontend_url,
    )
    return {
        "request_id": stored_request.request_id,
        "authorize_url": stored_request.authorize_url,
        "expires_at": stored_request.expires_at,
    }

```

--- Google OAuth ---

```

@router.get("/login/google")
async def login_google(request: Request) -> Response:
    """Redirect to Google OAuth."""
    settings = get_settings()
    if not settings.google_client_id:
        return JSONResponse({"error": "Google OAuth not configured"})
    response = await oauth.google.authorize_redirect(
        request,
        settings.google_redirect_uri,
    )
    return cast(Response, response)

```

```

@router.get("/callback/google")
@limiter.limit("10/minute")
async def callback_google(request: Request) -> RedirectResponse:
    """Handle Google OAuth callback."""
    try:
        token = await oauth.google.authorize_access_token(request)
    except OAuthError:
        logger.warning("Google OAuth rejected")
        return RedirectResponse(url="/?error=oauth_error")
    except (ConnectionError, httpx.HTTPError, OSError, TypeError, ValueError):

```

```

        logger.error("Google token exchange failed")
        return RedirectResponse(url="/?error=oauth_error")

    user_info = token.get("userinfo")
    if not user_info:
        return RedirectResponse(url="/?error=no_user_info")

    email = user_info.get("email")
    if not email or not user_info.get("email_verified"):
        return RedirectResponse(url="/?error=email_not_verified")

    try:
        tenant = resolve_or_create_user(
            provider="google",
            provider_user_id=user_info["sub"],
            email=email,
            display_name=user_info.get("name"),
            avatar_url=user_info.get("picture"),
            provider_data=dict(user_info),
        )
    except (sqlite3.Error, OSError):
        logger.error("Google account provisioning failed")
        return RedirectResponse(url="/?error=account_error")

    response = RedirectResponse(url="/app")
    _set_session_cookie(response, tenant.user_id)
    logger.info("Google login completed")
    return response

# --- GitHub OAuth ---

@router.get("/login/github")
async def login_github(request: Request) -> Response:
    """Redirect to GitHub OAuth."""
    settings = get_settings()
    if not settings.github_client_id:
        return JSONResponse({"error": "GitHub OAuth not configured"})
    response = await oauth.github.authorize_redirect(
        request,
        settings.github_redirect_uri,
    )
    return cast(Response, response)

```

```

@router.get("/callback/github")
@limiter.limit("10/minute")
async def callback_github(request: Request) -> RedirectResponse:
    """Handle GitHub OAuth callback."""
    try:
        token = await oauth.github.authorize_access_token(request)
    except OAuthError:
        logger.warning("GitHub OAuth rejected")
        return RedirectResponse(url="/?error=oauth_error")
    except (ConnectionError, httpx.HTTPError, OSError, TypeError, ValueError):
        logger.error("GitHub token exchange failed")
        return RedirectResponse(url="/?error=oauth_error")

    # GitHub doesn't include user info in the token; call the API.
    try:
        async with httpx.AsyncClient() as client:
            headers = {"Authorization": f"Bearer {token['access_token']}"}
            user_resp = await client.get("https://api.github.com/user", headers=headers)
            user_resp.raise_for_status()
            user_data = user_resp.json()

            emails_resp = await client.get("https://api.github.com/user/emails", headers=headers)
            emails_resp.raise_for_status()
            emails = emails_resp.json()
    except (httpx.HTTPError, KeyError):
        logger.error("GitHub API call failed")
        return RedirectResponse(url="/?error=github_api_error")

    # Find the primary verified email, falling back to any verified email.
    primary_email = next(
        (e["email"] for e in emails if e.get("primary") and e.get("verified")),
        None,
    )
    verified_email = next(
        (e["email"] for e in emails if e.get("verified")),
        None,
    )
    email = primary_email or verified_email
    if not email:
        return RedirectResponse(url="/?error=no_verified_email")

    try:
        tenant = resolve_or_create_user(
            provider="github",
            provider_user_id=str(user_data["id"]),
            email=email,

```



```

        display_name=user_data.get("name") or user_data.get("login"),
        avatar_url=user_data.get("avatar_url"),
        provider_data=user_data,
    )
except (sqlite3.Error, OSError):
    logger.error("GitHub account provisioning failed")
    return RedirectResponse(url="/?error=account_error")

response = RedirectResponse(url="/app")
_set_session_cookie(response, tenant.user_id)
logger.info("GitHub login completed")
return response

async def verify_github_user_installation(code: str, installation_id: int) -> bool:
    """Verify that the authorized GitHub user can access one installation."""
    if not code:
        raise ValueError("code must not be empty")
    if installation_id <= 0:
        raise ValueError("installation_id must be positive")
    settings = get_settings()
    if not settings.github_app_client_id:
        raise GitHubAppError("GitHub App client ID is not configured")
    if settings.github_app_client_secret is None:
        raise GitHubAppError("GitHub App client secret is not configured")
    if not settings.github_app_user_callback_url:
        raise GitHubAppError("GitHub App user callback URL is not configured")

    try:
        async with httpx.AsyncClient(timeout=15.0) as client:
            token_response = await client.post(
                "https://github.com/login/oauth/access_token",
                headers={"Accept": "application/json"},
                data={
                    "client_id": settings.github_app_client_id,
                    "client_secret": settings.github_app_client_secret.get_secret_value(),
                    "code": code,
                    "redirect_uri": settings.github_app_user_callback_url,
                },
            )
            if token_response.status_code >= 400:
                return False
            token_payload = token_response.json()
            access_token = token_payload.get("access_token")
            if not isinstance(access_token, str) or not access_token:
                return False

```

```

        installation_response = await client.get(
            f"https://api.github.com/user/installations/"
            f"{installation_id}/repositories?per_page=1",
            headers={
                "Accept": "application/vnd.github+json",
                "Authorization": f"Bearer {access_token}",
                "X-GitHub-API-Version": "2022-11-28",
            },
        )
    except (httpx.HTTPError, ValueError) as exc:
        raise GitHubAppError("GitHub user authorization verification failed") from exc

    if installation_response.status_code in {401, 403, 404}:
        return False
    if installation_response.status_code >= 400:
        raise GitHubAppError("GitHub installation access verification failed")
    return True

@router.get("/github/install", response_model=None)
async def github_install(request: Request) -> Response:
    """Redirect an authenticated user to the GitHub App install flow."""
    settings = get_settings()
    if not settings.github_app_slug:
        return JSONResponse({"error": "GitHub App not configured"}, status_code=503)

    user_id = _current_user_id(request)
    if not user_id:
        return RedirectResponse(url="/?error=not_authenticated")

    state = _create_github_install_state(user_id)
    query = urlencode({"state": state})
    install_url = f"https://github.com/apps/{settings.github_app_slug}/installations/new?{q}"
    return RedirectResponse(url=install_url)

@router.get("/github/install/callback")
async def github_install_callback(request: Request) -> RedirectResponse:
    """Persist a GitHub installation after the GitHub App flow returns."""
    state = request.query_params.get("state")
    if not state:
        return RedirectResponse(url="/app?github_connect_error=invalid_state")

    state_payload = _verify_github_install_state(state)
    if state_payload is None:
        return RedirectResponse(url="/app?github_connect_error=invalid_state")

```

```

user_id, flow_token = state_payload
current_user_id = _current_user_id(request)
if current_user_id != user_id:
    return RedirectResponse(url="/app?github_connect_error=invalid_state")

installation_id_text = request.query_params.get("installation_id")
if not installation_id_text:
    return RedirectResponse(url="/app?github_connect_error=missing_installation")

try:
    installation_id = int(installation_id_text)
except ValueError:
    return RedirectResponse(url="/app?github_connect_error=missing_installation")

if installation_id <= 0:
    return RedirectResponse(url="/app?github_connect_error=missing_installation")

setup_action = request.query_params.get("setup_action")
if setup_action == "request":
    return RedirectResponse(url="/app?github_connect_error=not_granted")

settings = get_settings()
if not settings.github_app_client_id:
    return RedirectResponse(url="/app?github_connect_error=install_failed")
if settings.github_app_client_secret is None:
    return RedirectResponse(url="/app?github_connect_error=install_failed")
if not settings.github_app_user_callback_url:
    return RedirectResponse(url="/app?github_connect_error=install_failed")

state_digest = _github_install_flow_digest(flow_token)
with get_control_db() as db:
    cursor = db.execute(
        "UPDATE github_install_flows SET installation_id = ? "
        "WHERE state_digest = ? AND user_id = ? AND installation_id IS NULL "
        "AND expires_at >= ?",
        (installation_id, state_digest, user_id, int(time.time()))
    )
    db.commit()
if cursor.rowcount != 1:
    return RedirectResponse(url="/app?github_connect_error=invalid_state")

verification_state = _create_github_install_verification_state(
    user_id,
    installation_id,
    flow_token,
)

```

```

authorization_query = urlencode(
    {
        "client_id": settings.github_app_client_id,
        "redirect_uri": settings.github_app_user_callback_url,
        "state": verification_state,
    }
)
return RedirectResponse(url=f"https://github.com/login/oauth/authorize?{authorization_q

@router.get("/github/install/verify")
async def github_install_verify(request: Request) -> RedirectResponse:
    """Bind an installation after a GitHub user authorization callback."""
    state = request.query_params.get("state")
    code = request.query_params.get("code")
    if not state or not code:
        return RedirectResponse(url="/app?github_connect_error=invalid_state")

    state_payload = _verify_github_install_verification_state(state)
    if state_payload is None:
        return RedirectResponse(url="/app?github_connect_error=invalid_state")
    user_id, installation_id, state_digest = state_payload
    if _current_user_id(request) != user_id:
        return RedirectResponse(url="/app?github_connect_error=invalid_state")
    with get_control_db() as db:
        active_flow = db.execute(
            "SELECT 1 FROM github_install_flows "
            "WHERE state_digest = ? AND user_id = ? AND installation_id = ? "
            "AND expires_at >= ?",
            (state_digest, user_id, installation_id, int(time.time())),
        ).fetchone()
    if active_flow is None:
        return RedirectResponse(url="/app?github_connect_error=invalid_state")

    try:
        user_has_access = await verify_github_user_installation(code, installation_id)
    except GitHubAppError:
        logger.error("GitHub user installation verification failed")
        return RedirectResponse(url="/app?github_connect_error=install_failed")
    if not user_has_access:
        return RedirectResponse(url="/app?github_connect_error=not_granted")

    try:
        installation = await get_installation_details(installation_id)
    except GitHubInstallationUnusableError:
        return RedirectResponse(url="/app?github_connect_error=not_granted")

```

```

except GitHubAppError:
    logger.error("GitHub App installation verification failed")
    return HttpResponseRedirect(url="/app?github_connect_error=install_failed")

account = installation.get("account")
account_login = account.get("login") if isinstance(account, dict) else None
account_type = account.get("type") if isinstance(account, dict) else None
html_url = installation.get("html_url")
if installation.get("suspended_at"):
    return HttpResponseRedirect(url="/app?github_connect_error=not_granted")
if not isinstance(account_login, str) or not account_login:
    return HttpResponseRedirect(url="/app?github_connect_error=install_failed")
if not isinstance(account_type, str) or not account_type:
    return HttpResponseRedirect(url="/app?github_connect_error=install_failed")
if not isinstance(html_url, str) or not html_url:
    return HttpResponseRedirect(url="/app?github_connect_error=install_failed")

with get_control_db() as db:
    db.execute("BEGIN IMMEDIATE")
    active_flow = db.execute(
        "SELECT 1 FROM github_install_flows "
        "WHERE state_digest = ? AND user_id = ? AND installation_id = ? "
        "AND expires_at >= ?",
        (state_digest, user_id, installation_id, int(time.time())),
    ).fetchone()
    if active_flow is None:
        db.rollback()
        return HttpResponseRedirect(url="/app?github_connect_error=invalid_state")
    db.execute(
        """
        INSERT INTO github_installations (
            user_id, installation_id, account_login, account_type, html_url
        ) VALUES (?, ?, ?, ?, ?)
        ON CONFLICT(user_id, installation_id) DO UPDATE SET
            account_login = excluded.account_login,
            account_type = excluded.account_type,
            html_url = excluded.html_url
        """,
        (user_id, installation_id, account_login, account_type, html_url),
    )
    delete_cursor = db.execute(
        "DELETE FROM github_install_flows WHERE state_digest = ?",
        (state_digest,),
    )
    if delete_cursor.rowcount != 1:
        db.rollback()

```

```

        return RedirectResponse(url="/app?github_connect_error=invalid_state")
    db.commit()

    await _refresh_connected_github_repos(user_id, account_login)
    return RedirectResponse(url="/app?github_connected=1")

@provider_router.post("/providers/{provider}/start", response_model=ProviderAuthStartOut)
async def start_provider_auth(provider: str, request: Request) -> dict[str, Any]:
    """Start a provider OAuth flow through the sidecar."""
    user_id = _current_user_id(request)
    if user_id is None:
        raise HTTPException(status_code=401, detail="Not authenticated")
    tenant, socket_path = await _resolve_provider_sidecar_socket(request, user_id)
    sidecar = None
    try:
        async with tenant_container_activity(request, tenant):
            sidecar = await create_sidecar_connection(socket_path)
            flow = await sidecar.start_oauth_flow(provider)
            payload = _build_provider_auth_start_payload(flow)
            protect_tenant_container(
                request,
                tenant,
                lease_key=_provider_auth_lease_key(payload["flow_id"]),
                timeout_s=_PROVIDER_OAUTH_CONTAINER_LEASE_TIMEOUT_S,
            )
    except (ContainerNotReadyError, ContainerStartError, OSError, SidecarError) as error:
        raise _provider_auth_sidecar_http_error(error) from error
    finally:
        touch_tenant_container(request, tenant)
        if sidecar is not None:
            await sidecar.disconnect()
    return payload

@provider_router.get("/providers/{provider}/callback")
async def callback_provider_auth(provider: str, flow_id: str, request: Request) -> JSONRespo
    """Poll an OAuth flow and persist its credential when complete."""
    user_id = _current_user_id(request)
    if user_id is None:
        raise HTTPException(status_code=401, detail="Not authenticated")
    tenant, socket_path = await _resolve_provider_sidecar_socket(request, user_id)
    lease_key = _provider_auth_lease_key(flow_id)
    sidecar = None
    try:
        async with tenant_container_activity(request, tenant):

```

```

sidecar = await create_sidecar_connection(socket_path)
flow = await sidecar.get_oauth_flow_status(flow_id)
_require_provider_match(provider, flow)
status = _normalize_provider_flow_status(flow.get("status"))
if status in {"pending", "starting"}:
    protect_tenant_container(
        request,
        tenant,
        lease_key=lease_key,
        timeout_s=_PROVIDER_OAUTH_CONTAINER_LEASE_TIMEOUT_S,
    )
    return JSONResponse(
        status_code=202,
        content=_build_provider_auth_status_payload(flow, status=status),
    )
if status == "error":
    release_tenant_container(request, tenant, lease_key=lease_key)
    error_message = flow.get("error")
    if not isinstance(error_message, str) or not error_message:
        error_message = "OAuth flow failed"
    return JSONResponse(
        status_code=400,
        content=_build_provider_auth_status_payload(
            flow,
            status="error",
            error=error_message,
        ),
    )
if status != "complete":
    raise HTTPException(status_code=500, detail=f"Unexpected OAuth status: {status}")

await _persist_completed_provider_auth(
    user_id,
    provider,
    flow,
    sidecar,
)
release_tenant_container(request, tenant, lease_key=lease_key)
return JSONResponse(_build_provider_auth_status_payload(flow, status="complete"))
except (ContainerNotReadyError, ContainerStartError, OSError, SidecarError) as error:
    raise _provider_auth_sidecar_http_error(error) from error
finally:
    touch_tenant_container(request, tenant)
    if sidecar is not None:
        await sidecar.disconnect()

```

```

@provider_router.post("/providers/{provider}/callback", response_model=ProviderAuthStatusOut)
async def submit_provider_auth_callback(
    provider: str,
    payload: ProviderAuthInputIn,
    request: Request,
) -> JSONResponse:
    """Submit pasted OAuth callback data into an active provider auth flow."""
    user_id = _current_user_id(request)
    if user_id is None:
        raise HTTPException(status_code=401, detail="Not authenticated")

    tenant, socket_path = await _resolve_provider_sidecar_socket(request, user_id)
    lease_key = _provider_auth_lease_key(payload.flow_id)
    sidecar = None
    try:
        async with tenant_container_activity(request, tenant):
            sidecar = await create_sidecar_connection(socket_path)
            flow = await sidecar.get_oauth_flow_status(payload.flow_id)
            _require_provider_match(provider, flow)
            status = _normalize_provider_flow_status(flow.get("status"))
            if status == "complete":
                await _persist_completed_provider_auth(
                    user_id,
                    provider,
                    flow,
                    sidecar,
                )
                release_tenant_container(request, tenant, lease_key=lease_key)
                return JSONResponse(
                    _build_provider_auth_status_payload(flow, status="complete"),
                )
            if status == "error":
                release_tenant_container(request, tenant, lease_key=lease_key)
                error_message = flow.get("error")
                if not isinstance(error_message, str) or not error_message:
                    error_message = "OAuth flow failed"
                return JSONResponse(
                    status_code=400,
                    content=_build_provider_auth_status_payload(
                        flow,
                        status="error",
                        error=error_message,
                    ),
                )
    )

```



```

        await sidecar.submit_oauth_flow_input(
            payload.flow_id,
            payload.authorization_input,
        )
    protect_tenant_container(
        request,
        tenant,
        lease_key=lease_key,
        timeout_s=_PROVIDER_OAUTH_CONTAINER_LEASE_TIMEOUT_S,
    )
    return JSONResponse(
        status_code=202,
        content=_build_provider_auth_status_payload(
            {
                **flow,
                "manual_input_required": True,
                "manual_input_submitted": True,
            },
            status="pending",
        ),
    )
except (ContainerNotReadyError, ContainerStartError, OSError, SidecarError) as error:
    raise _provider_auth_sidecar_http_error(error) from error
finally:
    touch_tenant_container(request, tenant)
    if sidecar is not None:
        await sidecar.disconnect()

# --- Backward-compatible provider-neutral OAuth entrypoint ---

@router.get("/login")
async def login_redirect(request: Request) -> Response:
    """Redirect to a configured OAuth provider, preferring GitHub."""
    del request
    settings = get_settings()
    if settings.disable_auth:
        return RedirectResponse(url="/app", status_code=307)
    if settings.github_client_id and settings.github_client_secret:
        return RedirectResponse(url="/auth/login/github", status_code=307)
    if settings.google_client_id and settings.google_client_secret:
        return RedirectResponse(url="/auth/login/google", status_code=307)
    return JSONResponse({"error": "OAuth is not configured"}, status_code=503)

```

```

@router.get("/callback")
async def callback_redirect(request: Request) -> RedirectResponse:
    """Legacy /auth/callback redirects to Google callback.

    Preserves query parameters (state, code, scope) from the OAuth
    provider -- dropping them causes a state mismatch error.
    """
    target = "/auth/callback/google"
    query_string = request.url.query
    if query_string:
        target = f"{target}?{query_string}"
    return RedirectResponse(url=target, status_code=307)

# --- Common endpoints ---

@router.get("/me")
async def me(request: Request) -> dict[str, Any]:
    """Return current user info.

    This endpoint is under /auth/ (an open path), so the middleware
    doesn't populate request.state.tenant. We manually check the cookie.
    """
    if get_settings().disable_auth:
        return {"authenticated": False, "auth_disabled": True}

    token = request.cookies.get("yinshi_session")
    if not token:
        return {"authenticated": False}

    user_id = verify_session_token(token)
    if not user_id:
        return {"authenticated": False}

    tenant = _resolve_tenant_from_user_id(user_id)
    if not tenant:
        return {"authenticated": False}

    return {
        "authenticated": True,
        "email": tenant.email,
        "user_id": tenant.user_id,
    }

```

```

@router.post("/logout")
async def logout(request: Request) -> RedirectResponse:
    """Revoke the current auth session and clear the session cookie."""
    session_identity = _current_session_identity(request)
    if session_identity is not None:
        user_id, auth_session_id = session_identity
        revoke_auth_session(user_id, auth_session_id)
    response = RedirectResponse(url="/")
    _clear_session_cookie(response)
    return response

@router.post("/logout-all")
async def logout_all(request: Request) -> JSONResponse:
    """Revoke all auth sessions for the current user and clear the cookie."""
    user_id = _current_user_id(request)
    if user_id is None:
        raise HTTPException(status_code=401, detail="Not authenticated")

    revoke_auth_sessions(user_id)
    response = JSONResponse({"status": "ok"})
    _clear_session_cookie(response)
    return response

```

5.5 api/deps.py

Route dependencies centralize tenant extraction, database selection, and ownership checks. The root chunk below tangles back to backend/src/yinshi/api/deps.py.

```

"""Shared API dependency helpers (tenant extraction, DB context, legacy auth)."""

import sqlite3
from collections.abc import Iterator
from contextlib import contextmanager

from fastapi import HTTPException, Request

from yinshi.db import get_db
from yinshi.tenant import TenantContext, get_user_db

def get_tenant(request: Request) -> TenantContext | None:
    """Get the TenantContext from request state, or None if auth is disabled."""
    return getattr(request.state, "tenant", None)

```

```

def require_tenant(request: Request) -> TenantContext:
    """Get the TenantContext from request state, raising 401 if missing.

    Use this in endpoints that always require authentication.
    """
    tenant = get_tenant(request)
    if tenant is None:
        raise HTTPException(status_code=401, detail="Authentication required")
    return tenant

@contextmanager
def get_db_for_request(request: Request) -> Iterator[sqlite3.Connection]:
    """Return the correct DB connection for the current request.

    If a tenant is present (multi-tenant mode), returns the user's
    per-tenant database. Otherwise falls back to the shared legacy DB.
    """
    tenant = get_tenant(request)
    application_mode = getattr(request.app.state, "mode", None)
    if tenant and application_mode != "desktop":
        with get_user_db(tenant) as db:
            yield db
    else:
        with get_db() as db:
            yield db

# --- Legacy helpers (kept for backward compatibility during migration) ---

def get_user_email(request: Request) -> str | None:
    """Get authenticated user email, or None if auth is disabled."""
    return getattr(request.state, "user_email", None)

def check_owner(owner_email: str | None, user_email: str | None) -> None:
    """Raise 403 if authenticated user doesn't own the resource.

    Access is allowed when:
    - Auth is disabled (user_email is None)
    - Resource has no owner (owner_email is None, e.g. pre-migration data)
    - Owner matches the authenticated user
    """
    if user_email and owner_email and owner_email != user_email:
        raise HTTPException(status_code=403, detail="Not authorized")

```

```

def check_workspace_owner(
    db: sqlite3.Connection,
    workspace_id: str,
    request: Request,
) -> None:
    """In legacy mode, verify the authenticated user owns the workspace's repo."""
    if get_tenant(request):
        return
    ws = db.execute(
        "SELECT w.id, r.owner_email FROM workspaces w "
        "JOIN repos r ON w.repo_id = r.id WHERE w.id = ?",
        (workspace_id,),
    ).fetchone()
    if ws:
        check_owner(ws["owner_email"], get_user_email(request))
    else:
        raise HTTPException(status_code=404, detail="Workspace not found")

def check_session_owner(
    db: sqlite3.Connection,
    session_id: str,
    request: Request,
) -> None:
    """In legacy mode, verify the authenticated user owns the session's repo."""
    if get_tenant(request):
        return
    row = db.execute(
        "SELECT s.id, r.owner_email FROM sessions s "
        "JOIN workspaces w ON s.workspace_id = w.id "
        "JOIN repos r ON w.repo_id = r.id "
        "WHERE s.id = ?",
        (session_id,),
    ).fetchone()
    if row:
        check_owner(row["owner_email"], get_user_email(request))
    else:
        raise HTTPException(status_code=404, detail="Session not found")

```

6 Repository and Workspace Lifecycle

Repositories are imported once and then materialized into workspaces as Git worktrees. Runtime path helpers keep prompt execution, terminal access, and

file inspection pointed at the same trusted worktree.

6.1 api/repos.py

Repository routes import, list, update, and delete local or GitHub repositories after validating ownership and clone access. The root chunk below tangles back to `backend/src/yinshi/api/repos.py`.

```
"""CRUD endpoints for repositories."""

import logging
import sqlite3
import uuid
from contextlib import nullcontext
from pathlib import Path
from typing import Any

from fastapi import APIRouter, HTTPException, Request

from yinshi.api.deps import check_owner, get_db_for_request, get_tenant, get_user_email
from yinshi.config import get_settings
from yinshi.exceptions import (
    GitError,
    GitHubAccessError,
    GitHubAccessNotGrantedError,
    GitHubAppError,
    GitHubConnectRequiredError,
)
from yinshi.models import RepoCreate, RepoOut, RepoUpdate
from yinshi.rate_limit import limiter
from yinshi.services.git import (
    cleanup_repository_worktrees,
    clone_local_repo,
    clone_repo,
    validate_local_repo,
)
from yinshi.services.github_app import GitHubCloneAccess, resolve_github_clone_access
from yinshi.services.repository_lifecycle import (
    ManagedPathQuarantine,
    repository_lifecycle,
    repository_lifecycle_root,
)
from yinshi.services.run_coordinator import get_run_coordinator
from yinshi.services.sidecar import release_sessions
from yinshi.services.sidecar_runtime import (
    _workspace_home_expected_path,
```

```

        local_pi_session_file,
    )
    from yinshi.tenant import TenantContext, validate_user_path
    from yinshi.utils.paths import is_path_inside

    logger = logging.getLogger(__name__)
    router = APIRouter(prefix="/api/repos", tags=["repos"])

    # Only these columns can be updated via PATCH
    _UPDATABLE_COLUMNS = {"name", "custom_prompt", "agents_md"}
    _RUNTIME_BUSY_DETAIL = "Workspace is still stopping; deletion can be retried"

def _validate_local_path(path_str: str) -> str:
    """Validate and resolve a local path, checking against allowed base.

    Fail-closed: if ``allowed_repo_base`` is not configured, all local
    imports are rejected.
    """
    settings = get_settings()
    if not settings.allowed_repo_base:
        raise HTTPException(
            status_code=400,
            detail="Local repo imports are disabled (allowed_repo_base not set)",
        )
    resolved = str(Path(path_str).resolve())
    if not is_path_inside(resolved, settings.allowed_repo_base):
        raise HTTPException(status_code=400, detail="Path not in allowed directory")
    return resolved

def _check_repo_owner(row: sqlite3.Row, request: Request) -> None:
    """In legacy mode, verify the authenticated user owns the repo."""
    tenant = get_tenant(request)
    if not tenant:
        check_owner(row["owner_email"], get_user_email(request))

def _github_connect_url(request: Request) -> str | None:
    """Return the GitHub connect URL when the feature is usable for this request."""
    settings = get_settings()
    if not settings.github_app_slug:
        return None
    if get_tenant(request) is None:
        return None
    return "/auth/github/install"

```

```

def _github_http_exception(error: GitHubAccessError) -> HTTPException:
    """Convert a GitHub access error into a structured HTTP error."""
    detail = {
        "code": error.code,
        "message": str(error),
        "connect_url": error.connect_url,
        "manage_url": error.manage_url,
    }
    return HTTPException(status_code=400, detail=detail)

async def _resolve_clone_access(
    request: Request,
    remote_url: str,
) -> GitHubCloneAccess | None:
    """Resolve GitHub clone credentials for a remote, if applicable."""
    tenant = get_tenant(request)
    user_id = tenant.user_id if tenant else None
    try:
        return await resolve_github_clone_access(user_id, remote_url)
    except GitHubAccessError as error:
        raise _github_http_exception(error)
    except GitHubAppError as error:
        logger.error("GitHub integration failed during repository credential resolution")
        raise HTTPException(status_code=502, detail=str(error))

def _github_clone_failure(
    request: Request,
    clone_access: GitHubCloneAccess,
) -> HTTPException:
    """Translate an anonymous GitHub clone failure into an actionable error."""
    assert clone_access.access_token is None, "clone failure helper expects anonymous access"
    if clone_access.manage_url:
        return _github_http_exception(
            GitHubAccessNotGrantedError(
                "Grant this repository to the connected GitHub installation and try again.",
                manage_url=clone_access.manage_url,
            )
        )
    return _github_http_exception(
        GitHubConnectRequiredError(
            "Connect GitHub to import this private repository.",
            connect_url=_github_connect_url(request),
        )
    )

```



```

    )
)

@router.get("", response_model=list[RepoOut])
def list_repos(request: Request) -> list[dict[str, Any]]:
    """List all imported repositories."""
    tenant = get_tenant(request)
    email = None if tenant else get_user_email(request)

    with get_db_for_request(request) as db:
        if email:
            rows = db.execute(
                "SELECT * FROM repos WHERE owner_email = ? OR owner_email IS NULL "
                "ORDER BY created_at DESC",
                (email,),
            ).fetchall()
        else:
            rows = db.execute("SELECT * FROM repos ORDER BY created_at DESC").fetchall()
    return [dict(r) for r in rows]

@router.post("", response_model=RepoOut, status_code=201)
@limiter.limit("10/hour")
async def import_repo(body: RepoCreate, request: Request) -> dict[str, Any]:
    """Import a repository (clone from URL or register local path)."""
    tenant = get_tenant(request)
    settings = get_settings()
    normalized_remote_url = body.remote_url
    clone_access: GitHubCloneAccess | None = None
    repo_id: str | None = None

    if body.local_path:
        resolved = _validate_local_path(body.local_path)
        if not Path(resolved).is_dir():
            raise HTTPException(status_code=400, detail="Path does not exist")
        is_repo = await validate_local_repo(resolved)
        if not is_repo:
            raise HTTPException(status_code=400, detail="Not a valid git repository")
        if tenant and settings.container_enabled:
            repo_id = uuid.uuid4().hex
            root_path = str(Path(tenant.data_dir) / "repos" / repo_id)
            try:
                await clone_local_repo(resolved, root_path)
            except GitError as error:
                raise HTTPException(status_code=400, detail=str(error)) from error

```

```

        else:
            root_path = resolved
    elif body.remote_url:
        clone_access = await _resolve_clone_access(request, body.remote_url)
        access_token = None
        if clone_access is not None:
            normalized_remote_url = clone_access.clone_url
            access_token = clone_access.access_token

        # Sanitize name to prevent path traversal.
        safe_name = Path(body.name).name
        if not safe_name or safe_name != body.name or ".." in body.name:
            raise HTTPException(
                status_code=400,
                detail="Invalid repository name (must be a simple directory name)",
            )

        if tenant:
            clone_dir = str(Path(tenant.data_dir) / "repos" / safe_name)
        else:
            clone_dir = str(Path.home() / ".yinshi" / "repos" / safe_name)
        try:
            root_path = await clone_repo(
                normalized_remote_url or body.remote_url,
                clone_dir,
                access_token=access_token,
            )
        except GitError as e:
            if clone_access is not None and clone_access.access_token is None:
                if clone_access.repository_installation_id is not None:
                    raise _github_clone_failure(request, clone_access)
                if clone_access.manage_url is not None:
                    raise _github_clone_failure(request, clone_access)
            else:
                assert clone_access is None or clone_access.access_token is not None
                raise HTTPException(status_code=400, detail=str(e))
    else:
        raise HTTPException(status_code=400, detail="Either remote_url or local_path is required")

    installation_id = None
    if clone_access is not None and clone_access.access_token is not None:
        installation_id = clone_access.installation_id
    with get_db_for_request(request) as db:
        if tenant:
            if repo_id is None:
                cursor = db.execute(

```

```

        """INSERT INTO repos (name, remote_url, root_path, custom_prompt, agents_md,
        VALUES (?, ?, ?, ?, ?, ?)"""",
        (
            body.name,
            normalized_remote_url,
            root_path,
            body.custom_prompt,
            body.agents_md,
            installation_id,
        ),
    )
else:
    cursor = db.execute(
        """INSERT INTO repos (id, name, remote_url, root_path, custom_prompt, agents_md,
        VALUES (?, ?, ?, ?, ?, ?, ?)"""",
        (
            repo_id,
            body.name,
            normalized_remote_url,
            root_path,
            body.custom_prompt,
            body.agents_md,
            installation_id,
        ),
    )
else:
    email = get_user_email(request)
    cursor = db.execute(
        """INSERT INTO repos (name, remote_url, root_path, custom_prompt, agents_md, email,
        VALUES (?, ?, ?, ?, ?, ?, ?)"""",
        (
            body.name,
            normalized_remote_url,
            root_path,
            body.custom_prompt,
            body.agents_md,
            email,
            installation_id,
        ),
    )
db.commit()
row = db.execute("SELECT * FROM repos WHERE rowid = ?", (cursor.lastrowid,)).fetchone()
return dict(row)

```

```

@router.get("/{repo_id}", response_model=RepoOut)

```

```

def get_repo(repo_id: str, request: Request) -> dict[str, Any]:
    """Get a single repository by ID."""
    with get_db_for_request(request) as db:
        row = db.execute("SELECT * FROM repos WHERE id = ?", (repo_id,)).fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Repo not found")
        _check_repo_owner(row, request)
        return dict(row)

@router.patch("/{repo_id}", response_model=RepoOut)
def update_repo(
    repo_id: str,
    body: RepoUpdate,
    request: Request,
) -> dict[str, Any]:
    """Update a repository."""
    with get_db_for_request(request) as db:
        row = db.execute("SELECT * FROM repos WHERE id = ?", (repo_id,)).fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Repo not found")
        _check_repo_owner(row, request)

        updates = {
            k: v for k, v in body.model_dump(exclude_unset=True).items() if k in _UPDATABLES
        }
        if updates:
            set_clause = ", ".join(f"{k} = ?" for k in updates)
            values = list(updates.values()) + [repo_id]
            db.execute(f"UPDATE repos SET {set_clause} WHERE id = ?", values)
            db.commit()
        row = db.execute("SELECT * FROM repos WHERE id = ?", (repo_id,)).fetchone()
        return dict(row)

def _workspace_path_plans(
    repo: sqlite3.Row,
    workspaces: list[sqlite3.Row],
) -> list[tuple[Path, Path, str]]:
    """Return managed worktrees after checking their repository ownership."""
    repo_root = Path(str(repo["root_path"]))
    worktree_root = repo_root / ".worktrees"
    allowed_repo_base = Path(get_settings().allowed_repo_base)
    plans: list[tuple[Path, Path, str]] = []
    for workspace in workspaces:
        workspace_path = Path(str(workspace["path"]))

```

```

        if workspace_path.exists() and not _workspace_belongs_to_repo(
            repo_root,
            workspace_path,
        ):
            raise ValueError("workspace is not owned by its repository")
        branch = str(workspace["branch"])
        if is_path_inside(str(workspace_path), str(worktree_root)):
            plans.append((workspace_path, worktree_root, branch))
        elif get_settings().allowed_repo_base and is_path_inside(
            str(workspace_path),
            str(allowed_repo_base),
        ):
            plans.append((workspace_path, allowed_repo_base, branch))
        else:
            raise ValueError("workspace path is outside its managed root")
    return plans

def _workspace_belongs_to_repo(repo_root: Path, workspace_path: Path) -> bool:
    """Check a linked worktree's gitdir points into the selected repository."""
    git_file = workspace_path / ".git"
    if git_file.is_symlink() or not git_file.is_file():
        return False
    try:
        if git_file.stat().st_size > 4096:
            return False
        content = git_file.read_text(encoding="utf-8").strip()
    except (OSError, UnicodeError):
        return False
    prefix = "gitdir: "
    if not content.startswith(prefix):
        return False
    git_directory = Path(content[len(prefix) :])
    if not git_directory.is_absolute():
        git_directory = workspace_path / git_directory
    return is_path_inside(
        str(git_directory),
        str(repo_root / ".git" / "worktrees"),
    )

def _managed_repo_plan(
    repo: sqlite3.Row,
    tenant: TenantContext | None,
) -> tuple[Path, Path] | None:
    """Return a Yinshi-owned checkout and its trusted root."""

```

```

root_path = Path(str(repo["root_path"]))
if tenant is not None:
    if not is_path_inside(str(root_path), tenant.data_dir):
        return None
    validate_user_path(tenant, str(root_path))
    return root_path, Path(tenant.data_dir)

legacy_repo_directory = Path.home() / ".yinshi" / "repos"
if bool(repo["remote_url"]) and is_path_inside(
    str(root_path),
    str(legacy_repo_directory),
):
    return root_path, legacy_repo_directory
return None

@router.delete("/{repo_id}", status_code=204)
async def delete_repo(repo_id: str, request: Request) -> None:
    """Delete a repository while holding its lifecycle lock."""
    tenant = get_tenant(request)
    with get_db_for_request(request) as db:
        lock_root = repository_lifecycle_root(db, tenant)
        async with repository_lifecycle(repo_id, lock_root):
            await _delete_repo_locked(repo_id, request, db)

async def _delete_repo_locked(
    repo_id: str,
    request: Request,
    database: sqlite3.Connection,
) -> None:
    """Delete a repository after lifecycle serialization."""
    with nullcontext(database) as db:
        row = db.execute("SELECT * FROM repos WHERE id = ?", (repo_id,)).fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Repo not found")
        _check_repo_owner(row, request)
        tenant = get_tenant(request)
        workspace_rows = db.execute(
            "SELECT * FROM workspaces WHERE repo_id = ? ORDER BY rowid ASC",
            (repo_id,),
        ).fetchall()
        session_ids: list[str] = []
        try:
            coordinator = get_run_coordinator()
            for workspace in workspace_rows:

```

```

workspace_id = str(workspace["id"])
session_rows = db.execute(
    "SELECT id FROM sessions WHERE workspace_id = ?",
    (workspace_id,),
).fetchall()
for session_row in session_rows:
    session_id = str(session_row["id"])
    session_ids.append(session_id)
    await coordinator.request_cancel(session_id)

busy_workspace_found = False
if request.app.state.mode == "desktop":
    await release_sessions(get_settings().sidecar_socket_path, session_ids)
elif tenant is not None:
    container_manager = getattr(request.app.state, "container_manager", None)
    if container_manager is None and get_settings().container_enabled:
        raise RuntimeError("container manager is unavailable")
    if container_manager is not None:
        for workspace in workspace_rows:
            workspace_id = str(workspace["id"])
            container_removed = await container_manager.destroy_container(
                tenant.user_id,
                runtime_id=workspace_id,
            )
            if not container_removed:
                busy_workspace_found = True
except (OSError, RuntimeError, ValueError):
    logger.error("Failed to stop workspace runtime while deleting repository")
    raise HTTPException(
        status_code=500,
        detail="Repository cleanup failed; deletion can be retried",
    ) from None

if busy_workspace_found:
    raise HTTPException(status_code=409, detail=_RUNTIME_BUSY_DETAIL)

quarantine = ManagedPathQuarantine()
try:
    workspace_plans = _workspace_path_plans(row, list(workspace_rows))
    runtime_root = Path(tenant.data_dir) if tenant is not None else None
    runtime_paths = (
        [
            _workspace_home_expected_path(tenant, str(workspace["id"]))
            for workspace in workspace_rows
        ]
    )
    if tenant is not None

```

```

        else []
    )
    session_paths = (
        [Path(local_pi_session_file(session_id)) for session_id in session_ids]
        if tenant is None
        else []
    )
    managed_repo = _managed_repo_plan(row, tenant)
    cleanup_repo_root = Path(str(row["root_path"]))
    git_worktrees = [
        (str(workspace_path), branch)
        for workspace_path, _workspace_root, branch in workspace_plans
    ]
    if runtime_root is not None:
        for runtime_path in runtime_paths:
            quarantine.validate(runtime_path, runtime_root)
    for session_path in session_paths:
        quarantine.validate(session_path, session_path.parent)
    for workspace_path, workspace_root, _branch in workspace_plans:
        quarantine.validate(workspace_path, workspace_root)
    if managed_repo is not None:
        quarantine.validate(*managed_repo)
    if runtime_root is not None:
        for runtime_path in runtime_paths:
            quarantine.move(runtime_path, runtime_root)
    for session_path in session_paths:
        quarantine.move(session_path, session_path.parent)
    for workspace_path, workspace_root, _branch in workspace_plans:
        quarantine.move(workspace_path, workspace_root)
    if managed_repo is not None:
        quarantine.move(*managed_repo)
    for entry in reversed(quarantine.entries):
        if entry.source == managed_repo[0]:
            cleanup_repo_root = entry.target
            break
    db.execute("DELETE FROM repos WHERE id = ?", (repo_id,))
    db.commit()
except (OSError, RuntimeError, ValueError, sqlite3.Error):
    db.rollback()
    try:
        quarantine.restore()
    except (OSError, RuntimeError, ValueError):
        logger.error("Repository path restoration failed")
    logger.error("Repository deletion failed before database commit")
    raise HTTPException(
        status_code=500,

```



```

        detail="Repository cleanup failed; deletion can be retried",
    ) from None

    try:
        await cleanup_repository_worktrees(
            str(cleanup_repo_root),
            git_worktrees,
        )
    except (GitError, OSError, ValueError):
        logger.error("Repository Git cleanup failed")

    try:
        quarantine.discard()
    except (OSError, RuntimeError, ValueError):
        logger.error("Repository durable cleanup failed")

```

6.2 api/workspaces.py

Workspace routes create and remove Git worktrees while keeping branch state scoped to the owning repository. The root chunk below tangles back to `backend/src/yinshi/api/workspaces.py`.

"""Endpoints for workspace (worktree) management."""

```

import logging
import sqlite3
from typing import Any

from fastapi import APIRouter, HTTPException, Request

from yinshi.api.deps import (
    check_owner,
    check_workspace_owner,
    get_db_for_request,
    get_tenant,
    get_user_email,
)
from yinshi.config import get_settings
from yinshi.exceptions import GitError, RepoNotFoundError, WorkspaceNotFoundError
from yinshi.models import WorkspaceCreate, WorkspaceOut, WorkspaceUpdate
from yinshi.services.run_coordinator import get_run_coordinator
from yinshi.services.sidecar import release_sessions
from yinshi.services.workspace import create_workspace_for_repo, delete_workspace

logger = logging.getLogger(__name__)
router = APIRouter(tags=["workspaces"])

```

```

_UPDATABLE_COLUMNS = {"state"}
_RUNTIME_BUSY_DETAIL = "Workspace is still stopping; deletion can be retried"

def _check_repo_owner(
    db: sqlite3.Connection,
    repo_id: str,
    request: Request,
) -> None:
    """In legacy mode, verify the authenticated user owns the repo."""
    if get_tenant(request):
        return
    repo = db.execute("SELECT owner_email FROM repos WHERE id = ?", (repo_id,)).fetchone()
    if repo:
        check_owner(repo["owner_email"], get_user_email(request))
    else:
        raise HTTPException(status_code=404, detail="Repo not found")

@router.get("/api/repos/{repo_id}/workspaces", response_model=list[WorkspaceOut])
def list_workspaces(repo_id: str, request: Request) -> list[dict[str, Any]]:
    """List all workspaces for a repo."""
    with get_db_for_request(request) as db:
        _check_repo_owner(db, repo_id, request)
        rows = db.execute(
            "SELECT * FROM workspaces WHERE repo_id = ? ORDER BY created_at DESC",
            (repo_id,),
        ).fetchall()
        return [dict(r) for r in rows]

@router.post(
    "/api/repos/{repo_id}/workspaces",
    response_model=WorkspaceOut,
    status_code=201,
)
async def create_workspace(
    repo_id: str,
    body: WorkspaceCreate,
    request: Request,
) -> dict[str, Any]:
    """Create a new worktree workspace."""
    email = get_user_email(request)
    username = email.split("@")[0] if email else None
    tenant = get_tenant(request)

```

```

with get_db_for_request(request) as db:
    _check_repo_owner(db, repo_id, request)
    try:
        return await create_workspace_for_repo(
            db,
            repo_id,
            body.name,
            username=username,
            tenant=tenant,
        )
    except RepoNotFoundError:
        raise HTTPException(status_code=404, detail="Repo not found")
    except GitError as exc:
        raise HTTPException(status_code=409, detail=str(exc))

@router.patch("/api/workspaces/{workspace_id}", response_model=WorkspaceOut)
def update_workspace(
    workspace_id: str,
    body: WorkspaceUpdate,
    request: Request,
) -> dict[str, Any]:
    """Update workspace fields (currently only state)."""
    with get_db_for_request(request) as db:
        row = db.execute("SELECT * FROM workspaces WHERE id = ?", (workspace_id,)).fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Workspace not found")
        check_workspace_owner(db, workspace_id, request)

        updates = {
            k: v for k, v in body.model_dump(exclude_unset=True).items() if k in _UPDATABLE_
        }
        if updates:
            sets = ", ".join(f"{k} = ?" for k in updates)
            vals = list(updates.values()) + [workspace_id]
            db.execute(f"UPDATE workspaces SET {sets} WHERE id = ?", vals) # noqa: S608
            db.commit()
        updated = db.execute("SELECT * FROM workspaces WHERE id = ?", (workspace_id,)).fetchone()
        return dict(updated)

@router.delete("/api/workspaces/{workspace_id}", status_code=204)
async def remove_workspace(workspace_id: str, request: Request) -> None:
    """Stop runtime activity, then delete a workspace and its durable paths."""
    tenant = get_tenant(request)

```

```

with get_db_for_request(request) as db:
    check_workspace_owner(db, workspace_id, request)
    session_rows = db.execute(
        "SELECT id FROM sessions WHERE workspace_id = ?",
        (workspace_id,),
    ).fetchall()
    try:
        coordinator = get_run_coordinator()
        for session_row in session_rows:
            await coordinator.request_cancel(str(session_row["id"]))
        if request.app.state.mode == "desktop":
            # Desktop shares one long-lived sidecar, so these pi sessions
            # would otherwise stay resident until the app quits. Hosted
            # mode destroys the whole container below instead.
            await release_sessions(
                get_settings().sidecar_socket_path,
                [str(session_row["id"]) for session_row in session_rows],
            )
        elif tenant is not None:
            container_manager = getattr(request.app.state, "container_manager", None)
            if container_manager is not None:
                container_removed = await container_manager.destroy_container(
                    tenant.user_id,
                    runtime_id=workspace_id,
                )
                if not container_removed:
                    raise HTTPException(status_code=409, detail=_RUNTIME_BUSY_DETAIL)
            elif get_settings().container_enabled:
                raise RuntimeError("container manager is unavailable")
            await delete_workspace(db, workspace_id, tenant=tenant)
        except (WorkspaceNotFoundError, RepoNotFoundError):
            raise HTTPException(status_code=404, detail="Workspace not found")
        except HTTPException:
            raise
        except Exception:
            logger.error("Failed to delete workspace")
            raise HTTPException(status_code=500, detail="Failed to delete workspace")

```

6.3 services/git.py

Git services clone repositories, maintain remotes, create worktrees, and run git commands with explicit timeouts and auth environment. The root chunk below tangles back to backend/src/yinshi/services/git.py.

```

"""Git operations: clone repos and manage worktrees."""

```

```

import asyncio
import logging
import os
import secrets
import string
import tempfile
from collections.abc import Iterator
from contextlib import contextmanager
from pathlib import Path
from urllib.parse import urlparse

from yinshi.exceptions import GitError

logger = logging.getLogger(__name__)

_ADJECTIVES = [
    "swift",
    "bold",
    "calm",
    "dark",
    "keen",
    "warm",
    "cool",
    "pure",
    "wise",
    "fast",
    "bright",
    "quiet",
    "sharp",
    "smooth",
    "steady",
    "gentle",
    "vivid",
    "grand",
    "noble",
    "fresh",
    "prime",
    "lunar",
    "solar",
    "amber",
    "coral",
    "ivory",
    "olive",
    "azure",
]

_NOUNS = [

```

```

"fox",
"owl",
"elk",
"wolf",
"hawk",
"bear",
"lynx",
"crane",
"drake",
"finch",
"heron",
"raven",
"otter",
"tiger",
"eagle",
"falcon",
"panda",
"bison",
"cedar",
"maple",
"river",
"stone",
"flame",
"frost",
"storm",
"ridge",
"grove",
"brook",
]

```

```

_GIT_COMMAND_TIMEOUT_S = 300.0
_GIT_EXECUTABLE_PATH = "/usr/bin/git"
_GITHUB_HOST = "github.com"

```

```

def generate_branch_name(username: str | None = None) -> str:
    """Generate a random branch name like 'username/swift-fox-a3f2'."""
    adjective = secrets.choice(_ADJECTIVES)
    noun = secrets.choice(_NOUNS)
    suffix = "".join(secrets.choice(string.ascii_lowercase + string.digits) for _ in range(4))
    bare = f"{adjective}-{noun}-{suffix}"
    if username:
        return f"{username}/{bare}"
    return bare

```

```

def _validate_clone_url(url: str) -> None:
    """Allow only canonical GitHub HTTPS repository URLs on the host."""
    if not isinstance(url, str):
        raise TypeError("url must be a string")
    if url.startswith("-"):
        raise GitError("Invalid repository URL")
    if url.startswith(("ext:", "file://")):
        raise GitError("URL scheme not allowed")

    parsed = urlparse(url)
    try:
        port = parsed.port
    except ValueError as exc:
        raise GitError("Only canonical GitHub HTTPS repository URLs are allowed") from exc
    path_parts = [part for part in parsed.path.split("/") if part]
    canonical = (
        parsed.scheme == "https"
        and parsed.hostname == _GITHUB_HOST
        and parsed.username is None
        and parsed.password is None
        and port is None
        and len(path_parts) == 2
        and not parsed.query
        and not parsed.fragment
    )
    if not canonical:
        raise GitError("Only canonical GitHub HTTPS repository URLs are allowed")

@contextmanager
def _git_askpass_env(access_token: str | None) -> Iterator[dict[str, str] | None]:
    """Provide temporary environment variables for HTTPS token auth."""
    if access_token is None:
        yield None
        return

    if not access_token:
        raise GitError("Git access token must not be empty")

    with tempfile.TemporaryDirectory(prefix="yinshi-git-askpass-") as temp_dir:
        askpass_path = Path(temp_dir) / "askpass.sh"
        askpass_path.write_text(
            "#!/bin/sh\n"
            'case "$1" in\n'
            "  *Username*) printf '%s\\n' 'x-access-token' ;;\n"
            "  *) printf '%s\\n' \"${YINSHI_GIT_TOKEN}\" ;;\n"
        )

```

```

        "esac\n",
        encoding="utf-8",
    )
    askpass_path.chmod(0o700)
    yield {
        "GIT_ASKPASS": str(askpass_path),
        "GIT_TERMINAL_PROMPT": "0",
        "YINSHI_GIT_TOKEN": access_token,
    }

async def _run_git(
    args: list[str],
    cwd: str | None = None,
    env: dict[str, str] | None = None,
) -> str:
    """Run a git command asynchronously and return stdout."""
    if not args:
        raise ValueError("args must not be empty")
    cmd = [_GIT_EXECUTABLE_PATH, *args]
    logger.debug("Running git operation %s", args[0])
    child_env = {
        "GCM_INTERACTIVE": "Never",
        "GIT_CONFIG_GLOBAL": os.devnull,
        "GIT_CONFIG_NOSYSTEM": "1",
        "GIT_PAGER": "cat",
        "GIT_TERMINAL_PROMPT": "0",
        "HOME": "/nonexistent",
        "LANG": os.environ.get("LANG", "C.UTF-8"),
        "PATH": "/usr/bin:/bin",
    }
    if env is not None:
        child_env.update(env)
    proc = await asyncio.create_subprocess_exec(
        *cmd,
        cwd=cwd,
        env=child_env,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    try:
        stdout, _stderr = await asyncio.wait_for(
            proc.communicate(),
            timeout=_GIT_COMMAND_TIMEOUT_S,
        )
    except TimeoutError as exc:

```



```

        proc.kill()
        await proc.wait()
        raise GitError(f"git {args[0]} timed out") from exc
    if proc.returncode != 0:
        logger.error("Git operation %s failed", args[0])
        raise GitError(f"git {args[0]} failed")
    return stdout.decode().strip()

def _normalize_remote_url_for_compare(url: str) -> str:
    """Normalize a remote URL enough to compare logical equality."""
    if not url:
        raise ValueError("url must not be empty")
    normalized_url = url.strip()
    if not normalized_url:
        raise ValueError("url must not be blank")
    if normalized_url.endswith(".git"):
        normalized_url = normalized_url[:-4]
    return normalized_url.rstrip("/")

def _remote_urls_match(existing_remote_url: str, expected_remote_url: str) -> bool:
    """Return whether two remote URLs refer to the same repository."""
    if not isinstance(existing_remote_url, str):
        raise TypeError("existing_remote_url must be a string")
    if not isinstance(expected_remote_url, str):
        raise TypeError("expected_remote_url must be a string")
    if not existing_remote_url.strip():
        return False
    if not expected_remote_url.strip():
        raise ValueError("expected_remote_url must not be blank")
    return _normalize_remote_url_for_compare(
        existing_remote_url
    ) == _normalize_remote_url_for_compare(expected_remote_url)

async def _has_remote_refs(repo_path: str, remote_name: str = "origin") -> bool:
    """Return whether one local checkout has fetched refs for one remote."""
    if not isinstance(repo_path, str):
        raise TypeError("repo_path must be a string")
    if not isinstance(remote_name, str):
        raise TypeError("remote_name must be a string")
    normalized_repo_path = repo_path.strip()
    normalized_remote_name = remote_name.strip()
    if not normalized_repo_path:
        raise ValueError("repo_path must not be empty")

```

```

if not normalized_remote_name:
    raise ValueError("remote_name must not be empty")

refs_output = await _run_git(
    [
        "for-each-ref",
        "--format=%(refname)",
        f"refs/remotes/{normalized_remote_name}",
    ],
    cwd=normalized_repo_path,
)
for ref_name in refs_output.splitlines():
    normalized_ref_name = ref_name.strip()
    if not normalized_ref_name:
        continue
    if normalized_ref_name == f"refs/remotes/{normalized_remote_name}/HEAD":
        continue
    return True
return False

async def get_remote_url(
    repo_path: str,
    remote_name: str = "origin",
) -> str | None:
    """Return one configured remote URL, or None when it is missing."""
    if not repo_path:
        raise ValueError("repo_path must not be empty")
    if not remote_name:
        raise ValueError("remote_name must not be empty")

    try:
        remote_url = await _run_git(
            ["remote", "get-url", remote_name],
            cwd=repo_path,
        )
    except GitError:
        return None

    normalized_remote_url = remote_url.strip()
    if not normalized_remote_url:
        return None
    return normalized_remote_url

async def ensure_remote_url(

```

```

repo_path: str,
remote_url: str,
remote_name: str = "origin",
) -> bool:
    """Ensure a checkout points one named remote at the expected URL."""
    if not repo_path:
        raise ValueError("repo_path must not be empty")
    if not remote_name:
        raise ValueError("remote_name must not be empty")
    if not remote_url:
        raise ValueError("remote_url must not be empty")

    current_remote_url = await get_remote_url(repo_path, remote_name=remote_name)
    if current_remote_url is not None:
        if _normalize_remote_url_for_compare(
            current_remote_url
        ) == _normalize_remote_url_for_compare(remote_url):
            return False
        await _run_git(
            ["remote", "set-url", remote_name, remote_url],
            cwd=repo_path,
        )
        return True

    await _run_git(
        ["remote", "add", remote_name, remote_url],
        cwd=repo_path,
    )
    return True

async def clone_repo(
    url: str,
    dest: str,
    access_token: str | None = None,
) -> str:
    """Clone a git repository. Returns the clone path.

    If dest already exists and is a valid git repo with matching remote, reuse it.
    """
    _validate_clone_url(url)

    dest_path = Path(dest)
    if dest_path.exists():
        if await validate_local_repo(dest):
            # Verify the existing clone's remote matches the requested URL

```

```

        # before reusing it to prevent cross-repo data leakage.
    try:
        existing_remote = await _run_git(
            ["remote", "get-url", "origin"],
            cwd=dest,
        )
    except GitError:
        existing_remote = ""
    if not _remote_urls_match(existing_remote, url):
        raise GitError("Destination already contains a clone of a different repository")
    had_remote_refs_before_fetch = await _has_remote_refs(dest)
    logger.info("Reusing an existing repository clone")
    try:
        with _git_askpass_env(access_token) as env:
            await _run_git(["fetch", "--all"], cwd=dest, env=env)
    except GitError as error:
        if not had_remote_refs_before_fetch:
            raise GitError(
                "Existing clone is incomplete and could not be refreshed"
            ) from error
        logger.warning("Repository refresh failed; reusing existing refs")
        return dest
    if not had_remote_refs_before_fetch and not await _has_remote_refs(dest):
        raise GitError("Existing clone is incomplete and missing remote refs")
    return dest
    raise GitError("Destination already exists but is not a git repository")
dest_path.parent.mkdir(parents=True, exist_ok=True)
with _git_askpass_env(access_token) as env:
    await _run_git(["clone", url, dest], env=env)
logger.info("Repository clone completed")
return dest

async def clone_local_repo(
    source: str,
    dest: str,
    remote_url: str | None = None,
) -> str:
    """Clone a local git repository for tenant path repairs.

    Using the existing checkout as the clone source preserves local branches
    that may not have been pushed to the remote yet.
    """
    if not await validate_local_repo(source):
        raise GitError("Source repository is not a valid git repository")

```

```

dest_path = Path(dest)
if dest_path.exists():
    if not await validate_local_repo(dest):
        raise GitError("Destination already exists but is not a git repository")
else:
    dest_path.parent.mkdir(parents=True, exist_ok=True)
    await _run_git(["clone", "--no-hardlinks", source, dest])

if remote_url:
    await _run_git(["remote", "set-url", "origin", remote_url], cwd=dest)

logger.info("Local repository clone completed")
return dest

async def resolve_remote_base_ref(
    repo_path: str,
    access_token: str | None = None,
) -> str:
    """Fetch origin and return the tracked default remote branch reference."""
    assert repo_path, "repo_path must not be empty"

    with _git_askpass_env(access_token) as env:
        await _run_git(["fetch", "origin"], cwd=repo_path, env=env)
        try:
            symbolic_ref = await _run_git(
                ["symbolic-ref", "refs/remotes/origin/HEAD"],
                cwd=repo_path,
                env=env,
            )
        except GitError:
            symbolic_ref = ""

    normalized_symbolic_ref = symbolic_ref.strip()
    if normalized_symbolic_ref.startswith("refs/remotes/origin/"):
        remote_branch = normalized_symbolic_ref.removeprefix("refs/remotes/")
        assert remote_branch, "remote_branch must not be empty"
        return remote_branch

    for fallback_remote_branch in ("origin/main", "origin/master"):
        try:
            await _run_git(
                ["rev-parse", "--verify", fallback_remote_branch],
                cwd=repo_path,
            )
        except GitError:

```

```

        continue
    return fallback_remote_branch

raise GitError("Could not determine the remote default branch")

async def create_worktree(
    repo_path: str,
    worktree_path: str,
    branch: str,
    *,
    base_ref: str | None = None,
) -> str:
    """Create a git worktree with a new branch. Returns the worktree path."""
    assert repo_path, "repo_path must not be empty"
    assert worktree_path, "worktree_path must not be empty"
    assert branch, "branch must not be empty"

    Path(worktree_path).parent.mkdir(parents=True, exist_ok=True)
    worktree_add_args = ["worktree", "add", "-b", branch, worktree_path]
    if base_ref is not None:
        normalized_base_ref = base_ref.strip()
        if not normalized_base_ref:
            raise ValueError("base_ref must not be empty when provided")
        worktree_add_args.append(normalized_base_ref)
    await _run_git(worktree_add_args, cwd=repo_path)
    logger.info("Repository worktree created")
    return worktree_path

async def restore_worktree(repo_path: str, worktree_path: str, branch: str) -> str:
    """Restore a worktree for an existing branch, creating the branch if needed."""
    assert repo_path, "repo_path must not be empty"
    assert worktree_path, "worktree_path must not be empty"
    assert branch, "branch must not be empty"

    worktree_dir = Path(worktree_path)
    if worktree_dir.exists():
        if await validate_local_repo(worktree_path):
            return worktree_path
        raise GitError("Worktree path already exists but is not a git repository")

    worktree_dir.parent.mkdir(parents=True, exist_ok=True)
    try:
        await _run_git(["worktree", "add", worktree_path, branch], cwd=repo_path)
    except GitError:

```

```

        await _run_git(["worktree", "add", "-b", branch, worktree_path], cwd=repo_path)

    logger.info("Repository worktree restored")
    return worktree_path

async def delete_worktree(repo_path: str, worktree_path: str) -> None:
    """Remove a git worktree and its branch."""
    try:
        branch = await _run_git(
            ["rev-parse", "--abbrev-ref", "HEAD"],
            cwd=worktree_path,
        )
    except GitError:
        branch = None

    await _run_git(["worktree", "remove", "--force", worktree_path], cwd=repo_path)

    if branch and branch not in ("main", "master"):
        try:
            await _run_git(["branch", "-D", branch], cwd=repo_path)
        except GitError:
            pass

    logger.info("Repository worktree deleted")

async def cleanup_repository_worktrees(
    repo_path: str,
    worktrees: list[tuple[str, str]],
) -> None:
    """Remove selected linked-worktree metadata and local branches after commit."""
    if not repo_path:
        raise ValueError("repo_path must not be empty")
    for worktree_path, branch in worktrees:
        if not worktree_path or not branch:
            raise ValueError("worktree cleanup values must not be empty")

    first_error: GitError | None = None
    for worktree_path, _branch in worktrees:
        try:
            await _run_git(
                ["worktree", "remove", "--force", worktree_path],
                cwd=repo_path,
            )
        except GitError as exc:

```

```

        if first_error is None:
            first_error = exc

existing_refs: set[str] = set()
try:
    refs_output = await _run_git(
        ["for-each-ref", "--format=%(refname)", "refs/heads"],
        cwd=repo_path,
    )
    existing_refs = set(refs_output.splitlines())
except GitError as exc:
    if first_error is None:
        first_error = exc

for _worktree_path, branch in worktrees:
    if f"refs/heads/{branch}" not in existing_refs:
        continue
    try:
        await _run_git(["branch", "-D", "--", branch], cwd=repo_path)
    except GitError as exc:
        if first_error is None:
            first_error = exc

if first_error is not None:
    raise GitError("Repository worktree cleanup failed") from first_error
logger.info("Repository worktrees cleaned up")

async def validate_local_repo(path: str) -> bool:
    """Check if a path is a valid git repository."""
    if not Path(path).exists():
        return False
    try:
        await _run_git(["rev-parse", "--git-dir"], cwd=path)
        return True
    except GitError:
        return False

```

6.4 services/git_runtime.py

Runtime git helpers provide the sidecar with short-lived credentials when an agent or terminal must fetch from GitHub. The root chunk below tangles back to backend/src/yinshi/services/git_runtime.py.

```

"""Runtime git authentication helpers for sidecar-backed agent sessions."""

```



```

from __future__ import annotations

from dataclasses import dataclass

from yinshi.services.github_app import resolve_github_runtime_access_token

@dataclass(frozen=True, slots=True)
class GitRuntimeAuth:
    """One short-lived git credential payload for the sidecar."""

    strategy: str
    host: str
    access_token: str

    def as_sidecar_payload(self) -> dict[str, object]:
        """Serialize the runtime credential into the sidecar wire format."""
        if self.strategy != "github_app_https":
            raise ValueError(f"Unsupported git auth strategy: {self.strategy}")
        if self.host != "github.com":
            raise ValueError(f"Unsupported git auth host: {self.host}")
        if not self.access_token:
            raise ValueError("access_token must not be empty")
        return {
            "strategy": self.strategy,
            "host": self.host,
            "accessToken": self.access_token,
        }

    async def resolve_git_runtime_auth(
        user_id: str | None,
        remote_url: str | None,
        installation_id: int | None,
    ) -> GitRuntimeAuth | None:
        """Resolve one ephemeral git credential for the current prompt session."""
        if user_id is None:
            return None
        if not isinstance(user_id, str):
            raise TypeError("user_id must be a string or None")
        normalized_user_id = user_id.strip()
        if not normalized_user_id:
            raise ValueError("user_id must not be empty")
        if remote_url is None:
            return None
        if not isinstance(remote_url, str):

```

```

        raise TypeError("remote_url must be a string or None")
normalized_remote_url = remote_url.strip()
if not normalized_remote_url:
    return None
if installation_id is not None and not isinstance(installation_id, int):
    raise TypeError("installation_id must be an integer or None")
if installation_id is not None and installation_id <= 0:
    raise ValueError("installation_id must be positive")

access_token = await resolve_github_runtime_access_token(
    normalized_user_id,
    normalized_remote_url,
    installation_id,
)
if access_token is None:
    return None
return GitRuntimeAuth(
    strategy="github_app_https",
    host="github.com",
    access_token=access_token,
)

```

6.5 services/workspace.py

Workspace services materialize tenant checkouts, refresh GitHub repository metadata, and build trusted worktree paths. The root chunk below tangles back to backend/src/yinshi/services/workspace.py.

```

"""Workspace lifecycle management."""

import logging
import os
import sqlite3
from typing import Any, cast

from yinshi.config import get_settings
from yinshi.exceptions import (
    GitError,
    GitHubAccessError,
    GitHubAppError,
    RepoNotFoundError,
    WorkspaceNotFoundError,
)
from yinshi.services.git import (
    clone_local_repo,
    clone_repo,

```

```

        create_worktree,
        delete_worktree,
        ensure_remote_url,
        generate_branch_name,
        get_remote_url,
        resolve_remote_base_ref,
        restore_worktree,
        validate_local_repo,
    )
    from yinshi.services.github_app import normalize_github_remote, resolve_github_clone_access
    from yinshi.services.repository_lifecycle import (
        repository_lifecycle,
        repository_lifecycle_root,
    )
    from yinshi.services.sidecar_runtime import (
        delete_local_pi_session_file,
        delete_workspace_runtime_home,
    )
    from yinshi.services.workspace_files import ensure_secret_guardrails
    from yinshi.tenant import TenantContext
    from yinshi.utils.paths import is_path_inside

    logger = logging.getLogger(__name__)

    def _fetch_repo(db: sqlite3.Connection, repo_id: str) -> sqlite3.Row:
        """Load a repo row or raise RepoNotFoundError."""
        assert repo_id, "repo_id must not be empty"
        repo = db.execute("SELECT * FROM repos WHERE id = ?", (repo_id,)).fetchone()
        if not repo:
            raise RepoNotFoundError(f"Repo {repo_id} not found")
        return cast(sqlite3.Row, repo)

    def _fetch_workspace(db: sqlite3.Connection, workspace_id: str) -> sqlite3.Row:
        """Load a workspace row or raise WorkspaceNotFoundError."""
        assert workspace_id, "workspace_id must not be empty"
        workspace = db.execute(
            "SELECT * FROM workspaces WHERE id = ?",
            (workspace_id,),
        ).fetchone()
        if not workspace:
            raise WorkspaceNotFoundError(f"Workspace {workspace_id} not found")
        return cast(sqlite3.Row, workspace)

```

```

def _tenant_path_is_trusted(tenant: TenantContext, path: str) -> bool:
    """Return whether a tenant path is inside tenant-managed storage."""
    assert tenant.data_dir, "tenant.data_dir must not be empty"
    assert path, "path must not be empty"
    if is_path_inside(path, tenant.data_dir):
        return True

    settings = get_settings()
    if settings.container_enabled:
        return False
    if settings.allowed_repo_base and is_path_inside(path, settings.allowed_repo_base):
        return True
    return False

def _tenant_repo_path(tenant: TenantContext, repo_id: str) -> str:
    """Return the per-tenant repair target for a repo checkout."""
    assert tenant.data_dir, "tenant.data_dir must not be empty"
    assert repo_id, "repo_id must not be empty"
    return os.path.join(tenant.data_dir, "repos", repo_id)

def _workspace_path(repo_path: str, branch: str) -> str:
    """Build the canonical on-disk path for a worktree branch."""
    assert repo_path, "repo_path must not be empty"
    assert branch, "branch must not be empty"
    return os.path.join(repo_path, ".worktrees", branch)

async def _materialize_repo_checkout(
    source_path: str,
    target_path: str,
    remote_url: str | None,
    access_token: str | None = None,
) -> None:
    """Create or reuse a repaired repo checkout inside tenant storage."""
    assert target_path, "target_path must not be empty"

    if await validate_local_repo(target_path):
        return

    if source_path and await validate_local_repo(source_path):
        await clone_local_repo(source_path, target_path, remote_url=remote_url)
        return

    if remote_url:

```

```

        await clone_repo(remote_url, target_path, access_token=access_token)
        return

    raise RepoNotFoundError("Repo checkout is missing and cannot be repaired")

async def _sync_repo_checkout_remote(
    repo_path: str,
    remote_url: str | None,
) -> bool:
    """Ensure one valid checkout points at the canonical remote URL."""
    if not await validate_local_repo(repo_path):
        return False
    if remote_url is None:
        return False
    return await ensure_remote_url(repo_path, remote_url)

async def _resolve_remote_checkout(
    tenant: TenantContext,
    remote_url: str | None,
) -> tuple[str | None, str | None, int | None]:
    """Resolve a canonical remote URL plus any GitHub token for repairs."""
    if remote_url is None:
        return None, None, None

    try:
        clone_access = await resolve_github_clone_access(tenant.user_id, remote_url)
    except GitHubAccessError as exc:
        raise GitError(str(exc)) from exc
    except GitHubAppError as exc:
        raise GitError(str(exc)) from exc

    if clone_access is None:
        return remote_url, None, None

    return (
        clone_access.clone_url,
        clone_access.access_token,
        clone_access.installation_id,
    )

async def _refresh_repo_remote_metadata(
    tenant: TenantContext,
    repo_path: str,

```

```

        remote_url: str | None,
        installation_id: int | None,
    ) -> tuple[str | None, str | None, int | None]:
        """Refresh canonical remote metadata without breaking local-only recovery."""
        source_repo_is_available = await validate_local_repo(repo_path)
        access_token = None
        refreshed_remote_url = remote_url
        refreshed_installation_id = installation_id

        if remote_url:
            try:
                (
                    refreshed_remote_url,
                    access_token,
                    resolved_installation_id,
                ) = await _resolve_remote_checkout(tenant, remote_url)
            except GitError:
                if not source_repo_is_available:
                    raise
                logger.warning("Refreshing repository from local checkout after remote auth fail")
            else:
                if resolved_installation_id is not None:
                    refreshed_installation_id = resolved_installation_id

        return refreshed_remote_url, access_token, refreshed_installation_id

async def _trusted_repo_needs_refresh(
    repo_path: str,
    remote_url: str | None,
    installation_id: int | None,
) -> bool:
    """Return whether a trusted repo should refresh remote metadata."""
    if not await validate_local_repo(repo_path):
        return True
    if remote_url is None:
        return False
    if installation_id is None:
        return True

    normalized_remote = normalize_github_remote(remote_url)
    if normalized_remote is None:
        return False

    current_remote_url = await get_remote_url(repo_path)
    if current_remote_url is None:

```

```

        return True
    return current_remote_url.rstrip("/") != normalized_remote.clone_url.rstrip("/")

async def ensure_repo_checkout_for_tenant(
    db: sqlite3.Connection,
    tenant: TenantContext,
    repo_id: str,
) -> dict[str, Any]:
    """Repair migrated tenant repo/workspace paths into the tenant data directory.

    Legacy migrations copied root_path and worktree paths into the per-user DB
    without relocating them. This lazily repairs those records the first time
    the repo is used after migration.
    """
    repo = _fetch_repo(db, repo_id)
    repo_path = repo["root_path"]
    assert repo_path, "repo root_path must not be empty"
    remote_url = repo["remote_url"]
    installation_id = repo["installation_id"] if "installation_id" in repo.keys() else None
    source_repo_is_available = await validate_local_repo(repo_path)

    if _tenant_path_is_trusted(tenant, repo_path):
        if source_repo_is_available and not await _trusted_repo_needs_refresh(
            repo_path,
            remote_url,
            installation_id,
        ):
            return dict(repo)

        refreshed_remote_url, _, refreshed_installation_id = await _refresh_repo_remote_metadata(
            tenant,
            repo_path,
            remote_url,
            installation_id,
        )
        remote_was_updated = await _sync_repo_checkout_remote(repo_path, refreshed_remote_url)
        metadata_changed = (
            refreshed_remote_url != remote_url or refreshed_installation_id != installation_id
        )
        if not remote_was_updated and not metadata_changed:
            return dict(repo)

    db.execute(
        "UPDATE repos SET remote_url = ?, installation_id = ? WHERE id = ?",
        (refreshed_remote_url, refreshed_installation_id, repo_id),
    )

```

```

    )
    db.commit()
    return dict(_fetch_repo(db, repo_id))

target_repo_path = _tenant_repo_path(tenant, repo_id)
remote_url, access_token, installation_id = await _refresh_repo_remote_metadata(
    tenant,
    repo_path,
    remote_url,
    installation_id,
)
await _materialize_repo_checkout(
    repo_path,
    target_repo_path,
    remote_url,
    access_token=access_token,
)

workspaces = db.execute(
    "SELECT * FROM workspaces WHERE repo_id = ? ORDER BY created_at ASC",
    (repo_id,),
).fetchall()
for workspace in workspaces:
    branch = workspace["branch"]
    if not branch:
        raise WorkspaceNotFoundError(f"Workspace {workspace['id']} is missing its branch")
    target_workspace_path = _workspace_path(target_repo_path, branch)
    await restore_worktree(target_repo_path, target_workspace_path, branch)
    db.execute(
        "UPDATE workspaces SET path = ? WHERE id = ?",
        (target_workspace_path, workspace["id"]),
    )

db.execute(
    "UPDATE repos SET root_path = ?, remote_url = ?, installation_id = ? WHERE id = ?",
    (target_repo_path, remote_url, installation_id, repo_id),
)
db.commit()
logger.info("Repaired repository into tenant storage")

updated_repo = _fetch_repo(db, repo_id)
return dict(updated_repo)

async def relink_github_repos_for_tenant(
    db: sqlite3.Connection,

```



```

        tenant: TenantContext,
        owner_login: str,
    ) -> int:
        """Refresh existing tenant repos after one GitHub App installation is connected."""
        if not owner_login:
            raise ValueError("owner_login must not be empty")
        refreshed_repo_count = 0
        repos = db.execute(
            "SELECT id FROM repos WHERE remote_url IS NOT NULL ORDER BY created_at ASC"
        ).fetchall()

        for repo_row in repos:
            repo = _fetch_repo(db, repo_row["id"])
            remote_url = repo["remote_url"]
            if not isinstance(remote_url, str) or not remote_url:
                continue
            github_remote = normalize_github_remote(remote_url)
            if github_remote is None:
                continue
            if github_remote.owner.lower() != owner_login.lower():
                continue

            refreshed_repo = await ensure_repo_checkout_for_tenant(db, tenant, repo["id"])
            if refreshed_repo["remote_url"] != repo["remote_url"]:
                refreshed_repo_count += 1
                continue
            if refreshed_repo["installation_id"] != repo["installation_id"]:
                refreshed_repo_count += 1

        return refreshed_repo_count

    async def ensure_workspace_checkout_for_tenant(
        db: sqlite3.Connection,
        tenant: TenantContext,
        workspace_id: str,
    ) -> dict[str, Any]:
        """Repair the repo backing a workspace and return the updated workspace row."""
        workspace = _fetch_workspace(db, workspace_id)
        repo_id = workspace["repo_id"]
        assert repo_id, "workspace repo_id must not be empty"

        await ensure_repo_checkout_for_tenant(db, tenant, repo_id)
        updated_workspace = _fetch_workspace(db, workspace_id)
        return dict(updated_workspace)

```

```

async def create_workspace_for_repo(
    db: sqlite3.Connection,
    repo_id: str,
    name: str | None = None,
    username: str | None = None,
    tenant: TenantContext | None = None,
) -> dict[str, Any]:
    """Create a new worktree while holding its repository lifecycle lock."""
    lock_root = repository_lifecycle_root(db, tenant)
    async with repository_lifecycle(repo_id, lock_root):
        return await _create_workspace_for_repo_unlocked(
            db,
            repo_id,
            name=name,
            username=username,
            tenant=tenant,
        )

async def _create_workspace_for_repo_unlocked(
    db: sqlite3.Connection,
    repo_id: str,
    name: str | None = None,
    username: str | None = None,
    tenant: TenantContext | None = None,
) -> dict[str, Any]:
    """Create a new worktree after repository lifecycle serialization."""
    if tenant is not None:
        await ensure_repo_checkout_for_tenant(db, tenant, repo_id)

    repo = _fetch_repo(db, repo_id)

    branch = generate_branch_name(username=username)
    if not name:
        name = branch

    repo_path = repo["root_path"]
    assert repo_path, "repo_path must not be empty"
    worktree_dir = _workspace_path(repo_path, branch)
    base_ref: str | None = None

    remote_url = repo["remote_url"]
    if remote_url:
        access_token = None
        try:

```

```

        if tenant is not None:
            _, access_token, _ = await _resolve_remote_checkout(tenant, remote_url)
            base_ref = await resolve_remote_base_ref(repo_path, access_token=access_token)
        except GitError:
            logger.warning("Creating worktree from local HEAD after remote sync failure")
            base_ref = None

    await create_worktree(repo_path, worktree_dir, branch, base_ref=base_ref)
    ensure_secret_guardrails(repo_path)

    cursor = db.execute(
        """INSERT INTO workspaces (repo_id, name, branch, path, state)
        VALUES (?, ?, ?, ?, 'ready')""",
        (repo_id, name, branch, worktree_dir),
    )
    db.commit()

    row = db.execute("SELECT * FROM workspaces WHERE rowid = ?", (cursor.lastrowid,)).fetchone()
    return dict(row)

async def delete_workspace(
    db: sqlite3.Connection,
    workspace_id: str,
    *,
    tenant: TenantContext | None = None,
) -> None:
    """Delete a workspace while holding its repository lifecycle lock."""
    workspace = _fetch_workspace(db, workspace_id)
    repo_id = str(workspace["repo_id"])
    lock_root = repository_lifecycle_root(db, tenant)
    async with repository_lifecycle(repo_id, lock_root):
        await _delete_workspace_unlocked(db, workspace_id, tenant=tenant)

async def _delete_workspace_unlocked(
    db: sqlite3.Connection,
    workspace_id: str,
    *,
    tenant: TenantContext | None = None,
) -> None:
    """Delete one workspace after repository lifecycle serialization."""
    workspace = _fetch_workspace(db, workspace_id)
    session_rows = []
    if tenant is None:
        session_rows = db.execute(

```

```

        "SELECT id FROM sessions WHERE workspace_id = ?",
        (workspace_id,),
    ).fetchall()

repo = _fetch_repo(db, workspace["repo_id"])
await delete_worktree(repo["root_path"], workspace["path"])

if tenant is not None:
    delete_workspace_runtime_home(tenant, workspace_id)
else:
    for session_row in session_rows:
        try:
            delete_local_pi_session_file(session_row["id"])
        except OSError:
            logger.warning("Failed to delete Pi session file")

db.execute("DELETE FROM workspaces WHERE id = ?", (workspace_id,))
db.commit()

```

6.6 services/workspace_runtime_paths.py

Runtime path preparation returns the workspace, tenant home, and sidecar-visible paths used by prompts and terminals. The root chunk below tangles back to backend/src/yinshi/services/workspace_runtime_paths.py.

```

"""Shared workspace path preparation for tenant-scoped runtime features."""

from __future__ import annotations

import os
import sqlite3
from dataclasses import dataclass
from typing import cast

from yinshi.exceptions import WorkspaceNotFoundError
from yinshi.services.workspace import ensure_workspace_checkout_for_tenant
from yinshi.services.workspace_files import ensure_secret_guardrails
from yinshi.tenant import TenantContext
from yinshi.utils.paths import is_path_inside

@dataclass(frozen=True, slots=True)
class WorkspaceRuntimePaths:
    """Trusted host paths needed by workspace-scoped runtime features."""

    workspace_path: str

```

```

repo_root_path: str
agents_md: str | None

def _workspace_runtime_row(db: sqlite3.Connection, workspace_id: str) -> sqlite3.Row:
    """Load workspace plus repo path fields needed by runtime features."""
    if not isinstance(workspace_id, str):
        raise TypeError("workspace_id must be a string")
    normalized_workspace_id = workspace_id.strip()
    if not normalized_workspace_id:
        raise ValueError("workspace_id must not be empty")

    row = db.execute(
        "SELECT w.path, r.root_path, r.agents_md "
        "FROM workspaces w JOIN repos r ON w.repo_id = r.id WHERE w.id = ?",
        (normalized_workspace_id,),
    ).fetchone()
    if row is None:
        raise WorkspaceNotFoundError("Workspace not found")
    return cast(sqlite3.Row, row)

def _tenant_owned_path(path: str, tenant: TenantContext, path_name: str) -> str:
    """Return a real path after proving it stays inside tenant storage."""
    if not isinstance(path, str):
        raise TypeError(f"{path_name} must be a string")
    normalized_path = path.strip()
    if not normalized_path:
        raise ValueError(f"{path_name} must not be empty")
    real_path = os.path.realpath(normalized_path)
    if not is_path_inside(real_path, tenant.data_dir):
        raise PermissionError(f"{path_name} is outside tenant storage")
    return real_path

async def prepare_tenant_workspace_runtime_paths(
    db: sqlite3.Connection,
    tenant: TenantContext,
    workspace_id: str,
) -> WorkspaceRuntimePaths:
    """Repair, validate, and guard paths for one tenant workspace."""
    if tenant is None:
        raise TypeError("tenant must not be None")

    await ensure_workspace_checkout_for_tenant(db, tenant, workspace_id)
    row = _workspace_runtime_row(db, workspace_id)

```

```

workspace_path = _tenant_owned_path(str(row["path"]), tenant, "workspace path")
repo_root_path = _tenant_owned_path(str(row["root_path"]), tenant, "repo root path")
ensure_secret_guardrails(repo_root_path)
agents_md = row["agents_md"]
if agents_md is not None and not isinstance(agents_md, str):
    raise TypeError("agents_md must be a string or None")
return WorkspaceRuntimePaths(
    workspace_path=workspace_path,
    repo_root_path=repo_root_path,
    agents_md=agents_md,
)

```

7 Agent Streaming and Run Coordination

Prompt execution is streamed rather than polled. The backend composes provider settings and starts one active run per session. It stores each turn event and sends SSE events during sidecar work.

7.1 api/stream.py

The stream route builds effective model settings, launches prompt execution, persists turn events, and exposes cancellation. The root chunk below tangles back to `backend/src/yinshi/api/stream.py`.

```

"""SSE streaming endpoint for agent interaction.

Tests: test_prompt_session_not_found, test_prompt_streams_sidecar_events,
test_prompt_saves_partial_on_sidecar_error, test_cancel_session_not_found,
test_cancel_no_active_stream in tests/test_api.py
"""

import asyncio
import json
import logging
import sqlite3
import sys
import uuid
from collections.abc import AsyncGenerator, AsyncIterator, Callable
from dataclasses import dataclass
from typing import Any, Literal, cast

from fastapi import APIRouter, HTTPException, Request
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field, field_validator

from yinshi.api.deps import check_owner, get_db_for_request, get_tenant, get_user_email

```

```

from yinshi.auth import get_session_identity
from yinshi.config import get_settings
from yinshi.exceptions import (
    ContainerNotReadyError,
    ContainerStartError,
    GitError,
    KeyNotFoundError,
    RepoNotFoundError,
    SidecarError,
    WorkspaceNotFoundError,
)
from yinshi.model_catalog import get_provider_metadata, normalize_model_ref
from yinshi.rate_limit import limiter
from yinshi.services.container import (
    ContainerActivityReservation,
    ContainerManager,
)
from yinshi.services.desktop_devices import desktop_device_is_active
from yinshi.services.git_runtime import resolve_git_runtime_auth
from yinshi.services.keys import record_usage
from yinshi.services.live_auth_sessions import (
    LiveAuthSessionRegistration,
    register_live_auth_session,
    register_live_desktop_device,
)
from yinshi.services.provider_connections import (
    resolve_provider_connection,
    update_provider_connection_secret,
)
from yinshi.services.run_coordinator import get_run_coordinator
from yinshi.services.sidecar import SidecarClient, create_sidecar_connection
from yinshi.services.sidecar_runtime import (
    local_pi_session_file,
    remap_path_for_container,
    resolve_tenant_sidecar_context,
    touch_tenant_container,
)
from yinshi.services.workspace import ensure_workspace_checkout_for_tenant
from yinshi.utils.paths import is_path_inside

logger = logging.getLogger(__name__)
router = APIRouter()

# Batch DB writes every N chunks to reduce I/O
_PERSIST_BATCH_SIZE = 10
_STORED_TURN_SCHEMA = "yinshi.assistant_turn.v1"

```

```

ThinkingLevel = Literal["off", "minimal", "low", "medium", "high", "xhigh"]

_AUTH_SESSION_RECHECK_INTERVAL_S = 1.0
_STREAM_LIFETIME_S_MAX = 2 * 60 * 60
_THINKING_LEVEL_DEFAULT: ThinkingLevel = "medium"
_THINKING_LEVEL_OFF: ThinkingLevel = "off"
_THINKING_LEVEL_ORDER: tuple[ThinkingLevel, ...] = (
    "off",
    "minimal",
    "low",
    "medium",
    "high",
    "xhigh",
)
_THINKING_LEVELS = frozenset(_THINKING_LEVEL_ORDER)
_STANDARD_THINKING_LEVELS = _THINKING_LEVEL_ORDER[:-1]

@dataclass(frozen=True, slots=True)
class ExecutionContext:
    """Resolved sidecar execution inputs for a single prompt request."""

    sidecar_socket: str | None
    effective_cwd: str
    key_source: str
    provider: str
    provider_auth: dict[str, object] | None
    provider_config: dict[str, object] | None
    git_auth: dict[str, object] | None = None
    agent_dir: str | None = None
    settings_payload: dict[str, object] | None = None
    model_ref: str = ""
    runtime_id: str | None = None
    pi_session_file: str | None = None

@dataclass(frozen=True, slots=True)
class _ContainerActivity:
    """Bind one reservation to the manager that issued it."""

    manager: ContainerManager
    reservation: ContainerActivityReservation

async def _acquire_tenant_container_activity(
    request: Request,

```



```

    tenant: Any,
    *,
    runtime_id: str | None,
    required: bool,
) -> _ContainerActivity | None:
    """Acquire the current tenant runtime before any sidecar operation."""
    if tenant is None:
        return None
    manager = cast(
        ContainerManager | None,
        getattr(request.app.state, "container_manager", None),
    )
    if manager is None:
        if required:
            raise ContainerNotReadyError("Container manager is not initialized")
        return None
    reservation = await manager.acquire_activity(
        tenant.user_id,
        runtime_id=runtime_id,
    )
    if reservation is None:
        if required:
            raise ContainerNotReadyError("Container runtime is no longer available")
        return None
    return _ContainerActivity(manager, reservation)

async def _release_tenant_container_activity(
    activity: _ContainerActivity | None,
) -> None:
    """Release the exact container reservation acquired by this caller."""
    if activity is not None:
        await activity.manager.release_activity(activity.reservation)

class _AuthSessionRevoked(Exception):
    """Signal that an active stream's originating auth session was revoked."""

class _StreamLifetimeReached(Exception):
    """Signal that a response reached its independent connection deadline."""

async def _authority_bound_events(
    *,
    events: AsyncIterator[dict[str, Any]],

```

```

sidecar: SidecarClient,
session_id: str,
registration: LiveAuthSessionRegistration,
authority_is_active: Callable[[], bool],
) -> AsyncGenerator[dict[str, Any], None]:
    """Yield sidecar events while one durable authority remains active."""
    if not session_id:
        raise ValueError("session_id must not be empty")
    if not authority_is_active():
        registration.close()
        await sidecar.cancel(session_id)
        raise _AuthSessionRevoked

    loop = asyncio.get_running_loop()
    connection_deadline = loop.time() + _STREAM_LIFETIME_S_MAX
    event_iterator = aiter(events)
    next_event_task: asyncio.Future[dict[str, Any]] | None = None
    revocation_task = asyncio.create_task(registration.event.wait())
    try:
        while True:
            if loop.time() >= connection_deadline:
                await sidecar.cancel(session_id)
                raise _StreamLifetimeReached
            next_event_task = asyncio.ensure_future(anext(event_iterator))
            while not next_event_task.done():
                remaining_lifetime_s = connection_deadline - loop.time()
                if remaining_lifetime_s <= 0:
                    next_event_task.cancel()
                    await sidecar.cancel(session_id)
                    raise _StreamLifetimeReached
            done, _ = await asyncio.wait(
                {next_event_task, revocation_task},
                timeout=min(_AUTH_SESSION_RECHECK_INTERVAL_S, remaining_lifetime_s),
                return_when=asyncio.FIRST_COMPLETED,
            )
            if revocation_task in done:
                next_event_task.cancel()
                await sidecar.cancel(session_id)
                raise _AuthSessionRevoked
            if next_event_task in done:
                break
            if loop.time() >= connection_deadline:
                next_event_task.cancel()
                await sidecar.cancel(session_id)
                raise _StreamLifetimeReached
            if not authority_is_active():

```

```

        next_event_task.cancel()
        await sidecar.cancel(session_id)
        raise _AuthSessionRevoked
    try:
        event = next_event_task.result()
    except StopAsyncIteration:
        return
    finally:
        next_event_task = None
    if registration.event.is_set() or not authority_is_active():
        await sidecar.cancel(session_id)
        raise _AuthSessionRevoked
    if loop.time() >= connection_deadline:
        await sidecar.cancel(session_id)
        raise _StreamLifetimeReached
    yield event
finally:
    registration.close()
    revocation_task.cancel()
    if next_event_task is not None and not next_event_task.done():
        next_event_task.cancel()

async def _session_bound_events(
    events: AsyncIterator[dict[str, Any]],
    session_token: str,
    sidecar: SidecarClient,
    session_id: str,
) -> AsyncGenerator[dict[str, Any], None]:
    """Yield sidecar events until browser-session revocation occurs."""
    if not session_token:
        raise ValueError("session_token must not be empty")
    identity = get_session_identity(session_token)
    if identity is None:
        await sidecar.cancel(session_id)
        raise _AuthSessionRevoked
    user_id, auth_session_id = identity
    registration = register_live_auth_session(
        user_id=user_id,
        auth_session_id=auth_session_id,
    )
    async for event in _authority_bound_events(
        events=events,
        sidecar=sidecar,
        session_id=session_id,
        registration=registration,

```

```

        authority_is_active=lambda: get_session_identity(session_token) is not None,
    ):
        yield event

async def _desktop_device_bound_events(
    events: AsyncIterator[dict[str, Any]],
    *,
    user_id: str,
    device_id: str,
    sidecar: SidecarClient,
    session_id: str,
) -> AsyncGenerator[dict[str, Any], None]:
    """Yield sidecar events until desktop-device revocation occurs."""
    registration = register_live_desktop_device(
        user_id=user_id,
        device_id=device_id,
    )
    async for event in _authority_bound_events(
        events=events,
        sidecar=sidecar,
        session_id=session_id,
        registration=registration,
        authority_is_active=lambda: desktop_device_is_active(
            user_id=user_id,
            device_id=device_id,
        ),
    ):
        yield event

class PromptRequest(BaseModel):
    prompt: str = Field(..., max_length=100_000)
    model: str | None = None
    thinking: ThinkingLevel | None = None

    @field_validator("thinking", mode="before")
    @classmethod
    def validate_thinking(cls, value: object) -> str | None:
        """Normalize legacy booleans and explicit thinking levels."""
        if value is None:
            return None
        if isinstance(value, bool):
            return _THINKING_LEVEL_DEFAULT if value else _THINKING_LEVEL_OFF
        if not isinstance(value, str):
            raise ValueError("thinking must be a valid thinking level")

```

```

        normalized_value = value.strip().lower()
        if normalized_value in _THINKING_LEVELS:
            return normalized_value
        valid_levels = ", ".join(_THINKING_LEVEL_ORDER)
        raise ValueError(f"thinking must be one of {valid_levels}")

@field_validator("model")
@classmethod
def validate_model(cls, value: str | None) -> str | None:
    """Normalize optional model values into canonical refs."""
    if value is None:
        return None
    return normalize_model_ref(value)

def _catalog_model_thinking_levels(
    model_payload: dict[str, Any],
) -> tuple[ThinkingLevel, ...] | None:
    """Return model-specific thinking levels from one catalog row."""
    raw_levels = model_payload.get("thinking_levels")
    if isinstance(raw_levels, list):
        thinking_levels: list[ThinkingLevel] = []
        for raw_level in raw_levels:
            if raw_level not in _THINKING_LEVELS:
                continue
            level = cast(ThinkingLevel, raw_level)
            if level not in thinking_levels:
                thinking_levels.append(level)
        if thinking_levels:
            if _THINKING_LEVEL_OFF not in thinking_levels:
                thinking_levels.insert(0, _THINKING_LEVEL_OFF)
            return tuple(thinking_levels)

    reasoning_value = model_payload.get("reasoning")
    if isinstance(reasoning_value, bool):
        if reasoning_value:
            return _STANDARD_THINKING_LEVELS
        return (_THINKING_LEVEL_OFF,)
    return None

def _catalog_thinking_levels(
    catalog_payload: dict[str, Any],
    model_ref: str,
) -> tuple[ThinkingLevel, ...] | None:
    """Return available thinking levels for one catalog model."""

```

```

if not isinstance(catalog_payload, dict):
    raise TypeError("catalog_payload must be a dictionary")
if not isinstance(model_ref, str):
    raise TypeError("model_ref must be a string")
normalized_model_ref = model_ref.strip()
if not normalized_model_ref:
    raise ValueError("model_ref must not be empty")

models_payload = catalog_payload.get("models")
if not isinstance(models_payload, list):
    return None

for model_payload in models_payload:
    if not isinstance(model_payload, dict):
        continue
    if model_payload.get("ref") != normalized_model_ref:
        continue

    thinking_levels = _catalog_model_thinking_levels(model_payload)
    if thinking_levels is None:
        logger.warning(
            "Catalog thinking metadata missing for model %s",
            normalized_model_ref,
        )
    return thinking_levels

logger.warning("Requested catalog entry is unavailable")
return None

def _clamp_thinking_level(
    requested_level: ThinkingLevel,
    available_levels: tuple[ThinkingLevel, ...],
) -> ThinkingLevel:
    """Clamp one requested thinking level to model-supported levels."""
    if requested_level in available_levels:
        return requested_level
    available_level_set = set(available_levels)
    requested_index = _THINKING_LEVEL_ORDER.index(requested_level)

    for candidate in _THINKING_LEVEL_ORDER[requested_index:]:
        if candidate in available_level_set:
            return candidate
    for candidate in reversed(_THINKING_LEVEL_ORDER[:requested_index]):
        if candidate in available_level_set:
            return candidate

```

```

    return available_levels[0] if available_levels else _THINKING_LEVEL_OFF

def _build_effective_settings(
    settings_payload: dict[str, object] | None,
    thinking_override: ThinkingLevel | None,
    available_levels: tuple[ThinkingLevel, ...] | None,
) -> dict[str, object] | None:
    """Merge one prompt-scoped thinking level into Pi-compatible settings."""
    if thinking_override is None:
        return settings_payload
    if available_levels == (_THINKING_LEVEL_OFF,):
        return settings_payload

    effective_settings = dict(settings_payload or {})
    effective_level = thinking_override
    if available_levels is not None:
        effective_level = _clamp_thinking_level(thinking_override, available_levels)
    effective_settings["defaultThinkingLevel"] = effective_level
    return effective_settings

def _stored_turn_event(event: dict[str, Any]) -> dict[str, Any]:
    """Return one JSON-safe turn event for persisted assistant history."""
    if not isinstance(event, dict):
        raise TypeError("event must be a dictionary")
    event_type = event.get("type")
    if not isinstance(event_type, str):
        raise ValueError("event type must be a string")
    safe_event = json.loads(json.dumps(event))
    assert isinstance(safe_event, dict), "serialized event must remain an object"
    return cast(dict[str, Any], safe_event)

def _serialize_stored_turn(events: list[dict[str, Any]]) -> str | None:
    """Serialize turn events using an explicit replay schema."""
    if not events:
        return None
    payload = {
        "schema": _STORED_TURN_SCHEMA,
        "events": events,
    }
    return json.dumps(payload)

```

```

_FILLER_PREFIXES = [

```

```

    "please ",
    "can you ",
    "could you ",
    "would you ",
    "i want you to ",
    "i need you to ",
    "help me ",
    "i'd like you to ",
    "i would like you to ",
    "go ahead and ",
    "let's ",
    "we need to ",
    "we should ",
]

```

```

_STOP_WORDS = frozenset(
    {
        "the",
        "a",
        "an",
        "and",
        "or",
        "but",
        "in",
        "on",
        "at",
        "to",
        "for",
        "of",
        "with",
        "by",
        "from",
        "is",
        "are",
        "was",
        "were",
        "be",
        "been",
        "have",
        "has",
        "had",
        "do",
        "does",
        "did",
        "this",
        "that",
    }
)

```


"it",
"its",
"my",
"your",
"our",
"their",
"some",
"all",
"any",
"so",
"up",
"out",
"about",
"into",
"me",
"him",
"her",
"us",
"them",
"i",
"you",
"he",
"she",
"we",
"they",
"just",
"also",
"very",
"really",
"actually",
"basically",
"need",
"needs",
"want",
"make",
"sure",
"there",
"using",
"how",
"what",
"when",
"where",
"which",
"who",
"why",
"new",

```

        "now",
    }
)

def _summarize_prompt(prompt: str, max_words: int = 3) -> str:
    """Derive a 2-3 word workspace name from a user prompt."""
    text = prompt.strip()
    if not text:
        return ""

    lower = text.lower()
    for prefix in _FILLER_PREFIXES:
        if lower.startswith(prefix):
            text = text[len(prefix) :]
            break

    words = [w.strip(".,;:!?-\\'()[]{}") for w in text.split()]
    words = [w for w in words if w]
    significant = [w for w in words if w.lower() not in _STOP_WORDS]

    if not significant:
        collapsed_text = "-".join(text.split())
        significant = words[:max_words] if words else [collapsed_text[:30]]

    result = significant[:max_words]
    summary = "-".join(w.lower() for w in result)

    if len(summary) > 50:
        summary = summary[:50].rsplit("-", 1)[0]
    if not summary:
        summary = "-".join(text.lower().split())[:30]
    return summary

def _workspace_path_is_trusted(tenant: Any, workspace_path: str) -> bool:
    """Return whether a workspace path is inside tenant-managed storage."""
    assert workspace_path, "workspace_path must not be empty"
    if is_path_inside(workspace_path, tenant.data_dir):
        return True

    settings = get_settings()
    if settings.container_enabled:
        return False
    if settings.allowed_repo_base and is_path_inside(workspace_path, settings.allowed_repo_b
        return True

```

```

        return False

def _validate_workspace_path(tenant: Any, workspace_path: str) -> None:
    """Reject workspace paths that are outside trusted directories."""
    if _workspace_path_is_trusted(tenant, workspace_path):
        return

    raise HTTPException(
        status_code=403,
        detail="Workspace path outside allowed directories",
    )

def _remap_path(
    host_path: str,
    data_dir: str,
    mount: str = "/data",
) -> str:
    """Translate a host workspace path to the container's mount namespace."""
    return remap_path_for_container(host_path, data_dir, mount_path=mount)

def _session_pi_context_version(session: sqlite3.Row) -> int:
    """Return durable Pi context version for one session row."""
    if "pi_context_version" not in session.keys():
        return 0
    try:
        return int(session["pi_context_version"] or 0)
    except (TypeError, ValueError):
        return 0

def _message_count_for_session(db: sqlite3.Connection, session_id: str) -> int:
    """Return stored transcript message count for one session."""
    assert session_id, "session_id must not be empty"
    row = db.execute(
        "SELECT COUNT(*) AS message_count FROM messages WHERE session_id = ?",
        (session_id,),
    ).fetchone()
    assert row is not None, "message count query must return one row"
    return int(row["message_count"])

def _ensure_promptable_pi_context(db: sqlite3.Connection, session: sqlite3.Row) -> None:
    """Reject legacy transcript-only sessions that cannot resume exact Pi context."""

```

```

session_id = session["id"]
assert isinstance(session_id, str), "session id must be a string"
if _session_pi_context_version(session) >= 1:
    return

if _message_count_for_session(db, session_id) > 0:
    raise HTTPException(
        status_code=409,
        detail={
            "code": "legacy_pi_context",
            "message": (
                "This session predates durable Pi context and cannot continue "
                "with exact model context. Start a new session in this workspace."
            ),
        },
    )

db.execute(
    "UPDATE sessions SET pi_context_version = 1 WHERE id = ?",
    (session_id,),
)
db.commit()

def _lookup_session(
    db: sqlite3.Connection,
    session_id: str,
    request: Request,
) -> sqlite3.Row | None:
    """Look up a session with workspace info, including owner_email in legacy mode."""
    tenant = get_tenant(request)
    if tenant:
        row = db.execute(
            "SELECT s.*, w.path as workspace_path, w.id as workspace_id, "
            "w.name as workspace_name, w.branch as workspace_branch, "
            "r.remote_url, r.installation_id, r.agents_md, r.root_path as repo_root_path "
            "FROM sessions s "
            "JOIN workspaces w ON s.workspace_id = w.id "
            "JOIN repos r ON w.repo_id = r.id "
            "WHERE s.id = ?",
            (session_id,),
        ).fetchone()
        return cast(sqlite3.Row | None, row)

    row = db.execute(
        "SELECT s.*, w.path as workspace_path, w.id as workspace_id, "

```

```

        "w.name as workspace_name, w.branch as workspace_branch, "
        "r.owner_email, r.remote_url, r.installation_id, r.agents_md, r.root_path as repo_ro
        "FROM sessions s "
        "JOIN workspaces w ON s.workspace_id = w.id "
        "JOIN repos r ON w.repo_id = r.id "
        "WHERE s.id = ?",
        (session_id,),
    ).fetchone()
    return cast(sqlite3.Row | None, row)

async def _resolve_execution_context(
    request: Request,
    tenant: Any,
    runtime_session_id: str,
    workspace_id: str,
    workspace_path: str,
    model: str,
    repo_root_path: str | None = None,
    remote_url: str | None = None,
    installation_id: int | None = None,
    agents_md: str | None = None,
) -> ExecutionContext:
    """Resolve all sidecar execution inputs for the current request."""
    if not tenant:
        return ExecutionContext(
            sidecar_socket=None,
            effective_cwd=workspace_path,
            key_source="platform",
            provider="",
            provider_auth=None,
            provider_config=None,
            git_auth=None,
            model_ref=model,
            runtime_id=None,
            pi_session_file=local_pi_session_file(runtime_session_id),
        )

    _validate_workspace_path(tenant, workspace_path)

    try:
        tenant_sidecar_context = await resolve_tenant_sidecar_context(
            request,
            tenant,
            runtime_session_id=runtime_session_id,
            repo_agents_md=agents_md,

```

```

        repo_root_path=repo_root_path,
        workspace_path=workspace_path,
        workspace_id=workspace_id,
    )
except (ContainerStartError, ContainerNotReadyError):
    logger.error("Container start failed")
    raise HTTPException(
        status_code=503,
        detail="Agent environment temporarily unavailable",
    ) from None
sidecar_socket = tenant_sidecar_context.socket_path
effective_cwd = workspace_path
agent_dir = tenant_sidecar_context.agent_dir
settings_payload = tenant_sidecar_context.settings_payload

if sidecar_socket is not None:
    try:
        effective_cwd = _remap_path(workspace_path, tenant.data_dir)
    except ValueError as exc:
        raise HTTPException(
            status_code=403,
            detail="Workspace path outside allowed directories",
        ) from exc

sidecar_tmp = None
try:
    activity = await _acquire_tenant_container_activity(
        request,
        tenant,
        runtime_id=tenant_sidecar_context.runtime_id,
        required=sidecar_socket is not None,
    )
except (ContainerStartError, ContainerNotReadyError):
    logger.error("Container activity acquisition failed")
    raise HTTPException(
        status_code=503,
        detail="Agent environment temporarily unavailable",
    ) from None
try:
    sidecar_tmp = await create_sidecar_connection(sidecar_socket)
    resolved = await sidecar_tmp.resolve_model(model, agent_dir=agent_dir)
    provider: str | None = resolved["provider"]
    if not provider:
        raise HTTPException(
            status_code=400,
            detail="Could not determine provider for model",

```

```

    )
    provider_metadata = get_provider_metadata(provider)
    if not provider_metadata.supported:
        raise HTTPException(
            status_code=400,
            detail=f"Provider {provider} is not supported in Yinshi yet",
        )
    model_ref = cast(str, resolved["model"])
    connection = resolve_provider_connection(tenant.user_id, provider)
    provider_auth: dict[str, object] = {
        "provider": provider,
        "authStrategy": connection["auth_strategy"],
        "secret": cast(object, connection["secret"]),
    }
    provider_config = cast(dict[str, object], connection["config"])
    auth_resolved = await sidecar_tmp.resolve_provider_auth(
        provider=provider,
        model=model_ref,
        provider_auth=cast(dict[str, Any], provider_auth),
        provider_config=provider_config,
        agent_dir=agent_dir,
    )
    refreshed_auth = auth_resolved.get("auth")
    if refreshed_auth is not None and refreshed_auth != connection["secret"]:
        update_provider_connection_secret(
            tenant.user_id,
            connection["id"],
            connection["auth_strategy"],
            cast(str | dict[str, object], refreshed_auth),
        )
        provider_auth["secret"] = cast(object, refreshed_auth)
    resolved_model_ref = cast(str, auth_resolved.get("model_ref") or model_ref)
    resolved_provider_config = cast(
        dict[str, object] | None,
        auth_resolved.get("model_config"),
    )
    git_runtime_auth = await resolve_git_runtime_auth(
        tenant.user_id,
        remote_url,
        installation_id,
    )
except KeyNotFoundError as exc:
    raise HTTPException(status_code=402, detail=str(exc)) from exc
finally:
    try:
        if sidecar_tmp is not None:

```

```

        await sidecar_tmp.disconnect()
    finally:
        await _release_tenant_container_activity(activity)

    return ExecutionContext(
        sidecar_socket=sidecar_socket,
        effective_cwd=effective_cwd,
        key_source=connection["auth_strategy"],
        provider=provider,
        provider_auth=provider_auth,
        provider_config=resolved_provider_config or provider_config,
        git_auth=None if git_runtime_auth is None else git_runtime_auth.as_sidecar_payload(),
        agent_dir=agent_dir,
        settings_payload=settings_payload,
        model_ref=resolved_model_ref,
        runtime_id=tenant_sidecar_context.runtime_id,
        pi_session_file=tenant_sidecar_context.pi_session_file,
    )

@router.post("/api/sessions/{session_id}/prompt")
@limiter.limit("120/hour")
async def prompt_session(
    session_id: str,
    body: PromptRequest,
    request: Request,
) -> StreamingResponse:
    """Send a prompt and stream agent events as SSE."""
    tenant = get_tenant(request)
    desktop_device_id = getattr(request.state, "desktop_device_id", None)
    if desktop_device_id is not None and (
        not isinstance(desktop_device_id, str) or not desktop_device_id
    ):
        raise RuntimeError("desktop device authority is invalid")
    requires_auth_session = (
        tenant is not None and request.app.state.mode != "worker" and desktop_device_id is not None
    )
    auth_session_token = request.cookies.get("yinshi_session") if requires_auth_session else None
    with get_db_for_request(request) as db:
        session = _lookup_session(db, session_id, request)
        if session and tenant:
            try:
                await ensure_workspace_checkout_for_tenant(
                    db,
                    tenant,
                    session["workspace_id"],

```



```

    )
    except (GitError, RepoNotFoundError, WorkspaceNotFoundError) as exc:
        raise HTTPException(status_code=409, detail=str(exc))
    session = _lookup_session(db, session_id, request)

if not session:
    raise HTTPException(status_code=404, detail="Session not found")

if not tenant:
    check_owner(session["owner_email"], get_user_email(request))

if session["status"] == "running":
    raise HTTPException(status_code=409, detail="Session already has an active stream")

with get_db_for_request(request) as db:
    _ensure_promptable_pi_context(db, session)

workspace_path = session["workspace_path"]
remote_url = session["remote_url"] if "remote_url" in session.keys() else None
installation_id = session["installation_id"] if "installation_id" in session.keys() else None
model = normalize_model_ref(body.model or session["model"])
prompt = body.prompt
turn_id = uuid.uuid4().hex

# Atomically claim the session for this stream. The WHERE clause
# ensures only one concurrent request can transition idle -> running.
with get_db_for_request(request) as db:
    result = db.execute(
        "UPDATE sessions SET status = 'running' WHERE id = ? AND status = 'idle'",
        (session_id,),
    )
    if result.rowcount == 0:
        raise HTTPException(status_code=409, detail="Session already has an active stream")

    db.execute(
        "INSERT INTO messages (session_id, role, content, turn_id) VALUES (?, 'user', ?, ?)",
        (session_id, prompt, turn_id),
    )
    # Update workspace name on first prompt (when name == branch)
    if session["workspace_name"] == session["workspace_branch"]:
        display_name = _summarize_prompt(prompt)
        db.execute(
            "UPDATE workspaces SET name = ? WHERE id = ?",
            (display_name, session["workspace_id"]),
        )
    db.commit()

```

```

try:
    context = await _resolve_execution_context(
        request,
        tenant,
        session_id,
        session["workspace_id"],
        workspace_path,
        model,
        repo_root_path=(
            session["repo_root_path"] if "repo_root_path" in session.keys() else None
        ),
        remote_url=remote_url,
        installation_id=installation_id,
        agents_md=session["agents_md"] if "agents_md" in session.keys() else None,
    )
except asyncio.CancelledError:
    with get_db_for_request(request) as db:
        db.execute(
            "UPDATE sessions SET status = 'idle' WHERE id = ?",
            (session_id,)
        )
        db.commit()
    raise
except Exception:
    with get_db_for_request(request) as db:
        db.execute(
            "UPDATE sessions SET status = 'idle' WHERE id = ?",
            (session_id,)
        )
        db.commit()
    raise

logger.info(
    "Prompt received: prompt_len=%d model=%s provider=%s",
    len(prompt),
    model,
    context.provider,
)

async def event_stream() -> AsyncGenerator[str, None]:
    sidecar: SidecarClient | None = None
    coordinator = get_run_coordinator()
    accumulated = ""
    assistant_msg_id: str | None = None
    chunk_count = 0

```

```

usage_data: dict[str, Any] = {}
result_provider = context.provider or ""
turn_status = "completed"
turn_events: list[dict[str, Any]] = []
activity: _ContainerActivity | None = None

try:
    activity = await _acquire_tenant_container_activity(
        request,
        tenant,
        runtime_id=context.runtime_id,
        required=context.sidecar_socket is not None,
    )
    sidecar = await create_sidecar_connection(context.sidecar_socket)
    await coordinator.register(session_id, sidecar)

    available_thinking_levels: tuple[ThinkingLevel, ...] | None = None
    if body.thinking is not None:
        catalog_payload = await sidecar.get_catalog(agent_dir=context.agent_dir)
        available_thinking_levels = _catalog_thinking_levels(
            catalog_payload,
            context.model_ref or model,
        )
    if available_thinking_levels == (_THINKING_LEVEL_OFF,):
        logger.info(
            "Ignoring thinking override for non-reasoning model: model=%s",
            context.model_ref or model,
        )

    effective_settings = _build_effective_settings(
        context.settings_payload,
        body.thinking,
        available_thinking_levels,
    )

    await sidecar.warmup(
        session_id,
        model=context.model_ref or model,
        cwd=context.effective_cwd,
        provider_auth=cast(dict[str, Any] | None, context.provider_auth),
        provider_config=cast(dict[str, Any] | None, context.provider_config),
        git_auth=cast(dict[str, Any] | None, context.git_auth),
        agent_dir=context.agent_dir,
        settings_payload=effective_settings,
        pi_session_file=context.pi_session_file,
    )

```

```

logger.info("Prompt stream started")

sidecar_events = sidecar.query(
    session_id,
    prompt,
    model=context.model_ref or model,
    cwd=context.effective_cwd,
    provider_auth=cast(dict[str, Any] | None, context.provider_auth),
    provider_config=cast(dict[str, Any] | None, context.provider_config),
    git_auth=cast(dict[str, Any] | None, context.git_auth),
    agent_dir=context.agent_dir,
    settings_payload=effective_settings,
    pi_session_file=context.pi_session_file,
)
if desktop_device_id is not None:
    assert tenant is not None, "desktop authority requires a tenant"
    event_source = _desktop_device_bound_events(
        sidecar_events,
        user_id=tenant.user_id,
        device_id=desktop_device_id,
        sidecar=sidecar,
        session_id=session_id,
    )
elif requires_auth_session:
    if not auth_session_token:
        raise _AuthSessionRevoked
    event_source = _session_bound_events(
        sidecar_events,
        auth_session_token,
        sidecar,
        session_id,
    )
else:
    event_source = sidecar_events

async for event in event_source:
    event_type = event.get("type")
    if not isinstance(event_type, str):
        raise SidecarError("Sidecar event type must be a string")
    logger.debug("Sidecar event received")

    if event_type == "cancelled":
        turn_status = "cancelled"
        cancel_event = {"type": "cancelled", "reason": "user_stop"}
        turn_events.append(_stored_turn_event(cancel_event))

```

```

        yield f"data: {json.dumps(cancel_event)}\n\n"
        break

    if event_type == "message":
        data = event.get("data", {})
        if not isinstance(data, dict):
            raise SidecarError("Sidecar message event must contain object data")
        logger.debug("Sidecar message event received")
        turn_events.append(_stored_turn_event(data))

        # Extract assistant text for persistence
        if data.get("type") == "assistant":
            message_payload = data.get("message", {})
            if not isinstance(message_payload, dict):
                raise SidecarError("Assistant event message payload must be an object")
            content_blocks = message_payload.get("content", [])
            if not isinstance(content_blocks, list):
                raise SidecarError("Assistant event content must be a list")
            for block in content_blocks:
                if isinstance(block, dict) and block.get("type") == "text":
                    text = block.get("text", "")
                    if isinstance(text, str) and text:
                        accumulated += text
                        chunk_count += 1

        # Batched incremental persistence
        if accumulated and chunk_count % _PERSIST_BATCH_SIZE == 0:
            with get_db_for_request(request) as db:
                if assistant_msg_id is None:
                    assistant_msg_id = uuid.uuid4().hex
                    db.execute(
                        (
                            "INSERT INTO messages "
                            "(id, session_id, role, content, turn_id) "
                            "VALUES (?, ?, 'assistant', ?, ?)"
                        ),
                        (assistant_msg_id, session_id, accumulated, turn_id)
                    )
                else:
                    db.execute(
                        "UPDATE messages SET content = ? WHERE id = ?",
                        (accumulated, assistant_msg_id),
                    )
            db.commit()

        # On result, capture usage and finalize with full_message

```

```

if data.get("type") == "result":
    usage_payload = data.get("usage", {})
    usage_data = usage_payload if isinstance(usage_payload, dict) else {}
    provider_payload = data.get("provider")
    if isinstance(provider_payload, str):
        result_provider = provider_payload
    stored_turn = _serialize_stored_turn(turn_events)
    assert (
        stored_turn is not None
    ), "result event must be present in stored turn"
    # Ensure an assistant message row exists even for
    # short responses (< batch size) or tool-only turns.
    with get_db_for_request(request) as db:
        if assistant_msg_id is None:
            assistant_msg_id = uuid.uuid4().hex
            db.execute(
                (
                    "INSERT INTO messages "
                    "(id, session_id, role, content, turn_id) "
                    "VALUES (?, ?, 'assistant', ?, ?)"
                ),
                (assistant_msg_id, session_id, accumulated, turn_id),
            )
        db.execute(
            (
                "UPDATE messages SET full_message = ?, "
                "turn_status = ? WHERE id = ?"
            ),
            (stored_turn, turn_status, assistant_msg_id),
        )
        db.commit()

    # Yield the SSE event with the inner data
    yield f"data: {json.dumps(data)}\n\n"

elif event_type == "error":
    turn_status = "failed"
    error_value = event.get("error", "Unknown sidecar error")
    error_msg = error_value if isinstance(error_value, str) else str(error_value)
    error_event = {"type": "error", "error": error_msg}
    turn_events.append(_stored_turn_event(error_event))
    yield f"data: {json.dumps(error_event)}\n\n"

else:
    # Forward any other event types (content_block_start, tool_use, etc.)
    turn_events.append(_stored_turn_event(event))

```

```

        yield f"data: {json.dumps(event)}\n\n"

    except _AuthSessionRevoked:
        turn_status = "cancelled"
        logger.info("Prompt stream revoked")
    except _StreamLifetimeReached:
        turn_status = "cancelled"
        logger.info("Prompt stream lifetime reached")
    except (
        ConnectionError,
        ContainerNotReadyError,
        ContainerStartError,
        OSError,
        GitError,
        SidecarError,
        TypeError,
        ValueError,
    ):
        logger.error("Sidecar prompt execution failed")
        error_event = {
            "type": "error",
            "error": "An internal error occurred",
        }
        turn_events.append(_stored_turn_event(error_event))
        yield f"data: {json.dumps(error_event)}\n\n"
        turn_status = "failed"

    finally:
        active_error = sys.exception()
        finalization_error: BaseException | None = None

        try:
            with get_db_for_request(request) as db:
                stored_turn = _serialize_stored_turn(turn_events)
                if assistant_msg_id is None:
                    if accumulated or stored_turn is not None:
                        assistant_msg_id = uuid.uuid4().hex
                        db.execute(
                            (
                                "INSERT INTO messages "
                                "(id, session_id, role, content, full_message, "
                                "turn_id, turn_status) "
                                "VALUES (?, ?, 'assistant', ?, ?, ?, ?)"
                            ),
                            (
                                assistant_msg_id,

```

```

        session_id,
        accumulated,
        stored_turn,
        turn_id,
        turn_status,
    ),
)
else:
    db.execute(
        (
            "UPDATE messages SET content = ?, full_message = ?, "
            "turn_status = ? WHERE id = ?"
        ),
        (accumulated, stored_turn, turn_status, assistant_msg_id),
    )
    db.execute(
        "UPDATE sessions SET status = 'idle' WHERE id = ?",
        (session_id,),
    )
    db.commit()
except BaseException as exc:
    logger.error("Prompt persistence finalization failed")
    if active_error is None:
        finalization_error = exc

if tenant and usage_data:
    try:
        record_usage(
            user_id=tenant.user_id,
            session_id=session_id,
            provider=result_provider,
            model=context.model_ref or model,
            usage=usage_data,
            key_source=context.key_source,
        )
    except Exception:
        logger.error("Failed to record prompt usage")

try:
    touch_tenant_container(request, tenant, runtime_id=context.runtime_id)
except BaseException as exc:
    logger.error("Prompt container touch failed")
    if active_error is None and finalization_error is None:
        finalization_error = exc

try:

```



```

        await coordinator.release(session_id)
    except BaseException as exc:
        logger.error("Prompt run release failed")
        if active_error is None and finalization_error is None:
            finalization_error = exc

    if sidecar:
        try:
            await sidecar.disconnect()
        except BaseException as exc:
            logger.error("Prompt sidecar disconnect failed")
            if active_error is None and finalization_error is None:
                finalization_error = exc

    try:
        await _release_tenant_container_activity(activity)
    except BaseException as exc:
        logger.error("Prompt container activity release failed")
        if active_error is None and finalization_error is None:
            finalization_error = exc

    if finalization_error is not None:
        raise finalization_error

    logger.info(
        "Turn complete: chunks=%d content_len=%d turn_status=%s",
        chunk_count,
        len(accumulated),
        turn_status,
    )

    return StreamingResponse(
        event_stream(),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )

@router.post("/api/sessions/{session_id}/cancel")
async def cancel_session(session_id: str, request: Request) -> dict[str, str]:
    """Cancel the active sidecar operation for a session."""
    with get_db_for_request(request) as db:
        session = _lookup_session(db, session_id, request)

    if not session:
        raise HTTPException(status_code=404, detail="Session not found")

```

```

if not get_tenant(request):
    check_owner(session["owner_email"], get_user_email(request))

coordinator = get_run_coordinator()
found = await coordinator.request_cancel(session_id)
if not found:
    raise HTTPException(status_code=409, detail="No active stream for this session")

logger.info("Prompt cancellation requested")
return {"status": "stopping"}

```

7.2 services/sidecar.py

The sidecar client speaks the Unix-socket protocol used to call catalog, prompt, runtime, and terminal operations. The root chunk below tangles back to backend/src/yinshi/services/sidecar.py.

"""Async Unix socket client for communicating with the Node.js pi sidecar."""

```

import asyncio
import json
import logging
from collections.abc import AsyncGenerator, Iterable
from typing import Any, TypedDict

from yinshi.config import get_settings
from yinshi.exceptions import SidecarError, SidecarNotConnectedError
from yinshi.model_catalog import DEFAULT_SESSION_MODEL

class PiCommandPayload(TypedDict):
    """One slash command as serialized over the sidecar socket."""

    kind: str
    name: str
    description: str
    command_name: str

class PiCommandsPayload(TypedDict):
    """Wire shape returned by the sidecar's list_resources handler."""

    commands: list[PiCommandPayload]

```

```

class PiRuntimeVersionPayload(TypedDict):
    """Wire shape returned by the sidecar's version handler."""

    package_name: str
    installed_version: str
    node_version: str

logger = logging.getLogger(__name__)

_SIDECAR_MESSAGE_LIMIT_BYTES = 1024 * 1024 * 8
_SIDECAR_WARMUP_TIMEOUT_SECONDS = 30.0
_SIDECAR_CANCELLATION_TIMEOUT_SECONDS = 30.0

class SidecarClient:
    """Async client for a single sidecar connection via Unix domain socket.

Each instance owns one socket connection. Use one per active session
to avoid message interleaving between concurrent sessions.

Supports ``async with`` for automatic disconnect::

    async with await create_sidecar_connection() as sidecar:
        await sidecar.warmup(...)
    """

    def __init__(self) -> None:
        self._reader: asyncio.StreamReader | None = None
        self._writer: asyncio.StreamWriter | None = None
        self._connected = False
        self._active_query_ids: set[str] = set()
        self._pending_cancellations: dict[str, asyncio.Future[dict[str, Any] | None]] = {}
        self._pending_cancellation_waiters: dict[str, int] = {}
        self._cancelled_before_query: set[str] = set()

    @property
    def connected(self) -> bool:
        return self._connected

    async def __aenter__(self) -> "SidecarClient":
        return self

    async def __aexit__(self, *exc: object) -> None:
        await self.disconnect()

```

```

async def connect(self, socket_path: str | None = None) -> None:
    """Connect to a sidecar Unix socket.

    Args:
        socket_path: Explicit path to the Unix socket. When *None*,
            falls back to the global ``sidecar_socket_path`` setting
            used by host-side execution.
    """
    if socket_path is None:
        settings = get_settings()
        socket_path = settings.sidecar_socket_path
    assert isinstance(socket_path, str)
    assert socket_path
    try:
        self._reader, self._writer = await asyncio.open_unix_connection(
            socket_path,
            limit=_SIDE CAR_MESSAGE_LIMIT_BYTES,
        )
        self._connected = True

        init_line = await self._reader.readline()
        if init_line:
            init_msg = json.loads(init_line.decode())
            if init_msg.get("type") == "init_status" and init_msg.get("success"):
                logger.info("Connected to sidecar runtime")
            else:
                raise SidecarError(f"Sidecar init failed: {init_msg}")
    except FileNotFoundError:
        raise SidecarNotConnectedError(
            f"Sidecar socket not found at {socket_path}. Is the sidecar running?"
        )
    except ConnectionRefusedError:
        raise SidecarNotConnectedError("Sidecar connection refused")

async def disconnect(self) -> None:
    """Close the connection."""
    if self._writer:
        self._writer.close()
        try:
            await self._writer.wait_closed()
        except OSError:
            pass
    self._connected = False
    self._reader = None
    self._writer = None

```

```

async def _send(self, message: dict[str, Any]) -> None:
    """Send a JSON message to the sidecar."""
    if not self._connected or not self._writer:
        raise SidecarNotConnectedError("Not connected to sidecar")
    data = json.dumps(message) + "\n"
    self._writer.write(data.encode())
    await self._writer.drain()

async def _read_line(self) -> dict[str, Any] | None:
    """Read a single JSON line from the sidecar."""
    if not self._reader:
        return None
    try:
        line = await self._reader.readline()
    except ValueError as exc:
        message_text = str(exc)
        if "longer than limit" in message_text:
            raise SidecarError("Sidecar message exceeded the configured read limit") from exc
        raise
    if not line:
        return None
    if len(line) > _SIDECAR_MESSAGE_LIMIT_BYTES:
        raise SidecarError("Sidecar message exceeded the configured read limit")
    message = json.loads(line.decode())
    if not isinstance(message, dict):
        raise SidecarError("Sidecar returned a non-object response")
    return message

def _route_cancellation(self, session_id: str, message: dict[str, Any]) -> bool:
    """Route a cancellation response to waiters without reading concurrently."""
    acknowledgement = self._pending_cancellations.get(session_id)
    if message.get("type") != "cancel_status" or acknowledgement is None:
        return False
    if not acknowledgement.done():
        acknowledgement.set_result(message)
        if (
            message
            == {
                "id": session_id,
                "type": "cancel_status",
                "success": True,
            }
            and session_id not in self._active_query_ids
        ):
            self._cancelled_before_query.add(session_id)
    return True

```

```

def _fail_pending_cancellation(self, session_id: str) -> None:
    """Fail a cancellation when its socket reader exits first."""
    acknowledgement = self._pending_cancellations.get(session_id)
    if acknowledgement is not None and not acknowledgement.done():
        acknowledgement.set_result(None)

    @staticmethod
    def _build_options(
        model: str,
        cwd: str,
        provider_auth: dict[str, Any] | None = None,
        provider_config: dict[str, Any] | None = None,
        git_auth: dict[str, Any] | None = None,
        agent_dir: str | None = None,
        settings_payload: dict[str, Any] | None = None,
        pi_session_file: str | None = None,
    ) -> dict[str, Any]:
        """Build the options dict sent with warmup/query messages."""
        options: dict[str, Any] = {"model": model, "cwd": cwd}
        if provider_auth:
            options["providerAuth"] = provider_auth
        if provider_config:
            options["providerConfig"] = provider_config
        if git_auth:
            options["gitAuth"] = git_auth
        if agent_dir:
            options["agentDir"] = agent_dir
        if settings_payload:
            options["settings"] = settings_payload
        if pi_session_file:
            options["piSessionFile"] = pi_session_file
        return options

    async def warmup(
        self,
        session_id: str,
        model: str = DEFAULT_SESSION_MODEL,
        cwd: str = ".",
        provider_auth: dict[str, Any] | None = None,
        provider_config: dict[str, Any] | None = None,
        git_auth: dict[str, Any] | None = None,
        agent_dir: str | None = None,
        settings_payload: dict[str, Any] | None = None,
        pi_session_file: str | None = None,
    ) -> None:

```

```

"""Pre-create a pi session on the sidecar."""
await self._send(
    {
        "type": "warmup",
        "id": session_id,
        "options": self._build_options(
            model,
            cwd,
            provider_auth,
            provider_config,
            git_auth,
            agent_dir=agent_dir,
            settings_payload=settings_payload,
            pi_session_file=pi_session_file,
        ),
    }
)

try:
    async with asyncio.timeout(_SIDECAR_WARMUP_TIMEOUT_SECONDS):
        while True:
            msg = await self._read_line()
            if msg is None:
                raise SidecarError("Sidecar connection lost during warmup")
            if msg.get("id") != session_id:
                continue
            if self._route_cancellation(session_id, msg):
                continue
            if msg.get("type") != "warmup_status":
                raise SidecarError(f"Unexpected warmup response type: {msg.get('type')}")
            if msg.get("success") is not True:
                raise SidecarError(f"Sidecar warmup failed: {msg.get('error', 'unknown')}")
            return
except TimeoutError as exc:
    raise SidecarError("Sidecar warmup timed out") from exc
finally:
    self._fail_pending_cancellation(session_id)

async def query(
    self,
    session_id: str,
    prompt: str,
    model: str = DEFAULT_SESSION_MODEL,
    cwd: str = ".",
    provider_auth: dict[str, Any] | None = None,
    provider_config: dict[str, Any] | None = None,

```

```

git_auth: dict[str, Any] | None = None,
agent_dir: str | None = None,
settings_payload: dict[str, Any] | None = None,
pi_session_file: str | None = None,
) -> AsyncGenerator[dict[str, Any], None]:
    """Send a prompt and yield streaming events from the sidecar."""
    if session_id in self._cancelled_before_query:
        self._cancelled_before_query.discard(session_id)
        yield {"id": session_id, "type": "cancelled"}
        return

    self._active_query_ids.add(session_id)
    try:
        await self._send(
            {
                "type": "query",
                "id": session_id,
                "prompt": prompt,
                "options": self._build_options(
                    model,
                    cwd,
                    provider_auth,
                    provider_config,
                    git_auth,
                    agent_dir=agent_dir,
                    settings_payload=settings_payload,
                    pi_session_file=pi_session_file,
                ),
            }
        )

    while True:
        msg = await self._read_line()
        if msg is None:
            raise SidecarError("Sidecar connection lost")

        # On a dedicated connection, all messages should be for this session,
        # but filter defensively in case of protocol quirks.
        if msg.get("id") and msg.get("id") != session_id:
            continue

        if self._route_cancellation(session_id, msg):
            continue

        yield msg

```



```

        msg_type = msg.get("type")
        if msg_type in {"cancelled", "error"}:
            break
        if msg_type == "message":
            data = msg.get("data", {})
            if data.get("type") == "result":
                break
    finally:
        self._active_query_ids.discard(session_id)
        self._fail_pending_cancellation(session_id)

    async def resolve_model(
        self,
        model_key: str,
        *,
        agent_dir: str | None = None,
        provider_auth: dict[str, Any] | None = None,
        provider_config: dict[str, Any] | None = None,
    ) -> dict[str, str | None]:
        """Ask the sidecar to resolve a model key.

        Returns {'provider': '...', 'model': '...'}.
        """
        request_id = f"resolve-{model_key}"
        await self._send(
            {
                "type": "resolve",
                "id": request_id,
                "model": model_key,
                "options": self._build_options(
                    model_key,
                    ".",
                    provider_auth,
                    provider_config,
                    agent_dir=agent_dir,
                ),
            }
        )

        msg = await self._read_line()
        if msg is None:
            raise SidecarError("Sidecar connection lost during model resolve")
        if msg.get("type") == "error":
            raise SidecarError(f"Model resolve failed: {msg.get('error', 'unknown')}")
        if msg.get("type") != "resolved":
            raise SidecarError(f"Unexpected response type: {msg.get('type')}")

```

```

        provider = msg.get("provider")
        if provider is not None and not isinstance(provider, str):
            raise SidecarError("Resolved provider must be a string or null")
        model = msg.get("model")
        if not isinstance(model, str) or not model:
            raise SidecarError("Resolved model must be a non-empty string")

        return {"provider": provider, "model": model}

    async def get_catalog(self, *, agent_dir: str | None = None) -> dict[str, Any]:
        """Request the provider/model catalog from the sidecar."""
        request_id = "catalog"
        await self._send(
            {
                "type": "catalog",
                "id": request_id,
                "options": {"agentDir": agent_dir} if agent_dir else {},
            }
        )
        msg = await self._read_line()
        if msg is None:
            raise SidecarError("Sidecar connection lost during catalog request")
        if msg.get("type") == "error":
            raise SidecarError(f"Catalog request failed: {msg.get('error', 'unknown')}")
        if msg.get("type") != "catalog":
            raise SidecarError(f"Unexpected response type: {msg.get('type')}")
        return msg

    async def get_runtime_version(self) -> PiRuntimeVersionPayload:
        """Request the installed pi package version from the sidecar runtime."""
        await self._send({"type": "version", "id": "version"})
        msg = await self._read_line()
        if msg is None:
            raise SidecarError("Sidecar connection lost during version request")
        if msg.get("type") == "error":
            raise SidecarError(f"Version request failed: {msg.get('error', 'unknown')}")
        if msg.get("type") != "version":
            raise SidecarError(f"Unexpected response type: {msg.get('type')}")

        package_name = msg.get("package_name")
        installed_version = msg.get("installed_version")
        node_version = msg.get("node_version")
        if not isinstance(package_name, str) or not package_name:
            raise SidecarError("Runtime package_name must be a non-empty string")
        if not isinstance(installed_version, str) or not installed_version:

```

```

        raise SidecarError("Runtime installed_version must be a non-empty string")
    if not isinstance(node_version, str) or not node_version:
        raise SidecarError("Runtime node_version must be a non-empty string")
    return {
        "package_name": package_name,
        "installed_version": installed_version,
        "node_version": node_version,
    }

async def list_imported_commands(self, *, agent_dir: str | None = None) -> PiCommandsPayload:
    """Request the slash commands discoverable from the user's imported Pi config.

    Returns a flat ``commands`` list where each entry carries ``kind`` to
    distinguish skills, prompts, and extension-registered commands.
    Requests run on a dedicated connection (see create_sidecar_connection)
    so the hardcoded request id never collides with concurrent calls.
    """
    await self._send(
        {
            "type": "list_resources",
            "id": "list_imported_commands",
            "options": {"agentDir": agent_dir} if agent_dir else {},
        }
    )
    msg = await self._read_line()
    if msg is None:
        raise SidecarError("Sidecar connection lost during list_imported_commands request")
    if msg.get("type") == "error":
        raise SidecarError(f"list_imported_commands failed: {msg.get('error', 'unknown')}")
    if msg.get("type") != "resources":
        raise SidecarError(f"Unexpected response type: {msg.get('type')}")
    raw_commands = msg.get("commands", [])
    assert isinstance(raw_commands, list), "sidecar commands must be a list"
    return {"commands": raw_commands}

async def resolve_provider_auth(
    self,
    *,
    provider: str,
    model: str,
    provider_auth: dict[str, Any],
    provider_config: dict[str, Any] | None = None,
    agent_dir: str | None = None,
) -> dict[str, Any]:
    """Resolve one provider auth payload into runtime-ready values."""
    request_id = f"auth-resolve-{provider}"

```

```

        await self._send(
            {
                "type": "auth_resolve",
                "id": request_id,
                "provider": provider,
                "model": model,
                "providerAuth": provider_auth,
                "providerConfig": provider_config,
                "agentDir": agent_dir,
            }
        )
    msg = await self._read_line()
    if msg is None:
        raise SidecarError("Sidecar connection lost during auth resolve")
    if msg.get("type") == "error":
        raise SidecarError(f"Provider auth resolve failed: {msg.get('error', 'unknown')}")
    if msg.get("type") != "auth_resolved":
        raise SidecarError(f"Unexpected response type: {msg.get('type')}")
    return msg

async def start_oauth_flow(self, provider: str) -> dict[str, Any]:
    """Start an OAuth connection flow for a provider."""
    request_id = f"oauth-start-{provider}"
    await self._send({"type": "oauth_start", "id": request_id, "provider": provider})
    msg = await self._read_line()
    if msg is None:
        raise SidecarError("Sidecar connection lost during OAuth start")
    if msg.get("type") == "error":
        raise SidecarError(f"OAuth start failed: {msg.get('error', 'unknown')}")
    if msg.get("type") != "oauth_started":
        raise SidecarError(f"Unexpected response type: {msg.get('type')}")
    return msg

async def get_oauth_flow_status(self, flow_id: str) -> dict[str, Any]:
    """Query the state of a started OAuth flow."""
    request_id = f"oauth-status-{flow_id}"
    await self._send({"type": "oauth_status", "id": request_id, "flowId": flow_id})
    msg = await self._read_line()
    if msg is None:
        raise SidecarError("Sidecar connection lost during OAuth status check")
    if msg.get("type") == "error":
        raise SidecarError(f"OAuth status failed: {msg.get('error', 'unknown')}")
    if msg.get("type") != "oauth_status":
        raise SidecarError(f"Unexpected response type: {msg.get('type')}")
    return msg

```

```

async def submit_oauth_flow_input(
    self,
    flow_id: str,
    authorization_input: str,
) -> dict[str, Any]:
    """Submit a pasted OAuth callback URL or authorization code."""
    if not isinstance(flow_id, str):
        raise TypeError("flow_id must be a string")
    normalized_flow_id = flow_id.strip()
    if not normalized_flow_id:
        raise ValueError("flow_id must not be empty")
    if not isinstance(authorization_input, str):
        raise TypeError("authorization_input must be a string")
    normalized_authorization_input = authorization_input.strip()
    if not normalized_authorization_input:
        raise ValueError("authorization_input must not be empty")

    request_id = f"oauth-submit-{normalized_flow_id}"
    await self._send(
        {
            "type": "oauth_submit",
            "id": request_id,
            "flowId": normalized_flow_id,
            "authorizationInput": normalized_authorization_input,
        }
    )
    msg = await self._read_line()
    if msg is None:
        raise SidecarError("Sidecar connection lost during OAuth input submission")
    if msg.get("type") == "error":
        raise SidecarError(f"OAuth input submission failed: {msg.get('error', 'unknown')}")
    if msg.get("type") != "oauth_submitted":
        raise SidecarError(f"Unexpected response type: {msg.get('type')}")
    return msg

async def clear_oauth_flow(self, flow_id: str) -> None:
    """Clear one completed or failed OAuth flow from the sidecar."""
    request_id = f"oauth-clear-{flow_id}"
    await self._send({"type": "oauth_clear", "id": request_id, "flowId": flow_id})
    msg = await self._read_line()
    if msg is None:
        raise SidecarError("Sidecar connection lost during OAuth cleanup")
    if msg.get("type") == "error":
        raise SidecarError(f"OAuth cleanup failed: {msg.get('error', 'unknown')}")
    if msg.get("type") != "oauth_cleared":
        raise SidecarError(f"Unexpected response type: {msg.get('type')}")

```

```

async def cancel(self, session_id: str) -> None:
    """Cancel a session after receiving its exact acknowledgement."""
    acknowledgement = self._pending_cancellations.get(session_id)
    should_send = acknowledgement is None
    if acknowledgement is None:
        acknowledgement = asyncio.get_running_loop().create_future()
        self._pending_cancellations[session_id] = acknowledgement
    self._pending_cancellation_waiters[session_id] = (
        self._pending_cancellation_waiters.get(session_id, 0) + 1
    )

    try:
        async with asyncio.timeout(_SIDECAR_CANCELLATION_TIMEOUT_SECONDS):
            if should_send:
                try:
                    await self._send({"type": "cancel", "id": session_id})
                except Exception:
                    if not acknowledgement.done():
                        acknowledgement.set_result(None)
                    raise
                response = await asyncio.shield(acknowledgement)
            if response != {
                "id": session_id,
                "type": "cancel_status",
                "success": True,
            }:
                raise SidecarError("Sidecar cancellation failed")
    except Exception as exc:
        raise SidecarError("Sidecar cancellation failed") from exc
    finally:
        remaining_waiters = self._pending_cancellation_waiters[session_id] - 1
        if remaining_waiters:
            self._pending_cancellation_waiters[session_id] = remaining_waiters
        else:
            self._pending_cancellation_waiters.pop(session_id, None)
            if self._pending_cancellations.get(session_id) is acknowledgement:
                self._pending_cancellations.pop(session_id, None)

async def release_session(self, session_id: str) -> None:
    """Drop one pi session so the sidecar can free its context.

    The sidecar also expires idle sessions on its own, so this is an
    optimisation rather than a correctness requirement. Callers therefore
    treat failure as harmless.
    """

```

```

        await self._send({"type": "session_release", "id": session_id})

    async def ping(self) -> bool:
        """Health check the sidecar."""
        try:
            await self._send({"type": "ping"})
            msg = await self._read_line()
            return msg is not None and msg.get("type") == "pong"
        except Exception:
            return False

    async def release_sessions(socket_path: str | None, session_ids: Iterable[str]) -> None:
        """Ask the sidecar to drop pi sessions that can never be used again.

        The sidecar expires idle sessions and caps how many it holds, so this only
        returns the memory sooner. Any failure is logged and swallowed.
        """
        ids = [session_id for session_id in session_ids if session_id]
        if not ids:
            return

        client = None
        try:
            client = await create_sidecar_connection(socket_path)
            for session_id in ids:
                await client.release_session(session_id)
        except Exception:
            logger.warning("Could not release pi sessions from the sidecar")
        finally:
            if client is not None:
                await client.disconnect()

    async def create_sidecar_connection(
        socket_path: str | None = None,
    ) -> SidecarClient:
        """Create a new sidecar connection.

        Args:
            socket_path: Explicit Unix socket path. *None* uses the global
                host-sidecar setting.
        """
        client = SidecarClient()
        await client.connect(socket_path)
        return client

```

7.3 services/run_coordinator.py

The run coordinator tracks active prompt tasks so duplicate runs and cancellation are handled consistently. The root chunk below tangles back to `backend/src/yinshi/services/run_coordinator.py`.

```
"""Run coordinator for managing active prompt runs and cancellation."""

import asyncio
import logging

from yinshi.services.sidecar import SidecarClient

logger = logging.getLogger(__name__)

class RunCoordinator:
    """Manages active sidecar runs keyed by session id."""

    def __init__(self) -> None:
        self._runs: dict[str, SidecarClient] = {}
        self._lock = asyncio.Lock()

    async def register(self, session_id: str, sidecar: SidecarClient) -> None:
        """Register a new active run."""
        if not session_id:
            raise ValueError("session_id must be non-empty")
        cancel_method = getattr(sidecar, "cancel", None)
        if not callable(cancel_method):
            raise TypeError("sidecar must expose a callable cancel method")

        async with self._lock:
            self._runs[session_id] = sidecar
            logger.debug("Run registered")

    async def request_cancel(self, session_id: str) -> bool:
        """Request cancellation for a run. Returns True when a run was found."""
        if not session_id:
            raise ValueError("session_id must be non-empty")

        async with self._lock:
            sidecar = self._runs.get(session_id)
            if sidecar is None:
                return False

        await sidecar.cancel(session_id)
```



```

        logger.info("Run cancellation requested")
        return True

    async def release(self, session_id: str) -> None:
        """Remove a run record."""
        if not session_id:
            raise ValueError("session_id must be non-empty")

        async with self._lock:
            self._runs.pop(session_id, None)
            logger.debug("Run released")

_coordinator: RunCoordinator | None = None

def get_run_coordinator() -> RunCoordinator:
    """Get the global run coordinator instance."""
    global _coordinator
    if _coordinator is None:
        _coordinator = RunCoordinator()
    return _coordinator

```

8 Container Isolation

Tenant mode runs the sidecar behind a per-user container boundary. The backend mounts only the active runtime paths that the sidecar needs. Explicit path mapping and activity tracking protect live prompts and terminals from cleanup.

8.1 services/container.py

Container management starts per-user Podman sidecars with narrow mounts instead of exposing the whole data directory. The root chunk below tangles back to backend/src/yinshi/services/container.py.

```

"""Per-user Podman container management for sidecar isolation."""

import asyncio
import json
import logging
import os
import re
import tempfile
import time
from contextlib import asynccontextmanager
from dataclasses import dataclass, field

```

```

from datetime import datetime, timedelta, timezone
from typing import IO, TYPE_CHECKING, Any, AsyncIterator

from yinshi.exceptions import ContainerNotReadyError, ContainerStartError

logger = logging.getLogger(__name__)

_SIDE CAR_NET = "yinshi-sidecar-net"
_PODMAN_RUN_TIMEOUT_S = 90.0
_USER_ID_RE = re.compile(r"[0-9a-f]{32}$")

if TYPE_CHECKING:
    from yinshi.config import Settings

@dataclass(frozen=True)
class ContainerMount:
    """One host path exposed to a sidecar container."""

    source_path: str
    target_path: str
    read_only: bool = False

@dataclass
class ContainerInfo:
    """Tracks one sidecar container runtime."""

    container_id: str
    user_id: str
    socket_path: str
    mounts: tuple[ContainerMount, ...] = field(default_factory=tuple)
    runtime_id: str | None = None
    environment: tuple[tuple[str, str], ...] = field(default_factory=lambda: (("HOME", "/tmp"),))
    created_at: datetime = field(default_factory=lambda: datetime.now(timezone.utc))
    last_activity: datetime = field(default_factory=lambda: datetime.now(timezone.utc))
    active_request_count: int = 0
    protected_operation_deadlines: dict[str, datetime] = field(default_factory=dict)

@dataclass(frozen=True, slots=True)
class ContainerActivityReservation:
    """Identifies one acquired container generation."""

    container_key: str
    container: ContainerInfo = field(repr=False, compare=False)

```

```

class ContainerManager:
    """Manages per-user Podman containers for sidecar isolation.

    Each user gets a dedicated container with only the paths required for the
    current sidecar operation mounted. Containers are reaped after an idle timeout.
    """

    def __init__(
        self,
        settings: "Settings", # noqa: F821 -- forward ref avoids circular import
        podman_binary: str = "podman",
    ) -> None:
        self._settings = settings
        self._podman_bin = podman_binary
        self._containers: dict[str, ContainerInfo] = {}
        self._pending_container_keys: set[str] = set()
        self._locks: dict[str, asyncio.Lock] = {}
        self._lock_ref_counts: dict[str, int] = {}
        self._global_lock = asyncio.Lock()
        self._initialization_lock = asyncio.Lock()
        self._socket_poll_timeout_s: float = 10.0
        self._socket_poll_interval_s: float = 0.1
        self._initialized = False

    @staticmethod
    def _now() -> datetime:
        """Return the current UTC timestamp."""
        return datetime.now(timezone.utc)

# -- Podman subprocess helper -----

    async def _run_podman(
        self,
        *args: str,
        check: bool = True,
        timeout: float = 30.0,
    ) -> tuple[int, str, str]:
        """Execute a podman command and return (returncode, stdout, stderr).

        Raises ``ContainerStartError`` when *check* is True and the
        command exits non-zero, or when the binary is missing / times out.
        """
        proc = await self._start_podman_process(
            *args,

```

```

        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )

    try:
        stdout_data, stderr_data = await asyncio.wait_for(
            proc.communicate(),
            timeout=timeout,
        )
    except asyncio.TimeoutError:
        await self._stop_process(proc)
        raise ContainerStartError(
            f"Podman command timed out: podman {' '.join(args)}"
        ) from None

    return self._checked_podman_result(
        args,
        proc.returncode,
        self._decode_process_output(stdout_data),
        self._decode_process_output(stderr_data),
        check=check,
    )

async def _run_podman_waiting_for_exit(
    self,
    *args: str,
    check: bool = True,
    timeout: float = 30.0,
) -> tuple[int, str, str]:
    """Execute Podman and wait on process exit instead of pipe EOF.

    Detached ``podman run`` can leave the captured pipes open in the
    spawned container monitor on some rootless runtimes. Waiting for the
    Podman process exit avoids turning a successful detached start into a
    false timeout while still preserving stdout and stderr for failures.
    """

    with tempfile.TemporaryFile() as stdout_file, tempfile.TemporaryFile() as stderr_file:
        proc = await self._start_podman_process(
            *args,
            stdout=stdout_file,
            stderr=stderr_file,
        )
        try:
            await asyncio.wait_for(proc.wait(), timeout=timeout)
        except asyncio.CancelledError:
            await asyncio.shield(self._stop_process(proc))

```

```

        raise
    except asyncio.TimeoutError:
        await self._stop_process(proc)
        raise ContainerStartError(
            f"Podman command timed out: podman {' '.join(args)}"
        ) from None

    stdout_file.seek(0)
    stderr_file.seek(0)
    stdout = self._decode_process_output(stdout_file.read())
    stderr = self._decode_process_output(stderr_file.read())

    return self._checked_podman_result(args, proc.returncode, stdout, stderr, check=check)

async def _start_podman_process(
    self,
    *args: str,
    stdout: int | IO[Any] | None,
    stderr: int | IO[Any] | None,
) -> asyncio.subprocess.Process:
    """Start one Podman subprocess with explicit output destinations."""
    try:
        proc = await asyncio.create_subprocess_exec(
            self._podman_bin,
            *args,
            stdout=stdout,
            stderr=stderr,
        )
    except FileNotFoundError:
        raise ContainerStartError("podman binary not found") from None

    assert proc is not None, "create_subprocess_exec must return a process"
    return proc

def _checked_podman_result(
    self,
    args: tuple[str, ...],
    returncode: int | None,
    stdout: str,
    stderr: str,
    *,
    check: bool,
) -> tuple[int, str, str]:
    """Validate one completed Podman process result."""
    if returncode is None:
        raise ContainerStartError("Podman process exited without a return code")

```

```

        if check:
            if returncode != 0:
                raise ContainerStartError(f"podman {args[0]} failed: {stderr}")

        return returncode, stdout, stderr

    @staticmethod
    def _decode_process_output(raw_data: bytes | str | None) -> str:
        """Decode subprocess output without truncating Podman's JSON responses."""
        if raw_data is None:
            return ""
        if isinstance(raw_data, str):
            return raw_data.strip()
        if not raw_data:
            return ""
        return raw_data.decode(errors="replace").strip()

    async def _stop_process(
        self,
        proc: asyncio.subprocess.Process,
    ) -> None:
        """Terminate one subprocess while tolerating races with an already-exited process."""
        if proc.returncode is not None:
            return

        try:
            proc.kill()
        except ProcessLookupError:
            return

        try:
            await asyncio.wait_for(proc.wait(), timeout=1.0)
        except asyncio.TimeoutError:
            logger.warning("Timed out waiting for Podman process shutdown")

# -- Initialization -----

    async def initialize(self) -> None:
        """Create the Podman network and clean up orphaned containers.

        Idempotent -- safe to call more than once. Called automatically
        on the first ``ensure_container`` invocation.
        """
        if self._initialized:
            return

```

```

    async with self._initialization_lock:
        if self._initialized:
            return
        self._ensure_socket_base_dir()
        await self._verify_podman_available()
        await self._ensure_network()
        await self._ensure_image()
        await self._cleanup_orphaned_containers()
        self._initialized = True

# -- Podman network -----

def _ensure_socket_base_dir(self) -> None:
    """Create the shared socket base directory with restricted permissions."""
    socket_base_dir = self._settings.container_socket_base
    if not isinstance(socket_base_dir, str):
        raise TypeError("container_socket_base must be a string")
    normalized_socket_base_dir = socket_base_dir.strip()
    if not normalized_socket_base_dir:
        raise ValueError("container_socket_base must not be empty")

    try:
        if os.path.exists(normalized_socket_base_dir):
            if not os.path.isdir(normalized_socket_base_dir):
                raise ContainerStartError("container socket base path must be a directory")
            else:
                os.makedirs(normalized_socket_base_dir, mode=0o700, exist_ok=True)
                os.chmod(normalized_socket_base_dir, 0o700)
        except OSError as exc:
            raise ContainerStartError(
                f"Failed to prepare container socket base: {normalized_socket_base_dir}"
            ) from exc

    async def _verify_podman_available(self) -> None:
        """Fail fast when the Podman CLI is missing or unhealthy."""
        await self._run_podman("--version")

    async def _ensure_network(self) -> None:
        """Create the tenant network and repair old internal-only variants."""
        rc, stdout, _ = await self._run_podman(
            "network",
            "inspect",
            _SIDECAR_NET,
            check=False,
        )
        if rc != 0:

```

```

        await self._create_network()
        return

    if self._network_is_internal(stdout):
        await self._run_podman("network", "rm", _SIDECAR_NET)
        await self._create_network()
        logger.info("Recreated Podman network %s without internal isolation", _SIDECAR_NET)

    async def _create_network(self) -> None:
        """Create one Podman network with outbound access for model providers."""
        await self._run_podman(
            "network",
            "create",
            _SIDECAR_NET,
        )
        logger.info("Created Podman network %s", _SIDECAR_NET)

    def _network_is_internal(self, inspect_output: str) -> bool:
        """Return whether one inspected Podman network blocks outbound traffic."""
        if not isinstance(inspect_output, str):
            raise TypeError("inspect_output must be a string")
        normalized_inspect_output = inspect_output.strip()
        if not normalized_inspect_output:
            return False

        try:
            parsed_output = json.loads(normalized_inspect_output)
        except json.JSONDecodeError as exc:
            raise ContainerStartError("Podman network inspect returned invalid JSON") from exc
        if not isinstance(parsed_output, list):
            raise ContainerStartError("Podman network inspect returned an invalid payload")
        if not parsed_output:
            return False

        network_info = parsed_output[0]
        if not isinstance(network_info, dict):
            raise ContainerStartError("Podman network inspect returned a non-object network")

        internal_value = network_info.get("internal")
        if isinstance(internal_value, bool):
            return internal_value
        return False

    async def _ensure_image(self) -> None:
        """Require the configured sidecar image to be present locally."""
        image_name = self._settings.container_image

```



```

        if not isinstance(image_name, str):
            raise TypeError("container_image must be a string")
        normalized_image_name = image_name.strip()
        if not normalized_image_name:
            raise ValueError("container_image must not be empty")

    rc, _, _ = await self._run_podman(
        "image",
        "exists",
        normalized_image_name,
        check=False,
    )
    if rc != 0:
        raise ContainerStartError(
            f"Configured sidecar image is not available locally: {normalized_image_name}"
        )

# -- Orphan cleanup -----

async def _cleanup_orphaned_containers(self) -> None:
    """Remove containers left over from a previous process crash."""
    try:
        rc, stdout, _ = await self._run_podman(
            "ps",
            "-a",
            "--filter",
            "label=yinshi.user_id",
            "--format",
            "json",
            check=False,
        )
        if rc != 0 or not stdout:
            return
        containers = json.loads(stdout)
        for c in containers:
            cid = c.get("Id", c.get("id", ""))
            if cid:
                await self._run_podman("rm", "-f", cid, check=False)
                logger.info("Removed orphaned sidecar container")
    except (json.JSONDecodeError, ContainerStartError):
        logger.warning("Failed to clean up orphaned containers")

# -- Per-user locks -----

@asynccontextmanager
async def _runtime_lock(self, container_key: str) -> AsyncIterator[None]:

```

```

"""Serialize work for one runtime without blocking unrelated runtimes."""
async with self._global_lock:
    lock = self._locks.setdefault(container_key, asyncio.Lock())
    self._lock_ref_counts[container_key] = self._lock_ref_counts.get(container_key,

acquired = False
try:
    await lock.acquire()
    acquired = True
    yield
finally:
    if acquired:
        lock.release()
    async with self._global_lock:
        remaining = self._lock_ref_counts[container_key] - 1
        if remaining:
            self._lock_ref_counts[container_key] = remaining
        else:
            del self._lock_ref_counts[container_key]
            if (
                container_key not in self._containers
                and container_key not in self._pending_container_keys
            ):
                self._locks.pop(container_key, None)

def _container_key(self, user_id: str, runtime_id: str | None = None) -> str:
    """Return the in-memory key for one user or workspace runtime."""
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    normalized_user_id = user_id.strip()
    if not normalized_user_id:
        raise ValueError("user_id must not be empty")
    if runtime_id is None:
        return normalized_user_id
    if not isinstance(runtime_id, str):
        raise TypeError("runtime_id must be a string or None")
    normalized_runtime_id = runtime_id.strip()
    if not normalized_runtime_id:
        raise ValueError("runtime_id must not be empty when provided")
    if not re.match(r"^[0-9a-f]{32}$", normalized_runtime_id):
        raise ValueError(f"Invalid runtime_id format: {runtime_id!r}")
    return f"{normalized_user_id}:{normalized_runtime_id}"

def _socket_dir(self, user_id: str, runtime_id: str | None = None) -> str:
    """Return the host socket directory for one sidecar runtime."""
    if runtime_id is None:

```

```

        return os.path.join(self._settings.container_socket_base, user_id)
    return os.path.join(self._settings.container_socket_base, user_id, runtime_id)

def _container_name(self, user_id: str, runtime_id: str | None = None) -> str:
    """Return a Podman-safe, stable container name for one runtime."""
    if runtime_id is None:
        return f"yinshi-sidecar-{user_id}"
    return f"yinshi-sidecar-{user_id[:12]}-{runtime_id}"

def _default_mounts(self, data_dir: str) -> tuple[ContainerMount, ...]:
    """Return the legacy tenant-data mount for compatibility paths."""
    real_data_dir = os.path.realpath(data_dir)
    return (ContainerMount(source_path=real_data_dir, target_path="/data", read_only=False),)

def _normalize_mounts(
    self,
    data_dir: str,
    mounts: tuple[ContainerMount, ...] | None,
) -> tuple[ContainerMount, ...]:
    """Validate and normalize the host paths mounted into a container."""
    if mounts is None:
        return self._default_mounts(data_dir)
    real_data_dir = os.path.realpath(data_dir)
    normalized_mounts: list[ContainerMount] = []
    seen_targets: set[str] = set()
    for mount in mounts:
        if not isinstance(mount, ContainerMount):
            raise TypeError("mounts must contain ContainerMount values")
        source_path = os.path.realpath(mount.source_path)
        target_path = mount.target_path.strip()
        if not source_path.startswith(real_data_dir + os.sep):
            if source_path != real_data_dir:
                raise ContainerStartError("container mount source must stay inside tenant data dir")
        if source_path == real_data_dir:
            raise ContainerStartError(
                "narrow container mounts must not expose the tenant data root"
            )
        if not os.path.exists(source_path):
            raise ContainerStartError(f"container mount source does not exist: {source_path}")
        if not os.path.isabs(target_path):
            raise ContainerStartError("container mount target must be an absolute path")
        if target_path in seen_targets:
            raise ContainerStartError("container mount targets must be unique")
        seen_targets.add(target_path)
        normalized_mounts.append(
            ContainerMount(

```

```

        source_path=source_path,
        target_path=target_path,
        read_only=mount.read_only,
    )
)
return tuple(sorted(normalized_mounts, key=lambda value: value.target_path))

def _normalize_environment(
    self,
    environment: dict[str, str] | None,
) -> tuple[tuple[str, str], ...]:
    """Validate and normalize environment variables for one runtime."""
    if environment is None:
        return (("HOME", "/tmp"),)
    normalized_environment: list[tuple[str, str]] = []
    for key, value in environment.items():
        if not isinstance(key, str):
            raise TypeError("environment keys must be strings")
        if not isinstance(value, str):
            raise TypeError("environment values must be strings")
        normalized_key = key.strip()
        if not normalized_key:
            raise ValueError("environment keys must not be empty")
        if "=" in normalized_key or "\x00" in normalized_key:
            raise ValueError("environment keys must not contain '=' or NUL")
        if "\x00" in value:
            raise ValueError("environment values must not contain NUL")
        normalized_environment.append((normalized_key, value))
    return tuple(sorted(normalized_environment, key=lambda item: item[0]))

def _container_has_busy_state(self, info: ContainerInfo) -> bool:
    """Return whether a container is unsafe to replace for mount changes."""
    if info.active_request_count > 0:
        return True
    return bool(info.protected_operation_deadlines)

# -- Public API -----

async def ensure_container(
    self,
    user_id: str,
    data_dir: str,
    mounts: tuple[ContainerMount, ...] | None = None,
    *,
    runtime_id: str | None = None,
    environment: dict[str, str] | None = None,

```

```

) -> ContainerInfo:
    """Get or create a sidecar container for a user or workspace runtime.

    Raises ``ValueError`` if *user_id* is not a valid 32-char hex string.
    Raises ``ContainerStartError`` if the max container limit is reached.
    """

    if not _USER_ID_RE.match(user_id):
        raise ValueError(f"Invalid user_id format: {user_id!r}")
    container_key = self._container_key(user_id, runtime_id)
    normalized_mounts = self._normalize_mounts(data_dir, mounts)
    normalized_environment = self._normalize_environment(environment)

    if not self._initialized:
        await self.initialize()

    async with self._runtime_lock(container_key):
        existing = self._containers.get(container_key)
        if existing:
            if await self._is_running(existing.container_id):
                if (
                    existing.mounts == normalized_mounts
                    and existing.environment == normalized_environment
                ):
                    existing.last_activity = datetime.now(timezone.utc)
                    return existing
                if self._container_has_busy_state(existing):
                    raise ContainerStartError(
                        "Existing sidecar container is busy with a different runtime con
                    )
            removed = await self._remove_container(existing.container_id)
            if not removed:
                raise ContainerStartError("Failed to remove existing sidecar container")
            self._reserve_replacement(container_key, existing)
        else:
            await self._reserve_container_slot(container_key)

    info: ContainerInfo | None = None
    try:
        info = await self._create_container(
            user_id,
            normalized_mounts,
            runtime_id=runtime_id,
            environment=normalized_environment,
        )
    self._publish_container(container_key, info)
    return info

```

```

        except BaseException:
            if info is not None:
                await asyncio.shield(self._remove_container(info.container_id))
                if self._containers.get(container_key) is info:
                    del self._containers[container_key]
            raise
        finally:
            self._release_container_slot(container_key)

    async def _reserve_container_slot(self, container_key: str) -> None:
        """Reserve quota before starting Podman for one new runtime."""
        max_count = getattr(self._settings, "container_max_count", 0)
        async with self._global_lock:
            tracked_count = len(self._containers) + len(self._pending_container_keys)
            if not max_count or tracked_count < max_count:
                self._pending_container_keys.add(container_key)
            return

        await self.reap_idle()

        async with self._global_lock:
            tracked_count = len(self._containers) + len(self._pending_container_keys)
            if max_count and tracked_count >= max_count:
                raise ContainerStartError("Maximum container limit reached")
            self._pending_container_keys.add(container_key)

    def _reserve_replacement(
        self,
        container_key: str,
        existing: ContainerInfo,
    ) -> None:
        """Convert one tracked container into a creation reservation."""
        if self._containers.get(container_key) is not existing:
            raise ContainerStartError("Container runtime changed during replacement")
        del self._containers[container_key]
        self._pending_container_keys.add(container_key)

    def _publish_container(
        self,
        container_key: str,
        info: ContainerInfo,
    ) -> None:
        """Replace one quota reservation with its ready container."""
        if container_key not in self._pending_container_keys:
            raise ContainerStartError("Container quota reservation was lost")
        self._pending_container_keys.remove(container_key)

```

```

        self._containers[container_key] = info

def _release_container_slot(self, container_key: str) -> None:
    """Release a failed or cancelled creation reservation."""
    self._pending_container_keys.discard(container_key)

def touch(self, user_id: str, *, runtime_id: str | None = None) -> None:
    """Update last activity timestamp for a runtime container."""
    info = self._containers.get(self._container_key(user_id, runtime_id))
    if info:
        info.last_activity = self._now()

async def acquire_activity(
    self,
    user_id: str,
    *,
    runtime_id: str | None = None,
) -> ContainerActivityReservation | None:
    """Reserve the current runtime generation before a caller uses it."""
    container_key = self._container_key(user_id, runtime_id)
    async with self._runtime_lock(container_key):
        info = self._containers.get(container_key)
        if info is None:
            logger.warning("Cannot acquire activity for missing container")
            return None
        info.active_request_count += 1
        info.last_activity = self._now()
        return ContainerActivityReservation(container_key, info)

async def release_activity(
    self,
    reservation: ContainerActivityReservation,
) -> None:
    """Release the exact runtime generation represented by a reservation."""
    if not isinstance(reservation, ContainerActivityReservation):
        raise TypeError("reservation must be a ContainerActivityReservation")
    async with self._runtime_lock(reservation.container_key):
        info = reservation.container
        if info.active_request_count == 0:
            logger.warning("Cannot release container activity without a matching acquire")
            return
        info.active_request_count -= 1
        info.last_activity = self._now()

def begin_activity(self, user_id: str, *, runtime_id: str | None = None) -> None:
    """Mark a container as busy for the lifetime of one request."""

```

```

        container_key = self._container_key(user_id, runtime_id)
        info = self._containers.get(container_key)
        if info is None:
            logger.warning("Cannot mark activity for missing container")
            return
        info.active_request_count += 1
        info.last_activity = self._now()

def end_activity(self, user_id: str, *, runtime_id: str | None = None) -> None:
    """Release one active request marker for a runtime container."""
    container_key = self._container_key(user_id, runtime_id)
    info = self._containers.get(container_key)
    if info is None:
        logger.warning("Cannot end activity for missing container")
        return
    if info.active_request_count == 0:
        logger.warning("Cannot end container activity without a matching begin")
        return
    info.active_request_count -= 1
    info.last_activity = self._now()

def protect(
    self,
    user_id: str,
    lease_key: str,
    timeout_s: int,
    *,
    runtime_id: str | None = None,
) -> None:
    """Keep one container alive for a named long-lived operation."""
    container_key = self._container_key(user_id, runtime_id)
    if not isinstance(lease_key, str):
        raise TypeError("lease_key must be a string")
    normalized_lease_key = lease_key.strip()
    if not normalized_lease_key:
        raise ValueError("lease_key must not be empty")
    if not isinstance(timeout_s, int):
        raise TypeError("timeout_s must be an integer")
    if timeout_s <= 0:
        raise ValueError("timeout_s must be positive")

    info = self._containers.get(container_key)
    if info is None:
        logger.warning("Cannot protect missing container")
        return
    info.protected_operation_deadlines[normalized_lease_key] = self._now() + timedelta(

```



```

        seconds=timeout_s
    )
    info.last_activity = self._now()

def unprotect(
    self,
    user_id: str,
    lease_key: str,
    *,
    runtime_id: str | None = None,
) -> None:
    """Remove one named long-lived operation lease from a runtime container."""
    container_key = self._container_key(user_id, runtime_id)
    if not isinstance(lease_key, str):
        raise TypeError("lease_key must be a string")
    normalized_lease_key = lease_key.strip()
    if not normalized_lease_key:
        raise ValueError("lease_key must not be empty")

    info = self._containers.get(container_key)
    if info is None:
        logger.warning("Cannot unprotect missing container")
        return
    info.protected_operation_deadlines.pop(normalized_lease_key, None)
    info.last_activity = self._now()

async def destroy_container(self, user_id: str, *, runtime_id: str | None = None) -> bool:
    """Stop the current runtime and report whether workspace deletion is safe."""
    container_key = self._container_key(user_id, runtime_id)
    async with self._runtime_lock(container_key):
        current = self._containers.get(container_key)
        if current is None:
            return True
        self._prune_expired_protection(current, self._now())
        if self._container_has_busy_state(current):
            return False
        removed = await self._remove_container(current.container_id)
        if not removed:
            return False
        del self._containers[container_key]
        logger.info("Destroyed container runtime")
        return True

async def reap_idle(self) -> int:
    """Destroy containers that remain idle after acquiring their runtime lock."""
    timeout = self._settings.container_idle_timeout_s

```

```

cutoff = self._now()
idle_candidates = [
    (container_key, info)
    for container_key, info in self._containers.items()
    if self._container_is_reapable(info, cutoff, timeout)
]
removed_count = 0
for container_key, expected in idle_candidates:
    async with self._runtime_lock(container_key):
        current = self._containers.get(container_key)
        if current is not expected:
            continue
        if not self._container_is_reapable(current, self._now(), timeout):
            continue
        removed = await self._remove_container(current.container_id)
        if not removed:
            continue
        if self._containers.get(container_key) is current:
            del self._containers[container_key]
            removed_count += 1
            logger.info("Destroyed container runtime")
return removed_count

async def run_reaper(self) -> None:
    """Background task that periodically reaps idle containers."""
    while True:
        await asyncio.sleep(60)
        try:
            count = await self.reap_idle()
            if count:
                logger.info("Reaped %d idle container(s)", count)
        except Exception:
            logger.error("Error in container reaper")

async def destroy_all(self) -> None:
    """Destroy all managed containers (shutdown hook)."""
    container_entries = list(self._containers.items())
    for container_key, expected in container_entries:
        await self._destroy_container_by_key(container_key, expected, allow_busy=True)
    logger.info("All sidecar containers destroyed")

async def _destroy_container_by_key(
    self,
    container_key: str,
    expected: ContainerInfo | None,
    *,

```

```

        allow_busy: bool = False,
    ) -> bool:
        """Remove the selected runtime container after serializing its lifecycle."""
        async with self._runtime_lock(container_key):
            current = self._containers.get(container_key)
            if expected is None or current is not expected:
                return False
            self._prune_expired_protection(current, self._now())
            if not allow_busy and self._container_has_busy_state(current):
                return False
            removed = await self._remove_container(current.container_id)
            if not removed:
                return False
            if self._containers.get(container_key) is not current:
                return False
            del self._containers[container_key]
            logger.info("Destroyed container runtime")
            return True

# -- Internal helpers -----

async def _is_running(self, container_id: str) -> bool:
    """Check if a container is still running."""
    rc, stdout, _ = await self._run_podman(
        "inspect",
        "--format",
        "{{.State.Status}}",
        container_id,
        check=False,
    )
    return rc == 0 and stdout == "running"

async def _remove_container(self, container_id: str) -> bool:
    """Force-remove a container and report whether Podman succeeded."""
    rc, _, _ = await self._run_podman(
        "rm",
        "-f",
        container_id,
        check=False,
    )
    if rc != 0:
        logger.warning("Failed to remove container")
        return False
    logger.info("Removed container")
    return True

```

```

async def _create_container(
    self,
    user_id: str,
    mounts: tuple[ContainerMount, ...],
    *,
    runtime_id: str | None = None,
    environment: tuple[tuple[str, str], ...] = (),
) -> ContainerInfo:
    """Start a new sidecar container for a user or workspace runtime."""
    s = self._settings
    socket_dir = self._socket_dir(user_id, runtime_id)
    socket_path = os.path.join(socket_dir, "sidecar.sock")
    cidfile_path = os.path.join(socket_dir, "container.cid")
    self._prepare_socket_dir(socket_dir, socket_path)
    self._remove_stale_file(cidfile_path, "container cidfile")

    cpus = str(s.container_cpu_quota / 100000)
    mount_args: list[str] = []
    for mount in mounts:
        mode = "ro" if mount.read_only else "rw"
        mount_args.extend(["-v", f"{mount.source_path}:{mount.target_path}:{mode}"])
    runtime_uid = os.getuid()
    runtime_gid = os.getgid()
    env_args = ["--env", "SIDE CAR_SOCKET_PATH=/run/sidecar/sidecar.sock"]
    for key, value in environment or (("HOME", "/tmp"),):
        env_args.extend(["--env", f"{key}={value}"])

    container_id: str | None = None
    container_name = self._container_name(user_id, runtime_id)
    try:
        _, run_stdout, _ = await self._run_podman_waiting_for_exit(
            "run",
            "-d",
            "--replace",
            "--cidfile",
            cidfile_path,
            "--name",
            container_name,
            "--userns",
            "keep-id",
            "--user",
            f"{runtime_uid}:{runtime_gid}",
            *env_args,
            "-v",
            f"{socket_dir}:/run/sidecar:rw",
            *mount_args,

```

```

        "--network",
        _SIDECAR_NET,
        "--memory",
        s.container_memory_limit,
        "--memory-swap",
        s.container_memory_limit,
        "--cpus",
        cpus,
        "--pids-limit",
        str(s.container_pids_limit),
        "--security-opt",
        "no-new-privileges",
        "--cap-drop",
        "ALL",
        "--label",
        f"yinshi.user_id={user_id}",
        "--label",
        f"yinshi.runtime_id={runtime_id or ''}",
        s.container_image,
        timeout=_PODMAN_RUN_TIMEOUT_S,
    )
    container_id = self._resolve_created_container_id(run_stdout, cidfile_path)
    self._discard_runtime_file(cidfile_path, "container cidfile")
    await self._wait_for_socket(socket_path)
except BaseException as error:
    if container_id is None:
        container_id = self._read_optional_container_id(cidfile_path)
    if container_id is not None:
        await asyncio.shield(self._remove_container(container_id))
    elif isinstance(error, asyncio.CancelledError):
        await asyncio.shield(self._remove_container(container_name))
    self._discard_runtime_file(cidfile_path, "container cidfile")
    self._discard_runtime_file(socket_path, "container socket")
    raise

info = ContainerInfo(
    container_id=container_id,
    user_id=user_id,
    socket_path=socket_path,
    mounts=mounts,
    runtime_id=runtime_id,
    environment=environment or (("HOME", "/tmp"),),
)
logger.info("Started container runtime")
return info

```

```

async def _wait_for_socket(self, socket_path: str) -> None:
    """Poll until the sidecar accepts a connection and sends its init message."""
    deadline = time.monotonic() + self._socket_poll_timeout_s
    while time.monotonic() < deadline:
        reader: asyncio.StreamReader | None = None
        writer: asyncio.StreamWriter | None = None
        try:
            reader, writer = await asyncio.open_unix_connection(socket_path)
            init_line = await asyncio.wait_for(
                reader.readline(),
                timeout=self._socket_poll_interval_s,
            )
            if init_line:
                init_message = json.loads(init_line.decode())
                if init_message.get("type") == "init_status" and init_message.get("success"):
                    return
            await asyncio.sleep(self._socket_poll_interval_s)
        except (
            asyncio.TimeoutError,
            ConnectionRefusedError,
            FileNotFoundError,
            OSError,
            json.JSONDecodeError,
        ):
            await asyncio.sleep(self._socket_poll_interval_s)
    finally:
        if writer is not None:
            writer.close()
            try:
                await writer.wait_closed()
            except OSError:
                pass

    raise ContainerNotReadyError(
        f"Sidecar socket not ready after {self._socket_poll_timeout_s}s"
    )

def _resolve_created_container_id(self, stdout: str, cidfile_path: str) -> str:
    """Return the created container id from Podman stdout or its cidfile."""
    if not isinstance(stdout, str):
        raise TypeError("stdout must be a string")
    if not isinstance(cidfile_path, str):
        raise TypeError("cidfile_path must be a string")

    normalized_stdout = stdout.strip()
    if normalized_stdout:

```

```

        return normalized_stdout

    try:
        with open(cidfile_path, encoding="utf-8") as cidfile:
            container_id = cidfile.read().strip()
    except OSError as exc:
        raise ContainerStartError("podman run did not report a container id") from exc

    if not container_id:
        raise ContainerStartError("podman run did not report a container id")
    return container_id

def _read_optional_container_id(self, cidfile_path: str) -> str | None:
    """Read a container id written before a failed or cancelled Podman run."""
    try:
        with open(cidfile_path, encoding="utf-8") as cidfile:
            container_id = cidfile.read().strip()
    except OSError:
        return None
    return container_id or None

def _discard_runtime_file(self, path: str, description: str) -> None:
    """Remove a runtime file without masking its primary lifecycle result."""
    try:
        if os.path.lexists(path) and not os.path.isdir(path):
            os.unlink(path)
    except OSError:
        logger.warning("Failed to remove container runtime file")

def _prepare_socket_dir(self, socket_dir: str, socket_path: str) -> None:
    """Create one user socket directory and remove any stale socket file."""
    if not isinstance(socket_dir, str):
        raise TypeError("socket_dir must be a string")
    normalized_socket_dir = socket_dir.strip()
    if not normalized_socket_dir:
        raise ValueError("socket_dir must not be empty")
    if not isinstance(socket_path, str):
        raise TypeError("socket_path must be a string")
    normalized_socket_path = socket_path.strip()
    if not normalized_socket_path:
        raise ValueError("socket_path must not be empty")

    try:
        os.makedirs(normalized_socket_dir, mode=0o700, exist_ok=True)
        os.chmod(normalized_socket_dir, 0o700)
        if os.path.lexists(normalized_socket_path):

```

```

        if os.path.isdir(normalized_socket_path):
            raise ContainerStartError("container socket path must not be a directory")
        os.unlink(normalized_socket_path)
    except OSError as exc:
        raise ContainerStartError(
            f"Failed to prepare socket directory for user container: {normalized_socket_path}"
        ) from exc

def _remove_stale_file(self, path: str, description: str) -> None:
    """Remove one stale filesystem entry before container creation."""
    if not isinstance(path, str):
        raise TypeError("path must be a string")
    if not isinstance(description, str):
        raise TypeError("description must be a string")
    normalized_path = path.strip()
    if not normalized_path:
        raise ValueError("path must not be empty")
    normalized_description = description.strip()
    if not normalized_description:
        raise ValueError("description must not be empty")

    try:
        if os.path.lexists(normalized_path):
            if os.path.isdir(normalized_path):
                raise ContainerStartError(
                    f"{normalized_description} path must not be a directory"
                )
            os.unlink(normalized_path)
    except OSError as exc:
        raise ContainerStartError(f"Failed to remove stale {normalized_description}") from exc

def _container_is_reapable(
    self,
    info: ContainerInfo,
    cutoff: datetime,
    timeout_s: int,
) -> bool:
    """Return whether one container can be safely reaped."""
    if not isinstance(timeout_s, int):
        raise TypeError("timeout_s must be an integer")
    if timeout_s < 0:
        raise ValueError("timeout_s must not be negative")

    self._prune_expired_protection(info, cutoff)
    if info.active_request_count > 0:
        return False

```



```

        if info.protected_operation_deadlines:
            return False
        return (cutoff - info.last_activity).total_seconds() > timeout_s

def _prune_expired_protection(self, info: ContainerInfo, cutoff: datetime) -> None:
    """Drop any expired protection lease from one container."""
    expired_lease_keys = [
        lease_key
        for lease_key, deadline in info.protected_operation_deadlines.items()
        if deadline <= cutoff
    ]
    for lease_key in expired_lease_keys:
        del info.protected_operation_deadlines[lease_key]

```

8.2 services/sidecar__runtime.py

Sidecar runtime resolution maps tenant paths into container paths and protects active containers from idle cleanup. The root chunk below tangles back to backend/src/yinshi/services/sidecar_runtime.py.

```

"""Shared tenant sidecar runtime resolution for containerized execution."""

from __future__ import annotations

import hashlib
import logging
import os
import posixpath
import re
import shutil
import stat
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager
from dataclasses import dataclass
from pathlib import Path
from typing import cast

from fastapi import Request

from yinshi.config import get_settings
from yinshi.exceptions import ContainerNotReadyError, ContainerStartError
from yinshi.services.container import (
    ContainerActivityReservation,
    ContainerManager,
    ContainerMount,
)

```

```

from yinshi.services.pi_config import resolve_effective_pi_runtime
from yinshi.tenant import TenantContext
from yinshi.utils.paths import is_path_inside

logger = logging.getLogger(__name__)

@dataclass(frozen=True, slots=True)
class TenantSidecarContext:
    """Resolved sidecar runtime inputs for one tenant-scoped request."""

    socket_path: str | None
    agent_dir: str | None
    settings_payload: dict[str, object] | None
    runtime_id: str | None = None
    pi_session_file: str | None = None

_RUNTIME_ID_RE = re.compile(r"^[0-9a-f]{32}$")
_WORKSPACE_HOME_TARGET = "/home/yinshi"
_PI_SESSION_DIRECTORY = ".yinshi/pi-sessions"
_PI_SESSION_PATH_PARTS = tuple(_PI_SESSION_DIRECTORY.split("/"))
_WORKSPACE_PATH = (
    f"{_WORKSPACE_HOME_TARGET}/bin:"
    f"{_WORKSPACE_HOME_TARGET}/.local/bin:"
    f"{_WORKSPACE_HOME_TARGET}/.npm-global/bin:"
    "/app/node_modules/.bin:"
    "/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
)

def remap_path_for_container(
    host_path: str,
    data_dir: str,
    *,
    mount_path: str = "/data",
) -> str:
    """Translate a host path into the tenant container mount namespace."""
    if not isinstance(host_path, str):
        raise TypeError("host_path must be a string")
    normalized_host_path = host_path.strip()
    if not normalized_host_path:
        raise ValueError("host_path must not be empty")
    if not isinstance(data_dir, str):
        raise TypeError("data_dir must be a string")
    normalized_data_dir = data_dir.strip()

```

```

if not normalized_data_dir:
    raise ValueError("data_dir must not be empty")
if not isinstance(mount_path, str):
    raise TypeError("mount_path must be a string")
normalized_mount_path = mount_path.strip()
if not normalized_mount_path:
    raise ValueError("mount_path must not be empty")

if not is_path_inside(normalized_host_path, normalized_data_dir):
    raise ValueError("Path outside user data directory")

resolved_host_path = os.path.realpath(normalized_host_path)
resolved_data_dir = os.path.realpath(normalized_data_dir)
if resolved_host_path == resolved_data_dir:
    return normalized_mount_path

relative_path = os.path.relpath(resolved_host_path, resolved_data_dir)
return os.path.join(normalized_mount_path, relative_path)

def _resolve_agent_dir_for_runtime(
    agent_dir: str | None,
    data_dir: str,
    *,
    container_enabled: bool,
) -> str | None:
    """Return the runtime-visible agent dir for the current execution mode."""
    if agent_dir is None:
        return None
    if not isinstance(agent_dir, str):
        raise TypeError("agent_dir must be a string or None")
    normalized_agent_dir = agent_dir.strip()
    if not normalized_agent_dir:
        raise ValueError("agent_dir must not be empty when provided")

    if not container_enabled:
        return normalized_agent_dir
    if not is_path_inside(normalized_agent_dir, data_dir):
        return normalized_agent_dir
    return remap_path_for_container(normalized_agent_dir, data_dir)

def _runtime_safe_id(value: str | None, name: str) -> str | None:
    """Return a stable 32-character hex id for one runtime-owned path segment."""
    if value is None:
        return None

```

```

    if not isinstance(value, str):
        raise TypeError(f"{name} must be a string or None")
    normalized_value = value.strip()
    if not normalized_value:
        raise ValueError(f"{name} must not be empty when provided")
    if _RUNTIME_ID_RE.match(normalized_value):
        return normalized_value
    return hashlib.sha256(normalized_value.encode("utf-8")).hexdigest()[:32]

def _workspace_runtime_id(workspace_id: str | None) -> str | None:
    """Return stable container id for one workspace, or None for legacy runtime."""
    return _runtime_safe_id(workspace_id, "workspace_id")

def _pi_session_file_name(session_id: str) -> str:
    """Return path-safe Pi session file name for one Yinshi session id."""
    normalized_session_id = _runtime_safe_id(session_id, "session_id")
    assert normalized_session_id is not None, "session_id must be normalized"
    return f"{normalized_session_id}.jsonl"

def _workspace_pi_session_directory(home_path: Path) -> Path:
    """Return the Pi session directory below one workspace home."""
    if not home_path.is_absolute():
        raise ValueError("home_path must be absolute")
    return home_path.joinpath(*_PI_SESSION_PATH_PARTS)

def _workspace_pi_session_host_directory(
    tenant: TenantContext,
    workspace_id: str,
    *,
    create: bool,
) -> Path:
    """Return host directory for workspace-scoped durable Pi session files."""
    if tenant is None:
        raise ValueError("tenant is required")
    if not tenant.data_dir:
        raise ValueError("tenant data_dir must not be empty")
    if create:
        home_path = Path(_workspace_home_source(tenant, workspace_id))
    else:
        home_path = _workspace_home_expected_path(tenant, workspace_id)
    session_directory = _workspace_pi_session_directory(home_path)
    if create:

```

```

        yinshi_directory = home_path / ".yinshi"
        if yinshi_directory.is_symlink() or session_directory.is_symlink():
            raise ValueError("workspace Pi session path must not contain symlinks")
        session_directory.mkdir(mode=0o700, parents=True, exist_ok=True)
    return session_directory

def _workspace_pi_session_host_file(
    tenant: TenantContext,
    workspace_id: str,
    session_id: str,
) -> str:
    """Return host path for a workspace-scoped durable Pi session file."""
    session_directory = _workspace_pi_session_host_directory(
        tenant,
        workspace_id,
        create=True,
    )
    return str(session_directory / _pi_session_file_name(session_id))

def _workspace_pi_session_runtime_file(
    tenant: TenantContext,
    workspace_id: str,
    session_id: str,
    *,
    container_enabled: bool,
    narrow_mounts: bool,
) -> str:
    """Return the path a sidecar process should use for Pi session persistence."""
    host_file = _workspace_pi_session_host_file(tenant, workspace_id, session_id)
    if not container_enabled:
        return host_file
    if narrow_mounts:
        return posixpath.join(
            _WORKSPACE_HOME_TARGET,
            _PI_SESSION_DIRECTORY,
            _pi_session_file_name(session_id),
        )
    return remap_path_for_container(host_file, tenant.data_dir)

def _local_pi_session_directory(*, create: bool) -> str:
    """Return the legacy no-tenant Pi session directory."""
    settings = get_settings()
    db_path = os.path.abspath(settings.db_path)

```

```

if not db_path:
    raise ValueError("settings.db_path must not be empty")
session_directory = os.path.join(os.path.dirname(db_path), "pi-sessions")
if create:
    os.makedirs(session_directory, mode=0o700, exist_ok=True)
return session_directory

def local_pi_session_file(session_id: str) -> str:
    """Return host path for durable Pi sessions in legacy no-tenant mode."""
    return os.path.join(
        _local_pi_session_directory(create=True),
        _pi_session_file_name(session_id),
    )

def delete_local_pi_session_file(session_id: str) -> None:
    """Delete one durable Pi session file in legacy no-tenant mode."""
    session_directory = _local_pi_session_directory(create=False)
    session_file = os.path.join(session_directory, _pi_session_file_name(session_id))
    if os.path.exists(session_file):
        os.unlink(session_file)

def _pi_session_directory_is_safe_for_delete(
    home_path: Path,
    session_directory: Path,
) -> bool:
    """Return whether one Pi session directory can be safely removed."""
    if not home_path.is_absolute():
        raise ValueError("home_path must be absolute")
    if not session_directory.is_absolute():
        raise ValueError("session_directory must be absolute")

    expected_session_directory = _workspace_pi_session_directory(home_path)
    if session_directory != expected_session_directory:
        raise ValueError("session_directory must belong to home_path")

    yinshi_directory = home_path / ".yinshi"
    for path in (home_path, yinshi_directory, session_directory):
        if path.is_symlink():
            logger.warning("Refusing to delete Pi session files through a symlink")
            return False

    try:
        resolved_session_directory = session_directory.resolve(strict=False)

```

```

except OSError:
    logger.warning("Refusing to delete Pi session files because path resolution failed")
    return False

if resolved_session_directory != expected_session_directory:
    logger.warning("Refusing to delete Pi session files outside workspace home")
    return False

return True

def delete_workspace_runtime_home(tenant: TenantContext, workspace_id: str) -> None:
    """Delete the complete persistent home for one workspace runtime."""
    if tenant is None:
        raise ValueError("tenant is required")
    if not workspace_id:
        raise ValueError("workspace_id must not be empty")
    home_path = _workspace_home_expected_path(tenant, workspace_id)
    runtime_directory = home_path.parent
    if not runtime_directory.exists():
        return
    if runtime_directory.is_symlink() or home_path.is_symlink():
        raise ValueError("workspace runtime path must not contain symlinks")
    if not shutil.rmtree.avoids_symlink_attacks:
        raise RuntimeError("safe descriptor-based directory deletion is unavailable")

    directory_flags = os.O_RDONLY | os.O_DIRECTORY | os.O_NOFOLLOW | os.O_CLOEXEC
    parent_fd = os.open(runtime_directory, directory_flags)
    try:
        try:
            home_stat = os.stat("home", dir_fd=parent_fd, follow_symlinks=False)
        except FileNotFoundError:
            return
        if not stat.S_ISDIR(home_stat.st_mode):
            raise ValueError("workspace home must be a directory")
        shutil.rmtree("home", dir_fd=parent_fd)
    finally:
        os.close(parent_fd)
    try:
        runtime_directory.rmdir()
    except OSError:
        logger.warning("Workspace runtime directory was not empty after cleanup")

def delete_workspace_pi_sessions(tenant: TenantContext | None, workspace_id: str) -> None:
    """Delete durable Pi session files for one workspace runtime."""

```

```

if tenant is None:
    return
if not workspace_id:
    raise ValueError("workspace_id must not be empty")
home_path = _workspace_home_expected_path(tenant, workspace_id)
session_directory = _workspace_pi_session_directory(home_path)
if not _pi_session_directory_is_safe_for_delete(home_path, session_directory):
    return
if session_directory.exists():
    shutil.rmtree(session_directory)

def _workspace_home_expected_path(tenant: TenantContext, workspace_id: str) -> Path:
    """Return the non-symlink workspace home path owned by Yinshi."""
    if not tenant.data_dir:
        raise ValueError("tenant data_dir must not be empty")
    normalized_workspace_id = _workspace_runtime_id(workspace_id)
    assert normalized_workspace_id is not None, "workspace_id must be normalized"
    data_dir = Path(tenant.data_dir).resolve(strict=False)
    return data_dir / "runtime" / "workspaces" / normalized_workspace_id / "home"

def _workspace_home_path(tenant: TenantContext, workspace_id: str) -> str:
    """Return the persistent host home path for one workspace runtime."""
    return str(_workspace_home_expected_path(tenant, workspace_id).resolve(strict=False))

def _workspace_home_source(tenant: TenantContext, workspace_id: str) -> str:
    """Return and create the persistent host home for one workspace runtime."""
    home_path = _workspace_home_expected_path(tenant, workspace_id)
    if home_path.is_symlink():
        raise ValueError("workspace home must not be a symlink")
    home_path.mkdir(mode=0o700, parents=True, exist_ok=True)
    for subdirectory in ("bin", ".local/bin", ".npm-global/bin"):
        (home_path / subdirectory).mkdir(mode=0o700, parents=True, exist_ok=True)
    return str(home_path)

def workspace_runtime_environment(workspace_id: str | None) -> dict[str, str] | None:
    """Return container environment variables for a workspace runtime."""
    normalized_workspace_id = _workspace_runtime_id(workspace_id)
    if normalized_workspace_id is None:
        return None
    return {
        "HOME": _WORKSPACE_HOME_TARGET,
        "PATH": _WORKSPACE_PATH,
    }

```



```

        "NPM_CONFIG_PREFIX": f"{_WORKSPACE_HOME_TARGET}/.npm-global",
        "PIPX_HOME": f"{_WORKSPACE_HOME_TARGET}/.local/pipx",
        "PIPX_BIN_DIR": f"{_WORKSPACE_HOME_TARGET}/.local/bin",
        "YINSHI_WORKSPACE_ID": normalized_workspace_id,
    }

def _append_runtime_mount(
    mounts: list[ContainerMount],
    mounts_by_target: dict[str, ContainerMount],
    *,
    source_path: str,
    target_path: str,
    read_only: bool,
) -> None:
    """Append one mount while preventing ambiguous target overlays."""
    if not os.path.isabs(source_path):
        raise ValueError("source_path must be absolute")
    if not os.path.isabs(target_path):
        raise ValueError("target_path must be absolute")

    mount = ContainerMount(
        source_path=source_path,
        target_path=target_path,
        read_only=read_only,
    )
    existing_mount = mounts_by_target.get(target_path)
    if existing_mount is not None:
        if existing_mount == mount:
            return
        raise ContainerStartError("Sidecar mount targets must be unique")

    mounts.append(mount)
    mounts_by_target[target_path] = mount

def _container_mounts_for_runtime(
    tenant: TenantContext,
    *,
    agent_dir: str | None,
    repo_root_path: str | None,
    workspace_path: str | None,
    workspace_id: str | None,
) -> tuple[ContainerMount, ...]:
    """Build the narrow mount set required by one sidecar operation."""
    mounts: list[ContainerMount] = []

```

```

mounts_by_target: dict[str, ContainerMount] = {}
normalized_workspace_id = _workspace_runtime_id(workspace_id)
if normalized_workspace_id is not None:
    _append_runtime_mount(
        mounts,
        mounts_by_target,
        source_path=_workspace_home_source(tenant, normalized_workspace_id),
        target_path=_WORKSPACE_HOME_TARGET,
        read_only=False,
    )

for source_path, read_only, mount_at_host_path in (
    (repo_root_path, False, True),
    (workspace_path, False, False),
    (agent_dir, True, False),
):
    if source_path is None:
        continue
    normalized_source_path = os.path.realpath(source_path)
    if not is_path_inside(normalized_source_path, tenant.data_dir):
        message = "Sidecar mount path is outside tenant data"
        raise ContainerStartError(message)
    _append_runtime_mount(
        mounts,
        mounts_by_target,
        source_path=normalized_source_path,
        target_path=remap_path_for_container(
            normalized_source_path,
            tenant.data_dir,
        ),
        read_only=read_only,
    )
    if mount_at_host_path:
        # Git worktrees store absolute gitdir pointers into the repo's
        # metadata. Mounting the repo at that same absolute path lets Git
        # resolve those pointers inside the sidecar while cwd stays in
        # /data.
        _append_runtime_mount(
            mounts,
            mounts_by_target,
            source_path=normalized_source_path,
            target_path=normalized_source_path,
            read_only=read_only,
        )
return tuple(mounts)

```

```

async def resolve_tenant_sidecar_context(
    request: Request,
    tenant: TenantContext | None,
    runtime_session_id: str | None = None,
    repo_agents_md: str | None = None,
    repo_root_path: str | None = None,
    workspace_path: str | None = None,
    workspace_id: str | None = None,
) -> TenantSidecarContext:
    """Resolve the socket path and Pi runtime inputs for one request."""
    if tenant is None:
        return TenantSidecarContext(
            socket_path=None,
            agent_dir=None,
            settings_payload=None,
        )

    settings = get_settings()
    runtime_inputs = resolve_effective_pi_runtime(
        tenant.user_id,
        tenant.data_dir,
        runtime_session_id=runtime_session_id,
        repo_agents_md=repo_agents_md,
    )
    narrow_mounts = settings.container_mount_mode == "narrow"
    runtime_id = _workspace_runtime_id(workspace_id)
    pi_session_file = None
    if runtime_session_id is not None and workspace_id is not None:
        pi_session_file = _workspace_pi_session_runtime_file(
            tenant,
            workspace_id,
            runtime_session_id,
            container_enabled=settings.container_enabled,
            narrow_mounts=narrow_mounts,
        )
    runtime_agent_dir = _resolve_agent_dir_for_runtime(
        runtime_inputs.agent_dir,
        tenant.data_dir,
        container_enabled=settings.container_enabled,
    )

    if not settings.container_enabled:
        return TenantSidecarContext(
            socket_path=None,
            agent_dir=runtime_agent_dir,

```

```

        settings_payload=runtime_inputs.settings_payload,
        runtime_id=runtime_id,
        pi_session_file=pi_session_file,
    )

    container_manager = getattr(request.app.state, "container_manager", None)
    if container_manager is None:
        raise ContainerStartError("Container manager is not initialized")

    container_mounts = None
    if narrow_mounts:
        container_mounts = _container_mounts_for_runtime(
            tenant,
            agent_dir=runtime_inputs.agent_dir,
            repo_root_path=repo_root_path,
            workspace_path=workspace_path,
            workspace_id=runtime_id,
        )
    container_info = await container_manager.ensure_container(
        tenant.user_id,
        tenant.data_dir,
        mounts=container_mounts,
        runtime_id=runtime_id,
        environment=workspace_runtime_environment(runtime_id),
    )
    return TenantSidecarContext(
        socket_path=container_info.socket_path,
        agent_dir=runtime_agent_dir,
        settings_payload=runtime_inputs.settings_payload,
        runtime_id=runtime_id,
        pi_session_file=pi_session_file,
    )

def _call_container_method(
    method: object,
    user_id: str,
    *args: object,
    runtime_id: str | None = None,
) -> None:
    """Call a container manager lifecycle method for one runtime."""
    if not callable(method):
        return
    method(user_id, *args, runtime_id=runtime_id)

```

```

@asynccontextmanager
async def tenant_container_activity(
    request: Request,
    tenant: TenantContext | None,
    *,
    runtime_id: str | None = None,
    protect_lease_key: str | None = None,
    protect_timeout_s: int | None = None,
) -> AsyncIterator[None]:
    """Mark a container busy, then optionally keep it alive after work ends."""
    if protect_lease_key is not None and protect_timeout_s is None:
        raise ValueError("protect_timeout_s is required when protect_lease_key is set")
    if protect_timeout_s is not None and protect_timeout_s < 0:
        raise ValueError("protect_timeout_s must not be negative")

    container_manager, user_id = _tenant_container_manager(request, tenant)
    reservation: ContainerActivityReservation | None = None
    if container_manager is not None and user_id is not None:
        reservation = await container_manager.acquire_activity(
            user_id,
            runtime_id=runtime_id,
        )
    if reservation is None:
        raise ContainerNotReadyError("Container runtime is no longer available")
    elif tenant is not None and get_settings().container_enabled:
        raise ContainerNotReadyError("Container manager is not initialized")

    try:
        yield
    finally:
        try:
            if protect_lease_key is not None:
                assert protect_timeout_s is not None, "protect timeout must be validated"
                protect_tenant_container(
                    request,
                    tenant,
                    lease_key=protect_lease_key,
                    timeout_s=protect_timeout_s,
                    runtime_id=runtime_id,
                )
        finally:
            if reservation is not None:
                assert container_manager is not None
                await container_manager.release_activity(reservation)

```

```

def touch_tenant_container(
    request: Request,
    tenant: TenantContext | None,
    *,
    runtime_id: str | None = None,
) -> None:
    """Mark one tenant container as recently used when container mode is active."""
    container_manager, user_id = _tenant_container_manager(request, tenant)
    if container_manager is None or user_id is None:
        return
    _call_container_method(
        getattr(container_manager, "touch", None), user_id, runtime_id=runtime_id
    )

def begin_tenant_container_activity(
    request: Request,
    tenant: TenantContext | None,
    *,
    runtime_id: str | None = None,
) -> None:
    """Mark one tenant container as busy for the duration of a request step."""
    container_manager, user_id = _tenant_container_manager(request, tenant)
    if container_manager is None or user_id is None:
        return
    _call_container_method(
        getattr(container_manager, "begin_activity", None),
        user_id,
        runtime_id=runtime_id,
    )

def end_tenant_container_activity(
    request: Request,
    tenant: TenantContext | None,
    *,
    runtime_id: str | None = None,
) -> None:
    """Release one in-flight request marker from a tenant container."""
    container_manager, user_id = _tenant_container_manager(request, tenant)
    if container_manager is None or user_id is None:
        return
    _call_container_method(
        getattr(container_manager, "end_activity", None),
        user_id,
        runtime_id=runtime_id,
    )

```

```

    )

def protect_tenant_container(
    request: Request,
    tenant: TenantContext | None,
    *,
    lease_key: str,
    timeout_s: int,
    runtime_id: str | None = None,
) -> None:
    """Keep one tenant container alive for a named long-lived operation."""
    container_manager, user_id = _tenant_container_manager(request, tenant)
    if container_manager is None or user_id is None:
        return
    _call_container_method(
        getattr(container_manager, "protect", None),
        user_id,
        lease_key,
        timeout_s,
        runtime_id=runtime_id,
    )

def release_tenant_container(
    request: Request,
    tenant: TenantContext | None,
    *,
    lease_key: str,
    runtime_id: str | None = None,
) -> None:
    """Remove one named long-lived keepalive lease from a tenant container."""
    container_manager, user_id = _tenant_container_manager(request, tenant)
    if container_manager is None or user_id is None:
        return
    _call_container_method(
        getattr(container_manager, "unprotect", None),
        user_id,
        lease_key,
        runtime_id=runtime_id,
    )

def _tenant_container_manager(
    request: Request,
    tenant: TenantContext | None,

```

```

) -> tuple[ContainerManager | None, str | None]:
    """Return the container manager plus tenant user id when container mode is active."""
    if tenant is None:
        return None, None

    settings = get_settings()
    if not settings.container_enabled:
        return None, None

    container_manager = cast(
        ContainerManager | None,
        getattr(request.app.state, "container_manager", None),
    )
    if container_manager is None:
        return None, None
    return container_manager, tenant.user_id

```

9 Workspace Files and Terminals

The workspace inspector uses two transport styles. File tree, status, preview, diff, edit, and download operations are HTTP requests with path validation. The terminal is a WebSocket proxy to a sidecar-owned PTY so browser input and shell output stay low-latency.

9.1 api/workspace_files.py

Workspace file routes expose safe tree, status, preview, diff, edit, and download operations for visible paths only. The root chunk below tangles back to `backend/src/yinshi/api/workspace_files.py`.

```

"""Workspace file tree, status, preview, diff, edit, and download endpoints."""

from __future__ import annotations

import errno
import logging
import os
import sqlite3
import stat
from collections.abc import Iterator
from typing import Any, BinaryIO, cast
from urllib.parse import quote

from fastapi import APIRouter, HTTPException, Query, Request
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field

```



```

from yinshi.api.deps import check_workspace_owner, get_db_for_request, get_tenant
from yinshi.exceptions import GitError, WorkspaceNotFoundError
from yinshi.services.workspace_files import (
    _open_workspace_parent,
    build_file_tree,
    changed_files,
    changed_files_to_dicts,
    diff_file,
    ensure_secret_guardrails,
    file_tree_to_dicts,
    read_text_file,
    write_text_file,
)
from yinshi.services.workspace_runtime_paths import prepare_tenant_workspace_runtime_paths

logger = logging.getLogger(__name__)
router = APIRouter()

_EXPECTED_FILE_ERRORS = (
    FileNotFoundError,
    PermissionError,
    TypeError,
    ValueError,
    GitError,
    WorkspaceNotFoundError,
)

class FileEditRequest(BaseModel):
    """Request body for browser-based workspace file edits."""

    content: str = Field(..., max_length=512 * 1024)

def _workspace_row(db: sqlite3.Connection, workspace_id: str, request: Request) -> sqlite3.Row:
    """Load one workspace and its repo paths after owner validation."""
    check_workspace_owner(db, workspace_id, request)
    row = db.execute(
        "SELECT w.id, w.path, r.root_path "
        "FROM workspaces w JOIN repos r ON w.repo_id = r.id WHERE w.id = ?",
        (workspace_id,),
    ).fetchone()
    if row is None:
        raise HTTPException(status_code=404, detail="Workspace not found")
    return cast(sqlite3.Row, row)

```

```

async def _prepare_workspace_files(
    db: sqlite3.Connection,
    workspace_id: str,
    request: Request,
) -> str:
    """Return a trusted workspace path, installing Git secret guardrails."""
    tenant = get_tenant(request)
    if tenant is not None:
        try:
            paths = await prepare_tenant_workspace_runtime_paths(db, tenant, workspace_id)
        except PermissionError:
            raise
        except OSError as exc:
            raise HTTPException(
                status_code=409,
                detail="Failed to prepare workspace paths",
            ) from exc
        return paths.workspace_path

    row = _workspace_row(db, workspace_id, request)
    workspace_path = str(row["path"])
    repo_root_path = str(row["root_path"])
    try:
        ensure_secret_guardrails(repo_root_path)
    except OSError as exc:
        raise HTTPException(status_code=409, detail="Failed to prepare secret guardrails")
    return workspace_path

def _map_file_error(exc: Exception) -> HTTPException:
    """Convert file service exceptions into stable HTTP responses."""
    if isinstance(exc, (FileNotFoundError, WorkspaceNotFoundError)):
        return HTTPException(status_code=404, detail=str(exc) or "File not found")
    if isinstance(exc, PermissionError):
        return HTTPException(status_code=403, detail=str(exc) or "File is not available")
    if isinstance(exc, (TypeError, ValueError)):
        return HTTPException(status_code=400, detail=str(exc) or "Invalid file request")
    if isinstance(exc, GitError):
        return HTTPException(status_code=409, detail=str(exc) or "Git command failed")
    return HTTPException(status_code=500, detail="Workspace file operation failed")

def _http_file_error(exc: Exception, workspace_id: str) -> HTTPException:
    """Return an HTTP error, logging unexpected workspace file failures."""

```

```

    if isinstance(exc, HTTPException):
        return exc
    if not isinstance(exc, _EXPECTED_FILE_ERRORS):
        logger.error("Unexpected workspace file operation failure")
    return _map_file_error(exc)

@router.get("/api/workspaces/{workspace_id}/files/tree")
async def get_workspace_file_tree(workspace_id: str, request: Request) -> dict[str, Any]:
    """Return a bounded visible nested file tree for one workspace."""
    try:
        with get_db_for_request(request) as db:
            workspace_path = await _prepare_workspace_files(db, workspace_id, request)
            nodes = build_file_tree(workspace_path)
    except Exception as exc:
        raise _http_file_error(exc, workspace_id) from exc
    return {"files": file_tree_to_dicts(nodes)}

@router.get("/api/workspaces/{workspace_id}/files/changed")
async def get_workspace_changed_files(workspace_id: str, request: Request) -> dict[str, Any]:
    """Return visible Git status changes for one workspace."""
    try:
        with get_db_for_request(request) as db:
            workspace_path = await _prepare_workspace_files(db, workspace_id, request)
            changes = await changed_files(workspace_path)
    except Exception as exc:
        raise _http_file_error(exc, workspace_id) from exc
    return {"files": changed_files_to_dicts(changes)}

@router.get("/api/workspaces/{workspace_id}/files/preview")
async def preview_workspace_file(
    workspace_id: str,
    request: Request,
    path: str = Query(..., min_length=1, max_length=4096),
) -> dict[str, str]:
    """Return text content for one visible workspace file."""
    try:
        with get_db_for_request(request) as db:
            workspace_path = await _prepare_workspace_files(db, workspace_id, request)
            return {"path": path, "content": read_text_file(workspace_path, path)}
    except Exception as exc:
        raise _http_file_error(exc, workspace_id) from exc

```

```

@router.get("/api/workspaces/{workspace_id}/files/diff")
async def diff_workspace_file(
    workspace_id: str,
    request: Request,
    path: str = Query(..., min_length=1, max_length=4096),
) -> dict[str, str]:
    """Return a Git diff for one visible workspace file."""
    try:
        with get_db_for_request(request) as db:
            workspace_path = await _prepare_workspace_files(db, workspace_id, request)
            return {"path": path, "diff": await diff_file(workspace_path, path)}
    except Exception as exc:
        raise _http_file_error(exc, workspace_id) from exc

@router.put("/api/workspaces/{workspace_id}/files/content")
async def edit_workspace_file(
    workspace_id: str,
    body: FileEditRequest,
    request: Request,
    path: str = Query(..., min_length=1, max_length=4096),
) -> dict[str, str]:
    """Replace one visible workspace text file from the browser editor."""
    try:
        with get_db_for_request(request) as db:
            workspace_path = await _prepare_workspace_files(db, workspace_id, request)
            write_text_file(workspace_path, path, body.content)
    except Exception as exc:
        raise _http_file_error(exc, workspace_id) from exc
    return {"path": path, "status": "saved"}

def _stream_open_file(file_handle: BinaryIO) -> Iterator[bytes]:
    """Yield fixed-size chunks and close the pre-opened file on completion."""
    if file_handle.closed:
        raise ValueError("file_handle must be open")
    try:
        while True:
            chunk = file_handle.read(64 * 1024)
            if not chunk:
                break
            yield chunk
    finally:
        file_handle.close()

```

```

@router.get("/api/workspaces/{workspace_id}/files/download")
async def download_workspace_file(
    workspace_id: str,
    request: Request,
    path: str = Query(..., min_length=1, max_length=4096),
) -> StreamingResponse:
    """Download one visible workspace file through a stable descriptor."""
    file_handle: BinaryIO | None = None
    try:
        with get_db_for_request(request) as db:
            workspace_path = await _prepare_workspace_files(db, workspace_id, request)
            file_flags = os.O_RDONLY | os.O_NOFOLLOW | os.O_CLOEXEC
            with _open_workspace_parent(workspace_path, path) as (parent_fd, file_name):
                try:
                    file_descriptor = os.open(file_name, file_flags, dir_fd=parent_fd)
                except OSError as exc:
                    if exc.errno in {errno.ELOOP, errno.ENOTDIR}:
                        raise PermissionError("path contains a symlink") from exc
                    raise
            file_stat = os.fstat(file_descriptor)
            if not stat.S_ISREG(file_stat.st_mode):
                os.close(file_descriptor)
                raise FileNotFoundError("file does not exist")
            file_handle = os.fdopen(file_descriptor, "rb", closefd=True)
    except Exception as exc:
        if file_handle is not None:
            file_handle.close()
        raise _http_file_error(exc, workspace_id) from exc

    encoded_name = quote(file_name, safe="")
    return StreamingResponse(
        _stream_open_file(file_handle),
        media_type="application/octet-stream",
        headers={
            "Content-Disposition": f"attachment; filename*=UTF-8''{encoded_name}",
            "Content-Length": str(file_stat.st_size),
        },
    )

```

9.2 services/workspace_files.py

Workspace file services apply ignore rules, secret-path filters, text-size limits, git-status parsing, and edit validation. The root chunk below tangles back to backend/src/yinshi/services/workspace_files.py.

```

"""Workspace file tree, Git status, and safe file access helpers."""

from __future__ import annotations

import asyncio
import difflib
import errno
import os
import stat
from contextlib import contextmanager
from dataclasses import dataclass
from pathlib import Path
from typing import Iterator, Literal

from yinshi.exceptions import GitError

FileNodeType = Literal["file", "directory"]
ChangeKind = Literal["added", "copied", "deleted", "modified", "renamed", "untracked", "unkn

_EXCLUDED_DIRECTORY_NAMES = frozenset(
    {
        ".cache",
        ".git",
        ".hg",
        ".mypy_cache",
        ".next",
        ".parcel-cache",
        ".pytest_cache",
        ".ruff_cache",
        ".svn",
        ".tox",
        ".venv",
        "__pycache__",
        "build",
        "coverage",
        "dist",
        "htmlcov",
        "node_modules",
        "out",
        "target",
        "venv",
    }
)

_SECRET_FILE_NAMES = frozenset({".env"})
_SECRET_FILE_PREFIXES = (".env.",)
_MAX_TREE_ENTRIES = 5000

```

```

_MAX_TEXT_BYTES = 512 * 1024
_GUARDRAIL_MARKER = "# Yinshi secret guardrails"
_GUARDRAIL_PATTERNS = (".env", ".env.*")
_PRE_COMMIT_MARKER = "# Yinshi secret commit guard"
_PRE_PUSH_MARKER = "# Yinshi secret push guard"
_SECRET_PATH_GREP = "grep -E '(^|/)\\.env(\\.\\.*)?${' >/dev/null"
_PRE_COMMIT_GUARD = f""${_PRE_COMMIT_MARKER}
if git diff --cached --name-only --diff-filter=ACM | {_SECRET_PATH_GREP}; then
    echo 'Yinshi blocks committing .env files. Move secrets out of Git.' >&2
    exit 1
fi
"""

_PRE_PUSH_GUARD = f""${_PRE_PUSH_MARKER}
if git ls-files | {_SECRET_PATH_GREP}; then
    echo 'Yinshi blocks pushing tracked .env files. Move secrets out of Git.' >&2
    exit 1
fi
"""

@dataclass(frozen=True, slots=True)
class FileNode:
    """One visible file tree node."""

    name: str
    path: str
    type: FileNodeType
    children: tuple["FileNode", ...] = ()

@dataclass(frozen=True, slots=True)
class ChangedFile:
    """One visible changed file from Git status."""

    path: str
    status: str
    kind: ChangeKind
    original_path: str | None = None

def _workspace_root(workspace_path: str) -> Path:
    """Return a validated workspace root path."""
    if not isinstance(workspace_path, str):
        raise TypeError("workspace_path must be a string")
    normalized_workspace_path = workspace_path.strip()
    if not normalized_workspace_path:

```

```

        raise ValueError("workspace_path must not be empty")
    root = Path(normalized_workspace_path).resolve()
    if not root.is_dir():
        raise FileNotFoundError("workspace path does not exist")
    return root

def _repo_root(repo_root_path: str) -> Path | None:
    """Return a repository root path, or None when a mocked path is absent."""
    if not isinstance(repo_root_path, str):
        raise TypeError("repo_root_path must be a string")
    normalized_repo_root_path = repo_root_path.strip()
    if not normalized_repo_root_path:
        raise ValueError("repo_root_path must not be empty")
    root = Path(normalized_repo_root_path).resolve()
    if not root.is_dir():
        return None
    return root

def _is_secret_path(relative_path: str) -> bool:
    """Return whether a relative path points at a protected .env-style file."""
    parts = Path(relative_path).parts
    if not parts:
        return False
    for part in parts:
        if part in _SECRET_FILE_NAMES:
            return True
        if part.startswith(_SECRET_FILE_PREFIXES):
            return True
    return False

def _has_excluded_segment(relative_path: str) -> bool:
    """Return whether any path segment is intentionally hidden from the UI."""
    parts = Path(relative_path).parts
    return any(part in _EXCLUDED_DIRECTORY_NAMES for part in parts)

def _is_visible_relative_path(relative_path: str) -> bool:
    """Return whether a relative path is safe to show in workspace UI."""
    if not relative_path or relative_path == ".":
        return True
    if _is_secret_path(relative_path):
        return False
    return not _has_excluded_segment(relative_path)

```



```

def _visible_relative_path_parts(relative_path: str) -> tuple[str, ...]:
    """Return validated lexical components for one workspace-relative path."""
    if not isinstance(relative_path, str):
        raise TypeError("relative_path must be a string")
    normalized_relative_path = relative_path.strip()
    if not normalized_relative_path:
        raise ValueError("relative_path must not be empty")
    if os.path.isabs(normalized_relative_path):
        raise ValueError("relative_path must not be absolute")
    parts = Path(normalized_relative_path).parts
    if not parts or any(part in {"", ".", ".."} for part in parts):
        raise ValueError("path must stay inside workspace")
    display_path = Path(*parts).as_posix()
    if not _is_visible_relative_path(display_path):
        raise PermissionError("path is not available through the workspace UI")
    return parts

def validate_visible_relative_path(workspace_path: str, relative_path: str) -> Path:
    """Return one lexically valid workspace path without following child symlinks."""
    root = _workspace_root(workspace_path)
    parts = _visible_relative_path_parts(relative_path)
    return root.joinpath(*parts)

@contextmanager
def _open_workspace_parent(
    workspace_path: str,
    relative_path: str,
    *,
    create: bool = False,
) -> Iterator[tuple[int, str]]:
    """Open a stable parent directory descriptor beneath one workspace."""
    root = _workspace_root(workspace_path)
    parts = _visible_relative_path_parts(relative_path)
    directory_flags = os.O_RDONLY | os.O_DIRECTORY | os.O_NOFOLLOW | os.O_CLOEXEC
    current_fd = os.open(root, directory_flags)
    try:
        for part in parts[:-1]:
            if create:
                try:
                    os.mkdir(part, mode=0o700, dir_fd=current_fd)
                except FileExistsError:
                    pass

```

```

        try:
            next_fd = os.open(part, directory_flags, dir_fd=current_fd)
        except OSError as exc:
            if exc.errno in {errno.ELOOP, errno.ENOTDIR}:
                raise PermissionError("path contains a symlink") from exc
            raise
        os.close(current_fd)
        current_fd = next_fd
    yield current_fd, parts[-1]
finally:
    os.close(current_fd)

def _read_bounded_file_descriptor(file_descriptor: int) -> bytes:
    """Read one regular file up to the browser preview limit."""
    if file_descriptor < 0:
        raise ValueError("file_descriptor must be non-negative")
    file_stat = os.fstat(file_descriptor)
    if not stat.S_ISREG(file_stat.st_mode):
        raise FileNotFoundError("file does not exist")
    data = bytearray()
    while len(data) <= _MAX_TEXT_BYTES:
        chunk = os.read(file_descriptor, min(64 * 1024, _MAX_TEXT_BYTES + 1 - len(data)))
        if not chunk:
            break
        data.extend(chunk)
    return bytes(data)

def _node_to_dict(node: FileNode) -> dict[str, object]:
    """Serialize one file node for API responses."""
    return {
        "name": node.name,
        "path": node.path,
        "type": node.type,
        "children": [_node_to_dict(child) for child in node.children],
    }

def file_tree_to_dicts(nodes: tuple[FileNode, ...]) -> list[dict[str, object]]:
    """Serialize file tree nodes for API responses."""
    return [_node_to_dict(node) for node in nodes]

def build_file_tree(workspace_path: str) -> tuple[FileNode, ...]:
    """Build a bounded visible file tree through stable directory descriptors."""

```

```

root = _workspace_root(workspace_path)
directory_flags = os.O_RDONLY | os.O_DIRECTORY | os.O_NOFOLLOW | os.O_CLOEXEC
root_fd = os.open(root, directory_flags)
entry_count = 0

def build_directory(directory_fd: int, parent_path: str = "") -> tuple[FileNode, ...]:
    nonlocal entry_count
    classified_entries: list[tuple[str, str, FileNodeType]] = []
    for name in os.listdir(directory_fd):
        relative_path = f"{parent_path}/{name}" if parent_path else name
        if not _is_visible_relative_path(relative_path):
            continue
        try:
            child_stat = os.stat(name, dir_fd=directory_fd, follow_symlinks=False)
        except OSError:
            continue
        if stat.S_ISDIR(child_stat.st_mode):
            node_type: FileNodeType = "directory"
        elif stat.S_ISREG(child_stat.st_mode):
            node_type = "file"
        else:
            continue
        classified_entries.append((name, relative_path, node_type))

    classified_entries.sort(key=lambda entry: (entry[2] == "file", entry[0].lower()))
    children: list[FileNode] = []
    for name, relative_path, node_type in classified_entries:
        if entry_count >= _MAX_TREE_ENTRIES:
            break
        entry_count += 1
        if node_type == "file":
            children.append(FileNode(name=name, path=relative_path, type="file"))
            continue
        try:
            child_fd = os.open(name, directory_flags, dir_fd=directory_fd)
        except OSError as exc:
            if exc.errno in {errno.ELOOP, errno.ENOENT, errno.ENOTDIR}:
                continue
            raise
        try:
            child_nodes = build_directory(child_fd, relative_path)
        finally:
            os.close(child_fd)
        children.append(
            FileNode(
                name=name,

```

```

        path=relative_path,
        type="directory",
        children=child_nodes,
    )
    )
    return tuple(children)

try:
    return build_directory(root_fd)
finally:
    os.close(root_fd)

def _change_kind(status: str) -> ChangeKind:
    """Map Git porcelain status text to a UI change kind."""
    if "?" in status:
        return "untracked"
    if "R" in status:
        return "renamed"
    if "C" in status:
        return "copied"
    if "D" in status:
        return "deleted"
    if "A" in status:
        return "added"
    if "M" in status or "T" in status:
        return "modified"
    return "unknown"

def _parse_porcelain_z(output: bytes) -> tuple[ChangedFile, ...]:
    """Parse null-delimited Git porcelain v1 status output."""
    records = [
        record for record in output.decode("utf-8", errors="surrogateescape").split("\0") if record
    ]
    changes: list[ChangedFile] = []
    index = 0
    while index < len(records):
        record = records[index]
        if len(record) < 4:
            index += 1
            continue
        status = record[:2]
        path_text = record[3:]
        original_path = None
        index += 1

```

```

        if ("R" in status or "C" in status) and index < len(records):
            original_path = records[index]
            index += 1
        if _is_visible_relative_path(path_text):
            changes.append(
                ChangedFile(
                    path=path_text,
                    status=status,
                    kind=_change_kind(status),
                    original_path=original_path,
                )
            )
    return tuple(changes)

async def changed_files(workspace_path: str) -> tuple[ChangedFile, ...]:
    """Return visible changed files from Git status for one workspace."""
    root = _workspace_root(workspace_path)
    process = await asyncio.create_subprocess_exec(
        "/usr/bin/git",
        "-C",
        str(root),
        "status",
        "--porcelain=v1",
        "-z",
        "--untracked-files=all",
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    stdout, stderr = await process.communicate()
    if process.returncode != 0:
        raise GitError(stderr.decode("utf-8", errors="replace") or "git status failed")
    return _parse_porcelain_z(stdout)

def changed_files_to_dicts(changes: tuple[ChangedFile, ...]) -> list[dict[str, object]]:
    """Serialize changed files for API responses."""
    return [
        {
            "path": change.path,
            "status": change.status,
            "kind": change.kind,
            "original_path": change.original_path,
        }
        for change in changes
    ]

```

```

def read_text_file(workspace_path: str, relative_path: str) -> str:
    """Read a bounded text file through symlink-resistant descriptors."""
    file_flags = os.O_RDONLY | os.O_NOFOLLOW | os.O_CLOEXEC
    with _open_workspace_parent(workspace_path, relative_path) as (parent_fd, file_name):
        try:
            file_descriptor = os.open(file_name, file_flags, dir_fd=parent_fd)
        except OSError as exc:
            if exc.errno in {errno.ELOOP, errno.ENOTDIR}:
                raise PermissionError("path contains a symlink") from exc
            raise
        try:
            data = _read_bounded_file_descriptor(file_descriptor)
        finally:
            os.close(file_descriptor)
    if len(data) > _MAX_TEXT_BYTES:
        raise ValueError("file is too large to preview")
    if b"\x00" in data:
        raise ValueError("binary files cannot be previewed")
    return data.decode("utf-8", errors="replace")

def write_text_file(workspace_path: str, relative_path: str, content: str) -> None:
    """Atomically replace one text file through a stable parent descriptor."""
    if not isinstance(content, str):
        raise TypeError("content must be a string")
    encoded_content = content.encode("utf-8")
    if len(encoded_content) > _MAX_TEXT_BYTES:
        raise ValueError("file is too large to edit through the browser")

    with _open_workspace_parent(
        workspace_path,
        relative_path,
        create=True,
    ) as (parent_fd, file_name):
        temporary_name = f"{file_name}.yinshi-tmp-{os.getpid()}"
        file_flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL | os.O_NOFOLLOW | os.O_CLOEXEC
        file_descriptor = os.open(temporary_name, file_flags, 0o600, dir_fd=parent_fd)
        try:
            remaining_content = memoryview(encoded_content)
            while remaining_content:
                bytes_written = os.write(file_descriptor, remaining_content)
                if bytes_written <= 0:
                    raise OSError("failed to write workspace file")
                remaining_content = remaining_content[bytes_written:]

```

```

        os.fsync(file_descriptor)
    finally:
        os.close(file_descriptor)
    try:
        os.replace(temporary_name, file_name, src_dir_fd=parent_fd, dst_dir_fd=parent_fd)
        os.fsync(parent_fd)
    finally:
        try:
            os.unlink(temporary_name, dir_fd=parent_fd)
        except FileNotFoundError:
            pass

async def _changed_file_for_path(root: Path, display_path: str) -> ChangedFile | None:
    """Return Git status for one path without scanning the whole worktree."""
    process = await asyncio.create_subprocess_exec(
        "/usr/bin/git",
        "-C",
        str(root),
        "status",
        "--porcelain=v1",
        "-z",
        "--untracked-files=all",
        "--",
        display_path,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    stdout, stderr = await process.communicate()
    if process.returncode != 0:
        raise GitError(stderr.decode("utf-8", errors="replace") or "git status failed")
    return next(iter(_parse_porcelain_z(stdout)), None)

async def _head_file_text(root: Path, display_path: str) -> str | None:
    """Read one committed file from Git's object database without touching the worktree."""
    process = await asyncio.create_subprocess_exec(
        "/usr/bin/git",
        "-C",
        str(root),
        "show",
        f"HEAD:{display_path}",
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    stdout, _ = await process.communicate()

```

```

if process.returncode != 0:
    return None
if len(stdout) > _MAX_TEXT_BYTES:
    raise ValueError("file is too large to preview")
if b"\x00" in stdout:
    raise ValueError("binary files cannot be previewed")
return stdout.decode("utf-8", errors="replace")

async def diff_file(workspace_path: str, relative_path: str) -> str:
    """Return a text diff using stable worktree reads and Git object data."""
    root = _workspace_root(workspace_path)
    file_path = validate_visible_relative_path(workspace_path, relative_path)
    display_path = file_path.relative_to(root).as_posix()
    matching_change = await _changed_file_for_path(root, display_path)
    if matching_change is not None and matching_change.kind == "deleted":
        current_content = ""
    else:
        current_content = read_text_file(workspace_path, display_path)

    committed_content = await _head_file_text(root, display_path)
    if committed_content is None:
        if matching_change is None:
            raise GitError("file does not exist in Git HEAD")
        committed_content = ""

    diff_lines = difflib.unified_diff(
        committed_content.splitlines(),
        current_content.splitlines(),
        fromfile=f"a/{display_path}",
        tofile=f"b/{display_path}",
        lineterm="",
    )
    return "\n".join(diff_lines)

def _install_secret_hook_guard(hook_path: Path, marker: str, guard_script: str) -> None:
    """Install a secret guard before any existing hook body can exit."""
    existing_hook = hook_path.read_text(encoding="utf-8") if hook_path.exists() else "#!/bin/sh"
    if marker not in existing_hook:
        if existing_hook.startswith("#!"):
            shebang, separator, remainder = existing_hook.partition("\n")
            existing_body = remainder if separator else ""
            updated_hook = shebang + "\n" + guard_script + existing_body
        else:
            updated_hook = "#!/bin/sh\n" + guard_script + existing_hook

```



```

        hook_path.write_text(updated_hook, encoding="utf-8")
        hook_path.chmod(hook_path.stat().st_mode | stat.S_IXUSR)

def ensure_secret_guardrails(repo_root_path: str) -> None:
    """Install repo-local guardrails that keep .env files out of normal Git flow."""
    root = _repo_root(repo_root_path)
    if root is None:
        return
    git_dir = root / ".git"
    if not git_dir.is_dir():
        return
    info_dir = git_dir / "info"
    info_dir.mkdir(parents=True, exist_ok=True)
    exclude_path = info_dir / "exclude"
    existing_exclude = exclude_path.read_text(encoding="utf-8") if exclude_path.exists() else ""
    if _GUARDRAIL_MARKER not in existing_exclude:
        suffix = "" if existing_exclude.endswith("\n") or not existing_exclude else "\n"
        exclude_path.write_text(
            existing_exclude
            + suffix
            + _GUARDRAIL_MARKER
            + "\n"
            + "\n".join(_GUARDRAIL_PATTERNS)
            + "\n",
            encoding="utf-8",
        )

    hooks_dir = git_dir / "hooks"
    hooks_dir.mkdir(parents=True, exist_ok=True)
    _install_secret_hook_guard(hooks_dir / "pre-commit", _PRE_COMMIT_MARKER, _PRE_COMMIT_GUARD)
    _install_secret_hook_guard(hooks_dir / "pre-push", _PRE_PUSH_MARKER, _PRE_PUSH_GUARD)

```

9.3 api/terminals.py

The terminal WebSocket authenticates the browser, checks Origin, resolves the workspace runtime, and proxies PTY messages to the sidecar. The root chunk below tangles back to backend/src/yinshi/api/terminals.py.

```

"""Authenticated browser terminal WebSocket endpoint."""

from __future__ import annotations

import asyncio
import json
import logging

```

```

import os
import stat
from typing import Any, cast

from fastapi import APIRouter, WebSocket, WebSocketDisconnect

from yinshi.auth import auth_disabled, get_session_identity, resolve_tenant_from_session_token
from yinshi.config import get_settings
from yinshi.exceptions import (
    ContainerNotReadyError,
    ContainerStartError,
    GitError,
    WorkspaceNotFoundError,
)
from yinshi.services.desktop_terminal import (
    DesktopTerminalContext,
    resolve_desktop_terminal_context,
)
from yinshi.services.live_auth_sessions import (
    LiveAuthSessionRegistration,
    register_live_auth_session,
)
from yinshi.services.sidecar_runtime import (
    remap_path_for_container,
    resolve_tenant_sidecar_context,
    tenant_container_activity,
)
from yinshi.services.workspace_runtime_paths import prepare_tenant_workspace_runtime_paths
from yinshi.tenant import TenantContext, get_user_db

logger = logging.getLogger(__name__)
router = APIRouter()

_TERMINAL_CLOSE_POLICY = 1008
_TERMINAL_CLOSE_UNAVAILABLE = 1011

def _allowed_origins() -> set[str]:
    """Return browser origins allowed to open terminal WebSockets."""
    settings = get_settings()
    origins = {settings.frontend_url.rstrip("/")}
    if settings.debug:
        origins.add("http://localhost:5173")
        origins.add("http://127.0.0.1:5173")
    return origins

```

```

def _origin_allowed(origin: str | None) -> bool:
    """Return whether a WebSocket Origin header is acceptable."""
    if origin is None:
        return False
    return origin.rstrip("/") in _allowed_origins()

def _tenant_from_websocket(websocket: WebSocket) -> TenantContext | None:
    """Resolve the authenticated tenant from a WebSocket cookie."""
    if auth_disabled():
        return None
    token = websocket.cookies.get("yinshi_session")
    if not token:
        return None
    return resolve_tenant_from_session_token(token)

async def _send_sidecar(
    writer: asyncio.StreamWriter,
    message: dict[str, Any],
) -> None:
    """Send one JSON line to the sidecar."""
    writer.write((json.dumps(message) + "\n").encode("utf-8"))
    await writer.drain()

async def _read_sidecar(reader: asyncio.StreamReader) -> dict[str, Any] | None:
    """Read one JSON line from the sidecar."""
    line = await reader.readline()
    if not line:
        return None
    message = json.loads(line.decode("utf-8"))
    if not isinstance(message, dict):
        raise ValueError("sidecar terminal message must be an object")
    return message

async def _proxy_browser_to_sidecar(
    websocket: WebSocket,
    writer: asyncio.StreamWriter,
    terminal_id: str,
    attach_options: dict[str, Any],
    session_token: str | None,
) -> None:
    """Forward input while the originating auth session remains active."""

```

```

while True:
    payload = await websocket.receive_json()
    if session_token is not None and get_session_identity(session_token) is None:
        await websocket.close(code=_TERMINAL_CLOSE_POLICY)
        return
    if not isinstance(payload, dict):
        continue
    message_type = payload.get("type")
    if message_type == "input":
        data = payload.get("data", "")
        if isinstance(data, str):
            await _send_sidecar(
                writer,
                {"type": "terminal_input", "id": terminal_id, "data": data},
            )
    elif message_type == "resize":
        await _send_sidecar(
            writer,
            {
                "type": "terminal_resize",
                "id": terminal_id,
                "cols": payload.get("cols"),
                "rows": payload.get("rows"),
            },
        )
    elif message_type == "restart":
        await _send_sidecar(
            writer,
            {
                "type": "terminal_restart",
                "id": terminal_id,
                "options": attach_options,
            },
        )
    elif message_type == "kill":
        await _send_sidecar(writer, {"type": "terminal_kill", "id": terminal_id})
    elif message_type == "ping":
        await websocket.send_json({"type": "pong"})
    else:
        await websocket.send_json({"type": "error", "error": "Unknown terminal message"})

async def _monitor_terminal_session(
    websocket: WebSocket,
    session_token: str,
    revocation_event: asyncio.Event,

```

```

        connection_lifetime_s_max: int,
    ) -> None:
        """Close a revoked or expired terminal even while both proxies are idle."""
        if not session_token:
            raise ValueError("session_token must not be empty")
        if not isinstance(revocation_event, asyncio.Event):
            raise TypeError("revocation_event must be an asyncio.Event")
        if connection_lifetime_s_max <= 0:
            raise ValueError("connection_lifetime_s_max must be positive")
        loop = asyncio.get_running_loop()
        deadline = loop.time() + connection_lifetime_s_max
        while True:
            remaining_s = deadline - loop.time()
            session_revoked = revocation_event.is_set() or get_session_identity(session_token) is None
            if remaining_s <= 0 or session_revoked:
                await websocket.close(code=_TERMINAL_CLOSE_POLICY)
                return
            try:
                await asyncio.wait_for(revocation_event.wait(), timeout=min(1.0, remaining_s))
            except TimeoutError:
                continue

async def _proxy_sidecar_to_browser(
    websocket: WebSocket,
    reader: asyncio.StreamReader,
    session_token: str | None,
) -> None:
    """Forward terminal events while the originating session remains active."""
    while True:
        message = await _read_sidecar(reader)
        if session_token is not None and get_session_identity(session_token) is None:
            await websocket.close(code=_TERMINAL_CLOSE_POLICY)
            return
        if message is None:
            await websocket.send_json({"type": "error", "error": "Terminal runtime disconnected"})
            return
        if message.get("type") == "init_status":
            continue
        await websocket.send_json(message)

async def _run_desktop_terminal_proxy(
    websocket: WebSocket,
    context: DesktopTerminalContext,
    workspace_id: str,

```

```

) -> None:
    """Attach a bootstrapped desktop renderer directly to its local sidecar."""
    if not isinstance(context, DesktopTerminalContext):
        raise TypeError("context must be DesktopTerminalContext")
    if not workspace_id:
        raise ValueError("workspace_id must not be empty")
    try:
        socket_information = os.stat(context.socket_path, follow_symlinks=False)
    except OSError:
        await websocket.close(code=_TERMINAL_CLOSE_UNAVAILABLE)
        return
    if not stat.S_ISSOCK(socket_information.st_mode) or socket_information.st_mode & 0o077:
        await websocket.close(code=_TERMINAL_CLOSE_POLICY)
        return
    if hasattr(os, "getuid") and socket_information.st_uid != os.getuid():
        await websocket.close(code=_TERMINAL_CLOSE_POLICY)
        return

    settings = get_settings()
    await websocket.accept()
    reader: asyncio.StreamReader | None = None
    writer: asyncio.StreamWriter | None = None
    try:
        reader, writer = await asyncio.open_unix_connection(context.socket_path)
        init_message = await _read_sidecar(reader)
        if init_message is not None and init_message.get("type") != "init_status":
            await websocket.send_json(init_message)
        attach_options = {
            "workspaceId": workspace_id,
            "cwd": context.workspace_path,
            "cols": 100,
            "rows": 30,
            "scrollbackLines": settings.terminal_scrollback_lines,
        }
        await _send_sidecar(
            writer,
            {"type": "terminal_attach", "id": workspace_id, "options": attach_options},
        )
        browser_task = asyncio.create_task(
            _proxy_browser_to_sidecar(
                websocket,
                writer,
                workspace_id,
                attach_options,
                None,
            )
        )

```

```

    )
    sidecar_task = asyncio.create_task(_proxy_sidecar_to_browser(websocket, reader, None))
    done, pending = await asyncio.wait(
        {browser_task, sidecar_task},
        return_when=asyncio.FIRST_COMPLETED,
    )
    for task in pending:
        task.cancel()
    await asyncio.gather(*pending, return_exceptions=True)
    for task in done:
        task.result()
except WebSocketDisconnect:
    logger.info("Desktop terminal detached")
except (ConnectionError, OSError, ValueError, json.JSONDecodeError):
    logger.error("Desktop terminal proxy failed")
    try:
        await websocket.send_json({"type": "error", "error": "Terminal proxy failed"})
    except RuntimeError:
        pass
finally:
    if writer is not None:
        try:
            await _send_sidecar(
                writer,
                {"type": "terminal_detach", "id": workspace_id},
            )
        except (ConnectionError, OSError):
            pass
        writer.close()
        try:
            await writer.wait_closed()
        except OSError:
            pass

@router.websocket("/api/workspaces/{workspace_id}/terminal")
async def workspace_terminal(websocket: WebSocket, workspace_id: str) -> None:
    """Attach the browser to the persistent terminal for one workspace runtime."""
    if not _origin_allowed(websocket.headers.get("origin")):
        await websocket.close(code=_TERMINAL_CLOSE_POLICY)
        return

    if getattr(websocket.app.state, "mode", None) == "desktop":
        try:
            desktop_context = resolve_desktop_terminal_context(workspace_id)
        except (LookupError, PermissionError, TypeError, ValueError):

```

```

        await websocket.close(code=_TERMINAL_CLOSE_POLICY)
        return
    await _run_desktop_terminal_proxy(websocket, desktop_context, workspace_id)
    return

session_token = websocket.cookies.get("yinshi_session")
tenant = _tenant_from_websocket(websocket)
if tenant is None or not session_token:
    await websocket.close(code=_TERMINAL_CLOSE_POLICY)
    return

settings = get_settings()
if not settings.container_enabled:
    await websocket.close(code=_TERMINAL_CLOSE_UNAVAILABLE)
    return

try:
    with get_user_db(tenant) as db:
        paths = await prepare_tenant_workspace_runtime_paths(db, tenant, workspace_id)
except (PermissionError, WorkspaceNotFoundError, TypeError, ValueError):
    await websocket.close(code=_TERMINAL_CLOSE_POLICY)
    return
except (GitError, OSError):
    logger.error("Failed to prepare terminal workspace")
    await websocket.close(code=_TERMINAL_CLOSE_UNAVAILABLE)
    return

workspace_path = paths.workspace_path
repo_root_path = paths.repo_root_path

try:
    runtime = await resolve_tenant_sidecar_context(
        cast(Any, websocket),
        tenant,
        repo_agents_md=paths.agents_md,
        repo_root_path=repo_root_path,
        workspace_path=workspace_path,
        workspace_id=workspace_id,
    )
except (ContainerStartError, ContainerNotReadyError):
    logger.error("Failed to start terminal runtime")
    await websocket.close(code=_TERMINAL_CLOSE_UNAVAILABLE)
    return

if runtime.socket_path is None:
    await websocket.close(code=_TERMINAL_CLOSE_UNAVAILABLE)

```



```

        return

    try:
        effective_cwd = remap_path_for_container(workspace_path, tenant.data_dir)
    except ValueError:
        await websocket.close(code=_TERMINAL_CLOSE_POLICY)
        return

    terminal_id = runtime.runtime_id or workspace_id
    await websocket.accept()
    logger.info("Terminal attached")

    reader: asyncio.StreamReader | None = None
    writer: asyncio.StreamWriter | None = None
    registration: LiveAuthSessionRegistration | None = None
    try:
        async with tenant_container_activity(
            cast(Any, websocket),
            tenant,
            runtime_id=runtime.runtime_id,
            protect_lease_key=f"terminal:{workspace_id}",
            protect_timeout_s=settings.terminal_keepalive_s,
        ):
            reader, writer = await asyncio.open_unix_connection(runtime.socket_path)
            init_message = await _read_sidecar(reader)
            if init_message is not None and init_message.get("type") != "init_status":
                await websocket.send_json(init_message)

            identity = get_session_identity(session_token)
            if identity is None:
                await websocket.close(code=_TERMINAL_CLOSE_POLICY)
                return
            user_id, auth_session_id = identity
            registration = register_live_auth_session(
                user_id=user_id,
                auth_session_id=auth_session_id,
            )
            attach_options = {
                "workspaceId": terminal_id,
                "cwd": effective_cwd,
                "cols": 100,
                "rows": 30,
                "scrollbackLines": settings.terminal_scrollback_lines,
            }
            await _send_sidecar(
                writer,

```

```

        {"type": "terminal_attach", "id": terminal_id, "options": attach_options},
    )
    browser_task = asyncio.create_task(
        _proxy_browser_to_sidecar(
            websocket,
            writer,
            terminal_id,
            attach_options,
            session_token,
        )
    )
    sidecar_task = asyncio.create_task(
        _proxy_sidecar_to_browser(websocket, reader, session_token)
    )
    session_task = asyncio.create_task(
        _monitor_terminal_session(
            websocket,
            session_token,
            registration.event,
            settings.terminal_keepalive_s,
        )
    )
    done, pending = await asyncio.wait(
        {browser_task, sidecar_task, session_task},
        return_when=asyncio.FIRST_COMPLETED,
    )
    for task in pending:
        task.cancel()
    for task in done:
        task.result()
except WebSocketDisconnect:
    logger.info("Terminal detached")
except (ConnectionError, OSError, ValueError, json.JSONDecodeError):
    logger.error("Terminal proxy failed")
    try:
        await websocket.send_json({"type": "error", "error": "Terminal proxy failed"})
    except RuntimeError:
        pass
finally:
    if registration is not None:
        registration.close()
    if writer is not None:
        try:
            await _send_sidecar(writer, {"type": "terminal_detach", "id": terminal_id})
        except (ConnectionError, OSError):
            pass

```

```

writer.close()
try:
    await writer.wait_closed()
except OSError:
    pass

```

10 Encryption & Key Management

Yinshi is not zero-knowledge hosting: the server sees plaintext while it runs a user session. The storage model still limits damage at rest by wrapping per-user data keys, encrypting provider credentials, and optionally encrypting sensitive control-plane fields.

10.1 services/crypto.py

Cryptographic helpers wrap user data keys and encrypt provider secrets with AES-256-GCM and derived subkeys. The root chunk below tangles back to `backend/src/yinshi/services/crypto.py`.

```

"""Encryption services for per-user data encryption.

Uses HKDF for key derivation and AES-256-GCM for wrapping/unwrapping
Data Encryption Keys (DEKs), encrypting API keys, and encrypting small
control-plane fields under server-managed keys.
"""

from __future__ import annotations

import base64
import json
import os
from typing import Final

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

_DEK_ENVELOPE_PREFIX: Final[bytes] = b"yinshi-dek-v1:"
_TEXT_ENVELOPE_PREFIX: Final[str] = "enc:v1:"

def generate_dek() -> bytes:
    """Generate a random 256-bit Data Encryption Key."""
    return os.urandom(32)

```

```

def _require_bytes(value: bytes, name: str, expected_length: int | None = None) -> None:
    """Validate cryptographic byte strings before using them as keys."""
    if not isinstance(value, bytes):
        raise TypeError(f"{name} must be bytes")
    if expected_length is not None:
        if len(value) != expected_length:
            raise ValueError(f"{name} must be exactly {expected_length} bytes")
    else:
        if not value:
            raise ValueError(f"{name} must not be empty")

def _require_text(value: str, name: str) -> None:
    """Validate text used as envelope context or payload."""
    if not isinstance(value, str):
        raise TypeError(f"{name} must be a string")
    if not value:
        raise ValueError(f"{name} must not be empty")

def derive_subkey(master_key: bytes, *, purpose: str, context: str) -> bytes:
    """Derive a deterministic AES-256 subkey for one encryption purpose."""
    _require_bytes(master_key, "master_key")
    _require_text(purpose, "purpose")
    _require_text(context, "context")
    hkdf = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=f"yinshi:{purpose}".encode(),
        info=f"yinshi:{purpose}:{context}".encode(),
    )
    return hkdf.derive(master_key)

def _derive_kek(user_id: str, pepper: bytes) -> bytes:
    """Derive a legacy Key Encryption Key from user_id and server pepper."""
    _require_text(user_id, "user_id")
    _require_bytes(pepper, "pepper")
    hkdf = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=pepper,
        info=f"yinshi-kek-{user_id}".encode(),
    )
    return hkdf.derive(user_id.encode())

```

```

def wrap_dek(dek: bytes, user_id: str, pepper: bytes) -> bytes:
    """Wrap a DEK using the legacy user_id + pepper derivation scheme.

    Returns nonce (12 bytes) + ciphertext.
    """
    _require_bytes(dek, "dek", expected_length=32)
    kek = _derive_kek(user_id, pepper)
    aesgcm = AESGCM(kek)
    nonce = os.urandom(12)
    ciphertext = aesgcm.encrypt(nonce, dek, None)
    return nonce + ciphertext

def unwrap_dek(wrapped: bytes, user_id: str, pepper: bytes) -> bytes:
    """Unwrap a DEK using the legacy user_id + pepper derivation scheme."""
    _require_bytes(wrapped, "wrapped")
    if len(wrapped) <= 12:
        raise ValueError("wrapped DEK must include nonce and ciphertext")
    kek = _derive_kek(user_id, pepper)
    aesgcm = AESGCM(kek)
    nonce = wrapped[:12]
    ciphertext = wrapped[12:]
    return aesgcm.decrypt(nonce, ciphertext, None)

def _aad(*parts: str) -> bytes:
    """Build authenticated-data bytes from already validated context parts."""
    if not parts:
        raise ValueError("at least one AAD part is required")
    for part in parts:
        _require_text(part, "aad_part")
    return "\x1f".join(parts).encode()

def wrap_dek_with_kek(dek: bytes, user_id: str, key_id: str, kek: bytes) -> bytes:
    """Wrap a per-user DEK with a server-managed KEK and key id metadata."""
    _require_bytes(dek, "dek", expected_length=32)
    _require_text(user_id, "user_id")
    _require_text(key_id, "key_id")
    dek_wrapping_key = derive_subkey(kek, purpose="dek-wrap", context=key_id)
    nonce = os.urandom(12)
    ciphertext = AESGCM(dek_wrapping_key).encrypt(
        nonce,
        dek,
        _aad("dek", user_id, key_id),
    )

```

```

)
envelope = {
    "version": 1,
    "algorithm": "AES-256-GCM",
    "key_id": key_id,
    "nonce": base64.urlsafe_b64encode(nonce).decode(),
    "ciphertext": base64.urlsafe_b64encode(ciphertext).decode(),
}
return _DEK_ENVELOPE_PREFIX + json.dumps(envelope, sort_keys=True).encode()

def is_wrapped_dek_envelope(wrapped: bytes) -> bool:
    """Return whether a stored wrapped DEK uses the versioned envelope format."""
    if not isinstance(wrapped, bytes):
        return False
    return wrapped.startswith(_DEK_ENVELOPE_PREFIX)

def wrapped_dek_key_id(wrapped: bytes) -> str | None:
    """Return the key id from a wrapped DEK envelope, if present."""
    if not is_wrapped_dek_envelope(wrapped):
        return None
    envelope = _decode_dek_envelope(wrapped)
    key_id = envelope.get("key_id")
    if isinstance(key_id, str) and key_id:
        return key_id
    raise ValueError("wrapped DEK envelope is missing key_id")

def _decode_dek_envelope(wrapped: bytes) -> dict[str, object]:
    """Decode a versioned wrapped-DEK envelope from the control database."""
    _require_bytes(wrapped, "wrapped")
    if not wrapped.startswith(_DEK_ENVELOPE_PREFIX):
        raise ValueError("wrapped DEK is not a versioned envelope")
    payload = wrapped[len(_DEK_ENVELOPE_PREFIX) :]
    try:
        envelope = json.loads(payload.decode())
    except (UnicodeDecodeError, json.JSONDecodeError) as exc:
        raise ValueError("wrapped DEK envelope is invalid JSON") from exc
    if not isinstance(envelope, dict):
        raise ValueError("wrapped DEK envelope must be a JSON object")
    if envelope.get("version") != 1:
        raise ValueError("unsupported wrapped DEK envelope version")
    if envelope.get("algorithm") != "AES-256-GCM":
        raise ValueError("unsupported wrapped DEK envelope algorithm")
    return envelope

```

```

def unwrap_dek_with_keks(wrapped: bytes, user_id: str, keyring: dict[str, bytes]) -> bytes:
    """Unwrap a versioned DEK envelope using the matching key from a keyring."""
    _require_text(user_id, "user_id")
    if not keyring:
        raise ValueError("keyring must not be empty")
    envelope = _decode_dek_envelope(wrapped)
    key_id = envelope.get("key_id")
    if not isinstance(key_id, str) or not key_id:
        raise ValueError("wrapped DEK envelope is missing key_id")
    if key_id not in keyring:
        raise KeyError(f"No KEK configured for key id {key_id}")
    nonce_text = envelope.get("nonce")
    ciphertext_text = envelope.get("ciphertext")
    if not isinstance(nonce_text, str) or not isinstance(ciphertext_text, str):
        raise ValueError("wrapped DEK envelope is missing encrypted payload")
    dek_wrapping_key = derive_subkey(keyring[key_id], purpose="dek-wrap", context=key_id)
    nonce = base64.urlsafe_b64decode(nonce_text.encode())
    ciphertext = base64.urlsafe_b64decode(ciphertext_text.encode())
    return AESGCM(dek_wrapping_key).decrypt(
        nonce,
        ciphertext,
        _aad("dek", user_id, key_id),
    )

def encrypt_api_key(api_key: str, dek: bytes) -> bytes:
    """Encrypt an API key string using the user's DEK.

    Returns nonce (12 bytes) + ciphertext.
    """
    _require_text(api_key, "api_key")
    _require_bytes(dek, "dek", expected_length=32)
    aesgcm = AESGCM(dek)
    nonce = os.urandom(12)
    ciphertext = aesgcm.encrypt(nonce, api_key.encode(), None)
    return nonce + ciphertext

def decrypt_api_key(encrypted: bytes, dek: bytes) -> str:
    """Decrypt an API key using the user's DEK."""
    _require_bytes(encrypted, "encrypted")
    if len(encrypted) <= 12:
        raise ValueError("encrypted API key must include nonce and ciphertext")
    _require_bytes(dek, "dek", expected_length=32)

```

```

aesgcm = AESGCM(dek)
nonce = encrypted[:12]
ciphertext = encrypted[12:]
return aesgcm.decrypt(nonce, ciphertext, None).decode()

def encrypt_text(plaintext: str, key: bytes, *, aad: str) -> str:
    """Encrypt a small text field and return a portable envelope string."""
    _require_text(plaintext, "plaintext")
    _require_bytes(key, "key", expected_length=32)
    _require_text(aad, "aad")
    nonce = os.urandom(12)
    ciphertext = AESGCM(key).encrypt(nonce, plaintext.encode(), aad.encode())
    payload = base64.urlsafe_b64encode(nonce + ciphertext).decode()
    return f"({_TEXT_ENVELOPE_PREFIX}){payload}"

def decrypt_text(envelope: str, key: bytes, *, aad: str) -> str:
    """Decrypt an envelope string produced by encrypt_text."""
    _require_text(envelope, "envelope")
    _require_bytes(key, "key", expected_length=32)
    _require_text(aad, "aad")
    if not envelope.startswith(_TEXT_ENVELOPE_PREFIX):
        raise ValueError("text envelope has an unsupported prefix")
    payload = envelope[len(_TEXT_ENVELOPE_PREFIX) :]
    encrypted = base64.urlsafe_b64decode(payload.encode())
    if len(encrypted) <= 12:
        raise ValueError("text envelope must include nonce and ciphertext")
    nonce = encrypted[:12]
    ciphertext = encrypted[12:]
    return AESGCM(key).decrypt(nonce, ciphertext, aad.encode()).decode()

def is_encrypted_text(value: str) -> bool:
    """Return whether a text value is encrypted by encrypt_text."""
    if not isinstance(value, str):
        return False
    return value.startswith(_TEXT_ENVELOPE_PREFIX)

```

10.2 services/control_encryption.py

Control-plane encryption protects sensitive shared database fields when field encryption is enabled. The root chunk below tangles back to backend/src/yinshi/services/control_encryption.py.


```

"""Field-level encryption helpers for sensitive control-plane values."""

from __future__ import annotations

from cryptography.exceptions import InvalidTag

from yinshi.config import control_field_encryption_enabled, get_settings
from yinshi.exceptions import EncryptionNotConfiguredError
from yinshi.services.crypto import decrypt_text, derive_subkey, encrypt_text, is_encrypted_t

def _control_field_keys() -> tuple[bytes, ...]:
    """Derive current and previous AES keys for control-field rotation overlap."""
    settings = get_settings()
    master_key = settings.active_key_encryption_key_bytes
    if not master_key:
        raise EncryptionNotConfiguredError(
            "KEY_ENCRYPTION_KEY or ENCRYPTION_PEPPE is required for control-field encryption"
        )
    keys = [derive_subkey(master_key, purpose="control-field", context="v1")]
    for previous_key in settings.key_encryption_keyring_previous.values():
        derived_key = derive_subkey(previous_key, purpose="control-field", context="v1")
        if derived_key not in keys:
            keys.append(derived_key)
    return tuple(keys)

def _control_field_key() -> bytes:
    """Return the current AES key used for new control-field encryption."""
    return _control_field_keys()[0]

def _aad(field_name: str, user_id: str) -> str:
    """Build stable AAD so encrypted control fields cannot be copied between users."""
    if not isinstance(field_name, str):
        raise TypeError("field_name must be a string")
    normalized_field_name = field_name.strip()
    if not normalized_field_name:
        raise ValueError("field_name must not be empty")
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    normalized_user_id = user_id.strip()
    if not normalized_user_id:
        raise ValueError("user_id must not be empty")
    return f"{normalized_field_name}:{normalized_user_id}"

```

```

def encrypt_control_text(field_name: str, user_id: str, plaintext: str | None) -> str | None:
    """Encrypt a sensitive control-plane text field when policy requires it."""
    if plaintext is None:
        return None
    if not isinstance(plaintext, str):
        raise TypeError("plaintext must be a string or None")
    if is_encrypted_text(plaintext):
        return plaintext
    settings = get_settings()
    if not control_field_encryption_enabled(settings):
        return plaintext
    return encrypt_text(plaintext, _control_field_key(), aad=_aad(field_name, user_id))

def decrypt_control_text(field_name: str, user_id: str, stored_value: str | None) -> str | None:
    """Decrypt an encrypted control-plane text field and pass through plaintext legacy values"""
    if stored_value is None:
        return None
    if not isinstance(stored_value, str):
        raise TypeError("stored_value must be a string or None")
    if not is_encrypted_text(stored_value):
        return stored_value
    associated_data = _aad(field_name, user_id)
    for key in _control_field_keys():
        try:
            return decrypt_text(stored_value, key, aad=associated_data)
        except InvalidTag:
            continue
    raise InvalidTag("No configured control-field key could decrypt the value")

```

10.3 services/keys.py

Key services unwrap each user's data key, resolve provider credentials for prompts, estimate cost, and record usage. The root chunk below tangles back to backend/src/yinshi/services/keys.py.

```

"""BYOK key resolution, DEK wrapping, and usage logging."""

from __future__ import annotations

import logging
import uuid

from yinshi.config import get_settings
from yinshi.db import get_control_db

```

```

from yinshi.exceptions import EncryptionNotConfiguredError, KeyNotFoundError
from yinshi.services.crypto import (
    decrypt_api_key,
    generate_dek,
    is_wrapped_dek_envelope,
    unwrap_dek,
    unwrap_dek_with_keks,
    wrap_dek,
    wrap_dek_with_kek,
    wrapped_dek_key_id,
)

logger = logging.getLogger(__name__)

# MiniMax M2.5 Highspeed pricing (per 1M tokens, in cents)
_MINIMAX_COSTS = {
    "input": 30,  # $0.30/M
    "output": 120, # $1.20/M
    "cache_read": 3, # $0.03/M
    "cache_write": 3.75, # $0.0375/M
}

def _require_user_id(user_id: str) -> str:
    """Normalize user identifiers before using them in key derivation."""
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    normalized_user_id = user_id.strip()
    if not normalized_user_id:
        raise ValueError("user_id must not be empty")
    return normalized_user_id

def _wrap_user_dek(dek: bytes, user_id: str) -> bytes:
    """Wrap a user DEK with the strongest configured server key source."""
    settings = get_settings()
    normalized_user_id = _require_user_id(user_id)
    key_encryption_key = settings.key_encryption_key_bytes
    if key_encryption_key:
        return wrap_dek_with_kek(
            dek,
            normalized_user_id,
            settings.key_encryption_key_id,
            key_encryption_key,
        )
    pepper = settings.encryption_pepper_bytes

```

```

    if pepper:
        return wrap_dek(dek, normalized_user_id, pepper)
    raise EncryptionNotConfiguredError("KEY_ENCRYPTION_KEY or ENCRYPTION_PEPPER is required")

def _unwrap_user_dek(wrapped: bytes, user_id: str) -> bytes:
    """Unwrap a stored user DEK from either current or legacy storage."""
    settings = get_settings()
    normalized_user_id = _require_user_id(user_id)
    if is_wrapped_dek_envelope(wrapped):
        key_encryption_key = settings.key_encryption_key_bytes
        if not key_encryption_key:
            raise EncryptionNotConfiguredError(
                "KEY_ENCRYPTION_KEY is required to unwrap versioned DEK envelopes"
            )
        keyring = settings.key_encryption_keyring_previous
        keyring[settings.key_encryption_key_id] = key_encryption_key
        return unwrap_dek_with_keks(wrapped, normalized_user_id, keyring)

    pepper = settings.encryption_pepper_bytes
    if not pepper:
        raise EncryptionNotConfiguredError("ENCRYPTION_PEPPER is required for legacy DEK unwrapping")
    return unwrap_dek(wrapped, normalized_user_id, pepper)

def _stored_dek_needs_rewrap(wrapped: bytes) -> bool:
    """Return whether a DEK should be rewritten under the current KEK metadata."""
    settings = get_settings()
    if not settings.key_encryption_key_bytes:
        return False
    if not is_wrapped_dek_envelope(wrapped):
        return True
    return wrapped_dek_key_id(wrapped) != settings.key_encryption_key_id

def _store_wrapped_dek(user_id: str, encrypted_dek: bytes) -> None:
    """Persist a wrapped DEK for a user with a narrow update statement."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(encrypted_dek, bytes):
        raise TypeError("encrypted_dek must be bytes")
    if not encrypted_dek:
        raise ValueError("encrypted_dek must not be empty")
    with get_control_db() as db:
        db.execute(
            "UPDATE users SET encrypted_dek = ? WHERE id = ?",
            (encrypted_dek, normalized_user_id),
        )

```

```

    )
    db.commit()

def get_user_dek(user_id: str) -> bytes:
    """Retrieve and unwrap the user's DEK from the control DB."""
    normalized_user_id = _require_user_id(user_id)
    with get_control_db() as db:
        row = db.execute(
            "SELECT encrypted_dek FROM users WHERE id = ?",
            (normalized_user_id,),
        ).fetchone()
        if not row:
            raise KeyNotFoundError(f"User {normalized_user_id} not found")

    stored_dek = row["encrypted_dek"]
    if not stored_dek:
        # Serialize lazy creation, then recheck under the write lock.
        db.execute("BEGIN IMMEDIATE")
        row = db.execute(
            "SELECT encrypted_dek FROM users WHERE id = ?",
            (normalized_user_id,),
        ).fetchone()
        if not row:
            raise KeyNotFoundError(f"User {normalized_user_id} not found")
    stored_dek = row["encrypted_dek"]
    if not stored_dek:
        dek = generate_dek()
        encrypted_dek = _wrap_user_dek(dek, normalized_user_id)
        db.execute(
            "UPDATE users SET encrypted_dek = ? WHERE id = ?",
            (encrypted_dek, normalized_user_id),
        )
        db.commit()
        logger.info("Generated legacy account data-encryption key")
        return dek
    db.commit()

    dek = _unwrap_user_dek(stored_dek, normalized_user_id)
    if _stored_dek_needs_rewrap(stored_dek):
        _store_wrapped_dek(normalized_user_id, _wrap_user_dek(dek, normalized_user_id))
        logger.info("Rewrapped account data-encryption key")
    return dek

def wrap_new_user_dek(dek: bytes, user_id: str) -> bytes:

```

```

        """Wrap a freshly generated user DEK for account provisioning."""
        return _wrap_user_dek(dek, user_id)

def resolve_user_api_key(user_id: str, provider: str) -> str | None:
    """Look up and decrypt the user's stored BYOK key for a provider.

    Returns the plaintext API key, or None if no key is stored.
    """
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(provider, str):
        raise TypeError("provider must be a string")
    normalized_provider = provider.strip()
    if not normalized_provider:
        raise ValueError("provider must not be empty")

    with get_control_db() as db:
        row = db.execute(
            "SELECT encrypted_key FROM api_keys WHERE user_id = ? AND provider = ? "
            "ORDER BY created_at DESC LIMIT 1",
            (normalized_user_id, normalized_provider),
        ).fetchone()

        if not row:
            return None

        dek = get_user_dek(normalized_user_id)
        key = decrypt_api_key(row["encrypted_key"], dek)

        db.execute(
            "UPDATE api_keys SET last_used_at = CURRENT_TIMESTAMP "
            "WHERE user_id = ? AND provider = ?",
            (normalized_user_id, normalized_provider),
        )
        db.commit()

    return key

def resolve_api_key_for_prompt(user_id: str, provider: str) -> tuple[str, str]:
    """Resolve which API key to use for a prompt.

    Returns (api_key, key_source). Authenticated users must provide BYOK keys.
    """
    byok_key = resolve_user_api_key(user_id, provider)
    if byok_key:

```

```

        return byok_key, "byok"

    raise KeyNotFoundError(f"No API key found for {provider}. Add your own key in Settings.")

def estimate_cost_cents(provider: str, usage: dict[str, int]) -> float:
    """Estimate cost from token counts. Returns cents.

    MiniMax usage is estimated for reporting. Other providers return 0.
    """
    if provider != "minimax":
        return 0.0

    input_tokens = usage.get("input_tokens", 0)
    output_tokens = usage.get("output_tokens", 0)
    cache_read = usage.get("cache_read_tokens", 0)
    cache_write = usage.get("cache_write_tokens", 0)

    cost = (
        input_tokens * _MINIMAX_COSTS["input"]
        + output_tokens * _MINIMAX_COSTS["output"]
        + cache_read * _MINIMAX_COSTS["cache_read"]
        + cache_write * _MINIMAX_COSTS["cache_write"]
    ) / 1_000_000

    return cost

def record_usage(
    user_id: str,
    session_id: str,
    provider: str,
    model: str,
    usage: dict[str, int],
    key_source: str,
) -> None:
    """Insert a usage_log row for the completed prompt."""
    cost = estimate_cost_cents(provider, usage)
    row_id = uuid.uuid4().hex

    with get_control_db() as db:
        db.execute(
            "INSERT INTO usage_log "
            "(id, user_id, session_id, provider, model, "
            "input_tokens, output_tokens, cache_read_tokens, cache_write_tokens, "
            "cost_cents, key_source) "

```

```

        "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
        (
            row_id,
            user_id,
            session_id,
            provider,
            model,
            usage.get("input_tokens", 0),
            usage.get("output_tokens", 0),
            usage.get("cache_read_tokens", 0),
            usage.get("cache_write_tokens", 0),
            cost,
            key_source,
        ),
    )
    db.commit()

logger.info(
    "Usage recorded: provider=%s model=%s cost=%.2fc source=%s",
    provider,
    model,
    cost,
    key_source,
)

```

11 Provider Connections and Pi Configuration

Users bring model-provider access and optional Pi configuration. Settings code separates credential storage, command import, release notes, catalog metadata, and sidecar payloads. Each path removes secrets before storage or execution.

11.1 api/settings.py

Settings routes manage BYOK keys, provider connections, imported Pi config, command discovery, and release notes. The root chunk below tangles back to `backend/src/yinshi/api/settings.py`.

```

"""API key management endpoints (BYOK -- Bring Your Own Key)."""

import asyncio
import json
import logging
from typing import Any

from fastapi import APIRouter, BackgroundTasks, HTTPException, Request, UploadFile

```



```

from yinshi.api.deps import require_tenant
from yinshi.exceptions import (
    ContainerNotReadyError,
    ContainerStartError,
    GitHubAccessError,
    GitHubAppError,
    KeyNotFoundError,
    PiConfigError,
    PiConfigNotFoundError,
    SidecarError,
)
from yinshi.models import (
    ApiKeyCreate,
    ApiKeyOut,
    PiConfigCategoryUpdate,
    PiConfigCommandsOut,
    PiConfigImport,
    PiConfigOut,
    PiReleaseNotesOut,
    ProviderConnectionCreate,
    ProviderConnectionOut,
)
from yinshi.rate_limit import limiter
from yinshi.services.pi_config import (
    MAX_UPLOAD_BYTES,
    get_pi_config,
    import_from_github,
    import_from_upload,
    remove_pi_config,
    sync_pi_config,
    update_enabled_categories,
)
from yinshi.services.pi_releases import get_pi_release_notes
from yinshi.services.provider_connections import (
    create_provider_connection,
    delete_provider_connection,
    list_provider_connections,
)
from yinshi.services.sidecar import create_sidecar_connection
from yinshi.services.sidecar_runtime import (
    resolve_tenant_sidecar_context,
    tenant_container_activity,
    touch_tenant_container,
)

logger = logging.getLogger(__name__)

```

```

router = APIRouter(prefix="/api/settings", tags=["settings"])
_UPLOAD_READ_CHUNK_BYTES = 1024 * 1024
_UPLOAD_TOO_LARGE_DETAIL = "Uploaded archive exceeds the 50MB size limit"

def _github_http_exception(error: GitHubAccessError) -> HTTPException:
    """Convert a GitHub access error into a structured HTTP response."""
    detail = {
        "code": error.code,
        "message": str(error),
        "connect_url": error.connect_url,
        "manage_url": error.manage_url,
    }
    return HTTPException(status_code=400, detail=detail)

def _pi_config_http_exception(error: PiConfigError) -> HTTPException:
    """Convert Pi config service errors into stable HTTP status codes."""
    if isinstance(error, PiConfigNotFoundError):
        return HTTPException(status_code=404, detail=str(error))

    message = str(error)
    if "already exists" in message:
        return HTTPException(status_code=409, detail=message)
    if "still cloning" in message:
        return HTTPException(status_code=409, detail=message)
    return HTTPException(status_code=400, detail=message)

def _connection_http_exception(error: Exception) -> HTTPException:
    """Convert provider connection validation errors into user-facing 400s."""
    return HTTPException(status_code=400, detail=str(error))

def _upload_too_large_http_exception() -> HTTPException:
    """Return the stable 413 used for oversized Pi config uploads."""
    return HTTPException(status_code=413, detail=_UPLOAD_TOO_LARGE_DETAIL)

async def _read_upload_bytes(
    file: UploadFile,
    *,
    max_bytes: int,
) -> bytes:
    """Read one uploaded file while enforcing a hard byte limit."""
    if not isinstance(max_bytes, int):

```

```

        raise TypeError("max_bytes must be an integer")
    if max_bytes <= 0:
        raise ValueError("max_bytes must be positive")

    reported_size = getattr(file, "size", None)
    if isinstance(reported_size, int):
        if reported_size > max_bytes:
            raise _upload_too_large_http_exception()

    upload_bytes = bytearray()
    total_bytes_read = 0
    while True:
        chunk = await file.read(_UPLOAD_READ_CHUNK_BYTES)
        if not chunk:
            break
        total_bytes_read += len(chunk)
        if total_bytes_read > max_bytes:
            raise _upload_too_large_http_exception()
        upload_bytes.extend(chunk)

    assert total_bytes_read == len(upload_bytes)
    return bytes(upload_bytes)

@router.get("/keys", response_model=list[ApiKeyOut])
def list_keys(request: Request) -> list[dict[str, Any]]:
    """List API keys (provider + label only, never the key value)."""
    tenant = require_tenant(request)
    connections = list_provider_connections(tenant.user_id)
    return [
        {
            "id": connection["id"],
            "created_at": connection["created_at"],
            "provider": connection["provider"],
            "label": connection["label"],
            "last_used_at": connection["last_used_at"],
        }
        for connection in connections
        if connection["auth_strategy"] in {"api_key", "api_key_with_config"}
    ]

@router.post("/keys", response_model=ApiKeyOut, status_code=201)
def add_key(body: ApiKeyCreate, request: Request) -> dict[str, Any]:
    """Store an encrypted API key."""
    tenant = require_tenant(request)

```

```

    try:
        connection = create_provider_connection(
            tenant.user_id,
            body.provider,
            "api_key",
            body.key,
            label=body.label,
        )
    except (TypeError, ValueError) as error:
        raise _connection_http_exception(error) from error
    return {
        "id": connection["id"],
        "created_at": connection["created_at"],
        "provider": connection["provider"],
        "label": connection["label"],
        "last_used_at": connection["last_used_at"],
    }

@router.delete("/keys/{key_id}", status_code=204)
def delete_key(key_id: str, request: Request) -> None:
    """Revoke an API key."""
    tenant = require_tenant(request)
    try:
        delete_provider_connection(tenant.user_id, key_id)
    except KeyNotFoundError as error:
        raise HTTPException(status_code=404, detail="Key not found") from error

@router.get("/connections", response_model=list[ProviderConnectionOut])
def list_connections(request: Request) -> list[dict[str, Any]]:
    """List generic provider connections for the authenticated user."""
    tenant = require_tenant(request)
    return list_provider_connections(tenant.user_id)

@router.post("/connections", response_model=ProviderConnectionOut, status_code=201)
def add_connection(body: ProviderConnectionCreate, request: Request) -> dict[str, Any]:
    """Store one generic provider connection."""
    tenant = require_tenant(request)
    try:
        return create_provider_connection(
            tenant.user_id,
            body.provider,
            body.auth_strategy,
            body.secret,

```

```

        label=body.label,
        config=body.config,
    )
except (TypeError, ValueError) as error:
    raise _connection_http_exception(error) from error

@router.delete("/connections/{connection_id}", status_code=204)
def delete_connection(connection_id: str, request: Request) -> None:
    """Delete one generic provider connection."""
    tenant = require_tenant(request)
    try:
        delete_provider_connection(tenant.user_id, connection_id)
    except KeyNotFoundError as error:
        raise HTTPException(status_code=404, detail="Connection not found") from error

@router.get("/pi-config", response_model=PiConfigOut)
def get_pi_config_route(request: Request) -> dict[str, Any]:
    """Return the current user's imported Pi config metadata."""
    tenant = require_tenant(request)
    config = get_pi_config(tenant.user_id)
    if config is None:
        raise HTTPException(status_code=404, detail="Pi config not found")
    return config

@router.get("/pi-release-notes", response_model=PiReleaseNotesOut)
async def get_pi_release_notes_route(request: Request) -> dict[str, Any]:
    """Return installed pi package metadata and recent upstream release notes."""
    require_tenant(request)
    return await get_pi_release_notes()

_SIDE CAR_UNAVAILABLE_DETAIL = "Agent environment temporarily unavailable"

@router.get("/pi-config/commands", response_model=PiConfigCommandsOut)
async def list_pi_config_commands(request: Request) -> dict[str, Any]:
    """Return the slash commands discoverable from the user's imported Pi config."""
    tenant = require_tenant(request)
    assert tenant is not None, "require_tenant must raise rather than return None"

    try:
        tenant_sidecar_context = await resolve_tenant_sidecar_context(request, tenant)
    except (ContainerStartError, ContainerNotReadyError) as error:

```

```

        logger.warning("Pi config command runtime is unavailable")
        raise HTTPException(status_code=503, detail=_SIDECAR_UNAVAILABLE_DETAIL) from error

    # No imported config means no custom commands -- skip the sidecar round trip.
    if tenant_sidecar_context.agent_dir is None:
        return {"commands": []}

    resources = await _fetch_imported_commands(
        request, tenant, tenant_sidecar_context.socket_path, tenant_sidecar_context.agent_dir
    )
    assert isinstance(resources, dict), "sidecar list_imported_commands must return a dict"
    commands = resources.get("commands", [])
    logger.info(
        "pi-config/commands: returning %d commands",
        len(commands) if isinstance(commands, list) else -1,
    )
    return {"commands": commands}

async def _fetch_imported_commands(
    request: Request,
    tenant: Any,
    socket_path: str | None,
    agent_dir: str,
) -> dict[str, Any]:
    """Open a sidecar connection, fetch commands, guarantee cleanup even on errors."""
    sidecar = None
    try:
        async with tenant_container_activity(request, tenant):
            sidecar = await create_sidecar_connection(socket_path)
            resources = await sidecar.list_imported_commands(agent_dir=agent_dir)
            return {"commands": resources["commands"]}
    except (
        ContainerNotReadyError,
        ContainerStartError,
        OSError,
        SidecarError,
        json.JSONDecodeError,
        asyncio.TimeoutError,
    ) as error:
        logger.warning("Pi config command runtime request failed")
        raise HTTPException(status_code=503, detail=_SIDECAR_UNAVAILABLE_DETAIL) from error
    finally:
        # Each cleanup call is independent. Guard individually so one failure
        # does not skip the rest. A missed disconnect leaks the socket.
        try:

```

```

        touch_tenant_container(request, tenant)
    except Exception:
        logger.error("touch_tenant_container failed")
    if sidecar is not None:
        try:
            await sidecar.disconnect()
        except Exception:
            logger.error("sidecar disconnect failed")

@router.post("/pi-config/github", response_model=PiConfigOut, status_code=201)
async def import_github_pi_config(
    body: PiConfigImport,
    request: Request,
    background_tasks: BackgroundTasks,
) -> dict[str, Any]:
    """Start importing a Pi config from GitHub."""
    tenant = require_tenant(request)
    try:
        return await import_from_github(
            user_id=tenant.user_id,
            data_dir=tenant.data_dir,
            repo_url=body.repo_url,
            background_tasks=background_tasks,
        )
    except GitHubAccessError as error:
        raise _github_http_exception(error) from error
    except GitHubAppError as error:
        logger.error("GitHub Pi config import failed during credential resolution")
        raise HTTPException(status_code=502, detail=str(error)) from error
    except PiConfigError as error:
        raise _pi_config_http_exception(error) from error

@router.post("/pi-config/upload", response_model=PiConfigOut, status_code=201)
@limiter.limit("10/hour")
async def upload_pi_config(file: UploadFile, request: Request) -> dict[str, Any]:
    """Import a Pi config from an uploaded zip archive."""
    tenant = require_tenant(request)
    filename = file.filename or "pi-config.zip"
    try:
        zip_data = await _read_upload_bytes(file, max_bytes=MAX_UPLOAD_BYTES)
        return await import_from_upload(
            user_id=tenant.user_id,
            data_dir=tenant.data_dir,
            zip_data=zip_data,

```

```

        filename=filename,
    )
except PiConfigError as error:
    raise _pi_config_http_exception(error) from error
finally:
    await file.close()

@router.patch("/pi-config/categories", response_model=PiConfigOut)
def update_pi_config_categories(
    body: PiConfigCategoryUpdate,
    request: Request,
) -> dict[str, Any]:
    """Enable and disable available Pi config categories."""
    tenant = require_tenant(request)
    try:
        return update_enabled_categories(
            user_id=tenant.user_id,
            data_dir=tenant.data_dir,
            categories=body.enabled_categories,
        )
    except PiConfigError as error:
        raise _pi_config_http_exception(error) from error

@router.post("/pi-config/sync", response_model=PiConfigOut)
async def sync_pi_config_route(request: Request) -> dict[str, Any]:
    """Sync the current user's GitHub-backed Pi config."""
    tenant = require_tenant(request)
    try:
        return await sync_pi_config(
            user_id=tenant.user_id,
            data_dir=tenant.data_dir,
        )
    except GitHubAccessError as error:
        raise _github_http_exception(error) from error
    except GitHubAppError as error:
        logger.error("GitHub Pi config sync failed")
        raise HTTPException(status_code=502, detail=str(error)) from error
    except PiConfigError as error:
        raise _pi_config_http_exception(error) from error

@router.delete("/pi-config", status_code=204)
async def delete_pi_config_route(request: Request) -> None:
    """Remove the current user's imported Pi config."""

```



```

tenant = require_tenant(request)
try:
    await remove_pi_config(
        user_id=tenant.user_id,
        data_dir=tenant.data_dir,
    )
except PiConfigError as error:
    raise _pi_config_http_exception(error) from error

```

11.2 services/provider_connections.py

Provider connection storage normalizes OAuth and API-key credentials before encrypting secrets in tenant storage. The root chunk below tangles back to backend/src/yinshi/services/provider_connections.py.

```

"""Persistent provider connection storage and resolution."""

from __future__ import annotations

import json
import logging
from datetime import datetime
from typing import Any, cast

from yinshi.db import get_control_db
from yinshi.exceptions import KeyNotFoundError
from yinshi.model_catalog import ProviderMetadata, get_provider_metadata
from yinshi.services.crypto import decrypt_api_key, encrypt_api_key
from yinshi.services.keys import get_user_dek

logger = logging.getLogger(__name__)

def _normalize_user_id(user_id: str) -> str:
    """Require a non-empty user identifier."""
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    normalized_user_id = user_id.strip()
    if not normalized_user_id:
        raise ValueError("user_id must not be empty")
    return normalized_user_id

def _normalize_provider(provider: str) -> str:
    """Require a non-empty provider identifier."""
    if not isinstance(provider, str):

```

```

        raise TypeError("provider must be a string")
    normalized_provider = provider.strip()
    if not normalized_provider:
        raise ValueError("provider must not be empty")
    return normalized_provider

def _normalize_auth_strategy(auth_strategy: str) -> str:
    """Require a supported auth strategy for a provider connection."""
    if not isinstance(auth_strategy, str):
        raise TypeError("auth_strategy must be a string")
    normalized_auth_strategy = auth_strategy.strip()
    if not normalized_auth_strategy:
        raise ValueError("auth_strategy must not be empty")
    if normalized_auth_strategy not in {"api_key", "api_key_with_config", "oauth"}:
        raise ValueError(f"Unsupported auth strategy: {auth_strategy}")
    return normalized_auth_strategy

def _serialize_secret(secret: str | dict[str, object], auth_strategy: str) -> str:
    """Serialize a provider secret into a stable encrypted payload."""
    normalized_auth_strategy = _normalize_auth_strategy(auth_strategy)
    if normalized_auth_strategy == "api_key":
        if not isinstance(secret, str):
            raise TypeError("API key secrets must be strings")
        normalized_secret = secret.strip()
        if not normalized_secret:
            raise ValueError("API key secret must not be empty")
        return normalized_secret
    if normalized_auth_strategy == "api_key_with_config":
        if not isinstance(secret, dict):
            raise TypeError("API key + config secrets must be objects")
        if not secret:
            raise ValueError("API key + config secret must not be empty")
        return json.dumps(secret, sort_keys=True)

    if not isinstance(secret, dict):
        raise TypeError("OAuth secrets must be objects")
    if not secret:
        raise ValueError("OAuth secrets must not be empty")
    return json.dumps(secret, sort_keys=True)

def _deserialize_secret(secret_payload: str, auth_strategy: str) -> str | dict[str, object]:
    """Decode an encrypted provider secret payload."""
    normalized_auth_strategy = _normalize_auth_strategy(auth_strategy)

```

```

if normalized_auth_strategy == "api_key":
    if not isinstance(secret_payload, str):
        raise TypeError("Stored API key payload must be a string")
    return secret_payload
if normalized_auth_strategy == "api_key_with_config":
    decoded_secret = json.loads(secret_payload)
    if not isinstance(decoded_secret, dict):
        raise ValueError("Stored API key + config payload must decode to an object")
    normalized_secret = {str(key): value for key, value in decoded_secret.items()}
    return cast(dict[str, object], normalized_secret)

decoded_secret = json.loads(secret_payload)
if not isinstance(decoded_secret, dict):
    raise ValueError("Stored OAuth payload must decode to an object")
normalized_secret = {str(key): value for key, value in decoded_secret.items()}
return cast(dict[str, object], normalized_secret)

def _normalize_text_setting(field_name: str, value: object) -> str:
    """Normalize one string-valued provider setting."""
    if not isinstance(field_name, str):
        raise TypeError("field_name must be a string")
    if not field_name:
        raise ValueError("field_name must not be empty")
    if not isinstance(value, str):
        raise TypeError(f"{field_name} must be a string")
    normalized_value = value.strip()
    if not normalized_value:
        raise ValueError(f"{field_name} must not be empty")
    return normalized_value

def _normalize_public_config(
    provider_metadata: ProviderMetadata,
    config: dict[str, object] | None,
) -> dict[str, object]:
    """Validate and normalize the non-secret config payload."""
    if config is None:
        config = {}
    if not isinstance(config, dict):
        raise TypeError("config must be a dictionary or None")
    secret_field_keys = {field.key for field in provider_metadata.setup_fields if field.secret}
    public_fields = [field for field in provider_metadata.setup_fields if not field.secret]
    public_field_keys = {field.key for field in public_fields}
    unexpected_keys = set(config) - public_field_keys
    if unexpected_keys:

```

```

        unexpected_key = sorted(unexpected_keys)[0]
        if unexpected_key in secret_field_keys:
            raise ValueError(f"{unexpected_key} is secret and must not be sent in config")
        raise ValueError(f"Unexpected config field for {provider_metadata.id}: {unexpected_key}")

    normalized_config: dict[str, object] = {}
    for field in public_fields:
        raw_value = config.get(field.key)
        if raw_value is None:
            if field.required:
                raise ValueError(f"{field.label} is required")
            continue
        normalized_value = _normalize_text_setting(field.label, raw_value)
        normalized_config[field.key] = normalized_value
    return normalized_config

def _normalize_api_key_with_config_secret(
    provider_metadata: ProviderMetadata,
    secret: str | dict[str, object],
) -> dict[str, object]:
    """Normalize encrypted secret payloads for api_key_with_config providers."""
    secret_field_keys = {field.key for field in provider_metadata.setup_fields if field.secret}
    required_secret_field_keys = {
        field.key for field in provider_metadata.setup_fields if field.secret and field.required
    }
    if isinstance(secret, str):
        normalized_api_key = _normalize_text_setting("API key", secret)
        normalized_secret: dict[str, object] = {"apiKey": normalized_api_key}
    else:
        if not isinstance(secret, dict):
            raise TypeError("API key + config secrets must be an object or string")
        normalized_secret = {}
        for key, value in secret.items():
            if key != "apiKey" and key not in secret_field_keys:
                raise ValueError(f"Unexpected secret field for {provider_metadata.id}: {key}")
            normalized_secret[key] = _normalize_text_setting(str(key), value)
        api_key = normalized_secret.get("apiKey")
        if not isinstance(api_key, str) or not api_key:
            raise ValueError("API key is required")

    missing_secret_keys = sorted(
        key for key in required_secret_field_keys if key not in normalized_secret
    )
    if missing_secret_keys:
        raise ValueError(f"Missing required secret field: {missing_secret_keys[0]}")

```

```

    return normalized_secret

def _normalize_connection_secret(
    provider_metadata: ProviderMetadata,
    auth_strategy: str,
    secret: str | dict[str, object],
) -> str | dict[str, object]:
    """Normalize one provider secret payload by auth strategy."""
    normalized_auth_strategy = _normalize_auth_strategy(auth_strategy)
    if normalized_auth_strategy == "api_key":
        return _normalize_text_setting("API key", secret)
    if normalized_auth_strategy == "api_key_with_config":
        return _normalize_api_key_with_config_secret(provider_metadata, secret)
    if normalized_auth_strategy != "oauth":
        raise ValueError(f"Unsupported auth strategy: {auth_strategy}")
    if not isinstance(secret, dict):
        raise TypeError("OAuth secrets must be objects")
    if not secret:
        raise ValueError("OAuth secrets must not be empty")
    normalized_secret = {str(key): value for key, value in secret.items()}
    return normalized_secret

def _encode_config(config: dict[str, object] | None) -> str:
    """Serialize non-secret provider config."""
    if config is None:
        return "{}"
    if not isinstance(config, dict):
        raise TypeError("config must be a dictionary or None")
    normalized_config = {str(key): value for key, value in config.items()}
    return json.dumps(normalized_config, sort_keys=True)

def _decode_config(config_json: str | None) -> dict[str, object]:
    """Deserialize non-secret provider config."""
    if not config_json:
        return {}
    decoded_config = json.loads(config_json)
    if not isinstance(decoded_config, dict):
        raise ValueError("Stored config must decode to an object")
    normalized_config = {str(key): value for key, value in decoded_config.items()}
    return cast(dict[str, object], normalized_config)

def list_provider_connections(user_id: str) -> list[dict[str, Any]]:

```

```

        """List provider connections for a user."""
        normalized_user_id = _normalize_user_id(user_id)
        with get_control_db() as db:
            rows = db.execute(
                "SELECT id, created_at, updated_at, provider, auth_strategy, label, "
                "config_json, status, last_used_at, expires_at "
                "FROM provider_connections WHERE user_id = ? "
                "ORDER BY updated_at DESC, created_at DESC",
                (normalized_user_id,),
            ).fetchall()
        connections: list[dict[str, Any]] = []
        for row in rows:
            connection = dict(row)
            connection["config"] = _decode_config(connection.pop("config_json", "{}"))
            connections.append(connection)
        return connections

def create_provider_connection(
    user_id: str,
    provider: str,
    auth_strategy: str,
    secret: str | dict[str, object],
    *,
    label: str = "",
    config: dict[str, object] | None = None,
    status: str = "connected",
    expires_at: datetime | None = None,
) -> dict[str, Any]:
    """Store an encrypted provider connection for a user."""
    normalized_user_id = _normalize_user_id(user_id)
    normalized_provider = _normalize_provider(provider)
    normalized_auth_strategy = _normalize_auth_strategy(auth_strategy)
    provider_metadata = get_provider_metadata(normalized_provider)
    if normalized_auth_strategy not in provider_metadata.auth_strategies:
        raise ValueError(
            f"{normalized_provider} does not support auth strategy {normalized_auth_strategy}"
        )
    if not isinstance(label, str):
        raise TypeError("label must be a string")
    if not isinstance(status, str):
        raise TypeError("status must be a string")
    normalized_secret_object = _normalize_connection_secret(
        provider_metadata,
        normalized_auth_strategy,
        secret,
    )

```

```

)
normalized_config = _normalize_public_config(provider_metadata, config)
normalized_secret = _serialize_secret(normalized_secret_object, normalized_auth_strategy)
encrypted_secret = encrypt_api_key(normalized_secret, get_user_dek(normalized_user_id))
config_json = _encode_config(normalized_config)

with get_control_db() as db:
    cursor = db.execute(
        "INSERT INTO provider_connections "
        "(user_id, provider, auth_strategy, encrypted_secret, label, config_json, status, "
        "VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
        (
            normalized_user_id,
            normalized_provider,
            normalized_auth_strategy,
            encrypted_secret,
            label,
            config_json,
            status,
            expires_at.isoformat() if expires_at else None,
        ),
    )
    db.commit()
    row = db.execute(
        "SELECT id, created_at, updated_at, provider, auth_strategy, label, "
        "config_json, status, last_used_at, expires_at "
        "FROM provider_connections WHERE rowid = ?",
        (cursor.lastrowid,),
    ).fetchone()
    assert row is not None, "provider connection insert must return a row"
    connection = dict(row)
    connection["config"] = _decode_config(connection.pop("config_json", "{}"))
    return connection


def delete_provider_connection(user_id: str, connection_id: str) -> None:
    """Delete one provider connection belonging to a user."""
    normalized_user_id = _normalize_user_id(user_id)
    if not isinstance(connection_id, str):
        raise TypeError("connection_id must be a string")
    normalized_connection_id = connection_id.strip()
    if not normalized_connection_id:
        raise ValueError("connection_id must not be empty")
    with get_control_db() as db:
        row = db.execute(
            "SELECT id FROM provider_connections WHERE id = ? AND user_id = ?",

```

```

        (normalized_connection_id, normalized_user_id),
    ).fetchone()
    if row is None:
        raise KeyNotFoundError("Connection not found")
    db.execute("DELETE FROM provider_connections WHERE id = ?", (normalized_connection_id,))
    db.commit()

def resolve_provider_connection(
    user_id: str,
    provider: str,
) -> dict[str, Any]:
    """Return the newest provider connection for a provider, including decrypted secret."""
    normalized_user_id = _normalize_user_id(user_id)
    normalized_provider = _normalize_provider(provider)
    with get_control_db() as db:
        row = db.execute(
            "SELECT id, provider, auth_strategy, encrypted_secret, label, config_json, "
            "status, last_used_at, expires_at "
            "FROM provider_connections WHERE user_id = ? AND provider = ? "
            "ORDER BY updated_at DESC, created_at DESC LIMIT 1",
            (normalized_user_id, normalized_provider),
        ).fetchone()
        if row is not None:
            secret_payload = decrypt_api_key(
                row["encrypted_secret"],
                get_user_dek(normalized_user_id),
            )
            db.execute(
                "UPDATE provider_connections SET last_used_at = CURRENT_TIMESTAMP WHERE id = "
                "(row[\"id\"],),",
            )
            db.commit()
            connection = dict(row)
            connection["secret"] = _deserialize_secret(secret_payload, connection["auth_strategy"])
            connection["config"] = _decode_config(connection.pop("config_json", "{}"))
            return connection

    # Keep reading legacy BYOK rows during the migration window so existing
    # users and tests continue to work before they are backfilled.
    legacy_row = db.execute(
        "SELECT rowid, encrypted_key FROM api_keys WHERE user_id = ? AND provider = ? "
        "ORDER BY created_at DESC LIMIT 1",
        (normalized_user_id, normalized_provider),
    ).fetchone()
    if legacy_row is None:

```



```

        raise KeyNotFoundError(
            f"No provider connection found for {normalized_provider}. Add it in Settings"
        )

    legacy_secret = decrypt_api_key(
        legacy_row["encrypted_key"],
        get_user_dek(normalized_user_id),
    )
    db.execute(
        "UPDATE api_keys SET last_used_at = CURRENT_TIMESTAMP WHERE rowid = ?",
        (legacy_row["rowid"],),
    )
    db.commit()

    return {
        "id": f"legacy-{normalized_provider}",
        "provider": normalized_provider,
        "auth_strategy": "api_key",
        "label": "",
        "status": "connected",
        "last_used_at": None,
        "expires_at": None,
        "secret": legacy_secret,
        "config": {},
    }

def update_provider_connection_secret(
    user_id: str,
    connection_id: str,
    auth_strategy: str,
    secret: str | dict[str, object],
) -> None:
    """Persist a refreshed provider secret payload."""
    normalized_user_id = _normalize_user_id(user_id)
    if not isinstance(connection_id, str):
        raise TypeError("connection_id must be a string")
    normalized_connection_id = connection_id.strip()
    if not normalized_connection_id:
        raise ValueError("connection_id must not be empty")
    normalized_auth_strategy = _normalize_auth_strategy(auth_strategy)
    with get_control_db() as db:
        row = db.execute(
            "SELECT provider FROM provider_connections WHERE id = ? AND user_id = ?",
            (normalized_connection_id, normalized_user_id),
        ).fetchone()

```

```

if row is None:
    logger.warning("Skipping refresh for missing provider connection")
    return
normalized_secret = _normalize_connection_secret(
    get_provider_metadata(row["provider"]),
    normalized_auth_strategy,
    secret,
)
encrypted_secret = encrypt_api_key(
    _serialize_secret(normalized_secret, normalized_auth_strategy),
    get_user_dek(normalized_user_id),
)
with get_control_db() as db:
    db.execute(
        "UPDATE provider_connections SET encrypted_secret = ?, last_used_at = CURRENT_TIMESTAMP
        WHERE id = ? AND user_id = ?",
        (encrypted_secret, normalized_connection_id, normalized_user_id),
    )
    db.commit()
logger.info("Refreshed provider connection")

```

11.3 services/pi_config.py

Pi config services import user agent configuration, scrub secret-looking fields, store files, and prepare runtime settings. The root chunk below tangles back to backend/src/yinshi/services/pi_config.py.

"""Import, store, and toggle per-user Pi config resources."""

```

from __future__ import annotations

import io
import json
import logging
import os
import shutil
import sqlite3
import stat
import tempfile
import zipfile
from dataclasses import dataclass
from pathlib import Path, PurePosixPath
from typing import Any, cast

from fastapi import BackgroundTasks

```

```

from yinshi.config import get_settings
from yinshi.db import get_control_db
from yinshi.exceptions import PiConfigError, PiConfigNotFoundError
from yinshi.models import PI_CONFIG_CATEGORIES, PI_CONFIG_CATEGORY_ORDER
from yinshi.services.control_encryption import decrypt_control_text, encrypt_control_text
from yinshi.services.git import _git_askpass_env, _run_git, _validate_clone_url, clone_repo
from yinshi.services.github_app import resolve_github_clone_access
from yinshi.services.user_settings import (
    _sanitize_pi_settings,
    clear_pi_settings,
    get_sidecar_settings_payload,
    set_pi_settings_enabled,
    store_pi_settings,
)

logger = logging.getLogger(__name__)

_PI_CONFIG_DIRECTORY_NAME = "pi-config"
_PI_RUNTIME_DIRECTORY_NAME = "pi-runtime"
_AGENT_DIRECTORY_NAME = "agent"
_SESSION_RUNTIME_DIRECTORY_NAME = "sessions"
MAX_UPLOAD_BYTES = 50 * 1024 * 1024
_MAX_EXTRACTED_BYTES = 200 * 1024 * 1024
_MAX_ARCHIVE_MEMBERS = 4096
_EXTRACT_CHUNK_BYTES = 64 * 1024
_INSTRUCTION_FILENAMES = ("AGENTS.md", "CLAUDE.md")
_SENSITIVE_JSON_FILENAMES = frozenset(
    {
        "auth.json",
        "credentials.json",
        "oauth.json",
        "tokens.json",
        "secrets.json",
    }
)

_DIRECTORY_CATEGORIES = frozenset(
    {"skills", "extensions", "prompts", "agents", "themes", "sessions"}
)

_FILE_CATEGORIES = frozenset({"settings", "models"})
_CATEGORY_PATHS = {
    "skills": Path("agent/skills"),
    "extensions": Path("agent/extensions"),
    "prompts": Path("agent/prompts"),
    "agents": Path("agent/agents"),
    "themes": Path("agent/themes"),
    "settings": Path("agent/settings.json"),
}

```

```

        "models": Path("agent/models.json"),
        "sessions": Path("agent/sessions"),
    }

@dataclass(frozen=True, slots=True)
class PiRuntimeInputs:
    """One resolved Pi runtime configuration for sidecar execution."""

    agent_dir: str | None
    settings_payload: dict[str, object] | None

    def _validate_user_id(user_id: str) -> str:
        """Require a non-empty user identifier."""
        if not isinstance(user_id, str):
            raise TypeError("user_id must be a string")
        normalized_user_id = user_id.strip()
        if not normalized_user_id:
            raise ValueError("user_id must not be empty")
        return normalized_user_id

    def _validate_data_dir(data_dir: str) -> Path:
        """Require a non-empty data directory path."""
        if not isinstance(data_dir, str):
            raise TypeError("data_dir must be a string")
        normalized_data_dir = data_dir.strip()
        if not normalized_data_dir:
            raise ValueError("data_dir must not be empty")
        return Path(normalized_data_dir)

    def _validate_runtime_session_id(runtime_session_id: str) -> str:
        """Require a non-empty session identifier for runtime override paths."""
        if not isinstance(runtime_session_id, str):
            raise TypeError("runtime_session_id must be a string")
        normalized_session_id = runtime_session_id.strip()
        if not normalized_session_id:
            raise ValueError("runtime_session_id must not be empty")
        return normalized_session_id

    def _pi_config_root_path(data_dir: str) -> Path:
        """Return the root directory that stores a user's imported Pi config."""
        data_root = _validate_data_dir(data_dir)

```

```

    return data_root / _PI_CONFIG_DIRECTORY_NAME

def _pi_runtime_root_path(data_dir: str) -> Path:
    """Return the root directory that stores derived runtime-only Pi data."""
    data_root = _validate_data_dir(data_dir)
    return data_root / _PI_RUNTIME_DIRECTORY_NAME

def _pi_agent_dir_path(data_dir: str) -> Path:
    """Return the agentDir path for a user's imported Pi config."""
    return _pi_config_root_path(data_dir) / _AGENT_DIRECTORY_NAME

def _session_runtime_root_path(data_dir: str, runtime_session_id: str) -> Path:
    """Return the per-session runtime directory for repo instruction overlays."""
    normalized_session_id = _validate_runtime_session_id(runtime_session_id)
    return _pi_runtime_root_path(data_dir) / _SESSION_RUNTIME_DIRECTORY_NAME / normalized_session_id

def _ordered_categories(categories: set[str]) -> list[str]:
    """Return categories in a stable display and storage order."""
    return [category for category in PI_CONFIG_CATEGORY_ORDER if category in categories]

def _load_categories_json(encoded_categories: str) -> list[str]:
    """Decode stored categories and validate every entry."""
    if not isinstance(encoded_categories, str):
        raise TypeError("encoded_categories must be a string")
    decoded_categories = json.loads(encoded_categories)
    if not isinstance(decoded_categories, list):
        raise ValueError("Category JSON must decode to a list")

    normalized_categories: set[str] = set()
    for category in decoded_categories:
        if not isinstance(category, str):
            raise ValueError("Category names must be strings")
        if category not in PI_CONFIG_CATEGORIES:
            raise ValueError(f"Unsupported stored category: {category}")
        normalized_categories.add(category)
    return _ordered_categories(normalized_categories)

def _is_sensitive_filename(path: Path) -> bool:
    """Return whether one path name is a known secret-bearing config file."""
    file_name = path.name

```

```

assert file_name, "path.name must not be empty"

if file_name in _SENSITIVE_JSON_FILENAMES:
    return True
if file_name == ".env":
    return True
if file_name.startswith(".env."):
    return True
return False

def _normalize_secret_key_name(key: str) -> str:
    """Collapse a JSON key into a token-comparable secret marker string."""
    if not isinstance(key, str):
        raise TypeError("key must be a string")
    return "".join(character for character in key.lower() if character.isalnum())

def _looks_like_secret_key(key: str) -> bool:
    """Return whether one JSON key name is likely to hold a credential."""
    normalized_key = _normalize_secret_key_name(key)
    if not normalized_key:
        return False
    if "secret" in normalized_key:
        return True
    if normalized_key.endswith("apikey"):
        return True
    if normalized_key.endswith("token"):
        return True
    if normalized_key.endswith("privatekey"):
        return True
    if normalized_key.endswith("clientkey"):
        return True
    if normalized_key in {
        "apikey",
        "accesskey",
        "accesstoken",
        "refreshtoken",
        "authtoken",
        "bearertoken",
        "clientsecret",
        "privatekey",
    }:
        return True
    return False

```

```

def _scrub_json_payload_secrets(payload: object) -> tuple[object, bool]:
    """Drop secret-bearing keys from one decoded JSON payload."""
    if isinstance(payload, dict):
        scrubbed_payload: dict[str, object] = {}
        did_remove_secret = False
        for key, value in payload.items():
            if not isinstance(key, str):
                raise TypeError("JSON object keys must be strings")
            if _looks_like_secret_key(key):
                did_remove_secret = True
                continue
            scrubbed_value, value_removed_secret = _scrub_json_payload_secrets(value)
            if value_removed_secret:
                did_remove_secret = True
            scrubbed_payload[key] = scrubbed_value
        return scrubbed_payload, did_remove_secret

    if isinstance(payload, list):
        scrubbed_items: list[object] = []
        did_remove_secret = False
        for item in payload:
            scrubbed_item, item_removed_secret = _scrub_json_payload_secrets(item)
            if item_removed_secret:
                did_remove_secret = True
            scrubbed_items.append(scrubbed_item)
        return scrubbed_items, did_remove_secret

    return payload, False


def _scrub_json_file_secrets(json_path: Path) -> None:
    """Rewrite one JSON file after removing secret-bearing keys."""
    if not json_path.is_file():
        raise ValueError("json_path must point to an existing file")
    if json_path.suffix.lower() != ".json":
        raise ValueError("json_path must be a JSON file")

    try:
        payload = json.loads(json_path.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        return

    scrubbed_payload, did_remove_secret = _scrub_json_payload_secrets(payload)
    if not did_remove_secret:
        return

```

```

        json_path.write_text(
            json.dumps(scrubbed_payload, indent=2, sort_keys=True) + "\n",
            encoding="utf-8",
        )

def _scrub_secret_files(config_root: Path) -> None:
    """Delete imported files that are primarily used to store credentials."""
    if not config_root.is_dir():
        raise ValueError("config_root must point to a directory")

    for candidate_path in config_root.rglob("*"):
        if not candidate_path.is_file():
            continue
        if not _is_sensitive_filename(candidate_path):
            continue
        _remove_path(candidate_path)

def _scrub_json_secrets(config_root: Path) -> None:
    """Rewrite imported JSON files after removing nested credential fields."""
    if not config_root.is_dir():
        raise ValueError("config_root must point to a directory")

    for json_path in config_root.rglob("*.json"):
        if not json_path.is_file():
            continue
        _scrub_json_file_secrets(json_path)

def _scrub_settings_payload(config_root: Path) -> None:
    """Rewrite settings.json so on-disk runtime config matches stored sanitized settings."""
    if not config_root.is_dir():
        raise ValueError("config_root must point to a directory")

    settings_path = config_root / _CATEGORY_PATHS["settings"]
    if not settings_path.is_file():
        return

    settings_payload = _read_settings_json(settings_path)
    sanitized_settings = _sanitize_pi_settings(settings_payload)
    settings_path.write_text(
        json.dumps(sanitized_settings, indent=2, sort_keys=True) + "\n",
        encoding="utf-8",
    )

```



```

def _dump_categories_json(categories: list[str]) -> str:
    """Serialize categories with a stable ordering."""
    normalized_categories = set(categories)
    return json.dumps(_ordered_categories(normalized_categories))

def _decrypt_pi_config_row(row: sqlite3.Row) -> dict[str, Any]:
    """Return a Pi config row with sensitive control fields decrypted."""
    config_dict = dict(row)
    user_id = str(config_dict["user_id"])
    for field_name in ("source_label", "repo_url", "error_message"):
        field_value = config_dict.get(field_name)
        if field_value is None:
            continue
        if not isinstance(field_value, str):
            raise ValueError(f"Pi config field {field_name} must be text or NULL")
        config_dict[field_name] = decrypt_control_text(
            f"pi_configs.{field_name}",
            user_id,
            field_value,
        )
    return config_dict

def _row_to_pi_config(row: dict[str, Any]) -> dict[str, Any]:
    """Convert a database row into an API-facing config dictionary."""
    config_dict = dict(row)
    config_dict["available_categories"] = _load_categories_json(row["available_categories"])
    config_dict["enabled_categories"] = _load_categories_json(row["enabled_categories"])
    return config_dict

def _get_pi_config_row(user_id: str) -> dict[str, Any] | None:
    """Return the decrypted Pi config database row for a user."""
    normalized_user_id = _validate_user_id(user_id)
    with get_control_db() as db:
        row = cast(
            sqlite3.Row | None,
            db.execute(
                "SELECT * FROM pi_configs WHERE user_id = ?",
                (normalized_user_id,),
            ).fetchone(),
        )
    if row is None:

```

```

        return None
    return _decrypt_pi_config_row(row)

def _require_pi_config_row(user_id: str) -> dict[str, Any]:
    """Return the Pi config row or raise when it does not exist."""
    row = _get_pi_config_row(user_id)
    if row is None:
        raise PiConfigNotFoundError("Pi config not found")
    return row

def _insert_pi_config_row(
    user_id: str,
    *,
    source_type: str,
    source_label: str,
    repo_url: str | None,
    status: str,
    available_categories: list[str],
    enabled_categories: list[str],
) -> dict[str, Any]:
    """Insert a new Pi config row and return it as a dictionary."""
    normalized_user_id = _validate_user_id(user_id)
    if not source_label.strip():
        raise ValueError("source_label must not be empty")

    with get_control_db() as db:
        cursor = db.execute(
            "INSERT INTO pi_configs (user_id, source_type, source_label, repo_url, "
            "available_categories, enabled_categories, status) "
            "VALUES (?, ?, ?, ?, ?, ?, ?)",
            (
                normalized_user_id,
                source_type,
                encrypt_control_text("pi_configs.source_label", normalized_user_id, source_label),
                encrypt_control_text("pi_configs.repo_url", normalized_user_id, repo_url),
                _dump_categories_json(available_categories),
                _dump_categories_json(enabled_categories),
                status,
            ),
        )
    db.commit()
    row = cast(
        sqlite3.Row | None,
        db.execute(

```

```

        "SELECT * FROM pi_configs WHERE rowid = ?",
        (cursor.lastrowid,),
    ).fetchone(),
)
assert row is not None, "inserted Pi config row must be readable"
return _row_to_pi_config(_decrypt_pi_config_row(row))

def _update_pi_config_row(user_id: str, **fields: object) -> None:
    """Update the stored Pi config row using an allow-listed field set."""
    normalized_user_id = _validate_user_id(user_id)
    if not fields:
        raise ValueError("fields must not be empty")

    allowed_fields = {
        "source_label",
        "repo_url",
        "available_categories",
        "enabled_categories",
        "last_synced_at",
        "status",
        "error_message",
    }
    invalid_fields = set(fields) - allowed_fields
    if invalid_fields:
        raise ValueError(f"Unsupported update fields: {sorted(invalid_fields)}")

    assignments: list[str] = []
    values: list[object] = []
    encrypted_field_names = {"source_label", "repo_url", "error_message"}
    for field_name, field_value in fields.items():
        assignments.append(f"{field_name} = ?")
        if field_name in encrypted_field_names:
            if field_value is not None and not isinstance(field_value, str):
                raise TypeError(f"{field_name} must be text or None")
            values.append(
                encrypt_control_text(
                    f"pi_configs.{field_name}",
                    normalized_user_id,
                    field_value,
                )
            )
        else:
            values.append(field_value)
    values.append(normalized_user_id)

```

```

with get_control_db() as db:
    db.execute(
        f"UPDATE pi_configs SET {'', ' '.join(assignments)} WHERE user_id = ?", # noqa: S
        values,
    )
    db.commit()

def _delete_pi_config_row(user_id: str) -> None:
    """Remove the stored Pi config and imported settings rows for a user."""
    normalized_user_id = _validate_user_id(user_id)
    with get_control_db() as db:
        db.execute("DELETE FROM pi_configs WHERE user_id = ?", (normalized_user_id,))
        db.execute("DELETE FROM user_settings WHERE user_id = ?", (normalized_user_id,))
        db.commit()

def _remove_path(path: Path) -> None:
    """Delete a file or directory path when it exists."""
    if path.is_symlink() or path.is_file():
        path.unlink(missing_ok=True)
        return
    if path.is_dir():
        shutil.rmtree(path)

def _safe_zip_target(root_path: Path, archive_name: str) -> Path:
    """Resolve a zip member path and reject traversal or absolute paths."""
    normalized_name = archive_name.strip()
    if not normalized_name:
        raise PiConfigError("Archive contains an empty path entry")

    archive_path = PurePosixPath(normalized_name)
    if archive_path.is_absolute():
        raise PiConfigError("Archive contains an absolute path")
    if ".." in archive_path.parts:
        raise PiConfigError("Archive contains a parent directory traversal")

    relative_parts = [part for part in archive_path.parts if part not in ("", ".")]
    if not relative_parts:
        raise PiConfigError("Archive contains an invalid path entry")
    return root_path.joinpath(*relative_parts)

def _extract_archive(zip_data: bytes, temp_root: Path) -> None:
    """Extract a validated zip archive into a temporary directory."""

```

```

if len(zip_data) > MAX_UPLOAD_BYTES:
    raise PiConfigError("Uploaded archive exceeds the 50MB size limit")
if not zip_data.startswith(b"PK"):
    raise PiConfigError("Uploaded file is not a zip archive")

total_extracted_bytes = 0
with zipfile.ZipFile(io.BytesIO(zip_data)) as archive:
    members = archive.infolist()
    if not members:
        raise PiConfigError("Uploaded archive is empty")
    if len(members) > _MAX_ARCHIVE_MEMBERS:
        raise PiConfigError("Uploaded archive contains too many files")

    for member in members:
        mode_bits = member.external_attr >> 16
        if stat.S_ISLNK(mode_bits):
            raise PiConfigError("Archive must not contain symbolic links")

        target_path = _safe_zip_target(temp_root, member.filename)
        if member.is_dir():
            target_path.mkdir(parents=True, exist_ok=True)
            continue

        target_path.parent.mkdir(parents=True, exist_ok=True)
        with archive.open(member) as source_file:
            with target_path.open("wb") as target_file:
                while True:
                    chunk = source_file.read(_EXTRACT_CHUNK_BYTES)
                    if not chunk:
                        break
                    total_extracted_bytes += len(chunk)
                    if total_extracted_bytes > _MAX_EXTRACTED_BYTES:
                        raise PiConfigError("Archive expands beyond the allowed size limit")
                    target_file.write(chunk)

def _find_extracted_config_root(extracted_root: Path) -> Path:
    """Locate the extracted Pi config root that contains an agent directory."""
    candidate_paths: dict[Path, int] = {}
    for path in sorted(extracted_root.rglob("*")):
        if not path.is_dir():
            continue
        if (path / _AGENT_DIRECTORY_NAME).is_dir():
            candidate_paths[path] = len(path.relative_to(extracted_root).parts)

    if (extracted_root / _AGENT_DIRECTORY_NAME).is_dir():

```

```

        candidate_paths[extracted_root] = 0
    if (extracted_root / ".pi" / _AGENT_DIRECTORY_NAME).is_dir():
        candidate_paths[extracted_root / ".pi"] = 1

    if not candidate_paths:
        raise PiConfigError("Archive must contain an agent/ directory")

    shallowest_depth = min(candidate_paths.values())
    shallowest_paths = [
        path for path, depth in candidate_paths.items() if depth == shallowest_depth
    ]
    if len(shallowest_paths) != 1:
        raise PiConfigError("Archive contains multiple candidate Pi config roots")
    return shallowest_paths[0]

def _prepare_destination(config_root: Path) -> None:
    """Clear any leftover config directory and recreate its parent path."""
    _remove_path(config_root)
    config_root.parent.mkdir(parents=True, exist_ok=True)

def _recreate_directory_root(root_path: Path) -> None:
    """Replace one directory tree with a new empty directory."""
    _remove_path(root_path)
    root_path.mkdir(parents=True, exist_ok=True)

def _mirror_instruction_files(config_root: Path) -> None:
    """Copy supported root instruction files into agent/ for SDK discovery."""
    agent_dir = config_root / _AGENT_DIRECTORY_NAME
    if not agent_dir.is_dir():
        raise PiConfigError("Imported Pi config is missing the agent directory")

    for filename in _INSTRUCTION_FILENAMES:
        source_path = config_root / filename
        if not source_path.is_file():
            continue
        shutil.copyfile(source_path, agent_dir / filename)

def _clear_instruction_runtime_files(config_root: Path) -> None:
    """Remove mirrored instruction files so sync can rebuild them from root files."""
    for instruction_path in _instruction_runtime_paths(config_root):
        disabled_path = instruction_path.with_name(f"{instruction_path.name}.disabled")
        _remove_path(instruction_path)

```

```

_remove_path(disabled_path)

def _clear_disabled_category_artifacts(config_root: Path) -> None:
    """Remove local disabled artifacts so sync starts from the fetched tree."""
    for relative_path in _CATEGORY_PATHS.values():
        actual_path = config_root / relative_path
        disabled_path = actual_path.with_name(f"{actual_path.name}.disabled")
        _remove_path(disabled_path)

def _reset_runtime_artifacts_for_sync(config_root: Path) -> None:
    """Delete local runtime artifacts that are derived from toggle state."""
    _clear_disabled_category_artifacts(config_root)
    _clear_instruction_runtime_files(config_root)

def _instruction_runtime_paths(config_root: Path) -> list[Path]:
    """Return the mirrored instruction file paths inside agent/."""
    agent_dir = config_root / _AGENT_DIRECTORY_NAME
    return [agent_dir / filename for filename in _INSTRUCTION_FILENAMES]

def _link_runtime_agent_children(source_agent_dir: Path, target_agent_dir: Path) -> None:
    """Link all base agent assets into a runtime override directory except AGENTS.md."""
    if not source_agent_dir.exists():
        raise PiConfigError(f"Base agent directory does not exist: {source_agent_dir}")
    if not source_agent_dir.is_dir():
        raise PiConfigError(f"Base agent directory is not a directory: {source_agent_dir}")
    if not target_agent_dir.exists():
        raise PiConfigError(f"Target agent directory does not exist: {target_agent_dir}")
    if not target_agent_dir.is_dir():
        raise PiConfigError(f"Target agent directory is not a directory: {target_agent_dir}")

    for source_path in sorted(source_agent_dir.iterdir(), key=lambda path: path.name):
        if source_path.name == "AGENTS.md":
            continue
        target_path = target_agent_dir / source_path.name
        relative_source_path = os.path.relpath(source_path, target_agent_dir)
        target_path.symlink_to(
            relative_source_path,
            target_is_directory=source_path.is_dir(),
        )

def _materialize_repo_agent_override(

```

```

data_dir: str,
runtime_session_id: str,
repo_agents_md: str,
base_agent_dir: str | None,
) -> str:
    """Build one per-session agentDir that overlays repo instructions onto the base config."""
    if not isinstance(repo_agents_md, str):
        raise TypeError("repo_agents_md must be a string")

    runtime_root = _session_runtime_root_path(data_dir, runtime_session_id)
    _recreate_directory_root(runtime_root)

    runtime_agent_dir = runtime_root / _AGENT_DIRECTORY_NAME
    runtime_agent_dir.mkdir(parents=True, exist_ok=True)

    if base_agent_dir is not None:
        base_agent_path = Path(base_agent_dir)
        _link_runtime_agent_children(base_agent_path, runtime_agent_dir)

    # The runtime directory overlays only AGENTS.md so all other Pi assets stay shared.
    runtime_agents_md_path = runtime_agent_dir / "AGENTS.md"
    runtime_agents_md_path.write_text(repo_agents_md, encoding="utf-8")
    return str(runtime_agent_dir)

def _scan_categories(config_root: Path) -> list[str]:
    """Return available categories, including currently disabled ones."""
    if not config_root.is_dir():
        raise PiConfigError("Pi config directory does not exist")

    available_categories: set[str] = set()
    for category, relative_path in _CATEGORY_PATHS.items():
        actual_path = config_root / relative_path
        disabled_path = actual_path.with_name(f"{actual_path.name}.disabled")
        if actual_path.exists() or disabled_path.exists():
            available_categories.add(category)

    for instruction_path in _instruction_runtime_paths(config_root):
        disabled_instruction_path = instruction_path.with_name(f"{instruction_path.name}.disabled")
        if instruction_path.exists() or disabled_instruction_path.exists():
            available_categories.add("instructions")
            break

    return _ordered_categories(available_categories)

```



```

def _rename_if_present(source_path: Path, target_path: Path) -> None:
    """Rename a path when the source exists and the target does not."""
    if not source_path.exists():
        return
    if target_path.exists():
        raise PiConfigError(f"Target path already exists: {target_path}")
    source_path.rename(target_path)

def _set_category_enabled(config_root: Path, category: str, *, enabled: bool) -> None:
    """Rename a category path to either its enabled or disabled form."""
    if category in _DIRECTORY_CATEGORIES or category in _FILE_CATEGORIES:
        actual_path = config_root / _CATEGORY_PATHS[category]
        disabled_path = actual_path.with_name(f"{actual_path.name}.disabled")
        if enabled:
            _rename_if_present(disabled_path, actual_path)
        else:
            _rename_if_present(actual_path, disabled_path)
        return

    if category != "instructions":
        raise ValueError(f"Unsupported category: {category}")

    for instruction_path in _instruction_runtime_paths(config_root):
        disabled_path = instruction_path.with_name(f"{instruction_path.name}.disabled")
        if enabled:
            _rename_if_present(disabled_path, instruction_path)
        else:
            _rename_if_present(instruction_path, disabled_path)

def _apply_enabled_categories(
    config_root: Path,
    available_categories: list[str],
    enabled_categories: list[str],
) -> None:
    """Apply enabled category state by renaming category paths on disk."""
    available_set = set(available_categories)
    enabled_set = set(enabled_categories)
    for category in available_set:
        _set_category_enabled(config_root, category, enabled=category in enabled_set)

def _scrub_pi_config(config_root: Path, *, keep_git: bool) -> None:
    """Remove sensitive or host-specific files from an imported config."""
    if not config_root.is_dir():

```

```

        raise ValueError("config_root must point to a directory")
    _scrub_secret_files(config_root)
    _scrub_json_secrets(config_root)
    _scrub_settings_payload(config_root)
    _remove_path(config_root / "bin")
    _remove_path(config_root / _AGENT_DIRECTORY_NAME / "bin")
    if not keep_git:
        _remove_path(config_root / ".git")

def _read_settings_json(settings_path: Path) -> dict[str, object]:
    """Parse a settings.json file into a JSON object."""
    if not settings_path.is_file():
        raise PiConfigError("settings.json does not exist")
    try:
        payload = json.loads(settings_path.read_text(encoding="utf-8"))
    except json.JSONDecodeError as exc:
        raise PiConfigError("settings.json contains invalid JSON") from exc
    if not isinstance(payload, dict):
        raise PiConfigError("settings.json must contain a JSON object")
    return cast(dict[str, object], payload)

def _extract_and_store_settings(
    user_id: str,
    config_root: Path,
    *,
    settings_enabled: bool,
) -> None:
    """Persist imported settings or clear them when the file is absent."""
    settings_path = config_root / _CATEGORY_PATHS["settings"]
    if not settings_path.is_file():
        clear_pi_settings(user_id)
        return

    settings_payload = _read_settings_json(settings_path)
    store_pi_settings(user_id, settings_payload)
    set_pi_settings_enabled(user_id, enabled=settings_enabled)

async def _resolve_clone_details(user_id: str, repo_url: str) -> tuple[str, str | None]:
    """Normalize clone details and obtain credentials when GitHub access is available."""
    clone_access = await resolve_github_clone_access(user_id, repo_url)
    if clone_access is None:
        _validate_clone_url(repo_url)
        return repo_url, None

```

```

_validate_clone_url(clone_access.clone_url)
return clone_access.clone_url, clone_access.access_token

def _set_last_synced_at_now(user_id: str) -> None:
    """Update the sync timestamp using SQLite's current timestamp function."""
    normalized_user_id = _validate_user_id(user_id)
    with get_control_db() as db:
        db.execute(
            "UPDATE pi_configs SET last_synced_at = CURRENT_TIMESTAMP WHERE user_id = ?",
            (normalized_user_id,),
        )
        db.commit()

def get_pi_config(user_id: str) -> dict[str, Any] | None:
    """Return the stored Pi config metadata for a user."""
    row = _get_pi_config_row(user_id)
    if row is None:
        return None
    return _row_to_pi_config(row)

async def _finalize_github_import(
    user_id: str,
    data_dir: str,
    clone_url: str,
    source_label: str,
    access_token: str | None,
) -> None:
    """Clone, scrub, and index a GitHub-backed Pi config in the background."""
    config_root = _pi_config_root_path(data_dir)
    try:
        _prepare_destination(config_root)
        await clone_repo(clone_url, str(config_root), access_token=access_token)
        _scrub_pi_config(config_root, keep_git=True)
        _mirror_instruction_files(config_root)
        available_categories = _scan_categories(config_root)
        enabled_categories = list(available_categories)
        _extract_and_store_settings(
            user_id,
            config_root,
            settings_enabled="settings" in enabled_categories,
        )
        _update_pi_config_row(
            user_id,

```

```

        source_label=source_label,
        repo_url=clone_url,
        available_categories=_dump_categories_json(available_categories),
        enabled_categories=_dump_categories_json(enabled_categories),
        status="ready",
        error_message=None,
    )
except Exception:
    logger.error("Pi config GitHub import failed")
    _remove_path(config_root)
    clear_pi_settings(user_id)
    _update_pi_config_row(
        user_id,
        status="error",
        error_message="Import failed. Check server logs for details.",
        available_categories=_dump_categories_json([]),
        enabled_categories=_dump_categories_json([]),
    )

async def import_from_github(
    user_id: str,
    data_dir: str,
    repo_url: str,
    background_tasks: BackgroundTasks,
    access_token: str | None = None,
) -> dict[str, Any]:
    """Create a Pi config record and clone it in the background."""
    normalized_user_id = _validate_user_id(user_id)
    normalized_repo_url = repo_url.strip()
    if get_pi_config(normalized_user_id) is not None:
        raise PiConfigError("Pi config already exists")

    clone_url, resolved_access_token = await _resolve_clone_details(
        normalized_user_id,
        normalized_repo_url,
    )
    # An explicit token must win over any token resolved from the GitHub app flow.
    effective_access_token = access_token or resolved_access_token
    config = _insert_pi_config_row(
        normalized_user_id,
        source_type="github",
        source_label=normalized_repo_url,
        repo_url=clone_url,
        status="cloning",
        available_categories=[],

```

```

        enabled_categories=[],
    )
    background_tasks.add_task(
        _finalize_github_import,
        normalized_user_id,
        data_dir,
        clone_url,
        normalized_repo_url,
        effective_access_token,
    )
    return config

async def import_from_upload(
    user_id: str,
    data_dir: str,
    zip_data: bytes,
    filename: str,
) -> dict[str, Any]:
    """Import a Pi config from an uploaded archive."""
    normalized_user_id = _validate_user_id(user_id)
    if get_pi_config(normalized_user_id) is not None:
        raise PiConfigError("Pi config already exists")
    if not isinstance(zip_data, bytes):
        raise TypeError("zip_data must be bytes")
    if not isinstance(filename, str) or not filename.strip():
        raise ValueError("filename must not be empty")

    config_root = _pi_config_root_path(data_dir)
    try:
        with tempfile.TemporaryDirectory(prefix="yinshi-pi-upload-") as temp_dir_text:
            temp_dir = Path(temp_dir_text)
            _extract_archive(zip_data, temp_dir)
            extracted_root = _find_extracted_config_root(temp_dir)
            _prepare_destination(config_root)
            shutil.move(str(extracted_root), str(config_root))

            _scrub_pi_config(config_root, keep_git=False)
            _mirror_instruction_files(config_root)
            available_categories = _scan_categories(config_root)
            enabled_categories = list(available_categories)
            _extract_and_store_settings(
                normalized_user_id,
                config_root,
                settings_enabled="settings" in enabled_categories,
            )

```

```

        return _insert_pi_config_row(
            normalized_user_id,
            source_type="upload",
            source_label=filename.strip(),
            repo_url=None,
            status="ready",
            available_categories=available_categories,
            enabled_categories=enabled_categories,
        )
    except Exception:
        _remove_path(config_root)
        clear_pi_settings(normalized_user_id)
        raise

async def sync_pi_config(
    user_id: str,
    data_dir: str,
    access_token: str | None = None,
) -> dict[str, Any]:
    """Sync an existing GitHub-backed Pi config with its remote origin."""
    normalized_user_id = _validate_user_id(user_id)
    row = _require_pi_config_row(normalized_user_id)
    if row["source_type"] != "github":
        raise PiConfigError("Only GitHub-backed Pi configs can be synced")
    if row["status"] in ("cloning", "syncing"):
        raise PiConfigError(f"Pi config is currently {row['status']}")

    repo_url = row["repo_url"]
    if not isinstance(repo_url, str) or not repo_url:
        raise PiConfigError("GitHub Pi config is missing its repository URL")

    previous_available = _load_categories_json(row["available_categories"])
    previous_enabled = _load_categories_json(row["enabled_categories"])
    disabled_categories = set(previous_available) - set(previous_enabled)
    config_root = _pi_config_root_path(data_dir)

    # Atomically transition to syncing. Only one caller can win this
    # race -- the rest will see rowcount == 0 and raise.
    with get_control_db() as db:
        result = db.execute(
            "UPDATE pi_configs SET status = 'syncing', error_message = NULL "
            "WHERE user_id = ? AND status = 'ready'",
            (normalized_user_id,),
        )
    if result.rowcount == 0:

```

```

        raise PiConfigError("Pi config is not ready for sync")
    db.commit()
try:
    clone_url, resolved_access_token = await _resolve_clone_details(
        normalized_user_id, repo_url
    )
    effective_access_token = access_token or resolved_access_token
    await clone_repo(clone_url, str(config_root), access_token=effective_access_token)
    with _git_askpass_env(effective_access_token) as git_env:
        await _run_git(["reset", "--hard", "origin/HEAD"], cwd=str(config_root), env=git_env)

    _reset_runtime_artifacts_for_sync(config_root)
    _scrub_pi_config(config_root, keep_git=True)
    _mirror_instruction_files(config_root)
    available_categories = _scan_categories(config_root)
    enabled_categories = [
        category for category in available_categories if category not in disabled_categories
    ]
    _extract_and_store_settings(
        normalized_user_id,
        config_root,
        settings_enabled="settings" in enabled_categories,
    )
    _apply_enabled_categories(config_root, available_categories, enabled_categories)
    _update_pi_config_row(
        normalized_user_id,
        repo_url=clone_url,
        available_categories=_dump_categories_json(available_categories),
        enabled_categories=_dump_categories_json(enabled_categories),
        status="ready",
        error_message=None,
    )
    _set_last_synced_at_now(normalized_user_id)
    return cast(dict[str, Any], get_pi_config(normalized_user_id))
except Exception:
    logger.error("Pi config sync failed")
    _update_pi_config_row(
        normalized_user_id,
        status="error",
        error_message="Sync failed. Check server logs for details.",
    )
    raise

async def remove_pi_config(user_id: str, data_dir: str) -> None:
    """Delete the imported Pi config directory and database rows."""

```

```

        _require_pi_config_row(user_id)
        config_root = _pi_config_root_path(data_dir)
        _remove_path(config_root)
        _delete_pi_config_row(user_id)

def update_enabled_categories(
    user_id: str,
    data_dir: str,
    categories: list[str],
) -> dict[str, Any]:
    """Enable and disable discovered categories by renaming their paths."""
    normalized_user_id = _validate_user_id(user_id)
    requested_categories = set(categories)
    unknown_categories = requested_categories - PI_CONFIG_CATEGORIES
    if unknown_categories:
        raise ValueError(f"Unsupported categories: {sorted(unknown_categories)}")

    row = _require_pi_config_row(normalized_user_id)
    if row["status"] != "ready":
        raise PiConfigError("Pi config is not ready")

    config_root = _pi_config_root_path(data_dir)
    if not config_root.is_dir():
        raise PiConfigNotFoundError("Pi config directory does not exist")

    available_categories = _scan_categories(config_root)
    available_set = set(available_categories)
    if not requested_categories.issubset(available_set):
        unavailable_categories = sorted(requested_categories - available_set)
        raise PiConfigError(f"Categories not available: {unavailable_categories}")

    enabled_categories = _ordered_categories(requested_categories)
    _apply_enabled_categories(config_root, available_categories, enabled_categories)
    set_pi_settings_enabled(
        normalized_user_id,
        enabled="settings" in enabled_categories and "settings" in available_set,
    )
    _update_pi_config_row(
        normalized_user_id,
        available_categories=_dump_categories_json(available_categories),
        enabled_categories=_dump_categories_json(enabled_categories),
        error_message=None,
    )
    return cast(dict[str, Any], get_pi_config(normalized_user_id))

```



```

def resolve_pi_runtime(user_id: str, data_dir: str) -> PiRuntimeInputs:
    """Return the active Pi runtime inputs when container mode is enabled."""
    settings = get_settings()
    if not settings.container_enabled:
        return PiRuntimeInputs(agent_dir=None, settings_payload=None)

    config = get_pi_config(user_id)
    if config is None:
        return PiRuntimeInputs(agent_dir=None, settings_payload=None)
    if config["status"] != "ready":
        return PiRuntimeInputs(agent_dir=None, settings_payload=None)

    agent_dir = _pi_agent_dir_path(data_dir)
    if not agent_dir.is_dir():
        return PiRuntimeInputs(agent_dir=None, settings_payload=None)

    settings_payload = get_sidecar_settings_payload(user_id)
    return PiRuntimeInputs(
        agent_dir=str(agent_dir),
        settings_payload=settings_payload,
    )

def resolve_agent_dir(user_id: str, data_dir: str) -> str | None:
    """Return the agentDir path when a ready Pi config should be active."""
    runtime_inputs = resolve_pi_runtime(user_id, data_dir)
    return runtime_inputs.agent_dir

def resolve_effective_pi_runtime(
    user_id: str,
    data_dir: str,
    *,
    runtime_session_id: str | None = None,
    repo_agents_md: str | None = None,
) -> PiRuntimeInputs:
    """Return runtime inputs with an optional repo-level AGENTS.md overlay applied."""
    runtime_inputs = resolve_pi_runtime(user_id, data_dir)

    if repo_agents_md is None:
        return runtime_inputs

    if runtime_session_id is None:
        raise ValueError("runtime_session_id must be provided when repo_agents_md is set")

```

```

override_agent_dir = _materialize_repo_agent_override(
    data_dir,
    runtime_session_id,
    repo_agents_md,
    runtime_inputs.agent_dir,
)
return PiRuntimeInputs(
    agent_dir=override_agent_dir,
    settings_payload=runtime_inputs.settings_payload,
)

```

11.4 services/user_settings.py

User settings decide which imported Pi settings are enabled and which sanitized payload reaches the sidecar. The root chunk below tangles back to `backend/src/yinshi/services/user_settings.py`.

```

"""Persist imported Pi settings and build safe sidecar settings payloads."""

from __future__ import annotations

import json
import sqlite3
from typing import cast

from yinshi.db import get_control_db
from yinshi.services.control_encryption import decrypt_control_text, encrypt_control_text

_BLOCKED_SETTING_KEYS = frozenset(
    {
        "packages",
        "extensions",
        "skills",
        "prompts",
        "themes",
        "enabledModels",
    }
)

def _validate_user_id(user_id: str) -> str:
    """Require a non-empty user identifier."""
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    normalized_user_id = user_id.strip()
    if not normalized_user_id:

```

```

        raise ValueError("user_id must not be empty")
    return normalized_user_id

def _decode_settings_json(user_id: str, settings_json: str) -> dict[str, object]:
    """Decode a stored settings JSON object."""
    if not isinstance(settings_json, str):
        raise TypeError("settings_json must be a string")
    decrypted_settings_json = decrypt_control_text(
        "user_settings.pi_settings_json",
        user_id,
        settings_json,
    )
    if decrypted_settings_json is None:
        raise ValueError("Stored Pi settings must not be NULL")
    payload = json.loads(decrypted_settings_json)
    if not isinstance(payload, dict):
        raise ValueError("Stored Pi settings must decode to an object")
    normalized_payload = {str(key): value for key, value in payload.items()}
    return cast(dict[str, object], normalized_payload)

def _sanitize_pi_settings(settings_payload: dict[str, object]) -> dict[str, object]:
    """Drop resource-loading settings that would bypass agentDir controls."""
    if not isinstance(settings_payload, dict):
        raise TypeError("settings_payload must be a dictionary")
    sanitized_payload: dict[str, object] = {}
    for key, value in settings_payload.items():
        if not isinstance(key, str):
            raise TypeError("settings_payload keys must be strings")
        if key in _BLOCKED_SETTING_KEYS:
            continue
        sanitized_payload[key] = value
    return sanitized_payload

def _upsert_settings_row(
    user_id: str,
    settings_payload: dict[str, object],
    *,
    enabled: bool,
) -> None:
    """Insert or update the persisted settings row for a user."""
    normalized_user_id = _validate_user_id(user_id)
    sanitized_payload = _sanitize_pi_settings(settings_payload)
    serialized_payload = json.dumps(sanitized_payload, sort_keys=True)

```

```

stored_payload = encrypt_control_text(
    "user_settings.pi_settings_json",
    normalized_user_id,
    serialized_payload,
)
assert stored_payload is not None, "serialized settings payload must not encrypt to NULL"

with get_control_db() as db:
    db.execute(
        "INSERT INTO user_settings (user_id, pi_settings_json, pi_settings_enabled) "
        "VALUES (?, ?, ?) "
        "ON CONFLICT(user_id) DO UPDATE SET "
        "pi_settings_json = excluded.pi_settings_json, "
        "pi_settings_enabled = excluded.pi_settings_enabled",
        (normalized_user_id, stored_payload, int(enabled)),
    )
    db.commit()

def store_pi_settings(user_id: str, settings_payload: dict[str, object]) -> None:
    """Persist imported Pi settings with the enabled state turned on."""
    _upsert_settings_row(user_id, settings_payload, enabled=True)

def clear_pi_settings(user_id: str) -> None:
    """Clear stored Pi settings and disable forwarding."""
    _upsert_settings_row(user_id, {}, enabled=False)

def get_pi_settings(user_id: str) -> dict[str, object] | None:
    """Return the stored Pi settings payload for a user, if any."""
    normalized_user_id = _validate_user_id(user_id)
    with get_control_db() as db:
        row = cast(
            sqlite3.Row | None,
            db.execute(
                "SELECT pi_settings_json FROM user_settings WHERE user_id = ?",
                (normalized_user_id,),
            ).fetchone(),
        )
    if row is None:
        return None
    return _decode_settings_json(normalized_user_id, row["pi_settings_json"])

def set_pi_settings_enabled(user_id: str, enabled: bool) -> None:

```

```

        """Enable or disable forwarding of imported Pi settings."""
    normalized_user_id = _validate_user_id(user_id)
    if not isinstance(enabled, bool):
        raise TypeError("enabled must be a boolean")

    with get_control_db() as db:
        if enabled:
            db.execute(
                "INSERT INTO user_settings (user_id, pi_settings_json, pi_settings_enabled) "
                "VALUES (?, ?, ?) "
                "ON CONFLICT(user_id) DO UPDATE SET "
                "pi_settings_enabled = excluded.pi_settings_enabled",
                (
                    normalized_user_id,
                    encrypt_control_text(
                        "user_settings.pi_settings_json", normalized_user_id, "{}"
                    ),
                    1,
                ),
            )
        else:
            db.execute(
                "UPDATE user_settings SET pi_settings_enabled = 0 WHERE user_id = ?",
                (normalized_user_id,),
            )
        db.commit()

def get_sidecar_settings_payload(user_id: str) -> dict[str, object] | None:
    """Return the enabled, sidecar-safe settings payload for a user."""
    normalized_user_id = _validate_user_id(user_id)
    with get_control_db() as db:
        row = cast(
            sqlite3.Row | None,
            db.execute(
                "SELECT pi_settings_json, pi_settings_enabled "
                "FROM user_settings WHERE user_id = ?",
                (normalized_user_id,),
            ).fetchone(),
        )
    if row is None:
        return None
    if not row["pi_settings_enabled"]:
        return None

    settings_payload = _decode_settings_json(normalized_user_id, row["pi_settings_json"])

```

```

if not settings_payload:
    return None
return settings_payload

```

11.5 services/pi_releases.py

Release helpers combine installed runtime information with GitHub release notes for the settings page. The root chunk below tangles back to backend/src/yinshi/services/pi_releases.py.

```

"""Release-note and runtime-version helpers for the bundled pi package."""

from __future__ import annotations

import asyncio
import json
import logging
import time
from typing import Any

import httpx

from yinshi.config import get_settings
from yinshi.exceptions import SidecarError
from yinshi.services.sidecar import (
    PiRuntimeVersionPayload,
    SidecarClient,
    create_sidecar_connection,
)

logger = logging.getLogger(__name__)

_RELEASES_PER_PAGE = 8
_RELEASE_BODY_LENGTH_MAX = 12_000
_RELEASE_CACHE_TTL_S = 300.0
_SIDECAR_VERSION_TIMEOUT_S = 5.0
_GITHUB_API_BASE_URL = "https://api.github.com"
_RUNTIME_VERSION_ERROR = "Failed to read sidecar pi package version"
_DISCONNECT_ERROR = "sidecar disconnect failed after version request"
_UPDATE_POLICY = "Updated through reviewed lockfile deployments"

_release_cache_lock = asyncio.Lock()
_release_cache_payload: dict[str, Any] | None = None
_release_cache_expires_at = 0.0

```

```

def _string_or_none(value: Any) -> str | None:
    """Return a non-empty string value or ``None`` for absent metadata."""
    if isinstance(value, str):
        normalized_value = value.strip()
        if normalized_value:
            return normalized_value
    return None

def _normalize_release_version(tag_name: str) -> str:
    """Convert GitHub release tags such as ``v0.70.2`` to package versions."""
    if not isinstance(tag_name, str):
        raise TypeError("tag_name must be a string")
    normalized_tag = tag_name.strip()
    if not normalized_tag:
        raise ValueError("tag_name must not be empty")
    if normalized_tag.startswith("v"):
        return normalized_tag[1:]
    return normalized_tag

def _truncate_release_body(body: str) -> str:
    """Cap release body size so one note cannot bloat Settings JSON."""
    if not isinstance(body, str):
        raise TypeError("body must be a string")
    if len(body) <= _RELEASE_BODY_LENGTH_MAX:
        return body
    return f"{body[:_RELEASE_BODY_LENGTH_MAX]}\n\n[Release notes truncated.]"

def _normalize_github_release(raw_release: Any) -> dict[str, Any]:
    """Validate and normalize one GitHub release object."""
    if not isinstance(raw_release, dict):
        raise TypeError("GitHub release must be an object")

    tag_name = _string_or_none(raw_release.get("tag_name"))
    html_url = _string_or_none(raw_release.get("html_url"))
    if tag_name is None:
        raise ValueError("GitHub release is missing tag_name")
    if html_url is None:
        raise ValueError("GitHub release is missing html_url")

    name = _string_or_none(raw_release.get("name")) or tag_name
    body = raw_release.get("body")
    body_text = body if isinstance(body, str) else ""
    body_markdown = _truncate_release_body(body_text)

```

```

return {
    "tag_name": tag_name,
    "version": _normalize_release_version(tag_name),
    "name": name,
    "published_at": _string_or_none(raw_release.get("published_at")),
    "html_url": html_url,
    "body_markdown": body_markdown,
}

def _normalize_repository(repository: str) -> str:
    """Reject malformed GitHub repository strings."""
    if not isinstance(repository, str):
        raise TypeError("repository must be a string")
    normalized_repository = repository.strip()
    if normalized_repository.count("/") != 1:
        raise ValueError("repository must be in owner/name form")
    owner, name = normalized_repository.split("/", maxsplit=1)
    if not owner:
        raise ValueError("repository owner must not be empty")
    if not name:
        raise ValueError("repository name must not be empty")
    return normalized_repository

async def _fetch_github_releases(repository: str) -> dict[str, Any]:
    """Fetch recent release notes from GitHub's releases API."""
    normalized_repository = _normalize_repository(repository)
    url = f"{_GITHUB_API_BASE_URL}/repos/{normalized_repository}/releases"
    timeout = httpx.Timeout(timeout=5.0, connect=2.0)
    headers = {
        "Accept": "application/vnd.github+json",
        "User-Agent": "yinshi-pi-release-notes",
    }
    async with httpx.AsyncClient(
        follow_redirects=True,
        headers=headers,
        timeout=timeout,
    ) as client:
        response = await client.get(
            url,
            params={"per_page": _RELEASES_PER_PAGE},
        )
        response.raise_for_status()
        payload = response.json()

```



```

if not isinstance(payload, list):
    raise ValueError("GitHub releases response must be a list")

releases: list[dict[str, Any]] = []
for raw_release in payload:
    try:
        releases.append(_normalize_github_release(raw_release))
    except (TypeError, ValueError):
        logger.warning("Skipping malformed GitHub release")

latest_version = releases[0]["version"] if releases else None
return {"latest_version": latest_version, "releases": releases}

async def _get_cached_github_releases(
    repository: str,
) -> tuple[dict[str, Any], str | None]:
    """Return cached releases and preserve stale data after failures."""
    global _release_cache_expires_at, _release_cache_payload

    now = time.monotonic()
    async with _release_cache_lock:
        if _release_cache_payload is not None:
            if now < _release_cache_expires_at:
                return _release_cache_payload, None

        try:
            fetched_payload = await _fetch_github_releases(repository)
        except (httpx.HTTPError, TypeError, ValueError) as error:
            logger.warning("Failed to fetch pi release notes")
            if _release_cache_payload is not None:
                return _release_cache_payload, str(error)
            return {"latest_version": None, "releases": []}, str(error)

        _release_cache_payload = fetched_payload
        _release_cache_expires_at = time.monotonic() + _RELEASE_CACHE_TTL_S
        return fetched_payload, None

async def _read_runtime_version() -> tuple[PiRuntimeVersionPayload | None, str | None]:
    """Ask the live sidecar which pi package version it has loaded."""
    sidecar: SidecarClient | None = None
    try:
        sidecar = await asyncio.wait_for(
            create_sidecar_connection(),
            timeout=_SIDECAR_VERSION_TIMEOUT_S,

```

```

    )
    runtime_version = await asyncio.wait_for(
        sidecar.get_runtime_version(),
        timeout=_SIDECAR_VERSION_TIMEOUT_S,
    )
    return runtime_version, None
except (
    OSError,
    SidecarError,
    json.JSONDecodeError,
    asyncio.TimeoutError,
) as error:
    logger.warning(_RUNTIME_VERSION_ERROR)
    return None, str(error)
finally:
    if sidecar is not None:
        try:
            await sidecar.disconnect()
        except OSError:
            logger.error(_DISCONNECT_ERROR)

async def get_pi_release_notes() -> dict[str, Any]:
    """Return runtime pi package metadata and recent upstream release notes."""
    settings = get_settings()
    runtime_version, runtime_error = await _read_runtime_version()
    release_payload, release_error = await _get_cached_github_releases(
        settings.pi_release_repository
    )
    installed_version = None
    node_version = None
    package_name = settings.pi_package_name
    if runtime_version is not None:
        installed_version = runtime_version.get("installed_version")
        node_version = runtime_version.get("node_version")
        package_name = runtime_version.get("package_name") or package_name

    release_notes_url = "/".join(
        (
            "https://github.com",
            settings.pi_release_repository,
            "releases",
        )
    )
    return {
        "package_name": package_name,

```

```

        "installed_version": installed_version,
        "latest_version": release_payload.get("latest_version"),
        "node_version": node_version,
        "release_notes_url": release_notes_url,
        "update_policy": _UPDATE_POLICY,
        "runtime_error": runtime_error,
        "release_error": release_error,
        "releases": release_payload.get("releases", []),
    }

```

11.6 api/catalog.py

Catalog routes ask the sidecar for available providers and fall back to backend metadata when needed. The root chunk below tangles back to backend/src/yinshi/api/catalog.py.

```

"""Catalog endpoints for provider and model discovery."""

from __future__ import annotations

from typing import Any

from fastapi import APIRouter, HTTPException, Request

from yinshi.api.deps import require_tenant
from yinshi.exceptions import ContainerNotReadyError, ContainerStartError, SidecarError
from yinshi.model_catalog import get_provider_metadata
from yinshi.models import ProviderCatalogOut
from yinshi.services.pi_config import resolve_effective_pi_runtime
from yinshi.services.provider_connections import list_provider_connections
from yinshi.services.sidecar import create_sidecar_connection
from yinshi.services.sidecar_runtime import (
    resolve_tenant_sidecar_context,
    tenant_container_activity,
    touch_tenant_container,
)
from yinshi.tenant import TenantContext

router = APIRouter(tags=["catalog"])

def _build_provider_entry(
    provider_row: dict[str, Any],
    connected_provider_ids: set[str],
) -> dict[str, Any]:
    """Merge sidecar provider rows with Yinshi metadata."""

```

```

provider_id = provider_row["id"]
metadata = get_provider_metadata(provider_id)
return {
    "id": provider_id,
    "label": metadata.label,
    "auth_strategies": list(metadata.auth_strategies),
    "setup_fields": [
        {
            "key": field.key,
            "label": field.label,
            "required": field.required,
            "secret": field.secret,
        }
        for field in metadata.setup_fields
    ],
    "docs_url": metadata.docs_url,
    "connected": provider_id in connected_provider_ids,
    "model_count": provider_row["model_count"],
}

async def _load_catalog_from_sidecar(
    *,
    socket_path: str | None,
    agent_dir: str | None,
) -> dict[str, Any]:
    """Load model catalog data from one sidecar socket."""
    sidecar = await create_sidecar_connection(socket_path)
    try:
        return await sidecar.get_catalog(agent_dir=agent_dir)
    finally:
        await sidecar.disconnect()

async def _load_catalog_with_tenant_container(
    request: Request,
    tenant: TenantContext,
) -> dict[str, Any]:
    """Fall back to the tenant container when no host sidecar is available."""
    try:
        tenant_sidecar_context = await resolve_tenant_sidecar_context(request, tenant)
    except (ContainerStartError, ContainerNotReadyError) as error:
        raise HTTPException(
            status_code=503,
            detail="Agent environment temporarily unavailable",
        ) from error

```

```

try:
    async with tenant_container_activity(request, tenant):
        return await _load_catalog_from_sidecar(
            socket_path=tenant_sidecar_context.socket_path,
            agent_dir=tenant_sidecar_context.agent_dir,
        )
except (
    ContainerNotReadyError,
    ContainerStartError,
    OSError,
    SidecarError,
) as error:
    raise HTTPException(
        status_code=503,
        detail="Agent environment temporarily unavailable",
    ) from error
finally:
    touch_tenant_container(request, tenant)

@router.get("/api/catalog", response_model=ProviderCatalogOut)
async def get_catalog(request: Request) -> dict[str, Any]:
    """Return the current user's provider/model catalog."""
    tenant = require_tenant(request)
    connections = list_provider_connections(tenant.user_id)
    connected_provider_ids = {connection["provider"] for connection in connections}
    runtime_inputs = resolve_effective_pi_runtime(tenant.user_id, tenant.data_dir)

    try:
        # Catalog construction reads model metadata only. Use the persistent host
        # sidecar so settings/model dropdowns do not wait for a tenant execution
        # container to cold-start. Execution paths still use tenant containers.
        catalog = await _load_catalog_from_sidecar(
            socket_path=None,
            agent_dir=runtime_inputs.agent_dir,
        )
    except (OSError, SidecarError):
        catalog = await _load_catalog_with_tenant_container(request, tenant)

    supported_provider_ids = {
        provider_row["id"]
        for provider_row in catalog["providers"]
        if get_provider_metadata(provider_row["id"]).supported
    }
    providers = [

```

```

        _build_provider_entry(provider_row, connected_provider_ids)
        for provider_row in catalog["providers"]
        if provider_row["id"] in supported_provider_ids
    ]
    models = [
        model_row
        for model_row in catalog["models"]
        if model_row["provider"] in supported_provider_ids
    ]
    return {
        "default_model": catalog["default_model"],
        "providers": providers,
        "models": models,
    }

```

12 GitHub App Integration

GitHub access is intentionally installation-scoped. OAuth identifies the user, while the GitHub App grants repository clone access through installation tokens that can be refreshed and revoked independently of Yinshi sessions.

12.1 api/github.py

GitHub status routes expose App installation information for connected users. The root chunk below tangles back to `backend/src/yinshi/api/github.py`.

```

"""GitHub App status endpoints."""

from typing import Any

from fastapi import APIRouter, Request

from yinshi.api.deps import get_tenant
from yinshi.services.github_app import list_user_installations

router = APIRouter(prefix="/api/github", tags=["github"])

@router.get("/installations")
def list_installations(request: Request) -> list[dict[str, Any]]:
    """Return the saved GitHub installations for the current user."""
    tenant = get_tenant(request)
    if tenant is None:
        return []

    installations = list_user_installations(tenant.user_id)

```

```

    return [
        {
            "installation_id": installation.installation_id,
            "account_login": installation.account_login,
            "account_type": installation.account_type,
            "html_url": installation.html_url,
        }
        for installation in installations
    ]

```

12.2 services/github_app.py

GitHub App helpers sign app JWTs, find usable installations, cache installation tokens, and build authenticated clone URLs. The root chunk below tangles back to `backend/src/yinshi/services/github_app.py`.

```

"""GitHub App helpers for installation lookup and authenticated clone access."""

from __future__ import annotations

import logging
import re
import time
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any
from urllib.parse import urlparse

import httpx
from authlib.jose import jwt

from yinshi.config import get_settings
from yinshi.db import get_control_db
from yinshi.exceptions import (
    GitHubAppError,
    GitHubInstallationUnusableError,
)

logger = logging.getLogger(__name__)

_GITHUB_API_BASE_URL = "https://api.github.com"
_GITHUB_API_TIMEOUT_S = 15.0
_GITHUB_API_VERSION = "2022-11-28"
_INSTALLATION_REFRESH_WINDOW_S = 300
_GITHUB_SHORTHAND_RE = re.compile(

```

```

        r"^(?P<owner>[A-Za-z0-9](?:[A-Za-z0-9-]{0,38}))/" r"^(?P<repo>[A-Za-z0-9._-]+?)(?:\.git)"
    )
    _GITHUB_SCP_RE = re.compile(r"^git@github\.com:(?P<owner>[^\s]+)/(?P<repo>[^\s]+?)(?:\.git)"

    @dataclass(frozen=True)
    class GitHubRemote:
        """A normalized GitHub repository reference."""

        owner: str
        repo: str
        clone_url: str

    @dataclass(frozen=True)
    class GitHubInstallation:
        """A saved GitHub installation associated with a Yinshi user."""

        installation_id: int
        account_login: str
        account_type: str
        html_url: str

    @dataclass(frozen=True)
    class GitHubCloneAccess:
        """The clone URL and optional credentials for a GitHub repository."""

        clone_url: str
        repository_installation_id: int | None
        installation_id: int | None
        access_token: str | None
        manage_url: str | None

    @dataclass(frozen=True)
    class _CachedInstallationToken:
        """A cached installation token plus its expiry timestamp."""

        token: str
        expires_at_epoch: float

    _INSTALLATION_TOKEN_CACHE: dict[tuple[int, str], _CachedInstallationToken] = {}

```



```

def _build_clone_url(owner: str, repo: str) -> str:
    """Build the canonical HTTPS clone URL for a GitHub repo."""
    assert owner, "owner must not be empty"
    assert repo, "repo must not be empty"
    return f"https://github.com/{owner}/{repo}.git"

def _strip_dot_git(repo: str) -> str:
    """Remove a trailing .git suffix from a repository name."""
    assert repo, "repo must not be empty"
    if repo.endswith(".git"):
        return repo[:-4]
    return repo

def normalize_github_remote(value: str) -> GitHubRemote | None:
    """Normalize supported GitHub inputs to a canonical HTTPS remote."""
    assert value is not None, "value must not be None"

    candidate = value.strip()
    if not candidate:
        return None

    shorthand_match = _GITHUB_SHORTHAND_RE.fullmatch(candidate)
    if shorthand_match is not None:
        owner = shorthand_match.group("owner")
        repo = _strip_dot_git(shorthand_match.group("repo"))
        return GitHubRemote(owner=owner, repo=repo, clone_url=_build_clone_url(owner, repo))

    scp_match = _GITHUB_SCP_RE.fullmatch(candidate)
    if scp_match is not None:
        owner = scp_match.group("owner")
        repo = _strip_dot_git(scp_match.group("repo"))
        return GitHubRemote(owner=owner, repo=repo, clone_url=_build_clone_url(owner, repo))

    parsed = urlparse(candidate)
    if parsed.scheme == "https":
        if parsed.netloc.lower() != "github.com":
            return None
        parts = [part for part in parsed.path.split("/") if part]
        if len(parts) != 2:
            return None
        owner = parts[0]
        repo = _strip_dot_git(parts[1])
        if not owner or not repo:
            return None

```

```

        return GitHubRemote(owner=owner, repo=repo, clone_url=_build_clone_url(owner, repo))

if parsed.scheme == "ssh":
    if parsed.netloc.lower() != "git@github.com":
        return None
    parts = [part for part in parsed.path.split("/") if part]
    if len(parts) != 2:
        return None
    owner = parts[0]
    repo = _strip_dot_git(parts[1])
    if not owner or not repo:
        return None
    return GitHubRemote(owner=owner, repo=repo, clone_url=_build_clone_url(owner, repo))

return None

def _github_app_is_configured() -> bool:
    """Return whether the server has GitHub App settings configured."""
    settings = get_settings()
    if not settings.github_app_id:
        return False
    if not settings.github_app_private_key_path:
        return False
    if not settings.github_app_slug:
        return False
    return True

def _load_private_key_pem() -> str:
    """Load the configured GitHub App private key from disk."""
    settings = get_settings()
    if not settings.github_app_id:
        raise GitHubAppError("GitHub App ID is not configured")
    if not settings.github_app_private_key_path:
        raise GitHubAppError("GitHub App private key path is not configured")

    private_key_path = Path(settings.github_app_private_key_path)
    if not private_key_path.is_file():
        raise GitHubAppError("GitHub App private key file does not exist")

    private_key_pem = private_key_path.read_text(encoding="utf-8")
    if not private_key_pem.strip():
        raise GitHubAppError("GitHub App private key file is empty")
    return private_key_pem

```

```

def generate_app_jwt() -> str:
    """Generate a short-lived JWT for GitHub App API requests."""
    settings = get_settings()
    if not _github_app_is_configured():
        raise GitHubAppError("GitHub App is not fully configured")

    private_key_pem = _load_private_key_pem()
    issued_at_epoch = int(time.time()) - 60
    expires_at_epoch = issued_at_epoch + 540
    header = {"alg": "RS256", "typ": "JWT"}
    claims = {
        "iat": issued_at_epoch,
        "exp": expires_at_epoch,
        "iss": settings.github_app_id,
    }
    token = jwt.encode(header, claims, private_key_pem)
    if isinstance(token, bytes):
        return token.decode("utf-8")
    return str(token)

def _github_headers(bearer_token: str) -> dict[str, str]:
    """Build standard headers for GitHub API requests."""
    assert bearer_token, "bearer_token must not be empty"
    return {
        "Accept": "application/vnd.github+json",
        "Authorization": f"Bearer {bearer_token}",
        "X-GitHub-API-Version": _GITHUB_API_VERSION,
    }

async def _request_github_json(
    method: str,
    path: str,
    *,
    bearer_token: str,
    json_payload: dict[str, Any] | None = None,
) -> tuple[int, dict[str, Any]]:
    """Issue a GitHub API request and return status plus parsed JSON."""
    assert method, "method must not be empty"
    assert path.startswith("/"), "path must start with /"

    try:
        async with httpx.AsyncClient(timeout=_GITHUB_API_TIMEOUT_S) as client:
            request_arguments: dict[str, Any] = {

```

```

        "method": method,
        "url": f"{_GITHUB_API_BASE_URL}{path}",
        "headers": _github_headers(bearer_token),
    }
    if json_payload is not None:
        request_arguments["json"] = json_payload
        response = await client.request(**request_arguments)
    except httpx.HTTPError as exc:
        logger.error("GitHub API request failed for method %s", method)
        raise GitHubAppError("GitHub API request failed") from exc
    if response.status_code == 204:
        return response.status_code, {}
    try:
        payload = response.json()
    except ValueError as exc:
        raise GitHubAppError("GitHub API returned invalid JSON") from exc
    return response.status_code, payload

def _parse_github_timestamp(timestamp_text: str) -> float:
    """Parse a GitHub ISO-8601 timestamp into epoch seconds."""
    assert timestamp_text, "timestamp_text must not be empty"
    parsed = datetime.fromisoformat(timestamp_text.replace("Z", "+00:00"))
    if parsed.tzinfo is None:
        parsed = parsed.replace(tzinfo=timezone.utc)
    return parsed.timestamp()

async def get_installation_details(installation_id: int) -> dict[str, Any]:
    """Fetch account details for a GitHub App installation."""
    assert installation_id > 0, "installation_id must be positive"

    app_jwt = generate_app_jwt()
    status_code, payload = await _request_github_json(
        "GET",
        f"/app/installations/{installation_id}",
        bearer_token=app_jwt,
    )
    if status_code == 404:
        raise GitHubInstallationUnusableError("The GitHub installation is no longer available")
    if status_code >= 400:
        raise GitHubAppError("Failed to fetch GitHub installation details")
    return payload

async def get_repo_installation(owner: str, repo: str) -> int | None:

```

```

"""Return the GitHub App installation id for a repo, if present."""
assert owner, "owner must not be empty"
assert repo, "repo must not be empty"

app_jwt = generate_app_jwt()
status_code, payload = await _request_github_json(
    "GET",
    f"/repos/{owner}/{repo}/installation",
    bearer_token=app_jwt,
)
if status_code == 404:
    return None
if status_code >= 400:
    raise GitHubAppError("Failed to look up the repository installation")
installation_id = payload.get("id")
if not isinstance(installation_id, int):
    raise GitHubAppError("GitHub installation response is missing id")
return installation_id

async def get_installation_token(installation_id: int, repository_name: str) -> str:
    """Mint or reuse a token restricted to one GitHub repository."""
    if installation_id <= 0:
        raise ValueError("installation_id must be positive")
    normalized_repository_name = repository_name.strip()
    if not normalized_repository_name:
        raise ValueError("repository_name must not be empty")
    if "/" in normalized_repository_name or "\\\" in normalized_repository_name:
        raise ValueError("repository_name must not contain a path separator")

    cache_key = (installation_id, normalized_repository_name.lower())
    cached_entry = _INSTALLATION_TOKEN_CACHE.get(cache_key)
    if cached_entry is not None:
        refresh_before_epoch = cached_entry.expires_at_epoch - _INSTALLATION_REFRESH_WINDOW
        if time.time() < refresh_before_epoch:
            return cached_entry.token

    app_jwt = generate_app_jwt()
    status_code, payload = await _request_github_json(
        "POST",
        f"/app/installations/{installation_id}/access_tokens",
        bearer_token=app_jwt,
        json_payload={
            "repositories": [normalized_repository_name],
            "permissions": {"contents": "write"},
        },
    ),

```

```

    )
    if status_code in (403, 404):
        raise GitHubInstallationUnusableError(
            "The connected GitHub installation is no longer usable."
        )
    if status_code >= 400:
        raise GitHubAppError("Failed to mint a GitHub installation token")

    access_token = payload.get("token")
    expires_at_text = payload.get("expires_at")
    if not isinstance(access_token, str):
        raise GitHubAppError("GitHub installation token response is missing token")
    if not isinstance(expires_at_text, str):
        raise GitHubAppError("GitHub installation token response is missing expiry")

    expires_at_epoch = _parse_github_timestamp(expires_at_text)
    _INSTALLATION_TOKEN_CACHE[cache_key] = _CachedInstallationToken(
        token=access_token,
        expires_at_epoch=expires_at_epoch,
    )
    return access_token

def list_user_installations(user_id: str) -> list[GitHubInstallation]:
    """Return the saved GitHub installations for a Yinshi user."""
    assert user_id, "user_id must not be empty"

    with get_control_db() as db:
        rows = db.execute(
            "SELECT installation_id, account_login, account_type, html_url "
            "FROM github_installations WHERE user_id = ? ORDER BY account_login ASC",
            (user_id,),
        ).fetchall()

    installations: list[GitHubInstallation] = []
    for row in rows:
        installations.append(
            GitHubInstallation(
                installation_id=row["installation_id"],
                account_login=row["account_login"],
                account_type=row["account_type"],
                html_url=row["html_url"],
            )
        )
    return installations

```

```

def _find_user_installation(
    user_id: str,
    installation_id: int,
) -> GitHubInstallation | None:
    """Return a user's saved installation row, if present."""
    assert user_id, "user_id must not be empty"
    assert installation_id > 0, "installation_id must be positive"

    with get_control_db() as db:
        row = db.execute(
            "SELECT installation_id, account_login, account_type, html_url "
            "FROM github_installations WHERE user_id = ? AND installation_id = ?",
            (user_id, installation_id),
        ).fetchone()
    if row is None:
        return None
    return GitHubInstallation(
        installation_id=row["installation_id"],
        account_login=row["account_login"],
        account_type=row["account_type"],
        html_url=row["html_url"],
    )

def _find_installation_manage_url_for_owner(
    user_id: str,
    owner_login: str,
) -> str | None:
    """Return the manage URL for an installation whose account matches the owner."""
    assert user_id, "user_id must not be empty"
    assert owner_login, "owner_login must not be empty"

    owner_login_lower = owner_login.lower()
    for installation in list_user_installations(user_id):
        if installation.account_login.lower() == owner_login_lower:
            return installation.html_url
    return None

async def resolve_github_clone_access(
    user_id: str | None,
    remote_url: str,
) -> GitHubCloneAccess | None:
    """Resolve the canonical URL plus any GitHub App credentials for a remote."""
    assert remote_url, "remote_url must not be empty"

```

```

github_remote = normalize_github_remote(remote_url)
if github_remote is None:
    return None

if not _github_app_is_configured():
    return GitHubCloneAccess(
        clone_url=github_remote.clone_url,
        repository_installation_id=None,
        installation_id=None,
        access_token=None,
        manage_url=None,
    )

installation_id = await get_repo_installation(github_remote.owner, github_remote.repo)
if installation_id is None:
    manage_url = None
    if user_id:
        manage_url = _find_installation_manage_url_for_owner(user_id, github_remote.owner)
    return GitHubCloneAccess(
        clone_url=github_remote.clone_url,
        repository_installation_id=None,
        installation_id=None,
        access_token=None,
        manage_url=manage_url,
    )

# A repo can have an app installation and still be public. Callers probe
# anonymous clone first and only gate when that clone actually fails.
if not user_id:
    return GitHubCloneAccess(
        clone_url=github_remote.clone_url,
        repository_installation_id=installation_id,
        installation_id=None,
        access_token=None,
        manage_url=None,
    )

installation = _find_user_installation(user_id, installation_id)
if installation is None:
    return GitHubCloneAccess(
        clone_url=github_remote.clone_url,
        repository_installation_id=installation_id,
        installation_id=None,
        access_token=None,
        manage_url=None,
    )

```



```

    )

    try:
        access_token = await get_installation_token(installation_id, github_remote.repo)
    except GitHubInstallationUnusableError as exc:
        raise GitHubInstallationUnusableError(
            str(exc),
            manage_url=installation.html_url,
        ) from exc
    return GitHubCloneAccess(
        clone_url=github_remote.clone_url,
        repository_installation_id=installation_id,
        installation_id=installation_id,
        access_token=access_token,
        manage_url=installation.html_url,
    )

async def resolve_github_runtime_access_token(
    user_id: str,
    remote_url: str,
    installation_id: int | None,
) -> str | None:
    """Resolve a short-lived GitHub App token for runtime git operations.

    This helper returns ``None`` when token refresh is unavailable so prompt
    execution can continue even if git push/fetch credentials are temporarily
    missing.
    """
    assert user_id, "user_id must not be empty"
    assert remote_url, "remote_url must not be empty"

    github_remote = normalize_github_remote(remote_url)
    if github_remote is None:
        return None
    if not _github_app_is_configured():
        return None

    if installation_id is not None:
        installation = _find_user_installation(user_id, installation_id)
        if installation is None:
            return None
        try:
            return await get_installation_token(installation_id, github_remote.repo)
        except (GitHubAppError, GitHubInstallationUnusableError):
            logger.warning("Failed to authorize a runtime Git operation")

```

```

        return None

    try:
        clone_access = await resolve_github_clone_access(user_id, remote_url)
    except (GitHubAppError, GitHubInstallationUnusableError):
        logger.warning("Failed to resolve runtime GitHub clone access")
        return None
    if clone_access is None:
        return None
    return clone_access.access_token

```

13 Accounts, Runners, and Observability

The remaining backend modules cover account provisioning, bring-your-own cloud runners, and telemetry intake. They are operational support code, but each still follows the same tenant and validation boundaries as the core workspace flow.

13.1 services/accounts.py

Account services provision user records, migrate legacy data, create tenant directories, and store encrypted data keys. The root chunk below tangles back to `backend/src/yinshi/services/accounts.py`.

```

"""Account provisioning and resolution for multi-tenant users."""

import json
import logging
import os
import secrets
import sqlite3
from typing import Any, cast

from yinshi.config import get_settings
from yinshi.db import get_control_db
from yinshi.services.crypto import generate_dek
from yinshi.services.keys import wrap_new_user_dek
from yinshi.tenant import TenantContext, get_user_db, init_user_db, user_data_dir

logger = logging.getLogger(__name__)

def make_tenant(user_id: str, email: str) -> TenantContext:
    """Build a TenantContext from user_id and email."""
    settings = get_settings()
    data_dir = user_data_dir(settings.user_data_dir, user_id)

```

```

    return TenantContext(
        user_id=user_id,
        email=email,
        data_dir=data_dir,
        db_path=os.path.join(data_dir, "yinshi.db"),
    )

def provision_user(user_id: str, email: str) -> TenantContext:
    """Create the data directory and initialize the user's database."""
    tenant = make_tenant(user_id, email)
    repos_dir = os.path.join(tenant.data_dir, "repos")
    os.makedirs(repos_dir, exist_ok=True)
    init_user_db(tenant.db_path, tenant=tenant)

    logger.info("Provisioned account storage")
    return tenant

def _ensure_tenant_provisioned(user_id: str, email: str) -> TenantContext:
    """Return one tenant context backed by an initialized user database."""
    tenant = make_tenant(user_id, email)
    if os.path.isfile(tenant.db_path):
        return tenant
    return provision_user(user_id, email)

def _migrate_legacy_data(tenant: TenantContext) -> None:
    """Copy repos/workspaces/sessions/messages from the legacy DB to the user's DB.

    Runs once on first login. Skips silently if no legacy DB exists or if the
    user has no data in it.
    """
    settings = get_settings()
    legacy_path = settings.db_path
    if not os.path.exists(legacy_path):
        return

    try:
        source = sqlite3.connect(legacy_path)
        source.row_factory = sqlite3.Row
    except sqlite3.Error:
        logger.warning("Could not open legacy account database")
        return

    try:

```

```

repos = source.execute(
    "SELECT * FROM repos WHERE owner_email = ?",
    (tenant.email,),
).fetchall()

if not repos:
    return

with get_user_db(tenant) as dest:
    for repo in repos:
        r = dict(repo)
        dest.execute(
            (
                "INSERT OR IGNORE INTO repos "
                "(id, created_at, updated_at, name, remote_url, root_path, custom_prompt, agents_md, installation_id)"
                "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"
            ),
            (
                r["id"],
                r["created_at"],
                r["updated_at"],
                r["name"],
                r["remote_url"],
                r["root_path"],
                r.get("custom_prompt"),
                r.get("agents_md"),
                r.get("installation_id"),
            ),
        )

    for ws in source.execute(
        "SELECT * FROM workspaces WHERE repo_id = ?", (r["id"],)
    ).fetchall():
        w = dict(ws)
        dest.execute(
            (
                "INSERT OR IGNORE INTO workspaces "
                "(id, created_at, updated_at, repo_id, name, branch, path, state)"
                "VALUES (?, ?, ?, ?, ?, ?, ?, ?)"
            ),
            (
                w["id"],
                w["created_at"],
                w["updated_at"],
                w["repo_id"],
                w["name"],
            ),
        )

```

```

        w.get("branch", ""),
        w.get("path", ""),
        w["state"],
    ),
)

for sess in source.execute(
    "SELECT * FROM sessions WHERE workspace_id = ?", (w["id"],)
).fetchall():
    s = dict(sess)
    dest.execute(
        (
            "INSERT OR IGNORE INTO sessions "
            "(id, created_at, updated_at, workspace_id, status, model) "
            "VALUES (?, ?, ?, ?, ?, ?)"
        ),
        (
            s["id"],
            s["created_at"],
            s["updated_at"],
            s["workspace_id"],
            s["status"],
            s.get("model"),
        ),
    )

for msg in source.execute(
    "SELECT * FROM messages WHERE session_id = ?", (s["id"],)
).fetchall():
    m = dict(msg)
    dest.execute(
        (
            "INSERT OR IGNORE INTO messages "
            "(id, created_at, session_id, role, "
            "content, full_message, turn_id, turn_status) "
            "VALUES (?, ?, ?, ?, ?, ?, ?, ?)"
        ),
        (
            m["id"],
            m["created_at"],
            m["session_id"],
            m["role"],
            m["content"],
            m.get("full_message"),
            m.get("turn_id"),
            m.get("turn_status"),
        ),
    )

```

```

        ),
    )

    dest.commit()
    logger.info("Migrated %d legacy repository rows", len(repos))
except sqlite3.Error:
    logger.error("Failed to migrate legacy account data")
finally:
    source.close()

def _touch_last_login(db: sqlite3.Connection, user_id: str) -> None:
    """Update last_login_at for an existing user."""
    db.execute(
        "UPDATE users SET last_login_at = CURRENT_TIMESTAMP WHERE id = ?",
        (user_id,),
    )

def _find_user_by_identity(
    db: sqlite3.Connection,
    provider: str,
    provider_user_id: str,
) -> sqlite3.Row | None:
    """Return one user row for one OAuth identity, or None when missing."""
    row = db.execute(
        "SELECT oi.user_id, u.email FROM oauth_identities oi "
        "JOIN users u ON oi.user_id = u.id "
        "WHERE oi.provider = ? AND oi.provider_user_id = ?",
        (provider, provider_user_id),
    ).fetchone()
    return cast(sqlite3.Row | None, row)

def _find_user_by_email(
    db: sqlite3.Connection,
    email: str,
) -> sqlite3.Row | None:
    """Return one user row for one email, or None when missing."""
    row = db.execute(
        "SELECT id, email FROM users WHERE email = ?",
        (email,),
    ).fetchone()
    return cast(sqlite3.Row | None, row)

```

```

def _insert_oauth_identity(
    db: sqlite3.Connection,
    *,
    user_id: str,
    provider: str,
    provider_user_id: str,
    email: str,
    provider_data_json: str | None,
) -> None:
    """Insert one OAuth identity, tolerating concurrent duplicate inserts."""
    try:
        db.execute(
            "INSERT INTO oauth_identities "
            "(user_id, provider, provider_user_id, provider_email, provider_data) "
            "VALUES (?, ?, ?, ?, ?)",
            (user_id, provider, provider_user_id, email, provider_data_json),
        )
    except sqlite3.IntegrityError as error:
        existing_identity_row = _find_user_by_identity(db, provider, provider_user_id)
        if existing_identity_row is None:
            raise
        if existing_identity_row["user_id"] != user_id:
            raise RuntimeError("OAuth identity already belongs to a different user") from error

def _generate_encrypted_dek(user_id: str) -> bytes | None:
    """Return a wrapped DEK for one user when encryption is configured."""
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    normalized_user_id = user_id.strip()
    if not normalized_user_id:
        raise ValueError("user_id must not be empty")

    settings = get_settings()
    if not settings.active_key_encryption_key_bytes:
        return None

    dek = generate_dek()
    encrypted_dek = wrap_new_user_dek(dek, normalized_user_id)
    if not encrypted_dek:
        raise RuntimeError("Wrapped DEK must not be empty")
    return encrypted_dek

def _store_user_encrypted_dek(
    db: sqlite3.Connection,

```

```

        *,
        user_id: str,
    ) -> None:
        """Populate encrypted_dek for one newly inserted user row."""
        encrypted_dek = _generate_encrypted_dek(user_id)
        if encrypted_dek is None:
            return

        cursor = db.execute(
            "UPDATE users SET encrypted_dek = ? WHERE id = ? AND encrypted_dek IS NULL",
            (encrypted_dek, user_id),
        )
        if cursor.rowcount != 1:
            raise RuntimeError("Newly created user row must exist before storing encrypted DEK")

def _create_user_record(
    db: sqlite3.Connection,
    *,
    email: str,
    display_name: str | None,
    avatar_url: str | None,
) -> tuple[str, bool]:
    """Insert one user row or load the concurrently created row by email."""
    candidate_user_id = secrets.token_hex(16)

    try:
        db.execute(
            "INSERT INTO users (id, email, display_name, avatar_url, encrypted_dek, last_log",
            "VALUES (?, ?, ?, ?, ?, CURRENT_TIMESTAMP)",
            (candidate_user_id, email, display_name, avatar_url, None),
        )
    except sqlite3.IntegrityError:
        existing_user_row = _find_user_by_email(db, email)
        if existing_user_row is None:
            raise
        return existing_user_row["id"], False

    _store_user_encrypted_dek(db, user_id=candidate_user_id)
    return candidate_user_id, True

def resolve_or_create_user(
    provider: str,
    provider_user_id: str,
    email: str,

```



```

display_name: str | None = None,
avatar_url: str | None = None,
provider_data: dict[str, Any] | None = None,
) -> TenantContext:
    """Resolve an existing user or create a new one.

    1. Look up oauth_identities by (provider, provider_user_id) -- return if found
    2. Look up users by email -- link new identity if found
    3. Otherwise, provision a new user
    """

    provider_data_json = json.dumps(provider_data) if provider_data is not None else None

    with get_control_db() as db:
        # 1. Check existing identity
        row = _find_user_by_identity(db, provider, provider_user_id)

        if row:
            _touch_last_login(db, row["user_id"])
            db.commit()
            return _ensure_tenant_provisioned(row["user_id"], row["email"])

        # 2. Check existing user by email
        user_row = _find_user_by_email(db, email)

        if user_row:
            user_id = user_row["id"]
            _insert_oauth_identity(
                db,
                user_id=user_id,
                provider=provider,
                provider_user_id=provider_user_id,
                email=email,
                provider_data_json=provider_data_json,
            )
            _touch_last_login(db, user_id)
            db.commit()
            return _ensure_tenant_provisioned(user_id, email)

        # 3. Create new user
        user_id, user_was_created = _create_user_record(
            db,
            email=email,
            display_name=display_name,
            avatar_url=avatar_url,
        )
        _insert_oauth_identity(

```

```

        db,
        user_id=user_id,
        provider=provider,
        provider_user_id=provider_user_id,
        email=email,
        provider_data_json=provider_data_json,
    )
    if not user_was_created:
        _touch_last_login(db, user_id)
    db.commit()

tenant = _ensure_tenant_provisioned(user_id, email)
if not user_was_created:
    return tenant

# Provision outside the control DB transaction.
_migrate_legacy_data(tenant)
return tenant

```

13.2 api/runners.py

Runner routes register, heartbeat, revoke, and describe bring-your-own cloud runners. The root chunk below tangles back to `backend/src/yinshi/api/runners.py`.

```

"""Cloud runner registration and heartbeat API routes."""

from __future__ import annotations

import logging
from typing import Any
from urllib.parse import urlsplit

from fastapi import APIRouter, HTTPException, Request, Response

from yinshi.api.deps import require_tenant
from yinshi.config import get_settings
from yinshi.exceptions import RunnerAuthenticationError, RunnerRegistrationError
from yinshi.models import (
    CloudRunnerCreate,
    CloudRunnerOut,
    CloudRunnerRegistrationOut,
    RunnerCapabilityCreateIn,
    RunnerCapabilityOut,
    RunnerHeartbeatIn,
    RunnerHeartbeatOut,
    RunnerNoiseKeyConfirmationIn,

```

```

        RunnerRegisterIn,
        RunnerRegisterOut,
    )
    from yinshi.rate_limit import limiter
    from yinshi.services.runner_capabilities import (
        RUNNER_PROTOCOL_VERSION,
        create_runner_capability,
    )
    from yinshi.services.runner_relay import (
        runner_relay_broker,
        store_runner_transfer_grant,
    )
    from yinshi.services.runners import (
        confirm_runner_noise_key,
        create_runner_registration,
        get_runner_for_user,
        record_runner_heartbeat,
        register_runner,
        revoke_runner_for_user,
    )

    logger = logging.getLogger(__name__)
    router = APIRouter(tags=["runners"])
    _RUNNER_BEARER_REQUIRED = "Runner bearer token is required"

    def _request_control_url(request: Request) -> str:
        """Return the external origin, honoring forwarding only from configured proxies."""
        scheme = request.url.scheme
        netloc = request.url.netloc
        client_host = request.client.host.lower() if request.client is not None else ""
        if client_host in get_settings().trusted_proxy_ip_set:
            forwarded_scheme = request.headers.get("x-forwarded-proto")
            forwarded_host = request.headers.get("x-forwarded-host")
            if forwarded_scheme:
                scheme = forwarded_scheme.split(",", maxsplit=1)[0].strip().lower()
            if forwarded_host:
                netloc = forwarded_host.split(",", maxsplit=1)[0].strip()
        if scheme not in {"http", "https"}:
            raise HTTPException(status_code=400, detail="Could not determine control URL scheme")
        candidate = urlsplit(f"{scheme}://{netloc}")
        try:
            candidate_port = candidate.port
        except ValueError as error:
            raise HTTPException(
                status_code=400, detail="Could not determine control URL host"
            )

```

```

        ) from error
    if (
        not candidate.hostname
        or candidate.username is not None
        or candidate.password is not None
        or candidate.path
        or candidate.query
        or candidate.fragment
        or any(character.isspace() for character in netloc)
    ):
        raise HTTPException(status_code=400, detail="Could not determine control URL host")
    if candidate_port is not None and not 1 <= candidate_port <= 65_535:
        raise HTTPException(status_code=400, detail="Could not determine control URL host")
    return f"{scheme}://{netloc}"

def _request_relay_url(request: Request, transfer_id: str) -> str:
    """Return an externally visible WebSocket URL without embedding credentials."""
    control_url = _request_control_url(request)
    if control_url.startswith("https://"):
        relay_origin = f"wss://{control_url.removeprefix('https://')}"
    elif control_url.startswith("http://"):
        relay_origin = f"ws://{control_url.removeprefix('http://')}"
    else:
        raise HTTPException(status_code=400, detail="Could not determine relay URL scheme")
    return f"{relay_origin}/api/runner/relay/{transfer_id}"

def _bearer_token(request: Request) -> str:
    """Extract a bearer token from a runner Authorization header."""
    authorization = request.headers.get("authorization")
    if authorization is None:
        raise HTTPException(status_code=401, detail=_RUNNER_BEARER_REQUIRED)
    if not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail=_RUNNER_BEARER_REQUIRED)
    token = authorization.removeprefix("Bearer ").strip()
    if not token:
        raise HTTPException(status_code=401, detail=_RUNNER_BEARER_REQUIRED)
    return token

@router.get("/api/settings/runner", response_model=CloudRunnerOut | None)
def get_cloud_runner(request: Request) -> dict[str, Any] | None:
    """Return the current user's cloud runner status, if configured."""
    tenant = require_tenant(request)
    runner = get_runner_for_user(tenant.user_id)

```

```

        return runner

@router.post(
    "/api/settings/runner",
    response_model=CloudRunnerRegistrationOut,
    status_code=201,
)
def create_cloud_runner(
    body: CloudRunnerCreate,
    request: Request,
) -> dict[str, Any]:
    """Create a one-time registration token for a user-owned cloud runner."""
    tenant = require_tenant(request)
    try:
        return create_runner_registration(
            tenant.user_id,
            name=body.name,
            cloud_provider=body.cloud_provider,
            region=body.region,
            storage_profile=body.storage_profile,
            control_url=_request_control_url(request),
        )
    except (TypeError, ValueError) as error:
        raise HTTPException(status_code=400, detail=str(error)) from error

@router.post(
    "/api/settings/runner/noise-key/confirm",
    response_model=CloudRunnerOut,
)
@limiter.limit("10/minute")
def confirm_cloud_runner_noise_key(
    body: RunnerNoiseKeyConfirmationIn,
    request: Request,
) -> dict[str, Any]:
    """Record explicit confirmation of the runner's displayed Noise fingerprint."""
    tenant = require_tenant(request)
    try:
        return confirm_runner_noise_key(tenant.user_id, body.noise_public_key)
    except (RunnerRegistrationError, ValueError) as error:
        raise HTTPException(status_code=409, detail=str(error)) from error

@router.post(
    "/api/settings/runner/capabilities",

```

```

        response_model=RunnerCapabilityOut,
        status_code=201,
    )
    @limiter.limit("60/minute")
    def issue_cloud_runner_capability(
        body: RunnerCapabilityCreateIn,
        request: Request,
    ) -> RunnerCapabilityOut:
        """Issue least-privilege authority for one paired encrypted runner session."""
        tenant = require_tenant(request)
        runner = get_runner_for_user(tenant.user_id)
        if runner is None:
            raise HTTPException(status_code=404, detail="Runner not configured")
        if runner["status"] != "online":
            raise HTTPException(status_code=409, detail="Runner is not online")
        if not runner["noise_key_confirmed"]:
            raise HTTPException(status_code=409, detail="Runner Noise key must be confirmed")
        runner_public_key = runner["noise_public_key"]
        if not isinstance(runner_public_key, str) or not runner_public_key:
            raise HTTPException(status_code=409, detail="Runner Noise key is not available")
        try:
            capability, claims = create_runner_capability(
                user_id=tenant.user_id,
                runner_id=str(runner["id"]),
                runner_public_key=runner_public_key,
                initiator_public_key=body.initiator_public_key,
                scopes=body.scopes,
                max_session_bytes=body.max_session_bytes,
            )
        except (TypeError, ValueError) as error:
            raise HTTPException(status_code=400, detail=str(error)) from error
        store_runner_transfer_grant(capability, claims)
        return RunnerCapabilityOut(
            capability=capability,
            transfer_id=claims.transfer_id,
            runner_id=claims.runner_id,
            runner_public_key=claims.runner_public_key,
            protocol=RUNNER_PROTOCOL_VERSION,
            issued_at=claims.issued_at,
            expires_at=claims.expires_at,
            max_frame_bytes=claims.max_frame_bytes,
            max_session_bytes=claims.max_session_bytes,
            relay_url=_request_relay_url(request, claims.transfer_id),
        )

```

```

@router.delete("/api/settings/runner", status_code=204)
async def revoke_cloud_runner(request: Request) -> Response:
    """Revoke all runner authority and immediately close its outbound relay."""
    tenant = require_tenant(request)
    runner = get_runner_for_user(tenant.user_id)
    revoked = revoke_runner_for_user(tenant.user_id)
    if not revoked or runner is None:
        raise HTTPException(status_code=404, detail="Cloud runner not found")
    await runner_relay_broker.disconnect_runner(str(runner["id"]))
    return Response(status_code=204)

@router.post("/runner/register", response_model=RunnerRegisterOut, status_code=201)
@limiter.limit("30/minute")
def register_cloud_runner(body: RunnerRegisterIn, request: Request) -> dict[str, Any]:
    """Consume a one-time registration token from a freshly booted runner."""
    try:
        registered = register_runner(
            body.registration_token,
            runner_version=body.runner_version,
            capabilities=body.capabilities,
            data_dir=body.data_dir,
            sqlite_dir=body.sqlite_dir,
            shared_files_dir=body.shared_files_dir,
            storage_profile=body.storage_profile,
            noise_public_key=body.noise_public_key,
        )
    except RunnerRegistrationError as error:
        raise HTTPException(status_code=401, detail=str(error)) from error
    except (TypeError, ValueError) as error:
        raise HTTPException(status_code=400, detail=str(error)) from error

    logger.info("Cloud runner registered")
    return registered

@router.post("/runner/heartbeat", response_model=RunnerHeartbeatOut)
@limiter.limit("120/minute")
def heartbeat_cloud_runner(body: RunnerHeartbeatIn, request: Request) -> dict[str, Any]:
    """Record liveness and capabilities from a registered cloud runner."""
    runner_token = _bearer_token(request)
    try:
        heartbeat = record_runner_heartbeat(
            runner_token,
            runner_version=body.runner_version,
            capabilities=body.capabilities,

```

```

        data_dir=body.data_dir,
        sqlite_dir=body.sqlite_dir,
        shared_files_dir=body.shared_files_dir,
        storage_profile=body.storage_profile,
    )
except RunnerAuthenticationError as error:
    raise HTTPException(status_code=401, detail=str(error)) from error
except (TypeError, ValueError) as error:
    raise HTTPException(status_code=400, detail=str(error)) from error
return heartbeat

```

13.3 services/runners.py

Runner services validate storage profiles, hash tokens, persist runner capabilities, and track heartbeat freshness. The root chunk below tangles back to backend/src/yinshi/services/runners.py.

```

"""Control-plane records and tokens for bring-your-own-cloud runners."""

from __future__ import annotations

import base64
import binascii
import hashlib
import json
import secrets
import sqlite3
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from pathlib import PurePosixPath
from typing import Any, Literal, cast

from cryptography.hazmat.primitives.asymmetric.x25519 import (
    X25519PrivateKey,
    X25519PublicKey,
)

from yinshi.db import get_control_db
from yinshi.exceptions import RunnerAuthenticationError, RunnerRegistrationError
from yinshi.services.runner_capabilities import runner_capability_signing_public_key

RunnerKind = Literal["byoc", "managed", "managed_restore", "managed_retired"]
RunnerStorageProfile = Literal[
    "aws_ebs_s3_files",
    "archil_shared_files",
    "archil_all_posix",

```



```

    "fly_sprites_posix",
]

_REGISTRATION_TOKEN_TTL_MINUTES = 60
_HEARTBEAT_ONLINE_WINDOW_SECONDS = 120
_DEFAULT_RUNNER_DATA_DIR = "/var/lib/yinshi"
_DEFAULT_SQLITE_DIR = f"{_DEFAULT_RUNNER_DATA_DIR}/sqlite"
_DEFAULT_SHARED_FILES_DIR = "/mnt/yinshi-s3-files"
_DEFAULT_FLY_SPRITES_SHARED_FILES_DIR = f"{_DEFAULT_RUNNER_DATA_DIR}/files"
_DEFAULT_ARCHIL_SHARED_FILES_DIR = "/mnt/archil/yinshi"
_DEFAULT_ARCHIL_SQLITE_DIR = f"{_DEFAULT_ARCHIL_SHARED_FILES_DIR}/sqlite"
_DEFAULT_RUNNER_TOKEN_FILE = f"{_DEFAULT_RUNNER_DATA_DIR}/runner-token"
_DEFAULT_RUNNER_ENV_FILE = "/etc/yinshi-runner.env"
_REGISTRATION_TOKEN_EXPIRED = "Runner registration token has expired"
_AWS_STORAGE_PROFILE: RunnerStorageProfile = "aws_ebs_s3_files"
_ARCHIL_SHARED_FILES_PROFILE: RunnerStorageProfile = "archil_shared_files"
_ARCHIL_ALL_POSIX_PROFILE: RunnerStorageProfile = "archil_all_posix"
_FLY_SPRITES_POSIX_PROFILE: RunnerStorageProfile = "fly_sprites_posix"
_STORAGE_ARCHIL = "archil"
_STORAGE_RUNNER_EBS = "runner_ebs"
_STORAGE_S3_FILES_OR_LOCAL_POSIX = "s3_files_or_local_posix"
_STORAGE_S3_FILES_MOUNT = "s3_files_mount"
_STORAGE_LOCAL_POSIX = "local_posix"
_RUNNER_NOISE_PUBLIC_KEY_LENGTH = 32
_RUNNER_NOISE_VALIDATION_PRIVATE_KEY = X25519PrivateKey.from_private_bytes(b"\x01" * 32)
_BASE_CAPABILITIES = {
    "posix_storage": True,
    "sqlite": True,
    "git_worktrees": True,
    "pi_sidecar": True,
}

@dataclass(frozen=True, slots=True)
class RunnerStorageProfileSpec:
    """Validation and default path facts for one runner storage profile."""

    value: RunnerStorageProfile
    sqlite_storage: str
    shared_files_storage: str
    requires_explicit_storage: bool
    default_sqlite_dir: str
    default_shared_files_dir: str
    live_sqlite_on_shared_files: bool
    experimental: bool
    allow_sqlite_under_shared_files: bool

```

```

allowed_sqlite_storage: frozenset[str]
allowed_shared_files_storage: frozenset[str]

_STORAGE_PROFILES: dict[RunnerStorageProfile, RunnerStorageProfileSpec] = {
    _AWS_STORAGE_PROFILE: RunnerStorageProfileSpec(
        value=_AWS_STORAGE_PROFILE,
        sqlite_storage=_STORAGE_RUNNER_EBS,
        shared_files_storage=_STORAGE_S3_FILES_OR_LOCAL_POSIX,
        requires_explicit_storage=False,
        default_sqlite_dir=_DEFAULT_SQLITE_DIR,
        default_shared_files_dir=_DEFAULT_SHARED_FILES_DIR,
        live_sqlite_on_shared_files=False,
        experimental=False,
        allow_sqlite_under_shared_files=False,
        allowed_sqlite_storage=frozenset({_STORAGE_RUNNER_EBS}),
        allowed_shared_files_storage=frozenset(
            {
                _STORAGE_S3_FILES_OR_LOCAL_POSIX,
                _STORAGE_S3_FILES_MOUNT,
                _STORAGE_LOCAL_POSIX,
            }
        ),
    ),
    _ARCHIL_SHARED_FILES_PROFILE: RunnerStorageProfileSpec(
        value=_ARCHIL_SHARED_FILES_PROFILE,
        sqlite_storage=_STORAGE_RUNNER_EBS,
        shared_files_storage=_STORAGE_ARCHIL,
        requires_explicit_storage=True,
        default_sqlite_dir=_DEFAULT_SQLITE_DIR,
        default_shared_files_dir=_DEFAULT_ARCHIL_SHARED_FILES_DIR,
        live_sqlite_on_shared_files=False,
        experimental=True,
        allow_sqlite_under_shared_files=False,
        allowed_sqlite_storage=frozenset({_STORAGE_RUNNER_EBS}),
        allowed_shared_files_storage=frozenset({_STORAGE_ARCHIL}),
    ),
    _ARCHIL_ALL_POSIX_PROFILE: RunnerStorageProfileSpec(
        value=_ARCHIL_ALL_POSIX_PROFILE,
        sqlite_storage=_STORAGE_ARCHIL,
        shared_files_storage=_STORAGE_ARCHIL,
        requires_explicit_storage=True,
        default_sqlite_dir=_DEFAULT_ARCHIL_SQLITE_DIR,
        default_shared_files_dir=_DEFAULT_ARCHIL_SHARED_FILES_DIR,
        live_sqlite_on_shared_files=True,
        experimental=True,
    ),

```

```

        allow_sqlite_under_shared_files=True,
        allowed_sqlite_storage=frozenset({_STORAGE_ARCHIL}),
        allowed_shared_files_storage=frozenset({_STORAGE_ARCHIL}),
    ),
    _FLY_SPRITES_POSIX_PROFILE: RunnerStorageProfileSpec(
        value=_FLY_SPRITES_POSIX_PROFILE,
        sqlite_storage=_STORAGE_LOCAL_POSIX,
        shared_files_storage=_STORAGE_LOCAL_POSIX,
        requires_explicit_storage=False,
        default_sqlite_dir=_DEFAULT_SQLITE_DIR,
        default_shared_files_dir=_DEFAULT_FLY_SPRITES_SHARED_FILES_DIR,
        live_sqlite_on_shared_files=False,
        experimental=False,
        allow_sqlite_under_shared_files=False,
        allowed_sqlite_storage=frozenset({_STORAGE_LOCAL_POSIX}),
        allowed_shared_files_storage=frozenset({_STORAGE_LOCAL_POSIX}),
    ),
}

```

```

def _require_user_id(user_id: str) -> str:
    """Return a normalized user id or raise for invalid caller state."""
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    normalized_user_id = user_id.strip()
    if not normalized_user_id:
        raise ValueError("user_id must not be empty")
    return normalized_user_id

```

```

def _require_non_empty_text(value: str, name: str) -> str:
    """Return normalized text for fields that must be present."""
    if not isinstance(value, str):
        raise TypeError(f"{name} must be a string")
    normalized_value = value.strip()
    if not normalized_value:
        raise ValueError(f"{name} must not be empty")
    return normalized_value

```

```

def _optional_capability_text(capabilities: dict[str, Any], key: str) -> str | None:
    """Return one optional string capability after rejecting malformed values."""
    value = capabilities.get(key)
    if value is None:
        return None
    if not isinstance(value, str):

```

```

        raise ValueError(f"{key} capability must be a string")
    normalized_value = value.strip()
    if not normalized_value:
        return None
    return normalized_value

def _storage_profile_spec(storage_profile: str) -> RunnerStorageProfileSpec:
    """Return profile metadata after validating the stable profile value."""
    normalized_profile = _require_non_empty_text(storage_profile, "storage_profile")
    if normalized_profile not in _STORAGE_PROFILES:
        raise ValueError(f"Unsupported runner storage_profile: {normalized_profile}")
    return _STORAGE_PROFILES[normalized_profile]

def _storage_profile_from_capabilities(capabilities: dict[str, Any]) -> RunnerStorageProfileSpec:
    """Read a persisted storage profile, defaulting prerelease rows to AWS BYOC."""
    storage_profile = _optional_capability_text(capabilities, "storage_profile")
    if storage_profile is None:
        return _AWS_STORAGE_PROFILE
    return _storage_profile_spec(storage_profile).value

def _require_storage_path(value: str, name: str) -> PurePosixPath:
    """Return a normalized absolute runner storage path."""
    normalized_value = _require_non_empty_text(value, name)
    path = PurePosixPath(normalized_value)
    if not path.is_absolute():
        raise ValueError(f"{name} must be an absolute path")
    if ".." in path.parts:
        raise ValueError(f"{name} must not contain parent directory references")
    return path

def _path_contains(parent: PurePosixPath, child: PurePosixPath) -> bool:
    """Return whether child is equal to or nested under parent."""
    if child == parent:
        return True
    return parent in child.parents

def _utc_now() -> datetime:
    """Return the current UTC time with stable second precision."""
    return datetime.now(timezone.utc).replace(microsecond=0)

```

```

def _datetime_to_storage(value: datetime) -> str:
    """Serialize a timezone-aware timestamp for lexical SQLite comparisons."""
    if not isinstance(value, datetime):
        raise TypeError("value must be a datetime")
    if value.tzinfo is None:
        raise ValueError("value must be timezone-aware")
    return value.astimezone(timezone.utc).replace(microsecond=0).isoformat()

def _datetime_from_storage(value: object) -> datetime | None:
    """Parse SQLite timestamp strings produced by Yinshi schemas."""
    if value is None:
        return None
    if not isinstance(value, str):
        raise TypeError("stored datetime values must be strings or None")
    normalized_value = value.strip()
    if not normalized_value:
        return None
    parsed_value = datetime.fromisoformat(normalized_value.replace(" ", "T"))
    if parsed_value.tzinfo is None:
        parsed_value = parsed_value.replace(tzinfo=timezone.utc)
    return parsed_value.astimezone(timezone.utc)

def _hash_token(token: str) -> str:
    """Hash a token before storage so bearer values are never persisted."""
    normalized_token = _require_non_empty_text(token, "token")
    return hashlib.sha256(normalized_token.encode("utf-8")).hexdigest()

def _new_token() -> str:
    """Generate one URL-safe high-entropy bearer token."""
    return secrets.token_urlsafe(32)

def _validate_noise_public_key(value: str | None) -> str | None:
    """Return one canonical usable X25519 public key from runner input."""
    if value is None:
        return None
    normalized_value = _require_non_empty_text(value, "noise_public_key")
    try:
        public_key_bytes = base64.b64decode(
            normalized_value + "=",
            altchars=b" _",
            validate=True,
        )
    )

```

```

except (binascii.Error, ValueError) as exc:
    raise ValueError("noise_public_key must be canonical base64url") from exc
canonical_value = base64.urlsafe_b64encode(public_key_bytes).rstrip(b"=").decode("ascii")
if canonical_value != normalized_value:
    raise ValueError("noise_public_key must be canonical base64url")
if len(public_key_bytes) != _RUNNER_NOISE_PUBLIC_KEY_LENGTH:
    raise ValueError("noise_public_key must contain exactly 32 bytes")
try:
    public_key = X25519PublicKey.from_public_bytes(public_key_bytes)
    _RUNNER_NOISE_VALIDATION_PRIVATE_KEY.exchange(public_key)
except ValueError as exc:
    raise ValueError("noise_public_key is not a usable X25519 key") from exc
return canonical_value


def _capabilities_json(capabilities: dict[str, Any]) -> str:
    """Serialize runner capabilities after rejecting non-object payloads."""
    if not isinstance(capabilities, dict):
        raise TypeError("capabilities must be a dictionary")
    try:
        serialized = json.dumps(capabilities, sort_keys=True, separators=(",", ":"))
    except (TypeError, ValueError) as exc:
        raise ValueError("capabilities must be JSON serializable") from exc
    if len(serialized.encode("utf-8")) > 16_384:
        raise ValueError("capabilities payload is too large")
    return serialized


def _decode_capabilities(capabilities_json: object) -> dict[str, Any]:
    """Decode a capabilities JSON object from the control database."""
    if not isinstance(capabilities_json, str):
        raise TypeError("capabilities_json must be a string")
    payload = json.loads(capabilities_json or "{}")
    if not isinstance(payload, dict):
        raise ValueError("capabilities_json must decode to an object")
    return cast(dict[str, Any], payload)


def _validated_storage_class(
    capabilities: dict[str, Any],
    *,
    key: str,
    profile: RunnerStorageProfileSpec,
    expected_value: str,
    allowed_values: frozenset[str],
    required: bool,

```

```

) -> str:
    """Return a canonical storage-class string for one profile-bound capability."""
    requested_value = _optional_capability_text(capabilities, key)
    if requested_value is None:
        if required:
            raise ValueError(f"{key} must be {expected_value} for {profile.value}")
        return expected_value
    if requested_value not in allowed_values:
        allowed_text = ", ".join(sorted(allowed_values))
        raise ValueError(f"{key} must be one of {allowed_text} for {profile.value}")
    return requested_value

def _storage_capabilities(
    capabilities: dict[str, Any],
    *,
    data_dir: str,
    sqlite_dir: str | None,
    shared_files_dir: str | None,
    storage_profile: str,
) -> dict[str, Any]:
    """Merge runner facts with profile defaults and profile-aware path checks."""
    if not isinstance(capabilities, dict):
        raise TypeError("capabilities must be a dictionary")

    profile = _storage_profile_spec(storage_profile)
    normalized_data_dir = _require_storage_path(data_dir, "data_dir")
    normalized_sqlite_dir = _require_storage_path(
        sqlite_dir or profile.default_sqlite_dir,
        "sqlite_dir",
    )
    normalized_shared_files_dir = _require_storage_path(
        shared_files_dir or profile.default_shared_files_dir,
        "shared_files_dir",
    )
    if not profile.allow_sqlite_under_shared_files:
        if _path_contains(normalized_shared_files_dir, normalized_sqlite_dir):
            raise ValueError("sqlite_dir must not live under shared_files_dir")

    sqlite_storage = _validated_storage_class(
        capabilities,
        key="sqlite_storage",
        profile=profile,
        expected_value=profile.sqlite_storage,
        allowed_values=profile.allowed_sqlite_storage,
        required=profile.requires_explicit_storage,
    )

```

```

)
shared_files_storage = _validated_storage_class(
    capabilities,
    key="shared_files_storage",
    profile=profile,
    expected_value=profile.shared_files_storage,
    allowed_values=profile.allowed_shared_files_storage,
    required=profile.requires_explicit_storage,
)

merged_capabilities = {**_BASE_CAPABILITIES, **capabilities}
merged_capabilities.update(
    {
        "data_dir": str(normalized_data_dir),
        "sqlite_dir": str(normalized_sqlite_dir),
        "shared_files_dir": str(normalized_shared_files_dir),
        "storage_profile": profile.value,
        "storage_profile_experimental": profile.experimental,
        "sqlite_storage": sqlite_storage,
        "shared_files_storage": shared_files_storage,
        "live_sqlite_on_shared_files": profile.live_sqlite_on_shared_files,
    }
)
return merged_capabilities


def _serialized_capabilities(capabilities_json: object) -> dict[str, Any]:
    """Return capabilities with profile defaults filled for legacy rows."""
    capabilities = _decode_capabilities(capabilities_json)
    storage_profile = _storage_profile_from_capabilities(capabilities)
    profile = _storage_profile_spec(storage_profile)
    data_dir = _optional_capability_text(capabilities, "data_dir") or _DEFAULT_RUNNER_DATA_DIR
    sqlite_dir = _optional_capability_text(capabilities, "sqlite_dir") or profile.default_sqlite_dir
    shared_files_dir = (
        _optional_capability_text(capabilities, "shared_files_dir")
        or profile.default_shared_files_dir
    )
    return _storage_capabilities(
        capabilities,
        data_dir=data_dir,
        sqlite_dir=sqlite_dir,
        shared_files_dir=shared_files_dir,
        storage_profile=storage_profile,
    )

```



```

def _requested_profile_matches(
    *,
    requested_profile: str,
    stored_capabilities: dict[str, Any],
) -> RunnerStorageProfile:
    """Return the stored profile, rejecting runner attempts to change it."""
    stored_profile = _storage_profile_from_capabilities(stored_capabilities)
    supplied_profile = _storage_profile_spec(requested_profile).value
    if supplied_profile != stored_profile:
        raise ValueError("storage_profile must match requested runner profile")
    return stored_profile

def _display_status(row: Any, *, now: datetime | None = None) -> str:
    """Compute the user-facing runner status from registration and heartbeat fields."""
    assert row is not None, "row must not be None"
    current_time = now or _utc_now()
    if row["revoked_at"] is not None:
        return "revoked"
    if row["registered_at"] is None:
        return "pending"

    last_heartbeat_at = _datetime_from_storage(row["last_heartbeat_at"])
    if last_heartbeat_at is None:
        return "offline"
    heartbeat_age = (current_time - last_heartbeat_at).total_seconds()
    if heartbeat_age <= _HEARTBEAT_ONLINE_WINDOW_SECONDS:
        return "online"
    return "offline"

def _noise_key_fingerprint(noise_public_key: str | None) -> str | None:
    """Return a full SHA-256 fingerprint suitable for explicit user pairing."""
    if noise_public_key is None:
        return None
    validated_key = _validate_noise_public_key(noise_public_key)
    assert validated_key is not None
    key_bytes = base64.urlsafe_b64decode(validated_key + "=")
    return f"SHA256:{hashlib.sha256(key_bytes).hexdigest()}"

def _serialize_runner(row: Any) -> dict[str, Any]:
    """Return the safe API representation for one runner row."""
    assert row is not None, "row must not be None"
    noise_public_key = row["noise_public_key"]
    return {

```

```

        "id": row["id"],
        "kind": row["kind"],
        "created_at": row["created_at"],
        "updated_at": row["updated_at"],
        "name": row["name"],
        "cloud_provider": row["cloud_provider"],
        "region": row["region"],
        "status": _display_status(row),
        "registered_at": row["registered_at"],
        "last_heartbeat_at": row["last_heartbeat_at"],
        "runner_version": row["runner_version"],
        "capabilities": _serialized_capabilities(row["capabilities_json"]),
        "data_dir": row["data_dir"],
        "noise_public_key": noise_public_key,
        "noise_key_fingerprint": _noise_key_fingerprint(noise_public_key),
        "noise_key_confirmed": bool(noise_public_key and row["noise_public_key_confirmed_at"]),
        "restore_job_id": row["restore_job_id"] if "restore_job_id" in row.keys() else None,
    }

def get_runner_for_user(user_id: str) -> dict[str, Any] | None:
    """Return the current BYOC runner status for a user, including revoked state."""
    normalized_user_id = _require_user_id(user_id)
    with get_control_db() as db:
        row = db.execute(
            "SELECT * FROM user_runners WHERE user_id = ? AND kind = 'byoc'",
            (normalized_user_id,),
        ).fetchone()
    if row is None:
        return None
    return _serialize_runner(row)

def get_managed_runner_for_user(user_id: str) -> dict[str, Any] | None:
    """Return the internal managed runner status for a user, including revoked state."""
    normalized_user_id = _require_user_id(user_id)
    with get_control_db() as db:
        row = db.execute(
            "SELECT * FROM user_runners WHERE user_id = ? AND kind = 'managed'",
            (normalized_user_id,),
        ).fetchone()
    if row is None:
        return None
    return _serialize_runner(row)

```

```

def confirm_runner_noise_key(user_id: str, noise_public_key: str) -> dict[str, Any]:
    """Confirm the exact current runner identity after an out-of-band fingerprint check."""
    normalized_user_id = _require_user_id(user_id)
    normalized_public_key = _validate_noise_public_key(noise_public_key)
    assert normalized_public_key is not None
    confirmed_at = _datetime_to_storage(_utc_now())
    with get_control_db() as db:
        row = db.execute(
            "SELECT * FROM user_runners "
            "WHERE user_id = ? AND kind = 'byoc' AND revoked_at IS NULL",
            (normalized_user_id,),
        ).fetchone()
        if row is None or row["noise_public_key"] is None:
            raise RunnerRegistrationError("Runner Noise key is not available")
        if not secrets.compare_digest(row["noise_public_key"], normalized_public_key):
            raise RunnerRegistrationError("Runner Noise key does not match")
        db.execute(
            "UPDATE user_runners SET noise_public_key_confirmed_at = ? WHERE id = ?",
            (confirmed_at, row["id"]),
        )
        db.commit()
        confirmed_row = db.execute(
            "SELECT * FROM user_runners WHERE id = ?",
            (row["id"],),
        ).fetchone()
    assert confirmed_row is not None, "confirmed runner row must still exist"
    return _serialize_runner(confirmed_row)

def _runner_environment(
    *,
    control_url: str,
    registration_token: str,
    profile: RunnerStorageProfileSpec,
) -> dict[str, str]:
    """Return the systemd environment values needed to bootstrap one runner."""
    return {
        "YINSHI_CONTROL_URL": control_url,
        "YINSHI_REGISTRATION_TOKEN": registration_token,
        "YINSHI_RUNNER_STORAGE_PROFILE": profile.value,
        "YINSHI_RUNNER_SQLITE_STORAGE": profile.sqlite_storage,
        "YINSHI_RUNNER_SHARED_FILES_STORAGE": profile.shared_files_storage,
        "YINSHI_RUNNER_DATA_DIR": _DEFAULT_RUNNER_DATA_DIR,
        "YINSHI_RUNNER_SQLITE_DIR": profile.default_sqlite_dir,
        "YINSHI_RUNNER_DATA_PROTECTION_KEY_FILE": (
            f"{profile.default_sqlite_dir}/.yinshi-data-protection-key"
        )
    }

```

```

    ),
    "YINSHI_RUNNER_SHARED_FILES_DIR": profile.default_shared_files_dir,
    "YINSHI_RUNNER_TOKEN_FILE": _DEFAULT_RUNNER_TOKEN_FILE,
    "YINSHI_RUNNER_NOISE_KEY_FILE": f"{_DEFAULT_RUNNER_DATA_DIR}/runner-noise.key",
    "YINSHI_RUNNER_CAPABILITY_SIGNING_KEY_FILE": (
        f"{_DEFAULT_RUNNER_DATA_DIR}/control-capability-signing.pub"
    ),
    "YINSHI_RUNNER_REPLAY_DATABASE_FILE": (
        f"{_DEFAULT_RUNNER_DATA_DIR}/runner-capability-replay.sqlite3"
    ),
    "YINSHI_RUNNER_ENV_FILE": _DEFAULT_RUNNER_ENV_FILE,
}

def _create_runner_registration_in_connection(
    db: sqlite3.Connection,
    user_id: str,
    *,
    name: str,
    cloud_provider: str,
    region: str,
    storage_profile: str,
    control_url: str,
    runner_kind: RunnerKind = "byoc",
    now: datetime | None = None,
) -> dict[str, Any]:
    """Upsert registration state without committing the caller-owned transaction."""
    normalized_user_id = _require_user_id(user_id)
    normalized_name = _require_non_empty_text(name, "name")
    normalized_provider = _require_non_empty_text(cloud_provider, "cloud_provider")
    normalized_region = _require_non_empty_text(region, "region")
    normalized_control_url = _require_non_empty_text(control_url, "control_url").rstrip("/")
    normalized_kind = _require_non_empty_text(runner_kind, "runner_kind")
    profile = _storage_profile_spec(storage_profile)
    if normalized_kind == "byoc":
        if normalized_provider != "aws":
            raise ValueError("Only AWS runners are supported")
    elif normalized_kind in {"managed", "managed_restore"}:
        if normalized_provider != "fly_sprites":
            raise ValueError("Managed runners require fly_sprites")
    else:
        raise ValueError(f"Unsupported runner kind: {normalized_kind}")

    registration_token = _new_token()
    registration_token_hash = _hash_token(registration_token)
    expires_at = (now or _utc_now()) + timedelta(minutes=_REGISTRATION_TOKEN_TTL_MINUTES)

```

```

expires_at_text = _datetime_to_storage(expires_at)
pending_capabilities = _capabilities_json(
    _storage_capabilities(
        {
            "sqlite_storage": profile.sqlite_storage,
            "shared_files_storage": profile.shared_files_storage,
        },
        data_dir=_DEFAULT_RUNNER_DATA_DIR,
        sqlite_dir=profile.default_sqlite_dir,
        shared_files_dir=profile.default_shared_files_dir,
        storage_profile=profile.value,
    )
)
db.execute(
    """
    INSERT INTO user_runners (
        user_id, kind, name, cloud_provider, region, status,
        registration_token_hash, registration_token_expires_at,
        runner_token_hash, registered_at, last_heartbeat_at,
        runner_version, capabilities_json, data_dir, revoked_at
    )
    VALUES (?, ?, ?, ?, ?, 'pending', ?, ?, NULL, NULL, NULL, NULL, ?, NULL, NULL)
    ON CONFLICT(user_id, kind) DO UPDATE SET
        name = excluded.name,
        cloud_provider = excluded.cloud_provider,
        region = excluded.region,
        status = 'pending',
        registration_token_hash = excluded.registration_token_hash,
        registration_token_expires_at = excluded.registration_token_expires_at,
        runner_token_hash = NULL,
        registered_at = NULL,
        last_heartbeat_at = NULL,
        runner_version = NULL,
        capabilities_json = excluded.capabilities_json,
        data_dir = NULL,
        revoked_at = NULL,
        noise_public_key = CASE
            WHEN excluded.kind = 'managed' THEN user_runners.noise_public_key
            ELSE NULL
        END,
        noise_public_key_confirmed_at = CASE
            WHEN excluded.kind = 'managed'
            THEN user_runners.noise_public_key_confirmed_at
            ELSE NULL
        END
    """
),

```

```

        (
            normalized_user_id,
            normalized_kind,
            normalized_name,
            normalized_provider,
            normalized_region,
            registration_token_hash,
            expires_at_text,
            pending_capabilities,
        ),
    )
    row = db.execute(
        "SELECT * FROM user_runners WHERE user_id = ? AND kind = ?",
        (normalized_user_id, normalized_kind),
    ).fetchone()
    assert row is not None, "runner row must exist after registration upsert"
    return {
        "runner": _serialize_runner(row),
        "registration_token": registration_token,
        "registration_token_expires_at": expires_at_text,
        "control_url": normalized_control_url,
        "environment": _runner_environment(
            control_url=normalized_control_url,
            registration_token=registration_token,
            profile=profile,
        ),
    }
}

def create_managed_restore_registration(
    user_id: str,
    *,
    job_id: str,
    name: str,
    region: str,
    control_url: str,
) -> dict[str, Any]:
    """Create replacement authority bound to one exact restore job."""
    normalized_user_id = _require_user_id(user_id)
    normalized_job_id = _require_non_empty_text(job_id, "job_id")
    with get_control_db() as database:
        registration = _create_runner_registration_in_connection(
            database,
            normalized_user_id,
            name=name,
            cloud_provider="fly_sprites",

```

```

        region=region,
        storage_profile="fly_sprites_posix",
        control_url=control_url,
        runner_kind="managed_restore",
    )
    result = database.execute(
        "UPDATE user_runners SET restore_job_id = ? WHERE id = ?",
        (normalized_job_id, registration["runner"]["id"]),
    )
    if result.rowcount != 1:
        database.rollback()
        raise RunnerRegistrationError("Restore runner registration could not be bound")
    database.commit()
    return registration

def get_managed_restore_runner_for_job(
    user_id: str,
    job_id: str,
) -> dict[str, Any] | None:
    """Return only replacement authority bound to one exact restore job."""
    normalized_user_id = _require_user_id(user_id)
    normalized_job_id = _require_non_empty_text(job_id, "job_id")
    with get_control_db() as database:
        row = database.execute(
            """SELECT * FROM user_runners
               WHERE user_id = ? AND kind = 'managed_restore' AND restore_job_id = ?""",
            (normalized_user_id, normalized_job_id),
        ).fetchone()
    return _serialize_runner(row) if row is not None else None

def revoke_managed_restore_runner_for_job(user_id: str, job_id: str) -> bool:
    """Revoke only replacement authority bound to one exact restore job."""
    normalized_user_id = _require_user_id(user_id)
    normalized_job_id = _require_non_empty_text(job_id, "job_id")
    revoked_at = _datetime_to_storage(utc_now())
    with get_control_db() as database:
        result = database.execute(
            """UPDATE user_runners
               SET status = 'revoked', revoked_at = ?, registration_token_hash = NULL,
                 registration_token_expires_at = NULL, runner_token_hash = NULL
               WHERE user_id = ? AND kind = 'managed_restore'
                 AND restore_job_id = ? AND revoked_at IS NULL""",
            (revoked_at, normalized_user_id, normalized_job_id),
        )

```

```

        database.commit()
    return result.rowcount == 1

def get_managed_restore_runner_for_user(user_id: str) -> dict[str, Any] | None:
    """Return the non-active replacement runner for one managed restore."""
    normalized_user_id = _require_user_id(user_id)
    with get_control_db() as database:
        row = database.execute(
            "SELECT * FROM user_runners WHERE user_id = ? AND kind = 'managed_restore'",
            (normalized_user_id,),
        ).fetchone()
    return _serialize_runner(row) if row is not None else None

def create_runner_registration(
    user_id: str,
    *,
    name: str,
    cloud_provider: str,
    region: str,
    storage_profile: str,
    control_url: str,
    runner_kind: RunnerKind = "byoc",
) -> dict[str, Any]:
    """Create or rotate a one-time BYOC or internal managed registration token."""
    with get_control_db() as db:
        registration = _create_runner_registration_in_connection(
            db,
            user_id,
            name=name,
            cloud_provider=cloud_provider,
            region=region,
            storage_profile=storage_profile,
            control_url=control_url,
            runner_kind=runner_kind,
        )
        db.commit()
    return registration

def revoke_managed_restore_runner_for_user(user_id: str) -> bool:
    """Revoke the non-active replacement runner and all of its credentials."""
    normalized_user_id = _require_user_id(user_id)
    revoked_at = _datetime_to_storage(_utc_now())
    with get_control_db() as database:

```



```

        result = database.execute(
            """UPDATE user_runners
               SET status = 'revoked', revoked_at = ?,
                 registration_token_hash = NULL,
                 registration_token_expires_at = NULL,
                 runner_token_hash = NULL
               WHERE user_id = ? AND kind = 'managed_restore' AND revoked_at IS NULL""",
            (revoked_at, normalized_user_id),
        )
        database.commit()
    return result.rowcount == 1

def revoke_runner_for_user(user_id: str) -> bool:
    """Revoke the current runner and all of its outstanding tokens."""
    normalized_user_id = _require_user_id(user_id)
    revoked_at = _datetime_to_storage(_utc_now())
    with get_control_db() as db:
        result = db.execute(
            """
            UPDATE user_runners
            SET status = 'revoked',
              revoked_at = ?,
              registration_token_hash = NULL,
              registration_token_expires_at = NULL,
              runner_token_hash = NULL
            WHERE user_id = ? AND kind = 'byoc' AND revoked_at IS NULL
            """,
            (revoked_at, normalized_user_id),
        )
        db.commit()
    return result.rowcount > 0

def register_runner(
    registration_token: str,
    *,
    runner_version: str,
    capabilities: dict[str, Any],
    data_dir: str,
    sqlite_dir: str | None,
    shared_files_dir: str | None,
    storage_profile: str,
    noise_public_key: str | None,
) -> dict[str, Any]:
    """Consume a one-time registration token and issue a runner bearer token."""

```

```

token_hash = _hash_token(registration_token)
normalized_version = _require_non_empty_text(runner_version, "runner_version")
normalized_noise_public_key = _validate_noise_public_key(noise_public_key)
now_text = _datetime_to_storage(_utc_now())
runner_token = _new_token()
runner_token_hash = _hash_token(runner_token)

with get_control_db() as db:
    row = db.execute(
        """
        SELECT * FROM user_runners
        WHERE registration_token_hash = ? AND revoked_at IS NULL
        """,
        (token_hash,),
    ).fetchone()
    if row is None:
        raise RunnerRegistrationError("Runner registration token is invalid")

    expires_at = _datetime_from_storage(row["registration_token_expires_at"])
    if expires_at is None:
        raise RunnerRegistrationError(_REGISTRATION_TOKEN_EXPIRED)
    if expires_at <= _utc_now():
        raise RunnerRegistrationError(_REGISTRATION_TOKEN_EXPIRED)

    stored_noise_public_key = row["noise_public_key"]
    if row["kind"] in {"managed", "managed_restore"} and stored_noise_public_key is not None:
        if normalized_noise_public_key is None or not secrets.compare_digest(
            stored_noise_public_key,
            normalized_noise_public_key,
        ):
            raise RunnerRegistrationError("Managed runner Noise identity does not match")

    stored_capabilities = _decode_capabilities(row["capabilities_json"])
    stored_profile = _requested_profile_matches(
        requested_profile=storage_profile,
        stored_capabilities=stored_capabilities,
    )
    merged_capabilities = _storage_capabilities(
        capabilities,
        data_dir=data_dir,
        sqlite_dir=sqlite_dir,
        shared_files_dir=shared_files_dir,
        storage_profile=stored_profile,
    )
    capabilities_text = _capabilities_json(merged_capabilities)
    normalized_data_dir = str(merged_capabilities["data_dir"])

```

```

db.execute(
    """
    UPDATE user_runners
    SET status = 'online',
        registration_token_hash = NULL,
        registration_token_expires_at = NULL,
        runner_token_hash = ?,
        registered_at = ?,
        last_heartbeat_at = ?,
        runner_version = ?,
        capabilities_json = ?,
        data_dir = ?,
        noise_public_key = ?,
        noise_public_key_confirmed_at = CASE
            WHEN kind IN ('managed', 'managed_restore') THEN ?
            WHEN noise_public_key = ? THEN noise_public_key_confirmed_at
            ELSE NULL
        END
    WHERE id = ?
    """,
    (
        runner_token_hash,
        now_text,
        now_text,
        normalized_version,
        capabilities_text,
        normalized_data_dir,
        normalized_noise_public_key,
        now_text,
        normalized_noise_public_key,
        row["id"],
    ),
)
db.commit()

return {
    "runner_id": row["id"],
    "runner_token": runner_token,
    "capability_signing_public_key": runner_capability_signing_public_key(),
    "status": "online",
}

def authenticate_runner_token(runner_token: str) -> dict[str, str]:
    """Return relay identity for one current non-revoked runner bearer token."""

```

```

token_hash = _hash_token(runner_token)
with get_control_db() as database:
    row = database.execute(
        """
        SELECT id, user_id, noise_public_key
        FROM user_runners
        WHERE runner_token_hash = ? AND revoked_at IS NULL
        """,
        (token_hash,),
    ).fetchone()
if row is None:
    raise RunnerAuthenticationError("Runner token is invalid")
noise_public_key = row["noise_public_key"]
if not isinstance(noise_public_key, str) or not noise_public_key:
    raise RunnerAuthenticationError("Runner Noise identity is unavailable")
return {
    "runner_id": row["id"],
    "user_id": row["user_id"],
    "noise_public_key": noise_public_key,
}

def record_runner_heartbeat(
    runner_token: str,
    *,
    runner_version: str,
    capabilities: dict[str, Any],
    data_dir: str,
    sqlite_dir: str | None,
    shared_files_dir: str | None,
    storage_profile: str,
) -> dict[str, Any]:
    """Record a heartbeat from a registered runner bearer token."""
    token_hash = _hash_token(runner_token)
    normalized_version = _require_non_empty_text(runner_version, "runner_version")
    now_text = _datetime_to_storage(_utc_now())

    with get_control_db() as db:
        row = db.execute(
            """
            SELECT * FROM user_runners
            WHERE runner_token_hash = ? AND revoked_at IS NULL
            """,
            (token_hash,),
        ).fetchone()
        if row is None:

```

```

        raise RunnerAuthenticationError("Runner token is invalid")

    stored_capabilities = _decode_capabilities(row["capabilities_json"])
    stored_profile = _requested_profile_matches(
        requested_profile=storage_profile,
        stored_capabilities=stored_capabilities,
    )
    merged_capabilities = _storage_capabilities(
        capabilities,
        data_dir=data_dir,
        sqlite_dir=sqlite_dir,
        shared_files_dir=shared_files_dir,
        storage_profile=stored_profile,
    )
    capabilities_text = _capabilities_json(merged_capabilities)
    normalized_data_dir = str(merged_capabilities["data_dir"])

    db.execute(
        """
        UPDATE user_runners
        SET status = 'online',
            last_heartbeat_at = ?,
            runner_version = ?,
            capabilities_json = ?,
            data_dir = ?
        WHERE id = ?
        """,
        (
            now_text,
            normalized_version,
            capabilities_text,
            normalized_data_dir,
            row["id"],
        ),
    )
    db.commit()

    return {
        "runner_id": row["id"],
        "capability_signing_public_key": runner_capability_signing_public_key(),
        "status": "online",
    }

```

13.4 runner_agent.py

The runner agent is the small process a cloud runner runs to register itself and send heartbeats to Yinshi. The root chunk below tangles back to backend/src/yinshi/runner_agent.py.

```
"""Cloud runner agent for registration and encrypted restricted worker RPC."""

from __future__ import annotations

import asyncio
import base64
import binascii
import json
import logging
import math
import os
import re
import secrets
import stat
import time
import uuid

from dataclasses import dataclass
from pathlib import Path
from typing import Any, Literal
from urllib.parse import urlsplit, urlunsplit

import httpx
from websockets.asyncio.client import ClientConnection, connect
from websockets.exceptions import ConnectionClosed, InvalidStatus
from websockets.typing import Origin

from yinshi.runner_worker import (
    RunnerUserDataEncryptionMode,
    RunnerWorkerManager,
    validate_user_data_encryption_mode,
)
from yinshi.services.runner_agent_relay import (
    RunnerAgentRelayRuntime,
    RunnerRelaySessionError,
)
from yinshi.services.runner_data_key import load_or_create_runner_data_key
from yinshi.services.runner_noise import load_or_create_runner_noise_keypair
from yinshi.services.sprite_task_lease import SpriteTaskLease

logger = logging.getLogger(__name__)
```

```

RUNNER_VERSION = "0.2.0"
RunnerStorageProfile = Literal[
    "aws_ebs_s3_files",
    "archil_shared_files",
    "archil_all_posix",
    "fly_sprites_posix",
]
_DEFAULT_CONTROL_URL = "http://localhost:8000"
_DEFAULT_DATA_DIR = "/var/lib/yinshi"
_DEFAULT_SQLITE_DIR = f"{_DEFAULT_DATA_DIR}/sqlite"
_DEFAULT_SHARED_FILES_DIR = "/mnt/yinshi-s3-files"
_DEFAULT_FLY_SPRITES_SHARED_FILES_DIR = f"{_DEFAULT_DATA_DIR}/files"
_DEFAULT_ARCHIL_SHARED_FILES_DIR = "/mnt/archil/yinshi"
_DEFAULT_ARCHIL_SQLITE_DIR = f"{_DEFAULT_ARCHIL_SHARED_FILES_DIR}/sqlite"
_DEFAULT_TOKEN_FILE = "/var/lib/yinshi/runner-token"
_DEFAULT_HEARTBEAT_INTERVAL_S = 30.0
_REQUEST_TIMEOUT_S = 15.0
_REGISTRATION_TOKEN_ENV_PREFIX = "YINSHI_REGISTRATION_TOKEN="
_SHA256_PATTERN = re.compile(r"[0-9a-f]{64}\Z")
_AWS_STORAGE_PROFILE: RunnerStorageProfile = "aws_ebs_s3_files"
_ARCHIL_SHARED_FILES_PROFILE: RunnerStorageProfile = "archil_shared_files"
_ARCHIL_ALL_POSIX_PROFILE: RunnerStorageProfile = "archil_all_posix"
_FLY_SPRITES_POSIX_PROFILE: RunnerStorageProfile = "fly_sprites_posix"
_STORAGE_ARCHIL = "archil"
_STORAGE_RUNNER_EBS = "runner_ebs"
_STORAGE_S3_FILES_OR_LOCAL_POSIX = "s3_files_or_local_posix"
_STORAGE_S3_FILES_MOUNT = "s3_files_mount"
_STORAGE_LOCAL_POSIX = "local_posix"

@dataclass(frozen=True, slots=True)
class RunnerStorageProfileSpec:
    """Environment defaults and validation rules for one runner storage profile."""

    value: RunnerStorageProfile
    sqlite_storage: str
    shared_files_storage: str
    requires_explicit_storage: bool
    default_sqlite_dir: str
    default_shared_files_dir: str
    live_sqlite_on_shared_files: bool
    experimental: bool
    allow_sqlite_under_shared_files: bool
    allowed_sqlite_storage: frozenset[str]
    allowed_shared_files_storage: frozenset[str]

```

```

_STORAGE_PROFILES: dict[RunnerStorageProfile, RunnerStorageProfileSpec] = {
    _AWS_STORAGE_PROFILE: RunnerStorageProfileSpec(
        value=_AWS_STORAGE_PROFILE,
        sqlite_storage=_STORAGE_RUNNER_EBS,
        shared_files_storage=_STORAGE_S3_FILES_OR_LOCAL_POSIX,
        requires_explicit_storage=False,
        default_sqlite_dir=_DEFAULT_SQLITE_DIR,
        default_shared_files_dir=_DEFAULT_SHARED_FILES_DIR,
        live_sqlite_on_shared_files=False,
        experimental=False,
        allow_sqlite_under_shared_files=False,
        allowed_sqlite_storage=frozenset({_STORAGE_RUNNER_EBS}),
        allowed_shared_files_storage=frozenset(
            {
                _STORAGE_S3_FILES_OR_LOCAL_POSIX,
                _STORAGE_S3_FILES_MOUNT,
                _STORAGE_LOCAL_POSIX,
            }
        ),
    ),
    _ARCHIL_SHARED_FILES_PROFILE: RunnerStorageProfileSpec(
        value=_ARCHIL_SHARED_FILES_PROFILE,
        sqlite_storage=_STORAGE_RUNNER_EBS,
        shared_files_storage=_STORAGE_ARCHIL,
        requires_explicit_storage=True,
        default_sqlite_dir=_DEFAULT_SQLITE_DIR,
        default_shared_files_dir=_DEFAULT_ARCHIL_SHARED_FILES_DIR,
        live_sqlite_on_shared_files=False,
        experimental=True,
        allow_sqlite_under_shared_files=False,
        allowed_sqlite_storage=frozenset({_STORAGE_RUNNER_EBS}),
        allowed_shared_files_storage=frozenset({_STORAGE_ARCHIL}),
    ),
    _ARCHIL_ALL_POSIX_PROFILE: RunnerStorageProfileSpec(
        value=_ARCHIL_ALL_POSIX_PROFILE,
        sqlite_storage=_STORAGE_ARCHIL,
        shared_files_storage=_STORAGE_ARCHIL,
        requires_explicit_storage=True,
        default_sqlite_dir=_DEFAULT_ARCHIL_SQLITE_DIR,
        default_shared_files_dir=_DEFAULT_ARCHIL_SHARED_FILES_DIR,
        live_sqlite_on_shared_files=True,
        experimental=True,
        allow_sqlite_under_shared_files=True,
        allowed_sqlite_storage=frozenset({_STORAGE_ARCHIL}),
        allowed_shared_files_storage=frozenset({_STORAGE_ARCHIL}),
    ),

```



```

    ),
    _FLY_SPRITES_POSIX_PROFILE: RunnerStorageProfileSpec(
        value=_FLY_SPRITES_POSIX_PROFILE,
        sqlite_storage=_STORAGE_LOCAL_POSIX,
        shared_files_storage=_STORAGE_LOCAL_POSIX,
        requires_explicit_storage=False,
        default_sqlite_dir=_DEFAULT_SQLITE_DIR,
        default_shared_files_dir=_DEFAULT_FLY_SPRITES_SHARED_FILES_DIR,
        live_sqlite_on_shared_files=False,
        experimental=False,
        allow_sqlite_under_shared_files=False,
        allowed_sqlite_storage=frozenset({_STORAGE_LOCAL_POSIX}),
        allowed_shared_files_storage=frozenset({_STORAGE_LOCAL_POSIX}),
    ),
}

```

```

@dataclass(frozen=True, slots=True)
class RunnerAgentConfig:
    """Environment-derived configuration for the cloud runner agent."""

    control_url: str
    registration_token: str | None
    runner_token_file: Path
    noise_private_key_file: Path
    data_protection_key_file: Path
    capability_signing_key_file: Path
    replay_database_file: Path
    data_dir: Path
    sqlite_dir: Path
    shared_files_dir: Path
    storage_profile: RunnerStorageProfile
    sqlite_storage: str
    shared_files_storage: str | None
    user_data_encryption: RunnerUserDataEncryptionMode
    heartbeat_interval_s: float
    relay_idle_timeout_seconds: float | None
    sprite_task_lease: bool
    env_file: Path | None
    artifact_sha256: str | None = None
    artifact_attestation_file: Path | None = None

    def _env_text(name: str, default: str | None = None) -> str | None:
        """Read and normalize an optional environment value."""
        value = os.environ.get(name, default)

```

```

    if value is None:
        return None
    normalized_value = value.strip()
    if not normalized_value:
        return None
    return normalized_value

def _env_user_data_encryption() -> RunnerUserDataEncryptionMode:
    """Read the dedicated runner user-data encryption setting."""
    env_name = "YINSHI_RUNNER_USER_DATA_ENCRYPTION"
    raw_value = _env_text(env_name, "disabled")
    if raw_value is None:
        raise RuntimeError(f"{env_name} must be disabled or required")
    try:
        return validate_user_data_encryption_mode(raw_value)
    except (TypeError, ValueError) as exc:
        raise RuntimeError(f"{env_name} must be disabled or required") from exc

def _env_float(name: str, default: float) -> float:
    """Read a positive float from the environment with explicit validation."""
    raw_value = _env_text(name)
    if raw_value is None:
        return default
    try:
        value = float(raw_value)
    except ValueError as exc:
        raise RuntimeError(f"{name} must be a number") from exc
    if value <= 0:
        raise RuntimeError(f"{name} must be positive")
    return value

def _env_optional_positive_float(name: str) -> float | None:
    """Read an absent or positive finite float without accepting empty text."""
    raw_value = os.environ.get(name)
    if raw_value is None:
        return None
    try:
        value = float(raw_value.strip())
    except ValueError:
        raise RuntimeError(f"{name} must be a positive finite number") from None
    if not math.isfinite(value) or value <= 0:
        raise RuntimeError(f"{name} must be a positive finite number")
    return value

```

```

def _env_disabled_or_enabled(name: str) -> bool:
    """Read one exact disabled or enabled feature setting."""
    raw_value = os.environ.get(name, "disabled").strip()
    if raw_value == "disabled":
        return False
    if raw_value == "enabled":
        return True
    raise RuntimeError(f"{name} must be disabled or enabled")

def _env_path(name: str, default: str) -> Path:
    """Read a required absolute filesystem path from the environment."""
    path_text = _env_text(name, default)
    if path_text is None:
        raise RuntimeError(f"{name} must not be empty")
    path = Path(path_text)
    if not path.is_absolute():
        raise RuntimeError(f"{name} must be an absolute path")
    if ".." in path.parts:
        raise RuntimeError(f"{name} must not contain parent directory references")
    return path

def _storage_profile_spec(storage_profile: str) -> RunnerStorageProfileSpec:
    """Return storage profile metadata after validating the profile value."""
    normalized_profile = storage_profile.strip()
    if not normalized_profile:
        raise RuntimeError("YINSHI_RUNNER_STORAGE_PROFILE must not be empty")
    if normalized_profile not in _STORAGE_PROFILES:
        raise RuntimeError(f"Unsupported YINSHI_RUNNER_STORAGE_PROFILE: {normalized_profile}")
    return _STORAGE_PROFILES[normalized_profile]

def _validate_storage_class(
    *,
    env_name: str,
    value: str | None,
    profile: RunnerStorageProfileSpec,
    expected_value: str,
    allowed_values: frozenset[str],
    required: bool,
) -> str | None:
    """Validate one storage-class environment value against the selected profile."""
    if value is None:

```

```

        if required:
            raise RuntimeError(f"{env_name} must be {expected_value} for {profile.value}")
        return None
    if value not in allowed_values:
        allowed_text = ", ".join(sorted(allowed_values))
        raise RuntimeError(f"{env_name} must be one of {allowed_text} for {profile.value}")
    return value

def _load_storage_profile() -> RunnerStorageProfileSpec:
    """Read the selected runner storage profile from the environment."""
    storage_profile = _env_text("YINSHI_RUNNER_STORAGE_PROFILE", _AWS_STORAGE_PROFILE)
    assert storage_profile is not None, "default storage profile must be non-empty"
    return _storage_profile_spec(storage_profile)

def load_config() -> RunnerAgentConfig:
    """Build runner agent config from environment variables."""
    control_url = _env_text("YINSHI_CONTROL_URL", _DEFAULT_CONTROL_URL)
    assert control_url is not None, "default control URL must be non-empty"
    profile = _load_storage_profile()
    sqlite_storage = _validate_storage_class(
        env_name="YINSHI_RUNNER_SQLITE_STORAGE",
        value=_env_text("YINSHI_RUNNER_SQLITE_STORAGE", profile.sqlite_storage),
        profile=profile,
        expected_value=profile.sqlite_storage,
        allowed_values=profile.allowed_sqlite_storage,
        required=profile.requires_explicit_storage,
    )
    assert sqlite_storage is not None, "SQLite storage has a profile default"
    shared_files_storage_default = (
        profile.shared_files_storage if profile.value == _FLY_SPRITES_POSIX_PROFILE else None
    )
    shared_files_storage = _validate_storage_class(
        env_name="YINSHI_RUNNER_SHARED_FILES_STORAGE",
        value=_env_text(
            "YINSHI_RUNNER_SHARED_FILES_STORAGE",
            shared_files_storage_default,
        ),
        profile=profile,
        expected_value=profile.shared_files_storage,
        allowed_values=profile.allowed_shared_files_storage,
        required=profile.requires_explicit_storage,
    )
    runner_token_file = _env_path("YINSHI_RUNNER_TOKEN_FILE", _DEFAULT_TOKEN_FILE)
    data_dir = _env_path("YINSHI_RUNNER_DATA_DIR", _DEFAULT_DATA_DIR)

```

```

noise_private_key_file = _env_path(
    "YINSHI_RUNNER_NOISE_KEY_FILE",
    str(data_dir / "runner-noise.key"),
)
capability_signing_key_file = _env_path(
    "YINSHI_RUNNER_CAPABILITY_SIGNING_KEY_FILE",
    str(data_dir / "control-capability-signing.pub"),
)
replay_database_file = _env_path(
    "YINSHI_RUNNER_REPLAY_DATABASE_FILE",
    str(data_dir / "runner-capability-replay.sqlite3"),
)
sqlite_dir = _env_path("YINSHI_RUNNER_SQLITE_DIR", profile.default_sqlite_dir)
data_protection_key_file = _env_path(
    "YINSHI_RUNNER_DATA_PROTECTION_KEY_FILE",
    str(sqlite_dir / ".yinshi-data-protection-key"),
)
if data_protection_key_file.parent != sqlite_dir:
    raise RuntimeError(
        "YINSHI_RUNNER_DATA_PROTECTION_KEY_FILE must be inside YINSHI_RUNNER_SQLITE_DIR"
    )
shared_files_dir = _env_path(
    "YINSHI_RUNNER_SHARED_FILES_DIR",
    profile.default_shared_files_dir,
)
env_file_text = _env_text("YINSHI_RUNNER_ENV_FILE")
env_file = Path(env_file_text) if env_file_text else None
artifact_sha256 = _env_text("YINSHI_RUNNER_ARTIFACT_SHA256")
artifact_attestation_text = _env_text("YINSHI_RUNNER_ARTIFACT_ATTESTATION_FILE")
artifact_attestation_file = (
    Path(artifact_attestation_text) if artifact_attestation_text is not None else None
)
artifact_sha256 = _verify_artifact_attestation(
    profile.value,
    artifact_sha256,
    artifact_attestation_file,
)
relay_idle_timeout_seconds = _env_optional_positive_float(
    "YINSHI_RUNNER_RELAY_IDLE_TIMEOUT_SECONDS"
)
sprite_task_lease = _env_disabled_or_enabled("YINSHI_RUNNER_SPRITE_TASK_LEASE")
if sprite_task_lease and profile.value != _FLY_SPRITES_POSIX_PROFILE:
    raise RuntimeError(
        "YINSHI_RUNNER_SPRITE_TASK_LEASE requires fly_sprites_posix storage profile"
    )
return RunnerAgentConfig(

```

```

control_url=control_url.rstrip("/"),
registration_token=_env_text("YINSHI_REGISTRATION_TOKEN"),
runner_token_file=runner_token_file,
noise_private_key_file=noise_private_key_file,
data_protection_key_file=data_protection_key_file,
capability_signing_key_file=capability_signing_key_file,
replay_database_file=replay_database_file,
data_dir=data_dir,
sqlite_dir=sqlite_dir,
shared_files_dir=shared_files_dir,
storage_profile=profile.value,
sqlite_storage=sqlite_storage,
shared_files_storage=shared_files_storage,
user_data_encryption=_env_user_data_encryption(),
heartbeat_interval_s=_env_float(
    "YINSHI_RUNNER_HEARTBEAT_INTERVAL_S",
    _DEFAULT_HEARTBEAT_INTERVAL_S,
),
relay_idle_timeout_seconds=relay_idle_timeout_seconds,
sprite_task_lease=sprite_task_lease,
env_file=env_file,
artifact_sha256=artifact_sha256,
artifact_attestation_file=artifact_attestation_file,
)

```

```

def _probe_writable_directory(directory: Path, label: str) -> None:
    """Create and probe a POSIX directory required by the runner."""
    directory.mkdir(mode=0o700, parents=True, exist_ok=True)
    directory_flags = os.O_RDONLY | getattr(os, "O_DIRECTORY", 0)
    directory_flags |= getattr(os, "O_NOFOLLOW", 0) | getattr(os, "O_CLOEXEC", 0)
    try:
        directory_descriptor = os.open(directory, directory_flags)
    except OSError:
        raise RuntimeError(f"Runner {label} path is not a directory: {directory}") from None
    probe_name = f".yinshi-runner-write-check.{secrets.token_hex(16)}"
    failure_message = f"Runner {label} directory failed read-after-write check"
    probe_descriptor: int | None = None
    primary_error: BaseException | None = None
    cleanup_failed = False
    try:
        metadata = os.fstat(directory_descriptor)
        if not stat.S_ISDIR(metadata.st_mode):
            raise RuntimeError(f"Runner {label} path is not a directory: {directory}")
        if metadata.st_uid != os.geteuid():
            raise RuntimeError(f"Runner {label} directory is not owned by the runner user")
    
```

```

os.fchmod(directory_descriptor, 0o700)
probe_flags = os.O_RDWR | os.O_CREAT | os.O_EXCL
probe_flags |= getattr(os, "O_NOFOLLOW", 0) | getattr(os, "O_CLOEXEC", 0)
probe_descriptor = os.open(
    probe_name,
    probe_flags,
    0o600,
    dir_fd=directory_descriptor,
)
expected = b"ok\n"
if os.write(probe_descriptor, expected) != len(expected):
    raise RuntimeError(failure_message)
os.fsync(probe_descriptor)
os.lseek(probe_descriptor, 0, os.SEEK_SET)
if os.read(probe_descriptor, len(expected) + 1) != expected:
    raise RuntimeError(failure_message)
except OSError:
    primary_error = RuntimeError(failure_message)
except BaseException as error:
    primary_error = error
finally:
    if probe_descriptor is not None:
        try:
            os.close(probe_descriptor)
        except OSError:
            cleanup_failed = True
        try:
            os.unlink(probe_name, dir_fd=directory_descriptor)
        except OSError:
            cleanup_failed = True
    try:
        os.close(directory_descriptor)
    except OSError:
        cleanup_failed = True
if primary_error is not None:
    raise primary_error from None
if cleanup_failed:
    raise RuntimeError(failure_message)

def _shared_files_storage(shared_files_dir: Path) -> str:
    """Describe whether the shared file path is a mounted filesystem."""
    if shared_files_dir.is_mount():
        return _STORAGE_S3_FILES_MOUNT
    return _STORAGE_LOCAL_POSIX

```

```

def _validate_storage_layout(config: RunnerAgentConfig) -> RunnerStorageProfileSpec:
    """Reject path layouts that violate the selected storage profile."""
    profile = _storage_profile_spec(config.storage_profile)
    if profile.allow_sqlite_under_shared_files:
        return profile
    try:
        config.sqlite_dir.relative_to(config.shared_files_dir)
    except ValueError:
        return profile
    raise RuntimeError(
        "YINSHI_RUNNER_SQLITE_DIR must not live under YINSHI_RUNNER_SHARED_FILES_DIR"
    )

def _resolved_shared_files_storage(config: RunnerAgentConfig) -> str:
    """Return explicit shared storage, or detect AWS mount/local storage."""
    if config.shared_files_storage is not None:
        return config.shared_files_storage
    return _shared_files_storage(config.shared_files_dir)

def _capabilities(config: RunnerAgentConfig) -> dict[str, Any]:
    """Return storage and execution capabilities advertised to the control plane."""
    profile = _validate_storage_layout(config)
    _probe_writable_directory(config.data_dir, "data")
    _probe_writable_directory(config.sqlite_dir, "sqlite")
    _probe_writable_directory(config.shared_files_dir, "shared files")
    capabilities: dict[str, Any] = {
        "posix_storage": True,
        "sqlite": True,
        "git_worktrees": True,
        "pi_sidecar": True,
        "data_dir": str(config.data_dir),
        "sqlite_dir": str(config.sqlite_dir),
        "shared_files_dir": str(config.shared_files_dir),
        "storage_profile": profile.value,
        "storage_profile_experimental": profile.experimental,
        "sqlite_storage": config.sqlite_storage,
        "shared_files_storage": _resolved_shared_files_storage(config),
        "live_sqlite_on_shared_files": profile.live_sqlite_on_shared_files,
    }
    if config.artifact_sha256 is not None:
        capabilities["artifact_sha256"] = config.artifact_sha256
    return capabilities

```



```

def _read_owner_only_text_file(path: Path, label: str) -> str | None:
    """Read a regular owner-only text file without following symlinks."""
    if not path.exists() and not path.is_symlink():
        return None
    no_follow = getattr(os, "O_NOFOLLOW", 0)
    try:
        descriptor = os.open(path, os.O_RDONLY | no_follow)
    except OSError as exc:
        raise RuntimeError(f"{label} file could not be opened safely") from exc
    try:
        metadata = os.fstat(descriptor)
        if not stat.S_ISREG(metadata.st_mode):
            raise RuntimeError(f"{label} file must be regular")
        if metadata.st_uid != os.geteuid():
            raise RuntimeError(f"{label} file must be owned by the runner user")
        if stat.S_IMODE(metadata.st_mode) != 0o600:
            raise RuntimeError(f"{label} file must have owner-only permissions")
        encoded_value = os.read(descriptor, 8_193)
    finally:
        os.close(descriptor)
    if len(encoded_value) > 8_192:
        raise RuntimeError(f"{label} file is too large")
    try:
        value = encoded_value.decode("ascii").strip()
    except UnicodeDecodeError as exc:
        raise RuntimeError(f"{label} file must contain ASCII text") from exc
    if not value:
        raise RuntimeError(f"{label} file is empty")
    return value

def _verify_artifact_attestation(
    storage_profile: str,
    expected_sha256: str | None,
    attestation_file: Path | None,
) -> str | None:
    """Return artifact digest only after validating its private local attestation."""
    required = storage_profile == _FLY_SPRITES_POSIX_PROFILE
    if expected_sha256 is None:
        if required:
            raise RuntimeError("YINSHI_RUNNER_ARTIFACT_SHA256 is required for fly_sprites_p")
        if attestation_file is not None:
            raise RuntimeError(
                "YINSHI_RUNNER_ARTIFACT_SHA256 is required when "
                "YINSHI_RUNNER_ARTIFACT_ATTESTATION_FILE is set"
            )

```

```

        )
        return None
    if _SHA256_PATTERN.fullmatch(expected_sha256) is None:
        raise RuntimeError(
            "YINSHI_RUNNER_ARTIFACT_SHA256 must contain exactly "
            "64 lowercase hexadecimal characters"
        )
    if attestation_file is None:
        profile_suffix = " for fly_sprites_posix" if required else ""
        raise RuntimeError("YINSHI_RUNNER_ARTIFACT_ATTESTATION_FILE is required" + profile_suffix)
    if not attestation_file.is_absolute():
        raise RuntimeError("YINSHI_RUNNER_ARTIFACT_ATTESTATION_FILE must be an absolute path")
    if ".." in attestation_file.parts:
        raise RuntimeError(
            "YINSHI_RUNNER_ARTIFACT_ATTESTATION_FILE must not contain "
            "parent directory references"
        )
    attested_sha256 = _read_owner_only_text_file(
        attestation_file,
        "Runner artifact attestation",
    )
    if attested_sha256 is None:
        raise RuntimeError("Runner artifact attestation file is missing")
    if _SHA256_PATTERN.fullmatch(attested_sha256) is None:
        raise RuntimeError(
            "Runner artifact attestation must contain exactly "
            "64 lowercase hexadecimal characters"
        )
    if not secrets.compare_digest(attested_sha256, expected_sha256):
        raise RuntimeError("Runner artifact attestation does not match expected SHA-256")
    return expected_sha256

def _write_owner_only_text_file(path: Path, value: str, label: str) -> None:
    """Atomically persist one owner-only ASCII value."""
    if not isinstance(value, str) or not value.strip():
        raise RuntimeError(f"{label} must not be empty")
    try:
        encoded_value = f"{value.strip()}\n".encode("ascii")
    except UnicodeEncodeError as exc:
        raise RuntimeError(f"{label} must contain ASCII text") from exc
    path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    temporary_path = path.with_name(f"{path.name}.{secrets.token_hex(8)}.tmp")
    no_follow = getattr(os, "O_NOFOLLOW", 0)
    descriptor = os.open(
        temporary_path,

```

```

        os.O_WRONLY | os.O_CREAT | os.O_EXCL | no_follow,
        0o600,
    )
    try:
        written = os.write(descriptor, encoded_value)
        if written != len(encoded_value):
            raise RuntimeError(f"{label} write was incomplete")
        os.fsync(descriptor)
    except BaseException:
        temporary_path.unlink(missing_ok=True)
        raise
    finally:
        os.close(descriptor)
    os.replace(temporary_path, path)
    directory_flags = os.O_RDONLY | getattr(os, "O_DIRECTORY", 0)
    directory_descriptor = os.open(path.parent, directory_flags)
    try:
        os.fsync(directory_descriptor)
    finally:
        os.close(directory_descriptor)

def _read_runner_token(token_file: Path) -> str | None:
    """Read a previously issued runner bearer token from owner-only storage."""
    return _read_owner_only_text_file(token_file, "Runner token")

def _write_runner_token(token_file: Path, runner_token: str) -> None:
    """Persist the runner bearer token with owner-only permissions."""
    _write_owner_only_text_file(token_file, runner_token, "Runner token")

def _validate_capability_signing_public_key(value: object) -> str:
    """Return a canonical raw Ed25519 public key from a control response."""
    if not isinstance(value, str) or not value:
        raise RuntimeError("Control response did not include a capability signing key")
    try:
        key_bytes = base64.b64decode(value + "=", altchars=b"-_ ", validate=True)
    except (binascii.Error, ValueError) as exc:
        raise RuntimeError("Control capability signing key is not valid base64url") from exc
    canonical_value = base64.urlsafe_b64encode(key_bytes).rstrip(b"=").decode("ascii")
    if canonical_value != value or len(key_bytes) != 32:
        raise RuntimeError("Control capability signing key is not a canonical 32-byte key")
    return canonical_value

```

```

def _pin_capability_signing_key(path: Path, value: object) -> str:
    """Persist first control key and reject every later key change."""
    validated_value = _validate_capability_signing_public_key(value)
    existing_value = _read_owner_only_text_file(path, "Control capability signing key")
    if existing_value is None:
        _write_owner_only_text_file(path, validated_value, "Control capability signing key")
        return validated_value
    if not secrets.compare_digest(existing_value, validated_value):
        raise RuntimeError("Control capability signing key changed; runner re-pairing is required")
    return existing_value

def _scrub_registration_token(env_file: Path | None) -> None:
    """Remove the consumed one-time token from the systemd environment file."""
    if env_file is None:
        return
    if not env_file.exists():
        return
    lines = env_file.read_text(encoding="utf-8").splitlines()
    filtered_lines = [line for line in lines if not line.startswith(_REGISTRATION_TOKEN_ENV)]
    if filtered_lines == lines:
        return
    env_file.write_text("\n".join(filtered_lines) + "\n", encoding="utf-8")
    env_file.chmod(0o600)

def _runner_status_payload(config: RunnerAgentConfig) -> dict[str, Any]:
    """Build the runner status fields shared by registration and heartbeats."""
    return {
        "runner_version": RUNNER_VERSION,
        "capabilities": _capabilities(config),
        "data_dir": str(config.data_dir),
        "sqlite_dir": str(config.sqlite_dir),
        "shared_files_dir": str(config.shared_files_dir),
        "storage_profile": config.storage_profile,
    }

def _runner_registration_payload(config: RunnerAgentConfig) -> dict[str, Any]:
    """Build registration fields including the persistent Noise responder identity."""
    if config.registration_token is None:
        raise RuntimeError("YINSHI_REGISTRATION_TOKEN is required until a runner token file is provided")
    noise_keypair = load_or_create_runner_noise_keypair(config.noise_private_key_file)
    return {
        "registration_token": config.registration_token,
        "noise_public_key": noise_keypair.public_key_base64url,
    }

```

```

        **_runner_status_payload(config),
    }

async def _register(config: RunnerAgentConfig, client: httpx.AsyncClient) -> str:
    """Register this runner and return the issued bearer token."""
    payload = _runner_registration_payload(config)
    response = await client.post("/runner/register", json=payload)
    response.raise_for_status()
    body = response.json()
    runner_token = body.get("runner_token")
    if not isinstance(runner_token, str) or not runner_token.strip():
        raise RuntimeError("Runner registration response did not include a bearer token")
    _pin_capability_signing_key(
        config.capability_signing_key_file,
        body.get("capability_signing_public_key"),
    )
    _write_runner_token(config.runner_token_file, runner_token)
    _scrub_registration_token(config.env_file)
    logger.info("Registered Yinshi cloud runner")
    return runner_token

async def _heartbeat(
    config: RunnerAgentConfig,
    client: httpx.AsyncClient,
    runner_token: str,
) -> None:
    """Send one heartbeat to the control plane."""
    payload = _runner_status_payload(config)
    response = await client.post(
        "/runner/heartbeat",
        json=payload,
        headers={"Authorization": f"Bearer {runner_token}"},
    )
    response.raise_for_status()
    body = response.json()
    _pin_capability_signing_key(
        config.capability_signing_key_file,
        body.get("capability_signing_public_key"),
    )
    logger.info("Heartbeat accepted for Yinshi cloud runner")

def _runner_relay_url(control_url: str) -> str:
    """Convert one validated HTTP control origin to its WebSocket relay URL."""

```

```

parsed_url = urlsplit(control_url)
if parsed_url.scheme not in {"http", "https"} or not parsed_url.netloc:
    raise ValueError("YINSHI_CONTROL_URL must be an HTTP or HTTPS origin")
if parsed_url.path not in {"", "/"} or parsed_url.query or parsed_url.fragment:
    raise ValueError("YINSHI_CONTROL_URL must not include a path, query, or fragment")
websocket_scheme = "wss" if parsed_url.scheme == "https" else "ws"
return urlunsplit((websocket_scheme, parsed_url.netloc, "/runner/relay", "", ""))

def _runner_relay_runtime(
    config: RunnerAgentConfig,
    worker_manager: RunnerWorkerManager,
    task_lease: SpriteTaskLease | None,
) -> RunnerAgentRelayRuntime:
    """Load pinned key material for one fresh relay connection."""
    noise_keypair = load_or_create_runner_noise_keypair(config.noise_private_key_file)
    signing_key_text = _read_owner_only_text_file(
        config.capability_signing_key_file,
        "Control capability signing key",
    )
    if signing_key_text is None:
        raise RuntimeError("Control capability signing key is missing")
    validated_signing_key = _validate_capability_signing_public_key(signing_key_text)
    signing_key_bytes = base64.urlsafe_b64decode(validated_signing_key + "=")
    assert len(signing_key_bytes) == 32
    return RunnerAgentRelayRuntime(
        runner_static_private_key=noise_keypair.private_key,
        capability_signing_public_key=signing_key_bytes,
        replay_database_path=config.replay_database_file,
        dispatcher_factory=worker_manager.dispatcher,
        task_lease=task_lease,
        maintenance_handler=worker_manager.quiesce,
    )

async def _consume_runner_relay_messages(
    runtime: RunnerAgentRelayRuntime,
    websocket: ClientConnection,
    *,
    idle_timeout_seconds: float | None = None,
) -> bool:
    """Consume controls while dispatching one binary operation per transfer."""
    send_lock = asyncio.Lock()
    binary_tasks: dict[str, asyncio.Task[None]] = {}
    all_binary_tasks: set[asyncio.Task[None]] = set()
    protocol_error = asyncio.Event()

```

```

async def send(message: str | bytes) -> None:
    """Serialize writes from control and transfer tasks."""
    async with send_lock:
        await websocket.send(message)

async def cancel_binary_tasks(tasks: tuple[asyncio.Task[None], ...]) -> None:
    """Stop selected operations before their control state is retired."""
    for task in tasks:
        task.cancel()
    if tasks:
        await asyncio.gather(*tasks, return_exceptions=True)

async def apply_control(message: str) -> str | None:
    """Make close and maintenance controls barriers for active operations."""
    payload = json.loads(message)
    if isinstance(payload, dict) and payload.get("type") == "close":
        if set(payload) == {"transfer_id", "type"}:
            raw_transfer_id = payload.get("transfer_id")
            if isinstance(raw_transfer_id, str):
                try:
                    transfer_id = str(uuid.UUID(raw_transfer_id))
                except ValueError:
                    transfer_id = ""
                if transfer_id == raw_transfer_id:
                    task = binary_tasks.get(transfer_id)
                    await cancel_binary_tasks((task if task is None else (task,)))
    elif isinstance(payload, dict) and payload.get("type") == "quiesce":
        if set(payload) == {"job_id", "type"}:
            await cancel_binary_tasks(tuple(binary_tasks.values()))
    return await runtime.handle_control(message)

async def dispatch_binary(transfer_id: str, frame: bytes) -> None:
    """Run one transfer operation without blocking unrelated controls."""
    try:
        try:
            response = await runtime.handle_binary(
                frame,
                current_time=int(time.time()),
            )
        except RunnerRelaySessionError as error:
            await send(
                json.dumps(
                    {"transfer_id": error.transfer_id, "type": "close"},
                    separators=(",", ":"),
                    sort_keys=True,
                )
            )

```

```

        )
    )
    else:
        await send(response)
except asyncio.CancelledError:
    raise
except Exception:
    protocol_error.set()
    try:
        await websocket.close(code=1008, reason="Runner relay frame rejected")
    except Exception:
        pass
finally:
    current_task = asyncio.current_task()
    if binary_tasks.get(transfer_id) is current_task:
        binary_tasks.pop(transfer_id, None)

try:
    while True:
        try:
            if idle_timeout_seconds is not None and not runtime.active_transfer_ids:
                message = await asyncio.wait_for(
                    websocket.recv(),
                    timeout=idle_timeout_seconds,
                )
            else:
                message = await websocket.recv()
        except TimeoutError:
            return True
        return True
        if isinstance(message, str):
            acknowledgement = await apply_control(message)
            if acknowledgement is not None:
                await send(acknowledgement)
            continue
        frame = bytes(message)
        if len(frame) <= 16:
            raise ValueError("Runner relay frame has an invalid length")
        transfer_id = str(uuid.UUID(bytes=frame[:16]))
        if transfer_id in binary_tasks:
            raise ValueError("Runner relay transfer already has an active operation")
        task = asyncio.create_task(dispatch_binary(transfer_id, frame))
        binary_tasks[transfer_id] = task
        all_binary_tasks.add(task)
        task.add_done_callback(all_binary_tasks.discard)
except (RuntimeError, TypeError, ValueError):
    try:

```



```

        await websocket.close(code=1008, reason="Runner relay frame rejected")
    except Exception:
        pass
    raise RuntimeError("Runner relay protocol rejected a frame") from None
finally:
    await cancel_binary_tasks(tuple(all_binary_tasks))
    if protocol_error.is_set():
        raise RuntimeError("Runner relay protocol rejected a frame")

async def _serve_runner_relay_connection(
    config: RunnerAgentConfig,
    runner_token: str,
    worker_manager: RunnerWorkerManager,
    task_lease: SpriteTaskLease | None,
) -> bool:
    """Serve one outbound relay connection until transport closure or idle expiry."""
    runtime = _runner_relay_runtime(config, worker_manager, task_lease)
    try:
        async with connect(
            _runner_relay_url(config.control_url),
            origin=Origin(config.control_url.rstrip("/")),
            compression=None,
            additional_headers={"Authorization": f"Bearer {runner_token}"},
            proxy=True,
            open_timeout=_REQUEST_TIMEOUT_S,
            ping_interval=20.0,
            ping_timeout=20.0,
            close_timeout=5.0,
            max_size=65_551,
            max_queue=16,
        ) as websocket:
            return await _consume_runner_relay_messages(
                runtime,
                websocket,
                idle_timeout_seconds=config.relay_idle_timeout_seconds,
            )
    finally:
        await runtime.aclose()

async def _runner_relay_loop(config: RunnerAgentConfig, runner_token: str) -> None:
    """Reconnect the outbound relay with bounded backoff until cancelled."""
    task_lease = SpriteTaskLease() if config.sprite_task_lease else None
    try:
        noise_keypair = load_or_create_runner_noise_keypair(config.noise_private_key_file)

```

```

data_protection_key = load_or_create_runner_data_key(
    config.data_protection_key_file,
    config.sqlite_dir,
    noise_keypair.private_key,
)
worker_manager = RunnerWorkerManager(
    data_directory=config.data_dir / "worker-runtime",
    database_directory=config.sqlite_dir,
    user_data_directory=config.shared_files_dir / "users",
    data_protection_key=data_protection_key,
    user_data_encryption=config.user_data_encryption,
)
reconnect_delay_seconds = 1.0
while True:
    try:
        idle_expired = await _serve_runner_relay_connection(
            config,
            runner_token,
            worker_manager,
            task_lease,
        )
        if idle_expired:
            return
        reconnect_delay_seconds = 1.0
    except InvalidStatus as error:
        status_code = error.response.status_code
        if status_code in {401, 403}:
            raise RuntimeError("Runner relay authentication was rejected") from None
        logger.warning("Runner relay unavailable with HTTP status %s", status_code)
    except (ConnectionClosed, OSError, TimeoutError):
        logger.warning("Runner relay connection unavailable; retrying")
        await asyncio.sleep(reconnect_delay_seconds)
        reconnect_delay_seconds = min(reconnect_delay_seconds * 2, 30.0)
finally:
    if task_lease is not None:
        await task_lease.aclose()

async def _heartbeat_loop(
    config: RunnerAgentConfig,
    client: httpx.AsyncClient,
    runner_token: str,
) -> None:
    """Send recurring authenticated heartbeats until cancelled or revoked."""
    while True:
        try:

```

```

        await _heartbeat(config, client, runner_token)
    except httpx.HTTPStatusError as exc:
        if exc.response.status_code == 401:
            raise RuntimeError("Runner token was rejected by the control plane") from exc
        raise
    await asyncio.sleep(config.heartbeat_interval_s)

async def run_agent(config: RunnerAgentConfig) -> None:
    """Run heartbeats and outbound encrypted relay until either fails."""
    limits = httpx.Limits(max_connections=4, max_keepalive_connections=2)
    async with httpx.AsyncClient(
        base_url=config.control_url,
        timeout=_REQUEST_TIMEOUT_S,
        limits=limits,
        follow_redirects=False,
    ) as client:
        runner_token = _read_runner_token(config.runner_token_file)
        if runner_token is None:
            runner_token = await _register(config, client)
        else:
            pinned_key = _read_owner_only_text_file(
                config.capability_signing_key_file,
                "Control capability signing key",
            )
            if pinned_key is None:
                raise RuntimeError(
                    "Control capability signing key is missing; runner re-registration is required"
                )
            _validate_capability_signing_public_key(pinned_key)

        logger.info("Runner Noise identity loaded")
        heartbeat_task = asyncio.create_task(
            _heartbeat_loop(config, client, runner_token),
            name="runner-heartbeat",
        )
        relay_task = asyncio.create_task(
            _runner_relay_loop(config, runner_token),
            name="runner-relay",
        )
        tasks = (heartbeat_task, relay_task)
    try:
        done, pending = await asyncio.wait(
            tasks,
            return_when=asyncio.FIRST_COMPLETED,
        )

```

```

        for task in pending:
            task.cancel()
        await asyncio.gather(*pending, return_exceptions=True)
    for task in tasks:
        if task in done:
            task.result()
    finally:
        for task in tasks:
            if not task.done():
                task.cancel()
        await asyncio.gather(*tasks, return_exceptions=True)

def main() -> None:
    """Load configuration and run the cloud runner agent."""
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    )
    config = load_config()
    logger.info(
        "Starting Yinshi cloud runner agent against %s with profile %s",
        config.control_url,
        config.storage_profile,
    )
    asyncio.run(run_agent(config))

if __name__ == "__main__":
    main()

```

13.5 api/datadog_proxy.py

The Datadog proxy keeps browser telemetry intake on the Yinshi domain while validating the forwarded path. The root chunk below tangles back to backend/src/yinshi/api/datadog_proxy.py.

```

"""Proxy Datadog browser RUM/session-replay intake through our own domain.

```

```

Purpose: Some DNS-level blockers (e.g. NextDNS, Pi-hole) and ad-blockers target`browser-intake-datadoghq.com` directly, returning a blockpage certificate that breaks the browser SDK with `ERR_CERT_AUTHORITY_INVALID`. Proxying the intake through our own origin keeps telemetry working without exposing the Datadog hostname to clients.
"""

```

```

from __future__ import annotations

import asyncio
import logging
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager

import httpx
from fastapi import APIRouter, Depends, HTTPException, Request, Response

from yinshi.rate_limit import limiter

logger = logging.getLogger(__name__)

router = APIRouter(tags=["datadog-proxy"])

# Site-specific intake host. This proxy pins to US1 (`datadoghq.com`) because
# that is what `frontend/src/main.tsx` configures. If the frontend site ever
# changes, update this constant in the same commit.
DATADOG_INTAKE_HOST = "browser-intake-datadoghq.com"

# Only forward to well-known intake endpoints so the proxy cannot be abused as
# an open redirect to arbitrary Datadog paths.
ALLOWED_INTAKE_PATHS = frozenset({"rum", "replay", "logs"})

# Upper bound on request body size. RUM batches are small; session replay
# segments can be larger but rarely exceed a few hundred KB.
MAX_BODY_BYTES = 5 * 1024 * 1024 # 5 MiB

# Connect quickly but allow slower uploads for replay segments on poor links.
INTAKE_TIMEOUT_S = httpx.Timeout(connect=5.0, read=15.0, write=15.0, pool=5.0)
_INTAKE_CONCURRENCY_MAX = 16
_INTAKE_SLOT_WAIT_TIMEOUT_S = 0.1
_intake_concurrency = asyncio.Semaphore(_INTAKE_CONCURRENCY_MAX)

# Hop-by-hop and transport-specific headers that must not be relayed to the
# caller. See RFC 7230 section 6.1; `content-encoding`/`content-length` are
# dropped because the downstream framework re-computes them for our response.
HOP_BY_HOP_HEADERS = frozenset(
    {
        "connection",
        "content-encoding",
        "content-length",
        "keep-alive",
        "proxy-authenticate",
        "proxy-authorization",
    }

```

```

        "te",
        "trailer",
        "transfer-encoding",
        "upgrade",
    }
)

def _validate_intake_path(intake_path: str) -> str:
    """Return the validated intake path or raise 404."""
    if intake_path not in ALLOWED_INTAKE_PATHS:
        raise HTTPException(status_code=404, detail="Unknown intake path")
    return intake_path

@asynccontextmanager
async def _intake_concurrency_slot() -> AsyncIterator[None]:
    """Bound concurrent body reads and upstream requests across this process."""
    try:
        await asyncio.wait_for(
            _intake_concurrency.acquire(),
            timeout=_INTAKE_SLOT_WAIT_TIMEOUT_S,
        )
    except TimeoutError:
        raise HTTPException(
            status_code=503,
            detail="Intake concurrency limit reached",
            headers={"Retry-After": "1"},
        ) from None
    try:
        yield
    finally:
        _intake_concurrency.release()

async def _intake_concurrency_dependency() -> AsyncIterator[None]:
    """Hold one global intake slot for the duration of a proxy request."""
    async with _intake_concurrency_slot():
        yield

async def _read_bounded_body(request: Request) -> bytes:
    """Read request chunks while enforcing the intake memory boundary."""
    if not isinstance(request, Request):
        raise TypeError("request must be a Request")
    body = bytearray()

```

```

    async for chunk in request.stream():
        if len(body) + len(chunk) > MAX_BODY_BYTES:
            raise HTTPException(status_code=413, detail="Intake payload too large")
        body.extend(chunk)
    return bytes(body)

@router.post("/rum/api/v2/{intake_path}")
@limiter.limit("120/minute")
async def proxy_datadog_intake(
    intake_path: str,
    request: Request,
    _concurrency_slot: None = Depends(_intake_concurrency_dependency),
) -> Response:
    """Forward a browser SDK intake POST to Datadog and relay the response."""
    validated_intake_path = _validate_intake_path(intake_path)

    # Enforce a hard upper bound before buffering the body in memory.
    content_length_header = request.headers.get("content-length")
    if content_length_header is not None:
        try:
            content_length_bytes = int(content_length_header)
        except ValueError as error:
            raise HTTPException(status_code=400, detail="Invalid Content-Length") from error
        if content_length_bytes < 0:
            raise HTTPException(status_code=400, detail="Negative Content-Length")
        if content_length_bytes > MAX_BODY_BYTES:
            raise HTTPException(status_code=413, detail="Intake payload too large")

    body_bytes = await _read_bounded_body(request)

    # Preserve the original query string -- the SDK signs batches with
    # per-request parameters (e.g. `dd-api-key`, `dd-request-id`, `ddsource`).
    query_string = request.url.query

    target_url = f"https://{DATADOG_INTAKE_HOST}/api/v2/{validated_intake_path}"
    if query_string:
        target_url = f"{target_url}?{query_string}"

    # Only forward headers that Datadog actually uses. In particular, drop
    # `Host`, `Cookie`, and auth headers so we never leak app state upstream.
    content_type_header = request.headers.get("content-type", "text/plain; charset=UTF-8")
    forwarded_headers = {
        "Content-Type": content_type_header,
        "User-Agent": request.headers.get("user-agent", "yinshi-rum-proxy"),
    }

```

```

try:
    async with httpx.AsyncClient(timeout=INTAKE_TIMEOUT_S) as client:
        upstream_response = await client.post(
            target_url,
            content=body_bytes,
            headers=forwarded_headers,
        )
except httpx.TimeoutException:
    logger.warning("Datadog intake request timed out")
    raise HTTPException(status_code=504, detail="Intake timeout") from None
except httpx.HTTPError as error:
    logger.warning("Datadog intake transport error")
    raise HTTPException(status_code=502, detail="Intake unreachable") from error

# Strip hop-by-hop and transport-specific headers before relaying.
relayed_headers = {
    name: value
    for name, value in upstream_response.headers.items()
    if name.lower() not in HOP_BY_HOP_HEADERS
}

return Response(
    content=upstream_response.content,
    status_code=upstream_response.status_code,
    headers=relayed_headers,
    media_type=upstream_response.headers.get("content-type"),
)

```

14 Desktop Runtime Bridge

Desktop mode moves local privileges into a supervised helper. Device registration, short-lived access, terminal ownership, and helper startup remain explicit backend responsibilities.

14.1 yinshi/api/desktop_devices.py

These routes register desktop devices and exchange approved device identity for short-lived local access.

```

"""Hosted account API for registered desktop devices."""

from __future__ import annotations

from fastapi import APIRouter, HTTPException, Request, Response
from pydantic import BaseModel, ConfigDict

```



```

from yinshi.api.deps import require_tenant
from yinshi.rate_limit import limiter
from yinshi.services.desktop_devices import list_desktop_devices, revoke_desktop_device

router = APIRouter(prefix="/api/account/desktop-devices", tags=["desktop-devices"])

class DesktopDeviceOut(BaseModel):
    """Account-visible metadata for one registered desktop device."""

    model_config = ConfigDict(extra="forbid")

    id: str
    name: str
    created_at: int
    last_seen_at: int | None
    revoked_at: int | None

@router.get("", response_model=list[DesktopDeviceOut])
@limiter.limit("60/minute")
async def get_desktop_devices(request: Request) -> list[DesktopDeviceOut]:
    """List desktop registrations belonging to the authenticated account."""
    tenant = require_tenant(request)
    devices = list_desktop_devices(user_id=tenant.user_id)
    return [DesktopDeviceOut.model_validate(device, from_attributes=True) for device in devices]

@router.delete("/{device_id}", status_code=204)
@limiter.limit("30/minute")
async def delete_desktop_device(request: Request, device_id: str) -> Response:
    """Revoke one owned desktop registration without exposing foreign IDs."""
    tenant = require_tenant(request)
    if len(device_id) != 32:
        raise HTTPException(status_code=404, detail="Desktop device not found")
    revoked = revoke_desktop_device(user_id=tenant.user_id, device_id=device_id)
    if not revoked:
        raise HTTPException(status_code=404, detail="Desktop device not found")
    return Response(status_code=204)

```

14.2 yinshi/desktop_bootstrap.py

This bootstrap process validates desktop launch inputs before it starts the local backend and emits its connection record.

```

"""One-time inherited-capability bootstrap for the loopback desktop helper."""

from __future__ import annotations

import asyncio
import hashlib
import hmac
import re
import secrets
from http.cookies import CookieError, SimpleCookie

from starlette.types import ASGIApp, Message, Receive, Scope, Send

_BOOTSTRAP_HEADER = b"x-yinshi-bootstrap"
_BOOTSTRAP_PATH = "/desktop/bootstrap"
_SESSION_COOKIE = "yinshi_desktop_session"
_TOKEN_PATTERN = re.compile(r"^[A-Za-z0-9_-]{32,128}$")

def _hash_token(token: str) -> bytes:
    """Hash a validated capability before retaining it in helper memory."""
    if not isinstance(token, str):
        raise TypeError("token must be a string")
    if _TOKEN_PATTERN.fullmatch(token) is None:
        raise ValueError("token must be a 32-128 character base64url value")
    return hashlib.sha256(token.encode("ascii")).digest()

def _header_values(scope: Scope, name: bytes) -> list[bytes]:
    """Return every exact ASGI header value for duplicate rejection."""
    headers = scope.get("headers")
    if not isinstance(headers, list):
        return []
    return [value for header_name, value in headers if header_name.lower() == name]

def _cookie_token(scope: Scope) -> str | None:
    """Parse one host-only desktop session cookie without accepting duplicates."""
    cookie_values = _header_values(scope, b"cookie")
    if not cookie_values:
        return None
    try:
        cookie_text = b"; ".join(cookie_values).decode("ascii")
        cookies = SimpleCookie(cookie_text)
    except (CookieError, UnicodeDecodeError):
        return None

```

```

morsel = cookies.get(_SESSION_COOKIE)
if morsel is None or _TOKEN_PATTERN.fullmatch(morsel.value) is None:
    return None
return morsel.value

async def _send_http_response(
    send: Send,
    *,
    status_code: int,
    headers: list[tuple[bytes, bytes]] | None = None,
) -> None:
    """Send one empty, non-cacheable bootstrap boundary response."""
    response_headers = [(b"cache-control", b"no-store")]
    if headers is not None:
        response_headers.extend(headers)
    response_start: Message = {
        "type": "http.response.start",
        "status": status_code,
        "headers": response_headers,
    }
    response_body: Message = {"type": "http.response.body", "body": b""}
    await send(response_start)
    await send(response_body)

class DesktopBootstrapMiddleware:
    """Exchange one inherited nonce for a random HttpOnly helper session."""

    def __init__(self, application: ASGIApp, *, instance_nonce: str) -> None:
        if not callable(application):
            raise TypeError("application must be callable")
        self._application = application
        self._nonce_hash = _hash_token(instance_nonce)
        self._session_hash: bytes | None = None
        self._nonce_consumed = False
        self._lock = asyncio.Lock()

    def _session_is_valid(self, scope: Scope) -> bool:
        """Compare a presented session cookie to the in-memory session hash."""
        if self._session_hash is None:
            return False
        token = _cookie_token(scope)
        if token is None:
            return False
        return hmac.compare_digest(_hash_token(token), self._session_hash)

```

```

async def _bootstrap(self, scope: Scope, send: Send) -> None:
    """Consume one exact nonce and return a fresh browser session cookie."""
    if scope.get("method") != "POST":
        await _send_http_response(send, status_code=405)
        return
    nonce_values = _header_values(scope, _BOOTSTRAP_HEADER)
    if len(nonce_values) != 1:
        await _send_http_response(send, status_code=403)
        return
    try:
        nonce = nonce_values[0].decode("ascii")
        nonce_hash = _hash_token(nonce)
    except (UnicodeDecodeError, ValueError):
        await _send_http_response(send, status_code=403)
        return

    async with self._lock:
        if self._nonce_consumed:
            await _send_http_response(send, status_code=409)
            return
        if not hmac.compare_digest(nonce_hash, self._nonce_hash):
            await _send_http_response(send, status_code=403)
            return
        session_token = secrets.token_urlsafe(32)
        self._session_hash = _hash_token(session_token)
        self._nonce_consumed = True

    cookie = (f'{{_SESSION_COOKIE}}={{session_token}}; HttpOnly; Path=/; SameSite=Strict').encode("ascii")
    )
    await _send_http_response(
        send,
        status_code=204,
        headers=[(b"set-cookie", cookie)],
    )

async def __call__(self, scope: Scope, receive: Receive, send: Send) -> None:
    """Protect every helper HTTP and WebSocket route after bootstrap."""
    scope_type = scope.get("type")
    if scope_type not in {"http", "websocket"}:
        await self._application(scope, receive, send)
        return
    if scope_type == "http" and scope.get("path") == _BOOTSTRAP_PATH:
        await self._bootstrap(scope, send)
        return

```

```

    if self._session_is_valid(scope):
        await self._application(scope, receive, send)
        return
    if scope_type == "websocket":
        close_message: Message = {
            "type": "websocket.close",
            "code": 4401,
            "reason": "Not authenticated",
        }
        await send(close_message)
        return
    await _send_http_response(send, status_code=401)

```

14.3 yinshi/desktop_runtime.py

Desktop runtime assembly constrains local paths, selects loopback endpoints, and creates the helper configuration consumed by the Electron process.

"""Desktop helper process protocol and loopback runtime primitives."""

```

from __future__ import annotations

import argparse
import asyncio
import json
import os
import re
import secrets
import socket
from collections.abc import Sequence
from pathlib import Path

import uvicorn

DESKTOP_HELPER_PROTOCOL_VERSION = 1
_INSTANCE_NONCE_PATTERN = re.compile(r"^[A-Za-z0-9_-]{32,128}$")
_LOOPBACK_HOST = "127.0.0.1"

def serialize_ready_message(*, port: int, instance_nonce: str) -> bytes:
    """Return one validated newline-delimited readiness message."""
    if isinstance(port, bool) or not isinstance(port, int):
        raise TypeError("port must be an integer")
    if port < 1 or port > 65535:
        raise ValueError("port must be between 1 and 65535")
    if not isinstance(instance_nonce, str):

```

```

        raise TypeError("instance_nonce must be a string")
    if _INSTANCE_NONCE_PATTERN.fullmatch(instance_nonce) is None:
        raise ValueError("instance_nonce must be 32-128 base64url characters")

    payload = {
        "type": "ready",
        "protocolVersion": DESKTOP_HELPER_PROTOCOL_VERSION,
        "port": port,
        "instanceNonce": instance_nonce,
    }
    message = json.dumps(
        payload,
        ensure_ascii=True,
        separators=(",", ":"),
        sort_keys=True,
    )
    return f"{message}\n".encode("ascii")

def _write_all(file_descriptor: int, payload: bytes) -> None:
    """Write a small protocol message completely, then close the inherited pipe."""
    if isinstance(file_descriptor, bool) or not isinstance(file_descriptor, int):
        raise TypeError("file_descriptor must be an integer")
    if file_descriptor < 0:
        raise ValueError("file_descriptor must not be negative")
    if not payload:
        raise ValueError("payload must not be empty")

    remaining = memoryview(payload)
    try:
        while remaining:
            written = os.write(file_descriptor, remaining)
            if written <= 0:
                raise OSError("readiness pipe write made no progress")
            remaining = remaining[written:]
    finally:
        os.close(file_descriptor)

def _bind_loopback_socket() -> tuple[socket.socket, int]:
    """Bind an inheritable-disabled TCP socket to an OS-assigned loopback port."""
    listener = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        listener.set_inheritable(False)
        listener.bind((_LOOPBACK_HOST, 0))
        listener.listen(socket.SOMAXCONN)

```

```

        address = listener.getsockname()
        port = int(address[1])
        if port < 1 or port > 65535:
            raise RuntimeError("operating system returned an invalid loopback port")
        return listener, port
    except (OSError, RuntimeError):
        listener.close()
        raise

async def _wait_until_started(server: uvicorn.Server, task: asyncio.Task[None]) -> None:
    """Wait until Uvicorn finishes startup or propagate an early exit."""
    while not server.started:
        if task.done():
            await task
            raise RuntimeError("desktop helper exited before readiness")
        await asyncio.sleep(0.01)

async def serve_desktop_helper(
    *,
    ready_fd: int,
    asset_dir: str | Path | None = None,
) -> None:
    """Serve the restricted desktop app and signal readiness over one pipe."""
    if isinstance(ready_fd, bool) or not isinstance(ready_fd, int):
        raise TypeError("ready_fd must be an integer")
    if ready_fd < 0:
        raise ValueError("ready_fd must not be negative")

    from yinshi.config import get_settings
    from yinshi.desktop_bootstrap import DesktopBootstrapMiddleware
    from yinshi.main import create_app

    listener, port = _bind_loopback_socket()
    os.environ["FRONTEND_URL"] = f"http://127.0.0.1:{port}"
    get_settings.cache_clear()
    instance_nonce = secrets.token_urlsafe(32)
    application = DesktopBootstrapMiddleware(
        create_app(mode="desktop", desktop_asset_dir=asset_dir),
        instance_nonce=instance_nonce,
    )
    config = uvicorn.Config(
        application,
        host=_LOOPBACK_HOST,
        port=port,

```

```

        log_level="warning",
        access_log=False,
    )
    server = uvicorn.Server(config)
    server_task = asyncio.create_task(server.serve(sockets=[listener]))
    try:
        await _wait_until_started(server, server_task)
        ready_message = serialize_ready_message(
            port=port,
            instance_nonce=instance_nonce,
        )
        _write_all(ready_fd, ready_message)
        await server_task
    finally:
        if not server_task.done():
            server.should_exit = True
            await server_task
        listener.close()

def _parse_args(arguments: Sequence[str] | None) -> argparse.Namespace:
    """Parse the private desktop helper command-line contract."""
    parser = argparse.ArgumentParser(prog="yinshi-desktop-helper")
    parser.add_argument("--ready-fd", required=True, type=int)
    parser.add_argument("--asset-dir", type=Path)
    parsed = parser.parse_args(arguments)
    if parsed.ready_fd < 0:
        parser.error("--ready-fd must not be negative")
    return parsed

def main(arguments: Sequence[str] | None = None) -> None:
    """Run the desktop helper until terminated by its Electron parent."""
    parsed = _parse_args(arguments)
    asyncio.run(
        serve_desktop_helper(
            ready_fd=parsed.ready_fd,
            asset_dir=parsed.asset_dir,
        )
    )

if __name__ == "__main__":
    main()

```


14.4 yinshi/services/desktop_auth.py

Desktop authentication verifies device-bound bearer credentials without reusing browser session cookies.

```
"""Hosted account authorization records for desktop clients."""

from __future__ import annotations

import base64
import hashlib
import secrets
import time
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from urllib.parse import urlencode

from yinshi.db import get_control_db
from yinshi.services.desktop_tokens import (
    create_desktop_token,
    desktop_signing_public_key,
)
from yinshi.services.live_auth_sessions import signal_desktop_device_revoked

_AUTHORIZATION_TTL = timedelta(minutes=10)

class DesktopAuthorizationNotFoundError(Exception):
    """Raised when an opaque request id has no matching stored record."""

class DesktopAuthorizationExpiredError(Exception):
    """Raised when a stored request is past its authorization window."""

class DesktopAuthorizationUsedError(Exception):
    """Raised when a stored request or code has already been consumed."""

class DesktopCodeInvalidError(Exception):
    """Raised when a desktop authorization code cannot identify an account."""

class DesktopPkceMismatchError(Exception):
    """Raised when the submitted verifier does not match the stored challenge."""
```

```

class DesktopAccountUnavailableError(Exception):
    """Raised when the approved account can no longer receive credentials."""

class DesktopRefreshInvalidError(Exception):
    """Raised after an invalid, expired, revoked, or replayed refresh credential."""

@dataclass(frozen=True, slots=True)
class DesktopTokenExchange:
    """Initial account and credential material for one registered desktop."""

    access_token: str
    access_token_expires_at: int
    refresh_token: str
    refresh_token_expires_at: int
    account_lease: str
    account_lease_expires_at: int
    device_id: str
    signing_public_key: str
    user_id: str
    user_email: str

def _issue_desktop_tokens(
    *,
    user_id: str,
    user_email: str,
    device_id: str,
    issued_at: int,
) -> DesktopTokenExchange:
    """Create one internally consistent access, refresh, and lease credential set."""
    if not user_id or not user_email or not device_id:
        raise ValueError("desktop token identity fields must not be empty")
    if not isinstance(issued_at, int) or issued_at < 1:
        raise ValueError("issued_at must be a positive integer")

    access_expires_at = issued_at + 15 * 60
    lease_expires_at = issued_at + 30 * 24 * 60 * 60
    refresh_expires_at = issued_at + 90 * 24 * 60 * 60
    return DesktopTokenExchange(
        access_token=create_desktop_token(
            token_type="access",
            user_id=user_id,
            device_id=device_id,

```

```

        issued_at=issued_at,
        expires_at=access_expires_at,
    ),
    access_token_expires_at=access_expires_at,
    refresh_token=secrets.token_urlsafe(48),
    refresh_token_expires_at=refresh_expires_at,
    account_lease=create_desktop_token(
        token_type="lease",
        user_id=user_id,
        device_id=device_id,
        issued_at=issued_at,
        expires_at=lease_expires_at,
    ),
    account_lease_expires_at=lease_expires_at,
    device_id=device_id,
    signing_public_key=desktop_signing_public_key(),
    user_id=user_id,
    user_email=user_email,
)

@dataclass(frozen=True, slots=True)
class ApprovedDesktopAuthorization:
    """One callback URL containing a newly issued authorization code."""

    callback_url: str

@dataclass(frozen=True, slots=True)
class DesktopAuthorizationRequest:
    """One newly stored opaque desktop authorization request."""

    request_id: str
    authorize_url: str
    expires_at: datetime

def exchange_desktop_authorization_code(
    *,
    authorization_code: str,
    code_verifier: str,
    device_name: str,
) -> DesktopTokenExchange:
    """Atomically consume a PKCE code and store one desktop refresh credential."""
    for name, value in (
        ("authorization_code", authorization_code),

```

```

        ("code_verifier", code_verifier),
        ("device_name", device_name),
    ):
        if not isinstance(value, str):
            raise TypeError(f"{name} must be a string")
        if not value.strip():
            raise ValueError(f"{name} must not be empty")

    code_hash = hashlib.sha256(authorization_code.encode("utf-8")).hexdigest()
    verifier_challenge = (
        base64.urlsafe_b64encode(hashlib.sha256(code_verifier.encode("ascii")).digest())
        .rstrip(b"=")
        .decode("ascii")
    )
    current_time = int(time.time())

    with get_control_db() as database:
        database.execute("BEGIN IMMEDIATE")
        row = database.execute(
            """
            SELECT request_id_hash, user_id, code_challenge, expires_at, consumed_at
            FROM desktop_authorization_requests
            WHERE authorization_code_hash = ?
            """,
            (code_hash,),
        ).fetchone()
        if row is None or row["user_id"] is None:
            raise DesktopCodeInvalidError
        if row["consumed_at"] is not None:
            raise DesktopAuthorizationUsedError
        if int(row["expires_at"]) <= current_time:
            raise DesktopAuthorizationExpiredError
        if not secrets.compare_digest(row["code_challenge"], verifier_challenge):
            raise DesktopPkceMismatchError
        user = database.execute(
            "SELECT id, email FROM users WHERE id = ? AND status = 'active'",
            (row["user_id"],),
        ).fetchone()
        if user is None:
            raise DesktopAccountUnavailableError

    credentials = _issue_desktop_tokens(
        user_id=user["id"],
        user_email=user["email"],
        device_id=secrets.token_hex(16),
        issued_at=current_time,
    )

```

```

)
refresh_token_hash = hashlib.sha256(credentials.refresh_token.encode("utf-8")).hexdigest()
device_result = database.execute(
    """
    INSERT INTO desktop_devices (
        id, user_id, name, created_at,
        refresh_token_hash, refresh_token_expires_at, last_seen_at
    ) VALUES (?, ?, ?, ?, ?, ?, ?)
    """,
    (
        credentials.device_id,
        user["id"],
        device_name.strip(),
        current_time,
        refresh_token_hash,
        credentials.refresh_token_expires_at,
        current_time,
    ),
)
if device_result.rowcount != 1:
    raise RuntimeError("desktop device was not stored")
consume_result = database.execute(
    """
    UPDATE desktop_authorization_requests
    SET consumed_at = ?
    WHERE request_id_hash = ? AND consumed_at IS NULL
    """,
    (current_time, row["request_id_hash"]),
)
if consume_result.rowcount != 1:
    raise DesktopAuthorizationUsedError
database.commit()

return credentials


def rotate_desktop_refresh_token(*, refresh_token: str) -> DesktopTokenExchange:
    """Rotate a current refresh token or revoke its device when an old token reappears."""
    if not isinstance(refresh_token, str):
        raise TypeError("refresh_token must be a string")
    if len(refresh_token) < 32 or len(refresh_token) > 256:
        raise DesktopRefreshInvalidError

    refresh_token_hash = hashlib.sha256(refresh_token.encode("utf-8")).hexdigest()
    current_time = int(time.time())
    with get_control_db() as database:

```

```

database.execute("BEGIN IMMEDIATE")
device = database.execute(
    """
        SELECT d.*, u.email, u.status AS user_status
        FROM desktop_devices d
        JOIN users u ON u.id = d.user_id
        WHERE d.refresh_token_hash = ?
    """,
    (refresh_token_hash,),
).fetchone()
if device is None:
    used_token = database.execute(
        """
            SELECT device_id FROM desktop_used_refresh_tokens
            WHERE token_hash = ?
        """,
        (refresh_token_hash,),
    ).fetchone()
    if used_token is not None:
        database.execute(
            """
                UPDATE desktop_devices
                SET revoked_at = ?
                WHERE id = ? AND revoked_at IS NULL
            """,
            (current_time, used_token["device_id"]),
        )
        database.commit()
        signal_desktop_device_revoked(used_token["device_id"])
        raise DesktopRefreshInvalidError
    if device["user_status"] != "active" or device["revoked_at"] is not None:
        raise DesktopRefreshInvalidError
    if int(device["refresh_token_expires_at"]) <= current_time:
        raise DesktopRefreshInvalidError

credentials = _issue_desktop_tokens(
    user_id=device["user_id"],
    user_email=device["email"],
    device_id=device["id"],
    issued_at=current_time,
)
new_refresh_token_hash = hashlib.sha256(
    credentials.refresh_token.encode("utf-8")
).hexdigest()
database.execute(
    """

```

```

        INSERT INTO desktop_used_refresh_tokens (token_hash, device_id, rotated_at)
        VALUES (?, ?, ?)
        """
        (refresh_token_hash, device["id"], current_time),
    )
    result = database.execute(
        """
        UPDATE desktop_devices
        SET refresh_token_hash = ?, refresh_token_expires_at = ?, last_seen_at = ?
        WHERE id = ? AND refresh_token_hash = ? AND revoked_at IS NULL
        """
        (
            new_refresh_token_hash,
            credentials.refresh_token_expires_at,
            current_time,
            device["id"],
            refresh_token_hash,
        ),
    )
    if result.rowcount != 1:
        raise DesktopRefreshInvalidError
    database.commit()
    return credentials

def approve_desktop_authorization_request(
    *,
    request_id: str,
    user_id: str,
) -> ApprovedDesktopAuthorization:
    """Bind one pending request to a user and issue its one-time callback code."""
    if not isinstance(request_id, str) or len(request_id) < 32 or len(request_id) > 128:
        raise DesktopAuthorizationNotFoundError
    if not isinstance(user_id, str) or not user_id:
        raise ValueError("user_id must not be empty")

    request_id_hash = hashlib.sha256(request_id.encode("utf-8")).hexdigest()
    with get_control_db() as database:
        database.execute("BEGIN IMMEDIATE")
        row = database.execute(
            """
            SELECT redirect_uri, state, expires_at, approved_at,
                   authorization_code_hash
            FROM desktop_authorization_requests
            WHERE request_id_hash = ?
            """

```

```

        (request_id_hash,),
    ).fetchone()
    if row is None:
        raise DesktopAuthorizationNotFoundError
    if row["approved_at"] is not None or row["authorization_code_hash"] is not None:
        raise DesktopAuthorizationUsedError
    current_time = int(time.time())
    if int(row["expires_at"]) <= current_time:
        raise DesktopAuthorizationExpiredError

    authorization_code = secrets.token_urlsafe(32)
    authorization_code_hash = hashlib.sha256(authorization_code.encode("utf-8")).hexdigest()
    result = database.execute(
        """
        UPDATE desktop_authorization_requests
        SET user_id = ?, authorization_code_hash = ?, approved_at = ?
        WHERE request_id_hash = ?
        AND approved_at IS NULL
        AND authorization_code_hash IS NULL
        """,
        (user_id, authorization_code_hash, current_time, request_id_hash),
    )
    if result.rowcount != 1:
        raise DesktopAuthorizationUsedError
    database.commit()

    callback_query = urlencode({"code": authorization_code, "state": row["state"]})
    return ApprovedDesktopAuthorization(
        callback_url=f"{row['redirect_uri']}?{callback_query}",
    )

def create_desktop_authorization_request(
    *,
    redirect_uri: str,
    code_challenge: str,
    state: str,
    frontend_url: str,
) -> DesktopAuthorizationRequest:
    """Store and return one short-lived PKCE-bound desktop request."""
    for name, value in (
        ("redirect_uri", redirect_uri),
        ("code_challenge", code_challenge),
        ("state", state),
        ("frontend_url", frontend_url),
    ):

```



```

        if not isinstance(value, str):
            raise TypeError(f"{name} must be a string")
        if not value:
            raise ValueError(f"{name} must not be empty")

request_id = secrets.token_urlsafe(32)
if len(request_id) < 32:
    raise RuntimeError("generated desktop request id was unexpectedly short")
created_at = datetime.now(timezone.utc)
expires_at = created_at + _AUTHORIZATION_TTL
request_id_hash = hashlib.sha256(request_id.encode("utf-8")).hexdigest()

with get_control_db() as database:
    cursor = database.execute(
        """
        INSERT INTO desktop_authorization_requests (
            request_id_hash, created_at, expires_at,
            redirect_uri, code_challenge, state
        ) VALUES (?, ?, ?, ?, ?, ?)
        """,
        (
            request_id_hash,
            int(created_at.timestamp()),
            int(expires_at.timestamp()),
            redirect_uri,
            code_challenge,
            state,
        ),
    )
    if cursor.rowcount != 1:
        raise RuntimeError("desktop authorization request was not stored")
    database.commit()

authorize_url = f"{frontend_url.rstrip('/')}/auth/desktop/authorize/{request_id}"
return DesktopAuthorizationRequest(
    request_id=request_id,
    authorize_url=authorize_url,
    expires_at=expires_at,
)

```

14.5 yinshi/services/desktop_devices.py

Device records bind one desktop installation to its owner, revocation state, and public verification material.

```

"""Hosted desktop device listing and account-scoped revocation."""

from __future__ import annotations

import time
from dataclasses import dataclass

from yinshi.db import get_control_db
from yinshi.services.live_auth_sessions import signal_desktop_device_revoked

@dataclass(frozen=True, slots=True)
class DesktopDevice:
    """Persisted desktop device metadata safe to return to its account owner."""

    id: str
    name: str
    created_at: int
    last_seen_at: int | None
    revoked_at: int | None

def list_desktop_devices(*, user_id: str) -> list[DesktopDevice]:
    """Return every desktop device owned by one account, newest first."""
    if not isinstance(user_id, str):
        raise TypeError("user_id must be a string")
    if not user_id:
        raise ValueError("user_id must not be empty")
    with get_control_db() as database:
        rows = database.execute(
            """
            SELECT id, name, created_at, last_seen_at, revoked_at
            FROM desktop_devices
            WHERE user_id = ?
            ORDER BY created_at DESC, id ASC
            """,
            (user_id,),
        ).fetchall()
    return [
        DesktopDevice(
            id=row["id"],
            name=row["name"],
            created_at=int(row["created_at"]),
            last_seen_at=(int(row["last_seen_at"]) if row["last_seen_at"] is not None else None),
            revoked_at=(int(row["revoked_at"]) if row["revoked_at"] is not None else None),
        )
    ]

```

```

        for row in rows
    ]

def desktop_device_is_active(*, user_id: str, device_id: str) -> bool:
    """Return whether one desktop device still grants account authority."""
    for name, value in (("user_id", user_id), ("device_id", device_id)):
        if not isinstance(value, str):
            raise TypeError(f"{name} must be a string")
        if not value:
            raise ValueError(f"{name} must not be empty")
    with get_control_db() as database:
        row = database.execute(
            """
            SELECT d.revoked_at, u.status AS user_status
            FROM desktop_devices d
            JOIN users u ON u.id = d.user_id
            WHERE d.id = ? AND d.user_id = ?
            """,
            (device_id, user_id),
        ).fetchone()
    return row is not None and row["revoked_at"] is None and row["user_status"] == "active"

def revoke_desktop_device(*, user_id: str, device_id: str) -> bool:
    """Revoke one owned device and return false without exposing foreign devices."""
    for name, value in (("user_id", user_id), ("device_id", device_id)):
        if not isinstance(value, str):
            raise TypeError(f"{name} must be a string")
        if not value:
            raise ValueError(f"{name} must not be empty")

    with get_control_db() as database:
        database.execute("BEGIN IMMEDIATE")
        device = database.execute(
            "SELECT revoked_at FROM desktop_devices WHERE id = ? AND user_id = ?",
            (device_id, user_id),
        ).fetchone()
        if device is None:
            return False
        if device["revoked_at"] is None:
            result = database.execute(
                """
                UPDATE desktop_devices
                SET revoked_at = ?
                WHERE id = ? AND user_id = ? AND revoked_at IS NULL
            """

```

```

        """
        (int(time.time()), device_id, user_id),
    )
    if result.rowcount != 1:
        raise RuntimeError("desktop device revocation was not stored")
    database.commit()
    signal_desktop_device_revoked(device_id)
    return True

```

14.6 yinshi/services/desktop_terminal.py

Desktop terminal grants restrict channel access to the authenticated local device and active tenant.

```

"""Desktop-local terminal context resolution."""

from __future__ import annotations

import os
from dataclasses import dataclass
from pathlib import Path

from yinshi.config import get_settings
from yinshi.db import get_db
from yinshi.utils.paths import is_path_inside

@dataclass(frozen=True, slots=True)
class DesktopTerminalContext:
    """Validated host paths and sidecar socket for one local workspace terminal."""

    workspace_path: str
    repo_root_path: str
    socket_path: str

def _managed_directory(path_value: str, *, base_path: str) -> str:
    """Resolve an existing directory and require it to remain inside managed storage."""
    if not path_value or not os.path.isabs(path_value):
        raise PermissionError("Desktop terminal path is outside managed storage")
    try:
        resolved_path = str(Path(path_value).resolve(strict=True))
        resolved_base = str(Path(base_path).resolve(strict=True))
    except OSError as error:
        raise PermissionError("Desktop terminal path is outside managed storage") from error
    if not os.path.isdir(resolved_path) or not is_path_inside(resolved_path, resolved_base):

```

```

        raise PermissionError("Desktop terminal path is outside managed storage")
    return resolved_path

def resolve_desktop_terminal_context(workspace_id: str) -> DesktopTerminalContext:
    """Resolve one local workspace terminal context from app-managed storage."""
    if not isinstance(workspace_id, str):
        raise TypeError("workspace_id must be a string")
    if not workspace_id or len(workspace_id) > 128:
        raise ValueError("workspace_id must contain 1-128 characters")
    settings = get_settings()
    if not settings.allowed_repo_base or not os.path.isabs(settings.allowed_repo_base):
        raise PermissionError("Desktop repository storage is not configured")
    if not os.path.isabs(settings.sidecar_socket_path):
        raise PermissionError("Desktop sidecar socket path is invalid")

    with get_db() as database:
        row = database.execute(
            """
            SELECT w.path AS workspace_path, r.root_path AS repo_root_path
            FROM workspaces w
            JOIN repos r ON r.id = w.repo_id
            WHERE w.id = ?
            """,
            (workspace_id,),
        ).fetchone()
    if row is None:
        raise LookupError("Desktop workspace not found")

    profile_directory = str(Path(settings.db_path).resolve().parent)
    socket_path = str(Path(settings.sidecar_socket_path).resolve())
    if not is_path_inside(socket_path, profile_directory):
        raise PermissionError("Desktop sidecar socket is outside managed storage")
    return DesktopTerminalContext(
        workspace_path=_managed_directory(
            row["workspace_path"],
            base_path=settings.allowed_repo_base,
        ),
        repo_root_path=_managed_directory(
            row["repo_root_path"],
            base_path=settings.allowed_repo_base,
        ),
        socket_path=socket_path,
    )

```

14.7 yinshi/services/desktop_tokens.py

Short-lived desktop tokens limit credential exposure while preserving automatic local reconnection.

```
"""Asymmetric compact tokens for desktop access and offline account leases."""

from __future__ import annotations

import base64
import binascii
import json
import time
from dataclasses import dataclass
from typing import Literal

from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

from yinshi.config import get_settings

DesktopTokenType = Literal["access", "lease"]
_TOKEN_TYPE_HEADERS: dict[DesktopTokenType, str] = {
    "access": "YINSHI-ACCESS",
    "lease": "YINSHI-LEASE",
}

@dataclass(frozen=True, slots=True)
class VerifiedDesktopAccess:
    """Identity claims from one valid unexpired desktop access token."""

    user_id: str
    device_id: str

def _decode_base64url(value: str) -> bytes:
    """Strictly decode one unpadded compact-token segment."""
    if not isinstance(value, str) or not value:
        raise ValueError("compact token segment must not be empty")
    padding = "=" * (-len(value) % 4)
    try:
        decoded = base64.b64decode(
            f"{value}{padding}",

```

```

        altchars=b"-_",
        validate=True,
    )
except (binascii.Error, ValueError) as error:
    raise ValueError("compact token segment is not valid base64url") from error
canonical_value = base64.urlsafe_b64encode(decoded).rstrip(b"=").decode("ascii")
if canonical_value != value:
    raise ValueError("compact token segment is not canonical base64url")
return decoded

def _encode_base64url(value: bytes) -> str:
    """Encode one compact-token segment without padding."""
    if not isinstance(value, bytes):
        raise TypeError("value must be bytes")
    if not value:
        raise ValueError("value must not be empty")
    return base64.urlsafe_b64encode(value).rstrip(b"=").decode("ascii")

def _signing_key() -> Ed25519PrivateKey:
    """Derive a domain-separated Ed25519 key from the stable session secret."""
    secret = get_settings().secret_key.encode("utf-8")
    if len(secret) < 32:
        raise RuntimeError("desktop token signing requires a 32-byte session secret")
    seed = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=b"yinshi-desktop-token-signing-v1",
        info=b"Ed25519 compact tokens",
    ).derive(secret)
    if len(seed) != 32:
        raise RuntimeError("desktop token signing key derivation failed")
    return Ed25519PrivateKey.from_private_bytes(seed)

def verify_desktop_access_token(
    token: str,
    *,
    current_time: int | None = None,
) -> VerifiedDesktopAccess | None:
    """Verify one compact access token and return only its identity claims."""
    if not isinstance(token, str) or not token:
        return None
    segments = token.split(".")
    if len(segments) != 3:

```

```

        return None
    encoded_header, encoded_payload, encoded_signature = segments
    try:
        header = json.loads(_decode_base64url(encoded_header))
        payload = json.loads(_decode_base64url(encoded_payload))
        signature = _decode_base64url(encoded_signature)
    except (UnicodeDecodeError, ValueError, json.JSONDecodeError):
        return None
    if header != {"alg": "EdDSA", "typ": "YINSHI-ACCESS", "v": 1}:
        return None
    if not isinstance(payload, dict):
        return None
    if payload.get("aud") != "yinshi-desktop" or payload.get("v") != 1:
        return None

    user_id = payload.get("sub")
    device_id = payload.get("device_id")
    issued_at = payload.get("iat")
    expires_at = payload.get("exp")
    if not isinstance(user_id, str) or not user_id:
        return None
    if not isinstance(device_id, str) or not device_id:
        return None
    if not isinstance(issued_at, int) or not isinstance(expires_at, int):
        return None
    now = int(time.time()) if current_time is None else current_time
    if issued_at > now + 60 or expires_at <= now or expires_at - issued_at > 15 * 60:
        return None

    signing_input = f"{encoded_header}.{encoded_payload}".encode("ascii")
    try:
        _signing_key().public_key().verify(signature, signing_input)
    except InvalidSignature:
        return None
    return VerifiedDesktopAccess(user_id=user_id, device_id=device_id)

def desktop_signing_public_key() -> str:
    """Return the unpadded raw Ed25519 public key for desktop verification."""
    public_key = (
        _signing_key()
        .public_key()
        .public_bytes(
            encoding=serialization.Encoding.Raw,
            format=serialization.PublicFormat.Raw,
        )
    )

```



```

    )
    if len(public_key) != 32:
        raise RuntimeError("desktop token public key must contain 32 bytes")
    return _encode_base64url(public_key)

def create_desktop_token(
    *,
    token_type: DesktopTokenType,
    user_id: str,
    device_id: str,
    issued_at: int,
    expires_at: int,
) -> str:
    """Create one Ed25519-signed access token or offline account lease."""
    if token_type not in _TOKEN_TYPE_HEADERS:
        raise ValueError(f"Unsupported desktop token type: {token_type}")
    if not isinstance(user_id, str) or not user_id:
        raise ValueError("user_id must not be empty")
    if not isinstance(device_id, str) or not device_id:
        raise ValueError("device_id must not be empty")
    if not isinstance(issued_at, int) or not isinstance(expires_at, int):
        raise TypeError("token timestamps must be integers")
    if expires_at <= issued_at:
        raise ValueError("expires_at must be after issued_at")

    header = {
        "alg": "EdDSA",
        "typ": _TOKEN_TYPE_HEADERS[token_type],
        "v": 1,
    }
    payload = {
        "aud": "yinshi-desktop",
        "device_id": device_id,
        "exp": expires_at,
        "iat": issued_at,
        "sub": user_id,
        "v": 1,
    }
    encoded_header = _encode_base64url(
        json.dumps(header, separators=(",", ":"), sort_keys=True).encode("utf-8")
    )
    encoded_payload = _encode_base64url(
        json.dumps(payload, separators=(",", ":"), sort_keys=True).encode("utf-8")
    )
    signing_input = f"{encoded_header}.{encoded_payload}"

```

```

signature = _signing_key().sign(signing_input.encode("ascii"))
if len(signature) != 64:
    raise RuntimeError("desktop token signature must contain 64 bytes")
return f"{signing_input}.{_encode_base64url(signature)}"

```

15 Managed Runtime Control Plane

Hosted users receive isolated Fly Sprites. The control plane owns durable runtime identity, exact provider inventory, artifact rollout, network policy, task leases, reconciliation, and guarded deletion.

15.1 yinshi/api/managed_runtime.py

Managed runtime routes expose readiness, activation, backup requests, and sanitized operator status without returning provider credentials.

```

"""Authenticated managed runtime status and capability routes."""

from __future__ import annotations

from typing import Literal

from fastapi import APIRouter, HTTPException, Request
from pydantic import AliasChoices, BaseModel, ConfigDict, Field

from yinshi.api.deps import require_tenant
from yinshi.api.runners import _request_relay_url
from yinshi.models import RunnerCapabilityCreateIn, RunnerCapabilityOut
from yinshi.rate_limit import limiter
from yinshi.services.managed_backups import (
    ManagedBackupConflictError,
    get_managed_backup_operation,
    list_managed_backup_archives,
)
from yinshi.services.managed_runners import (
    ManagedRuntimeStatus,
    get_managed_runtime_status,
)
from yinshi.services.managed_runtime_manager import (
    ManagedRuntimeIdentityError,
    ManagedRuntimeProviderError,
    ManagedRuntimeStateError,
    ManagedRuntimeTimeoutError,
)
from yinshi.services.runner_capabilities import (
    RUNNER_PROTOCOL_VERSION,

```

```

        create_runner_capability,
    )
    from yinshi.services.runner_relay import store_runner_transfer_grant
    from yinshi.services.runners import get_managed_runner_for_user

class ManagedBackupOut(BaseModel):
    """Safe archive state without storage location or key material."""

    id: str
    status: str
    size_bytes: int | None
    created_at: str
    completed_at: str | None
    last_error: str | None

class ManagedBackupJobOut(BaseModel):
    """Safe maintenance progress without worker or provider ownership."""

    model_config = ConfigDict(from_attributes=True)

    id: str = Field(validation_alias=AliasChoices("id", "job_id"))
    archive_id: str
    operation: str
    status: str
    phase: str
    started_at: str
    updated_at: str
    last_error: str | None

class ManagedRuntimeOut(BaseModel):
    """Public runtime state without provider authority or tenant identity."""

    provider: Literal["local", "fly_sprites"]
    status: str
    artifact_version: str | None = None
    last_error: str | None = None
    runner_public_key: str | None = None

router = APIRouter(tags=["runtime"])

def _safe_runtime_status(

```

```

        user_id: str,
        runtime: ManagedRuntimeStatus | None,
    ) -> ManagedRuntimeOut:
        """Convert persisted state to fixed public response fields."""
        if runtime is None:
            return ManagedRuntimeOut(provider="fly_sprites", status="absent")
        runner = get_managed_runner_for_user(user_id)
        runner_public_key = None
        if (
            runner is not None
            and runner.get("id") == runtime.runner_id
            and runner.get("kind") == "managed"
            and runner.get("cloud_provider") == runtime.provider_name
            and runner.get("noise_key_confirmed")
        ):
            candidate = runner.get("noise_public_key")
            if isinstance(candidate, str) and candidate:
                runner_public_key = candidate
        return ManagedRuntimeOut(
            provider=runtime.provider_name,
            status=runtime.lifecycle_status,
            artifact_version=runtime.artifact_version,
            last_error=runtime.last_error,
            runner_public_key=runner_public_key,
        )

```

```

@router.get("/api/runtime/backups", response_model=list[ManagedBackupOut])
def list_runtime_backups(request: Request) -> list[ManagedBackupOut]:
    """Return bounded safe archive states owned by the authenticated tenant."""
    tenant = require_tenant(request)
    return [
        ManagedBackupOut(
            id=archive.id,
            status=archive.status,
            size_bytes=archive.size_bytes,
            created_at=archive.created_at,
            completed_at=archive.completed_at,
            last_error=archive.last_error,
        )
        for archive in list_managed_backup_archives(tenant.user_id)
    ]

```

```

@router.post(
    "/api/runtime/backups",

```

```

        response_model=ManagedBackupJobOut,
        status_code=202,
    )
    @limiter.limit("5/hour")
    def create_runtime_backup(request: Request) -> ManagedBackupJobOut:
        """Queue one encrypted managed backup for the authenticated tenant."""
        tenant = require_tenant(request)
        manager = getattr(request.app.state, "managed_backup_manager", None)
        if manager is None:
            raise HTTPException(status_code=503, detail="Managed backups are unavailable")
        try:
            job = manager.enqueue_create(tenant.user_id)
        except (ManagedBackupConflictError, ValueError) as error:
            raise HTTPException(status_code=409, detail="Managed backup state is invalid") from error
        manager.wake()
        return ManagedBackupJobOut.model_validate(job)

    def _queue_backup_mutation(
        request: Request,
        archive_id: str,
        method_name: str,
    ) -> ManagedBackupJobOut:
        tenant = require_tenant(request)
        manager = getattr(request.app.state, "managed_backup_manager", None)
        if manager is None:
            raise HTTPException(status_code=503, detail="Managed backups are unavailable")
        try:
            job = getattr(manager, method_name)(tenant.user_id, archive_id)
        except LookupError as error:
            raise HTTPException(status_code=404, detail="Managed backup was not found") from error
        except (ManagedBackupConflictError, ValueError) as error:
            raise HTTPException(status_code=409, detail="Managed backup state is invalid") from error
        manager.wake()
        return ManagedBackupJobOut.model_validate(job)

    @router.post(
        "/api/runtime/backups/{archive_id}/restore",
        response_model=ManagedBackupJobOut,
        status_code=202,
    )
    @limiter.limit("3/hour")
    def restore_runtime_backup(archive_id: str, request: Request) -> ManagedBackupJobOut:
        """Queue one tenant-owned replacement restore."""
        return _queue_backup_mutation(request, archive_id, "enqueue_restore")

```

```

@router.delete(
    "/api/runtime/backups/{archive_id}",
    response_model=ManagedBackupJobOut,
    status_code=202,
)
@limiter.limit("5/hour")
def delete_runtime_backup(archive_id: str, request: Request) -> ManagedBackupJobOut:
    """Queue one tenant-owned exact-version archive deletion."""
    return _queue_backup_mutation(request, archive_id, "enqueue_delete")


@router.get(
    "/api/runtime/backup-jobs/{job_id}",
    response_model=ManagedBackupJobOut,
)
def get_runtime_backup_job(job_id: str, request: Request) -> ManagedBackupJobOut:
    """Return safe progress for one tenant-owned managed maintenance job."""
    tenant = require_tenant(request)
    operation = get_managed_backup_operation(tenant.user_id, job_id)
    if operation is None:
        raise HTTPException(status_code=404, detail="Managed backup job was not found")
    return ManagedBackupJobOut(
        id=operation.job_id,
        archive_id=operation.archive_id,
        operation=operation.operation,
        status=operation.status,
        phase=operation.phase,
        started_at=operation.started_at,
        updated_at=operation.updated_at,
        last_error=operation.last_error,
    )


@router.get("/api/runtime", response_model=ManagedRuntimeOut)
def get_runtime(request: Request) -> ManagedRuntimeOut:
    """Return the authenticated tenant's safe runtime state."""
    tenant = require_tenant(request)
    manager = getattr(request.app.state, "managed_runtime_manager", None)
    if manager is None:
        return ManagedRuntimeOut(provider="local", status="ready")
    return _safe_runtime_status(
        tenant.user_id,
        get_managed_runtime_status(tenant.user_id),
    )

```

```

@router.post("/api/runtime/provision", response_model=ManagedRuntimeOut)
@limiter.limit("10/minute")
async def provision_runtime(request: Request) -> ManagedRuntimeOut:
    """Provision the tenant's managed runtime and return safe state."""
    tenant = require_tenant(request)
    public_launch_enabled = getattr(
        request.app.state,
        "sprites_public_launch_enabled",
        False,
    )
    if public_launch_enabled is not True:
        raise HTTPException(
            status_code=503,
            detail="Managed runtime public launch is disabled",
        )
    manager = getattr(request.app.state, "managed_runtime_manager", None)
    if manager is None:
        raise HTTPException(status_code=503, detail="Managed runtime is unavailable")
    try:
        runtime = await manager.provision(tenant.user_id)
    except ManagedRuntimeStateError as error:
        raise HTTPException(
            status_code=409,
            detail="Managed runtime state is invalid",
        ) from error
    except ManagedRuntimeIdentityError as error:
        raise HTTPException(
            status_code=409,
            detail="Managed runtime identity changed",
        ) from error
    except ManagedRuntimeProviderError as error:
        raise HTTPException(
            status_code=503,
            detail="Managed runtime provider unavailable",
        ) from error
    except ManagedRuntimeTimeoutError as error:
        raise HTTPException(
            status_code=503,
            detail="Managed runtime wake timed out",
        ) from error
    return _safe_runtime_status(tenant.user_id, runtime)

@router.post(

```

```

        "/api/runtime/capabilities",
        response_model=RunnerCapabilityOut,
        status_code=201,
    )
    @limiter.limit("60/minute")
    async def issue_managed_runtime_capability(
        body: RunnerCapabilityCreateIn,
        request: Request,
    ) -> RunnerCapabilityOut:
        """Wake and revalidate one managed runner before signing authority."""
        tenant = require_tenant(request)
        manager = getattr(request.app.state, "managed_runtime_manager", None)
        if manager is None:
            raise HTTPException(status_code=503, detail="Managed runtime is unavailable")
        try:
            runner = await manager.ensure_online(tenant.user_id)
        except ManagedRuntimeStateError as error:
            raise HTTPException(status_code=409, detail="Managed runtime state is invalid") from error
        except ManagedRuntimeIdentityError as error:
            raise HTTPException(status_code=409, detail="Managed runtime identity changed") from error
        except ManagedRuntimeProviderError as error:
            raise HTTPException(
                status_code=503,
                detail="Managed runtime provider unavailable",
            ) from error
        except ManagedRuntimeTimeoutError as error:
            raise HTTPException(status_code=503, detail="Managed runtime wake timed out") from error

        try:
            capability, claims = create_runner_capability(
                user_id=tenant.user_id,
                runner_id=runner.runner_id,
                runner_public_key=runner.runner_public_key,
                initiator_public_key=body.initiator_public_key,
                scopes=body.scopes,
                max_session_bytes=body.max_session_bytes,
            )
        except (TypeError, ValueError) as error:
            raise HTTPException(status_code=400, detail=str(error)) from error
        store_runner_transfer_grant(capability, claims)
        return RunnerCapabilityOut(
            capability=capability,
            transfer_id=claims.transfer_id,
            runner_id=claims.runner_id,
            runner_public_key=claims.runner_public_key,
            protocol=RUNNER_PROTOCOL_VERSION,

```



```

        issued_at=claims.issued_at,
        expires_at=claims.expires_at,
        max_frame_bytes=claims.max_frame_bytes,
        max_session_bytes=claims.max_session_bytes,
        relay_url=_request_relay_url(request, claims.transfer_id),
    )

```

15.2 yinshi/managed_sprite_cleanup.py

The cleanup command reports candidates by default and requires explicit confirmation before deleting durably owned Sprites.

```

"""Guarded operator command for managed Sprite cleanup."""

from __future__ import annotations

import argparse
import asyncio
import json
from collections.abc import Sequence
from datetime import timedelta
from typing import Any

import httpx

from yinshi.config import get_settings
from yinshi.services.managed_sprite_reconciliation import (
    ManagedSpriteReconciler,
    ManagedSpriteReconciliationResult,
)
from yinshi.services.sprites import SpritesClient


def _parser() -> argparse.ArgumentParser:
    """Build cleanup arguments without reading settings."""
    parser = argparse.ArgumentParser(prog="yinshi-managed-sprite-cleanup")
    parser.add_argument("--execute", action="store_true")
    parser.add_argument(
        "--confirm-delete-unreferenced-managed-sprites",
        action="store_true",
    )
    return parser


async def _run_cleanup(*, execute: bool) -> ManagedSpriteReconciliationResult:
    """Build validated provider dependencies and run one reconciliation."""

```

```

settings = get_settings()
api_token = settings.sprites_api_token
name_key = settings.sprites_name_key
if settings.managed_runtime_provider != "fly_sprites" or api_token is None or name_key is None:
    raise RuntimeError("Managed Sprite cleanup is unavailable")
restore_name_prefix = f"{settings.sprites_name_prefix}-restore"[:30].rstrip("-")
async with httpx.AsyncClient(
    base_url=settings.sprites_api_url,
    follow_redirects=False,
) as http_client:
    provider = SpritesClient(
        api_token=api_token.get_secret_value(),
        http_client=http_client,
    )
    reconciler = ManagedSpriteReconciler(
        provider=provider,
        name_prefix=settings.sprites_name_prefix,
        restore_name_prefix=restore_name_prefix,
        restore_name_key=name_key.get_secret_value(),
        grace=timedelta(seconds=settings.sprites_reconcile_grace_seconds),
    )
    return await reconciler.reconcile_once(dry_run=not execute)

def _result_payload(
    result: ManagedSpriteReconciliationResult,
    *,
    execute: bool,
) -> dict[str, Any]:
    """Return count-only output for one successful command."""
    return {
        "deleted": len(result.deleted),
        "deferred": result.deferred,
        "eligible": len(result.eligible),
        "examined": result.examined,
        "mode": "execute" if execute else "dry-run",
        "retained": result.retained,
        "status": "ok",
    }

def main(arguments: Sequence[str] | None = None) -> int:
    """Run one dry or explicitly confirmed cleanup pass."""
    parser = _parser()
    options = parser.parse_args(arguments)
    if options.execute != options.confirm_delete_unreferenced_managed_sprites:

```

```

        parser.error(
            "--execute and --confirm-delete-unreferenced-managed-sprites must be used together"
        )
    execute = bool(options.execute)
    try:
        result = asyncio.run(_run_cleanup(execute=execute))
    except Exception:
        print(json.dumps({"status": "failed"}, separators=(",", ":"), sort_keys=True))
        return 1
    print(
        json.dumps(
            _result_payload(result, execute=execute),
            separators=(",", ":"),
            sort_keys=True,
        )
    )
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

15.3 yinshi/services/managed_artifacts.py

Artifact metadata pins each guest release to an immutable version and expected digest before installation.

```

"""Secure retrieval for pinned managed guest artifacts."""

```

```

from __future__ import annotations

import asyncio
import hashlib
import hmac
import re
from urllib.parse import urlsplit

import httpx

MAX_ARTIFACT_BYTES = 10 * 1024 * 1024
CONNECT_TIMEOUT_SECONDS = 5.0
READ_TIMEOUT_SECONDS = 10.0
TOTAL_TIMEOUT_SECONDS = 30.0

class ManagedArtifactError(Exception):

```

```

        """Base error for managed artifact retrieval."""

class ManagedArtifactValidationError(ManagedArtifactError):
    """Raised when artifact metadata is invalid."""

class ManagedArtifactHTTPError(ManagedArtifactError):
    """Raised when the artifact server returns a non-success response."""

class ManagedArtifactSizeError(ManagedArtifactError):
    """Raised when artifact size constraints fail."""

class ManagedArtifactTransportError(ManagedArtifactError):
    """Raised when artifact transport fails."""

class ManagedArtifactTimeoutError(ManagedArtifactError):
    """Raised when artifact retrieval times out."""

class ManagedArtifactDigestError(ManagedArtifactError):
    """Raised when artifact digest verification fails."""

def _validate_artifact_url(artifact_url: str) -> None:
    """Validate an artifact URL without including it in failure details."""
    try:
        parsed_url = urlsplit(artifact_url)
        has_user_information = parsed_url.username is not None or parsed_url.password is not None
        hostname = parsed_url.hostname
    except (TypeError, ValueError):
        raise ManagedArtifactValidationError("Invalid artifact URL") from None

    if (
        parsed_url.scheme != "https"
        or hostname is None
        or has_user_information
        or "#" in artifact_url
    ):
        raise ManagedArtifactValidationError("Invalid artifact URL")

def _validate_expected_sha256(expected_sha256: str) -> None:

```

```

        """Require a canonical lowercase SHA-256 digest."""
    if (
        not isinstance(expected_sha256, str)
        or re.fullmatch(r"[0-9a-f]{64}", expected_sha256) is None
    ):
        raise ManagedArtifactValidationError("Invalid artifact digest")

    async def fetch_pinned_artifact(
        http_client: httpx.AsyncClient,
        artifact_url: str,
        expected_sha256: str,
    ) -> bytes:
        """Fetch one bounded HTTPS artifact and verify its pinned SHA-256 digest."""
        _validate_artifact_url(artifact_url)
        _validate_expected_sha256(expected_sha256)
        request_timeout = httpx.Timeout(
            connect=CONNECT_TIMEOUT_SECONDS,
            read=READ_TIMEOUT_SECONDS,
            write=CONNECT_TIMEOUT_SECONDS,
            pool=CONNECT_TIMEOUT_SECONDS,
        )
        try:
            async with asyncio.timeout(TOTAL_TIMEOUT_SECONDS):
                async with http_client.stream(
                    "GET",
                    artifact_url,
                    follow_redirects=False,
                    timeout=request_timeout,
                ) as response:
                    response.raise_for_status()
                    content_length_values = response.headers.get_list("content-length")
                    declared_length: int | None = None
                    if content_length_values:
                        declared_value = content_length_values[0]
                        if (
                            len(content_length_values) != 1
                            or re.fullmatch(r"[0-9]+", declared_value) is None
                        ):
                            raise ManagedArtifactSizeError("Invalid artifact size")
                        declared_length = int(declared_value)
                        if declared_length > MAX_ARTIFACT_BYTES:
                            raise ManagedArtifactSizeError("Invalid artifact size")
                    content_buffer = bytearray()
                    async for chunk in response.aiter_bytes():
                        content_buffer.extend(chunk)

```

```

        if len(content_buffer) > MAX_ARTIFACT_BYTES:
            raise ManagedArtifactSizeError("Invalid artifact size")
        content = bytes(content_buffer)
        if not content:
            raise ManagedArtifactSizeError("Invalid artifact size")
        if declared_length is not None and len(content) != declared_length:
            raise ManagedArtifactSizeError("Invalid artifact size")
    except httpx.HTTPStatusError:
        raise ManagedArtifactHTTPError("Artifact HTTP failure") from None
    except httpx.TimeoutException:
        raise ManagedArtifactTimeoutError("Artifact retrieval timed out") from None
    except httpx.RequestError:
        raise ManagedArtifactTransportError("Artifact transport failure") from None
    except TimeoutError:
        raise ManagedArtifactTimeoutError("Artifact retrieval timed out") from None

    actual_sha256 = hashlib.sha256(content).hexdigest()
    if not hmac.compare_digest(actual_sha256, expected_sha256):
        raise ManagedArtifactDigestError("Artifact digest mismatch")
    return content

```

15.4 yinshi/services/managed_guest_installer.py

Guest installation verifies the release archive, installs runtime files atomically, and preserves persistent tenant data across upgrades.

"""Install managed runner services inside a private Fly Sprite."""

```

from __future__ import annotations

import hashlib
import hmac
import logging
import re
import shlex
from collections.abc import Awaitable, Callable
from math import isfinite
from typing import Any, Literal, TypeVar

_CONFIG_ROOT = "/home/sprite/.config/yinshi"
_ARTIFACT_PATH = f"{_CONFIG_ROOT}/artifact.tar.gz"
_BOOTSTRAP_PATH = f"{_CONFIG_ROOT}/bootstrap.sh"
_RUNNER_ENV_PATH = f"{_CONFIG_ROOT}/runner.env"
_STORAGE_ENCRYPTION_MARKER_PATH = "/var/lib/yinshi/.yinshi-encrypted-storage"
_ARTIFACT_ATTESTATION_PATH = "/opt/yinshi/current/.artifact-sha256"
_MAX_FILE_BYTES = 10 * 1024 * 1024

```

```

_MAX_ENVIRONMENT_VALUE_CHARS = 4096
_MIN_BOOTSTRAP_TIMEOUT_SECONDS = 600.0
_MAX_BOOTSTRAP_TIMEOUT_SECONDS = 86400.0
_SPRITE_NAME_PATTERN = re.compile(r"[a-z0-9](?:[a-z0-9]{0,61}[a-z0-9])?\Z")
_ARTIFACT_VERSION_PATTERN = re.compile(r"[A-Za-z0-9][A-Za-z0-9._-]{0,127}\Z")
_SHA256_PATTERN = re.compile(r"[0-9a-f]{64}\Z")
_VARIABLE_CLAIM_KEYS = {"YINSHI_CONTROL_URL", "YINSHI_REGISTRATION_TOKEN"}
_FIXED_CLAIM_ENVIRONMENT = {
    "YINSHI_RUNNER_STORAGE_PROFILE": "fly_sprites_posix",
    "YINSHI_RUNNER_SQLITE_STORAGE": "local_posix",
    "YINSHI_RUNNER_SHARED_FILES_STORAGE": "local_posix",
    "YINSHI_RUNNER_DATA_DIR": "/var/lib/yinshi",
    "YINSHI_RUNNER_SQLITE_DIR": "/var/lib/yinshi/sqlite",
    "YINSHI_RUNNER_DATA_PROTECTION_KEY_FILE": (
        "/var/lib/yinshi/sqlite/.yinshi-data-protection-key"
    ),
    "YINSHI_RUNNER_SHARED_FILES_DIR": "/var/lib/yinshi/files",
    "YINSHI_RUNNER_TOKEN_FILE": "/var/lib/yinshi/runner-token",
    "YINSHI_RUNNER_NOISE_KEY_FILE": "/var/lib/yinshi/runner-noise.key",
    "YINSHI_RUNNER_CAPABILITY_SIGNING_KEY_FILE": ("/var/lib/yinshi/control-capability-signing-key"),
    "YINSHI_RUNNER_REPLAY_DATABASE_FILE": ("/var/lib/yinshi/runner-capability-replay.sqlite"),
    "YINSHI_RUNNER_ENV_FILE": "/etc/yinshi-runner.env",
}
_CLAIM_KEYS = _VARIABLE_CLAIM_KEYS | _FIXED_CLAIM_ENVIRONMENT.keys()
_PROVIDER_ERROR = "Managed Sprite installation failed"
_ProviderStage = Literal[
    "write_storage_marker",
    "write_artifact",
    "write_bootstrap",
    "write_runner_environment",
    "delete_stale_runner_token",
    "configure_bootstrap",
    "configure_sidecar",
    "configure_runner",
]
_Result = TypeVar("_Result")

logger = logging.getLogger(__name__)

def _validate_claim_environment(environment: dict[str, str]) -> None:
    """Reject claims outside the exact managed Fly Sprite runner contract."""
    if not isinstance(environment, dict) or environment.keys() != _CLAIM_KEYS:
        raise ValueError("environment must contain exactly the managed runner claim keys")
    if any(
        not isinstance(value, str)
    ):

```

```

        or not value
        or len(value) > _MAX_ENVIRONMENT_VALUE_CHARS
        or any(character in value for character in "\0\r\n")
        for value in environment.values()
    ):
        raise ValueError("environment values must be valid bounded strings")
    if any(environment[key] != value for key, value in _FIXED_CLAIM_ENVIRONMENT.items()):
        raise ValueError("environment fixed values do not match the managed runner profile")

async def _provider_call(
    call: Callable[[], Awaitable[_Result]],
    *,
    stage: _ProviderStage,
) -> _Result:
    """Run one provider call and log only its fixed stage on failure."""
    try:
        return await call()
    except Exception:
        logger.error("managed_sprite_installation_failed stage=%s", stage)
        raise RuntimeError(_PROVIDER_ERROR) from None

class ManagedGuestInstaller:
    """Write verified inputs and configure private managed Sprite services."""

    def __init__(
        self,
        *,
        client: Any,
        bootstrap_script: bytes,
        bootstrap_timeout_seconds: float,
        storage_encryption_confirmed: bool,
        clock: Callable[[], float],
        sleep: Callable[[float], Awaitable[None]],
    ) -> None:
        if (
            not isinstance(bootstrap_script, bytes)
            or not bootstrap_script
            or len(bootstrap_script) > _MAX_FILE_BYTES
        ):
            raise ValueError("bootstrap_script must be non-empty bounded bytes")
        if isinstance(bootstrap_timeout_seconds, bool) or not isinstance(
            bootstrap_timeout_seconds, (int, float)
        ):
            raise ValueError("bootstrap_timeout_seconds must be between 600 and 86400")

```



```

try:
    bootstrap_timeout = float(bootstrap_timeout_seconds)
except OverflowError:
    raise ValueError("bootstrap_timeout_seconds must be between 600 and 86400") from None
if (
    not isfinite(bootstrap_timeout)
    or not _MIN_BOOTSTRAP_TIMEOUT_SECONDS
    <= bootstrap_timeout
    <= _MAX_BOOTSTRAP_TIMEOUT_SECONDS
):
    raise ValueError("bootstrap_timeout_seconds must be between 600 and 86400")
if storage_encryption_confirmed is not True:
    raise ValueError("storage_encryption_confirmed must be explicitly true")
if not callable(clock):
    raise ValueError("clock must be callable")
if not callable(sleep):
    raise ValueError("sleep must be callable")
self._client = client
self._bootstrap_script = bootstrap_script
self._bootstrap_timeout_seconds = bootstrap_timeout
self._clock = clock
self._sleep = sleep

async def install(
    self,
    *,
    sprite_name: str,
    artifact: bytes,
    environment: dict[str, str],
    artifact_version: str,
    artifact_sha256: str,
) -> None:
    """Install one verified release and start its private services."""
    if _SPRITE_NAME_PATTERN.fullmatch(sprite_name) is None:
        raise ValueError("sprite_name must be a lowercase DNS label of 1 to 63 characters")
    if not isinstance(artifact, bytes) or not artifact or len(artifact) > _MAX_FILE_BYTES:
        raise ValueError("artifact must be non-empty bytes within the 10 MiB limit")
    if (
        not isinstance(artifact_version, str)
        or _ARTIFACT_VERSION_PATTERN.fullmatch(artifact_version) is None
    ):
        raise ValueError("artifact_version must be a valid release identifier")
    if _SHA256_PATTERN.fullmatch(artifact_sha256) is None:
        raise ValueError("artifact_sha256 must be exactly 64 lowercase hexadecimal characters")
    actual_sha256 = hashlib.sha256(artifact).hexdigest()
    if not hmac.compare_digest(actual_sha256, artifact_sha256):

```

```

        raise ValueError("artifact does not match artifact_sha256")
    _validate_claim_environment(environment)
    await _provider_call(
        lambda: self._client.write_file(
            sprite_name,
            path=_STORAGE_ENCRYPTION_MARKER_PATH,
            content=b"fly-sprites-encrypted-storage\n",
            mode="0600",
            mkdir=True,
        ),
        stage="write_storage_marker",
    )
    await _provider_call(
        lambda: self._client.write_file(
            sprite_name,
            path=_ARTIFACT_PATH,
            content=artifact,
            mode="0600",
            mkdir=True,
        ),
        stage="write_artifact",
    )
    await _provider_call(
        lambda: self._client.write_file(
            sprite_name,
            path=_BOOTSTRAP_PATH,
            content=self._bootstrap_script,
            mode="0700",
            mkdir=True,
        ),
        stage="write_bootstrap",
    )
    runner_environment = dict(environment)
    runner_environment.update(
        {
            "YINSHI_RUNNER_STORAGE_PROFILE": "fly_sprites_posix",
            "YINSHI_RUNNER_DATA_DIR": "/var/lib/yinshi",
            "YINSHI_RUNNER_ARTIFACT_SHA256": artifact_sha256,
            "YINSHI_RUNNER_ARTIFACT_ATTESTATION_FILE": _ARTIFACT_ATTESTATION_PATH,
            "YINSHI_RUNNER_USER_DATA_ENCRYPTION": "required",
            "YINSHI_RUNNER_SPRITE_TASK_LEASE": "enabled",
            "YINSHI_RUNNER_ENV_FILE": _RUNNER_ENV_PATH,
            "SIDECAR_SOCKET_PATH": "/var/lib/yinshi/sidecar.sock",
        }
    )
    runner_env = "".join(

```

```

        f"{key}={shlex.quote(runner_environment[key])}\\n" for key in sorted(runner_environment.items())
    ).encode("utf-8")
    await _provider_call(
        lambda: self._client.write_file(
            sprite_name,
            path=_RUNNER_ENV_PATH,
            content=runner_env,
            mode="0600",
            mkdir=True,
        ),
        stage="write_runner_environment",
    )
    await _provider_call(
        lambda: self._client.delete_file(
            sprite_name,
            path=_FIXED_CLAIM_ENVIRONMENT["YINSHI_RUNNER_TOKEN_FILE"],
        ),
        stage="delete_stale_runner_token",
    )
    await _provider_call(
        lambda: self._client.configure_service(
            sprite_name,
            service_name="yinshi-bootstrap",
            command="/bin/bash",
            args=(
                _BOOTSTRAP_PATH,
                _ARTIFACT_PATH,
                artifact_sha256,
                artifact_version,
            ),
            environment={},
            directory=_CONFIG_ROOT,
            needs=(),
            http_port=None,
            monitor_duration=self._bootstrap_timeout_seconds,
        ),
        stage="configure_bootstrap",
    )
    await _provider_call(
        lambda: self._client.configure_service(
            sprite_name,
            service_name="yinshi-sidecar",
            command="/usr/bin/env",
            args=("node", "/opt/yinshi/current/sidecar/src/index.js"),
            environment={"SIDECAR_SOCKET_PATH": "/var/lib/yinshi/sidecar.sock"},
            directory="/opt/yinshi/current/sidecar",
        ),

```

```

        needs=(),
        http_port=None,
        monitor_duration=None,
    ),
    stage="configure_sidecar",
)
runner_command = (
    "set -a; . /home/sprite/.config/yinshi/runner.env; set +a; "
    "exec /opt/yinshi/current/venv/bin/python -m yinshi.runner_agent"
)
await _provider_call(
    lambda: self._client.configure_service(
        sprite_name,
        service_name="yinshi-runner",
        command="/bin/bash",
        args=("-lc", runner_command),
        environment={},
        directory="/opt/yinshi/current/backend",
        needs=("yinshi-sidecar",),
        http_port=None,
        monitor_duration=None,
    ),
    stage="configure_runner",
)

```

15.5 yinshi/services/managed_hosted_runtime.py

This coordinator joins provider operations, registry ownership, activation, reconciliation, and backup capabilities behind one hosted runtime interface.

```

"""Compose hosted managed runtime capabilities and owned resources."""

from __future__ import annotations

from dataclasses import dataclass
from typing import Any

@dataclass(frozen=True, slots=True)
class HostedManagedRuntime:
    """Own hosted runtime services and narrow provider capabilities."""

    runtime_manager: Any
    backup_provider: Any
    inventory_provider: Any
    provider_http_client: Any

```

```

async def aclose(self) -> None:
    """Close runtime work before its provider transport."""
    error: BaseException | None = None
    try:
        await self.runtime_manager.aclose()
    except BaseException as caught:
        error = caught
    try:
        await self.provider_http_client.aclose()
    except BaseException as caught:
        if error is None:
            error = caught
    if error is not None:
        raise error

```

15.6 yinshi/services/managed_operation_failures.py

Typed failure records retain sanitized operational faults for health checks and later recovery.

```

"""Persist typed managed operation failures."""

from __future__ import annotations

from datetime import datetime
from typing import Literal

from yinshi.db import get_control_db
from yinshi.services.managed_backups import _timestamp

ManagedOperationFailureClass = Literal["restore_failed", "deletion_failed"]

def fail_managed_backup_operation(
    *,
    job_id: str,
    runtime_generation: int,
    lease_owner: str | None,
    lease_token: str | None,
    failure_class: ManagedOperationFailureClass,
    error_code: str,
    now: datetime,
) -> bool:
    """Persist one exact semantic operation failure for monitoring."""
    timestamp = _timestamp(now)

```

```

with get_control_db() as database:
    result = database.execute(
        """UPDATE managed_backup_operations
           SET status = 'failed', failure_class = ?, last_error = ?, updated_at = ?
           WHERE job_id = ? AND runtime_generation = ? AND status = 'running'
              AND lease_owner IS ? AND lease_token IS ?""",
        (
            failure_class,
            error_code,
            timestamp,
            job_id,
            runtime_generation,
            lease_owner,
            lease_token,
        ),
    )
    database.commit()
return result.rowcount == 1

```

15.7 yinshi/services/managed_runners.py

Managed runner records bind a tenant runtime to its relay identity, token generation, heartbeat state, and artifact version.

```

"""Atomic lifecycle state for managed runner infrastructure."""

from __future__ import annotations

import base64
import hashlib
import hmac
import re
import sqlite3
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from typing import Any, Literal, cast, get_args

from yinshi.db import get_control_db
from yinshi.services.runners import (
    _HEARTBEAT_ONLINE_WINDOW_SECONDS,
    _create_runner_registration_in_connection,
    _datetime_from_storage,
    _datetime_to_storage,
    _decode_capabilities,
    _require_user_id,
)

```

```

ManagedRuntimeLifecycleStatus = Literal["provisioning", "ready", "failed", "deleting"]
ManagedRuntimeErrorCode = Literal[
    "artifact_invalid",
    "provider_unavailable",
    "network_policy_failed",
    "bootstrap_failed",
    "runner_registration_failed",
    "runner_identity_changed",
    "wake_timeout",
    "checkpoint_failed",
    "delete_failed",
]

_MANAGED_RUNTIME_ERROR_CODES = frozenset(get_args(ManagedRuntimeErrorCode))
_SPRITE_PREFIX_PATTERN = re.compile(r"^[a-z0-9](?:[a-z0-9]{0,28}[a-z0-9])?$")
_SPRITE_DIGEST_LENGTH = 32
_PROVISIONING_STALE_AFTER = timedelta(minutes=10)

@dataclass(frozen=True, slots=True)
class ManagedRuntimeStatus:
    """Safe persisted status for one user's managed runtime."""

    user_id: str
    runner_id: str
    provider_name: Literal["fly_sprites"]
    sprite_name: str
    lifecycle_status: ManagedRuntimeLifecycleStatus
    generation: int
    artifact_version: str
    created_at: str
    updated_at: str
    last_error: ManagedRuntimeErrorCode | None

@dataclass(frozen=True, slots=True)
class ProvisioningClaimResult:
    """Provisioning ownership plus authority returned only to claim owners."""

    claimed: bool
    runtime: ManagedRuntimeStatus
    registration_token: str | None = None
    registration_token_expires_at: str | None = None
    control_url: str | None = None
    environment: dict[str, str] | None = None

```

```

@dataclass(frozen=True, slots=True)
class DeletionClaimResult:
    """Deletion ownership and persisted runtime state."""

    claimed: bool
    runtime: ManagedRuntimeStatus

ManagedRuntimeProvisioningClaim = ProvisioningClaimResult

def managed_sprite_name(user_id: str, *, prefix: str, secret_key: str) -> str:
    """Return one deterministic provider-safe name without exposing user identity."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(prefix, str) or _SPRITE_PREFIX_PATTERN.fullmatch(prefix) is None:
        raise ValueError("prefix must be a lowercase DNS label of 1 to 30 characters")
    if not isinstance(secret_key, str) or not secret_key:
        raise ValueError("secret_key must not be empty")
    digest = hmac.new(
        secret_key.encode("utf-8"),
        normalized_user_id.encode("utf-8"),
        hashlib.sha256,
    ).digest()
    encoded_digest = base64.b32encode(digest).decode("ascii").lower().rstrip("=")
    return f"{prefix}-{encoded_digest[:_SPRITE_DIGEST_LENGTH]}"

def _normalized_now(now: datetime | None) -> datetime:
    """Return injected or current UTC time at stable second precision."""
    value = now or datetime.now(timezone.utc)
    if not isinstance(value, datetime):
        raise TypeError("now must be a datetime")
    if value.tzinfo is None:
        raise ValueError("now must be timezone-aware")
    return value.astimezone(timezone.utc).replace(microsecond=0)

def _required_text(value: str, name: str) -> str:
    """Return normalized required text."""
    if not isinstance(value, str):
        raise TypeError(f"{name} must be a string")
    normalized = value.strip()
    if not normalized:
        raise ValueError(f"{name} must not be empty")

```



```

        return normalized

def _runtime_status(row: sqlite3.Row) -> ManagedRuntimeStatus:
    """Convert one runtime row to its safe typed representation."""
    return ManagedRuntimeStatus(
        user_id=row["user_id"],
        runner_id=row["runner_id"],
        provider_name=cast(Literal["fly_sprites"], row["provider_name"]),
        sprite_name=row["sprite_external_id"],
        lifecycle_status=cast(ManagedRuntimeLifecycleStatus, row["lifecycle_status"]),
        generation=row["generation"],
        artifact_version=row["artifact_version"],
        created_at=str(row["created_at"]),
        updated_at=str(row["updated_at"]),
        last_error=cast(ManagedRuntimeErrorCode | None, row["last_error"]),
    )

def activate_managed_restore_candidate(
    user_id: str,
    *,
    source_generation: int,
    candidate_runner_id: str,
    candidate_sprite_id: str,
    artifact_version: str,
    now: datetime,
    job_id: str,
    lease_token: str,
) -> bool:
    """Atomically promote one replacement runner and revoke old authority."""
    normalized_user_id = _require_user_id(user_id)
    if type(source_generation) is not int or source_generation <= 0:
        raise ValueError("source_generation must be a positive integer")
    candidate_runner = _required_text(candidate_runner_id, "candidate_runner_id")
    candidate_sprite = _required_text(candidate_sprite_id, "candidate_sprite_id")
    artifact = _required_text(artifact_version, "artifact_version")
    normalized_job_id = _required_text(job_id, "job_id")
    normalized_lease_token = _required_text(lease_token, "lease_token")
    now_text = _datetime_to_storage(_normalized_now(now)).replace("+00:00", "Z")
    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            runtime = database.execute(
                """SELECT runner_id FROM managed_runtimes
                WHERE user_id = ? AND generation = ? AND lifecycle_status = 'ready'""",

```

```

        (normalized_user_id, source_generation),
    ).fetchone()
    candidate = database.execute(
        """SELECT id FROM user_runners
           WHERE id = ? AND user_id = ? AND kind = 'managed_restore'
              AND status = 'online' AND runner_token_hash IS NOT NULL
              AND registered_at IS NOT NULL AND last_heartbeat_at IS NOT NULL
              AND noise_public_key IS NOT NULL
              AND noise_public_key_confirmed_at IS NOT NULL
              AND revoked_at IS NULL""",
        (candidate_runner, normalized_user_id),
    ).fetchone()
    if runtime is None or candidate is None:
        database.rollback()
        return False
    operation = database.execute(
        """SELECT 1 FROM managed_backup_operations
           WHERE user_id = ? AND job_id = ? AND operation = 'restore'
              AND status = 'running' AND runtime_generation = ?
              AND lease_token = ? AND lease_expires_at > ?""",
        (
            normalized_user_id,
            normalized_job_id,
            source_generation,
            normalized_lease_token,
            now_text,
        ),
    ).fetchone()
    if operation is None:
        database.rollback()
        return False
    old_runner_id = runtime["runner_id"]
    database.execute(
        """UPDATE user_runners
           SET status = 'revoked', revoked_at = ?, registration_token_hash = NULL,
              registration_token_expires_at = NULL, runner_token_hash = NULL
           WHERE id = ? AND user_id = ? AND kind = 'managed'""",
        (now_text, old_runner_id, normalized_user_id),
    )
    database.execute(
        """INSERT INTO managed_runtime_activation_guards (
           user_id, job_id, lease_token
        ) VALUES (?, ?, ?)""",
        (normalized_user_id, normalized_job_id, normalized_lease_token),
    )
    database.execute(

```

```

        "DELETE FROM user_runners WHERE user_id = ? AND kind = 'managed_retired'",
        (normalized_user_id,)),
    )
    database.execute(
        "UPDATE user_runners SET kind = 'managed_retired' WHERE id = ?",
        (old_runner_id,)),
    )
    database.execute(
        "UPDATE user_runners SET kind = 'managed' WHERE id = ?",
        (candidate_runner,)),
    )
    result = database.execute(
        """UPDATE managed_runtimes
        SET runner_id = ?, sprite_external_id = ?, generation = ?,
            artifact_version = ?, updated_at = ?, last_error = NULL
        WHERE user_id = ? AND generation = ? AND lifecycle_status = 'ready'""",
        (
            candidate_runner,
            candidate_sprite,
            source_generation + 1,
            artifact,
            now_text,
            normalized_user_id,
            source_generation,
        ),
    )
    if result.rowcount != 1:
        database.rollback()
        return False
    operation_result = database.execute(
        """UPDATE managed_backup_operations
        SET phase = 'activated', activation_generation = ?, updated_at = ?
        WHERE user_id = ? AND job_id = ? AND operation = 'restore'
            AND status = 'running' AND runtime_generation = ?
            AND lease_token = ? AND lease_expires_at > ?""",
        (
            source_generation + 1,
            now_text,
            normalized_user_id,
            normalized_job_id,
            source_generation,
            normalized_lease_token,
            now_text,
        ),
    )
    if operation_result.rowcount != 1:

```

```

        database.rollback()
        return False
    database.execute(
        "DELETE FROM managed_runtime_activation_guards WHERE user_id = ?",
        (normalized_user_id,),
    )
    database.commit()
    return True
except sqlite3.IntegrityError:
    database.rollback()
    return False
except Exception:
    database.rollback()
    raise

def get_managed_runtime_status(user_id: str) -> ManagedRuntimeStatus | None:
    """Return safe persisted state without registration authority."""
    normalized_user_id = _require_user_id(user_id)
    with get_control_db() as database:
        row = database.execute(
            "SELECT * FROM managed_runtimes WHERE user_id = ?",
            (normalized_user_id,),
        ).fetchone()
    if row is None:
        return None
    return _runtime_status(row)

def claim_managed_runtime_deletion(
    user_id: str,
    now: datetime,
) -> DeletionClaimResult | None:
    """Claim deletion and revoke linked managed runner authority atomically."""
    normalized_user_id = _require_user_id(user_id)
    current_time = _normalized_now(now)
    now_text = _datetime_to_storage(current_time).replace("+00:00", "Z")

    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            maintenance = database.execute(
                """SELECT 1 FROM managed_backup_operations
                WHERE user_id = ? AND status = 'running'""",
                (normalized_user_id,),
            ).fetchone()

```

```

if maintenance is not None:
    raise RuntimeError("managed runtime maintenance is active")
row = database.execute(
    """
    SELECT runtime.*, runner.kind AS runner_kind
    FROM managed_runtimes AS runtime
    JOIN user_runners AS runner ON runner.id = runtime.runner_id
    WHERE runtime.user_id = ?
    """,
    (normalized_user_id,),
).fetchone()
if row is None:
    database.commit()
    return None
if row["lifecycle_status"] == "deleting":
    database.commit()
    return DeletionClaimResult(claimed=False, runtime=_runtime_status(row))

runtime_result = database.execute(
    """
    INSERT OR REPLACE INTO managed_runtimes (
        user_id, runner_id, provider_name, sprite_external_id,
        lifecycle_status, generation, artifact_version,
        created_at, updated_at, last_error
    )
    SELECT user_id, runner_id, provider_name, sprite_external_id,
        'deleting', generation + 1, artifact_version,
        created_at, ?, NULL
    FROM managed_runtimes
    WHERE user_id = ? AND lifecycle_status != 'deleting'
    """,
    (now_text, normalized_user_id),
)
runner_result = database.execute(
    """
    UPDATE user_runners
    SET status = 'revoked', revoked_at = ?,
        registration_token_hash = NULL,
        registration_token_expires_at = NULL,
        runner_token_hash = NULL
    WHERE id = ? AND user_id = ? AND kind = 'managed'
    """,
    (now_text, row["runner_id"], normalized_user_id),
)
if runtime_result.rowcount != 1 or runner_result.rowcount != 1:
    raise RuntimeError("managed runtime deletion claim was not stored")

```

```

        claimed_row = database.execute(
            "SELECT * FROM managed_runtimes WHERE user_id = ?",
            (normalized_user_id,),
        ).fetchone()
        assert claimed_row is not None, "managed runtime deletion claim must exist"
        database.commit()
        return DeletionClaimResult(claimed=True, runtime=_runtime_status(claimed_row))
    except Exception:
        database.rollback()
        raise

def finalize_managed_runtime_deletion(user_id: str, generation: int) -> bool:
    """Delete one matching deleting runtime followed by its managed runner."""
    normalized_user_id = _require_user_id(user_id)
    if type(generation) is not int or generation <= 0:
        raise ValueError("generation must be a positive integer")

    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            row = database.execute(
                """
                SELECT runner_id
                FROM managed_runtimes
                WHERE user_id = ? AND generation = ?
                  AND lifecycle_status = 'deleting'
                """,
                (normalized_user_id, generation),
            ).fetchone()
            if row is None:
                database.rollback()
                return False
            runtime_result = database.execute(
                """
                DELETE FROM managed_runtimes
                WHERE user_id = ? AND generation = ?
                  AND lifecycle_status = 'deleting'
                """,
                (normalized_user_id, generation),
            )
            runner_result = database.execute(
                """
                DELETE FROM user_runners
                WHERE id = ? AND user_id = ? AND kind = 'managed'
                """,

```

```

        (row["runner_id"], normalized_user_id),
    )
    if runtime_result.rowcount != 1 or runner_result.rowcount != 1:
        raise RuntimeError("managed runtime deletion was not finalized")
    database.commit()
    return True
except Exception:
    database.rollback()
    raise

def refresh_managed_runtime_provisioning(
    user_id: str,
    generation: int,
    now: datetime,
) -> bool:
    """Refresh one current provisioning generation's ownership timestamp."""
    normalized_user_id = _require_user_id(user_id)
    if type(generation) is not int or generation <= 0:
        raise ValueError("generation must be a positive integer")
    current_time = _normalized_now(now)
    now_text = _datetime_to_storage(current_time).replace("+00:00", "Z")

    with get_control_db() as database:
        result = database.execute(
            """
            INSERT OR REPLACE INTO managed_runtimes (
                user_id, runner_id, provider_name, sprite_external_id,
                lifecycle_status, generation, artifact_version,
                created_at, updated_at, last_error
            )
            SELECT user_id, runner_id, provider_name, sprite_external_id,
                lifecycle_status, generation, artifact_version,
                created_at, ?, last_error
            FROM managed_runtimes
            WHERE user_id = ? AND generation = ?
                AND lifecycle_status = 'provisioning'
            """,
            (now_text, normalized_user_id, generation),
        )
        database.commit()
        return result.rowcount == 1

def mark_managed_runtime_ready(
    user_id: str,

```

```

        generation: int,
        now: datetime,
    ) -> bool:
        """Mark one current provisioning generation ready after runner validation."""
        normalized_user_id = _require_user_id(user_id)
        if type(generation) is not int or generation <= 0:
            raise ValueError("generation must be a positive integer")
        current_time = _normalized_now(now)
        now_text = _datetime_to_storage(current_time).replace("+00:00", "Z")

        with get_control_db() as database:
            try:
                database.execute("BEGIN IMMEDIATE")
                row = database.execute(
                    """
                    SELECT runtime.lifecycle_status, runtime.generation,
                           runtime.provider_name, runner.kind,
                           runner.cloud_provider, runner.status AS runner_status,
                           runner.last_heartbeat_at, runner.capabilities_json,
                           runner.noise_public_key,
                           runner.noise_public_key_confirmed_at
                    FROM managed_runtimes AS runtime
                    JOIN user_runners AS runner ON runner.id = runtime.runner_id
                    WHERE runtime.user_id = ?
                    """,
                    (normalized_user_id,),
                ).fetchone()
                heartbeat_at = None if row is None else _datetime_from_storage(row["last_heartbeat_at"])
                capabilities = {} if row is None else _decode_capabilities(row["capabilities_json"])
                heartbeat_age = (
                    None if heartbeat_at is None else (current_time - heartbeat_at).total_seconds()
                )
                heartbeat_is_current = (
                    heartbeat_age is not None and 0 <= heartbeat_age <= _HEARTBEAT_ONLINE_WINDOW_SECONDS
                )
                if (
                    row is None
                    or row["lifecycle_status"] != "provisioning"
                    or row["generation"] != generation
                    or row["provider_name"] != "fly_sprites"
                    or row["kind"] != "managed"
                    or row["cloud_provider"] != "fly_sprites"
                    or row["runner_status"] != "online"
                    or not heartbeat_is_current
                    or capabilities.get("storage_profile") != "fly_sprites_posix"
                    or not row["noise_public_key"]
                )

```



```

        or row["noise_public_key_confirmed_at"] is None
    ):
        database.rollback()
        return False
    result = database.execute(
        """
        INSERT OR REPLACE INTO managed_runtimes (
            user_id, runner_id, provider_name, sprite_external_id,
            lifecycle_status, generation, artifact_version,
            created_at, updated_at, last_error
        )
        SELECT user_id, runner_id, provider_name, sprite_external_id,
            'ready', generation, artifact_version,
            created_at, ?, NULL
        FROM managed_runtimes
        WHERE user_id = ? AND generation = ?
            AND lifecycle_status = 'provisioning'
        """,
        (now_text, normalized_user_id, generation),
    )
    database.commit()
    return result.rowcount == 1
except Exception:
    database.rollback()
    raise


def reconcile_managed_runtime_provisioning(
    active_owners: set[tuple[str, int]],
    now: datetime,
) -> int:
    """Fail abandoned provisioning while preserving current process ownership."""
    current_time = _normalized_now(now)
    now_text = _datetime_to_storage(current_time).replace("+00:00", "Z")

    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            rows = database.execute("""
                SELECT user_id, runner_id, generation
                FROM managed_runtimes
                WHERE lifecycle_status = 'provisioning'
            """).fetchall()
            changed = 0
            for row in rows:
                owner = (row["user_id"], row["generation"])

```

```

        if owner in active_owners:
            continue
        runtime_result = database.execute(
            """
            UPDATE managed_runtimes
            SET lifecycle_status = 'failed', updated_at = ?,
                last_error = 'provider_unavailable'
            WHERE user_id = ? AND generation = ?
              AND lifecycle_status = 'provisioning'
            """,
            (now_text, row["user_id"], row["generation"]),
        )
        if runtime_result.rowcount != 1:
            continue
        runner_result = database.execute(
            """
            UPDATE user_runners
            SET status = 'revoked', revoked_at = ?,
                registration_token_hash = NULL,
                registration_token_expires_at = NULL,
                runner_token_hash = NULL
            WHERE id = ? AND user_id = ? AND kind = 'managed'
            """,
            (now_text, row["runner_id"], row["user_id"]),
        )
        if runner_result.rowcount != 1:
            raise RuntimeError("managed runtime reconciliation was not stored")
        changed += 1
    database.commit()
    return changed
except Exception:
    database.rollback()
    raise

def mark_managed_runtime_failed(
    user_id: str,
    generation: int,
    error_code: str,
    now: datetime,
) -> bool:
    """Fail one current provisioning generation and revoke its runner authority."""
    normalized_user_id = _require_user_id(user_id)
    if type(generation) is not int or generation <= 0:
        raise ValueError("generation must be a positive integer")
    if error_code not in _MANAGED_RUNTIME_ERROR_CODES:

```

```

        raise ValueError("error_code is not an allowed managed runtime failure code")
    current_time = _normalized_now(now)
    now_text = _datetime_to_storage(current_time).replace("+00:00", "Z")

    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            row = database.execute(
                """
                SELECT runtime.*, runner.kind AS runner_kind
                FROM managed_runtimes AS runtime
                JOIN user_runners AS runner ON runner.id = runtime.runner_id
                WHERE runtime.user_id = ?
                """,
                (normalized_user_id,),
            ).fetchone()
            if (
                row is None
                or row["lifecycle_status"] != "provisioning"
                or row["generation"] != generation
                or row["runner_kind"] != "managed"
            ):
                database.rollback()
                return False
            runtime_result = database.execute(
                """
                INSERT OR REPLACE INTO managed_runtimes (
                    user_id, runner_id, provider_name, sprite_external_id,
                    lifecycle_status, generation, artifact_version,
                    created_at, updated_at, last_error
                )
                SELECT user_id, runner_id, provider_name, sprite_external_id,
                    'failed', generation, artifact_version,
                    created_at, ?, ?
                FROM managed_runtimes
                WHERE user_id = ? AND generation = ?
                    AND lifecycle_status = 'provisioning'
                """,
                (now_text, error_code, normalized_user_id, generation),
            )
            runner_result = database.execute(
                """
                UPDATE user_runners
                SET status = 'revoked', revoked_at = ?,
                    registration_token_hash = NULL,
                    registration_token_expires_at = NULL,

```

```

        runner_token_hash = NULL
        WHERE id = ? AND user_id = ? AND kind = 'managed'
        """
        (now_text, row["runner_id"], normalized_user_id),
    )
    if runtime_result.rowcount != 1 or runner_result.rowcount != 1:
        raise RuntimeError("managed runtime failure transition was not stored")
    database.commit()
    return True
except Exception:
    database.rollback()
    raise

def _observer(row: sqlite3.Row) -> ProvisioningClaimResult:
    """Return runtime state without bearer authority."""
    return ProvisioningClaimResult(claimed=False, runtime=_runtime_status(row))

def _owner(row: sqlite3.Row, registration: dict[str, Any]) -> ProvisioningClaimResult:
    """Return runtime state plus newly generated registration authority."""
    return ProvisioningClaimResult(
        claimed=True,
        runtime=_runtime_status(row),
        registration_token=registration["registration_token"],
        registration_token_expires_at=registration["registration_token_expires_at"],
        control_url=registration["control_url"],
        environment=registration["environment"],
    )

def claim_managed_runtime_provisioning(
    user_id: str,
    *,
    name_prefix: str,
    name_key: str,
    artifact_version: str,
    region: str,
    control_url: str,
    allow_upgrade: bool = False,
    now: datetime | None = None,
    stale_after: timedelta = _PROVISIONING_STALE_AFTER,
) -> ProvisioningClaimResult:
    """Claim provisioning with registration state in one immediate transaction."""
    if type(allow_upgrade) is not bool:
        raise TypeError("allow_upgrade must be a boolean")

```

```

normalized_user_id = _require_user_id(user_id)
normalized_artifact = _required_text(artifact_version, "artifact_version")
normalized_region = _required_text(region, "region")
current_time = _normalized_now(now)
if not isinstance(stale_after, timedelta) or stale_after.total_seconds() <= 0:
    raise ValueError("stale_after must be a positive timedelta")
generated_name = managed_sprite_name(
    normalized_user_id,
    prefix=name_prefix,
    secret_key=name_key,
)
now_text = _datetime_to_storage(current_time)

with get_control_db() as database:
    try:
        database.execute("BEGIN IMMEDIATE")
        row = database.execute(
            "SELECT * FROM managed_runtimes WHERE user_id = ?",
            (normalized_user_id,),
        ).fetchone()
        if row is not None:
            row_time = _datetime_from_storage(row["updated_at"])
            assert row_time is not None, "managed runtime timestamp must exist"
            is_stale = current_time - row_time >= stale_after
            should_claim = row["lifecycle_status"] == "failed" or (
                row["lifecycle_status"] == "provisioning" and is_stale
            )
            should_claim = should_claim or (
                allow_upgrade
                and row["lifecycle_status"] == "ready"
                and row["artifact_version"] != normalized_artifact
            )
            if not should_claim:
                database.commit()
                return _observer(row)
            sprite_name = row["sprite_external_id"]
            generation = int(row["generation"]) + 1
        else:
            sprite_name = generated_name
            generation = 1

    registration = _create_runner_registration_in_connection(
        database,
        normalized_user_id,
        name="Managed Fly Sprite",
        cloud_provider="fly_sprites",

```

```

        region=normalized_region,
        storage_profile="fly_sprites_posix",
        control_url=control_url,
        runner_kind="managed",
        now=current_time,
    )
runner_id = registration["runner"]["id"]
if row is None:
    database.execute(
        """
        INSERT INTO managed_runtimes (
            user_id, runner_id, provider_name, sprite_external_id,
            lifecycle_status, generation, artifact_version,
            created_at, updated_at, last_error
        ) VALUES (?, ?, 'fly_sprites', ?, 'provisioning', 1, ?, ?, ?, NULL)
        """,
        (
            normalized_user_id,
            runner_id,
            sprite_name,
            normalized_artifact,
            now_text,
            now_text,
        ),
    )
else:
    database.execute(
        """
        UPDATE managed_runtimes
        SET runner_id = ?, lifecycle_status = 'provisioning',
            generation = ?, artifact_version = ?, updated_at = ?,
            last_error = NULL
        WHERE user_id = ?
        """,
        (
            runner_id,
            generation,
            normalized_artifact,
            now_text,
            normalized_user_id,
        ),
    )
claimed_row = database.execute(
    "SELECT * FROM managed_runtimes WHERE user_id = ?",
    (normalized_user_id,),
).fetchone()

```

```

        assert claimed_row is not None, "managed runtime claim must exist"
        database.commit()
        return _owner(claimed_row, registration)
    except Exception:
        database.rollback()
        raise

```

15.8 yinshi/services/managed_runtime_manager.py

The manager serializes provisioning and upgrades while enforcing artifact, network, encryption, and ownership prerequisites.

```

"""Coordinate managed Fly Sprite provisioning."""

from __future__ import annotations

import asyncio
import hmac
from collections.abc import Awaitable, Callable
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from typing import Any, Protocol, TypeVar

import httpx

from yinshi.services.managed_artifacts import fetch_pinned_artifact
from yinshi.services.managed_backups import managed_backup_operation_is_running
from yinshi.services.managed_runners import (
    ManagedRuntimeStatus,
    ProvisioningClaimResult,
    claim_managed_runtime_provisioning,
    get_managed_runtime_status,
    mark_managed_runtime_failed,
    mark_managed_runtime_ready,
    reconcile_managed_runtime_provisioning,
    refresh_managed_runtime_provisioning,
)
from yinshi.services.managed_sprite_registry import register_managed_sprite_identity
from yinshi.services.runners import (
    _HEARTBEAT_ONLINE_WINDOW_SECONDS,
    _datetime_from_storage,
    create_managed_restore_registration,
    get_managed_restore_runner_for_job,
    get_managed_runner_for_user,
)
from yinshi.services.sprites import SpriteRecord

```

```

class ManagedRuntimeWakeError(RuntimeError):
    """Base error for safe managed runtime wake failures."""

class ManagedRuntimeProviderError(ManagedRuntimeWakeError):
    """The managed runtime provider could not restart the service."""

class ManagedRuntimeTimeoutError(ManagedRuntimeWakeError):
    """The managed runner did not become reachable before the deadline."""

class ManagedRuntimeStateError(ManagedRuntimeWakeError):
    """Persisted managed runtime state cannot authorize a wake."""

class ManagedRuntimeIdentityError(ManagedRuntimeWakeError):
    """The managed runner identity changed during a wake."""

_PROVIDER_ERROR_MESSAGE = "Managed runtime provider unavailable"
_TIMEOUT_ERROR_MESSAGE = "Managed runtime wake timed out"
_STATE_ERROR_MESSAGE = "Managed runtime state is invalid"
_MAINTENANCE_ERROR_MESSAGE = "Managed runtime maintenance is active"
_IDENTITY_ERROR_MESSAGE = "Managed runtime identity changed"
_PROVISIONING_HEARTBEAT_INTERVAL_SECONDS = 30.0

_Result = TypeVar("_Result")

@dataclass(frozen=True, slots=True)
class OnlineManagedRunner:
    """Validated managed runner identity authorized for capability issue."""

    runner_id: str
    runner_public_key: str

class ManagedRuntimeProvider(Protocol):
    """Provider operations needed during provisioning and wake."""

    async def get_sprite(self, name: str) -> SpriteRecord | None: ...

    async def create_sprite(self, name: str) -> SpriteRecord: ...

```



```

    async def delete_sprite(self, name: str) -> None: ...

    async def set_network_policy(
        self,
        name: str,
        *,
        allowed_domains: tuple[str, ...],
    ) -> None: ...

    async def restart_service(
        self,
        name: str,
        *,
        service_name: str,
        monitor_duration: float | None,
    ) -> None: ...

    async def stop_service(
        self,
        name: str,
        *,
        service_name: str,
        timeout_seconds: int,
    ) -> None: ...

class ManagedGuestInstaller(Protocol):
    """Install verified managed runner content inside a Sprite."""

    async def install(
        self,
        *,
        sprite_name: str,
        artifact: bytes,
        environment: dict[str, str],
        artifact_version: str,
        artifact_sha256: str,
    ) -> None: ...

class ManagedRuntimeManager:
    """Coordinate managed runtime provisioning."""

    def __init__(
        self,

```

```

*,
provider: ManagedRuntimeProvider,
guest_installer: ManagedGuestInstaller,
http_client: httpx.AsyncClient,
name_prefix: str,
name_key: str,
artifact_url: str,
artifact_sha256: str,
artifact_version: str,
allowed_domains: tuple[str, ...],
region: str,
control_url: str,
readiness_timeout_seconds: float,
is_runner_connected: Callable[[str], bool],
poll_interval_seconds: float = 1.0,
clock: Callable[[], datetime] | None = None,
sleep: Callable[[float], Awaitable[None]] = asyncio.sleep,
heartbeat_sleep: Callable[[float], Awaitable[None]] = asyncio.sleep,
register_sprite_identity: Callable[..., None] = register_managed_sprite_identity,
) -> None:
    self._provider = provider
    self._guest_installer = guest_installer
    self._http_client = http_client
    self._name_prefix = name_prefix
    self._name_key = name_key
    self._artifact_url = artifact_url
    self._artifact_sha256 = artifact_sha256
    self._artifact_version = artifact_version
    self._allowed_domains = allowed_domains
    self._region = region
    self._control_url = control_url
    if readiness_timeout_seconds <= 0:
        raise ValueError("readiness_timeout_seconds must be positive")
    if poll_interval_seconds <= 0:
        raise ValueError("poll_interval_seconds must be positive")
    self._readiness_timeout = timedelta(seconds=readiness_timeout_seconds)
    self._poll_interval_seconds = poll_interval_seconds
    self._is_runner_connected = is_runner_connected
    self._clock = clock or (lambda: datetime.now(timezone.utc))
    self._sleep = sleep
    self._heartbeat_sleep = heartbeat_sleep
    self._register_sprite_identity = register_sprite_identity
    self._fetch_restore_artifact: Callable[[], Awaitable[bytes]] = (
        lambda: fetch_pinned_artifact(
            self._http_client,
            self._artifact_url,

```

```

        self._artifact_sha256,
    )
)
self._create_restore_registration: Callable[[str, str], dict[str, Any]] = (
    self._new_restore_registration
)
self._get_restore_runner: Callable[[str, str], dict[str, Any] | None] = (
    get_managed_restore_runner_for_job
)
self._provisioning_tasks: dict[asyncio.Task[ManagedRuntimeStatus], tuple[str, int]]
self._online_lock = asyncio.Lock()
self._online_tasks: dict[str, asyncio.Task[OnlineManagedRunner]] = {}
self._closing = False

@property
def artifact_version(self) -> str:
    """Return the exact installed artifact version for replacement activation."""
    return self._artifact_version

async def reconcile_startup(self) -> int:
    """Fail abandoned provisioning before this manager serves requests."""
    active_owners = set(self._provisioning_tasks.values())
    return reconcile_managed_runtime_provisioning(active_owners, self._now())

async def aclose(self) -> None:
    """Cancel manager-owned work before closing the artifact client."""
    async with self._online_lock:
        self._closing = True
        online_tasks = list(self._online_tasks.values())
    for online_task in online_tasks:
        online_task.cancel()
    if online_tasks:
        await asyncio.gather(*online_tasks, return_exceptions=True)

    tracked = list(self._provisioning_tasks.items())
    for provisioning_task, _ in tracked:
        provisioning_task.cancel()
    if tracked:
        await asyncio.gather(
            *(provisioning_task for provisioning_task, _ in tracked),
            return_exceptions=True,
        )
    for provisioning_task, (user_id, generation) in tracked:
        if provisioning_task.cancelled():
            try:
                mark_managed_runtime_failed(

```

```

        user_id,
        generation,
        "provider_unavailable",
        self._now(),
    )
    except Exception:
        pass
    await self._http_client.aclose()

async def provision_restore_candidate(
    self,
    user_id: str,
    *,
    job_id: str,
    candidate_sprite_name: str,
    candidate_runner_id: str | None = None,
) -> OnlineManagedRunner:
    """Install a fresh candidate or resume one exact persisted replacement."""
    if not user_id or not job_id or not candidate_sprite_name:
        raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
    self._register_sprite_identity(
        sprite_name=candidate_sprite_name,
        identity_kind="restore_candidate",
        user_id=user_id,
        job_id=job_id,
        lifecycle_status="creating",
        now=self._now(),
    )
    sprite = await self._provider.get_sprite(candidate_sprite_name)
    if candidate_runner_id is not None:
        runner = self._get_restore_runner(user_id, job_id)
        if (
            sprite is None
            or runner is None
            or runner.get("id") != candidate_runner_id
            or runner.get("kind") != "managed_restore"
            or runner.get("status") != "online"
            or runner.get("noise_key_confirmed") is not True
            or not self._artifact_attestation_matches(runner)
            or not self._is_runner_connected(candidate_runner_id)
        ):
            raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
        noise_key = runner.get("noise_public_key")
        if not isinstance(noise_key, str) or not noise_key:
            raise ManagedRuntimeIdentityError(_IDENTITY_ERROR_MESSAGE)
        return OnlineManagedRunner(candidate_runner_id, noise_key)

```

```

registration = self._create_restore_registration(user_id, job_id)
candidate_runner_id = registration["runner"]["id"]
environment = registration.get("environment")
if not isinstance(environment, dict):
    raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
artifact = await self._fetch_restore_artifact()
if sprite is None:
    sprite = await self._provider.create_sprite(candidate_sprite_name)
if sprite.name != candidate_sprite_name:
    raise ManagedRuntimeProviderError(_PROVIDER_ERROR_MESSAGE)
await self._provider.set_network_policy(
    candidate_sprite_name,
    allowed_domains=self._allowed_domains,
)
await self._guest_installer.install(
    sprite_name=candidate_sprite_name,
    artifact=artifact,
    environment=dict(environment),
    artifact_version=self._artifact_version,
    artifact_sha256=self._artifact_sha256,
)
deadline = self._now() + self._readiness_timeout
while True:
    runner = self._get_restore_runner(user_id, job_id)
    if runner is not None:
        noise_key = runner.get("noise_public_key")
        ready = (
            runner.get("id") == candidate_runner_id
            and runner.get("kind") == "managed_restore"
            and runner.get("status") == "online"
            and runner.get("registered_at") is not None
            and isinstance(noise_key, str)
            and runner.get("noise_key_confirmed") is True
            and self._artifact_attestation_matches(runner)
            and self._is_runner_connected(candidate_runner_id)
        )
        if ready:
            for service in ("yinshi-runner", "yinshi-sidecar"):
                await self._provider.stop_service(
                    candidate_sprite_name,
                    service_name=service,
                    timeout_seconds=30,
                )
            assert isinstance(noise_key, str)
            return OnlineManagedRunner(candidate_runner_id, noise_key)
    if self._now() >= deadline:

```

```

        raise ManagedRuntimeTimeoutError(_TIMEOUT_ERROR_MESSAGE)
    await self._sleep(self._poll_interval_seconds)

def _new_restore_registration(self, user_id: str, job_id: str) -> dict[str, Any]:
    """Create candidate authority bound to one exact restore job."""
    return create_managed_restore_registration(
        user_id,
        job_id=job_id,
        name="Managed restore candidate",
        region=self._region,
        control_url=self._control_url,
    )

async def verify_restore_candidate(
    self,
    user_id: str,
    *,
    job_id: str,
    candidate_runner_id: str,
    candidate_sprite_name: str,
    expected_public_key: str,
) -> None:
    """Revalidate candidate identity, attestation, heartbeat, and relay connection."""
    if not job_id or not candidate_sprite_name:
        raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
    deadline = self._now() + self._readiness_timeout
    while True:
        runner = self._get_restore_runner(user_id, job_id)
        if runner is not None:
            noise_key = runner.get("noise_public_key")
            if noise_key != expected_public_key:
                raise ManagedRuntimeIdentityError(_IDENTITY_ERROR_MESSAGE)
            if (
                runner.get("id") == candidate_runner_id
                and runner.get("kind") == "managed_restore"
                and runner.get("status") == "online"
                and runner.get("noise_key_confirmed") is True
                and self._artifact_attestation_matches(runner)
                and self._is_runner_connected(candidate_runner_id)
            ):
                return
        if self._now() >= deadline:
            raise ManagedRuntimeTimeoutError(_TIMEOUT_ERROR_MESSAGE)
        await self._sleep(self._poll_interval_seconds)

async def ensure_online(self, user_id: str) -> OnlineManagedRunner:

```

```

        """Join one manager-owned wake operation for the user's runtime."""
    if not isinstance(user_id, str) or not user_id:
        raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
    if managed_backup_operation_is_running(user_id):
        raise ManagedRuntimeStateError(_MAINTENANCE_ERROR_MESSAGE)
    async with self._online_lock:
        if self._closing:
            raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
        task = self._online_tasks.get(user_id)
        if task is None:
            task = asyncio.create_task(self._ensure_online(user_id))
            self._online_tasks[user_id] = task
            task.add_done_callback(self._online_task_finished)
    return await asyncio.shield(task)

async def _ensure_online(self, user_id: str) -> OnlineManagedRunner:
    """Wake one ready managed runtime and return its validated identity."""
    runtime, runner, expected_noise_key = self._load_ready_runner(user_id)
    if self._runner_is_connected(runtime, runner):
        return OnlineManagedRunner(runtime.runner_id, expected_noise_key)

    try:
        await self._provider.restart_service(
            runtime.sprite_name,
            service_name="yinshi-runner",
            monitor_duration=None,
        )
    except Exception:
        raise ManagedRuntimeProviderError(_PROVIDER_ERROR_MESSAGE) from None

    deadline = self._now() + self._readiness_timeout
    while True:
        current_runtime, runner, current_noise_key = self._load_ready_runner(user_id)
        if (
            current_runtime.runner_id != runtime.runner_id
            or current_runtime.sprite_name != runtime.sprite_name
        ):
            raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
        if current_noise_key != expected_noise_key:
            raise ManagedRuntimeIdentityError(_IDENTITY_ERROR_MESSAGE)
        if self._runner_is_connected(current_runtime, runner):
            return OnlineManagedRunner(runtime.runner_id, expected_noise_key)
        if self._now() >= deadline:
            raise ManagedRuntimeTimeoutError(_TIMEOUT_ERROR_MESSAGE)
        await self._sleep(self._poll_interval_seconds)

```

```

def _load_ready_runner(
    self,
    user_id: str,
) -> tuple[ManagedRuntimeStatus, dict[str, Any], str]:
    """Load one internally consistent ready runtime and confirmed runner."""
    try:
        runtime = get_managed_runtime_status(user_id)
        runner = get_managed_runner_for_user(user_id)
    except Exception:
        raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE) from None
    if (
        runtime is None
        or runtime.lifecycle_status != "ready"
        or runtime.provider_name != "fly_sprites"
        or runner is None
        or runner.get("id") != runtime.runner_id
        or runner.get("kind") != "managed"
        or runner.get("cloud_provider") != runtime.provider_name
    ):
        raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
    noise_key = runner.get("noise_public_key")
    if not isinstance(noise_key, str) or not runner.get("noise_key_confirmed"):
        raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
    return runtime, runner, noise_key

def _runner_is_connected(
    self,
    runtime: ManagedRuntimeStatus,
    runner: dict[str, Any],
) -> bool:
    """Return whether heartbeat and relay state authorize immediate use."""
    now = self._now()
    try:
        heartbeat_at = _datetime_from_storage(runner.get("last_heartbeat_at"))
        connected = self._is_runner_connected(runtime.runner_id)
    except Exception:
        raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE) from None
    if heartbeat_at is None:
        return False
    heartbeat_age = (now - heartbeat_at).total_seconds()
    return (
        runner.get("status") == "online"
        and 0 <= heartbeat_age <= _HEARTBEAT_ONLINE_WINDOW_SECONDS
        and connected
    )

```



```

def _online_task_finished(
    self,
    task: asyncio.Task[OnlineManagedRunner],
) -> None:
    """Consume one wake result and remove only its current registry entry."""
    for user_id, current_task in tuple(self._online_tasks.items()):
        if current_task is task:
            self._online_tasks.pop(user_id, None)
            break
    if not task.cancelled():
        task.exception()

async def provision(
    self,
    user_id: str,
    allow_upgrade: bool = False,
) -> ManagedRuntimeStatus:
    """Return observer state without external calls when another caller owns claim."""
    if managed_backup_operation_is_running(user_id):
        raise ManagedRuntimeStateError(_MAINTENANCE_ERROR_MESSAGE)
    claim = claim_managed_runtime_provisioning(
        user_id,
        name_prefix=self._name_prefix,
        name_key=self._name_key,
        artifact_version=self._artifact_version,
        region=self._region,
        control_url=self._control_url,
        allow_upgrade=allow_upgrade,
        now=self._now(),
    )
    if not claim.claimed:
        return claim.runtime

    generation = claim.runtime.generation
    task = asyncio.create_task(self._complete_provision(user_id, claim))
    self._provisioning_tasks[task] = (user_id, generation)
    task.add_done_callback(self._provisioning_task_finished)
    return await asyncio.shield(task)

async def _complete_provision(
    self,
    user_id: str,
    claim: ProvisioningClaimResult,
) -> ManagedRuntimeStatus:
    """Complete manager-owned provisioning after a successful claim."""
    generation = claim.runtime.generation

```

```

ownership_lost = asyncio.Event()
heartbeat = asyncio.create_task(
    self._maintain_provisioning_heartbeat(
        user_id,
        generation,
        ownership_lost,
    )
)
try:
    return await self._run_provisioning(user_id, claim, ownership_lost)
finally:
    heartbeat.cancel()
    await asyncio.gather(heartbeat, return_exceptions=True)

async def _run_provisioning(
    self,
    user_id: str,
    claim: ProvisioningClaimResult,
    ownership_lost: asyncio.Event,
) -> ManagedRuntimeStatus:
    """Run external provisioning work for one current owner."""
    generation = claim.runtime.generation
    if claim.runtime.provider_name != "fly_sprites":
        return self._fail(user_id, generation, "provider_unavailable", claim.runtime)
    if claim.environment is None:
        return self._fail(
            user_id,
            generation,
            "runner_registration_failed",
            claim.runtime,
        )
    try:
        runner = get_managed_runner_for_user(user_id)
    except Exception:
        return self._fail(
            user_id,
            generation,
            "runner_registration_failed",
            claim.runtime,
        )
    if (
        runner is None
        or runner.get("id") != claim.runtime.runner_id
        or runner.get("kind") != "managed"
        or runner.get("cloud_provider") != "fly_sprites"
    ):

```

```

        return self._fail(
            user_id,
            generation,
            "runner_registration_failed",
            claim.runtime,
        )
    expected_noise_key = (
        runner.get("noise_public_key") if runner.get("noise_key_confirmed") else None
    )
    try:
        artifact = await self._await_owned(
            fetch_pinned_artifact(
                self._http_client,
                self._artifact_url,
                self._artifact_sha256,
            ),
            user_id,
            generation,
            ownership_lost,
        )
    except ManagedRuntimeStateError:
        raise
    except Exception:
        return self._fail(user_id, generation, "artifact_invalid", claim.runtime)

    try:
        self._register_sprite_identity(
            sprite_name=claim.runtime.sprite_name,
            identity_kind="runtime",
            user_id=user_id,
            job_id=None,
            lifecycle_status="creating",
            now=self._now(),
        )
        sprite = await self._await_owned(
            self._provider.get_sprite(claim.runtime.sprite_name),
            user_id,
            generation,
            ownership_lost,
        )
    if sprite is None:
        sprite = await self._await_owned(
            self._provider.create_sprite(claim.runtime.sprite_name),
            user_id,
            generation,
            ownership_lost,

```

```

        )
        if sprite.name != claim.runtime.sprite_name:
            raise ValueError("inconsistent Sprite name")
    except ManagedRuntimeStateError:
        raise
    except Exception:
        return self._fail(user_id, generation, "provider_unavailable", claim.runtime)

    try:
        await self._await_owned(
            self._provider.set_network_policy(
                claim.runtime.sprite_name,
                allowed_domains=self._allowed_domains,
            ),
            user_id,
            generation,
            ownership_lost,
        )
    except ManagedRuntimeStateError:
        raise
    except Exception:
        return self._fail(
            user_id,
            generation,
            "network_policy_failed",
            claim.runtime,
        )

    try:
        await self._await_owned(
            self._guest_installer.install(
                sprite_name=claim.runtime.sprite_name,
                artifact=artifact,
                environment=dict(claim.environment),
                artifact_version=self._artifact_version,
                artifact_sha256=self._artifact_sha256,
            ),
            user_id,
            generation,
            ownership_lost,
        )
    except ManagedRuntimeStateError:
        raise
    except Exception:
        return self._fail(user_id, generation, "bootstrap_failed", claim.runtime)

```

```

deadline = self._now() + self._readiness_timeout
while True:
    now = self._now()
    try:
        runner = get_managed_runner_for_user(user_id)
    except Exception:
        return self._fail(
            user_id,
            generation,
            "runner_registration_failed",
            claim.runtime,
        )
    if runner is None or runner.get("id") != claim.runtime.runner_id:
        return self._fail(
            user_id,
            generation,
            "runner_registration_failed",
            claim.runtime,
        )
    noise_key = runner.get("noise_public_key")
    if expected_noise_key is not None and noise_key != expected_noise_key:
        return self._fail(
            user_id,
            generation,
            "runner_identity_changed",
            claim.runtime,
        )
    identity_is_confirmed = isinstance(noise_key, str) and bool(
        runner.get("noise_key_confirmed")
    )
    if runner.get("status") == "online" and not identity_is_confirmed:
        return self._fail(
            user_id,
            generation,
            "runner_registration_failed",
            claim.runtime,
        )
    if runner.get("registered_at") is not None and identity_is_confirmed:
        if not self._artifact_attestation_matches(runner):
            return self._fail(
                user_id,
                generation,
                "bootstrap_failed",
                claim.runtime,
            )
    if mark_managed_runtime_ready(

```

```

        user_id,
        claim.runtime.generation,
        now,
    ):
        self._register_sprite_identity(
            sprite_name=claim.runtime.sprite_name,
            identity_kind="runtime",
            user_id=user_id,
            job_id=None,
            lifecycle_status="active",
            now=now,
        )
        status = get_managed_runtime_status(user_id)
        assert status is not None
        return status
    self._refresh_ownership(user_id, generation, ownership_lost)
    if now >= deadline:
        return self._fail(user_id, generation, "wake_timeout", claim.runtime)
    await self._await_owned(
        self._sleep(self._poll_interval_seconds),
        user_id,
        generation,
        ownership_lost,
    )

async def _maintain_provisioning_heartbeat(
    self,
    user_id: str,
    generation: int,
    ownership_lost: asyncio.Event,
) -> None:
    """Refresh ownership until completion or generation loss."""
    while True:
        await self._heartbeat_sleep(_PROVISIONING_HEARTBEAT_INTERVAL_SECONDS)
        try:
            refreshed = refresh_managed_runtime_provisioning(
                user_id,
                generation,
                self._now(),
            )
        except Exception:
            refreshed = False
        if not refreshed:
            ownership_lost.set()
        return

```

```

async def _await_owned(
    self,
    awaitable: Awaitable[_Result],
    user_id: str,
    generation: int,
    ownership_lost: asyncio.Event,
) -> _Result:
    """Cancel one operation when its provisioning ownership is lost."""
    operation = asyncio.ensure_future(awaitable)
    lost_waiter = asyncio.create_task(ownership_lost.wait())
    try:
        done, _ = await asyncio.wait(
            (operation, lost_waiter),
            return_when=asyncio.FIRST_COMPLETED,
        )
        if lost_waiter in done:
            operation.cancel()
            await asyncio.gather(operation, return_exceptions=True)
            raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)
        lost_waiter.cancel()
        await asyncio.gather(lost_waiter, return_exceptions=True)
        self._refresh_ownership(user_id, generation, ownership_lost)
        return operation.result()
    finally:
        if not operation.done():
            operation.cancel()
        if not lost_waiter.done():
            lost_waiter.cancel()
        await asyncio.gather(operation, lost_waiter, return_exceptions=True)

def _refresh_ownership(
    self,
    user_id: str,
    generation: int,
    ownership_lost: asyncio.Event,
) -> None:
    """Refresh current ownership or raise the fixed state error."""
    try:
        refreshed = refresh_managed_runtime_provisioning(
            user_id,
            generation,
            self._now(),
        )
    except Exception:
        refreshed = False
    if not refreshed:

```

```

        ownership_lost.set()
        raise ManagedRuntimeStateError(_STATE_ERROR_MESSAGE)

def _provisioning_task_finished(
    self,
    task: asyncio.Task[ManagedRuntimeStatus],
) -> None:
    """Forget finished work and consume its fixed completion result."""
    self._provisioning_tasks.pop(task, None)
    if not task.cancelled():
        task.exception()

def _artifact_attestation_matches(self, runner: dict[str, Any]) -> bool:
    """Return whether managed guest reports configured artifact digest."""
    capabilities = runner.get("capabilities")
    if not isinstance(capabilities, dict):
        return False
    artifact_sha256 = capabilities.get("artifact_sha256")
    return isinstance(artifact_sha256, str) and hmac.compare_digest(
        artifact_sha256,
        self._artifact_sha256,
    )

def _fail(
    self,
    user_id: str,
    generation: int,
    error_code: str,
    fallback: ManagedRuntimeStatus,
) -> ManagedRuntimeStatus:
    """Fail only the matching generation and return safe current state."""
    mark_managed_runtime_failed(user_id, generation, error_code, self._now())
    return get_managed_runtime_status(user_id) or fallback

def _now(self) -> datetime:
    """Return one validated injected UTC clock value."""
    now = self._clock()
    if not isinstance(now, datetime):
        raise TypeError("clock must return a datetime")
    if now.tzinfo is None:
        raise ValueError("clock must return a timezone-aware datetime")
    return now.astimezone(timezone.utc)

```


15.9 yinshi/services/managed_sprite_reconciliation.py

Reconciliation compares durable ownership with provider inventory and repairs only states that have one unambiguous owner.

```
"""Reconcile managed provider Sprites against durable control-plane ownership."""

from __future__ import annotations

import asyncio
import logging
from collections.abc import Awaitable, Callable
from dataclasses import dataclass
from datetime import datetime, timedelta, timezone
from typing import Protocol

from yinshi.db import get_control_db
from yinshi.services import managed_operational_failures
from yinshi.services.managed_runners import managed_sprite_name
from yinshi.services.managed_sprite_registry import (
    list_managed_sprite_identities,
    remove_managed_sprite_identity,
)
from yinshi.services.runners import revoke_managed_restore_runner_for_job
from yinshi.services.sprites import SpriteInventoryRecord, SpriteRecord

logger = logging.getLogger(__name__)

class ManagedSpriteInventoryProvider(Protocol):
    """Provider operations required for inventory reconciliation."""

    async def list_sprites(self, *, prefix: str) -> tuple[SpriteInventoryRecord, ...]: ...

    async def get_sprite(self, name: str) -> SpriteRecord | None: ...

    async def delete_sprite(self, name: str) -> None: ...

@dataclass(frozen=True, slots=True)
class ManagedSpriteReconciliationResult:
    """Sanitized result for one complete reconciliation pass."""

    examined: int
    retained: int
    deleted: tuple[str, ...]
```

```

deferred: int
eligible: tuple[str, ...] = ()

@dataclass(frozen=True, slots=True)
class _ManagedSpriteReferences:
    names: frozenset[str]
    restore_jobs_by_name: dict[str, tuple[str, str]]

def _parse_registry_created_at(value: str) -> datetime | None:
    """Parse one registry timestamp without treating malformed state as old."""
    try:
        created_at = datetime.fromisoformat(value.replace("Z", "+00:00"))
    except ValueError:
        return None
    if created_at.tzinfo is None:
        return None
    return created_at.astimezone(timezone.utc)

def _managed_sprite_references(
    *,
    restore_name_prefix: str,
    restore_name_key: str,
) -> _ManagedSpriteReferences:
    """Read all durable runtime and running-operation provider references."""
    names: set[str] = set()
    restore_jobs: dict[str, tuple[str, str]] = {}
    with get_control_db() as database:
        runtime_rows = database.execute(
            """SELECT sprite_external_id FROM managed_runtimes
               WHERE provider_name = ?""",
            ("fly_sprites",),
        ).fetchall()
        operation_rows = database.execute(
            """SELECT user_id, job_id, operation, source_sprite_id, candidate_sprite_id
               FROM managed_backup_operations WHERE status = ?""",
            ("running",),
        ).fetchall()
        restore_runner_rows = database.execute("""SELECT user_id, restore_job_id FROM user_r
               WHERE kind = 'managed_restore' AND restore_job_id IS NOT NULL
               AND revoked_at IS NULL""").fetchall()
    names.update(row["sprite_external_id"] for row in runtime_rows)
    for row in operation_rows:
        source_name = row["source_sprite_id"]

```

```

candidate_name = row["candidate_sprite_id"]
if source_name:
    names.add(source_name)
if candidate_name:
    names.add(candidate_name)
    if row["operation"] == "restore":
        restore_jobs[candidate_name] = (row["user_id"], row["job_id"])
if row["operation"] == "restore":
    deterministic_name = managed_sprite_name(
        f"{row['user_id']}:{row['job_id']}",
        prefix=restore_name_prefix,
        secret_key=restore_name_key,
    )
    names.add(deterministic_name)
    restore_jobs[deterministic_name] = (row["user_id"], row["job_id"])
for row in restore_runner_rows:
    deterministic_name = managed_sprite_name(
        f"{row['user_id']}:{row['restore_job_id']}",
        prefix=restore_name_prefix,
        secret_key=restore_name_key,
    )
    restore_jobs[deterministic_name] = (row["user_id"], row["restore_job_id"])
return _ManagedSpriteReferences(frozenset(names), restore_jobs)

```

```

class ManagedSpriteReconciler:

```

```

    """Delete old managed Sprites that have no durable owner."""

```

```

def __init__(
    self,
    *,
    provider: ManagedSpriteInventoryProvider,
    name_prefix: str,
    restore_name_prefix: str,
    restore_name_key: str,
    grace: timedelta,
    clock: Callable[[], datetime] | None = None,
    sleep: Callable[[float], Awaitable[None]] = asyncio.sleep,
) -> None:
    if not name_prefix or not restore_name_prefix or not restore_name_key:
        raise ValueError("Sprite reconciliation names and key must not be empty")
    if grace.total_seconds() <= 0:
        raise ValueError("Sprite reconciliation grace must be positive")
    self._provider = provider
    self._name_prefix = name_prefix
    self._restore_name_prefix = restore_name_prefix

```

```

self._restore_name_key = restore_name_key
self._grace = grace
self._clock = clock or (lambda: datetime.now(timezone.utc))
self._sleep = sleep
self._task: asyncio.Task[None] | None = None

async def reconcile_once(
    self,
    *,
    dry_run: bool = False,
) -> ManagedSpriteReconciliationResult:
    """Reconcile one fully validated provider inventory."""
    inventory_names: set[str] = set()
    for prefix in self._inventory_prefixes():
        for inventory_record in await self._provider.list_sprites(prefix=prefix):
            inventory_names.add(inventory_record.name)
    references = _managed_sprite_references(
        restore_name_prefix=self._restore_name_prefix,
        restore_name_key=self._restore_name_key,
    )
    registered_by_name = {
        identity.sprite_name: identity
        for identity in list_managed_sprite_identities()
        if self._owns_name(identity.sprite_name)
    }
    registered_names = set(registered_by_name)
    absent_registered_names = registered_names - inventory_names
    candidate_names = (inventory_names & registered_names) | absent_registered_names
    records_by_name = {
        name: await self._provider.get_sprite(name) for name in sorted(candidate_names)
    }
    now = self._clock().astimezone(timezone.utc)
    deleted: list[str] = []
    eligible: list[str] = []
    retained = 0
    deferred = 0
    for name in sorted(candidate_names):
        if name in references.names:
            retained += 1
            continue
        record = records_by_name[name]
        provider_absent = name in absent_registered_names and record is None
        if name in absent_registered_names and record is not None:
            deferred += 1
            continue
        if record is None and not provider_absent:

```

```

        deferred += 1
        continue
    created_at = (
        _parse_registry_created_at(registered_by_name[name].created_at)
        if provider_absent
        else record.created_at if record is not None else None
    )
    if created_at is None or now - created_at < self._grace:
        deferred += 1
        continue
    current = _managed_sprite_references(
        restore_name_prefix=self._restore_name_prefix,
        restore_name_key=self._restore_name_key,
    )
    if name in current.names:
        retained += 1
        continue
    if dry_run:
        eligible.append(name)
        continue
    restore_job = current.restore_jobs_by_name.get(name)
    if restore_job is not None:
        revoke_managed_restore_runner_for_job(*restore_job)
    if not provider_absent:
        await self._provider.delete_sprite(name)
    remove_managed_sprite_identity(name)
    deleted.append(name)
    return ManagedSpriteReconciliationResult(
        examined=len(inventory_names | absent_registered_names),
        retained=retained,
        deleted=tuple(deleted),
        deferred=deferred,
        eligible=tuple(eligible),
    )

async def reconcile_classified(
    self,
    *,
    raise_on_failure: bool,
) -> ManagedSpriteReconciliationResult | None:
    """Run one pass with the same stable failure classification for every caller."""
    alert_class = (
        managed_operational_failures.ManagedPersistentAlertClass.SPRITE_RECONCILIATION_F
    )
    try:
        result = await self.reconcile_once()

```

```

except asyncio.CancelledError:
    raise
except Exception:
    managed_operational_failures.record_managed_operational_failure(alert_class)
    logger.exception("managed_sprite_reconciliation_failed")
    if raise_on_failure:
        raise
    return None
managed_operational_failures.clear_managed_operational_failure(alert_class)
return result

async def run(self, *, interval_seconds: float) -> None:
    """Run recurring passes until cancelled."""
    if interval_seconds <= 0:
        raise ValueError("interval_seconds must be positive")
    while True:
        await self._sleep(interval_seconds)
        await self.reconcile_classified(raise_on_failure=False)

def _inventory_prefixes(self) -> tuple[str, ...]:
    """Return distinct provider query prefixes including delimiter."""
    prefixes = (f"{self._name_prefix}-", f"{self._restore_name_prefix}-")
    return tuple(dict.fromkeys(prefixes))

def _owns_name(self, name: str) -> bool:
    """Return whether a name belongs to an exact configured namespace."""
    return any(name.startswith(prefix) for prefix in self._inventory_prefixes())

```

15.10 yinshi/services/managed_sprite_registry.py

The Sprite registry preserves provider ownership after user deletion so later cleanup never relies on name matching.

```

"""Persist provider identities owned by this Yinshi deployment."""

```

```

from __future__ import annotations

from dataclasses import dataclass
from datetime import UTC, datetime
from typing import Literal, cast

from yinshi.db import get_control_db

```

```

ManagedSpriteIdentityKind = Literal["runtime", "restore_candidate"]
ManagedSpriteIdentityStatus = Literal["creating", "active", "retired", "deleting"]

```

```

@dataclass(frozen=True, slots=True)
class ManagedSpriteIdentity:
    """One deployment-owned provider identity."""

    sprite_name: str
    provider_name: str
    identity_kind: ManagedSpriteIdentityKind
    user_id: str
    job_id: str | None
    lifecycle_status: ManagedSpriteIdentityStatus
    created_at: str
    updated_at: str

def _timestamp(now: datetime) -> str:
    if not isinstance(now, datetime) or now.tzinfo is None:
        raise ValueError("now must be timezone-aware")
    return now.astimezone(UTC).isoformat()

def register_managed_sprite_identity(
    *,
    sprite_name: str,
    identity_kind: ManagedSpriteIdentityKind,
    user_id: str,
    job_id: str | None,
    lifecycle_status: ManagedSpriteIdentityStatus,
    now: datetime,
) -> None:
    """Register one intended provider identity before external creation."""
    if not sprite_name or not user_id:
        raise ValueError("Sprite identity and owner must not be empty")
    if identity_kind == "runtime" and job_id is not None:
        raise ValueError("Runtime Sprite identity must not have a job")
    if identity_kind == "restore_candidate" and not job_id:
        raise ValueError("Restore Sprite identity must have a job")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        result = database.execute(
            """INSERT INTO managed_sprite_identities
            (sprite_name, provider_name, identity_kind, user_id, job_id,
            lifecycle_status, created_at, updated_at)
            VALUES (?, 'fly_sprites', ?, ?, ?, ?, ?, ?)
            ON CONFLICT(sprite_name) DO UPDATE SET
                lifecycle_status = excluded.lifecycle_status,

```

```

        updated_at = excluded.updated_at
    WHERE managed_sprite_identities.provider_name = 'fly_sprites'
        AND managed_sprite_identities.identity_kind = excluded.identity_kind
        AND managed_sprite_identities.user_id = excluded.user_id
        AND managed_sprite_identities.job_id IS excluded.job_id""",
    (
        sprite_name,
        identity_kind,
        user_id,
        job_id,
        lifecycle_status,
        timestamp,
        timestamp,
    ),
)
if result.rowcount != 1:
    database.rollback()
    raise ValueError("Sprite identity conflicts with another owner")
database.commit()

def list_managed_sprite_identities() -> tuple[ManagedSpriteIdentity, ...]:
    """Return all provider identities owned by this deployment."""
    with get_control_db() as database:
        rows = database.execute(
            """SELECT sprite_name, provider_name, identity_kind, user_id, job_id,
                lifecycle_status, created_at, updated_at
            FROM managed_sprite_identities ORDER BY sprite_name"""
        ).fetchall()
    return tuple(
        ManagedSpriteIdentity(
            sprite_name=row["sprite_name"],
            provider_name=row["provider_name"],
            identity_kind=cast(ManagedSpriteIdentityKind, row["identity_kind"]),
            user_id=row["user_id"],
            job_id=row["job_id"],
            lifecycle_status=cast(ManagedSpriteIdentityStatus, row["lifecycle_status"]),
            created_at=row["created_at"],
            updated_at=row["updated_at"],
        )
        for row in rows
    )

def remove_managed_sprite_identity(sprite_name: str) -> bool:
    """Forget one identity only after confirmed provider absence."""

```



```

if not sprite_name:
    raise ValueError("sprite_name must not be empty")
with get_control_db() as database:
    result = database.execute(
        "DELETE FROM managed_sprite_identities WHERE sprite_name = ?",
        (sprite_name,),
    )
    database.commit()
return result.rowcount == 1

```

15.11 yinshi/services/sprite_task_lease.py

Task leases prevent concurrent maintenance operations from changing one Sprite at the same time.

```

"""Reference-counted local task lease for managed Fly Sprites."""

from __future__ import annotations

import asyncio
import logging
from collections.abc import Awaitable, Callable

import httpx

logger = logging.getLogger(__name__)

_TASK_PATH = "/v1/tasks/yinshi-active"
_TASK_PAYLOAD = {"expire": "5m"}
_REFRESH_INTERVAL_SECONDS = 60.0
_DEFAULT_REQUEST_TIMEOUT_SECONDS = 5.0
_SPRITE_API_SOCKET = "/.sprite/api.sock"

Sleep = Callable[[float], Awaitable[None]]

class SpriteTaskLeaseError(RuntimeError):
    """Raised when a managed Sprite task cannot be acquired."""

class SpriteTaskLease:
    """Keep one bounded Sprite task alive while relay transfers exist."""

    def __init__(
        self,
        *,

```

```

transport: httpx.AsyncBaseTransport | None = None,
sleep: Sleep = asyncio.sleep,
request_timeout_seconds: float = _DEFAULT_REQUEST_TIMEOUT_SECONDS,
) -> None:
    if not callable(sleep):
        raise TypeError("sleep must be callable")
    if (
        isinstance(request_timeout_seconds, bool)
        or not isinstance(request_timeout_seconds, (int, float))
        or request_timeout_seconds <= 0
    ):
        raise ValueError("request_timeout_seconds must be positive")
    if transport is None:
        transport = httpx.AsyncHTTPTransport(uds=_SPRITE_API_SOCKET)
    self._client = httpx.AsyncClient(
        base_url="http://sprite",
        transport=transport,
        follow_redirects=False,
    )
    self._sleep = sleep
    self._request_timeout_seconds = float(request_timeout_seconds)
    self._lock = asyncio.Lock()
    self._references = 0
    self._refresh_task: asyncio.Task[None] | None = None
    self._closed = False

    async def acquire(self) -> None:
        """Acquire one reference and create the local task when needed."""
        async with self._lock:
            if self._closed:
                raise RuntimeError("Sprite task lease is closed")
            if self._references == 0:
                await self._put_task()
                self._refresh_task = asyncio.create_task(
                    self._refresh_loop(),
                    name="sprite-task-lease-refresh",
                )
            self._references += 1

    async def release(self) -> None:
        """Release one reference and delete the task after the final reference."""
        async with self._lock:
            if self._references == 0:
                raise RuntimeError("Sprite task lease has no references")
            self._references -= 1
            if self._references != 0:

```

```

        return
    await self._stop_refresh()
    await self._delete_task()

async def aclose(self) -> None:
    """Delete any held task and close the local HTTP client."""
    async with self._lock:
        if self._closed:
            return
        self._closed = True
        had_references = self._references > 0
        self._references = 0
        await self._stop_refresh()
        if had_references:
            await self._delete_task()
    await self._client.aclose()

async def _refresh_loop(self) -> None:
    """Refresh the task expiry once per minute while references remain."""
    while True:
        await self._sleep(_REFRESH_INTERVAL_SECONDS)
        async with self._lock:
            if self._references == 0 or self._closed:
                return
            try:
                await self._put_task()
            except SpriteTaskLeaseError:
                logger.warning("Could not refresh managed Sprite task lease")

async def _stop_refresh(self) -> None:
    """Stop and join the current refresh task."""
    refresh_task = self._refresh_task
    self._refresh_task = None
    if refresh_task is None or refresh_task is asyncio.current_task():
        return
    refresh_task.cancel()
    try:
        await refresh_task
    except asyncio.CancelledError:
        pass

async def _put_task(self) -> None:
    """Create or refresh the fixed local task without exposing HTTP errors."""
    failed = False
    try:
        response = await self._client.put(

```

```

        _TASK_PATH,
        json=_TASK_PAYLOAD,
        timeout=self._request_timeout_seconds,
    )
    failed = not 200 <= response.status_code < 300
except (httpx.RequestError, TimeoutError):
    failed = True
if failed:
    raise SpriteTaskLeaseError("Could not acquire managed Sprite task lease")

async def _delete_task(self) -> None:
    """Best-effort delete the task while retaining expiry as crash fallback."""
    failed = False
    try:
        response = await self._client.delete(
            _TASK_PATH,
            timeout=self._request_timeout_seconds,
        )
        failed = not 200 <= response.status_code < 300 and response.status_code != 404
    except (httpx.RequestError, TimeoutError):
        failed = True
    if failed:
        logger.warning("Could not delete managed Sprite task lease; expiry remains active")

```

15.12 yinshi/services/sprites.py

The provider client validates identifiers, bounds responses, sanitizes failures, and implements exact Sprite and service lifecycle calls.

"""Validated asynchronous client for the Fly Sprites HTTP API."""

```

from __future__ import annotations

import asyncio
import hashlib
import ipaddress
import json
import os
import re
from collections.abc import AsyncIterator, Iterator, Mapping
from contextlib import contextmanager
from dataclasses import dataclass
from datetime import datetime, timezone
from math import isfinite
from pathlib import Path
from typing import Literal, cast

```

```

import httpx

class SpritesProtocolError(RuntimeError):
    """Raised when Fly returns an invalid Sprite response."""

class SpritesProviderError(RuntimeError):
    """Raised when Fly rejects or cannot complete a request."""

_SPRITE_NAME_PATTERN = re.compile(r"[a-z0-9](?:[a-z0-9]{0,61}[a-z0-9])?\Z")
_SERVICE_NAME_PATTERN = _SPRITE_NAME_PATTERN
_CHECKPOINT_ID_PATTERN = re.compile(r"v[0-9]+\Z")
_FILE_MODE_PATTERN = re.compile(r"0[0-7]{3}\Z")
_DNS_LABEL_PATTERN = re.compile(r"[a-z0-9](?:[a-z0-9]{0,61}[a-z0-9])?\Z")
_POSIX_ENVIRONMENT_NAME_PATTERN = re.compile(r"[A-Za-z_][A-Za-z0-9_]*\Z")
_STANDARD_OPERATION_TIMEOUT_SECONDS = 30.0
_LONG_OPERATION_TIMEOUT_SECONDS = 120.0
_SERVICE_STREAM_TRANSPORT_MARGIN_SECONDS = 30.0
_MAX_STREAM_RESPONSE_BYTES = 1024 * 1024
_MAX_STREAM_EVENTS = 1000
_MAX_PATH_LENGTH = 4096
_MAX_FILE_CONTENT_BYTES = 10 * 1024 * 1024
_MAX_SPRITE_ID_LENGTH = 256
_MAX_SPRITE_STATUS_LENGTH = 64
_MAX_SPRITE_INVENTORY_PAGES = 100
_MAX_SPRITE_INVENTORY_ITEMS = 5000
_MAX_CONTINUATION_TOKEN_LENGTH = 4096
_MAX_SERVICE_COMMAND_LENGTH = 4096
_MAX_SERVICE_LIST_ITEMS = 256
_MAX_SERVICE_VALUE_LENGTH = 4096
_MAX_ALLOWED_DOMAINS = 256
_MAX_DNS_NAME_LENGTH = 253
_MAX_CHECKPOINT_COMMENT_LENGTH = 4096
_MAX_MONITOR_DURATION_SECONDS = 86400.0
_MAX_STATE_STARTED_AT_LENGTH = 128
_MAX_STATE_ERROR_LENGTH = 4096
_FILE_TRANSFER_CHUNK_BYTES = 4 * 1024 * 1024
_FILE_TRANSFER_BYTES_MAX = 200 * 1024 * 1024 * 1024
_CONTENT_RANGE_PATTERN = re.compile(r"bytes ([0-9]+)-([0-9]+)/([0-9]+)\Z")

async def _read_bounded_response(
    response: httpx.Response,

```

```

description: str,
*,
max_bytes: int = _MAX_STREAM_RESPONSE_BYTES,
maximum: int = _MAX_STREAM_RESPONSE_BYTES,
) -> bytes:
    """Read one provider response without exceeding its byte limit."""
    if type(maximum) is not int or not 1 <= maximum <= _MAX_FILE_CONTENT_BYTES:
        raise ValueError("maximum is outside the provider response limit")
    if type(max_bytes) is not int or not 1 <= max_bytes <= maximum:
        raise ValueError("max_bytes is outside the provider response limit")
    content_length = response.headers.get("Content-Length")
    if content_length is not None:
        try:
            declared_length = int(content_length)
        except ValueError:
            declared_length = 0
        if declared_length > max_bytes:
            raise SpritesProtocolError(f"{description} exceeds size limit")
    body = bytearray()
    async for chunk in response.aiter_bytes():
        if len(body) + len(chunk) > max_bytes:
            raise SpritesProtocolError(f"{description} exceeds size limit")
        body.extend(chunk)
    return bytes(body)

def _validate_sprite_name(name: str) -> str:
    """Require one lowercase provider-safe DNS label."""
    if not isinstance(name, str) or _SPRITE_NAME_PATTERN.fullmatch(name) is None:
        raise ValueError("Sprite name must be a lowercase DNS label of 1 to 63 characters")
    return name

def _validate_service_name(name: str) -> str:
    """Require one lowercase provider-safe service label."""
    if not isinstance(name, str) or _SERVICE_NAME_PATTERN.fullmatch(name) is None:
        raise ValueError("Service name must be a lowercase DNS label of 1 to 63 characters")
    return name

def _validate_monitor_duration(monitor_duration: float | None) -> float | None:
    """Require a finite positive provider monitoring duration within one day."""
    if monitor_duration is None:
        return None
    if isinstance(monitor_duration, bool) or not isinstance(monitor_duration, (int, float)):
        raise ValueError("Monitor duration must be between 0 and 86400 seconds")

```

```

try:
    validated_duration = float(monitor_duration)
except OverflowError:
    raise ValueError("Monitor duration must be between 0 and 86400 seconds") from None
if (
    not isfinite(validated_duration)
    or not 0 < validated_duration <= _MAX_MONITOR_DURATION_SECONDS
):
    raise ValueError("Monitor duration must be between 0 and 86400 seconds")
return validated_duration

def _service_stream_timeout(monitor_duration: float | None) -> float:
    """Allow provider monitoring plus a bounded transport completion margin."""
    if monitor_duration is None:
        return _LONG_OPERATION_TIMEOUT_SECONDS
    return max(
        _LONG_OPERATION_TIMEOUT_SECONDS,
        monitor_duration + _SERVICE_STREAM_TRANSPORT_MARGIN_SECONDS,
    )

def _validate_service_command(command: str) -> str:
    """Require one bounded nonblank command without a null byte."""
    if (
        not isinstance(command, str)
        or not command.strip()
        or len(command) > _MAX_SERVICE_COMMAND_LENGTH
        or "\x00" in command
    ):
        raise ValueError("Command must be bounded nonblank text without null bytes")
    return command

def _validate_service_args(args: tuple[str, ...]) -> tuple[str, ...]:
    """Require a bounded tuple of bounded string arguments."""
    if (
        not isinstance(args, tuple)
        or len(args) > _MAX_SERVICE_LIST_ITEMS
        or not all(
            isinstance(value, str)
            and len(value) <= _MAX_SERVICE_VALUE_LENGTH
            and "\x00" not in value
            for value in args
        )
    ):

```

```

        raise ValueError("Service args must be a bounded tuple of strings")
    return args

def _validate_service_environment(environment: Mapping[str, str]) -> dict[str, str]:
    """Require bounded POSIX environment names and bounded string values."""
    if not isinstance(environment, Mapping) or len(environment) > _MAX_SERVICE_LIST_ITEMS:
        raise ValueError("Service environment must be a bounded mapping")
    for key, value in environment.items():
        if (
            not isinstance(key, str)
            or len(key) > _MAX_SERVICE_VALUE_LENGTH
            or _POSIX_ENVIRONMENT_NAME_PATTERN.fullmatch(key) is None
            or not isinstance(value, str)
            or len(value) > _MAX_SERVICE_VALUE_LENGTH
            or "\x00" in value
        ):
            raise ValueError("Service environment keys and values are invalid")
    return dict(environment)

def _validate_service_dependencies(needs: tuple[str, ...]) -> tuple[str, ...]:
    """Require unique bounded service labels in one tuple."""
    if not isinstance(needs, tuple) or len(needs) > _MAX_SERVICE_LIST_ITEMS:
        raise ValueError("Service dependencies must be a bounded tuple")
    validated = tuple(_validate_service_name(value) for value in needs)
    if len(set(validated)) != len(validated):
        raise ValueError("Service dependencies must not contain duplicates")
    return validated

def _validate_http_port(http_port: int | None) -> int | None:
    """Require an exact TCP port integer when present."""
    if http_port is not None and (type(http_port) is not int or not 1 <= http_port <= 65535):
        raise ValueError("HTTP port must be an integer between 1 and 65535")
    return http_port

def _validate_checkpoint_comment(comment: str) -> str:
    """Require bounded printable nonblank checkpoint text."""
    if (
        not isinstance(comment, str)
        or not comment.strip()
        or len(comment) > _MAX_CHECKPOINT_COMMENT_LENGTH
        or not all(character.isprintable() for character in comment)
    ):

```



```

        raise ValueError("Checkpoint comment must be bounded safe nonblank text")
    return comment

def _validate_checkpoint_id(checkpoint_id: str) -> str:
    """Require the provider checkpoint identifier format."""
    if (
        not isinstance(checkpoint_id, str)
        or _CHECKPOINT_ID_PATTERN.fullmatch(checkpoint_id) is None
    ):
        raise ValueError("Checkpoint ID must use the provider v<number> format")
    return checkpoint_id

def _validate_file_path(path: str, *, field: str, allow_root: bool) -> str:
    """Require a bounded absolute POSIX path without traversal."""
    if (
        not isinstance(path, str)
        or not path.startswith("/")
        or len(path) > _MAX_PATH_LENGTH
        or "\x00" in path
        or any(part in {".", ".."} for part in path.split("/"))
        or (not allow_root and path == "/")
    ):
        raise ValueError(f"{field} must be a safe absolute path")
    return path

def _validate_allowed_domains(domains: tuple[str, ...]) -> tuple[str, ...]:
    """Require a bounded tuple of unique public DNS allow rules."""
    if not isinstance(domains, tuple) or len(domains) > _MAX_ALLOWED_DOMAINS:
        raise ValueError("Allowed domains must be a bounded tuple")
    if not all(isinstance(entry, str) for entry in domains):
        raise ValueError("Allowed domains must contain public DNS names")
    if len(set(domains)) != len(domains):
        raise ValueError("Allowed domains must not contain duplicates")
    for entry in domains:
        if entry == "*":
            raise ValueError("Allowed domains must contain public DNS names")
        domain = entry[2:] if entry.startswith("*.") else entry
        labels = domain.split(".")
        if (
            "*" in domain
            or len(domain) > _MAX_DNS_NAME_LENGTH
            or len(labels) < 2
            or any(_DNS_LABEL_PATTERN.fullmatch(label) is None for label in labels)
        )

```

```

        or labels[-1].isdigit()
    ):
        raise ValueError("Allowed domains must contain public DNS names")
    try:
        ipaddress.ip_address(domain)
    except ValueError:
        continue
    raise ValueError("Allowed domains must not contain IP addresses")
return domains

@contextmanager
def _translate_transport_errors(operation: str) -> Iterator[None]:
    """Replace request-bearing transport errors with a stable provider error."""
    try:
        yield
    except (httpx.RequestError, TimeoutError):
        raise SpritesProviderError(f"Fly could not {operation}") from None

@dataclass(frozen=True, slots=True)
class SpriteFileTransfer:
    """Validated metadata for one completed Sprite file transfer."""

    size_bytes: int
    sha256: str

@dataclass(frozen=True, slots=True)
class SpriteInventoryRecord:
    """Minimal provider inventory identity from the list endpoint."""

    name: str

@dataclass(frozen=True, slots=True)
class SpriteRecord:
    """Provider identity and lifecycle state for one Sprite."""

    id: str
    name: str
    status: str
    created_at: datetime | None = None

ServiceStatus = Literal["stopped", "starting", "running", "stopping", "failed"]

```

```
_SERVICE_STATUSES = {"stopped", "starting", "running", "stopping", "failed"}
```

```
@dataclass(frozen=True, slots=True)
```

```
class ServiceState:
```

```
    """Current provider runtime state for one Sprite service."""
```

```
    name: str
    status: ServiceStatus
    pid: int | None
    started_at: str | None
    error: str | None
```

```
@dataclass(frozen=True, slots=True)
```

```
class ServiceRecord:
```

```
    """Provider definition and runtime state for one Sprite service."""
```

```
    name: str
    command: str
    args: tuple[str, ...]
    needs: tuple[str, ...]
    http_port: int | None
    state: ServiceState | None
```

```
def _raise_for_stream_error(response: httpx.Response, operation: str) -> None:
```

```
    """Translate an NDJSON error event without copying provider details."""
```

```
    if len(response.content) > _MAX_STREAM_RESPONSE_BYTES:
```

```
        raise SpritesProtocolError(f"Sprite {operation} response exceeds size limit")
```

```
    lines = [line for line in response.text.splitlines() if line.strip()]
```

```
    if len(lines) > _MAX_STREAM_EVENTS:
```

```
        raise SpritesProtocolError(f"Sprite {operation} response exceeds event limit")
```

```
    try:
```

```
        events = [json.loads(line) for line in lines]
```

```
    except json.JSONDecodeError:
```

```
        raise SpritesProtocolError(f"Sprite {operation} response is not valid NDJSON") from
```

```
    if any(isinstance(event, dict) and event.get("type") == "error" for event in events):
```

```
        raise SpritesProtocolError(f"Sprite {operation} failed")
```

```
    if any(
```

```
        isinstance(event, dict)
```

```
        and event.get("type") == "exit"
```

```
        and (type(event.get("exit_code")) is not int or event.get("exit_code") != 0)
```

```
        for event in events
```

```
    ):
```

```
        raise SpritesProtocolError(f"Sprite {operation} failed")
```

```

        if not events or not isinstance(events[-1], dict) or events[-1].get("type") != "complete":
            raise SpritesProtocolError(f"Sprite {operation} response did not complete")

def _parse_sprite_response(
    response: httpx.Response,
    expected_name: str,
) -> SpriteRecord:
    """Decode and validate one provider Sprite response."""
    try:
        payload = response.json()
    except ValueError:
        raise SpritesProtocolError("Sprite response is not valid JSON") from None
    return _parse_sprite_record(payload, expected_name)

def _parse_provider_timestamp(value: object) -> datetime | None:
    """Parse one optional provider timestamp into an aware UTC value."""
    if value is None:
        return None
    if not isinstance(value, str) or not value or len(value) > 128:
        raise SpritesProtocolError("Sprite response creation timestamp is invalid")
    try:
        parsed = datetime.fromisoformat(value.replace("Z", "+00:00"))
    except ValueError:
        raise SpritesProtocolError("Sprite response creation timestamp is invalid") from None
    if parsed.tzinfo is None:
        raise SpritesProtocolError("Sprite response creation timestamp is invalid")
    return parsed.astimezone(timezone.utc)

def _parse_sprite_record(payload: object, expected_name: str) -> SpriteRecord:
    """Validate one provider Sprite record."""
    if not isinstance(payload, dict):
        raise SpritesProtocolError("Sprite response record is invalid")
    values = (payload.get("id"), payload.get("name"), payload.get("status"))
    if any(not isinstance(value, str) or not value for value in values):
        raise SpritesProtocolError("Sprite response record is invalid")
    if (
        len(payload["id"]) > _MAX_SPRITE_ID_LENGTH
        or len(payload["status"]) > _MAX_SPRITE_STATUS_LENGTH
    ):
        raise SpritesProtocolError("Sprite response record is invalid")
    if payload["name"] != expected_name:
        raise SpritesProtocolError("Sprite response name does not match the request")
    return SpriteRecord(

```

```

        id=payload["id"],
        name=payload["name"],
        status=payload["status"],
        created_at=_parse_provider_timestamp(payload.get("created_at")),
    )

def _parse_service_response(response: httpx.Response, expected_name: str) -> ServiceRecord:
    """Decode and validate one provider service response."""
    try:
        payload = response.json()
    except ValueError:
        raise SpritesProtocolError("Sprite service response is not valid JSON") from None
    if not isinstance(payload, dict):
        raise SpritesProtocolError("Sprite service response is invalid")
    name = payload.get("name")
    command = payload.get("cmd")
    args = payload.get("args")
    needs = payload.get("needs")
    http_port = payload.get("http_port")
    if (
        name != expected_name
        or not isinstance(command, str)
        or not command
        or len(command) > _MAX_SERVICE_COMMAND_LENGTH
    ):
        raise SpritesProtocolError("Sprite service response is invalid")
    if (
        not isinstance(args, list)
        or len(args) > _MAX_SERVICE_LIST_ITEMS
        or not all(
            isinstance(value, str) and len(value) <= _MAX_SERVICE_VALUE_LENGTH for value in
        )
    ):
        raise SpritesProtocolError("Sprite service response is invalid")
    if needs is None:
        needs = []
    if (
        not isinstance(needs, list)
        or len(needs) > _MAX_SERVICE_LIST_ITEMS
        or not all(
            isinstance(value, str) and len(value) <= _MAX_SERVICE_VALUE_LENGTH for value in
        )
    ):
        raise SpritesProtocolError("Sprite service response is invalid")
    if http_port is not None and (

```

```

        not isinstance(http_port, int) or isinstance(http_port, bool) or not 1 <= http_port
    ):
        raise SpritesProtocolError("Sprite service response is invalid")
    return ServiceRecord(
        name=name,
        command=command,
        args=tuple(args),
        needs=tuple(needs),
        http_port=http_port,
        state=_parse_service_state(payload.get("state"), expected_name),
    )

def _parse_service_state(payload: object, expected_name: str) -> ServiceState | None:
    """Validate optional runtime state from a service response."""
    if payload is None:
        return None
    if not isinstance(payload, dict):
        raise SpritesProtocolError("Sprite service state is invalid")
    name = payload.get("name")
    status = payload.get("status")
    pid = payload.get("pid")
    started_at = payload.get("started_at")
    error = payload.get("error")
    if name != expected_name or status not in _SERVICE_STATUSES:
        raise SpritesProtocolError("Sprite service state is invalid")
    if pid is not None and (not isinstance(pid, int) or isinstance(pid, bool)):
        raise SpritesProtocolError("Sprite service state is invalid")
    if started_at is not None and (
        not isinstance(started_at, str) or len(started_at) > _MAX_STATE_STARTED_AT_LENGTH
    ):
        raise SpritesProtocolError("Sprite service state is invalid")
    if error is not None and (not isinstance(error, str) or len(error) > _MAX_STATE_ERROR_LENGTH):
        raise SpritesProtocolError("Sprite service state is invalid")
    return ServiceState(
        name=name,
        status=cast(ServiceStatus, status),
        pid=pid,
        started_at=started_at,
        error=error,
    )

class SpritesClient:
    """Call the Fly Sprites API without exposing provider details to callers."""

```

```

def __init__(self, *, api_token: str, http_client: httpx.AsyncClient) -> None:
    if not isinstance(api_token, str) or not api_token.strip():
        raise ValueError("api_token must be a non-empty string")
    self._api_token = api_token
    self._http_client = http_client

async def list_sprites(self, *, prefix: str) -> tuple[SpriteInventoryRecord, ...]:
    """Return complete provider inventory for one exact managed prefix."""
    if not isinstance(prefix, str) or not prefix or len(prefix) > 63:
        raise ValueError("prefix must be bounded non-empty text")
    records: list[SpriteInventoryRecord] = []
    names: set[str] = set()
    tokens: set[str] = set()
    continuation_token: str | None = None
    for _ in range(_MAX_SPRITE_INVENTORY_PAGES):
        params = {"prefix": prefix, "max_results": "50"}
        if continuation_token is not None:
            params["continuation_token"] = continuation_token
        with _translate_transport_errors("list Sprites"):
            async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
                async with self._http_client.stream(
                    "GET",
                    "/v1/sprites",
                    headers={"Authorization": f"Bearer {self._api_token}"},
                    params=params,
                    timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
                ) as response:
                    body = await _read_bounded_response(response, "Sprite inventory response")
                    status_code = response.status_code
                if not 200 <= status_code < 300:
                    raise SpritesProviderError(
                        f"Fly could not list Sprites (status {status_code})"
                    ) from None
                try:
                    payload = json.loads(body)
                except (json.JSONDecodeError, UnicodeDecodeError):
                    raise SpritesProtocolError("Sprite inventory response is not valid JSON") from None
                if not isinstance(payload, dict):
                    raise SpritesProtocolError("Sprite inventory response is invalid")
                entries = payload.get("sprites")
                has_more = payload.get("has_more")
                has_next_token_field = "next_continuation_token" in payload
                next_token = payload.get("next_continuation_token")
                if not isinstance(entries, list) or len(entries) > 50 or type(has_more) is not bool:
                    raise SpritesProtocolError("Sprite inventory response is invalid")
                page_names: list[str] = []

```

```

for entry in entries:
    if not isinstance(entry, dict):
        raise SpritesProtocolError("Sprite inventory record is invalid")
    name = entry.get("name")
    if not isinstance(name, str) or _SPRITE_NAME_PATTERN.fullmatch(name) is None:
        raise SpritesProtocolError("Sprite inventory record is invalid")
    if not name.startswith(prefix) or name in names:
        raise SpritesProtocolError("Sprite inventory record is invalid")
    names.add(name)
    page_names.append(name)
if len(names) > _MAX_SPRITE_INVENTORY_ITEMS:
    raise SpritesProtocolError("Sprite inventory exceeds item limit")
records.extend(SpriteInventoryRecord(name=name) for name in page_names)
if not has_more or (has_next_token_field and next_token is None):
    if not has_more and next_token is not None:
        raise SpritesProtocolError("Sprite inventory continuation is invalid")
    return tuple(records)
if (
    not isinstance(next_token, str)
    or not next_token
    or len(next_token) > _MAX_CONTINUATION_TOKEN_LENGTH
    or next_token in tokens
):
    raise SpritesProtocolError("Sprite inventory continuation is invalid")
tokens.add(next_token)
continuation_token = next_token
raise SpritesProtocolError("Sprite inventory exceeds page limit")

async def create_sprite(self, name: str) -> SpriteRecord:
    """Create one private Sprite and return its validated provider record."""
    name = _validate_sprite_name(name)
    with _translate_transport_errors("create Sprite"):
        async with asyncio.timeout(_LONG_OPERATION_TIMEOUT_SECONDS):
            async with self._http_client.stream(
                "POST",
                "/v1/sprites",
                headers={"Authorization": f"Bearer {self._api_token}"},
                json={
                    "name": name,
                    "url_settings": {"auth": "sprite"},
                    "wait_for_capacity": True,
                },
                timeout=_LONG_OPERATION_TIMEOUT_SECONDS,
            ) as response:
                body = await _read_bounded_response(response, "Sprite response")
                status_code = response.status_code

```



```

        if not 200 <= status_code < 300:
            raise SpritesProviderError(
                f"Fly could not create Sprite (status {status_code})"
            ) from None
        return _parse_sprite_response(httpx.Response(status_code, content=body), name)

    async def get_sprite(self, name: str) -> SpriteRecord | None:
        """Return one Sprite or None when Fly reports it missing."""
        name = _validate_sprite_name(name)
        with _translate_transport_errors("get Sprite"):
            async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
                async with self._http_client.stream(
                    "GET",
                    f"/v1/sprites/{name}",
                    headers={"Authorization": f"Bearer {self._api_token}"},
                    timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
                ) as response:
                    body = await _read_bounded_response(response, "Sprite response")
                    status_code = response.status_code
        if status_code == 404:
            return None
        if not 200 <= status_code < 300:
            raise SpritesProviderError(f"Fly could not get Sprite (status {status_code})")
        return _parse_sprite_response(httpx.Response(status_code, content=body), name)

    async def set_network_policy(
        self,
        name: str,
        *,
        allowed_domains: tuple[str, ...],
    ) -> None:
        """Restrict Sprite egress to explicit domains."""
        name = _validate_sprite_name(name)
        allowed_domains = _validate_allowed_domains(allowed_domains)
        rules = [{"action": "allow", "domain": domain} for domain in allowed_domains]
        rules.append({"action": "deny", "domain": "*"})
        with _translate_transport_errors("set network policy"):
            async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
                async with self._http_client.stream(
                    "POST",
                    f"/v1/sprites/{name}/policy/network",
                    headers={"Authorization": f"Bearer {self._api_token}"},
                    json={"rules": rules},
                    timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
                ) as response:
                    body = await _read_bounded_response(

```

```

        response,
        "Sprite network policy response",
    )
    status_code = response.status_code
    if not 200 <= status_code < 300:
        raise SpritesProviderError(
            f"Fly could not set network policy (status {status_code})"
        ) from None
    if not body:
        return
    try:
        payload = httpx.Response(status_code, content=body).json()
    except ValueError:
        raise SpritesProtocolError("Sprite network policy response is not valid JSON")
    if not isinstance(payload, dict) or payload.get("rules") != rules:
        raise SpritesProtocolError("Sprite network policy response is invalid")

async def write_file(
    self,
    name: str,
    *,
    path: str,
    content: bytes,
    mode: str,
    mkdir: bool,
) -> None:
    """Write raw bytes to one Sprite filesystem path."""
    name = _validate_sprite_name(name)
    path = _validate_file_path(path, field="File path", allow_root=False)
    if not isinstance(content, bytes) or len(content) > _MAX_FILE_CONTENT_BYTES:
        raise ValueError("File content must be bytes within the 10 MiB limit")
    if not isinstance(mode, str) or _FILE_MODE_PATTERN.fullmatch(mode) is None:
        raise ValueError("File mode must be a four-digit octal permission string")
    if type(mkdir) is not bool:
        raise ValueError("mkdir must be a Boolean")
    with _translate_transport_errors("write Sprite file"):
        async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
            async with self._http_client.stream(
                "PUT",
                f"/v1/sprites/{name}/fs/write",
                headers={
                    "Authorization": f"Bearer {self._api_token}",
                    "Content-Type": "application/octet-stream",
                },
                params={
                    "path": path,

```

```

        "workingDir": "/",
        "mode": mode,
        "mkdir": str(mkdir).lower(),
    },
    content=content,
    timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
) as response:
    await _read_bounded_response(response, "Sprite file write response")
    status_code = response.status_code
if not 200 <= status_code < 300:
    raise SpritesProviderError(
        f"Fly could not write Sprite file (status {status_code})"
    ) from None

async def read_file(
    self,
    name: str,
    *,
    path: str,
    max_bytes: int,
) -> bytes:
    """Read one small guest file under an explicit byte limit."""
    name = _validate_sprite_name(name)
    path = _validate_file_path(path, field="File path", allow_root=False)
    if type(max_bytes) is not int or not 1 <= max_bytes <= _MAX_FILE_CONTENT_BYTES:
        raise ValueError("max_bytes is outside the small-file limit")
    with _translate_transport_errors("read Sprite file"):
        async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
            async with self._http_client.stream(
                "GET",
                f"/v1/sprites/{name}/fs/read",
                headers={"Authorization": f"Bearer {self._api_token}"},
                params={"path": path, "workingDir": "/"},
                timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
            ) as response:
                body = await _read_bounded_response(
                    response,
                    "Sprite file read response",
                    max_bytes=max_bytes,
                    maximum=_MAX_FILE_CONTENT_BYTES,
                )
                status_code = response.status_code
if not 200 <= status_code < 300:
    raise SpritesProviderError(
        f"Fly could not read Sprite file (status {status_code})"
    ) from None

```

```

        return body

    async def delete_file(
        self,
        name: str,
        *,
        path: str,
    ) -> None:
        """Delete one exact guest file without recursive behavior."""
        name = _validate_sprite_name(name)
        path = _validate_file_path(path, field="File path", allow_root=False)
        with _translate_transport_errors("delete Sprite file"):
            async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
                async with self._http_client.stream(
                    "DELETE",
                    f"/v1/sprites/{name}/fs/delete",
                    headers={"Authorization": f"Bearer {self._api_token}"},
                    params={"path": path, "workingDir": "/", "recursive": "false"},
                    timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
                ) as response:
                    await _read_bounded_response(response, "Sprite file delete response")
                    status_code = response.status_code
        if status_code == 404:
            return
        if not 200 <= status_code < 300:
            raise SpritesProviderError(
                f"Fly could not delete Sprite file (status {status_code})"
            )
        return None

    async def upload_file(
        self,
        name: str,
        *,
        source_path: Path,
        path: str,
        expected_size: int,
        expected_sha256: str,
        mode: str,
    ) -> SpriteFileTransfer:
        """Upload one verified ciphertext file without loading it into memory."""
        name = _validate_sprite_name(name)
        path = _validate_file_path(path, field="File path", allow_root=False)
        if (
            not isinstance(source_path, Path)
            or not source_path.is_file()
            or source_path.is_symlink()

```

```

):
    raise ValueError("source_path must be a regular pathlib.Path")
if type(expected_size) is not int or not 1 <= expected_size <= _FILE_TRANSFER_BYTES:
    raise ValueError("expected_size is outside the transfer limit")
if source_path.stat().st_size != expected_size:
    raise ValueError("source file size does not match expected_size")
if (
    not isinstance(expected_sha256, str)
    or len(expected_sha256) != 64
    or any(character not in "0123456789abcdef" for character in expected_sha256)
):
    raise ValueError("expected_sha256 must be 64 lowercase hexadecimal characters")
if not isinstance(mode, str) or _FILE_MODE_PATTERN.fullmatch(mode) is None:
    raise ValueError("File mode must be a four-digit octal permission string")
digest = hashlib.sha256()

async def content() -> AsyncIterator[bytes]:
    source_flags = os.O_RDONLY | getattr(os, "O_NOFOLLOW", 0)
    descriptor = os.open(source_path, source_flags)
    try:
        with os.fdopen(descriptor, "rb", closefd=False) as source:
            while chunk := await asyncio.to_thread(source.read, _FILE_TRANSFER_CHUNK):
                digest.update(chunk)
            yield chunk
    finally:
        os.close(descriptor)

with _translate_transport_errors("write Sprite file"):
    async with asyncio.timeout(_LONG_OPERATION_TIMEOUT_SECONDS):
        async with self._http_client.stream(
            "PUT",
            f"/v1/sprites/{name}/fs/write",
            headers={
                "Authorization": f"Bearer {self._api_token}",
                "Content-Type": "application/octet-stream",
            },
            params={
                "path": path,
                "workingDir": "/",
                "mode": mode,
                "mkdir": "true",
            },
            content=content(),
            timeout=_LONG_OPERATION_TIMEOUT_SECONDS,
        ) as response:
            await _read_bounded_response(response, "Sprite file write response")

```

```

        status_code = response.status_code
    if not 200 <= status_code < 300:
        raise SpritesProviderError(
            f"Fly could not write Sprite file (status {status_code})"
        ) from None
    actual_sha256 = digest.hexdigest()
    if actual_sha256 != expected_sha256:
        raise SpritesProtocolError("Sprite upload source checksum did not match")
    return SpriteFileTransfer(size_bytes=expected_size, sha256=actual_sha256)

async def download_file(
    self,
    name: str,
    *,
    path: str,
    target_path: Path,
    expected_size: int,
    expected_sha256: str,
) -> SpriteFileTransfer:
    """Download one guest ciphertext file through validated byte ranges."""
    name = _validate_sprite_name(name)
    path = _validate_file_path(path, field="File path", allow_root=False)
    if not isinstance(target_path, Path) or not target_path.is_absolute():
        raise ValueError("target_path must be an absolute pathlib.Path")
    if target_path.exists() or target_path.is_symlink():
        raise FileExistsError(target_path)
    if type(expected_size) is not int or not 1 <= expected_size <= _FILE_TRANSFER_BYTES:
        raise ValueError("expected_size is outside the transfer limit")
    if (
        not isinstance(expected_sha256, str)
        or len(expected_sha256) != 64
        or any(character not in "0123456789abcdef" for character in expected_sha256)
    ):
        raise ValueError("expected_sha256 must be 64 lowercase hexadecimal characters")
    target_path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    descriptor = os.open(target_path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    digest = hashlib.sha256()
    written = 0
    try:
        with os.fdopen(descriptor, "wb", closefd=False) as output:
            while written < expected_size:
                end = min(written + _FILE_TRANSFER_CHUNK_BYTES, expected_size) - 1
                with _translate_transport_errors("read Sprite file"):
                    async with asyncio.timeout(_LONG_OPERATION_TIMEOUT_SECONDS):
                        async with self._http_client.stream(
                            "GET",

```

```

        f"/v1/sprites/{name}/fs/read",
        headers={
            "Authorization": f"Bearer {self._api_token}",
            "Range": f"bytes={written}-{end}",
        },
        params={"path": path, "workingDir": "/"},
        timeout=_LONG_OPERATION_TIMEOUT_SECONDS,
    ) as response:
        if response.status_code != 206:
            raise SpritesProtocolError(
                "Sprite file read did not honor the byte range"
            )
        content_range = response.headers.get("Content-Range", "")
        match = _CONTENT_RANGE_PATTERN.fullmatch(content_range)
        if (
            match is None
            or int(match.group(1)) != written
            or int(match.group(2)) != end
            or int(match.group(3)) != expected_size
        ):
            raise SpritesProtocolError(
                "Sprite file read returned an invalid content range"
            )
        range_bytes = 0
        async for chunk in response.aiter_bytes():
            range_bytes += len(chunk)
            if range_bytes > end - written + 1:
                raise SpritesProtocolError(
                    "Sprite file read exceeded the requested range"
                )
            output.write(chunk)
            digest.update(chunk)
        if range_bytes != end - written + 1:
            raise SpritesProtocolError(
                "Sprite file read returned an incomplete range"
            )
        written = end + 1
        output.flush()
        os.fsync(output.fileno())
    except BaseException:
        target_path.unlink(missing_ok=True)
        raise
    finally:
        os.close(descriptor)
    actual_sha256 = digest.hexdigest()
    if actual_sha256 != expected_sha256:

```

```

        target_path.unlink(missing_ok=True)
        raise SpritesProtocolError("Sprite file checksum did not match")
    os.chmod(target_path, 0o600)
    return SpriteFileTransfer(size_bytes=written, sha256=actual_sha256)

async def configure_service(
    self,
    name: str,
    *,
    service_name: str,
    command: str,
    args: tuple[str, ...],
    environment: Mapping[str, str],
    directory: str,
    needs: tuple[str, ...],
    http_port: int | None = None,
    monitor_duration: float | None = None,
) -> None:
    """Create or update one named Sprite service."""
    name = _validate_sprite_name(name)
    service_name = _validate_service_name(service_name)
    monitor_duration = _validate_monitor_duration(monitor_duration)
    command = _validate_service_command(command)
    args = _validate_service_args(args)
    validated_environment = _validate_service_environment(environment)
    directory = _validate_file_path(directory, field="Service directory", allow_root=True)
    needs = _validate_service_dependencies(needs)
    http_port = _validate_http_port(http_port)
    payload: dict[str, object] = {
        "cmd": command,
        "args": list(args),
        "env": validated_environment,
        "dir": directory,
        "needs": list(needs),
    }
    if http_port is not None:
        payload["http_port"] = http_port
    params = None
    if monitor_duration is not None:
        params = {"duration": f"{monitor_duration:g}s"}
    operation = (
        "configure runner service" if service_name == "yinshi-runner" else "configure service"
    )
    stream_operation = "runner service" if service_name == "yinshi-runner" else "service"
    stream_timeout = _service_stream_timeout(monitor_duration)
    with _translate_transport_errors(operation):

```



```

        async with asyncio.timeout(stream_timeout):
            async with self._http_client.stream(
                "PUT",
                f"/v1/sprites/{name}/services/{service_name}",
                headers={"Authorization": f"Bearer {self._api_token}"},
                params=params,
                json=payload,
                timeout=stream_timeout,
            ) as response:
                try:
                    response.raise_for_status()
                except httpx.HTTPStatusError:
                    raise SpritesProviderError(
                        f"Fly could not {operation} (status {response.status_code})"
                    ) from None
                body = bytearray()
                async for chunk in response.aiter_bytes():
                    if len(body) + len(chunk) > _MAX_STREAM_RESPONSE_BYTES:
                        raise SpritesProtocolError(
                            f"Sprite {stream_operation} response exceeds size limit"
                        )
                    body.extend(chunk)
            _raise_for_stream_error(
                httpx.Response(200, content=bytes(body)),
                stream_operation,
            )

    async def start_service(
        self,
        name: str,
        *,
        service_name: str,
        monitor_duration: float | None,
    ) -> None:
        """Start one service and require started plus complete progress events."""
        name = _validate_sprite_name(name)
        service_name = _validate_service_name(service_name)
        monitor_duration = _validate_monitor_duration(monitor_duration)
        params = None
        if monitor_duration is not None:
            params = {"duration": f"{monitor_duration:g}s"}
        stream_timeout = _service_stream_timeout(monitor_duration)
        with _translate_transport_errors("start service"):
            async with asyncio.timeout(stream_timeout):
                async with self._http_client.stream(
                    "POST",

```

```

        f"/v1/sprites/{name}/services/{service_name}/start",
        headers={"Authorization": f"Bearer {self._api_token}"},
        params=params,
        timeout=stream_timeout,
    ) as response:
        try:
            response.raise_for_status()
        except httpx.HTTPStatusError:
            raise SpritesProviderError(
                f"Fly could not start service (status {response.status_code})"
            ) from None
        body = await _read_bounded_response(response, "Sprite service start resp
    parsed = httpx.Response(200, content=body)
    _raise_for_stream_error(parsed, "service start")
    events = [json.loads(line) for line in parsed.text.splitlines() if line.strip()]
    if not any(isinstance(event, dict) and event.get("type") == "started" for event in e
        raise SpritesProtocolError("Sprite service start response did not start")

async def stop_service(
    self,
    name: str,
    *,
    service_name: str,
    timeout_seconds: int,
) -> None:
    """Stop one service and require stopped plus complete progress events."""
    name = _validate_sprite_name(name)
    service_name = _validate_service_name(service_name)
    if type(timeout_seconds) is not int or not 1 <= timeout_seconds <= 300:
        raise ValueError("timeout_seconds must be between 1 and 300")
    stream_timeout = float(timeout_seconds + 15)
    with _translate_transport_errors("stop service"):
        async with asyncio.timeout(stream_timeout):
            async with self._http_client.stream(
                "POST",
                f"/v1/sprites/{name}/services/{service_name}/stop",
                headers={"Authorization": f"Bearer {self._api_token}"},
                params={"timeout": f"{timeout_seconds}s"},
                timeout=stream_timeout,
            ) as response:
                try:
                    response.raise_for_status()
                except httpx.HTTPStatusError:
                    raise SpritesProviderError(
                        f"Fly could not stop service (status {response.status_code})"
                    ) from None

```

```

        body = await _read_bounded_response(response, "Sprite service stop response")
    parsed = httpx.Response(200, content=body)
    _raise_for_stream_error(parsed, "service stop")
    events = [json.loads(line) for line in parsed.text.splitlines() if line.strip()]
    if not any(isinstance(event, dict) and event.get("type") == "stopped" for event in events):
        raise SpritesProtocolError("Sprite service stop response did not stop")

async def delete_service(
    self,
    name: str,
    *,
    service_name: str,
) -> None:
    """Delete one exact service definition with a bounded response."""
    name = _validate_sprite_name(name)
    service_name = _validate_service_name(service_name)
    with _translate_transport_errors("delete service"):
        async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
            async with self._http_client.stream(
                "DELETE",
                f"/v1/sprites/{name}/services/{service_name}",
                headers={"Authorization": f"Bearer {self._api_token}"},
                timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
            ) as response:
                if response.status_code == 404:
                    return
                try:
                    response.raise_for_status()
                except httpx.HTTPStatusError:
                    raise SpritesProviderError(
                        f"Fly could not delete service (status {response.status_code})"
                    ) from None
            await _read_bounded_response(response, "Sprite service delete response")

async def restart_service(
    self,
    name: str,
    *,
    service_name: str,
    monitor_duration: float | None,
) -> None:
    """Restart one service and consume bounded provider progress."""
    name = _validate_sprite_name(name)
    service_name = _validate_service_name(service_name)
    monitor_duration = _validate_monitor_duration(monitor_duration)
    params = None

```

```

if monitor_duration is not None:
    params = {"duration": f"{monitor_duration:g}s"}
    stream_timeout = _service_stream_timeout(monitor_duration)
    with _translate_transport_errors("restart service"):
        async with asyncio.timeout(stream_timeout):
            async with self._http_client.stream(
                "POST",
                f"/v1/sprites/{name}/services/{service_name}/restart",
                headers={"Authorization": f"Bearer {self._api_token}"},
                params=params,
                timeout=stream_timeout,
            ) as response:
                try:
                    response.raise_for_status()
                except httpx.HTTPStatusError:
                    raise SpritesProviderError(
                        f"Fly could not restart service (status {response.status_code})"
                    ) from None
                body = bytearray()
                async for chunk in response.aiter_bytes():
                    if len(body) + len(chunk) > _MAX_STREAM_RESPONSE_BYTES:
                        raise SpritesProtocolError(
                            "Sprite service restart response exceeds size limit"
                        )
                    body.extend(chunk)
            _raise_for_stream_error(
                httpx.Response(200, content=bytes(body)),
                "service restart",
            )

async def get_service(
    self,
    name: str,
    *,
    service_name: str,
) -> ServiceRecord | None:
    """Return one typed Sprite service record when it exists."""
    name = _validate_sprite_name(name)
    service_name = _validate_service_name(service_name)
    with _translate_transport_errors("get service"):
        async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
            async with self._http_client.stream(
                "GET",
                f"/v1/sprites/{name}/services/{service_name}",
                headers={"Authorization": f"Bearer {self._api_token}"},
                timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
            ) as response:

```

```

        ) as response:
            body = await _read_bounded_response(
                response,
                "Sprite service response",
            )
            status_code = response.status_code
    if status_code == 404:
        return None
    if not 200 <= status_code < 300:
        raise SpritesProviderError(
            f"Fly could not get service (status {status_code})"
        ) from None
    return _parse_service_response(
        httpx.Response(status_code, content=body),
        service_name,
    )

async def configure_private_runner(
    self,
    name: str,
    *,
    command: str,
    args: tuple[str, ...],
    environment: Mapping[str, str],
    working_directory: str,
) -> None:
    """Create or update the private Yinshi runner service."""
    await self.configure_service(
        name,
        service_name="yinshi-runner",
        command=command,
        args=args,
        environment=environment,
        directory=working_directory,
        needs=(),
        http_port=None,
        monitor_duration=None,
    )

async def wake_sprite(self, name: str) -> None:
    """Wake one cold Sprite through the simple HTTP Exec endpoint."""
    name = _validate_sprite_name(name)
    with _translate_transport_errors("wake Sprite"):
        async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
            async with self._http_client.stream(
                "POST",

```

```

        f"/v1/sprites/{name}/exec",
        headers={"Authorization": f"Bearer {self._api_token}"},
        params=[("cmd", "true")],
        timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
    ) as response:
        await _read_bounded_response(response, "Sprite wake response")
        status_code = response.status_code
    if not 200 <= status_code < 300:
        raise SpritesProviderError(
            f"Fly could not wake Sprite (status {status_code})"
        ) from None

    async def delete_sprite(self, name: str) -> None:
        """Permanently delete one Sprite."""
        name = _validate_sprite_name(name)
        with _translate_transport_errors("delete Sprite"):
            async with asyncio.timeout(_STANDARD_OPERATION_TIMEOUT_SECONDS):
                async with self._http_client.stream(
                    "DELETE",
                    f"/v1/sprites/{name}",
                    headers={"Authorization": f"Bearer {self._api_token}"},
                    timeout=_STANDARD_OPERATION_TIMEOUT_SECONDS,
                ) as response:
                    await _read_bounded_response(response, "Sprite delete response")
                    status_code = response.status_code
        if status_code == 404:
            return
        if not 200 <= status_code < 300:
            raise SpritesProviderError(
                f"Fly could not delete Sprite (status {status_code})"
            ) from None

    async def restore_checkpoint(self, name: str, *, checkpoint_id: str) -> None:
        """Restore one checkpoint and consume bounded provider progress."""
        name = _validate_sprite_name(name)
        checkpoint_id = _validate_checkpoint_id(checkpoint_id)
        with _translate_transport_errors("restore checkpoint"):
            async with asyncio.timeout(_LONG_OPERATION_TIMEOUT_SECONDS):
                async with self._http_client.stream(
                    "POST",
                    f"/v1/sprites/{name}/checkpoints/{checkpoint_id}/restore",
                    headers={"Authorization": f"Bearer {self._api_token}"},
                    timeout=_LONG_OPERATION_TIMEOUT_SECONDS,
                ) as response:
                    try:
                        response.raise_for_status()

```

```

        except httpx.HTTPStatusError:
            raise SpritesProviderError(
                f"Fly could not restore checkpoint (status {response.status_code})"
            ) from None
        body = bytearray()
        async for chunk in response.aiter_bytes():
            if len(body) + len(chunk) > _MAX_STREAM_RESPONSE_BYTES:
                raise SpritesProtocolError(
                    "Sprite checkpoint restore response exceeds size limit"
                )
            body.extend(chunk)
        _raise_for_stream_error(
            httpx.Response(200, content=bytes(body)),
            "checkpoint restore",
        )

    async def create_checkpoint(self, name: str, *, comment: str) -> None:
        """Create a filesystem checkpoint after reading its progress response."""
        name = _validate_sprite_name(name)
        comment = _validate_checkpoint_comment(comment)
        with _translate_transport_errors("create checkpoint"):
            async with asyncio.timeout(_LONG_OPERATION_TIMEOUT_SECONDS):
                async with self._http_client.stream(
                    "POST",
                    f"/v1/sprites/{name}/checkpoint",
                    headers={"Authorization": f"Bearer {self._api_token}"},
                    json={"comment": comment},
                    timeout=_LONG_OPERATION_TIMEOUT_SECONDS,
                ) as response:
                    try:
                        response.raise_for_status()
                    except httpx.HTTPStatusError:
                        raise SpritesProviderError(
                            f"Fly could not create checkpoint (status {response.status_code})"
                        ) from None
                    body = bytearray()
                    async for chunk in response.aiter_bytes():
                        if len(body) + len(chunk) > _MAX_STREAM_RESPONSE_BYTES:
                            raise SpritesProtocolError(
                                "Sprite checkpoint response exceeds size limit"
                            )
                        body.extend(chunk)
                    _raise_for_stream_error(httpx.Response(200, content=bytes(body)), "checkpoint")

```

16 Managed Backup and Recovery

Managed storage uses encrypted archives, immutable object versions, persisted operation state, and staged recovery. Recovery drills use separate identities and retain enough state to prevent an unsafe restart.

16.1 yinshi/api/managed_recovery_drills.py

Recovery drill routes start controlled exercises and return sanitized progress for authorized operators.

```
"""Staging-only operator boundary for destructive managed recovery drills."""

from __future__ import annotations

import hashlib
import hmac

from fastapi import APIRouter, Depends, HTTPException, Request
from pydantic import BaseModel, Field

router = APIRouter(prefix="/internal/managed-recovery", tags=["managed-recovery"])

class ManagedRecoveryDrillStartIn(BaseModel):
    """Bounded source revision attached to one retained drill result."""

    commit_sha: str = Field(pattern=r"^[0-9a-f]{40}$")

def _require_operator(request: Request) -> None:
    """Require one exact dedicated staging bearer token."""
    authorization = request.headers.get("Authorization", "")
    scheme, separator, token = authorization.partition(" ")
    configured_hash = request.app.state.managed_recovery_operator_token_hash
    presented_hash = hashlib.sha256(token.encode("utf-8")).hexdigest()
    if (
        scheme.lower() != "bearer"
        or separator != " "
        or not token
        or not hmac.compare_digest(presented_hash, configured_hash)
    ):
        raise HTTPException(status_code=401, detail="Invalid operator token")

@router.post("/drills", status_code=202, dependencies=[Depends(_require_operator)])
```



```

async def start_managed_recovery_drill(
    body: ManagedRecoveryDrillStartIn,
    request: Request,
) -> dict[str, object]:
    """Start one application-owned staging recovery drill."""
    controller = getattr(request.app.state, "managed_recovery_drill_controller", None)
    if controller is None:
        raise HTTPException(status_code=503, detail="Managed recovery drill is unavailable")
    try:
        result = await controller.start(commit_sha=body.commit_sha)
    except RuntimeError as error:
        raise HTTPException(status_code=409, detail="Managed recovery drill is active") from error
    if not isinstance(result, dict):
        raise HTTPException(status_code=503, detail="Managed recovery drill is unavailable")
    return result

@router.get("/drills/latest", dependencies=[Depends(_require_operator)])
def get_managed_recovery_drill_status(request: Request) -> dict[str, object]:
    """Return the latest sanitized aggregate drill state."""
    controller = getattr(request.app.state, "managed_recovery_drill_controller", None)
    if controller is None:
        raise HTTPException(status_code=503, detail="Managed recovery drill is unavailable")
    result = controller.status()
    if not isinstance(result, dict):
        raise HTTPException(status_code=503, detail="Managed recovery drill is unavailable")
    return result

```

16.2 yinshi/managed_backup_guest.py

The in-guest backup worker quiesces services, copies encrypted state, publishes one archive, and records restart markers.

```

"""Create and restore encrypted managed guest archives."""

from __future__ import annotations

import argparse
import asyncio
import base64
import binascii
import hashlib
import io
import json
import os
import re

```

```

import shutil
import stat
import tarfile
import tempfile
from collections.abc import Callable
from dataclasses import asdict, dataclass
from pathlib import Path, PurePosixPath
from typing import BinaryIO

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

from yinshi.services.managed_backup_crypto import open_managed_backup_job
from yinshi.services.runner_noise import load_or_create_runner_noise_keypair
from yinshi.services.sprite_task_lease import SpriteTaskLease

_ARCHIVE_MAGIC = b"YINSHI-MANAGED-BACKUP-V1\n"
_CHUNK_BYTES = 1024 * 1024
_NONCE_BYTES = 12
_TAG_BYTES = 16
_MANIFEST_NAME = "manifest.json"
_PORTABLE_DATA_KEY_MEMBER = "sqlite/.yinshi-data-protection-key"
_PORTABLE_DATA_KEY_ERROR = "managed backup portable data-protection key is invalid"
_MEMBER_COUNT_MAX = 200_000
_MEMBER_BYTES_MAX = 64 * 1024 * 1024 * 1024
_EXPANDED_BYTES_MAX = 200 * 1024 * 1024 * 1024
_STATE_ROOT = Path("/var/lib/yinshi")
_RESTORE_TRANSACTION_NAME = ".yinshi-restore-active"
_RESTORE_CLEANUP_NAME = ".yinshi-restore-cleanup"
_JOB_ID_PATTERN = re.compile(r"[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}")

@dataclass(frozen=True, slots=True)
class ManagedArchiveContext:
    """Authenticated identity and lifecycle binding for one guest archive."""

    archive_id: str
    created_at: str
    owner_digest: str
    runtime_generation: int

@dataclass(frozen=True, slots=True)
class ManagedArchiveRecord:
    """Ciphertext metadata returned after durable archive creation."""

    size_bytes: int

```

```

sha256: str

@dataclass(frozen=True, slots=True)
class ManagedRestoreResult:
    """Durable restore commit outcome with independent cleanup state."""

    committed: bool
    cleanup_pending: bool

def _require_context(context: ManagedArchiveContext) -> None:
    """Validate bounded archive context before writing authenticated metadata."""
    if not isinstance(context, ManagedArchiveContext):
        raise TypeError("context must be ManagedArchiveContext")
    if not context.archive_id or len(context.archive_id) > 128:
        raise ValueError("archive_id must be bounded non-empty text")
    if not context.created_at or len(context.created_at) > 128:
        raise ValueError("created_at must be bounded non-empty text")
    if len(context.owner_digest) != 64 or any(
        character not in "0123456789abcdef" for character in context.owner_digest
    ):
        raise ValueError("owner_digest must be 64 lowercase hexadecimal characters")
    if type(context.runtime_generation) is not int or context.runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")

def _require_key(key: bytes) -> bytes:
    """Copy one exact AES-256 archive key."""
    if not isinstance(key, bytes) or len(key) != 32:
        raise ValueError("archive_key must contain exactly 32 bytes")
    return bytes(key)

def _regular_files(root: Path, archive_root: str) -> list[tuple[Path, str, os.stat_result]]:
    """List regular files without following links or accepting special entries."""
    if not isinstance(root, Path) or not root.is_absolute():
        raise ValueError(f"{archive_root}_root must be an absolute path")
    metadata = root.lstat()
    if not stat.S_ISDIR(metadata.st_mode) or root.is_symlink():
        raise ValueError(f"{archive_root}_root must be a real directory")
    files: list[tuple[Path, str, os.stat_result]] = []
    for directory_name, directory_names, file_names in os.walk(root, followlinks=False):
        directory = Path(directory_name)
        for name in tuple(directory_names):
            candidate = directory / name

```

```

        candidate_metadata = candidate.lstat()
        if not stat.S_ISDIR(candidate_metadata.st_mode) or candidate.is_symlink():
            raise ValueError("managed backup roots must not contain links or special ent
    for name in file_names:
        candidate = directory / name
        candidate_metadata = candidate.lstat()
        if not stat.S_ISREG(candidate_metadata.st_mode) or candidate.is_symlink():
            raise ValueError("managed backup roots must contain regular files only")
        relative = candidate.relative_to(root)
        member_name = str(PurePosixPath(archive_root, *relative.parts))
        files.append((candidate, member_name, candidate_metadata))
    return sorted(files, key=lambda item: item[1])

def _validate_portable_data_key_descriptor(descriptor: int) -> None:
    """Validate exact portable storage-key descriptor and reset its offset."""
    metadata = os.fstat(descriptor)
    if (
        not stat.S_ISREG(metadata.st_mode)
        or metadata.st_uid != os.geteuid()
        or metadata.st_nlink != 1
        or stat.S_IMODE(metadata.st_mode) != 0o600
        or metadata.st_size != 32
    ):
        raise ValueError(_PORTABLE_DATA_KEY_ERROR)
    if len(os.read(descriptor, 32)) != 32 or os.read(descriptor, 1):
        raise ValueError(_PORTABLE_DATA_KEY_ERROR)
    os.lseek(descriptor, 0, os.SEEK_SET)

def _validate_portable_data_key(source_path: Path) -> None:
    """Require exact portable storage-key metadata without following links."""
    try:
        descriptor = os.open(source_path, os.O_RDONLY | getattr(os, "O_NOFOLLOW", 0))
    except OSError as exc:
        raise ValueError(_PORTABLE_DATA_KEY_ERROR) from exc
    try:
        _validate_portable_data_key_descriptor(descriptor)
    finally:
        os.close(descriptor)

def _manifest(context: ManagedArchiveContext, member_names: tuple[str, ...]) -> bytes:
    """Serialize exact authenticated archive metadata."""
    return (
        json.dumps(

```

```

        {
            "context": asdict(context),
            "format": "yinshi-managed-backup-v2",
            "members": list(member_names),
        },
        separators=(",", ":"),
        sort_keys=True,
    )
    + "\n"
).encode("utf-8")

def _write_tar(
    target: BinaryIO,
    files: list[tuple[Path, str, os.stat_result]],
    manifest: bytes,
) -> None:
    """Write a deterministic uncompressed archive with bounded copy buffers."""
    with tarfile.open(fileobj=target, mode="w") as archive:
        manifest_member = tarfile.TarInfo(_MANIFEST_NAME)
        manifest_member.size = len(manifest)
        manifest_member.mode = 0o600
        manifest_member.mtime = 0
        archive.addfile(manifest_member, io.BytesIO(manifest))
        for source_path, member_name, listed_metadata in files:
            descriptor = os.open(source_path, os.O_RDONLY | getattr(os, "O_NOFOLLOW", 0))
            try:
                source_metadata = os.fstat(descriptor)
                source_changed = (
                    not stat.S_ISREG(source_metadata.st_mode)
                    or source_metadata.st_dev != listed_metadata.st_dev
                    or source_metadata.st_ino != listed_metadata.st_ino
                    or source_metadata.st_size != listed_metadata.st_size
                    or stat.S_IMODE(source_metadata.st_mode)
                    != stat.S_IMODE(listed_metadata.st_mode)
                )
                if member_name == _PORTABLE_DATA_KEY_MEMBER:
                    if source_changed:
                        raise ValueError(_PORTABLE_DATA_KEY_ERROR)
                    _validate_portable_data_key_descriptor(descriptor)
                elif source_changed:
                    raise ValueError("managed backup source changed during creation")
                member = tarfile.TarInfo(member_name)
                member.size = source_metadata.st_size
                member.mode = 0o600
                member.mtime = 0

```

```

        with os.fdopen(descriptor, "rb", closefd=False) as source:
            archive.addfile(member, source)
    finally:
        os.close(descriptor)

def _encrypt_file(source_path: Path, archive_path: Path, key: bytes) -> None:
    """Encrypt one staged tar with AES-256-GCM and durable private output."""
    archive_path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    if archive_path.exists() or archive_path.is_symlink():
        raise FileExistsError(archive_path)
    nonce = os.urandom(_NONCE_BYTES)
    encryptor = Cipher(algorithms.AES(key), modes.GCM(nonce)).encryptor()
    encryptor.authenticate_additional_data(_ARCHIVE_MAGIC)
    descriptor = os.open(archive_path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    try:
        with source_path.open("rb") as source, os.fdopen(descriptor, "wb", closefd=False) as output:
            output.write(_ARCHIVE_MAGIC)
            output.write(nonce)
            for chunk in iter(lambda: source.read(_CHUNK_BYTES), b''):
                output.write(encryptor.update(chunk))
            output.write(encryptor.finalize())
            output.write(encryptor.tag)
            output.flush()
            os.fsync(output.fileno())
    except BaseException:
        archive_path.unlink(missing_ok=True)
        raise
    finally:
        os.close(descriptor)

def create_managed_backup_archive(
    *,
    sqlite_root: Path,
    files_root: Path,
    archive_path: Path,
    archive_key: bytes,
    context: ManagedArchiveContext,
) -> ManagedArchiveRecord:
    """Create one encrypted archive from the managed guest's durable roots."""
    _require_context(context)
    key = _require_key(archive_key)
    sqlite_files = _regular_files(sqlite_root, "sqlite")
    portable_keys = [
        source_path

```

```

        for source_path, member_name, _metadata in sqlite_files
        if member_name == _PORTABLE_DATA_KEY_MEMBER
    ]
    if len(portable_keys) != 1:
        raise ValueError(_PORTABLE_DATA_KEY_ERROR)
    shared_files = _regular_files(files_root, "files")
    files = sorted(sqlite_files + shared_files, key=lambda item: item[1])
    member_names = tuple(member_name for _path, member_name, _metadata in files)
    manifest = _manifest(context, member_names)
    temporary_parent = archive_path.parent
    temporary_parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    descriptor, temporary_name = tempfile.mkstemp(prefix=".archive-", dir=temporary_parent)
    os.close(descriptor)
    temporary_path = Path(temporary_name)
    os.chmod(temporary_path, 0o600)
    try:
        with temporary_path.open("wb") as target:
            _write_tar(target, files, manifest)
            target.flush()
            os.fsync(target.fileno())
            _encrypt_file(temporary_path, archive_path, key)
    finally:
        temporary_path.unlink(missing_ok=True)
    digest = hashlib.sha256()
    with archive_path.open("rb") as source:
        for chunk in iter(lambda: source.read(_CHUNK_BYTES), b''):
            digest.update(chunk)
    return ManagedArchiveRecord(
        size_bytes=archive_path.stat().st_size,
        sha256=digest.hexdigest(),
    )

def _decrypt_to_temporary(archive_path: Path, key: bytes, target_path: Path) -> None:
    """Authenticate and decrypt one managed archive into a private tar file."""
    source_size = archive_path.stat().st_size
    minimum = len(_ARCHIVE_MAGIC) + _NONCE_BYTES + _TAG_BYTES
    if source_size <= minimum:
        raise ValueError("managed backup archive is truncated")
    with archive_path.open("rb") as source:
        if source.read(len(_ARCHIVE_MAGIC)) != _ARCHIVE_MAGIC:
            raise ValueError("managed backup archive header is invalid")
        nonce = source.read(_NONCE_BYTES)
        source.seek(-_TAG_BYTES, os.SEEK_END)
        tag = source.read(_TAG_BYTES)
        remaining = source_size - minimum

```

```

source.seek(len(_ARCHIVE_MAGIC) + _NONCE_BYTES)
decryptor = Cipher(algorithms.AES(key), modes.GCM(nonce, tag)).decryptor()
decryptor.authenticate_additional_data(_ARCHIVE_MAGIC)
with target_path.open("wb") as output:
    while remaining:
        chunk = source.read(min(_CHUNK_BYTES, remaining))
        if not chunk:
            raise ValueError("managed backup archive is truncated")
        remaining -= len(chunk)
        output.write(decryptor.update(chunk))
    output.write(decryptor.finalize())
    output.flush()
    os.fsync(output.fileno())
os.chmod(target_path, 0o600)

def _inspect_tar(tar_path: Path, expected_context: ManagedArchiveContext) -> tuple[str, ...]:
    """Validate exact archive layout and return ordered payload member names."""
    members: dict[str, tarfile.TarInfo] = {}
    expanded_bytes = 0
    with tarfile.open(tar_path, mode="r") as archive:
        for index, member in enumerate(archive, start=1):
            if index > _MEMBER_COUNT_MAX:
                raise ValueError("managed backup contains too many members")
            path = PurePosixPath(member.name)
            if (
                not member.isfile()
                or member.name in members
                or member.name.startswith("/")
                or "\\" in member.name
                or any(part in {"", ".", ".."} for part in path.parts)
            ):
                raise ValueError("managed backup contains an unsafe member")
            if member.size < 0 or member.size > _MEMBER_BYTES_MAX:
                raise ValueError("managed backup member exceeds the size limit")
            expanded_bytes += member.size
            if expanded_bytes > _EXPANDED_BYTES_MAX:
                raise ValueError("managed backup exceeds the expanded size limit")
            members[member.name] = member
    manifest_member = members.pop(_MANIFEST_NAME, None)
    if manifest_member is None or manifest_member.size > 64 * 1024:
        raise ValueError("managed backup manifest is missing or too large")
    manifest_source = archive.extractfile(manifest_member)
    if manifest_source is None:
        raise ValueError("managed backup manifest cannot be read")
    try:

```



```

        manifest = json.loads(manifest_source.read().decode("utf-8"))
    except (UnicodeDecodeError, json.JSONDecodeError):
        raise ValueError("managed backup manifest is invalid") from None
    if isinstance(manifest, dict) and manifest.get("format") == "yinshi-managed-backup-v1":
        raise ValueError("managed backup archive predates portable data-key support")
    expected_manifest = {
        "context": asdict(expected_context),
        "format": "yinshi-managed-backup-v2",
        "members": sorted(members),
    }
    if manifest != expected_manifest:
        raise ValueError("managed backup manifest does not match expected context")
    portable_key = members.get(_PORTABLE_DATA_KEY_MEMBER)
    if portable_key is None or portable_key.size != 32 or stat.S_IMODE(portable_key.mode) != 0o600:
        raise ValueError(_PORTABLE_DATA_KEY_ERROR)
    for name in members:
        root = PurePosixPath(name).parts[0]
        if root not in {"sqlite", "files"}:
            raise ValueError("managed backup contains an unexpected root")
    return tuple(sorted(members))

def inspect_managed_backup_archive(
    archive_path: Path,
    *,
    archive_key: bytes,
    expected_context: ManagedArchiveContext,
) -> tuple[str, ...]:
    """Authenticate one managed archive and return its validated members."""
    _require_context(expected_context)
    key = _require_key(archive_key)
    if not isinstance(archive_path, Path) or not archive_path.is_file():
        raise FileNotFoundError(archive_path)
    with tempfile.TemporaryDirectory(prefix="yinshi-managed-inspect-") as directory_name:
        tar_path = Path(directory_name) / "archive.tar"
        _decrypt_to_temporary(archive_path, key, tar_path)
        return _inspect_tar(tar_path, expected_context)

def _extract_validated_tar(tar_path: Path, stage_root: Path) -> None:
    """Copy previously validated members into private restore staging."""
    with tarfile.open(tar_path, mode="r") as archive:
        for member in archive:
            if member.name == _MANIFEST_NAME:
                continue
            target = stage_root.joinpath(*PurePosixPath(member.name).parts)

```

```

target.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
parent = target.parent
while parent != stage_root:
    os.chmod(parent, 0o700)
    parent = parent.parent
source = archive.extractfile(member)
if source is None:
    raise ValueError("managed backup member cannot be read")
descriptor = os.open(target, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
try:
    with os.fdopen(descriptor, "wb", closefd=False) as output:
        remaining = member.size
        while remaining:
            chunk = source.read(min(_CHUNK_BYTES, remaining))
            if not chunk:
                raise ValueError("managed backup member is truncated")
            output.write(chunk)
            remaining -= len(chunk)
        output.flush()
        os.fsync(output.fileno())
finally:
    os.close(descriptor)

def _sync_directory(path: Path) -> None:
    """Persist directory entry changes before the next restore transition."""
    descriptor = os.open(path, os.O_RDONLY | getattr(os, "O_DIRECTORY", 0))
    try:
        os.fsync(descriptor)
    finally:
        os.close(descriptor)

def _sync_directory_tree(root: Path) -> None:
    """Persist a validated directory tree from leaves through its root."""
    if root.is_symlink() or not root.is_dir():
        raise ValueError("managed restore staging root must be a real directory")
    directories: list[Path] = []
    for directory_name, child_names, file_names in os.walk(root, followlinks=False):
        directory = Path(directory_name)
        for child_name in child_names:
            child = directory / child_name
            if child.is_symlink() or not child.is_dir():
                raise ValueError("managed restore staging contains unsafe directory")
        for file_name in file_names:
            file_path = directory / file_name

```

```

        if file_path.is_symlink() or not file_path.is_file():
            raise ValueError("managed restore staging contains unsafe file")
        directories.append(directory)
    for directory in reversed(directories):
        _sync_directory(directory)

def _write_restore_state(transaction_root: Path, phase: str) -> None:
    """Persist one exact restore phase inside the fixed transaction directory."""
    state_path = transaction_root / "state.json"
    temporary_path = transaction_root / ".state.json.tmp"
    payload = json.dumps(
        {"phase": phase, "version": 1},
        separators=(",", ":"),
        sort_keys=True,
    )
    with temporary_path.open("w", encoding="utf-8") as output:
        output.write(payload + "\n")
        output.flush()
        os.fsync(output.fileno())
    os.chmod(temporary_path, 0o600)
    os.replace(temporary_path, state_path)
    _sync_directory(transaction_root)

def _publish_restore_transaction(state_root: Path) -> Path:
    """Publish a complete prepared journal before changing live roots."""
    transaction_root = state_root / _RESTORE_TRANSACTION_NAME
    if transaction_root.exists() or transaction_root.is_symlink():
        raise FileExistsError(transaction_root)
    preparation_root = Path(tempfile.mkdtemp(prefix=".yinshi-restore-prepare-", dir=state_root))
    try:
        (preparation_root / "rollback").mkdir(mode=0o700)
        _write_restore_state(preparation_root, "prepared")
        _sync_directory(preparation_root)
        os.replace(preparation_root, transaction_root)
        _sync_directory(state_root)
    except BaseException:
        if preparation_root.exists() and not preparation_root.is_symlink():
            shutil.rmtree(preparation_root)
        raise
    if not transaction_root.is_dir() or transaction_root.is_symlink():
        raise RuntimeError("managed restore transaction was not published")
    return transaction_root

```

```

def _remove_restore_cleanup(state_root: Path) -> None:
    """Finish removal of a committed restore journal."""
    cleanup_root = state_root / _RESTORE_CLEANUP_NAME
    if not cleanup_root.exists() and not cleanup_root.is_symlink():
        return
    if cleanup_root.is_symlink() or not cleanup_root.is_dir():
        raise ValueError("managed restore cleanup path is unsafe")
    shutil.rmtree(cleanup_root)
    _sync_directory(state_root)

def _recover_restore_transaction(sqlite_root: Path, files_root: Path) -> None:
    """Recover old roots from one interrupted pre-commit replacement."""
    transaction_root = sqlite_root.parent / _RESTORE_TRANSACTION_NAME
    if not transaction_root.exists() and not transaction_root.is_symlink():
        return
    if transaction_root.is_symlink() or not transaction_root.is_dir():
        raise ValueError("managed restore transaction path is unsafe")
    state_path = transaction_root / "state.json"
    if state_path.is_symlink() or not state_path.is_file():
        raise ValueError("managed restore transaction state is missing")
    try:
        state = json.loads(state_path.read_text(encoding="utf-8"))
    except (UnicodeDecodeError, json.JSONDecodeError):
        raise ValueError("managed restore transaction state is invalid") from None
    if (
        not isinstance(state, dict)
        or state.get("version") != 1
        or state.get("phase")
        not in {"prepared", "old_sqlite_moved", "old_roots_moved", "new_sqlite_installed"}
    ):
        raise ValueError("managed restore transaction state is invalid")
    rollback_root = transaction_root / "rollback"
    for live_root in (sqlite_root, files_root):
        rollback = rollback_root / live_root.name
        if rollback.exists():
            if rollback.is_symlink() or not rollback.is_dir():
                raise ValueError("managed restore rollback root is unsafe")
            if live_root.exists() or live_root.is_symlink():
                failed_root = transaction_root / f"failed-{live_root.name}"
                if failed_root.exists() or failed_root.is_symlink():
                    shutil.rmtree(failed_root)
                    os.replace(live_root, failed_root)
                os.replace(rollback, live_root)
            _sync_directory(sqlite_root.parent)
    if not sqlite_root.is_dir() or not files_root.is_dir():

```

```

        raise ValueError("managed restore rollback is incomplete")
    shutil.rmtree(transaction_root)
    _sync_directory(sqlite_root.parent)

def restore_managed_backup_archive(
    archive_path: Path,
    *,
    archive_key: bytes,
    expected_context: ManagedArchiveContext,
    sqlite_root: Path,
    files_root: Path,
) -> ManagedRestoreResult:
    """Replace both guest data roots after full authentication and staging."""
    _require_context(expected_context)
    key = _require_key(archive_key)
    if sqlite_root.parent != files_root.parent or sqlite_root == files_root:
        raise ValueError("managed restore roots must be distinct siblings")
    state_root = sqlite_root.parent
    if not state_root.is_dir() or state_root.is_symlink():
        raise ValueError("managed restore state root must be a real directory")
    _remove_restore_cleanup(state_root)
    _recover_restore_transaction(sqlite_root, files_root)
    for root in (sqlite_root, files_root):
        if root.is_symlink() or not root.is_dir():
            raise ValueError("managed restore targets must be real directories")
    with tempfile.TemporaryDirectory(prefix=".yinshi-restore-stage-", dir=state_root) as stage_root:
        stage_root = Path(stage_root)
        tar_path = stage_root / "archive.tar"
        _decrypt_to_temporary(archive_path, key, tar_path)
        _inspect_tar(tar_path, expected_context)
        extracted_root = stage_root / "extracted"
        extracted_root.mkdir(mode=0o700)
        staged_sqlite = extracted_root / "sqlite"
        staged_files = extracted_root / "files"
        staged_sqlite.mkdir(mode=0o700)
        staged_files.mkdir(mode=0o700)
        _extract_validated_tar(tar_path, extracted_root)
        if not staged_sqlite.is_dir() or not staged_files.is_dir():
            raise ValueError("managed backup is missing required data roots")
        _validate_portable_data_key(staged_sqlite / ".yinshi-data-protection-key")
        _sync_directory_tree(staged_sqlite)
        _sync_directory_tree(staged_files)
        _sync_directory(extracted_root)
        transaction_root = _publish_restore_transaction(state_root)
        rollback_root = transaction_root / "rollback"

```

```

rollback_sqlite = rollback_root / "sqlite"
rollback_files = rollback_root / "files"
try:
    os.replace(sqlite_root, rollback_sqlite)
    _sync_directory(state_root)
    _write_restore_state(transaction_root, "old_sqlite_moved")
    os.replace(files_root, rollback_files)
    _sync_directory(state_root)
    _write_restore_state(transaction_root, "old_roots_moved")
    os.replace(staged_sqlite, sqlite_root)
    _sync_directory(state_root)
    _write_restore_state(transaction_root, "new_sqlite_installed")
    os.replace(staged_files, files_root)
    _sync_directory(state_root)
except BaseException:
    _recover_restore_transaction(sqlite_root, files_root)
    raise
else:
    cleanup_root = state_root / _RESTORE_CLEANUP_NAME
    if cleanup_root.exists() or cleanup_root.is_symlink():
        raise FileExistsError(cleanup_root)
    os.replace(transaction_root, cleanup_root)
    _sync_directory(state_root)
    try:
        _remove_restore_cleanup(state_root)
    except OSError:
        return ManagedRestoreResult(committed=True, cleanup_pending=True)
return ManagedRestoreResult(committed=True, cleanup_pending=False)

def _decode_archive_key(value: object) -> bytes:
    """Decode one canonical unpadded archive key from a sealed job."""
    if not isinstance(value, str) or not value:
        raise ValueError("managed backup job archive_key is invalid")
    try:
        decoded = base64.b64decode(
            value + "=" * (-len(value) % 4),
            altchars=b"-_",
            validate=True,
        )
    except (binascii.Error, ValueError):
        raise ValueError("managed backup job archive_key is invalid") from None
    canonical = base64.urlsafe_b64encode(decoded).rstrip(b"=").decode("ascii")
    if len(decoded) != 32 or canonical != value:
        raise ValueError("managed backup job archive_key is invalid")
    return decoded

```

```

def _job_context(value: object) -> ManagedArchiveContext:
    """Parse exact authenticated archive context from one sealed job."""
    if not isinstance(value, dict) or set(value) != {
        "archive_id",
        "created_at",
        "owner_digest",
        "runtime_generation",
    }:
        raise ValueError("managed backup job archive context is invalid")
    context = ManagedArchiveContext(
        archive_id=value["archive_id"],
        created_at=value["created_at"],
        owner_digest=value["owner_digest"],
        runtime_generation=value["runtime_generation"],
    )
    _require_context(context)
    return context

def _read_existing_job_result(
    path: Path,
    *,
    archive_path: Path,
    job_id: str,
    operation: str,
) -> bool:
    """Accept only complete same-job output from an interrupted control transfer."""
    if not path.exists() and not path.is_symlink():
        return False
    if path.is_symlink() or not path.is_file():
        raise ValueError("managed backup result path is unsafe")
    try:
        result = json.loads(path.read_text(encoding="utf-8"))
    except (UnicodeDecodeError, json.JSONDecodeError):
        raise ValueError("managed backup result is invalid") from None
    if operation == "restore":
        if result != {
            "cleanup_pending": False,
            "job_id": job_id,
            "status": "restored",
        }:
            raise ValueError("managed backup result is invalid")
        return True
    if (

```

```

        not isinstance(result, dict)
        or set(result) != {"job_id", "sha256", "size_bytes", "status"}
        or result.get("job_id") != job_id
        or result.get("status") != "ready"
        or type(result.get("size_bytes")) is not int
        or result["size_bytes"] <= 0
        or not isinstance(result.get("sha256"), str)
        or len(result["sha256"]) != 64
        or any(character not in "0123456789abcdef" for character in result["sha256"])
        or archive_path.is_symlink()
        or not archive_path.is_file()
        or archive_path.stat().st_size != result["size_bytes"]
    ):
        raise ValueError("managed backup result is invalid")
    digest = hashlib.sha256()
    with archive_path.open("rb") as source:
        for chunk in iter(lambda: source.read(_CHUNK_BYTES), b""):
            digest.update(chunk)
    if digest.hexdigest() != result["sha256"]:
        raise ValueError("managed backup result is invalid")
    return True

def _write_job_result(path: Path, payload: dict[str, object]) -> None:
    """Atomically write one private fixed-shape maintenance result."""
    if path.exists() or path.is_symlink():
        raise FileExistsError(path)
    path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    descriptor, temporary_name = tempfile.mkstemp(prefix=".result-", dir=path.parent)
    temporary_path = Path(temporary_name)
    try:
        with os.fdopen(descriptor, "w", encoding="utf-8") as output:
            json.dump(payload, output, separators=(",", ":"), sort_keys=True)
            output.write("\n")
            output.flush()
            os.fsync(output.fileno())
        os.chmod(temporary_path, 0o600)
        os.replace(temporary_path, path)
        _sync_directory(path.parent)
    finally:
        temporary_path.unlink(missing_ok=True)

def run_managed_backup_job(
    *,
    job_path: Path,

```



```

result_path: Path,
runner_private_key: bytes,
sqlite_root: Path,
files_root: Path,
maintenance_root: Path,
expected_job_id: str,
) -> None:
    """Open and execute one exact managed backup maintenance job."""
    if job_path.parent != maintenance_root or result_path.parent != maintenance_root:
        raise ValueError("managed backup job paths must be inside maintenance_root")
    if job_path.is_symlink() or not job_path.is_file():
        raise ValueError("managed backup job must be a regular file")
    payload = open_managed_backup_job(
        job_path.read_bytes(),
        runner_private_key=runner_private_key,
        expected_job_id=expected_job_id,
    )
    if not isinstance(payload, dict) or set(payload) != {
        "archive_context",
        "archive_key",
        "job_id",
        "operation",
        "version",
    }:
        raise ValueError("managed backup job has an invalid shape")
    operation = payload["operation"]
    if (
        payload["version"] != 1
        or payload["job_id"] != expected_job_id
        or operation not in {"create", "restore"}
    ):
        raise ValueError("managed backup job is invalid")
    context = _job_context(payload["archive_context"])
    archive_path = maintenance_root / f"{expected_job_id}.archive.enc"
    if _read_existing_job_result(
        result_path,
        archive_path=archive_path,
        job_id=expected_job_id,
        operation=operation,
    ):
        return
    archive_key = _decode_archive_key(payload["archive_key"])
    if operation == "create":
        record = create_managed_backup_archive(
            sqlite_root=sqlite_root,
            files_root=files_root,

```

```

        archive_path=archive_path,
        archive_key=archive_key,
        context=context,
    )
    result = {
        "job_id": expected_job_id,
        "sha256": record.sha256,
        "size_bytes": record.size_bytes,
        "status": "ready",
    }
else:
    restore_result = restore_managed_backup_archive(
        archive_path,
        archive_key=archive_key,
        expected_context=context,
        sqlite_root=sqlite_root,
        files_root=files_root,
    )
    result = {
        "cleanup_pending": restore_result.cleanup_pending,
        "job_id": expected_job_id,
        "status": "restored",
    }
_write_job_result(result_path, result)

async def hold_managed_backup_job(
    *,
    job_id: str,
    maintenance_root: Path,
    run_job: Callable[[], None],
    timeout_seconds: int,
) -> None:
    """Hold one expiring Sprite task through control-plane transfer completion."""
    if _JOB_ID_PATTERN.fullmatch(job_id) is None:
        raise ValueError("job_id must be a canonical lowercase UUID")
    if maintenance_root.is_symlink() or not maintenance_root.is_dir():
        raise ValueError("maintenance_root must be a real directory")
    if not callable(run_job):
        raise TypeError("run_job must be callable")
    if type(timeout_seconds) is not int or not 1 <= timeout_seconds <= 3600:
        raise ValueError("timeout_seconds must be between 1 and 3600")
    release_path = maintenance_root / f"{job_id}.release"
    tracked_paths = (
        maintenance_root / f"{job_id}.job",
        maintenance_root / f"{job_id}.result",
    )

```

```

        maintenance_root / f"{job_id}.archive.enc",
        release_path,
    )
    lease = SpriteTaskLease()
    await lease.acquire()
    released = False
    job_completed = False
    try:
        await asyncio.to_thread(run_job)
        job_completed = True
        deadline = asyncio.get_running_loop().time() + timeout_seconds
        while not release_path.is_file() or release_path.is_symlink():
            if asyncio.get_running_loop().time() >= deadline:
                raise TimeoutError("managed backup release timed out")
            await asyncio.sleep(1.0)
        released = True
    finally:
        await lease.aclose()
        if released or not job_completed:
            for path in tracked_paths:
                if path.is_file() and not path.is_symlink():
                    path.unlink(missing_ok=True)

def main(argv: list[str] | None = None) -> int:
    """Run one sealed guest job using only fixed managed paths."""
    parser = argparse.ArgumentParser(prog="yinshi-managed-backup")
    parser.add_argument("operation", choices=("create", "restore"))
    parser.add_argument("--job-id", required=True)
    parser.add_argument("--hold-seconds", type=int)
    arguments = parser.parse_args(argv)
    if _JOB_ID_PATTERN.fullmatch(arguments.job_id) is None:
        parser.error("--job-id must be a canonical lowercase UUID")
    maintenance_root = _STATE_ROOT / "maintenance"

def run_job() -> None:
    run_managed_backup_job(
        job_path=maintenance_root / f"{arguments.job_id}.job",
        result_path=maintenance_root / f"{arguments.job_id}.result",
        runner_private_key=load_or_create_runner_noise_keypair(
            _STATE_ROOT / "runner-noise.key"
        ).private_key,
        sqlite_root=_STATE_ROOT / "sqlite",
        files_root=_STATE_ROOT / "files",
        maintenance_root=maintenance_root,
        expected_job_id=arguments.job_id,
    )

```

```

    )

    if arguments.hold_seconds is None:
        run_job()
    else:
        asyncio.run(
            hold_managed_backup_job(
                job_id=arguments.job_id,
                maintenance_root=maintenance_root,
                run_job=run_job,
                timeout_seconds=arguments.hold_seconds,
            )
        )
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

16.3 yinshi/managed_operations_check.py

This operator check combines runtime, backup, recovery, alert, and cleanup status into one release gate.

```

"""Command-line managed runtime operational status check."""

from __future__ import annotations

import argparse
import json
import sqlite3
from collections.abc import Sequence
from datetime import UTC, datetime

from yinshi.services.managed_operational_status import collect_managed_operational_status


def _parse_utc_timestamp(value: str) -> datetime:
    """Parse one explicit UTC timestamp."""
    try:
        timestamp = datetime.fromisoformat(value.replace("Z", "+00:00"))
    except ValueError as exc:
        raise argparse.ArgumentTypeError("timestamp must be ISO 8601") from exc
    if timestamp.tzinfo is None:
        raise argparse.ArgumentTypeError("timestamp must include timezone information")
    return timestamp.astimezone(UTC)

```

```

def _parser() -> argparse.ArgumentParser:
    """Build checker arguments without reading application settings."""
    parser = argparse.ArgumentParser(prog="yinshi-managed-operations-check")
    parser.add_argument("--control-db", required=True)
    parser.add_argument("--backup-stale-seconds", type=int, default=86_400)
    parser.add_argument("--operation-stuck-seconds", type=int, default=3_600)
    parser.add_argument("--now", type=_parse_utc_timestamp)
    return parser

def main(arguments: Sequence[str] | None = None) -> int:
    """Print sanitized JSON and return a monitoring-compatible status."""
    options = _parser().parse_args(arguments)
    now = options.now or datetime.now(UTC)
    database = sqlite3.connect(f"file:{options.control_db}?mode=ro", uri=True)
    database.row_factory = sqlite3.Row
    try:
        status = collect_managed_operational_status(
            database,
            now=now,
            backup_stale_seconds=options.backup_stale_seconds,
            operation_stuck_seconds=options.operation_stuck_seconds,
        )
    finally:
        database.close()
    print(json.dumps(status.to_dict(), separators=(",", ":"), sort_keys=True))
    return 2 if status.critical else 0

if __name__ == "__main__":
    raise SystemExit(main())

```

16.4 yinshi/managed_recovery_drill_controller.py

The drill controller allocates isolated identities, follows staged recovery, records results, and cleans only resources created by the exercise.

```

"""Application-owned execution state for staging recovery drills."""

from __future__ import annotations

import asyncio
import logging
from collections.abc import Awaitable, Callable

```

```

from datetime import UTC, datetime

from yinshi.managed_source_loss_recovery import (
    ManagedSourceLossReceipt,
    ManagedSourceLossResult,
)

logger = logging.getLogger(__name__)

class ManagedRecoveryDrillController:
    """Own one drill task and expose only sanitized aggregate state."""

    def __init__(
        self,
        *,
        run_drill: Callable[[], Awaitable[ManagedSourceLossReceipt]],
    ) -> None:
        if not callable(run_drill):
            raise TypeError("run_drill must be callable")
        self._run_drill = run_drill
        self._task: asyncio.Task[None] | None = None
        self._result: dict[str, object] = {
            "schema_version": 1,
            "status": "not_started",
        }

    async def start(self, *, commit_sha: str) -> dict[str, object]:
        """Start one run unless current work is still active."""
        if len(commit_sha) != 40 or any(
            character not in "0123456789abcdef" for character in commit_sha
        ):
            raise ValueError("commit_sha must be 40 lowercase hexadecimal characters")
        if self._task is not None and not self._task.done():
            raise RuntimeError("managed recovery drill is active")
        checks = self._result.get("checks")
        if (
            isinstance(checks, dict)
            and checks.get("cleanup_verified") is not True
            and self._result.get("status") in {"failed", "passed"}
        ):
            raise RuntimeError("managed recovery cleanup must be retried")
        started_at = datetime.now(UTC).isoformat().replace("+00:00", "Z")
        self._result = {"schema_version": 1, "status": "running"}
        self._task = asyncio.create_task(
            self._execute(commit_sha=commit_sha, started_at=started_at),

```

```

        name="managed-recovery-drill",
    )
    return dict(self._result)

def status(self) -> dict[str, object]:
    """Return a detached copy of current aggregate state."""
    result = dict(self._result)
    checks = result.get("checks")
    if isinstance(checks, dict):
        result["checks"] = dict(checks)
    return result

async def aclose(self) -> None:
    """Cancel unfinished work during application shutdown."""
    task = self._task
    if task is None or task.done():
        return
    task.cancel()
    await asyncio.gather(task, return_exceptions=True)

async def _execute(self, *, commit_sha: str, started_at: str) -> None:
    try:
        receipt = await self._run_drill()
        result = ManagedSourceLossResult(
            commit_sha=commit_sha,
            started_at=started_at,
            checks={
                "archive_version_count": receipt.archive_version_count,
                "cleanup_verified": receipt.cleanup_verified,
                "data_verified": receipt.data_verified,
                "multipart_upload_count": receipt.multipart_upload_count,
                "replacement_authority_verified": (receipt.replacement_authority_verified,
            },
        )
        self._result = result.to_dict()
    except asyncio.CancelledError:
        raise
    except Exception:
        logger.exception("managed_recovery_drill_failed")
        self._result = {
            "schema_version": 1,
            "status": "failed",
            "commit_sha": commit_sha,
            "started_at": started_at,
            "checks": {
                "archive_version_count": 0,

```

```

        "cleanup_verified": False,
        "data_verified": False,
        "multipart_upload_count": 0,
        "replacement_authority_verified": False,
    },
}

```

16.5 yinshi/managed_recovery_live.py

Live recovery restores an archive into a replacement runtime while preserving durable ownership and restart fencing.

```

"""Destructive staging recovery runner with guaranteed cleanup."""

from __future__ import annotations

from typing import Protocol

from yinshi.managed_source_loss_recovery import ManagedSourceLossReceipt

class ManagedRecoveryLiveBoundary(Protocol):
    """Narrow live capabilities required by one destructive drill."""

    async def provision(self) -> None: ...
    async def write_fixtures(self) -> None: ...
    async def backup_with_lost_completion(self) -> None: ...
    async def delete_source(self) -> None: ...
    async def restore(self) -> None: ...
    async def verify(self) -> tuple[int, int, bool, bool]: ...
    async def cleanup(self) -> bool: ...

class ManagedRecoveryLiveRunner:
    """Run one complete source-loss drill and emit aggregate checks only."""

    def __init__(self, *, boundary: ManagedRecoveryLiveBoundary) -> None:
        self._boundary = boundary

    async def run(self) -> ManagedSourceLossReceipt:
        """Execute destructive recovery and require cleanup before success."""
        try:
            await self._boundary.provision()
            await self._boundary.write_fixtures()
            await self._boundary.backup_with_lost_completion()
            await self._boundary.delete_source()

```



```

        await self._boundary.restore()
    (
        archive_version_count,
        multipart_upload_count,
        data_verified,
        replacement_authority_verified,
    ) = await self._boundary.verify()
    finally:
        cleanup_verified = await self._boundary.cleanup()
    return ManagedSourceLossReceipt(
        archive_version_count=archive_version_count,
        cleanup_verified=cleanup_verified,
        data_verified=data_verified,
        multipart_upload_count=multipart_upload_count,
        replacement_authority_verified=replacement_authority_verified,
    )

```

16.6 yinshi/managed_recovery_staging.py

Staging recovery validates archives and data keys away from the source runtime before any authority transfer.

"""Concrete isolated staging boundary for destructive managed recovery drills."""

```

from __future__ import annotations

import asyncio
import hashlib
import shutil
import sqlite3
import tempfile
import uuid
from datetime import UTC, datetime
from pathlib import Path
from typing import Any

from yinshi.db import get_control_db
from yinshi.services.accounts import resolve_or_create_user
from yinshi.services.managed_backups import (
    get_managed_backup_operation,
    list_managed_backup_archives,
)
from yinshi.services.managed_runners import (
    claim_managed_runtime_deletion,
    finalize_managed_runtime_deletion,
    get_managed_runtime_status,

```

```

)
from yinshi.services.managed_sprite_registry import (
    list_managed_sprite_identities,
    remove_managed_sprite_identity,
)

_SQLITE_PATH = "/var/lib/yinshi/sqlite/drill.db"
_SQLITE_FIXTURE_MAX_BYTES = 10 * 1024 * 1024
_TEXT_PATH = "/var/lib/yinshi/files/nested/canary.txt"
_BINARY_PATH = "/var/lib/yinshi/files/canary.bin"
_EMPTY_PATH = "/var/lib/yinshi/files/empty"
_POLL_SECONDS = 2.0
_TIMEOUT_SECONDS = 900.0

def list_retained_managed_recovery_tenants() -> tuple[str, ...]:
    """Return durable internal drill tenant IDs across a process restart."""
    with get_control_db() as database:
        rows = database.execute("""SELECT user_id FROM oauth_identities
                                WHERE provider = 'managed_recovery_drill' ORDER BY user_id""").fetchall()
    return tuple(str(row["user_id"]) for row in rows)

class StagingManagedRecoveryBoundary:
    """Compose existing hosted managers for one isolated internal tenant."""

    def __init__(
        self,
        *,
        runtime_manager: Any,
        backup_manager: Any,
        provider: Any,
        store: Any,
    ) -> None:
        self._runtime_manager = runtime_manager
        self._backup_manager = backup_manager
        self._provider = provider
        self._store = store
        self._tenant: Any | None = None
        self._source_name: str | None = None
        self._archive_id: str | None = None
        self._object_key: str | None = None
        self._fixtures = self._make_fixtures()

    @staticmethod
    def _make_fixtures() -> dict[str, bytes]:

```

```

"""Build valid representative SQLite, text, binary, and empty fixtures."""
with tempfile.TemporaryDirectory(prefix="yinshi-drill-sqlite-") as directory:
    database_path = Path(directory) / "drill.db"
    with sqlite3.connect(database_path) as database:
        database.execute("CREATE TABLE canary (value TEXT NOT NULL)")
        database.execute("INSERT INTO canary (value) VALUES (?)", ("restored",))
        database.commit()
    sqlite_payload = database_path.read_bytes()
    return {
        _SQLITE_PATH: sqlite_payload,
        _TEXT_PATH: b"nested managed recovery canary\n",
        _BINARY_PATH: bytes(range(256)) * 4,
        _EMPTY_PATH: b"",
    }

async def recover_retained_cleanup(self) -> None:
    """Fail closed before new work when an earlier drill still exists."""
    if list_retained_managed_recovery_tenants():
        raise RuntimeError("managed recovery retained cleanup failed")

async def provision(self) -> None:
    """Create one internal tenant and wait for its managed runtime."""
    identity = uuid.uuid4().hex
    self._tenant = await asyncio.to_thread(
        resolve_or_create_user,
        "managed_recovery_drill",
        identity,
        f"managed-recovery-{identity}@invalid.local",
    )
    await self._runtime_manager.provision(self._required_user_id())
    runtime = await self._wait_runtime("ready")
    self._source_name = runtime.sprite_name

async def write_fixtures(self) -> None:
    """Write representative bounded state and verify exact source bytes."""
    sprite_name = self._required_source_name()
    for path, content in self._fixtures.items():
        await self._provider.write_file(
            sprite_name,
            path=path,
            content=content,
            mode="0600",
            mkdir=True,
        )
    await self._verify_fixture_bytes(sprite_name)

```

```

async def backup_with_lost_completion(self) -> None:
    """Create one encrypted backup while losing one successful completion response."""
    user_id = self._required_user_id()
    reservation = self._backup_manager.reserve_create()
    self._archive_id = reservation.archive_id
    self._object_key = reservation.object_key
    self._store.arm_lost_completion_response(
        object_key=reservation.object_key,
        archive_id=reservation.archive_id,
    )
    operation = self._backup_manager.enqueue_create(
        user_id,
        reservation=reservation,
    )
    self._backup_manager.wake()
    await self._wait_operation_complete(operation.job_id)
    archive = next(
        (
            value
            for value in list_managed_backup_archives(user_id)
            if value.id == operation.archive_id
        ),
        None,
    )
    if archive is None or archive.status != "ready" or archive.object_version is None:
        raise RuntimeError("managed recovery backup did not become ready")
    self._object_key = archive.object_key
    inventory = await self._store.inspect_object(object_key=archive.object_key)
    if inventory.version_count != 1 or inventory.multipart_upload_ids:
        raise RuntimeError("managed recovery backup object inventory is invalid")

async def delete_source(self) -> None:
    """Delete the exact source Sprite after a durable ready archive exists."""
    source_name = self._required_source_name()
    await self._provider.delete_sprite(source_name)
    if await self._provider.get_sprite(source_name) is not None:
        raise RuntimeError("managed recovery source Sprite still exists")

async def restore(self) -> None:
    """Claim explicit source loss and wait for replacement publication."""
    operation = self._backup_manager.enqueue_source_loss_restore(
        self._required_user_id(),
        self._required_archive_id(),
    )
    self._backup_manager.wake()
    await self._wait_operation_complete(operation.job_id)

```

```

runtime = await self._wait_runtime("ready")
if runtime.sprite_name == self._required_source_name():
    raise RuntimeError("managed recovery replacement did not change authority")

async def verify(self) -> tuple[int, int, bool, bool]:
    """Verify restored bytes, SQLite integrity, and sole replacement authority."""
    runtime = await self._wait_runtime("ready")
    await self._verify_fixture_bytes(runtime.sprite_name)
    database_payload = await self._provider.read_file(
        runtime.sprite_name,
        path=_SQLITE_PATH,
        max_bytes=_SQLITE_FIXTURE_MAX_BYTES,
    )
    data_verified = await asyncio.to_thread(self._verify_sqlite, database_payload)
    source_absent = await self._provider.get_sprite(self._required_source_name()) is None
    inventory = await self._store.inspect_object(object_key=self._required_object_key())
    authority_verified = source_absent and runtime.sprite_name != self._required_source_name()
    return (
        inventory.version_count,
        len(inventory.multipart_upload_ids),
        data_verified,
        authority_verified,
    )

async def cleanup(self) -> bool:
    """Delete archive, replacement, registry state, and internal tenant data."""
    if self._tenant is None:
        return True
    user_id = self._tenant.user_id
    try:
        if self._object_key is None and self._archive_id is not None:
            archives = list_managed_backup_archives(user_id)
            archive = next((value for value in archives if value.id == self._archive_id), None)
            if archive is not None:
                self._object_key = archive.object_key
        if self._object_key is not None:
            await self._store.purge_object(object_key=self._object_key)
            inventory = await self._store.inspect_object(object_key=self._object_key)
            if (
                inventory.version_count
                or inventory.delete_marker_count
                or inventory.multipart_upload_ids
            ):
                return False
        runtime = get_managed_runtime_status(user_id)
        if runtime is not None:

```

```

        claim = claim_managed_runtime_deletion(user_id, datetime.now(UTC))
        if claim is not None:
            await self._provider.delete_sprite(claim.runtime.sprite_name)
            if await self._provider.get_sprite(claim.runtime.sprite_name) is not None:
                raise RuntimeError("managed recovery replacement still exists")
            if not finalize_managed_runtime_deletion(user_id, claim.runtime.generator):
                raise RuntimeError("managed recovery runtime cleanup was not stored")
        owned_identities = tuple(
            identity
            for identity in list_managed_sprite_identities()
            if identity.user_id == user_id
        )
        for identity in owned_identities:
            if await self._provider.get_sprite(identity.sprite_name) is not None:
                return False
            remove_managed_sprite_identity(identity.sprite_name)
        if any(identity.user_id == user_id for identity in list_managed_sprite_identities()):
            return False
        with get_control_db() as database:
            database.execute("DELETE FROM users WHERE id = ?", (user_id,))
            database.commit()
        shutil.rmtree(self._tenant.data_dir, ignore_errors=True)
        return get_managed_runtime_status(user_id) is None
    except Exception:
        return False

async def _wait_runtime(self, status: str) -> Any:
    deadline = asyncio.get_running_loop().time() + _TIMEOUT_SECONDS
    while True:
        runtime = get_managed_runtime_status(self._required_user_id())
        if runtime is not None and runtime.lifecycle_status == status:
            return runtime
        if runtime is not None and runtime.lifecycle_status == "failed":
            raise RuntimeError("managed recovery runtime failed")
        if asyncio.get_running_loop().time() >= deadline:
            raise TimeoutError("managed recovery runtime timed out")
        await asyncio.sleep(_POLL_SECONDS)

async def _wait_operation_complete(self, job_id: str) -> None:
    deadline = asyncio.get_running_loop().time() + _TIMEOUT_SECONDS
    while True:
        operation = get_managed_backup_operation(self._required_user_id(), job_id)
        if operation is None:
            return
        if operation.status == "failed":
            raise RuntimeError("managed recovery operation failed")

```

```

        if asyncio.get_running_loop().time() >= deadline:
            raise TimeoutError("managed recovery operation timed out")
        await asyncio.sleep(_POLL_SECONDS)

    async def _verify_fixture_bytes(self, sprite_name: str) -> None:
        for path, expected in self._fixtures.items():
            actual = await self._provider.read_file(
                sprite_name,
                path=path,
                max_bytes=max(len(expected), 1) + 1024,
            )
            if not hashlib.sha256(actual).digest() == hashlib.sha256(expected).digest():
                raise RuntimeError("managed recovery fixture digest did not match")

    @staticmethod
    def _verify_sqlite(payload: bytes) -> bool:
        with tempfile.TemporaryDirectory(prefix="yinshi-drill-verify-") as directory:
            path = Path(directory) / "drill.db"
            path.write_bytes(payload)
            with sqlite3.connect(path) as database:
                integrity = database.execute("PRAGMA integrity_check").fetchone()
                canary = database.execute("SELECT value FROM canary").fetchone()
            return bool(integrity == ("ok",) and canary == ("restored",))

    def _required_user_id(self) -> str:
        if self._tenant is None:
            raise RuntimeError("managed recovery tenant is unavailable")
        return str(self._tenant.user_id)

    def _required_source_name(self) -> str:
        if self._source_name is None:
            raise RuntimeError("managed recovery source is unavailable")
        return self._source_name

    def _required_archive_id(self) -> str:
        if self._archive_id is None:
            raise RuntimeError("managed recovery archive is unavailable")
        return self._archive_id

    def _required_object_key(self) -> str:
        if self._object_key is None:
            raise RuntimeError("managed recovery object is unavailable")
        return self._object_key

```

16.7 yinshi/managed_source_loss_recovery.py

Source-loss recovery claims replacement authority atomically and rejects archives that cannot restore encrypted persistent state.

```
"""Typed staging harness for destructive managed source-loss drills."""

from __future__ import annotations

import json
import os
from collections.abc import Mapping
from dataclasses import dataclass
from datetime import UTC, datetime
from typing import Any, Protocol

_REQUIRED_ENVIRONMENT_NAMES = (
    "STAGING_CONTROL_URL",
    "STAGING_OPERATOR_TOKEN",
    "STAGING_SPRITES_API_TOKEN",
    "STAGING_SPRITES_NAME_KEY",
    "STAGING_BACKUP_BUCKET",
    "STAGING_BACKUP_ENDPOINT_URL",
    "STAGING_BACKUP_REGION",
    "STAGING_BACKUP_ACCESS_KEY_ID",
    "STAGING_BACKUP_SECRET_ACCESS_KEY",
    "STAGING_BACKUP_ENCRYPTION_KEY",
)

class DrillConfigurationError(RuntimeError):
    """Raised when a staging drill cannot safely start."""

_COUNT_CHECK_NAMES = ("archive_version_count", "multipart_upload_count")
_BOOLEAN_CHECK_NAMES = (
    "cleanup_verified",
    "data_verified",
    "replacement_authority_verified",
)
_CHECK_NAMES = _COUNT_CHECK_NAMES + _BOOLEAN_CHECK_NAMES

@dataclass(frozen=True, slots=True)
class ManagedRecoveryControl:
    """Control-plane capability used by the staging drill."""
```



```

url: str
operator_token: str

@dataclass(frozen=True, slots=True)
class ManagedRecoveryProvider:
    """Sprite provider capability used by the staging drill."""

    api_token: str
    name_key: str

@dataclass(frozen=True, slots=True)
class ManagedRecoveryStorage:
    """Versioned backup-store capability used by the staging drill."""

    bucket: str
    endpoint_url: str
    region: str
    access_key_id: str
    secret_access_key: str
    encryption_key: str

@dataclass(frozen=True, slots=True)
class ManagedSourceLossConfiguration:
    """Complete typed capabilities kept outside drill output."""

    control: ManagedRecoveryControl
    provider: ManagedRecoveryProvider
    storage: ManagedRecoveryStorage

@dataclass(frozen=True, slots=True)
class ManagedSourceLossReceipt:
    """Typed aggregate checks returned after destructive cleanup."""

    archive_version_count: int
    cleanup_verified: bool
    data_verified: bool
    multipart_upload_count: int
    replacement_authority_verified: bool

    def __post_init__(self) -> None:
        if type(self.archive_version_count) is not int or self.archive_version_count < 0:

```

```

        raise ValueError("archive_version_count must be a nonnegative integer")
    if type(self.multipart_upload_count) is not int or self.multipart_upload_count < 0:
        raise ValueError("multipart_upload_count must be a nonnegative integer")
    for name in (
        "cleanup_verified",
        "data_verified",
        "replacement_authority_verified",
    ):
        if type(getattr(self, name)) is not bool:
            raise TypeError(f"{name} must be Boolean")

class ManagedSourceLossBoundary(Protocol):
    """Run provider-specific destructive drill actions."""

    def run(self, configuration: ManagedSourceLossConfiguration) -> ManagedSourceLossReceipt:
        """Return required verification fields after cleanup."""
        ...

@dataclass(frozen=True, slots=True)
class ManagedSourceLossResult:
    """Sanitized drill result suitable for a retained CI artifact."""

    commit_sha: str
    started_at: str
    checks: dict[str, bool | int]

    def to_dict(self) -> dict[str, Any]:
        """Return only allow-listed, non-sensitive drill fields."""
        passed = (
            self.checks["archive_version_count"] == 1
            and self.checks["multipart_upload_count"] == 0
            and self.checks["cleanup_verified"] is True
            and self.checks["data_verified"] is True
            and self.checks["replacement_authority_verified"] is True
        )
        return {
            "schema_version": 1,
            "status": "passed" if passed else "failed",
            "commit_sha": self.commit_sha,
            "started_at": self.started_at,
            "checks": dict(self.checks),
        }

```

```

def load_drill_configuration(
    environment: Mapping[str, str],
) -> ManagedSourceLossConfiguration:
    """Copy required staging settings for boundary use."""
    values: dict[str, str] = {}
    for name in _REQUIRED_ENVIRONMENT_NAMES:
        value = environment.get(name, "").strip()
        if not value:
            raise DrillConfigurationError(f"missing required staging setting: {name}")
        values[name] = value
    return ManagedSourceLossConfiguration(
        control=ManagedRecoveryControl(
            url=values["STAGING_CONTROL_URL"],
            operator_token=values["STAGING_OPERATOR_TOKEN"],
        ),
        provider=ManagedRecoveryProvider(
            api_token=values["STAGING_SPRITES_API_TOKEN"],
            name_key=values["STAGING_SPRITES_NAME_KEY"],
        ),
        storage=ManagedRecoveryStorage(
            bucket=values["STAGING_BACKUP_BUCKET"],
            endpoint_url=values["STAGING_BACKUP_ENDPOINT_URL"],
            region=values["STAGING_BACKUP_REGION"],
            access_key_id=values["STAGING_BACKUP_ACCESS_KEY_ID"],
            secret_access_key=values["STAGING_BACKUP_SECRET_ACCESS_KEY"],
            encryption_key=values["STAGING_BACKUP_ENCRYPTION_KEY"],
        ),
    )

def sanitized_configuration_status(
    configuration: ManagedSourceLossConfiguration,
) -> dict[str, bool | int | str]:
    """Describe workflow readiness without exposing configured values."""
    if not configuration.control.url or not configuration.storage.bucket:
        raise ValueError("configuration is incomplete")
    return {
        "schema_version": 1,
        "status": "pending_live_staging_integration",
        "required_settings_configured": True,
        "live_staging_execution": False,
    }

def configuration_check_main() -> int:
    """Validate workflow settings and report that live work remains pending."""

```

```

configuration = load_drill_configuration(os.environ)
payload = sanitized_configuration_status(configuration)
print(json.dumps(payload, separators=(",", ":"), sort_keys=True))
return 3

```

```

class ManagedSourceLossDrill:
    """Emit only allow-listed recovery results from a drill boundary."""

    def __init__(self, boundary: ManagedSourceLossBoundary) -> None:
        self._boundary = boundary

    def run(
        self,
        configuration: ManagedSourceLossConfiguration,
        *,
        commit_sha: str,
    ) -> ManagedSourceLossResult:
        """Run staging boundary and retain only aggregate checks."""
        receipt = self._boundary.run(configuration)
        checks: dict[str, bool | int] = {}
        for name in _CHECK_NAMES:
            value = getattr(receipt, name)
            if name in _COUNT_CHECK_NAMES:
                if type(value) is not int or value < 0:
                    raise RuntimeError(f"drill boundary returned invalid check: {name}")
            elif type(value) is not bool:
                raise RuntimeError(f"drill boundary returned invalid check: {name}")
            checks[name] = value
        return ManagedSourceLossResult(
            commit_sha=commit_sha,
            started_at=datetime.now(UTC).isoformat().replace("+00:00", "Z"),
            checks=checks,
        )

```

16.8 yinshi/services/managed_backup_crypto.py

Backup cryptography derives archive keys from portable data-protection material and authenticates each encrypted payload.

```

"""Seal managed-backup jobs to one persisted runner identity."""

from __future__ import annotations

import base64
import binascii

```

```

import json
import os
from typing import Any

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric.x25519 import (
    X25519PrivateKey,
    X25519PublicKey,
)
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

_ENVELOPE_MAGIC = b"YINSHI-MANAGED-JOB-V1\n"
_X25519_KEY_BYTES = 32
_NONCE_BYTES = 12
_ARCHIVE_KEY_ENVELOPE = b"YINSHI-MANAGED-ARCHIVE-KEY-V1\n"

def _decode_public_key(value: str) -> bytes:
    """Decode one canonical unpadded X25519 public key."""
    if not isinstance(value, str) or not value:
        raise ValueError("runner_public_key must not be empty")
    try:
        decoded = base64.b64decode(
            value + "=" * (-len(value) % 4),
            altchars=b"._",
            validate=True,
        )
    except (binascii.Error, ValueError) as error:
        raise ValueError("runner_public_key is not canonical base64url") from error
    canonical = base64.urlsafe_b64encode(decoded).rstrip(b"=").decode("ascii")
    if len(decoded) != _X25519_KEY_BYTES or canonical != value:
        raise ValueError("runner_public_key must contain 32 canonical bytes")
    return decoded

def _job_aad(job_id: str) -> bytes:
    """Return authenticated context for one exact maintenance job."""
    if not isinstance(job_id, str) or not job_id or len(job_id) > 128:
        raise ValueError("job_id must be bounded non-empty text")
    return _ENVELOPE_MAGIC + job_id.encode("utf-8")

def _job_key(shared_secret: bytes, ephemeral_public_key: bytes, runner_public_key: bytes) ->
    """Derive one job key from both envelope identities."""
    return HKDF(

```

```

        algorithm=hashes.SHA256(),
        length=32,
        salt=b"yinshi-managed-job-seal-v1",
        info=ephemeral_public_key + runner_public_key,
    ).derive(shared_secret)

def _archive_key_aad(user_id: str, archive_id: str, key_id: str) -> bytes:
    """Build exact authenticated context for one stored archive key."""
    values = (user_id, archive_id, key_id)
    if any(not isinstance(value, str) or not value or len(value) > 256 for value in values):
        raise ValueError("archive key context must contain bounded non-empty text")
    return _ARCHIVE_KEY_ENVELOPE + "\x1f".join(values).encode("utf-8")

def wrap_managed_archive_key(
    archive_key: bytes,
    *,
    user_id: str,
    archive_id: str,
    key_id: str,
    wrapping_key: bytes,
) -> bytes:
    """Wrap one random archive key under server-controlled key material."""
    if not isinstance(archive_key, bytes) or len(archive_key) != 32:
        raise ValueError("archive_key must contain exactly 32 bytes")
    if not isinstance(wrapping_key, bytes) or len(wrapping_key) < 32:
        raise ValueError("wrapping_key must contain at least 32 bytes")
    aad = _archive_key_aad(user_id, archive_id, key_id)
    derived_key = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=b"yinshi-managed-archive-key-wrap-v1",
        info=key_id.encode("utf-8"),
    ).derive(wrapping_key)
    nonce = os.urandom(_NONCE_BYTES)
    ciphertext = AESGCM(derived_key).encrypt(nonce, archive_key, aad)
    payload = {
        "ciphertext": base64.urlsafe_b64encode(ciphertext).decode("ascii"),
        "key_id": key_id,
        "nonce": base64.urlsafe_b64encode(nonce).decode("ascii"),
        "version": 1,
    }
    return _ARCHIVE_KEY_ENVELOPE + json.dumps(
        payload, separators=(",", ":"), sort_keys=True
    ).encode("utf-8")

```

```

def unwrap_managed_archive_key(
    envelope: bytes,
    *,
    user_id: str,
    archive_id: str,
    keyring: dict[str, bytes],
) -> bytes:
    """Unwrap one archive key after checking every stored context field."""
    if not isinstance(envelope, bytes) or not envelope.startswith(_ARCHIVE_KEY_ENVELOPE):
        raise ValueError("managed archive key could not be unwrapped")
    try:
        payload = json.loads(envelope[len(_ARCHIVE_KEY_ENVELOPE) :].decode("utf-8"))
        if not isinstance(payload, dict) or set(payload) != {
            "ciphertext",
            "key_id",
            "nonce",
            "version",
        }:
            raise ValueError
        key_id = payload["key_id"]
        if payload["version"] != 1 or not isinstance(key_id, str) or key_id not in keyring:
            raise ValueError
        wrapping_key = keyring[key_id]
        if not isinstance(wrapping_key, bytes) or len(wrapping_key) < 32:
            raise ValueError
        derived_key = HKDF(
            algorithm=hashes.SHA256(),
            length=32,
            salt=b"yinshi-managed-archive-key-wrap-v1",
            info=key_id.encode("utf-8"),
        ).derive(wrapping_key)
        nonce = base64.b64decode(payload["nonce"], altchars=b"_" , validate=True)
        ciphertext = base64.b64decode(payload["ciphertext"], altchars=b"_" , validate=True)
        archive_key = AESGCM(derived_key).decrypt(
            nonce,
            ciphertext,
            _archive_key_aad(user_id, archive_id, key_id),
        )
    except Exception:
        raise ValueError("managed archive key could not be unwrapped") from None
    if len(archive_key) != 32:
        raise ValueError("managed archive key could not be unwrapped")
    return archive_key

```

```

def seal_managed_backup_job(
    payload: dict[str, Any],
    *,
    runner_public_key: str,
    job_id: str,
) -> bytes:
    """Encrypt one JSON job so only the intended runner can open it."""
    if not isinstance(payload, dict):
        raise TypeError("payload must be a dictionary")
    try:
        plaintext = json.dumps(payload, separators=(",", ":"), sort_keys=True).encode("utf-8")
    except (TypeError, ValueError) as error:
        raise ValueError("payload must be JSON-serializable") from error
    runner_public_bytes = _decode_public_key(runner_public_key)
    runner_public = X25519PublicKey.from_public_bytes(runner_public_bytes)
    ephemeral_private = X25519PrivateKey.generate()
    ephemeral_public = ephemeral_private.public_key().public_bytes_raw()
    shared_secret = ephemeral_private.exchange(runner_public)
    nonce = os.urandom(_NONCE_BYTES)
    ciphertext = AESGCM(_job_key(shared_secret, ephemeral_public, runner_public_bytes)).encrypt(
        nonce, plaintext, _job_aad(job_id)
    )
    return _ENVELOPE_MAGIC + ephemeral_public + nonce + ciphertext


def open_managed_backup_job(
    envelope: bytes,
    *,
    runner_private_key: bytes,
    expected_job_id: str,
) -> dict[str, Any]:
    """Open one identity-bound job and return its strict JSON object."""
    if not isinstance(envelope, bytes) or not envelope.startswith(_ENVELOPE_MAGIC):
        raise ValueError("managed backup job could not be opened")
    if not isinstance(runner_private_key, bytes) or len(runner_private_key) != 32:
        raise ValueError("runner_private_key must contain exactly 32 bytes")
    minimum = len(_ENVELOPE_MAGIC) + _X25519_KEY_BYTES + _NONCE_BYTES + 16
    if len(envelope) <= minimum:
        raise ValueError("managed backup job could not be opened")
    offset = len(_ENVELOPE_MAGIC)
    ephemeral_public_bytes = envelope[offset : offset + _X25519_KEY_BYTES]
    offset += _X25519_KEY_BYTES
    nonce = envelope[offset : offset + _NONCE_BYTES]
    ciphertext = envelope[offset + _NONCE_BYTES :]
    try:

```



```

runner_private = X25519PrivateKey.from_private_bytes(runner_private_key)
runner_public_bytes = runner_private.public_key().public_bytes_raw()
shared_secret = runner_private.exchange(
    X25519PublicKey.from_public_bytes(ephemeral_public_bytes)
)
plaintext = AESGCM(
    _job_key(shared_secret, ephemeral_public_bytes, runner_public_bytes)
).decrypt(nonce, ciphertext, _job_aad(expected_job_id))
payload = json.loads(plaintext.decode("utf-8"))
except Exception:
    raise ValueError("managed backup job could not be opened") from None
if not isinstance(payload, dict):
    raise ValueError("managed backup job must contain a JSON object")
return payload

```

16.9 yinshi/services/managed_backup_manager.py

The backup manager coordinates quiescence, upload, durable publication, maintenance cleanup, restart, and interrupted-operation recovery.

"""Lifecycle owner for durable managed backup work."""

```

from __future__ import annotations

import asyncio
import base64
import hashlib
import json
import logging
import os
import stat
import tempfile
import time
import uuid
from collections.abc import Awaitable, Callable
from contextlib import suppress
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Any, NamedTuple

from yinshi.services.managed_backup_crypto import (
    seal_managed_backup_job,
    unwrap_managed_archive_key,
    wrap_managed_archive_key,
)
from yinshi.services.managed_backup_store import (

```

```

        PendingManagedBackupUploads,
        StoredManagedBackup,
    )
    from yinshi.services.managed_backups import (
        ManagedBackupArchive,
        ManagedBackupCreationClaim,
        ManagedBackupOperation,
        claim_due_managed_backup_operation,
        clear_managed_backup_candidate,
        complete_managed_backup_creation,
        complete_managed_backup_deletion,
        get_managed_backup_archive,
        list_managed_backup_retention_candidates,
        record_managed_backup_candidate,
        record_managed_backup_upload,
        record_managed_backup_upload_intent,
        renew_managed_backup_operation_lease,
        start_managed_backup_creation,
        start_managed_source_loss_restore,
    )
    from yinshi.services.managed_operation_failures import fail_managed_backup_operation
    from yinshi.services.managed_runners import (
        ManagedRuntimeStatus,
        activate_managed_restore_candidate,
        get_managed_runtime_status,
        managed_sprite_name,
    )
    from yinshi.services.runners import (
        get_managed_runner_for_user,
        revoke_managed_restore_runner_for_job,
    )
    from yinshi.services.sprites import SpritesProviderError

    _MAINTENANCE_ROOT = "/var/lib/yinshi/maintenance"
    _RESULT_BYTES_MAX = 4096

    logger = logging.getLogger(__name__)

    def _prepare_staging_root(staging_root: Path | None) -> Path | None:
        """Create and validate one owner-only local ciphertext staging directory."""
        if staging_root is None:
            return None
        try:
            staging_root.mkdir(mode=0o700, parents=True, exist_ok=True)
            metadata = staging_root.lstat()

```

```

except OSError:
    raise ValueError("managed backup staging root is invalid") from None
if not stat.S_ISDIR(metadata.st_mode):
    raise ValueError("managed backup staging root is invalid")
if hasattr(os, "geteuid") and metadata.st_uid != os.geteuid():
    raise ValueError("managed backup staging root is invalid")
try:
    staging_root.chmod(0o700)
except OSError:
    raise ValueError("managed backup staging root is invalid") from None
return staging_root

class ManagedBackupReservation(NamedTuple):
    """Non-runnable exact backup identity reserved by one manager call."""

    archive_id: str
    job_id: str
    object_key: str

class ManagedBackupManager:
    """Own managed backup startup and shutdown outside request handlers."""

    def __init__(
        self,
        *,
        reconcile_once: Callable[[], Awaitable[bool]] | None = None,
        interval_seconds: float = 5.0,
        start_creation: Callable[..., Any] = start_managed_backup_creation,
        start_restore: Callable[..., Any] | None = None,
        start_source_loss_restore: Callable[..., Any] | None = start_managed_source_loss_restore,
        start_deletion: Callable[..., Any] | None = None,
        get_runtime: Callable[[str], ManagedRuntimeStatus | None] = get_managed_runtime_status,
        get_runner: Callable[[str], dict[str, Any] | None] = get_managed_runner_for_user,
        wrapping_key: bytes | None = None,
        key_id: str = "backup-v1",
        object_prefix: str = "yinshi-managed-v1",
        now: Callable[[], datetime] | None = None,
        new_id: Callable[[], str] | None = None,
        provider: Any | None = None,
        store: Any | None = None,
        relay: Any | None = None,
        worker_id: str = "managed-backup-worker",
        claim_operation: Callable[
            ..., ManagedBackupOperation | None
        ]
    )

```

```

] = claim_due_managed_backup_operation,
renew_lease: Callable[..., bool] = renew_managed_backup_operation_lease,
fail_operation: Callable[..., bool] = fail_managed_backup_operation,
lease_renew_interval_seconds: float = 60.0,
get_archive: Callable[[str, str], ManagedBackupArchive | None] = get_managed_backup,
unwrap_key: Callable[..., bytes] = unwrap_managed_archive_key,
record_upload_intent: Callable[..., bool] = record_managed_backup_upload_intent,
record_upload: Callable[..., bool] = record_managed_backup_upload,
complete_creation: Callable[..., bool] = complete_managed_backup_creation,
complete_deletion: Callable[..., bool] = complete_managed_backup_deletion,
runtime_service: Any | None = None,
restore_name_prefix: str = "managed-restore",
restore_name_key: str | None = None,
record_candidate: Callable[..., bool] = record_managed_backup_candidate,
clear_candidate: Callable[..., bool] = clear_managed_backup_candidate,
activate_candidate: Callable[..., bool] = activate_managed_restore_candidate,
complete_restore: Callable[..., bool] | None = None,
revoke_restore_runner: Callable[[str, str], bool] = (revoke_managed_restore_runner_1,
coordinate_restore: (
    Callable[[ManagedBackupOperation, ManagedBackupArchive], Awaitable[None]] | None
) = None,
new_lease_token: Callable[[], str] | None = None,
staging_root: Path | None = None,
retention_days: int = 30,
retention_batch_size: int = 20,
list_retention: Callable[..., tuple[ManagedBackupArchive, ...]] = (
    list_managed_backup_retention_candidates
),
enqueue_retention: Callable[[str, str], Any] | None = None,
sleep: Callable[[float], Awaitable[None]] = asyncio.sleep,
monotonic: Callable[[], float] = time.monotonic,
create_result_timeout_seconds: float = 240.0,
) -> None:
    if interval_seconds <= 0:
        raise ValueError("interval_seconds must be positive")
    if create_result_timeout_seconds <= 0:
        raise ValueError("create_result_timeout_seconds must be positive")
    if wrapping_key is not None and len(wrapping_key) != 32:
        raise ValueError("wrapping_key must contain exactly 32 bytes")
    if not key_id or not object_prefix:
        raise ValueError("managed backup key and object prefix are required")
    self._reconcile_once = reconcile_once or self.run_once
    self._interval_seconds = interval_seconds
    self._start_creation = start_creation
    self._start_restore = start_restore
    self._start_source_loss_restore = start_source_loss_restore

```

```

self._start_deletion = start_deletion
self._get_runtime = get_runtime
self._get_runner = get_runner
self._wrapping_key = wrapping_key
self._key_id = key_id
self._object_prefix = object_prefix.rstrip("/")
self._now = now or (lambda: datetime.now(timezone.utc))
self._new_id = new_id or (lambda: str(uuid.uuid4()))
self._provider = provider
self._sleep = sleep
self._monotonic = monotonic
self._create_result_timeout_seconds = create_result_timeout_seconds
self._store = store
self._relay = relay
self._worker_id = worker_id
self._claim_operation = claim_operation
if lease_renew_interval_seconds <= 0:
    raise ValueError("lease_renew_interval_seconds must be positive")
self._renew_lease = renew_lease
self._fail_operation = fail_operation
self._lease_renew_interval_seconds = lease_renew_interval_seconds
self._get_archive = get_archive
self._unwrap_key = unwrap_key
self._record_upload_intent = record_upload_intent
self._record_upload = record_upload
self._complete_creation = complete_creation
self._complete_deletion = complete_deletion
self._runtime_service = runtime_service
self._restore_name_prefix = restore_name_prefix
self._restore_name_key = restore_name_key
self._record_candidate = record_candidate
self._clear_candidate = clear_candidate
self._activate_candidate = activate_candidate
self._complete_restore = complete_restore
self._revoke_restore_runner = revoke_restore_runner
self._coordinate_restore_job = coordinate_restore or self._coordinate_restore
self._new_lease_token = new_lease_token or (lambda: uuid.uuid4().hex)
if not 1 <= retention_days <= 3650 or not 1 <= retention_batch_size <= 100:
    raise ValueError("managed backup retention bounds are invalid")
self._staging_root = _prepare_staging_root(staging_root)
self._retention_days = retention_days
self._retention_batch_size = retention_batch_size
self._list_retention = list_retention
self._enqueue_retention = enqueue_retention or self.enqueue_delete
self._next_retention_at: datetime | None = None
self._wake_event = asyncio.Event()

```

```

self._wake_generation = 0
self._handled_wake_generation = 0
self._stop_event = asyncio.Event()
self._task: asyncio.Task[None] | None = None

def reserve_create(self) -> ManagedBackupReservation:
    """Reserve exact identifiers before publishing a runnable operation."""
    archive_id = self._new_id()
    return ManagedBackupReservation(
        archive_id=archive_id,
        job_id=self._new_id(),
        object_key=f"{self._object_prefix}/{archive_id}.enc",
    )

def enqueue_create(
    self,
    user_id: str,
    *,
    reservation: ManagedBackupReservation | None = None,
) -> ManagedBackupOperation:
    """Persist one server-authorized encrypted backup request."""
    if self._wrapping_key is None:
        raise ValueError("managed backups are unavailable")
    runtime = self._get_runtime(user_id)
    runner = self._get_runner(user_id)
    if (
        runtime is None
        or runtime.lifecycle_status != "ready"
        or runner is None
        or runner.get("id") != runtime.runner_id
        or runner.get("noise_key_confirmed") is not True
    ):
        raise ValueError("managed runtime is not ready for backup")
    reserved = reservation or self.reserve_create()
    archive_id = reserved.archive_id
    job_id = reserved.job_id
    archive_key = os.urandom(32)
    owner_digest = hashlib.sha256(user_id.encode("utf-8")).hexdigest()
    wrapped_key = wrap_managed_archive_key(
        archive_key,
        user_id=user_id,
        archive_id=archive_id,
        key_id=self._key_id,
        wrapping_key=self._wrapping_key,
    )
    claim = self._start_creation(

```

```

        user_id,
        runtime_generation=runtime.generation,
        archive_id=archive_id,
        job_id=job_id,
        object_key=reserved.object_key,
        wrapped_key=wrapped_key,
        key_id=self._key_id,
        owner_digest=owner_digest,
        now=self._now(),
    )
    if isinstance(claim, ManagedBackupCreationClaim):
        return claim.operation
    if isinstance(claim, ManagedBackupOperation):
        return claim
    return ManagedBackupOperation(
        user_id=user_id,
        job_id=job_id,
        archive_id=archive_id,
        operation="create",
        status="running",
        runtime_generation=runtime.generation,
        started_at=self._now().isoformat(),
        updated_at=self._now().isoformat(),
        last_error=None,
    )

def enqueue_restore(self, user_id: str, archive_id: str) -> ManagedBackupOperation:
    """Persist one tenant-owned replacement restore request."""
    runtime = self._get_runtime(user_id)
    archive = self._get_archive(user_id, archive_id)
    if archive is None:
        raise LookupError("managed backup archive was not found")
    if (
        runtime is None
        or runtime.lifecycle_status != "ready"
        or archive.status != "ready"
        or archive.object_version is None
        or self._start_restore is None
    ):
        raise ValueError("managed backup archive is not restorable")
    job_id = self._new_id()
    claim = self._start_restore(
        user_id,
        archive_id=archive_id,
        runtime_generation=runtime.generation,
        job_id=job_id,

```

```

        now=self._now(),
    )
    if isinstance(claim, ManagedBackupCreationClaim):
        return claim.operation
    if isinstance(claim, ManagedBackupOperation):
        return claim
    return ManagedBackupOperation(
        user_id=user_id,
        job_id=job_id,
        archive_id=archive_id,
        operation="restore",
        status="running",
        runtime_generation=runtime.generation,
        started_at=self._now().isoformat(),
        updated_at=self._now().isoformat(),
        last_error=None,
    )

def enqueue_source_loss_restore(
    self,
    user_id: str,
    archive_id: str,
) -> ManagedBackupOperation:
    """Persist one explicitly destructive source-loss restore request."""
    runtime = self._get_runtime(user_id)
    archive = self._get_archive(user_id, archive_id)
    if archive is None:
        raise LookupError("managed backup archive was not found")
    if (
        runtime is None
        or runtime.lifecycle_status != "ready"
        or archive.status != "ready"
        or archive.object_version is None
        or self._start_source_loss_restore is None
    ):
        raise ValueError("managed backup archive is not restorable after source loss")
    job_id = self._new_id()
    claim = self._start_source_loss_restore(
        user_id,
        archive_id=archive_id,
        runtime_generation=runtime.generation,
        job_id=job_id,
        now=self._now(),
    )
    if isinstance(claim, ManagedBackupCreationClaim):
        return claim.operation

```



```

        if isinstance(claim, ManagedBackupOperation):
            return claim
        raise RuntimeError("managed source-loss restore claim is invalid")

def enqueue_delete(self, user_id: str, archive_id: str) -> ManagedBackupOperation:
    """Persist one tenant-owned exact-version deletion request."""
    archive = self._get_archive(user_id, archive_id)
    if archive is None:
        raise LookupError("managed backup archive was not found")
    if (
        archive.status != "ready"
        or archive.object_version is None
        or self._start_deletion is None
    ):
        raise ValueError("managed backup archive is not deletable")
    runtime = self._get_runtime(user_id)
    deletion_generation = (
        runtime.generation if runtime is not None else archive.runtime_generation
    )
    job_id = self._new_id()
    claim = self._start_deletion(
        user_id,
        archive_id=archive_id,
        runtime_generation=deletion_generation,
        job_id=job_id,
        now=self._now(),
    )
    if isinstance(claim, ManagedBackupCreationClaim):
        return claim.operation
    if isinstance(claim, ManagedBackupOperation):
        return claim
    return ManagedBackupOperation(
        user_id=user_id,
        job_id=job_id,
        archive_id=archive_id,
        operation="delete",
        status="running",
        runtime_generation=deletion_generation,
        started_at=self._now().isoformat(),
        updated_at=self._now().isoformat(),
        last_error=None,
    )

def schedule_retention(self) -> int:
    """Queue bounded old archives no more than once per minute."""
    now = self._now()

```

```

if self._next_retention_at is not None and now < self._next_retention_at:
    return 0
self._next_retention_at = now + timedelta(minutes=1)
cutoff = (now - timedelta(days=self._retention_days)).isoformat()
candidates = self._list_retention(
    cutoff=cutoff,
    limit=self._retention_batch_size,
)
bounded_candidates = candidates[: self._retention_batch_size]
for archive in bounded_candidates:
    self._enqueue_retention(archive.user_id, archive.id)
return len(bounded_candidates)

async def run_once(self) -> bool:
    """Claim and complete one exact managed backup creation."""
    if self._provider is None or self._store is None or self._relay is None:
        return False
    now = self._now()
    operation = self._claim_operation(
        worker_id=self._worker_id,
        lease_token=self._new_lease_token(),
        now=now,
        lease_expires_at=now + timedelta(minutes=15),
    )
    if operation is None:
        return False
    work_task = asyncio.create_task(
        self._coordinate_operation(operation),
        name="managed-backup-operation",
    )
    lease_task = asyncio.create_task(
        self._renew_operation_lease(operation),
        name="managed-backup-lease-renewal",
    )
    try:
        done, _pending = await asyncio.wait(
            (work_task, lease_task),
            return_when=asyncio.FIRST_COMPLETED,
        )
        if lease_task in done:
            work_task.cancel()
            await asyncio.gather(work_task, return_exceptions=True)
            lease_task.result()
            raise RuntimeError("managed backup lease task ended unexpectedly")
        lease_task.cancel()
        await asyncio.gather(lease_task, return_exceptions=True)

```

```

        return work_task.result()
    finally:
        work_task.cancel()
        lease_task.cancel()
        await asyncio.gather(work_task, lease_task, return_exceptions=True)

async def _coordinate_operation(self, operation: ManagedBackupOperation) -> bool:
    """Dispatch one leased operation while the caller supervises ownership."""
    archive = self._get_archive(operation.user_id, operation.archive_id)
    if operation.operation == "delete":
        try:
            if archive is not None:
                await self._coordinate_delete(operation, archive)
        except Exception:
            with suppress(Exception):
                self._persist_operation_failure(operation, "deletion_failed")
            raise
        return True
    if operation.operation == "restore":
        try:
            if archive is not None and self._coordinate_restore_job is not None:
                await self._coordinate_restore_job(operation, archive)
        except Exception:
            with suppress(Exception):
                self._persist_operation_failure(operation, "restore_failed")
            raise
        return True
    if operation.operation != "create":
        return True
    runtime = self._get_runtime(operation.user_id)
    runner = self._get_runner(operation.user_id)
    assert self._provider is not None
    assert self._store is not None
    assert self._relay is not None
    if archive is None or runtime is None or runner is None:
        return True
    await self._coordinate_create(operation, archive, runtime, runner)
    return True

def _persist_operation_failure(
    self,
    operation: ManagedBackupOperation,
    failure_class: str,
) -> None:
    """Persist a sanitized semantic class at the dispatch boundary."""
    self._fail_operation(

```

```

        job_id=operation.job_id,
        runtime_generation=operation.runtime_generation,
        lease_owner=operation.lease_owner,
        lease_token=operation.lease_token,
        failure_class=failure_class,
        error_code=f"{operation.operation}_coordination_failed",
        now=self._now(),
    )

    async def _renew_operation_lease(self, operation: ManagedBackupOperation) -> None:
        """Keep exact job ownership alive while external work remains active."""
        if operation.lease_token is None or operation.lease_owner is None:
            raise RuntimeError("managed backup operation lease is incomplete")
        while True:
            await asyncio.sleep(self._lease_renew_interval_seconds)
            now = self._now()
            renewed = await asyncio.to_thread(
                self._renew_lease,
                job_id=operation.job_id,
                worker_id=operation.lease_owner,
                lease_token=operation.lease_token,
                runtime_generation=operation.runtime_generation,
                now=now,
                lease_expires_at=now + timedelta(minutes=15),
            )
            if not renewed:
                raise RuntimeError("managed backup operation lease was lost")

    async def _coordinate_restore(
        self,
        operation: ManagedBackupOperation,
        archive: ManagedBackupArchive,
    ) -> None:
        """Restore one exact archive into a private replacement before cutover."""
        assert self._provider is not None
        assert self._store is not None
        assert self._relay is not None
        if (
            self._runtime_service is None
            or self._restore_name_key is None
            or self._wrapping_key is None
            or operation.lease_token is None
            or operation.source_runner_id is None
            or operation.source_sprite_id is None
            or archive.object_version is None
            or archive.size_bytes is None

```

```

        or archive.sha256 is None
    ):
        return
    if operation.phase == "activated":
        if operation.candidate_sprite_id is None:
            raise RuntimeError("activated restore candidate is missing")
        await self._finish_activated_restore(
            operation,
            operation.candidate_sprite_id,
        )
        return
    candidate_name = operation.candidate_sprite_id or managed_sprite_name(
        f"{operation.user_id}:{operation.job_id}",
        prefix=self._restore_name_prefix,
        secret_key=self._restore_name_key,
    )
    try:
        candidate = await self._runtime_service.provision_restore_candidate(
            operation.user_id,
            job_id=operation.job_id,
            candidate_sprite_name=candidate_name,
            candidate_runner_id=operation.candidate_runner_id,
        )
    except BaseException:
        await self._recover_failed_restore(operation, candidate_name)
        raise
    if operation.candidate_sprite_id is None and not self._record_candidate(
        job_id=operation.job_id,
        lease_token=operation.lease_token,
        runtime_generation=operation.runtime_generation,
        candidate_runner_id=candidate.runner_id,
        candidate_sprite_id=candidate_name,
        now=self._now(),
    ):
        await self._recover_failed_restore(operation, candidate_name)
        return
    try:
        activated = await self._prepare_restore_candidate(
            operation,
            archive,
            candidate_name,
            candidate,
        )
    except BaseException:
        await self._recover_failed_restore(operation, candidate_name)
        raise

```

```

        if not activated:
            await self._recover_failed_restore(operation, candidate_name)
            return
        await self._finish_activated_restore(operation, candidate_name)

    async def _prepare_restore_candidate(
        self,
        operation: ManagedBackupOperation,
        archive: ManagedBackupArchive,
        candidate_name: str,
        candidate: Any,
    ) -> bool:
        """Run replacement restore and report whether cutover became irreversible."""
        assert self._provider is not None
        assert self._store is not None
        assert self._relay is not None
        assert self._wrapping_key is not None
        assert operation.lease_token is not None
        assert operation.source_runner_id is not None
        assert operation.source_sprite_id is not None
        assert archive.object_version is not None
        assert archive.size_bytes is not None
        assert archive.sha256 is not None
        with tempfile.TemporaryDirectory(
            prefix="yinshi-managed-restore-", dir=self._staging_root
        ) as directory:
            local_path = Path(directory) / "archive.enc"
            await self._store.get_file(
                object_key=archive.object_key,
                object_version=archive.object_version,
                target_path=local_path,
                expected_size=archive.size_bytes,
                expected_sha256=archive.sha256,
            )
            root = f"{_MAINTENANCE_ROOT}/{operation.job_id}"
            await self._provider.upload_file(
                candidate_name,
                path=f"{root}.archive.enc",
                source_path=local_path,
                expected_size=archive.size_bytes,
                expected_sha256=archive.sha256,
                mode="0600",
            )
            archive_key = self._unwrap_key(
                envelope=archive.wrapped_key,
                user_id=operation.user_id,

```

```

        archive_id=archive.id,
        keyring={archive.key_id: self._wrapping_key},
    )
    sealed_job = seal_managed_backup_job(
        {
            "archive_context": {
                "archive_id": archive.id,
                "created_at": archive.created_at,
                "owner_digest": archive.owner_digest,
                "runtime_generation": archive.runtime_generation,
            },
            "archive_key": base64.urlsafe_b64encode(archive_key).rstrip(b "=").decode("as
            "job_id": operation.job_id,
            "operation": "restore",
            "version": 1,
        },
        runner_public_key=candidate.runner_public_key,
        job_id=operation.job_id,
    )
    await self._provider.write_file(
        candidate_name,
        path=f"{root}.job",
        content=sealed_job,
        mode="0600",
        mkdir=True,
    )
    await self._provider.configure_service(
        candidate_name,
        service_name="yinshi-maintenance",
        command="/opt/yinshi/current/venv/bin/python",
        args=(-m, "yinshi.managed_backup_guest", "restore", "--job-id", operation.job
        environment={},
        directory="/opt/yinshi/current/backend",
        needs=(),
        http_port=None,
        monitor_duration=None,
    )
    await self._provider.start_service(
        candidate_name,
        service_name="yinshi-maintenance",
        monitor_duration=300,
    )
    self._parse_restore_result(
        await self._provider.read_file(
            candidate_name,
            path=f"{root}.result",

```

```

        max_bytes=_RESULT_BYTES_MAX,
    ),
    operation.job_id,
)
for service in ("yinshi-sidecar", "yinshi-runner"):
    await self._provider.start_service(
        candidate_name,
        service_name=service,
        monitor_duration=30,
    )
assert self._runtime_service is not None
await self._runtime_service.verify_restore_candidate(
    operation.user_id,
    job_id=operation.job_id,
    candidate_runner_id=candidate.runner_id,
    candidate_sprite_name=candidate_name,
    expected_public_key=candidate.runner_public_key,
)
if not operation.source_lost:
    await self._relay.quiesce_runner(
        operation.source_runner_id,
        job_id=operation.job_id,
        timeout_seconds=30,
    )
artifact_version = self._runtime_service.artifact_version
if not self._activate_candidate(
    operation.user_id,
    source_generation=operation.runtime_generation,
    candidate_runner_id=candidate.runner_id,
    candidate_sprite_id=candidate_name,
    artifact_version=artifact_version,
    now=self._now(),
    job_id=operation.job_id,
    lease_token=operation.lease_token,
):
    return False
return True

async def _finish_activated_restore(
    self,
    operation: ManagedBackupOperation,
    candidate_name: str,
) -> None:
    """Finish post-cutover cleanup without reverting active authority."""
    assert self._provider is not None
    assert operation.source_sprite_id is not None

```



```

        if not operation.source_lost:
            await self._provider.delete_sprite(operation.source_sprite_id)
        root = f"{_MAINTENANCE_ROOT}/{operation.job_id}"
        for suffix in (".job", ".result", ".archive.enc", ".release"):
            with suppress(Exception):
                await self._provider.delete_file(candidate_name, path=f"{root}{suffix}")
        if self._complete_restore is not None:
            self._complete_restore(
                job_id=operation.job_id,
                lease_token=operation.lease_token,
                runtime_generation=operation.runtime_generation,
                now=self._now(),
            )

    async def _recover_failed_restore(
        self,
        operation: ManagedBackupOperation,
        candidate_name: str,
    ) -> None:
        """Delete failed replacement and restore source availability before cutover."""
        assert self._provider is not None
        assert self._relay is not None
        with suppress(Exception):
            self._revoke_restore_runner(operation.user_id, operation.job_id)
        candidate_deleted = False
        try:
            await self._provider.delete_sprite(candidate_name)
            candidate_deleted = True
        except Exception:
            pass
        if (
            candidate_deleted
            and operation.lease_token is not None
            and operation.candidate_runner_id is not None
            and operation.candidate_sprite_id is not None
        ):
            with suppress(Exception):
                self._clear_candidate(
                    job_id=operation.job_id,
                    lease_token=operation.lease_token,
                    runtime_generation=operation.runtime_generation,
                    candidate_runner_id=operation.candidate_runner_id,
                    candidate_sprite_id=operation.candidate_sprite_id,
                    now=self._now(),
                )
        if operation.source_sprite_id is not None and not operation.source_lost:

```

```

        for service in ("yinshi-sidecar", "yinshi-runner"):
            with suppress(Exception):
                await self._provider.start_service(
                    operation.source_sprite_id,
                    service_name=service,
                    monitor_duration=30,
                )
    if operation.source_runner_id is not None and not operation.source_lost:
        with suppress(Exception):
            await self._relay.release_maintenance(
                operation.source_runner_id,
                job_id=operation.job_id,
            )

    @staticmethod
    def _parse_restore_result(payload: bytes, job_id: str) -> None:
        try:
            result = json.loads(payload.decode("utf-8"))
        except (UnicodeDecodeError, json.JSONDecodeError):
            raise ValueError("managed restore guest result is invalid") from None
        current_result = {
            "cleanup_pending": False,
            "job_id": job_id,
            "status": "restored",
        }
        if result != current_result:
            raise ValueError("managed restore guest result is invalid")

    async def _coordinate_delete(
        self,
        operation: ManagedBackupOperation,
        archive: ManagedBackupArchive,
    ) -> None:
        """Delete one exact version before cryptographic catalog erasure."""
        assert self._store is not None
        if archive.status != "deleting" or archive.object_version is None:
            return
        await self._store.delete_file(
            object_key=archive.object_key,
            object_version=archive.object_version,
        )
        self._complete_deletion(
            operation.user_id,
            job_id=operation.job_id,
            lease_token=operation.lease_token,
            runtime_generation=operation.runtime_generation,

```

```

        now=self._now(),
    )

    async def _coordinate_create(
        self,
        operation: ManagedBackupOperation,
        archive: ManagedBackupArchive,
        runtime: ManagedRuntimeStatus,
        runner: dict[str, Any],
    ) -> None:
        assert self._provider is not None
        assert self._store is not None
        assert self._relay is not None
        if runtime.generation != operation.runtime_generation or runtime.runner_id != runner["id"]
    ):
        return
    if archive.status == "uploaded":
        await self._recover_uploaded_create(operation, archive, runtime)
        return
    if operation.phase == "object_uploading":
        if archive.size_bytes is None or archive.sha256 is None:
            raise RuntimeError("managed backup upload metadata is incomplete")
        try:
            stored = await self._store.reconcile_upload(
                object_key=archive.object_key,
                archive_id=archive.id,
                expected_size=archive.size_bytes,
                expected_sha256=archive.sha256,
            )
        except asyncio.CancelledError:
            await self._restore_create_runtime_if_owned(operation, runtime)
            raise
        except Exception:
            await self._restore_create_runtime(operation, runtime)
            raise
    if isinstance(stored, PendingManagedBackupUploads):
        if operation.lease_owner is None or operation.lease_token is None:
            raise RuntimeError("managed backup operation lease is incomplete")
        renewed_at = self._now()
        renewed = await asyncio.to_thread(
            self._renew_lease,
            job_id=operation.job_id,
            worker_id=operation.lease_owner,
            lease_token=operation.lease_token,
            runtime_generation=operation.runtime_generation,

```

```

        now=renewed_at,
        lease_expires_at=renewed_at + timedelta(minutes=15),
    )
    if not renewed:
        raise RuntimeError("managed backup operation lease was lost")
    try:
        await self._store.abort_uploads(
            object_key=archive.object_key,
            upload_ids=stored.upload_ids,
        )
        stored = await self._store.reconcile_upload(
            object_key=archive.object_key,
            archive_id=archive.id,
            expected_size=archive.size_bytes,
            expected_sha256=archive.sha256,
        )
        if isinstance(stored, PendingManagedBackupUploads):
            raise RuntimeError("managed backup upload cleanup did not converge")
    except asyncio.CancelledError:
        await self._restore_create_runtime_if_owned(operation, runtime)
        raise
    except Exception:
        await self._restore_create_runtime(operation, runtime)
        raise
    if stored is None:
        try:
            stored = await self._upload_preserved_create(
                operation,
                archive,
                runtime,
            )
        except asyncio.CancelledError:
            await self._restore_create_runtime_if_owned(operation, runtime)
            raise
        except Exception:
            await self._restore_create_runtime(operation, runtime)
            raise
    if not self._record_upload(
        operation.user_id,
        job_id=operation.job_id,
        lease_token=operation.lease_token,
        runtime_generation=operation.runtime_generation,
        size_bytes=stored.size_bytes,
        sha256=stored.sha256,
        object_version=stored.version,
        now=self._now(),
    ):

```

```

):
    return
    await self._recover_recorded_create(operation, stored, runtime)
    return
assert self._wrapping_key is not None
archive_key = self._unwrap_key(
    envelope=archive.wrapped_key,
    user_id=operation.user_id,
    archive_id=archive.id,
    keyring={self._key_id: self._wrapping_key},
)
payload = {
    "archive_context": {
        "archive_id": archive.id,
        "created_at": archive.created_at,
        "owner_digest": archive.owner_digest,
        "runtime_generation": operation.runtime_generation,
    },
    "archive_key": base64.urlsafe_b64encode(archive_key).rstrip(b"=").decode("ascii"),
    "job_id": operation.job_id,
    "operation": "create",
    "version": 1,
}
sealed_job = seal_managed_backup_job(
    payload,
    runner_public_key=str(runner["noise_public_key"]),
    job_id=operation.job_id,
)
root = f"{_MAINTENANCE_ROOT}/{operation.job_id}"
maintenance_started = False
try:
    await self._relay.quiesce_runner(
        runtime.runner_id, job_id=operation.job_id, timeout_seconds=30
    )
    maintenance_started = True
    for service in ("yinshi-runner", "yinshi-sidecar"):
        await self._provider.stop_service(
            runtime.sprite_name,
            service_name=service,
            timeout_seconds=30,
        )
    await self._provider.write_file(
        runtime.sprite_name,
        path=f"{root}.job",
        content=sealed_job,
        mode="0600",

```

```

        mkdir=True,
    )
    await self._provider.configure_service(
        runtime.sprite_name,
        service_name="yinshi-maintenance",
        command="/opt/yinshi/current/venv/bin/python",
        args=(
            "-m",
            "yinshi.managed_backup_guest",
            "create",
            "--job-id",
            operation.job_id,
            "--hold-seconds",
            "300",
        ),
        environment={},
        directory="/opt/yinshi/current/backend",
        needs=(),
        http_port=None,
        monitor_duration=None,
    )
    await self._provider.start_service(
        runtime.sprite_name,
        service_name="yinshi-maintenance",
        monitor_duration=5,
    )
    result = await self._wait_for_create_result(
        runtime.sprite_name,
        root,
        operation.job_id,
    )
except BaseException:
    if maintenance_started:
        await self._recover_failed_create(operation, runtime, root)
    raise
try:
    with tempfile.TemporaryDirectory(
        prefix="yinshi-managed-backup-", dir=self._staging_root
    ) as directory:
        local_path = Path(directory) / "archive.enc"
        await self._provider.download_file(
            runtime.sprite_name,
            path=f"{root}.archive.enc",
            target_path=local_path,
            expected_size=result["size_bytes"],
            expected_sha256=result["sha256"],

```

```

    )
    if not self._record_upload_intent(
        job_id=operation.job_id,
        lease_token=operation.lease_token,
        runtime_generation=operation.runtime_generation,
        size_bytes=result["size_bytes"],
        sha256=result["sha256"],
        now=self._now(),
    ):
        await self._recover_failed_create(operation, runtime, root)
        return
    stored = await self._store.put_file(
        local_path,
        object_key=archive.object_key,
        expected_size=result["size_bytes"],
        expected_sha256=result["sha256"],
        archive_id=archive.id,
    )
except BaseException:
    await self._restore_create_runtime(operation, runtime)
    raise
if not self._record_upload(
    operation.user_id,
    job_id=operation.job_id,
    lease_token=operation.lease_token,
    runtime_generation=operation.runtime_generation,
    size_bytes=stored.size_bytes,
    sha256=stored.sha256,
    object_version=stored.version,
    now=self._now(),
):
    await self._recover_failed_create(operation, runtime, root)
    return
await self._provider.write_file(
    runtime.sprite_name,
    path=f"{root}.release",
    content=b"release\n",
    mode="0600",
    mkdir=True,
)
await self._provider.delete_service(
    runtime.sprite_name,
    service_name="yinshi-maintenance",
)
for service in ("yinshi-sidecar", "yinshi-runner"):
    await self._provider.start_service(

```

```

        runtime.sprite_name, service_name=service, monitor_duration=30
    )
    await self._relay.release_maintenance(runtime.runner_id, job_id=operation.job_id)
    self._complete_creation(
        operation.user_id,
        job_id=operation.job_id,
        lease_token=operation.lease_token,
        runtime_generation=operation.runtime_generation,
        size_bytes=stored.size_bytes,
        sha256=stored.sha256,
        object_version=stored.version,
        now=self._now(),
    )
    for suffix in (".job", ".result", ".archive.enc", ".release"):
        await self._provider.delete_file(runtime.sprite_name, path=f"{root}/{suffix}")

    async def _wait_for_create_result(
        self,
        sprite_name: str,
        root: str,
        job_id: str,
    ) -> dict[str, Any]:
        """Wait a bounded interval for one valid maintenance result file."""
        assert self._provider is not None
        deadline = self._monotonic() + self._create_result_timeout_seconds
        while True:
            remaining = deadline - self._monotonic()
            if remaining <= 0:
                raise TimeoutError("managed backup result timed out") from None
            try:
                async with asyncio.timeout(remaining):
                    payload = await self._provider.read_file(
                        sprite_name,
                        path=f"{root}.result",
                        max_bytes=_RESULT_BYTES_MAX,
                    )
                return self._parse_create_result(payload, job_id)
            except TimeoutError:
                raise TimeoutError("managed backup result timed out") from None
            except SpritesProviderError:
                remaining = deadline - self._monotonic()
                if remaining <= 0:
                    raise TimeoutError("managed backup result timed out") from None
                try:
                    async with asyncio.timeout(remaining):
                        await self._sleep(min(1.0, remaining))

```



```

        except TimeoutError:
            raise TimeoutError("managed backup result timed out") from None

    async def _upload_preserved_create(
        self,
        operation: ManagedBackupOperation,
        archive: ManagedBackupArchive,
        runtime: ManagedRuntimeStatus,
    ) -> StoredManagedBackup:
        """Conditionally re-upload trusted guest output after confirmed absence."""
        assert self._provider is not None
        assert self._store is not None
        assert archive.size_bytes is not None
        assert archive.sha256 is not None
        root = f"{_MAINTENANCE_ROOT}/{operation.job_id}"
        with tempfile.TemporaryDirectory(
            prefix="yinshi-managed-backup-retry-",
            dir=self._staging_root,
        ) as directory:
            local_path = Path(directory) / "archive.enc"
            await self._provider.download_file(
                runtime.sprite_name,
                path=f"{root}.archive.enc",
                target_path=local_path,
                expected_size=archive.size_bytes,
                expected_sha256=archive.sha256,
            )
            stored = await self._store.put_file(
                local_path,
                object_key=archive.object_key,
                expected_size=archive.size_bytes,
                expected_sha256=archive.sha256,
                archive_id=archive.id,
            )
            if not isinstance(stored, StoredManagedBackup):
                raise RuntimeError("backup upload returned invalid object metadata")
            return stored

    async def _recover_failed_create(
        self,
        operation: ManagedBackupOperation,
        runtime: ManagedRuntimeStatus,
        root: str,
    ) -> None:
        """Restore source availability after a failed pre-upload create step."""
        assert self._provider is not None

```

```

        await self._restore_create_runtime(operation, runtime)
    for suffix in (".job", ".result", ".archive.enc", ".release"):
        with suppress(Exception):
            await self._provider.delete_file(
                runtime.sprite_name,
                path=f"{root}/{suffix}",
            )

    async def _restore_create_runtime_if_owned(
        self,
        operation: ManagedBackupOperation,
        runtime: ManagedRuntimeStatus,
    ) -> None:
        """Restore source access only after extending the exact current lease."""
        if operation.lease_owner is None or operation.lease_token is None:
            return
        renewed_at = self._now()
        try:
            renewed = await asyncio.to_thread(
                self._renew_lease,
                job_id=operation.job_id,
                worker_id=operation.lease_owner,
                lease_token=operation.lease_token,
                runtime_generation=operation.runtime_generation,
                now=renewed_at,
                lease_expires_at=renewed_at + timedelta(minutes=15),
            )
        except Exception:
            return
        if renewed:
            await self._restore_create_runtime(operation, runtime)

    async def _restore_create_runtime(
        self,
        operation: ManagedBackupOperation,
        runtime: ManagedRuntimeStatus,
    ) -> None:
        """Restore source access without changing durable guest backup output."""
        assert self._provider is not None
        assert self._relay is not None
        with suppress(Exception):
            await self._provider.stop_service(
                runtime.sprite_name,
                service_name="yinshi-maintenance",
                timeout_seconds=30,
            )

```

```

        with suppress(Exception):
            await self._provider.delete_service(
                runtime.sprite_name,
                service_name="yinshi-maintenance",
            )
    for service in ("yinshi-sidecar", "yinshi-runner"):
        with suppress(Exception):
            await self._provider.start_service(
                runtime.sprite_name,
                service_name=service,
                monitor_duration=30,
            )
    with suppress(Exception):
        await self._relay.release_maintenance(
            runtime.runner_id,
            job_id=operation.job_id,
        )

async def _recover_uploaded_create(
    self,
    operation: ManagedBackupOperation,
    archive: ManagedBackupArchive,
    runtime: ManagedRuntimeStatus,
) -> None:
    """Resume the exact source and publish one previously recorded object."""
    if archive.object_version is None or archive.size_bytes is None or archive.sha256 is None:
        return
    await self._recover_recorded_create(
        operation,
        StoredManagedBackup(
            version=archive.object_version,
            size_bytes=archive.size_bytes,
            sha256=archive.sha256,
        ),
        runtime,
    )

async def _recover_recorded_create(
    self,
    operation: ManagedBackupOperation,
    stored: StoredManagedBackup,
    runtime: ManagedRuntimeStatus,
) -> None:
    """Resume one source runtime after exact object metadata is durable."""
    assert self._provider is not None
    assert self._relay is not None

```

```

if operation.lease_token is None:
    return
root = f"{_MAINTENANCE_ROOT}/{operation.job_id}"
await self._provider.write_file(
    runtime.sprite_name,
    path=f"{root}.release",
    content=b"release\n",
    mode="0600",
    mkdir=True,
)
await self._provider.delete_service(
    runtime.sprite_name,
    service_name="yinshi-maintenance",
)
for service in ("yinshi-sidecar", "yinshi-runner"):
    await self._provider.start_service(
        runtime.sprite_name,
        service_name=service,
        monitor_duration=30,
    )
await self._relay.release_maintenance(
    runtime.runner_id,
    job_id=operation.job_id,
)
completed = self._complete_creation(
    operation.user_id,
    job_id=operation.job_id,
    lease_token=operation.lease_token,
    runtime_generation=operation.runtime_generation,
    size_bytes=stored.size_bytes,
    sha256=stored.sha256,
    object_version=stored.version,
    now=self._now(),
)
if not completed:
    return
for suffix in (".job", ".result", ".archive.enc", ".release"):
    await self._provider.delete_file(runtime.sprite_name, path=f"{root}{suffix}")

@staticmethod
def _parse_create_result(payload: bytes, job_id: str) -> dict[str, Any]:
    try:
        result = json.loads(payload.decode("utf-8"))
    except (UnicodeDecodeError, json.JSONDecodeError):
        raise ValueError("managed backup guest result is invalid") from None
    if not isinstance(result, dict) or set(result) != {

```

```

        "job_id",
        "sha256",
        "size_bytes",
        "status",
    }:
        raise ValueError("managed backup guest result is invalid")
    size = result["size_bytes"]
    digest = result["sha256"]
    if result["job_id"] != job_id or result["status"] != "ready":
        raise ValueError("managed backup guest result is invalid")
    if type(size) is not int or size <= 0 or not isinstance(digest, str) or len(digest) > 64:
        raise ValueError("managed backup guest result is invalid")
    if any(character not in "0123456789abcdef" for character in digest):
        raise ValueError("managed backup guest result is invalid")
    return result

async def start(self) -> None:
    """Start background maintenance ownership."""
    if self._task is not None:
        raise RuntimeError("managed backup manager is already started")
    self._stop_event.clear()
    self._task = asyncio.create_task(self._run(), name="managed-backup-manager")

def wake(self) -> None:
    """Request prompt reconciliation."""
    if self._task is None:
        raise RuntimeError("managed backup manager is not started")
    self._wake_generation += 1
    self._wake_event.set()

async def aclose(self) -> None:
    """Stop background maintenance ownership."""
    task = self._task
    if task is None:
        return
    self._stop_event.set()
    self._wake_event.set()
    task.cancel()
    with suppress(asyncio.CancelledError):
        await task
    self._task = None

async def _run(self) -> None:
    while not self._stop_event.is_set():
        reconcile_generation = self._wake_generation
        try:

```

```

        self.schedule_retention()
        did_work = await self._reconcile_once()
    except asyncio.CancelledError:
        raise
    except Exception:
        logger.exception("Managed backup reconciliation failed")
        did_work = False
    self._handled_wake_generation = reconcile_generation
    if did_work:
        continue
    self._wake_event.clear()
    if self._wake_generation != self._handled_wake_generation:
        continue
    try:
        await asyncio.wait_for(
            self._wake_event.wait(),
            timeout=self._interval_seconds,
        )
    except TimeoutError:
        continue

async def _idle_reconcile(self) -> bool:
    return False

```

16.10 yinshi/services/managed_backup_store.py

Object storage access uses exact versions, remote digest checks, retention rules, and explicit provider capability profiles.

```

"""S3-compatible storage for encrypted managed guest backups."""

from __future__ import annotations

import asyncio
import base64
import hashlib
import os
import re
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Protocol

_BUCKET_PATTERN = re.compile(r"[a-z0-9][a-z0-9.-]{1,61}[a-z0-9]\Z")
_OBJECT_KEY_PATTERN = re.compile(r"[A-Za-z0-9][A-Za-z0-9._/-]{0,1023}\Z")
_SHA256_PATTERN = re.compile(r"[0-9a-f]{64}\Z")
_PART_BYTES_MIN = 5 * 1024 * 1024

```

```

_PART_BYTES_MAX = 5 * 1024 * 1024 * 1024
_OBJECT_BYTES_MAX = 200 * 1024 * 1024 * 1024
_RECONCILIATION_PAGES_MAX = 8

def create_managed_backup_store(settings: Any) -> "S3ManagedBackupStore":
    """Build one bounded S3 backup store from validated application settings."""
    import boto3
    from botocore.config import Config

    client_options: dict[str, Any] = {
        "endpoint_url": settings.managed_backup_endpoint_url,
        "region_name": settings.managed_backup_region,
    }
    if settings.managed_backup_access_key_id is not None:
        client_options["aws_access_key_id"] = (
            settings.managed_backup_access_key_id.get_secret_value().strip()
        )
        client_options["aws_secret_access_key"] = (
            settings.managed_backup_secret_access_key.get_secret_value().strip()
        )
    client = boto3.client(
        "s3",
        **client_options,
        config=Config(
            connect_timeout=5,
            read_timeout=30,
            retries={"max_attempts": 3, "mode": "standard"},
            s3={"addressing_style": "path"},
        ),
    )
    return S3ManagedBackupStore(
        client=client,
        bucket=settings.managed_backup_bucket,
        server_side_encryption="AES256",
        require_bucket_default_encryption=(settings.managed_backup_provider == "aws_s3"),
        require_part_checksum_confirmation=(settings.managed_backup_provider == "aws_s3"),
        require_object_encryption_confirmation=(settings.managed_backup_provider == "aws_s3"),
        part_bytes=settings.managed_backup_part_bytes,
    )

class S3Client(Protocol):
    def create_multipart_upload(self, **request: Any) -> dict[str, Any]: ...
    def upload_part(self, **request: Any) -> dict[str, Any]: ...
    def complete_multipart_upload(self, **request: Any) -> dict[str, Any]: ...

```

```

def abort_multipart_upload(self, **request: Any) -> dict[str, Any]: ...
def head_object(self, **request: Any) -> dict[str, Any]: ...
def list_object_versions(self, **request: Any) -> dict[str, Any]: ...
def list_multipart_uploads(self, **request: Any) -> dict[str, Any]: ...
def get_object(self, **request: Any) -> dict[str, Any]: ...
def delete_object(self, **request: Any) -> dict[str, Any]: ...
def get_bucket_versioning(self, **request: Any) -> dict[str, Any]: ...
def get_bucket_encryption(self, **request: Any) -> dict[str, Any]: ...

@dataclass(frozen=True, slots=True)
class StoredManagedBackup:
    """Verified immutable object metadata after upload completion."""

    version: str
    size_bytes: int
    sha256: str

@dataclass(frozen=True, slots=True)
class PendingManagedBackupUploads:
    """Exact unfinished upload IDs found during read-only reconciliation."""

    upload_ids: tuple[str, ...]

@dataclass(frozen=True, slots=True)
class ManagedBackupObjectInventory:
    """Bounded exact-key counts used by staging cleanup checks."""

    version_count: int
    delete_marker_count: int
    multipart_upload_ids: tuple[str, ...]

class S3ManagedBackupStore:
    """Store already encrypted archives under immutable versioned object keys."""

    def __init__(
        self,
        *,
        client: S3Client,
        bucket: str,
        server_side_encryption: str,
        require_bucket_default_encryption: bool = True,
        require_part_checksum_confirmation: bool = True,

```



```

        require_object_encryption_confirmation: bool = True,
        part_bytes: int = 16 * 1024 * 1024,
    ) -> None:
        if _BUCKET_PATTERN.fullmatch(bucket) is None:
            raise ValueError("bucket must be a valid S3 bucket name")
        if server_side_encryption != "AES256":
            raise ValueError("server_side_encryption must be AES256")
        if type(require_bucket_default_encryption) is not bool:
            raise TypeError("require_bucket_default_encryption must be Boolean")
        if type(require_part_checksum_confirmation) is not bool:
            raise TypeError("require_part_checksum_confirmation must be Boolean")
        if type(require_object_encryption_confirmation) is not bool:
            raise TypeError("require_object_encryption_confirmation must be Boolean")
        if type(part_bytes) is not int or not _PART_BYTES_MIN <= part_bytes <= _PART_BYTES_MAX:
            raise ValueError("part_bytes is outside S3 multipart limits")
        self._client = client
        self._bucket = bucket
        self._server_side_encryption = server_side_encryption
        self._require_bucket_default_encryption = require_bucket_default_encryption
        self._require_part_checksum_confirmation = require_part_checksum_confirmation
        self._require_object_encryption_confirmation = require_object_encryption_confirmation
        self._part_bytes = part_bytes
        self._lost_completion_response_target: tuple[str, str] | None = None

    @property
    def lost_completion_response_target(self) -> tuple[str, str] | None:
        """Return the exact staging fault target without mutating it."""
        return self._lost_completion_response_target

    def arm_lost_completion_response(self, *, object_key: str, archive_id: str) -> None:
        """Lose one completion response only for one exact staging archive."""
        key = self._object_key(object_key)
        if not isinstance(archive_id, str) or not archive_id or len(archive_id) > 128:
            raise ValueError("archive_id must be bounded non-empty text")
        if self._lost_completion_response_target is not None:
            raise RuntimeError("backup completion response fault is already armed")
        self._lost_completion_response_target = (key, archive_id)

    @staticmethod
    def _object_key(value: str) -> str:
        if _OBJECT_KEY_PATTERN.fullmatch(value) is None or "." in value.split("/"):
            raise ValueError("object_key is invalid")
        return value

    @staticmethod
    def _expected(expected_size: int, expected_sha256: str) -> None:

```

```

if type(expected_size) is not int or not 1 <= expected_size <= _OBJECT_BYTES_MAX:
    raise ValueError("expected_size is outside the backup object limit")
if _SHA256_PATTERN.fullmatch(expected_sha256) is None:
    raise ValueError("expected_sha256 must be 64 lowercase hexadecimal characters")

async def preflight(self) -> None:
    """Require enabled versioning and AES256 bucket default encryption."""
    versioning = await asyncio.to_thread(
        self._client.get_bucket_versioning,
        Bucket=self._bucket,
    )
    if versioning.get("Status") != "Enabled":
        raise RuntimeError("managed backup bucket versioning is not enabled")
    if not self._require_bucket_default_encryption:
        return
    encryption = await asyncio.to_thread(
        self._client.get_bucket_encryption,
        Bucket=self._bucket,
    )
    configuration = encryption.get("ServerSideEncryptionConfiguration")
    rules = configuration.get("Rules") if isinstance(configuration, dict) else None
    if not isinstance(rules, list) or not any(
        isinstance(rule, dict)
        and isinstance(rule.get("ApplyServerSideEncryptionByDefault"), dict)
        and rule["ApplyServerSideEncryptionByDefault"].get("SSEAlgorithm") == "AES256"
        for rule in rules
    ):
        raise RuntimeError("managed backup bucket encryption is not AES256")

async def put_file(
    self,
    source_path: Path,
    *,
    object_key: str,
    expected_size: int,
    expected_sha256: str,
    archive_id: str,
) -> StoredManagedBackup:
    """Upload one file with multipart checksums and conditional completion."""
    key = self._object_key(object_key)
    self._expected(expected_size, expected_sha256)
    if not source_path.is_file() or source_path.is_symlink():
        raise ValueError("source_path must be a regular file")
    if source_path.stat().st_size != expected_size:
        raise ValueError("source file size does not match expected_size")
    if not archive_id or len(archive_id) > 128:

```

```

        raise ValueError("archive object metadata is invalid")
    metadata = {
        "archive-id": archive_id,
        "format": "yinshi-managed-backup-v1",
        "sha256": expected_sha256,
    }
    created = await asyncio.to_thread(
        self._client.create_multipart_upload,
        Bucket=self._bucket,
        Key=key,
        ServerSideEncryption=self._server_side_encryption,
        ChecksumAlgorithm="SHA256",
        ContentType="application/octet-stream",
        Metadata=metadata,
    )
    upload_id = created.get("UploadId")
    if not isinstance(upload_id, str) or not upload_id:
        raise RuntimeError("backup object upload did not return an upload ID")
    completed_parts: list[dict[str, object]] = []
    digest = hashlib.sha256()
    total = 0
    completion_response_lost = False
    try:
        with source_path.open("rb") as source:
            part_number = 1
            while chunk := await asyncio.to_thread(source.read, self._part_bytes):
                total += len(chunk)
                digest.update(chunk)
                checksum = base64.b64encode(hashlib.sha256(chunk).digest()).decode("ascii")
                uploaded = await asyncio.to_thread(
                    self._client.upload_part,
                    Bucket=self._bucket,
                    Key=key,
                    UploadId=upload_id,
                    PartNumber=part_number,
                    Body=chunk,
                    ContentLength=len(chunk),
                    ChecksumSHA256=checksum,
                )
                if not isinstance(uploaded.get("ETag"), str):
                    raise RuntimeError("backup object part ETag was not returned")
                if (
                    self._require_part_checksum_confirmation
                    and uploaded.get("ChecksumSHA256") != checksum
                ):
                    raise RuntimeError("backup object part checksum was not confirmed")
            part_number += 1

```

```

        completed_parts.append(
            {
                "ETag": uploaded["ETag"],
                "PartNumber": part_number,
                "ChecksumSHA256": checksum,
            }
        )
        part_number += 1
    if total != expected_size or digest.hexdigest() != expected_sha256:
        raise RuntimeError("backup object source checksum did not match")
    completed = await asyncio.to_thread(
        self._client.complete_multipart_upload,
        Bucket=self._bucket,
        Key=key,
        UploadId=upload_id,
        IfNoneMatch="*",
        MultipartUpload={"Parts": completed_parts},
        ChecksumType="COMPOSITE",
    )
    if self._lost_completion_response_target == (key, archive_id):
        self._lost_completion_response_target = None
        completion_response_lost = True
except BaseException:
    await asyncio.to_thread(
        self._client.abort_multipart_upload,
        Bucket=self._bucket,
        Key=key,
        UploadId=upload_id,
    )
    raise
if completion_response_lost:
    reconciled = await self.reconcile_upload(
        object_key=object_key,
        archive_id=archive_id,
        expected_size=expected_size,
        expected_sha256=expected_sha256,
    )
    if isinstance(reconciled, StoredManagedBackup):
        return reconciled
    raise RuntimeError("backup completion response could not be reconciled")
version = completed.get("VersionId")
if not isinstance(version, str) or not version:
    raise RuntimeError("backup object upload did not return a version")
head = await asyncio.to_thread(
    self._client.head_object,
    Bucket=self._bucket,

```

```

        Key=key,
        VersionId=version,
    )
    if (
        head.get("ContentLength") != expected_size
        or head.get("Metadata") != metadata
        or (
            self._require_object_encryption_confirmation
            and head.get("ServerSideEncryption") != self._server_side_encryption
        )
        or head.get("VersionId") != version
    ):
        await self._delete_invalid_version(key, version)
        raise RuntimeError("backup object metadata validation failed")
    if not self._require_object_encryption_confirmation:
        try:
            await self._verify_remote_digest(
                key=key,
                version=version,
                expected_size=expected_size,
                expected_sha256=expected_sha256,
            )
        except RuntimeError:
            await self._delete_invalid_version(key, version)
            raise
    return StoredManagedBackup(
        version=version, size_bytes=expected_size, sha256=expected_sha256
    )

async def _delete_invalid_version(self, key: str, version: str) -> None:
    """Remove one uploaded version that failed validation."""
    await asyncio.to_thread(
        self._client.delete_object,
        Bucket=self._bucket,
        Key=key,
        VersionId=version,
    )

async def _verify_remote_digest(
    self,
    *,
    key: str,
    version: str,
    expected_size: int,
    expected_sha256: str,
) -> None:

```

```

        """Confirm exact remote ciphertext when provider metadata is insufficient."""
    response = await asyncio.to_thread(
        self._client.get_object,
        Bucket=self._bucket,
        Key=key,
        VersionId=version,
    )
    body = response.get("Body")
    read = getattr(body, "read", None)
    close = getattr(body, "close", None)
    try:
        if response.get("ContentLength") != expected_size or not callable(read):
            raise RuntimeError("backup object checksum could not be confirmed")
        digest = hashlib.sha256()
        total = 0
        while chunk := await asyncio.to_thread(read, self._part_bytes):
            total += len(chunk)
            digest.update(chunk)
    finally:
        if callable(close):
            close()
    if total != expected_size or digest.hexdigest() != expected_sha256:
        raise RuntimeError("backup object checksum did not match")

    async def reconcile_upload(
        self,
        *,
        object_key: str,
        archive_id: str,
        expected_size: int,
        expected_sha256: str,
    ) -> StoredManagedBackup | PendingManagedBackupUploads | None:
        """Recover one exact version, pending uploads, or confirmed absence."""
        key = self._object_key(object_key)
        self._expected(expected_size, expected_sha256)
        if not isinstance(archive_id, str) or not archive_id or len(archive_id) > 128:
            raise ValueError("archive_id must be bounded non-empty text")
        matching_versions, matching_markers = await self._list_exact_versions(key)
        if not matching_versions and not matching_markers:
            matching_uploads = await self._list_exact_uploads(key)
            if matching_uploads:
                return PendingManagedBackupUploads(tuple(matching_uploads))
            matching_versions, matching_markers = await self._list_exact_versions(key)
        if not matching_versions and not matching_markers:
            matching_uploads = await self._list_exact_uploads(key)
            if matching_uploads:

```

```

        return PendingManagedBackupUploads(tuple(matching_uploads))
    matching_versions, matching_markers = await self._list_exact_versions(key)
    if not matching_versions and not matching_markers:
        return None
    if len(matching_versions) != 1 or matching_markers:
        raise RuntimeError("backup upload reconciliation was ambiguous")
    version = matching_versions[0]
    head = await asyncio.to_thread(
        self._client.head_object,
        Bucket=self._bucket,
        Key=key,
        VersionId=version,
    )
    metadata = head.get("Metadata")
    size_bytes = head.get("ContentLength")
    if (
        size_bytes != expected_size
        or not isinstance(metadata, dict)
        or metadata.get("archive-id") != archive_id
        or metadata.get("format") != "yinshi-managed-backup-v1"
        or metadata.get("sha256") != expected_sha256
        or (
            self._require_object_encryption_confirmation
            and head.get("ServerSideEncryption") != self._server_side_encryption
        )
        or head.get("VersionId") != version
    ):
        raise RuntimeError("backup upload reconciliation validation failed")
    if not self._require_object_encryption_confirmation:
        await self._verify_remote_digest(
            key=key,
            version=version,
            expected_size=expected_size,
            expected_sha256=expected_sha256,
        )
    return StoredManagedBackup(
        version=version,
        size_bytes=size_bytes,
        sha256=expected_sha256,
    )

async def _list_exact_versions(self, key: str) -> tuple[list[str], list[dict[str, Any]]]:
    request: dict[str, Any] = {
        "Bucket": self._bucket,
        "Prefix": key,
        "MaxKeys": 3,
    }

```

```

    }
    matching_versions: list[str] = []
    matching_markers: list[dict[str, Any]] = []
    for _page in range(_RECONCILIATION_PAGES_MAX):
        response = await asyncio.to_thread(
            self._client.list_object_versions,
            **request,
        )
        versions = response.get("Versions")
        markers = response.get("DeleteMarkers")
        version_entries = versions if isinstance(versions, list) else []
        marker_entries = markers if isinstance(markers, list) else []
        matching_versions.extend(
            str(version["VersionId"])
            for version in version_entries
            if isinstance(version, dict)
            and version.get("Key") == key
            and isinstance(version.get("VersionId"), str)
            and version.get("VersionId")
        )
        matching_markers.extend(
            marker
            for marker in marker_entries
            if isinstance(marker, dict) and marker.get("Key") == key
        )
        returned_keys = [
            str(entry["Key"])
            for entry in (*version_entries, *marker_entries)
            if isinstance(entry, dict) and isinstance(entry.get("Key"), str)
        ]
        if any(returned_key > key for returned_key in returned_keys):
            return matching_versions, matching_markers
        if response.get("IsTruncated") is not True:
            return matching_versions, matching_markers
        next_key = response.get("NextKeyMarker")
        next_version = response.get("NextVersionIdMarker")
        if next_key != key or not isinstance(next_version, str) or not next_version:
            raise RuntimeError("backup upload reconciliation was ambiguous")
        request["KeyMarker"] = next_key
        request["VersionIdMarker"] = next_version
    raise RuntimeError("backup upload reconciliation was ambiguous")

async def _list_exact_uploads(self, key: str) -> list[str]:
    request: dict[str, Any] = {
        "Bucket": self._bucket,
        "Prefix": key,
    }

```



```

        "MaxUploads": 2,
    }
    matching_uploads: list[str] = []
    for _page in range(_RECONCILIATION_PAGES_MAX):
        response = await asyncio.to_thread(
            self._client.list_multipart_uploads,
            **request,
        )
        uploads = response.get("Uploads")
        upload_entries = uploads if isinstance(uploads, list) else []
        matching_uploads.extend(
            str(upload["UploadId"])
            for upload in upload_entries
            if isinstance(upload, dict)
            and upload.get("Key") == key
            and isinstance(upload.get("UploadId"), str)
            and upload.get("UploadId")
        )
        returned_keys = [
            str(upload["Key"])
            for upload in upload_entries
            if isinstance(upload, dict) and isinstance(upload.get("Key"), str)
        ]
        if any(returned_key > key for returned_key in returned_keys):
            return matching_uploads
        if response.get("IsTruncated") is not True:
            return matching_uploads
        next_key = response.get("NextKeyMarker")
        next_upload = response.get("NextUploadIdMarker")
        if next_key != key or not isinstance(next_upload, str) or not next_upload:
            raise RuntimeError("backup upload reconciliation was ambiguous")
        request["KeyMarker"] = next_key
        request["UploadIdMarker"] = next_upload
    raise RuntimeError("backup upload reconciliation was ambiguous")

    async def inspect_object(self, *, object_key: str) -> ManagedBackupObjectInventory:
        """Return bounded exact-key version and multipart state."""
        key = self._object_key(object_key)
        versions, markers = await self._list_exact_versions(key)
        uploads = await self._list_exact_uploads(key)
        return ManagedBackupObjectInventory(
            version_count=len(versions),
            delete_marker_count=len(markers),
            multipart_upload_ids=tuple(uploads),
        )

```

```

async def purge_object(self, *, object_key: str) -> None:
    """Remove every version, delete marker, and upload for one exact key."""
    key = self._object_key(object_key)
    versions, markers = await self._list_exact_versions(key)
    uploads = await self._list_exact_uploads(key)
    for upload_id in uploads:
        await asyncio.to_thread(
            self._client.abort_multipart_upload,
            Bucket=self._bucket,
            Key=key,
            UploadId=upload_id,
        )
    marker_versions = tuple(
        str(marker["VersionId"])
        for marker in markers
        if isinstance(marker.get("VersionId"), str) and marker["VersionId"]
    )
    for version in (*versions, *marker_versions):
        await asyncio.to_thread(
            self._client.delete_object,
            Bucket=self._bucket,
            Key=key,
            VersionId=version,
        )

async def abort_uploads(
    self,
    *,
    object_key: str,
    upload_ids: tuple[str, ...],
) -> None:
    """Abort a bounded set of exact multipart upload IDs."""
    key = self._object_key(object_key)
    if (
        not isinstance(upload_ids, tuple)
        or not upload_ids
        or len(upload_ids) > _RECONCILIATION_PAGES_MAX * 2
        or len(set(upload_ids)) != len(upload_ids)
    ):
        raise ValueError("upload_ids must be a bounded unique tuple")
    if any(
        not isinstance(upload_id, str) or not upload_id or len(upload_id) > 2048
        for upload_id in upload_ids
    ):
        raise ValueError("upload_ids contain invalid text")
    for upload_id in upload_ids:

```

```

        await asyncio.to_thread(
            self._client.abort_multipart_upload,
            Bucket=self._bucket,
            Key=key,
            UploadId=upload_id,
        )

    async def delete_file(
        self,
        *,
        object_key: str,
        object_version: str,
    ) -> None:
        """Delete one exact immutable object version."""
        key = self._object_key(object_key)
        if not isinstance(object_version, str) or not object_version or len(object_version) > 1024:
            raise ValueError("object_version must be bounded non-empty text")
        deleted = await asyncio.to_thread(
            self._client.delete_object,
            Bucket=self._bucket,
            Key=key,
            VersionId=object_version,
        )
        if isinstance(deleted, dict) and deleted.get("VersionId") == object_version:
            return
        try:
            await asyncio.to_thread(
                self._client.head_object,
                Bucket=self._bucket,
                Key=key,
                VersionId=object_version,
            )
        except Exception as error:
            response = getattr(error, "response", None)
            code = response.get("Error", {}).get("Code") if isinstance(response, dict) else None
            if code in {"404", "NoSuchKey", "NoSuchVersion", "NotFound"}:
                return
            raise RuntimeError("backup object version deletion could not be confirmed") from error
        raise RuntimeError("backup object version deletion was not confirmed")

    async def get_file(
        self,
        *,
        object_key: str,
        object_version: str,
        target_path: Path,
    ) -> None:

```

```

        expected_size: int,
        expected_sha256: str,
    ) -> StoredManagedBackup:
        """Download one exact object version and validate its ciphertext digest."""
        key = self._object_key(object_key)
        self._expected(expected_size, expected_sha256)
        if not object_version or target_path.exists() or target_path.is_symlink():
            raise ValueError("object version or target path is invalid")
        response = await asyncio.to_thread(
            self._client.get_object,
            Bucket=self._bucket,
            Key=key,
            VersionId=object_version,
        )
        if response.get("ContentLength") != expected_size:
            raise RuntimeError("backup object size did not match")
        body = response.get("Body")
        if body is None or not callable(getattr(body, "read", None)):
            raise RuntimeError("backup object body is unavailable")
        target_path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
        descriptor = os.open(target_path, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
        digest = hashlib.sha256()
        total = 0
        try:
            with os.fdopen(descriptor, "wb", closefd=False) as output:
                while chunk := await asyncio.to_thread(body.read, self._part_bytes):
                    total += len(chunk)
                    digest.update(chunk)
                    output.write(chunk)
                    output.flush()
                os.fsync(output.fileno())
        except BaseException:
            target_path.unlink(missing_ok=True)
            raise
        finally:
            os.close(descriptor)
            close = getattr(body, "close", None)
            if callable(close):
                close()
        if total != expected_size or digest.hexdigest() != expected_sha256:
            target_path.unlink(missing_ok=True)
            raise RuntimeError("backup object checksum did not match")
        return StoredManagedBackup(
            version=object_version,
            size_bytes=expected_size,
            sha256=expected_sha256,

```

)

16.11 yinshi/services/managed_backups.py

Backup records track reservation, publication, restore claims, purge state, and archive lineage in control storage.

```
"""Durable managed backup catalog and operation claims."""

from __future__ import annotations

import sqlite3
from dataclasses import dataclass
from datetime import datetime, timezone
from typing import Literal, cast

from yinshi.db import get_control_db
from yinshi.services.runners import _require_user_id

ArchiveStatus = Literal["creating", "uploaded", "ready", "failed", "deleting", "deleted"]
OperationStatus = Literal["running", "failed"]
OperationKind = Literal["create", "restore", "delete"]
ManagedOperationFailureClass = Literal["restore_failed", "deletion_failed"]

class ManagedBackupConflictError(RuntimeError):
    """The account already owns an active managed backup operation."""

@dataclass(frozen=True, slots=True)
class ManagedBackupArchive:
    id: str
    user_id: str
    runtime_generation: int
    status: ArchiveStatus
    object_key: str
    object_version: str | None
    size_bytes: int | None
    sha256: str | None
    wrapped_key: bytes
    key_id: str
    owner_digest: str
    created_at: str
    completed_at: str | None
    last_error: str | None
```

```

@dataclass(frozen=True, slots=True)
class ManagedBackupOperation:
    user_id: str
    job_id: str
    archive_id: str
    operation: OperationKind
    status: OperationStatus
    runtime_generation: int
    started_at: str
    updated_at: str
    last_error: str | None
    phase: str = "claimed"
    lease_owner: str | None = None
    lease_token: str | None = None
    lease_expires_at: str | None = None
    attempt_count: int = 0
    next_attempt_at: str | None = None
    source_runner_id: str | None = None
    source_sprite_id: str | None = None
    source_lost: bool = False
    candidate_runner_id: str | None = None
    candidate_sprite_id: str | None = None
    activation_generation: int | None = None
    failure_class: ManagedOperationFailureClass | None = None

@dataclass(frozen=True, slots=True)
class ManagedBackupCreationClaim:
    archive: ManagedBackupArchive
    operation: ManagedBackupOperation

def _timestamp(now: datetime) -> str:
    """Normalize one aware time for durable catalog comparisons."""
    if not isinstance(now, datetime) or now.tzinfo is None:
        raise ValueError("now must be a timezone-aware datetime")
    return now.astimezone(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "")

def _archive(row: sqlite3.Row) -> ManagedBackupArchive:
    """Convert one trusted catalog row to its public typed form."""
    return ManagedBackupArchive(
        id=row["id"],
        user_id=row["user_id"],
        runtime_generation=row["runtime_generation"],

```

```

        status=cast(ArchiveStatus, row["status"]),
        object_key=row["object_key"],
        object_version=row["object_version"],
        size_bytes=row["size_bytes"],
        sha256=row["sha256"],
        wrapped_key=row["wrapped_key"],
        key_id=row["key_id"],
        owner_digest=row["owner_digest"],
        created_at=row["created_at"],
        completed_at=row["completed_at"],
        last_error=row["last_error"],
    )

def _operation(row: sqlite3.Row) -> ManagedBackupOperation:
    """Convert one trusted operation row to its public typed form."""
    return ManagedBackupOperation(
        user_id=row["user_id"],
        job_id=row["job_id"],
        archive_id=row["archive_id"],
        operation=cast(OperationKind, row["operation"]),
        status=cast(OperationStatus, row["status"]),
        runtime_generation=row["runtime_generation"],
        started_at=row["started_at"],
        updated_at=row["updated_at"],
        last_error=row["last_error"],
        phase=row["phase"],
        lease_owner=row["lease_owner"],
        lease_token=row["lease_token"],
        lease_expires_at=row["lease_expires_at"],
        attempt_count=row["attempt_count"],
        next_attempt_at=row["next_attempt_at"],
        source_runner_id=row["source_runner_id"],
        source_sprite_id=row["source_sprite_id"],
        source_lost=bool(row["source_lost"]),
        candidate_runner_id=row["candidate_runner_id"],
        candidate_sprite_id=row["candidate_sprite_id"],
        activation_generation=row["activation_generation"],
        failure_class=cast(ManagedOperationFailureClass | None, row["failure_class"]),
    )

def claim_due_managed_backup_operation(
    *,
    worker_id: str,
    lease_token: str,

```

```

now: datetime,
lease_expires_at: datetime,
) -> ManagedBackupOperation | None:
    """Claim one due running operation with an exact expiring owner token."""
    for value, name in ((worker_id, "worker_id"), (lease_token, "lease_token")):
        if not isinstance(value, str) or not value or len(value) > 128:
            raise ValueError(f"{name} must be bounded non-empty text")
    now_text = _timestamp(now)
    expiry_text = _timestamp(lease_expires_at)
    if expiry_text <= now_text:
        raise ValueError("lease_expires_at must be after now")
    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            row = database.execute(
                """SELECT job_id FROM managed_backup_operations
                WHERE status = 'running'
                AND (next_attempt_at IS NULL OR next_attempt_at <= ?)
                AND (lease_expires_at IS NULL OR lease_expires_at <= ?)
                ORDER BY started_at, job_id LIMIT 1""",
                (now_text, now_text),
            ).fetchone()
            if row is None:
                database.rollback()
                return None
            result = database.execute(
                """UPDATE managed_backup_operations
                SET lease_owner = ?, lease_token = ?, lease_expires_at = ?,
                attempt_count = attempt_count + 1, updated_at = ?
                WHERE job_id = ? AND status = 'running'
                AND (lease_expires_at IS NULL OR lease_expires_at <= ?)""",
                (
                    worker_id,
                    lease_token,
                    expiry_text,
                    now_text,
                    row["job_id"],
                    now_text,
                ),
            )
            if result.rowcount != 1:
                database.rollback()
                return None
            claimed = database.execute(
                "SELECT * FROM managed_backup_operations WHERE job_id = ?",
                (row["job_id"],),
            )

```



```

        ).fetchone()
        assert claimed is not None
        database.commit()
    except Exception:
        database.rollback()
        raise
    return _operation(claimed)

def renew_managed_backup_operation_lease(
    *,
    job_id: str,
    worker_id: str,
    lease_token: str,
    runtime_generation: int,
    now: datetime,
    lease_expires_at: datetime,
) -> bool:
    """Extend one unexpired operation lease held by its exact current owner."""
    for value, name in (
        (job_id, "job_id"),
        (worker_id, "worker_id"),
        (lease_token, "lease_token"),
    ):
        if not isinstance(value, str) or not value or len(value) > 128:
            raise ValueError(f"{name} must be bounded non-empty text")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    now_text = _timestamp(now)
    expiry_text = _timestamp(lease_expires_at)
    if expiry_text <= now_text:
        raise ValueError("lease_expires_at must be after now")
    with get_control_db() as database:
        result = database.execute(
            """UPDATE managed_backup_operations
            SET lease_expires_at = ?, updated_at = ?
            WHERE job_id = ? AND status = 'running' AND runtime_generation = ?
            AND lease_owner = ? AND lease_token = ? AND lease_expires_at > ?""",
            (
                expiry_text,
                now_text,
                job_id,
                runtime_generation,
                worker_id,
                lease_token,
                now_text,
            )
        )

```

```

        ),
    )
    database.commit()
    return result.rowcount == 1

def record_managed_backup_candidate(
    *,
    job_id: str,
    lease_token: str,
    runtime_generation: int,
    candidate_runner_id: str,
    candidate_sprite_id: str,
    now: datetime,
) -> bool:
    """Persist exact restore candidate identity for the current leased job."""
    for value, name in (
        (job_id, "job_id"),
        (lease_token, "lease_token"),
        (candidate_runner_id, "candidate_runner_id"),
        (candidate_sprite_id, "candidate_sprite_id"),
    ):
        if not isinstance(value, str) or not value or len(value) > 128:
            raise ValueError(f"{name} must be bounded non-empty text")
        if type(runtime_generation) is not int or runtime_generation <= 0:
            raise ValueError("runtime_generation must be a positive integer")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        result = database.execute(
            """UPDATE managed_backup_operations
            SET candidate_runner_id = ?, candidate_sprite_id = ?,
              phase = 'candidate_provisioning', updated_at = ?
            WHERE job_id = ? AND lease_token = ? AND operation = 'restore'
              AND status = 'running' AND runtime_generation = ?
              AND (
                  phase = 'claimed'
                  OR (
                      phase = 'candidate_provisioning'
                      AND candidate_runner_id = ? AND candidate_sprite_id = ?
                  )
              )
            AND lease_expires_at > ?""",
            (
                candidate_runner_id,
                candidate_sprite_id,
                timestamp,

```

```

        job_id,
        lease_token,
        runtime_generation,
        candidate_runner_id,
        candidate_sprite_id,
        timestamp,
    ),
)
database.commit()
return result.rowcount == 1

def clear_managed_backup_candidate(
    *,
    job_id: str,
    lease_token: str,
    runtime_generation: int,
    candidate_runner_id: str,
    candidate_sprite_id: str,
    now: datetime,
) -> bool:
    """Clear one failed replacement identity from its exact leased restore job."""
    for value, name in (
        (job_id, "job_id"),
        (lease_token, "lease_token"),
        (candidate_runner_id, "candidate_runner_id"),
        (candidate_sprite_id, "candidate_sprite_id"),
    ):
        if not isinstance(value, str) or not value or len(value) > 128:
            raise ValueError(f"{name} must be bounded non-empty text")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        result = database.execute(
            """UPDATE managed_backup_operations
            SET candidate_runner_id = NULL, candidate_sprite_id = NULL,
              phase = 'claimed', updated_at = ?
            WHERE job_id = ? AND lease_token = ? AND operation = 'restore'
              AND status = 'running' AND runtime_generation = ?
              AND phase = 'candidate_provisioning'
              AND candidate_runner_id = ? AND candidate_sprite_id = ?
              AND lease_expires_at > ?""",
            (
                timestamp,
                job_id,

```

```

        lease_token,
        runtime_generation,
        candidate_runner_id,
        candidate_sprite_id,
        timestamp,
    ),
)
    database.commit()
    return result.rowcount == 1

def advance_managed_backup_operation(
    *,
    job_id: str,
    lease_token: str,
    runtime_generation: int,
    expected_phase: str,
    next_phase: str,
    now: datetime,
) -> bool:
    """Advance one owned job only across its expected durable boundary."""
    for value, name in (
        (job_id, "job_id"),
        (lease_token, "lease_token"),
        (expected_phase, "expected_phase"),
        (next_phase, "next_phase"),
    ):
        if not isinstance(value, str) or not value or len(value) > 128:
            raise ValueError(f"{name} must be bounded non-empty text")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        result = database.execute(
            """UPDATE managed_backup_operations
            SET phase = ?, updated_at = ?
            WHERE job_id = ? AND lease_token = ? AND status = 'running'
            AND runtime_generation = ? AND phase = ?
            AND lease_expires_at > ?""",
            (
                next_phase,
                timestamp,
                job_id,
                lease_token,
                runtime_generation,
                expected_phase,
            )
        )

```

```

        timestamp,
    ),
)
database.commit()
return result.rowcount == 1

def record_managed_backup_upload_intent(
    *,
    job_id: str,
    lease_token: str,
    runtime_generation: int,
    size_bytes: int,
    sha256: str,
    now: datetime,
) -> bool:
    """Persist trusted guest metadata before immutable object publication."""
    for value, name in ((job_id, "job_id"), (lease_token, "lease_token")):
        if not isinstance(value, str) or not value or len(value) > 128:
            raise ValueError(f"{name} must be bounded non-empty text")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    if type(size_bytes) is not int or size_bytes <= 0:
        raise ValueError("size_bytes must be a positive integer")
    if len(sha256) != 64 or any(character not in "0123456789abcdef" for character in sha256):
        raise ValueError("sha256 must be 64 lowercase hexadecimal characters")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            operation = database.execute(
                """SELECT archive_id FROM managed_backup_operations
                WHERE job_id = ? AND lease_token = ? AND operation = 'create'
                AND status = 'running' AND runtime_generation = ?
                AND phase = 'claimed' AND lease_expires_at > ?""",
                (job_id, lease_token, runtime_generation, timestamp),
            ).fetchone()
            if operation is None:
                database.rollback()
                return False
            archive_result = database.execute(
                """UPDATE managed_backup_archives
                SET size_bytes = ?, sha256 = ?, last_error = NULL
                WHERE id = ? AND status = 'creating'
                AND runtime_generation = ? AND object_version IS NULL""",
                (size_bytes, sha256, operation["archive_id"], runtime_generation),
            )

```

```

    )
    operation_result = database.execute(
        """UPDATE managed_backup_operations
           SET phase = 'object_uploading', updated_at = ?
           WHERE job_id = ? AND lease_token = ? AND operation = 'create'
              AND status = 'running' AND runtime_generation = ?
              AND phase = 'claimed' AND lease_expires_at > ?""",
        (
            timestamp,
            job_id,
            lease_token,
            runtime_generation,
            timestamp,
        ),
    )
    if archive_result.rowcount != 1 or operation_result.rowcount != 1:
        database.rollback()
        return False
    database.commit()
    return True
except Exception:
    database.rollback()
    raise

def complete_managed_backup_restore(
    *,
    job_id: str,
    lease_token: str,
    runtime_generation: int,
    now: datetime,
) -> bool:
    """Complete one activated restore after exact old-Sprite deletion."""
    for value, name in ((job_id, "job_id"), (lease_token, "lease_token")):
        if not isinstance(value, str) or not value or len(value) > 128:
            raise ValueError(f"{name} must be bounded non-empty text")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        result = database.execute(
            """DELETE FROM managed_backup_operations
               WHERE job_id = ? AND lease_token = ? AND operation = 'restore'
                  AND status = 'running' AND runtime_generation = ?
                  AND phase = 'activated' AND lease_expires_at > ?""",
            (job_id, lease_token, runtime_generation, timestamp),
        )

```

```

        )
        database.commit()
    return result.rowcount == 1

def start_managed_backup_creation(
    user_id: str,
    *,
    runtime_generation: int,
    archive_id: str,
    job_id: str,
    object_key: str,
    wrapped_key: bytes,
    key_id: str,
    owner_digest: str,
    now: datetime,
) -> ManagedBackupCreationClaim:
    """Atomically create one archive row and exclusive active operation."""
    normalized_user_id = _require_user_id(user_id)
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    for value, name, limit in (
        (archive_id, "archive_id", 128),
        (job_id, "job_id", 128),
        (object_key, "object_key", 1024),
        (key_id, "key_id", 128),
    ):
        if not isinstance(value, str) or not value or len(value) > limit:
            raise ValueError(f"{name} must be bounded non-empty text")
    if not isinstance(wrapped_key, bytes) or not wrapped_key:
        raise ValueError("wrapped_key must be non-empty bytes")
    if len(owner_digest) != 64 or any(
        character not in "0123456789abcdef" for character in owner_digest
    ):
        raise ValueError("owner_digest must be 64 lowercase hexadecimal characters")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            active = database.execute(
                """SELECT status FROM managed_backup_operations
                WHERE user_id = ?""",
                (normalized_user_id,),
            ).fetchone()
            if active is not None and active["status"] == "running":
                raise ManagedBackupConflictError("Managed backup operation is already active")

```

```

if active is not None:
    database.execute(
        "DELETE FROM managed_backup_operations WHERE user_id = ?",
        (normalized_user_id,),
    )
runtime = database.execute(
    """SELECT generation, lifecycle_status, runner_id, sprite_external_id
       FROM managed_runtimes WHERE user_id = ?""",
    (normalized_user_id,),
).fetchone()
if (
    runtime is None
    or runtime["generation"] != runtime_generation
    or runtime["lifecycle_status"] != "ready"
):
    raise ManagedBackupConflictError("Managed runtime is not ready for backup")
database.execute(
    """INSERT INTO managed_backup_archives (
        id, user_id, runtime_generation, status, object_key,
        wrapped_key, key_id, owner_digest, created_at
    ) VALUES (?, ?, ?, 'creating', ?, ?, ?, ?, ?)""",
    (
        archive_id,
        normalized_user_id,
        runtime_generation,
        object_key,
        wrapped_key,
        key_id,
        owner_digest,
        timestamp,
    ),
)
database.execute(
    """INSERT INTO managed_backup_operations (
        user_id, job_id, archive_id, operation, status,
        runtime_generation, source_runner_id, source_sprite_id,
        started_at, updated_at
    ) VALUES (?, ?, ?, 'create', 'running', ?, ?, ?, ?, ?)""",
    (
        normalized_user_id,
        job_id,
        archive_id,
        runtime_generation,
        runtime["runner_id"],
        runtime["sprite_external_id"],
        timestamp,
    ),
)

```



```

        timestamp,
    ),
)
archive_row = database.execute(
    "SELECT * FROM managed_backup_archives WHERE id = ?",
    (archive_id,),
).fetchone()
operation_row = database.execute(
    "SELECT * FROM managed_backup_operations WHERE user_id = ?",
    (normalized_user_id,),
).fetchone()
assert archive_row is not None and operation_row is not None
database.commit()
except ManagedBackupConflictError:
    database.rollback()
    raise
except Exception:
    database.rollback()
    raise
return ManagedBackupCreationClaim(
    archive=_archive(archive_row),
    operation=_operation(operation_row),
)

def _start_managed_backup_restore(
    user_id: str,
    *,
    archive_id: str,
    runtime_generation: int,
    job_id: str,
    now: datetime,
    source_lost: bool,
) -> ManagedBackupCreationClaim:
    """Claim one tenant-owned ready archive with explicit source state."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(archive_id, str) or not archive_id:
        raise ValueError("archive_id must not be empty")
    if not isinstance(job_id, str) or not job_id:
        raise ValueError("job_id must not be empty")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")

```

```

runtime = database.execute(
    """SELECT generation, runner_id, sprite_external_id
       FROM managed_runtimes
       WHERE user_id = ? AND lifecycle_status = 'ready'""",
    (normalized_user_id,),
).fetchone()
archive_row = database.execute(
    """SELECT * FROM managed_backup_archives
       WHERE id = ? AND user_id = ? AND status = 'ready'
       AND object_version IS NOT NULL""",
    (archive_id, normalized_user_id),
).fetchone()
if runtime is None or runtime["generation"] != runtime_generation:
    raise ManagedBackupConflictError("managed runtime generation changed")
if archive_row is None:
    raise ManagedBackupConflictError("managed backup archive is not ready")
database.execute(
    """INSERT INTO managed_backup_operations (
        user_id, job_id, archive_id, operation, status,
        runtime_generation, source_runner_id, source_sprite_id,
        source_lost, started_at, updated_at, last_error
    ) VALUES (?, ?, ?, 'restore', 'running', ?, ?, ?, ?, ?, ?, NULL)""",
    (
        normalized_user_id,
        job_id,
        archive_id,
        runtime_generation,
        runtime["runner_id"],
        runtime["sprite_external_id"],
        int(source_lost),
        timestamp,
        timestamp,
    ),
)
database.commit()
except sqlite3.IntegrityError:
    database.rollback()
    raise ManagedBackupConflictError("managed backup operation is active") from None
except Exception:
    database.rollback()
    raise
return ManagedBackupCreationClaim(
    archive=_archive(archive_row),
    operation=ManagedBackupOperation(
        user_id=normalized_user_id,
        job_id=job_id,

```

```

        archive_id=archive_id,
        operation="restore",
        status="running",
        runtime_generation=runtime_generation,
        started_at=timestamp,
        updated_at=timestamp,
        last_error=None,
        source_runner_id=runtime["runner_id"],
        source_sprite_id=runtime["sprite_external_id"],
        source_lost=source_lost,
    ),
)

def start_managed_backup_restore(
    user_id: str,
    *,
    archive_id: str,
    runtime_generation: int,
    job_id: str,
    now: datetime,
) -> ManagedBackupCreationClaim:
    """Claim one tenant-owned ready archive for replacement-runtime restore."""
    return _start_managed_backup_restore(
        user_id,
        archive_id=archive_id,
        runtime_generation=runtime_generation,
        job_id=job_id,
        now=now,
        source_lost=False,
    )

def start_managed_source_loss_restore(
    user_id: str,
    *,
    archive_id: str,
    runtime_generation: int,
    job_id: str,
    now: datetime,
) -> ManagedBackupCreationClaim:
    """Atomically claim one ready archive after confirmed source loss."""
    return _start_managed_backup_restore(
        user_id,
        archive_id=archive_id,
        runtime_generation=runtime_generation,

```

```

        job_id=job_id,
        now=now,
        source_lost=True,
    )

def start_managed_backup_deletion(
    user_id: str,
    *,
    archive_id: str,
    runtime_generation: int,
    job_id: str,
    now: datetime,
) -> ManagedBackupCreationClaim:
    """Claim exact-version deletion while retaining wrapped recovery material."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(archive_id, str) or not archive_id:
        raise ValueError("archive_id must not be empty")
    if not isinstance(job_id, str) or not job_id:
        raise ValueError("job_id must not be empty")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            result = database.execute(
                """UPDATE managed_backup_archives SET status = 'deleting'
                WHERE id = ? AND user_id = ? AND status = 'ready'
                AND object_version IS NOT NULL""",
                (archive_id, normalized_user_id),
            )
            if result.rowcount != 1:
                raise ManagedBackupConflictError("managed backup archive is not ready")
            database.execute(
                """INSERT INTO managed_backup_operations (
                user_id, job_id, archive_id, operation, status,
                runtime_generation, started_at, updated_at, last_error
                ) VALUES (?, ?, ?, 'delete', 'running', ?, ?, ?, NULL)""",
                (
                    normalized_user_id,
                    job_id,
                    archive_id,
                    runtime_generation,
                    timestamp,
                    timestamp,
                )
            )

```

```

        ),
    )
    archive_row = database.execute(
        "SELECT * FROM managed_backup_archives WHERE id = ?",
        (archive_id,),
    ).fetchone()
    assert archive_row is not None
    database.commit()
except sqlite3.IntegrityError:
    database.rollback()
    raise ManagedBackupConflictError("managed backup operation is active") from None
except Exception:
    database.rollback()
    raise
return ManagedBackupCreationClaim(
    archive=_archive(archive_row),
    operation=ManagedBackupOperation(
        user_id=normalized_user_id,
        job_id=job_id,
        archive_id=archive_id,
        operation="delete",
        status="running",
        runtime_generation=runtime_generation,
        started_at=timestamp,
        updated_at=timestamp,
        last_error=None,
    ),
)

def complete_managed_backup_deletion(
    user_id: str,
    *,
    job_id: str,
    lease_token: str,
    runtime_generation: int,
    now: datetime,
) -> bool:
    """Retire one exact remotely deleted archive and release its fence."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(job_id, str) or not job_id:
        raise ValueError("job_id must not be empty")
    if not isinstance(lease_token, str) or not lease_token:
        raise ValueError("lease_token must not be empty")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")

```

```

timestamp = _timestamp(now)
with get_control_db() as database:
    try:
        database.execute("BEGIN IMMEDIATE")
        operation = database.execute(
            """SELECT archive_id FROM managed_backup_operations
               WHERE user_id = ? AND job_id = ? AND operation = 'delete'
                  AND status = 'running' AND runtime_generation = ?
                  AND lease_token = ? AND lease_expires_at > ?""",
            (
                normalized_user_id,
                job_id,
                runtime_generation,
                lease_token,
                timestamp,
            ),
        ).fetchone()
        if operation is None:
            database.rollback()
            return False
        result = database.execute(
            """UPDATE managed_backup_archives
               SET status = 'deleted', object_version = NULL, size_bytes = NULL,
                 sha256 = NULL, wrapped_key = X'', completed_at = ?,
                 last_error = NULL
               WHERE id = ? AND user_id = ? AND status = 'deleting'""",
            (timestamp, operation["archive_id"], normalized_user_id),
        )
        if result.rowcount != 1:
            database.rollback()
            return False
        database.execute(
            "DELETE FROM managed_backup_operations WHERE user_id = ? AND job_id = ?",
            (normalized_user_id, job_id),
        )
        database.commit()
        return True
    except Exception:
        database.rollback()
        raise

def fail_managed_backup_creation(
    user_id: str,
    *,
    job_id: str,

```

```

runtime_generation: int,
error_code: str,
now: datetime,
) -> bool:
    """Fail one matching create operation and release its maintenance fence."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(job_id, str) or not job_id:
        raise ValueError("job_id must not be empty")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    if (
        not isinstance(error_code, str)
        or not error_code
        or len(error_code) > 100
        or any(character not in "abcdefghijklmnopqrstuvwxyz_" for character in error_code)
    ):
        raise ValueError("error_code must be bounded lowercase identifier text")
    timestamp = _timestamp(now)
    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            operation = database.execute(
                """SELECT archive_id FROM managed_backup_operations
                WHERE user_id = ? AND job_id = ? AND operation = 'create'
                AND status = 'running' AND runtime_generation = ?""",
                (normalized_user_id, job_id, runtime_generation),
            ).fetchone()
            if operation is None:
                database.rollback()
                return False
            result = database.execute(
                """UPDATE managed_backup_archives
                SET status = 'failed', completed_at = ?, last_error = ?
                WHERE id = ? AND user_id = ? AND status = 'creating'
                AND runtime_generation = ?""",
                (
                    timestamp,
                    error_code,
                    operation["archive_id"],
                    normalized_user_id,
                    runtime_generation,
                ),
            )
            if result.rowcount != 1:
                database.rollback()
                return False

```

```

        database.execute(
            """UPDATE managed_backup_operations
               SET status = 'failed', updated_at = ?, last_error = ?
               WHERE user_id = ? AND job_id = ? AND status = 'running'""",
            (timestamp, error_code, normalized_user_id, job_id),
        )
        database.commit()
        return True
    except Exception:
        database.rollback()
        raise

def list_managed_backup_retention_candidates(
    *,
    cutoff: str,
    limit: int,
) -> tuple[ManagedBackupArchive, ...]:
    """Return bounded old ready archives not owned by active operations."""
    if not isinstance(cutoff, str) or not cutoff:
        raise ValueError("cutoff must not be empty")
    if type(limit) is not int or not 1 <= limit <= 100:
        raise ValueError("limit must be between 1 and 100")
    with get_control_db() as database:
        rows = database.execute(
            """SELECT archive.* FROM managed_backup_archives AS archive
               WHERE archive.status = 'ready' AND archive.created_at < ?
               AND archive.object_version IS NOT NULL
               AND NOT EXISTS (
                   SELECT 1 FROM managed_backup_operations AS operation
                   WHERE operation.archive_id = archive.id
                   AND operation.status = 'running'
               )
               ORDER BY archive.created_at, archive.id LIMIT ?""",
            (cutoff, limit),
        ).fetchall()
        return tuple(_archive(row) for row in rows)

def get_managed_backup_operation(
    user_id: str,
    job_id: str,
) -> ManagedBackupOperation | None:
    """Return one tenant-owned operation without exposing another account."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(job_id, str) or not job_id:

```



```

        raise ValueError("job_id must not be empty")
    with get_control_db() as database:
        row = database.execute(
            """SELECT * FROM managed_backup_operations
               WHERE user_id = ? AND job_id = ?""",
            (normalized_user_id, job_id),
        ).fetchone()
    return None if row is None else _operation(row)

def managed_backup_operation_is_running(user_id: str) -> bool:
    """Return whether one account has a durable managed maintenance fence."""
    normalized_user_id = _require_user_id(user_id)
    with get_control_db() as database:
        row = database.execute(
            """SELECT 1 FROM managed_backup_operations
               WHERE user_id = ? AND status = 'running'""",
            (normalized_user_id,),
        ).fetchone()
    return row is not None

def get_running_managed_backup_operation_for_runner(
    runner_id: str,
) -> ManagedBackupOperation | None:
    """Return the running maintenance job that fences one exact source runner."""
    if not isinstance(runner_id, str) or not runner_id or len(runner_id) > 128:
        raise ValueError("runner_id must be bounded non-empty text")
    with get_control_db() as database:
        row = database.execute(
            """SELECT * FROM managed_backup_operations
               WHERE source_runner_id = ? AND status = 'running'""",
            (runner_id,),
        ).fetchone()
    return None if row is None else _operation(row)

def list_managed_backup_archives(
    user_id: str,
    *,
    limit: int = 100,
) -> tuple[ManagedBackupArchive, ...]:
    """Return bounded newest-first archive rows owned by one account."""
    normalized_user_id = _require_user_id(user_id)
    if type(limit) is not int or not 1 <= limit <= 100:
        raise ValueError("limit must be between 1 and 100")

```

```

with get_control_db() as database:
    rows = database.execute(
        """SELECT * FROM managed_backup_archives
           WHERE user_id = ? ORDER BY created_at DESC, id DESC LIMIT ?""",
        (normalized_user_id, limit),
    ).fetchall()
return tuple(_archive(row) for row in rows)

def get_managed_backup_archive(
    user_id: str,
    archive_id: str,
) -> ManagedBackupArchive | None:
    """Return one tenant-owned archive without exposing another account."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(archive_id, str) or not archive_id:
        raise ValueError("archive_id must not be empty")
    with get_control_db() as database:
        row = database.execute(
            "SELECT * FROM managed_backup_archives WHERE user_id = ? AND id = ?",
            (normalized_user_id, archive_id),
        ).fetchone()
    return None if row is None else _archive(row)

def record_managed_backup_upload(
    user_id: str,
    *,
    job_id: str,
    lease_token: str,
    runtime_generation: int,
    size_bytes: int,
    sha256: str,
    object_version: str,
    now: datetime,
) -> bool:
    """Persist exact uploaded object metadata before runtime recovery."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(job_id, str) or not job_id:
        raise ValueError("job_id must not be empty")
    if not isinstance(lease_token, str) or not lease_token:
        raise ValueError("lease_token must not be empty")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    if type(size_bytes) is not int or size_bytes <= 0:
        raise ValueError("size_bytes must be a positive integer")

```

```

if len(sha256) != 64 or any(character not in "0123456789abcdef" for character in sha256):
    raise ValueError("sha256 must be 64 lowercase hexadecimal characters")
if not isinstance(object_version, str) or not object_version or len(object_version) > 10:
    raise ValueError("object_version must be bounded non-empty text")
timestamp = _timestamp(now)
with get_control_db() as database:
    try:
        database.execute("BEGIN IMMEDIATE")
        operation = database.execute(
            """SELECT archive_id FROM managed_backup_operations
               WHERE user_id = ? AND job_id = ? AND operation = 'create'
                  AND status = 'running' AND runtime_generation = ?
                  AND lease_token = ? AND lease_expires_at > ?""",
            (
                normalized_user_id,
                job_id,
                runtime_generation,
                lease_token,
                timestamp,
            ),
        ).fetchone()
        if operation is None:
            database.rollback()
            return False
        result = database.execute(
            """UPDATE managed_backup_archives
               SET status = 'uploaded', object_version = ?, size_bytes = ?,
                  sha256 = ?, completed_at = ?, last_error = NULL
               WHERE id = ? AND user_id = ? AND status = 'creating'
                  AND runtime_generation = ?""",
            (
                object_version,
                size_bytes,
                sha256,
                timestamp,
                operation["archive_id"],
                normalized_user_id,
                runtime_generation,
            ),
        )
        database.commit()
        return result.rowcount == 1
    except Exception:
        database.rollback()
        raise

```

```

def complete_managed_backup_creation(
    user_id: str,
    *,
    job_id: str,
    lease_token: str | None = None,
    runtime_generation: int,
    size_bytes: int,
    sha256: str,
    object_version: str | None,
    now: datetime,
) -> bool:
    """Publish verified object metadata for the matching active creation."""
    normalized_user_id = _require_user_id(user_id)
    if not isinstance(job_id, str) or not job_id:
        raise ValueError("job_id must not be empty")
    if not isinstance(lease_token, str) or not lease_token:
        raise ValueError("lease_token must not be empty")
    if type(runtime_generation) is not int or runtime_generation <= 0:
        raise ValueError("runtime_generation must be a positive integer")
    if type(size_bytes) is not int or size_bytes <= 0:
        raise ValueError("size_bytes must be a positive integer")
    if len(sha256) != 64 or any(character not in "0123456789abcdef" for character in sha256):
        raise ValueError("sha256 must be 64 lowercase hexadecimal characters")
    if not isinstance(object_version, str) or not object_version or len(object_version) > 10:
        return False
    timestamp = _timestamp(now)
    with get_control_db() as database:
        try:
            database.execute("BEGIN IMMEDIATE")
            operation = database.execute(
                """SELECT archive_id FROM managed_backup_operations
                WHERE user_id = ? AND job_id = ? AND operation = 'create'
                AND status = 'running' AND runtime_generation = ?
                AND lease_token = ? AND lease_expires_at > ?""",
                (
                    normalized_user_id,
                    job_id,
                    runtime_generation,
                    lease_token,
                    timestamp,
                ),
            ).fetchone()
            runtime = database.execute(
                "SELECT generation, lifecycle_status FROM managed_runtimes WHERE user_id = ?
                (normalized_user_id,)",

```

```

).fetchone()
if (
    operation is None
    or runtime is None
    or runtime["generation"] != runtime_generation
    or runtime["lifecycle_status"] != "ready"
):
    database.rollback()
    return False
result = database.execute(
    """UPDATE managed_backup_archives
    SET status = 'ready', object_version = ?, size_bytes = ?,
        sha256 = ?, completed_at = ?, last_error = NULL
    WHERE id = ? AND user_id = ?
        AND status IN ('creating', 'uploaded')
        AND runtime_generation = ?""",
    (
        object_version,
        size_bytes,
        sha256,
        timestamp,
        operation["archive_id"],
        normalized_user_id,
        runtime_generation,
    ),
)
if result.rowcount != 1:
    database.rollback()
    return False
database.execute(
    "DELETE FROM managed_backup_operations WHERE user_id = ? AND job_id = ?",
    (normalized_user_id, job_id),
)
database.commit()
return True
except Exception:
    database.rollback()
    raise

```

16.12 yinshi/services/managed_operational_failures.py

Operational failure storage keeps current sanitized faults separate from normal runtime lifecycle state.

"""Durable active failures used by managed operational monitoring."""

```

from __future__ import annotations

from datetime import UTC, datetime
from enum import Enum

from yinshi.db import get_control_db


class ManagedPersistentAlertClass(str, Enum):
    """Service failures approved for durable operational state."""

    SPRITE_RECONCILIATION_FAILED = "managed_sprite_reconciliation_failed"
    STORAGE_PREFLIGHT_FAILED = "managed_storage_preflight_failed"

def _validate_alert_class(
    alert_class: ManagedPersistentAlertClass,
) -> ManagedPersistentAlertClass:
    if not isinstance(alert_class, ManagedPersistentAlertClass):
        raise ValueError("alert class is not approved for durable monitoring")
    return alert_class

def _format_utc(timestamp: datetime) -> str:
    if timestamp.tzinfo is None:
        raise ValueError("failure timestamp must include timezone information")
    return timestamp.astimezone(UTC).isoformat().replace("+00:00", "Z")

def record_managed_operational_failure(
    alert_class: ManagedPersistentAlertClass,
    *,
    now: datetime | None = None,
) -> None:
    """Record one active service failure without storing provider details."""
    approved_class = _validate_alert_class(alert_class)
    timestamp = _format_utc(now or datetime.now(UTC))
    with get_control_db() as database:
        database.execute(
            """INSERT INTO managed_operational_failures (
                alert_class, first_seen_at, last_seen_at
            ) VALUES (?, ?, ?)
            ON CONFLICT(alert_class) DO UPDATE SET last_seen_at = excluded.last_seen_at"""
            (approved_class.value, timestamp, timestamp),
        )
        database.commit()

```

```

def clear_managed_operational_failure(
    alert_class: ManagedPersistentAlertClass,
) -> None:
    """Clear one active service failure after a complete successful operation."""
    approved_class = _validate_alert_class(alert_class)
    with get_control_db() as database:
        database.execute(
            "DELETE FROM managed_operational_failures WHERE alert_class = ?",
            (approved_class.value,),
        )
        database.commit()

```

16.13 yinshi/services/managed_operational_status.py

Operational status reduces detailed runtime findings into safe public health and richer authenticated diagnostics.

```

"""Sanitized operational findings for managed runtime monitoring."""

from __future__ import annotations

import sqlite3
from dataclasses import dataclass
from datetime import UTC, datetime, timedelta
from enum import Enum
from typing import Any

class ManagedAlertClass(str, Enum):
    """Stable alert identifiers consumed by external monitoring."""

    BACKUP_STALE = "managed_backup_stale"
    OPERATION_STUCK = "managed_operation_stuck"
    OPERATION_LEASE_EXPIRED = "managed_operation_lease_expired"
    RESTORE_FAILED = "managed_restore_failed"
    SPRITE_RECONCILIATION_FAILED = "managed_sprite_reconciliation_failed"
    STORAGE_PREFLIGHT_FAILED = "managed_storage_preflight_failed"
    DELETION_FAILED = "managed_deletion_failed"

@dataclass(frozen=True, slots=True)
class ManagedOperationalAlert:
    """One aggregated finding with no resource identifiers."""

```

```

alert_class: ManagedAlertClass
count: int
oldest_age_seconds: int

def __post_init__(self) -> None:
    if self.count < 1:
        raise ValueError("alert count must be positive")
    if self.oldest_age_seconds < 0:
        raise ValueError("alert age must not be negative")

def to_dict(self) -> dict[str, int | str]:
    """Return bounded public monitoring fields."""
    return {
        "alert_class": self.alert_class.value,
        "count": self.count,
        "oldest_age_seconds": self.oldest_age_seconds,
    }

@dataclass(frozen=True, slots=True)
class ManagedOperationalStatus:
    """Aggregated status returned by monitoring commands."""

    generated_at: str
    alerts: tuple[ManagedOperationalAlert, ...]

    @property
    def critical(self) -> bool:
        """Return whether monitoring must alert."""
        return bool(self.alerts)

    def to_dict(self) -> dict[str, Any]:
        """Serialize only sanitized aggregate data."""
        return {
            "schema_version": 1,
            "generated_at": self.generated_at,
            "status": "critical" if self.critical else "ok",
            "alerts": [alert.to_dict() for alert in self.alerts],
        }

def _parse_timestamp(value: object, *, now: datetime) -> datetime:
    """Parse one database timestamp into UTC or mark it maximally stale."""
    if not isinstance(value, str) or not value.strip():
        return datetime.min.replace(tzinfo=UTC)
    try:

```



```

        parsed = datetime.fromisoformat(value.strip().replace("Z", "+00:00"))
    except ValueError:
        return datetime.min.replace(tzinfo=UTC)
    if parsed.tzinfo is None:
        parsed = parsed.replace(tzinfo=UTC)
    return min(parsed.astimezone(UTC), now)

def _age_seconds(timestamp: datetime, *, now: datetime) -> int:
    """Return a nonnegative bounded age."""
    seconds = int((now - timestamp).total_seconds())
    return min(max(seconds, 0), 2_147_483_647)

def _append_alert(
    alerts: list[ManagedOperationalAlert],
    alert_class: ManagedAlertClass,
    timestamps: list[object],
    *,
    now: datetime,
) -> None:
    """Append one finding when matching records exist."""
    if not timestamps:
        return
    oldest = min(_parse_timestamp(value, now=now) for value in timestamps)
    alerts.append(
        ManagedOperationalAlert(
            alert_class=alert_class,
            count=len(timestamps),
            oldest_age_seconds=_age_seconds(oldest, now=now),
        )
    )

def collect_managed_operational_status(
    database: sqlite3.Connection,
    *,
    now: datetime,
    backup_stale_seconds: int,
    operation_stuck_seconds: int,
) -> ManagedOperationalStatus:
    """Read control state and return sanitized critical findings."""
    if now.tzinfo is None:
        raise ValueError("now must include timezone information")
    if backup_stale_seconds < 60 or operation_stuck_seconds < 60:
        raise ValueError("operational thresholds must be at least 60 seconds")

```

```

now = now.astimezone(UTC)
backup_cutoff = (now - timedelta(seconds=backup_stale_seconds)).isoformat()
operation_cutoff = (now - timedelta(seconds=operation_stuck_seconds)).isoformat()
alerts: list[ManagedOperationalAlert] = []

stale_backup_rows = database.execute(
    """SELECT MAX(archive.completed_at) AS finding_at
    FROM managed_runtimes AS runtime
    LEFT JOIN managed_backup_archives AS archive
    ON archive.user_id = runtime.user_id AND archive.status = ?
    WHERE runtime.lifecycle_status = ?
    GROUP BY runtime.user_id
    HAVING finding_at IS NULL OR finding_at < ?""",
    ("ready", "ready", backup_cutoff),
).fetchall()
_append_alert(
    alerts,
    ManagedAlertClass.BACKUP_STALE,
    [row["finding_at"] for row in stale_backup_rows],
    now=now,
)

stuck_rows = database.execute(
    """SELECT updated_at FROM managed_backup_operations
    WHERE status = ? AND updated_at < ?""",
    ("running", operation_cutoff),
).fetchall()
_append_alert(
    alerts,
    ManagedAlertClass.OPERATION_STUCK,
    [row["updated_at"] for row in stuck_rows],
    now=now,
)

expired_lease_rows = database.execute(
    """SELECT lease_expires_at FROM managed_backup_operations
    WHERE status = ? AND lease_token IS NOT NULL AND lease_expires_at <= ?""",
    ("running", now.isoformat()),
).fetchall()
_append_alert(
    alerts,
    ManagedAlertClass.OPERATION_LEASE_EXPIRED,
    [row["lease_expires_at"] for row in expired_lease_rows],
    now=now,
)

```

```

restore_rows = database.execute(
    """SELECT updated_at FROM managed_backup_operations
       WHERE failure_class = ? AND status = ?""",
    ("restore_failed", "failed"),
).fetchall()
_append_alert(
    alerts,
    ManagedAlertClass.RESTORE_FAILED,
    [row["updated_at"] for row in restore_rows],
    now=now,
)

deletion_rows = database.execute(
    """SELECT updated_at AS finding_at FROM managed_backup_operations
       WHERE failure_class = ? AND status = ?""",
    ("deletion_failed", "failed"),
).fetchall()
_append_alert(
    alerts,
    ManagedAlertClass.DELETION_FAILED,
    [row["finding_at"] for row in deletion_rows],
    now=now,
)

persistent_rows = database.execute(
    "SELECT alert_class, first_seen_at FROM managed_operational_failures"
).fetchall()
for row in persistent_rows:
    _append_alert(
        alerts,
        ManagedAlertClass(row["alert_class"]),
        [row["first_seen_at"]],
        now=now,
    )

alerts.sort(key=lambda alert: alert.alert_class.value)
return ManagedOperationalStatus(
    generated_at=now.isoformat().replace("+00:00", "Z"),
    alerts=tuple(alerts),
)

```

17 Encrypted Runner Relay

The browser reaches hosted workspaces through Noise-encrypted RPC messages. A long-lived runner relay carries repository, prompt, file, upload, and terminal

operations without exposing runtime credentials in control-plane responses.

17.1 yinshi/api/prompt_runs.py

Prompt routes start one journaled run and reconnect clients to stored events after transport interruption.

```
"""Reconnectable JSON prompt-run routes for local, hosted, and BYOC transports."""

from __future__ import annotations

import uuid
from typing import Any, Literal, cast

from fastapi import APIRouter, HTTPException, Request, status
from pydantic import BaseModel, Field, field_validator

from yinshi.api.deps import check_session_owner, get_db_for_request
from yinshi.api.stream import PromptRequest
from yinshi.services.prompt_journal import (
    PromptEventBatch,
    PromptJournal,
    PromptRun,
    PromptRunConflictError,
    PromptRunNotFoundError,
)

router = APIRouter()
RunStatus = Literal[
    "starting",
    "running",
    "stopping",
    "completed",
    "failed",
    "cancelled",
    "interrupted",
]

class PromptRunStart(PromptRequest):
    """Prompt content plus a client-stable idempotency key."""

    idempotency_key: str = Field(..., max_length=36)

    @field_validator("idempotency_key")
    @classmethod
```

```

def validate_idempotency_key(cls, value: str) -> str:
    """Require one canonical UUID so retries cannot alias."""
    try:
        normalized = str(uuid.UUID(value))
    except ValueError as exc:
        raise ValueError("idempotency_key must be a UUID") from exc
    if normalized != value:
        raise ValueError("idempotency_key must be canonical")
    return normalized

class PromptRunResponse(BaseModel):
    id: str
    session_id: str
    status: RunStatus

class PromptEventBatchResponse(BaseModel):
    run_id: str
    status: RunStatus
    events: list[dict[str, Any]]
    next_sequence: int

def _prompt_journal(request: Request) -> PromptJournal:
    journal = getattr(request.app.state, "prompt_journal", None)
    if not isinstance(journal, PromptJournal):
        raise RuntimeError("prompt journal is unavailable")
    return journal

def _require_session(request: Request, session_id: str) -> None:
    with get_db_for_request(request) as database:
        row = database.execute(
            "SELECT id FROM sessions WHERE id = ?",
            (session_id,),
        ).fetchone()
        if row is None:
            raise HTTPException(status_code=404, detail="Session not found")
        check_session_owner(database, session_id, request)

def _run_response(run: PromptRun) -> PromptRunResponse:
    return PromptRunResponse(
        id=run.id,
        session_id=run.session_id,

```

```

        status=cast(RunStatus, run.status),
    )

def _event_response(batch: PromptEventBatch) -> PromptEventBatchResponse:
    return PromptEventBatchResponse(
        run_id=batch.run_id,
        status=cast(RunStatus, batch.status),
        events=list(batch.events),
        next_sequence=batch.next_sequence,
    )

@router.post(
    "/api/sessions/{session_id}/runs",
    response_model=PromptRunResponse,
    status_code=status.HTTP_202_ACCEPTED,
)
async def start_prompt_run(
    session_id: str,
    body: PromptRunStart,
    request: Request,
) -> PromptRunResponse:
    """Start one durable prompt run or return its idempotent predecessor."""
    _require_session(request, session_id)
    prompt_body = PromptRequest(
        prompt=body.prompt,
        model=body.model,
        thinking=body.thinking,
    )
    try:
        run = await _prompt_journal(request).start(
            request=request,
            session_id=session_id,
            idempotency_key=body.idempotency_key,
            body=prompt_body,
        )
    except PromptRunNotFoundError as exc:
        raise HTTPException(status_code=404, detail="Session not found") from exc
    except PromptRunConflictError as exc:
        raise HTTPException(status_code=409, detail=str(exc)) from exc
    return _run_response(run)

@router.get(
    "/api/sessions/{session_id}/runs/{run_id}/events/{next_sequence}",

```

```

        response_model=PromptEventBatchResponse,
    )
    async def get_prompt_events(
        session_id: str,
        run_id: str,
        next_sequence: int,
        request: Request,
    ) -> PromptEventBatchResponse:
        """Return one bounded contiguous journal page from the supplied cursor."""
        _require_session(request, session_id)
        try:
            batch = await _prompt_journal(request).events(
                request=request,
                session_id=session_id,
                run_id=run_id,
                next_sequence=next_sequence,
            )
        except (PromptRunNotFoundError, ValueError) as exc:
            raise HTTPException(status_code=404, detail="Prompt run not found") from exc
        return _event_response(batch)

    @router.post(
        "/api/sessions/{session_id}/runs/{run_id}/cancel",
        response_model=PromptRunResponse,
    )
    async def cancel_prompt_run(
        session_id: str,
        run_id: str,
        request: Request,
    ) -> PromptRunResponse:
        """Cancel one run idempotently without depending on a live HTTP stream."""
        _require_session(request, session_id)
        try:
            run = await _prompt_journal(request).cancel(
                request=request,
                session_id=session_id,
                run_id=run_id,
            )
        except (PromptRunNotFoundError, ValueError) as exc:
            raise HTTPException(status_code=404, detail="Prompt run not found") from exc
        return _run_response(run)

```

17.2 yinshi/api/runner_relay.py

The relay endpoint authenticates one browser or runner role, then forwards opaque encrypted frames without inspecting payload content.

```
"""Opaque WebSocket relay between browser clients and outbound BYOC runners."""

from __future__ import annotations

import asyncio
import contextlib
import json

from fastapi import APIRouter, WebSocket, WebSocketDisconnect

from yinshi.exceptions import RunnerAuthenticationError
from yinshi.services.runner_relay import (
    RunnerRelayAuthorizationError,
    claim_runner_transfer_grant,
    runner_relay_broker,
)
from yinshi.services.runners import authenticate_runner_token

router = APIRouter(tags=["runner-relay"])
_CLIENT_AUTH_TIMEOUT_SECONDS = 10.0
_CLIENT_CONNECTION_MAX_SECONDS = 3_600.0
_CLIENT_CONNECTION_LIMIT = asyncio.Semaphore(128)
_CLIENT_SLOT_TIMEOUT_SECONDS = 0.05

def _runner_bearer_token(websocket: WebSocket) -> str:
    """Extract one runner bearer token without accepting the WebSocket first."""
    authorization = websocket.headers.get("authorization")
    if authorization is None or not authorization.startswith("Bearer "):
        raise RunnerAuthenticationError("Runner bearer token is required")
    token = authorization.removeprefix("Bearer ").strip()
    if not token:
        raise RunnerAuthenticationError("Runner bearer token is required")
    return token

@router.websocket("/runner/relay")
async def runner_relay_socket(websocket: WebSocket) -> None:
    """Accept one authenticated outbound runner and route only framed ciphertext."""
    try:
        runner = authenticate_runner_token(_runner_bearer_token(websocket))
```



```

except (RunnerAuthenticationError, TypeError, ValueError):
    await websocket.close(code=4401, reason="Runner authentication failed")
    return

runner_id = runner["runner_id"]
await websocket.accept()
await websocket.send_json(
    {
        "runner_id": runner_id,
        "type": "welcome",
    }
)
await runner_relay_broker.register_runner(runner_id, websocket)
try:
    while True:
        message = await websocket.receive()
        if message["type"] == "websocket.disconnect":
            break
        control_text = message.get("text")
        if isinstance(control_text, str):
            try:
                control = json.loads(control_text)
                if not isinstance(control, dict):
                    raise ValueError("invalid runner relay control")
                if (
                    set(control) == {"transfer_id", "type"}
                    and control.get("type") == "close"
                    and isinstance(control.get("transfer_id"), str)
                ):
                    await runner_relay_broker.runner_closed_transfer(
                        runner_id,
                        control["transfer_id"],
                    )
                    continue
                if (
                    set(control) == {"job_id", "type"}
                    and control.get("type") == "quiesced"
                    and isinstance(control.get("job_id"), str)
                ):
                    await runner_relay_broker.runner_quiesced(
                        runner_id,
                        control["job_id"],
                    )
                    continue
                raise ValueError("invalid runner relay control")
            except (json.JSONDecodeError, RunnerRelayAuthorizationError, ValueError):

```

```

        await websocket.close(code=4400, reason="Runner relay control was rejected")
        break
    ciphertext = message.get("bytes")
    if not isinstance(ciphertext, bytes):
        await websocket.close(code=4400, reason="Binary relay frames are required")
        break
    try:
        await runner_relay_broker.runner_frame(runner_id, ciphertext)
    except (RunnerRelayAuthorizationError, TypeError, ValueError):
        await websocket.close(code=4400, reason="Runner relay frame was rejected")
        break
except WebSocketDisconnect:
    pass
finally:
    await runner_relay_broker.unregister_runner(runner_id, websocket)

@router.websocket("/api/runner/relay/{transfer_id}")
async def client_relay_socket(websocket: WebSocket, transfer_id: str) -> None:
    """Authorize one capability and relay bounded ciphertext without persistence."""
    try:
        await asyncio.wait_for(
            _CLIENT_CONNECTION_LIMIT.acquire(),
            timeout=_CLIENT_SLOT_TIMEOUT_SECONDS,
        )
    except TimeoutError:
        await websocket.close(code=4429, reason="Runner relay connection limit reached")
        return
    sender_task: asyncio.Task[None] | None = None
    attached = False
    try:
        await websocket.accept()
        async with asyncio.timeout(_CLIENT_AUTH_TIMEOUT_SECONDS):
            capability = await websocket.receive_text()
        grant = claim_runner_transfer_grant(transfer_id, capability)
        await runner_relay_broker.attach_client(grant, websocket)
        attached = True
        await websocket.send_json({"type": "ready"})
        sender_task = asyncio.create_task(
            runner_relay_broker.send_client_frames(transfer_id,
            name=f"runner-relay-client-{transfer_id}",
        )
    except asyncio.timeout:
        pass
    async with asyncio.timeout(_CLIENT_CONNECTION_MAX_SECONDS):
        while True:
            message = await websocket.receive()
            if message["type"] == "websocket.disconnect":

```

```

        break
    ciphertext = message.get("bytes")
    if not isinstance(ciphertext, bytes):
        await websocket.close(code=4400, reason="Binary relay frames are required")
        break
    await runner_relay_broker.client_frame(transfer_id, ciphertext)
except TimeoutError:
    await websocket.close(code=4408, reason="Runner relay timed out")
except (RunnerRelayAuthorizationError, TypeError, ValueError):
    await websocket.close(code=4403, reason="Runner relay authorization failed")
except WebSocketDisconnect:
    pass
finally:
    if attached:
        await runner_relay_broker.detach_client(transfer_id, websocket)
    if sender_task is not None:
        sender_task.cancel()
        with contextlib.suppress(asyncio.CancelledError):
            await sender_task
    _CLIENT_CONNECTION_LIMIT.release()

```

17.3 yinshi/api/runtime__uploads.py

Upload routes reserve bounded transfers and attach encrypted payload meta-data to the authorized runtime.

```

"""Chunked encrypted upload routes for browser-to-runner file operations."""

from __future__ import annotations

import base64
import binascii
from typing import Any

from fastapi import APIRouter, HTTPException, Request, status
from pydantic import BaseModel, Field

from yinshi.api.deps import require_tenant
from yinshi.exceptions import PiConfigError
from yinshi.services.encrypted_uploads import (
    EncryptedUpload,
    EncryptedUploadManager,
)
from yinshi.services.pi_config import import_from_upload

router = APIRouter()

```

```

class EncryptedUploadStartRequest(BaseModel):
    purpose: str = Field(..., min_length=1, max_length=32)
    filename: str = Field(..., min_length=1, max_length=255)
    size_bytes: int = Field(..., ge=1, le=50 * 1024 * 1024)
    sha256: str = Field(..., min_length=64, max_length=64)

class EncryptedUploadChunkRequest(BaseModel):
    data: str = Field(..., min_length=1, max_length=32_000)

class EncryptedUploadResponse(BaseModel):
    id: str
    purpose: str
    filename: str
    size_bytes: int
    next_chunk_index: int

def _manager(request: Request) -> EncryptedUploadManager:
    manager = getattr(request.app.state, "encrypted_upload_manager", None)
    if not isinstance(manager, EncryptedUploadManager):
        raise RuntimeError("encrypted upload manager is unavailable")
    return manager

def _response(upload: EncryptedUpload) -> EncryptedUploadResponse:
    return EncryptedUploadResponse(
        id=upload.id,
        purpose=upload.purpose,
        filename=upload.filename,
        size_bytes=upload.size_bytes,
        next_chunk_index=upload.next_chunk_index,
    )

def _decode_chunk(value: str) -> bytes:
    if not isinstance(value, str) or not value or "=" in value:
        raise ValueError("encrypted upload chunk encoding is invalid")
    padded = value + "=" * (-len(value) % 4)
    try:
        decoded = base64.b64decode(
            padded.encode("ascii"),
            altchars=b"_"

```

```

        validate=True,
    )
except (UnicodeEncodeError, binascii.Error) as exc:
    raise ValueError("encrypted upload chunk encoding is invalid") from exc
canonical = base64.urlsafe_b64encode(decoded).rstrip(b"=").decode("ascii")
if canonical != value:
    raise ValueError("encrypted upload chunk encoding is not canonical")
return decoded

@router.post(
    "/api/settings/pi-config/uploads",
    response_model=EncryptedUploadResponse,
    status_code=status.HTTP_201_CREATED,
)
def start_encrypted_upload(
    body: EncryptedUploadStartRequest,
    request: Request,
) -> EncryptedUploadResponse:
    """Reserve one bounded owner-scoped Pi config upload."""
    tenant = require_tenant(request)
    try:
        upload = _manager(request).start(
            user_id=tenant.user_id,
            purpose=body.purpose,
            filename=body.filename,
            size_bytes=body.size_bytes,
            sha256_hex=body.sha256,
        )
    except (RuntimeError, ValueError) as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    return _response(upload)

@router.post(
    "/api/settings/pi-config/uploads/{upload_id}/chunks/{chunk_index}",
    response_model=EncryptedUploadResponse,
)
def append_encrypted_upload_chunk(
    upload_id: str,
    chunk_index: int,
    body: EncryptedUploadChunkRequest,
    request: Request,
) -> EncryptedUploadResponse:
    """Append or idempotently retry one exact encrypted upload chunk."""
    tenant = require_tenant(request)

```

```

    try:
        upload = _manager(request).append(
            user_id=tenant.user_id,
            upload_id=upload_id,
            chunk_index=chunk_index,
            chunk=_decode_chunk(body.data),
        )
    except LookupError as exc:
        raise HTTPException(status_code=404, detail="Encrypted upload not found") from exc
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    return _response(upload)

@router.post(
    "/api/settings/pi-config/uploads/{upload_id}/complete",
    status_code=status.HTTP_201_CREATED,
)
async def complete_encrypted_upload(upload_id: str, request: Request) -> dict[str, Any]:
    """Verify and consume one upload before invoking the existing Pi importer."""
    tenant = require_tenant(request)
    try:
        completed = _manager(request).complete(
            user_id=tenant.user_id,
            upload_id=upload_id,
        )
    except LookupError as exc:
        raise HTTPException(status_code=404, detail="Encrypted upload not found") from exc
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    if completed.purpose != "pi_config":
        raise RuntimeError("encrypted upload purpose changed before completion")
    try:
        return await import_from_upload(
            user_id=tenant.user_id,
            data_dir=tenant.data_dir,
            zip_data=completed.data,
            filename=completed.filename,
        )
    except PiConfigError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc

@router.delete(
    "/api/settings/pi-config/uploads/{upload_id}",
    status_code=status.HTTP_204_NO_CONTENT,
)

```

```

)
def cancel_encrypted_upload(upload_id: str, request: Request) -> None:
    """Discard one incomplete upload idempotently."""
    tenant = require_tenant(request)
    try:
        _manager(request).cancel(user_id=tenant.user_id, upload_id=upload_id)
    except LookupError as exc:
        raise HTTPException(status_code=404, detail="Encrypted upload not found") from exc

```

17.4 yinshi/api/sessions.py

Session routes create, list, rename, and remove agent conversations within tenant-owned workspaces.

```

"""Endpoints for agent sessions."""

import logging
import os
from typing import Any

from fastapi import APIRouter, HTTPException, Request

from yinshi.api.deps import (
    check_session_owner,
    check_workspace_owner,
    get_db_for_request,
)
from yinshi.model_catalog import normalize_model_ref
from yinshi.models import MessageOut, SessionCreate, SessionOut, SessionUpdate

logger = logging.getLogger(__name__)
router = APIRouter(tags=["sessions"])

_UPDATABLE_COLUMNS = {"model"}
_EXCLUDED_DIRS = frozenset(
    {
        ".git",
        "node_modules",
        ".venv",
        "venv",
        "__pycache__",
        ".tox",
        ".mypy_cache",
        "vendor",
        ".next",
        "dist",
    }
)

```

```

        "build",
    }
)
_TREE_FILE_LIMIT = 5000

def _normalize_session_row(db: Any, row: Any) -> dict[str, Any]:
    """Normalize stored session models and persist repairs on read."""
    normalized_row = dict(row)
    original_model = normalized_row["model"]
    normalized_model = normalize_model_ref(original_model)
    if normalized_model != original_model:
        db.execute(
            "UPDATE sessions SET model = ? WHERE id = ?", (normalized_model, normalized_row["id"])
        )
        db.commit()
        normalized_row["model"] = normalized_model
    if "pi_context_version" not in normalized_row or normalized_row["pi_context_version"] is None:
        normalized_row["pi_context_version"] = 0
    return normalized_row

def _list_workspace_files(workspace_path: str) -> list[str]:
    """List workspace files while excluding bulky build directories."""
    if not os.path.isdir(workspace_path):
        return []

    files: list[str] = []
    for dirpath, dirnames, filenames in os.walk(workspace_path):
        dirnames[:] = sorted(dirname for dirname in dirnames if dirname not in _EXCLUDED_DIRS)
        for filename in sorted(filenames):
            relative_path = os.path.relpath(
                os.path.join(dirpath, filename),
                workspace_path,
            )
            files.append(relative_path)
        if len(files) >= _TREE_FILE_LIMIT:
            files.sort()
            return files

    files.sort()
    return files

@router.get("/api/workspaces/{workspace_id}/sessions", response_model=list[SessionOut])
def list_sessions(workspace_id: str, request: Request) -> list[dict[str, Any]]:

```



```

        """List all sessions for a workspace."""
    with get_db_for_request(request) as db:
        check_workspace_owner(db, workspace_id, request)
        rows = db.execute(
            "SELECT * FROM sessions WHERE workspace_id = ? ORDER BY created_at DESC",
            (workspace_id,),
        ).fetchall()
        return [_normalize_session_row(db, row) for row in rows]

@router.post(
    "/api/workspaces/{workspace_id}/sessions",
    response_model=SessionOut,
    status_code=201,
)
def create_session(
    workspace_id: str,
    body: SessionCreate,
    request: Request,
) -> dict[str, Any]:
    """Create a new agent session for a workspace."""
    with get_db_for_request(request) as db:
        ws = db.execute("SELECT id FROM workspaces WHERE id = ?", (workspace_id,)).fetchone()
        if not ws:
            raise HTTPException(status_code=404, detail="Workspace not found")
        check_workspace_owner(db, workspace_id, request)

        cursor = db.execute(
            """INSERT INTO sessions (workspace_id, status, model, pi_context_version)
            VALUES (?, 'idle', ?, 1)""",
            (workspace_id, body.model),
        )
        db.commit()
        row = db.execute("SELECT * FROM sessions WHERE rowid = ?", (cursor.lastrowid,)).fetchone()
        assert row is not None, "created session must be queryable"
        return _normalize_session_row(db, row)

@router.get("/api/sessions/{session_id}", response_model=SessionOut)
def get_session(session_id: str, request: Request) -> dict[str, Any]:
    """Get a session by ID."""
    with get_db_for_request(request) as db:
        row = db.execute("SELECT * FROM sessions WHERE id = ?", (session_id,)).fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Session not found")
        check_session_owner(db, session_id, request)

```

```

        return _normalize_session_row(db, row)

@router.patch("/api/sessions/{session_id}", response_model=SessionOut)
def update_session(
    session_id: str,
    body: SessionUpdate,
    request: Request,
) -> dict[str, Any]:
    """Update session fields (currently only model)."""
    with get_db_for_request(request) as db:
        row = db.execute("SELECT * FROM sessions WHERE id = ?", (session_id,)).fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Session not found")
        check_session_owner(db, session_id, request)

        updates = {
            k: v for k, v in body.model_dump(exclude_unset=True).items() if k in _UPDATABLES
        }
        if updates:
            sets = ", ".join(f"{k} = ?" for k in updates)
            vals = list(updates.values()) + [session_id]
            db.execute(f"UPDATE sessions SET {sets} WHERE id = ?", vals) # noqa: S608
            db.commit()
        updated = db.execute("SELECT * FROM sessions WHERE id = ?", (session_id,)).fetchone()
        assert updated is not None, "updated session must be queryable"
        return _normalize_session_row(db, updated)

@router.get("/api/sessions/{session_id}/messages", response_model=list[MessageOut])
def get_messages(session_id: str, request: Request) -> list[dict[str, Any]]:
    """Get all messages for a session."""
    with get_db_for_request(request) as db:
        sess = db.execute("SELECT id FROM sessions WHERE id = ?", (session_id,)).fetchone()
        if not sess:
            raise HTTPException(status_code=404, detail="Session not found")
        check_session_owner(db, session_id, request)

        rows = db.execute(
            "SELECT * FROM messages WHERE session_id = ? ORDER BY created_at",
            (session_id,),
        ).fetchall()
        return [dict(r) for r in rows]

@router.get("/api/sessions/{session_id}/tree")

```

```

def get_session_tree(session_id: str, request: Request) -> dict[str, list[str]]:
    """Return the workspace file tree for a session."""
    with get_db_for_request(request) as db:
        row = db.execute(
            "SELECT s.id, w.path as workspace_path "
            "FROM sessions s "
            "JOIN workspaces w ON s.workspace_id = w.id "
            "WHERE s.id = ?",
            (session_id,),
        ).fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Session not found")
        check_session_owner(db, session_id, request)

        workspace_path = row["workspace_path"]
        assert isinstance(workspace_path, str)
        return {"files": _list_workspace_files(workspace_path)}

```

17.5 yinshi/api/terminal_channels.py

Terminal routes allocate reconnectable channels and enforce workspace ownership before input or resize operations.

```

"""Reconnectable terminal channels for encrypted and JSON runtime transports."""

from __future__ import annotations

from collections.abc import AsyncIterator
from contextlib import asynccontextmanager
from typing import Any, Literal

from fastapi import APIRouter, HTTPException, Request, status
from pydantic import BaseModel, Field

from yinshi.api.deps import get_db_for_request, get_tenant
from yinshi.config import get_settings
from yinshi.exceptions import (
    ContainerNotReadyError,
    ContainerStartError,
    GitError,
    SidecarError,
    SidecarNotConnectedError,
    WorkspaceNotFoundError,
)
from yinshi.services.sidecar_runtime import (
    protect_tenant_container,

```

```

        release_tenant_container,
        remap_path_for_container,
        resolve_tenant_sidecar_context,
        tenant_container_activity,
    )
    from yinshi.services.terminal_journal import (
        TerminalCursorExpiredError,
        TerminalEventBatch,
        TerminalJournal,
        TerminalLimitError,
        TerminalNotFoundError,
    )
    from yinshi.services.workspace_runtime_paths import prepare_tenant_workspace_runtime_paths

    router = APIRouter()

    class TerminalStartRequest(BaseModel):
        cols: int = Field(default=100, ge=2, le=500)
        rows: int = Field(default=30, ge=2, le=500)

    class TerminalStartResponse(BaseModel):
        id: str
        workspace_id: str
        status: Literal["attached"]

    class TerminalInputRequest(BaseModel):
        data: str = Field(..., min_length=1, max_length=16_384)

    class TerminalResizeRequest(BaseModel):
        cols: int = Field(..., ge=2, le=500)
        rows: int = Field(..., ge=2, le=500)

    class TerminalEventBatchResponse(BaseModel):
        terminal_id: str
        events: list[dict[str, Any]]
        next_sequence: int
        closed: bool

    def _journal(request: Request) -> TerminalJournal:
        terminal_journal = getattr(request.app.state, "terminal_journal", None)

```

```

    if not isinstance(terminal_journal, TerminalJournal):
        raise RuntimeError("terminal journal is unavailable")
    return terminal_journal

def _tenant_identity(request: Request) -> tuple[str, Any]:
    tenant = get_tenant(request)
    if tenant is None:
        raise HTTPException(status_code=403, detail="Terminal account context is required")
    if not tenant.user_id:
        raise RuntimeError("terminal tenant user ID is empty")
    return tenant.user_id, tenant

async def _terminal_context(
    request: Request,
    workspace_id: str,
) -> tuple[str, Any, str, str, str | None]:
    user_id, tenant = _tenant_identity(request)
    try:
        with get_db_for_request(request) as database:
            paths = await prepare_tenant_workspace_runtime_paths(
                database,
                tenant,
                workspace_id,
            )
        runtime = await resolve_tenant_sidecar_context(
            request,
            tenant,
            repo_agents_md=paths.agents_md,
            repo_root_path=paths.repo_root_path,
            workspace_path=paths.workspace_path,
            workspace_id=workspace_id,
        )
        if runtime.socket_path is None:
            socket_path = get_settings().sidecar_socket_path
            effective_cwd = paths.workspace_path
        else:
            socket_path = runtime.socket_path
            effective_cwd = remap_path_for_container(paths.workspace_path, tenant.data_dir)
    except (PermissionError, WorkspaceNotFoundError):
        raise HTTPException(status_code=404, detail="Workspace not found") from None
    except (ContainerStartError, ContainerNotReadyError, GitError, OSError, ValueError):
        raise HTTPException(status_code=503, detail="Terminal runtime unavailable") from None
    return user_id, tenant, socket_path, effective_cwd, runtime.runtime_id

```

```

def _batch_response(batch: TerminalEventBatch) -> TerminalEventBatchResponse:
    return TerminalEventBatchResponse(
        terminal_id=batch.terminal_id,
        events=list(batch.events),
        next_sequence=batch.next_sequence,
        closed=batch.closed,
    )

def _terminal_lease_key(workspace_id: str, terminal_id: str) -> str:
    return f"terminal:{workspace_id}:{terminal_id}"

def _refresh_terminal_lease(
    request: Request,
    tenant: Any,
    workspace_id: str,
    terminal_id: str,
) -> None:
    protect_tenant_container(
        request,
        tenant,
        lease_key=_terminal_lease_key(workspace_id, terminal_id),
        timeout_s=get_settings().terminal_keepalive_s,
        runtime_id=workspace_id,
    )

def _release_terminal_lease(
    request: Request,
    tenant: Any,
    workspace_id: str,
    terminal_id: str,
) -> None:
    release_tenant_container(
        request,
        tenant,
        lease_key=_terminal_lease_key(workspace_id, terminal_id),
        runtime_id=workspace_id,
    )

@asynccontextmanager
async def _terminal_runtime_activity(
    request: Request,

```

```

        tenant: Any,
        workspace_id: str,
    ) -> AsyncIterator[None]:
        try:
            async with tenant_container_activity(
                request,
                tenant,
                runtime_id=workspace_id,
            ):
                yield
        except ContainerNotReadyError:
            raise HTTPException(
                status_code=503,
                detail="Terminal runtime unavailable",
            ) from None

@router.post(
    "/api/workspaces/{workspace_id}/terminals",
    response_model=TerminalStartResponse,
    status_code=status.HTTP_201_CREATED,
)
async def start_terminal_channel(
    workspace_id: str,
    body: TerminalStartRequest,
    request: Request,
) -> TerminalStartResponse:
    """Attach one account-scoped terminal to the workspace sidecar."""
    user_id, tenant, socket_path, cwd, runtime_id = await _terminal_context(
        request,
        workspace_id,
    )
    try:
        async with tenant_container_activity(
            request,
            tenant,
            runtime_id=runtime_id,
        ):
            terminal_id = await _journal(request).start(
                user_id=user_id,
                workspace_id=workspace_id,
                socket_path=socket_path,
                cwd=cwd,
                cols=body.cols,
                rows=body.rows,
            )

```

```

        _refresh_terminal_lease(request, tenant, workspace_id, terminal_id)
    except TerminalLimitError as exc:
        raise HTTPException(status_code=429, detail=str(exc)) from exc
    except (
        ConnectionError,
        ContainerNotReadyError,
        OSError,
        SidecarError,
        SidecarNotConnectedError,
    ):
        raise HTTPException(status_code=503, detail="Terminal runtime unavailable") from None
    return TerminalStartResponse(
        id=terminal_id,
        workspace_id=workspace_id,
        status="attached",
    )

@router.post("/api/workspaces/{workspace_id}/terminals/{terminal_id}/input", status_code=200)
async def send_terminal_input(
    workspace_id: str,
    terminal_id: str,
    body: TerminalInputRequest,
    request: Request,
) -> None:
    """Forward one bounded input fragment to an attached terminal."""
    user_id, tenant = _tenant_identity(request)
    try:
        async with _terminal_runtime_activity(request, tenant, workspace_id):
            await _journal(request).input(
                user_id=user_id,
                workspace_id=workspace_id,
                terminal_id=terminal_id,
                data=body.data,
            )
        _refresh_terminal_lease(request, tenant, workspace_id, terminal_id)
    except TerminalNotFoundError as exc:
        raise HTTPException(status_code=404, detail="Terminal not found") from exc

@router.post("/api/workspaces/{workspace_id}/terminals/{terminal_id}/resize", status_code=200)
async def resize_terminal_channel(
    workspace_id: str,
    terminal_id: str,
    body: TerminalResizeRequest,
    request: Request,

```



```

) -> None:
    """Resize one attached terminal."""
    user_id, tenant = _tenant_identity(request)
    try:
        async with _terminal_runtime_activity(request, tenant, workspace_id):
            await _journal(request).resize(
                user_id=user_id,
                workspace_id=workspace_id,
                terminal_id=terminal_id,
                cols=body.cols,
                rows=body.rows,
            )
            _refresh_terminal_lease(request, tenant, workspace_id, terminal_id)
    except TerminalNotFoundError as exc:
        raise HTTPException(status_code=404, detail="Terminal not found") from exc

@router.post("/api/workspaces/{workspace_id}/terminals/{terminal_id}/restart", status_code=200)
async def restart_terminal_channel(
    workspace_id: str,
    terminal_id: str,
    request: Request,
) -> None:
    """Restart one attached terminal with its original options."""
    user_id, tenant = _tenant_identity(request)
    try:
        async with _terminal_runtime_activity(request, tenant, workspace_id):
            await _journal(request).restart(
                user_id=user_id,
                workspace_id=workspace_id,
                terminal_id=terminal_id,
            )
            _refresh_terminal_lease(request, tenant, workspace_id, terminal_id)
    except TerminalNotFoundError as exc:
        raise HTTPException(status_code=404, detail="Terminal not found") from exc

@router.get(
    "/api/workspaces/{workspace_id}/terminals/{terminal_id}/events/{next_sequence}",
    response_model=TerminalEventBatchResponse,
)
async def get_terminal_events(
    workspace_id: str,
    terminal_id: str,
    next_sequence: int,
    request: Request,

```

```

) -> TerminalEventBatchResponse:
    """Poll one contiguous bounded terminal-output page."""
    user_id, tenant = _tenant_identity(request)
    try:
        async with _terminal_runtime_activity(request, tenant, workspace_id):
            batch = await _journal(request).events(
                user_id=user_id,
                workspace_id=workspace_id,
                terminal_id=terminal_id,
                next_sequence=next_sequence,
            )
            if batch.closed:
                _release_terminal_lease(request, tenant, workspace_id, terminal_id)
            else:
                _refresh_terminal_lease(request, tenant, workspace_id, terminal_id)
    except TerminalNotFoundError as exc:
        raise HTTPException(status_code=404, detail="Terminal not found") from exc
    except TerminalCursorExpiredError as exc:
        raise HTTPException(status_code=409, detail="Terminal output cursor expired") from exc
    return _batch_response(batch)

@router.delete(
    "/api/workspaces/{workspace_id}/terminals/{terminal_id}",
    status_code=204,
)
async def close_terminal_channel(
    workspace_id: str,
    terminal_id: str,
    request: Request,
) -> None:
    """Detach one terminal channel idempotently."""
    user_id, tenant = _tenant_identity(request)
    await _journal(request).close(
        user_id=user_id,
        workspace_id=workspace_id,
        terminal_id=terminal_id,
    )
    _release_terminal_lease(request, tenant, workspace_id, terminal_id)

```

17.6 yinshi/runner_worker.py

The Sprite worker dispatches validated RPC methods to tenant-local services and keeps the relay connection alive between requests.

```

"""Stable runner-local configuration and restricted worker dispatcher factory."""

from __future__ import annotations

import base64
import hashlib
import json
import os
import secrets
import sqlite3
import stat
import uuid
from collections.abc import Callable
from pathlib import Path
from typing import Literal

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

from yinshi.tenant import TenantContext, get_user_db, init_user_db
from yinshi.worker_auth import WorkerPrincipal, prepare_worker_principal_storage
from yinshi.worker_runtime import WorkerHttpDispatcher

EnvironmentSetter = Callable[[str, str], None]
RunnerUserDataEncryptionMode = Literal["disabled", "required"]
_X25519_PRIVATE_KEY_LENGTH = 32
_PROMPT_EVENT_COUNT_MAX = 100_000

def validate_user_data_encryption_mode(
    value: object,
) -> RunnerUserDataEncryptionMode:
    """Return one supported runner user-data encryption mode."""
    if not isinstance(value, str):
        raise TypeError("user_data_encryption must be a string")
    if value == "disabled":
        return "disabled"
    if value == "required":
        return "required"
    raise ValueError("user_data_encryption must be disabled or required")

def _derive_worker_secrets(data_protection_key: bytes) -> tuple[str, str]:
    """Derive stable domain-separated application and field-encryption secrets."""
    if not isinstance(data_protection_key, bytes):
        raise TypeError("data_protection_key must be bytes")

```

```

if len(data_protection_key) != _X25519_PRIVATE_KEY_LENGTH:
    raise ValueError("data_protection_key must contain exactly 32 bytes")
key_material = HKDF(
    algorithm=hashes.SHA256(),
    length=64,
    salt=b"yinshi-runner-worker-storage-v1",
    info=b"runner-local database and field keys",
).derive(data_protection_key)
secret_key = base64.urlsafe_b64encode(key_material[:32]).decode("ascii")
encryption_pepper = key_material[32:].hex()
assert len(secret_key) >= 32
assert len(encryption_pepper) == 64
return secret_key, encryption_pepper


def _derive_worker_bearer_root(data_protection_key: bytes) -> bytes:
    """Derive a domain-separated root from the persistent storage key."""
    if len(data_protection_key) != _X25519_PRIVATE_KEY_LENGTH:
        raise ValueError("data_protection_key must contain exactly 32 bytes")
    bearer_root = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=b"yinshi-runner-worker-bearer-root-v1",
        info=b"runner-local account bearer derivation",
    ).derive(data_protection_key)
    assert len(bearer_root) == 32
    return bearer_root


def _prepare_owner_directory(path: Path) -> None:
    """Create one real owner-only runner directory or reject unsafe storage."""
    path.mkdir(mode=0o700, parents=True, exist_ok=True)
    metadata = path.lstat()
    if not stat.S_ISDIR(metadata.st_mode) or path.is_symlink():
        raise RuntimeError("runner worker data directory must be a real directory")
    if metadata.st_uid != os.getuid():
        raise RuntimeError("runner worker data directory must be owned by the runner user")
    if stat.S_IMODE(metadata.st_mode) != 0o700:
        raise RuntimeError("runner worker data directory must have owner-only permissions")


def _bind_runner_account(binding_path: Path, user_id: str) -> None:
    """Persist an opaque owner-only account binding and compare it on restart."""
    expected_digest = hashlib.sha256(user_id.encode("utf-8")).hexdigest().encode("ascii")
    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL
    flags |= getattr(os, "O_NOFOLLOW", 0)

```

```

try:
    file_descriptor = os.open(binding_path, flags, 0o600)
except FileNotFoundError:
    metadata = binding_path.lstat()
    if not stat.S_ISREG(metadata.st_mode) or binding_path.is_symlink():
        raise RuntimeError("runner account binding must be a real file")
    if metadata.st_uid != os.geteuid() or stat.S_IMODE(metadata.st_mode) != 0o600:
        raise RuntimeError("runner account binding must be owner-only")
    stored_digest = binding_path.read_bytes()
    if len(stored_digest) != len(expected_digest):
        raise RuntimeError("runner account binding is corrupt")
    if not secrets.compare_digest(stored_digest, expected_digest):
        raise ValueError("runner worker cannot accept a different account")
    return

try:
    written_bytes = 0
    while written_bytes < len(expected_digest):
        chunk_bytes = os.write(file_descriptor, expected_digest[written_bytes:])
        if chunk_bytes <= 0:
            raise RuntimeError("runner account binding write made no progress")
        written_bytes += chunk_bytes
    os.fsync(file_descriptor)
finally:
    os.close(file_descriptor)
metadata = binding_path.stat()
if metadata.st_size != len(expected_digest):
    raise RuntimeError("runner account binding write was incomplete")

def _recover_interrupted_prompt_runs(tenant: TenantContext) -> None:
    """Fail orphaned journal runs closed after a runner process restart."""
    event_json = json.dumps(
        {"type": "error", "error": "Prompt run was interrupted"},
        separators=(",", ":"),
        sort_keys=True,
    )
    with get_user_db(tenant) as database:
        database.execute("BEGIN IMMEDIATE")
        try:
            rows = database.execute("""SELECT id, session_id FROM prompt_runs
                                     WHERE status IN ('starting', 'running', 'stopping')""").fetchall()
            for row in rows:
                sequence_row = database.execute(
                    """SELECT COALESCE(MAX(sequence), -1) + 1 AS next_sequence
                       FROM prompt_events WHERE run_id = ?""",

```

```

        (row["id"],),
    ).fetchone()
    if sequence_row is None or type(sequence_row["next_sequence"]) is not int:
        raise RuntimeError("prompt journal recovery sequence is invalid")
    if sequence_row["next_sequence"] < _PROMPT_EVENT_COUNT_MAX:
        database.execute(
            """INSERT INTO prompt_events (run_id, sequence, event_json)
            VALUES (?, ?, ?)""",
            (row["id"], sequence_row["next_sequence"], event_json),
        )
        database.execute(
            "UPDATE sessions SET status = 'idle' WHERE id = ? AND status = 'running'"
            (row["session_id"],),
        )
    database.execute("""UPDATE prompt_runs
        SET status = 'interrupted', updated_at = CURRENT_TIMESTAMP
        WHERE status IN ('starting', 'running', 'stopping')""")
    database.commit()
except (RuntimeError, sqlite3.Error):
    database.rollback()
    raise

```

```

class RunnerWorkerManager:

```

```

    """Create one account-bound worker app with stable encrypted local stores."""

```

```

    def __init__(
        self,
        *,
        data_directory: Path,
        data_protection_key: bytes,
        database_directory: Path | None = None,
        user_data_directory: Path | None = None,
        user_data_encryption: RunnerUserDataEncryptionMode = "disabled",
        environment_setter: EnvironmentSetter | None = None,
    ) -> None:
        if not isinstance(data_directory, Path):
            raise TypeError("data_directory must be a pathlib.Path")
        if not data_directory.is_absolute():
            raise ValueError("runner worker data directory must be absolute")
        validated_user_data_encryption = validate_user_data_encryption_mode(user_data_encryption)
        storage_key = bytes(data_protection_key)
        secret_key, encryption_pepper = _derive_worker_secrets(storage_key)
        set_environment = environment_setter or os.environ.__setitem__
        if not callable(set_environment):
            raise TypeError("environment_setter must be callable")

```

```

selected_database_directory = database_directory or data_directory
selected_user_data_directory = user_data_directory or data_directory / "users"
for selected_path, name in (
    (selected_database_directory, "database_directory"),
    (selected_user_data_directory, "user_data_directory"),
):
    if not isinstance(selected_path, Path) or not selected_path.is_absolute():
        raise ValueError(f"runner worker {name} must be an absolute pathlib.Path")
_prepare_owner_directory(data_directory)
import_directory = selected_user_data_directory / "imports"
_prepare_owner_directory(selected_database_directory)
_prepare_owner_directory(selected_user_data_directory)
_prepare_owner_directory(import_directory)
environment = {
    "ALLOWED_REPO_BASE": str(import_directory),
    "CONTAINER_ENABLED": "false",
    "CONTROL_DB_PATH": str(selected_database_directory / "control.db"),
    "CONTROL_FIELD_ENCRYPTION": "required",
    "DB_PATH": str(selected_database_directory / "legacy.db"),
    "DISABLE_AUTH": "true",
    "ENCRYPTION_PEPPEER": encryption_pepper,
    "HOST": "127.0.0.1",
    "REQUIRE_HTTPS": "disabled",
    "SECRET_KEY": secret_key,
    "TENANT_DB_ENCRYPTION": "required",
    "TRUSTED_HOSTS": "localhost,127.0.0.1,[::1]",
    "USER_DATA_DIR": str(selected_user_data_directory),
    "USER_DATA_ENCRYPTION": validated_user_data_encryption,
}
for name, value in environment.items():
    set_environment(name, value)

from yinshi.config import get_settings
from yinshi.db import init_control_db, init_db

get_settings.cache_clear()
settings = get_settings()
if settings.control_db_path != environment["CONTROL_DB_PATH"]:
    raise RuntimeError("runner worker control database configuration did not apply")
if settings.user_data_dir != environment["USER_DATA_DIR"]:
    raise RuntimeError("runner worker user data configuration did not apply")
if settings.user_data_encryption_mode != validated_user_data_encryption:
    raise RuntimeError("runner worker user data encryption configuration did not apply")
init_control_db()
init_db()

```

```

self._data_directory = data_directory
self._database_directory = selected_database_directory
self._user_data_directory = selected_user_data_directory
self._account_binding_path = data_directory / "account.binding"
self._bearer_root = _derive_worker_bearer_root(storage_key)
self._account_id: str | None = None
self._dispatcher: WorkerHttpDispatcher | None = None

def dispatcher(self, user_id: str) -> WorkerHttpDispatcher:
    """Return the sole account dispatcher, rejecting cross-account capabilities."""
    if not isinstance(user_id, str) or not user_id or len(user_id) > 256:
        raise ValueError("runner worker user_id has an invalid length")
    if self._account_id is not None and not secrets.compare_digest(
        self._account_id,
        user_id,
    ):
        raise ValueError("runner worker cannot accept a different account")
    if self._dispatcher is not None:
        return self._dispatcher

    _bind_runner_account(self._account_binding_path, user_id)
    account_directory_name = hashlib.sha256(user_id.encode("utf-8")).hexdigest()
    account_data_prefix = self._user_data_directory / account_directory_name[:2]
    account_directory = account_data_prefix / account_directory_name
    _prepare_owner_directory(account_data_prefix)
    _prepare_owner_directory(account_directory)
    database_users_directory = self._database_directory / "users"
    account_database_prefix = database_users_directory / account_directory_name[:2]
    account_database_directory = account_database_prefix / account_directory_name
    _prepare_owner_directory(database_users_directory)
    _prepare_owner_directory(account_database_prefix)
    _prepare_owner_directory(account_database_directory)
    database_path = account_database_directory / "yinshi.db"
    bearer_key = HKDF(
        algorithm=hashes.SHA256(),
        length=32,
        salt=b"yinshi-runner-worker-bearer-v1",
        info=user_id.encode("utf-8"),
    ).derive(self._bearer_root)
    bearer_token = base64.urlsafe_b64encode(bearer_key).rstrip(b"=").decode("ascii")
    principal = WorkerPrincipal(
        tenant=TenantContext(
            user_id=user_id,
            email=f"{account_directory_name[:16]}@runner.invalid",
            data_dir=str(account_directory),

```



```

        db_path=str(database_path),
    ),
    bearer_token=bearer_token,
    database_root=str(self._database_directory),
)

from yinshi.db import get_control_db
from yinshi.main import create_app

with get_control_db() as control_database:
    control_database.execute(
        """
        INSERT INTO users (id, email, status)
        VALUES (?, ?, 'active')
        ON CONFLICT(id) DO NOTHING
        """,
        (user_id, principal.tenant.email),
    )
    control_database.commit()
    user_row = control_database.execute(
        "SELECT email, status FROM users WHERE id = ?",
        (user_id,),
    ).fetchone()
    if user_row is None or user_row["email"] != principal.tenant.email:
        raise RuntimeError("runner worker account metadata conflicts with local storage")
    if user_row["status"] != "active":
        raise RuntimeError("runner worker account is not active")

prepare_worker_principal_storage(principal)
init_user_db(str(database_path), tenant=principal.tenant)
_recover_interrupted_prompt_runs(principal.tenant)
worker_app = create_app(mode="worker", worker_principal=principal)
worker_app.state.container_manager = None
dispatcher = WorkerHttpDispatcher(app=worker_app, principal=principal)
self._account_id = user_id
self._dispatcher = dispatcher
return dispatcher

async def quiesce(self, job_id: str) -> None:
    """Close worker resources and checkpoint all runner-owned databases."""
    try:
        normalized_job_id = str(uuid.UUID(job_id))
    except (AttributeError, ValueError) as exc:
        raise ValueError("job_id must be a UUID") from exc
    if normalized_job_id != job_id:
        raise ValueError("job_id must be canonical")

```

```

dispatcher = self._dispatcher
if dispatcher is None:
    return
prompt_journal = dispatcher.app.state.prompt_journal
terminal_journal = dispatcher.app.state.terminal_journal
await prompt_journal.close()
await terminal_journal.close_all()
from yinshi.db import get_control_db

for connection_factory in (
    get_control_db,
    lambda: get_user_db(dispatcher.tenant),
):
    with connection_factory() as database:
        result = database.execute("PRAGMA wal_checkpoint(TRUNCATE)").fetchone()
        if result is None or len(result) != 3 or result[0] != 0:
            raise RuntimeError("runner database checkpoint failed")

```

17.7 yinshi/services/encrypted_uploads.py

Encrypted upload records bind ciphertext limits, content digests, expiry, and one-time consumption to a runtime capability.

"""Owner-scoped in-memory assembly for encrypted runner file uploads."""

```

from __future__ import annotations

import hashlib
import time
import uuid
from collections.abc import Callable
from dataclasses import dataclass, field

_UPLOAD_BYTES_MAX = 50 * 1024 * 1024
_UPLOAD_BYTES_RESERVED_MAX = 100 * 1024 * 1024
_UPLOAD_COUNT_MAX = 8
_UPLOAD_CHUNK_BYTES_MAX = 24_000
_UPLOAD_LIFETIME_SECONDS = 15 * 60

@dataclass(frozen=True, slots=True)
class EncryptedUpload:
    id: str
    purpose: str
    filename: str
    size_bytes: int

```

```

        next_chunk_index: int

@dataclass(frozen=True, slots=True)
class CompletedEncryptedUpload:
    purpose: str
    filename: str
    data: bytes

@dataclass(slots=True)
class _UploadState:
    id: str
    user_id: str
    purpose: str
    filename: str
    size_bytes: int
    sha256_hex: str
    expires_at: float
    data: bytearray = field(default_factory=bytearray)
    chunk_digests: list[tuple[int, str]] = field(default_factory=list)

class EncryptedUploadManager:
    """Assemble retry-safe chunks only after Noise has decrypted each request."""

    def __init__(self, *, clock: Callable[[], float] = time.monotonic) -> None:
        if not callable(clock):
            raise TypeError("clock must be callable")
        self._clock = clock
        self._uploads: dict[str, _UploadState] = {}

    def start(
        self,
        *,
        user_id: str,
        purpose: str,
        filename: str,
        size_bytes: int,
        sha256_hex: str,
    ) -> EncryptedUpload:
        """Reserve one bounded upload after validating immutable metadata."""
        self._validate_user_id(user_id)
        self._validate_metadata(purpose, filename, size_bytes, sha256_hex)
        self._reap_expired()
        if len(self._uploads) >= _UPLOAD_COUNT_MAX:

```

```

        raise RuntimeError("encrypted upload count limit reached")
    reserved_bytes = sum(upload.size_bytes for upload in self._uploads.values())
    if reserved_bytes + size_bytes > _UPLOAD_BYTES_RESERVED_MAX:
        raise RuntimeError("encrypted upload byte limit reached")
    upload_id = uuid.uuid4().hex
    if upload_id in self._uploads:
        raise RuntimeError("encrypted upload ID collision")
    state = _UploadState(
        id=upload_id,
        user_id=user_id,
        purpose=purpose,
        filename=filename,
        size_bytes=size_bytes,
        sha256_hex=sha256_hex,
        expires_at=self._clock() + _UPLOAD_LIFETIME_SECONDS,
    )
    self._uploads[upload_id] = state
    return self._public_upload(state)

def append(
    self,
    *,
    user_id: str,
    upload_id: str,
    chunk_index: int,
    chunk: bytes,
) -> EncryptedUpload:
    """Append the next chunk or accept an exact digest-matching retry."""
    state = self._owned_upload(user_id, upload_id)
    if type(chunk_index) is not int or chunk_index < 0:
        raise ValueError("encrypted upload chunk_index must be non-negative")
    if not isinstance(chunk, bytes) or not chunk or len(chunk) > _UPLOAD_CHUNK_BYTES_MAX:
        raise ValueError("encrypted upload chunk has an invalid length")
    chunk_digest = hashlib.sha256(chunk).hexdigest()
    next_chunk_index = len(state.chunk_digests)
    if chunk_index < next_chunk_index:
        stored_length, stored_digest = state.chunk_digests[chunk_index]
        if stored_length != len(chunk) or stored_digest != chunk_digest:
            raise ValueError("encrypted upload retry does not match stored chunk")
        return self._public_upload(state)
    if chunk_index != next_chunk_index:
        raise ValueError("encrypted upload chunk sequence is not contiguous")
    if len(state.data) + len(chunk) > state.size_bytes:
        raise ValueError("encrypted upload exceeds declared size")
    state.data.extend(chunk)
    state.chunk_digests.append((len(chunk), chunk_digest))

```

```

state.expires_at = self._clock() + _UPLOAD_LIFETIME_SECONDS
return self._public_upload(state)

def complete(self, *, user_id: str, upload_id: str) -> CompletedEncryptedUpload:
    """Consume one upload only when size and SHA-256 match declared metadata."""
    state = self._owned_upload(user_id, upload_id)
    self._uploads.pop(upload_id, None)
    if len(state.data) != state.size_bytes:
        raise ValueError("encrypted upload size does not match declaration")
    if hashlib.sha256(state.data).hexdigest() != state.sha256_hex:
        state.data[:] = b"\x00" * len(state.data)
        raise ValueError("encrypted upload digest does not match declaration")
    data = bytes(state.data)
    state.data[:] = b"\x00" * len(state.data)
    return CompletedEncryptedUpload(
        purpose=state.purpose,
        filename=state.filename,
        data=data,
    )

def cancel(self, *, user_id: str, upload_id: str) -> None:
    """Discard one upload idempotently without exposing another owner's state."""
    self._validate_user_id(user_id)
    self._validate_upload_id(upload_id)
    state = self._uploads.get(upload_id)
    if state is None:
        return
    if state.user_id != user_id:
        raise LookupError("encrypted upload not found")
    self._uploads.pop(upload_id, None)
    state.data[:] = b"\x00" * len(state.data)

def _owned_upload(self, user_id: str, upload_id: str) -> _UploadState:
    self._validate_user_id(user_id)
    self._validate_upload_id(upload_id)
    self._reap_expired()
    state = self._uploads.get(upload_id)
    if state is None or state.user_id != user_id:
        raise LookupError("encrypted upload not found")
    return state

def _reap_expired(self) -> None:
    current_time = self._clock()
    expired_ids = [
        upload_id
        for upload_id, state in self._uploads.items()

```

```

        if state.expires_at <= current_time
    ]
    for upload_id in expired_ids:
        state = self._uploads.pop(upload_id)
        state.data[:] = b"\x00" * len(state.data)

    @staticmethod
    def _public_upload(state: _UploadState) -> EncryptedUpload:
        return EncryptedUpload(
            id=state.id,
            purpose=state.purpose,
            filename=state.filename,
            size_bytes=state.size_bytes,
            next_chunk_index=len(state.chunk_digests),
        )

    @staticmethod
    def _validate_metadata(
        purpose: str,
        filename: str,
        size_bytes: int,
        sha256_hex: str,
    ) -> None:
        if purpose != "pi_config":
            raise ValueError("encrypted upload purpose is unsupported")
        if (
            not isinstance(filename, str)
            or not filename
            or len(filename) > 255
            or filename != filename.strip()
            or "/" in filename
            or "\\" in filename
            or not filename.lower().endswith(".zip")
        ):
            raise ValueError("encrypted upload filename must be a simple zip name")
        if type(size_bytes) is not int or not 1 <= size_bytes <= _UPLOAD_BYTES_MAX:
            raise ValueError("encrypted upload size is invalid")
        if (
            not isinstance(sha256_hex, str)
            or len(sha256_hex) != 64
            or any(character not in "0123456789abcdef" for character in sha256_hex)
        ):
            raise ValueError("encrypted upload SHA-256 is invalid")

    @staticmethod
    def _validate_user_id(user_id: str) -> None:

```

```

        if not isinstance(user_id, str) or not user_id or len(user_id) > 256:
            raise ValueError("encrypted upload user_id is invalid")

    @staticmethod
    def _validate_upload_id(upload_id: str) -> None:
        if (
            not isinstance(upload_id, str)
            or len(upload_id) != 32
            or any(character not in "0123456789abcdef" for character in upload_id)
        ):
            raise ValueError("encrypted upload ID is invalid")

```

17.8 yinshi/services/prompt_journal.py

The prompt journal gives each run ordered events, resumable cursors, terminal status, and bounded retention.

"""Durable prompt-event journal used by reconnectable runtime transports."""

```

from __future__ import annotations

import asyncio
import json
import sqlite3
import uuid
from collections.abc import AsyncIterator, Callable
from contextlib import suppress
from dataclasses import dataclass
from pathlib import Path
from typing import Any

from fastapi import Request

from yinshi.api.deps import get_db_for_request
from yinshi.config import get_settings
from yinshi.services.run_coordinator import get_run_coordinator

PromptExecutor = Callable[[Request, str, Any], AsyncIterator[dict[str, Any]]]
_EVENT_BYTES_MAX = 1_048_576
_EVENT_BATCH_BYTES_MAX = 48_000
_EVENT_COUNT_MAX = 100_000
_EVENT_BATCH_MAX = 100
_ACTIVE_STATUSES = frozenset({"starting", "running", "stopping"})
_TERMINAL_STATUSES = frozenset({"completed", "failed", "cancelled", "interrupted"})
_RESOURCE_ID_LENGTH = 32

```

```

class PromptRunNotFoundError(LookupError):
    """Requested session or prompt run does not exist in the active tenant."""

class PromptRunConflictError(RuntimeError):
    """Session already owns a different active prompt run."""

@dataclass(frozen=True, slots=True)
class PromptRun:
    """Public durable state for one prompt execution."""

    id: str
    session_id: str
    status: str

@dataclass(frozen=True, slots=True)
class PromptEventBatch:
    """Ordered journal events and next reconnect cursor."""

    run_id: str
    status: str
    events: tuple[dict[str, Any], ...]
    next_sequence: int

async def _default_prompt_executor(
    request: Request,
    session_id: str,
    body: Any,
) -> AsyncIterator[dict[str, Any]]:
    """Adapt the existing SSE route generator to structured journal events."""
    from yinshi.api.stream import prompt_session

    response = await prompt_session(session_id, body, request)
    body_iterator = response.body_iterator
    buffer = ""
    try:
        async for chunk in body_iterator:
            if isinstance(chunk, str):
                text = chunk
            elif isinstance(chunk, bytes):
                text = chunk.decode("utf-8", errors="strict")
            else:

```



```

        raise TypeError("prompt stream yielded an invalid chunk type")
    buffer += text
    while "\n\n" in buffer:
        frame, buffer = buffer.split("\n\n", maxsplit=1)
        data_lines = [line[6:] for line in frame.splitlines() if line.startswith("da
        if len(data_lines) != 1:
            raise ValueError("prompt stream yielded an invalid SSE frame")
        event = json.loads(data_lines[0])
        if not isinstance(event, dict):
            raise ValueError("prompt stream event must be an object")
        yield event
    if buffer.strip():
        raise ValueError("prompt stream ended with an incomplete SSE frame")
finally:
    close_body_iterator = getattr(body_iterator, "aclose", None)
    if close_body_iterator is not None:
        await close_body_iterator()

```

```

class PromptJournal:

```

```

    """Run prompts in background tasks while durably journaling ordered events."""

```

```

    def __init__(self, *, executor: PromptExecutor | None = None) -> None:
        selected_executor = executor or _default_prompt_executor
        if not callable(selected_executor):
            raise TypeError("prompt executor must be callable")
        self._executor = selected_executor
        self._tasks: dict[str, asyncio.Task[None]] = {}
        self._tasks_lock = asyncio.Lock()
        self._start_lock = asyncio.Lock()
        self._recovery_lock = asyncio.Lock()
        self._recovered_database_paths: set[str] = set()
        self._closing = False

```

```

    async def start(
        self,
        *,
        request: Request,
        session_id: str,
        idempotency_key: str,
        body: Any,
    ) -> PromptRun:
        """Create one durable run or return its idempotent predecessor."""
        self._validate_resource_id(session_id, "session_id")
        self._validate_idempotency_key(idempotency_key)
        if self._closing:

```

```

        raise RuntimeError("prompt journal is closing")
    if body is None:
        raise TypeError("prompt body must not be None")
    await self._recover_database(request)
    async with self._start_lock:
        if self._closing:
            raise RuntimeError("prompt journal is closing")
        return await self._start_serialized(
            request=request,
            session_id=session_id,
            idempotency_key=idempotency_key,
            body=body,
        )

    async def _start_serialized(
        self,
        *,
        request: Request,
        session_id: str,
        idempotency_key: str,
        body: Any,
    ) -> PromptRun:
        """Commit and register one prompt task while concurrent starts are excluded."""
        with get_db_for_request(request) as database:
            existing = database.execute(
                """SELECT id, session_id, status FROM prompt_runs
                WHERE session_id = ? AND idempotency_key = ?""",
                (session_id, idempotency_key),
            ).fetchone()
            if existing is not None:
                return self._row_to_run(existing)
            session = database.execute(
                "SELECT id FROM sessions WHERE id = ?",
                (session_id,),
            ).fetchone()
            if session is None:
                raise PromptRunNotFoundError("session not found")
            run_id = uuid.uuid4().hex
            try:
                database.execute(
                    """INSERT INTO prompt_runs
                    (id, session_id, idempotency_key, status)
                    VALUES (?, ?, ?, 'starting')""",
                    (run_id, session_id, idempotency_key),
                )
            database.commit()

```

```

except sqlite3.IntegrityError as exc:
    database.rollback()
    existing = database.execute(
        """SELECT id, session_id, status FROM prompt_runs
           WHERE session_id = ? AND idempotency_key = ?""",
        (session_id, idempotency_key),
    ).fetchone()
    if existing is not None:
        return self._row_to_run(existing)
    active = database.execute(
        """SELECT id FROM prompt_runs
           WHERE session_id = ?
              AND status IN ('starting', 'running', 'stopping')""",
        (session_id,)),
    ).fetchone()
    if active is not None:
        raise PromptRunConflictError(
            "session already has an active prompt run"
        ) from exc
    raise

task = asyncio.create_task(
    self._consume(request=request, session_id=session_id, run_id=run_id, body=body),
    name=f"prompt-journal-{run_id}",
)
async with self._tasks_lock:
    previous_task = self._tasks.setdefault(run_id, task)
    assert previous_task is task, "new run ID must not collide with an active task"
    return PromptRun(id=run_id, session_id=session_id, status="starting")

async def events(
    self,
    *,
    request: Request,
    session_id: str,
    run_id: str,
    next_sequence: int,
) -> PromptEventBatch:
    """Read one bounded ordered event page from an exact reconnect cursor."""
    self._validate_resource_id(session_id, "session_id")
    self._validate_resource_id(run_id, "run_id")
    if type(next_sequence) is not int or next_sequence < 0:
        raise ValueError("next_sequence must be a non-negative integer")
    await self._recover_database(request)

    with get_db_for_request(request) as database:

```

```

run = database.execute(
    "SELECT id, status FROM prompt_runs WHERE id = ? AND session_id = ?",
    (run_id, session_id),
).fetchone()
if run is None:
    raise PromptRunNotFoundError("prompt run not found")
rows = database.execute(
    """SELECT sequence, event_json FROM prompt_events
    WHERE run_id = ? AND sequence >= ?
    ORDER BY sequence ASC LIMIT ?""",
    (run_id, next_sequence, _EVENT_BATCH_MAX),
)
events: list[dict[str, Any]] = []
cursor = next_sequence
event_bytes = 0
for row in rows:
    sequence = row["sequence"]
    if type(sequence) is not int or sequence != cursor:
        raise RuntimeError("prompt journal sequence is not contiguous")
    serialized_event = row["event_json"]
    if not isinstance(serialized_event, str):
        raise RuntimeError("prompt journal event JSON is invalid")
    serialized_bytes = len(serialized_event.encode("utf-8"))
    if events and event_bytes + serialized_bytes > _EVENT_BATCH_BYTES_MAX:
        break
    event = json.loads(serialized_event)
    if not isinstance(event, dict):
        raise RuntimeError("prompt journal event is not an object")
    events.append(event)
    event_bytes += serialized_bytes
    cursor += 1
status = run["status"]

return PromptEventBatch(
    run_id=run_id,
    status=status,
    events=tuple(events),
    next_sequence=cursor,
)

async def cancel(
    self,
    *,
    request: Request,
    session_id: str,
    run_id: str,

```

```

) -> PromptRun:
    """Request cancellation once and return stable terminal state on retries."""
    self._validate_resource_id(session_id, "session_id")
    self._validate_resource_id(run_id, "run_id")
    await self._recover_database(request)
    with get_db_for_request(request) as database:
        row = database.execute(
            "SELECT id, session_id, status FROM prompt_runs WHERE id = ? AND session_id = ?"
            (run_id, session_id),
        ).fetchone()
        if row is None:
            raise PromptRunNotFoundError("prompt run not found")
        run = self._row_to_run(row)
        if run.status in _TERMINAL_STATUSES or run.status == "stopping":
            return run
        if run.status not in _ACTIVE_STATUSES:
            raise RuntimeError("prompt run has an invalid status")
        database.execute(
            """UPDATE prompt_runs SET status = 'stopping', updated_at = CURRENT_TIMESTAMP
            WHERE id = ?""",
            (run_id,),
        )
        database.commit()

    cancellation_found = await get_run_coordinator().request_cancel(session_id)
    if not cancellation_found:
        async with self._tasks_lock:
            task = self._tasks.get(run_id)
            if task is not None and not task.done():
                task.cancel()
                with suppress(asyncio.CancelledError):
                    await task
        await self._append_terminal_event_safely(
            request,
            run_id,
            {"type": "cancelled", "reason": "user_stop"},
        )
        self._set_terminal_status(request, run_id, "cancelled")
    return await self._load_run(request=request, session_id=session_id, run_id=run_id)

async def close(self) -> None:
    """Cancel active tasks and wait for their cleanup before app shutdown."""
    async with self._start_lock:
        async with self._tasks_lock:
            self._closing = True
            tasks = tuple(self._tasks.values())

```

```

        for task in tasks:
            if not task.done():
                task.cancel()

        try:
            await asyncio.gather(*tasks, return_exceptions=True)
        finally:
            async with self._tasks_lock:
                self._tasks.clear()

    async def _recover_database(self, request: Request) -> None:
        """Mark active rows from a previous process as interrupted once per tenant DB."""
        database_path = self._database_path(request)
        async with self._recovery_lock:
            if database_path in self._recovered_database_paths:
                return
            event_json = json.dumps(
                {"type": "error", "error": "Prompt run was interrupted"},
                separators=(",", ":"),
                sort_keys=True,
            )
            with get_db_for_request(request) as database:
                database.execute("BEGIN IMMEDIATE")
                try:
                    rows = database.execute("""SELECT id, session_id FROM prompt_runs
                                            WHERE status IN ('starting', 'running', 'stopping')""").fetchall()
                    for row in rows:
                        sequence_row = database.execute(
                            """SELECT COALESCE(MAX(sequence), -1) + 1 AS next_sequence
                                FROM prompt_events WHERE run_id = ?""",
                            (row["id"],),
                        ).fetchone()
                        if sequence_row is None or type(sequence_row["next_sequence"]) is not int:
                            raise RuntimeError("prompt journal recovery sequence is invalid")
                        if sequence_row["next_sequence"] < _EVENT_COUNT_MAX:
                            database.execute(
                                """INSERT INTO prompt_events (run_id, sequence, event_json)
                                    VALUES (?, ?, ?)""",
                                (row["id"], sequence_row["next_sequence"], event_json),
                            )
                        database.execute(
                            "UPDATE sessions SET status = 'idle' WHERE id = ? AND status = 'running'",
                            (row["session_id"],),
                        )
                database.execute("""UPDATE prompt_runs
                                SET status = 'interrupted', updated_at = CURRENT_TIMESTAMP
                                WHERE status IN ('starting', 'running', 'stopping')""")

```

```

        database.commit()
    except (RuntimeError, sqlite3.Error):
        database.rollback()
        raise
    self._recovered_database_paths.add(database_path)

async def _rearm_database_recovery(self, request: Request) -> None:
    """Require orphan recovery again after a terminal status write fails."""
    database_path = self._database_path(request)
    async with self._recovery_lock:
        self._recovered_database_paths.discard(database_path)

async def _consume(
    self,
    *,
    request: Request,
    session_id: str,
    run_id: str,
    body: Any,
) -> None:
    final_status = "completed"
    event_source: AsyncIterator[dict[str, Any]] | None = None
    try:
        self._set_status(request, run_id, "running", expected={"starting"})
        event_source = self._executor(request, session_id, body)
        async for event in event_source:
            await self._append_event(request, run_id, event)
            event_type = event.get("type")
            if event_type == "cancelled":
                final_status = "cancelled"
            elif event_type == "error":
                final_status = "failed"
    except asyncio.CancelledError:
        if self._closing:
            final_status = "interrupted"
            event = {"type": "error", "error": "Prompt run was interrupted"}
        else:
            final_status = "cancelled"
            event = {"type": "cancelled", "reason": "user_stop"}
        await self._append_terminal_event_safely(request, run_id, event)
        raise
    except Exception:
        final_status = "failed"
        await self._append_terminal_event_safely(
            request,
            run_id,

```

```

        {"type": "error", "error": "Prompt execution failed"},
    )
    finally:
        try:
            if event_source is not None:
                close_event_source = getattr(event_source, "aclose", None)
                if close_event_source is not None:
                    with suppress(Exception):
                        await close_event_source()
        finally:
            try:
                try:
                    self._set_terminal_status(request, run_id, final_status)
                except Exception:
                    with suppress(BaseException):
                        await self._rearm_database_recovery(request)
                    raise
            finally:
                async with self._tasks_lock:
                    current_task = asyncio.current_task()
                    if self._tasks.get(run_id) is current_task:
                        self._tasks.pop(run_id, None)

    async def _append_terminal_event_safely(
        self,
        request: Request,
        run_id: str,
        event: dict[str, Any],
    ) -> None:
        """Best-effort terminal event persistence without blocking status cleanup."""
        try:
            await self._append_event(request, run_id, event, unless_terminal=True)
        except (PromptRunNotFoundError, RuntimeError, TypeError, ValueError, sqlite3.Error):
            return

    async def _append_event(
        self,
        request: Request,
        run_id: str,
        event: dict[str, Any],
        *,
        unless_terminal: bool = False,
    ) -> None:
        if not isinstance(event, dict) or not isinstance(event.get("type"), str):
            raise ValueError("prompt journal event must have a string type")
        serialized = json.dumps(event, separators=(",", ":"), sort_keys=True)

```



```

if len(serialized.encode("utf-8")) > _EVENT_BYTES_MAX:
    raise ValueError("prompt journal event exceeds the byte limit")
with get_db_for_request(request) as database:
    run = database.execute(
        "SELECT status FROM prompt_runs WHERE id = ?",
        (run_id,),
    ).fetchone()
    if run is None:
        raise PromptRunNotFoundError("prompt run not found")
    if unless_terminal and run["status"] in _TERMINAL_STATUSES:
        return
    sequence_row = database.execute(
        "SELECT MAX(sequence) AS sequence_max FROM prompt_events WHERE run_id = ?",
        (run_id,),
    ).fetchone()
    assert sequence_row is not None
    sequence_max = sequence_row["sequence_max"]
    if sequence_max is not None and type(sequence_max) is not int:
        raise RuntimeError("prompt journal sequence is invalid")
    sequence = 0 if sequence_max is None else sequence_max + 1
    if sequence >= _EVENT_COUNT_MAX:
        raise RuntimeError("prompt journal event count exceeded the limit")
    database.execute(
        "INSERT INTO prompt_events (run_id, sequence, event_json) VALUES (?, ?, ?)",
        (run_id, sequence, serialized),
    )
    database.execute(
        "UPDATE prompt_runs SET updated_at = CURRENT_TIMESTAMP WHERE id = ?",
        (run_id,),
    )
    database.commit()

def _set_status(
    self,
    request: Request,
    run_id: str,
    status: str,
    *,
    expected: set[str],
) -> None:
    with get_db_for_request(request) as database:
        placeholders = ",".join("?" for _ in expected)
        result = database.execute(
            f"""UPDATE prompt_runs SET status = ?, updated_at = CURRENT_TIMESTAMP
                WHERE id = ? AND status IN ({placeholders})""",
            (status, run_id, *sorted(expected)),
        )

```

```

        )
        database.commit()
    if result.rowcount != 1:
        raise RuntimeError("prompt run status transition was rejected")

def _set_terminal_status(self, request: Request, run_id: str, status: str) -> None:
    if status not in _TERMINAL_STATUSES:
        raise ValueError("prompt terminal status is invalid")
    with get_db_for_request(request) as database:
        database.execute(
            """UPDATE prompt_runs SET status = ?, updated_at = CURRENT_TIMESTAMP
               WHERE id = ? AND status IN ('starting', 'running', 'stopping')""",
            (status, run_id),
        )
        database.commit()

async def _load_run(
    self,
    *,
    request: Request,
    session_id: str,
    run_id: str,
) -> PromptRun:
    with get_db_for_request(request) as database:
        row = database.execute(
            "SELECT id, session_id, status FROM prompt_runs WHERE id = ? AND session_id = ?",
            (run_id, session_id),
        ).fetchone()
    if row is None:
        raise PromptRunNotFoundError("prompt run not found")
    return self._row_to_run(row)

@staticmethod
def _database_path(request: Request) -> str:
    tenant = getattr(request.state, "tenant", None)
    database_path = getattr(tenant, "db_path", None)
    if database_path is None:
        database_path = str(Path(get_settings().db_path).resolve())
    if not isinstance(database_path, str) or not database_path:
        raise RuntimeError("prompt journal request database is unavailable")
    return database_path

@staticmethod
def _row_to_run(row: sqlite3.Row) -> PromptRun:
    run = PromptRun(id=row["id"], session_id=row["session_id"], status=row["status"])
    if len(run.id) != _RESOURCE_ID_LENGTH or run.status not in (

```

```

        _ACTIVE_STATUSES | _TERMINAL_STATUSES
    ):
        raise RuntimeError("prompt run row is invalid")
    return run

    @staticmethod
    def _validate_resource_id(value: str, name: str) -> None:
        if not isinstance(value, str) or len(value) != _RESOURCE_ID_LENGTH:
            raise ValueError(f"{name} must contain exactly 32 characters")
        if any(character not in "0123456789abcdef" for character in value):
            raise ValueError(f"{name} must be lowercase hexadecimal")

    @staticmethod
    def _validate_idempotency_key(value: str) -> None:
        if not isinstance(value, str):
            raise TypeError("idempotency_key must be a string")
        try:
            normalized = str(uuid.UUID(value))
        except ValueError as exc:
            raise ValueError("idempotency_key must be a UUID") from exc
        if normalized != value:
            raise ValueError("idempotency_key must be canonical")

```

17.9 yinshi/services/relay__process__lock.py

A process lock prevents two runner agents from claiming the same persistent relay identity.

```

"""Host-local process lock for the in-memory opaque runner relay."""

```

```

from __future__ import annotations

```

```

import fcntl
import os
import stat
import threading
from dataclasses import dataclass
from pathlib import Path

```

```

@dataclass(slots=True)
class _HeldProcessLock:
    file_descriptor: int
    reference_count: int
    process_id: int

```

```

_REGISTRY_LOCK = threading.RLock()
_HELD_PROCESS_LOCKS: dict[Path, _HeldProcessLock] = {}

class RelayProcessLock:
    """Prevent unsupported multi-process relay routing on one deployment host."""

    def __init__(self, path: Path) -> None:
        if not isinstance(path, Path) or not path.is_absolute():
            raise ValueError("relay process lock path must be absolute")
        self._path = path
        self._acquired = False

    def acquire(self) -> None:
        """Acquire exclusive ownership without blocking another API process."""
        with _REGISTRY_LOCK:
            if self._acquired:
                return
            process_id = os.getpid()
            held_lock = _HELD_PROCESS_LOCKS.get(self._path)
            if held_lock is not None and held_lock.process_id == process_id:
                held_lock.reference_count += 1
                self._acquired = True
                return
            if held_lock is not None:
                # A fork inherits Python memory and file descriptors. The child must
                # discard that inherited descriptor before testing the kernel lock.
                os.close(held_lock.file_descriptor)
                del _HELD_PROCESS_LOCKS[self._path]

            file_descriptor = self._acquire_file_descriptor(process_id)
            _HELD_PROCESS_LOCKS[self._path] = _HeldProcessLock(
                file_descriptor=file_descriptor,
                reference_count=1,
                process_id=process_id,
            )
            self._acquired = True

    def _acquire_file_descriptor(self, process_id: int) -> int:
        """Open, validate, and lock the owner-controlled relay lock file."""
        self._path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
        flags = os.O_RDWR | os.O_CREAT | os.O_CLOEXEC | getattr(os, "O_NOFOLLOW", 0)
        file_descriptor = os.open(self._path, flags, 0o600)
        try:
            metadata = os.fstat(file_descriptor)

```

```

if not stat.S_ISREG(metadata.st_mode) or metadata.st_uid != os.geteuid():
    raise RuntimeError("relay process lock must be an owner-controlled file")
if stat.S_IMODE(metadata.st_mode) != 0o600:
    os.fchmod(file_descriptor, 0o600)
try:
    fcntl.flock(file_descriptor, fcntl.LOCK_EX | fcntl.LOCK_NB)
except BlockingIOError as exc:
    raise RuntimeError(
        "runner relay requires a single hosted process or affinity routing"
    ) from exc
os.ftruncate(file_descriptor, 0)
os.write(file_descriptor, str(process_id).encode("ascii"))
os.fsync(file_descriptor)
except (OSError, RuntimeError):
    os.close(file_descriptor)
    raise
return file_descriptor

def release(self) -> None:
    """Release process ownership during graceful application shutdown."""
    with _REGISTRY_LOCK:
        if not self._acquired:
            return
        self._acquired = False
        held_lock = _HELD_PROCESS_LOCKS.get(self._path)
        if held_lock is None or held_lock.process_id != os.getpid():
            return
        held_lock.reference_count -= 1
        assert held_lock.reference_count >= 0
        if held_lock.reference_count > 0:
            return
        del _HELD_PROCESS_LOCKS[self._path]
        try:
            fcntl.flock(held_lock.file_descriptor, fcntl.LOCK_UN)
        finally:
            os.close(held_lock.file_descriptor)

```

17.10 yinshi/services/repository_lifecycle.py

Repository lifecycle rules serialize clone, upgrade, removal, and worktree cleanup for tenant-owned checkouts.

```

"""Repository lifecycle locking and reversible managed-path moves."""

from __future__ import annotations

```

```

import asyncio
import errno
import fcntl
import hashlib
import os
import shutil
import sqlite3
import stat
import threading
import uuid
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager
from dataclasses import dataclass
from pathlib import Path

from yinshi.config import get_settings
from yinshi.tenant import TenantContext
from yinshi.utils.paths import is_path_inside

@dataclass(slots=True)
class _LockState:
    """Track one keyed lock while callers hold or await it."""

    lock: asyncio.Lock
    references: int = 0

@dataclass(frozen=True, slots=True)
class QuarantinedPath:
    """Record one reversible managed-path move."""

    source: Path
    target: Path
    run_directory: Path

_LOCK_STATES: dict[str, _LockState] = {}
_LOCK_STATES_GUARD = threading.Lock()
_LOCK_DIRECTORY = ".repository-lifecycle-locks"
_LOCK_POLL_INTERVAL_SECONDS = 0.05

def _validate_directory(path_stat: os.stat_result, *, private: bool) -> None:
    """Require an owner-controlled directory."""
    if not stat.S_ISDIR(path_stat.st_mode):

```

```

        raise ValueError("repository lock path must be a directory")
    if path_stat.st_uid != os.geteuid():
        raise ValueError("repository lock directory must be owned by this user")
    mode = stat.S_IMODE(path_stat.st_mode)
    if mode & 0o022 or private and mode != 0o700:
        raise ValueError("repository lock directory permissions are unsafe")

def _directory_open_flags() -> int:
    """Return secure flags for opening a directory descriptor."""
    flags = os.O_RDONLY
    flags |= getattr(os, "O_DIRECTORY", 0)
    flags |= getattr(os, "O_NOFOLLOW", 0)
    flags |= getattr(os, "O_CLOEXEC", 0)
    return flags

def _open_lock_file(lock_root: Path, repo_id: str) -> int:
    """Open a stable owner-only lock file below a trusted root."""
    if not lock_root.is_absolute():
        raise ValueError("repository lock root must be absolute")
    try:
        root_path_stat = os.lstat(lock_root)
    except OSError as exc:
        raise ValueError("repository lock root is unavailable") from exc
    if stat.S_ISLNK(root_path_stat.st_mode):
        raise ValueError("repository lock root must not be a link")
    _validate_directory(root_path_stat, private=False)

    root_fd = -1
    lock_directory_fd = -1
    try:
        root_fd = os.open(lock_root, _directory_open_flags())
        root_fd_stat = os.fstat(root_fd)
        if (root_fd_stat.st_dev, root_fd_stat.st_ino) != (
            root_path_stat.st_dev,
            root_path_stat.st_ino,
        ):
            raise ValueError("repository lock root changed during validation")
        _validate_directory(root_fd_stat, private=False)

    try:
        os.mkdir(_LOCK_DIRECTORY, mode=0o700, dir_fd=root_fd)
    except FileExistsError:
        pass
    lock_directory_stat = os.stat(

```

```

        _LOCK_DIRECTORY,
        dir_fd=root_fd,
        follow_symlinks=False,
    )
    if stat.S_ISLNK(lock_directory_stat.st_mode):
        raise ValueError("repository lock directory must not be a link")
    _validate_directory(lock_directory_stat, private=True)

    lock_directory_fd = os.open(
        _LOCK_DIRECTORY,
        _directory_open_flags(),
        dir_fd=root_fd,
    )
    opened_directory_stat = os.fstat(lock_directory_fd)
    if (opened_directory_stat.st_dev, opened_directory_stat.st_ino) != (
        lock_directory_stat.st_dev,
        lock_directory_stat.st_ino,
    ):
        raise ValueError("repository lock directory changed during validation")
    _validate_directory(opened_directory_stat, private=True)

    lock_name = f"{hashlib.sha256(repo_id.encode('utf-8')).hexdigest()}.lock"
    file_flags = os.O_RDWR | os.O_CREAT
    file_flags |= getattr(os, "O_NOFOLLOW", 0)
    file_flags |= getattr(os, "O_CLOEXEC", 0)
    lock_fd = os.open(lock_name, file_flags, 0o600, dir_fd=lock_directory_fd)
    try:
        lock_path_stat = os.stat(
            lock_name,
            dir_fd=lock_directory_fd,
            follow_symlinks=False,
        )
        lock_fd_stat = os.fstat(lock_fd)
        if stat.S_ISLNK(lock_path_stat.st_mode) or not stat.S_ISREG(lock_fd_stat.st_mode):
            raise ValueError("repository lock file must be a regular non-link file")
        if (lock_fd_stat.st_dev, lock_fd_stat.st_ino) != (
            lock_path_stat.st_dev,
            lock_path_stat.st_ino,
        ):
            raise ValueError("repository lock file changed during validation")
        if lock_fd_stat.st_uid != os.geteuid():
            raise ValueError("repository lock file must be owned by this user")
        if stat.S_IMODE(lock_fd_stat.st_mode) != 0o600:
            raise ValueError("repository lock file permissions are unsafe")
        return lock_fd
    except BaseException:

```



```

        os.close(lock_fd)
        raise
    except OSError as exc:
        raise ValueError("repository lock path validation failed") from exc
    finally:
        if lock_directory_fd >= 0:
            os.close(lock_directory_fd)
        if root_fd >= 0:
            os.close(root_fd)

def repository_lifecycle_root(
    db: sqlite3.Connection,
    tenant: TenantContext | None,
) -> Path:
    """Choose a shared lock root from the operation database and tenant."""
    application_root = Path(get_settings().db_path).expanduser().absolute().parent
    if tenant is None:
        return application_root
    database_row = db.execute("PRAGMA database_list").fetchone()
    if database_row is None or not database_row[2]:
        return application_root
    database_path = Path(str(database_row[2])).expanduser().absolute()
    if not is_path_inside(str(database_path), tenant.data_dir):
        return application_root
    return Path(tenant.data_dir)

async def _acquire_file_lock(lock_fd: int) -> None:
    """Acquire an advisory lock with cancellation-friendly polling."""
    while True:
        try:
            fcntl.flock(lock_fd, fcntl.LOCK_EX | fcntl.LOCK_NB)
            return
        except OSError as exc:
            if exc.errno not in {errno.EACCES, errno.EAGAIN}:
                raise
            await asyncio.sleep(_LOCK_POLL_INTERVAL_SECONDS)

@asynccontextmanager
async def repository_lifecycle(repo_id: str, lock_root: Path) -> AsyncIterator[None]:
    """Serialize lifecycle work for one repository across worker processes."""
    if not repo_id:
        raise ValueError("repo_id must not be empty")

```

```

state_key = f"{lock_root.absolute()}:{repo_id}"
with _LOCK_STATES_GUARD:
    state = _LOCK_STATES.setdefault(state_key, _LockState(lock=asyncio.Lock()))
    state.references += 1

acquired = False
lock_fd = -1
try:
    await state.lock.acquire()
    acquired = True
    lock_fd = _open_lock_file(lock_root, repo_id)
    await _acquire_file_lock(lock_fd)
    yield
finally:
    if lock_fd >= 0:
        os.close(lock_fd)
    if acquired:
        state.lock.release()
    with _LOCK_STATES_GUARD:
        state.references -= 1
        if state.references == 0 and not state.lock.locked():
            _LOCK_STATES.pop(state_key, None)

class ManagedPathQuarantine:
    """Move trusted managed paths aside until database deletion succeeds."""

    def __init__(self) -> None:
        self._token = uuid.uuid4().hex
        self._entries: list[QuarantinedPath] = []

    @property
    def entries(self) -> tuple[QuarantinedPath, ...]:
        """Return current moved-path records for cleanup reporting."""
        return tuple(self._entries)

    def validate(self, source: Path, trusted_root: Path) -> None:
        """Validate one existing source against its trusted owner-controlled root."""
        if not source.is_absolute() or not trusted_root.is_absolute():
            raise ValueError("managed deletion paths must be absolute")
        if not source.exists() and not source.is_symlink():
            return
        if source.is_symlink():
            raise ValueError("managed deletion source must not be a symlink")
        if trusted_root.is_symlink() or not trusted_root.is_dir():
            raise ValueError("managed deletion root must be a real directory")

```

```

        if not is_path_inside(str(source), str(trusted_root)):
            raise ValueError("managed deletion source is outside its trusted root")

    root_stat = trusted_root.stat()
    if root_stat.st_uid != os.geteuid():
        raise ValueError("managed deletion root must be owned by this user")

def move(self, source: Path, trusted_root: Path) -> None:
    """Move one validated source into an owner-only directory on its filesystem."""
    self.validate(source, trusted_root)
    if not source.exists():
        return

    parent = trusted_root / ".yinshi-delete-quarantine"
    run_directory = parent / self._token
    self._ensure_private_directory(parent)
    self._ensure_private_directory(run_directory)
    target = run_directory / f"{len(self._entries):04d}"
    if target.exists() or target.is_symlink():
        raise RuntimeError("managed deletion target already exists")

    os.rename(source, target)
    self._entries.append(
        QuarantinedPath(
            source=source,
            target=target,
            run_directory=run_directory,
        )
    )

def restore(self) -> None:
    """Restore every moved path in reverse order."""
    first_error: OSError | RuntimeError | ValueError | None = None
    for entry in reversed(self._entries.copy()):
        try:
            if entry.source.exists() or entry.source.is_symlink():
                raise RuntimeError("managed deletion source was recreated during rollback")
            if entry.target.exists() or entry.target.is_symlink():
                os.rename(entry.target, entry.source)
            self._entries.remove(entry)
            self._remove_empty_directories(entry.run_directory)
        except (OSError, RuntimeError, ValueError) as exc:
            if first_error is None:
                first_error = exc
    if first_error is not None:
        raise first_error

```

```

def discard(self) -> None:
    """Permanently remove every moved path after database commit."""
    first_error: OSError | RuntimeError | ValueError | None = None
    for entry in reversed(self._entries.copy()):
        try:
            self._remove_target(entry.target)
            self._entries.remove(entry)
            self._remove_empty_directories(entry.run_directory)
        except (OSError, RuntimeError, ValueError) as exc:
            if first_error is None:
                first_error = exc
    if first_error is not None:
        raise first_error

    @staticmethod
    def _ensure_private_directory(path: Path) -> None:
        """Create or validate one owner-only non-symlink directory."""
        if path.is_symlink():
            raise ValueError("managed deletion directory must not be a symlink")
        path.mkdir(mode=0o700, parents=True, exist_ok=True)
        path_stat = path.stat()
        if not stat.S_ISDIR(path_stat.st_mode):
            raise ValueError("managed deletion path must be a directory")
        if path_stat.st_uid != os.geteuid():
            raise ValueError("managed deletion directory must be owned by this user")
        if stat.S_IMODE(path_stat.st_mode) != 0o700:
            os.chmod(path, 0o700)

    @staticmethod
    def _remove_target(target: Path) -> None:
        """Remove one non-symlink target without following directory links."""
        if target.is_symlink():
            raise ValueError("managed deletion target must not be a symlink")
        if not target.exists():
            return
        if target.is_dir():
            if not shutil.rmtree.avoids_symlink_attacks:
                raise RuntimeError("safe descriptor-based directory deletion is unavailable")
            shutil.rmtree(target)
            return
        target.unlink()

    @staticmethod
    def _remove_empty_directories(run_directory: Path) -> None:
        """Remove empty per-run and shared quarantine directories."""

```

```

try:
    run_directory.rmdir()
except OSError:
    return
try:
    run_directory.parent.rmdir()
except OSError:
    return

```

17.11 yinshi/services/runner_agent_relay.py

This client maintains authenticated runner WebSocket state and translates connection loss into bounded retries.

```

"""Runner-side multiplexing for control messages and encrypted RPC frames."""

from __future__ import annotations

import json
import time
import uuid
from collections import OrderedDict
from collections.abc import Awaitable, Callable
from pathlib import Path
from typing import Protocol

from yinshi.services.runner_noise_session import (
    RunnerCapabilityReplayStore,
    RunnerNoiseSession,
)
from yinshi.services.runner_rpc import DispatcherFactory, EncryptedRunnerRpcSession

_UUID_BYTES_LENGTH = 16
_RELAY_FRAME_BYTES_MAX = 65_535
_ACTIVE_TRANSFERS_MAX = 32
_RETIRED_TRANSFERS_MAX = 128
_RETIRED_TRANSFER_TTL_SECONDS = 300.0
_WELCOME_KEYS = {"runner_id", "type"}
_TRANSFER_CONTROL_KEYS = {"transfer_id", "type"}
_MAINTENANCE_CONTROL_KEYS = {"job_id", "type"}
MaintenanceHandler = Callable[[str], Awaitable[None]]

class RunnerTaskLease(Protocol):
    """Task reference operations needed by managed relay transfers."""

```

```

    async def acquire(self) -> None:
        """Acquire one active-transfer reference."""
        ...

    async def release(self) -> None:
        """Release one active-transfer reference."""
        ...

class RunnerRelaySessionError(ValueError):
    """One known transfer failed without invalidating the shared runner socket."""

    def __init__(self, transfer_id: str) -> None:
        super().__init__("Runner relay transfer frame was rejected")
        self.transfer_id = transfer_id

def _canonical_uuid(value: object, name: str) -> str:
    """Return a canonical UUID string from one untrusted control value."""
    if not isinstance(value, str) or not value:
        raise ValueError(f"{name} must not be empty")
    try:
        normalized_value = str(uuid.UUID(value))
    except ValueError as exc:
        raise ValueError(f"{name} must be a UUID") from exc
    if normalized_value != value:
        raise ValueError(f"{name} must be canonical")
    return normalized_value

class RunnerAgentRelayRuntime:
    """Own connection-scoped runner identity and per-transfer encrypted sessions."""

    def __init__(
        self,
        *,
        runner_static_private_key: bytes,
        capability_signing_public_key: bytes,
        replay_database_path: Path,
        dispatcher_factory: DispatcherFactory | None = None,
        task_lease: RunnerTaskLease | None = None,
        maintenance_handler: MaintenanceHandler | None = None,
    ) -> None:
        if not isinstance(runner_static_private_key, bytes) or len(runner_static_private_key) != 32:
            raise ValueError("runner_static_private_key must contain exactly 32 bytes")
        if not isinstance(capability_signing_public_key, bytes):

```

```

        raise TypeError("capability_signing_public_key must be bytes")
    if len(capability_signing_public_key) != 32:
        raise ValueError("capability_signing_public_key must contain exactly 32 bytes")
    if not isinstance(replay_database_path, Path):
        raise TypeError("replay_database_path must be a pathlib.Path")
    if dispatcher_factory is not None and not callable(dispatcher_factory):
        raise TypeError("dispatcher_factory must be callable or None")
    if maintenance_handler is not None and not callable(maintenance_handler):
        raise TypeError("maintenance_handler must be callable or None")

    self._runner_static_private_key = bytes(runner_static_private_key)
    self._capability_signing_public_key = bytes(capability_signing_public_key)
    self._replay_store = RunnerCapabilityReplayStore(replay_database_path)
    self._dispatcher_factory = dispatcher_factory
    self._task_lease = task_lease
    self._maintenance_handler = maintenance_handler
    self._runner_id: str | None = None
    self._maintenance_job_id: str | None = None
    self._sessions: dict[str, EncryptedRunnerRpcSession] = {}
    self._retired_transfers: OrderedDict[str, float] = OrderedDict()

    @property
    def active_transfer_ids(self) -> tuple[str, ...]:
        """Return random routing IDs only, excluding capability and payload data."""
        return tuple(sorted(self._sessions))

    async def aclose(self) -> None:
        """Clear open transfers and release their managed task references once."""
        reference_count = len(self._sessions)
        self._sessions.clear()
        self._retired_transfers.clear()
        if self._task_lease is not None:
            for _ in range(reference_count):
                await self._task_lease.release()

    async def handle_control(self, message: str) -> str | None:
        """Apply one exact welcome, transfer, or maintenance control message."""
        if not isinstance(message, str) or not message or len(message) > 1_024:
            raise ValueError("Runner relay control message has an invalid length")
        try:
            payload = json.loads(message)
        except json.JSONDecodeError as exc:
            raise ValueError("Runner relay control message is not valid JSON") from exc
        if not isinstance(payload, dict):
            raise ValueError("Runner relay control message has an invalid shape")
        message_type = payload.get("type")

```

```

if message_type == "welcome":
    self._handle_welcome(payload)
    return None
if message_type in {"open", "close"}:
    await self._handle_transfer_control(payload, message_type=message_type)
    return None
if message_type == "quiesce":
    return await self._handle_maintenance_control(payload)
raise ValueError("Runner relay control message type is unsupported")

async def handle_binary(self, frame: bytes, *, current_time: int) -> bytes:
    """Route one UUID-prefixed ciphertext frame through its encrypted RPC session."""
    if not isinstance(frame, bytes):
        raise TypeError("Runner relay frame must be bytes")
    if not _UUID_BYTES_LENGTH < len(frame) <= _UUID_BYTES_LENGTH + _RELAY_FRAME_BYTES_MAX:
        raise ValueError("Runner relay frame has an invalid length")
    if type(current_time) is not int or current_time < 0:
        raise ValueError("current_time must be a non-negative integer")
    transfer_id = str(uuid.UUID(bytes=frame[:_UUID_BYTES_LENGTH]))
    session = self._sessions.get(transfer_id)
    if session is None:
        raise ValueError("Runner relay transfer is not open")
    try:
        response = await session.handle_frame(
            frame[_UUID_BYTES_LENGTH:],
            current_time=current_time,
        )
    except (RuntimeError, TypeError, ValueError) as exc:
        removed_session = self._sessions.pop(transfer_id, None)
        self._retire_transfer(transfer_id)
        if removed_session is not None and self._task_lease is not None:
            await self._task_lease.release()
        raise RunnerRelaySessionError(transfer_id) from exc
    return uuid.UUID(transfer_id).bytes + response

def _handle_welcome(self, payload: dict[str, object]) -> None:
    """Set runner identity exactly once from authenticated relay state."""
    if set(payload) != _WELCOME_KEYS:
        raise ValueError("Runner relay welcome message has an invalid shape")
    runner_id = payload.get("runner_id")
    if not isinstance(runner_id, str) or not runner_id:
        raise ValueError("Runner relay welcome runner_id must not be empty")
    if self._runner_id is not None:
        raise ValueError("Runner relay welcome was already received")
    self._runner_id = runner_id

```



```

async def _handle_transfer_control(
    self,
    payload: dict[str, object],
    *,
    message_type: str,
) -> None:
    """Open or close one bounded transfer after relay welcome."""
    if set(payload) != _TRANSFER_CONTROL_KEYS:
        raise ValueError("Runner relay transfer control has an invalid shape")
    if self._runner_id is None:
        raise ValueError("Runner relay welcome is required before transfers")
    if self._maintenance_job_id is not None:
        raise ValueError("Runner relay is in maintenance")
    transfer_id = _canonical_uuid(payload.get("transfer_id"), "transfer_id")
    if message_type == "close":
        if self._sessions.pop(transfer_id, None) is None:
            if self._is_retired_transfer(transfer_id):
                return
            raise ValueError("Runner relay transfer is not open")
        self._retire_transfer(transfer_id)
        if self._task_lease is not None:
            await self._task_lease.release()
        return
    if transfer_id in self._sessions:
        raise ValueError("Runner relay transfer is already open")
    self._retired_transfers.pop(transfer_id, None)
    if len(self._sessions) >= _ACTIVE_TRANSFERS_MAX:
        raise ValueError("Runner relay active transfer limit was reached")
    if self._task_lease is not None:
        await self._task_lease.acquire()
    try:
        noise_session = RunnerNoiseSession(
            runner_id=self._runner_id,
            runner_static_private_key=self._runner_static_private_key,
            capability_signing_public_key=self._capability_signing_public_key,
            replay_store=self._replay_store,
        )
        session = EncryptedRunnerRpcSession(
            transfer_id=transfer_id,
            noise_session=noise_session,
            dispatcher_factory=self._dispatcher_factory,
        )
    except Exception:
        if self._task_lease is not None:
            await self._task_lease.release()
        raise

```

```

self._sessions[transfer_id] = session

def _retire_transfer(self, transfer_id: str) -> None:
    """Retain one bounded marker for transfer-local teardown races."""
    now = time.monotonic()
    self._prune_retired_transfers(now=now)
    self._retired_transfers[transfer_id] = now
    self._retired_transfers.move_to_end(transfer_id)
    while len(self._retired_transfers) > _RETIRED_TRANSFERS_MAX:
        self._retired_transfers.popitem(last=False)

def _is_retired_transfer(self, transfer_id: str) -> bool:
    """Return whether this connection recently retired one known transfer."""
    self._prune_retired_transfers(now=time.monotonic())
    return transfer_id in self._retired_transfers

def _prune_retired_transfers(self, *, now: float) -> None:
    """Remove expired markers from their ordered prefix."""
    while self._retired_transfers:
        _, retired_at = next(iter(self._retired_transfers.items()))
        if now - retired_at <= _RETIRED_TRANSFER_TTL_SECONDS:
            return
        self._retired_transfers.popitem(last=False)

async def _handle_maintenance_control(self, payload: dict[str, object]) -> str:
    """Close transfer authority, drain workers, and acknowledge one job."""
    if set(payload) != _MAINTENANCE_CONTROL_KEYS:
        raise ValueError("Runner relay maintenance control has an invalid shape")
    if self._runner_id is None:
        raise ValueError("Runner relay welcome is required before maintenance")
    job_id = _canonical_uuid(payload.get("job_id"), "job_id")
    if self._maintenance_job_id is not None:
        if self._maintenance_job_id != job_id:
            raise ValueError("Runner relay is already in maintenance")
    return json.dumps(
        {"job_id": job_id, "type": "quiesced"},
        separators=(",", ":"),
        sort_keys=True,
    )
self._maintenance_job_id = job_id
try:
    reference_count = len(self._sessions)
    for transfer_id in self._sessions:
        self._retire_transfer(transfer_id)
    self._sessions.clear()
    if self._task_lease is not None:

```

```

        for _ in range(reference_count):
            await self._task_lease.release()
        if self._maintenance_handler is None:
            raise ValueError("Runner relay maintenance handler is unavailable")
        await self._maintenance_handler(job_id)
    except BaseException:
        self._maintenance_job_id = None
        raise
    return json.dumps(
        {"job_id": job_id, "type": "quiesced"},
        separators=(",", ":"),
        sort_keys=True,
    )

```

17.12 yinshi/services/runner_capabilities.py

Signed capabilities restrict each RPC request by tenant, runtime, method, expiry, and replay-resistant identifier.

"""Short-lived Ed25519 capabilities for end-to-end encrypted runner sessions."""

```

from __future__ import annotations

import base64
import binascii
import json
import secrets
import time
import uuid
from dataclasses import dataclass

from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric.ed25519 import (
    Ed25519PrivateKey,
    Ed25519PublicKey,
)
from cryptography.hazmat.primitives.asymmetric.x25519 import (
    X25519PrivateKey,
    X25519PublicKey,
)
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

from yinshi.config import get_settings

RUNNER_PROTOCOL_VERSION = "yinshi-runner-v1"

```

```

RUNNER_CAPABILITY_TTL_SECONDS = 300
RUNNER_FRAME_BYTES_MAX = 65_535
RUNNER_SESSION_BYTES_MIN = 65_536
RUNNER_SESSION_BYTES_MAX = 1_073_741_824
RUNNER_SCOPES = frozenset(
    {
        "files.read",
        "files.write",
        "pi.configure",
        "provider.configure",
        "repository.read",
        "repository.write",
        "session.read",
        "session.stream",
        "session.write",
        "terminal",
        "transfer.read",
        "transfer.write",
        "worker.health",
        "workspace.read",
        "workspace.write",
    }
)
_HEADER = {"alg": "EdDSA", "typ": "YINSHI-RUNNER-CAP", "v": 1}
_PAYLOAD_KEYS = {
    "aud",
    "exp",
    "iat",
    "initiator_key",
    "jti",
    "max_frame_bytes",
    "max_session_bytes",
    "protocol",
    "runner_id",
    "runner_key",
    "scopes",
    "sub",
    "transfer_id",
    "v",
}
_X25519_VALIDATION_PRIVATE_KEY = X25519PrivateKey.from_private_bytes(b"\x01" * 32)

@dataclass(frozen=True, slots=True)
class VerifiedRunnerCapability:
    """Validated dispatch authority consumed by one runner Noise session."""

```

```

user_id: str
runner_id: str
runner_public_key: str
initiator_public_key: str
transfer_id: str
token_id: str
scopes: tuple[str, ...]
issued_at: int
expires_at: int
max_frame_bytes: int
max_session_bytes: int

def _decode_base64url(value: str) -> bytes:
    """Strictly decode one canonical unpadded base64url value."""
    if not isinstance(value, str) or not value:
        raise ValueError("base64url value must not be empty")
    padding = "=" * (-len(value) % 4)
    try:
        decoded = base64.b64decode(
            f"{value}{padding}",
            altchars=b"._",
            validate=True,
        )
    except (binascii.Error, ValueError) as exc:
        raise ValueError("value is not valid base64url") from exc
    if _encode_base64url(decoded) != value:
        raise ValueError("value is not canonical base64url")
    return decoded

def _encode_base64url(value: bytes) -> str:
    """Encode non-empty bytes as canonical unpadded base64url."""
    if not isinstance(value, bytes):
        raise TypeError("value must be bytes")
    if not value:
        raise ValueError("value must not be empty")
    return base64.urlsafe_b64encode(value).rstrip(b"=").decode("ascii")

def _signing_key() -> Ed25519PrivateKey:
    """Derive the runner-capability key in a domain separate from other tokens."""
    secret = get_settings().secret_key.encode("utf-8")
    if len(secret) < 32:
        raise RuntimeError("runner capability signing requires a 32-byte session secret")

```

```

seed = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=b"yinshi-runner-capability-signing-v1",
    info=b"Ed25519 runner dispatch capabilities",
).derive(secret)
if len(seed) != 32:
    raise RuntimeError("runner capability signing key derivation failed")
return Ed25519PrivateKey.from_private_bytes(seed)

def runner_capability_signing_public_key() -> str:
    """Return the control plane's canonical raw Ed25519 verification key."""
    public_key = (
        _signing_key()
        .public_key()
        .public_bytes(
            encoding=serialization.Encoding.Raw,
            format=serialization.PublicFormat.Raw,
        )
    )
    if len(public_key) != 32:
        raise RuntimeError("runner capability public key must contain 32 bytes")
    return _encode_base64url(public_key)

def _validate_x25519_public_key(value: str, name: str) -> str:
    """Validate canonical encoding and reject unusable low-order X25519 keys."""
    if not isinstance(value, str) or not value:
        raise ValueError(f"{name} must not be empty")
    key_bytes = _decode_base64url(value)
    if len(key_bytes) != 32:
        raise ValueError(f"{name} must contain exactly 32 bytes")
    try:
        public_key = X25519PublicKey.from_public_bytes(key_bytes)
        _X25519_VALIDATION_PRIVATE_KEY.exchange(public_key)
    except ValueError as exc:
        raise ValueError(f"{name} is not a usable X25519 key") from exc
    return value

def _validate_scopes(scopes: tuple[str, ...] | list[str]) -> tuple[str, ...]:
    """Normalize a non-empty unique allowlisted runner scope set."""
    if not isinstance(scopes, (tuple, list)) or not scopes:
        raise ValueError("runner capability scopes must not be empty")
    if any(not isinstance(scope, str) or not scope for scope in scopes):

```

```

        raise ValueError("runner capability scopes must be non-empty strings")
    normalized_scopes = tuple(sorted(scopes))
    if len(set(normalized_scopes)) != len(normalized_scopes):
        raise ValueError("runner capability scopes must not contain duplicates")
    if any(scope not in RUNNER_SCOPES for scope in normalized_scopes):
        raise ValueError("runner capability contains an unsupported scope")
    return normalized_scopes

def create_runner_capability(
    *,
    user_id: str,
    runner_id: str,
    runner_public_key: str,
    initiator_public_key: str,
    scopes: tuple[str, ...] | list[str],
    max_session_bytes: int,
    current_time: int | None = None,
) -> tuple[str, VerifiedRunnerCapability]:
    """Issue one five-minute capability bound to both Noise static identities."""
    if not isinstance(user_id, str) or not user_id:
        raise ValueError("user_id must not be empty")
    if not isinstance(runner_id, str) or not runner_id:
        raise ValueError("runner_id must not be empty")
    normalized_runner_key = _validate_x25519_public_key(runner_public_key, "runner_public_key")
    normalized_initiator_key = _validate_x25519_public_key(
        initiator_public_key,
        "initiator_public_key",
    )
    normalized_scopes = _validate_scopes(scopes)
    if type(max_session_bytes) is not int:
        raise TypeError("max_session_bytes must be an integer")
    if not RUNNER_SESSION_BYTES_MIN <= max_session_bytes <= RUNNER_SESSION_BYTES_MAX:
        raise ValueError("max_session_bytes is outside the allowed range")

    issued_at = int(time.time()) if current_time is None else current_time
    if type(issued_at) is not int or issued_at < 0:
        raise ValueError("current_time must be a non-negative integer")
    expires_at = issued_at + RUNNER_CAPABILITY_TTL_SECONDS
    transfer_id = str(uuid.uuid4())
    token_id = secrets.token_urlsafe(16)
    claims = VerifiedRunnerCapability(
        user_id=user_id,
        runner_id=runner_id,
        runner_public_key=normalized_runner_key,
        initiator_public_key=normalized_initiator_key,

```

```

        transfer_id=transfer_id,
        token_id=token_id,
        scopes=normalized_scopes,
        issued_at=issued_at,
        expires_at=expires_at,
        max_frame_bytes=RUNNER_FRAME_BYTES_MAX,
        max_session_bytes=max_session_bytes,
    )
    payload = {
        "aud": "yinshi-runner",
        "exp": claims.expires_at,
        "iat": claims.issued_at,
        "initiator_key": claims.initiator_public_key,
        "jti": claims.token_id,
        "max_frame_bytes": claims.max_frame_bytes,
        "max_session_bytes": claims.max_session_bytes,
        "protocol": RUNNER_PROTOCOL_VERSION,
        "runner_id": claims.runner_id,
        "runner_key": claims.runner_public_key,
        "scopes": list(claims.scopes),
        "sub": claims.user_id,
        "transfer_id": claims.transfer_id,
        "v": 1,
    }
    encoded_header = _encode_base64url(
        json.dumps(_HEADER, separators=(",", ":"), sort_keys=True).encode("utf-8")
    )
    encoded_payload = _encode_base64url(
        json.dumps(payload, separators=(",", ":"), sort_keys=True).encode("utf-8")
    )
    signing_input = f"{encoded_header}.{encoded_payload}"
    signature = _signing_key().sign(signing_input.encode("ascii"))
    if len(signature) != 64:
        raise RuntimeError("runner capability signature must contain 64 bytes")
    return f"{signing_input}.{_encode_base64url(signature)}", claims

def _parse_verified_payload(
    payload: object,
    *,
    expected_runner_id: str,
    expected_runner_public_key: str,
    current_time: int,
) -> VerifiedRunnerCapability | None:
    """Validate strict capability claims after Ed25519 verification."""
    if not isinstance(payload, dict) or set(payload) != _PAYLOAD_KEYS:

```



```

        return None
    if payload.get("aud") != "yinshi-runner" or payload.get("v") != 1:
        return None
    if payload.get("protocol") != RUNNER_PROTOCOL_VERSION:
        return None
    if payload.get("runner_id") != expected_runner_id:
        return None
    if payload.get("runner_key") != expected_runner_public_key:
        return None

    string_fields = ("sub", "initiator_key", "jti", "transfer_id")
    if any(
        not isinstance(payload.get(field), str) or not payload[field] for field in string_fields
    ):
        return None
    integer_fields = ("iat", "exp", "max_frame_bytes", "max_session_bytes")
    if any(type(payload.get(field)) is not int for field in integer_fields):
        return None
    issued_at = payload["iat"]
    expires_at = payload["exp"]
    if issued_at > current_time + 30 or expires_at <= current_time:
        return None
    if expires_at - issued_at != RUNNER_CAPABILITY_TTL_SECONDS:
        return None
    if payload["max_frame_bytes"] != RUNNER_FRAME_BYTES_MAX:
        return None
    if not RUNNER_SESSION_BYTES_MIN <= payload["max_session_bytes"] <= RUNNER_SESSION_BYTES_MAX:
        return None

    scopes_value = payload.get("scopes")
    if not isinstance(scopes_value, list):
        return None
    try:
        initiator_key = _validate_x25519_public_key(payload["initiator_key"], "initiator_key")
        normalized_scopes = _validate_scopes(scopes_value)
        transfer_id = str(uuid.UUID(payload["transfer_id"]))
        _decode_base64url(payload["jti"])
    except (TypeError, ValueError):
        return None
    if transfer_id != payload["transfer_id"]:
        return None
    return VerifiedRunnerCapability(
        user_id=payload["sub"],
        runner_id=expected_runner_id,
        runner_public_key=expected_runner_public_key,
        initiator_public_key=initiator_key,

```

```

        transfer_id=transfer_id,
        token_id=payload["jti"],
        scopes=normalized_scopes,
        issued_at=issued_at,
        expires_at=expires_at,
        max_frame_bytes=payload["max_frame_bytes"],
        max_session_bytes=payload["max_session_bytes"],
    )

def verify_runner_capability(
    token: str,
    *,
    signing_public_key: bytes,
    expected_runner_id: str,
    expected_runner_public_key: str,
    current_time: int | None = None,
) -> VerifiedRunnerCapability | None:
    """Verify one capability against pinned control-plane and runner identities."""
    if not isinstance(token, str) or not token or len(token) > 8_192:
        return None
    if not isinstance(signing_public_key, bytes) or len(signing_public_key) != 32:
        return None
    if not isinstance(expected_runner_id, str) or not expected_runner_id:
        return None
    try:
        normalized_runner_key = _validate_x25519_public_key(
            expected_runner_public_key,
            "expected_runner_public_key",
        )
    except (TypeError, ValueError):
        return None

    segments = token.split(".")
    if len(segments) != 3:
        return None
    encoded_header, encoded_payload, encoded_signature = segments
    try:
        header_bytes = _decode_base64url(encoded_header)
        payload_bytes = _decode_base64url(encoded_payload)
        signature = _decode_base64url(encoded_signature)
        header = json.loads(header_bytes)
    except (UnicodeDecodeError, ValueError, json.JSONDecodeError):
        return None
    if header != _HEADER or len(signature) != 64:
        return None

```

```

signing_input = f"{encoded_header}.{encoded_payload}".encode("ascii")
try:
    public_key = Ed25519PublicKey.from_public_bytes(signing_public_key)
    public_key.verify(signature, signing_input)
except (InvalidSignature, ValueError):
    return None
try:
    payload = json.loads(payload_bytes)
except (UnicodeDecodeError, json.JSONDecodeError):
    return None
now = int(time.time()) if current_time is None else current_time
if type(now) is not int or now < 0:
    return None
return _parse_verified_payload(
    payload,
    expected_runner_id=expected_runner_id,
    expected_runner_public_key=normalized_runner_key,
    current_time=now,
)

```

17.13 yinshi/services/runner_data_key.py

The runner data key protects persistent tenant secrets independently from replaceable relay transport identity.

```

"""Portable runner data-protection key storage."""

from __future__ import annotations

import os
import re
import secrets
import stat
import time
from pathlib import Path

_KEY_BYTES = 32
_ERROR = "Runner data-protection key is invalid"
_PUBLISH_READ_ATTEMPTS = 100
_PUBLISH_READ_DELAY_SECONDS = 0.01
_TEMPORARY_NAME_PATTERN = re.compile(
    rf"~{re.escape('.yinshi-data-protection-key.tmp-')}[0-9a-f]{{16}}$"
)

def _require_key(value: bytes, name: str) -> bytes:

```

```

    if not isinstance(value, bytes):
        raise TypeError(f"{name} must be bytes")
    if len(value) != _KEY_BYTES:
        raise ValueError(f"{name} must contain exactly 32 bytes")
    return bytes(value)

def _read_key(path: Path) -> bytes:
    try:
        descriptor = os.open(path, os.O_RDONLY | getattr(os, "O_NOFOLLOW", 0))
    except OSError as exc:
        raise RuntimeError(_ERROR) from exc
    try:
        metadata = os.fstat(descriptor)
        if (
            not stat.S_ISREG(metadata.st_mode)
            or metadata.st_uid != os.geteuid()
            or stat.S_IMODE(metadata.st_mode) != 0o600
            or metadata.st_nlink != 1
        ):
            raise RuntimeError(_ERROR)
        key = os.read(descriptor, _KEY_BYTES + 1)
    finally:
        os.close(descriptor)
    if len(key) != _KEY_BYTES:
        raise RuntimeError(_ERROR)
    return bytes(key)

def _valid_key_metadata(metadata: os.stat_result) -> bool:
    return (
        stat.S_ISREG(metadata.st_mode)
        and metadata.st_uid == os.geteuid()
        and stat.S_IMODE(metadata.st_mode) == 0o600
        and metadata.st_size == _KEY_BYTES
    )

def _recover_published_key(path: Path) -> bytes:
    """Finish durable cleanup left between publication directory syncs."""
    directory_descriptor = os.open(
        path.parent,
        os.O_RDONLY | os.O_DIRECTORY | getattr(os, "O_NOFOLLOW", 0),
    )
    final_descriptor: int | None = None
    temporary_descriptors: list[tuple[str, int]] = []

```

```

try:
    final_descriptor = os.open(
        path.name,
        os.O_RDONLY | getattr(os, "O_NOFOLLOW", 0),
        dir_fd=directory_descriptor,
    )
    final_metadata = os.fstat(final_descriptor)
    if not _valid_key_metadata(final_metadata):
        raise RuntimeError(_ERROR)
    temporary_names = sorted(
        name
        for name in os.listdir(directory_descriptor)
        if _TEMPORARY_NAME_PATTERN.fullmatch(name)
    )
    if not temporary_names:
        raise RuntimeError(_ERROR)
    for name in temporary_names:
        descriptor = os.open(
            name,
            os.O_RDONLY | getattr(os, "O_NOFOLLOW", 0),
            dir_fd=directory_descriptor,
        )
        temporary_descriptors.append((name, descriptor))
        metadata = os.fstat(descriptor)
        if (
            not _valid_key_metadata(metadata)
            or (metadata.st_dev, metadata.st_ino)
            != (final_metadata.st_dev, final_metadata.st_ino)
            or metadata.st_nlink != final_metadata.st_nlink
        ):
            raise RuntimeError(_ERROR)
    if final_metadata.st_nlink != len(temporary_descriptors) + 1:
        raise RuntimeError(_ERROR)
    key = os.read(final_descriptor, _KEY_BYTES + 1)
    if len(key) != _KEY_BYTES:
        raise RuntimeError(_ERROR)
    for name, _descriptor in temporary_descriptors:
        os.unlink(name, dir_fd=directory_descriptor)
    os.fsync(directory_descriptor)
    if os.fstat(final_descriptor).st_nlink != 1:
        raise RuntimeError(_ERROR)
    return bytes(key)
except OSError as exc:
    raise RuntimeError(_ERROR) from exc
finally:
    for _name, descriptor in temporary_descriptors:

```

```

        os.close(descriptor)
    if final_descriptor is not None:
        os.close(final_descriptor)
    os.close(directory_descriptor)

def _read_published_key(path: Path) -> bytes:
    """Read a winner or recover its durable interrupted cleanup."""
    for attempt in range(_PUBLISH_READ_ATTEMPTS):
        try:
            return _read_key(path)
        except RuntimeError:
            try:
                metadata = path.lstat()
            except FileNotFoundError:
                raise
            publishing = stat.S_ISREG(metadata.st_mode) and metadata.st_nlink > 1
            if not publishing:
                raise
            if attempt + 1 == _PUBLISH_READ_ATTEMPTS:
                return _recover_published_key(path)
            time.sleep(_PUBLISH_READ_DELAY_SECONDS)
    raise RuntimeError(_ERROR)

def _create_key(path: Path, key: bytes) -> bool:
    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL | getattr(os, "O_NOFOLLOW", 0)
    temporary_path: Path | None = None
    descriptor: int | None = None
    for _attempt in range(16):
        candidate = path.with_name(f"{path.name}.tmp-{secrets.token_hex(8)}")
        try:
            descriptor = os.open(candidate, flags, 0o600)
        except FileExistsError:
            continue
        temporary_path = candidate
        break
    if temporary_path is None or descriptor is None:
        raise RuntimeError(_ERROR)

    published = False
    publication_durable = False
    try:
        try:
            remaining = memoryview(key)
            while remaining:

```

```

        written = os.write(descriptor, remaining)
        if written <= 0:
            raise RuntimeError(_ERROR)
        remaining = remaining[written:]
    os.fsync(descriptor)
finally:
    os.close(descriptor)
    descriptor = None
try:
    os.link(temporary_path, path, follow_symlinks=False)
    published = True
except FileExistsError:
    published = False
if not published:
    os.unlink(temporary_path)
    temporary_path = None
    return False
directory_descriptor = os.open(path.parent, os.O_RDONLY | os.O_DIRECTORY)
try:
    os.fsync(directory_descriptor)
    publication_durable = True
    os.unlink(temporary_path)
    temporary_path = None
    os.fsync(directory_descriptor)
finally:
    os.close(directory_descriptor)
return True
except BaseException:
    if descriptor is not None:
        try:
            os.close(descriptor)
        except OSError:
            pass
    if published and not publication_durable:
        try:
            os.unlink(path)
        except OSError:
            pass
    if temporary_path is not None:
        try:
            os.unlink(temporary_path)
            temporary_path = None
        except OSError:
            pass
raise

```

```

def load_or_create_runner_data_key(
    key_path: Path,
    database_root: Path,
    legacy_noise_private_key: bytes,
) -> bytes:
    """Load or create the backed-up runner storage key."""
    if not isinstance(key_path, Path) or not isinstance(database_root, Path):
        raise TypeError("key paths must be pathlib.Path values")
    if not key_path.is_absolute() or not database_root.is_absolute():
        raise ValueError("runner data-protection key paths must be absolute")
    if key_path.parent != database_root:
        raise ValueError("runner data-protection key must be inside database root")
    legacy_key = _require_key(legacy_noise_private_key, "legacy_noise_private_key")
    database_root.mkdir(mode=0o700, parents=True, exist_ok=True)
    metadata = database_root.lstat()
    if not stat.S_ISDIR(metadata.st_mode) or metadata.st_uid != os.geteuid():
        raise RuntimeError(_ERROR)
    database_root.chmod(0o700)
    if key_path.exists() or key_path.is_symlink():
        return _read_published_key(key_path)
    initialized = False
    for name in ("control.db", "legacy.db"):
        durable_path = database_root / name
        try:
            durable_metadata = durable_path.lstat()
        except FileNotFoundError:
            continue
        if not stat.S_ISREG(durable_metadata.st_mode) or durable_metadata.st_uid != os.geteuid():
            raise RuntimeError(_ERROR)
        initialized = True
    key = legacy_key if initialized else secrets.token_bytes(_KEY_BYTES)
    if _create_key(key_path, key):
        return key
    return _read_published_key(key_path)

```

17.14 yinshi/services/runner_noise.py

Noise key utilities validate transport identities and derive the browser-to-runner encrypted channel.

```

"""Noise IK identity storage and encrypted runner transport primitives."""

from __future__ import annotations

import base64

```



```

import os
import stat
from dataclasses import dataclass
from pathlib import Path

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from noise.connection import Keypair, NoiseConnection
from noise.exceptions import (
    NoiseHandshakeError,
    NoiseInvalidMessage,
    NoiseMaxNonceError,
    NoiseValidationError,
    NoiseValueError,
)

_NOISE_PROTOCOL_NAME = b"Noise_IK_25519_ChaChaPoly_SHA256"
_X25519_KEY_LENGTH = 32
_NOISE_IK_FIRST_MESSAGE_MIN_LENGTH = 96
_NOISE_IK_SECOND_MESSAGE_MIN_LENGTH = 48
_NOISE_MESSAGE_MAX_LENGTH = 65_535

@dataclass(frozen=True, slots=True)
class RunnerNoiseKeypair:
    """Persistent runner static identity represented as raw X25519 keys."""

    private_key: bytes
    public_key: bytes

    @property
    def public_key_base64url(self) -> str:
        """Return the canonical unpadded base64url public key."""
        assert len(self.public_key) == _X25519_KEY_LENGTH
        return base64.urlsafe_b64encode(self.public_key).rstrip(b"=").decode("ascii")

    def _require_key_bytes(value: bytes, name: str) -> bytes:
        """Copy one exact-length X25519 key or reject malformed caller state."""
        if not isinstance(value, bytes):
            raise TypeError(f"{name} must be bytes")
        if len(value) != _X25519_KEY_LENGTH:
            raise ValueError(f"{name} must contain exactly {_X25519_KEY_LENGTH} bytes")
        return bytes(value)

```

```

def _read_owner_only_key(path: Path) -> bytes:
    """Read a regular owner-only key file without following symlinks."""
    no_follow = getattr(os, "O_NOFOLLOW", 0)
    try:
        descriptor = os.open(path, os.O_RDONLY | no_follow)
    except OSError as exc:
        raise RuntimeError(f"Runner Noise private key could not be opened: {path}") from exc
    try:
        metadata = os.fstat(descriptor)
        if not stat.S_ISREG(metadata.st_mode):
            raise RuntimeError("Runner Noise private key must be a regular file")
        if metadata.st_uid != os.geteuid():
            raise RuntimeError("Runner Noise private key must be owned by the runner user")
        if stat.S_IMODE(metadata.st_mode) != 0o600:
            raise RuntimeError("Runner Noise private key must have owner-only permissions")
        private_key = os.read(descriptor, _X25519_KEY_LENGTH + 1)
    finally:
        os.close(descriptor)
    return _require_key_bytes(private_key, "Runner Noise private key")

def _create_owner_only_key(path: Path, private_key: bytes) -> bool:
    """Atomically create one owner-only key, returning false on a creation race."""
    flags = os.O_WRONLY | os.O_CREAT | os.O_EXCL
    no_follow = getattr(os, "O_NOFOLLOW", 0)
    try:
        descriptor = os.open(path, flags | no_follow, 0o600)
    except FileExistsError:
        return False
    try:
        written = os.write(descriptor, private_key)
        if written != len(private_key):
            raise RuntimeError("Runner Noise private key write was incomplete")
        os.fsync(descriptor)
    except BaseException:
        path.unlink(missing_ok=True)
        raise
    finally:
        os.close(descriptor)
    return True

def load_or_create_runner_noise_keypair(path: Path) -> RunnerNoiseKeypair:
    """Load or atomically create the runner's owner-only static Noise keypair."""
    if not isinstance(path, Path):
        raise TypeError("path must be a pathlib.Path")

```

```

if not path.is_absolute():
    raise ValueError("Runner Noise private key path must be absolute")

if path.exists() or path.is_symlink():
    private_key_bytes = _read_owner_only_key(path)
else:
    path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
    generated_key = X25519PrivateKey.generate()
    generated_bytes = generated_key.private_bytes_raw()
    if _create_owner_only_key(path, generated_bytes):
        private_key_bytes = generated_bytes
    else:
        private_key_bytes = _read_owner_only_key(path)

private_key = X25519PrivateKey.from_private_bytes(private_key_bytes)
public_key_bytes = private_key.public_key().public_bytes_raw()
assert len(public_key_bytes) == _X25519_KEY_LENGTH
return RunnerNoiseKeypair(
    private_key=private_key_bytes,
    public_key=public_key_bytes,
)

class NoiseIkResponder:
    """Strict two-message Noise IK responder with authenticated transport ciphers."""

    def __init__(
        self,
        *,
        static_private_key: bytes,
        prologue: bytes = b"",
        ephemeral_private_key: bytes | None = None,
    ) -> None:
        static_key = _require_key_bytes(static_private_key, "static_private_key")
        if not isinstance(prologue, bytes):
            raise TypeError("prologue must be bytes")
        if len(prologue) > _NOISE_MESSAGE_MAX_LENGTH:
            raise ValueError("prologue is too large")

        connection = NoiseConnection.from_name(_NOISE_PROTOCOL_NAME)
        connection.set_as_responder()
        connection.set_keypair_from_private_bytes(Keypair.STATIC, static_key)
        if ephemeral_private_key is not None:
            ephemeral_key = _require_key_bytes(ephemeral_private_key, "ephemeral_private_key")
            connection.set_keypair_from_private_bytes(Keypair.EPHEMERAL, ephemeral_key)
        connection.set_prologue(prologue)

```

```

connection.start_handshake()

self._connection = connection
self._first_message_read = False
self._handshake_complete = False
self._initiator_static_public_key: bytes | None = None

@property
def initiator_static_public_key(self) -> bytes:
    """Return authenticated initiator identity after reading the first message."""
    if self._initiator_static_public_key is None:
        raise RuntimeError("Noise IK initiator identity is not authenticated")
    return bytes(self._initiator_static_public_key)

@property
def handshake_hash(self) -> bytes:
    """Return channel-binding hash after handshake completion."""
    if not self._handshake_complete:
        raise RuntimeError("Noise IK handshake is not complete")
    handshake_hash = self._connection.get_handshake_hash()
    if not isinstance(handshake_hash, bytes) or len(handshake_hash) != 32:
        raise RuntimeError("Noise IK handshake hash is invalid")
    return bytes(handshake_hash)

def read_handshake_message(self, message: bytes) -> bytes:
    """Authenticate and decrypt the initiator's single IK handshake message."""
    if self._first_message_read:
        raise RuntimeError("Noise IK initiator message was already read")
    if not isinstance(message, bytes):
        raise TypeError("message must be bytes")
    if not _NOISE_IK_FIRST_MESSAGE_MIN_LENGTH <= len(message) <= _NOISE_MESSAGE_MAX_LENGTH:
        raise ValueError("Noise IK initiator message has an invalid length")

    try:
        payload = bytes(self._connection.read_message(message))
    except (
        InvalidTag,
        NoiseHandshakeError,
        NoiseInvalidMessage,
        NoiseMaxNonceError,
        NoiseValidationError,
        NoiseValueError,
    ) as exc:
        raise ValueError("Noise IK initiator message failed authentication") from exc
    handshake_state = self._connection.noise_protocol.handshake_state
    remote_static = handshake_state.rs.public_bytes

```

```

self._initiator_static_public_key = _require_key_bytes(
    remote_static,
    "Noise IK initiator public key",
)
self._first_message_read = True
return payload

def write_handshake_message(self, payload: bytes) -> bytes:
    """Encrypt the responder payload and finish the IK handshake."""
    if not self._first_message_read or self._handshake_complete:
        raise RuntimeError("Noise IK responder is not ready to finish the handshake")
    if not isinstance(payload, bytes):
        raise TypeError("payload must be bytes")
    if len(payload) > _NOISE_MESSAGE_MAX_LENGTH - _NOISE_IK_SECOND_MESSAGE_MIN_LENGTH:
        raise ValueError("Noise IK responder payload is too large")

    try:
        message = bytes(self._connection.write_message(payload))
    except (
        NoiseHandshakeError,
        NoiseInvalidMessage,
        NoiseMaxNonceError,
        NoiseValidationError,
        NoiseValueError,
    ) as exc:
        raise ValueError("Noise IK responder message could not be created") from exc
    if not _NOISE_IK_SECOND_MESSAGE_MIN_LENGTH <= len(message) <= _NOISE_MESSAGE_MAX_LENGTH:
        raise RuntimeError("Noise IK responder produced an invalid message")
    self._handshake_complete = True
    return message

def encrypt(self, plaintext: bytes) -> bytes:
    """Encrypt one ordered runner-to-client transport message."""
    if not self._handshake_complete:
        raise RuntimeError("Noise IK transport is not ready")
    if not isinstance(plaintext, bytes):
        raise TypeError("plaintext must be bytes")
    if len(plaintext) > _NOISE_MESSAGE_MAX_LENGTH - 16:
        raise ValueError("Noise transport plaintext is too large")
    try:
        return bytes(self._connection.encrypt(plaintext))
    except (NoiseMaxNonceError, NoiseValidationError, NoiseValueError) as exc:
        raise ValueError("Noise transport encryption failed") from exc

def decrypt(self, ciphertext: bytes) -> bytes:
    """Authenticate and decrypt one ordered client-to-runner transport message."""

```

```

if not self._handshake_complete:
    raise RuntimeError("Noise IK transport is not ready")
if not isinstance(ciphertext, bytes):
    raise TypeError("ciphertext must be bytes")
if not 16 <= len(ciphertext) <= _NOISE_MESSAGE_MAX_LENGTH:
    raise ValueError("Noise transport ciphertext has an invalid length")
try:
    return bytes(self._connection.decrypt(ciphertext))
except (
    InvalidTag,
    NoiseInvalidMessage,
    NoiseMaxNonceError,
    NoiseValidationError,
    NoiseValueError,
) as exc:
    raise ValueError("Noise transport ciphertext failed authentication") from exc

```

17.15 yinshi/services/runner_noise_session.py

Noise sessions enforce handshake order, message counters, replay limits, and authenticated encryption for RPC envelopes.

"""Capability-bound Noise sessions for the outbound BYOC runner channel."""

```

from __future__ import annotations

import base64
import json
import os
import secrets
import sqlite3
import stat
from pathlib import Path

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey

from yinshi.services.runner_capabilities import (
    RUNNER_PROTOCOL_VERSION,
    VerifiedRunnerCapability,
    verify_runner_capability,
)
from yinshi.services.runner_noise import NoiseIkResponder

RUNNER_NOISE_PROLOGUE = b"yinshi-runner-v1"
_MESSAGES_BEFORE_REHANDSHAKE = 1_048_576

```

```
_X25519_KEY_LENGTH = 32
_ED25519_KEY_LENGTH = 32
```

```
class RunnerCapabilityReplayStore:
    """Durably consume capability IDs without retaining request or response content."""

    def __init__(self, path: Path) -> None:
        if not isinstance(path, Path):
            raise TypeError("path must be a pathlib.Path")
        if not path.is_absolute():
            raise ValueError("Runner capability replay database path must be absolute")
        self._path = path
        self._prepare_file()
        self._initialize_schema()

    def _prepare_file(self) -> None:
        """Create or validate the owner-only regular SQLite file."""
        self._path.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
        no_follow = getattr(os, "O_NOFOLLOW", 0)
        try:
            descriptor = os.open(
                self._path,
                os.O_WRONLY | os.O_CREAT | os.O_EXCL | no_follow,
                0o600,
            )
        except FileExistsError:
            try:
                descriptor = os.open(self._path, os.O_RDONLY | no_follow)
            except OSError as exc:
                raise RuntimeError("Runner replay database could not be opened safely") from exc
        try:
            metadata = os.fstat(descriptor)
            if not stat.S_ISREG(metadata.st_mode):
                raise RuntimeError("Runner replay database must be a regular file")
            if metadata.st_uid != os.geteuid():
                raise RuntimeError("Runner replay database must be owned by the runner user")
            if stat.S_IMODE(metadata.st_mode) != 0o600:
                raise RuntimeError("Runner replay database must have owner-only permissions")
        finally:
            os.close(descriptor)

    def _connect(self) -> sqlite3.Connection:
        """Open one bounded SQLite transaction connection."""
        connection = sqlite3.connect(self._path, timeout=5.0)
        connection.execute("PRAGMA busy_timeout = 5000")
```

```

        connection.execute("PRAGMA journal_mode = DELETE")
        connection.execute("PRAGMA synchronous = FULL")
        return connection

def _initialize_schema(self) -> None:
    """Create the minimal replay table before accepting capabilities."""
    with self._connect() as connection:
        connection.execute("""
            CREATE TABLE IF NOT EXISTS consumed_runner_capabilities (
                token_id TEXT PRIMARY KEY,
                expires_at INTEGER NOT NULL,
                consumed_at INTEGER NOT NULL,
                CHECK (expires_at > consumed_at)
            )
        """)
        connection.commit()

def consume(self, token_id: str, *, expires_at: int, current_time: int) -> bool:
    """Atomically consume one unexpired token ID, returning false for replay."""
    if not isinstance(token_id, str) or not token_id:
        raise ValueError("token_id must not be empty")
    if len(token_id) > 128:
        raise ValueError("token_id is too large")
    if type(expires_at) is not int or type(current_time) is not int:
        raise TypeError("replay timestamps must be integers")
    if expires_at <= current_time:
        raise ValueError("capability must be unexpired when consumed")

    connection = self._connect()
    try:
        connection.execute("BEGIN IMMEDIATE")
        connection.execute(
            "DELETE FROM consumed_runner_capabilities WHERE expires_at <= ?",
            (current_time,)
        )
        try:
            connection.execute(
                """
                INSERT INTO consumed_runner_capabilities (
                    token_id, expires_at, consumed_at
                ) VALUES (?, ?, ?)
                """
                ,
                (token_id, expires_at, current_time),
            )
        except sqlite3.IntegrityError:
            connection.rollback()

```



```

        return False
    connection.commit()
    return True
finally:
    connection.close()

class RunnerNoiseSession:
    """One authenticated, bounded, ordered browser-to-runner Noise channel."""

    def __init__(
        self,
        *,
        runner_id: str,
        runner_static_private_key: bytes,
        capability_signing_public_key: bytes,
        replay_store: RunnerCapabilityReplayStore,
    ) -> None:
        if not isinstance(runner_id, str) or not runner_id:
            raise ValueError("runner_id must not be empty")
        if not isinstance(runner_static_private_key, bytes):
            raise TypeError("runner_static_private_key must be bytes")
        if len(runner_static_private_key) != _X25519_KEY_LENGTH:
            raise ValueError("runner_static_private_key must contain exactly 32 bytes")
        if not isinstance(capability_signing_public_key, bytes):
            raise TypeError("capability_signing_public_key must be bytes")
        if len(capability_signing_public_key) != _ED25519_KEY_LENGTH:
            raise ValueError("capability_signing_public_key must contain exactly 32 bytes")
        if not isinstance(replay_store, RunnerCapabilityReplayStore):
            raise TypeError("replay_store must be RunnerCapabilityReplayStore")

        runner_public_key = (
            X25519PrivateKey.from_private_bytes(runner_static_private_key)
            .public_key()
            .public_bytes(
                encoding=serialization.Encoding.Raw,
                format=serialization.PublicFormat.Raw,
            )
        )
        self._runner_id = runner_id
        self._runner_public_key = (
            base64.urlsafe_b64encode(runner_public_key).rstrip(b"=").decode("ascii")
        )
        self._capability_signing_public_key = bytes(capability_signing_public_key)
        self._replay_store = replay_store
        self._responder = NoiseIkResponder(

```

```

        static_private_key=runner_static_private_key,
        prologue=RUNNER_NOISE_PROLOGUE,
    )
    self._capability: VerifiedRunnerCapability | None = None
    self._failed = False
    self._messages_sent = 0
    self._messages_received = 0
    self._ciphertext_bytes = 0

@property
def capability(self) -> VerifiedRunnerCapability:
    """Return verified authority after successful handshake acceptance."""
    if self._capability is None:
        raise RuntimeError("Runner Noise capability is not authenticated")
    return self._capability

def accept_handshake(self, message: bytes, *, current_time: int) -> bytes:
    """Verify client identity, signed scope, expiry, and one-time use."""
    if self._failed or self._capability is not None:
        raise RuntimeError("Runner Noise session cannot accept another handshake")
    if type(current_time) is not int or current_time < 0:
        raise ValueError("current_time must be a non-negative integer")
    try:
        token_bytes = self._responder.read_handshake_message(message)
        token = token_bytes.decode("ascii")
        verified = verify_runner_capability(
            token,
            signing_public_key=self._capability_signing_public_key,
            expected_runner_id=self._runner_id,
            expected_runner_public_key=self._runner_public_key,
            current_time=current_time,
        )
        if verified is None:
            raise ValueError("Runner capability is invalid or expired")
        initiator_public_key = (
            base64.urlsafe_b64encode(self._responder.initiator_static_public_key)
            .rstrip(b"=")
            .decode("ascii")
        )
        if not secrets.compare_digest(
            initiator_public_key,
            verified.initiator_public_key,
        ):
            raise ValueError("Runner capability initiator identity does not match")
        consumed = self._replay_store.consume(
            verified.token_id,

```

```

        expires_at=verified.expires_at,
        current_time=current_time,
    )
    if not consumed:
        raise ValueError("Runner capability has already been consumed")
    response_payload = json.dumps(
        {
            "protocol": RUNNER_PROTOCOL_VERSION,
            "transfer_id": verified.transfer_id,
        },
        separators=(",", ":"),
        sort_keys=True,
    ).encode("ascii")
    response = self._responder.write_handshake_message(response_payload)
    self._capability = verified
    return response
except (UnicodeDecodeError, ValueError):
    self._failed = True
    raise

def encrypt(self, plaintext: bytes) -> bytes:
    """Encrypt one runner response within session byte and message limits."""
    capability = self._require_transport()
    if not isinstance(plaintext, bytes):
        raise TypeError("plaintext must be bytes")
    if self._messages_sent >= _MESSAGES_BEFORE_REHANDSHAKE:
        raise RuntimeError("Runner Noise transport requires a fresh handshake")
    predicted_ciphertext_length = len(plaintext) + 16
    self._require_session_capacity(capability, predicted_ciphertext_length)
    try:
        ciphertext = self._responder.encrypt(plaintext)
    except BaseException:
        self._failed = True
        raise
    self._ciphertext_bytes += len(ciphertext)
    self._messages_sent += 1
    return ciphertext

def decrypt(self, ciphertext: bytes) -> bytes:
    """Decrypt one client request within session byte and message limits."""
    capability = self._require_transport()
    if not isinstance(ciphertext, bytes):
        raise TypeError("ciphertext must be bytes")
    if self._messages_received >= _MESSAGES_BEFORE_REHANDSHAKE:
        raise RuntimeError("Runner Noise transport requires a fresh handshake")
    self._require_session_capacity(capability, len(ciphertext))

```

```

    try:
        plaintext = self._responder.decrypt(ciphertext)
    except BaseException:
        self._failed = True
        raise
    self._ciphertext_bytes += len(ciphertext)
    self._messages_received += 1
    return plaintext

def _require_transport(self) -> VerifiedRunnerCapability:
    """Return authority only while the established channel remains usable."""
    if self._failed:
        raise RuntimeError("Runner Noise transport failed and cannot be reused")
    if self._capability is None:
        raise RuntimeError("Runner Noise transport is not ready")
    return self._capability

def _require_session_capacity(
    self,
    capability: VerifiedRunnerCapability,
    ciphertext_length: int,
) -> None:
    """Reject per-frame and cumulative ciphertext limit violations."""
    if ciphertext_length < 16 or ciphertext_length > capability.max_frame_bytes:
        raise ValueError("Runner Noise ciphertext exceeds the frame limit")
    if self._ciphertext_bytes + ciphertext_length > capability.max_session_bytes:
        raise ValueError("Runner Noise session exceeds the byte limit")

```

17.16 yinshi/services/runner_relay.py

Relay state tracks connected roles and routes opaque frames only between matching tenant runtime peers.

"""Bounded opaque relay authorization and in-memory connection broker."""

```

from __future__ import annotations

import asyncio
import hashlib
import secrets
import time
import uuid
from collections import OrderedDict
from collections.abc import Callable
from dataclasses import dataclass, field
from typing import Protocol

```

```

from yinshi.db import get_control_db
from yinshi.services.managed_backups import ManagedBackupOperation
from yinshi.services.runner_capabilities import VerifiedRunnerCapability

_RUNNER_PREFIX_LENGTH = 16
_RELAY_FRAME_BYTES_MAX = 65_535
_RELAY_QUEUE_FRAMES_MAX = 16
_RUNNER_SESSIONS_MAX = 32
_RETIRED_TRANSFERS_MAX = 128
_RETIRED_TRANSFER_TTL_SECONDS = 300.0

class RunnerRelayAuthorizationError(ValueError):
    """A relay grant is missing, stale, mismatched, or already consumed."""

@dataclass(frozen=True, slots=True)
class RunnerTransferGrant:
    """Metadata needed to route bounded ciphertext without retaining content."""

    transfer_id: str
    runner_id: str
    expires_at: int
    max_session_bytes: int

class RelayWebSocket(Protocol):
    """Small WebSocket surface used by the relay broker and test fakes."""

    async def send_text(self, data: str) -> None: ...

    async def send_bytes(self, data: bytes) -> None: ...

    async def close(self, code: int = 1000, reason: str | None = None) -> None: ...

def _require_transfer_id(transfer_id: str) -> str:
    """Return one canonical UUID transfer ID."""
    if not isinstance(transfer_id, str) or not transfer_id:
        raise ValueError("transfer_id must not be empty")
    try:
        normalized_id = str(uuid.UUID(transfer_id))
    except ValueError as exc:
        raise ValueError("transfer_id must be a UUID") from exc
    if normalized_id != transfer_id:

```

```

        raise ValueError("transfer_id must be canonical")
    return normalized_id

def _capability_hash(capability: str) -> str:
    """Hash a bounded capability before control-plane persistence or comparison."""
    if not isinstance(capability, str) or not capability:
        raise ValueError("capability must not be empty")
    if len(capability) > 8_192:
        raise ValueError("capability is too large")
    return hashlib.sha256(capability.encode("ascii")).hexdigest()

def store_runner_transfer_grant(
    capability: str,
    claims: VerifiedRunnerCapability,
) -> None:
    """Persist only routing limits and a capability hash for one issued grant."""
    if not isinstance(claims, VerifiedRunnerCapability):
        raise TypeError("claims must be VerifiedRunnerCapability")
    capability_hash = _capability_hash(capability)
    with get_control_db() as database:
        database.execute(
            "DELETE FROM runner_transfer_grants WHERE expires_at <= ?",
            (claims.issued_at,),
        )
        database.execute(
            """
            INSERT INTO runner_transfer_grants (
                transfer_id, user_id, runner_id, capability_hash,
                expires_at, max_session_bytes, claimed_at
            ) VALUES (?, ?, ?, ?, ?, ?, NULL)
            """,
            (
                claims.transfer_id,
                claims.user_id,
                claims.runner_id,
                capability_hash,
                claims.expires_at,
                claims.max_session_bytes,
            ),
        )
        database.commit()

def claim_runner_transfer_grant(

```

```

transfer_id: str,
capability: str,
*,
current_time: int | None = None,
) -> RunnerTransferGrant:
    """Atomically consume one exact unexpired capability hash for relay use."""
    normalized_transfer_id = _require_transfer_id(transfer_id)
    capability_hash = _capability_hash(capability)
    now = int(time.time()) if current_time is None else current_time
    if type(now) is not int or now < 0:
        raise ValueError("current_time must be a non-negative integer")

    with get_control_db() as database:
        database.execute("BEGIN IMMEDIATE")
        row = database.execute(
            """
            SELECT grant.*, runner.revoked_at, runner.runner_token_hash
            FROM runner_transfer_grants AS grant
            JOIN user_runners AS runner ON runner.id = grant.runner_id
            WHERE grant.transfer_id = ?
            """,
            (normalized_transfer_id,),
        ).fetchone()
        if row is None:
            database.rollback()
            raise RunnerRelayAuthorizationError("Runner relay grant was not found")
        if row["expires_at"] <= now:
            database.rollback()
            raise RunnerRelayAuthorizationError("Runner relay grant has expired")
        if row["claimed_at"] is not None:
            database.rollback()
            raise RunnerRelayAuthorizationError("Runner relay grant was already claimed")
        if row["revoked_at"] is not None or row["runner_token_hash"] is None:
            database.rollback()
            raise RunnerRelayAuthorizationError("Runner relay grant was revoked")
        if not secrets.compare_digest(row["capability_hash"], capability_hash):
            database.rollback()
            raise RunnerRelayAuthorizationError("Runner relay capability does not match")
        result = database.execute(
            """
            UPDATE runner_transfer_grants
            SET claimed_at = ?
            WHERE transfer_id = ? AND claimed_at IS NULL
            """,
            (now, normalized_transfer_id),
        )

```

```

        if result.rowcount != 1:
            database.rollback()
            raise RunnerRelayAuthorizationError("Runner relay grant was already claimed")
        database.commit()
    return RunnerTransferGrant(
        transfer_id=normalized_transfer_id,
        runner_id=row["runner_id"],
        expires_at=row["expires_at"],
        max_session_bytes=row["max_session_bytes"],
    )

@dataclass(slots=True)
class _RunnerConnection:
    websocket: RelayWebSocket
    send_lock: asyncio.Lock = field(default_factory=asyncio.Lock)
    sessions: set[str] = field(default_factory=set)
    retired_transfers: OrderedDict[str, float] = field(default_factory=OrderedDict)

@dataclass(frozen=True, slots=True)
class _RunnerMaintenance:
    job_id: str
    acknowledgement: asyncio.Future[None] | None

@dataclass(slots=True)
class _ClientConnection:
    websocket: RelayWebSocket
    grant: RunnerTransferGrant
    outbound_queue: asyncio.Queue[bytes] = field(
        default_factory=lambda: asyncio.Queue(maxsize=_RELAY_QUEUE_FRAMES_MAX)
    )
    ciphertext_bytes: int = 0

def _get_running_maintenance_operation(
    runner_id: str,
) -> ManagedBackupOperation | None:
    """Load one durable source-runner fence without coupling relay import startup."""
    from yinshi.services.managed_backups import (
        get_running_managed_backup_operation_for_runner,
    )

    return get_running_managed_backup_operation_for_runner(runner_id)

```



```

class RunnerRelayBroker:
    """Route framed ciphertext between one outbound runner and bounded clients."""

    def __init__(
        self,
        *,
        get_running_operation: Callable[[str], ManagedBackupOperation | None] | None = None,
        monotonic: Callable[[], float] = time.monotonic,
    ) -> None:
        if get_running_operation is not None and not callable(get_running_operation):
            raise TypeError("get_running_operation must be callable or None")
        if not callable(monotonic):
            raise TypeError("monotonic must be callable")
        self._get_running_operation = get_running_operation
        self._monotonic = monotonic
        self._lock = asyncio.Lock()
        self._runners: dict[str, _RunnerConnection] = {}
        self._maintenance: dict[str, _RunnerMaintenance] = {}
        self._clients: dict[str, _ClientConnection] = {}

    def is_runner_connected(self, runner_id: str) -> bool:
        """Return whether this event loop currently owns a runner socket."""
        if not isinstance(runner_id, str) or not runner_id:
            raise ValueError("runner_id must not be empty")
        return runner_id in self._runners

    async def register_runner(self, runner_id: str, websocket: RelayWebSocket) -> None:
        """Register one current outbound runner connection, replacing stale state."""
        if not isinstance(runner_id, str) or not runner_id:
            raise ValueError("runner_id must not be empty")
        durable_operation = (
            None if self._get_running_operation is None else self._get_running_operation(runner_id)
        )
        durable_job_id = None if durable_operation is None else durable_operation.job_id
        clients_to_close: list[_ClientConnection] = []
        async with self._lock:
            previous = self._runners.get(runner_id)
            if previous is not None:
                for transfer_id in tuple(previous.sessions):
                    client = self._clients.pop(transfer_id, None)
                    if client is not None:
                        self._close_client_queue(client)
                        clients_to_close.append(client)
            self._runners[runner_id] = _RunnerConnection(websocket=websocket)
            maintenance = self._maintenance.get(runner_id)

```

```

        maintenance_job_id = None if maintenance is None else maintenance.job_id
        if durable_job_id is not None and maintenance is None:
            acknowledgement = asyncio.get_running_loop().create_future()
            self._maintenance[runner_id] = _RunnerMaintenance(
                durable_job_id,
                acknowledgement,
            )
            maintenance_job_id = durable_job_id
    if previous is not None:
        await previous.websocket.close(code=4001, reason="Runner connection replaced")
    for client in clients_to_close:
        await client.websocket.close(code=4002, reason="Runner connection replaced")
    if maintenance_job_id is not None:
        control_message = '{"job_id":"' + maintenance_job_id + '", "type": "quiesce"}'
        async with self._runners[runner_id].send_lock:
            await websocket.send_text(control_message)

async def unregister_runner(self, runner_id: str, websocket: RelayWebSocket) -> None:
    """Remove only the matching runner connection and close attached clients."""
    clients_to_close: list[_ClientConnection] = []
    async with self._lock:
        runner = self._runners.get(runner_id)
        if runner is None or runner.websocket is not websocket:
            return
        del self._runners[runner_id]
        for transfer_id in tuple(runner.sessions):
            client = self._clients.pop(transfer_id, None)
            if client is not None:
                self._close_client_queue(client)
                clients_to_close.append(client)
    for client in clients_to_close:
        await client.websocket.close(code=4002, reason="Runner disconnected")

async def quiesce_runner(
    self,
    runner_id: str,
    *,
    job_id: str,
    timeout_seconds: float,
) -> None:
    """Request exact runner quiescence and await its matching acknowledgement."""
    if not isinstance(runner_id, str) or not runner_id:
        raise ValueError("runner_id must not be empty")
    normalized_job_id = _require_transfer_id(job_id)
    if (
        isinstance(timeout_seconds, bool)

```

```

        or not isinstance(timeout_seconds, (int, float))
        or not 0 < timeout_seconds <= 300
    ):
        raise ValueError("timeout_seconds must be between 0 and 300")
    clients_to_close: list[_ClientConnection] = []
    async with self._lock:
        runner = self._runners.get(runner_id)
        if runner is None:
            raise RunnerRelayAuthorizationError("Runner is not connected")
        existing_maintenance = self._maintenance.get(runner_id)
        if existing_maintenance is not None:
            if existing_maintenance.job_id != normalized_job_id:
                raise RunnerRelayAuthorizationError("Runner maintenance is already active")
            acknowledgement = existing_maintenance.acknowledgement
            if acknowledgement is None or acknowledgement.done():
                return
            maintenance = existing_maintenance
        else:
            acknowledgement = asyncio.get_running_loop().create_future()
            maintenance = _RunnerMaintenance(normalized_job_id, acknowledgement)
            self._maintenance[runner_id] = maintenance
        for transfer_id in tuple(runner.sessions):
            client = self._clients.pop(transfer_id, None)
            runner.sessions.discard(transfer_id)
            self._retire_transfer(runner, transfer_id)
            if client is not None:
                self._close_client_queue(client)
                clients_to_close.append(client)
        for client in clients_to_close:
            await client.websocket.close(code=4006, reason="Runner entered maintenance")
        control_message = '{"job_id":"' + normalized_job_id + '", "type": "quiesce"}'
        try:
            async with runner.send_lock:
                await runner.websocket.send_text(control_message)
            await asyncio.wait_for(
                asyncio.shield(acknowledgement),
                timeout=float(timeout_seconds),
            )
        except BaseException:
            async with self._lock:
                current = self._maintenance.get(runner_id)
                if current is maintenance:
                    del self._maintenance[runner_id]
            raise

    async def runner_quiesced(self, runner_id: str, job_id: str) -> None:

```

```

        """Accept one exact maintenance acknowledgement from the current runner."""
        normalized_job_id = _require_transfer_id(job_id)
        async with self._lock:
            maintenance = self._maintenance.get(runner_id)
            if maintenance is None or maintenance.job_id != normalized_job_id:
                raise RunnerRelayAuthorizationError("Runner maintenance acknowledgement mismatch")
            acknowledgement = maintenance.acknowledgement
            if acknowledgement is not None and not acknowledgement.done():
                acknowledgement.set_result(None)

    async def release_maintenance(self, runner_id: str, *, job_id: str) -> None:
        """Release only the matching maintenance fence on one current runner."""
        if not isinstance(runner_id, str) or not runner_id:
            raise ValueError("runner_id must not be empty")
        normalized_job_id = _require_transfer_id(job_id)
        async with self._lock:
            maintenance = self._maintenance.get(runner_id)
            if maintenance is None or maintenance.job_id != normalized_job_id:
                raise RunnerRelayAuthorizationError("Runner maintenance release mismatched")
            acknowledgement = maintenance.acknowledgement
            if acknowledgement is not None and not acknowledgement.done():
                raise RunnerRelayAuthorizationError("Runner maintenance is not quiesced")
            del self._maintenance[runner_id]

    async def attach_client(
        self,
        grant: RunnerTransferGrant,
        websocket: RelayWebSocket,
    ) -> None:
        """Attach one claimed client only while its runner is connected and bounded."""
        if not isinstance(grant, RunnerTransferGrant):
            raise TypeError("grant must be RunnerTransferGrant")
        async with self._lock:
            runner = self._runners.get(grant.runner_id)
            if runner is None:
                raise RunnerRelayAuthorizationError("Runner is not connected")
            if grant.runner_id in self._maintenance:
                raise RunnerRelayAuthorizationError("Runner maintenance is active")
            if len(runner.sessions) >= _RUNNER_SESSIONS_MAX:
                raise RunnerRelayAuthorizationError("Runner session limit was reached")
            if grant.transfer_id in self._clients:
                raise RunnerRelayAuthorizationError("Runner relay client is already attached")
            runner.sessions.add(grant.transfer_id)
            self._clients[grant.transfer_id] = _ClientConnection(
                websocket=websocket,
                grant=grant,

```

```

    )
    control_message = '{"transfer_id":"' + grant.transfer_id + '","type":"open"}'
    try:
        async with runner.send_lock:
            await runner.websocket.send_text(control_message)
    except BaseException:
        await self.detach_client(grant.transfer_id, websocket)
        raise

async def detach_client(self, transfer_id: str, websocket: RelayWebSocket) -> None:
    """Detach one matching client and notify its current runner."""
    normalized_transfer_id = _require_transfer_id(transfer_id)
    runner: _RunnerConnection | None = None
    async with self._lock:
        client = self._clients.get(normalized_transfer_id)
        if client is None or client.websocket is not websocket:
            return
        del self._clients[normalized_transfer_id]
        self._close_client_queue(client)
        runner = self._runners.get(client.grant.runner_id)
        if runner is not None:
            runner.sessions.discard(normalized_transfer_id)
            self._retire_transfer(runner, normalized_transfer_id)
    if runner is not None:
        control_message = '{"transfer_id":"' + normalized_transfer_id + '","type":"close"}'
        async with runner.send_lock:
            await runner.websocket.send_text(control_message)

async def client_frame(self, transfer_id: str, ciphertext: bytes) -> None:
    """Forward one bounded client ciphertext frame with an opaque UUID prefix."""
    normalized_transfer_id = _require_transfer_id(transfer_id)
    if not isinstance(ciphertext, bytes):
        raise TypeError("ciphertext must be bytes")
    if not 1 <= len(ciphertext) <= _RELAY_FRAME_BYTES_MAX:
        raise ValueError("Relay ciphertext frame has an invalid length")
    async with self._lock:
        client = self._clients.get(normalized_transfer_id)
        if client is None:
            raise RunnerRelayAuthorizationError("Runner relay client is not attached")
        runner = self._runners.get(client.grant.runner_id)
        if runner is None:
            raise RunnerRelayAuthorizationError("Runner is not connected")
        self._record_bytes(client, len(ciphertext))
        framed_ciphertext = uuid.UUID(normalized_transfer_id).bytes + ciphertext
    async with runner.send_lock:
        await runner.websocket.send_bytes(framed_ciphertext)

```

```

async def runner_closed_transfer(self, runner_id: str, transfer_id: str) -> None:
    """Close one failed transfer without disrupting other runner sessions."""
    normalized_transfer_id = _require_transfer_id(transfer_id)
    async with self._lock:
        runner = self._runners.get(runner_id)
        if runner is None:
            raise RunnerRelayAuthorizationError("Runner relay transfer is not attached")
        client = self._clients.get(normalized_transfer_id)
        if client is None or client.grant.runner_id != runner_id:
            if self._is_retired_transfer(runner, normalized_transfer_id):
                return
            raise RunnerRelayAuthorizationError("Runner relay transfer is not attached")
        del self._clients[normalized_transfer_id]
        runner.sessions.discard(normalized_transfer_id)
        self._retire_transfer(runner, normalized_transfer_id)
        self._close_client_queue(client)
    await client.websocket.close(code=4004, reason="Runner rejected transfer")

async def runner_frame(self, runner_id: str, framed_ciphertext: bytes) -> None:
    """Queue one bounded runner ciphertext frame for its attached client."""
    if not isinstance(framed_ciphertext, bytes):
        raise TypeError("framed_ciphertext must be bytes")
    if (
        not _RUNNER_PREFIX_LENGTH
        < len(framed_ciphertext)
        <= (_RUNNER_PREFIX_LENGTH + _RELAY_FRAME_BYTES_MAX)
    ):
        raise ValueError("Runner relay frame has an invalid length")
    transfer_id = str(uuid.UUID(bytes=framed_ciphertext[:_RUNNER_PREFIX_LENGTH]))
    ciphertext = framed_ciphertext[_RUNNER_PREFIX_LENGTH:]
    close_reason: str | None = None
    async with self._lock:
        runner = self._runners.get(runner_id)
        if runner is None:
            raise RunnerRelayAuthorizationError("Runner relay frame has no attached client")
        client = self._clients.get(transfer_id)
        if client is None or client.grant.runner_id != runner_id:
            if self._is_retired_transfer(runner, transfer_id):
                return
            raise RunnerRelayAuthorizationError("Runner relay frame has no attached client")
        if client.ciphertext_bytes + len(ciphertext) > client.grant.max_session_bytes:
            close_reason = "Runner relay session exceeded byte limit"
        elif client.outbound_queue.full():
            close_reason = "Runner relay client exceeded backpressure"
        else:

```

```

        self._record_bytes(client, len(ciphertext))
        client.outbound_queue.put_nowait(ciphertext)
    if close_reason is not None:
        del self._clients[transfer_id]
        runner.sessions.discard(transfer_id)
        self._retire_transfer(runner, transfer_id)
        self._close_client_queue(client)
    if close_reason is not None:
        control_message = '{"transfer_id":"' + transfer_id + '", "type": "close"}'
        async with runner.send_lock:
            await runner.websocket.send_text(control_message)
        await client.websocket.close(code=4005, reason=close_reason)

async def send_client_frames(self, transfer_id: str) -> None:
    """Drain runner ciphertext to one client until task cancellation."""
    normalized_transfer_id = _require_transfer_id(transfer_id)
    while True:
        async with self._lock:
            client = self._clients.get(normalized_transfer_id)
            if client is None:
                return
            queue = client.outbound_queue
            websocket = client.websocket
            ciphertext = await queue.get()
            if not ciphertext:
                return
            await websocket.send_bytes(ciphertext)

async def disconnect_runner(self, runner_id: str) -> None:
    """Remove and close a runner plus every client immediately after revocation."""
    clients_to_close: list[_ClientConnection] = []
    async with self._lock:
        runner = self._runners.pop(runner_id, None)
        self._maintenance.pop(runner_id, None)
        if runner is None:
            return
        for transfer_id in tuple(runner.sessions):
            client = self._clients.pop(transfer_id, None)
            if client is not None:
                self._close_client_queue(client)
                clients_to_close.append(client)
    await runner.websocket.close(code=4003, reason="Runner revoked")
    for client in clients_to_close:
        await client.websocket.close(code=4003, reason="Runner revoked")

def _retire_transfer(

```

```

        self,
        runner: _RunnerConnection,
        transfer_id: str,
    ) -> None:
        """Retain one bounded connection-local marker for late runner traffic."""
        now = self._monotonic()
        self._prune_retired_transfers(runner, now=now)
        runner.retired_transfers[transfer_id] = now
        runner.retired_transfers.move_to_end(transfer_id)
        while len(runner.retired_transfers) > _RETIRED_TRANSFERS_MAX:
            runner.retired_transfers.popitem(last=False)

    def _is_retired_transfer(
        self,
        runner: _RunnerConnection,
        transfer_id: str,
    ) -> bool:
        """Return whether this runner recently owned one detached transfer."""
        self._prune_retired_transfers(runner, now=self._monotonic())
        return transfer_id in runner.retired_transfers

    @staticmethod
    def _prune_retired_transfers(
        runner: _RunnerConnection,
        *,
        now: float,
    ) -> None:
        """Discard expired markers without scanning beyond their ordered prefix."""
        while runner.retired_transfers:
            _, retired_at = next(iter(runner.retired_transfers.items()))
            if now - retired_at <= _RETIRED_TRANSFER_TTL_SECONDS:
                return
            runner.retired_transfers.popitem(last=False)

    @staticmethod
    def _close_client_queue(client: _ClientConnection) -> None:
        """Wake one sender task, dropping stale queued frames during teardown."""
        while True:
            try:
                client.outbound_queue.put_nowait(b"")
                return
            except asyncio.QueueFull:
                client.outbound_queue.get_nowait()

    @staticmethod
    def _record_bytes(client: _ClientConnection, count: int) -> None:

```



```

        """Apply one shared bidirectional ciphertext byte budget."""
    if count < 1:
        raise ValueError("Relay ciphertext byte count must be positive")
    if client.ciphertext_bytes + count > client.grant.max_session_bytes:
        raise RunnerRelayAuthorizationError("Runner relay session exceeded byte limit")
    client.ciphertext_bytes += count

```

```
runner_relay_broker = RunnerRelayBroker(get_running_operation=_get_running_maintenance_operation)
```

17.17 yinshi/services/runner_rpc.py

RPC validation constrains method names, request identifiers, payload sizes, status codes, and serialized error forms.

```

"""Strict encrypted RPC envelope for the restricted BYOC worker surface."""

from __future__ import annotations

import json
import re
import uuid
from collections.abc import Callable
from dataclasses import dataclass
from typing import Any
from urllib.parse import urlsplit

from yinshi.services.runner_capabilities import RUNNER_PROTOCOL_VERSION
from yinshi.services.runner_noise_session import RunnerNoiseSession
from yinshi.services.runner_rpc_transport import (
    NOISE_CIPHERTEXT_BYTES_MAX as _NOISE_CIPHERTEXT_BYTES_MAX,
)
from yinshi.services.runner_rpc_transport import NOISE_TAG_BYTES as _NOISE_TAG_BYTES
from yinshi.services.runner_rpc_transport import TRANSPORT_ACK as _TRANSPORT_ACK
from yinshi.services.runner_rpc_transport import TRANSPORT_HEADER as _TRANSPORT_HEADER
from yinshi.services.runner_rpc_transport import TRANSPORT_MAGIC as _TRANSPORT_MAGIC
from yinshi.services.runner_rpc_transport import (
    TRANSPORT_PAYLOAD_BYTES_MAX as _TRANSPORT_PAYLOAD_BYTES_MAX,
)
from yinshi.services.runner_rpc_transport import TRANSPORT_PULL as _TRANSPORT_PULL
from yinshi.services.runner_rpc_transport import TRANSPORT_REQUEST as _TRANSPORT_REQUEST
from yinshi.services.runner_rpc_transport import TRANSPORT_RESPONSE as _TRANSPORT_RESPONSE
from yinshi.services.runner_rpc_transport import (
    fragment_count,
)
from yinshi.worker_runtime import WorkerHttpDispatcher

```

```

_REQUEST_KEYS_V1 = {"body", "method", "path", "request_id", "sequence", "type", "v"}
_REQUEST_KEYS_V2 = _REQUEST_KEYS_V1 | {"query"}
_REQUEST_BYTES_MAX = 2 * 1_024 * 1_024
_RESPONSE_BYTES_MAX = 10 * 1_024 * 1_024
_NOISE_PLAINTEXT_BYTES_MAX = _NOISE_CIPHERTEXT_BYTES_MAX - _NOISE_TAG_BYTES
# Capability budgets count ciphertext for requests, acknowledgements, pulls, and responses.
# Callers must include Noise tags and transport headers in their requested budget.
_RESOURCE_ID = r"[0-9a-f]{32}"
_REPOSITORY_MEMBER_PATH = re.compile(rf"^/api/repos/{_RESOURCE_ID}$")
_WORKSPACE_COLLECTION_PATH = re.compile(rf"^/api/repos/{_RESOURCE_ID}/workspaces$")
_WORKSPACE_MEMBER_PATH = re.compile(rf"^/api/workspaces/{_RESOURCE_ID}$")
_SESSION_COLLECTION_PATH = re.compile(rf"^/api/workspaces/{_RESOURCE_ID}/sessions$")
_SESSION_MEMBER_PATH = re.compile(rf"^/api/sessions/{_RESOURCE_ID}$")
_SESSION_READ_PATH = re.compile(rf"^/api/sessions/{_RESOURCE_ID}/(?:messages|tree)$")
_PROMPT_RUN_COLLECTION_PATH = re.compile(rf"^/api/sessions/{_RESOURCE_ID}/runs$")
_PROMPT_RUN_EVENTS_PATH = re.compile(
    rf"^/api/sessions/{_RESOURCE_ID}/runs/{_RESOURCE_ID}/events/[0-9]{{1,6}}$"
)
_PROMPT_RUN_CANCEL_PATH = re.compile(rf"^/api/sessions/{_RESOURCE_ID}/runs/{_RESOURCE_ID}/cancel$")
_WORKSPACE_FILE_READ_PATH = re.compile(
    rf"^/api/workspaces/{_RESOURCE_ID}/files/(?:changed|diff|preview|tree)$"
)
_WORKSPACE_FILE_WRITE_PATH = re.compile(rf"^/api/workspaces/{_RESOURCE_ID}/files/content$")
_TERMINAL_COLLECTION_PATH = re.compile(rf"^/api/workspaces/{_RESOURCE_ID}/terminals$")
_TERMINAL_MEMBER_PATH = re.compile(rf"^/api/workspaces/{_RESOURCE_ID}/terminals/{_RESOURCE_ID}$")
_TERMINAL_ACTION_PATH = re.compile(
    rf"^/api/workspaces/{_RESOURCE_ID}/terminals/{_RESOURCE_ID}/(?:input|resize|restart)$"
)
_TERMINAL_EVENTS_PATH = re.compile(
    rf"^/api/workspaces/{_RESOURCE_ID}/terminals/{_RESOURCE_ID}/events/[0-9]{{1,10}}$"
)
_PROVIDER_CONNECTION_COLLECTION_PATH = "/api/settings/connections"
_PROVIDER_CONNECTION_MEMBER_PATH = re.compile(rf"^/api/settings/connections/{_RESOURCE_ID}$")
_PROVIDER_AUTH_START_PATH = re.compile(rf"^/auth/providers/[a-z0-9-]{{1,64}}/start$")
_PROVIDER_AUTH_CALLBACK_PATH = re.compile(rf"^/auth/providers/[a-z0-9-]{{1,64}}/callback$")
_PI_CONFIG_COLLECTION_PATH = "/api/settings/pi-config"
_PI_UPLOAD_COLLECTION_PATH = "/api/settings/pi-config/uploads"
_PI_UPLOAD_CHUNK_PATH = re.compile(
    rf"^/api/settings/pi-config/uploads/{_RESOURCE_ID}/chunks/[0-9]{{1,5}}$"
)
_PI_UPLOAD_COMPLETE_PATH = re.compile(rf"^/api/settings/pi-config/uploads/{_RESOURCE_ID}/complete$")
_PI_UPLOAD_MEMBER_PATH = re.compile(rf"^/api/settings/pi-config/uploads/{_RESOURCE_ID}$")
DispatcherFactory = Callable[[str], WorkerHttpDispatcher]

```

```

@dataclass(frozen=True, slots=True)
class RunnerRpcRequest:
    """Validated ordered request decrypted only inside the user's runner."""

    version: int
    sequence: int
    request_id: str
    method: str
    path: str
    body: Any
    query: dict[str, str]

def _parse_request(plaintext: bytes, *, expected_sequence: int) -> RunnerRpcRequest:
    """Parse one exact RPC shape and enforce application ordering."""
    if not isinstance(plaintext, bytes):
        raise TypeError("Runner RPC plaintext must be bytes")
    if not plaintext or len(plaintext) > _REQUEST_BYTES_MAX:
        raise ValueError("Runner RPC request has an invalid length")
    try:
        payload = json.loads(plaintext)
    except (UnicodeDecodeError, json.JSONDecodeError) as exc:
        raise ValueError("Runner RPC request is not valid UTF-8 JSON") from exc
    if not isinstance(payload, dict):
        raise ValueError("Runner RPC request has an invalid shape")
    version = payload.get("v")
    expected_keys = _REQUEST_KEYS_V1 if version == 1 else _REQUEST_KEYS_V2
    if version not in {1, 2} or set(payload) != expected_keys:
        raise ValueError("Runner RPC request has an invalid shape or version")
    if payload.get("type") != "request":
        raise ValueError("Runner RPC request has an unsupported type")
    sequence = payload.get("sequence")
    if type(sequence) is not int or sequence != expected_sequence:
        raise ValueError("Runner RPC request sequence is out of order")
    request_id = payload.get("request_id")
    if not isinstance(request_id, str) or not request_id:
        raise ValueError("Runner RPC request_id must not be empty")
    try:
        normalized_request_id = str(uuid.UUID(request_id))
    except ValueError as exc:
        raise ValueError("Runner RPC request_id must be a UUID") from exc
    if normalized_request_id != request_id:
        raise ValueError("Runner RPC request_id must be canonical")
    method = payload.get("method")
    path = payload.get("path")
    if not isinstance(method, str) or not method:

```

```

        raise ValueError("Runner RPC method must not be empty")
    if not isinstance(path, str) or not path.startswith("/"):
        raise ValueError("Runner RPC path must be absolute")
    if "?" in path or "#" in path or "." in path or "\\" in path or "\\x00" in path:
        raise ValueError("Runner RPC path must be normalized")
    query_payload = payload.get("query", {})
    if not isinstance(query_payload, dict) or len(query_payload) > 16:
        raise ValueError("Runner RPC query must be a bounded object")
    query: dict[str, str] = {}
    for key, value in query_payload.items():
        if not isinstance(key, str) or not key or len(key) > 64:
            raise ValueError("Runner RPC query key is invalid")
        if not key.replace("_", "").isalnum():
            raise ValueError("Runner RPC query key is invalid")
        if not isinstance(value, str) or len(value) > 2_048:
            raise ValueError("Runner RPC query value is invalid")
        query[key] = value
    return RunnerRpcRequest(
        version=version,
        sequence=sequence,
        request_id=normalized_request_id,
        method=method,
        path=path,
        body=payload.get("body"),
        query=query,
    )

def _required_scope(request: RunnerRpcRequest) -> str:
    """Map one exact worker route to its least-privilege capability scope."""
    if request.method == "GET" and request.path == "/health":
        return "worker.health"
    is_repository_collection = request.path == "/api/repos"
    is_repository_member = _REPOSITORY_MEMBER_PATH.fullmatch(request.path) is not None
    if request.method == "GET" and (is_repository_collection or is_repository_member):
        return "repository.read"
    if request.method in {"POST", "PATCH", "DELETE"}:
        if is_repository_collection or is_repository_member:
            return "repository.write"

    is_workspace_collection = _WORKSPACE_COLLECTION_PATH.fullmatch(request.path) is not None
    is_workspace_member = _WORKSPACE_MEMBER_PATH.fullmatch(request.path) is not None
    if request.method == "GET" and (is_workspace_collection or is_workspace_member):
        return "workspace.read"
    if request.method in {"POST", "PATCH", "DELETE"} and (is_workspace_collection or is_workspace_member):
        return "workspace.write"

```

```

if request.method in {"PATCH", "DELETE"} and is_workspace_member:
    return "workspace.write"

is_session_collection = _SESSION_COLLECTION_PATH.fullmatch(request.path) is not None
is_session_member = _SESSION_MEMBER_PATH.fullmatch(request.path) is not None
is_session_read = _SESSION_READ_PATH.fullmatch(request.path) is not None
if request.method == "GET" and (is_session_collection or is_session_member or is_session_read):
    return "session.read"
if request.method == "POST" and is_session_collection:
    return "session.write"
if request.method == "PATCH" and is_session_member:
    return "session.write"

is_prompt_run_collection = _PROMPT_RUN_COLLECTION_PATH.fullmatch(request.path) is not None
is_prompt_run_events = _PROMPT_RUN_EVENTS_PATH.fullmatch(request.path) is not None
is_prompt_run_cancel = _PROMPT_RUN_CANCEL_PATH.fullmatch(request.path) is not None
if request.method == "POST" and (is_prompt_run_collection or is_prompt_run_cancel):
    return "session.stream"
if request.method == "GET" and is_prompt_run_events:
    return "session.stream"
if request.method == "GET" and _WORKSPACE_FILE_READ_PATH.fullmatch(request.path):
    return "files.read"
if request.method == "PUT" and _WORKSPACE_FILE_WRITE_PATH.fullmatch(request.path):
    return "files.write"

is_terminal_collection = _TERMINAL_COLLECTION_PATH.fullmatch(request.path) is not None
is_terminal_member = _TERMINAL_MEMBER_PATH.fullmatch(request.path) is not None
is_terminal_action = _TERMINAL_ACTION_PATH.fullmatch(request.path) is not None
is_terminal_events = _TERMINAL_EVENTS_PATH.fullmatch(request.path) is not None
if request.method == "POST" and (is_terminal_collection or is_terminal_action):
    return "terminal"
if request.method == "GET" and is_terminal_events:
    return "terminal"
if request.method == "DELETE" and is_terminal_member:
    return "terminal"

is_provider_collection = request.path == _PROVIDER_CONNECTION_COLLECTION_PATH
is_provider_member = _PROVIDER_CONNECTION_MEMBER_PATH.fullmatch(request.path) is not None
if request.method in {"GET", "POST"} and is_provider_collection:
    return "provider.configure"
if request.method == "DELETE" and is_provider_member:
    return "provider.configure"
if request.method == "GET" and request.path == "/api/catalog":
    return "provider.configure"
if request.method == "POST" and _PROVIDER_AUTH_START_PATH.fullmatch(request.path):
    return "provider.configure"
if request.method in {"GET", "POST"} and _PROVIDER_AUTH_CALLBACK_PATH.fullmatch(request.path):
    return "provider.configure"

```

```

if request.method in {"GET", "DELETE"} and request.path == _PI_CONFIG_COLLECTION_PATH:
    return "pi.configure"
if request.method == "GET" and request.path in {
    "/api/settings/pi-config/commands",
    "/api/settings/pi-release-notes",
}:
    return "pi.configure"
if request.method == "POST" and request.path in {
    "/api/settings/pi-config/github",
    "/api/settings/pi-config/sync",
}:
    return "pi.configure"
if request.method == "PATCH" and request.path == "/api/settings/pi-config/categories":
    return "pi.configure"
is_pi_upload_chunk = _PI_UPLOAD_CHUNK_PATH.fullmatch(request.path) is not None
is_pi_upload_complete = _PI_UPLOAD_COMPLETE_PATH.fullmatch(request.path) is not None
is_pi_upload_member = _PI_UPLOAD_MEMBER_PATH.fullmatch(request.path) is not None
if request.method == "POST" and (
    request.path == _PI_UPLOAD_COLLECTION_PATH or is_pi_upload_chunk or is_pi_upload_com
):
    return "pi.configure"
if request.method == "DELETE" and is_pi_upload_member:
    return "pi.configure"
raise ValueError("Runner RPC method or path is not allowed")

def _validate_route_query(request: RunnerRpcRequest) -> None:
    """Allow query values only on file routes whose contract requires a path."""
    if request.method == "GET" and _PROVIDER_AUTH_CALLBACK_PATH.fullmatch(request.path):
        flow_id = request.query.get("flow_id")
        if set(request.query) != {"flow_id"} or not isinstance(flow_id, str):
            raise ValueError("Runner provider callback requires one flow_id query value")
        try:
            normalized_flow_id = str(uuid.UUID(flow_id))
        except ValueError as exc:
            raise ValueError("Runner provider callback flow_id must be a UUID") from exc
        if normalized_flow_id != flow_id:
            raise ValueError("Runner provider callback flow_id must be canonical")
        return
    requires_file_path = request.path.endswith(("files/content", "files/diff", "files/pre
if requires_file_path:
    if set(request.query) != {"path"} or not request.query["path"]:
        raise ValueError("Runner file request requires one path query value")
    return
if request.query:
    raise ValueError("Runner RPC route does not accept query values")

```

```

def _validate_repository_import(request: RunnerRpcRequest) -> None:
    """Restrict direct worker imports to credential-free HTTPS repositories."""
    if request.method != "POST" or request.path != "/api/repos":
        return
    if not isinstance(request.body, dict):
        raise ValueError("Runner repository import body must be an object")
    if request.body.get("local_path") is not None:
        raise ValueError("Runner repository import cannot accept a local path")
    remote_url = request.body.get("remote_url")
    if not isinstance(remote_url, str) or not remote_url:
        raise ValueError("Runner repository import requires a remote URL")
    parsed_url = urlsplit(remote_url)
    if (
        parsed_url.scheme != "https"
        or not parsed_url.hostname
        or parsed_url.username is not None
        or parsed_url.password is not None
        or parsed_url.query
        or parsed_url.fragment
    ):
        raise ValueError("Runner repository import requires a credential-free HTTPS URL")

class EncryptedRunnerRpcSession:
    """Perform one capability handshake, then dispatch ordered encrypted RPCs."""

    def __init__(
        self,
        *,
        transfer_id: str,
        noise_session: RunnerNoiseSession,
        dispatcher_factory: DispatcherFactory | None = None,
    ) -> None:
        if not isinstance(transfer_id, str) or not transfer_id:
            raise ValueError("transfer_id must not be empty")
        try:
            normalized_transfer_id = str(uuid.UUID(transfer_id))
        except ValueError as exc:
            raise ValueError("transfer_id must be a UUID") from exc
        if normalized_transfer_id != transfer_id:
            raise ValueError("transfer_id must be canonical")
        if not isinstance(noise_session, RunnerNoiseSession):
            raise TypeError("noise_session must be RunnerNoiseSession")
        if dispatcher_factory is not None and not callable(dispatcher_factory):

```

```

        raise TypeError("dispatcher_factory must be callable or None")
self._transfer_id = normalized_transfer_id
self._noise_session = noise_session
self._dispatcher_factory = dispatcher_factory
self._dispatcher: WorkerHttpDispatcher | None = None
self._established = False
self._failed = False
self._next_request_sequence = 0
self._request_buffer: bytearray | None = None
self._request_fragment_count = 0
self._next_request_fragment = 0
self._response_payload: bytes | None = None
self._response_fragment_count = 0
self._next_response_fragment = 0

async def handle_frame(self, ciphertext: bytes, *, current_time: int) -> bytes:
    """Accept a handshake or decrypt, dispatch, and encrypt one RPC response."""
    if self._failed:
        raise RuntimeError("Runner RPC session failed and cannot be reused")
    if type(current_time) is not int or current_time < 0:
        raise ValueError("current_time must be a non-negative integer")
    if not self._established:
        return self._accept_handshake(ciphertext, current_time=current_time)

    try:
        plaintext = self._noise_session.decrypt(ciphertext)
        if plaintext.startswith(_TRANSPORT_MAGIC):
            response = await self._handle_transport_frame(plaintext)
            return self._noise_session.encrypt(response)
        if self._request_buffer is not None or self._response_payload is not None:
            raise ValueError("Runner RPC transport fragment is invalid")
        response = await self._dispatch_payload(plaintext)
        if len(response) <= _NOISE_PLAINTEXT_BYTES_MAX:
            return self._noise_session.encrypt(response)
        return self._noise_session.encrypt(self._begin_framed_response(response))
    except (TypeError, ValueError):
        self._failed = True
        self._clear_transport_buffers()
        raise

async def _handle_transport_frame(self, frame: bytes) -> bytes:
    """Validate one bounded transport fragment and advance exact stream state."""
    if len(frame) < _TRANSPORT_HEADER.size:
        raise ValueError("Runner RPC transport fragment is invalid")
    try:
        magic, kind, index, count, total = _TRANSPORT_HEADER.unpack(

```



```

        frame[: _TRANSPORT_HEADER.size]
    )
except ValueError as exc:
    raise ValueError("Runner RPC transport fragment is invalid") from exc
payload = frame[_TRANSPORT_HEADER.size :]
if magic != _TRANSPORT_MAGIC:
    raise ValueError("Runner RPC transport fragment is invalid")
if kind == _TRANSPORT_REQUEST:
    return await self._accept_request_fragment(
        index=index,
        count=count,
        total=total,
        payload=payload,
    )
if kind == _TRANSPORT_PULL:
    return self._accept_response_pull(
        index=index,
        count=count,
        total=total,
        payload=payload,
    )
raise ValueError("Runner RPC transport fragment is invalid")

async def _accept_request_fragment(
    self,
    *,
    index: int,
    count: int,
    total: int,
    payload: bytes,
) -> bytes:
    """Buffer one ordered request fragment and dispatch only when complete."""
    if self._response_payload is not None:
        raise ValueError("Runner RPC transport fragment is invalid")
    self._validate_fragment(
        index=index,
        count=count,
        total=total,
        payload_length=len(payload),
        total_limit=_REQUEST_BYTES_MAX,
    )
    if self._request_buffer is None:
        if index != 0:
            raise ValueError("Runner RPC transport fragment is invalid")
        self._request_buffer = bytearray(total)
        self._request_fragment_count = count

```

```

        self._next_request_fragment = 0
    if (
        count != self._request_fragment_count
        or total != len(self._request_buffer)
        or index != self._next_request_fragment
    ):
        raise ValueError("Runner RPC transport fragment is invalid")
    start = index * _TRANSPORT_PAYLOAD_BYTES_MAX
    self._request_buffer[start : start + len(payload)] = payload
    self._next_request_fragment += 1
    if self._next_request_fragment < count:
        return _TRANSPORT_HEADER.pack(
            _TRANSPORT_MAGIC,
            _TRANSPORT_ACK,
            index,
            count,
            total,
        )

    request_payload = bytes(self._request_buffer)
    self._request_buffer = None
    self._request_fragment_count = 0
    self._next_request_fragment = 0
    response = await self._dispatch_payload(request_payload)
    return self._begin_framed_response(response)

def _begin_framed_response(self, response: bytes) -> bytes:
    """Return first framed response and retain bounded remaining data for pulls."""
    if len(response) > _RESPONSE_BYTES_MAX:
        raise ValueError("Runner RPC response exceeded transport limit")
    response_count = self._fragment_count(len(response))
    first = self._pack_response_fragment(response, index=0, count=response_count)
    if response_count > 1:
        self._response_payload = response
        self._response_fragment_count = response_count
        self._next_response_fragment = 1
    return first

def _accept_response_pull(
    self,
    *,
    index: int,
    count: int,
    total: int,
    payload: bytes,
) -> bytes:

```

```

        """Return one requested response fragment in exact order."""
        response = self._response_payload
        if (
            self._request_buffer is not None
            or response is None
            or payload
            or index != self._next_response_fragment
            or count != self._response_fragment_count
            or total != len(response)
        ):
            raise ValueError("Runner RPC transport fragment is invalid")
        fragment = self._pack_response_fragment(response, index=index, count=count)
        self._next_response_fragment += 1
        if self._next_response_fragment == count:
            self._response_payload = None
            self._response_fragment_count = 0
            self._next_response_fragment = 0
        return fragment

    async def _dispatch_payload(self, plaintext: bytes) -> bytes:
        """Parse and dispatch one complete bounded RPC request payload."""
        request = _parse_request(
            plaintext,
            expected_sequence=self._next_request_sequence,
        )
        status_code, response_body = await self._dispatch(request)
        response = json.dumps(
            {
                "body": response_body,
                "request_id": request.request_id,
                "sequence": request.sequence,
                "status": status_code,
                "type": "response",
                "v": request.version,
            },
            separators=(",", ":"),
            sort_keys=True,
        ).encode("utf-8")
        self._next_request_sequence += 1
        return response

    @staticmethod
    def _fragment_count(total: int) -> int:
        """Return canonical fragment count for one payload length."""
        return fragment_count(total)

```

```

@classmethod
def _validate_fragment(
    cls,
    *,
    index: int,
    count: int,
    total: int,
    payload_length: int,
    total_limit: int,
) -> None:
    """Reject noncanonical fragment metadata before buffering payload bytes."""
    if total < 1 or total > total_limit or count != cls._fragment_count(total):
        raise ValueError("Runner RPC transport fragment is invalid")
    if index >= count:
        raise ValueError("Runner RPC transport fragment is invalid")
    expected_length = min(
        _TRANSPORT_PAYLOAD_BYTES_MAX,
        total - index * _TRANSPORT_PAYLOAD_BYTES_MAX,
    )
    if payload_length != expected_length:
        raise ValueError("Runner RPC transport fragment is invalid")

@staticmethod
def _pack_response_fragment(response: bytes, *, index: int, count: int) -> bytes:
    """Pack one canonical response fragment without copying remaining payload."""
    start = index * _TRANSPORT_PAYLOAD_BYTES_MAX
    end = min(len(response), start + _TRANSPORT_PAYLOAD_BYTES_MAX)
    return (
        _TRANSPORT_HEADER.pack(
            _TRANSPORT_MAGIC,
            _TRANSPORT_RESPONSE,
            index,
            count,
            len(response),
        )
        + response[start:end]
    )

def _clear_transport_buffers(self) -> None:
    """Release bounded request and response buffers after terminal failure."""
    self._request_buffer = None
    self._request_fragment_count = 0
    self._next_request_fragment = 0
    self._response_payload = None
    self._response_fragment_count = 0
    self._next_response_fragment = 0

```

```

def _accept_handshake(self, message: bytes, *, current_time: int) -> bytes:
    """Bind relay routing and any worker dispatcher to signed capability identity."""
    try:
        response = self._noise_session.accept_handshake(
            message,
            current_time=current_time,
        )
        capability = self._noise_session.capability
        if capability.transfer_id != self._transfer_id:
            raise ValueError("Runner capability transfer_id does not match relay routing")
        if self._dispatcher_factory is not None:
            dispatcher = self._dispatcher_factory(capability.user_id)
            if not isinstance(dispatcher, WorkerHttpDispatcher):
                raise TypeError("dispatcher_factory must return WorkerHttpDispatcher")
            if dispatcher.user_id != capability.user_id:
                raise ValueError("Runner worker tenant does not match capability identity")
            self._dispatcher = dispatcher
        self._established = True
        return response
    except (TypeError, ValueError):
        self._failed = True
        raise

async def _dispatch(self, request: RunnerRpcRequest) -> tuple[int, Any]:
    """Authorize and invoke an exact restricted worker operation."""
    required_scope = _required_scope(request)
    if required_scope not in self._noise_session.capability.scopes:
        raise ValueError(f"Runner capability does not allow {required_scope}")
    if request.path == "/health":
        if request.body is not None or request.query:
            raise ValueError("Runner health request body and query must be empty")
        return 200, {
            "protocol": RUNNER_PROTOCOL_VERSION,
            "status": "ok",
        }
    if self._dispatcher is None:
        raise ValueError("Runner worker dispatcher is unavailable")
    _validate_route_query(request)
    _validate_repository_import(request)
    worker_response = await self._dispatcher.request(
        method=request.method,
        path=request.path,
        body=request.body,
        query=request.query,
    )

```

```
        return worker_response.status_code, worker_response.body
```

17.18 yinshi/services/runner_rpc_transport.py

This service contains the policy and lifecycle rules for runner rpc transport.

```
"""Canonical encrypted RPC fragmentation constants and header codec."""
```

```
from __future__ import annotations
```

```
import struct
```

```
NOISE_CIPHERTEXT_BYTES_MAX = 65_535
```

```
NOISE_TAG_BYTES = 16
```

```
TRANSPORT_HEADER = struct.Struct(">4sBIII")
```

```
TRANSPORT_MAGIC = b"YRP1"
```

```
TRANSPORT_REQUEST = 1
```

```
TRANSPORT_ACK = 2
```

```
TRANSPORT_RESPONSE = 3
```

```
TRANSPORT_PULL = 4
```

```
TRANSPORT_PAYLOAD_BYTES_MAX = NOISE_CIPHERTEXT_BYTES_MAX - NOISE_TAG_BYTES - TRANSPORT_HEADER.size
```

```
def fragment_count(total: int) -> int:
```

```
    """Return canonical fragment count for one positive payload length."""
```

```
    if type(total) is not int or total < 1:
```

```
        raise ValueError("Runner RPC transport total must be positive")
```

```
    return max(1, (total + TRANSPORT_PAYLOAD_BYTES_MAX - 1) // TRANSPORT_PAYLOAD_BYTES_MAX)
```

17.19 yinshi/services/terminal_journal.py

This service contains the policy and lifecycle rules for terminal journal.

```
"""Bounded multiplexed terminal channels for reconnectable JSON transports."""
```

```
from __future__ import annotations
```

```
import asyncio
```

```
import json
```

```
import time
```

```
import uuid
```

```
from collections.abc import Awaitable, Callable
```

```
from dataclasses import dataclass, field
```

```
from typing import Any, Protocol
```

```
from yinshi.config import get_settings
```

```

_TERMINAL_ID_LENGTH = 32
_OUTPUT_EVENT_BYTES_MAX = 24_000
_OUTPUT_BUFFER_BYTES_MAX = 1_048_576
_OUTPUT_EVENT_COUNT_MAX = 1_000
_TERMINALS_PER_USER_MAX = 8
_EVENT_BATCH_BYTES_MAX = 48_000
_EVENT_BATCH_COUNT_MAX = 100

```

```

class TerminalWriter(Protocol):
    def write(self, data: bytes) -> None: ...

    async def drain(self) -> None: ...

    def close(self) -> None: ...

    async def wait_closed(self) -> None: ...

```

```

TerminalConnector = Callable[[str], Awaitable[tuple[asyncio.StreamReader, TerminalWriter]]]

```

```

class TerminalNotFoundError(LookupError):
    """Terminal does not exist for the selected account and workspace."""

```

```

class TerminalLimitError(RuntimeError):
    """Account already owns the maximum number of live terminals."""

```

```

class TerminalCursorExpiredError(RuntimeError):
    """Requested output cursor has fallen behind the bounded memory journal."""

```

```

@dataclass(frozen=True, slots=True)
class TerminalEventBatch:
    terminal_id: str
    events: tuple[dict[str, Any], ...]
    next_sequence: int
    closed: bool

```

```

@dataclass(slots=True)
class _TerminalChannel:
    id: str
    user_id: str

```

```

workspace_id: str
reader: asyncio.StreamReader
writer: TerminalWriter
attach_options: dict[str, Any]
events: list[tuple[int, dict[str, Any], int]] = field(default_factory=list)
next_sequence: int = 0
event_bytes: int = 0
closed: bool = False
write_lock: asyncio.Lock = field(default_factory=asyncio.Lock)
reader_task: asyncio.Task[None] | None = None
activity_monotonic: float = field(default_factory=time.monotonic)
output_available: asyncio.Event = field(default_factory=asyncio.Event)

async def _connect_sidecar(
    socket_path: str,
) -> tuple[asyncio.StreamReader, TerminalWriter]:
    reader, writer = await asyncio.open_unix_connection(
        socket_path,
        limit=8 * 1024 * 1024,
    )
    return reader, writer

class TerminalJournal:
    """Own sidecar terminal sockets while encrypted clients poll bounded output."""

    def __init__(
        self,
        *,
        connector: TerminalConnector | None = None,
        scrollback_lines: int | None = None,
        idle_seconds: int | None = None,
    ) -> None:
        selected_connector = connector or _connect_sidecar
        if not callable(selected_connector):
            raise TypeError("terminal connector must be callable")
        selected_scrollback_lines = (
            get_settings().terminal_scrollback_lines
            if scrollback_lines is None
            else scrollback_lines
        )
        if (
            type(selected_scrollback_lines) is not int
            or not 100 <= selected_scrollback_lines <= 100_000
        ):

```



```

        raise ValueError("terminal scrollbar_lines must be between 100 and 100000")
    selected_idle_seconds = (
        get_settings().terminal_keepalive_s if idle_seconds is None else idle_seconds
    )
    if type(selected_idle_seconds) is not int or not 60 <= selected_idle_seconds <= 86400:
        raise ValueError("terminal idle_seconds must be between 60 and 86400")
    self._connector = selected_connector
    self._scrollback_lines = selected_scrollback_lines
    self._idle_seconds = selected_idle_seconds
    self._channels: dict[str, _TerminalChannel] = {}
    self._starting_by_user: dict[str, int] = {}
    self._channels_lock = asyncio.Lock()
    self._closing = False

    async def start(
        self,
        *,
        user_id: str,
        workspace_id: str,
        socket_path: str,
        cwd: str,
        cols: int,
        rows: int,
    ) -> str:
        """Attach one terminal channel after enforcing per-account limits."""
        self._validate_identity(user_id, "user_id")
        self._validate_resource_id(workspace_id, "workspace_id")
        if not isinstance(socket_path, str) or not socket_path:
            raise ValueError("socket_path must not be empty")
        if not isinstance(cwd, str) or not cwd:
            raise ValueError("cwd must not be empty")
        self._validate_size(cols, rows)
        await self._reap_idle_channels()
        async with self._channels_lock:
            if self._closing:
                raise RuntimeError("terminal journal is closing")
            closed_terminal_ids = tuple(
                terminal_id
                for terminal_id, channel in self._channels.items()
                if channel.user_id == user_id and channel.closed
            )
            for terminal_id in closed_terminal_ids:
                self._channels.pop(terminal_id, None)
            active_count = sum(
                1
                for channel in self._channels.values()

```

```

        if channel.user_id == user_id and not channel.closed
    )
    starting_count = self._starting_by_user.get(user_id, 0)
    if active_count + starting_count >= _TERMINALS_PER_USER_MAX:
        raise TerminalLimitError("terminal limit reached")
    self._starting_by_user[user_id] = starting_count + 1

    try:
        reader, writer = await self._connector(socket_path)
    except BaseException:
        await self._release_start_reservation(user_id)
        raise
    try:
        init_line = await asyncio.wait_for(reader.readline(), timeout=10)
        if not init_line:
            raise ConnectionError("sidecar disconnected before terminal attach")
        init_message = self._decode_message(init_line)
        if init_message.get("type") != "init_status" or not init_message.get("success"):
            raise ConnectionError("sidecar terminal initialization failed")
        terminal_id = uuid.uuid4().hex
        attach_options = {
            "workspaceId": workspace_id,
            "cwd": cwd,
            "cols": cols,
            "rows": rows,
            "scrollbackLines": self._scrollback_lines,
        }
        channel = _TerminalChannel(
            id=terminal_id,
            user_id=user_id,
            workspace_id=workspace_id,
            reader=reader,
            writer=writer,
            attach_options=attach_options,
        )
        await self._send(
            channel,
            {"type": "terminal_attach", "id": workspace_id, "options": attach_options},
        )
    async with self._channels_lock:
        if self._closing:
            raise RuntimeError("terminal journal is closing")
        if terminal_id in self._channels:
            raise RuntimeError("terminal ID collision")
        self._channels[terminal_id] = channel
        channel.reader_task = asyncio.create_task(

```

```

        self._read_output(channel),
        name=f"terminal-journal-{terminal_id}",
    )
    return terminal_id
except BaseException:
    writer.close()
    await writer.wait_closed()
    raise
finally:
    await self._release_start_reservation(user_id)

async def input(
    self,
    *,
    user_id: str,
    workspace_id: str,
    terminal_id: str,
    data: str,
) -> None:
    """Forward bounded terminal input to one exact account channel."""
    channel = await self._channel(user_id, workspace_id, terminal_id)
    if not isinstance(data, str) or not data or len(data.encode("utf-8")) > 16_384:
        raise ValueError("terminal input has an invalid length")
    await self._send(
        channel,
        {"type": "terminal_input", "id": workspace_id, "data": data},
    )

async def resize(
    self,
    *,
    user_id: str,
    workspace_id: str,
    terminal_id: str,
    cols: int,
    rows: int,
) -> None:
    """Resize one terminal using bounded positive dimensions."""
    self._validate_size(cols, rows)
    channel = await self._channel(user_id, workspace_id, terminal_id)
    await self._send(
        channel,
        {
            "type": "terminal_resize",
            "id": workspace_id,
            "cols": cols,

```

```

        "rows": rows,
    },
)

async def restart(
    self,
    *,
    user_id: str,
    workspace_id: str,
    terminal_id: str,
) -> None:
    """Restart one sidecar terminal with its original reviewed options."""
    channel = await self._channel(user_id, workspace_id, terminal_id)
    await self._send(
        channel,
        {
            "type": "terminal_restart",
            "id": workspace_id,
            "options": channel.attach_options,
        },
    )

async def events(
    self,
    *,
    user_id: str,
    workspace_id: str,
    terminal_id: str,
    next_sequence: int,
    wait_seconds: float = 10.0,
) -> TerminalEventBatch:
    """Return one contiguous output page from a reconnect cursor."""
    if type(next_sequence) is not int or next_sequence < 0:
        raise ValueError("next_sequence must be a non-negative integer")
    if not isinstance(wait_seconds, (int, float)) or not 0 <= wait_seconds <= 10:
        raise ValueError("wait_seconds must be between 0 and 10")
    channel = await self._channel(user_id, workspace_id, terminal_id, allow_closed=True)
    if next_sequence == channel.next_sequence and not channel.closed and wait_seconds:
        try:
            await asyncio.wait_for(
                channel.output_available.wait(),
                timeout=float(wait_seconds),
            )
        except TimeoutError:
            pass
    channel.output_available.clear()

```

```

earliest_sequence = channel.events[0][0] if channel.events else channel.next_sequence
if next_sequence < earliest_sequence:
    raise TerminalCursorExpiredError("terminal output cursor expired")
if next_sequence > channel.next_sequence:
    raise ValueError("terminal output cursor is ahead of the journal")

selected_events: list[dict[str, Any]] = []
selected_bytes = 0
cursor = next_sequence
for sequence, event, event_bytes in channel.events:
    if sequence < cursor:
        continue
    if sequence != cursor:
        raise RuntimeError("terminal journal sequence is not contiguous")
    if selected_events and selected_bytes + event_bytes > _EVENT_BATCH_BYTES_MAX:
        break
    selected_events.append(event)
    selected_bytes += event_bytes
    cursor += 1
    if len(selected_events) >= _EVENT_BATCH_COUNT_MAX:
        break
channel.activity_monotonic = time.monotonic()
return TerminalEventBatch(
    terminal_id=terminal_id,
    events=tuple(selected_events),
    next_sequence=cursor,
    closed=channel.closed,
)

async def close(
    self,
    *,
    user_id: str,
    workspace_id: str,
    terminal_id: str,
) -> None:
    """Detach and remove one terminal channel idempotently."""
    try:
        channel = await self._channel(
            user_id,
            workspace_id,
            terminal_id,
            allow_closed=True,
        )
    except TerminalNotFoundError:
        return

```

```

        await self._close_channel(channel, detach=True)
    async with self._channels_lock:
        self._channels.pop(channel_id, None)

    async def _release_start_reservation(self, user_id: str) -> None:
        """Release one in-flight terminal slot after connection succeeds or fails."""
        async with self._channels_lock:
            count = self._starting_by_user.get(user_id, 0)
            if count <= 0:
                raise RuntimeError("terminal start reservation is missing")
            if count == 1:
                self._starting_by_user.pop(user_id, None)
            else:
                self._starting_by_user[user_id] = count - 1

    async def _reap_idle_channels(self) -> None:
        """Close channels that no client has polled or written within the idle limit."""
        cutoff = time.monotonic() - self._idle_seconds
        async with self._channels_lock:
            stale_channels = tuple(
                channel
                for channel in self._channels.values()
                if channel.activity_monotonic < cutoff
            )
            for channel in stale_channels:
                self._channels.pop(channel.id, None)
        for channel in stale_channels:
            await self._close_channel(channel, detach=True)

    async def close_all(self) -> None:
        """Detach all channels before application shutdown."""
        async with self._channels_lock:
            self._closing = True
            channels = tuple(self._channels.values())
            self._channels.clear()
        for channel in channels:
            await self._close_channel(channel, detach=True)

    async def _read_output(self, channel: _TerminalChannel) -> None:
        try:
            while True:
                line = await channel.reader.readline()
                if not line:
                    self._append_event(
                        channel,
                        {"type": "error", "error": "Terminal runtime disconnected"},

```

```

        )
        return
    message = self._decode_message(line)
    if message.get("type") == "init_status":
        continue
    for event in self._split_event(message):
        self._append_event(channel, event)
except (ConnectionError, OSError, UnicodeDecodeError, ValueError, json.JSONDecodeError):
    self._append_event(
        channel,
        {"type": "error", "error": "Terminal runtime disconnected"},
    )
finally:
    channel.closed = True
    channel.output_available.set()
    channel.writer.close()
    try:
        await channel.writer.wait_closed()
    except OSError:
        pass

def _append_event(self, channel: _TerminalChannel, event: dict[str, Any]) -> None:
    serialized = json.dumps(event, separators=(",", ":"), sort_keys=True)
    event_bytes = len(serialized.encode("utf-8"))
    if event_bytes > _OUTPUT_EVENT_BYTES_MAX:
        raise ValueError("terminal event exceeds the byte limit")
    sequence = channel.next_sequence
    channel.next_sequence += 1
    channel.events.append((sequence, event, event_bytes))
    channel.event_bytes += event_bytes
    while (
        channel.event_bytes > _OUTPUT_BUFFER_BYTES_MAX
        or len(channel.events) > _OUTPUT_EVENT_COUNT_MAX
    ):
        _sequence, _event, removed_bytes = channel.events.pop(0)
        channel.event_bytes -= removed_bytes
    assert channel.event_bytes >= 0
    channel.activity_monotonic = time.monotonic()
    channel.output_available.set()

@staticmethod
def _split_event(event: dict[str, Any]) -> tuple[dict[str, Any], ...]:
    event_type = event.get("type")
    if event_type == "terminal_ready" and isinstance(event.get("replay"), str):
        replay = event["replay"]
        ready_event = {key: value for key, value in event.items() if key != "replay"}

```

```

        ready_event["replay"] = ""
        if not replay:
            return (ready_event,)
        replay_events = TerminalJournal._split_terminal_data(replay)
        return (ready_event, *replay_events)
    if event_type != "terminal_data" or not isinstance(event.get("data"), str):
        return (event,)
    return TerminalJournal._split_terminal_data(event["data"])

    @staticmethod
    def _split_terminal_data(data: str) -> tuple[dict[str, Any], ...]:
        chunks: list[dict[str, Any]] = []
        current = ""
        current_bytes = 0
        for character in data:
            character_bytes = len(character.encode("utf-8"))
            if current and current_bytes + character_bytes > 20_000:
                chunks.append({"type": "terminal_data", "data": current})
                current = ""
                current_bytes = 0
            current += character
            current_bytes += character_bytes
        if current or not chunks:
            chunks.append({"type": "terminal_data", "data": current})
        return tuple(chunks)

    async def _channel(
        self,
        user_id: str,
        workspace_id: str,
        terminal_id: str,
        *,
        allow_closed: bool = False,
    ) -> _TerminalChannel:
        self._validate_identity(user_id, "user_id")
        self._validate_resource_id(workspace_id, "workspace_id")
        self._validate_resource_id(terminal_id, "terminal_id")
        async with self._channels_lock:
            channel = self._channels.get(terminal_id)
            if (
                channel is None
                or channel.user_id != user_id
                or channel.workspace_id != workspace_id
                or (channel.closed and not allow_closed)
            ):
                raise TerminalNotFoundError("terminal not found")

```



```

        return channel

    @staticmethod
    async def _send(channel: _TerminalChannel, message: dict[str, Any]) -> None:
        if channel.closed:
            raise TerminalNotFoundError("terminal is closed")
        async with channel.write_lock:
            channel.writer.write(
                (json.dumps(message, separators=(",", ":")) + "\n").encode("utf-8")
            )
            await channel.writer.drain()
        channel.activity_monotonic = time.monotonic()

    async def _close_channel(self, channel: _TerminalChannel, *, detach: bool) -> None:
        if detach and not channel.closed:
            try:
                await self._send(
                    channel,
                    {"type": "terminal_detach", "id": channel.workspace_id},
                )
            except (ConnectionError, OSError, TerminalNotFoundError):
                pass
        channel.closed = True
        channel.output_available.set()
        if channel.reader_task is not None and not channel.reader_task.done():
            channel.reader_task.cancel()
            try:
                await channel.reader_task
            except asyncio.CancelledError:
                pass
        channel.writer.close()
        try:
            await channel.writer.wait_closed()
        except OSError:
            pass

    @staticmethod
    def _decode_message(line: bytes) -> dict[str, Any]:
        if not isinstance(line, bytes) or not line or len(line) > 8 * 1024 * 1024:
            raise ValueError("sidecar terminal message has an invalid length")
        message = json.loads(line.decode("utf-8", errors="strict"))
        if not isinstance(message, dict) or not isinstance(message.get("type"), str):
            raise ValueError("sidecar terminal message must be a typed object")
        return message

    @staticmethod

```

```

def _validate_resource_id(value: str, name: str) -> None:
    if not isinstance(value, str) or len(value) != _TERMINAL_ID_LENGTH:
        raise ValueError(f"{name} must contain exactly 32 characters")
    if any(character not in "0123456789abcdef" for character in value):
        raise ValueError(f"{name} must be lowercase hexadecimal")

    @staticmethod
    def _validate_identity(value: str, name: str) -> None:
        if not isinstance(value, str) or not value or len(value) > 256:
            raise ValueError(f"{name} has an invalid length")

    @staticmethod
    def _validate_size(cols: int, rows: int) -> None:
        if type(cols) is not int or not 2 <= cols <= 500:
            raise ValueError("terminal cols must be between 2 and 500")
        if type(rows) is not int or not 2 <= rows <= 500:
            raise ValueError("terminal rows must be between 2 and 500")

```

17.20 yinshi/worker_auth.py

Worker authentication accepts only the configured shared token and rejects malformed or duplicate headers before adapter access.

```

"""Connection-scoped authentication for the in-process restricted worker app."""

from __future__ import annotations

import os
import secrets
import stat
from dataclasses import dataclass
from pathlib import Path

from starlette.types import ASGIApp, Receive, Scope, Send

from yinshi.tenant import TenantContext

@dataclass(frozen=True, slots=True)
class WorkerPrincipal:
    """One tenant and high-entropy bearer accepted by a worker application."""

    tenant: TenantContext
    bearer_token: str
    database_root: str | None = None

```

```

def __post_init__(self) -> None:
    if not isinstance(self.tenant, TenantContext):
        raise TypeError("worker tenant must be TenantContext")
    if not self.tenant.user_id or not self.tenant.email:
        raise ValueError("worker tenant identity must not be empty")
    data_directory = Path(self.tenant.data_dir)
    database_path = Path(self.tenant.db_path)
    if not data_directory.is_absolute() or not database_path.is_absolute():
        raise ValueError("worker tenant paths must be absolute")
    if database_path.name != "yinshi.db":
        raise ValueError("worker tenant database filename is invalid")
    if self.database_root is None:
        if database_path.parent != data_directory:
            raise ValueError("worker tenant database must live directly in its data directory")
        else:
            database_root = Path(self.database_root)
            if not database_root.is_absolute():
                raise ValueError("worker database root must be absolute")
            try:
                database_path.relative_to(database_root)
            except ValueError as exc:
                raise ValueError("worker tenant database must stay under database root") from exc
    if not isinstance(self.bearer_token, str):
        raise TypeError("worker bearer token must be a string")
    if not 32 <= len(self.bearer_token) <= 256:
        raise ValueError("worker bearer token must contain at least 32 characters")
    if not self.bearer_token.isascii() or any(
        character.isspace() for character in self.bearer_token
    ):
        raise ValueError("worker bearer token must be non-whitespace ASCII")

def prepare_worker_principal_storage(principal: WorkerPrincipal) -> None:
    """Create and validate the worker tenant's owner-only data directory."""
    if not isinstance(principal, WorkerPrincipal):
        raise TypeError("principal must be WorkerPrincipal")
    storage_directories = {Path(principal.tenant.data_dir)}
    if principal.database_root is not None:
        storage_directories.add(Path(principal.tenant.db_path).parent)
    for storage_directory in storage_directories:
        storage_directory.mkdir(mode=0o700, parents=True, exist_ok=True)
        metadata = storage_directory.lstat()
        if not stat.S_ISDIR(metadata.st_mode) or storage_directory.is_symlink():
            raise RuntimeError("worker tenant storage must be a real directory")
        if metadata.st_uid != os.geteuid():
            raise RuntimeError("worker tenant storage must be owned by the worker user")

```

```

        if stat.S_IMODE(metadata.st_mode) != 0o700:
            raise RuntimeError("worker tenant storage must have owner-only permissions")

class WorkerPrincipalMiddleware:
    """Authenticate every worker HTTP or WebSocket scope and inject its tenant."""

    def __init__(self, app: ASGIApp, *, principal: WorkerPrincipal) -> None:
        if not callable(app):
            raise TypeError("app must be an ASGI application")
        if not isinstance(principal, WorkerPrincipal):
            raise TypeError("principal must be WorkerPrincipal")
        self._app = app
        self._principal = principal

    async def __call__(self, scope: Scope, receive: Receive, send: Send) -> None:
        if scope["type"] not in {"http", "websocket"}:
            await self._app(scope, receive, send)
            return
        bearer_token = self._bearer_token(scope)
        if bearer_token is None or not secrets.compare_digest(
            bearer_token,
            self._principal.bearer_token,
        ):
            await self._reject(scope, send)
            return
        state = scope.setdefault("state", {})
        state["tenant"] = self._principal.tenant
        state["user_email"] = self._principal.tenant.email
        await self._app(scope, receive, send)

    @staticmethod
    def _bearer_token(scope: Scope) -> str | None:
        """Return one exact Authorization bearer, rejecting duplicates and invalid text."""
        authorization_values = [
            value for name, value in scope.get("headers", []) if name.lower() == b"authorization"
        ]
        if len(authorization_values) != 1:
            return None
        raw_authorization = authorization_values[0]
        if not isinstance(raw_authorization, bytes):
            return None
        try:
            authorization = raw_authorization.decode("ascii")
        except UnicodeDecodeError:
            return None

```

```

    scheme, separator, token = authorization.partition(" ")
    if scheme.lower() != "bearer" or separator != " " or not token:
        return None
    if token != token.strip() or " " in token:
        return None
    return token

    @staticmethod
    async def _reject(scope: Scope, send: Send) -> None:
        """Return a minimal protocol-appropriate authentication failure."""
        if scope["type"] == "websocket":
            await send({"type": "websocket.close", "code": 4401})
            return
        await send(
            {
                "type": "http.response.start",
                "status": 401,
                "headers": [(b"content-length", b"0")],
            }
        )
        await send({"type": "http.response.body", "body": b""})

```

17.21 yinshi/worker_runtime.py

The worker adapter exposes a restricted HTTP interface for runner-local repository, session, file, prompt, and terminal operations.

```

"""In-process HTTP adapter for the restricted worker application contract."""

from __future__ import annotations

import json
from dataclasses import dataclass
from typing import Any

import httpx
from fastapi import FastAPI

from yinshi.tenant import TenantContext
from yinshi.worker_auth import WorkerPrincipal

_WORKER_RESPONSE_BYTES_MAX = 1_048_576
_WORKER_PATH_LENGTH_MAX = 2_048
_WORKER_METHODS = frozenset({"DELETE", "GET", "PATCH", "POST", "PUT"})

```

```

@dataclass(frozen=True, slots=True)
class WorkerHttpResponse:
    """Bounded JSON response returned by the internal worker application."""

    status_code: int
    body: Any
    content_type: str | None

class WorkerHttpDispatcher:
    """Invoke execution routes through ASGI without opening a network listener."""

    def __init__(self, *, app: FastAPI, principal: WorkerPrincipal) -> None:
        if not isinstance(app, FastAPI):
            raise TypeError("app must be FastAPI")
        if app.state.mode != "worker":
            raise ValueError("app must use worker mode")
        if not isinstance(principal, WorkerPrincipal):
            raise TypeError("principal must be WorkerPrincipal")
        self._app = app
        self._principal = principal

    @property
    def app(self) -> FastAPI:
        """Return the worker application for runner lifecycle management."""
        return self._app

    @property
    def user_id(self) -> str:
        """Return the single tenant identity bound to this dispatcher."""
        return self._principal.tenant.user_id

    @property
    def tenant(self) -> TenantContext:
        """Return immutable account storage coordinates without bearer authority."""
        return self._principal.tenant

    @property
    def data_directory(self) -> str:
        """Return the runner-local opaque tenant directory."""
        return self._principal.tenant.data_dir

    @property
    def database_path(self) -> str:
        """Return the runner-local encrypted tenant database path."""
        return self._principal.tenant.db_path

```

```

async def request(
    self,
    *,
    method: str,
    path: str,
    body: Any,
    query: dict[str, str] | None = None,
) -> WorkerHttpResponse:
    """Dispatch one normalized HTTP-like request and return bounded JSON."""
    normalized_method = self._validate_method(method)
    normalized_path = self._validate_path(path)
    if normalized_method in {"GET", "DELETE"} and body is not None:
        raise ValueError(f"{normalized_method} worker request body must be null")
    request_kwargs: dict[str, Any] = {}
    normalized_query = self._validate_query(query)
    if normalized_query:
        request_kwargs["params"] = normalized_query
    if body is not None:
        try:
            json.dumps(body, separators=","), sort_keys=True)
        except (TypeError, ValueError) as exc:
            raise ValueError("worker request body must be JSON-serializable") from exc
    request_kwargs["json"] = body

    transport = httpx.ASGITransport(app=self._app, raise_app_exceptions=False)
    headers = {
        "Authorization": f"Bearer {self._principal.bearer_token}",
        "X-Requested-With": "XMLHttpRequest",
    }
    async with httpx.AsyncClient(
        transport=transport,
        base_url="http://localhost",
        headers=headers,
        follow_redirects=False,
        timeout=httpx.Timeout(30.0),
    ) as client:
        response = await client.request(
            normalized_method,
            normalized_path,
            **request_kwargs,
        )
    content = response.content
    if len(content) > _WORKER_RESPONSE_BYTES_MAX:
        raise RuntimeError("worker response exceeded the size limit")
    content_type = response.headers.get("content-type")

```

```

normalized_content_type = (
    content_type.split(";", maxsplit=1)[0].strip().lower()
    if content_type is not None
    else None
)
if response.status_code == 204:
    if content:
        raise RuntimeError("worker 204 response included a body")
    return WorkerHttpResponse(
        status_code=204,
        body=None,
        content_type=normalized_content_type,
    )
if normalized_content_type != "application/json":
    raise RuntimeError("worker response must use application/json")
try:
    response_body = response.json()
except json.JSONDecodeError as exc:
    raise RuntimeError("worker response contained invalid JSON") from exc
return WorkerHttpResponse(
    status_code=response.status_code,
    body=response_body,
    content_type=normalized_content_type,
)

@staticmethod
def _validate_query(query: dict[str, str] | None) -> dict[str, str]:
    """Return bounded scalar query data for internal ASGI dispatch."""
    if query is None:
        return {}
    if not isinstance(query, dict) or len(query) > 16:
        raise ValueError("worker request query must be a bounded object")
    normalized: dict[str, str] = {}
    for key, value in query.items():
        if not isinstance(key, str) or not key or len(key) > 64:
            raise ValueError("worker request query key is invalid")
        if not key.replace("_", "").isalnum():
            raise ValueError("worker request query key is invalid")
        if not isinstance(value, str) or len(value) > 2_048:
            raise ValueError("worker request query value is invalid")
        normalized[key] = value
    return normalized

@staticmethod
def _validate_method(method: str) -> str:
    """Return one allowlisted uppercase HTTP method."""

```



```

        if not isinstance(method, str) or method not in _WORKER_METHODS:
            raise ValueError("worker request method is not allowed")
        return method

    @staticmethod
    def _validate_path(path: str) -> str:
        """Return one relative normalized application path without query data."""
        if not isinstance(path, str) or not path or len(path) > _WORKER_PATH_LENGTH_MAX:
            raise ValueError("worker request path has an invalid length")
        if not path.startswith("/") or path.startswith("//"):
            raise ValueError("worker request path must be a relative application path")
        if "?" in path or "#" in path or "\\" in path or "\x00" in path:
            raise ValueError("worker request path must be normalized")
        if any(segment in {"", ".", ".."} for segment in path.split("/")[1:]):
            raise ValueError("worker request path must be normalized")
        return path

```

18 Desktop Application

Electron provides a local runtime without weakening hosted boundaries. The main process stores credentials in the operating-system keychain, supervises helper and sidecar processes, applies navigation and shell policy, and exposes a narrow preload API.

18.1 accountLease.ts

Account leases keep one desktop owner active and expire stale local authorization after bounded inactivity.

```

import { createPublicKey, timingSafeEqual, verify } from "node:crypto";

const LEASE_DURATION_SECONDS_MAX = 30 * 24 * 60 * 60;
const ED25519_SPKI_PREFIX = Buffer.from("302a300506032b6570032100", "hex");

export interface AccountLeaseClaims {
    readonly userId: string;
    readonly deviceId: string;
    readonly issuedAt: number;
    readonly expiresAt: number;
}

export interface VerifyAccountLeaseOptions {
    readonly token: string;
    readonly signingPublicKey: string;
    readonly currentTimeSeconds?: number;
}

```

```

function invalidLease(): never {
    throw new Error("account lease is invalid");
}

function decodeCanonicalBase64url(value: string): Buffer {
    if (typeof value !== "string" || value.length === 0 || !/^[A-Za-z0-9_-]+$/.test(value)) {
        return invalidLease();
    }
    const decoded = Buffer.from(value, "base64url");
    if (decoded.toString("base64url") !== value) {
        return invalidLease();
    }
    return decoded;
}

function parseJsonObject(encodedValue: string): Record<string, unknown> {
    let value: unknown;
    try {
        value = JSON.parse(decodeCanonicalBase64url(encodedValue).toString("utf8"));
    } catch {
        return invalidLease();
    }
    if (typeof value !== "object" || value === null || Array.isArray(value)) {
        return invalidLease();
    }
    return value as Record<string, unknown>;
}

export function assertSigningKeyContinuity(
    pinnedSigningPublicKey: string | undefined,
    receivedSigningPublicKey: string,
): string {
    if (typeof receivedSigningPublicKey !== "string") {
        throw new TypeError("receivedSigningPublicKey must be a string");
    }
    let receivedKey: Buffer;
    try {
        receivedKey = decodeCanonicalBase64url(receivedSigningPublicKey);
    } catch {
        throw new Error("desktop signing key is invalid");
    }
    if (receivedKey.length !== 32) {
        throw new Error("desktop signing key is invalid");
    }
    if (pinnedSigningPublicKey === undefined) {

```

```

        return receivedSigningPublicKey;
    }
    if (typeof pinnedSigningPublicKey !== "string") {
        throw new TypeError("pinnedSigningPublicKey must be a string or undefined");
    }
    let pinnedKey: Buffer;
    try {
        pinnedKey = decodeCanonicalBase64url(pinnedSigningPublicKey);
    } catch {
        throw new Error("pinned desktop signing key is invalid");
    }
    if (pinnedKey.length !== 32) {
        throw new Error("pinned desktop signing key is invalid");
    }
    if (!timingSafeEqual(pinnedKey, receivedKey)) {
        throw new Error("desktop signing key changed");
    }
    return pinnedSigningPublicKey;
}

export function verifyAccountLease(options: VerifyAccountLeaseOptions): AccountLeaseClaims {
    if (typeof options.token !== "string" || typeof options.signingPublicKey !== "string") {
        return invalidLease();
    }
    const segments = options.token.split(".");
    if (segments.length !== 3) {
        return invalidLease();
    }
    const [encodedHeader, encodedPayload, encodedSignature] = segments;
    if (encodedHeader === undefined || encodedPayload === undefined || encodedSignature === undefined) {
        return invalidLease();
    }

    const header = parseJsonObject(encodedHeader);
    if (header.alg !== "EdDSA" || header.typ !== "YINSHI-LEASE" || header.v !== 1) {
        return invalidLease();
    }
    const payload = parseJsonObject(encodedPayload);
    const userId = payload.sub;
    const deviceId = payload.device_id;
    const issuedAt = payload.iat;
    const expiresAt = payload.exp;
    if (payload.aud !== "yinshi-desktop" || payload.v !== 1) {
        return invalidLease();
    }
}

```

```

if (typeof userId !== "string" || userId.length === 0) {
    return invalidLease();
}
if (typeof deviceId !== "string" || deviceId.length === 0) {
    return invalidLease();
}
if (typeof issuedAt !== "number" || !Number.isSafeInteger(issuedAt)) {
    return invalidLease();
}
if (typeof expiresAt !== "number" || !Number.isSafeInteger(expiresAt)) {
    return invalidLease();
}

const currentTimeSeconds = options.currentTimeSeconds ?? Math.floor(Date.now() / 1_000);
if (!Number.isSafeInteger(currentTimeSeconds) || currentTimeSeconds < 1) {
    throw new TypeError("currentTimeSeconds must be a positive integer");
}
if (issuedAt > currentTimeSeconds + 60 || expiresAt <= currentTimeSeconds) {
    return invalidLease();
}
if (expiresAt <= issuedAt || expiresAt - issuedAt > LEASE_DURATION_SECONDS_MAX) {
    return invalidLease();
}

const rawPublicKey = decodeCanonicalBase64url(options.signingPublicKey);
const signature = decodeCanonicalBase64url(encodedSignature);
if (rawPublicKey.length !== 32 || signature.length !== 64) {
    return invalidLease();
}
let validSignature = false;
try {
    const publicKey = createPublicKey({
        key: Buffer.concat([ED25519_SPKI_PREFIX, rawPublicKey]),
        format: "der",
        type: "spki",
    });
    validSignature = verify(
        null,
        Buffer.from(`${encodedHeader}.${encodedPayload}`, "ascii"),
        publicKey,
        signature,
    );
} catch {
    return invalidLease();
}
if (!validSignature) {

```

```

    return invalidLease();
  }
  return { userId, deviceId, issuedAt, expiresAt };
}

```

18.2 accountSession.ts

The account session stores current sign-in state and coordinates renewal, logout, and hosted account transitions.

```

import { verifyAccountLease } from "../accountLease.js";
import type { DesktopCredentialProfile } from "../credentialStore.js";
import {
  readHostedDesktopTokenResponse,
  type HostedAuthCredentialStore,
} from "../hostedAuth.js";

export class DesktopSignInRequiredError extends Error {
  constructor() {
    super("desktop sign-in is required");
    this.name = "DesktopSignInRequiredError";
  }
}

export interface ResumeDesktopAccountOptions {
  readonly apiBaseUrl: string;
  readonly fetch: (
    input: string | URL,
    init?: RequestInit,
  ) => Promise<Response>;
  readonly credentialStore: HostedAuthCredentialStore;
  readonly currentTimeSeconds?: number;
}

export type DesktopAccountSession =
  | { readonly mode: "signed-out" }
  | { readonly mode: "offline"; readonly profile: DesktopCredentialProfile }
  | {
    readonly mode: "online";
    readonly accessToken: string;
    readonly accessTokenExpiresAt: number;
    readonly profile: DesktopCredentialProfile;
  };

function validateApiBaseUrl(value: string): URL {
  let url: URL;

```

```

    try {
        url = new URL(value);
    } catch {
        throw new TypeError("apiBaseUrl must be a valid URL");
    }
    if (url.protocol !== "https:" || url.username !== "" || url.password !== "") {
        throw new TypeError("apiBaseUrl must be an HTTPS URL without credentials");
    }
    url.pathname = `${url.pathname.replace(/\+/+$/, "")}/`;
    url.search = "";
    url.hash = "";
    return url;
}

function assertOfflineLease(
    profile: DesktopCredentialProfile,
    currentTimeSeconds: number,
): void {
    let claims;
    try {
        claims = verifyAccountLease({
            token: profile.accountLease,
            signingPublicKey: profile.signingPublicKey,
            currentTimeSeconds,
        });
    } catch {
        throw new DesktopSignInRequiredError();
    }
    if (
        claims.userId !== profile.user.id ||
        claims.deviceId !== profile.deviceId ||
        claims.expiresAt !== profile.accountLeaseExpiresAt
    ) {
        throw new DesktopSignInRequiredError();
    }
}

export async function resumeDesktopAccount(
    options: ResumeDesktopAccountOptions,
): Promise<DesktopAccountSession> {
    const apiBaseUrl = validateApiBaseUrl(options.apiBaseUrl);
    const currentTimeSeconds = (): number => {
        const value = options.currentTimeSeconds ?? Math.floor(Date.now() / 1_000);
        if (!Number.isSafeInteger(value) || value < 1) {
            throw new TypeError("currentTimeSeconds must be a positive integer");
        }
    }
}

```

```

        return value;
    };
    currentTimeSeconds();
    const profile = await options.credentialStore.load();
    if (profile === null) {
        return { mode: "signed-out" };
    }

    let refreshResponse: Response;
    try {
        refreshResponse = await options.fetch(
            new URL("auth/desktop/refresh", apiBaseUrl),
            {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ refresh_token: profile.refreshToken }),
                redirect: "error",
                signal: AbortSignal.timeout(15_000),
            },
        );
    } catch {
        assertOfflineLease(profile, currentTimeSeconds());
        return { mode: "offline", profile };
    }
    if (refreshResponse.status >= 500 && refreshResponse.status <= 599) {
        assertOfflineLease(profile, currentTimeSeconds());
        return { mode: "offline", profile };
    }
    let hostedSession;
    try {
        hostedSession = await readHostedDesktopTokenResponse({
            response: refreshResponse,
            currentTimeSeconds: currentTimeSeconds(),
            pinnedSigningPublicKey: profile.signingPublicKey,
            expectedUserId: profile.user.id,
            expectedDeviceId: profile.deviceId,
        });
    } catch {
        throw new DesktopSignInRequiredError();
    }
    await options.credentialStore.save(hostedSession.profile);
    return { mode: "online", ...hostedSession };
}

```

18.3 appController.ts

The application controller orders window creation, authentication, helper startup, local runtime selection, and shutdown.

```
import {
    DesktopSignInRequiredError,
    type DesktopAccountSession,
} from "./accountSession.js";
import type { DesktopCredentialProfile } from "./credentialStore.js";
import type { ManagedHelper } from "./helperSupervisor.js";
import type { HostedDesktopSession } from "./hostedAuth.js";

export interface DesktopAppControllerDependencies {
    resumeAccount(): Promise<DesktopAccountSession>;
    signIn(): Promise<HostedDesktopSession>;
    clearCredentials(): Promise<void>;
    startHelper(profile: DesktopCredentialProfile): Promise<ManagedHelper>;
    bootstrapHelper(helper: ManagedHelper): Promise<string>;
    showSignIn(): Promise<void>;
    loadApplication(origin: string): Promise<void>;
}

function assertDependencies(dependencies: DesktopAppControllerDependencies): void {
    const methods: Array<keyof DesktopAppControllerDependencies> = [
        "resumeAccount",
        "signIn",
        "clearCredentials",
        "startHelper",
        "bootstrapHelper",
        "showSignIn",
        "loadApplication",
    ];
    for (const method of methods) {
        if (typeof dependencies[method] !== "function") {
            throw new TypeError(`desktop app controller ${method} dependency is invalid`);
        }
    }
}

export class DesktopAppController {
    readonly #dependencies: DesktopAppControllerDependencies;
    #helper: ManagedHelper | undefined;
    #operation: Promise<void> = Promise.resolve();
    #started = false;
    #stopped = false;
}
```



```

constructor(dependencies: DesktopAppControllerDependencies) {
    assertDependencies(dependencies);
    this.#dependencies = dependencies;
}

#enqueue(operation: () => Promise<void>): Promise<void> {
    const result = this.#operation.then(operation);
    this.#operation = result.catch(() => undefined);
    return result;
}

async #stopHelper(): Promise<void> {
    const helper = this.#helper;
    this.#helper = undefined;
    if (helper !== undefined) {
        await helper.stop();
    }
}

async #loadProfile(profile: DesktopCredentialProfile): Promise<void> {
    await this.#stopHelper();
    const helper = await this.#dependencies.startHelper(profile);
    if (!helper.running || helper.processId < 1) {
        await helper.stop();
        throw new Error("desktop helper failed to start");
    }
    this.#helper = helper;
    try {
        const origin = await this.#dependencies.bootstrapHelper(helper);
        await this.#dependencies.loadApplication(origin);
    } catch (error) {
        await this.#stopHelper();
        throw error;
    }
}

start(): Promise<void> {
    return this.#enqueue(async () => {
        if (this.#stopped) {
            throw new Error("desktop app controller is stopped");
        }
        if (this.#started) {
            return;
        }
        this.#started = true;
    });
}

```

```

    let session: DesktopAccountSession;
    try {
        session = await this.#dependencies.resumeAccount();
    } catch (error) {
        if (error instanceof DesktopSignInRequiredError) {
            await this.#dependencies.showSignIn();
            return;
        }
        throw error;
    }
    if (session.mode === "signed-out") {
        await this.#dependencies.showSignIn();
        return;
    }
    await this.#loadProfile(session.profile);
});
}

signIn(): Promise<void> {
    return this.#enqueue(async () => {
        if (!this.#started || this.#stopped) {
            throw new Error("desktop app controller is not active");
        }
        const session = await this.#dependencies.signIn();
        await this.#loadProfile(session.profile);
    });
}

switchProfile(): Promise<void> {
    return this.#enqueue(async () => {
        if (!this.#started || this.#stopped) {
            throw new Error("desktop app controller is not active");
        }
        const session = await this.#dependencies.resumeAccount();
        if (session.mode === "signed-out") {
            throw new DesktopSignInRequiredError();
        }
        await this.#loadProfile(session.profile);
    });
}

signOut(): Promise<void> {
    return this.#enqueue(async () => {
        if (!this.#started || this.#stopped) {
            throw new Error("desktop app controller is not active");
        }
    });
}

```

```

        await this.#stopHelper();
        await this.#dependencies.clearCredentials();
        await this.#dependencies.showSignIn();
    });
}

stop(): Promise<void> {
    return this.#enqueue(async () => {
        if (this.#stopped) {
            return;
        }
        this.#stopped = true;
        await this.#stopHelper();
    });
}
}

```

18.4 authCallbackListener.ts

A loopback callback listener accepts only the expected authorization response and closes after one valid exchange.

```

import { createServer, type Server } from "node:http";
import type { AddressInfo } from "node:net";

const CALLBACK_PATH = "/auth/desktop/callback";
const REQUEST_TARGET_LENGTH_MAX = 2_048;

export interface AuthCallbackListener {
    readonly callbackUri: string;
    waitForCallback(): Promise<URL>;
    close(): Promise<void>;
}

export interface StartAuthCallbackListenerOptions {
    readonly timeoutMs: number;
}

function writeTextResponse(
    response: import("node:http").ServerResponse,
    statusCode: number,
    message: string,
): void {
    response.writeHead(statusCode, {
        "Cache-Control": "no-store",
        "Content-Security-Policy": "default-src 'none'",
    });
}

```

```

        "Content-Type": "text/plain; charset=utf-8",
        "Referrer-Policy": "no-referrer",
        "X-Content-Type-Options": "nosniff",
        Connection: "close",
    });
    response.end(message);
}

function closeServer(server: Server): Promise<void> {
    if (!server.listening) {
        return Promise.resolve();
    }
    return new Promise((resolve, reject) => {
        server.close((error) => {
            if (error !== undefined) {
                reject(error);
                return;
            }
            resolve();
        });
        server.closeIdleConnections();
    });
}

export async function startAuthCallbackListener(
    options: StartAuthCallbackListenerOptions,
): Promise<AuthCallbackListener> {
    if (!Number.isInteger(options.timeoutMs) || options.timeoutMs < 1) {
        throw new TypeError("timeoutMs must be a positive integer");
    }
    if (options.timeoutMs > 10 * 60 * 1_000) {
        throw new TypeError("timeoutMs must not exceed ten minutes");
    }

    let callbackOrigin: string | undefined;
    let settled = false;
    let resolveCallback: ((callback: URL) => void) | undefined;
    let rejectCallback: ((error: Error) => void) | undefined;
    const callbackPromise = new Promise<URL>((resolve, reject) => {
        resolveCallback = resolve;
        rejectCallback = reject;
    });
    void callbackPromise.catch(() => undefined);
    let timeout: NodeJS.Timeout | undefined;

    const server = createServer((request, response) => {

```

```

if (settled) {
    writeTextResponse(response, 410, "Authentication callback is no longer active.");
    return;
}
if (request.method !== "GET") {
    writeTextResponse(response, 405, "Method not allowed.");
    return;
}
if (request.url === undefined || request.url.length > REQUEST_TARGET_LENGTH_MAX) {
    writeTextResponse(response, 414, "Authentication callback is invalid.");
    return;
}
if (callbackOrigin === undefined) {
    writeTextResponse(response, 503, "Authentication callback is not ready.");
    return;
}

let callback: URL;
try {
    callback = new URL(request.url, callbackOrigin);
} catch {
    writeTextResponse(response, 400, "Authentication callback is invalid.");
    return;
}
if (callback.origin !== callbackOrigin || callback.pathname !== CALLBACK_PATH) {
    writeTextResponse(response, 404, "Not found.");
    return;
}
const codes = callback.searchParams.getAll("code");
const states = callback.searchParams.getAll("state");
if (codes.length !== 1 || states.length !== 1 || !codes[0] || !states[0]) {
    writeTextResponse(response, 400, "Authentication callback is invalid.");
    return;
}

settled = true;
if (timeout !== undefined) {
    clearTimeout(timeout);
}
writeTextResponse(
    response,
    200,
    "Authentication complete. You can close this window and return to Yinshi.",
);
resolveCallback?.(callback);
void closeServer(server).catch(() => undefined);

```

```

});

await new Promise<void>((resolve, reject) => {
    const onError = (error: Error): void => {
        server.off("listening", onListening);
        reject(error);
    };
    const onListening = (): void => {
        server.off("error", onError);
        resolve();
    };
    server.once("error", onError);
    server.once("listening", onListening);
    server.listen({ host: "127.0.0.1", port: 0, exclusive: true });
});
const address = server.address();
if (address === null || typeof address === "string") {
    await closeServer(server);
    throw new Error("desktop auth callback listener address is unavailable");
}
const callbackPort = (address as AddressInfo).port;
if (!Number.isInteger(callbackPort) || callbackPort < 1 || callbackPort > 65_535) {
    await closeServer(server);
    throw new Error("desktop auth callback listener port is invalid");
}
callbackOrigin = `http://127.0.0.1:${callbackPort}`;
const callbackUri = `${callbackOrigin}${CALLBACK_PATH}`;

timeout = setTimeout(() => {
    if (settled) {
        return;
    }
    settled = true;
    rejectCallback?.(new Error("desktop auth callback timed out"));
    void closeServer(server).catch(() => undefined);
}, options.timeoutMs);
timeout.unref();

let explicitClose: Promise<void> | undefined;
return {
    callbackUri,
    waitForCallback(): Promise<URL> {
        return callbackPromise;
    },
    close(): Promise<void> {
        explicitClose ??= (async () => {

```

```

        if (timeout !== undefined) {
            clearTimeout(timeout);
        }
        if (!settled) {
            settled = true;
            rejectCallback?.(new Error("desktop auth callback listener closed"));
        }
        await closeServer(server);
    })();
    return explicitClose;
},
};
}

```

18.5 automaticUpdates.ts

Update handling checks signed releases, reports progress, and defers installation until the application can restart safely.

```

const UPDATE_CHECK_DELAY_INITIAL_MS = 30_000;
const UPDATE_CHECK_INTERVAL_MS = 6 * 60 * 60 * 1_000;

export interface UpdateAdapter {
    autoDownload: boolean;
    autoInstallOnAppQuit: boolean;
    allowDowngrade: boolean;
    allowPrerelease: boolean;
    disableWebInstaller: boolean;
    logger: unknown;
    on(event: "update-downloaded" | "error", listener: () => void): void;
    checkForUpdates(): Promise<unknown>;
}

export interface ScheduledUpdateTask {
    cancel(): void;
}

export interface AutomaticUpdateOptions {
    readonly isPackaged: boolean;
    readonly updater: UpdateAdapter;
    readonly schedule: (delayMs: number, callback: () => void) => ScheduledUpdateTask;
    readonly onDownloaded: () => void;
}

export interface AutomaticUpdateController {
    stop(): void;
}

```

```

}

const sanitizedLogger = Object.freeze({
  debug: (): void => undefined,
  error: (): void => undefined,
  info: (): void => undefined,
  warn: (): void => undefined,
});

export function startAutomaticUpdates(
  options: AutomaticUpdateOptions,
): AutomaticUpdateController {
  if (!options.isPackaged) {
    return Object.freeze({ stop: (): void => undefined });
  }
  if (typeof options.schedule !== "function" || typeof options.onDownloaded !== "function")
    throw new TypeError("automatic update callbacks are invalid");
}

options.updater.autoDownload = true;
options.updater.autoInstallOnAppQuit = true;
options.updater.allowDowngrade = false;
options.updater.allowPrerelease = false;
options.updater.disableWebInstaller = true;
options.updater.logger = sanitizedLogger;

let stopped = false;
let scheduledTask: ScheduledUpdateTask | undefined;
const scheduleCheck = (delayMs: number): void => {
  if (stopped) {
    return;
  }
  scheduledTask = options.schedule(delayMs, () => {
    void (async () => {
      if (stopped) {
        return;
      }
      try {
        await options.updater.checkForUpdates();
      } catch {
        // Update errors are intentionally sanitized and retried on the fixed interval.
      }
      scheduleCheck(UPDATE_CHECK_INTERVAL_MS);
    })();
  });
};

```



```

options.updater.on("update-downloaded", () => {
  if (!stopped) {
    options.onDownloaded();
  }
});
options.updater.on("error", () => undefined);
scheduleCheck(UPDATE_CHECK_DELAY_INITIAL_MS);

return Object.freeze({
  stop(): void {
    if (stopped) {
      return;
    }
    stopped = true;
    scheduledTask?.cancel();
    scheduledTask = undefined;
  },
});
}

```

18.6 credentialStore.ts

The credential store places account secrets in the operating-system keychain instead of application files.

```

import {
  SecureJsonStore,
  type SafeStorageAdapter,
} from "../secureJsonStore.js";

export type { SafeStorageAdapter } from "../secureJsonStore.js";

export interface DesktopCredentialProfile {
  readonly version: 1;
  readonly refreshToken: string;
  readonly refreshTokenExpiresAt: number;
  readonly accountLease: string;
  readonly accountLeaseExpiresAt: number;
  readonly signingPublicKey: string;
  readonly deviceId: string;
  readonly user: {
    readonly id: string;
    readonly email: string;
  };
}

```

```

export interface DesktopProfileSummary {
    readonly user: DesktopCredentialProfile["user"];
    readonly hasCredentials: boolean;
    readonly active: boolean;
}

interface DesktopCredentialVaultEntry {
    readonly user: DesktopCredentialProfile["user"];
    readonly profile: DesktopCredentialProfile | null;
}

interface DesktopCredentialVault {
    readonly version: 2;
    readonly activeUserId: string | null;
    readonly entries: readonly DesktopCredentialVaultEntry[];
}

export interface DesktopCredentialStoreOptions {
    readonly directoryPath: string;
    readonly safeStorage: SafeStorageAdapter;
}

function requiredString(value: unknown, name: string): string {
    if (typeof value !== "string" || value.length === 0) {
        throw new Error(`desktop credential ${name} is invalid`);
    }
    return value;
}

function positiveInteger(value: unknown, name: string): number {
    if (typeof value !== "number" || !Number.isSafeInteger(value) || value < 1) {
        throw new Error(`desktop credential ${name} is invalid`);
    }
    return value;
}

function validateProfile(value: unknown): DesktopCredentialProfile {
    if (typeof value !== "object" || value === null || Array.isArray(value)) {
        throw new Error("desktop credential profile is invalid");
    }
    const profile = value as Record<string, unknown>;
    if (profile.version !== 1) {
        throw new Error("desktop credential profile version is unsupported");
    }
    const userValue = profile.user;
    if (typeof userValue !== "object" || userValue === null || Array.isArray(userValue)) {

```

```

        throw new Error("desktop credential user is invalid");
    }
    const user = userValue as Record<string, unknown>;
    const refreshTokenExpiresAt = positiveInteger(
        profile.refreshTokenExpiresAt,
        "refreshTokenExpiresAt",
    );
    const accountLeaseExpiresAt = positiveInteger(
        profile.accountLeaseExpiresAt,
        "accountLeaseExpiresAt",
    );
    if (accountLeaseExpiresAt > refreshTokenExpiresAt) {
        throw new Error("desktop credential expiry order is invalid");
    }
    return {
        version: 1,
        refreshToken: requiredString(profile.refreshToken, "refreshToken"),
        refreshTokenExpiresAt,
        accountLease: requiredString(profile.accountLease, "accountLease"),
        accountLeaseExpiresAt,
        signingPublicKey: requiredString(profile.signingPublicKey, "signingPublicKey"),
        deviceId: requiredString(profile.deviceId, "deviceId"),
        user: {
            id: requiredString(user.id, "user.id"),
            email: requiredString(user.email, "user.email"),
        },
    };
}

function validateVault(value: unknown): DesktopCredentialVault {
    if (typeof value === "object" && value !== null && !Array.isArray(value)) {
        const candidate = value as Record<string, unknown>;
        if (candidate.version === 1) {
            const profile = validateProfile(value);
            return {
                version: 2,
                activeUserId: profile.user.id,
                entries: [{ user: profile.user, profile }],
            };
        }
        if (candidate.version === 2) {
            if (!Array.isArray(candidate.entries) || candidate.entries.length > 32) {
                throw new Error("desktop credential vault entries are invalid");
            }
            const entries = candidate.entries.map((entryValue) => {
                if (typeof entryValue !== "object" || entryValue === null || Array.isArray(entryValue))

```

```

        throw new Error("desktop credential vault entry is invalid");
    }
    const entry = entryValue as Record<string, unknown>;
    const userValue = entry.user;
    if (typeof userValue !== "object" || userValue === null || Array.isArray(userValue)) {
        throw new Error("desktop credential vault user is invalid");
    }
    const userRecord = userValue as Record<string, unknown>;
    const user = {
        id: requiredString(userRecord.id, "vault.user.id"),
        email: requiredString(userRecord.email, "vault.user.email"),
    };
    const profile = entry.profile === null ? null : validateProfile(entry.profile);
    if (
        profile !== null &&
        (profile.user.id !== user.id || profile.user.email !== user.email)
    ) {
        throw new Error("desktop credential vault profile identity changed");
    }
    return { user, profile };
});
const userIds = entries.map((entry) => entry.user.id);
if (new Set(userIds).size !== userIds.length) {
    throw new Error("desktop credential vault contains duplicate users");
}
const activeUserId = candidate.activeUserId;
if (activeUserId !== null && typeof activeUserId !== "string") {
    throw new Error("desktop credential vault active user is invalid");
}
if (
    activeUserId !== null &&
    !entries.some(
        (entry) => entry.user.id === activeUserId && entry.profile !== null,
    )
) {
    throw new Error("desktop credential vault active profile is unavailable");
}
return { version: 2, activeUserId, entries };
}
}
throw new Error("desktop credential vault is invalid");
}

const EMPTY_VAULT: DesktopCredentialVault = {
    version: 2,
    activeUserId: null,

```

```

    entries: [],
  };

export class DesktopCredentialStore {
  readonly #store: SecureJsonStore<DesktopCredentialVault>;
  #operation: Promise<void> = Promise.resolve();

  constructor(options: DesktopCredentialStoreOptions) {
    this.#store = new SecureJsonStore({
      directoryPath: options.directoryPath,
      fileName: "credentials.bin",
      safeStorage: options.safeStorage,
      validate: validateVault,
    });
  }

  #enqueue<T>(operation: () => Promise<T>): Promise<T> {
    const result = this.#operation.then(operation, operation);
    this.#operation = result.then(
      () => undefined,
      () => undefined,
    );
    return result;
  }

  save(profileValue: DesktopCredentialProfile): Promise<void> {
    const profile = validateProfile(profileValue);
    return this.#enqueue(async () => {
      const vault = (await this.#store.load()) ?? EMPTY_VAULT;
      const entries = vault.entries.filter((entry) => entry.user.id !== profile.user.id);
      entries.push({ user: profile.user, profile });
      await this.#store.save({
        version: 2,
        activeUserId: profile.user.id,
        entries,
      });
    });
  }

  load(): Promise<DesktopCredentialProfile | null> {
    return this.#enqueue(async () => {
      const vault = await this.#store.load();
      if (vault === null || vault.activeUserId === null) return null;
      return (
        vault.entries.find((entry) => entry.user.id === vault.activeUserId)?.profile ??
        null
      );
    });
  }
}

```

```

    );
  });
}

list(): Promise<DesktopProfileSummary[]> {
  return this.#enqueue(async () => {
    const vault = await this.#store.load();
    if (vault === null) return [];
    return vault.entries.map((entry) => ({
      user: entry.user,
      hasCredentials: entry.profile !== null,
      active: entry.user.id === vault.activeUserId,
    }));
  });
}

select(userIdValue: string): Promise<DesktopCredentialProfile | null> {
  const userId = requiredString(userIdValue, "selectedUserId");
  return this.#enqueue(async () => {
    const vault = await this.#store.load();
    if (vault === null) return null;
    const selectedEntry = vault.entries.find((entry) => entry.user.id === userId);
    if (selectedEntry?.profile === null || selectedEntry === undefined) return null;
    await this.#store.save({ ...vault, activeUserId: userId });
    return selectedEntry.profile;
  });
}

clear(): Promise<void> {
  return this.#enqueue(async () => {
    const vault = await this.#store.load();
    if (vault === null || vault.activeUserId === null) return;
    const entries = vault.entries.map((entry) =>
      entry.user.id === vault.activeUserId ? { ...entry, profile: null } : entry,
    );
    await this.#store.save({ version: 2, activeUserId: null, entries });
  });
}

remove(userIdValue: string): Promise<void> {
  const userId = requiredString(userIdValue, "removedUserId");
  return this.#enqueue(async () => {
    const vault = await this.#store.load();
    if (vault === null) return;
    const entries = vault.entries.filter((entry) => entry.user.id !== userId);
    if (entries.length === 0) {

```

```

        await this.#store.clear();
        return;
    }
    await this.#store.save({
        version: 2,
        activeUserId: vault.activeUserId === userId ? null : vault.activeUserId,
        entries,
    });
});
}
}

```

18.7 desktopApi.ts

The main-process API validates renderer requests before invoking privileged desktop operations.

```

export const DESKTOP_IPC_CHANNELS = Object.freeze({
    signIn: "desktop:account:sign-in",
    signOut: "desktop:account:sign-out",
    importLocalRepository: "desktop:repository:import-local",
    hostedRequest: "desktop:hosted-api:request",
    fileVaultStatus: "desktop:security:file-vault-status",
    listProfiles: "desktop:profiles:list",
    switchProfile: "desktop:profiles:switch",
    removeProfile: "desktop:profiles:remove",
});

export interface HostedApiRequest {
    readonly method: "DELETE" | "GET" | "PATCH" | "POST" | "PUT";
    readonly path: string;
    readonly body?: Readonly<Record<string, unknown>>;
}

export interface HostedApiResponse {
    readonly status: number;
    readonly body: unknown;
}

export type LocalRepositoryImportResult =
    | { readonly status: "cancelled" }
    | {
        readonly status: "imported";
        readonly repository: { readonly id: string; readonly name: string };
    };

```

```

export interface DesktopProfileSummary {
  readonly user: { readonly id: string; readonly email: string };
  readonly hasCredentials: boolean;
  readonly active: boolean;
}

export interface YinshiDesktopApi {
  signIn(): Promise<void>;
  signOut(): Promise<void>;
  importLocalRepository(): Promise<LocalRepositoryImportResult>;
  hostedRequest(request: HostedApiRequest): Promise<HostedApiResponse>;
  fileVaultStatus(): Promise<{ readonly status: "disabled" | "enabled" | "unknown" }>;
  listProfiles(): Promise<readonly DesktopProfileSummary[]>;
  switchProfile(userId: string): Promise<void>;
  removeProfile(userId: string): Promise<void>;
}

declare global {
  interface Window {
    readonly yinshiDesktop: YinshiDesktopApi;
  }
}

```

18.8 diskEncryption.ts

Disk encryption checks block local workspace use when required storage protection is unavailable.

```

import { execFile } from "node:child_process";
import { promisify } from "node:util";

export interface FileVaultStatus {
  readonly status: "disabled" | "enabled" | "unknown";
}

interface ExecuteResult {
  readonly stdout: string;
  readonly stderr: string;
}

type Execute = (
  file: string,
  arguments_: readonly string[],
  options: { readonly timeout: number; readonly maxBuffer: number; readonly windowsHide: boolean }
) => Promise<ExecuteResult>;

```



```

const executeFile = promisify(execFile) as unknown as Execute;

export async function detectFileVaultStatus(
  execute: Execute = executeFile,
): Promise<FileVaultStatus> {
  if (typeof execute !== "function") {
    throw new TypeError("FileVault execute dependency must be callable");
  }
  try {
    const result = await execute("/usr/bin/fdesetup", ["status"], {
      timeout: 5_000,
      maxBuffer: 16_384,
      windowsHide: true,
    });
    if (result.stderr.trim()) return { status: "unknown" };
    const output = result.stdout.trim();
    if (output === "FileVault is On.") return { status: "enabled" };
    if (output === "FileVault is Off.") return { status: "disabled" };
    return { status: "unknown" };
  } catch {
    return { status: "unknown" };
  }
}

```

18.9 helperBootstrap.ts

Helper bootstrap verifies packaged binaries and starts the local backend with a minimal inherited environment.

```

import type { HelperReadyMessage } from "./helperProtocol.js";

const INSTANCE_NONCE_PATTERN = /^[A-Za-z0-9_-]{32,128}$/;

export interface BootstrapHelperSessionOptions {
  readonly ready: HelperReadyMessage;
  readonly fetch: (input: string | URL, init?: RequestInit) => Promise<Response>;
}

export async function bootstrapHelperSession(
  options: BootstrapHelperSessionOptions,
): Promise<string> {
  if (!Number.isInteger(options.ready.port) || options.ready.port < 1 || options.ready.port > 65535) {
    throw new TypeError("helper bootstrap port is invalid");
  }
  if (!INSTANCE_NONCE_PATTERN.test(options.ready.instanceNonce)) {
    throw new TypeError("helper bootstrap nonce is invalid");
  }
}

```

```

}
if (typeof options.fetch !== "function") {
  throw new TypeError("helper bootstrap fetch adapter is invalid");
}

const origin = `http://127.0.0.1:${options.ready.port}`;
const bootstrapResponse = await options.fetch(`${origin}/desktop/bootstrap`, {
  method: "POST",
  headers: { "X-Yinshi-Bootstrap": options.ready.instanceNonce },
  redirect: "error",
  signal: AbortSignal.timeout(5_000),
});
if (bootstrapResponse.status !== 204) {
  throw new Error("desktop helper bootstrap was rejected");
}

const healthResponse = await options.fetch(`${origin}/health`, {
  redirect: "error",
  signal: AbortSignal.timeout(5_000),
});
if (healthResponse.status !== 200) {
  throw new Error("desktop helper session was not established");
}
let health: unknown;
try {
  health = await healthResponse.json();
} catch {
  throw new Error("desktop helper health response was invalid");
}
if (
  typeof health !== "object" ||
  health === null ||
  Array.isArray(health) ||
  (health as Record<string, unknown>).status !== "ok"
) {
  throw new Error("desktop helper health response was invalid");
}
return origin;
}

```

18.10 helperProtocol.ts

The helper protocol defines typed startup, readiness, shutdown, and fault messages between supervised processes.

```

export const HELPER_PROTOCOL_VERSION = 1;

export interface HelperReadyMessage {
  port: number;
  instanceNonce: string;
}

const READY_FIELDS = new Set(["type", "protocolVersion", "port", "instanceNonce"]);
const INSTANCE_NONCE_PATTERN = /^[A-Za-z0-9_-]{32,128}$/;

export function parseHelperReadyLine(line: string): HelperReadyMessage {
  if (typeof line !== "string" || line.length === 0) {
    throw new TypeError("helper readiness must be valid JSON");
  }

  let value: unknown;
  try {
    value = JSON.parse(line);
  } catch {
    throw new TypeError("helper readiness must be valid JSON");
  }

  if (value === null || typeof value !== "object" || Array.isArray(value)) {
    throw new TypeError("helper readiness must be a JSON object");
  }

  const record = value as Record<string, unknown>;
  const fields = Object.keys(record);
  if (fields.length !== READY_FIELDS.size || fields.some((field) => !READY_FIELDS.has(field))) {
    throw new TypeError("helper readiness contains unexpected fields");
  }
  if (record.type !== "ready") {
    throw new TypeError("helper readiness type must be ready");
  }
  if (record.protocolVersion !== HELPER_PROTOCOL_VERSION) {
    throw new TypeError("helper readiness protocol version is unsupported");
  }
  if (!Number.isInteger(record.port) || (record.port as number) < 1 || (record.port as number) > 65535) {
    throw new TypeError("helper readiness port must be an integer between 1 and 65535");
  }
  if (typeof record.instanceNonce !== "string" || !INSTANCE_NONCE_PATTERN.test(record.instanceNonce)) {
    throw new TypeError("helper readiness instance nonce is invalid");
  }

  return {
    port: record.port as number,
    instanceNonce: record.instanceNonce,
  };
}

```

```
};
}
```

18.11 helperSupervisor.ts

The supervisor restarts failed helpers within limits and performs ordered shutdown without leaving child processes behind.

```
import { spawn, type ChildProcess } from "node:child_process";
import { createInterface } from "node:readline";
import { Readable } from "node:stream";

import { parseHelperReadyLine, type HelperReadyMessage } from "../helperProtocol.js";

export interface ManagedHelper {
  readonly ready: HelperReadyMessage;
  readonly processId: number;
  readonly running: boolean;
  stop(): Promise<void>;
}

export interface StartManagedHelperOptions {
  command: string;
  arguments: string[];
  environment: Record<string, string>;
  workingDirectory?: string;
  readinessTimeoutMs: number;
  shutdownTimeoutMs: number;
}

function childIsRunning(child: ChildProcess): boolean {
  return child.exitCode === null && child.signalCode === null;
}

function waitForExit(child: ChildProcess, timeoutMs: number): Promise<boolean> {
  if (!childIsRunning(child)) {
    return Promise.resolve(true);
  }
  return new Promise((resolve) => {
    const timeout = setTimeout(() => finish(false), timeoutMs);
    const onExit = () => finish(true);

    function finish(exited: boolean): void {
      clearTimeout(timeout);
      child.off("exit", onExit);
      resolve(exited);
    }
  });
}
```

```

    }

    child.once("exit", onExit);
  });
}

async function stopChild(child: ChildProcess, timeoutMs: number): Promise<void> {
  if (!childIsRunning(child)) {
    return;
  }

  const gracefulExit = waitForExit(child, timeoutMs);
  child.kill("SIGTERM");
  if (await gracefulExit) {
    return;
  }

  const forcedExit = waitForExit(child, timeoutMs);
  child.kill("SIGKILL");
  if (!(await forcedExit)) {
    throw new Error("helper did not stop after SIGKILL");
  }
}

async function readFirstReadyLine(readyPipe: Readable, timeoutMs: number): Promise<string> {
  const lines = createInterface({ input: readyPipe, crlfDelay: Infinity });
  let timeout: NodeJS.Timeout | undefined;
  const timeoutResult = new Promise<never>((_ , reject) => {
    timeout = setTimeout(
      () => reject(new Error("helper readiness timed out")),
      timeoutMs,
    );
  });

  try {
    const firstLine = await Promise.race([
      lines[Symbol.asyncIterator]() .next(),
      timeoutResult,
    ]);
    if (firstLine.done || typeof firstLine.value !== "string") {
      throw new Error("helper closed readiness pipe without a message");
    }
    return firstLine.value;
  } finally {
    if (timeout !== undefined) {
      clearTimeout(timeout);
    }
  }
}

```

```

    }
    lines.close();
  }
}

export async function startManagedHelper(
  options: StartManagedHelperOptions,
): Promise<ManagedHelper> {
  if (!Number.isInteger(options.readinessTimeoutMs) || options.readinessTimeoutMs < 1) {
    throw new TypeError("readinessTimeoutMs must be a positive integer");
  }
  if (!Number.isInteger(options.shutdownTimeoutMs) || options.shutdownTimeoutMs < 1) {
    throw new TypeError("shutdownTimeoutMs must be a positive integer");
  }

  const child = spawn(options.command, options.arguments, {
    cwd: options.workingDirectory,
    env: options.environment,
    stdio: ["ignore", "ignore", "ignore", "pipe"],
  });
  const readyPipe = child.stdio[3];
  if (!(readyPipe instanceof Readable)) {
    await stopChild(child, options.shutdownTimeoutMs);
    throw new Error("helper readiness pipe was unavailable");
  }

  let ready: HelperReadyMessage;
  try {
    const readyLine = await readFirstReadyLine(readyPipe, options.readinessTimeoutMs);
    ready = parseHelperReadyLine(readyLine);
  } catch (error) {
    await stopChild(child, options.shutdownTimeoutMs);
    throw error;
  }

  if (child.pid === undefined || child.pid < 1) {
    await stopChild(child, options.shutdownTimeoutMs);
    throw new Error("helper process id was unavailable");
  }

  let stopPromise: Promise<void> | undefined;
  return {
    ready,
    processId: child.pid,
    get running(): boolean {
      return childIsRunning(child);
    },
  },

```

```

    stop(): Promise<void> {
      stopPromise ??= stopChild(child, options.shutdownTimeoutMs);
      return stopPromise;
    },
  };
}

```

18.12 hostedAccessSession.ts

Hosted access sessions refresh browser-compatible account credentials without exposing them to local workspace commands.

```

import type { DesktopAccountSession } from "../accountSession.js";
import type { DesktopCredentialProfile } from "../credentialStore.js";

export interface HostedAccessSessionOptions {
  readonly resume: () => Promise<DesktopAccountSession>;
}

export class HostedAccessSession {
  readonly #resume: HostedAccessSessionOptions["resume"];
  #profile: DesktopCredentialProfile | undefined;
  #accessToken: string | undefined;
  #accessTokenExpiresAt: number | undefined;
  #refresh: Promise<DesktopAccountSession> | undefined;
  #epoch = 0;

  constructor(options: HostedAccessSessionOptions) {
    if (typeof options.resume !== "function") {
      throw new TypeError("HostedAccessSession requires a resume function");
    }
    this.#resume = options.resume;
  }

  get profile(): DesktopCredentialProfile | undefined {
    return this.#profile;
  }

  setAccount(account: DesktopAccountSession): void {
    if (account === null || typeof account !== "object") {
      throw new TypeError("Hosted account session must be an object");
    }
    this.#epoch += 1;
    this.#applyAccount(account);
  }
}

```

```

clear(): void {
    this.#epoch += 1;
    this.#profile = undefined;
    this.#accessToken = undefined;
    this.#accessTokenExpiresAt = undefined;
    this.#refresh = undefined;
}

getCachedAccessToken(currentTimeSeconds: number): string | undefined {
    this.#requireCurrentTime(currentTimeSeconds);
    if (
        this.#accessToken === undefined ||
        this.#accessTokenExpiresAt === undefined ||
        this.#accessTokenExpiresAt <= currentTimeSeconds + 30
    ) {
        return undefined;
    }
    return this.#accessToken;
}

async getAccessToken(currentTimeSeconds: number): Promise<string> {
    const cachedToken = this.getCachedAccessToken(currentTimeSeconds);
    if (cachedToken !== undefined) {
        return cachedToken;
    }
    const account = await this.resumeAccount();
    if (account.mode !== "online") {
        throw new Error("Hosted account is offline");
    }
    return account.accessToken;
}

/**
 * Single entry point for spending the stored refresh token. The control
 * plane treats a replayed refresh token as device compromise and revokes the
 * device, so every caller shares one in-flight attempt.
 */
async resumeAccount(): Promise<DesktopAccountSession> {
    const inFlight = this.#refresh;
    if (inFlight !== undefined) {
        return inFlight;
    }

    const refreshEpoch = this.#epoch;
    const refreshPromise = (async () => {
        const account = await this.#resume();

```



```

        if (this.#epoch !== refreshEpoch) {
            throw new Error("Hosted account changed while access was refreshing");
        }
        this.#applyAccount(account);
        return account;
    })();
    this.#refresh = refreshPromise;
    try {
        return await refreshPromise;
    } finally {
        if (this.#refresh === refreshPromise) {
            this.#refresh = undefined;
        }
    }
}

#applyAccount(account: DesktopAccountSession): void {
    this.#profile = account.mode === "signed-out" ? undefined : account.profile;
    this.#accessToken =
        account.mode === "online" ? account.accessToken : undefined;
    this.#accessTokenExpiresAt =
        account.mode === "online" ? account.accessTokenExpiresAt : undefined;
}

#requireCurrentTime(currentTimeSeconds: number): void {
    if (!Number.isSafeInteger(currentTimeSeconds) || currentTimeSeconds < 0) {
        throw new RangeError("currentTimeSeconds must be a non-negative integer");
    }
}
}

```

18.13 hostedApiGateway.ts

The gateway forwards approved hosted API calls and removes headers that the renderer must not control.

```

import type { HostedApiRequest, HostedApiResponse } from "../desktopApi.js";

const RESPONSE_BYTES_MAX = 1_048_576;
const REQUEST_BYTES_MAX = 65_536;
const REQUEST_TIMEOUT_MS = 15_000;

const RESOURCE_ID = "[0-9a-f]{32}";
const RUNNER_REQUESTS = new Set([
    "DELETE /api/settings/runner",
    "GET /api/settings/runner",

```

```

    "POST /api/settings/runner",
    "POST /api/settings/runner/capabilities",
    "POST /api/settings/runner/noise-key/confirm",
  ]);

function routeAllowed(
  method: HostedApiRequest["method"],
  pathValue: string,
): boolean {
  if (
    !pathValue.startsWith("/") ||
    pathValue.startsWith("//") ||
    pathValue.includes("\\")
  ) {
    return false;
  }
  let routeUrl: URL;
  try {
    routeUrl = new URL(pathValue, "https://hosted.invalid");
  } catch {
    return false;
  }
  if (routeUrl.origin !== "https://hosted.invalid" || routeUrl.hash)
    return false;
  const path = routeUrl.pathname;
  if (pathValue.split("?")[0] !== path) return false;
  const queryKeys = Array.from(routeUrl.searchParams.keys());
  const isFileRoute = new RegExp(
    `^/api/workspaces/${RESOURCE_ID}/files/(?::content|diff|preview)$`,
  ).test(path);
  const isProviderCallback = new RegExp(
    `^/auth/providers/[a-z0-9-]{1,64}/callback$`,
  ).test(path);
  if (queryKeys.length > 0) {
    const fileQuery =
      isFileRoute &&
      queryKeys.length === 1 &&
      queryKeys[0] === "path" &&
      routeUrl.searchParams.getAll("path").length === 1 &&
      Boolean(routeUrl.searchParams.get("path")) &&
      (routeUrl.searchParams.get("path")?.length ?? 0) <= 2_048;
    const providerQuery =
      isProviderCallback &&
      queryKeys.length === 1 &&
      queryKeys[0] === "flow_id" &&
      routeUrl.searchParams.getAll("flow_id").length === 1 &&

```

```

        /^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/ .test(
            routeUrl.searchParams.get("flow_id") ?? "",
        );
        if (!fileQuery && !providerQuery) return false;
    }
    if (RUNNER_REQUESTS.has(`${method} ${path}`)) return queryKeys.length === 0;
    if (
        method === "GET" &&
        ["/api/catalog", "/api/github/installations"].includes(path)
    ) {
        return queryKeys.length === 0;
    }
    if (method === "GET" && path === "/api/repos") return true;
    if (method === "POST" && path === "/api/repos") return true;
    if (new RegExp(`~/api/repos/${RESOURCE_ID}$`).test(path)) {
        return ["DELETE", "GET", "PATCH"].includes(method);
    }
    if (new RegExp(`~/api/repos/${RESOURCE_ID}/workspaces$`).test(path)) {
        return method === "GET" || method === "POST";
    }
    if (new RegExp(`~/api/workspaces/${RESOURCE_ID}$`).test(path)) {
        return method === "DELETE" || method === "PATCH";
    }
    if (new RegExp(`~/api/workspaces/${RESOURCE_ID}/sessions$`).test(path)) {
        return method === "GET" || method === "POST";
    }
    if (new RegExp(`~/api/sessions/${RESOURCE_ID}$`).test(path)) {
        return method === "GET" || method === "PATCH";
    }
    if (
        new RegExp(`~/api/sessions/${RESOURCE_ID}/(?:messages|tree)$`).test(path)
    ) {
        return method === "GET";
    }
    if (new RegExp(`~/api/sessions/${RESOURCE_ID}/runs$`).test(path))
        return method === "POST";
    if (
        new RegExp(
            `~/api/sessions/${RESOURCE_ID}/runs/${RESOURCE_ID}/events/[0-9]{1,6}$`,
        ).test(path)
    ) {
        return method === "GET";
    }
    if (
        new RegExp(
            `~/api/sessions/${RESOURCE_ID}/runs/${RESOURCE_ID}/cancel$`,

```

```

    ).test(path)
  ) {
    return method === "POST";
  }
  if (
    new RegExp(
      `~/api/workspaces/${RESOURCE_ID}/files/(?::changed|diff|preview|tree)$`,
    ).test(path)
  ) {
    return method === "GET";
  }
  if (new RegExp(`~/api/workspaces/${RESOURCE_ID}/files/content$`).test(path)) {
    return method === "PUT";
  }
  if (new RegExp(`~/api/workspaces/${RESOURCE_ID}/terminals$`).test(path)) {
    return method === "POST";
  }
  if (
    new RegExp(
      `~/api/workspaces/${RESOURCE_ID}/terminals/${RESOURCE_ID}$`,
    ).test(path)
  ) {
    return method === "DELETE";
  }
  if (
    new RegExp(
      `~/api/workspaces/${RESOURCE_ID}/terminals/${RESOURCE_ID}/(?::input|resize|restart)$`,
    ).test(path)
  ) {
    return method === "POST";
  }
  if (
    new RegExp(
      `~/api/workspaces/${RESOURCE_ID}/terminals/${RESOURCE_ID}/events/[0-9]{1,10}$`,
    ).test(path)
  ) {
    return method === "GET";
  }
  if (path === "/api/settings/connections")
    return method === "GET" || method === "POST";
  if (new RegExp(`~/api/settings/connections/${RESOURCE_ID}$`).test(path)) {
    return method === "DELETE";
  }
  if (path === "/api/settings/pi-config")
    return method === "GET" || method === "DELETE";
  if (

```

```

[
    "/api/settings/pi-config/commands",
    "/api/settings/pi-release-notes",
].includes(path)
) {
    return method === "GET";
}
if (
    ["/api/settings/pi-config/github", "/api/settings/pi-config/sync"].includes(
        path,
    )
) {
    return method === "POST";
}
if (path === "/api/settings/pi-config/categories") return method === "PATCH";
if (path === "/api/settings/pi-config/uploads") return method === "POST";
if (
    new RegExp(
        `~/api/settings/pi-config/uploads/${RESOURCE_ID}/chunks/[0-9]{1,5}$`,
    ).test(path)
) {
    return method === "POST";
}
if (
    new RegExp(
        `~/api/settings/pi-config/uploads/${RESOURCE_ID}/complete$`,
    ).test(path)
) {
    return method === "POST";
}
if (
    new RegExp(`~/api/settings/pi-config/uploads/${RESOURCE_ID}$`).test(path)
) {
    return method === "DELETE";
}
if (new RegExp(`~/auth/providers/[a-z0-9-]{1,64}/start$`).test(path)) {
    return method === "POST";
}
if (isProviderCallback) {
    if (method === "POST") return queryKeys.length === 0;
    if (method !== "GET") return false;
    return (
        queryKeys.length === 1 &&
        queryKeys[0] === "flow_id" &&
        routeUrl.searchParams.getAll("flow_id").length === 1 &&
        /^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/.test(

```

```

        routeUrl.searchParams.get("flow_id") ?? "",
    )
    );
}
return false;
}

export interface HostedApiGatewayOptions {
    readonly apiBaseUrl: string;
    readonly fetch: (
        input: string | URL,
        init?: RequestInit,
    ) => Promise<Response>;
    readonly getAccessToken: () => Promise<string>;
}

function validateBaseUrl(value: string): URL {
    let url: URL;
    try {
        url = new URL(value);
    } catch {
        throw new TypeError("apiBaseUrl must be a valid URL");
    }
    if (
        url.protocol !== "https:" ||
        url.username !== "" ||
        url.password !== "" ||
        url.search !== "" ||
        url.hash !== ""
    ) {
        throw new TypeError(
            "apiBaseUrl must be an HTTPS URL without credentials, query, or fragment",
        );
    }
    url.pathname = `${url.pathname.replace(/\+/+$/, "")}`;
    return url;
}

function validateRequest(value: HostedApiRequest): {
    method: HostedApiRequest["method"];
    path: string;
    bodyText: string | undefined;
} {
    if (value === null || typeof value !== "object") {
        throw new TypeError("Hosted API request must be an object");
    }
}

```

```

const keys = Object.keys(value);
if (keys.some((key) => !["body", "method", "path"].includes(key))) {
  throw new TypeError("Hosted API request contains unsupported fields");
}
if (!routeAllowed(value.method, value.path)) {
  throw new Error("Hosted API route is not allowed");
}
if (
  (value.method === "GET" || value.method === "DELETE") &&
  value.body !== undefined
) {
  throw new TypeError(
    `${value.method} hosted requests cannot include a body`,
  );
}
if (value.body === undefined) {
  if (value.method === "PATCH" || value.method === "PUT") {
    throw new TypeError(
      `${value.method} hosted requests require a JSON object body`,
    );
  }
  return { method: value.method, path: value.path, bodyText: undefined };
}
if (
  value.body === null ||
  typeof value.body !== "object" ||
  Array.isArray(value.body)
) {
  throw new TypeError("Hosted API request body must be a JSON object");
}
const bodyText = JSON.stringify(value.body);
if (new TextEncoder().encode(bodyText).length > REQUEST_BYTES_MAX) {
  throw new RangeError("Hosted API request exceeded the size limit");
}
return { method: value.method, path: value.path, bodyText };
}

async function readBoundedBody(response: Response): Promise<Uint8Array> {
  const declaredLength = response.headers.get("content-length");
  if (declaredLength !== null) {
    const parsedLength = Number.parseInt(declaredLength, 10);
    if (!Number.isSafeInteger(parsedLength) || parsedLength < 0) {
      throw new Error("Hosted API response Content-Length is invalid");
    }
    if (parsedLength > RESPONSE_BYTES_MAX) {
      throw new Error("Hosted API response exceeded the size limit");
    }
  }
}

```

```

    }
  }
  if (response.body === null) {
    return new Uint8Array();
  }
  const reader = response.body.getReader();
  const chunks: Uint8Array[] = [];
  let bytesRead = 0;
  try {
    while (true) {
      const { done, value } = await reader.read();
      if (done) {
        break;
      }
      bytesRead += value.length;
      if (bytesRead > RESPONSE_BYTES_MAX) {
        await reader.cancel();
        throw new Error("Hosted API response exceeded the size limit");
      }
      chunks.push(value);
    }
  } finally {
    reader.releaseLock();
  }
  const body = new Uint8Array(bytesRead);
  let offset = 0;
  for (const chunk of chunks) {
    body.set(chunk, offset);
    offset += chunk.length;
  }
  return body;
}

export class HostedApiGateway {
  readonly #apiBaseUrl: URL;
  readonly #fetch: HostedApiGatewayOptions["fetch"];
  readonly #getAccessToken: HostedApiGatewayOptions["getAccessToken"];

  constructor(options: HostedApiGatewayOptions) {
    this.#apiBaseUrl = validateBaseUrl(options.apiBaseUrl);
    if (
      typeof options.fetch !== "function" ||
      typeof options.getAccessToken !== "function"
    ) {
      throw new TypeError(
        "HostedApiGateway requires fetch and getAccessToken functions",
      );
    }
  }
}

```



```

    );
}
this.#fetch = options.fetch;
this.#getAccessToken = options.getAccessToken;
}

async request(requestValue: HostedApiRequest): Promise<HostedApiResponse> {
    const request = validateRequest(requestValue);
    const accessToken = await this.#getAccessToken();
    if (typeof accessToken !== "string" || accessToken.length < 32) {
        throw new Error("Hosted API access token is unavailable");
    }
    const headers: Record<string, string> = {
        Accept: "application/json",
        Authorization: `Bearer ${accessToken}`,
    };
    if (request.method !== "GET") {
        headers["X-Requested-With"] = "XMLHttpRequest";
    }
    if (request.bodyText !== undefined) {
        headers["Content-Type"] = "application/json";
    }
    const requestInit: RequestInit = {
        method: request.method,
        headers,
        redirect: "error",
        signal: AbortSignal.timeout(REQUEST_TIMEOUT_MS),
    };
    if (request.bodyText !== undefined) {
        requestInit.body = request.bodyText;
    }
    const response = await this.#fetch(
        new URL(request.path.slice(1), this.#apiBaseUrl),
        requestInit,
    );
    const bodyBytes = await readBoundedBody(response);
    if (response.status === 204) {
        if (bodyBytes.length !== 0) {
            throw new Error("Hosted API 204 response included a body");
        }
        return { status: response.status, body: null };
    }
    const contentType = response.headers
        .get("content-type")
        ?.split("; ", 1)[0]
        ?.trim();

```

```

    if (contentType !== "application/json") {
      throw new Error("Hosted API response must be JSON");
    }
    let body: unknown;
    try {
      body = JSON.parse(
        new TextDecoder("utf-8", { fatal: true }).decode(bodyBytes),
      );
    } catch {
      throw new Error("Hosted API response contained invalid JSON");
    }
    return { status: response.status, body };
  }
}

```

18.14 hostedAuth.ts

Hosted authentication opens system-browser sign-in and binds the callback result to the initiating desktop request.

```

import { createHash, randomBytes } from "node:crypto";

import {
  assertSigningKeyContinuity,
  verifyAccountLease,
} from "../accountLease.js";
import type { DesktopCredentialProfile } from "../credentialStore.js";

const ACCESS_DURATION_SECONDS_MAX = 15 * 60;
const AUTHORIZATION_DURATION_SECONDS_MAX = 10 * 60;
const AUTHORIZATION_EXPIRY_ROUNDING_MILLISECONDS = 1_000;
const REFRESH_DURATION_SECONDS_MAX = 90 * 24 * 60 * 60;

export interface HostedAuthCredentialStore {
  load(): Promise<DesktopCredentialProfile | null>;
  save(profile: DesktopCredentialProfile): Promise<void>;
}

export type HostedSignInStage =
  | "loading-profile"
  | "requesting-authorization"
  | "opening-browser"
  | "waiting-callback"
  | "exchanging-token"
  | "saving-profile";

```

```

export interface StartHostedSignInOptions {
  readonly apiBaseUrl: string;
  readonly callbackUri: string;
  readonly deviceName: string;
  readonly fetch: (
    input: string | URL,
    init?: RequestInit,
  ) => Promise<Response>;
  readonly openExternal: (url: string) => Promise<void>;
  readonly waitForCallback: (expectedState: string) => Promise<URL>;
  readonly credentialStore: HostedAuthCredentialStore;
  readonly currentTimeSeconds?: number;
  readonly onProgress?: (stage: HostedSignInStage) => void;
}

export interface HostedDesktopSession {
  readonly accessToken: string;
  readonly accessTokenExpiresAt: number;
  readonly profile: DesktopCredentialProfile;
}

function invalidResponse(): never {
  throw new Error("hosted desktop authentication response is invalid");
}

function requiredString(value: unknown): string {
  if (typeof value !== "string" || value.length === 0) {
    return invalidResponse();
  }
  return value;
}

function safeInteger(value: unknown): number {
  if (typeof value !== "number" || !Number.isSafeInteger(value)) {
    return invalidResponse();
  }
  return value;
}

function parseApiBaseUrl(value: string): URL {
  let url: URL;
  try {
    url = new URL(value);
  } catch {
    throw new TypeError("apiBaseUrl must be a valid URL");
  }
}

```

```

    if (
        url.protocol !== "https:" ||
        url.username !== "" ||
        url.password !== "" ||
        url.search !== "" ||
        url.hash !== ""
    ) {
        throw new TypeError(
            "apiBaseUrl must be an HTTPS URL without credentials or query data",
        );
    }
    url.pathname = `${url.pathname.replace(/\+/g, "%20")}/`;
    return url;
}

function parseCallbackUri(value: string): URL {
    let url: URL;
    try {
        url = new URL(value);
    } catch {
        throw new TypeError("callbackUri must be a valid URL");
    }
    if (
        url.protocol !== "http:" ||
        url.hostname !== "127.0.0.1" ||
        url.port !== "" ||
        url.pathname !== "/auth/desktop/callback" ||
        url.username !== "" ||
        url.password !== "" ||
        url.search !== "" ||
        url.hash !== ""
    ) {
        throw new TypeError(
            "callbackUri must be an exact loopback desktop callback URL",
        );
    }
    return url;
}

async function readJsonObject(
    response: Response,
    expectedStatus: number,
): Promise<Record<string, unknown>> {
    if (!(response instanceof Response) || response.status !== expectedStatus) {
        return invalidResponse();
    }
}

```

```

let value: unknown;
try {
    value = await response.json();
} catch {
    return invalidResponse();
}
if (typeof value !== "object" || value === null || Array.isArray(value)) {
    return invalidResponse();
}
return value as Record<string, unknown>;
}

function validateAuthorizeUrl(value: unknown): string {
    const urlValue = requiredString(value);
    let url: URL;
    try {
        url = new URL(urlValue);
    } catch {
        return invalidResponse();
    }
    if (url.protocol !== "https:" || url.username !== "" || url.password !== "") {
        return invalidResponse();
    }
    return url.toString();
}

function validateCallback(
    callbackUrl: URL,
    expectedCallback: URL,
    expectedState: string,
): string {
    if (!(callbackUrl instanceof URL)) {
        throw new Error("desktop authorization callback is invalid");
    }
    if (
        callbackUrl.origin !== expectedCallback.origin ||
        callbackUrl.pathname !== expectedCallback.pathname
    ) {
        throw new Error("desktop authorization callback is invalid");
    }
    const states = callbackUrl.searchParams.getAll("state");
    const codes = callbackUrl.searchParams.getAll("code");
    if (
        states.length !== 1 ||
        states[0] !== expectedState ||
        codes.length !== 1

```

```

    ) {
        throw new Error("desktop authorization callback is invalid");
    }
    const code = codes[0];
    if (code === undefined || code.length < 32 || code.length > 256) {
        throw new Error("desktop authorization callback is invalid");
    }
    return code;
}

export interface ReadHostedDesktopTokenResponseOptions {
    readonly response: Response;
    readonly currentTimeSeconds: number;
    readonly pinnedSigningPublicKey?: string;
    readonly expectedUserId?: string;
    readonly expectedDeviceId?: string;
}

export async function readHostedDesktopTokenResponse(
    options: ReadHostedDesktopTokenResponseOptions,
): Promise<HostedDesktopSession> {
    if (
        !Number.isSafeInteger(options.currentTimeSeconds) ||
        options.currentTimeSeconds < 1
    ) {
        throw new TypeError("currentTimeSeconds must be a positive integer");
    }
    const tokenBody = await readJsonObject(options.response, 200);
    if (tokenBody.token_type !== "Bearer") {
        return invalidResponse();
    }
    const accessToken = requiredString(tokenBody.access_token);
    const accessTokenExpiresAt = safeInteger(tokenBody.access_token_expires_at);
    const refreshToken = requiredString(tokenBody.refresh_token);
    const refreshTokenExpiresAt = safeInteger(tokenBody.refresh_token_expires_at);
    const accountLease = requiredString(tokenBody.account_lease);
    const accountLeaseExpiresAt = safeInteger(tokenBody.account_lease_expires_at);
    const deviceId = requiredString(tokenBody.device_id);
    const receivedSigningPublicKey = requiredString(tokenBody.signing_public_key);
    const signingPublicKey = assertSigningKeyContinuity(
        options.pinnedSigningPublicKey,
        receivedSigningPublicKey,
    );
    const userValue = tokenBody.user;
    if (
        typeof userValue !== "object" ||

```

```

    userValue === null ||
    Array.isArray(userValue)
  ) {
    return invalidResponse();
  }
  const user = userValue as Record<string, unknown>;
  const userId = requiredString(user.id);
  const userEmail = requiredString(user.email);
  if (
    accessTokenExpiresAt <= options.currentTimeSeconds ||
    accessTokenExpiresAt - options.currentTimeSeconds >
      ACCESS_DURATION_SECONDS_MAX
  ) {
    return invalidResponse();
  }
  if (
    refreshTokenExpiresAt <= options.currentTimeSeconds ||
    refreshTokenExpiresAt - options.currentTimeSeconds >
      REFRESH_DURATION_SECONDS_MAX
  ) {
    return invalidResponse();
  }

  const leaseClaims = verifyAccountLease({
    token: accountLease,
    signingPublicKey,
    currentTimeSeconds: options.currentTimeSeconds,
  });
  if (
    leaseClaims.expiresAt !== accountLeaseExpiresAt ||
    leaseClaims.userId !== userId ||
    leaseClaims.deviceId !== deviceId
  ) {
    return invalidResponse();
  }
  if (
    options.expectedUserId !== undefined &&
    options.expectedUserId !== userId
  ) {
    return invalidResponse();
  }
  if (
    options.expectedDeviceId !== undefined &&
    options.expectedDeviceId !== deviceId
  ) {
    return invalidResponse();
  }

```

```

    }
    const profile: DesktopCredentialProfile = {
      version: 1,
      refreshToken,
      refreshTokenExpiresAt,
      accountLease,
      accountLeaseExpiresAt,
      signingPublicKey,
      deviceId,
      user: { id: userId, email: userEmail },
    };
    return { accessToken, accessTokenExpiresAt, profile };
  }

export async function startHostedSignIn(
  options: StartHostedSignInOptions,
): Promise<HostedDesktopSession> {
  const apiBaseUrl = parseApiBaseUrl(options.apiBaseUrl);
  const callbackUrl = parseCallbackUri(options.callbackUri);
  if (
    typeof options.deviceName !== "string" ||
    options.deviceName.trim().length === 0
  ) {
    throw new TypeError("deviceName must be a non-empty string");
  }
  if (options.deviceName.trim().length > 100) {
    throw new TypeError("deviceName must not exceed 100 characters");
  }
  const authorizationCurrentTimeSeconds =
    options.currentTimeSeconds ?? Math.floor(Date.now() / 1_000);
  if (
    !Number.isSafeInteger(authorizationCurrentTimeSeconds) ||
    authorizationCurrentTimeSeconds < 1
  ) {
    throw new TypeError("currentTimeSeconds must be a positive integer");
  }

  options.onProgress?.("loading-profile");
  const existingProfile = await options.credentialStore.load();
  const codeVerifier = randomBytes(64).toString("base64url");
  const codeChallenge = createHash("sha256")
    .update(codeVerifier, "ascii")
    .digest("base64url");
  const state = randomBytes(32).toString("base64url");
  options.onProgress?.("requesting-authorization");
  const requestResponse = await options.fetch(

```



```

    new URL("auth/desktop/requests", apiBaseUrl),
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        redirect_uri: callbackUrl.toString(),
        code_challenge: codeChallenge,
        state,
      }),
      redirect: "error",
      signal: AbortSignal.timeout(15_000),
    },
  );
  const requestBody = await readJsonObject(requestResponse, 201);
  requiredString(requestBody.request_id);
  const authorizeUrl = validateAuthorizeUrl(requestBody.authorize_url);
  const authorizationExpiresAtMilliseconds = Date.parse(
    requiredString(requestBody.expires_at),
  );
  const currentTimeMilliseconds = authorizationCurrentTimeSeconds * 1_000;
  if (
    !Number.isSafeInteger(authorizationExpiresAtMilliseconds) ||
    authorizationExpiresAtMilliseconds <= currentTimeMilliseconds ||
    authorizationExpiresAtMilliseconds - currentTimeMilliseconds >
      AUTHORIZATION_DURATION_SECONDS_MAX * 1_000 +
      AUTHORIZATION_EXPIRY_ROUNDING_MILLISECONDS
  ) {
    return invalidResponse();
  }

  options.onProgress?.("opening-browser");
  await options.openExternal(authorizeUrl);
  options.onProgress?.("waiting-callback");
  const callback = await options.waitForCallback(state);
  const authorizationCode = validateCallback(callback, callbackUrl, state);
  options.onProgress?.("exchanging-token");
  const tokenResponse = await options.fetch(
    new URL("auth/desktop/token", apiBaseUrl),
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        authorization_code: authorizationCode,
        code_verifier: codeVerifier,
        device_name: options.deviceName.trim(),
      }),
    },
  );

```

```

        redirect: "error",
        signal: AbortSignal.timeout(15_000),
    },
);
const tokenCurrentTimeSeconds =
    options.currentTimeSeconds ?? Math.floor(Date.now() / 1_000);
const session = await readHostedDesktopTokenResponse({
    response: tokenResponse,
    currentTimeSeconds: tokenCurrentTimeSeconds,
    ...(existingProfile === null
        ? {}
        : { pinnedSigningPublicKey: existingProfile.signingPublicKey }),
});
options.onProgress?.("saving-profile");
await options.credentialStore.save(session.profile);
return session;
}

```

18.15 localRepositoryImport.ts

Local import validates repository roots and records an approved checkout without copying unrelated filesystem content.

```

import { execFile } from "node:child_process";
import { constants } from "node:fs";
import { access, lstat, mkdir, realpath, rm } from "node:fs/promises";
import path from "node:path";
import { promisify } from "node:util";

const executeFile = promisify(execFile);

export class DirtyRepositoryError extends Error {
    constructor() {
        super("selected repository has uncommitted changes");
        this.name = "DirtyRepositoryError";
    }
}

export class InvalidRepositoryError extends Error {
    constructor() {
        super("selected directory is not a safe Git repository");
        this.name = "InvalidRepositoryError";
    }
}

export interface CloneRepositoryOptions {

```

```

    readonly gitCommand: string;
    readonly sourcePath: string;
    readonly managedBasePath: string;
    readonly destinationPath: string;
    readonly allowDirty: boolean;
}

export interface ClonedRepository {
    readonly name: string;
    readonly dirty: boolean;
    readonly path: string;
}

function isInside(candidatePath: string, basePath: string): boolean {
    const relativePath = path.relative(basePath, candidatePath);
    return relativePath !== "" && !relativePath.startsWith(`.${path.sep}`) && relativePath !==
}

async function pathExists(candidatePath: string): Promise<boolean> {
    try {
        await lstat(candidatePath);
        return true;
    } catch (error) {
        if (error instanceof Error && "code" in error && error.code === "ENOENT") {
            return false;
        }
        throw error;
    }
}

async function runGit(
    gitCommand: string,
    arguments_: readonly string[],
): Promise<{ stdout: string; stderr: string }> {
    try {
        return await executeFile(gitCommand, [...arguments_], {
            encoding: "utf8",
            env: {
                HOME: process.env.HOME ?? "/var/empty",
                PATH: process.env.PATH ?? "/usr/bin:/bin",
                GIT_CONFIG_GLOBAL: "/dev/null",
                GIT_CONFIG_NOSYSTEM: "1",
                GIT_TERMINAL_PROMPT: "0",
            },
            maxBuffer: 2 * 1024 * 1024,
            timeout: 120_000,
        });
    }
}

```

```

    });
  } catch {
    throw new InvalidRepositoryError();
  }
}

export async function cloneRepositoryIntoProfile(
  options: CloneRepositoryOptions,
): Promise<ClonedRepository> {
  for (const [name, value] of [
    ["gitCommand", options.gitCommand],
    ["sourcePath", options.sourcePath],
    ["managedBasePath", options.managedBasePath],
    ["destinationPath", options.destinationPath],
  ] as const) {
    if (typeof value !== "string" || !path.isAbsolute(value) || value.includes("\0")) {
      throw new TypeError(`${name} must be an absolute path`);
    }
  }

  await access(options.gitCommand, constants.X_OK);
  await mkdir(options.managedBasePath, { mode: 0o700, recursive: true });
  const managedBasePath = await realpath(options.managedBasePath);
  const sourcePath = await realpath(options.sourcePath);
  const destinationParent = await realpath(path.dirname(options.destinationPath));
  const destinationPath = path.join(destinationParent, path.basename(options.destinationPath));
  if (!isInside(destinationPath, managedBasePath) || destinationParent !== managedBasePath) {
    throw new TypeError("destinationPath must be a direct child of managedBasePath");
  }

  if (await pathExists(destinationPath)) {
    throw new Error("managed repository destination already exists");
  }

  const topLevel = (
    await runGit(options.gitCommand, [
      "-C",
      sourcePath,
      "rev-parse",
      "--show-toplevel",
    ])
  ).stdout.trim();
  if (!topLevel || (await realpath(topLevel)) !== sourcePath) {
    throw new InvalidRepositoryError();
  }

  const status = await runGit(options.gitCommand, [
    "-C",
    sourcePath,

```

```

    "status",
    "--porcelain=v1",
    "--untracked-files=all",
  ]);
  const dirty = status.stdout.trim().length > 0;
  if (dirty && !options.allowDirty) {
    throw new DirtyRepositoryError();
  }

  try {
    await runGit(options.gitCommand, [
      "clone",
      "--local",
      "--no-hardlinks",
      "--no-checkout",
      "--",
      sourcePath,
      destinationPath,
    ]);
    await runGit(options.gitCommand, [
      "-C",
      destinationPath,
      "-c",
      "core.hooksPath=/dev/null",
      "checkout",
      "--force",
      "HEAD",
    ]);
  } catch (error) {
    await rm(destinationPath, { recursive: true, force: true });
    throw error;
  }

  const repositoryName = path.basename(sourcePath);
  if (!repositoryName || repositoryName === "." || repositoryName === "..") {
    await rm(destinationPath, { recursive: true, force: true });
    throw new InvalidRepositoryError();
  }

  return { name: repositoryName, dirty, path: destinationPath };
}

```

18.16 localRuntime.ts

Local runtime state exposes the supervised helper endpoint through the same contract used by hosted workspaces.

```

import type {
  ManagedHelper,
  StartManagedHelperOptions,
} from "./helperSupervisor.js";
import type { ChildLaunchConfig } from "./runtimeLaunchConfig.js";
import type { ManagedSidecar, SidecarOptions } from "./sidecarSupervisor.js";

export interface StartLocalRuntimeOptions {
  readonly helper: ChildLaunchConfig;
  readonly sidecar: ChildLaunchConfig;
  readonly socketPath: string;
  readonly startSidecar: (options: SidecarOptions) => Promise<ManagedSidecar>;
  readonly startHelper: (
    options: StartManagedHelperOptions,
  ) => Promise<ManagedHelper>;
}

export async function startLocalRuntime(
  options: StartLocalRuntimeOptions,
): Promise<ManagedHelper> {
  if (options.helper.environment.SIDECAR_SOCKET_PATH !== options.socketPath) {
    throw new Error(
      "helper sidecar socket path does not match runtime socket path",
    );
  }
  if (options.sidecar.environment.SIDECAR_SOCKET_PATH !== options.socketPath) {
    throw new Error("sidecar socket path does not match runtime socket path");
  }
  console.info("Desktop local runtime starting sidecar");
  const sidecar = await options.startSidecar({
    command: options.sidecar.command,
    args: options.sidecar.args,
    environment: options.sidecar.environment,
    workingDirectory: options.sidecar.workingDirectory,
    socketPath: options.socketPath,
    startupTimeoutMs: 15_000,
    shutdownTimeoutMs: 5_000,
  });

  console.info("Desktop local runtime sidecar ready");
  let helper: ManagedHelper;
  try {
    console.info("Desktop local runtime starting helper");
    helper = await options.startHelper({
      command: options.helper.command,
      arguments: [...options.helper.args],
    });
  }

```

```

        environment: { ...options.helper.environment },
        workingDirectory: options.helper.workingDirectory,
        readinessTimeoutMs: 15_000,
        shutdownTimeoutMs: 5_000,
    });
} catch (startupError) {
    try {
        await sidecar.stop();
    } catch (cleanupError) {
        throw new AggregateError(
            [startupError, cleanupError],
            "Local helper startup and sidecar cleanup failed",
        );
    }
    throw startupError;
}

console.info("Desktop local runtime helper ready");
let stopOperation: Promise<void> | undefined;
return {
    ready: helper.ready,
    processId: helper.processId,
    get running(): boolean {
        return helper.running && sidecar.running;
    },
    stop(): Promise<void> {
        stopOperation ??= (async () => {
            let helperError: unknown;
            try {
                await helper.stop();
            } catch (error) {
                helperError = error;
            }
            try {
                await sidecar.stop();
            } catch (sidecarError) {
                if (helperError !== undefined) {
                    throw new AggregateError(
                        [helperError, sidecarError],
                        "Local helper and sidecar shutdown failed",
                    );
                }
                throw sidecarError;
            }
        })
        if (helperError !== undefined) {
            throw helperError;
        }
    }
}

```

```

    }
  })();
  return stopOperation;
},
};
}

```

18.17 main.ts

The Electron entry point applies security policy before it delegates lifecycle work to the application controller.

```

import { createHash, randomUUID } from "node:crypto";
import { accessSync, constants } from "node:fs";
import { chmod, mkdir, rm } from "node:fs/promises";
import os from "node:os";
import path from "node:path";
import { fileURLToPath, pathToFileURL } from "node:url";

import {
  app,
  BrowserWindow,
  dialog,
  ipcMain,
  net,
  safeStorage,
  session,
  shell,
  type IpcMainInvokeEvent,
} from "electron";
import updaterPackage from "electron-updater";

import { resumeDesktopAccount } from "./accountSession.js";
import { DesktopAppController } from "./appController.js";
import {
  startAutomaticUpdates,
  type AutomaticUpdateController,
} from "./automaticUpdates.js";
import { startAuthCallbackListener } from "./authCallbackListener.js";
import { DesktopCredentialStore } from "./credentialStore.js";
import { detectFileVaultStatus } from "./diskEncryption.js";
import { DESKTOP_IPC_CHANNELS, type HostedApiRequest } from "./desktopApi.js";
import { bootstrapHelperSession } from "./helperBootstrap.js";
import { startManagedHelper, type ManagedHelper } from "./helperSupervisor.js";
import { HostedAccessSession } from "./hostedAccessSession.js";
import { HostedApiGateway } from "./hostedApiGateway.js";

```



```

import { startHostedSignIn, type HostedSignInStage } from "../hostedAuth.js";
import { startLocalRuntime } from "../localRuntime.js";
import {
  cloneRepositoryIntoProfile,
  DirtyRepositoryError,
} from "../localRepositoryImport.js";
import {
  buildRuntimeLaunchConfig,
  type RuntimeLaunchConfig,
} from "../runtimeLaunchConfig.js";
import { RuntimeSecretStore } from "../runtimeSecrets.js";
import { createBrowserWindowOptions } from "../security.js";
import { createShellPolicy } from "../shellPolicy.js";
import { startSidecar } from "../sidecarSupervisor.js";

const { autoUpdater } = updaterPackage;

const HOSTED_API_BASE_URL = "https://yinshi.io";
const EXTERNAL_ORIGINS = Object.freeze([
  "https://yinshi.io",
  "https://docs.yinshi.io",
  "https://github.com",
  "https://docs.github.com",
]);

const moduleDirectory = path.dirname(fileURLToPath(import.meta.url));
let mainWindow: BrowserWindow | undefined;
let controller: DesktopAppController | undefined;
let applicationOrigin: string | undefined;
let updateController: AutomaticUpdateController | undefined;
let quitting = false;

function shellEnvironment(): Record<string, string | undefined> {
  const user = os.userInfo();
  return {
    HOME: process.env.HOME ?? user.homedir,
    LANG: process.env.LANG ?? "en_US.UTF-8",
    LC_ALL: process.env.LC_ALL,
    LC_CTYPE: process.env.LC_CTYPE,
    LOGNAME: process.env.LOGNAME ?? user.username,
    PATH:
      process.env.PATH ??
      "/opt/homebrew/bin:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin",
    SHELL: process.env.SHELL ?? user.shell ?? "/bin/zsh",
    SSH_AUTH_SOCK: process.env.SSH_AUTH_SOCK,
    TMPDIR: process.env.TMPDIR ?? os.tmpdir(),
  };
}

```

```

    USER: process.env.USER ?? user.username,
  };
}

function profileDirectoryPath(userId: string): string {
  if (typeof userId !== "string" || userId.length === 0) {
    throw new TypeError("desktop profile user ID is invalid");
  }
  const profileId = createHash("sha256").update(userId, "utf8").digest("hex");
  return path.join(app.getPath("userData"), "profiles", profileId);
}

async function ensureProfileDirectories(directoryPath: string): Promise<void> {
  const directories = [
    directoryPath,
    path.join(directoryPath, "backups"),
    path.join(directoryPath, "data"),
    path.join(directoryPath, "repositories"),
    path.join(directoryPath, "run"),
  ];
  for (const target of directories) {
    await mkdir(target, { mode: 0o700, recursive: true });
    await chmod(target, 0o700);
  }
}

function developmentExecutablePath(executableName: string): string {
  if (!/^([A-Za-z0-9._-]+)$/.test(executableName)) {
    throw new TypeError("development executable name is invalid");
  }
  const searchPath = shellEnvironment().PATH;
  if (searchPath === undefined) {
    throw new Error("development Node search path is unavailable");
  }
  for (const directoryPath of searchPath.split(path.delimiter)) {
    if (!path.isAbsolute(directoryPath)) {
      continue;
    }
    const candidate = path.join(directoryPath, executableName);
    try {
      accessSync(candidate, constants.X_OK);
      return candidate;
    } catch {
      continue;
    }
  }
}

```

```

    throw new Error(`development ${executableName} executable is unavailable`);
}

function developmentLaunchConfig(
  packagedConfig: RuntimeLaunchConfig,
  profileDirectory: string,
): RuntimeLaunchConfig {
  if (app.isPackaged) {
    return packagedConfig;
  }
  const projectRoot = path.resolve(app.getAppPath(), "..");
  return {
    helper: {
      ...packagedConfig.helper,
      command: path.join(projectRoot, "backend", ".venv", "bin", "python"),
      workingDirectory: profileDirectory,
      args: [
        "-m",
        "yinshi.desktop_runtime",
        "--ready-fd",
        "3",
        "--asset-dir",
        path.join(projectRoot, "frontend", "dist"),
      ],
    },
    sidecar: {
      ...packagedConfig.sidecar,
      command: developmentExecutablePath("node"),
      workingDirectory: profileDirectory,
      args: [path.join(projectRoot, "sidecar", "src", "index.js")],
      environment: {
        ...packagedConfig.sidecar.environment,
        NODE_ENV: "development",
      },
    },
  };
}

function gitExecutablePath(): string {
  const gitCommand = app.isPackaged
    ? path.join(process.resourcesPath, "bin", "git")
    : developmentExecutablePath("git");
  accessSync(gitCommand, constants.X_OK);
  return gitCommand;
}

```

```

function signInFilePath(): string {
  if (app.isPackaged) {
    return path.join(app.getAppPath(), "assets", "signin.html");
  }
  return path.resolve(moduleDirectory, "..", "assets", "signin.html");
}

function currentShellPolicy() {
  return createShellPolicy({
    applicationOrigin,
    signInUrl: pathToFileURL(signInFilePath()).href,
    externalOrigins: EXTERNAL_ORIGINS,
  });
}

function installWindowSecurity(window: BrowserWindow): void {
  window.webContents.on("will-navigate", (event, url) => {
    if (!currentShellPolicy().navigationAllowed(url)) {
      event.preventDefault();
    }
  });
  window.webContents.on("will-redirect", (event, url) => {
    if (!currentShellPolicy().navigationAllowed(url)) {
      event.preventDefault();
    }
  });
  window.webContents.on("will-attach-webview", (event) => {
    event.preventDefault();
  });
  window.webContents.setWindowOpenHandler(({ url }) => {
    if (currentShellPolicy().externalAllowed(url)) {
      void shell.openExternal(url, { activate: true }).catch(() => undefined);
    }
    return { action: "deny" };
  });
}

function createMainWindow(): BrowserWindow {
  const preloadPath = path.join(moduleDirectory, "preload.cjs");
  const window = new BrowserWindow({
    ...createBrowserWindowOptions(preloadPath),
    width: 1_080,
    height: 760,
    minWidth: 760,
    minHeight: 560,
    titleBarStyle: "hiddenInset",
  });
}

```

```

        backgroundColor: "#f4f1e9",
    });
    installWindowSecurity(window);
    window.on("closed", () => {
        if (mainWindow === window) {
            mainWindow = undefined;
        }
    });
    return window;
}

function requireWindow(): BrowserWindow {
    if (mainWindow === undefined || mainWindow.isDestroyed()) {
        throw new Error("desktop window is unavailable");
    }
    return mainWindow;
}

function requestFromAllowedPage(
    event: IpcMainInvokeEvent,
    channel: string,
): boolean {
    const sourceUrl = event.senderFrame?.url;
    if (sourceUrl === undefined) {
        return false;
    }
    const fromSignInPage = sourceUrl === pathToFileURL(signInFilePath()).href;
    if (channel === DESKTOP_IPC_CHANNELS.signIn) {
        return fromSignInPage;
    }
    if (
        fromSignInPage &&
        (channel === DESKTOP_IPC_CHANNELS.listProfiles ||
            channel === DESKTOP_IPC_CHANNELS.switchProfile)
    ) {
        return true;
    }
    if (applicationOrigin === undefined) {
        return false;
    }
    try {
        return new URL(sourceUrl).origin === applicationOrigin;
    } catch {
        return false;
    }
}

```

```

async function configureApplication(): Promise<DesktopAppController> {
  const secureDirectory = path.join(app.getPath("userData"), "secure");
  const credentialStore = new DesktopCredentialStore({
    directoryPath: secureDirectory,
    safeStorage,
  });
  const runtimeSecretStore = new RuntimeSecretStore({
    directoryPath: secureDirectory,
    safeStorage,
  });
  const fetchAdapter = (
    input: string | URL,
    init?: RequestInit,
  ): Promise<Response> => net.fetch(input.toString(), init);
  const electronSessionFetch = (
    input: string | URL,
    init?: RequestInit,
  ): Promise<Response> => session.defaultSession.fetch(input.toString(), init);

  const hostedAccessSession = new HostedAccessSession({
    resume: () =>
      resumeDesktopAccount({
        apiBaseUrl: HOSTED_API_BASE_URL,
        fetch: fetchAdapter,
        credentialStore,
      }),
  });
  const hostedApiGateway = new HostedApiGateway({
    apiBaseUrl: HOSTED_API_BASE_URL,
    fetch: fetchAdapter,
    getAccessToken: () =>
      hostedAccessSession.getAccessToken(Math.floor(Date.now() / 1_000)),
  });

  const appController = new DesktopAppController({
    resumeAccount: () => hostedAccessSession.resumeAccount(),
    signIn: async () => {
      let stage: HostedSignInStage | "starting-listener" = "starting-listener";
      let listener;
      try {
        listener = await startAuthCallbackListener({
          timeoutMs: 5 * 60 * 1_000,
        });
      } catch (error) {
        const errorName = error instanceof Error ? error.name : "UnknownError";

```

```

        console.error(`Desktop sign-in failed during ${stage}: ${errorName}`);
        throw new Error("Desktop sign-in failed");
    }
    try {
        const account = await startHostedSignIn({
            apiBaseUrl: HOSTED_API_BASE_URL,
            callbackUri: listener.callbackUri,
            deviceName: `Mac (${os.hostname().split(".", 1)[0] || "Yinshi")`,
            fetch: fetchAdapter,
            openExternal: async (url) => {
                await shell.openExternal(url, { activate: true });
            },
            waitForCallback: async () => listener.waitForCallback(),
            credentialStore,
            onProgress: (nextStage) => {
                stage = nextStage;
                console.info(`Desktop sign-in entered ${nextStage}`);
            },
        });
        hostedAccessSession.setAccount({ mode: "online", ...account });
        return account;
    } catch {
        console.error(`Desktop sign-in failed during ${stage}`);
        throw new Error("Desktop sign-in failed");
    } finally {
        await listener.close();
    }
},
clearCredentials: async () => {
    hostedAccessSession.clear();
    await credentialStore.clear();
},
startHelper: async (profile): Promise<ManagedHelper> => {
    console.info("Desktop local runtime preparing profile");
    const profileDirectory = profileDirectoryPath(profile.user.id);
    await ensureProfileDirectories(profileDirectory);
    console.info("Desktop local runtime loading secrets");
    const runtimeSecrets = await runtimeSecretStore.loadOrCreate();
    console.info("Desktop local runtime building launch configuration");
    const packagedConfig = buildRuntimeLaunchConfig({
        resourcesPath: process.resourcesPath,
        profileDirectoryPath: profileDirectory,
        socketDirectoryPath: path.join(app.getPath("userData"), "run"),
        runtimeSecrets,
        shellEnvironment: shellEnvironment(),
    });
}

```

```

const launchConfig = developmentLaunchConfig(
  packagedConfig,
  profileDirectory,
);
const socketPath = launchConfig.helper.environment.SIDECAR_SOCKET_PATH;
if (socketPath === undefined) {
  throw new Error("desktop sidecar socket path is unavailable");
}
console.info("Desktop local runtime starting child processes");
return startLocalRuntime({
  helper: launchConfig.helper,
  sidecar: launchConfig.sidecar,
  socketPath,
  startSidecar,
  startHelper: startManagedHelper,
});
},
bootstrapHelper: async (helper) =>
  bootstrapHelperSession({
    ready: helper.ready,
    fetch: electronSessionFetch,
  }),
showSignIn: async () => {
  applicationOrigin = undefined;
  const window = requireWindow();
  await window.loadFile(signInFilePath());
  if (!window.isVisible()) {
    window.show();
  }
},
loadApplication: async (origin) => {
  applicationOrigin = origin;
  const window = requireWindow();
  await window.loadURL(`${origin}/app`);
  if (!window.isVisible()) {
    window.show();
  }
},
});

ipcMain.handle(DESKTOP_IPC_CHANNELS.signIn, async (event) => {
  if (!requestFromAllowedPage(event, DESKTOP_IPC_CHANNELS.signIn)) {
    throw new Error("Desktop IPC request denied");
  }
  try {
    await appController.signIn();
  }
});

```



```

    } catch {
        throw new Error("Desktop sign-in failed");
    }
});
ipcMain.handle(DESKTOP_IPC_CHANNELS.signOut, async (event) => {
    if (!requestFromAllowedPage(event, DESKTOP_IPC_CHANNELS.signOut)) {
        throw new Error("Desktop IPC request denied");
    }
    try {
        await appController.signOut();
    } catch {
        throw new Error("Desktop sign-out failed");
    }
});
ipcMain.handle(
    DESKTOP_IPC_CHANNELS.hostedRequest,
    async (event, request: HostedApiRequest) => {
        if (!requestFromAllowedPage(event, DESKTOP_IPC_CHANNELS.hostedRequest)) {
            throw new Error("Desktop IPC request denied");
        }
        try {
            return await hostedApiGateway.request(request);
        } catch {
            throw new Error("Hosted API request failed");
        }
    },
);
ipcMain.handle(DESKTOP_IPC_CHANNELS.fileVaultStatus, async (event) => {
    if (!requestFromAllowedPage(event, DESKTOP_IPC_CHANNELS.fileVaultStatus)) {
        throw new Error("Desktop IPC request denied");
    }
    return detectFileVaultStatus();
});
ipcMain.handle(DESKTOP_IPC_CHANNELS.listProfiles, async (event) => {
    if (!requestFromAllowedPage(event, DESKTOP_IPC_CHANNELS.listProfiles)) {
        throw new Error("Desktop IPC request denied");
    }
    return credentialStore.list();
});
ipcMain.handle(
    DESKTOP_IPC_CHANNELS.switchProfile,
    async (event, userId: string) => {
        if (!requestFromAllowedPage(event, DESKTOP_IPC_CHANNELS.switchProfile)) {
            throw new Error("Desktop IPC request denied");
        }
        if (typeof userId !== "string" || !userId || userId.length > 256) {

```

```

        throw new Error("Desktop profile ID is invalid");
    }
    const previousProfile = await credentialStore.load();
    const selectedProfile = await credentialStore.select(userId);
    if (selectedProfile === null) {
        throw new Error("Desktop profile requires sign-in");
    }
    try {
        await appController.switchProfile();
    } catch {
        if (previousProfile === null) {
            throw new Error("Desktop profile switch failed");
        }
        await credentialStore.select(previousProfile.user.id);
        try {
            await appController.switchProfile();
        } catch {
            throw new Error(
                "Desktop profile switch failed and the previous profile could not restart",
            );
        }
        throw new Error("Desktop profile switch failed");
    }
},
);
ipcMain.handle(
    DESKTOP_IPC_CHANNELS.removeProfile,
    async (event, userId: string) => {
        if (!requestFromAllowedPage(event, DESKTOP_IPC_CHANNELS.removeProfile)) {
            throw new Error("Desktop IPC request denied");
        }
        if (typeof userId !== "string" || !userId || userId.length > 256) {
            throw new Error("Desktop profile ID is invalid");
        }
        const profile = (await credentialStore.list()).find(
            (candidate) => candidate.user.id === userId,
        );
        if (profile === undefined) return;
        if (profile.active) {
            await appController.signOut();
        }
        await rm(profileDirectoryPath(userId), { force: true, recursive: true });
        await credentialStore.remove(userId);
    },
);
ipcMain.handle(DESKTOP_IPC_CHANNELS.importLocalRepository, async (event) => {

```

```

if (
  !requestFromAllowedPage(event, DESKTOP_IPC_CHANNELS.importLocalRepository)
) {
  throw new Error("Desktop IPC request denied");
}
const profile = hostedAccessSession.profile;
const origin = applicationOrigin;
if (profile === undefined || origin === undefined) {
  throw new Error("Desktop account session is unavailable");
}
const window = requireWindow();
const selection = await dialog.showOpenDialog(window, {
  title: "Import a local Git repository",
  buttonLabel: "Choose Repository",
  properties: ["openDirectory", "dontAddToRecent"],
});
const sourcePath = selection.filePaths[0];
if (selection.canceled || sourcePath === undefined) {
  return { status: "cancelled" } as const;
}

const repositoryBasePath = path.join(
  profileDirectoryPath(profile.user.id),
  "repositories",
);
const destinationPath = path.join(repositoryBasePath, randomUUID());
let clonedRepository;
try {
  clonedRepository = await cloneRepositoryIntoProfile({
    gitCommand: gitExecutablePath(),
    sourcePath,
    managedBasePath: repositoryBasePath,
    destinationPath,
    allowDirty: false,
  });
} catch (error) {
  if (!(error instanceof DirtyRepositoryError)) {
    throw new Error("Local repository import failed");
  }
  const confirmation = await dialog.showMessageBox(window, {
    type: "warning",
    title: "Repository has uncommitted changes",
    message: "Import committed state only?",
    detail:
      "Yinshi will leave the selected checkout untouched. Uncommitted and untracked files will be lost.",
    buttons: ["Import Committed State", "Cancel"],
  });
  if (confirmation === "Import Committed State") {
    clonedRepository = await cloneRepositoryIntoProfile({
      gitCommand: gitExecutablePath(),
      sourcePath,
      managedBasePath: repositoryBasePath,
      destinationPath,
      allowDirty: false,
    });
  }
}

```

```

        defaultId: 1,
        cancelId: 1,
        noLink: true,
    });
    if (confirmation.response !== 0) {
        return { status: "cancelled" } as const;
    }
    try {
        clonedRepository = await cloneRepositoryIntoProfile({
            gitCommand: gitExecutablePath(),
            sourcePath,
            managedBasePath: repositoryBasePath,
            destinationPath,
            allowDirty: true,
        });
    } catch {
        throw new Error("Local repository import failed");
    }
}

try {
    const response = await electronSessionFetch(`${origin}/api/repos`, {
        method: "POST",
        headers: {
            "Content-Type": "application/json",
            "X-Requested-With": "XMLHttpRequest",
        },
        body: JSON.stringify({
            name: clonedRepository.name,
            local_path: clonedRepository.path,
        }),
        redirect: "error",
        signal: AbortSignal.timeout(30_000),
    });
    if (response.status !== 201) {
        throw new Error("Local repository registration failed");
    }
    const value: unknown = await response.json();
    if (typeof value !== "object" || value === null || Array.isArray(value)) {
        throw new Error("Local repository registration response is invalid");
    }
    const repository = value as Record<string, unknown>;
    if (
        typeof repository.id !== "string" ||
        typeof repository.name !== "string"
    ) {

```

```

        throw new Error("Local repository registration response is invalid");
    }
    return {
        status: "imported",
        repository: { id: repository.id, name: repository.name },
    } as const;
} catch {
    await rm(clonedRepository.path, { recursive: true, force: true });
    throw new Error("Local repository import failed");
}
});
return appController;
}

async function startApplication(): Promise<void> {
    session.defaultSession.setPermissionCheckHandler(() => false);
    session.defaultSession.setPermissionRequestHandler(
        (_webContents, _permission, callback) => {
            callback(false);
        },
    );
};
session.defaultSession.on("will-download", (event) => {
    event.preventDefault();
});
mainWindow = createMainWindow();
controller = await configureApplication();
await controller.start();
updateController = startAutomaticUpdates({
    isPackaged: app.isPackaged,
    updater: autoUpdater,
    schedule: (delayMs, callback) => {
        const timer = setTimeout(callback, delayMs);
        timer.unref();
        return { cancel: () => clearTimeout(timer) };
    },
    onDownloaded: () => {
        const window = mainWindow;
        if (window === undefined || window.isDestroyed()) {
            return;
        }
        void dialog
            .showMessageBox(window, {
                type: "info",
                title: "Yinshi update ready",
                message: "A signed Yinshi update is ready to install.",
                detail: "Restart now, or install automatically when you quit Yinshi.",
            })
    },
});

```

```

        buttons: ["Restart now", "Later"],
        defaultId: 0,
        cancelId: 1,
        noLink: true,
    })
    .then((result) => {
        if (result.response === 0) {
            autoUpdater.quitAndInstall(false, true);
        }
    })
    .catch(() => undefined);
    },
    });
}

if (!app.requestSingleInstanceLock()) {
    app.quit();
} else {
    app.on("second-instance", () => {
        const window = mainWindow;
        if (window !== undefined && !window.isDestroyed()) {
            if (window.isMinimized()) {
                window.restore();
            }
            window.show();
            window.focus();
        }
    });

    app.on("activate", () => {
        if (mainWindow === undefined) {
            mainWindow = createMainWindow();
            if (applicationOrigin === undefined) {
                void mainWindow.loadFile(signInFilePath());
            } else {
                void mainWindow.loadURL(`${applicationOrigin}/`);
            }
        }
    });

    app.on("window-all-closed", () => {
        if (process.platform !== "darwin") {
            app.quit();
        }
    });
}

```

```

app.on("before-quit", (event) => {
  if (quitting) {
    return;
  }
  event.preventDefault();
  quitting = true;
  void (async () => {
    try {
      updateController?.stop();
      await controller?.stop();
    } finally {
      app.quit();
    }
  })();
});

void app
  .whenReady()
  .then(startApplication)
  .catch(() => {
    dialog.showErrorBox(
      "Yinshi could not start",
      "The desktop runtime could not be started safely. No local workspace was opened.",
    );
    app.quit();
  });
}

```

18.18 preload.ts

The preload bridge exposes a narrow typed API while keeping Node.js access outside renderer pages.

```

import { contextBridge, ipcRenderer } from "electron";

import {
  DESKTOP_IPC_CHANNELS,
  type HostedApiRequest,
  type YinshiDesktopApi,
} from "../desktopApi.js";

const desktopApi: YinshiDesktopApi = Object.freeze({
  signIn: async (): Promise<void> => {
    await ipcRenderer.invoke(DESKTOP_IPC_CHANNELS.signIn);
  },
  signOut: async (): Promise<void> => {

```

```

        await ipcRenderer.invoke(DESKTOP_IPC_CHANNELS.signOut);
    },
    importLocalRepository: async () =>
        ipcRenderer.invoke(DESKTOP_IPC_CHANNELS.importLocalRepository),
    hostedRequest: async (request: HostedApiRequest) =>
        ipcRenderer.invoke(DESKTOP_IPC_CHANNELS.hostedRequest, request),
    fileVaultStatus: async () =>
        ipcRenderer.invoke(DESKTOP_IPC_CHANNELS.fileVaultStatus),
    listProfiles: async () =>
        ipcRenderer.invoke(DESKTOP_IPC_CHANNELS.listProfiles),
    switchProfile: async (userId: string) =>
        ipcRenderer.invoke(DESKTOP_IPC_CHANNELS.switchProfile, userId),
    removeProfile: async (userId: string) =>
        ipcRenderer.invoke(DESKTOP_IPC_CHANNELS.removeProfile, userId),
    });

contextBridge.exposeInMainWorld("yinshiDesktop", desktopApi);

```

18.19 runtimeLaunchConfig.ts

Launch configuration resolves packaged paths, loopback ports, data roots, and mode-specific helper arguments.

```

import path from "node:path";

import type { RuntimeSecrets } from "../runtimeSecrets.js";

const SHELL_ENVIRONMENT_KEYS = [
    "HOME",
    "LANG",
    "LC_ALL",
    "LC_CTYPE",
    "LOGNAME",
    "PATH",
    "SHELL",
    "SSH_AUTH_SOCK",
    "TMPDIR",
    "USER",
] as const;

export interface RuntimeLaunchConfigOptions {
    readonly resourcesPath: string;
    readonly profileDirectoryPath: string;
    readonly socketDirectoryPath: string;
    readonly runtimeSecrets: RuntimeSecrets;
    readonly shellEnvironment: Readonly<Record<string, string | undefined>>;
}

```



```

}

export interface ChildLaunchConfig {
  readonly command: string;
  readonly workingDirectory: string;
  readonly args: readonly string[];
  readonly environment: Readonly<Record<string, string>>;
}

export interface RuntimeLaunchConfig {
  readonly helper: ChildLaunchConfig;
  readonly sidecar: ChildLaunchConfig;
}

function requireAbsolutePath(value: string, name: string): string {
  if (
    typeof value !== "string" ||
    !path.isAbsolute(value) ||
    value.includes("\0")
  ) {
    throw new TypeError(`${name} must be an absolute path`);
  }
  return path.normalize(value);
}

function copyShellEnvironment(
  source: Readonly<Record<string, string | undefined>>,
): Record<string, string> {
  if (typeof source !== "object" || source === null || Array.isArray(source)) {
    throw new TypeError("shellEnvironment must be an object");
  }
  const environment: Record<string, string> = {};
  for (const key of SHELL_ENVIRONMENT_KEYS) {
    const value = source[key];
    if (value === undefined || value === "") {
      continue;
    }
    if (typeof value !== "string" || value.includes("\0")) {
      throw new TypeError(`shell environment ${key} is invalid`);
    }
    environment[key] = value;
  }
  for (const requiredKey of ["HOME", "PATH", "SHELL"] as const) {
    if (environment[requiredKey] === undefined) {
      throw new Error(`shell environment ${requiredKey} is required`);
    }
  }
}

```

```

    }
    return environment;
}

function validateRuntimeSecrets(secrets: RuntimeSecrets): void {
    if (secrets.version !== 1 || !/^[A-Za-z0-9_-]{64}$/.test(secrets.secretKey)) {
        throw new Error("runtime session secret is invalid");
    }
    if (!/^[a-f0-9]{64}$/.test(secrets.encryptionPepper)) {
        throw new Error("runtime encryption pepper is invalid");
    }
    if (!/^[a-f0-9]{64}$/.test(secrets.keyEncryptionKey)) {
        throw new Error("runtime key-encryption key is invalid");
    }
}

export function buildRuntimeLaunchConfig(
    options: RuntimeLaunchConfigOptions,
): RuntimeLaunchConfig {
    const resourcesPath = requireAbsolutePath(
        options.resourcesPath,
        "resourcesPath",
    );
    const profileDirectoryPath = requireAbsolutePath(
        options.profileDirectoryPath,
        "profileDirectoryPath",
    );
    const socketDirectoryPath = requireAbsolutePath(
        options.socketDirectoryPath,
        "socketDirectoryPath",
    );
    validateRuntimeSecrets(options.runtimeSecrets);
    const shellEnvironment = copyShellEnvironment(options.shellEnvironment);

    const runDirectoryPath = path.join(profileDirectoryPath, "run");
    const profileName = path.basename(profileDirectoryPath);
    if (!/^[A-Za-z0-9_-]{1,64}$/.test(profileName)) {
        throw new Error(
            "profile directory name is unsafe for runtime socket selection",
        );
    }
    const sidecarSocketPath = path.join(
        socketDirectoryPath,
        `${profileName.slice(0, 16)}.sock`,
    );
    if (Buffer.byteLength(sidecarSocketPath, "utf-8") > 100) {

```

```

    throw new Error("sidecar socket path exceeds the macOS Unix-socket limit");
}
const bundledBinaryPath = path.join(resourcesPath, "bin");
const inheritedPath = shellEnvironment.PATH;
if (inheritedPath === undefined) {
    throw new Error("shell environment PATH is required");
}
const childPath = `${bundledBinaryPath}:${inheritedPath}`;
const sharedEnvironment = {
    ...shellEnvironment,
    PATH: childPath,
    SIDECAR_SOCKET_PATH: sidecarSocketPath,
    YINSHI_DESKTOP: "1",
};
const helperEnvironment = Object.freeze({
    ...sharedEnvironment,
    ALLOWED_REPO_BASE: path.join(profileDirectoryPath, "repositories"),
    APP_NAME: "Yinshi",
    BACKUP_DIR: path.join(profileDirectoryPath, "backups"),
    CONTAINER_ENABLED: "false",
    CONTAINER_SOCKET_BASE: runDirectoryPath,
    CONTROL_DB_PATH: path.join(profileDirectoryPath, "control.db"),
    CONTROL_FIELD_ENCRYPTION: "required",
    DB_PATH: path.join(profileDirectoryPath, "local.db"),
    DEBUG: "false",
    DISABLE_AUTH: "true",
    ENCRYPTION_PEPPEP: options.runtimeSecrets.encryptionPepper,
    HOST: "127.0.0.1",
    HSTS_ENABLED: "false",
    KEY_ENCRYPTION_KEY: options.runtimeSecrets.keyEncryptionKey,
    KEY_ENCRYPTION_KEY_ID: "desktop-v1",
    PYTHONUNBUFFERED: "1",
    REQUIRE_HTTPS: "disabled",
    SECRET_KEY: options.runtimeSecrets.secretKey,
    TENANT_DB_ENCRYPTION: "required",
    TRUSTED_HOSTS: "127.0.0.1,localhost",
    USER_DATA_DIR: path.join(profileDirectoryPath, "data"),
    USER_DATA_ENCRYPTION: "disabled",
});
const sidecarEnvironment = Object.freeze({
    ...sharedEnvironment,
    NODE_ENV: "production",
    SIDECAR_LOAD_DOTENV: "0",
});
return {
    helper: {

```

```

        command: path.join(resourcesPath, "helper", "yinshi-desktop-helper"),
        workingDirectory: profileDirectoryPath,
        args: Object.freeze([
            "--ready-fd",
            "3",
            "--asset-dir",
            path.join(resourcesPath, "frontend"),
        ]),
        environment: helperEnvironment,
    },
    sidecar: {
        command: path.join(resourcesPath, "node", "bin", "node"),
        workingDirectory: profileDirectoryPath,
        args: Object.freeze([
            path.join(resourcesPath, "sidecar", "src", "index.js"),
        ]),
        environment: sidecarEnvironment,
    },
};
}

```

18.20 runtimeSecrets.ts

Runtime secret handling generates transient credentials and removes them from logs and persistent launch metadata.

```

import { randomBytes } from "node:crypto";

import type { SafeStorageAdapter } from "../credentialStore.js";
import { SecureJsonStore } from "../secureJsonStore.js";

export interface RuntimeSecrets {
    readonly version: 1;
    readonly secretKey: string;
    readonly encryptionPepper: string;
    readonly keyEncryptionKey: string;
}

export interface RuntimeSecretStoreOptions {
    readonly directoryPath: string;
    readonly safeStorage: SafeStorageAdapter;
}

function validateRuntimeSecrets(value: unknown): RuntimeSecrets {
    if (typeof value !== "object" || value === null || Array.isArray(value)) {
        throw new Error("desktop runtime secrets are invalid");
    }
}

```

```

    }
    const secrets = value as Record<string, unknown>;
    if (secrets.version !== 1) {
        throw new Error("desktop runtime secret version is unsupported");
    }
    if (typeof secrets.secretKey !== "string" || !/^[A-Za-z0-9_-]{64}$/.test(secrets.secretKey)) {
        throw new Error("desktop runtime session secret is invalid");
    }
    if (
        typeof secrets.encryptionPepper !== "string" ||
        !/^[a-f0-9]{64}$/.test(secrets.encryptionPepper)
    ) {
        throw new Error("desktop runtime encryption pepper is invalid");
    }
    if (
        typeof secrets.keyEncryptionKey !== "string" ||
        !/^[a-f0-9]{64}$/.test(secrets.keyEncryptionKey)
    ) {
        throw new Error("desktop runtime key-encryption key is invalid");
    }
    return {
        version: 1,
        secretKey: secrets.secretKey,
        encryptionPepper: secrets.encryptionPepper,
        keyEncryptionKey: secrets.keyEncryptionKey,
    };
}

export class RuntimeSecretStore {
    readonly #store: SecureJsonStore<RuntimeSecrets>;
    #pending: Promise<RuntimeSecrets> | undefined;

    constructor(options: RuntimeSecretStoreOptions) {
        this.#store = new SecureJsonStore({
            directoryPath: options.directoryPath,
            fileName: "runtime-secrets.bin",
            safeStorage: options.safeStorage,
            validate: validateRuntimeSecrets,
        });
    }

    async loadOrCreate(): Promise<RuntimeSecrets> {
        if (this.#pending !== undefined) {
            return this.#pending;
        }
        const operation = (async () => {

```

```

    const stored = await this.#store.load();
    if (stored !== null) {
        return stored;
    }
    const created: RuntimeSecrets = {
        version: 1,
        secretKey: randomBytes(48).toString("base64url"),
        encryptionPepper: randomBytes(32).toString("hex"),
        keyEncryptionKey: randomBytes(32).toString("hex"),
    };
    await this.#store.save(created);
    return created;
  })();
  this.#pending = operation;
  try {
    return await operation;
  } finally {
    if (this.#pending === operation) {
      this.#pending = undefined;
    }
  }
}
}
}

```

18.21 secureJsonStore.ts

The JSON store uses atomic writes, strict permissions, schema checks, and corruption-safe reads for non-keychain state.

```

import { randomUUID } from "node:crypto";
import { constants } from "node:fs";
import {
  chmod,
  lstat,
  mkdir,
  open,
  rename,
  rm,
} from "node:fs/promises";
import path from "node:path";

export interface SafeStorageAdapter {
  isEncryptionAvailable(): boolean;
  encryptString(value: string): Buffer;
  decryptString(value: Buffer): string;
}

```

```

export interface SecureJsonStoreOptions<Value> {
  readonly directoryPath: string;
  readonly fileName: string;
  readonly safeStorage: SafeStorageAdapter;
  readonly validate: (value: unknown) => Value;
}

function isNodeError(error: unknown): error is NodeJS.ErrnoException {
  return error instanceof Error && "code" in error;
}

async function ensureOwnerOnlyDirectory(directoryPath: string): Promise<void> {
  await mkdir(directoryPath, { mode: 0o700, recursive: true });
  const information = await lstat(directoryPath);
  if (!information.isDirectory() || information.isSymbolicLink()) {
    throw new Error("secure storage directory must be a real directory");
  }
  await chmod(directoryPath, 0o700);
}

async function assertOwnerOnlyFile(filePath: string): Promise<boolean> {
  let information;
  try {
    information = await lstat(filePath);
  } catch (error) {
    if (isNodeError(error) && error.code === "ENOENT") {
      return false;
    }
    throw error;
  }
  if (!information.isFile() || information.isSymbolicLink()) {
    throw new Error("secure storage path must be a regular file");
  }
  if ((information.mode & 0o077) !== 0) {
    throw new Error("secure storage file permissions are unsafe");
  }
  return true;
}

async function syncDirectory(directoryPath: string): Promise<void> {
  const directory = await open(directoryPath, constants.O_RDONLY);
  try {
    await directory.sync();
  } finally {
    await directory.close();
  }
}

```

```

    }
  }

export class SecureJsonStore<Value> {
  readonly #directoryPath: string;
  readonly #filePath: string;
  readonly #safeStorage: SafeStorageAdapter;
  readonly #validate: (value: unknown) => Value;

  constructor(options: SecureJsonStoreOptions<Value>) {
    if (typeof options.directoryPath !== "string" || !path.isAbsolute(options.directoryPath)) {
      throw new TypeError("directoryPath must be an absolute path");
    }
    if (
      typeof options.fileName !== "string" ||
      !/^[A-Za-z0-9][A-Za-z0-9._-]{0,99}$/.test(options.fileName) ||
      options.fileName === "." ||
      options.fileName === ".."
    ) {
      throw new TypeError("fileName must be one safe relative file name");
    }
    if (typeof options.safeStorage?.isEncryptionAvailable !== "function") {
      throw new TypeError("safeStorage must implement Electron safeStorage methods");
    }
    if (typeof options.validate !== "function") {
      throw new TypeError("validate must be a function");
    }
    this.#directoryPath = options.directoryPath;
    this.#filePath = path.join(options.directoryPath, options.fileName);
    this.#safeStorage = options.safeStorage;
    this.#validate = options.validate;
  }

  async save(value: Value): Promise<void> {
    const validatedValue = this.#validate(value);
    if (!this.#safeStorage.isEncryptionAvailable()) {
      throw new Error("Keychain encryption is unavailable");
    }
    await ensureOwnerOnlyDirectory(this.#directoryPath);
    await assertOwnerOnlyFile(this.#filePath);

    const encryptedValue = this.#safeStorage.encryptString(JSON.stringify(validatedValue));
    if (!Buffer.isBuffer(encryptedValue) || encryptedValue.length === 0) {
      throw new Error("Keychain encryption returned no data");
    }
    const temporaryPath = path.join(this.#directoryPath, `${randomUUID()}.tmp`);
  }
}

```



```

try {
  const temporaryFile = await open(
    temporaryPath,
    constants.O_CREAT | constants.O_EXCL | constants.O_WRONLY,
    0o600,
  );
  try {
    await temporaryFile.writeFile(encryptedValue);
    await temporaryFile.sync();
  } finally {
    await temporaryFile.close();
  }
  await rename(temporaryPath, this.#filePath);
  await chmod(this.#filePath, 0o600);
  await syncDirectory(this.#directoryPath);
} finally {
  await rm(temporaryPath, { force: true });
}
}

async load(): Promise<Value | null> {
  await ensureOwnerOnlyDirectory(this.#directoryPath);
  if (!(await assertOwnerOnlyFile(this.#filePath))) {
    return null;
  }
  if (!this.#safeStorage.isEncryptionAvailable()) {
    throw new Error("Keychain encryption is unavailable");
  }

  const storedFile = await open(this.#filePath, constants.O_RDONLY | constants.O_NOFOLLOW);
  let encryptedValue: Buffer;
  try {
    const information = await storedFile.stat();
    if (!information.isFile() || (information.mode & 0o077) !== 0) {
      throw new Error("secure storage file changed during read");
    }
  }
  encryptedValue = await storedFile.readFile();
} finally {
  await storedFile.close();
}
if (encryptedValue.length === 0) {
  throw new Error("secure storage file is empty");
}
let parsedValue: unknown;
try {
  parsedValue = JSON.parse(this.#safeStorage.decryptString(encryptedValue));
}

```

```

    } catch {
      throw new Error("secure storage file cannot be decrypted");
    }
    return this.#validate(parsedValue);
  }

  async clear(): Promise<void> {
    await ensureOwnerOnlyDirectory(this.#directoryPath);
    if (!(await assertOwnerOnlyFile(this.#filePath))) {
      return;
    }
    await rm(this.#filePath);
    await syncDirectory(this.#directoryPath);
  }
}

```

18.22 security.ts

Central security policy configures isolation, permissions, navigation, external links, and trusted content origins.

```

import path from "node:path";

import type { BrowserWindowConstructorOptions } from "electron";

export function createBrowserWindowOptions(
  preloadPath: string,
): BrowserWindowConstructorOptions {
  if (typeof preloadPath !== "string" || preloadPath.length === 0) {
    throw new TypeError("preloadPath must be a non-empty string");
  }
  if (!path.isAbsolute(preloadPath)) {
    throw new TypeError("preloadPath must be absolute");
  }

  return {
    show: false,
    webPreferences: {
      preload: preloadPath,
      contextIsolation: true,
      nodeIntegration: false,
      sandbox: true,
      webSecurity: true,
      allowRunningInsecureContent: false,
      webviewTag: false,
      devTools: false,
    },
  };
}

```

```

        navigateOnDragDrop: false,
        spellcheck: false,
    },
};
}

```

18.23 shellPolicy.ts

Shell policy permits only approved external URLs and rejects local, privileged, or malformed targets.

```

export interface ShellPolicyOptions {
    readonly applicationOrigin: string | undefined;
    readonly signInUrl: string;
    readonly externalOrigins: readonly string[];
}

export interface ShellPolicy {
    navigationAllowed(url: string): boolean;
    externalAllowed(url: string): boolean;
}

function parseApplicationOrigin(value: string | undefined): string | undefined {
    if (value === undefined) {
        return undefined;
    }
    const url = new URL(value);
    if (
        url.protocol !== "http:" ||
        url.hostname !== "127.0.0.1" ||
        url.port === "" ||
        url.pathname !== "/" ||
        url.search !== "" ||
        url.hash !== "" ||
        url.username !== "" ||
        url.password !== ""
    ) {
        throw new TypeError("applicationOrigin must be an exact loopback HTTP origin");
    }
    return url.origin;
}

export function createShellPolicy(options: ShellPolicyOptions): ShellPolicy {
    const signInUrl = new URL(options.signInUrl);
    if (
        signInUrl.protocol !== "file:" ||

```

```

    signInUrl.search !== "" ||
    signInUrl.hash !== "" ||
    signInUrl.username !== "" ||
    signInUrl.password !== ""
  ) {
    throw new TypeError("signInUrl must be an exact file URL");
  }
  const applicationOrigin = parseApplicationOrigin(options.applicationOrigin);
  const externalOrigins = new Set(
    options.externalOrigins.map((value) => {
      const url = new URL(value);
      if (
        url.protocol !== "https:" ||
        url.pathname !== "/" ||
        url.search !== "" ||
        url.hash !== "" ||
        url.username !== "" ||
        url.password !== ""
      ) {
        throw new TypeError("external origins must be exact HTTPS origins");
      }
      return url.origin;
    })
  );

  return Object.freeze({
    navigationAllowed(value: string): boolean {
      let url: URL;
      try {
        url = new URL(value);
      } catch {
        return false;
      }
      if (url.protocol === "file:") {
        return url.href === signInUrl.href;
      }
      return applicationOrigin !== undefined && url.origin === applicationOrigin;
    },
    externalAllowed(value: string): boolean {
      let url: URL;
      try {
        url = new URL(value);
      } catch {
        return false;
      }
      return url.protocol === "https:" && externalOrigins.has(url.origin);
    }
  });

```

```

    },
  });
}

```

18.24 sidecarSupervisor.ts

The sidecar supervisor ties agent process lifetime to the selected local runtime and active account lease.

```

import { spawn, type ChildProcess } from "node:child_process";
import { lstat, mkdir, rm } from "node:fs/promises";
import path from "node:path";

const READINESS_LINE_LENGTH_MAX = 4_096;

export interface SidecarOptions {
  readonly command: string;
  readonly args: readonly string[];
  readonly environment: Readonly<Record<string, string>>;
  readonly workingDirectory?: string;
  readonly socketPath: string;
  readonly startupTimeoutMs: number;
  readonly shutdownTimeoutMs: number;
}

export interface ManagedSidecar {
  readonly processId: number;
  readonly running: boolean;
  readonly socketPath: string;
  stop(): Promise<void>;
}

function validateOptions(options: SidecarOptions): void {
  if (!path.isAbsolute(options.command) || options.command.includes("\0")) {
    throw new TypeError("sidecar command must be an absolute path");
  }
  if (!path.isAbsolute(options.socketPath) || options.socketPath.includes("\0")) {
    throw new TypeError("sidecar socketPath must be an absolute path");
  }
  if (!Number.isInteger(options.startupTimeoutMs) || options.startupTimeoutMs < 1) {
    throw new TypeError("sidecar startupTimeoutMs must be a positive integer");
  }
  if (!Number.isInteger(options.shutdownTimeoutMs) || options.shutdownTimeoutMs < 1) {
    throw new TypeError("sidecar shutdownTimeoutMs must be a positive integer");
  }
  if (options.args.some((argument) => typeof argument !== "string" || argument.includes("\0"))

```

```

        throw new TypeError("sidecar arguments must be strings without null bytes");
    }
}

async function removeOwnedStaleSocket(socketPath: string): Promise<void> {
    let information;
    try {
        information = await lstat(socketPath);
    } catch (error) {
        if (error instanceof Error && "code" in error && error.code === "ENOENT") {
            return;
        }
        throw error;
    }
    const userId = process.getuid?();
    if (!information.isSocket() || (userId !== undefined && information.uid !== userId)) {
        throw new Error("sidecar socket path is occupied by an unsafe file");
    }
    await rm(socketPath);
}

async function assertSocketSecurity(socketPath: string): Promise<void> {
    const information = await lstat(socketPath);
    const userId = process.getuid?();
    if (!information.isSocket()) {
        throw new Error("sidecar did not create a Unix socket");
    }
    if ((information.mode & 0o077) !== 0) {
        throw new Error("sidecar socket permissions are unsafe");
    }
    if (userId !== undefined && information.uid !== userId) {
        throw new Error("sidecar socket owner is invalid");
    }
}

function waitForExit(child: ChildProcess, timeoutMs: number): Promise<boolean> {
    if (child.exitCode !== null || child.signalCode !== null) {
        return Promise.resolve(true);
    }
    return new Promise((resolve) => {
        const timeout = setTimeout(() => {
            child.off("exit", onExit);
            resolve(false);
        }, timeoutMs);
        const onExit = (): void => {
            clearTimeout(timeout);

```

```

        resolve(true);
    };
    child.once("exit", onExit);
  });
}

async function terminateChild(child: ChildProcess, timeoutMs: number): Promise<void> {
  if (child.exitCode !== null || child.signalCode !== null) {
    return;
  }
  child.kill("SIGTERM");
  if (await waitForExit(child, timeoutMs)) {
    return;
  }
  child.kill("SIGKILL");
  if (!(await waitForExit(child, timeoutMs))) {
    throw new Error("sidecar process did not exit after SIGKILL");
  }
}

async function waitForReadiness(
  child: ChildProcess,
  expectedLine: string,
  timeoutMs: number,
): Promise<void> {
  const stdout = child.stdout;
  const stderr = child.stderr;
  if (stdout === null || stderr === null) {
    throw new Error("sidecar output pipes are unavailable");
  }
  stderr.on("data", () => undefined);
  return new Promise((resolve, reject) => {
    let buffer = "";
    let settled = false;
    const finish = (error?: Error): void => {
      if (settled) {
        return;
      }
      settled = true;
      clearTimeout(timeout);
      child.off("error", onError);
      child.off("exit", onExit);
      stdout.off("data", onData);
      if (error === undefined) {
        stdout.on("data", () => undefined);
        resolve();
      }
    }
  });
}

```

```

    } else {
      reject(error);
    }
  };
  const onError = (): void => finish(new Error("sidecar process failed during startup"));
  const onExit = (): void => finish(new Error("sidecar process exited before readiness"));
  const onData = (chunk: Buffer | string): void => {
    buffer += chunk.toString();
    if (buffer.length > READINESS_LINE_LENGTH_MAX) {
      finish(new Error("sidecar readiness output exceeded its limit"));
      return;
    }
    const newline = buffer.indexOf("\n");
    if (newline < 0) {
      return;
    }
    const line = buffer.slice(0, newline).replace(/\r$/, "");
    if (line !== expectedLine) {
      finish(new Error("sidecar readiness output was invalid"));
      return;
    }
    finish();
  };
  const timeout = setTimeout(
    () => finish(new Error("sidecar readiness timed out")),
    timeoutMs,
  );
  child.once("error", onError);
  child.once("exit", onExit);
  stdout.on("data", onData);
});
}

export async function startSidecar(options: SidecarOptions): Promise<ManagedSidecar> {
  validateOptions(options);
  const socketDirectory = path.dirname(options.socketPath);
  await mkdir(socketDirectory, { mode: 0o700, recursive: true });
  await removeOwnedStaleSocket(options.socketPath);

  const child = spawn(options.command, [...options.args], {
    cwd: options.workingDirectory,
    env: { ...options.environment },
    stdio: ["ignore", "pipe", "pipe"],
    windowsHide: true,
  });
  try {

```



```

    await waitForReadiness(
      child,
      `SOCKET_PATH=${options.socketPath}`,
      options.startupTimeoutMs,
    );
    await assertSocketSecurity(options.socketPath);
  } catch (error) {
    await terminateChild(child, options.shutdownTimeoutMs);
    await removeOwnedStaleSocket(options.socketPath);
    throw error;
  }
  if (child.pid === undefined || child.pid < 1) {
    await terminateChild(child, options.shutdownTimeoutMs);
    throw new Error("sidecar process ID is unavailable");
  }

  let stopOperation: Promise<void> | undefined;
  return {
    processId: child.pid,
    get running(): boolean {
      return child.exitCode === null && child.signalCode === null;
    },
    socketPath: options.socketPath,
    stop(): Promise<void> {
      stopOperation ??= (async () => {
        await terminateChild(child, options.shutdownTimeoutMs);
        await removeOwnedStaleSocket(options.socketPath);
      })();
      return stopOperation;
    },
  };
}

```

18.25 signInRenderer.ts

The sign-in renderer presents account state and sends authentication actions through the preload bridge.

```

import "../desktopApi.js";

function requiredElement<ElementType extends HTMLElement>(
  selector: string,
  expectedType: new () => ElementType,
): ElementType {
  const element = document.querySelector(selector);
  if (!(element instanceof expectedType)) {

```

```

        throw new Error(`Required sign-in element is missing: ${selector}`);
    }
    return element;
}

window.addEventListener("DOMContentLoaded", () => {
    const signInButton = requiredElement("[data-sign-in]", HTMLButtonElement);
    const status = requiredElement("[data-status]", HTMLParagraphElement);
    const profileList = requiredElement("[data-profile-list]", HTMLDivElement);
    const profileButtons = requiredElement("[data-profile-buttons]", HTMLDivElement);

    void window.yinshiDesktop
        .listProfiles()
        .then((profiles) => {
            const availableProfiles = profiles.filter((profile) => profile.hasCredentials);
            if (availableProfiles.length === 0) return;
            for (const profile of availableProfiles) {
                const button = document.createElement("button");
                button.type = "button";
                button.className = "profile-button";
                button.textContent = profile.user.email;
                button.addEventListener("click", async () => {
                    button.disabled = true;
                    status.textContent = "Opening local profile...";
                    status.dataset.state = "working";
                    try {
                        await window.yinshiDesktop.switchProfile(profile.user.id);
                    } catch {
                        button.disabled = false;
                        status.textContent = "This profile could not be opened. Sign in again to refresh";
                        status.dataset.state = "error";
                    }
                });
                profileButtons.append(button);
            }
            profileList.hidden = false;
        })
        .catch(() => undefined);

    signInButton.addEventListener("click", async () => {
        signInButton.disabled = true;
        signInButton.textContent = "Opening browser...";
        status.textContent = "Complete sign-in in your browser. This window will update automatically";
        status.dataset.state = "working";
        try {
            await window.yinshiDesktop.signIn();
        }
    });
});

```

```

    } catch {
      signInButton.disabled = false;
      signInButton.textContent = "Sign in with Yinshi";
      status.textContent = "Sign-in did not complete. Check your connection and try again.";
      status.dataset.state = "error";
    }
  });
});
});

```

19 Frontend Shell and Routing

The frontend begins with a small router and a global stylesheet. Authenticated screens reuse the same layout and recover from stale deployed chunks without asking the user to understand Vite build artifacts.

19.1 main.tsx

The frontend entry point installs browser observability and mounts React into the static shell. The root chunk below tangles back to `frontend/src/main.tsx`.

```

import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import { datadogRum } from "@datadog/browser-rum";
import App from "./App";
import { AuthProvider } from "./hooks/useAuth";
import { createRumConfiguration } from "./rum";
import "./index.css";

declare const __GIT_COMMIT_HASH__: string;

/* Defer Datadog RUM so it does not compete with first paint. */
const initDatadogRum = () => {
  datadogRum.init(createRumConfiguration(__GIT_COMMIT_HASH__));
};

if ("requestIdleCallback" in window) {
  window.requestIdleCallback(initDatadogRum);
} else {
  setTimeout(initDatadogRum, 0);
}

ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>

```

```

        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>,
);

```

19.2 rum.ts

Browser observability is isolated from application startup so privacy defaults, sampling, and transport settings can be tested without mounting React.

```

import type { RumBeforeSend } from "@datadog/browser-rum";
import { reactPlugin } from "@datadog/browser-rum-react";

function normalizeRumViewUrl(rawUrl: string): string {
  if (typeof rawUrl !== "string" || rawUrl.length === 0) {
    return "/other";
  }

  let pathname: string;
  try {
    pathname = new URL(rawUrl, "https://yinshi.invalid").pathname;
  } catch {
    return "/other";
  }

  if (/^\/app\/session\/[~\/]+\w?$/.test(pathname)) {
    return "/app/session/:sessionId";
  }

  return "/other";
}

const filterRumEvent: RumBeforeSend = (event) => {
  if (!("type" in event) || event.type !== "view") {
    return false;
  }

  if (event.usr !== undefined || event.account !== undefined) {
    return false;
  }

  if (event.context !== undefined && Object.keys(event.context).length > 0) {
    return false;
  }

  const normalizedPath = normalizeRumViewUrl(event.view.url);
  event.view.url = normalizedPath;
  event.view.name = normalizedPath;
  event.view.referrer = "";
}

```

```

    return true;
  };

  export function createRumConfiguration(version: string) {
    if (typeof version !== "string" || version.trim().length === 0) {
      throw new Error("version must be a non-empty string");
    }

    return {
      applicationId: "6ca07893-ea15-4577-88cb-ef72b856ad3e",
      clientToken: "pubbe7e2760d9e429d5cda2d2eb49a408be", // gitleaks:allow -- public browser
      site: "datadoghq.com",
      service: "yinshi",
      env: "prod",
      version,
      sessionSampleRate: 10,
      // Coding sessions render private source, prompts, and tool output. Replay
      // remains disabled even though the SDK package supports it.
      sessionReplaySampleRate: 0,
      trackResources: false,
      trackUserInteractions: false,
      trackLongTasks: false,
      defaultPrivacyLevel: "mask" as const,
      beforeSend: filterRumEvent,
      proxy: (options: { path: string; parameters: string }) =>
        `/rum${options.path}?${options.parameters}`,
      plugins: [reactPlugin({ router: false })],
    };
  }
}

```

19.3 App.tsx

The router defines the public landing page, auth callback pages, the session workbench, settings, and the empty state. The root chunk below tangles back to frontend/src/App.tsx.

```

import React, { Suspense } from "react";
import { Route, Routes } from "react-router-dom";
import ChunkErrorBoundary from "../components/ChunkErrorBoundary";
import Layout from "../components/Layout";
import RequireAuth from "../components/RequireAuth";
import Landing from "../pages/Landing";

```

```

/* Code-split authenticated routes so landing page visitors download only what they need. */
const EmptyState = React.lazy(() => import("../pages/EmptyState"));
const Session = React.lazy(() => import("../pages/Session"));

```

```

const Settings = React.lazy(() => import("./pages/Settings"));

export default function App() {
  return (
    <Routes>
      <Route path="/" element={<Landing />} />
      <Route element={<RequireAuth />}>
        <Route element={<Layout />}>
          <Route
            path="/app"
            element={
              <ChunkErrorBoundary><Suspense><EmptyState /></Suspense></ChunkErrorBoundary>
            }
          />
          <Route
            path="/app/session/:id"
            element={
              <ChunkErrorBoundary><Suspense><Session /></Suspense></ChunkErrorBoundary>
            }
          />
          <Route
            path="/app/settings"
            element={
              <ChunkErrorBoundary><Suspense><Settings /></Suspense></ChunkErrorBoundary>
            }
          />
        </Route>
      </Route>
    </Routes>
  );
}

```

19.4 index.css

Global CSS defines the light and dark visual language, landing-page styling, chat surfaces, and responsive workspace layout. The root chunk below tangles back to `frontend/src/index.css`.

```

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  html {
    -webkit-tap-highlight-color: transparent;
    overscroll-behavior: none;
  }
}

```

```

}

body {
  @apply bg-gray-900 text-gray-100;
  padding-top: env(safe-area-inset-top);
  padding-left: env(safe-area-inset-left);
  padding-right: env(safe-area-inset-right);
}

/* Prevent iOS zoom on input focus */
input,
textarea,
select {
  font-size: 16px;
}
}

@layer components {
  .btn-primary {
    @apply flex items-center justify-center rounded-lg bg-blue-500 px-4 py-3
      font-medium text-white transition-colors active:bg-blue-600
      min-h-touch min-w-touch;
  }

  .btn-secondary {
    @apply flex items-center justify-center rounded-lg border border-gray-600
      bg-gray-800 px-4 py-3 font-medium text-gray-200
      transition-colors active:bg-gray-700 min-h-touch min-w-touch;
  }

  .card {
    @apply rounded-xl bg-gray-800 p-4;
  }
}

@layer utilities {
  .scrollbar-hide {
    -ms-overflow-style: none;
    scrollbar-width: none;
  }
  .scrollbar-hide::-webkit-scrollbar {
    display: none;
  }

  .animate-message-in {
    animation: message-in 0.2s ease-out;
  }
}

```

```

    }
  }

  /* Markdown prose styles for assistant messages */
  .markdown-prose {
    line-height: 1.65;
  }

  .markdown-prose p {
    margin: 0.5em 0;
  }

  .markdown-prose p:first-child {
    margin-top: 0;
  }

  .markdown-prose p:last-child {
    margin-bottom: 0;
  }

  .markdown-prose h1,
  .markdown-prose h2,
  .markdown-prose h3,
  .markdown-prose h4 {
    font-weight: 600;
    margin: 1em 0 0.5em;
    color: var(--prose-heading);
  }

  .markdown-prose h1 { font-size: 1.25em; }
  .markdown-prose h2 { font-size: 1.125em; }
  .markdown-prose h3 { font-size: 1em; }

  .markdown-prose ul,
  .markdown-prose ol {
    margin: 0.5em 0;
    padding-left: 1.5em;
  }

  .markdown-prose ul { list-style-type: disc; }
  .markdown-prose ol { list-style-type: decimal; }

  .markdown-prose li {
    margin: 0.25em 0;
  }

```



```

.markdown-prose code {
  background: var(--prose-code-bg);
  border-radius: 4px;
  padding: 0.15em 0.35em;
  font-size: 0.875em;
  font-family: ui-monospace, SFMono-Regular, "SF Mono", Menlo, monospace;
}

.markdown-prose pre {
  background: var(--prose-pre-bg);
  border-radius: 8px;
  padding: 0.75em 1em;
  margin: 0.5em 0;
  overflow-x: auto;
}

.markdown-prose pre code {
  background: none;
  padding: 0;
  border-radius: 0;
  font-size: 0.8125em;
}

.markdown-prose blockquote {
  border-left: 3px solid var(--prose-border);
  padding-left: 0.75em;
  margin: 0.5em 0;
  color: var(--prose-muted);
}

.markdown-prose a {
  color: #d4543d;
  text-decoration: underline;
  text-underline-offset: 2px;
}

.markdown-prose hr {
  border: none;
  border-top: 1px solid var(--prose-border);
  margin: 0.75em 0;
}

.markdown-prose table {
  border-collapse: collapse;
  margin: 0.5em 0;
  font-size: 0.875em;
}

```

```

    width: 100%;
}

.markdown-prose th,
.markdown-prose td {
    border: 1px solid var(--prose-border);
    padding: 0.35em 0.6em;
    text-align: left;
}

.markdown-prose th {
    background: var(--prose-th-bg);
    font-weight: 600;
}

.markdown-prose strong {
    font-weight: 600;
    color: var(--prose-heading);
}

@keyframes message-in {
    from {
        opacity: 0;
        transform: translateY(8px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

/* ---- Theme: gray scale CSS variables ---- */
:root {
    /* Light mode (default): parchment background, ink text */
    --gray-50: 15 12 9;
    --gray-100: 26 20 16;
    --gray-200: 45 37 32;
    --gray-300: 74 63 53;
    --gray-400: 107 93 79;
    --gray-500: 140 122 100;
    --gray-600: 168 148 120;
    --gray-700: 203 184 154;
    --gray-800: 224 209 184;
    --gray-900: 240 230 211;
    --gray-950: 247 240 227;

```

```

    /* Prose theme tokens */
    --prose-heading: rgb(var(--gray-100));
    --prose-code-bg: rgba(0, 0, 0, 0.07);
    --prose-pre-bg: rgba(0, 0, 0, 0.06);
    --prose-border: rgb(var(--gray-700));
    --prose-muted: rgb(var(--gray-400));
    --prose-th-bg: rgba(0, 0, 0, 0.04);
  }

  /* Dark mode: invert the gray scale */
  .dark {
    --gray-50: 247 240 227;
    --gray-100: 240 230 211;
    --gray-200: 224 209 184;
    --gray-300: 203 184 154;
    --gray-400: 168 148 120;
    --gray-500: 140 122 100;
    --gray-600: 107 93 79;
    --gray-700: 74 63 53;
    --gray-800: 45 37 32;
    --gray-900: 26 20 16;
    --gray-950: 15 12 9;

    --prose-code-bg: rgba(255, 255, 255, 0.1);
    --prose-pre-bg: rgba(0, 0, 0, 0.3);
    --prose-th-bg: rgba(255, 255, 255, 0.05);
  }

```

19.5 components/Layout.tsx

The layout component wraps authenticated pages with the sidebar and main content frame. The root chunk below tangles back to `frontend/src/components/Layout.tsx`.

```

import { Outlet } from "react-router-dom";
import Sidebar from "../Sidebar";
import { useMobileDrawer } from "../hooks/useMobileDrawer";

export default function Layout() {
  const { open, toggle, close, panelClassName } = useMobileDrawer();

  return (
    <div className="flex h-screen bg-gray-900">
      /* Mobile hamburger */
      <button
        onClick={toggle}
        aria-label="Toggle sidebar"

```

```

        className="fixed top-2 left-2 z-50 flex h-10 w-10 items-center justify-center rounded
    >
    <svg className="h-5 w-5" fill="none" viewBox="0 0 24 24" strokeWidth={2} stroke="cur
        <path strokeLinecap="round" strokeLinejoin="round" d="M3.75 6.75h16.5M3.75 12h16.5
    </svg>
</button>

{/* Overlay */}
{open && (
    <div
        data-testid="sidebar-overlay"
        className="fixed inset-0 z-40 bg-black/60 md:hidden"
        onClick={close}
    />
)}

{/* Sidebar panel: off-screen on mobile, static on desktop */}
<div data-testid="sidebar-panel" className={panelClassName}>
    <Sidebar onNavigate={close} />
</div>

<main className="flex min-h-0 flex-1 flex-col overflow-hidden">
    <Outlet />
</main>
</div>
);
}

```

19.6 components/RequireAuth.tsx

The auth guard blocks private routes until the current-user request has resolved.
The root chunk below tangles back to frontend/src/components/RequireAuth.tsx.

```

import { Navigate, Outlet } from "react-router-dom";
import { useAuth } from "../hooks/useAuth";

export default function RequireAuth() {
    const { status } = useAuth();

    if (status === "unauthenticated") {
        return <Navigate to="/" replace />;
    }

    return <Outlet />;
}

```

19.7 components/ChunkErrorBoundary.tsx

The chunk boundary performs one safe reload when a deployed frontend references stale Vite chunks. The root chunk below tangles back to frontend/src/components/ChunkErrorBoundary.tsx.

```
import React from "react";

interface State {
  hasError: boolean;
  reloadRecommended: boolean;
}

const CHUNK_RELOAD_STORAGE_KEY = "yinshi.chunk-reload-signature";
const CHUNK_ERROR_PATTERNS = [
  "chunkloaderror",
  "failed to fetch dynamically imported module",
  "error loading dynamically imported module",
  "importing a module script failed",
  "loading chunk",
] as const;

function readErrorSignature(error: unknown): string {
  if (error instanceof Error) {
    return `${error.name} ${error.message}`.toLowerCase();
  }
  return String(error).toLowerCase();
}

export function isChunkLoadError(error: unknown): boolean {
  const errorSignature = readErrorSignature(error);
  return CHUNK_ERROR_PATTERNS.some((pattern) => errorSignature.includes(pattern));
}

function isStorageAccessError(error: unknown): boolean {
  if (!(error instanceof DOMException)) {
    return false;
  }
  if (error.name === "QuotaExceededError") {
    return true;
  }
  if (error.name === "SecurityError") {
    return true;
  }
  return false;
}
```

```

function readEntryScriptSignature(): string {
  if (typeof document === "undefined") {
    return "server";
  }
  const entryScript = document.querySelector<HTMLScriptElement>('script[type="module"] [src]
  const entryScriptSource = entryScript?.src;
  if (typeof entryScriptSource === "string" && entryScriptSource.length > 0) {
    return entryScriptSource;
  }
  return "unknown-entry-script";
}

export function shouldReloadForChunkError(
  storage: Pick<Storage, "getItem" | "setItem">,
  pathname: string,
  entryScriptSignature: string,
): boolean {
  if (typeof pathname !== "string") {
    throw new TypeError("pathname must be a string");
  }
  if (!pathname) {
    throw new Error("pathname must not be empty");
  }
  if (typeof entryScriptSignature !== "string") {
    throw new TypeError("entryScriptSignature must be a string");
  }
  if (!entryScriptSignature) {
    throw new Error("entryScriptSignature must not be empty");
  }
  const reloadSignature = `${pathname}:${entryScriptSignature}`;
  if (storage.getItem(CHUNK_RELOAD_STORAGE_KEY) === reloadSignature) {
    return false;
  }
  storage.setItem(CHUNK_RELOAD_STORAGE_KEY, reloadSignature);
  return true;
}

/**
 * Catches errors thrown by React.lazy() when a code-split chunk fails to
 * load (network error, deploy mismatch, etc.) and renders a retry prompt
 * instead of crashing the entire app to a white screen.
 */
export default class ChunkErrorBoundary extends React.Component<
  React.PropsWithChildren,
  State

```

```

> {
  state: State = { hasError: false, reloadRecommended: false };

  static getDerivedStateFromError(): State {
    return { hasError: true, reloadRecommended: false };
  }

  componentDidCatch(error: unknown): void {
    if (typeof window === "undefined") {
      return;
    }
    if (!isChunkLoadError(error)) {
      return;
    }
    let shouldReload = false;
    try {
      shouldReload = shouldReloadForChunkError(
        window.sessionStorage,
        window.location.pathname,
        readEntryScriptSignature(),
      );
    } catch (storageError: unknown) {
      if (!isStorageAccessError(storageError)) {
        throw storageError;
      }
      this.setState({ reloadRecommended: true });
      return;
    }
    if (shouldReload) {
      window.location.reload();
      return;
    }
    this.setState({ reloadRecommended: true });
  }

  handleRetry = () => {
    if (this.state.reloadRecommended && typeof window !== "undefined") {
      window.location.reload();
      return;
    }
    this.setState({ hasError: false, reloadRecommended: false });
  };

  render() {
    if (this.state.hasError) {
      return (

```

```

    <div style={{ padding: "2rem", textAlign: "center" }}>
      <p>
        {this.state.reloadRecommended
          ? "This page needs a refresh after the latest deploy."
          : "Something went wrong loading this page."}
      </p>
      <button onClick={this.handleRetry} style={{ marginTop: "1rem" }}>
        {this.state.reloadRecommended ? "Reload page" : "Try again"}
      </button>
    </div>
  );
}
return this.props.children;
}
}

```

20 Frontend Pages

Pages hold product-level workflows: public explanation, OAuth entry, workspace sessions, settings, and the no-selection state. Most reusable behavior stays below them in hooks and components.

20.1 pages/Landing.tsx

The landing page explains the product and links directly to the architecture anchors for isolation, GitHub access, and encryption. The root chunk below tangles back to `frontend/src/pages/Landing.tsx`.

```

import { useSearchParams } from "react-router-dom";
import "./Landing.css";

const ERROR_MESSAGES: Record<string, string> = {
  oauth_error: "Sign-in was cancelled or failed. Please try again.",
  github_api_error: "Could not retrieve your GitHub account details. Please try again.",
  account_error: "Account setup failed. Please try again or contact support.",
  no_user_info: "Could not retrieve your profile information. Please try again.",
  no_verified_email: "No verified email found on your GitHub account.",
};

type Capability = {
  title: string;
  description: string;
  links?: Array<{
    href: string;
    label: string;
  }>;
};

```



```

};

const CAPABILITIES: Capability[] = [
  {
    title: "Tenant isolation",
    description:
      "Each account runs in its own tenant boundary. Yinshi keeps per-user data separate and",
    links: [
      {
        href: "/architecture.html#container-isolation",
        label: "Container isolation",
      },
      {
        href: "/architecture.html#github-app-integration",
        label: "GitHub App integration",
      },
    ],
  },
  {
    title: "AI agent sessions",
    description:
      "Converse with a coding agent that reads, writes, and refactors code inside your works",
  },
  {
    title: "Mobile-first interface",
    description:
      "Work from anywhere. The responsive interface adapts from phone to desktop, keeping yo",
  },
  {
    title: "Encrypted secrets",
    description:
      "Provider keys and connection secrets are encrypted at rest with AES-256-GCM. Per-user",
    links: [
      {
        href: "/architecture.html#encryption-key-management",
        label: "Encryption and key management",
      },
    ],
  },
],

function Navigation() {
  return (
    <header className="landing-header">
      <nav className="landing-nav" aria-label="Primary">
        <a className="landing-brand" href="/" aria-label="Yinshi home">

```

```

        Yinshi
      </a>
      <a className="landing-nav-link" href="/auth/login">
        Sign in
      </a>
    </nav>
  </header>
);
}

function WorkspacePreview() {
  return (
    <figure className="landing-preview" aria-label="Example Yinshi coding workspace">
      <div className="landing-preview-bar">
        <strong>Web app</strong>
        <span className="landing-preview-status">
          <span aria-hidden="true" />
          pi connected
        </span>
      </div>
      <div className="landing-preview-body">
        <aside className="landing-preview-sidebar" aria-label="Example worktree">
          <span aria-hidden="true" className="landing-preview-branch-mark" />
          <code>session/quiet-pine</code>
        </aside>
        <div className="landing-preview-chat">
          <div className="landing-preview-message landing-preview-message-user">
            <span aria-hidden="true">Y</span>
            <p>Add retry feedback when a repository import fails.</p>
          </div>
          <div className="landing-preview-message landing-preview-message-agent">
            <span aria-hidden="true"></span>
            <code>6 focused tests passed</code>
          </div>
        </div>
      </div>
    </figure>
  );
}

function Hero() {
  return (
    <section className="landing-hero" aria-labelledby="landing-title">
      <div className="landing-hero-mark">
        
</div>
<div className="landing-hero-text">
  <p className="landing-subtitle">Browser-based coding workspace</p>
  <h1 id="landing-title" className="landing-title">
    Run coding agents against your repositories from any browser.
  </h1>
  <p className="landing-desc">
    Import a GitHub or allowed local repository. Yinshi creates an isolated git worktree,
    connects a pi agent, and streams the session to your browser.
  </p>
  <div className="landing-cta-group">
    <a href="/auth/login" className="landing-cta">
      Start a workspace
    </a>
    <a href="/architecture.html" className="landing-cta landing-cta-secondary">
      Read the architecture
    </a>
  </div>
</div>
</section>
);
}

function TechnicalFoundation() {
  return (
    <section className="landing-foundation" aria-label="Technical foundation">
      <ul>
        <li>
          <span>Workspace isolation</span>
          <strong>One git worktree per session</strong>
        </li>
        <li>
          <span>Agent runtime</span>
          <strong>pi coding agent</strong>
        </li>
        <li>
          <span>Repository access</span>
          <strong>GitHub App or allowed local path</strong>
        </li>
      </ul>
    </section>
  );
}

```

```

    );
}

function Workflow() {
  return (
    <section className="landing-workflow" aria-labelledby="workflow-title">
      <h2 id="workflow-title">From repository to reviewable branch</h2>
      <ol>
        <li>
          <strong>Connect a repository</strong>
          <p>Choose a GitHub repository or an allowed local path available to the server.</p>
        </li>
        <li>
          <strong>Give pi a task</strong>
          <p>Yinshi creates a named worktree and streams pi's messages, tool calls, and file<
        </li>
        <li>
          <strong>Review the branch</strong>
          <p>Inspect the result, then merge or discard the workspace through your existing C
        </li>
      </ol>
    </section>
  );
}

function Capabilities() {
  return (
    <section className="landing-capabilities" aria-labelledby="capabilities-title">
      <h2 id="capabilities-title" className="landing-section-title">
        Capabilities
      </h2>
      <div className="landing-cap-grid">
        {CAPABILITIES.map((capability) => (
          <article key={capability.title} className="landing-cap-card">
            <h3 className="landing-cap-title">{capability.title}</h3>
            <p className="landing-cap-desc">{capability.description}</p>
            {capability.links ? (
              <div className="landing-cap-links">
                {capability.links.map((link) => (
                  <a key={link.href} href={link.href} className="landing-cap-link">
                    {link.label}
                  </a>
                ))}
              </div>
            ) : null}
          </article>
        ))}
      </div>
    </section>
  );
}

```

```

        ))}
      </div>
    </section>
  );
}

function FinalAction() {
  return (
    <section className="landing-final">
      <p className="landing-final-text">
        No IDE or app required. Fire up your browser, import your repos and pi configs, and
        work.
      </p>
      <a href="/auth/login" className="landing-cta">
        Start a workspace
      </a>
    </section>
  );
}

function Footer() {
  return (
    <footer className="landing-footer">
      <span>Yinshi</span>
      <span className="landing-footer-sep">·</span>
      <span>Code with AI agents, anywhere.</span>
    </footer>
  );
}

export default function Landing() {
  const [searchParams] = useSearchParams();
  const errorCode = searchParams.get("error");
  const errorMessage = errorCode
    ? ERROR_MESSAGES[errorCode] ?? "Something went wrong. Please try again."
    : null;

  return (
    <div className="landing-page">
      <a className="landing-skip-link" href="#landing-main">
        Skip to main content
      </a>
      <Navigation />
      <main id="landing-main" tabIndex={-1}>
        {errorMessage ? (
          <div className="landing-alert" role="alert">

```

```

        {errorMessage}
      </div>
    ) : null}
    <Hero />
    <WorkspacePreview />
    <TechnicalFoundation />
    <Workflow />
    <Capabilities />
    <FinalAction />
  </main>
  <Footer />
</div>
);
}

```

20.2 pages/Login.tsx

The login page starts OAuth and shows user-facing messages for expired, denied, or failed auth attempts. The root chunk below tangles back to `frontend/src/pages/Login.tsx`.

```

import { Navigate, useSearchParams } from "react-router-dom";
import { useAuth } from "../hooks/useAuth";

const ERROR_MESSAGES: Record<string, string> = {
  oauth_error: "Sign-in was cancelled or failed. Please try again.",
  github_api_error: "Could not retrieve your GitHub account details. Please try again.",
  account_error: "Account setup failed. Please try again or contact support.",
  no_user_info: "Could not retrieve your profile information. Please try again.",
  no_verified_email: "No verified email found on your GitHub account.",
};

export default function Login() {
  const { status } = useAuth();
  const [searchParams] = useSearchParams();
  const errorCode = searchParams.get("error");
  const errorMessage = errorCode
    ? ERROR_MESSAGES[errorCode] ?? "Something went wrong. Please try again."
    : null;

  if (status === "authenticated") {
    return <Navigate to="/" replace />;
  }

  return (
    <div className="flex min-h-screen flex-col items-center justify-center bg-gray-900 px-6">

```

```

<div className="w-full max-w-sm space-y-8 text-center">
  <div>
    <h1 className="text-3xl font-bold text-gray-100">Yinshi</h1>
    <p className="mt-2 text-sm text-gray-500">
      Code with AI agents, anywhere.
    </p>
  </div>

  {errorMessage && (
    <div className="rounded-md bg-red-900/50 px-4 py-3 text-sm text-red-200">
      {errorMessage}
    </div>
  )}

  <div className="space-y-3">
    <button
      onClick={() => { window.location.href = "/auth/login/google"; }}
      className="btn-primary w-full gap-3"
    >
      <svg className="h-5 w-5" viewBox="0 0 24 24">
        <path
          fill="currentColor"
          d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92a5.06 5.06 0 0 1-2.2 3.32"
        />
        <path
          fill="currentColor"
          d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.97 0-5.46-.98-7.28-2.66"
        />
        <path
          fill="currentColor"
          d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.54 0 12 0 15.46"
        />
        <path
          fill="currentColor"
          d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 9.03 1 6.54 2.09 5.38 3.15"
        />
      </svg>
      Continue with Google
    </button>

    <button
      onClick={() => { window.location.href = "/auth/login/github"; }}
      className="btn-primary w-full gap-3 bg-gray-800 hover:bg-gray-700"
    >
      <svg className="h-5 w-5" viewBox="0 0 24 24" fill="currentColor">
        <path d="M12 0C5.374 0 0 5.373 0 12c0 6.626 5.373 12 12 12c6.626 0 12-5.373 12-12 0-6.626-5.373-12-12-12"
        />
      </svg>
      Continue with GitHub
    </button>
  </div>

```

```

        </svg>
        Continue with GitHub
      </button>
    </div>

    <p className="text-xs text-gray-600">
      Sign in or create an account. By continuing, you agree to the terms of service.
    </p>
  </div>
</div>
);
}

```

20.3 pages/EmptyState.tsx

The empty state gives authenticated users a landing point when no session is selected. The root chunk below tangles back to `frontend/src/pages/EmptyState.tsx`.

```

export default function EmptyState() {
  return (
    <div className="flex h-full items-center justify-center">
      <p className="text-sm text-gray-600">
        Select a workspace from the sidebar to start coding.
      </p>
    </div>
  );
}

```

20.4 pages/Session.tsx

The session page coordinates chat, sidebar selection, prompt submission, cancellation, and the resizable workspace inspector. The root chunk below tangles back to `frontend/src/pages/Session.tsx`.

```

import { useCallback, useEffect, useMemo, useRef, useState, type PointerEvent as ReactPointerEvent } from "react";
import { useParams } from "react-router-dom";
import { type Message, type SessionInfo, type ThinkingLevel } from "../api/client";
import ChatView from "../components/ChatView";
import WorkspaceInspector from "../components/WorkspaceInspector";
import { useAgentStream, type ChatMessage } from "../hooks/useAgentStream";
import { useCatalog } from "../hooks/useCatalog";
import { usePiCommands } from "../hooks/usePiCommands";
import {
  DEFAULT_SESSION_MODEL,
  availableSessionModelsMarkdown,
  describeSessionModel,
  formatSessionModelOptionLabel,
}

```



```

    formatThinkingLevelLabel,
    getModelThinkingLevels,
    getSessionModelOption,
    resolveSessionModelKey,
  } from "../models/sessionModels";
import { useRuntimeResource } from "../runtime/useRuntimeResource";
import { parseStoredTurnBlocks } from "../utils/turnEvents";

let cmdIdCounter = 0;
function nextCmdId(): string {
  return `cmd-${Date.now()}-${++cmdIdCounter}`;
}

const INSPECTOR_WIDTH_DEFAULT = 420;
const INSPECTOR_WIDTH_MIN = 320;
const INSPECTOR_WIDTH_MAX = 760;
const DESKTOP_INSPECTOR_QUERY = "(min-width: 1024px)";
const LEGACY_PI_CONTEXT_MESSAGE =
  "This session predates durable Pi context and cannot continue with exact model context. St

function useMediaQuery(query: string): boolean {
  const [matches, setMatches] = useState(() => {
    if (typeof window === "undefined" || typeof window.matchMedia !== "function") {
      return false;
    }
    return window.matchMedia(query).matches;
  });

  useEffect(() => {
    if (typeof window === "undefined" || typeof window.matchMedia !== "function") {
      return;
    }
    const mediaQuery = window.matchMedia(query);
    const updateMatches = () => setMatches(mediaQuery.matches);
    updateMatches();
    mediaQuery.addEventListener("change", updateMatches);
    return () => mediaQuery.removeEventListener("change", updateMatches);
  }, [query]);

  return matches;
}

function storedInspectorWidth(): number {
  const raw = sessionStorage.getItem("yinshi-inspector-width");
  const value = Number(raw);
  if (Number.isFinite(value)) {

```

```

        return Math.min(INSPECTOR_WIDTH_MAX, Math.max(INSPECTOR_WIDTH_MIN, value));
    }
    return INSPECTOR_WIDTH_DEFAULT;
}

export default function Session() {
    const { id: encodedSessionId } = useParams<{ id: string }>();
    const runtimeState = useRuntimeResource(encodedSessionId);
    const runtimeResource = runtimeState.resource;
    const id = runtimeResource?.resourceId;
    const transport = runtimeResource?.transport;
    const { messages, sendPrompt, cancel, streaming, setMessages } =
        useAgentStream(id, transport);
    const { catalog, loading: loadingCatalog } = useCatalog(transport);
    const piCommands = usePiCommands(transport);
    const [sessionModel, setSessionModel] = useState(DEFAULT_SESSION_MODEL);
    const [loadingHistory, setLoadingHistory] = useState(true);
    const [historyError, setHistoryError] = useState<string | null>(null);
    const [updatingModel, setUpdatingModel] = useState(false);
    const [pendingModelSelection, setPendingModelSelection] = useState<
        string | null
    >(null);
    const [thinkingOverride, setThinkingOverride] =
        useState<ThinkingLevel | null>(null);
    const [workspaceId, setWorkspaceId] = useState<string | null>(null);
    const [piContextVersion, setPiContextVersion] = useState(0);
    const [workspacePanelOpen, setWorkspacePanelOpen] = useState(false);
    const [inspectorWidth, setInspectorWidth] = useState(storedInspectorWidth);
    const [fileRefreshKey, setFileRefreshKey] = useState(0);
    const isDesktopInspectorVisible = useMediaQuery(DESKTOP_INSPECTOR_QUERY);
    const wasStreamingRef = useRef(false);

    useEffect(() => {
        setLoadingHistory(true);
        setHistoryError(null);
    }, [encodedSessionId]);

    useEffect(() => {
        if (runtimeState.error) {
            setHistoryError(runtimeState.error);
            setLoadingHistory(false);
        }
    }, [runtimeState.error]);

    // Load existing message history
    useEffect(() => {

```

```

    if (!id || !transport) return;
    let cancelled = false;
    const runtimeTransport = transport;

    async function loadHistory() {
      try {
        const history = await runtimeTransport.get<Message[]>(
          `/api/sessions/${id}/messages`,
        );
        if (cancelled) return;
        const mapped: ChatMessage[] = history.map((m) => {
          let blockIndex = 0;
          const blocks =
            m.role === "assistant"
              ? parseStoredTurnBlocks(
                  m.full_message,
                  () => `${m.id}-block-${++blockIndex}`,
                )
              : [];
          return {
            id: m.id,
            role: m.role as ChatMessage["role"],
            content: m.content || "",
            blocks,
            timestamp: new Date(m.created_at).getTime(),
          };
        });
        setMessages(mapped);
        setHistoryError(null);
      } catch {
        if (!cancelled) {
          setHistoryError("Failed to load session history.");
        }
      } finally {
        if (!cancelled) setLoadingHistory(false);
      }
    }

    loadHistory();
    return () => {
      cancelled = true;
    };
  }, [id, setMessages, transport]);

  useEffect(() => {
    if (!id || !transport) return;

```

```

let cancelled = false;
const runtimeTransport = transport;

async function loadSession() {
  try {
    const session = await runtimeTransport.get<SessionInfo>(`/api/sessions/${id}`);
    if (cancelled) return;
    setSessionModel(session.model);
    setWorkspaceId(session.workspace_id);
    setPiContextVersion(session.pi_context_version);
  } catch {
    if (!cancelled) setHistoryError("Failed to load session metadata.");
  }
}

loadSession();
return () => {
  cancelled = true;
};
}, [id, transport]);

useEffect(() => {
  setPendingModelSelection(null);
  setThinkingOverride(null);
}, [id]);

useEffect(() => {
  if (wasStreamingRef.current && !streaming) {
    setFileRefreshKey((value) => value + 1);
  }
  wasStreamingRef.current = streaming;
}, [streaming]);

const addSystemMessage = useCallback(
  (content: string) => {
    setMessages((prev) => [
      ...prev,
      {
        id: nextCmdId(),
        role: "assistant" as const,
        content,
        blocks: [{ id: nextCmdId(), type: "text" as const, text: content }],
        timestamp: Date.now(),
      },
    ]);
  },
  [],
);

```

```

    [setMessages],
  );

const updateSessionModel = useCallback(
  async (requestedModel: string, announce: boolean) => {
    if (!id || !transport) return false;
    if (!catalog) return false;

    const connectedProviderIds = new Set(
      catalog.providers
        .filter((provider) => provider.connected)
        .map((provider) => provider.id),
    );
    const providerLabelById = new Map(
      catalog.providers.map(
        (provider) => [provider.id, provider.label] as const,
      ),
    );
    const resolvedModel = resolveSessionModelKey(
      requestedModel,
      catalog.models,
      connectedProviderIds,
    );
    if (!resolvedModel) {
      if (announce) {
        addSystemMessage(
          "Unknown model. Available models:\n\n" +
            availableSessionModelsMarkdown(catalog.models),
        );
      }
      return false;
    }
    const resolvedModelOption = getSessionModelOption(
      resolvedModel,
      catalog.models,
    );
    if (!resolvedModelOption) {
      if (announce) {
        addSystemMessage("Failed to resolve the requested model.");
      }
      return false;
    }
    if (!connectedProviderIds.has(resolvedModelOption.provider)) {
      const providerLabel =
        providerLabelById.get(resolvedModelOption.provider) ||
        resolvedModelOption.provider;

```

```

        if (announce) {
            addSystemMessage(
                `Model ${describeSessionModel(resolvedModelOption.ref, catalog.models)} requires
            );
        }
        return false;
    }

    setUpdatingModel(true);
    try {
        const updated = await transport.patch<{ model: string }>(
            `/api/sessions/${id}`,
            { model: resolvedModel },
        );
        setSessionModel(updated.model);
        if (announce) {
            addSystemMessage(
                `Model changed to ${describeSessionModel(updated.model, catalog.models)}`,
            );
        }
        return true;
    } catch {
        if (announce) {
            addSystemMessage("Failed to update model.");
        }
        return false;
    } finally {
        setUpdatingModel(false);
    }
},
[addSystemMessage, catalog, id, transport],
);

const handleCommand = useCallback(
    async (name: string, args: string) => {
        if (!id) return;

        const availableModelKeys = catalog
            ? catalog.models.map((model) => `${model.ref}`).join(", ")
            : "";

        switch (name) {
            case "help":
                addSystemMessage(
                    `**Available commands:**\n\n` +
                    `-- /help -- List available commands\n` +

```

```

        "- `/model [name]` -- Show or change the AI model\n" +
        "- `/tree` -- Show workspace file tree\n" +
        "- `/export` -- Download chat as markdown\n" +
        "- `/clear` -- Clear chat display\n\n" +
        `Available model keys: ${availableModelKeys}`,
    );
    break;

case "model":
    if (args.trim()) {
        await updateSessionModel(args, true);
    } else {
        addSystemMessage(
            `Current model: ${describeSessionModel(sessionModel, catalog?.models || [])}\n` +
            "Available models:\n\n" +
            availableSessionModelsMarkdown(catalog?.models || []),
        );
    }
    break;

case "tree":
    try {
        if (!transport) {
            throw new Error("Runtime transport is unavailable");
        }
        const data = await transport.get<{ files: string[] }>(
            `/api/sessions/${id}/tree`,
        );
        if (data.files.length === 0) {
            addSystemMessage("Workspace is empty.");
        } else {
            const tree = data.files.map((f) => `- \`${f}\``).join("\n");
            addSystemMessage(
                `**Workspace files** (${data.files.length}):\n\n${tree}`,
            );
        }
    } catch {
        addSystemMessage("Failed to load file tree.");
    }
    break;

case "export": {
    const md = messages
        .map((m) => {
            const label = m.role === "user" ? "**You**" : "**Assistant**";
            return `${label}:\n\n${m.content}\n`;
        })
    .join("\n\n");
    break;
}

```

```

    })
    .join("\n---\n\n");
    const blob = new Blob([md], { type: "text/markdown" });
    const url = URL.createObjectURL(blob);
    const a = document.createElement("a");
    a.href = url;
    a.download = `session-${id?.slice(0, 8)}.md`;
    a.click();
    URL.revokeObjectURL(url);
    addSystemMessage("Chat exported as markdown.");
    break;
  }

  case "clear":
    setMessages([]);
    break;
}
},
[
  addSystemMessage,
  catalog,
  id,
  messages,
  sessionModel,
  setMessages,
  transport,
  updateSessionModel,
],
);

const { catalogModels, connectedProviderIds, providerLabelById } =
  useMemo(() => {
    const connectedProviderIds = new Set<string>();
    const providerLabelById = new Map<string, string>();
    const models = [...(catalog?.models || [])];

    for (const provider of catalog?.providers || []) {
      providerLabelById.set(provider.id, provider.label);
      if (provider.connected) {
        connectedProviderIds.add(provider.id);
      }
    }
  })

models.sort((leftModel, rightModel) => {
  const leftConnectionRank = connectedProviderIds.has(leftModel.provider)
    ? 0

```



```

        : 1;
    const rightConnectionRank = connectedProviderIds.has(
        rightModel.provider,
    )
    ? 0
    : 1;
    if (leftConnectionRank !== rightConnectionRank) {
        return leftConnectionRank - rightConnectionRank;
    }

    const leftProviderLabel =
        providerLabelById.get(leftModel.provider) || leftModel.provider;
    const rightProviderLabel =
        providerLabelById.get(rightModel.provider) || rightModel.provider;
    const providerComparison =
        leftProviderLabel.localeCompare(rightProviderLabel);
    if (providerComparison !== 0) {
        return providerComparison;
    }
    return leftModel.label.localeCompare(rightModel.label);
});

return {
    catalogModels: models,
    connectedProviderIds,
    providerLabelById,
};
}, [catalog]);
const selectedModelRef = pendingModelSelection ?? sessionModel;
const selectedModelOption = getSessionModelOption(
    selectedModelRef,
    catalogModels,
);
const selectedModelValue = selectedModelOption?.ref || selectedModelRef;
const selectedProviderLabel = selectedModelOption
    ? providerLabelById.get(selectedModelOption.provider) ||
      selectedModelOption.provider
    : null;
const selectedModelRequiresConnection = selectedModelOption
    ? !connectedProviderIds.has(selectedModelOption.provider)
    : false;
const availableThinkingLevels = getModelThinkingLevels(selectedModelOption);
const canOverrideThinking = availableThinkingLevels.some(
    (level) => level !== "off",
);
const selectedThinkingOverride =

```

```

    thinkingOverride && availableThinkingLevels.includes(thinkingOverride)
      ? thinkingOverride
      : null;
const promptThinkingOverride = canOverrideThinking
  ? (selectedThinkingOverride ?? undefined)
  : undefined;
const legacyInputDisabledReason =
  piContextVersion < 1 && messages.length > 0
    ? LEGACY_PI_CONTEXT_MESSAGE
    : null;

const handleModelChange = useCallback(
  (requestedModel: string) => {
    setPendingModelSelection(requestedModel);
    void updateSessionModel(requestedModel, false).then((updated) => {
      setPendingModelSelection((currentModel) =>
        currentModel === requestedModel ? null : currentModel,
      );
      if (!updated) {
        addSystemMessage("Failed to update model.");
      }
    });
  },
  [addSystemMessage, updateSessionModel],
);

const handleSend = useCallback(
  async (prompt: string) => {
    if (legacyInputDisabledReason) return;
    if (piContextVersion < 1) {
      setPiContextVersion(1);
    }
    // If the user starts a prompt while the model save is still in flight,
    // include the selected model in this prompt so the run does not fall back
    // to the previously persisted session model.
    await sendPrompt(
      prompt,
      pendingModelSelection ?? undefined,
      promptThinkingOverride,
    );
  },
  [
    legacyInputDisabledReason,
    pendingModelSelection,
    piContextVersion,
    promptThinkingOverride,
  ]
);

```

```

        sendPrompt,
      ],
    );

const beginInspectorResize = useCallback(
  (event: ReactPointerEvent<HTMLDivElement>) => {
    event.currentTarget.setPointerCapture(event.pointerId);
    const startX = event.clientX;
    const startWidth = inspectorWidth;
    const onMove = (moveEvent: PointerEvent) => {
      const nextWidth = Math.min(
        INSPECTOR_WIDTH_MAX,
        Math.max(INSPECTOR_WIDTH_MIN, startWidth - (moveEvent.clientX - startX)),
      );
      setInspectorWidth(nextWidth);
      sessionStorage.setItem("yinshi-inspector-width", String(nextWidth));
    };
    const onUp = () => {
      window.removeEventListener("pointermove", onMove);
      window.removeEventListener("pointerup", onUp);
    };
    window.addEventListener("pointermove", onMove);
    window.addEventListener("pointerup", onUp);
  },
  [inspectorWidth],
);

return (
  <>
    {/* Header */}
    <header className="flex items-center gap-3 border-b border-gray-800 px-4 py-2 pl-14 m">
      <div className="flex-1 min-w-0">
        <div className="text-sm font-medium text-gray-100 truncate">
          Session {id?.slice(0, 8)}
        </div>
      </div>
      {workspaceId && (
        <button
          type="button"
          onClick={() => setWorkspacePanelOpen(true)}
          className="rounded-lg border border-gray-800 px-3 py-1 text-xs text-gray-300 ho
        >
          Workspace
        </button>
      )}
    </div className="flex items-center gap-2">

```

```

<label
  htmlFor="session-model"
  className="hidden text-xs text-gray-500 sm:block"
>
  Model
</label>
<select
  id="session-model"
  value={selectedModelValue}
  disabled={streaming || updatingModel || loadingCatalog}
  onChange={(event) => {
    handleModelChange(event.target.value);
  }}
  className="rounded-lg border border-gray-800 bg-gray-900 px-2 py-1 text-xs text-
>
  {!selectedModelOption && (
    <option value={sessionModel}>{sessionModel}</option>
  )}
  {catalogModels.map((model) => (
    <option
      key={model.ref}
      value={model.ref}
      disabled={!connectedProviderIds.has(model.provider)}
    >
      {formatSessionModelOptionLabel(
        model,
        providerLabelById.get(model.provider),
        connectedProviderIds.has(model.provider),
      )}
    </option>
  ))}
</select>
<label
  htmlFor="thinking-level"
  className="hidden text-xs text-gray-500 sm:block"
>
  Thinking
</label>
<select
  id="thinking-level"
  value={selectedThinkingOverride ?? "default"}
  disabled={streaming || !canOverrideThinking}
  onChange={(event) => {
    const value = event.target.value;
    setThinkingOverride(
      value === "default" ? null : (value as ThinkingLevel),

```

```

    );
  }}
  className="rounded-lg border border-gray-800 bg-gray-900 px-2 py-1 text-xs text-
  title={
    canOverrideThinking
      ? "Select a thinking level for the next prompt"
      : "This model does not support thinking"
  }
}
>
<option value="default">Model default</option>
{availableThinkingLevels.map((level) => (
  <option key={level} value={level}>
    {formatThinkingLevelLabel(level)}
  </option>
))}
</select>
</div>
{streaming && (
  <div className="flex items-center gap-2">
    <div className="h-2 w-2 animate-pulse rounded-full bg-blue-500" />
    <span className="text-xs text-gray-500">Streaming</span>
  </div>
)}
</header>

{/* Workspace */}
<div className="flex min-h-0 flex-1 overflow-hidden">
  <main className="min-w-0 flex-1 overflow-hidden">
    {loadingHistory ? (
      <div className="flex h-full items-center justify-center">
        <div className="h-6 w-6 animate-spin rounded-full border-2 border-blue-500 bor
      </div>
    ) : (
      <div className="flex h-full flex-col">
        {selectedModelRequiresConnection && selectedProviderLabel && (
          <div className="mx-4 mt-4 rounded-md border border-amber-500/40 bg-amber-500
            Selected model requires a {selectedProviderLabel} connection.
            Pick a connected provider from the model list or add{" "}
            {selectedProviderLabel} in Settings.
          </div>
        )}
        {historyError && (
          <div className="mx-4 mt-4 rounded-md border border-red-500/40 bg-red-500/10
            {historyError}
          </div>
        )}
      </div>
    )}
  </main>
</div>

```

```

    <div className="flex-1 overflow-hidden">
      <ChatView
        messages={messages}
        streaming={streaming}
        onSend={handleSend}
        onCancel={cancel}
        onCommand={handleCommand}
        inputDisabledReason={legacyInputDisabledReason}
        piCommands={piCommands}
      />
    </div>
  </div>
)}
</main>
{workspaceId && transport && isDesktopInspectorVisible && (
  <>
    <div
      role="separator"
      aria-label="Resize workspace panel"
      onPointerDown={beginInspectorResize}
      className="w-1.5 cursor-col-resize border-x border-gray-800 bg-gray-900 hover:"
    />
    <WorkspaceInspector
      workspaceId={workspaceId}
      transport={transport}
      refreshKey={fileRefreshKey}
      className="flex"
      style={{ width: inspectorWidth }}
    />
  </>
)}
</div>
{workspaceId && transport && workspacePanelOpen && (
  <div className="fixed inset-0 z-50 bg-gray-950 lg:hidden">
    <div className="flex h-full min-h-0 flex-col">
      <div className="flex items-center justify-between border-b border-gray-800 px-4">
        <div className="text-sm font-medium text-gray-100">Workspace</div>
        <button
          type="button"
          onClick={() => setWorkspacePanelOpen(false)}
          className="rounded-lg border border-gray-800 px-3 py-1 text-xs text-gray-300"
        >
          Close
        </button>
      </div>
      <WorkspaceInspector

```

```

        workspaceId={workspaceId}
        transport={transport}
        refreshKey={fileRefreshKey}
        className="flex-1"
      />
    </div>
  </div>
)}
</>
);
}

```

20.5 pages/Settings.tsx

Settings group provider credentials, imported Pi config, release notes, and cloud runner setup into a tabbed screen. The root chunk below tangles back to `frontend/src/pages/Settings.tsx`.

```

import { useEffect, useMemo, useRef, useState } from "react";

import {
  api,
  type CloudRunner,
  type ProviderAuthStart,
  type ProviderAuthStatus,
  type ProviderConnection,
  type ProviderDescriptor,
} from "../api/client";
import CloudRunnerSection from "../components/CloudRunnerSection";
import PiConfigSection from "../components/PiConfigSection";
import PiReleaseNotesSection from "../components/PiReleaseNotesSection";
import { useAuth } from "../hooks/useAuth";
import { useCatalog } from "../hooks/useCatalog";
import {
  provisionRuntimeRef,
  resolveRuntimeRef,
} from "../runtime/resolveRuntime";
import type { UnresolvedRuntimeRef } from "../runtime/runtimeRef";
import {
  createRuntimeTransport,
  type RuntimeRef,
  type RuntimeTransport,
} from "../runtime/runtimeTransport";

function formatTimestamp(timestamp: string | null): string {
  if (!timestamp) {

```

```

        return "Never used";
    }
    return new Date(timestamp).toLocaleString();
}

function buildInitialConfig(
    provider: ProviderDescriptor,
): Record<string, string> {
    const initialConfig: Record<string, string> = {};
    for (const field of provider.setup_fields) {
        initialConfig[field.key] = "";
    }
    return initialConfig;
}

function normalizeFieldValue(value: string | undefined): string {
    return (value || "").trim();
}

function providerAuthorizationUrl(value: string): string {
    let authorizationUrl: URL;
    try {
        authorizationUrl = new URL(value);
    } catch {
        throw new Error("Provider authorization URL is invalid");
    }
    if (
        authorizationUrl.protocol !== "https:" ||
        authorizationUrl.username !== "" ||
        authorizationUrl.password !== ""
    ) {
        throw new Error(
            "Provider authorization URL must use credential-free HTTPS",
        );
    }
    return authorizationUrl.href;
}

function defaultOAuthInstructions(): string {
    return "Open the provider authorization flow in a new window, complete sign-in, and Yinsh";
}

function ProviderCard({
    provider,
    connection,
    onConnectionChange,

```



```

    transport,
  }: {
    provider: ProviderDescriptor;
    connection: ProviderConnection | undefined;
    onConnectionChange: () => Promise<void>;
    transport: RuntimeTransport;
  }) {
    const [authStrategy, setAuthStrategy] = useState(
      provider.auth_strategies[0] || "api_key",
    );
    const [secret, setSecret] = useState("");
    const [label, setLabel] = useState("");
    const [config, setConfig] = useState<Record<string, string>>(() =>
      buildInitialConfig(provider),
    );
    const [saving, setSaving] = useState(false);
    const [connecting, setConnecting] = useState(false);
    const [oauthFlowId, setOauthFlowId] = useState<string | null>(null);
    const [oauthInstructions, setOauthInstructions] = useState<string | null>(
      null,
    );
    const [oauthProgress, setOauthProgress] = useState<string[]>([]);
    const [oauthManualInputRequired, setOauthManualInputRequired] =
      useState(false);
    const [oauthManualInputPrompt, setOauthManualInputPrompt] = useState<
      string | null
    >(null);
    const [oauthManualInputSubmitted, setOauthManualInputSubmitted] =
      useState(false);
    const [oauthManualInputValue, setOauthManualInputValue] = useState("");
    const oauthClipboardRequestRef = useRef(0);
    const [readingOauthClipboard, setReadingOauthClipboard] = useState(false);
    const [submittingOauthInput, setSubmittingOauthInput] = useState(false);
    const [error, setError] = useState<string | null>(null);

    function resetOauthFlowState() {
      setOauthFlowId(null);
      setOauthInstructions(null);
      setOauthProgress([]);
      setOauthManualInputRequired(false);
      setOauthManualInputPrompt(null);
      setOauthManualInputSubmitted(false);
      oauthClipboardRequestRef.current += 1;
      setOauthManualInputValue("");
      setReadingOauthClipboard(false);
      setSubmittingOauthInput(false);
    }
  }

```

```

}

function applyOAuthFlowState(flow: ProviderAuthStart | ProviderAuthStatus) {
  setOAuthFlowId(flow.flow_id);
  if ("instructions" in flow) {
    setOAuthInstructions(flow.instructions ?? null);
  }
  if ("progress" in flow && Array.isArray(flow.progress)) {
    setOAuthProgress(flow.progress);
  }
  if ("manual_input_required" in flow) {
    setOAuthManualInputRequired(Boolean(flow.manual_input_required));
  }
  if ("manual_input_prompt" in flow) {
    setOAuthManualInputPrompt(flow.manual_input_prompt ?? null);
  }
  if ("manual_input_submitted" in flow) {
    setOAuthManualInputSubmitted(Boolean(flow.manual_input_submitted));
  }
}

useEffect(() => {
  setAuthStrategy(provider.auth_strategies[0] || "api_key");
  setConfig(buildInitialConfig(provider));
  setSecret("");
  setLabel("");
  resetOAuthFlowState();
  setError(null);
}, [provider]);

const hasKeyForm =
  authStrategy === "api_key" || authStrategy === "api_key_with_config";
const hasOAuth = authStrategy === "oauth";
const secretSetupFields = useMemo(
  () => provider.setup_fields.filter((field) => field.secret),
  [provider.setup_fields],
);
const publicSetupFields = useMemo(
  () => provider.setup_fields.filter((field) => !field.secret),
  [provider.setup_fields],
);
const nonSecretConfig = useMemo(() => {
  const trimmedConfigEntries = publicSetupFields
    .map(
      (field) => [field.key, normalizeFieldValue(config[field.key])] as const,
    )

```

```

        .filter(([, value]) => value.length > 0);
    return Object.fromEntries(trimmedConfigEntries);
}, [config, publicSetupFields]);
const structuredSecret = useMemo(() => {
    const normalizedSecret = normalizeFieldValue(secret);
    if (authStrategy !== "api_key_with_config") {
        return normalizedSecret;
    }
    const secretPayload: Record<string, string> = { apiKey: normalizedSecret };
    for (const field of secretSetupFields) {
        const fieldValue = normalizeFieldValue(config[field.key]);
        if (fieldValue) {
            secretPayload[field.key] = fieldValue;
        }
    }
    return secretPayload;
}, [authStrategy, config, secret, secretSetupFields]);
const missingRequiredField = useMemo(() => {
    if (!hasKeyForm) {
        return null;
    }
    if (!normalizeFieldValue(secret)) {
        return "API key";
    }
    for (const field of provider.setup_fields) {
        if (!field.required) {
            continue;
        }
        if (!normalizeFieldValue(config[field.key])) {
            return field.label;
        }
    }
    return null;
}, [config, hasKeyForm, provider.setup_fields, secret]);

async function saveConnection() {
    if (!hasKeyForm || missingRequiredField) {
        return;
    }
    setSaving(true);
    setError(null);
    try {
        await transport.post("/api/settings/connections", {
            provider: provider.id,
            auth_strategy: authStrategy,
            secret: structuredSecret,

```

```

        label,
        config: nonSecretConfig,
    });
    setSecret("");
    setLabel("");
    setConfig(buildInitialConfig(provider));
    await onConnectionChange();
} catch (saveError) {
    setError(
        saveError instanceof Error
        ? saveError.message
        : "Failed to save provider connection",
    );
} finally {
    setSaving(false);
}
}

async function connectProvider() {
    if (!hasOAuth) {
        return;
    }
    setConnecting(true);
    resetOAuthFlowState();
    setError(null);
    try {
        const started = await transport.post<ProviderAuthStart>(
            `/auth/providers/${provider.id}/start`,
        );
        applyOAuthFlowState(started);
        if (started.auth_url) {
            window.open(
                providerAuthorizationUrl(started.auth_url),
                "_blank",
                "noopener,noreferrer",
            );
        }
    }
    for (let attempt = 0; attempt < 600; attempt += 1) {
        await new Promise((resolve) => window.setTimeout(resolve, 1000));
        const status = await transport.get<ProviderAuthStatus>(
            `/auth/providers/${provider.id}/callback?flow_id=${encodeURIComponent(started.flow_id)}`,
        );
        applyOAuthFlowState(status);
        if (status.status === "complete") {
            resetOAuthFlowState();
            await onConnectionChange();
        }
    }
}

```

```

        return;
    }
    if (status.status === "error") {
        throw new Error(status.error || "Provider authorization failed");
    }
}
throw new Error("Provider authorization timed out");
} catch (connectError) {
    setError(
        connectError instanceof Error
        ? connectError.message
        : "Provider authorization failed",
    );
} finally {
    setConnecting(false);
}
}

async function pasteOAuthCallbackInput() {
    if (!navigator.clipboard?.readText) {
        setError("Clipboard access is unavailable. Paste the address manually.");
        return;
    }
    const clipboardRequest = oauthClipboardRequestRef.current + 1;
    oauthClipboardRequestRef.current = clipboardRequest;
    setReadingOAuthClipboard(true);
    setError(null);
    try {
        const clipboardValue = normalizeFieldValue(
            await navigator.clipboard.readText(),
        );
        if (oauthClipboardRequestRef.current !== clipboardRequest) {
            return;
        }
        if (!clipboardValue) {
            setError("Clipboard is empty. Copy the localhost address first.");
            return;
        }
        setOAuthManualInputValue(clipboardValue);
    } catch {
        if (oauthClipboardRequestRef.current === clipboardRequest) {
            setError("Clipboard access was denied. Paste the address manually.");
        }
    } finally {
        if (oauthClipboardRequestRef.current === clipboardRequest) {
            setReadingOAuthClipboard(false);
        }
    }
}

```

```

    }
  }
}

async function submitOAuthCallbackInput() {
  if (!oauthFlowId) {
    setError("Provider authorization flow is not active");
    return;
  }
  const normalizedAuthorizationInput = normalizeFieldValue(
    oauthManualInputValue,
  );
  if (!normalizedAuthorizationInput) {
    setError("Authorization input is required");
    return;
  }
  oauthClipboardRequestRef.current += 1;
  setReadingOAuthClipboard(false);
  setSubmittingOAuthInput(true);
  setError(null);
  try {
    const status = await transport.post<ProviderAuthStatus>(
      `/auth/providers/${provider.id}/callback`,
      {
        flow_id: oauthFlowId,
        authorization_input: normalizedAuthorizationInput,
      },
    );
    applyOAuthFlowState(status);
    setOAuthManualInputValue("");
  } catch (submitError) {
    setError(
      submitError instanceof Error
        ? submitError.message
        : "Failed to submit authorization input",
    );
  } finally {
    setSubmittingOAuthInput(false);
  }
}

async function removeConnection() {
  if (!connection) {
    return;
  }
  try {

```

```

        await transport.delete(`/api/settings/connections/${connection.id}`);
        await onConnectionChange();
    } catch (deleteError) {
        setError(
            deleteError instanceof Error
                ? deleteError.message
                : "Failed to remove provider connection",
        );
    }
}

return (
    <div className="rounded-xl border border-gray-800 bg-gray-900/70 p-4">
        <div className="flex flex-wrap items-start justify-between gap-3">
            <div>
                <h3 className="text-base font-semibold text-gray-100">
                    {provider.label}
                </h3>
                <p className="mt-1 text-sm text-gray-400">
                    {provider.model_count} models
                </p>
            </div>
            <div>
                {connection ? (
                    <div className="text-right text-xs text-gray-500">
                        <div className="text-green-400">Connected</div>
                        {connection.label ? (
                            <div className="text-gray-300">{connection.label}</div>
                        ) : null}
                        <div>{formatTimestamp(connection.last_used_at)}</div>
                    </div>
                ) : (
                    <div className="text-xs text-gray-500">Not connected</div>
                )}
            </div>
        </div>

        {provider.auth_strategies.length > 1 && (
            <div className="mt-4 flex gap-2">
                {provider.auth_strategies.map((strategy) => (
                    <button
                        key={strategy}
                        type="button"
                        onClick={() => setAuthStrategy(strategy)}
                        className={`rounded px-3 py-1 text-xs ${
                            authStrategy === strategy
                                ? "bg-gray-200 text-gray-900"
                                : "border border-gray-700 text-gray-300"
                        }`}
                    >

```

```

    }`}
  >
    {strategy === "oauth"
      ? "Connect"
      : strategy === "api_key_with_config"
        ? "Key + Config"
        : "API Key"}
  </button>
)}
</div>
)}

{hasKeyForm && (
  <div className="mt-4 space-y-3">
    <input
      type="text"
      value={label}
      onChange={(event) => setLabel(event.target.value)}
      placeholder="Label (optional)"
      className="w-full rounded border border-gray-700 bg-gray-800 px-3 py-2 text-sm t
    />
    <input
      type="password"
      value={secret}
      onChange={(event) => setSecret(event.target.value)}
      placeholder="Enter API key"
      className="w-full rounded border border-gray-700 bg-gray-800 px-3 py-2 text-sm t
    />
    {provider.setup_fields.map((field) => (
      <input
        key={field.key}
        type={field.secret ? "password" : "text"}
        value={config[field.key] || ""}
        onChange={(event) => {
          setConfig((previousConfig) => ({
            ...previousConfig,
            [field.key]: event.target.value,
          }));
        }}
        placeholder={field.label}
        className="w-full rounded border border-gray-700 bg-gray-800 px-3 py-2 text-sm t
      />
    ))}
    <button
      type="button"
      onClick={() => {

```



```

        void saveConnection();
    }}
    disabled={saving || missingRequiredField !== null}
    className="btn-primary px-4 py-2 text-sm disabled:opacity-50"
  >
    {saving ? "Saving..." : "Save Connection"}
  </button>
  {missingRequiredField ? (
    <p className="text-sm text-gray-500">
      {missingRequiredField} is required.
    </p>
  ) : null}
</div>
)}}

{hasOAuth && (
  <div className="mt-4 space-y-3">
    <p className="text-sm text-gray-400">
      {oauthInstructions || defaultOAuthInstructions()}
    </p>
    {oauthProgress.length > 0 ? (
      <p className="text-xs text-gray-500">
        {oauthProgress[oauthProgress.length - 1]}
      </p>
    ) : null}
    <button
      type="button"
      onClick={() => {
        void connectProvider();
      }}
      disabled={connecting}
      className="btn-primary px-4 py-2 text-sm disabled:opacity-50"
    >
      {connecting ? "Connecting..." : "Connect Provider"}
    </button>
    {oauthManualInputRequired && !connection ? (
      <div className="rounded-lg border border-gray-800 bg-gray-950/60 p-3">
        {window.yinshiDesktop === undefined ? (
          <p className="mb-3 rounded border border-amber-700/50 bg-amber-100/80 px-3">
            The localhost error page is expected. Copy its full address,
            return to Yinshi, paste it below, then submit.
          </p>
        ) : null}
        <p className="text-sm text-gray-300">
          {oauthManualInputPrompt ||
            "Paste the final redirect URL or authorization code here."}
        </p>
      </div>
    ) : null}
  </div>
)}

```

```

</p>
<textarea
  value={oauthManualInputValue}
  onChange={(event) => {
    oauthClipboardRequestRef.current += 1;
    setReadingOAuthClipboard(false);
    setOAuthManualInputValue(event.target.value);
  }}
  placeholder="http://localhost:1455/auth/callback?code=..."
  rows={3}
  className="mt-3 w-full rounded border border-gray-700 bg-gray-800 px-3 py-2"
/>
<div className="mt-3 flex flex-wrap items-center gap-3">
  {window.yinshiDesktop === undefined ? (
    <button
      type="button"
      onClick={() => {
        void pasteOAuthCallbackInput();
      }}
      disabled={readingOAuthClipboard || oauthManualInputSubmitted}
      className="rounded border border-gray-600 px-4 py-2 text-sm text-gray-100"
    >
      {readingOAuthClipboard
        ? "Reading clipboard..."
        : "Paste from clipboard"}
    </button>
  ) : null}
  <button
    type="button"
    onClick={() => {
      void submitOAuthCallbackInput();
    }}
    disabled={submittingOAuthInput || oauthManualInputSubmitted}
    className="rounded border border-gray-600 px-4 py-2 text-sm text-gray-100"
  >
    {submittingOAuthInput
      ? "Submitting..."
      : oauthManualInputSubmitted
        ? "Submitted"
        : "Submit Callback URL"}
    </button>
  {oauthManualInputSubmitted ? (
    <span className="text-xs text-gray-500">
      Waiting for the provider to finish the OAuth flow.
    </span>
  ) : null}

```

```

        </div>
      </div>
    ) : null}
  </div>
)}

<div className="mt-4 flex flex-wrap items-center justify-between gap-3">
  <a
    href={provider.docs_url}
    target="_blank"
    rel="noopener noreferrer"
    className="text-xs text-blue-400 hover:text-blue-300"
  >
    Provider docs
  </a>
  {connection && (
    <button
      type="button"
      onClick={() => {
        void removeConnection();
      }}
      className="text-sm text-red-400 hover:text-red-300"
    >
      Remove
    </button>
  )}
</div>

  {error && <p className="mt-3 text-sm text-red-400">{error}</p>}
</div>
);
}

type SettingsTab =
  "providers" | "cloud-runner" | "pi-config" | "pi-release-notes";

const SETTINGS_TABS: Array<{ id: SettingsTab; label: string }> = [
  { id: "providers", label: "Providers" },
  { id: "cloud-runner", label: "Cloud runner" },
  { id: "pi-config", label: "Pi config" },
  { id: "pi-release-notes", label: "Pi release notes" },
];

function ProvidersSection({ transport }: { transport: RuntimeTransport }) {
  const { catalog, loading, error: catalogError } = useCatalog(transport);
  const [connections, setConnections] = useState<ProviderConnection[]>([]);

```

```

const [loadingConnections, setLoadingConnections] = useState(true);
const [connectionsError, setConnectionsError] = useState<string | null>(null);

async function loadConnections() {
  setLoadingConnections(true);
  try {
    const loadedConnections = await transport.get<ProviderConnection[]>(
      "/api/settings/connections",
    );
    setConnections(loadedConnections);
    setConnectionsError(null);
  } catch (loadError) {
    setConnectionsError(
      loadError instanceof Error
        ? loadError.message
        : "Failed to load connections",
    );
  } finally {
    setLoadingConnections(false);
  }
}

useEffect(() => {
  void loadConnections();
}, [transport]);

const connectionByProviderId = useMemo(() => {
  const mappedConnections = new Map<string, ProviderConnection>();
  for (const connection of connections) {
    if (!mappedConnections.has(connection.provider)) {
      mappedConnections.set(connection.provider, connection);
    }
  }
  return mappedConnections;
}, [connections]);

return (
  <section aria-labelledby="providers-settings-heading">
    <h2
      id="providers-settings-heading"
      className="mb-4 text-lg font-semibold text-gray-200"
    >
      Providers
    </h2>
    <p className="mb-4 text-sm text-gray-400">
      Yinshi does not provide shared model credits. Connect your own model
    </p>
  </section>
)

```

providers here before starting sessions. Secrets are encrypted at rest and never shown again after saving.

```

</p>

{{(loading || loadingConnections) && (
  <div className="rounded border border-gray-700 bg-gray-800 px-4 py-3 text-sm text-gray-100">
    Loading provider catalog...
  </div>
)}}

{{(catalogError || connectionsError) && (
  <div className="mb-4 rounded border border-red-900/50 bg-gray-800 px-4 py-3 text-sm text-red-100">
    {{catalogError || connectionsError}}
  </div>
)}}

{{catalog && (
  <div className="grid gap-4 md:grid-cols-2">
    {{catalog.providers.map((provider) => (
      <ProviderCard
        key={`${transport.runtime.location}-${provider.id}`}
        provider={provider}
        connection={connectionByProviderId.get(provider.id)}
        onConnectionChange={loadConnections}
        transport={transport}
      />
    ))}}
  </div>
)}}
</section>
);
}

function RuntimeLocationSelector({
  runtime,
  primaryRuntime,
  byocRuntime,
  onSelect,
}): {
  runtime: RuntimeRef;
  primaryRuntime: RuntimeRef;
  byocRuntime: RuntimeRef | null;
  onSelect: (runtime: RuntimeRef) => void;
} {
  const selectedByoc = runtime.location === "byoc";
  const selectedPrimary = runtime.location === primaryRuntime.location;

```

```

return (
  <section className="mb-6" aria-labelledby="runtime-location-heading">
    <h2
      id="runtime-location-heading"
      className="mb-2 text-sm font-semibold text-gray-300"
    >
      Provider location
    </h2>
    <p className="mb-3 text-sm text-gray-500">
      Credentials stay in the selected execution location and are never copied
      automatically.
    </p>
    <div className="flex flex-wrap gap-2">
      <button
        type="button"
        onClick={() => onSelect(primaryRuntime)}
        className={`rounded border px-3 py-2 text-sm ${
          selectedPrimary
            ? "border-gray-500 bg-gray-800 text-gray-100"
            : "border-gray-800 text-gray-400 hover:border-gray-600"
          }`}
      >
        {window.yinshiDesktop ? "This Mac" : "Hosted Yinshi"}
      </button>
      {window.yinshiDesktop ? (
        <button
          type="button"
          onClick={() => onSelect({ location: "hosted" })}
          className={`rounded border px-3 py-2 text-sm ${
            runtime.location === "hosted"
              ? "border-gray-500 bg-gray-800 text-gray-100"
              : "border-gray-800 text-gray-400 hover:border-gray-600"
            }`}
        >
          Hosted Yinshi
        </button>
      ) : null}
    </div>
    {byocRuntime ? (
      <button
        type="button"
        onClick={() => onSelect(byocRuntime)}
        className={`rounded border px-3 py-2 text-sm ${
          selectedByoc
            ? "border-gray-500 bg-gray-800 text-gray-100"
            : "border-gray-800 text-gray-400 hover:border-gray-600"
          }`}
      >

```

```

        >
        BYOC runner
      </button>
    ) : null}
  </div>
</section>
);
}

function SettingsTabButton({
  tab,
  activeTab,
  onSelect,
}): {
  tab: { id: SettingsTab; label: string };
  activeTab: SettingsTab;
  onSelect: (tab: SettingsTab) => void;
}) {
  const selected = activeTab === tab.id;
  return (
    <button
      id={`settings-tab-${tab.id}`}
      type="button"
      role="tab"
      aria-selected={selected}
      aria-controls={`settings-panel-${tab.id}`}
      onClick={() => onSelect(tab.id)}
      className={`rounded-lg px-3 py-2 text-sm transition-colors ${
        selected
          ? "bg-gray-200 text-gray-950"
          : "border border-gray-800 text-gray-300 hover:bg-gray-800"
      }`}
    >
      {tab.label}
    </button>
  );
}

export default function Settings() {
  const { email } = useAuth();
  const requestedPrimaryRuntime = useMemo<UnresolvedRuntimeRef>(
    () =>
      window.yinshiDesktop ? { location: "local" } : { location: "hosted" },
    [],
  );
  const [activeTab, setActiveTab] = useState<SettingsTab>("providers");

```

```

const [primaryRuntime, setPrimaryRuntime] = useState<RuntimeRef | null>(null);
const [selectedRuntime, setSelectedRuntime] = useState<RuntimeRef | null>(null);
const [runtimeResolutionError, setRuntimeResolutionError] = useState<
  string | null
>(null);
const [byocRuntime, setByocRuntime] = useState<RuntimeRef | null>(null);
const [fileVaultStatus, setFileVaultStatus] = useState<
  "disabled" | "enabled" | "unknown" | null
>(null);
const [desktopProfiles, setDesktopProfiles] = useState<
  readonly {
    readonly user: { readonly id: string; readonly email: string };
    readonly hasCredentials: boolean;
    readonly active: boolean;
  }[]
>([]);
const [profileError, setProfileError] = useState<string | null>(null);
const [runtimeProvisioning, setRuntimeProvisioning] = useState(false);
const runtimeTransport = useMemo(
  () =>
    selectedRuntime ? createRuntimeTransport(selectedRuntime) : null,
  [selectedRuntime],
);

useEffect(
  () => () => {
    runtimeTransport?.close();
  },
  [runtimeTransport],
);

async function provisionManagedRuntime() {
  if (runtimeProvisioning) return;
  setRuntimeProvisioning(true);
  setRuntimeResolutionError(null);
  try {
    const resolvedRuntime = await provisionRuntimeRef(requestedPrimaryRuntime);
    setPrimaryRuntime(resolvedRuntime);
    setSelectedRuntime(resolvedRuntime);
  } catch (provisionError) {
    setRuntimeResolutionError(
      provisionError instanceof Error
        ? provisionError.message
        : "Managed runtime setup failed",
    );
  } finally {

```



```

        setRuntimeProvisioning(false);
    }
}

useEffect(() => {
    let cancelled = false;
    setRuntimeResolutionError(null);
    void resolveRuntimeRef(requestedPrimaryRuntime)
        .then((resolvedRuntime) => {
            if (cancelled) return;
            setPrimaryRuntime(resolvedRuntime);
            setSelectedRuntime(resolvedRuntime);
        })
        .catch((resolutionError) => {
            if (cancelled) return;
            setRuntimeResolutionError(
                resolutionError instanceof Error
                    ? resolutionError.message
                    : "Failed to resolve runtime",
            );
        });
    return () => {
        cancelled = true;
    };
}, [requestedPrimaryRuntime]);

useEffect(() => {
    const desktopBridge = window.yinshiDesktop;
    if (!desktopBridge) return;
    let cancelled = false;
    void desktopBridge
        .listProfiles()
        .then((profiles) => {
            if (!cancelled) setDesktopProfiles(profiles);
        })
        .catch(() => {
            if (!cancelled) setProfileError("Local profiles could not be loaded.");
        });
    void desktopBridge
        .fileVaultStatus()
        .then((result) => {
            if (!cancelled) setFileVaultStatus(result.status);
        })
        .catch(() => {
            if (!cancelled) setFileVaultStatus("unknown");
        });
});

```

```

    return () => {
        cancelled = true;
    };
}, []);

useEffect(() => {
    let cancelled = false;
    void api
        .get<CloudRunner | null>("/api/settings/runner")
        .then((runner) => {
            if (cancelled) return;
            if (
                runner?.noise_key_confirmed &&
                runner.noise_public_key &&
                runner.status !== "revoked"
            ) {
                setByocRuntime({
                    location: "byoc",
                    runnerId: runner.id,
                    runnerPublicKey: runner.noise_public_key,
                });
            } else {
                setByocRuntime(null);
                setSelectedRuntime((currentRuntime) =>
                    currentRuntime?.location === "byoc"
                    ? primaryRuntime
                    : currentRuntime,
                );
            }
        })
        .catch(() => {
            if (!cancelled) setByocRuntime(null);
        });
    return () => {
        cancelled = true;
    };
}, [activeTab, primaryRuntime]);

async function switchDesktopProfile(userId: string) {
    const desktopBridge = window.yinshiDesktop;
    if (!desktopBridge) return;
    setProfileError(null);
    try {
        await desktopBridge.switchProfile(userId);
    } catch {
        setProfileError(

```

```

        "Profile switch failed. Sign in to refresh this profile.",
    );
}
}

async function removeDesktopProfile(userId: string, emailAddress: string) {
    const desktopBridge = window.yinshiDesktop;
    if (!desktopBridge) return;
    const confirmed = window.confirm(
        `Remove local profile ${emailAddress} and all of its local workspaces?`,
    );
    if (!confirmed) return;
    setProfileError(null);
    try {
        await desktopBridge.removeProfile(userId);
        setDesktopProfiles((profiles) =>
            profiles.filter((profile) => profile.user.id !== userId),
        );
    } catch {
        setProfileError("Local profile removal failed.");
    }
}

return (
    <div className="h-full overflow-y-auto">
        <div className="mx-auto max-w-5xl p-6 pb-12 pt-12 md:pt-6">
            <h1 className="mb-6 text-2xl font-bold text-gray-100">Settings</h1>

            <section className="mb-8">
                <h2 className="mb-2 text-lg font-semibold text-gray-200">Account</h2>
                <p className="text-sm text-gray-400">{email}</p>
            </section>

            {window.yinshiDesktop ? (
                <section className="mb-6" aria-labelledby="local-profiles-heading">
                    <h2
                        id="local-profiles-heading"
                        className="mb-2 text-sm font-semibold text-gray-300"
                    >
                        Local profiles
                    </h2>
                    <div className="space-y-2">
                        {desktopProfiles.map((profile) => (
                            <div
                                key={profile.user.id}
                                className="flex flex-wrap items-center justify-between gap-3 rounded-xl b

```

```

>
<div>
  <p className="text-sm font-medium text-gray-200">
    {profile.user.email}
  </p>
  <p className="mt-1 text-xs text-gray-500">
    {profile.active
      ? "Active on this Mac"
      : profile.hasCredentials
        ? "Ready to switch"
        : "Signed out; local data retained"}
  </p>
</div>
<div className="flex items-center gap-2">
  {!profile.active ? (
    <button
      type="button"
      disabled={!profile.hasCredentials}
      onClick={() =>
        void switchDesktopProfile(profile.user.id)
      }
      className="rounded border border-gray-700 px-3 py-1.5 text-xs text-g
    >
      Switch
    </button>
  ) : null}
  <button
    type="button"
    onClick={() =>
      void removeDesktopProfile(
        profile.user.id,
        profile.user.email,
      )
    }
    className="rounded border border-red-900/60 px-3 py-1.5 text-xs text-1
  >
    Remove local data
  </button>
</div>
</div>
)))}
{desktopProfiles.length === 0 ? (
  <p className="text-sm text-gray-500">
    No retained local profiles.
  </p>
) : null}

```

```

        {profileError ? (
          <p className="text-sm text-red-400">{profileError}</p>
        ) : null}
      </div>
    </section>
  ) : null}

{window.yinshiDesktop ? (
  <section
    className="mb-6"
    aria-labelledby="local-storage-security-heading"
  >
    <h2
      id="local-storage-security-heading"
      className="mb-2 text-sm font-semibold text-gray-300"
    >
      Local storage security
    </h2>
    <div className="rounded-xl border border-gray-800 bg-gray-900/70 px-4 py-3">
      <div className="flex items-center justify-between gap-4">
        <div>
          <p className="text-sm font-medium text-gray-200">FileVault</p>
          <p className="mt-1 text-xs text-gray-500">
            Yinshi encrypts databases and secrets. FileVault also
            protects repository files at rest.
          </p>
        </div>
        <div>
          <span
            className={`text-xs font-medium ${
              fileVaultStatus === "enabled"
                ? "text-green-400"
                : fileVaultStatus === "disabled"
                  ? "text-amber-300"
                  : "text-gray-500"
            }`}
          >
            {fileVaultStatus === "enabled"
              ? "Enabled"
              : fileVaultStatus === "disabled"
                ? "Disabled"
                : "Status unavailable"}
          </span>
        </div>
      </div>
    </div>
  </section>
) : null}

```

```

<div
  className="mb-6 flex flex-wrap gap-2"
  role="tablist"
  aria-label="Settings sections"
>
  {SETTINGS_TABS.map((tab) => (
    <SettingsTabButton
      key={tab.id}
      tab={tab}
      activeTab={activeTab}
      onSelect={setActiveTab}
    />
  ))}
</div>

<div
  id={`settings-panel-${activeTab}`}
  role="tabpanel"
  aria-labelledby={`settings-tab-${activeTab}`}
>
  {(activeTab === "providers" ||
    activeTab === "pi-config" ||
    activeTab === "pi-release-notes") &&
    primaryRuntime &&
    selectedRuntime ? (
    <RuntimeLocationSelector
      runtime={selectedRuntime}
      primaryRuntime={primaryRuntime}
      byocRuntime={byocRuntime}
      onSelect={setSelectedRuntime}
    />
    ) : null}
  {(activeTab === "providers" ||
    activeTab === "pi-config" ||
    activeTab === "pi-release-notes") &&
    !runtimeTransport &&
    !runtimeResolutionError ? (
    <div className="rounded border border-gray-700 bg-gray-800 px-4 py-3 text-sm text-white">
      Loading provider catalog...
    </div>
    ) : null}
  {runtimeResolutionError ? (
    <div className="rounded border border-red-900/50 bg-gray-800 px-4 py-3 text-sm text-white">
      <p>{runtimeResolutionError}</p>
      {requestedPrimaryRuntime.location === "hosted" &&

```

```

        window.yinshiDesktop === undefined ? (
          <button
            type="button"
            onClick={() => void provisionManagedRuntime()}
            disabled={runtimeProvisioning}
            className="mt-3 rounded border border-gray-600 px-3 py-1.5 text-sm text-gray-600"
          >
            {runtimeProvisioning
              ? "Provisioning managed runtime..."
              : "Provision managed runtime"}
          </button>
        ) : null}
      </div>
    ) : null}
    {activeTab === "providers" && runtimeTransport ? (
      <ProvidersSection transport={runtimeTransport} />
    ) : null}
    {activeTab === "cloud-runner" ? <CloudRunnerSection /> : null}
    {activeTab === "pi-config" && runtimeTransport ? (
      <PiConfigSection transport={runtimeTransport} />
    ) : null}
    {activeTab === "pi-release-notes" && runtimeTransport ? (
      <PiReleaseNotesSection transport={runtimeTransport} />
    ) : null}
  </div>
</div>
</div>
);
}

```

21 Frontend API and State Hooks

Network code is typed at the boundary, then hooks turn it into page-level state. This keeps rendering code from duplicating fetch details, SSE parsing, retry behavior, or local-storage synchronization.

21.1 api/client.ts

The API client centralizes typed HTTP calls, SSE parsing, WebSocket URL creation, and backend error translation. The root chunk below tangles back to `frontend/src/api/client.ts`.

```

export interface Repo {
  id: string;
  created_at: string;
  updated_at: string;
}

```

```

    name: string;
    remote_url: string | null;
    root_path: string;
    custom_prompt: string | null;
    agents_md: string | null;
}

export interface GitHubInstallation {
    installation_id: number;
    account_login: string;
    account_type: string;
    html_url: string;
}

export interface ApiKey {
    id: string;
    created_at: string;
    provider: string;
    label: string;
    last_used_at: string | null;
}

export interface ProviderSetupField {
    key: string;
    label: string;
    required: boolean;
    secret: boolean;
}

export interface ProviderDescriptor {
    id: string;
    label: string;
    auth_strategies: string[];
    setup_fields: ProviderSetupField[];
    docs_url: string;
    connected: boolean;
    model_count: number;
}

export type ThinkingLevel =
    "off" | "minimal" | "low" | "medium" | "high" | "xhigh";

export interface ModelDescriptor {
    ref: string;
    provider: string;
    id: string;
}

```



```

    label: string;
    api: string;
    reasoning: boolean;
    thinking_levels?: ThinkingLevel[];
    inputs: string[];
    context_window: number;
    max_tokens: number;
}

export interface ProviderCatalog {
    default_model: string;
    providers: ProviderDescriptor[];
    models: ModelDescriptor[];
}

export interface ProviderConnection {
    id: string;
    created_at: string;
    updated_at: string;
    provider: string;
    auth_strategy: string;
    label: string;
    config: Record<string, unknown>;
    status: string;
    last_used_at: string | null;
    expires_at: string | null;
}

export type CloudRunnerStatus = "pending" | "online" | "offline" | "revoked";
export type RunnerStorageProfile =
    "aws_ebs_s3_files" | "archil_shared_files" | "archil_all_posix";

export interface CloudRunner {
    id: string;
    created_at: string;
    updated_at: string;
    name: string;
    cloud_provider: string;
    region: string;
    status: CloudRunnerStatus;
    registered_at: string | null;
    last_heartbeat_at: string | null;
    runner_version: string | null;
    capabilities: Record<string, unknown>;
    data_dir: string | null;
    noise_public_key: string | null;
}

```

```

    noise_key_fingerprint: string | null;
    noise_key_confirmed: boolean;
}

export interface CloudRunnerRegistration {
    runner: CloudRunner;
    registration_token: string;
    registration_token_expires_at: string;
    control_url: string;
    environment: Record<string, string>;
}

export interface ProviderAuthStart {
    flow_id: string;
    provider: string;
    auth_url: string;
    instructions: string | null;
    manual_input_required: boolean;
    manual_input_prompt: string | null;
    manual_input_submitted: boolean;
}

export interface ProviderAuthStatus {
    status: string;
    provider: string;
    flow_id: string;
    instructions?: string | null;
    progress?: string[];
    manual_input_required?: boolean;
    manual_input_prompt?: string | null;
    manual_input_submitted?: boolean;
    error?: string | null;
}

export interface PiConfig {
    id: string;
    created_at: string;
    updated_at: string;
    source_type: "upload" | "github";
    source_label: string;
    last_synced_at: string | null;
    status: "ready" | "cloning" | "syncing" | "error";
    error_message: string | null;
    available_categories: string[];
    enabled_categories: string[];
}

```

```

export type PiCommandKind = "skill" | "prompt" | "extension";

export interface PiCommand {
  kind: PiCommandKind;
  name: string;
  description: string;
  command_name: string;
}

export interface PiConfigCommands {
  commands: PiCommand[];
}

export interface PiPackageRelease {
  tag_name: string;
  version: string;
  name: string;
  published_at: string | null;
  html_url: string;
  body_markdown: string;
}

export interface PiReleaseNotes {
  package_name: string;
  installed_version: string | null;
  latest_version: string | null;
  node_version: string | null;
  release_notes_url: string;
  update_policy: string;
  runtime_error: string | null;
  release_error: string | null;
  releases: PiPackageRelease[];
}

export interface Workspace {
  id: string;
  created_at: string;
  updated_at: string;
  repo_id: string;
  name: string;
  branch: string;
  path: string;
  state: string;
}

```

```

export interface SessionInfo {
  id: string;
  created_at: string;
  updated_at: string;
  workspace_id: string;
  status: string;
  model: string;
  pi_context_version: number;
}

export interface Message {
  id: string;
  created_at: string;
  session_id: string;
  role: string;
  content: string | null;
  full_message: string | null;
  turn_id: string | null;
  turn_status: string | null;
}

export interface WorkspaceFileNode {
  name: string;
  path: string;
  type: "file" | "directory";
  children: WorkspaceFileNode[];
}

export interface WorkspaceChangedFile {
  path: string;
  status: string;
  kind:
    | "added"
    | "copied"
    | "deleted"
    | "modified"
    | "renamed"
    | "untracked"
    | "unknown";
  original_path: string | null;
}

interface StructuredApiErrorPayload {
  code?: string;
  message?: string;
  connect_url?: string | null;
}

```

```

    manage_url?: string | null;
}

export class ApiError extends Error {
    public code?: string;
    public connect_url?: string | null;
    public manage_url?: string | null;

    constructor(
        public status: number,
        message: string,
        payload?: StructuredApiErrorPayload,
    ) {
        super(message);
        this.name = "ApiError";
        this.code = payload?.code;
        this.connect_url = payload?.connect_url;
        this.manage_url = payload?.manage_url;
    }
}

function _normalizeErrorPayload(
    payload: unknown,
): StructuredApiErrorPayload | null {
    if (!payload || typeof payload !== "object") {
        return null;
    }

    const detail = "detail" in payload ? payload.detail : payload;
    if (!detail || typeof detail !== "object") {
        return null;
    }

    const candidate = detail as Record<string, unknown>;
    const normalized: StructuredApiErrorPayload = {};
    if (typeof candidate.code === "string") {
        normalized.code = candidate.code;
    }
    if (typeof candidate.message === "string") {
        normalized.message = candidate.message;
    }
    if (typeof candidate.connect_url === "string") {
        normalized.connect_url = candidate.connect_url;
    }
    if (candidate.connect_url === null) {
        normalized.connect_url = null;
    }
}

```

```

    }
    if (typeof candidate.manage_url === "string") {
        normalized.manage_url = candidate.manage_url;
    }
    if (candidate.manage_url === null) {
        normalized.manage_url = null;
    }
    return normalized;
}

function _apiErrorFromPayload(status: number, payload: unknown): ApiError {
    const normalized = _normalizeErrorPayload(payload);
    if (normalized?.message) {
        return new ApiError(status, normalized.message, normalized);
    }
    if (
        payload &&
        typeof payload === "object" &&
        "detail" in payload &&
        typeof payload.detail === "string"
    ) {
        return new ApiError(status, payload.detail);
    }
    return new ApiError(status, `API request failed with status ${status}`);
}

async function _readApiError(response: Response): Promise<ApiError> {
    const contentType = response.headers.get("content-type") || "";
    if (contentType.includes("application/json")) {
        const payload = await response.json().catch(() => null);
        const error = _apiErrorFromPayload(response.status, payload);
        if (payload !== null) {
            return error;
        }
    }

    const text = await response.text().catch(() => "");
    return new ApiError(response.status, text || response.statusText);
}

const DESKTOP_HOSTED_RUNNER_PATHS = new Set([
    "/api/settings/runner",
    "/api/settings/runner/capabilities",
    "/api/settings/runner/noise-key/confirm",
]);

```

```

async function desktopHostedRequest<T>(
  method: DesktopHostedApiRequest["method"],
  path: string,
  body: unknown,
  forceHosted = false,
): Promise<T | undefined> {
  const bridge = window.yinshiDesktop;
  if (
    bridge === undefined ||
    (!forceHosted && !DESKTOP_HOSTED_RUNNER_PATHS.has(path))
  ) {
    return undefined;
  }
  if (!["DELETE", "GET", "PATCH", "POST", "PUT"].includes(method)) {
    throw new TypeError("Desktop hosted API method is not supported");
  }
  if (
    body !== undefined &&
    (body === null || typeof body !== "object" || Array.isArray(body))
  ) {
    throw new TypeError("Desktop hosted API body must be an object");
  }
  const hostedRequest: DesktopHostedApiRequest =
    body === undefined
      ? { method, path }
      : { method, path, body: body as Readonly<Record<string, unknown>> };
  const response = await bridge.hostedRequest(hostedRequest);
  if (
    !Number.isInteger(response.status) ||
    response.status < 100 ||
    response.status > 599
  ) {
    throw new Error("Desktop hosted API returned an invalid status");
  }
  if (response.status < 200 || response.status > 299) {
    throw _apiErrorFromPayload(response.status, response.body);
  }
  if (response.status === 204) {
    return undefined as T;
  }
  return response.body as T;
}

async function request<T>(
  method: DesktopHostedApiRequest["method"],
  path: string,

```

```

    body?: unknown,
  ): Promise<T> {
    const usesDesktopHostedApi =
      window.yinshiDesktop !== undefined && DESKTOP_HOSTED_RUNNER_PATHS.has(path);
    const hostedResponse = await desktopHostedRequest<T>(method, path, body);
    if (usesDesktopHostedApi) {
      return hostedResponse as T;
    }
    const opts: RequestInit = {
      method,
      headers: {
        "Content-Type": "application/json",
        "X-Requested-With": "XMLHttpRequest",
      },
      credentials: "include",
    };
    if (body !== undefined) {
      opts.body = JSON.stringify(body);
    }
    const res = await fetch(path, opts);
    if (!res.ok) {
      if (res.status === 401 && window.location.pathname.startsWith("/app")) {
        window.location.href = "/";
      }
      throw await _readApiError(res);
    }
    if (res.status === 204) return undefined as T;
    return res.json();
  }

  async function hostedRequest<T>(
    method: DesktopHostedApiRequest["method"],
    path: string,
    body?: unknown,
  ): Promise<T> {
    if (window.yinshiDesktop !== undefined) {
      return (await desktopHostedRequest<T>(method, path, body, true)) as T;
    }
    return request<T>(method, path, body);
  }

  export const api = {
    get: <T>(path: string) => request<T>("GET", path),
    post: <T>(path: string, body?: unknown) => request<T>("POST", path, body),
    patch: <T>(path: string, body?: unknown) => request<T>("PATCH", path, body),
    put: <T>(path: string, body?: unknown) => request<T>("PUT", path, body),
  }

```



```

delete: (path: string) => request<void>("DELETE", path),
upload: async <T>(path: string, file: File): Promise<T> => {
  const form = new FormData();
  form.append("file", file);
  const response = await fetch(path, {
    method: "POST",
    credentials: "include",
    headers: { "X-Requested-With": "XMLHttpRequest" },
    body: form,
  });
  if (!response.ok) {
    if (
      response.status === 401 &&
      window.location.pathname.startsWith("/app")
    ) {
      window.location.href = "/";
    }
    throw await _readApiError(response);
  }
  return response.json();
},
};

export const hostedApi = {
  get: <T>(path: string) => hostedRequest<T>("GET", path),
  post: <T>(path: string, body?: unknown) =>
    hostedRequest<T>("POST", path, body),
  patch: <T>(path: string, body?: unknown) =>
    hostedRequest<T>("PATCH", path, body),
  put: <T>(path: string, body?: unknown) => hostedRequest<T>("PUT", path, body),
  delete: (path: string) => hostedRequest<void>("DELETE", path),
  upload: <T>(path: string, file: File) => api.upload<T>(path, file),
};

export type SSEEvent =
  | { type: "assistant"; message: { content: ContentBlock[] } }
  | {
    type: "tool_use";
    name: string;
    tool_name?: string;
    id?: string;
    input: unknown;
  }
  | {
    type: "tool_result";
    tool_use_id: string;
  };

```

```

        content: string | ContentBlock[] | unknown;
        is_error?: boolean;
    }
| { type: "content_block_start"; content_block: ContentBlock; index?: number }
| {
    type: "content_block_delta";
    delta: {
        type: string;
        text?: string;
        partial_json?: string;
        thinking?: string;
    };
    index?: number;
}
| { type: "content_block_stop"; index?: number }
| { type: "message_start"; message?: unknown }
| { type: "message_delta"; delta?: unknown }
| { type: "message_stop" }
| { type: "result"; [key: string]: unknown }
| {
    type: "status";
    status: string;
    message?: string;
    severity?: "info" | "warning" | "error";
    [key: string]: unknown;
}
| { type: "cancelled"; reason?: string }
| { type: "error"; error: string };

export interface ContentBlock {
    type: string;
    text?: string;
    thinking?: string;
    id?: string;
    name?: string;
    input?: unknown;
}

export function normalizeEvent(raw: Record<string, unknown>): SSEEvent {
    if (raw.type === "tool_use") {
        return {
            type: "tool_use",
            name: (raw.toolName || raw.name || raw.tool_name || "unknown") as string,
            id: raw.id as string,
            input: raw.toolInput ?? raw.input,
        };
    }
}

```

```

    }
    return raw as SSEEvent;
}

export async function* streamPrompt(
  sessionId: string,
  prompt: string,
  model?: string,
  thinking?: ThinkingLevel,
  signal?: AbortSignal,
): AsyncGenerator<SSEEvent> {
  const res = await fetch(`/api/sessions/${sessionId}/prompt`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-Requested-With": "XMLHttpRequest",
    },
    credentials: "include",
    body: JSON.stringify({ prompt, model, thinking }),
    signal,
  });

  if (!res.ok) {
    if (res.status === 401 && window.location.pathname.startsWith("/app")) {
      window.location.href = "/";
    }
    throw await _readApiError(res);
  }

  if (!res.body) {
    throw new ApiError(res.status, "Response body is null");
  }

  const reader = res.body.getReader();
  const decoder = new TextDecoder();
  let buffer = "";

  try {
    while (true) {
      const { done, value } = await reader.read();
      if (done) break;
      buffer += decoder.decode(value, { stream: true });

      // Parse complete SSE lines from buffer
      const lines = buffer.split("\n");
      buffer = lines.pop() ?? "";
    }
  }

```

```

    for (const line of lines) {
      const trimmed = line.trim();
      if (trimmed.startsWith("data: ")) {
        try {
          yield normalizeEvent(JSON.parse(trimmed.slice(6)));
        } catch {
          /* ignore malformed events */
        }
      }
    }
  }
} finally {
  reader.releaseLock();
}
}

export async function cancelSession(sessionId: string): Promise<void> {
  await request<{ status: string }>({
    "POST",
    `/api/sessions/${sessionId}/cancel`,
  });
}

```

21.2 api/piCommandsCache.ts

The command cache keeps imported slash-command metadata fresh without refetching on every chat render. The root chunk below tangles back to frontend/src/api/piCommandsCache.ts.

```

import { ApiError, api, type PiConfigCommands } from "../client";
import type { SlashCommand } from "../components/SlashCommandMenu";
import type { RuntimeTransport } from "../runtime/runtimeTransport";

// Module-level cache shared across every Session mount. Without this, every
// session navigation triggers a new HTTP request which triggers the sidecar
// to re-evaluate every extension module (moduleCache:false in pi-mono).
const cachedPromises = new Map<string, Promise<SlashCommand[]>>();
const subscribers = new Set<() => void>();

function toSlashCommand(command: PiConfigCommands["commands"][number]): SlashCommand {
  return {
    name: command.command_name,
    description: command.description,
    source: "pi",
  };
}

```

```

}

function transportKey(runtimeTransport?: RuntimeTransport): string {
  const runtime = runtimeTransport?.runtime;
  if (!runtime) return "default";
  return runtime.location === "byoc" ? `byoc:${runtime.runnerId}` : runtime.location;
}

async function fetchCommands(runtimeTransport?: RuntimeTransport): Promise<SlashCommand[]> {
  try {
    const commandClient = runtimeTransport ?? api;
    const payload = await commandClient.get<PiConfigCommands>(
      "/api/settings/pi-config/commands",
    );
    return payload.commands.map(toSlashCommand);
  } catch (error) {
    // A missing Pi config has no imported commands. A 503 is different: the
    // sidecar or tenant container is still warming up, so callers must retry
    // rather than cache an empty palette for the rest of the browser session.
    if (error instanceof ApiError && error.status === 404) {
      return [];
    }
    throw error;
  }
}

export function getCachedPiCommands(
  runtimeTransport?: RuntimeTransport,
): Promise<SlashCommand[]> {
  const key = transportKey(runtimeTransport);
  const cachedPromise = cachedPromises.get(key);
  if (cachedPromise) return cachedPromise;
  const promise = fetchCommands(runtimeTransport).catch((error) => {
    // Clear only this location after failure so another runtime's cache survives.
    if (cachedPromises.get(key) === promise) {
      cachedPromises.delete(key);
    }
    throw error;
  });
  cachedPromises.set(key, promise);
  return promise;
}

export function invalidatePiCommands(runtimeTransport?: RuntimeTransport): void {
  if (runtimeTransport) {
    cachedPromises.delete(transportKey(runtimeTransport));
  }
}

```

```

    } else {
      cachedPromises.clear();
    }
    for (const notify of subscribers) {
      notify();
    }
  }
}

export function subscribePiCommands(notify: () => void): () => void {
  subscribers.add(notify);
  return () => {
    subscribers.delete(notify);
  };
}

```

21.3 hooks/useAuth.tsx

Auth state is fetched once and distributed through React context. The root chunk below tangles back to `frontend/src/hooks/useAuth.tsx`.

```

import {
  createContext,
  useContext,
  useEffect,
  useState,
  type ReactNode,
} from "react";

type AuthStatus = "loading" | "authenticated" | "unauthenticated" | "disabled";

interface AuthState {
  status: AuthStatus;
  email: string | null;
  userId: string | null;
  logout: () => Promise<void>;
}

interface AuthProviderProps {
  children: ReactNode;
}

const AuthContext = createContext<AuthState>({
  status: "loading",
  email: null,
  userId: null,
  logout: async () => {},

```

```

});

export function AuthProvider({ children }: AuthProviderProps) {
  const [status, setStatus] = useState<AuthStatus>("loading");
  const [email, setEmail] = useState<string | null>(null);
  const [userId, setUserId] = useState<string | null>(null);

  useEffect(() => {
    async function checkAuth() {
      try {
        const desktopApi = window.yinshiDesktop;
        if (desktopApi !== undefined) {
          const profiles = await desktopApi.listProfiles();
          const activeProfile = profiles.find(
            (profile) => profile.active && profile.hasCredentials,
          );
          if (activeProfile === undefined) {
            setStatus("unauthenticated");
            return;
          }
          setEmail(activeProfile.user.email);
          setUserId(activeProfile.user.id);
          setStatus("authenticated");
          return;
        }
        const res = await fetch("/auth/me", { credentials: "include" });
        if (!res.ok) {
          setStatus("unauthenticated");
          return;
        }
        const data = await res.json();
        if (data.authenticated) {
          setEmail(data.email);
          setUserId(data.user_id || null);
          setStatus("authenticated");
        } else if (data.auth_disabled === true) {
          setStatus("disabled");
        } else {
          setStatus("unauthenticated");
        }
      } catch {
        setStatus("unauthenticated");
      }
    }
    checkAuth();
  }, []);

```

```

async function logout() {
  if (window.yinshiDesktop !== undefined) {
    await window.yinshiDesktop.signOut();
  } else {
    await fetch("/auth/logout", {
      method: "POST",
      credentials: "include",
      headers: { "X-Requested-With": "XMLHttpRequest" },
    }).catch(() => {});
  }
  setStatus("unauthenticated");
  setEmail(null);
  setUserId(null);
  window.location.href = "/";
}

if (status === "loading") {
  return (
    <div className="flex min-h-screen items-center justify-center bg-gray-900">
      <div className="h-6 w-6 animate-spin rounded-full border-2 border-blue-500 border-t-
    </div>
    </div>
  );
}

return (
  <AuthContext.Provider value={{ status, email, userId, logout }}>
    {children}
  </AuthContext.Provider>
);
}

export function useAuth(): AuthState {
  return useContext(AuthContext);
}

```

21.4 hooks/useTheme.tsx

Theme state synchronizes the app's light or dark mode with local storage and the document root. The root chunk below tangles back to frontend/src/hooks/useTheme.tsx.

```

import { useCallback, useEffect, useState } from "react";

type Theme = "light" | "dark";

```



```

function getInitialTheme(): Theme {
  const stored = localStorage.getItem("theme");
  if (stored === "dark" || stored === "light") return stored;
  return "light";
}

export function useTheme() {
  const [theme, setTheme] = useState<Theme>(getInitialTheme);

  useEffect(() => {
    const root = document.documentElement;
    if (theme === "dark") {
      root.classList.add("dark");
    } else {
      root.classList.remove("dark");
    }
    localStorage.setItem("theme", theme);
  }, [theme]);

  const toggle = useCallback(() => {
    setTheme((t) => (t === "light" ? "dark" : "light"));
  }, []);

  return { theme, toggle };
}

```

21.5 hooks/useCatalog.ts

The catalog hook loads provider and model metadata used by prompt settings. The root chunk below tangles back to frontend/src/hooks/useCatalog.ts.

```

import { useEffect, useState } from "react";

import { api, type ProviderCatalog } from "../api/client";
import type { RuntimeTransport } from "../runtime/runtimeTransport";

export function useCatalog(runtimeTransport?: RuntimeTransport) {
  const [catalog, setCatalog] = useState<ProviderCatalog | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let cancelled = false;
    setLoading(true);
    setError(null);
    const catalogClient = runtimeTransport ?? api;

```

```

async function loadCatalog() {
  try {
    const loadedCatalog = await catalogClient.get<ProviderCatalog>("/api/catalog");
    if (cancelled) {
      return;
    }
    setCatalog(loadedCatalog);
    setError(null);
  } catch (loadError) {
    if (cancelled) {
      return;
    }
    setError(loadError instanceof Error ? loadError.message : "Failed to load catalog");
  } finally {
    if (!cancelled) {
      setLoading(false);
    }
  }
}

void loadCatalog();
return () => {
  cancelled = true;
};
}, [runtimeTransport]);

return { catalog, loading, error, setCatalog };
}

```

21.6 hooks/usePiCommands.ts

The Pi command hook retries command discovery and subscribes to cache invalidation. The root chunk below tangles back to `frontend/src/hooks/usePiCommands.ts`.

```

import { useEffect, useState } from "react";

import {
  getCachedPiCommands,
  subscribePiCommands,
} from "../api/piCommandsCache";
import type { SlashCommand } from "../components/SlashCommandMenu";
import type { RuntimeTransport } from "../runtime/runtimeTransport";

const EMPTY_COMMANDS: SlashCommand[] = [];
const COMMAND_RETRY_DELAY_MS = 3000;

```

```

export function usePiCommands(runtimeTransport?: RuntimeTransport): SlashCommand[] {
  const [commands, setCommands] = useState<SlashCommand[]>(EMPTY_COMMANDS);

  useEffect(() => {
    let disposed = false;
    let retryTimer: number | null = null;
    let loadVersion = 0;

    function clearRetry(): void {
      if (retryTimer === null) {
        return;
      }
      window.clearTimeout(retryTimer);
      retryTimer = null;
    }

    function scheduleRetry(): void {
      if (disposed) {
        return;
      }
      if (retryTimer !== null) {
        return;
      }
      retryTimer = window.setTimeout(() => {
        retryTimer = null;
        void load();
      }, COMMAND_RETRY_DELAY_MS);
    }

    async function load(): Promise<void> {
      const currentVersion = loadVersion + 1;
      loadVersion = currentVersion;
      try {
        const loaded = await getCachedPiCommands(runtimeTransport);
        if (!disposed && currentVersion === loadVersion) {
          clearRetry();
          setCommands(loaded);
        }
      } catch {
        // Network/server failure -- fall back to no pi commands rather than
        // block the palette entirely. Transient sidecar startup errors retry so
        // a cold container does not leave the slash palette permanently empty.
        if (!disposed && currentVersion === loadVersion) {
          setCommands(EMPTY_COMMANDS);
          scheduleRetry();
        }
      }
    }
  });
}

```

```

    }
  }
}

void load();
const unsubscribe = subscribePiCommands(() => {
  clearRetry();
  void load();
});

return () => {
  disposed = true;
  loadVersion += 1;
  clearRetry();
  unsubscribe();
};
}, [runtimeTransport]);

return commands;
}

```

21.7 hooks/usePiConfig.ts

The Pi config hook owns upload, GitHub import, enablement, deletion, and category toggles for imported agent config. The root chunk below tangles back to frontend/src/hooks/usePiConfig.ts.

```

import { useEffect, useRef, useState } from "react";

import { ApiError, type PiConfig } from "../api/client";
import type { RuntimeTransport } from "../runtime/runtimeTransport";
import { invalidatePiCommands } from "../api/piCommandsCache";

export interface UsePiConfigReturn {
  config: PiConfig | null;
  loading: boolean;
  error: string | null;
  importing: boolean;
  syncing: boolean;
  updatingCategories: boolean;
  loadConfig: () => Promise<void>;
  importFromGithub: (repoUrl: string) => Promise<boolean>;
  importFromUpload: (file: File) => Promise<boolean>;
  syncConfig: () => Promise<boolean>;
  removeConfig: () => Promise<boolean>;
  toggleCategory: (category: string, enabled: boolean) => Promise<boolean>;
}

```

```

}

function errorStatus(error: unknown): number | null {
  if (error instanceof ApiError) return error.status;
  if (error !== null && typeof error === "object" && "status" in error) {
    const status = (error as { status?: unknown }).status;
    return typeof status === "number" ? status : null;
  }
  return null;
}

function getErrorMessage(error: unknown, fallback: string): string {
  if (error instanceof Error && error.message) {
    return error.message;
  }
  return fallback;
}

function buildEnabledCategories(
  currentCategories: string[],
  category: string,
  enabled: boolean,
): string[] {
  const nextCategories = new Set(currentCategories);
  if (enabled) {
    nextCategories.add(category);
  } else {
    nextCategories.delete(category);
  }
  return Array.from(nextCategories);
}

export function usePiConfig(transport: RuntimeTransport): UsePiConfigReturn {
  const [config, setConfig] = useState<PiConfig | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [importing, setImporting] = useState(false);
  const [syncing, setSyncing] = useState(false);
  const [updatingCategories, setUpdatingCategories] = useState(false);
  const isMountedRef = useRef(true);

  async function loadConfigInternal(polling: boolean): Promise<void> {
    if (!polling) {
      setLoading(true);
    }
    try {

```

```

    const nextConfig = await transport.get<PiConfig>("/api/settings/pi-config");
    if (!isMountedRef.current) {
        return;
    }
    setConfig(nextConfig);
    setError(null);
} catch (requestError) {
    if (!isMountedRef.current) {
        return;
    }
    if (errorStatus(requestError) === 404) {
        setConfig(null);
        setError(null);
    } else if (!polling) {
        setError(getErrorMessage(requestError, "Failed to load Pi config"));
    }
} finally {
    if (!polling && isMountedRef.current) {
        setLoading(false);
    }
}
}

async function loadConfig(): Promise<void> {
    await loadConfigInternal(false);
}

async function importFromGithub(repoUrl: string): Promise<boolean> {
    setImporting(true);
    setError(null);
    try {
        const nextConfig = await transport.post<PiConfig>("/api/settings/pi-config/github", {
            repo_url: repoUrl,
        });
        if (isMountedRef.current) {
            setConfig(nextConfig);
        }
        invalidatePiCommands(transport);
        return true;
    } catch (requestError) {
        if (isMountedRef.current) {
            setError(getErrorMessage(requestError, "Failed to import from GitHub"));
        }
        return false;
    } finally {
        if (isMountedRef.current) {

```

```

        setImporting(false);
    }
}

}

async function importFromUpload(file: File): Promise<boolean> {
    setImporting(true);
    setError(null);
    try {
        const nextConfig = await transport.upload<PiConfig>(
            "/api/settings/pi-config/upload",
            file,
        );
        if (isMountedRef.current) {
            setConfig(nextConfig);
        }
        invalidatePiCommands(transport);
        return true;
    } catch (requestError) {
        if (isMountedRef.current) {
            setError(getErrorMessage(requestError, "Failed to upload Pi config"));
        }
        return false;
    } finally {
        if (isMountedRef.current) {
            setImporting(false);
        }
    }
}

async function syncConfig(): Promise<boolean> {
    setSyncing(true);
    setError(null);
    try {
        const nextConfig = await transport.post<PiConfig>("/api/settings/pi-config/sync");
        if (isMountedRef.current) {
            setConfig(nextConfig);
        }
        invalidatePiCommands(transport);
        return true;
    } catch (requestError) {
        if (isMountedRef.current) {
            setError(getErrorMessage(requestError, "Failed to sync Pi config"));
        }
        return false;
    } finally {

```

```

        if (isMountedRef.current) {
            setSyncing(false);
        }
    }
}

async function removeConfig(): Promise<boolean> {
    setError(null);
    try {
        await transport.delete("/api/settings/pi-config");
        if (isMountedRef.current) {
            setConfig(null);
        }
        invalidatePiCommands(transport);
        return true;
    } catch (requestError) {
        if (isMountedRef.current) {
            setError(getErrorMessage(requestError, "Failed to remove Pi config"));
        }
        return false;
    }
}

async function toggleCategory(category: string, enabled: boolean): Promise<boolean> {
    if (!config) {
        return false;
    }
    if (updatingCategories || syncing) {
        return false;
    }
    setError(null);
    setUpdatingCategories(true);
    const previousConfig = config;
    const enabledCategories = buildEnabledCategories(
        previousConfig.enabled_categories,
        category,
        enabled,
    );
    const optimisticConfig: PiConfig = {
        ...previousConfig,
        enabled_categories: enabledCategories,
    };
    if (isMountedRef.current) {
        setConfig(optimisticConfig);
    }
    try {

```



```

const nextConfig = await transport.patch<PiConfig>(
  "/api/settings/pi-config/categories",
  {
    enabled_categories: enabledCategories,
  },
);
if (isMountedRef.current) {
  setConfig(nextConfig);
}
invalidatePiCommands(transport);
return true;
} catch (requestError) {
  if (isMountedRef.current) {
    setConfig(previousConfig);
    setError(getErrorMessage(requestError, "Failed to update Pi config categories"));
  }
  return false;
} finally {
  if (isMountedRef.current) {
    setUpdatingCategories(false);
  }
}
}

useEffect(() => {
  isMountedRef.current = true;
  void loadConfigInternal(false);
  return () => {
    isMountedRef.current = false;
  };
}, [transport]);

useEffect(() => {
  if (config?.status !== "cloning") {
    return undefined;
  }
  const intervalId = window.setInterval(() => {
    void loadConfigInternal(true);
  }, 2000);
  return () => {
    window.clearInterval(intervalId);
  };
}, [config?.status, transport]);

return {
  config,

```

```

    loading,
    error,
    importing,
    syncing,
    updatingCategories,
    loadConfig,
    importFromGithub,
    importFromUpload,
    syncConfig,
    removeConfig,
    toggleCategory,
  };
}

```

21.8 hooks/usePiReleaseNotes.ts

Release-note state is isolated so the settings page can refresh it independently.
The root chunk below tangles back to frontend/src/hooks/usePiReleaseNotes.ts.

```

import { useCallback, useEffect, useState } from "react";

import { type PiReleaseNotes } from "../api/client";
import type { RuntimeTransport } from "../runtime/runtimeTransport";

export function usePiReleaseNotes(transport: RuntimeTransport) {
  const [releaseNotes, setReleaseNotes] = useState<PiReleaseNotes | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  const loadReleaseNotes = useCallback(async (isCancelled: () => boolean = () => false) => {
    setLoading(true);
    try {
      const loadedReleaseNotes = await transport.get<PiReleaseNotes>(
        "/api/settings/pi-release-notes",
      );
      if (isCancelled()) {
        return;
      }
      setReleaseNotes(loadedReleaseNotes);
      setError(null);
    } catch (loadError) {
      if (isCancelled()) {
        return;
      }
      setError(loadError instanceof Error ? loadError.message : "Failed to load pi release notes");
    } finally {

```

```

        if (!isCancelled()) {
            setLoading(false);
        }
    }, [transport]);

    useEffect(() => {
        let cancelled = false;
        void loadReleaseNotes(() => cancelled);
        return () => {
            cancelled = true;
        };
    }, [loadReleaseNotes]);

    return {
        releaseNotes,
        loading,
        error,
        refresh: loadReleaseNotes,
    };
}

```

21.9 hooks/useAgentStream.ts

The stream hook turns backend SSE events into chat messages and run-state transitions. The root chunk below tangles back to frontend/src/hooks/useAgentStream.ts.

```

import { useCallback, useRef, useState } from "react";
import { cancelSession, streamPrompt, type ThinkingLevel } from "../api/client";
import {
    startRuntimePrompt,
    type RuntimePromptHandle,
} from "../runtime/promptStream";
import type { RuntimeTransport } from "../runtime/runtimeTransport";
import {
    applyTurnEventToBlocks,
    blocksToContent,
    type TurnBlock,
    type TurnStatus,
} from "../utils/turnEvents";

export type { TurnBlock } from "../utils/turnEvents";

export interface ChatMessage {
    id: string;
    role: "user" | "assistant" | "error";
}

```

```

    content: string;
    blocks: TurnBlock[];
    streaming?: boolean;
    turnStatus?: TurnStatus;
    timestamp: number;
  }

  let messageIdCounter = 0;
  function nextId(): string {
    return `msg-${Date.now()}-${++messageIdCounter}`;
  }

  export type RunState = "idle" | "running" | "stopping";

  const CANCELLATION_ERROR_MESSAGE =
    "Could not stop the current response. Try again.";

  export function useAgentStream(
    sessionId: string | undefined,
    runtimeTransport?: RuntimeTransport,
  ) {
    const [messages, setMessages] = useState<ChatMessage[]>([]);
    const [runState, setRunState] = useState<RunState>("idle");
    const abortRef = useRef<AbortController | null>(null);
    const promptHandleRef = useRef<RuntimePromptHandle | null>(null);
    const promptStartRef = useRef<Promise<RuntimePromptHandle> | null>(null);
    const cancelInFlightRef = useRef(false);
    const queuedPromptRef = useRef<{
      prompt: string;
      model?: string;
      thinking?: ThinkingLevel;
    } | null>(null);

    const startPrompt = useCallback(
      async (prompt: string, model?: string, thinking?: ThinkingLevel) => {
        if (!sessionId) return;
        const normalizedPrompt = prompt.trim();
        if (!normalizedPrompt) return;

        setMessages((prev) => [
          ...prev,
          {
            id: nextId(),
            role: "user",
            content: normalizedPrompt,
            blocks: [],

```

```

        timestamp: Date.now(),
    },
]);

setRunState("running");
const controller = new AbortController();
abortRef.current = controller;

const turnId = nextId();
const blocks: TurnBlock[] = [];
let rafId: number | null = null;
let turnStatus: TurnStatus = "completed";

function upsertTurn(done = false) {
    const allText = blocksToContent(blocks);
    const snapshot = blocks.map((b) => ({ ...b }));

    setMessages((prev) => {
        const msg: ChatMessage = {
            id: turnId,
            role: "assistant",
            content: allText,
            blocks: snapshot,
            streaming: !done,
            turnStatus: done ? turnStatus : undefined,
            timestamp: Date.now(),
        };
        const idx = prev.findIndex((m) => m.id === turnId);
        if (idx >= 0) {
            const next = [...prev];
            next[idx] = msg;
            return next;
        }
        return [...prev, msg];
    });
}

function scheduleUpsert(done = false) {
    if (done) {
        if (rafId) cancelAnimationFrame(rafId);
        rafId = null;
        upsertTurn(true);
        return;
    }
    if (rafId) return;
    rafId = requestAnimationFrame(() => {

```

```

        rafId = null;
        upsertTurn(false);
    });
}

try {
    const promptStart = runtimeTransport
        ? startRuntimePrompt(runtimeTransport, sessionId, {
            prompt: normalizedPrompt,
            model,
            thinking,
            signal: controller.signal,
        })
        : null;
    promptStartRef.current = promptStart;
    const runtimePrompt = promptStart ? await promptStart : null;
    promptHandleRef.current = runtimePrompt;
    const eventSource = runtimePrompt
        ? runtimePrompt.events()
        : streamPrompt(
            sessionId,
            normalizedPrompt,
            model,
            thinking,
            controller.signal,
        );
    for await (const event of eventSource) {
        const applyResult = applyTurnEventToBlocks(blocks, event, nextId);
        if (applyResult.status) {
            turnStatus = applyResult.status;
            scheduleUpsert(true);
            if (applyResult.status !== "completed" || event.type === "result") {
                break;
            }
            continue;
        }
        if (applyResult.changed) {
            scheduleUpsert();
        }
    }
} catch (e) {
    if (!(e instanceof DOMException && e.name === "AbortError")) {
        blocks.push({
            id: nextId(),
            type: "error",
            text: e instanceof Error ? e.message : "Stream failed",
        });
    }
}

```

```

    });
    turnStatus = "failed";
    scheduleUpsert(true);
  }
} finally {
  if (rafId) cancelAnimationFrame(rafId);
  abortRef.current = null;
  promptHandleRef.current = null;
  promptStartRef.current = null;

  const queuedPrompt = queuedPromptRef.current;
  queuedPromptRef.current = null;
  setRunState("idle");

  // Always replay a queued steering prompt once the active run finishes.
  if (queuedPrompt) {
    void startPrompt(
      queuedPrompt.prompt,
      queuedPrompt.model,
      queuedPrompt.thinking,
    );
  }
},
[runtimeTransport, sessionId],
);

const cancel = useCallback(async () => {
  if (runState !== "running" || cancelInFlightRef.current) return;
  cancelInFlightRef.current = true;
  const activeController = abortRef.current;
  setRunState("stopping");
  try {
    if (sessionId) {
      if (runtimeTransport) {
        const promptHandle =
          promptHandleRef.current ?? (await promptStartRef.current);
        if (promptHandle) {
          await promptHandle.cancel();
        }
      } else {
        await cancelSession(sessionId);
      }
    }
  }
} catch {
  if (

```

```

        activeController !== null &&
        abortRef.current === activeController
    ) {
        setRunState("running");
    }
    setMessages((prev) => [
        ...prev,
        {
            id: nextId(),
            role: "error",
            content: CANCELLATION_ERROR_MESSAGE,
            blocks: [],
            timestamp: Date.now(),
        },
    ]);
    } finally {
        cancelInFlightRef.current = false;
    }
}, [runtimeTransport, sessionId, runState]);

const sendPrompt = useCallback(
    async (prompt: string, model?: string, thinking?: ThinkingLevel) => {
        if (!sessionId) return;
        const normalizedPrompt = prompt.trim();
        if (!normalizedPrompt) return;

        if (runState === "running") {
            // Queue steering prompt and request stop
            queuedPromptRef.current = { prompt: normalizedPrompt, model, thinking };
            await cancel();
            return;
        }

        if (runState === "stopping") {
            // Replace queued steering prompt
            queuedPromptRef.current = { prompt: normalizedPrompt, model, thinking };
            return;
        }

        // runState === "idle", start normally
        await startPrompt(normalizedPrompt, model, thinking);
    },
    [cancel, runState, startPrompt],
);

return {

```



```

    messages,
    sendPrompt,
    cancel,
    runState,
    setMessages,
    streaming: runState === "running" || runState === "stopping",
  };
}

```

21.10 hooks/useMobileDrawer.ts

The mobile drawer hook keeps sidebar visibility local to narrow screens. The root chunk below tangles back to frontend/src/hooks/useMobileDrawer.ts.

```

import { useCallback, useState } from "react";

export function useMobileDrawer() {
  const [open, setOpen] = useState(false);
  const toggle = useCallback(() => setOpen((o) => !o), []);
  const close = useCallback(() => setOpen(false), []);
  const panelClassName = `fixed inset-y-0 left-0 z-40 w-72 transition-transform duration-200`;
  return { open, toggle, close, panelClassName };
}

```

21.11 models/sessionModels.ts

Session model helpers normalize model references, labels, provider preference, and thinking-level options. The root chunk below tangles back to frontend/src/models/sessionModels.ts.

```

import type { ModelDescriptor, ThinkingLevel } from "../api/client";

export const DEFAULT_SESSION_MODEL = "minimax/MiniMax-M2.7";

export const STANDARD_THINKING_LEVELS: ThinkingLevel[] = [
  "off",
  "minimal",
  "low",
  "medium",
  "high",
];

const LEGACY_MODEL_ALIASES = new Map<string, string>([
  ["haiku", "anthropic/claude-haiku-4-5-20251001"],
  ["minimax", DEFAULT_SESSION_MODEL],
  ["minimax-m2.5-highspeed", "minimax/MiniMax-M2.5-highspeed"],
  ["minimax-m2.7", DEFAULT_SESSION_MODEL],
  ["minimax-m2.7-highspeed", "minimax/MiniMax-M2.7-highspeed"],
]);

```

```

    ["opus", "anthropic/claude-opus-4-20250514"],
    ["sonnet", "anthropic/claude-sonnet-4-20250514"],
  ]);

function normalizeModelValue(value: string): string {
  return value.trim().toLowerCase();
}

function normalizePreferredProviderIds(
  preferredProviderIds: Iterable<string> | undefined,
): Set<string> {
  const normalizedProviderIds = new Set<string>();
  if (!preferredProviderIds) {
    return normalizedProviderIds;
  }
  for (const providerId of preferredProviderIds) {
    const normalizedProviderId = normalizeModelValue(providerId);
    if (normalizedProviderId) {
      normalizedProviderIds.add(normalizedProviderId);
    }
  }
  return normalizedProviderIds;
}

export function resolveSessionModelKey(
  model: string,
  models: ModelDescriptor[],
  preferredProviderIds?: Iterable<string>,
): string | null {
  const normalizedModel = normalizeModelValue(model);
  if (!normalizedModel) {
    return null;
  }
  const aliasMatch = LEGACY_MODEL_ALIASES.get(normalizedModel);
  if (aliasMatch) {
    return aliasMatch;
  }
  const directRefMatch = models.find((candidate) => {
    if (normalizeModelValue(candidate.ref) === normalizedModel) {
      return true;
    }
    return false;
  });
  if (directRefMatch) {
    return directRefMatch.ref;
  }
}

```

```

const matchingModels = models.filter((candidate) => {
  if (normalizeModelValue(candidate.id) === normalizedModel) {
    return true;
  }
  return normalizeModelValue(candidate.label) === normalizedModel;
});
if (matchingModels.length === 0) {
  return null;
}
if (matchingModels.length === 1) {
  return matchingModels[0]?.ref || null;
}

const normalizedPreferredProviderIds =
  normalizePreferredProviderIds(preferredProviderIds);
if (normalizedPreferredProviderIds.size === 0) {
  return null;
}
const preferredMatches = matchingModels.filter((candidate) =>
  normalizedPreferredProviderIds.has(normalizeModelValue(candidate.provider)),
);
if (preferredMatches.length === 1) {
  return preferredMatches[0]?.ref || null;
}
return null;
}

export function getSessionModelOption(
  model: string | null | undefined,
  models: ModelDescriptor[],
): ModelDescriptor | null {
  if (!model) {
    return null;
  }
  const resolvedKey = resolveSessionModelKey(model, models);
  if (!resolvedKey) {
    return null;
  }
  return models.find((candidate) => candidate.ref === resolvedKey) || null;
}

export function formatSessionModelOptionLabel(
  model: ModelDescriptor,
  providerLabel: string | undefined,
  connected: boolean,

```

```

): string {
    const normalizedProviderLabel =
        typeof providerLabel === "string" && providerLabel.trim()
        ? providerLabel.trim()
        : model.provider;
    const connectionSuffix = connected ? "" : " (not connected)";
    return `${normalizedProviderLabel} - ${model.label}${connectionSuffix}`;
}

export function getModelThinkingLevels(
    model: ModelDescriptor | null,
): ThinkingLevel[] {
    if (!model) {
        return ["off"];
    }
    if (!model.reasoning) {
        return ["off"];
    }
    if (model.thinking_levels?.length) {
        return model.thinking_levels;
    }
    return STANDARD_THINKING_LEVELS;
}

export function formatThinkingLevelLabel(level: ThinkingLevel): string {
    if (level === "xhigh") {
        return "XHigh";
    }
    return level.charAt(0).toUpperCase() + level.slice(1);
}

export function getSessionModelLabel(
    model: string,
    models: ModelDescriptor[],
): string {
    const matchingModel = getSessionModelOption(model, models);
    if (!matchingModel) {
        return model;
    }
    return matchingModel.label;
}

export function describeSessionModel(
    model: string,
    models: ModelDescriptor[],
): string {

```

```

    const matchingModel = getSessionModelOption(model, models);
    if (!matchingModel) {
        return ``${model}``;
    }
    return `**${matchingModel.label}** (\`${matchingModel.ref}\`);
}

export function availableSessionModelsMarkdown(
    models: ModelDescriptor[],
): string {
    return models
        .map((model) => ` - **${model.label}** (\`${model.ref}\`);
        .join("\n");
}

```

21.12 utils/repo.ts

Repository utilities recognize Git URLs, GitHub shorthand, and local paths before import. The root chunk below tangles back to `frontend/src/utils/repo.ts`.

```

const GITHUB_SHORTHAND_RE =
    /^[A-Za-z0-9](?:[A-Za-z0-9-]{0,38})\/[A-Za-z0-9._-]+(?:\.git)?$/;

export function isGitUrl(s: string): boolean {
    return /^https?:\/\/\./+/.test(s) || s.startsWith("git@");
}

export function isLocalPath(s: string): boolean {
    return s.startsWith("/") || s.startsWith("~") || s.startsWith("./");
}

export function isGithubShorthand(s: string): boolean {
    return GITHUB_SHORTHAND_RE.test(s);
}

export function deriveRepoName(input: string): string {
    const match = input.match(/\/([~\/]+?)(?:\.git)?$/);
    if (match) return match[1];
    const parts = input.replace(/\/+$/, "").split("/");
    return parts[parts.length - 1] || input;
}

```

21.13 utils/turnEvents.ts

Turn-event utilities rebuild assistant responses, tool calls, and stored transcript blocks from incremental stream events. The root chunk below tangles back to

```

frontend/src/utils/turnEvents.ts.

import type { SSEEvent } from "../api/client";

export type TurnStatus = "completed" | "cancelled" | "failed";

export interface TurnBlock {
  id: string;
  type: "text" | "thinking" | "tool_use" | "error" | "status";
  text?: string;
  severity?: "info" | "warning" | "error";
  toolName?: string;
  toolInput?: unknown;
  toolId?: string;
  toolOutput?: string;
  toolError?: boolean;
}

export interface TurnEventApplyResult {
  changed: boolean;
  status?: TurnStatus;
}

const STORED_TURN_SCHEMA = "yinshi.assistant_turn.v1";

function appendOrCreate(
  blocks: TurnBlock[],
  type: "text" | "thinking",
  text: string,
  createBlockId: () => string,
): boolean {
  if (!text) {
    return false;
  }
  const last = blocks[blocks.length - 1];
  if (last && last.type === type) {
    last.text = (last.text || "") + text;
  } else {
    blocks.push({ id: createBlockId(), type, text });
  }
  return true;
}

function stringifyOutput(content: unknown): string {
  if (typeof content === "string") {
    return content;
  }

```

```

    }
    if (content === null || content === undefined) {
        return "";
    }
    return JSON.stringify(content, null, 2);
}

function parseToolInput(block: TurnBlock): void {
    if (block.type !== "tool_use") {
        return;
    }
    if (typeof block.toolInput !== "string") {
        return;
    }
    try {
        block.toolInput = JSON.parse(block.toolInput);
    } catch {
        // Partial tool-call JSON is still useful to show as raw text.
    }
}

function applyAssistantEvent(
    blocks: TurnBlock[],
    event: Extract<SSEvent, { type: "assistant" }>,
    createBlockId: () => string,
): boolean {
    let changed = false;
    for (const block of event.message?.content ?? []) {
        if (block.type === "text") {
            changed = appendOrCreate(blocks, "text", block.text || "", createBlockId) || changed;
        } else if (block.type === "thinking") {
            changed = appendOrCreate(
                blocks,
                "thinking",
                block.thinking || block.text || "",
                createBlockId,
            ) || changed;
        } else if (block.type === "tool_use") {
            blocks.push({
                id: block.id || createBlockId(),
                type: "tool_use",
                toolName: block.name || "unknown",
                toolInput: block.input,
                toolId: block.id,
            });
            changed = true;
        }
    }
}

```

```

    }
  }
  return changed;
}

function applyContentBlockStart(
  blocks: TurnBlock[],
  event: Extract<SSEEvent, { type: "content_block_start" }>,
  createBlockId: () => string,
): boolean {
  const contentBlock = event.content_block;
  if (contentBlock.type === "tool_use") {
    blocks.push({
      id: contentBlock.id || createBlockId(),
      type: "tool_use",
      toolName: contentBlock.name || "unknown",
      toolInput: contentBlock.input,
      toolId: contentBlock.id,
    });
    return true;
  }
  if (contentBlock.type === "thinking") {
    blocks.push({
      id: createBlockId(),
      type: "thinking",
      text: contentBlock.thinking || "",
    });
    return true;
  }
  if (contentBlock.type === "text") {
    blocks.push({
      id: createBlockId(),
      type: "text",
      text: contentBlock.text || "",
    });
    return true;
  }
  return false;
}

function applyContentBlockDelta(
  blocks: TurnBlock[],
  event: Extract<SSEEvent, { type: "content_block_delta" }>,
  createBlockId: () => string,
): boolean {
  const delta = event.delta;

```



```

    if (delta.type === "text_delta") {
      return appendOrCreate(blocks, "text", delta.text || "", createBlockId);
    }
    if (delta.type === "thinking_delta") {
      return appendOrCreate(blocks, "thinking", delta.thinking || delta.text || "", createBlockId);
    }
    if (delta.type === "input_json_delta") {
      const last = blocks[blocks.length - 1];
      if (last && last.type === "tool_use") {
        const partial = delta.partial_json || "";
        last.toolInput = ((last.toolInput as string) || "") + partial;
        return true;
      }
    }
    return false;
  }

function applyToolResult(
  blocks: TurnBlock[],
  event: Extract<SSEEvent, { type: "tool_result" }>,
): boolean {
  const matching = blocks.find(
    (block) => block.type === "tool_use" && block.toolId === event.tool_use_id,
  );
  if (!matching) {
    return false;
  }
  matching.toolOutput = stringifyOutput(event.content);
  matching.toolError = event.is_error;
  return true;
}

export function applyTurnEventToBlocks(
  blocks: TurnBlock[],
  event: SSEEvent,
  createBlockId: () => string,
): TurnEventApplyResult {
  switch (event.type) {
    case "assistant":
      return { changed: applyAssistantEvent(blocks, event, createBlockId) };
    case "tool_use":
      blocks.push({
        id: event.id || createBlockId(),
        type: "tool_use",
        toolName: event.name || event.tool_name || "unknown",
        toolInput: event.input,
      });
  }
}

```

```

        toolId: event.id,
    });
    return { changed: true };
case "content_block_start":
    return { changed: applyContentBlockStart(blocks, event, createBlockId) };
case "content_block_delta":
    return { changed: applyContentBlockDelta(blocks, event, createBlockId) };
case "content_block_stop": {
    const last = blocks[blocks.length - 1];
    if (last) {
        parseToolInput(last);
    }
    return { changed: true };
}
case "tool_result":
    return { changed: applyToolResult(blocks, event) };
case "result":
case "message_stop":
    return { changed: false, status: "completed" };
case "cancelled":
    return { changed: false, status: "cancelled" };
case "status":
    blocks.push({
        id: createBlockId(),
        type: "status",
        text: event.message || event.status,
        severity: event.severity || "info",
    });
    return { changed: true };
case "error":
    blocks.push({ id: createBlockId(), type: "error", text: event.error });
    return { changed: true, status: "failed" };
case "message_start":
case "message_delta":
    return { changed: false };
default:
    return { changed: false };
}
}

export function blocksToContent(blocks: TurnBlock[]): string {
    return blocks
        .filter((block) => block.type === "text")
        .map((block) => block.text || "")
        .join("");
}

```

```

function normalizeStoredEvent(rawEvent: unknown): SSEEvent | null {
  if (!rawEvent || typeof rawEvent !== "object") {
    return null;
  }
  const record = rawEvent as Record<string, unknown>;
  if (record.type === "message") {
    return normalizeStoredEvent(record.data);
  }
  if (record.type === "tool_use") {
    return {
      type: "tool_use",
      name: String(record.toolName || record.name || record.tool_name || "unknown"),
      id: typeof record.id === "string" ? record.id : undefined,
      input: record.toolInput ?? record.input,
    };
  }
  return record as SSEEvent;
}

function extractStoredEvents(fullMessage: string): SSEEvent[] {
  const parsed = JSON.parse(fullMessage) as unknown;
  if (!parsed || typeof parsed !== "object") {
    return [];
  }
  const record = parsed as Record<string, unknown>;
  if (record.schema === STORED_TURN_SCHEMA && Array.isArray(record.events)) {
    return record.events.flatMap((event) => {
      const normalizedEvent = normalizeStoredEvent(event);
      return normalizedEvent ? [normalizedEvent] : [];
    });
  }
  const normalizedEvent = normalizeStoredEvent(record);
  return normalizedEvent ? [normalizedEvent] : [];
}

export function parseStoredTurnBlocks(
  fullMessage: string | null | undefined,
  createBlockId: () => string,
): TurnBlock[] {
  if (!fullMessage) {
    return [];
  }

  let events: SSEEvent[];
  try {

```

```

    events = extractStoredEvents(fullMessage);
  } catch {
    return [];
  }

  const blocks: TurnBlock[] = [];
  for (const event of events) {
    applyTurnEventToBlocks(blocks, event, createBlockId);
  }
  return blocks;
}

```

22 Frontend Chat and Workspace UI

The workbench presents streamed agent output and workspace state side by side. Chat components focus on turns and tool output, while the inspector focuses on files, diffs, edits, and terminal reconnect behavior.

22.1 components/ChatView.tsx

The chat view manages prompt input, slash-command search, run controls, and message rendering. The root chunk below tangles back to frontend/src/components/ChatView.tsx.

```

import { useCallback, useEffect, useLayoutEffect, useMemo, useRef, useState } from "react";
import type { ChatMessage } from "../hooks/useAgentStream";
import AssistantTurn from "../AssistantTurn";
import MessageBubble from "../MessageBubble";
import SlashCommandMenu, { type SlashCommand } from "../SlashCommandMenu";
import StreamingDots from "../StreamingDots";

// Locate the slash-command token under the caret. A valid token is a "/" that
// is at the very start of the input OR immediately preceded by whitespace,
// followed by non-whitespace non-"/" characters up to the caret. Returns the
// starting index of the "/" and the partial token after it.
function computeSlashRegion(
  input: string,
  caret: number,
): { start: number; token: string } | null {
  let scanIndex = caret;
  while (
    scanIndex > 0 &&
    !/\s/.test(input[scanIndex - 1]) &&
    input[scanIndex - 1] !== "/"
  ) {
    scanIndex--;
  }

```

```

    }
    if (scanIndex === 0 || input[scanIndex - 1] !== "/") {
      return null;
    }
    const slashIndex = scanIndex - 1;
    // A "/" must start a fresh token to qualify -- reject things like "a/b".
    if (slashIndex > 0 && !/\s/.test(input[slashIndex - 1])) {
      return null;
    }
    return { start: slashIndex, token: input.slice(scanIndex, caret) };
  }

const SLASH_COMMANDS: SlashCommand[] = [
  { name: "help", description: "List available commands", source: "builtin" },
  { name: "model", description: "Show or change the AI model", source: "builtin" },
  { name: "tree", description: "Show workspace file tree", source: "builtin" },
  { name: "export", description: "Download chat as markdown", source: "builtin" },
  { name: "clear", description: "Clear chat display", source: "builtin" },
];

const BUILTIN_COMMAND_NAMES = new Set(SLASH_COMMANDS.map((c) => c.name));

// The textarea grows with content up to this height (in px); beyond that it scrolls.
const INPUT_HEIGHT_MAX = 120;

function resizeInput(element: HTMLTextAreaElement | null): void {
  if (element === null) return;
  element.style.height = "auto";
  element.style.height = Math.min(element.scrollHeight, INPUT_HEIGHT_MAX) + "px";
}

interface ChatViewProps {
  messages: ChatMessage[];
  streaming: boolean;
  onSend: (prompt: string) => void | Promise<void>;
  onCancel: () => void | Promise<void>;
  onCommand?: (name: string, args: string) => void | Promise<void>;
  inputDisabledReason?: string | null;
  // Pi-provided slash commands (skills, prompts, extension commands). These
  // are inserted into the input on click so the user can supply arguments,
  // then submitted as a regular prompt that pi handles internally.
  piCommands?: SlashCommand[];
}

export default function ChatView({
  messages,

```

```

streaming,
onSend,
onCancel,
onCommand,
inputDisabledReason,
piCommands,
}: ChatViewProps) {
  const [input, setInput] = useState("");
  const [caret, setCaret] = useState(0);
  const [showMenu, setShowMenu] = useState(false);
  const [menuIndex, setMenuIndex] = useState(0);
  const scrollRef = useRef<HTMLDivElement>(null);
  const inputRef = useRef<HTMLTextAreaElement>(null);
  const isNearBottom = useRef(true);
  // When selectCommand mutates the input, it also needs to move the caret to
  // land right after the inserted command. React resets selection when the
  // textarea's value changes, so we record the desired caret here and apply
  // it in a useLayoutEffect after the value prop has been committed.
  const pendingCaretRef = useRef<number | null>(null);

  useLayoutEffect(() => {
    if (pendingCaretRef.current === null || inputRef.current === null) {
      return;
    }
    const desiredCaret = pendingCaretRef.current;
    pendingCaretRef.current = null;
    inputRef.current.focus();
    inputRef.current.setSelectionRange(desiredCaret, desiredCaret);
    setCaret(desiredCaret);
  }, [input]);

  const syncCaretFromInput = useCallback(() => {
    const pos = inputRef.current?.selectionStart ?? input.length;
    setCaret(pos);
  }, [input.length]);

  const handleScroll = useCallback(() => {
    const el = scrollRef.current;
    if (el) {
      isNearBottom.current = el.scrollTop + el.clientHeight >= el.scrollHeight - 100;
    }
  }, []);

  // Auto-scroll to bottom only when near bottom
  useEffect(() => {
    if (isNearBottom.current) {

```

```

        requestAnimationFrame(() => {
            const el = scrollRef.current;
            if (el) el.scrollTop = el.scrollHeight;
        });
    }
}, [messages]);

// Full palette = Yinshi UI commands + any imported pi skills/prompts/extensions.
const allCommands = useMemo<SlashCommand[]>(
    () => [...SLASH_COMMANDS, ...(piCommands ?? [])],
    [piCommands],
);

// Slash palette fires on a "/" token wherever the caret sits, not just at
// the very start of the input. That lets users compose prompts that reference
// multiple commands ("run /skill:foo then /skill:bar on file.py").
const slashRegion = useMemo(
    () => computeSlashRegion(input, caret),
    [input, caret],
);
const slashFilter = slashRegion?.token ?? null;
const filteredCommands =
    slashFilter !== null
        ? allCommands.filter((c) =>
            c.name.startsWith(slashFilter.toLowerCase()),
        )
        : [];
const menuVisible = slashFilter !== null && filteredCommands.length > 0;

const selectCommand = useCallback(
    (command: SlashCommand) => {
        setShowMenu(false);
        setMenuIndex(0);
        const region = computeSlashRegion(input, caret);
        if (region === null) {
            return;
        }
    }

    // A builtin dispatched when the slash token is the entire input keeps
    // the existing "click to execute" behavior. Anywhere else (mid-prompt,
    // or a pi command) we insert text so multiple commands can be chained.
    const selectionCoversEntireInput =
        region.start === 0 && caret === input.length;
    if (command.source === "builtin" && selectionCoversEntireInput) {
        setInput("");
        pendingCaretRef.current = 0;
    }

```

```

        resizeInput(inputRef.current);
        onCommand?.(command.name, "");
        return;
    }

    // Only append a trailing space when the caret isn't already followed by
    // whitespace -- avoids "/name more" double-space when inserting into
    // the middle of existing text.
    const nextChar = input.charAt(caret);
    const needsTrailingSpace = nextChar === "" || !/\s/.test(nextChar);
    const replacement = `/${command.name}${needsTrailingSpace ? " " : ""}`;
    const newText =
        input.slice(0, region.start) + replacement + input.slice(caret);
    pendingCaretRef.current = region.start + replacement.length;
    setInput(newText);
},
[input, caret, onCommand],
);

const handleSubmit = useCallback(
    (e?: React.FormEvent) => {
        e?.preventDefault();
        const text = input.trim();
        if (!text || inputDisabledReason) return;

        // Only intercept slash commands whose first token matches a builtin Yinshi
        // UI command. Pi skill / prompt / extension commands pass through to onSend
        // and pi executes them internally via session.prompt(). Builtins win on name
        // collisions with imported pi commands by intent.
        if (text.startsWith("/")) {
            const parts = text.slice(1).split(/\s+/);
            const cmdName = parts[0]?.toLowerCase() ?? "";
            const cmdArgs = parts.slice(1).join(" ");
            if (BUILTIN_COMMAND_NAMES.has(cmdName)) {
                setInput("");
                setCaret(0);
                setShowMenu(false);
                setMenuIndex(0);
                resizeInput(inputRef.current);
                onCommand?.(cmdName, cmdArgs);
                return;
            }
        }
    }
);

void onSend(text);
setInput("");

```



```

        setCaret(0);
        setShowMenu(false);
        resizeInput(inputRef.current);
    },
    [input, inputDisabledReason, streaming, onSend, onCommand],
);

const hasInput = input.trim().length > 0;
const inputDisabled = Boolean(inputDisabledReason);

const handleKeyDown = useCallback(
    (e: React.KeyboardEvent) => {
        if (menuVisible) {
            if (e.key === "ArrowDown") {
                e.preventDefault();
                setMenuIndex((i) => Math.min(i + 1, filteredCommands.length - 1));
                return;
            }
            if (e.key === "ArrowUp") {
                e.preventDefault();
                setMenuIndex((i) => Math.max(i - 1, 0));
                return;
            }
            if (e.key === "Tab" || (e.key === "Enter" && !e.shiftKey)) {
                e.preventDefault();
                const cmd = filteredCommands[menuIndex];
                if (cmd) selectCommand(cmd);
                return;
            }
            if (e.key === "Escape") {
                e.preventDefault();
                setShowMenu(false);
                return;
            }
        }
        if (e.key === "Enter" && !e.shiftKey) {
            e.preventDefault();
            handleSubmit();
        }
    },
    [handleSubmit, menuVisible, filteredCommands, menuIndex, selectCommand],
);

const handleInputChange = useCallback(
    (e: React.ChangeEvent<HTMLTextAreaElement>) => {
        setInput(e.target.value);
    }

```

```

        setCaret(e.target.selectionStart);
        setMenuIndex(0);
        resizeInput(e.target);
    },
    [],
);

return (
    <div className="flex h-full flex-col">
        {/* Messages area */}
        <div
            ref={scrollRef}
            onScroll={handleScroll}
            className="flex-1 overflow-y-auto scrollbar-hide px-3 py-4 space-y-3"
        >
            {messages.length === 0 && (
                <div className="flex h-full items-center justify-center">
                    <p className="text-gray-600 text-sm">
                        Send a message to start coding.
                    </p>
                </div>
            )}

            {messages.map((msg) => {
                if (msg.role === "user") {
                    return (
                        <MessageBubble
                            key={msg.id}
                            role="user"
                            content={msg.content}
                        />
                    );
                }
                if (msg.role === "assistant") {
                    return (
                        <AssistantTurn
                            key={msg.id}
                            blocks={msg.blocks ?? []}
                            streaming={msg.streaming}
                            fallbackContent={
                                (msg.blocks ?? []).length === 0 ? msg.content : undefined
                            }
                        />
                    );
                }
                if (msg.role === "error") {

```

```

        return (
            <MessageBubble
                key={msg.id}
                role="error"
                content={msg.content}
            />
        );
    }
    return null;
  })}

  {streaming &&
    !messages.some((m) => m.role === "assistant" && m.streaming) && (
      <div className="flex items-center gap-2 px-2 py-1">
        <StreamingDots />
      </div>
    )}
</div>

{/* Input bar */}
<div
  className="relative border-t border-gray-800 bg-gray-900 px-3 py-2"
  style={{ paddingBottom: "max(0.5rem, env(safe-area-inset-bottom))" }}
>
  {menuVisible && (
    <SlashCommandMenu
      filter={slashFilter ?? ""}
      commands={allCommands}
      selectedIndex={menuIndex}
      onSelect={selectCommand}
    />
  )}
  {inputDisabledReason && (
    <div className="mb-2 rounded-lg border border-amber-800/50 bg-amber-950/30 px-3 py-2">
      {inputDisabledReason}
    </div>
  )}
  <form onSubmit={handleSubmit} className="flex items-end gap-2">
    <textarea
      ref={inputRef}
      value={input}
      onChange={handleInputChange}
      onKeyDown={handleKeyDown}
      onKeyUp={syncCaretFromInput}
      onClick={syncCaretFromInput}
      onSelect={syncCaretFromInput}
    />
  </form>
</div>

```

```

placeholder={inputDisabledReason || "Describe what to build..."}
disabled={inputDisabled}
rows={1}
className="flex-1 resize-none rounded-xl bg-gray-800 px-4 py-3 text-sm text-gray-500"
style={{ maxHeight: "120px" }}
/>
{streaming && !hasInput ? (
  <button
    type="button"
    onClick={() => {
      void onCancel();
    }}
    className="flex h-11 w-11 items-center justify-center rounded-xl bg-red-500/20"
    aria-label="Cancel"
  >
    <svg
      className="h-5 w-5"
      fill="none"
      viewBox="0 0 24 24"
      strokeWidth={2}
      stroke="currentColor"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        d="M6 18 18 6 6 12 12 18"
      />
    </svg>
  </button>
) : (
  <button
    type="submit"
    disabled={!hasInput || inputDisabled}
    className="flex h-11 w-11 items-center justify-center rounded-xl bg-blue-500"
    aria-label={streaming ? "Steer" : "Send"}
  >
    <svg
      className="h-5 w-5"
      fill="none"
      viewBox="0 0 24 24"
      strokeWidth={2}
      stroke="currentColor"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"

```

```

        d="M6 12 3.269 3.125A59.769 59.769 0 0 1 21.485 12 59.768 59.768 0 0 1 3.269 12"
      />
    </svg>
  </button>
  })
</form>
</div>
</div>
);
}

```

22.2 components/AssistantTurn.tsx

Assistant turns render streamed content blocks, trace blocks, and response markdown. The root chunk below tangles back to `frontend/src/components/AssistantTurn.tsx`.

```

import { memo, useState } from "react";
import ReactMarkdown from "react-markdown";
import remarkGfm from "remark-gfm";
import type { TurnBlock } from "../hooks/useAgentStream";
import StreamingDots from "../StreamingDots";
import ThinkingBlock from "../ThinkingBlock";
import ToolCallBlock, { ToolIcon } from "../ToolCallBlock";

interface AssistantTurnProps {
  blocks: TurnBlock[];
  streaming?: boolean;
  /** Fallback plain text when no blocks (e.g., loaded from history) */
  fallbackContent?: string;
}

function isTraceBlock(block: TurnBlock): boolean {
  return block.type === "tool_use" || block.type === "thinking";
}

function isResponseBlock(block: TurnBlock): boolean {
  return block.type === "text" || block.type === "error" || block.type === "status";
}

function statusClassName(severity: TurnBlock["severity"]): string {
  if (severity === "warning") {
    return "border-amber-800/50 bg-amber-950/30 text-amber-200";
  }
  if (severity === "error") {
    return "border-red-800/50 bg-red-900/30 text-red-300";
  }
}

```

```

    return "border-blue-900/50 bg-blue-950/30 text-blue-200";
  }

function turnSummary(blocks: TurnBlock[]) {
  const tools = blocks.filter((block) => block.type === "tool_use");
  const thinking = blocks.filter((block) => block.type === "thinking");
  const toolNames = [
    ...new Set(
      tools
        .map((tool) => tool.toolName || "")
        .filter((toolName) => toolName.length > 0),
    ),
  ];

  const parts: string[] = [];
  if (tools.length > 0) {
    parts.push(`${tools.length} tool call${tools.length !== 1 ? "s" : ""}`);
  }
  if (thinking.length > 0) {
    parts.push("reasoning");
  }

  return {
    text: parts.join(", ") || "no reasoning or tool use recorded",
    toolNames,
  };
}

function renderResponseBlock(block: TurnBlock) {
  switch (block.type) {
    case "text":
      return (
        <div key={block.id} className="markdown-prose px-2 text-sm text-gray-200">
          <ReactMarkdown remarkPlugins={[remarkGfm]}>
            {block.text || ""}
          </ReactMarkdown>
        </div>
      );
    case "status":
      return (
        <div
          key={block.id}
          className={`mx-2 rounded-lg border px-3 py-2 text-sm ${statusClassName(block.sever
        >
          {block.text}
        </div>
      );

```

```

    );
    case "error":
        return (
            <div
                key={block.id}
                className="mx-2 rounded-lg border border-red-800/50 bg-red-900/30 px-3 py-2 text-s
            >
                <div className="flex items-center gap-2">
                    <svg
                        className="h-4 w-4 shrink-0"
                        fill="none"
                        viewBox="0 0 24 24"
                        strokeWidth={2}
                        stroke="currentColor"
                    >
                        <path
                            strokeLinecap="round"
                            strokeLinejoin="round"
                            d="M12 9v3.75m9-.75a9 9 0 1 1-18 0 9 9 0 0 1 18 0Zm-9 3.75h.008v.008H12v-.008"
                        />
                    </svg>
                    <span>{block.text}</span>
                </div>
            </div>
        );
    default:
        return null;
}
}

function renderTraceBlock(block: TurnBlock) {
    switch (block.type) {
        case "thinking":
            return <ThinkingBlock key={block.id} text={block.text || ""} />;
        case "tool_use":
            return (
                <ToolCallBlock
                    key={block.id}
                    toolName={block.toolName || "unknown"}
                    toolInput={block.toolInput}
                    toolOutput={block.toolOutput}
                    toolError={block.toolError}
                />
            );
        default:
            return null;
    }
}

```

```

    }
  }

const AssistantTurn = memo(function AssistantTurn({
  blocks,
  streaming,
  fallbackContent,
}: AssistantTurnProps) {
  const [traceExpanded, setTraceExpanded] = useState(false);
  const traceBlocks = blocks.filter(isTraceBlock);
  const responseBlocks = blocks.filter(isResponseBlock);
  const hasTraceBlocks = traceBlocks.length > 0;
  const hasResponseContent = responseBlocks.length > 0 || Boolean(fallbackContent);
  const summary = turnSummary(traceBlocks);

  return (
    <div className="animate-message-in space-y-1">
      <button
        type="button"
        aria-expanded={traceExpanded}
        onClick={() => setTraceExpanded((expanded) => !expanded)}
        className="flex max-w-full items-center gap-2 rounded-lg border border-gray-800 bg-gray-100"
      >
        <svg
          className={`h-3 w-3 shrink-0 transition-transform ${traceExpanded ? "rotate-180" : ""}`}
          fill="none"
          viewBox="0 0 24 24"
          strokeWidth={2}
          stroke="currentColor"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="m19.5 8.25-7.5 7.5-7.5-7.5"
          />
        </svg>
        <span className="truncate">
          {traceExpanded ? "Hide" : "Show"} trace: {summary.text}
        </span>
        {summary.toolNames.length > 0 && (
          <span className="flex shrink-0 gap-1 text-gray-600">
            {summary.toolNames.map((name) => (
              <ToolIcon key={name} name={name} />
            ))}
          </span>
        )}
      </div>
    )}
  )}

```



```

    {streaming && (
      <span className="ml-1 shrink-0">
        <StreamingDots size="sm" />
      </span>
    )}
  </button>

  {traceExpanded && (
    <div className="space-y-1">
      {hasTraceBlocks ? (
        traceBlocks.map(renderTraceBlock)
      ) : (
        <div className="mx-2 rounded-lg border border-gray-800/70 bg-gray-900/50 px-3 py-2">
          No reasoning or tool use was recorded for this response.
        </div>
      )}
    </div>
  )}

  {responseBlocks.map(renderResponseBlock)}

  {responseBlocks.length === 0 && fallbackContent && (
    <div className="px-2 py-1">
      <div className="markdown-prose text-sm text-gray-200">
        <ReactMarkdown remarkPlugins={[remarkGfm]}>
          {fallbackContent}
        </ReactMarkdown>
      </div>
    </div>
  )}

  {streaming && !hasResponseContent && (
    <div className="flex items-center gap-1 px-2 py-1">
      <StreamingDots />
    </div>
  )}
</div>
);
});

export default AssistantTurn;

```

22.3 components/MessageBubble.tsx

Message bubbles give user and system messages consistent spacing and color.
 The root chunk below tangles back to `frontend/src/components/MessageBubble.tsx`.

```

import ReactMarkdown from "react-markdown";
import remarkGfm from "remark-gfm";

interface MessageBubbleProps {
  role: string;
  content: string;
  streaming?: boolean;
}

function bubbleStyle(role: string): string {
  if (role === "user") {
    return "bg-blue-500 text-white rounded-br-md";
  }
  if (role === "error") {
    return "bg-red-500/20 text-red-300 rounded-bl-md";
  }
  return "bg-gray-800 text-gray-200 rounded-bl-md";
}

export default function MessageBubble({
  role,
  content,
  streaming,
}: MessageBubbleProps) {
  const isUser = role === "user";

  return (
    <div
      className={`animate-message-in flex ${isUser ? "justify-end" : "justify-start"}`}
    >
      <div
        className={`max-w-[85%] rounded-2xl px-4 py-2.5 text-sm ${bubbleStyle(role)}`}
      >
        {isUser ? (
          <div className="whitespace-pre-wrap break-words leading-relaxed">
            {content}
          </div>
        ) : (
          <div className="markdown-prose break-words">
            <ReactMarkdown remarkPlugins={[remarkGfm]}>
              {content}
            </ReactMarkdown>
          </div>
        )}
        {streaming && (
          <span className="inline-block h-3 w-0.5 animate-pulse bg-blue-400 ml-0.5 align-mid"

```

```

    })
  </div>
</div>
);
}

```

22.4 components/ThinkingBlock.tsx

Thinking blocks hide or reveal model reasoning traces without losing stream context. The root chunk below tangles back to `frontend/src/components/ThinkingBlock.tsx`.

```

import { useState } from "react";

interface ThinkingBlockProps {
  text: string;
}

export default function ThinkingBlock({ text }: ThinkingBlockProps) {
  const [expanded, setExpanded] = useState(false);

  // Show a short preview when collapsed
  const preview =
    text.length > 80 ? text.slice(0, 80).trimEnd() + "..." : text;

  return (
    <div className="mx-2">
      <button
        onClick={() => setExpanded(!expanded)}
        className="flex w-full items-start gap-2 rounded-lg bg-gray-800/40 px-3 py-2 text-1
      >
        <svg
          className="mt-0.5 h-4 w-4 shrink-0 text-purple-400"
          fill="none"
          viewBox="0 0 24 24"
          strokeWidth={1.5}
          stroke="currentColor"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="M9.813 15.904 9 18.751-.813-2.846a4.5 4.5 0 0 0-3.09-3.09L2.25 12.846-.813
          />
        </svg>
        <div className="flex-1 min-w-0">
          <div className="flex items-center gap-2">
            <span className="text-xs font-medium text-purple-400">

```

```

        Thinking
      </span>
      {!expanded && (
        <span className="truncate text-xs text-gray-600">
          {preview}
        </span>
      )}
    </div>
  </div>
  <svg
    className={`mt-0.5 h-3 w-3 shrink-0 text-gray-600 transition-transform ${expanded
    fill="none"
    viewBox="0 0 24 24"
    strokeWidth={2}
    stroke="currentColor"
  >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      d="m19.5 8.25-7.5 7.5-7.5-7.5"
    />
  </svg>
</button>

  {expanded && (
    <div className="mt-1 rounded-lg bg-gray-800/30 px-3 py-2">
      <pre className="whitespace-pre-wrap break-words text-xs text-gray-500 font-mono 1e
        {text}
      </pre>
    </div>
  )}
</div>
);
}

```

22.5 components/ToolCallBlock.tsx

Tool-call blocks summarize command execution, file edits, diffs, and expanded tool output. The root chunk below tangles back to `frontend/src/components/ToolCallBlock.tsx`.

```
import { memo, useState } from "react";
```

```
interface ToolCallBlockProps {
  toolName: string;
  toolInput: unknown;
  toolOutput?: string;

```

```

    toolError?: boolean;
}

/** Extract a human-readable summary from tool input */
function toolSummary(
  name: string,
  input: unknown,
): { label: string; added?: number; removed?: number } {
  const inp = input as Record<string, unknown> | null;
  if (!inp) return { label: "" };

  switch (name) {
    case "Read": {
      const path = (inp.file_path || inp.path || "") as string;
      return { label: shortPath(path) };
    }
    case "Edit":
    case "MultiEdit": {
      const path = (inp.file_path || "") as string;
      const oldStr = ((inp.old_string || "") as string).split("\n").length;
      const newStr = ((inp.new_string || "") as string).split("\n").length;
      return {
        label: shortPath(path),
        added: Math.max(0, newStr - 1),
        removed: Math.max(0, oldStr - 1),
      };
    }
    case "Write": {
      const path = (inp.file_path || "") as string;
      const lines = ((inp.content || "") as string).split("\n").length;
      return { label: shortPath(path), added: lines };
    }
    case "Bash": {
      const cmd = (inp.command || "") as string;
      const preview = cmd.length > 60 ? cmd.slice(0, 60) + "..." : cmd;
      return { label: preview };
    }
    case "Glob":
    case "Grep": {
      return { label: (inp.pattern || "") as string };
    }
    default: {
      const path = (inp.file_path || inp.path || inp.url || "") as string;
      return { label: path ? shortPath(path) : "" };
    }
  }
}

```

```

}

function shortPath(path: string): string {
  if (!path) return "";
  const parts = path.split("/");
  if (parts.length <= 3) return path;
  return parts.slice(-3).join("/");
}

/** Icon for tool type */
export function ToolIcon({ name }: { name: string }) {
  switch (name) {
    case "Read":
      return (
        <svg
          className="h-3.5 w-3.5"
          fill="none"
          viewBox="0 0 24 24"
          strokeWidth={2}
          stroke="currentColor"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="M19.5 14.25v-2.625a3.375 3.375 0 0 0-3.375-3.375h-1.5A1.125 1.125 0 0 1 13.5
          />
        </svg>
      );
    case "Edit":
    case "MultiEdit":
      return (
        <svg
          className="h-3.5 w-3.5"
          fill="none"
          viewBox="0 0 24 24"
          strokeWidth={2}
          stroke="currentColor"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="m16.862 4.487 1.687-1.688a1.875 1.875 0 1 1 2.652 2.652L10.582 16.07a4.5 4.5
          />
        </svg>
      );
    case "Write":

```

```

return (
  <svg
    className="h-3.5 w-3.5"
    fill="none"
    viewBox="0 0 24 24"
    strokeWidth={2}
    stroke="currentColor"
  >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      d="M19.5 14.25v-2.625a3.375 3.375 0 0 0-3.375-3.375h-1.5A1.125 1.125 0 0 1 13.5
    />
  </svg>
);
case "Bash":
  return (
    <svg
      className="h-3.5 w-3.5"
      fill="none"
      viewBox="0 0 24 24"
      strokeWidth={2}
      stroke="currentColor"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        d="m6.75 7.5 3 2.25-3 2.25m4.5 0h3m-9 8.25h13.5A2.25 2.25 0 0 0 21 18V6a2.25 2.2
      />
    </svg>
  );
case "Glob":
case "Grep":
case "Search":
  return (
    <svg
      className="h-3.5 w-3.5"
      fill="none"
      viewBox="0 0 24 24"
      strokeWidth={2}
      stroke="currentColor"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        d="m21 21-5.197-5.197m0 0A7.5 7.5 0 1 0 5.196 5.196a7.5 7.5 0 0 0 10.607 10.607

```

```

        />
      </svg>
    );
    case "Agent":
      return (
        <svg
          className="h-3.5 w-3.5"
          fill="none"
          viewBox="0 0 24 24"
          strokeWidth={2}
          stroke="currentColor"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="M15.75 6a3.75 3.75 0 1 1-7.5 0 3.75 3.75 0 0 1 7.5 0ZM4.501 20.118a7.5 7.5 0
          />
        </svg>
      );
    default:
      return (
        <svg
          className="h-3.5 w-3.5"
          fill="none"
          viewBox="0 0 24 24"
          strokeWidth={2}
          stroke="currentColor"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="M11.42 15.17 17.25 21A2.652 2.652 0 0 0 21 17.25l-5.877-5.877M11.42 15.17l2.4
          />
        </svg>
      );
  }
}

```

```

/** Render a simple inline diff for Edit tool */
function InlineDiff({ input }: { input: Record<string, unknown> }) {
  const oldStr = (input.old_string || "") as string;
  const newStr = (input.new_string || "") as string;

  if (!oldStr && !newStr) return null;

  return (

```



```

<div className="rounded-md bg-gray-900/80 text-xs font-mono overflow-x-auto">
  {oldStr && (
    <div className="border-b border-gray-800/50">
      {oldStr.split("\n").map((line, i) => (
        <div key={`old-${i}`} className="flex">
          <span className="w-8 shrink-0 select-none text-right pr-2 text-red-700/70">
            -
          </span>
          <span className="flex-1 bg-red-900/20 text-red-300/80 px-2 whitespace-pre">
            {line}
          </span>
        </div>
      ))}
    </div>
  )}
  {newStr && (
    <div>
      {newStr.split("\n").map((line, i) => (
        <div key={`new-${i}`} className="flex">
          <span className="w-8 shrink-0 select-none text-right pr-2 text-green-700/70">
            +
          </span>
          <span className="flex-1 bg-green-900/20 text-green-300/80 px-2 whitespace-pre">
            {line}
          </span>
        </div>
      ))}
    </div>
  )}
</div>
);
}

/** Render Bash command and output */
function BashContent({
  input,
  output,
}): {
  input: Record<string, unknown>;
  output?: string;
} {
  const cmd = (input.command || "") as string;
  return (
    <div className="space-y-1">
      <div className="rounded-md bg-gray-900/80 px-3 py-2 text-xs font-mono text-gray-300 ov
        $ {cmd}

```

```

    </div>
    {output && (
      <div className="rounded-md bg-gray-900/60 px-3 py-2 text-xs font-mono text-gray-500"
        {output.length > 2000 ? output.slice(0, 2000) + "\n..." : output}
      </div>
    )}
  </div>
);
}

function ExpandedContent({
  toolName,
  toolInput,
  toolOutput,
  toolError,
}: ToolCallBlockProps) {
  const inp = toolInput as Record<string, unknown> | null;

  if ((toolName === "Edit" || toolName === "MultiEdit") && inp) {
    return (
      <>
        <InlineDiff input={inp} />
        {toolOutput && (
          <div
            className={`rounded-md px-3 py-2 text-xs font-mono ${
              toolError
                ? "bg-red-900/30 text-red-300"
                : "bg-gray-900/40 text-gray-600"
            }`}
          >
            {toolOutput.length > 500
              ? toolOutput.slice(0, 500) + "..."
              : toolOutput}
          </div>
        )}
      </>
    );
  }

  if (toolName === "Bash" && inp) {
    return <BashContent input={inp} output={toolOutput} />;
  }

  return (
    <>
      {inp && (

```



```

>
  {toolName}
</span>
{summary.label && (
  <span className="truncate text-xs font-mono text-gray-500">
    {summary.label}
  </span>
)}
{(summary.added !== undefined || summary.removed !== undefined) && (
  <span className="shrink-0 text-xs font-mono">
    {summary.added !== undefined && (
      <span className="text-green-500">+{summary.added}</span>
    )}
    {summary.removed !== undefined && summary.removed > 0 && (
      <span className="text-red-500 ml-1">--{summary.removed}</span>
    )}
  </span>
)}
)}
{toolError && (
  <span className="shrink-0 flex items-center gap-1 text-xs text-red-400">
    <svg
      className="h-3 w-3"
      fill="none"
      viewBox="0 0 24 24"
      strokeWidth={2}
      stroke="currentColor"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        d="M12 9v3.75m9-.75a9 9 0 1 1-18 0 9 9 0 0 1 18 0Zm-9 3.75h.008v.008H12v-.008"
      />
    </svg>
    Error
  </span>
)}
<svg
  className={`ml-auto h-3 w-3 shrink-0 text-gray-600 transition-transform ${expanded ? 'scale-110' : 'scale-100'}`}
  fill="none"
  viewBox="0 0 24 24"
  strokeWidth={2}
  stroke="currentColor"
>
  <path
    strokeLinecap="round"
    strokeLinejoin="round"

```

```

        d="m19.5 8.25-7.5 7.5-7.5-7.5"
      />
    </svg>
  </button>

  {/* Expanded content */}
  {expanded && (
    <div className="mt-1 space-y-1">
      <ExpandedContent
        toolName={toolName}
        toolInput={toolInput}
        toolOutput={toolOutput}
        toolError={toolError}
      />
    </div>
  )}
</div>
);
});

export default ToolCallBlock;

```

22.6 components/StreamingDots.tsx

Streaming dots provide the small in-progress affordance used while the agent is still emitting events. The root chunk below tangles back to frontend/src/components/StreamingDots.tsx.

```

import { memo } from "react";

interface StreamingDotsProps {
  size?: "sm" | "md";
}

const StreamingDots = memo(function StreamingDots({ size = "md" }: StreamingDotsProps) {
  const s = size === "sm" ? "h-1 w-1" : "h-1.5 w-1.5";
  return (
    <span className="flex gap-0.5">
      <span className={` ${s} animate-pulse rounded-full bg-blue-400`} />
      <span className={` ${s} animate-pulse rounded-full bg-blue-400 [animation-delay:0.2s]`} />
      <span className={` ${s} animate-pulse rounded-full bg-blue-400 [animation-delay:0.4s]`} />
    </span>
  );
});

export default StreamingDots;

```

22.7 components/SlashCommandMenu.tsx

The slash-command menu merges built-in commands with imported Pi commands and supports keyboard selection. The root chunk below tangles back to `frontend/src/components/SlashCommandMenu.tsx`.

```
import { useEffect, useRef } from "react";

export type SlashCommandSource = "builtin" | "pi";

export interface SlashCommand {
  name: string;
  description: string;
  // "builtin" dispatches to a Yinshi UI handler on click; "pi" inserts the
  // command into the input so the user can supply arguments before submit.
  source: SlashCommandSource;
}

interface SlashCommandMenuProps {
  filter: string;
  commands: SlashCommand[];
  selectedIndex: number;
  onSelect: (command: SlashCommand) => void;
}

export default function SlashCommandMenu({
  filter,
  commands,
  selectedIndex,
  onSelect,
}: SlashCommandMenuProps) {
  const selectedRef = useRef<HTMLDivElement>(null);

  const filtered = commands.filter((cmd) =>
    cmd.name.startsWith(filter.toLowerCase()),
  );

  // When arrow-key navigation moves the highlight past the visible fold,
  // keep the selected row on-screen by scrolling the container. Using
  // block:"nearest" avoids yanking the whole list on small moves. The
  // existence check covers jsdom, which does not implement scrollIntoView.
  useEffect(() => {
    const selected = selectedRef.current;
    if (selected === null || typeof selected.scrollIntoView !== "function") {
      return;
    }
  })
```

```

    selected.scrollToView({ block: "nearest" });
  }, [selectedIndex, filtered.length]);

  if (filtered.length === 0) return null;

  const piCount = filtered.filter((cmd) => cmd.source === "pi").length;
  const builtinCount = filtered.length - piCount;

  return (
    <div
      role="listbox"
      // max-h-80 (20rem = 320px, ~8 items) with overflow scroll gives users a
      // clear signal that the list continues. Smaller caps (max-h-48) hid the
      // pi skills below the fold and made 130+ entries invisible without a
      // discoverable scroll affordance.
      className="absolute bottom-full left-0 right-0 mb-1 max-h-80 overflow-y-auto rounded-2xl"
    >
      <div className="sticky top-0 flex items-center justify-between border-b border-gray-800"
        >
        <span>
          {filtered.length} command{filtered.length === 1 ? "" : "s"}
          {piCount > 0 && builtinCount > 0 && (
            <span className="ml-1 text-gray-600">
              ({builtinCount} builtin, {piCount} from Pi config)
            </span>
          )}
        </span>
        <span className="font-mono text-gray-600">
          ⚡; navigate ⋅; tab to pick
        </span>
      </div>
      {filtered.map((cmd, i) => (
        <div
          key={`${cmd.source}:${cmd.name}`}
          ref={i === selectedIndex ? selectedRef : null}
          role="option"
          aria-selected={i === selectedIndex}
          onMouseDown={(e) => {
            e.preventDefault();
            onSelect(cmd);
          }}
          className={`flex cursor-pointer items-center gap-3 px-4 py-2 text-sm ${
            i === selectedIndex
              ? "bg-gray-800 text-gray-100"
              : "text-gray-300 hover:bg-gray-800/50"
          }`}
        >

```

```

        <span
          className={`font-mono font-medium ${
            cmd.source === "builtin" ? "text-blue-500" : "text-emerald-400"
          }`}
        >
          /{cmd.name}
        </span>
        <span className="truncate text-gray-500">{cmd.description}</span>
      </div>
    )})
  </div>
);
}

```

22.8 components/Sidebar.tsx

The sidebar owns repository import, workspace selection, theme switching, and responsive navigation state. The root chunk below tangles back to `frontend/src/components/Sidebar.tsx`.

```

import { useEffect, useMemo, useState } from "react";
import { useLocation, useNavigate, useParams } from "react-router-dom";
import {
  ApiError,
  api,
  type CloudRunner,
  type GitHubInstallation,
  type Repo,
  type SessionInfo,
  type Workspace,
} from "../api/client";
import { useAuth } from "../hooks/useAuth";
import { useTheme } from "../hooks/useTheme";
import { DEFAULT_SESSION_MODEL } from "../models/sessionModels";
import { listRunnerRepositories } from "../runner/repositories";
import { resolveRuntimeRef } from "../runtime/resolveRuntime";
import {
  runtimeResourceId,
  type UnresolvedRuntimeRef,
} from "../runtime/runtimeRef";
import {
  createRuntimeTransport,
  type RuntimeRef,
  type RuntimeTransport,
} from "../runtime/runtimeTransport";
import {

```



```

    deriveRepoName,
    isGithubShorthand,
    isGitUrl,
    isLocalPath,
} from "../utils/repo";

interface LocatedRepo {
    readonly repository: Repo;
    readonly runtime: RuntimeRef;
}

const COLORS = [
    "bg-[#c23b22]",
    "bg-[#8c6d3f]",
    "bg-[#5a7247]",
    "bg-[#7a5230]",
    "bg-[#985a4a]",
    "bg-[#4a6b5a]",
    "bg-[#6b5040]",
    "bg-[#8a6848]",
];

function runtimeIdentity(runtime: RuntimeRef): string {
    if (runtime.location === "byoc") {
        return `byoc:${runtime.runnerId}`;
    }
    return runtime.location;
}

function runtimeLabel(runtime: RuntimeRef): string {
    if (runtime.location === "byoc") return "BYOC";
    if (runtime.location === "local") return "Local";
    return "Hosted";
}

function repoColor(name: string): string {
    let hash = 0;
    for (const ch of name) hash = (hash * 31 + ch.charCodeAt(0)) | 0;
    return COLORS[Math.abs(hash) % COLORS.length];
}

function statusDotClass(hasRunning: boolean, workspaceState: string): string {
    if (hasRunning) return "bg-[#d4543d] animate-pulse";
    if (workspaceState === "ready") return "bg-[#5a7247]";
    return "bg-[#b8963e]";
}

```

```

const PlusIcon = (
  <svg
    className="h-4 w-4"
    fill="none"
    viewBox="0 0 24 24"
    strokeWidth={2}
    stroke="currentColor"
  >
    <path
      strokeLinecap="round"
      strokeLinejoin="round"
      d="M12 4.5v15m7.5-7.5h-15"
    />
  </svg>
);

type SidebarNotice = {
  message: string;
  tone: "error" | "success";
};

type ImportAction = {
  label: string;
  href: string;
  external: boolean;
};

function githubNoticeFromSearch(search: string): SidebarNotice | null {
  const params = new URLSearchParams(search);
  if (params.get("github_connected") === "1") {
    return { message: "GitHub connected.", tone: "success" };
  }

  const errorCode = params.get("github_connect_error");
  if (errorCode === "not_granted") {
    return {
      message: "GitHub access was not granted for that installation.",
      tone: "error",
    };
  }
  if (errorCode === "invalid_state") {
    return {
      message: "GitHub connect session expired. Try again.",
      tone: "error",
    };
  }
}

```

```

    }
    if (errorCode === "missing_installation") {
        return {
            message: "GitHub did not return an installation for this request.",
            tone: "error",
        };
    }
    if (errorCode === "install_failed") {
        return {
            message: "GitHub installation could not be completed.",
            tone: "error",
        };
    }
    return null;
}

function ThemeIcon({ theme }: { theme: string }) {
    if (theme === "light") {
        return (
            <svg
                className="h-4 w-4"
                fill="none"
                viewBox="0 0 24 24"
                strokeWidth={1.5}
                stroke="currentColor"
            >
                <path
                    strokeLinecap="round"
                    strokeLinejoin="round"
                    d="M21.752 15.002A9.72 9.72 0 0 1 18 15.75c-5.385 0-9.75-4.365-9.75-9.75 0-1.332"
                />
            </svg>
        );
    }
    return (
        <svg
            className="h-4 w-4"
            fill="none"
            viewBox="0 0 24 24"
            strokeWidth={1.5}
            stroke="currentColor"
        >
            <path
                strokeLinecap="round"
                strokeLinejoin="round"
                d="M12 3v2.25m6.364 3.86-1.591 1.591M21 12h-2.25m-.386 6.364-1.591-1.591M12 18.75V21"
            />
        </svg>
    );
}

```

```

        />
      </svg>
    );
  }

export default function Sidebar({ onNavigate }: { onNavigate?: () => void }) {
  const { id: activeSessionId } = useParams<{ id: string }>();
  const location = useLocation();
  const navigate = useNavigate();
  const { status, email, logout } = useAuth();
  const { theme, toggle: toggleTheme } = useTheme();
  const requestedPrimaryRuntime = useMemo<UnresolvedRuntimeRef>(
    () =>
      window.yinshiDesktop ? { location: "local" } : { location: "hosted" },
    [],
  );
  const [primaryRuntime, setPrimaryRuntime] = useState<RuntimeRef | null>(null);
  const [repos, setRepos] = useState<LocatedRepo[]>([]);
  const [availableRuntimes, setAvailableRuntimes] = useState<RuntimeRef[]>([]);
  const [githubInstallations, setGithubInstallations] = useState<
    GitHubInstallation[]
  >([]);
  const [githubNotice, setGithubNotice] = useState<SidebarNotice | null>(null);
  const [loading, setLoading] = useState(true);
  const [repoLoadError, setRepoLoadError] = useState<string | null>(null);
  const [locationErrors, setLocationErrors] = useState<string[]>([]);
  const [showImport, setShowImport] = useState(false);

  async function loadRepos() {
    setLoading(true);
    setRepoLoadError(null);
    setLocationErrors([]);
    try {
      const resolvedPrimary = await resolveRuntimeRef(requestedPrimaryRuntime);
      const primaryTransport = createRuntimeTransport(resolvedPrimary);
      const data = await primaryTransport.get<Repo[]>("/api/repos").finally(() => {
        primaryTransport.close();
      });
      setPrimaryRuntime(resolvedPrimary);
      const locatedRepositories: LocatedRepo[] = data.map((repository) => ({
        repository,
        runtime: resolvedPrimary,
      }));
      const unavailableLocations: string[] = [];
      const discoveredRuntimes: RuntimeRef[] = [resolvedPrimary];
      if (window.yinshiDesktop !== undefined) {

```

```

try {
  const hostedTransport = createRuntimeTransport({
    location: "hosted",
  });
  const hostedRepositories = await hostedTransport
    .get<Repo[]>("/api/repos")
    .finally(() => {
      hostedTransport.close();
    });
  discoveredRuntimes.push({ location: "hosted" });
  locatedRepositories.push(
    ...hostedRepositories.map((repository) => ({
      repository,
      runtime: { location: "hosted" as const },
    })),
  );
} catch {
  unavailableLocations.push("Hosted repositories are unavailable.");
}
}
try {
  const runner = await api.get<CloudRunner | null>(
    "/api/settings/runner",
  );
  if (
    runner?.noise_key_confirmed &&
    runner.noise_public_key &&
    runner.status !== "revoked"
  ) {
    const runnerTarget = {
      runnerId: runner.id,
      runnerPublicKey: runner.noise_public_key,
    };
    discoveredRuntimes.push({
      location: "byoc",
      runnerId: runner.id,
      runnerPublicKey: runner.noise_public_key,
    });
    const runnerRepositories = await listRunnerRepositories(runnerTarget);
    locatedRepositories.push(
      ...runnerRepositories.map((repository) => ({
        repository,
        runtime: {
          location: "byoc" as const,
          runnerId: runner.id,
          runnerPublicKey: runner.noise_public_key as string,

```

```

        },
      })),
    );
  }
} catch {
  unavailableLocations.push("BYOC repositories are unavailable.");
}
setLocationErrors(unavailableLocations);
setAvailableRuntimes(discoveredRuntimes);
setRepos(locatedRepositories);
} catch {
  setRepoLoadError("Failed to load repositories.");
} finally {
  setLoading(false);
}
}

useEffect(() => {
  void loadRepos();
}, []);

async function loadGitHubInstallations() {
  if (status !== "authenticated") {
    setGithubInstallations([]);
    return;
  }

  try {
    let data: GitHubInstallation[];
    if (window.yinshiDesktop) {
      const hostedTransport = createRuntimeTransport({ location: "hosted" });
      data = await hostedTransport
        .get<GitHubInstallation[]>("/api/github/installations")
        .finally(() => {
          hostedTransport.close();
        });
    } else {
      data = await api.get<GitHubInstallation[]>("/api/github/installations");
    }
    setGithubInstallations(data);
  } catch {
    setGithubInstallations([]);
  }
}

useEffect(() => {

```

```

        void loadGitHubInstallations();
    }, [status]);

useEffect(() => {
    const notice = githubNoticeFromSearch(location.search);
    if (!notice) {
        return;
    }

    setGithubNotice(notice);
    if (status === "authenticated") {
        void loadGitHubInstallations();
    }

    const params = new URLSearchParams(location.search);
    params.delete("github_connected");
    params.delete("github_connect_error");
    const nextSearch = params.toString();
    navigate(
        {
            pathname: location.pathname,
            search: nextSearch ? `?${nextSearch}` : "",
        },
        { replace: true },
    );
}, [location.pathname, location.search, navigate, status]);

function handleImported(repo: Repo | null, runtime?: RuntimeRef) {
    if (repo && runtime) {
        setRepos((prev) => [{ repository: repo, runtime }, ...prev]);
        setRepoLoadError(null);
    }
    setShowImport(false);
}

function handleRepoUpdated(updatedRepo: Repo, runtime: RuntimeRef) {
    setRepos((previousRepositories) =>
        previousRepositories.map((locatedRepository) => {
            const sameRuntime =
                runtimeIdentity(locatedRepository.runtime) ===
                runtimeIdentity(runtime);
            if (sameRuntime && locatedRepository.repository.id === updatedRepo.id) {
                return { repository: updatedRepo, runtime };
            }
            return locatedRepository;
        }),
    ),

```

```

    );
}

return (
  <aside className="flex h-full w-72 flex-col border-r border-gray-800 bg-gray-900">
    <div
      className={`flex items-center justify-between border-b border-gray-800 py-3 ${
        window.yinshiDesktop !== undefined ? "pl-24 pr-4" : "px-4"
      }`}
    >
      <span className="text-xs font-semibold uppercase tracking-wider text-gray-500">
        Workspaces
      </span>
      <button
        onClick={() => setShowImport(true)}
        disabled={!primaryRuntime || loading}
        className="text-gray-500 hover:text-gray-300 disabled:opacity-40"
        title="Add repository"
      >
        {PlusIcon}
      </button>
    </div>

    {showImport && availableRuntimes.length > 0 && (
      <ImportForm
        onDone={handleImported}
        canConnectGitHub={status === "authenticated"}
        githubInstallations={githubInstallations}
        runtimes={availableRuntimes}
      />
    )}

    <div className="flex-1 overflow-y-auto scrollbar-hide py-1">
      {githubNotice && (
        <div
          className={`mx-4 mt-4 rounded-md border px-3 py-2 text-xs ${
            githubNotice.tone === "success"
              ? "border-emerald-500/30 bg-emerald-500/10 text-emerald-200"
              : "border-red-500/40 bg-red-500/10 text-red-300"
          }`}
        >
          {githubNotice.message}
        </div>
      )}

      {loading && (

```



```

    <div className="flex justify-center py-8">
      <div className="h-5 w-5 animate-spin rounded-full border-2 border-blue-500 border">
      </div>
    </div>
  )}

  {!loading && repoLoadError && (
    <div className="mx-4 mt-4 rounded-md border border-red-500/40 bg-red-500/10 px-3 py-3">
      <div>{repoLoadError}</div>
      <button
        onClick={() => void loadRepos()}
        className="mt-2 text-red-200 underline underline-offset-2 hover:text-red-100"
        type="button">
      >
        Retry
      </button>
    </div>
  )}

  {!loading &&
    locationErrors.map((locationError) => (
      <div
        key={locationError}
        className="mx-4 mt-3 rounded-md border border-amber-500/30 bg-amber-500/10 px-3 py-3">
      >
        {locationError}
      </div>
    ))}

  {!loading && !repoLoadError && repos.length === 0 && !showImport && (
    <div className="px-4 py-8 text-center text-sm text-gray-600">
      No repositories yet.
    </div>
  )}

  {repos.map(({ repository, runtime }) => (
    <RepoSection
      key={`_${runtimeIdentity(runtime)}:${repository.id}`}
      repo={repository}
      runtime={runtime}
      activeSessionId={activeSessionId}
      onRepoUpdated={(updatedRepository) =>
        handleRepoUpdated(updatedRepository, runtime)
      }
      onNavigate={onNavigate}
    />
  ))}

```

```

</div>

<button
  onClick={() => setShowImport(true)}
  disabled={!primaryRuntime || loading}
  className="flex items-center gap-2 border-t border-gray-800 px-4 py-3 text-sm text-gray-800"
>
  {PlusIcon}
  Add repository
</button>

<div className="flex items-center justify-between border-t border-gray-800 px-4 py-3">
  {status === "authenticated" && email ? (
    <>
      <span className="truncate text-xs text-gray-500" title={email}>
        {email}
      </span>
      <div className="flex items-center gap-2">
        <button
          onClick={() => {
            navigate("/app/settings");
            onNavigate?.();
          }}
          className="shrink-0 text-gray-600 hover:text-gray-400"
          title="Settings"
        >
          <svg
            className="h-4 w-4"
            fill="none"
            viewBox="0 0 24 24"
            strokeWidth={1.5}
            stroke="currentColor"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M9.594 3.94c.09-.542.542-.94 1.11-.94h2.593c.55 0 1.02.398 1.11.941.293.866.293 1.732 0 2.593-.398.55-1.02.94-1.11.941h-2.593c-.55 0-1.02-.398-1.11-.941z"
            />
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M15 12a3 3 0 1 1-6 0 3 3 0 0 1 6 0Z"
            />
          </svg>
        </button>
        <button

```

```

        onClick={toggleTheme}
        className="shrink-0 text-gray-600 hover:text-gray-400"
        title={
            theme === "light"
                ? "Switch to dark mode"
                : "Switch to light mode"
        }
    >
        <ThemeIcon theme={theme} />
    </button>
    <button
        onClick={logout}
        className="shrink-0 text-xs text-gray-600 hover:text-gray-400"
    >
        Logout
    </button>
</div>
</>
) : (
    <button
        onClick={toggleTheme}
        className="text-gray-600 hover:text-gray-400"
        title={
            theme === "light" ? "Switch to dark mode" : "Switch to light mode"
        }
    >
        <ThemeIcon theme={theme} />
    </button>
    </div>
</aside>
);
}

function RepoSection({
    repo,
    runtime,
    activeSessionId,
    onRepoUpdated,
    onNavigate,
}): {
    repo: Repo;
    runtime: RuntimeRef;
    activeSessionId: string | undefined;
    onRepoUpdated: (repo: Repo) => void;
    onNavigate?: () => void;

```

```

}) {
  const navigate = useNavigate();
  const transport = useMemo(() => createRuntimeTransport(runtime), [runtime]);
  useEffect(
    () => () => {
      transport.close();
    },
    [transport],
  );
  const [workspaces, setWorkspaces] = useState<Workspace[]>([]);
  const [expanded, setExpanded] = useState(runtime.location !== "byoc");
  const [loaded, setLoaded] = useState(false);
  const [creating, setCreating] = useState(false);
  const [showArchived, setShowArchived] = useState(false);
  const [workspaceError, setWorkspaceError] = useState<string | null>(null);
  const [showSettings, setShowSettings] = useState(false);
  const [agentsMdDraft, setAgentsMdDraft] = useState(repo.agents_md ?? "");
  const [settingsSaving, setSettingsSaving] = useState(false);
  const [settingsError, setSettingsError] = useState<string | null>(null);
  const [settingsNotice, setSettingsNotice] = useState<string | null>(null);

  useEffect(() => {
    setAgentsMdDraft(repo.agents_md ?? "");
  }, [repo.agents_md]);

  useEffect(() => {
    if (expanded && !loaded) {
      transport
        .get<Workspace[]>(`/api/repos/${repo.id}/workspaces`)
        .then((data) => {
          setWorkspaces(data);
          setWorkspaceError(null);
          setLoaded(true);
        })
        .catch(() => {
          setWorkspaceError("Failed to load workspaces.");
        });
    }
  }, [expanded, loaded, repo.id, transport]);

  async function createBranch(e: React.MouseEvent) {
    e.stopPropagation();
    setCreating(true);
    setWorkspaceError(null);
    try {
      const ws = await transport.post<Workspace>(

```

```

        `/api/repos/${repo.id}/workspaces`,
        {},
    );
    setWorkspaces((prev) => [ws, ...prev]);
    setExpanded(true);
    // Auto-create a session and navigate to it
    const session = await transport.post<SessionInfo>(
        `/api/workspaces/${ws.id}/sessions`,
        { model: DEFAULT_SESSION_MODEL },
    );
    navigate(
        `/app/session/${runtimeResourceId(runtime, session.id, {
            desktop: window.yinshiDesktop !== undefined,
        })}`,
    );
    onNavigate?.();
} catch {
    setWorkspaceError("Failed to create workspace.");
} finally {
    setCreating(false);
}
}

async function handleStateChange(workspaceId: string, newState: string) {
    try {
        const updated = await transport.patch<Workspace>(
            `/api/workspaces/${workspaceId}`,
            { state: newState },
        );
        setWorkspaces((prev) =>
            prev.map((ws) => (ws.id === workspaceId ? updated : ws)),
        );
    } catch {
        setWorkspaceError("Failed to update workspace state.");
    }
}

async function saveRepoSettings() {
    setSettingsSaving(true);
    setSettingsError(null);
    setSettingsNotice(null);
    try {
        const updatedRepo = await transport.patch<Repo>(`/api/repos/${repo.id}`, {
            agents_md: agentsMdDraft.trim() ? agentsMdDraft : null,
        });
        onRepoUpdated(updatedRepo);
    }
}

```

```

        setAgentsMdDraft(updatedRepo.agents_md ?? "");
        setSettingsNotice("Repo instructions saved.");
    } catch {
        setSettingsError("Failed to save repo instructions.");
    } finally {
        setSettingsSaving(false);
    }
}

function cancelRepoSettings() {
    setAgentsMdDraft(repo.agents_md ?? "");
    setSettingsError(null);
    setSettingsNotice(null);
    setShowSettings(false);
}

function toggleRepoSettings(event: React.MouseEvent<HTMLButtonElement>) {
    event.stopPropagation();
    setSettingsError(null);
    setSettingsNotice(null);

    if (showSettings) {
        setShowSettings(false);
    } else {
        setExpanded(true);
        setShowSettings(true);
    }
}

const activeWorkspaces = workspaces.filter((ws) => ws.state !== "archived");
const archivedWorkspaces = workspaces.filter((ws) => ws.state === "archived");
const initial = repo.name.charAt(0).toUpperCase();

return (
    <div className="py-0.5">
        <div className="flex items-center hover:bg-gray-800/50 group">
            <button
                onClick={() => setExpanded(!expanded)}
                className="flex flex-1 items-center gap-2.5 px-4 py-2 min-w-0"
            >
                <span
                    className={`flex h-6 w-6 shrink-0 items-center justify-center rounded text-xs font-medium`}
                >
                    {initial}
                </span>
                <span className="flex-1 truncate text-left text-sm font-medium text-gray-200">

```

```

        {repo.name}
      </span>
      <span className="rounded border border-gray-700 px-1.5 py-0.5 text-[9px] font-sans"
        {runtimeLabel(runtime)}
      </span>
    </button>
    <button
      onClick={createBranch}
      disabled={creating}
      className="shrink-0 px-3 py-2 text-gray-600 opacity-0 group-hover:opacity-100 hover:scale-105"
      title="New branch"
    >
      {creating ? (
        <div className="h-3.5 w-3.5 animate-spin rounded-full border-2 border-gray-500" />
      ) : (
        <svg
          className="h-3.5 w-3.5"
          fill="none"
          viewBox="0 0 24 24"
          strokeWidth={2}
          stroke="currentColor"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            d="M12 4.5v15m7.5-7.5h-15"
          />
        </svg>
      )}
    </button>
    <button
      onClick={toggleRepoSettings}
      className="shrink-0 px-3 py-2 text-gray-600 opacity-0 group-hover:opacity-100 hover:scale-105"
      title="Repo settings"
    >
      <svg
        className="h-3.5 w-3.5"
        fill="none"
        viewBox="0 0 24 24"
        strokeWidth={1.5}
        stroke="currentColor"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          d="M4.5 12a7.5 7.5 0 1 1 15 0 7.5 7.5 0 0 1-15 0Zm7.5-3.75v3.75l2.25 2.25"
        >

```

```

        />
      </svg>
    </button>
  </div>

  {expanded && (
    <>
      {showSettings && (
        <div className="space-y-2 border-b border-gray-800/80 bg-gray-950/50 px-4 py-3">
          <div className="pl-9">
            <label
              htmlFor={`repo-agents-md-${repo.id}`}
              className="text-[11px] font-semibold uppercase tracking-wide text-gray-500"
            >
              AGENTS.md override
            </label>
            <p className="mt-1 text-xs text-gray-400">
              Used as the repo-level runtime instructions for new sessions
              from this repo.
            </p>
            <textarea
              id={`repo-agents-md-${repo.id}`}
              value={agentsMdDraft}
              onChange={(event) => setAgentsMdDraft(event.target.value)}
              rows={10}
              spellCheck={false}
              className="mt-2 w-full rounded-md border border-gray-800 bg-gray-900 px-3"
              placeholder="Leave blank to disable the repo-level override."
            />
            {settingsError && (
              <div className="mt-2 rounded-md border border-red-500/40 bg-red-500/10 px-3">
                {settingsError}
              </div>
            )}
            {settingsNotice && !settingsError && (
              <div className="mt-2 rounded-md border border-emerald-500/30 bg-emerald-500/10 px-3">
                {settingsNotice}
              </div>
            )}
          <div className="mt-3 flex gap-2">
            <button
              type="button"
              onClick={() => void saveRepoSettings()}
              disabled={settingsSaving}
              className="rounded-md bg-blue-500 px-3 py-1.5 text-xs font-medium text-white"
            >

```



```

        {settingsSaving ? "Saving..." : "Save AGENTS.md"}
    </button>
    <button
        type="button"
        onClick={cancelRepoSettings}
        disabled={settingsSaving}
        className="rounded-md bg-gray-800 px-3 py-1.5 text-xs text-gray-400 disa
    >
        Cancel
    </button>
</div>
</div>
</div>
)}
{workspaceError && (
    <div className="px-11 py-2 text-xs text-red-400">
        {workspaceError}
    </div>
)}
{activeWorkspaces.map((ws) => (
    <WorkspaceItem
        key={ws.id}
        workspace={ws}
        runtime={runtime}
        transport={transport}
        activeSessionId={activeSessionId}
        onNavigate={onNavigate}
        onArchive={() => handleStateChange(ws.id, "archived")}
    />
))}

{archivedWorkspaces.length > 0 && (
    <>
        <button
            onClick={() => setShowArchived(!showArchived)}
            className="flex w-full items-center gap-1 px-11 py-1 text-xs text-gray-600 h
        >
        <svg
            className={`h-3 w-3 transition-transform ${showArchived ? "rotate-90" : ""}
            fill="none"
            viewBox="0 0 24 24"
            strokeWidth={2}
            stroke="currentColor"
        >
            <path
                strokeLinecap="round"

```

```

        strokeLinejoin="round"
        d="M8.25 4.517.5 7.5-7.5 7.5"
      />
    </svg>
    Archived ({archivedWorkspaces.length})
  </button>
  {showArchived &&
    archivedWorkspaces.map((ws) => (
      <WorkspaceItem
        key={ws.id}
        workspace={ws}
        runtime={runtime}
        transport={transport}
        activeSessionId={activeSessionId}
        onNavigate={onNavigate}
        onRestore={() => handleStateChange(ws.id, "ready")}
      />
    ))}
</>
  )}
</>
  )}
</div>
);
}

function WorkspaceItem({
  workspace,
  runtime,
  transport,
  activeSessionId,
  onNavigate,
  onArchive,
  onRestore,
}: {
  workspace: Workspace;
  runtime: RuntimeRef;
  transport: RuntimeTransport;
  activeSessionId: string | undefined;
  onNavigate?: () => void;
  onArchive?: () => void;
  onRestore?: () => void;
}) {
  const navigate = useNavigate();
  const [sessions, setSessions] = useState<SessionInfo[]>([]);
  const [loadedSessions, setLoadedSessions] = useState(false);

```

```

const [sessionError, setSessionError] = useState<string | null>(null);

useEffect(() => {
  if (!loadedSessions) {
    transport
      .get<SessionInfo[]>(`/api/workspaces/${workspace.id}/sessions`)
      .then((data) => {
        setSessions(data);
        setSessionError(null);
        setLoadedSessions(true);
      })
      .catch(() => {
        setSessionError("Failed to load sessions.");
      });
  }
}, [workspace.id, loadedSessions, transport]);

async function openOrCreateSession() {
  if (sessions.length > 0) {
    navigate(
      `/app/session/${runtimeResourceId(runtime, sessions[0].id, {
        desktop: window.yinshiDesktop !== undefined,
      })}`,
    );
    onNavigate?.();
    return;
  }

  try {
    const session = await transport.post<SessionInfo>(
      `/api/workspaces/${workspace.id}/sessions`,
      { model: DEFAULT_SESSION_MODEL },
    );
    setSessions([session]);
    navigate(
      `/app/session/${runtimeResourceId(runtime, session.id, {
        desktop: window.yinshiDesktop !== undefined,
      })}`,
    );
    onNavigate?.();
  } catch {
    setSessionError("Failed to create a session.");
  }
}

const isActive = sessions.some(

```

```

(session) =>
  runtimeResourceId(runtime, session.id, {
    desktop: window.yinshiDesktop !== undefined,
  }) === activeSessionId,
);
const hasRunning = sessions.some((s) => s.status === "running");

return (
  <div
    className={`group flex w-full items-center py-1.5 pl-11 pr-4 hover:bg-gray-800/50 ${
      isActive ? "bg-gray-800/70" : ""
    }`}
  >
    <button
      onClick={openOrCreateSession}
      className="flex flex-1 items-center gap-2 min-w-0 text-left"
    >
      <span
        className={`h-2 w-2 shrink-0 rounded-full ${statusDotClass(hasRunning, workspace.name)}`
      >
        <div className="flex-1 min-w-0">
          <div className="truncate text-sm text-gray-300">{workspace.name}</div>
          {sessionError ? (
            <div className="truncate text-[10px] text-red-400">
              {sessionError}
            </div>
          ) : null}
        </div>
      </button>
      {onArchive && (
        <button
          onClick={(e) => {
            e.stopPropagation();
            onArchive();
          }}
          className="shrink-0 ml-1 text-gray-600 opacity-0 group-hover:opacity-100 hover:text-gray-800"
          title="Archive"
        >
          <svg
            className="h-3.5 w-3.5"
            fill="none"
            viewBox="0 0 24 24"
            strokeWidth={1.5}
            stroke="currentColor"
          >
            <path

```

```

        strokeLinecap="round"
        strokeLinejoin="round"
        d="m20.25 7.5-.625 10.632a2.25 2.25 0 0 1-2.247 2.118H6.622a2.25 2.25 0 0 1-2.
      />
    </svg>
  </button>
)}
{onRestore && (
  <button
    onClick={e => {
      e.stopPropagation();
      onRestore();
    }}
    className="shrink-0 ml-1 text-gray-600 opacity-0 group-hover:opacity-100 hover:tex
    title="Restore"
  >
    <svg
      className="h-3.5 w-3.5"
      fill="none"
      viewBox="0 0 24 24"
      strokeWidth={1.5}
      stroke="currentColor"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        d="M9 15 3 9m0 0 6-6M3 9h12a6 6 0 0 1 0 12h-3"
      />
    </svg>
  </button>
)}
</div>
);
}

function ImportForm({
  onDone,
  canConnectGitHub,
  githubInstallations,
  runtimes,
}: {
  onDone: (repo: Repo | null, runtime?: RuntimeRef) => void;
  canConnectGitHub: boolean;
  githubInstallations: GitHubInstallation[];
  runtimes: RuntimeRef[];
}) {

```

```

const [input, setInput] = useState("");
const [submitting, setSubmitting] = useState(false);
const [error, setError] = useState<string | null>(null);
const [errorAction, setErrorAction] = useState<ImportAction | null>(null);
const [selectedRuntimeIdentity, setSelectedRuntimeIdentity] = useState(
  runtimeIdentity(runtimes[0]),
);
const desktopBridge = window.yinshiDesktop;
const selectedRuntime =
  runtimes.find(
    (runtime) => runtimeIdentity(runtime) === selectedRuntimeIdentity,
  ) ?? runtimes[0];
const localPathImportEnabled =
  selectedRuntime.location === "local" ||
  (import.meta.env.DEV &&
    selectedRuntime.location === "hosted" &&
    !desktopBridge);

useEffect(() => {
  if (
    !runtimes.some(
      (runtime) => runtimeIdentity(runtime) === selectedRuntimeIdentity,
    )
  ) {
    setSelectedRuntimeIdentity(runtimeIdentity(runtimes[0]));
  }
}, [runtimes, selectedRuntimeIdentity]);

async function handleDesktopImport() {
  if (!desktopBridge) return;
  setSubmitting(true);
  setError(null);
  setErrorAction(null);
  try {
    const result = await desktopBridge.importLocalRepository();
    if (result.status === "cancelled") return;
    const repo = await api.get<Repo>(`/api/repos/${result.repository.id}`);
    onDone(repo, { location: "local" });
  } catch {
    setError(
      "Local repository import failed. Check the repository and try again.",
    );
  } finally {
    setSubmitting(false);
  }
}

```

```

async function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  const value = input.trim();
  if (!value) return;
  setSubmitting(true);
  setError(null);
  setErrorAction(null);
  try {
    const body: { name: string; remote_url?: string; local_path?: string } = {
      name: deriveRepoName(value),
    };
    if (isGitUrl(value)) {
      body.remote_url = value;
    } else if (isLocalPath(value)) {
      if (!localPathImportEnabled) {
        setError("Local paths can only be imported to This Mac.");
        return;
      }
      if (desktopBridge) {
        setError(
          "Use Choose local repository so Yinshi can keep the selected checkout untouched."
        );
        return;
      }
      body.local_path = value;
    } else if (isGithubShorthand(value)) {
      body.remote_url = `https://github.com/${value}`;
    } else {
      setError("Enter a GitHub URL, owner/repo, or local path.");
      return;
    }
  }
  const importTransport = createRuntimeTransport(selectedRuntime);
  const repo = await importTransport.post<Repo>("/api/repos", body).finally(() => {
    importTransport.close();
  });
  onDone(repo, selectedRuntime);
} catch (err) {
  if (err instanceof ApiError) {
    setError(err.message);
    if (err.manageUrl) {
      setErrorAction({
        label: "Manage GitHub",
        href: err.manageUrl,
        external: true,
      });
    }
  }
}

```

```

    } else if (err.connectUrl) {
      setErrorAction({
        label: "Connect GitHub",
        href: err.connectUrl,
        external: false,
      });
    }
  } else {
    setError(err instanceof Error ? err.message : "Import failed");
  }
} finally {
  setSubmitting(false);
}
}

return (
  <form
    onSubmit={handleSubmit}
    className="border-b border-gray-800 px-4 py-3 space-y-2"
  >
    <div>
      <p className="mb-1 text-[11px] font-semibold uppercase tracking-wide text-gray-500">
        Import to
      </p>
      <div className="flex flex-wrap gap-1.5">
        {runtimes.map((runtime) => {
          const identity = runtimeIdentity(runtime);
          const selected = identity === runtimeIdentity(selectedRuntime);
          return (
            <button
              key={identity}
              type="button"
              onClick={() => setSelectedRuntimeIdentity(identity)}
              className={`rounded border px-2 py-1 text-xs ${
                selected
                  ? "border-gray-500 bg-gray-800 text-gray-100"
                  : "border-gray-800 text-gray-500 hover:text-gray-300"
              }`}
            >
              {runtimeLabel(runtime)}
            </button>
          );
        })}
      </div>
    </div>
    {canConnectGitHub && selectedRuntime.location === "hosted" && (

```



```

<div className="rounded-md border border-gray-800 bg-gray-950/60 px-3 py-2">
  <div className="flex items-center justify-between gap-3">
    <div>
      <p className="text-[11px] font-semibold uppercase tracking-wide text-gray-500">
        Private GitHub repos
      </p>
      <p className="mt-1 text-xs text-gray-400">
        Connect GitHub once, then import private repos by URL.
      </p>
    </div>
    <a
      href="/auth/github/install"
      className="shrink-0 rounded-md border border-gray-700 px-2 py-1 text-[11px] text-gray-500"
    >
      Connect GitHub
    </a>
  </div>
  {githubInstallations.length > 0 && (
    <div className="mt-2 flex flex-wrap gap-1.5">
      {githubInstallations.map((installation) => (
        <a
          key={installation.installation_id}
          href={installation.html_url}
          target="_blank"
          rel="noreferrer"
          className="rounded-full border border-gray-700 px-2 py-0.5 text-[11px] text-gray-500"
          title={`${installation.account_login} (${installation.account_type})`}
        >
          {installation.account_login}
        </a>
      ))}
    </div>
  )}
</div>
)}
{desktopBridge && selectedRuntime.location === "local" && (
  <button
    type="button"
    onClick={handleDesktopImport}
    disabled={submitting}
    className="flex w-full items-center justify-center gap-2 rounded-md border border-gray-800 bg-gray-950/60 px-3 py-2"
  >
    <svg
      className="h-4 w-4"
      fill="none"
      viewBox="0 0 24 24"
    >

```

```

        strokeWidth={1.5}
        stroke="currentColor"
        aria-hidden="true"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          d="M3.75 9.75h16.5m-15-4.5h4.129c.414 0 .81.171 1.094.472 11.054 1.117c.283.3.6
        />
      </svg>
      {submitting
        ? "Importing local repository..."
        : "Choose local repository"}
    </button>
  )}
  <input
    type="text"
    placeholder={
      localPathImportEnabled
        ? "GitHub URL, user/repo, or local path"
        : "HTTPS GitHub URL or user/repo"
    }
    value={input}
    onChange={(e) => setInput(e.target.value)}
    className="w-full rounded-md bg-gray-800 px-3 py-2 text-sm text-gray-100 placeholder
    autoFocus
  />
  {error && (
    <div className="rounded-md border border-red-500/40 bg-red-500/10 px-3 py-2 text-xs
      <p>{error}</p>
      {errorAction && (
        <a
          href={errorAction.href}
          target={errorAction.external ? "_blank" : undefined}
          rel={errorAction.external ? "noreferrer" : undefined}
          className="mt-2 inline-flex text-red-100 underline underline-offset-2 hover:t
        >
          {errorAction.label}
        </a>
      )}
    </div>
  )}
  <div className="flex gap-2">
    <button
      type="submit"
      disabled={submitting || !input.trim()}

```

```

        className="flex-1 rounded-md bg-blue-500 px-2 py-1.5 text-xs font-medium text-white"
      >
        {submitting ? "Importing..." : "Import"}
      </button>
      <button
        type="button"
        onClick={() => onDone(null)}
        className="flex-1 rounded-md bg-gray-800 px-2 py-1.5 text-xs text-gray-400"
      >
        Cancel
      </button>
    </div>
  </form>
);
}

```

22.9 components/WorkspaceInspector.tsx

The workspace inspector shows changed files, previews, diffs, edits, downloads, and the reconnecting xterm terminal. The root chunk below tangles back to `frontend/src/components/WorkspaceInspector.tsx`.

```

import {
  useCallback,
  useEffect,
  useRef,
  useState,
  type CSSProperties,
  type PointerEvent as ReactPointerEvent,
} from "react";
import { Terminal, type ITheme } from "@xterm/xterm";
import { FitAddon } from "@xterm/addon-fit";
import "@xterm/xterm/css/xterm.css";
import type { WorkspaceChangedFile, WorkspaceFileNode } from "../api/client";
import {
  openRuntimeTerminal,
  type RuntimeTerminalChannel,
} from "../runtime/terminalChannel";
import type { RuntimeTransport } from "../runtime/runtimeTransport";

type InspectorTab = "files" | "changes";
type ViewerMode = "preview" | "diff" | "edit";

interface WorkspaceInspectorProps {
  workspaceId: string;
  transport: RuntimeTransport;
}

```

```

    refreshKey: number;
    active?: boolean;
    className?: string;
    style?: CSSProperties;
}

const TERMINAL_HEIGHT_DEFAULT = 260;
const TERMINAL_HEIGHT_MIN = 140;
const TERMINAL_HEIGHT_MAX = 620;
const FILE_STATUS_REFRESH_MS = 15000;
const TERMINAL_RECONNECT_DELAY_MS = 2000;
const TERMINAL_ACCENT_COLOR = "#c23b22";
const TERMINAL_SELECTION_BACKGROUND = "rgba(194, 59, 34, 0.28)";

const LIGHT_TERMINAL_ANSI_COLORS: ITheme = {
  black: "#1a1410",
  red: "#a02e18",
  green: "#4f6f37",
  yellow: "#8a681d",
  blue: "#75543c",
  magenta: "#8a4d58",
  cyan: "#4d6b64",
  white: "#3d3228",
  brightBlack: "#8c7a64",
  brightRed: TERMINAL_ACCENT_COLOR,
  brightGreen: "#5f7f45",
  brightYellow: "#b8963e",
  brightBlue: "#946846",
  brightMagenta: "#a15f6c",
  brightCyan: "#5d8179",
  brightWhite: "#1a1410",
};

const DARK_TERMINAL_ANSI_COLORS: ITheme = {
  black: "#4a3f35",
  red: "#d4543d",
  green: "#b0bf80",
  yellow: "#d7b95d",
  blue: "#c79b75",
  magenta: "#d38a9b",
  cyan: "#8fbbb0",
  white: "#e0d1b8",
  brightBlack: "#a89478",
  brightRed: "#e86a52",
  brightGreen: "#c3d38f",
  brightYellow: "#e7ca72",
};

```

```

    brightBlue: "#d7ad88",
    brightMagenta: "#e0a0ae",
    brightCyan: "#a6d0c5",
    brightWhite: "#f7f0e3",
  };

function storedTerminalHeight(): number {
  const raw = sessionStorage.getItem("yinshi-terminal-height");
  const value = Number(raw);
  if (Number.isFinite(value)) {
    return Math.min(TERMINAL_HEIGHT_MAX, Math.max(TERMINAL_HEIGHT_MIN, value));
  }
  return TERMINAL_HEIGHT_DEFAULT;
}

const CHANGE_LABELS: Partial<Record<WorkspaceChangedFile["kind"], string>> = {
  added: "A",
  copied: "C",
  deleted: "D",
  modified: "M",
  renamed: "R",
  untracked: "U",
};

function statusLabel(file: WorkspaceChangedFile): string {
  return (CHANGE_LABELS[file.kind] ?? file.status.trim()) || "?";
}

function countFiles(nodes: WorkspaceFileNode[]): number {
  return nodes.reduce((total, node) => {
    if (node.type === "file") return total + 1;
    return total + countFiles(node.children);
  }, 0);
}

function themeVariableColor(
  variableName: string,
  fallback: string,
  alpha?: number,
): string {
  const rawValue = getComputedStyle(document.documentElement)
    .getPropertyValue(variableName)
    .trim();
  const components = rawValue.split(/\s+/).map(Number);
  if (
    components.length !== 3 ||

```

```

        components.some((component) => !Number.isFinite(component))
    ) {
        return fallback;
    }
    const [red, green, blue] = components;
    if (alpha === undefined) {
        return `rgb(${red}, ${green}, ${blue})`;
    }
    return `rgba(${red}, ${green}, ${blue}, ${alpha})`;
}

function terminalTheme(): ITheme {
    const ansiColors = document.documentElement.classList.contains("dark")
        ? DARK_TERMINAL_ANSI_COLORS
        : LIGHT_TERMINAL_ANSI_COLORS;
    const background = themeVariableColor("--gray-900", "#f0e6d3");
    return {
        ...ansiColors,
        background,
        foreground: themeVariableColor("--gray-200", "#2d2520"),
        cursor: TERMINAL_ACCENT_COLOR,
        cursorAccent: background,
        selectionBackground: TERMINAL_SELECTION_BACKGROUND,
        selectionForeground: themeVariableColor("--gray-50", "#0f0c09"),
        selectionInactiveBackground: themeVariableColor(
            "--gray-600",
            "rgba(168, 148, 120, 0.24)",
            0.24,
        ),
        scrollbarSliderBackground: themeVariableColor(
            "--gray-400",
            "rgba(107, 93, 79, 0.24)",
            0.24,
        ),
        scrollbarSliderHoverBackground: themeVariableColor(
            "--gray-400",
            "rgba(107, 93, 79, 0.38)",
            0.38,
        ),
        scrollbarSliderActiveBackground: themeVariableColor(
            "--gray-400",
            "rgba(107, 93, 79, 0.5)",
            0.5,
        ),
    };
}

```

```

function FileTree({
  nodes,
  selectedPath,
  onSelect,
}): {
  nodes: WorkspaceFileNode[];
  selectedPath: string | null;
  onSelect: (path: string) => void;
}) {
  return (
    <ul className="space-y-0.5">
      {nodes.map((node) => (
        <li key={node.path}>
          {node.type === "directory" ? (
            <details open className="group">
              <summary className="cursor-pointer select-none rounded px-2 py-1 text-xs font-
                {node.name}
              </summary>
              <div className="ml-3 border-l border-gray-800 pl-2">
                <FileTree
                  nodes={node.children}
                  selectedPath={selectedPath}
                  onSelect={onSelect}
                />
              </div>
            </details>
          ) : (
            <button
              type="button"
              onClick={() => onSelect(node.path)}
              className={`block w-full truncate rounded px-2 py-1 text-left text-xs ${
                selectedPath === node.path
                  ? "bg-blue-500/15 text-blue-200"
                  : "text-gray-300 hover:bg-gray-800 hover:text-gray-100"
              }`}
              title={node.path}
            >
              {node.name}
            </button>
          )}
        </li>
      )}
    </ul>
  );
}

```

```

function FileViewer({
  workspaceId,
  transport,
  path,
  mode,
  onModeChange,
  onSaved,
}: {
  workspaceId: string;
  transport: RuntimeTransport;
  path: string | null;
  mode: ViewerMode;
  onModeChange: (mode: ViewerMode) => void;
  onSaved: () => void;
}) {
  const [content, setContent] = useState("");
  const [draft, setDraft] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    if (!path) {
      setContent("");
      setDraft("");
      setError(null);
      return;
    }
    let cancelled = false;
    setLoading(true);
    setError(null);
    const encodedPath = encodeURIComponent(path);
    const endpoint =
      mode === "diff"
        ? `~/api/workspaces/${workspaceId}/files/diff?path=${encodedPath}`
        : `~/api/workspaces/${workspaceId}/files/preview?path=${encodedPath}`;
    transport
      .get<{ content?: string; diff?: string }>(endpoint)
      .then((response) => {
        if (cancelled) return;
        const value =
          mode === "diff" ? (response.diff ?? "") : (response.content ?? "");
        setContent(value);
        setDraft(value);
      })
      .catch((apiError) => {

```



```

        if (cancelled) return;
        setError(
            apiError instanceof Error ? apiError.message : "Failed to load file",
        );
    })
    .finally(() => {
        if (!cancelled) setLoading(false);
    });
    return () => {
        cancelled = true;
    };
}, [mode, path, transport, workspaceId]);

const save = useCallback(async () => {
    if (!path) return;
    setLoading(true);
    setError(null);
    try {
        await transport.put(
            `/api/workspaces/${workspaceId}/files/content?path=${encodeURIComponent(path)}`,
            {
                content: draft,
            },
        );
        setContent(draft);
        onSaved();
    } catch (apiError) {
        setError(
            apiError instanceof Error ? apiError.message : "Failed to save file",
        );
    } finally {
        setLoading(false);
    }
}, [draft, onSaved, path, transport, workspaceId]);

if (!path) {
    return (
        <div className="p-3 text-xs text-gray-500">
            Select a file to preview it.
        </div>
    );
}

return (
    <div className="flex h-full min-h-0 flex-col border-t border-gray-800 bg-gray-950/60">
        <div className="flex items-center gap-1 border-b border-gray-800 px-2 py-2">

```

```

<div
  className="min-w-0 flex-1 truncate text-xs font-medium text-gray-300"
  title={path}
>
  {path}
</div>
{(["preview", "diff", "edit"] as ViewerMode[]).map((viewerMode) => (
  <button
    key={viewerMode}
    type="button"
    onClick={() => onModeChange(viewerMode)}
    className={`rounded px-2 py-1 text-[11px] capitalize ${
      mode === viewerMode
        ? "bg-gray-700 text-gray-100"
        : "text-gray-500 hover:bg-gray-800 hover:text-gray-200"
    }`}
  >
    {viewerMode}
  </button>
))}
{transport.runtime.location === "local" ||
transport.runtime.location === "hosted" ? (
  <a
    href={`~/api/workspaces/${workspaceId}/files/download?path=${encodeURIComponent(
      className="rounded px-2 py-1 text-[11px] text-gray-500 hover:bg-gray-800 hover:
  >
    Download
  </a>
) : null}
</div>
{error && (
  <div className="border-b border-red-900/40 bg-red-950/40 px-3 py-2 text-xs text-red-
    {error}
  </div>
)}
{loading && (
  <div className="px-3 py-2 text-xs text-gray-500">Loading...</div>
)}
{mode === "edit" ? (
  <div className="flex min-h-0 flex-1 flex-col">
    <textarea
      value={draft}
      onChange={(event) => setDraft(event.target.value)}
      spellCheck={false}
      className="min-h-0 flex-1 resize-none bg-gray-950 p-3 font-mono text-xs leading-
  />

```

```

        <div className="border-t border-gray-800 p-2 text-right">
          <button
            type="button"
            onClick={save}
            disabled={loading || draft === content}
            className="rounded bg-blue-600 px-3 py-1 text-xs font-medium text-white disabled"
          >
            Save
          </button>
        </div>
      </div>
    ) : (
      <pre className="min-h-0 flex-1 overflow-auto whitespace-pre-wrap p-3 font-mono text-gray-800"
        {content} || (mode === "diff" ? "No diff for this file." : "")
      </pre>
    )
  </div>
);
}

function RuntimeTerminalPane({
  workspaceId,
  transport,
  active,
}): {
  workspaceId: string;
  transport: RuntimeTransport;
  active: boolean;
} {
  const containerRef = useRef<HTMLDivElement | null>(null);
  const terminalRef = useRef<Terminal | null>(null);
  const fitRef = useRef<FitAddon | null>(null);
  const channelRef = useRef<RuntimeTerminalChannel | null>(null);
  const [status, setStatus] = useState(active ? "Connecting..." : "Paused");
  const [connectionVersion, setConnectionVersion] = useState(0);

  const fit = useCallback(() => {
    const terminal = terminalRef.current;
    const fitAddon = fitRef.current;
    if (!terminal || !fitAddon) return;
    try {
      fitAddon.fit();
      const channel = channelRef.current;
      if (channel) {
        void channel.resize(terminal.cols, terminal.rows).catch(() => {
          setStatus("Terminal resize failed");
        });
      }
    }
  }, [terminalRef, fitRef, channelRef]);

```

```

        });
    }
} catch {
    // The terminal can be hidden during mobile drawer transitions.
}
}, []);

useEffect(() => {
    if (!active) {
        setStatus("Paused");
        return;
    }
    const container = containerRef.current;
    if (!container) return;
    const abortController = new AbortController();
    let disposed = false;
    let reconnectTimer: number | null = null;
    setStatus("Connecting...");

    const terminal = new Terminal({
        cursorBlink: true,
        fontFamily: 'Menlo, Monaco, "Courier New", monospace',
        fontSize: 12,
        scrollbar: 1000,
        theme: terminalTheme(),
    });
    const fitAddon = new FitAddon();
    terminal.loadAddon(fitAddon);
    terminal.open(container);
    terminalRef.current = terminal;
    fitRef.current = fitAddon;
    try {
        fitAddon.fit();
    } catch {
        // Initial layout can still be hidden while a drawer opens.
    }

    const inputSubscription = terminal.onData((data) => {
        const channel = channelRef.current;
        if (channel) {
            void channel
                .sendInput(data)
                .catch(() => setStatus("Terminal input failed"));
        }
    });
    const observer = new ResizeObserver(() => fit());

```

```

observer.observe(container);
const themeObserver = new MutationObserver(() => {
  terminal.options.theme = terminalTheme();
});
themeObserver.observe(document.documentElement, {
  attributes: true,
  attributeFilter: ["class"],
});

void (async () => {
  try {
    const channel = await openRuntimeTerminal(transport, workspaceId, {
      cols: terminal.cols,
      rows: terminal.rows,
      signal: abortController.signal,
    });
    if (disposed) {
      await channel.close();
      return;
    }
    channelRef.current = channel;
    setStatus("Connected");
    for await (const event of channel.events()) {
      if (event.type === "terminal_ready") {
        if (typeof event.replay === "string" && event.replay) {
          terminal.write(event.replay);
        }
        setStatus("Connected");
        fit();
      } else if (
        event.type === "terminal_data" &&
        typeof event.data === "string"
      ) {
        terminal.write(event.data);
      } else if (event.type === "terminal_exit") {
        setStatus("Shell exited. Restart to open a new terminal.");
      } else if (event.type === "error") {
        setStatus(
          typeof event.error === "string" ? event.error : "Terminal error",
        );
      }
    }
  }
  if (!disposed) {
    setStatus("Disconnected. Retrying...");
    reconnectTimer = window.setTimeout(
      () => setConnectionVersion((value) => value + 1),
    );
  }
});

```

```

        TERMINAL_RECONNECT_DELAY_MS,
    );
}
} catch (error) {
    if (
        disposed ||
        (error instanceof DOMException && error.name === "AbortError")
    )
        return;
    setStatus(
        error instanceof Error ? error.message : "Terminal connection failed",
    );
    reconnectTimer = window.setTimeout(
        () => setConnectionVersion((value) => value + 1),
        TERMINAL_RECONNECT_DELAY_MS,
    );
}
})();

return () => {
    disposed = true;
    abortController.abort();
    if (reconnectTimer !== null) window.clearTimeout(reconnectTimer);
    themeObserver.disconnect();
    observer.disconnect();
    inputSubscription.dispose();
    const channel = channelRef.current;
    channelRef.current = null;
    if (channel) void channel.close().catch(() => undefined);
    terminal.dispose();
    terminalRef.current = null;
    fitRef.current = null;
};
}, [active, connectionVersion, fit, transport, workspaceId]);

const restart = useCallback(() => {
    terminalRef.current?.reset();
    const channel = channelRef.current;
    if (channel) {
        void channel.restart().catch(() => setStatus("Terminal restart failed"));
    } else {
        setConnectionVersion((value) => value + 1);
    }
}, []);

return (

```

```

<div className="flex h-full min-h-0 flex-col bg-gray-900">
  <div className="flex items-center justify-between border-b border-gray-800 px-3 py-2">
    <div className="text-xs font-medium text-gray-300">Terminal</div>
    <div className="flex items-center gap-2">
      <span className="text-[11px] text-gray-500">{status}</span>
      <button
        type="button"
        onClick={restart}
        className="rounded px-2 py-1 text-[11px] text-gray-400 hover:bg-gray-800 hover:
      >
        Restart
      </button>
    </div>
  </div>
  <div ref={containerRef} className="min-h-0 flex-1 overflow-hidden p-2" />
</div>
);
}

```

```

export default function WorkspaceInspector({
  workspaceId,
  transport,
  refreshKey,
  active = true,
  className = "",
  style,
}: WorkspaceInspectorProps) {
  const [tab, setTab] = useState<InspectorTab>("files");
  const [files, setFiles] = useState<WorkspaceFileNode[]>([]);
  const [changes, setChanges] = useState<WorkspaceChangedFile[]>([]);
  const [selectedPath, setSelectedPath] = useState<string | null>(null);
  const [viewerMode, setViewerMode] = useState<ViewerMode>("preview");
  const [terminalHeight, setTerminalHeight] = useState(storedTerminalHeight);
  const [error, setError] = useState<string | null>(null);

  const refresh = useCallback(async () => {
    if (!active) return;
    try {
      const [treeResponse, changedResponse] = await Promise.all([
        transport.get<{ files: WorkspaceFileNode[] }>(`
          ~/api/workspaces/${workspaceId}/files/tree`,
        ),
        transport.get<{ files: WorkspaceChangedFile[] }>(`
          ~/api/workspaces/${workspaceId}/files/changed`,
        ),
      ]);
    }
  });
}

```

```

        setFiles(treeResponse.files);
        setChanges(changedResponse.files);
        setError(null);
    } catch (apiError) {
        setError(
            apiError instanceof Error
                ? apiError.message
                : "Failed to load workspace files",
        );
    }
}, [active, transport, workspaceId]);

const refreshChanges = useCallback(async () => {
    if (!active) return;
    try {
        const changedResponse = await transport.get<{
            files: WorkspaceChangedFile[];
        }>(`/api/workspaces/${workspaceId}/files/changed`);
        setChanges(changedResponse.files);
        setError(null);
    } catch (apiError) {
        setError(
            apiError instanceof Error
                ? apiError.message
                : "Failed to load workspace changes",
        );
    }
}, [active, transport, workspaceId]);

useEffect(() => {
    void refresh();
}, [refresh, refreshKey]);

useEffect(() => {
    if (!active) return;
    const interval = window.setInterval(() => {
        if (document.visibilityState === "visible") {
            void refreshChanges();
        }
    }, FILE_STATUS_REFRESH_MS);
    const onFocus = () => void refresh();
    window.addEventListener("focus", onFocus);
    return () => {
        window.clearInterval(interval);
        window.removeEventListener("focus", onFocus);
    };
};

```



```

    }, [active, refresh, refreshChanges]);

const beginTerminalResize = useCallback(
  (event: ReactPointerEvent<HTMLDivElement>) => {
    event.currentTarget.setPointerCapture(event.pointerId);
    const startY = event.clientY;
    const startHeight = terminalHeight;
    const onMove = (moveEvent: PointerEvent) => {
      const nextHeight = Math.min(
        TERMINAL_HEIGHT_MAX,
        Math.max(
          TERMINAL_HEIGHT_MIN,
          startHeight - (moveEvent.clientY - startY),
        ),
      );
      setTerminalHeight(nextHeight);
      sessionStorage.setItem("yinshi-terminal-height", String(nextHeight));
    };
    const onUp = () => {
      window.removeEventListener("pointermove", onMove);
      window.removeEventListener("pointerup", onUp);
    };
    window.addEventListener("pointermove", onMove);
    window.addEventListener("pointerup", onUp);
  },
  [terminalHeight],
);

return (
  <aside
    style={style}
    className={`flex min-h-0 flex-col border-l border-gray-800 bg-gray-900 ${className}`}
  >
    <div className="flex items-center border-b border-gray-800 px-3 py-2">
      <button
        type="button"
        onClick={() => setTab("files")}
        className={`rounded px-2 py-1 text-xs font-medium ${tab === "files" ? "bg-gray-800"} `}
      >
        All files <span className="text-gray-500">{countFiles(files)}</span>
      </button>
      <button
        type="button"
        onClick={() => setTab("changes")}
        className={`ml-1 rounded px-2 py-1 text-xs font-medium ${tab === "changes" ? "bg-gray-800"} `}
      >

```

```

      Changes <span className="text-gray-500">{changes.length}</span>
    </button>
    <button
      type="button"
      onClick={() => void refresh()}
      className="ml-auto rounded px-2 py-1 text-[11px] text-gray-500 hover:bg-gray-800 h
    >
      Refresh
    </button>
  </div>

  {error && (
    <div className="border-b border-red-900/40 bg-red-950/40 px-3 py-2 text-xs text-red-
      {error}
    </div>
  )}

  <div className="flex min-h-0 flex-1 flex-col">
    <div className="min-h-0 flex-1 overflow-auto p-2">
      {tab === "files" ? (
        files.length ? (
          <FileTree
            nodes={files}
            selectedPath={selectedPath}
            onSelect={(path) => {
              setSelectedPath(path);
              setViewerMode("preview");
            }}
          />
        ) : (
          <div className="p-3 text-xs text-gray-500">No visible files.</div>
        )
      ) : changes.length ? (
        <div className="space-y-1">
          {changes.map((file) => (
            <button
              key={`${file.status}-${file.path}`}
              type="button"
              onClick={() => {
                setSelectedPath(file.path);
                setViewerMode("diff");
              }}
              className={`flex w-full items-center gap-2 rounded px-2 py-1 text-left tex
                selectedPath === file.path
                  ? "bg-blue-500/15 text-blue-200"
                  : "text-gray-300 hover:bg-gray-800 hover:text-gray-100"
            >

```

```

        }}
        title={file.path}
      >
        <span className="w-5 shrink-0 rounded bg-gray-800 px-1 text-center font-mono">
          {statusLabel(file)}
        </span>
        <span className="truncate">{file.path}</span>
      </button>
    )}}
  </div>
) : (
  <div className="p-3 text-xs text-gray-500">
    No uncommitted changes.
  </div>
)}
</div>
<div className="h-[42%] min-h-[180px] max-h-[50%]">
  <FileViewer
    workspaceId={workspaceId}
    transport={transport}
    path={selectedPath}
    mode={viewerMode}
    onModeChange={setViewerMode}
    onSaved={() => void refresh()}
  />
</div>
</div>

<div
  role="separator"
  aria-label="Resize terminal"
  onPointerDown={beginTerminalResize}
  className="h-1.5 cursor-row-resize border-y border-gray-800 bg-gray-800/80 hover:bg-gray-800/90"
/>
<div style={{ height: terminalHeight }} className="shrink-0">
  <RuntimeTerminalPane
    workspaceId={workspaceId}
    transport={transport}
    active={active}
  />
</div>
</aside>
);
}

```

23 Frontend Settings Components

Settings-specific components keep long forms and status panels out of the page container. They render cloud runners, imported Pi config, and release notes from the state hooks above.

23.1 components/CloudRunnerSection.tsx

Cloud runner settings present registration commands, storage profiles, heart-beat status, and revocation controls. The root chunk below tangles back to `frontend/src/components/CloudRunnerSection.tsx`.

```
import { useEffect, useState } from "react";

import {
  api,
  type CloudRunner,
  type CloudRunnerRegistration,
  type CloudRunnerStatus,
  type Repo,
  type RunnerStorageProfile,
} from "../api/client";
import { checkEncryptedRunnerHealth } from "../runner/encryptedRunnerClient";
import {
  importRunnerRepository,
  listRunnerRepositories,
} from "../runner/repositories";

type RunnerOptionId = "hosted" | RunnerStorageProfile;

type RunnerSetupOption = {
  id: RunnerOptionId;
  title: string;
  status: "Supported" | "Experimental";
  description: string;
  details: string[];
  warning?: string;
  runnerBacked: boolean;
};

const RUNNER_STORAGE_PROFILES: RunnerStorageProfile[] = [
  "aws_ebs_s3_files",
  "archil_shared_files",
  "archil_all_posix",
];
```

```

const RUNNER_OPTIONS: RunnerSetupOption[] = [
  {
    id: "hosted",
    title: "Hosted Yinshi",
    status: "Supported",
    description:
      "Use Yinshi exactly as it runs today with no cloud setup or runner token.",
    details: [
      "No AWS or Archil account required.",
      "Yinshi's hosted deployment owns the runtime storage.",
    ],
    runnerBacked: false,
  },
  {
    id: "aws_ebs_s3_files",
    title: "AWS BYOC: EBS plus S3 Files",
    status: "Supported",
    description:
      "Run sessions on user-owned AWS compute with active storage in the AWS account.",
    details: [
      "Live SQLite stays on encrypted runner EBS.",
      "Repos, worktrees, Pi config, sessions, and artifacts use S3 Files or local POSIX stor",
    ],
    runnerBacked: true,
  },
  {
    id: "archil_shared_files",
    title: "Archil shared-files mode",
    status: "Experimental",
    description:
      "Keep SQLite on runner EBS and place shared project files on Archil POSIX storage.",
    details: [
      "Requires a user Archil account or disk.",
      "Standard Archil uses Archil-managed active storage/cache even with a user-owned back",
    ],
    warning:
      "Experimental: Yinshi has not certified full Git and Pi workloads on Archil yet.",
    runnerBacked: true,
  },
  {
    id: "archil_all_posix",
    title: "Archil all-POSIX mode",
    status: "Experimental",
    description:
      "Put live SQLite and shared project files on one Archil POSIX tree.",
    details: [

```

```

        "Requires a user Archil account or disk.",
        "Standard Archil uses Archil-managed active storage/cache for active data.",
    ],
    warning:
        "Strong warning: live SQLite on Archil needs Yinshi-specific WAL, crash-recovery, and
runnerBacked: true,
    },
];

const STORAGE_LABELS: Record<string, string> = {
    archil: "Archil POSIX",
    archil_all_posix: "Archil all-POSIX",
    archil_shared_files: "Archil shared files",
    aws_ebs_s3_files: "AWS EBS plus S3 Files",
    local_posix: "Local POSIX",
    runner_ebs: "Runner EBS",
    s3_files_mount: "S3 Files mount",
    s3_files_or_local_posix: "S3 Files or local POSIX",
};

const RUNNER_STATUS_CLASSES: Record<CloudRunnerStatus, string> = {
    offline: "border-gray-600 bg-gray-800 text-gray-300",
    online: "border-emerald-700/60 bg-emerald-100/80 text-emerald-950",
    pending: "border-amber-700/60 bg-amber-100/80 text-amber-950",
    revoked: "border-red-700/60 bg-red-100/80 text-red-950",
};

const OPTION_STATUS_CLASSES: Record<RunnerSetupOption["status"], string> = {
    Experimental: "border-amber-700/60 bg-amber-100/80 text-amber-950",
    Supported: "border-emerald-700/60 bg-emerald-100/80 text-emerald-950",
};

function formatTimestamp(value: string | null): string {
    if (!value) {
        return "Never used";
    }
    return new Date(value).toLocaleString();
}

function runnerStatusClass(status: string): string {
    return (
        RUNNER_STATUS_CLASSES[status as CloudRunnerStatus] ??
        RUNNER_STATUS_CLASSES.offline
    );
}

```

```

function runnerEnvironmentText(registration: CloudRunnerRegistration): string {
    return Object.entries(registration.environment)
        .map(([key, value]) => `${key}=${value}`)
        .join("\n");
}

function errorMessage(error: unknown, fallback: string): string {
    return error instanceof Error ? error.message : fallback;
}

function runnerCapability(
    runner: CloudRunner,
    key: string,
    fallback: string,
): string {
    const value = runner.capabilities[key];
    return typeof value === "string" && value ? value : fallback;
}

function storageLabel(value: string): string {
    return STORAGE_LABELS[value] ?? value;
}

function isRunnerStorageProfile(value: unknown): value is RunnerStorageProfile {
    if (typeof value !== "string") {
        return false;
    }
    return RUNNER_STORAGE_PROFILES.includes(value as RunnerStorageProfile);
}

function optionForRunner(runner: CloudRunner | null): RunnerOptionId {
    if (!runner) {
        return "hosted";
    }
    if (runner.status === "revoked") {
        return "hosted";
    }

    const profile = runner.capabilities.storage_profile;
    if (isRunnerStorageProfile(profile)) {
        return profile;
    }
    return "aws_ebs_s3_files";
}

function optionById(id: RunnerOptionId): RunnerSetupOption {

```

```

    const option = RUNNER_OPTIONS.find((candidate) => candidate.id === id);
    if (!option) {
        throw new Error(`Unknown runner option: ${id}`);
    }
    return option;
}

function runnerProfileValue(
    optionId: RunnerOptionId,
): RunnerStorageProfile | null {
    if (optionId === "hosted") {
        return null;
    }
    return optionId;
}

function optionCardClass(isSelected: boolean): string {
    const baseClass =
        "block h-full cursor-pointer rounded-xl border bg-gray-900/70 p-4 transition focus-within:outline focus-within:outline-blue-500";
    if (isSelected) {
        return `${baseClass} border-blue-400/70 shadow-sm shadow-blue-900/20`;
    }
    return `${baseClass} border-gray-800 hover:border-gray-600`;
}

function CloudRunnerOptionCard({
    option,
    selectedOption,
    onSelect,
}): {
    option: RunnerSetupOption;
    selectedOption: RunnerOptionId;
    onSelect: (optionId: RunnerOptionId) => void;
} {
    const isSelected = option.id === selectedOption;
    const statusClass = OPTION_STATUS_CLASSES[option.status];

    return (
        <label className={optionCardClass(isSelected)}>
            <input
                type="radio"
                name="runner-storage-option"
                value={option.id}
                checked={isSelected}
                onChange={() => onSelect(option.id)}
                className="sr-only"
            />
        </label>
    );
}

```



```

    />
    <div className="flex items-start justify-between gap-3">
      <div>
        <div className="text-sm font-semibold text-gray-100">
          {option.title}
        </div>
        <div className="mt-1 text-xs text-gray-500">
          {option.runnerBacked ? "Runner-backed" : "No runner required"}
        </div>
      </div>
      <span
        className={`rounded-full border px-2 py-0.5 text-xs font-medium ${statusClass}`}
      >
        {option.status}
      </span>
    </div>
    <p className="mt-3 text-sm text-gray-300">{option.description}</p>
    <ul className="mt-3 space-y-1 text-xs text-gray-500">
      {option.details.map((detail) => (
        <li key={detail}>{detail}</li>
      ))}
    </ul>
    {option.warning ? (
      <p className="mt-3 rounded border border-amber-700/50 bg-amber-100/80 px-3 py-2 text-sm">
        {option.warning}
      </p>
    ) : null}
  </label>
);
}

export default function CloudRunnerSection() {
  const [runner, setRunner] = useState<CloudRunner | null>(null);
  const [registration, setRegistration] =
    useState<CloudRunnerRegistration | null>(null);
  const [selectedOption, setSelectedOption] =
    useState<RunnerOptionId>("hosted");
  const [name, setName] = useState("AWS runner");
  const [region, setRegion] = useState("us-east-1");
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [revoking, setRevoking] = useState(false);
  const [pairing, setPairing] = useState(false);
  const [checkingRunner, setCheckingRunner] = useState(false);
  const [runnerHealth, setRunnerHealth] = useState<string | null>(null);
  const [runnerRepositories, setRunnerRepositories] = useState<Repo[] | null>(

```

```

    null,
  );
  const [loadingRunnerRepositories, setLoadingRunnerRepositories] =
    useState(false);
  const [importingRunnerRepository, setImportingRunnerRepository] =
    useState(false);
  const [runnerRepositoryName, setRunnerRepositoryName] = useState("");
  const [runnerRepositoryUrl, setRunnerRepositoryUrl] = useState("");
  const [error, setError] = useState<string | null>(null);

  async function loadRunner() {
    setLoading(true);
    try {
      const loadedRunner = await api.get<CloudRunner | null>(
        "/api/settings/runner",
      );
      setRunner(loadedRunner);
      setSelectedOption(optionForRunner(loadedRunner));
      setError(null);
    } catch (loadError) {
      setError(errorMessage(loadError, "Failed to load cloud runner"));
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => {
    void loadRunner();
  }, []);

  useEffect(() => {
    setRunnerRepositories(null);
    setRunnerRepositoryName("");
    setRunnerRepositoryUrl("");
  }, [runner?.id]);

  async function createRunner() {
    const storageProfile = runnerProfileValue(selectedOption);
    if (!storageProfile) {
      setError("Hosted Yinshi does not need a runner registration token.");
      return;
    }

    const normalizedName = name.trim();
    const normalizedRegion = region.trim();
    if (!normalizedName || !normalizedRegion) {

```

```

        setError("Runner name and AWS region are required.");
        return;
    }

    setSaving(true);
    setError(null);
    try {
        const createdRegistration = await api.post<CloudRunnerRegistration>(
            "/api/settings/runner",
            {
                name: normalizedName,
                cloud_provider: "aws",
                region: normalizedRegion,
                storage_profile: storageProfile,
            },
        );
        setRegistration(createdRegistration);
        setRunner(createdRegistration.runner);
        setSelectedOption(optionForRunner(createdRegistration.runner));
    } catch (createError) {
        setError(errorMessage(createError, "Failed to create cloud runner"));
    } finally {
        setSaving(false);
    }
}

async function confirmRunnerIdentity() {
    if (!runner?.noise_public_key || !runner.noise_key_fingerprint) {
        setError("Runner identity is not available yet.");
        return;
    }
    setPairing(true);
    setError(null);
    try {
        const confirmedRunner = await api.post<CloudRunner>(
            "/api/settings/runner/noise-key/confirm",
            { noise_public_key: runner.noise_public_key },
        );
        setRunner(confirmedRunner);
    } catch (pairingError) {
        setError(errorMessage(pairingError, "Failed to pair runner identity"));
    } finally {
        setPairing(false);
    }
}

```

```

async function checkRunnerConnection() {
  if (!runner?.noise_public_key || !runner.noise_key_confirmed) {
    setError(
      "Pair the runner identity before testing the encrypted connection.",
    );
    return;
  }
  setCheckingRunner(true);
  setRunnerHealth(null);
  setError(null);
  try {
    await checkEncryptedRunnerHealth(runner.noise_public_key);
    setRunnerHealth("Encrypted runner connection is healthy.");
  } catch (healthError) {
    setError(errorMessage(healthError, "Encrypted runner connection failed"));
  } finally {
    setCheckingRunner(false);
  }
}

async function loadRunnerRepositories() {
  if (!runner?.noise_public_key || !runner.noise_key_confirmed) {
    setError("Pair the runner identity before loading BYOC repositories.");
    return;
  }
  setLoadingRunnerRepositories(true);
  setError(null);
  try {
    setRunnerRepositories(
      await listRunnerRepositories({
        runnerId: runner.id,
        runnerPublicKey: runner.noise_public_key,
      }),
    );
  } catch (repositoryError) {
    setError(
      errorMessage(repositoryError, "Failed to load BYOC repositories"),
    );
  } finally {
    setLoadingRunnerRepositories(false);
  }
}

async function importRepositoryToRunner() {
  if (!runner?.noise_public_key || !runner.noise_key_confirmed) {
    setError("Pair the runner identity before importing a BYOC repository.");
  }
}

```

```

        return;
    }
    setImportingRunnerRepository(true);
    setError(null);
    try {
        const importedRepository = await importRunnerRepository(
            {
                runnerId: runner.id,
                runnerPublicKey: runner.noise_public_key,
            },
            runnerRepositoryName,
            runnerRepositoryUrl,
        );
        setRunnerRepositories((currentRepositories) => {
            const repositories = currentRepositories ?? [];
            return [
                importedRepository,
                ...repositories.filter(
                    (repository) => repository.id !== importedRepository.id,
                ),
            ];
        });
        setRunnerRepositoryName("");
        setRunnerRepositoryUrl("");
    } catch (repositoryError) {
        setError(
            errorMessage(repositoryError, "Failed to import BYOC repository"),
        );
    } finally {
        setImportingRunnerRepository(false);
    }
}

async function revokeRunner() {
    if (!runner) {
        return;
    }
    setRevoking(true);
    setError(null);
    try {
        await api.delete("/api/settings/runner");
        setRegistration(null);
        await loadRunner();
    } catch (revokeError) {
        setError(errorMessage(revokeError, "Failed to revoke cloud runner"));
    } finally {

```

```

        setRevoking(false);
    }
}

const status = runner?.status ?? "offline";
const createButtonLabel =
    runner && runner.status !== "revoked" ? "Replace Runner" : "Create Token";
const statusLabel = loading
    ? "Loading"
    : (runner?.status ?? "Not configured");
const selectedRunnerProfile = runnerProfileValue(selectedOption);
const selectedOptionDetails = optionById(selectedOption);

return (
    <section className="mb-8 rounded-xl border border-gray-800 bg-gray-900/60 p-5">
        <div className="flex flex-wrap items-start justify-between gap-4">
            <div>
                <h2 className="text-lg font-semibold text-gray-200">Cloud Runner</h2>
                <p className="mt-2 max-w-3xl text-sm text-gray-400">
                    Choose hosted Yinshi, supported AWS BYOC storage, or experimental
                    Archil-backed POSIX storage. Archil options require the user to sign
                    up for Archil and accept Archil-managed active storage/cache.
                </p>
            </div>
            <span
                className={`rounded-full border px-3 py-1 text-xs ${runnerStatusClass(status)} `}
            >
                {statusLabel}
            </span>
        </div>

        <fieldset className="mt-5">
            <legend className="text-sm font-medium text-gray-200">
                Storage option
            </legend>
            <div className="mt-3 grid gap-3 lg:grid-cols-2">
                {RUNNER_OPTIONS.map((option) => (
                    <CloudRunnerOptionCard
                        key={option.id}
                        option={option}
                        selectedOption={selectedOption}
                        onSelect={setSelectedOption}
                    />
                ))}
            </div>
        </fieldset>
    </section>

```

```

{runner ? (
  <div className="mt-5 grid gap-3 text-sm text-gray-400 md:grid-cols-3 lg:grid-cols-6">
    <div>
      <div className="text-xs uppercase tracking-wide text-gray-500">
        Runner
      </div>
      <div className="mt-1 text-gray-200">{runner.name}</div>
    </div>
    <div>
      <div className="text-xs uppercase tracking-wide text-gray-500">
        Region
      </div>
      <div className="mt-1 text-gray-200">{runner.region}</div>
    </div>
    <div>
      <div className="text-xs uppercase tracking-wide text-gray-500">
        Profile
      </div>
      <div className="mt-1 text-gray-200">
        {storageLabel(
          runnerCapability(runner, "storage_profile", "aws_ebs_s3_files"),
        )}
      </div>
    </div>
    <div>
      <div className="text-xs uppercase tracking-wide text-gray-500">
        Last heartbeat
      </div>
      <div className="mt-1 text-gray-200">
        {formatTimestamp(runner.last_heartbeat_at)}
      </div>
    </div>
    <div>
      <div className="text-xs uppercase tracking-wide text-gray-500">
        SQLite
      </div>
      <div className="mt-1 text-gray-200">
        {storageLabel(
          runnerCapability(runner, "sqlite_storage", "Runner EBS"),
        )}
      </div>
    </div>
    <div>
      <div className="text-xs uppercase tracking-wide text-gray-500">
        Shared files

```

```

</div>
<div className="mt-1 text-gray-200">
  {storageLabel(
    runnerCapability(
      runner,
      "shared_files_storage",
      "S3 Files ready",
    ),
  )}
</div>
</div>
) : null}

{runner?.noise_key_fingerprint ? (
  <div className="mt-5 rounded-lg border border-gray-700 bg-gray-950/70 p-4">
    <div className="flex flex-wrap items-start justify-between gap-3">
      <div>
        <h3 className="text-sm font-semibold text-gray-100">
          Runner encryption identity
        </h3>
        <p className="mt-1 max-w-2xl text-sm text-gray-400">
          Compare this full fingerprint with the value printed by the
          runner service. Confirm only when every character matches.
        </p>
      </div>
      <span
        className={`rounded-full border px-2.5 py-1 text-xs font-medium ${
          runner.noise_key_confirmed
            ? "border-emerald-700/60 bg-emerald-100/80 text-emerald-950"
            : "border-amber-700/60 bg-amber-100/80 text-amber-950"
        }`}
      >
        {runner.noise_key_confirmed
          ? "Runner identity paired"
          : "Pairing required"}
      </span>
    </div>
    <code className="mt-3 block break-all rounded border border-gray-800 bg-gray-900 p-4">
      {runner.noise_key_fingerprint}
    </code>
    {runner.noise_key_confirmed ? (
      <div className="mt-3 flex flex-wrap items-center justify-between gap-3">
        <div>
          <p className="text-xs text-emerald-300">
            New encrypted sessions are pinned to this runner identity.

```



```

    </p>
    {runnerHealth ? (
      <p className="mt-1 text-xs text-gray-300">{runnerHealth}</p>
    ) : null}
  </div>
  <button
    type="button"
    onClick={() => {
      void checkRunnerConnection();
    }}
    disabled={checkingRunner}
    className="rounded border border-gray-700 px-3 py-2 text-sm text-gray-200 h-10"
  >
    {checkingRunner ? "Testing..." : "Test encrypted connection"}
  </button>
</div>
) : (
  <div className="mt-3 flex flex-wrap items-center justify-between gap-3">
    <p className="text-xs text-amber-200">
      Encrypted sessions are blocked until pairing is complete.
    </p>
    <button
      type="button"
      onClick={() => {
        void confirmRunnerIdentity();
      }}
      disabled={pairing}
      className="btn-primary px-3 py-2 text-sm disabled:opacity-50"
    >
      {pairing ? "Pairing..." : "I verified this fingerprint"}
    </button>
  </div>
  </div>
) : null}

{runner?.noise_key_confirmed && runner.noise_public_key ? (
  <RunnerRepositoryPanel
    repositories={runnerRepositories}
    loading={loadingRunnerRepositories}
    importing={importingRunnerRepository}
    repositoryName={runnerRepositoryName}
    repositoryUrl={runnerRepositoryUrl}
    onLoad={() => {
      void loadRunnerRepositories();
    }}
  >

```

```

        onNameChange={setRunnerRepositoryName}
        onUrlChange={setRunnerRepositoryUrl}
        onImport={() => {
            void importRepositoryToRunner();
        }}
    />
) : null}

{selectedRunnerProfile ? (
    <div className="mt-5 rounded-lg border border-gray-800 bg-gray-950/50 p-4">
        <div className="mb-3">
            <h3 className="text-sm font-semibold text-gray-100">
                Create a runner token for {selectedOptionDetails.title}
            </h3>
            <p className="mt-1 text-sm text-gray-400">
                The runner starts on AWS EC2. The selected storage profile
                controls validation, default paths, and advertised capabilities.
            </p>
        </div>
        <div className="grid gap-3 md:grid-cols-[1fr_180px_auto]">
            <input
                type="text"
                value={name}
                onChange={(event) => setName(event.target.value)}
                placeholder="Runner name"
                className="rounded border border-gray-700 bg-gray-800 px-3 py-2 text-sm text-g
            />
            <input
                type="text"
                value={region}
                onChange={(event) => setRegion(event.target.value)}
                placeholder="AWS region"
                className="rounded border border-gray-700 bg-gray-800 px-3 py-2 text-sm text-g
            />
            <button
                type="button"
                onClick={() => {
                    void createRunner();
                }}
                disabled={saving}
                className="btn-primary px-4 py-2 text-sm disabled:opacity-50"
            >
                {saving ? "Creating..." : createButtonLabel}
            </button>
        </div>
    </div>
</div>

```

```

) : (
  <div className="mt-5 rounded-lg border border-gray-800 bg-gray-950/50 p-4 text-sm text-gray-100">
    Hosted Yinshi does not create a runner record or registration token.
    Select a runner-backed option only when you want user-owned AWS
    compute.
  </div>
)}

{registration ? (
  <div className="mt-5 rounded-lg border border-blue-500/30 bg-blue-500/10 p-4">
    <h3 className="text-sm font-semibold text-blue-100">
      One-time registration values
    </h3>
    <p className="mt-1 text-sm text-blue-100/80">
      Store these in the AWS CloudFormation template or runner environment
      now. The token expires at{" "}
      {formatTimestamp(registration.registration_token_expires_at)}.
    </p>
    <textarea
      readOnly
      value={runnerEnvironmentText(registration)}
      rows={8}
      className="mt-3 w-full rounded border border-blue-400/30 bg-gray-950 px-3 py-2 text-gray-100"
    />
    <p className="mt-2 text-xs text-blue-100/70">
      Use docs/deployment/aws-runner-cloudformation.yaml and
      docs/deployment/runner-storage-options.md to launch the EC2 runner.
    </p>
  </div>
) : null}

<div className="mt-4 flex flex-wrap items-center justify-between gap-3">
  <p className="text-xs text-gray-500">
    Registration tokens are shown once. Runner bearer tokens are hashed
    before storage.
  </p>
  {runner && runner.status !== "revoked" ? (
    <button
      type="button"
      onClick={() => {
        void revokeRunner();
      }}
      disabled={revoking}
      className="text-sm text-red-400 hover:text-red-300 disabled:opacity-50"
    >
      {revoking ? "Revoking..." : "Revoke Runner"}
    </button>
  ) : null}
</div>

```

```

        </button>
      ) : null}
    </div>

    {error ? <p className="mt-3 text-sm text-red-400">{error}</p> : null}
  </section>
);
}

function RunnerRepositoryPanel({
  repositories,
  loading,
  importing,
  repositoryName,
  repositoryUrl,
  onLoad,
  onNameChange,
  onUrlChange,
  onImport,
}): {
  repositories: Repo[] | null;
  loading: boolean;
  importing: boolean;
  repositoryName: string;
  repositoryUrl: string;
  onLoad: () => void;
  onNameChange: (value: string) => void;
  onUrlChange: (value: string) => void;
  onImport: () => void;
}) {
  return (
    <div className="mt-5 rounded-lg border border-gray-700 bg-gray-950/70 p-4">
      <div className="flex flex-wrap items-start justify-between gap-3">
        <div>
          <h3 className="text-sm font-semibold text-gray-100">
            BYOC repositories
          </h3>
          <p className="mt-1 text-sm text-gray-400">
            Repository commands and responses stay inside the encrypted runner
            channel.
          </p>
        </div>
        <button
          type="button"
          onClick={onLoad}
          disabled={loading}

```

```

        className="rounded border border-gray-700 px-3 py-2 text-sm text-gray-200 hover:bo
    >
        {loading ? "Loading..." : "Load BYOC repositories"}
    </button>
</div>

{repositories !== null ? (
    <div className="mt-4 space-y-2">
        {repositories.length === 0 ? (
            <p className="text-sm text-gray-500">
                No repositories on this runner.
            </p>
        ) : (
            repositories.map((repository) => (
                <div
                    key={repository.id}
                    className="flex items-center justify-between rounded border border-gray-800
                >
                    <span className="text-sm font-medium text-gray-200">
                        {repository.name}
                    </span>
                    <span className="rounded border border-gray-700 px-2 py-0.5 text-[11px] font
                        BYOC
                    </span>
                </div>
            ))
        )}
    </div>
) : null}

<div className="mt-4 grid gap-3 border-t border-gray-800 pt-4 md:grid-cols-[minmax(0,0
    <label className="block text-sm text-gray-300">
        Repository name
        <input
            aria-label="Repository name"
            value={repositoryName}
            onChange={(event) => onNameChange(event.target.value)}
            className="input mt-1 w-full"
            placeholder="project"
            autoComplete="off"
        />
    </label>
    <label className="block text-sm text-gray-300">
        Repository HTTPS URL
        <input
            aria-label="Repository HTTPS URL"

```

```

        value={repositoryUrl}
        onChange={(event) => onUrlChange(event.target.value)}
        className="input mt-1 w-full"
        placeholder="https://example.com/team/project.git"
        autoComplete="off"
      />
    </label>
    <button
      type="button"
      onClick={onImport}
      disabled={
        importing || !repositoryName.trim() || !repositoryUrl.trim()
      }
      className="btn-primary px-3 py-2 text-sm disabled:opacity-50"
    >
      {importing ? "Importing..." : "Import to BYOC"}
    </button>
  </div>
</div>
);
}

```

23.2 components/PiConfigSection.tsx

Pi config settings show imported categories, upload state, GitHub import, and enablement controls. The root chunk below tangles back to frontend/src/components/PiConfigSection.tsx.

```

import { useState, type FormEvent } from "react";

import { usePiConfig } from "../hooks/usePiConfig";
import type { RuntimeTransport } from "../runtime/runtimeTransport";

const CATEGORY_LABELS: Record<string, string> = {
  skills: "Skills",
  extensions: "Extensions",
  prompts: "Prompts",
  agents: "Agents",
  themes: "Themes",
  settings: "Settings",
  models: "Models",
  sessions: "Sessions",
  instructions: "Instructions",
};

function getCategoryLabel(category: string): string {

```

```

    return CATEGORY_LABELS[category] ?? category;
}

function formatTimestamp(timestamp: string | null): string {
    if (!timestamp) {
        return "Never";
    }
    return new Date(timestamp).toLocaleString();
}

export default function PiConfigSection({
    transport,
}): {
    transport: RuntimeTransport;
}) {
    const {
        config,
        loading,
        error,
        importing,
        syncing,
        updatingCategories,
        importFromGithub,
        importFromUpload,
        syncConfig,
        removeConfig,
        toggleCategory,
    } = usePiConfig(transport);
    const [mode, setMode] = useState<"upload" | "github">("upload");
    const [repoUrl, setRepoUrl] = useState("");
    const [selectedFile, setSelectedFile] = useState<File | null>(null);

    async function handleGithubImport(event: FormEvent) {
        event.preventDefault();
        if (!repoUrl.trim()) {
            return;
        }
        const success = await importFromGithub(repoUrl);
        if (success) {
            setRepoUrl("");
        }
    }

    async function handleUpload(event: FormEvent) {
        event.preventDefault();
        if (!selectedFile) {

```

```

    return;
  }
  const success = await importFromUpload(selectedFile);
  if (success) {
    setSelectedFile(null);
  }
}

async function handleRetry() {
  if (!config || config.source_type !== "github") {
    return;
  }
  await syncConfig();
}

return (
  <section className="mt-8">
    <h2 className="mb-4 text-lg font-semibold text-gray-200">
      Pi Agent Configuration
    </h2>
    <p className="mb-4 text-sm text-gray-400">
      Bring your own Pi agent setup with imported skills, prompts,
      extensions, and settings.
    </p>

    {loading ? (
      <div className="rounded border border-gray-700 bg-gray-800 px-4 py-3 text-sm text-gray-400">
        Loading Pi config...
      </div>
    ) : null}

    {!loading && config?.status === "cloning" ? (
      <div className="rounded border border-gray-700 bg-gray-800 px-4 py-4">
        <p className="text-sm text-gray-300">
          Importing from: {config.source_label}
        </p>
        <p className="mt-2 text-sm text-gray-400">Cloning repository...</p>
      </div>
    ) : null}

    {!loading && config?.status === "error" ? (
      <div className="rounded border border-red-900/50 bg-gray-800 px-4 py-4">
        <p className="text-sm text-red-400">
          Import failed: {config.error_message || "Unknown error"}
        </p>
        <div className="mt-3 flex gap-3">

```



```

{config.source_type === "github" ? (
  <button
    type="button"
    onClick={handleRetry}
    disabled={syncing}
    className="rounded border border-gray-600 px-3 py-2 text-sm text-gray-200 d
  >
    {syncing ? "Retrying..." : "Retry"}
  </button>
) : null}
<button
  type="button"
  onClick={() => {
    void removeConfig();
  }}
  className="text-sm text-red-400 hover:text-red-300"
>
  Remove
</button>
</div>
</div>
) : null}

{!loading && config && config.status === "ready" ? (
  <div className="rounded border border-gray-700 bg-gray-800 px-4 py-4">
    <div className="flex flex-wrap items-center justify-between gap-3">
      <div>
        <p className="text-sm text-gray-300">
          Imported from: {config.source_label}
        </p>
        {config.source_type === "github" ? (
          <p className="mt-1 text-xs text-gray-500">
            Last synced: {formatTimestamp(config.last_synced_at)}
          </p>
        ) : null}
      </div>
      <div className="flex gap-3">
        {config.source_type === "github" ? (
          <button
            type="button"
            onClick={() => {
              void syncConfig();
            }}
            disabled={syncing}
            className="rounded border border-gray-600 px-3 py-2 text-sm text-gray-200
          >

```

```

        {syncing ? "Syncing..." : "Sync"}
      </button>
    ) : null}
    <button
      type="button"
      onClick={() => {
        void removeConfig();
      }}
      className="text-sm text-red-400 hover:text-red-300"
    >
      Remove
    </button>
  </div>
</div>

<div className="mt-4 grid gap-3 md:grid-cols-2">
  {config.available_categories.map((category) => (
    <label
      key={category}
      className="flex items-center justify-between rounded border border-gray-700"
    >
      <span className="text-sm text-gray-200">
        {getCategoryLabel(category)}
      </span>
      <input
        type="checkbox"
        checked={config.enabled_categories.includes(category)}
        disabled={updatingCategories}
        onChange={(event) => {
          void toggleCategory(category, event.target.checked);
        }}
      />
    </label>
  ))}
</div>
</div>
) : null}

{!loading && !config ? (
  <div className="rounded border border-gray-700 bg-gray-800 px-4 py-4">
    <div className="mb-4 flex gap-2">
      <button
        type="button"
        onClick={() => setMode("upload")}
        className={`rounded px-3 py-2 text-sm ${
          mode === "upload"

```

```

      ? "bg-gray-200 text-gray-900"
      : "border border-gray-700 text-gray-300"
    }`}
  >
    Upload
  </button>
  <button
    type="button"
    onClick={() => setMode("github")}
    className={`rounded px-3 py-2 text-sm ${
      mode === "github"
        ? "bg-gray-200 text-gray-900"
        : "border border-gray-700 text-gray-300"
    }`}
  >
    GitHub
  </button>
</div>

{mode === "upload" ? (
  <form onSubmit={handleUpload} className="space-y-3">
    <input
      type="file"
      accept=".zip"
      onChange={(event) => {
        setSelectedFile(event.target.files?.[0] ?? null);
      }}
      className="block w-full text-sm text-gray-300 file:mr-4 file:rounded file:b
    />
    <p className="text-xs text-gray-500">
      Upload a zip of your `.pi` directory or a partial archive rooted
      like `.pi`.
    </p>
    <button
      type="submit"
      disabled={!importing || !selectedFile}
      className="btn-primary px-4 py-2 text-sm disabled:opacity-50"
    >
      {importing ? "Uploading..." : "Upload Pi Config"}
    </button>
  </form>
) : (
  <form onSubmit={handleGithubImport} className="space-y-3">
    <div className="flex gap-3">
      <input
        type="text"

```

```

        value={repoUrl}
        onChange={(event) => setRepoUrl(event.target.value)}
        placeholder="owner/repo or https://github.com/owner/repo"
        className="flex-1 rounded border border-gray-700 bg-gray-900 px-3 py-2 text-white"
      />
      <button
        type="submit"
        disabled={importing || !repoUrl.trim()}
        className="btn-primary px-4 py-2 text-sm disabled:opacity-50"
      >
        {importing ? "Importing..." : "Import"}
      </button>
    </div>
    <p className="text-xs text-gray-500">
      Import a Pi config from a GitHub repository.
    </p>
  </form>
)}
</div>
) : null}

{error ? <p className="mt-3 text-sm text-red-400">{error}</p> : null}
</section>
);
}

```

23.3 components/PiReleaseNotesSection.tsx

Release-note cards compare bundled runtime version with available Pi releases.
 The root chunk below tangles back to frontend/src/components/PiReleaseNotesSection.tsx.

```

import ReactMarkdown from "react-markdown";
import remarkGfm from "remark-gfm";

import { type PiPackageRelease, type PiReleaseNotes } from "../api/client";
import { usePiReleaseNotes } from "../hooks/usePiReleaseNotes";
import type { RuntimeTransport } from "../runtime/runtimeTransport";

function formatDate(timestamp: string | null): string {
  if (!timestamp) {
    return "Unknown date";
  }
  return new Date(timestamp).toLocaleDateString(undefined, {
    year: "numeric",
    month: "short",
    day: "numeric",
  });
}

```

```

    });
  }

function formatVersion(version: string | null): string {
  return version || "Unknown";
}

function buildVersionStatus(releaseNotes: PiReleaseNotes): { text: string; tone: "current" | "warning" } {
  if (!releaseNotes.installed_version) {
    return { text: "Runtime version unavailable", tone: "warning" };
  }
  if (!releaseNotes.latest_version) {
    return { text: "Latest version unavailable", tone: "warning" };
  }
  if (releaseNotes.installed_version === releaseNotes.latest_version) {
    return { text: "Up to date", tone: "current" };
  }
  return { text: "Update available", tone: "warning" };
}

function ReleaseMarkdown({ body }: { body: string }) {
  if (!body.trim()) {
    return <p className="text-sm text-gray-500">No release notes published.</p>;
  }
  return (
    <div className="markdown-prose text-sm text-gray-300">
      <ReactMarkdown
        remarkPlugins={[remarkGfm]}
        components={{
          a: ({ href, children }) => (
            <a href={href} target="_blank" rel="noopener noreferrer">
              {children}
            </a>
          ),
        }}
      />
      {body}
    </ReactMarkdown>
  </div>
  );
}

function ReleaseCard({ release }: { release: PiPackageRelease }) {
  return (
    <article className="rounded-xl border border-gray-800 bg-gray-900/70 p-4">
      <div className="mb-3 flex flex-wrap items-start justify-between gap-3">

```

```

    <div>
      <h3 className="text-base font-semibold text-gray-100">{release.name}</h3>
      <p className="mt-1 text-xs text-gray-500">
        {release.tag_name} · {formatDate(release.published_at)}
      </p>
    </div>
    <a
      href={release.html_url}
      target="_blank"
      rel="noopener noreferrer"
      className="text-xs text-blue-400 hover:text-blue-300"
    >
      View on GitHub
    </a>
  </div>
  <ReleaseMarkdown body={release.body_markdown} />
</article>
);
}

function RuntimeSummary({ releaseNotes }: { releaseNotes: PiReleaseNotes }) {
  const status = buildVersionStatus(releaseNotes);
  const statusClassName = status.tone === "current" ? "text-green-400" : "text-amber-300";
  return (
    <div className="grid gap-3 md:grid-cols-3">
      <div className="rounded-xl border border-gray-800 bg-gray-900/70 p-4">
        <div className="text-xs uppercase tracking-wide text-gray-500">Installed</div>
        <div className="mt-2 font-mono text-lg text-gray-100">
          {formatVersion(releaseNotes.installed_version)}
        </div>
        <div>
          {releaseNotes.node_version ? (
            <div className="mt-1 text-xs text-gray-500">Node {releaseNotes.node_version}</div>
          ) : null}
        </div>
      </div>
      <div className="rounded-xl border border-gray-800 bg-gray-900/70 p-4">
        <div className="text-xs uppercase tracking-wide text-gray-500">Latest</div>
        <div className="mt-2 font-mono text-lg text-gray-100">
          {formatVersion(releaseNotes.latest_version)}
        </div>
        <div className={`mt-1 text-xs ${statusClassName}`}>{status.text}</div>
      </div>
      <div className="rounded-xl border border-gray-800 bg-gray-900/70 p-4">
        <div className="text-xs uppercase tracking-wide text-gray-500">Update policy</div>
        <div className="mt-2 text-sm text-gray-200">{releaseNotes.update_policy}</div>
      </div>
    </div>
  );
}

```

```

    );
}

export default function PiReleaseNotesSection({
  transport,
}: {
  transport: RuntimeTransport;
}) {
  const { releaseNotes, loading, error, refresh } = usePiReleaseNotes(transport);

  return (
    <section className="space-y-4" aria-labelledby="pi-release-notes-heading">
      <div className="flex flex-wrap items-start justify-between gap-3">
        <div>
          <h2 id="pi-release-notes-heading" className="text-lg font-semibold text-gray-200">
            Pi release notes
          </h2>
          <p className="mt-1 text-sm text-gray-400">
            Yinshi updates the bundled pi package through reviewed lockfile changes and immu
          </p>
          {releaseNotes ? (
            <a
              href={releaseNotes.release_notes_url}
              target="_blank"
              rel="noopener noreferrer"
              className="mt-2 inline-flex text-sm text-blue-400 hover:text-blue-300"
            >
              View all pi releases
            </a>
          ) : null}
        </div>
        <button
          type="button"
          onClick={() => {
            void refresh();
          }}
          disabled={loading}
          className="rounded border border-gray-700 px-3 py-2 text-sm text-gray-200 hover:bg-
        >
          {loading ? "Refreshing..." : "Refresh"}
        </button>
      </div>

      {loading && !releaseNotes ? (
        <div className="rounded border border-gray-700 bg-gray-800 px-4 py-3 text-sm text-gr
          Loading pi release notes...

```

```

    </div>
  ) : null}

  {error ? (
    <div className="rounded border border-red-900/50 bg-gray-800 px-4 py-3 text-sm text-red-500">
      {error}
    </div>
  ) : null}

  {releaseNotes ? (
    <>
      <RuntimeSummary releaseNotes={releaseNotes} />
      {releaseNotes.runtime_error ? (
        <div className="rounded border border-amber-900/50 bg-amber-950/20 px-4 py-3 text-sm text-amber-500">
          Runtime version unavailable: {releaseNotes.runtime_error}
        </div>
      ) : null}
      {releaseNotes.release_error ? (
        <div className="rounded border border-amber-900/50 bg-amber-950/20 px-4 py-3 text-sm text-amber-500">
          Showing cached release notes. Latest fetch failed: {releaseNotes.release_error}
        </div>
      ) : null}
      <div className="space-y-4">
        {releaseNotes.releases.map((release) => (
          <ReleaseCard key={release.tag_name} release={release} />
        ))}
        {releaseNotes.releases.length === 0 && !loading ? (
          <div className="rounded border border-gray-800 bg-gray-900/70 px-4 py-3 text-sm text-gray-500">
            No release notes are available yet.
          </div>
        ) : null}
      </div>
    </>
  ) : null}
</section>
);
}

```

24 Encrypted Browser Runtime

Runtime selection is explicit in the browser. Noise IK protects hosted RPC messages. Transport modules keep repository, prompt, upload, resource, and terminal behavior consistent across hosted and desktop execution.

24.1 crypto/noiseIk.ts

The browser Noise implementation creates IK handshakes and encrypts ordered messages with runner identity pinning.

```
import noiseWasmUrl from "@richardhopton/noise-c.wasm/src/noise-c.wasm?url";
import type {
  NoiseCipherState,
  NoiseHandshakeState,
  NoiseLibrary,
} from "@richardhopton/noise-c.wasm";

const NOISE_PROTOCOL_NAME = "Noise_IK_25519_ChaChaPoly_SHA256";
const X25519_KEY_LENGTH = 32;
const NOISE_TAG_LENGTH = 16;
const NOISE_MESSAGE_MAX_LENGTH = 65_535;
const IK_FIRST_MESSAGE_OVERHEAD = 96;
const IK_SECOND_MESSAGE_MIN_LENGTH = 48;
const TRANSPORT_PLAINTEXT_MAX_LENGTH = NOISE_MESSAGE_MAX_LENGTH - NOISE_TAG_LENGTH;
const TRANSPORT_MESSAGES_BEFORE_REHANDSHAKE = 1_048_576;
const WASM_LOAD_TIMEOUT_MS = 15_000;
const emptyBytes = new Uint8Array();

let libraryPromise: Promise<NoiseLibrary> | undefined;

export interface NoiseIkKeypair {
  readonly privateKey: Uint8Array;
  readonly publicKey: Uint8Array;
}

export interface NoiseIkInitiatorOptions {
  readonly staticPrivateKey: Uint8Array;
  readonly responderStaticPublicKey: Uint8Array;
  readonly prologue: Uint8Array;
}

export interface NoiseIkInitiator {
  readonly handshakeHash: Uint8Array;
  writeHandshakeMessage(payload: Uint8Array): Uint8Array;
  readHandshakeMessage(message: Uint8Array): Uint8Array;
  encrypt(plaintext: Uint8Array): Uint8Array;
  decrypt(ciphertext: Uint8Array): Uint8Array;
  dispose(): void;
}

function copyExact(value: Uint8Array, length: number, name: string): Uint8Array {
```

```

    if (!(value instanceof Uint8Array) || value.length !== length) {
        throw new TypeError(`${name} must contain exactly ${length} bytes`);
    }
    return Uint8Array.from(value);
}

function copyBytes(value: Uint8Array, name: string): Uint8Array {
    if (!(value instanceof Uint8Array)) {
        throw new TypeError(`${name} must be Uint8Array`);
    }
    return Uint8Array.from(value);
}

async function loadNoiseLibrary(): Promise<NoiseLibrary> {
    if (libraryPromise !== undefined) {
        return libraryPromise;
    }
    libraryPromise = import("@richardhopton/noise-c.wasm").then(
        ({ default: createNoise }) => {
            new Promise<NoiseLibrary>((resolve, reject) => {
                const timeout = globalThis.setTimeout(() => {
                    reject(new Error("Noise cryptography module did not load in time"));
                }, WASM_LOAD_TIMEOUT_MS);
                try {
                    const processValue = (
                        globalThis as typeof globalThis & {
                            process?: { versions?: { node?: string } }
                        }
                    ).process;
                    const moduleOptions =
                        typeof processValue?.versions?.node === "string"
                            ? {}
                            : {
                                locateFile: (path: string) => {
                                    if (path !== "noise-c.wasm") {
                                        throw new Error("Noise cryptography requested an unknown runtime asset");
                                    }
                                    return noiseWasmUrl;
                                },
                            };
                    createNoise(
                        moduleOptions,
                        (library) => {
                            globalThis.clearTimeout(timeout);
                            resolve(library);
                        },
                    );
                } catch {
                    reject(new Error("Noise cryptography module failed to load"));
                }
            });
        }
    );
}

```

```

    );
    } catch (error) {
        globalThis.clearTimeout(timeout);
        reject(error);
    }
    }),
    );
    return libraryPromise;
}

class NoiseIkInitiatorState implements NoiseIkInitiator {
    readonly #handshake: NoiseHandshakeState;
    #handshakeHashValue: Uint8Array | undefined;
    #sendCipher: NoiseCipherState | undefined;
    #receiveCipher: NoiseCipherState | undefined;
    #firstMessageWritten = false;
    #failed = false;
    #disposed = false;
    #messagesSent = 0;
    #messagesReceived = 0;

    constructor(library: NoiseLibrary, options: NoiseIkInitiatorOptions) {
        const staticPrivateKey = copyExact(
            options.staticPrivateKey,
            X25519_KEY_LENGTH,
            "staticPrivateKey",
        );
        const responderStaticPublicKey = copyExact(
            options.responderStaticPublicKey,
            X25519_KEY_LENGTH,
            "responderStaticPublicKey",
        );
        const prologue = copyBytes(options.prologue, "prologue");
        if (prologue.length > NOISE_MESSAGE_MAX_LENGTH) {
            throw new RangeError("Noise IK prologue is too large");
        }

        const handshake = new library.HandshakeState(
            NOISE_PROTOCOL_NAME,
            library.constants.NOISE_ROLE_INITIATOR,
        );
        try {
            handshake.Initialize(prologue, staticPrivateKey, responderStaticPublicKey);
        } catch (error) {
            handshake.free();
            throw error;
        }
    }
}

```

```

    } finally {
        staticPrivateKey.fill(0);
    }
    this.#handshake = handshake;
}

get handshakeHash(): Uint8Array {
    this.#requireUsable();
    if (this.#handshakeHashValue === undefined) {
        throw new Error("Noise IK handshake is not complete");
    }
    return Uint8Array.from(this.#handshakeHashValue);
}

writeHandshakeMessage(payloadValue: Uint8Array): Uint8Array {
    this.#requireUsable();
    if (this.#firstMessageWritten) {
        throw new Error("Noise IK initiator handshake message was already written");
    }
    const payload = copyBytes(payloadValue, "payload");
    if (payload.length > NOISE_MESSAGE_MAX_LENGTH - IK_FIRST_MESSAGE_OVERHEAD) {
        throw new RangeError("Noise IK initiator payload is too large");
    }
    try {
        const message = this.#handshake.WriteMessage(payload);
        if (message.length < IK_FIRST_MESSAGE_OVERHEAD || message.length > NOISE_MESSAGE_MAX_LENGTH) {
            throw new Error("Noise IK library produced an invalid initiator message");
        }
        this.#firstMessageWritten = true;
        return Uint8Array.from(message);
    } catch (error) {
        this.#failed = true;
        throw error;
    }
}

readHandshakeMessage(messageValue: Uint8Array): Uint8Array {
    this.#requireUsable();
    if (!this.#firstMessageWritten || this.#handshakeHashValue !== undefined) {
        throw new Error("Noise IK initiator is not ready for the responder message");
    }
    const message = copyBytes(messageValue, "message");
    if (message.length < IK_SECOND_MESSAGE_MIN_LENGTH || message.length > NOISE_MESSAGE_MAX_LENGTH) {
        throw new RangeError("Noise IK responder message has an invalid length");
    }
    try {

```

```

        const payload = this.#handshake.ReadMessage(message, true);
        if (payload === null) {
            throw new Error("Noise IK responder payload was not returned");
        }
        this.#handshakeHashValue = Uint8Array.from(this.#handshake.GetHandshakeHash());
        [this.#sendCipher, this.#receiveCipher] = this.#handshake.Split();
        return Uint8Array.from(payload);
    } catch (error) {
        this.#failed = true;
        throw error;
    }
}

encrypt(plaintextValue: Uint8Array): Uint8Array {
    const cipher = this.#transportCipher("send");
    const plaintext = copyBytes(plaintextValue, "plaintext");
    if (plaintext.length > TRANSPORT_PLAINTEXT_MAX_LENGTH) {
        throw new RangeError("Noise transport plaintext is too large");
    }
    if (this.#messagesSent >= TRANSPORT_MESSAGES_BEFORE_REHANDSHAKE) {
        throw new Error("Noise transport requires a fresh handshake");
    }
    try {
        const ciphertext = cipher.EncryptWithAd(emptyBytes, plaintext);
        this.#messagesSent += 1;
        return Uint8Array.from(ciphertext);
    } catch (error) {
        this.#failed = true;
        throw error;
    }
}

decrypt(ciphertextValue: Uint8Array): Uint8Array {
    const cipher = this.#transportCipher("receive");
    const ciphertext = copyBytes(ciphertextValue, "ciphertext");
    if (ciphertext.length < NOISE_TAG_LENGTH || ciphertext.length > NOISE_MESSAGE_MAX_LENGTH) {
        throw new RangeError("Noise transport ciphertext has an invalid length");
    }
    if (this.#messagesReceived >= TRANSPORT_MESSAGES_BEFORE_REHANDSHAKE) {
        throw new Error("Noise transport requires a fresh handshake");
    }
    try {
        const plaintext = cipher.DecryptWithAd(emptyBytes, ciphertext);
        this.#messagesReceived += 1;
        return Uint8Array.from(plaintext);
    } catch (error) {

```

```

        this.#failed = true;
        throw error;
    }
}

dispose(): void {
    if (this.#disposed) {
        return;
    }
    this.#disposed = true;
    if (this.#sendCipher !== undefined) {
        this.#sendCipher.free();
    }
    if (this.#receiveCipher !== undefined) {
        this.#receiveCipher.free();
    }
    if (this.#handshakeHashValue === undefined && !this.#failed) {
        this.#handshake.free();
    }
}

#transportCipher(direction: "send" | "receive"): NoiseCipherState {
    this.#requireUsable();
    const cipher = direction === "send" ? this.#sendCipher : this.#receiveCipher;
    if (cipher === undefined) {
        throw new Error("Noise IK transport is not ready");
    }
    return cipher;
}

#requireUsable(): void {
    if (this.#disposed) {
        throw new Error("Noise IK transport was disposed");
    }
    if (this.#failed) {
        throw new Error("Noise IK transport failed and cannot be reused");
    }
}

export async function createNoiseIkKeypair(): Promise<NoiseIkKeypair> {
    const library = await loadNoiseLibrary();
    const [privateKey, publicKey] = library.CreateKeyPair(
        library.constants.NOISE_DH_CURVE25519,
    );
    return {

```

```

        privateKey: copyExact(privateKey, X25519_KEY_LENGTH, "generated private key"),
        publicKey: copyExact(publicKey, X25519_KEY_LENGTH, "generated public key"),
    };
}

export async function createNoiseIkInitiator(
    options: NoiseIkInitiatorOptions,
): Promise<NoiseIkInitiator> {
    const library = await loadNoiseLibrary();
    return new NoiseIkInitiatorState(library, options);
}

```

24.2 runner/encryptedRunnerClient.ts

The encrypted client performs runner handshakes, sends RPC requests, and converts authenticated responses into browser errors.

```

import { api } from "../api/client";
import {
    createNoiseIkInitiator,
    createNoiseIkKeypair,
    type NoiseIkInitiator,
    type NoiseIkKeypair,
} from "../crypto/noiseIk";
import {
    hasTransportMagic,
    NOISE_CIPHERTEXT_BYTES_MAX,
    NOISE_PLAINTEXT_BYTES_MAX,
    packTransportFragment,
    parseTransportAck,
    parseTransportFragment,
    RPC_REQUEST_BYTES_MAX,
    RPC_RESPONSE_BYTES_MAX,
    TRANSPORT_PAYLOAD_BYTES_MAX,
    TRANSPORT_PULL,
    TRANSPORT_REQUEST,
    TRANSPORT_RESPONSE,
} from "../runnerRpcTransport";

const RUNNER_PROTOCOL = "yinshi-runner-v1";
const RUNNER_HEALTH_SESSION_BYTES = 65_536;
const SOCKET_TIMEOUT_MS = 15_000;
const RUNNER_PUBLIC_KEY_BYTES = 32;
const DEFAULT_CAPABILITY_ENDPOINT = "/api/settings/runner/capabilities";
// Capability budgets count ciphertext for requests, acknowledgements, pulls, and responses
// Callers must include Noise tags and transport headers in maxSessionBytes.

```

```

export type RunnerCapabilityEndpoint =
  typeof DEFAULT_CAPABILITY_ENDPOINT | "/api/runtime/capabilities";

export interface RunnerCapabilityResponse {
  readonly capability: string;
  readonly transfer_id: string;
  readonly runner_id: string;
  readonly runner_public_key: string;
  readonly protocol: string;
  readonly issued_at: number;
  readonly expires_at: number;
  readonly max_frame_bytes: number;
  readonly max_session_bytes: number;
  readonly relay_url: string;
}

export interface RunnerCapabilityRequest {
  readonly initiator_public_key: string;
  readonly scopes: readonly string[];
  readonly max_session_bytes: number;
}

export interface RunnerHealth {
  readonly protocol: string;
  readonly status: "ok";
}

export interface EncryptedRunnerConnectionOptions {
  readonly expectedRunnerPublicKey: string;
  readonly scopes: readonly string[];
  readonly maxSessionBytes?: number;
  readonly capabilityEndpoint?: RunnerCapabilityEndpoint;
}

export interface EncryptedRunnerOperation {
  readonly method: "DELETE" | "GET" | "PATCH" | "POST" | "PUT";
  readonly path: string;
  readonly query?: Readonly<Record<string, string>>;
  readonly body?: unknown;
}

export interface EncryptedRunnerRequest
  extends EncryptedRunnerConnectionOptions, EncryptedRunnerOperation {}

export interface EncryptedRunnerConnection {

```



```

    readonly expiresAtMs?: number;
    request<T>(operation: EncryptedRunnerOperation): Promise<T>;
    close(): void;
}

export class RunnerRpcError extends Error {
    readonly status: number;
    readonly body: unknown;

    constructor(status: number, body: unknown) {
        super(`Runner RPC failed with status ${status}`);
        this.name = "RunnerRpcError";
        this.status = status;
        this.body = body;
    }
}

export interface RunnerClientDependencies {
    createKeypair(): Promise<NoiseIkKeypair>;
    createInitiator(options: {
        readonly staticPrivateKey: Uint8Array;
        readonly responderStaticPublicKey: Uint8Array;
        readonly prologue: Uint8Array;
    }): Promise<NoiseIkInitiator>;
    issueCapability(
        request: RunnerCapabilityRequest,
        capabilityEndpoint?: RunnerCapabilityEndpoint,
    ): Promise<RunnerCapabilityResponse>;
    openWebSocket(url: string): WebSocket;
    createRequestId(): string;
}

function encodeBase64url(value: Uint8Array): string {
    if (!(value instanceof Uint8Array) || value.length === 0) {
        throw new TypeError("base64url input must be a non-empty Uint8Array");
    }
    let binary = "";
    for (const byte of value) {
        binary += String.fromCharCode(byte);
    }
    return btoa(binary)
        .replace(/\+/g, "-")
        .replace(/\//g, "_")
        .replace(/=+$/, "");
}

```

```

function decodePublicKey(value: string): Uint8Array {
  if (typeof value !== "string" || !/^[A-Za-z0-9_-]{43}$/.test(value)) {
    throw new Error("Runner public key is not canonical base64url");
  }
  let binary: string;
  try {
    binary = atob(`${value.replace(/-/g, "+").replace(/_/g, "/")}=`);
  } catch {
    throw new Error("Runner public key is not canonical base64url");
  }
  const key = Uint8Array.from(binary, (character) => character.charCodeAt(0));
  if (
    key.length !== RUNNER_PUBLIC_KEY_BYTES ||
    encodeBase64url(key) !== value
  ) {
    throw new Error("Runner public key must contain 32 canonical bytes");
  }
  return key;
}

function validateCapability(
  value: RunnerCapabilityResponse,
  expectedRunnerPublicKey: string,
  expectedSessionBytes: number,
): RunnerCapabilityResponse {
  if (value === null || typeof value !== "object") {
    throw new Error("Runner capability response is invalid");
  }
  if (value.protocol !== RUNNER_PROTOCOL) {
    throw new Error("Runner capability protocol is unsupported");
  }
  if (value.runner_public_key !== expectedRunnerPublicKey) {
    throw new Error("Runner identity changed; pairing is required again");
  }
  decodePublicKey(value.runner_public_key);
  if (
    !/^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/.test(
      value.transfer_id,
    )
  ) {
    throw new Error("Runner capability transfer ID is invalid");
  }
  if (
    !Number.isSafeInteger(value.issued_at) ||
    !Number.isSafeInteger(value.expires_at) ||
    value.expires_at <= Math.floor(Date.now() / 1_000)
  )

```

```

    ) {
        throw new Error("Runner capability is expired or invalid");
    }
    if (
        value.max_frame_bytes !== 65_535 ||
        value.max_session_bytes !== expectedSessionBytes
    ) {
        throw new Error("Runner capability limits are invalid");
    }
    const relayUrl = new URL(value.relay_url);
    if (
        !["ws:", "wss:"].includes(relayUrl.protocol) ||
        relayUrl.username !== "" ||
        relayUrl.password !== "" ||
        relayUrl.search !== "" ||
        relayUrl.hash !== "" ||
        relayUrl.pathname !== `/api/runner/relay/${value.transfer_id}`
    ) {
        throw new Error("Runner relay URL is invalid");
    }
    if (
        relayUrl.protocol !== "wss:" &&
        window.location.hostname !== "localhost"
    ) {
        throw new Error("Runner relay URL must use TLS");
    }
    if (typeof value.capability !== "string" || value.capability.length < 64) {
        throw new Error("Runner capability token is invalid");
    }
    return value;
}

function waitForOpen(socket: WebSocket): Promise<void> {
    if (socket.readyState === WebSocket.OPEN) {
        return Promise.resolve();
    }
    return new Promise((resolve, reject) => {
        const cleanup = () => {
            window.clearTimeout(timeout);
            socket.removeEventListener("open", handleOpen);
            socket.removeEventListener("error", handleError);
            socket.removeEventListener("close", handleClose);
        };
        const handleOpen = () => {
            cleanup();
            resolve();
        };
    });
}

```

```

    };
    const handleError = () => {
        cleanup();
        reject(new Error("Runner relay could not be opened"));
    };
    const handleClose = () => {
        cleanup();
        reject(new Error("Runner relay closed before opening"));
    };
    const timeout = window.setTimeout(() => {
        cleanup();
        reject(new Error("Runner relay open timed out"));
    }, SOCKET_TIMEOUT_MS);
    socket.addEventListener("open", handleOpen);
    socket.addEventListener("error", handleError);
    socket.addEventListener("close", handleClose);
  });
}

function receiveMessage(socket: WebSocket): Promise<string | ArrayBuffer> {
  return new Promise((resolve, reject) => {
    const cleanup = () => {
      window.clearTimeout(timeout);
      socket.removeEventListener("message", handleMessage);
      socket.removeEventListener("close", handleClose);
      socket.removeEventListener("error", handleError);
    };
    const handleMessage = (event: MessageEvent) => {
      cleanup();
      if (typeof event.data === "string" || event.data instanceof ArrayBuffer) {
        resolve(event.data);
        return;
      }
      reject(new Error("Runner relay returned an unsupported frame"));
    };
    const handleClose = () => {
      cleanup();
      reject(new Error("Runner relay closed before responding"));
    };
    const handleError = () => {
      cleanup();
      reject(new Error("Runner relay failed before responding"));
    };
    const timeout = window.setTimeout(() => {
      cleanup();
      reject(new Error("Runner relay response timed out"));
    }, SOCKET_TIMEOUT_MS);
  });
}

```

```

    }, SOCKET_TIMEOUT_MS);
    socket.addEventListener("message", handleMessage);
    socket.addEventListener("close", handleClose);
    socket.addEventListener("error", handleError);
  });
}

async function sendAndReceive(
  socket: WebSocket,
  message: string | Uint8Array,
): Promise<string | ArrayBuffer> {
  const response = receiveMessage(socket);
  socket.send(message);
  return response;
}

function parseHandshakeResponse(
  value: Uint8Array,
  expectedTransferId: string,
): void {
  let payload: unknown;
  try {
    payload = JSON.parse(
      new TextDecoder("utf-8", { fatal: true }).decode(value),
    );
  } catch {
    throw new Error("Runner handshake response is invalid");
  }
  if (
    payload === null ||
    typeof payload !== "object" ||
    Object.keys(payload).length !== 2 ||
    !("protocol" in payload) ||
    payload.protocol !== RUNNER_PROTOCOL ||
    !("transfer_id" in payload) ||
    payload.transfer_id !== expectedTransferId
  ) {
    throw new Error("Runner handshake binding did not match the capability");
  }
}

async function exchangeEncryptedFrame(
  socket: WebSocket,
  initiator: NoiseIkInitiator,
  plaintext: Uint8Array,
): Promise<Uint8Array> {

```

```

    const ciphertext = initiator.encrypt(plaintext);
    if (ciphertext.length > NOISE_CIPHERTEXT_BYTES_MAX) {
        throw new Error("Runner RPC encrypted fragment is too large");
    }
    const encryptedResponse = await sendAndReceive(socket, ciphertext);
    if (!(encryptedResponse instanceof ArrayBuffer)) {
        throw new Error("Runner RPC response must be binary");
    }
    return initiator.decrypt(new Uint8Array(encryptedResponse));
}

async function collectFramedResponse(
    socket: WebSocket,
    initiator: NoiseIcInitiator,
    firstResponse: Uint8Array,
): Promise<Uint8Array> {
    let fragment = parseTransportFragment(firstResponse, RPC_RESPONSE_BYTES_MAX);
    if (fragment.kind !== TRANSPORT_RESPONSE || fragment.index !== 0) {
        throw new Error("Runner RPC transport fragment is invalid");
    }
    const response = new Uint8Array(fragment.total);
    response.set(fragment.payload, 0);
    for (let index = 1; index < fragment.count; index += 1) {
        const plaintext = await exchangeEncryptedFrame(
            socket,
            initiator,
            packTransportFragment(
                TRANSPORT_PULL,
                index,
                fragment.count,
                fragment.total,
            ),
        );
        const nextFragment = parseTransportFragment(
            plaintext,
            RPC_RESPONSE_BYTES_MAX,
        );
        if (
            nextFragment.kind !== TRANSPORT_RESPONSE ||
            nextFragment.index !== index ||
            nextFragment.count !== fragment.count ||
            nextFragment.total !== fragment.total
        ) {
            throw new Error("Runner RPC transport fragment is invalid");
        }
        response.set(nextFragment.payload, index * TRANSPORT_PAYLOAD_BYTES_MAX);
    }
}

```

```

        fragment = nextFragment;
    }
    return response;
}

async function exchangeRpcPayload(
    socket: WebSocket,
    initiator: NoiseIkInitiator,
    request: Uint8Array,
): Promise<Uint8Array> {
    if (request.length < 1 || request.length > RPC_REQUEST_BYTES_MAX) {
        throw new Error("Runner RPC request is too large");
    }
    if (request.length <= NOISE_PLAINTEXT_BYTES_MAX) {
        const response = await exchangeEncryptedFrame(socket, initiator, request);
        return hasTransportMagic(response)
            ? collectFramedResponse(socket, initiator, response)
            : response;
    }

    const requestCount = Math.max(
        1,
        Math.ceil(request.length / TRANSPORT_PAYLOAD_BYTES_MAX),
    );
    let firstResponse: Uint8Array | undefined;
    for (let index = 0; index < requestCount; index += 1) {
        const start = index * TRANSPORT_PAYLOAD_BYTES_MAX;
        const end = Math.min(request.length, start + TRANSPORT_PAYLOAD_BYTES_MAX);
        const response = await exchangeEncryptedFrame(
            socket,
            initiator,
            packTransportFragment(
                TRANSPORT_REQUEST,
                index,
                requestCount,
                request.length,
                request.subarray(start, end),
            ),
        );
        if (index + 1 < requestCount) {
            parseTransportAck(response, index, requestCount, request.length);
        } else {
            firstResponse = response;
        }
    }
    if (firstResponse === undefined) {

```

```

        throw new Error("Runner RPC transport fragment is invalid");
    }
    return collectFramedResponse(socket, initiator, firstResponse);
}

function parseRpcResponse(
    value: Uint8Array,
    requestId: string,
    expectedSequence: number,
): { status: number; body: unknown } {
    let payload: unknown;
    try {
        payload = JSON.parse(
            new TextDecoder("utf-8", { fatal: true }).decode(value),
        );
    } catch {
        throw new Error("Runner RPC response is invalid");
    }
    if (payload === null || typeof payload !== "object") {
        throw new Error("Runner RPC response is invalid");
    }
    const response = payload as Record<string, unknown>;
    if (
        Object.keys(response).length !== 6 ||
        response.v !== 2 ||
        response.type !== "response" ||
        response.sequence !== expectedSequence ||
        response.request_id !== requestId ||
        !Number.isSafeInteger(response.status) ||
        (response.status as number) < 100 ||
        (response.status as number) > 599
    ) {
        throw new Error("Runner RPC response did not match the request");
    }
    return { status: response.status as number, body: response.body };
}

function validateHealth(value: unknown): RunnerHealth {
    if (value === null || typeof value !== "object") {
        throw new Error("Runner health response body is invalid");
    }
    const health = value as Record<string, unknown>;
    if (
        Object.keys(health).length !== 2 ||
        health.protocol !== RUNNER_PROTOCOL ||
        health.status !== "ok"
    )

```



```

    ) {
      throw new Error("Runner health response body is invalid");
    }
    return { protocol: RUNNER_PROTOCOL, status: "ok" };
  }

const defaultDependencies: RunnerClientDependencies = {
  createKeypair: createNoiseIkKeypair,
  createInitiator: createNoiseIkInitiator,
  issueCapability: (
    request,
    capabilityEndpoint = DEFAULT_CAPABILITY_ENDPOINT,
  ) => api.post<RunnerCapabilityResponse>(capabilityEndpoint, request),
  openWebSocket: (url) => new WebSocket(url),
  createRequestId: () => crypto.randomUUID(),
};

function validateOperation(operation: EncryptedRunnerOperation): {
  readonly method: EncryptedRunnerOperation["method"];
  readonly path: string;
  readonly query: Readonly<Record<string, string>>;
  readonly body: unknown;
} {
  const { method, path, query = {}, body = null } = operation;
  if (!path.startsWith("/") || path.includes("?") || path.includes("#")) {
    throw new TypeError("Runner request path must be normalized");
  }
  const queryEntries = Object.entries(query);
  if (queryEntries.length > 16)
    throw new TypeError("Runner request query is too large");
  for (const [key, value] of queryEntries) {
    if (
      !/^[A-Za-z0-9_]{1,64}$/u.test(key) ||
      typeof value !== "string" ||
      value.length > 2_048
    ) {
      throw new TypeError("Runner request query is invalid");
    }
  }
  if (!new Set(["DELETE", "GET", "PATCH", "POST", "PUT"]).has(method)) {
    throw new TypeError("Runner request method is invalid");
  }
  return { method, path, query, body };
}

export async function connectEncryptedRunner(

```

```

options: EncryptedRunnerConnectionOptions,
dependencies: RunnerClientDependencies = defaultDependencies,
): Promise<EncryptedRunnerConnection> {
  const {
    expectedRunnerPublicKey,
    scopes,
    maxSessionBytes = RUNNER_HEALTH_SESSION_BYTES,
    capabilityEndpoint = DEFAULT_CAPABILITY_ENDPOINT,
  } = options;
  if (
    capabilityEndpoint !== DEFAULT_CAPABILITY_ENDPOINT &&
    capabilityEndpoint !== "/api/runtime/capabilities"
  ) {
    throw new TypeError("Runner capability endpoint is invalid");
  }
  decodePublicKey(expectedRunnerPublicKey);
  if (
    !Array.isArray(scopes) ||
    scopes.length === 0 ||
    scopes.some((scope) => !scope)
  ) {
    throw new TypeError("Runner request scopes must not be empty");
  }
  if (
    !Number.isSafeInteger(maxSessionBytes) ||
    maxSessionBytes < 65_536 ||
    maxSessionBytes > 1_073_741_824
  ) {
    throw new RangeError("Runner session byte limit is invalid");
  }

  const keypair = await dependencies.createKeypair();
  if (keypair.privateKey.length !== 32 || keypair.publicKey.length !== 32) {
    keypair.privateKey.fill(0);
    throw new Error("Runner client keypair is invalid");
  }
  const clientPublicKey = encodeBase64url(keypair.publicKey);
  let capability: RunnerCapabilityResponse;
  let initiator: NoiseIkInitiator;
  try {
    capability = validateCapability(
      await dependencies.issueCapability(
        {
          initiator_public_key: clientPublicKey,
          scopes,
          max_session_bytes: maxSessionBytes,

```

```

        },
        capabilityEndpoint,
    ),
    expectedRunnerPublicKey,
    maxSessionBytes,
);
initiator = await dependencies.createInitiator({
    staticPrivateKey: keypair.privateKey,
    responderStaticPublicKey: decodePublicKey(capability.runner_public_key),
    prologue: new TextEncoder().encode(RUNNER_PROTOCOL),
});
} finally {
    keypair.privateKey.fill(0);
}

let socket: WebSocket;
try {
    socket = dependencies.openWebSocket(capability.relay_url);
} catch (error) {
    initiator.dispose();
    throw error;
}
socket.binaryType = "arraybuffer";
let closed = false;
let requestPending = false;
let nextSequence = 0;
const close = () => {
    if (closed) return;
    closed = true;
    initiator.dispose();
    if (
        socket.readyState === WebSocket.OPEN ||
        socket.readyState === WebSocket.CONNECTING
    ) {
        socket.close(1000, "Runner connection complete");
    }
};

try {
    await waitForOpen(socket);
    const ready = await sendAndReceive(socket, capability.capability);
    if (ready !== '{"type": "ready"}')
        throw new Error("Runner relay did not become ready");
    const handshakeMessage = initiator.writeHandshakeMessage(
        new TextEncoder().encode(capability.capability),
    );

```

```

const handshakeResponse = await sendAndReceive(socket, handshakeMessage);
if (!(handshakeResponse instanceof ArrayBuffer)) {
    throw new Error("Runner handshake response must be binary");
}
parseHandshakeResponse(
    initiator.readHandshakeMessage(new Uint8Array(handshakeResponse)),
    capability.transfer_id,
);
} catch (error) {
    close();
    throw error;
}

return {
    expiresAtMs: capability.expires_at * 1_000,
    async request<T>(operation: EncryptedRunnerOperation): Promise<T> {
        if (closed) throw new Error("Runner connection is closed");
        if (requestPending)
            throw new Error("Runner connection requests must be sequential");
        if (nextSequence >= 1_048_576) {
            close();
            throw new Error("Runner connection reached its message limit");
        }
        const { method, path, query, body } = validateOperation(operation);
        requestPending = true;
        const sequence = nextSequence;
        try {
            const requestId = dependencies.createRequestId();
            const request = new TextEncoder().encode(
                JSON.stringify({
                    body,
                    method,
                    path,
                    query,
                    request_id: requestId,
                    sequence,
                    type: "request",
                    v: 2,
                })
            );
            const response = parseRpcResponse(
                await exchangeRpcPayload(socket, initiator, request),
                requestId,
                sequence,
            );
            nextSequence += 1;

```

```

        if (response.status < 200 || response.status > 299) {
            throw new RunnerRpcError(response.status, response.body);
        }
        return response.body as T;
    } catch (error) {
        if (!(error instanceof RunnerRpcError)) close();
        throw error;
    } finally {
        requestPending = false;
    }
    },
    close,
};
}

export async function requestEncryptedRunner<T>(
    requestOptions: EncryptedRunnerRequest,
    dependencies: RunnerClientDependencies = defaultDependencies,
): Promise<T> {
    const {
        expectedRunnerPublicKey,
        scopes,
        maxSessionBytes,
        capabilityEndpoint,
        method,
        path,
        query,
        body,
    } = requestOptions;
    const connection = await connectEncryptedRunner(
        {
            expectedRunnerPublicKey,
            scopes,
            maxSessionBytes,
            capabilityEndpoint,
        },
        dependencies,
    );
    try {
        return await connection.request<T>({ method, path, query, body });
    } finally {
        connection.close();
    }
}

export async function checkEncryptedRunnerHealth(

```

```

    expectedRunnerPublicKey: string,
    dependencies: RunnerClientDependencies = defaultDependencies,
  ): Promise<RunnerHealth> {
    const response = await requestEncryptedRunner<unknown>(
      {
        expectedRunnerPublicKey,
        scopes: ["worker.health"],
        method: "GET",
        path: "/health",
      },
      dependencies,
    );
    return validateHealth(response);
  }

```

24.3 runner/repositories.ts

Repository helpers normalize browser operations before they cross the runner transport boundary.

```

import type { Repo } from "../api/client";
import { requestEncryptedRunner } from "../encryptedRunnerClient";

export interface RunnerRepositoryTarget {
  readonly runnerId: string;
  readonly runnerPublicKey: string;
}

function validateRepository(value: unknown): Repo {
  if (value === null || typeof value !== "object" || Array.isArray(value)) {
    throw new Error("Runner repository response is invalid");
  }
  const repo = value as Record<string, unknown>;
  const requiredStrings = ["id", "created_at", "updated_at", "name", "root_path"];
  if (requiredStrings.some((field) => typeof repo[field] !== "string" || !repo[field])) {
    throw new Error("Runner repository response is missing required fields");
  }
  for (const field of ["remote_url", "custom_prompt", "agents_md"]) {
    if (repo[field] !== null && typeof repo[field] !== "string") {
      throw new Error("Runner repository response has an invalid optional field");
    }
  }
  return repo as unknown as Repo;
}

function validateRepositoryName(value: string): string {

```

```

    if (typeof value !== "string") {
        throw new TypeError("Repository name must be a string");
    }
    const name = value.trim();
    if (!name || name.length > 255 || name !== value || /[\\\/]\.test(name) || name === "." ||
        throw new Error("Repository name must be a simple name");
    }
    return name;
}

function validateRepositoryUrl(value: string): string {
    let url: URL;
    try {
        url = new URL(value);
    } catch {
        throw new Error("Runner repository URL must be valid HTTPS");
    }
    if (url.protocol !== "https:" || !url.hostname) {
        throw new Error("Runner repository URL must use HTTPS");
    }
    if (url.username || url.password) {
        throw new Error("Runner repository URL must not include embedded credentials");
    }
    if (url.search || url.hash) {
        throw new Error("Runner repository URL must not include a query or fragment");
    }
    return url.toString();
}

function requestRunnerRepository<T>(
    target: RunnerRepositoryTarget,
    scope: "repository.read" | "repository.write",
    method: "GET" | "POST",
    body: unknown,
): Promise<T> {
    if (!target.runnerId || !target.runnerPublicKey) {
        throw new Error("Runner repository target is incomplete");
    }
    if (target.runnerId.length > 256) {
        throw new Error("BYOC runner ID is invalid");
    }
    return requestEncryptedRunner<T>({
        expectedRunnerPublicKey: target.runnerPublicKey,
        scopes: [scope],
        method,
        path: "/api/repos",
    });
}

```

```

        query: {},
        body,
        maxSessionBytes: 262_144,
    });
}

export async function listRunnerRepositories(
    target: RunnerRepositoryTarget,
): Promise<Repo[]> {
    const response = await requestRunnerRepository<unknown>(
        target,
        "repository.read",
        "GET",
        null,
    );
    if (!Array.isArray(response)) {
        throw new Error("Runner repository list response is invalid");
    }
    return response.map(validateRepository);
}

export async function importRunnerRepository(
    target: RunnerRepositoryTarget,
    nameValue: string,
    remoteUrlValue: string,
): Promise<Repo> {
    const name = validateRepositoryName(nameValue);
    const remoteUrl = validateRepositoryUrl(remoteUrlValue);
    const response = await requestRunnerRepository<unknown>(
        target,
        "repository.write",
        "POST",
        {
            name,
            remote_url: remoteUrl,
        },
    );
    return validateRepository(response);
}

```

24.4 runner/runnerRpcTransport.ts

RPC transport correlates requests with responses and rejects timeouts, disconnects, malformed frames, or mismatched identifiers.


```

export const RPC_REQUEST_BYTES_MAX = 2 * 1_024 * 1_024;
export const RPC_RESPONSE_BYTES_MAX = 10 * 1_024 * 1_024;
export const NOISE_CIPHertext_BYTES_MAX = 65_535;
export const NOISE_TAG_BYTES = 16;
export const NOISE_PLAINTEXT_BYTES_MAX =
    NOISE_CIPHertext_BYTES_MAX - NOISE_TAG_BYTES;
export const TRANSPORT_HEADER_BYTES = 17;
export const TRANSPORT_PAYLOAD_BYTES_MAX =
    NOISE_CIPHertext_BYTES_MAX - NOISE_TAG_BYTES - TRANSPORT_HEADER_BYTES;
export const TRANSPORT_REQUEST = 1;
export const TRANSPORT_ACK = 2;
export const TRANSPORT_RESPONSE = 3;
export const TRANSPORT_PULL = 4;
export const TRANSPORT_MAGIC = Uint8Array.of(89, 82, 80, 49);

export interface TransportFragment {
    readonly kind: number;
    readonly index: number;
    readonly count: number;
    readonly total: number;
    readonly payload: Uint8Array;
}

export function fragmentCount(total: number): number {
    if (!Number.isSafeInteger(total) || total < 1) {
        throw new RangeError("Runner RPC transport total must be positive");
    }
    return Math.max(1, Math.ceil(total / TRANSPORT_PAYLOAD_BYTES_MAX));
}

export function packTransportFragment(
    kind: number,
    index: number,
    count: number,
    total: number,
    payload?: Uint8Array,
): Uint8Array {
    const payloadLength = payload?.length ?? 0;
    const frame = new Uint8Array(TRANSPORT_HEADER_BYTES + payloadLength);
    frame.set(TRANSPORT_MAGIC, 0);
    const view = new DataView(frame.buffer);
    view.setUint8(4, kind);
    view.setUint32(5, index);
    view.setUint32(9, count);
    view.setUint32(13, total);
    if (payload !== undefined) frame.set(payload, TRANSPORT_HEADER_BYTES);
}

```

```

    return frame;
}

export function parseTransportFragment(
  frame: Uint8Array,
  totalLimit: number,
): TransportFragment {
  if (frame.length < TRANSPORT_HEADER_BYTES) {
    throw new Error("Runner RPC transport fragment is invalid");
  }
  for (let index = 0; index < TRANSPORT_MAGIC.length; index += 1) {
    if (frame[index] !== TRANSPORT_MAGIC[index]) {
      throw new Error("Runner RPC transport fragment is invalid");
    }
  }
  const view = new DataView(frame.buffer, frame.byteOffset, frame.byteLength);
  const kind = view.getUint8(4);
  const index = view.getUint32(5);
  const count = view.getUint32(9);
  const total = view.getUint32(13);
  if (
    total < 1 ||
    total > totalLimit ||
    count !== fragmentCount(total) ||
    index >= count
  ) {
    throw new Error("Runner RPC transport fragment is invalid");
  }
  const expectedPayloadLength = Math.min(
    TRANSPORT_PAYLOAD_BYTES_MAX,
    total - index * TRANSPORT_PAYLOAD_BYTES_MAX,
  );
  if (frame.length !== TRANSPORT_HEADER_BYTES + expectedPayloadLength) {
    throw new Error("Runner RPC transport fragment is invalid");
  }
  return {
    kind,
    index,
    count,
    total,
    payload: frame.subarray(TRANSPORT_HEADER_BYTES),
  };
}

export function parseTransportAck(
  frame: Uint8Array,

```

```

    expectedIndex: number,
    expectedCount: number,
    expectedTotal: number,
  ): void {
    if (frame.length !== TRANSPORT_HEADER_BYTES) {
      throw new Error("Runner RPC transport fragment is invalid");
    }
    const view = new DataView(frame.buffer, frame.byteOffset, frame.byteLength);
    for (let index = 0; index < TRANSPORT_MAGIC.length; index += 1) {
      if (frame[index] !== TRANSPORT_MAGIC[index]) {
        throw new Error("Runner RPC transport fragment is invalid");
      }
    }
    if (
      view.getUint8(4) !== TRANSPORT_ACK ||
      view.getUint32(5) !== expectedIndex ||
      view.getUint32(9) !== expectedCount ||
      view.getUint32(13) !== expectedTotal
    ) {
      throw new Error("Runner RPC transport fragment is invalid");
    }
  }
}

export function hasTransportMagic(value: Uint8Array): boolean {
  return (
    value.length >= TRANSPORT_MAGIC.length &&
    TRANSPORT_MAGIC.every((byte, index) => value[index] === byte)
  );
}

```

24.5 runtime/encryptedUpload.ts

Upload flow encrypts file content locally and sends only bounded ciphertext through authorized runtime endpoints.

```

import {
  connectEncryptedRunner,
  RunnerRpcError,
  type EncryptedRunnerConnection,
  type EncryptedRunnerOperation,
} from "../runner/encryptedRunnerClient";
import type { RuntimeRef } from "../runtimeTransport";

const FILE_BYTES_MAX = 50 * 1024 * 1024;
const CHUNK_BYTES = 24_000;
const SESSION_BYTES = 128 * 1024 * 1024;

```

```

const RESOURCE_ID_PATTERN = /^[0-9a-f]{32}$/;

interface UploadProgress {
  readonly id: string;
  readonly purpose: "pi_config";
  readonly filename: string;
  readonly size_bytes: number;
  readonly next_chunk_index: number;
}

function base64url(bytes: Uint8Array): string {
  let binary = "";
  for (const byte of bytes) binary += String.fromCharCode(byte);
  return btoa(binary).replace(/\+/gu, "-").replace(/\//gu, "_").replace(/=+$/u, "");
}

function hex(bytes: Uint8Array): string {
  return Array.from(bytes, (byte) => byte.toString(16).padStart(2, "0")).join("");
}

function validateProgress(
  value: unknown,
  expected: {
    readonly id?: string;
    readonly filename: string;
    readonly sizeBytes: number;
    readonly nextChunkIndex: number;
  },
): UploadProgress {
  if (value === null || typeof value !== "object" || Array.isArray(value)) {
    throw new Error("Encrypted upload response is invalid");
  }
  const progress = value as Record<string, unknown>;
  if (
    Object.keys(progress).length !== 5 ||
    typeof progress.id !== "string" ||
    !RESOURCE_ID_PATTERN.test(progress.id) ||
    (expected.id !== undefined && progress.id !== expected.id) ||
    progress.purpose !== "pi_config" ||
    progress.filename !== expected.filename ||
    progress.size_bytes !== expected.sizeBytes ||
    progress.next_chunk_index !== expected.nextChunkIndex
  ) {
    throw new Error("Encrypted upload response did not match the transfer");
  }
  return progress as unknown as UploadProgress;
}

```

```

}

function validateFile(file: File): void {
  if (!file || typeof file.name !== "string" || typeof file.size !== "number") {
    throw new TypeError("Pi config upload must be a file");
  }
  if (
    !file.name ||
    file.name.length > 255 ||
    file.name !== file.name.trim() ||
    file.name.includes("/") ||
    file.name.includes("\\") ||
    !file.name.toLowerCase().endsWith(".zip")
  ) {
    throw new Error("Pi config upload must have a simple .zip filename");
  }
  if (!Number.isSafeInteger(file.size) || file.size < 1 || file.size > FILE_BYTES_MAX) {
    throw new Error("Pi config upload must be between 1 byte and 50MB");
  }
}

type UploadInvoker = <Result>(
  operation: EncryptedRunnerOperation,
  retryConnectionFailures?: boolean,
) => Promise<Result>;

async function uploadPiConfigChunks<T>(file: File, invoke: UploadInvoker): Promise<T> {
  validateFile(file);
  const fileBytes = new Uint8Array(await file.arrayBuffer());
  if (fileBytes.length !== file.size) {
    fileBytes.fill(0);
    throw new Error("Pi config file size changed while reading");
  }
  const digest = new Uint8Array(await crypto.subtle.digest("SHA-256", fileBytes));
  let uploadId: string | null = null;
  try {
    const started = validateProgress(
      await invoke({
        method: "POST",
        path: "/api/settings/pi-config/uploads",
        body: {
          purpose: "pi_config",
          filename: file.name,
          size_bytes: fileBytes.length,
          sha256: hex(digest),
        },
      })
    );
  }
}

```

```

    }),
    {
        filename: file.name,
        sizeBytes: fileBytes.length,
        nextChunkIndex: 0,
    },
);
uploadId = started.id;
const chunkCount = Math.ceil(fileBytes.length / CHUNK_BYTES);
for (let chunkIndex = 0; chunkIndex < chunkCount; chunkIndex += 1) {
    const offset = chunkIndex * CHUNK_BYTES;
    const chunk = fileBytes.subarray(offset, Math.min(offset + CHUNK_BYTES, fileBytes.length));
    validateProgress(
        await invoke(
            {
                method: "POST",
                path: `/api/settings/pi-config/uploads/${uploadId}/chunks/${chunkIndex}`,
                body: { data: base64url(chunk) },
            },
            true,
        ),
        {
            id: uploadId,
            filename: file.name,
            sizeBytes: fileBytes.length,
            nextChunkIndex: chunkIndex + 1,
        },
    );
}
return await invoke<T>({
    method: "POST",
    path: `/api/settings/pi-config/uploads/${uploadId}/complete`,
});
} catch (error) {
    if (uploadId) {
        try {
            await invoke({
                method: "DELETE",
                path: `/api/settings/pi-config/uploads/${uploadId}`,
            });
        } catch {
            // Incomplete uploads expire from runtime memory after a bounded interval.
        }
    }
    throw error;
} finally {

```

```

        digest.fill(0);
        fileBytes.fill(0);
    }
}

export async function uploadEncryptedPiConfig<T>(
    runtime: Extract<RuntimeRef, { location: "byoc" | "managed" }>,
    file: File,
): Promise<T> {
    if (
        (runtime.location !== "byoc" && runtime.location !== "managed") ||
        !runtime.runnerPublicKey
    ) {
        throw new Error("Encrypted upload requires a remote runtime");
    }
    const connectionState: { current: EncryptedRunnerConnection | null } = {
        current: null,
    };
    const connect = async (): Promise<EncryptedRunnerConnection> =>
        connectEncryptedRunner({
            expectedRunnerPublicKey: runtime.runnerPublicKey,
            scopes: ["pi.configure"],
            maxSessionBytes: SESSION_BYTES,
            ...(runtime.location === "managed"
                ? { capabilityEndpoint: "/api/runtime/capabilities" as const }
                : {}),
        });
    const invoke: UploadInvoker = async <Result>(
        operation: EncryptedRunnerOperation,
        retryConnectionFailures = false,
    ): Promise<Result> => {
        for (let attempt = 0; attempt < 5; attempt += 1) {
            connectionState.current ??= await connect();
            try {
                return await connectionState.current.request<Result>(operation);
            } catch (error) {
                if (error instanceof RunnerRpcError) throw error;
                connectionState.current.close();
                connectionState.current = null;
                if (!retryConnectionFailures || attempt === 4) throw error;
            }
        }
        throw new Error("Encrypted upload retry limit reached");
    };
    try {
        return await uploadPiConfigChunks<T>(file, invoke);
    }
}

```

```

    } finally {
      connectionState.current?.close();
    }
  }

export async function uploadHostedPiConfig<T>(file: File): Promise<T> {
  const bridge = window.yinshiDesktop;
  if (!bridge) throw new Error("Hosted desktop API is unavailable");
  const invoke: UploadInvoker = async <Result>(operation: EncryptedRunnerOperation) => {
    const requestBody = operation.body;
    if (
      requestBody !== undefined &&
      (requestBody === null || typeof requestBody !== "object" || Array.isArray(requestBody))
    ) {
      throw new TypeError("Hosted upload request body must be an object");
    }
    const response = await bridge.hostedRequest(
      requestBody === undefined
        ? { method: operation.method, path: operation.path }
        : {
            method: operation.method,
            path: operation.path,
            body: requestBody as Readonly<Record<string, unknown>>,
          },
    );
    if (response.status < 200 || response.status > 299) {
      const error = new Error(`Hosted upload failed with status ${response.status}`);
      Object.assign(error, { status: response.status, body: response.body });
      throw error;
    }
    return response.body as Result;
  };
  return uploadPiConfigChunks<T>(file, invoke);
}

```

24.6 runtime/promptStream.ts

Prompt streaming reconnects from journal cursors and presents stored events through one async browser interface.

```

import {
  normalizeEvent,
  type SSEEvent,
  type ThinkingLevel,
} from "../api/client";
import type { RuntimeTransport } from "../runtimeTransport";

```



```

const RESOURCE_ID_PATTERN = /^[0-9a-f]{32}$/;
const TERMINAL_STATUSES = new Set([
  "completed",
  "failed",
  "cancelled",
  "interrupted",
]);
const RUN_STATUSES = new Set([
  "starting",
  "running",
  "stopping",
  ...TERMINAL_STATUSES,
]);

type PromptRunStatus =
  | "starting"
  | "running"
  | "stopping"
  | "completed"
  | "failed"
  | "cancelled"
  | "interrupted";

interface PromptRunResponse {
  readonly id: string;
  readonly session_id: string;
  readonly status: PromptRunStatus;
}

interface PromptEventBatchResponse {
  readonly run_id: string;
  readonly status: PromptRunStatus;
  readonly events: unknown[];
  readonly next_sequence: number;
}

export interface RuntimePromptOptions {
  readonly prompt: string;
  readonly model?: string;
  readonly thinking?: ThinkingLevel;
  readonly idempotencyKey?: string;
  readonly signal?: AbortSignal;
  readonly pollDelayMs?: number;
  readonly pollRetryLimit?: number;
}

```

```

export interface RuntimePromptHandle {
  readonly runId: string;
  events(): AsyncGenerator<SSEEvent>;
  cancel(): Promise<PromptRunStatus>;
}

function validateResourceId(value: unknown, field: string): string {
  if (typeof value !== "string" || !RESOURCE_ID_PATTERN.test(value)) {
    throw new Error(`Prompt ${field} is invalid`);
  }
  return value;
}

function validateStatus(value: unknown): PromptRunStatus {
  if (typeof value !== "string" || !RUN_STATUSES.has(value)) {
    throw new Error("Prompt run status is invalid");
  }
  return value as PromptRunStatus;
}

function validateRun(value: unknown, sessionId: string): PromptRunResponse {
  if (value === null || typeof value !== "object" || Array.isArray(value)) {
    throw new Error("Prompt run response is invalid");
  }
  const run = value as Record<string, unknown>;
  if (Object.keys(run).length !== 3 || run.session_id !== sessionId) {
    throw new Error("Prompt run response did not match the session");
  }
  return {
    id: validateResourceId(run.id, "run ID"),
    session_id: sessionId,
    status: validateStatus(run.status),
  };
}

function validateEvent(value: unknown): SSEEvent {
  if (value === null || typeof value !== "object" || Array.isArray(value)) {
    throw new Error("Prompt journal event is invalid");
  }
  const event = value as Record<string, unknown>;
  if (typeof event.type !== "string" || !event.type) {
    throw new Error("Prompt journal event type is invalid");
  }
  return normalizeEvent(event);
}

```

```

function validateBatch(
  value: unknown,
  runId: string,
  expectedSequence: number,
): { status: PromptRunStatus; events: SSEEvent[]; nextSequence: number } {
  if (value === null || typeof value !== "object" || Array.isArray(value)) {
    throw new Error("Prompt event batch is invalid");
  }
  const batch = value as Record<string, unknown>;
  if (
    Object.keys(batch).length !== 4 ||
    batch.run_id !== runId ||
    !Array.isArray(batch.events) ||
    !Number.isSafeInteger(batch.next_sequence)
  ) {
    throw new Error("Prompt event batch did not match the run");
  }
  const events = batch.events.map(validateEvent);
  const nextSequence = batch.next_sequence as number;
  if (nextSequence !== expectedSequence + events.length) {
    throw new Error("Prompt event sequence is not contiguous");
  }
  return {
    status: validateStatus(batch.status),
    events,
    nextSequence,
  };
}

function abortError(): DOMException {
  return new DOMException("Prompt polling aborted", "AbortError");
}

async function delay(milliseconds: number, signal?: AbortSignal): Promise<void> {
  if (signal?.aborted) throw abortError();
  if (milliseconds === 0) return;
  await new Promise<void>((resolve, reject) => {
    const timer = window.setTimeout(() => {
      signal?.removeEventListener("abort", abort);
      resolve();
    }, milliseconds);
    const abort = (): void => {
      window.clearTimeout(timer);
      reject(abortError());
    };
  });
}

```

```

        signal?.addEventListener("abort", abort, { once: true });
    });
}

function shouldRetry(error: unknown): boolean {
    if (error instanceof DOMException && error.name === "AbortError") return false;
    if (error instanceof TypeError) return true;
    if (error !== null && typeof error === "object" && "status" in error) {
        const status = (error as { status?: unknown }).status;
        return (
            typeof status === "number" &&
            (status === 429 || (status >= 500 && status < 600))
        );
    }
    return false;
}

export async function startRuntimePrompt(
    transport: RuntimeTransport,
    sessionIdValue: string,
    options: RuntimePromptOptions,
): Promise<RuntimePromptHandle> {
    if (!transport || typeof transport.post !== "function" || typeof transport.get !== "function") {
        throw new TypeError("Prompt runtime transport is invalid");
    }
    const sessionId = validateResourceId(sessionIdValue, "session ID");
    const prompt = options.prompt.trim();
    if (!prompt || prompt.length > 100_000) {
        throw new Error("Prompt content has an invalid length");
    }
    const idempotencyKey = options.idempotencyKey ?? crypto.randomUUID();
    if (typeof idempotencyKey !== "string" || idempotencyKey.length !== 36) {
        throw new Error("Prompt idempotency key is invalid");
    }
    const remoteRuntime =
        transport.runtime.location === "byoc" || transport.runtime.location === "managed";
    const pollDelayMs = options.pollDelayMs ?? (remoteRuntime ? 750 : 250);
    if (!Number.isSafeInteger(pollDelayMs) || pollDelayMs < 0 || pollDelayMs > 5_000) {
        throw new Error("Prompt poll delay is invalid");
    }
    const pollRetryLimit = options.pollRetryLimit ?? 5;
    if (
        !Number.isSafeInteger(pollRetryLimit) ||
        pollRetryLimit < 1 ||
        pollRetryLimit > 5
    ) {

```

```

    throw new Error("Prompt poll retry limit is invalid");
}
const started = validateRun(
  await transport.post<unknown>(`/api/sessions/${sessionId}/runs`, {
    prompt,
    model: options.model ?? null,
    thinking: options.thinking ?? null,
    idempotency_key: idempotencyKey,
  }),
  sessionId,
);
let consumed = false;

return {
  runId: started.id,
  async *events(): AsyncGenerator<SSEEvent> {
    if (consumed) {
      throw new Error("Prompt event stream can only be consumed once");
    }
    consumed = true;
    let nextSequence = 0;
    let retryDelayMs = pollDelayMs;
    let consecutiveTransientFailures = 0;
    let emittedTerminalEvent = false;
    while (true) {
      if (options.signal?.aborted) throw abortError();
      let rawBatch: unknown;
      try {
        rawBatch = await transport.get<unknown>(
          `/api/sessions/${sessionId}/runs/${started.id}/events/${nextSequence}`,
        );
      } catch (error) {
        if (!shouldRetry(error)) throw error;
        consecutiveTransientFailures += 1;
        if (consecutiveTransientFailures >= pollRetryLimit) throw error;
        retryDelayMs = Math.min(Math.max(retryDelayMs * 2, 250), 5_000);
        await delay(retryDelayMs, options.signal);
        continue;
      }
      const batch = validateBatch(rawBatch, started.id, nextSequence);
      consecutiveTransientFailures = 0;
      retryDelayMs = pollDelayMs;
      nextSequence = batch.nextSequence;
      for (const event of batch.events) {
        if (event.type === "error" || event.type === "cancelled" || event.type === "result")
          emittedTerminalEvent = true;
      }
    }
  }
};

```

```

    }
    yield event;
  }
  if (TERMINAL_STATUSES.has(batch.status)) {
    if (!emittedTerminalEvent && batch.status !== "completed") {
      yield {
        type: "error",
        error:
          batch.status === "interrupted"
            ? "Prompt run was interrupted"
            : `Prompt run ended with status ${batch.status}`,
      };
    }
    return;
  }
  await delay(pollDelayMs, options.signal);
}
},
async cancel(): Promise<PromptRunStatus> {
  const cancelled = validateRun(
    await transport.post<unknown>(
      `/api/sessions/${sessionId}/runs/${started.id}/cancel`,
    ),
    sessionId,
  );
  if (cancelled.id !== started.id) {
    throw new Error("Prompt cancellation response did not match the run");
  }
  return cancelled.status;
},
};
}

```

24.7 runtime/resolveRuntime.ts

Runtime resolution chooses hosted, desktop, or explicit no-auth behavior before workspace operations begin.

```

import { api, type CloudRunner } from "../api/client";
import type { UnresolvedRuntimeRef } from "../runtimeRef";
import type { RuntimeRef } from "../runtimeTransport";

interface RunnerIdentity {
  readonly id: string;
  readonly status: CloudRunner["status"];
  readonly noise_public_key: string | null;
}

```

```

    readonly noise_key_confirmed: boolean;
}

interface RuntimeResolverDependencies {
    readonly getRunner?: (signal?: AbortSignal) => Promise<RunnerIdentity | null>;
    readonly getRuntime?: (signal?: AbortSignal) => Promise<unknown>;
    readonly provisionRuntime?: (signal?: AbortSignal) => Promise<unknown>;
    readonly desktop?: boolean;
    readonly sleep?: (milliseconds: number, signal?: AbortSignal) => Promise<void>;
    readonly maxPollAttempts?: number;
}

const MANAGED_RUNTIME_POLL_INTERVAL_MS = 500;
const MANAGED_RUNTIME_MAX_POLL_ATTEMPTS = 20;
const MANAGED_RUNTIME_FAILURE_MESSAGES: Readonly<Record<string, string>> = {
    artifact_invalid: "Managed runtime artifact is invalid",
    provider_unavailable: "Managed runtime provider is unavailable",
    network_policy_failed: "Managed runtime network policy setup failed",
    bootstrap_failed: "Managed runtime setup failed",
    runner_registration_failed: "Managed runtime registration failed",
    runner_identity_changed: "Managed runtime identity changed",
    wake_timeout: "Managed runtime startup timed out",
    checkpoint_failed: "Managed runtime checkpoint failed",
    delete_failed: "Managed runtime deletion failed",
};

const defaultDependencies = {
    getRunner: () => api.get<CloudRunner | null>("/api/settings/runner"),
    getRuntime: () => api.get<unknown>("/api/runtime"),
    provisionRuntime: () => api.post<unknown>("/api/runtime/provision"),
    sleep: (milliseconds: number, signal?: AbortSignal) =>
        new Promise<void>((resolve, reject) => {
            throwIfAborted(signal);
            const timer = window.setTimeout(() => {
                signal?.removeEventListener("abort", abort);
                resolve();
            }, milliseconds);
            const abort = (): void => {
                window.clearTimeout(timer);
                reject(new Error());
            };
            signal?.addEventListener("abort", abort, { once: true });
        }),
};

function abortError(): DOMException {

```

```

    return new DOMException("Runtime resolution was aborted", "AbortError");
}

function throwIfAborted(signal?: AbortSignal): void {
    if (signal?.aborted) {
        throw abortError();
    }
}

function parseManagedRuntime(value: unknown): Record<string, unknown> {
    if (value === null || typeof value !== "object" || Array.isArray(value)) {
        throw new Error("Managed runtime response is invalid");
    }
    const response = value as Record<string, unknown>;
    if (
        typeof response.provider !== "string" ||
        typeof response.status !== "string" ||
        !["absent", "provisioning", "ready", "failed", "deleting"].includes(
            response.status,
        )
    ) {
        throw new Error("Managed runtime response is invalid");
    }
    return response;
}

function isCanonicalRunnerPublicKey(value: unknown): value is string {
    if (typeof value !== "string" || !/^[A-Za-z0-9_-]{43}$/u.test(value)) {
        return false;
    }
    try {
        const binary = atob(`${value.replace(/-/gu, "+").replace(/_/gu, "/")}=`);
        const bytes = Uint8Array.from(binary, (character) => character.charCodeAt(0));
        let canonical = "";
        for (const byte of bytes) {
            canonical += String.fromCharCode(byte);
        }
        return (
            bytes.length === 32 &&
            btoa(canonical)
                .replace(/\+/gu, "-")
                .replace(/\//gu, "_")
                .replace(/=+$/u, "") === value
        );
    } catch {
        return false;
    }
}

```



```

    }
  }

export async function provisionRuntimeRef(
  runtime: UnresolvedRuntimeRef,
  dependencies: RuntimeResolverDependencies = defaultDependencies,
  signal?: AbortSignal,
): Promise<RuntimeRef> {
  if (runtime.location !== "hosted") {
    throw new Error("Only a hosted runtime can be provisioned");
  }
  const desktop =
    dependencies.desktop ??
    (typeof window !== "undefined" && window.yinshiDesktop !== undefined);
  if (desktop) {
    throw new Error("Desktop runtime provisioning is unavailable");
  }
  const provisionRuntime =
    dependencies.provisionRuntime ?? defaultDependencies.provisionRuntime;
  const getRuntime = dependencies.getRuntime ?? defaultDependencies.getRuntime;
  throwIfAborted(signal);
  let managed = parseManagedRuntime(await provisionRuntime(signal));
  throwIfAborted(signal);
  if (managed.provider !== "fly_sprites") {
    throw new Error("Managed runtime provider is unsupported");
  }
  if (managed.status === "provisioning") {
    const sleep = dependencies.sleep ?? defaultDependencies.sleep;
    const maxPollAttempts =
      dependencies.maxPollAttempts ?? MANAGED_RUNTIME_MAX_POLL_ATTEMPTS;
    for (let attempt = 0; attempt < maxPollAttempts; attempt += 1) {
      await sleep(MANAGED_RUNTIME_POLL_INTERVAL_MS, signal);
      throwIfAborted(signal);
      managed = parseManagedRuntime(await getRuntime(signal));
      throwIfAborted(signal);
      if (managed.status !== "provisioning") break;
    }
  }
  if (managed.status === "provisioning") {
    throw new Error("Managed runtime provisioning timed out");
  }
  if (managed.status === "ready") {
    if (!isCanonicalRunnerPublicKey(managed.runner_public_key)) {
      throw new Error("Managed runtime runner public key is invalid");
    }
  }
  return {

```

```

        location: "managed",
        runnerPublicKey: managed.runner_public_key,
    };
}
if (managed.status === "failed") {
    const message =
        typeof managed.last_error === "string"
            ? MANAGED_RUNTIME_FAILURE_MESSAGES[managed.last_error]
            : undefined;
    throw new Error(message ?? "Managed runtime setup failed");
}
if (managed.status === "deleting") {
    throw new Error("Managed runtime is deleting");
}
throw new Error("Managed runtime response is invalid");
}

export async function resolveRuntimeRef(
    runtime: UnresolvedRuntimeRef,
    dependencies: RuntimeResolverDependencies = defaultDependencies,
    signal?: AbortSignal,
): Promise<RuntimeRef> {
    if (runtime.location === "local") {
        return runtime;
    }
    if (runtime.location === "hosted") {
        const desktop =
            dependencies.desktop ??
            (typeof window !== "undefined" && window.yinshiDesktop !== undefined);
        if (desktop) {
            return runtime;
        }
        const getRuntime = dependencies.getRuntime ?? defaultDependencies.getRuntime;
        throwIfAborted(signal);
        let managed = parseManagedRuntime(await getRuntime(signal));
        throwIfAborted(signal);
        if (managed.provider === "local") {
            return runtime;
        }
        if (managed.provider === "fly_sprites" && managed.status === "absent") {
            throw new Error("Managed runtime is not provisioned");
        }
        if (
            managed.provider === "fly_sprites" &&
            managed.status === "provisioning"
        ) {

```

```

const sleep = dependencies.sleep ?? defaultDependencies.sleep;
const maxPollAttempts =
  dependencies.maxPollAttempts ?? MANAGED_RUNTIME_MAX_POLL_ATTEMPTS;
for (let attempt = 0; attempt < maxPollAttempts; attempt += 1) {
  await sleep(MANAGED_RUNTIME_POLL_INTERVAL_MS, signal);
  throwIfAborted(signal);
  managed = parseManagedRuntime(await getRuntime(signal));
  throwIfAborted(signal);
  if (managed.status !== "provisioning") {
    break;
  }
}
if (managed.status === "provisioning") {
  throw new Error("Managed runtime provisioning timed out");
}
if (managed.provider === "fly_sprites" && managed.status === "deleting") {
  throw new Error("Managed runtime is deleting");
}
if (managed.provider !== "fly_sprites") {
  throw new Error("Managed runtime provider is unsupported");
}
if (managed.status === "ready") {
  if (!isCanonicalRunnerPublicKey(managed.runner_public_key)) {
    throw new Error("Managed runtime runner public key is invalid");
  }
  return {
    location: "managed",
    runnerPublicKey: managed.runner_public_key,
  };
}
if (managed.status === "failed") {
  const message =
    typeof managed.last_error === "string"
      ? MANAGED_RUNTIME_FAILURE_MESSAGES[managed.last_error]
      : undefined;
  throw new Error(message ?? "Managed runtime setup failed");
}
throw new Error("Managed runtime response is invalid");
}
if (!runtime.runnerId || runtime.runnerPublicKey !== null) {
  throw new Error("Unresolved BYOC runtime reference is invalid");
}
const getRunner = dependencies.getRunner ?? defaultDependencies.getRunner;
throwIfAborted(signal);
const runner = await getRunner(signal);

```

```

throwIfAborted(signal);
if (
  runner === null ||
  runner.id !== runtime.runnerId ||
  runner.status === "revoked" ||
  !runner.noise_key_confirmed ||
  !runner.noise_public_key
) {
  throw new Error("BYOC runner is unavailable or is not paired");
}
return {
  location: "byoc",
  runnerId: runner.id,
  runnerPublicKey: runner.noise_public_key,
};
}

```

24.8 runtime/runtimeRef.ts

A runtime reference gives hooks a stable identity without duplicating transport state across components.

```

import type { RuntimeRef } from "../runtimeTransport";

const RESOURCE_ID_PATTERN = /^[0-9a-f]{32}$/;
const RUNNER_ID_LENGTH_MAX = 256;

export type UnresolvedRuntimeRef =
  | { readonly location: "local" }
  | { readonly location: "hosted" }
  | {
    readonly location: "byoc";
    readonly runnerId: string;
    readonly runnerPublicKey: null;
  };

export interface ParsedRuntimeResourceId {
  readonly runtime: UnresolvedRuntimeRef;
  readonly resourceId: string;
}

function encodeBase64url(value: string): string {
  const bytes = new TextEncoder().encode(value);
  let binary = "";
  for (const byte of bytes) {
    binary += String.fromCharCode(byte);
  }
}

```

```

    }
    return btoa(binary).replace(/\+/gu, "-").replace(/\/gu, "_").replace(/=+$/u, "");
}

function decodeBase64url(value: string): string {
    if (!/^[A-Za-z0-9_-]+$/.test(value)) {
        throw new Error("Runtime runner qualifier is invalid");
    }
    const padded = value.replace(/-/gu, "+").replace(/_/gu, "/").padEnd(
        Math.ceil(value.length / 4) * 4,
        "=",
    );
    let binary: string;
    try {
        binary = atob(padded);
    } catch {
        throw new Error("Runtime runner qualifier is invalid");
    }
    const bytes = Uint8Array.from(binary, (character) => character.charCodeAt(0));
    let decoded: string;
    try {
        decoded = new TextDecoder("utf-8", { fatal: true }).decode(bytes);
    } catch {
        throw new Error("Runtime runner qualifier is invalid");
    }
    if (!decoded || decoded.length > RUNNER_ID_LENGTH_MAX || encodeBase64url(decoded) !== value) {
        throw new Error("Runtime runner qualifier is invalid");
    }
    return decoded;
}

function validateResourceId(resourceId: string): void {
    if (!RESOURCE_ID_PATTERN.test(resourceId)) {
        throw new Error("Runtime resource ID is invalid");
    }
}

export function defaultRuntimeRef({ desktop }: { desktop: boolean }): UnresolvedRuntimeRef {
    if (typeof desktop !== "boolean") {
        throw new TypeError("desktop must be a boolean");
    }
    return desktop ? { location: "local" } : { location: "hosted" };
}

export function runtimeResourceId(
    runtime: RuntimeRef,

```

```

    resourceId: string,
    environment: { readonly desktop: boolean } = { desktop: false },
  ): string {
    validateResourceId(resourceId);
    if (runtime.location === "managed") {
      return resourceId;
    }
    if (runtime.location === "hosted") {
      return environment.desktop ? `hosted.${resourceId}` : resourceId;
    }
    if (runtime.location === "local") {
      return `local.${resourceId}`;
    }
    if (!runtime.runnerId || runtime.runnerId.length > RUNNER_ID_LENGTH_MAX) {
      throw new Error("BYOC runner ID is invalid");
    }
    return `byoc.${encodeURIComponent(runtime.runnerId)}.${resourceId}`;
  }

export function parseRuntimeResourceId(
  encodedId: string,
  environment: { readonly desktop: boolean },
): ParsedRuntimeResourceId {
  if (typeof encodedId !== "string" || !encodedId) {
    throw new Error("Runtime resource ID is invalid");
  }
  if (!encodedId.includes(".")) {
    validateResourceId(encodedId);
    return {
      runtime: defaultRuntimeRef(environment),
      resourceId: encodedId,
    };
  }
  const parts = encodedId.split(".");
  if (parts.length === 2 && parts[0] === "hosted") {
    validateResourceId(parts[1]);
    return { runtime: { location: "hosted" }, resourceId: parts[1] };
  }
  if (parts.length === 2 && parts[0] === "local") {
    validateResourceId(parts[1]);
    return { runtime: { location: "local" }, resourceId: parts[1] };
  }
  if (parts.length === 3 && parts[0] === "byoc") {
    validateResourceId(parts[2]);
    return {
      runtime: {

```

```

        location: "byoc",
        runnerId: decodeBase64url(parts[1]),
        runnerPublicKey: null,
    },
    resourceId: parts[2],
};
}
throw new Error("Runtime resource ID qualifier is invalid");
}

```

24.9 runtime/runtimeTransport.ts

The transport contract unifies repository, file, prompt, model, provider, and terminal calls across execution modes.

```

import { api, hostedApi } from "../api/client";
import {
    uploadEncryptedPiConfig,
    uploadHostedPiConfig,
} from "../encryptedUpload";
import {
    connectEncryptedRunner,
    requestEncryptedRunner,
    type EncryptedRunnerConnection,
    type EncryptedRunnerConnectionOptions,
    type EncryptedRunnerRequest,
} from "../runner/encryptedRunnerClient";

export type RuntimeRef =
    | { readonly location: "local" }
    | { readonly location: "hosted" }
    | {
        readonly location: "managed";
        readonly runnerPublicKey: string;
    }
    | {
        readonly location: "byoc";
        readonly runnerId: string;
        readonly runnerPublicKey: string;
    };

type RuntimeMethod = "DELETE" | "GET" | "PATCH" | "POST" | "PUT";

interface JsonApiClient {
    get<T>(path: string): Promise<T>;
    post<T>(path: string, body?: unknown): Promise<T>;
}

```

```

    patch<T>(path: string, body?: unknown): Promise<T>;
    put<T>(path: string, body?: unknown): Promise<T>;
    delete(path: string): Promise<void>;
    upload<T>(path: string, file: File): Promise<T>;
}

type EncryptedRequest = <T>(request: EncryptedRunnerRequest) => Promise<T>;
type ConnectEncrypted = (
    options: EncryptedRunnerConnectionOptions,
) => Promise<EncryptedRunnerConnection>;

interface RuntimeTransportDependencies {
    readonly apiClient: JsonApiClient;
    readonly encryptedRequest: EncryptedRequest;
    readonly connectEncrypted?: ConnectEncrypted;
    readonly now?: () => number;
}

export interface RuntimeTransport {
    readonly runtime: RuntimeRef;
    get<T>(path: string): Promise<T>;
    post<T>(path: string, body?: unknown): Promise<T>;
    patch<T>(path: string, body?: unknown): Promise<T>;
    put<T>(path: string, body?: unknown): Promise<T>;
    delete(path: string): Promise<void>;
    upload<T>(path: string, file: File): Promise<T>;
    close(): void;
}

const RESOURCE_ID = "[0-9a-f]{32}";
const REPOSITORY_MEMBER_PATH = new RegExp(`~/api/repos/${RESOURCE_ID}$`);
const WORKSPACE_COLLECTION_PATH = new RegExp(
    `~/api/repos/${RESOURCE_ID}/workspaces$`,
);
const WORKSPACE_MEMBER_PATH = new RegExp(`~/api/workspaces/${RESOURCE_ID}$`);
const SESSION_COLLECTION_PATH = new RegExp(
    `~/api/workspaces/${RESOURCE_ID}/sessions$`,
);
const SESSION_MEMBER_PATH = new RegExp(`~/api/sessions/${RESOURCE_ID}$`);
const SESSION_READ_PATH = new RegExp(
    `~/api/sessions/${RESOURCE_ID}/(?:messages|tree)$`,
);
const PROMPT_RUN_COLLECTION_PATH = new RegExp(
    `~/api/sessions/${RESOURCE_ID}/runs$`,
);
const PROMPT_RUN_EVENTS_PATH = new RegExp(

```



```

    ``/api/sessions/${RESOURCE_ID}/runs/${RESOURCE_ID}/events/[0-9]{1,6}$`,
);
const PROMPT_RUN_CANCEL_PATH = new RegExp(
    ``/api/sessions/${RESOURCE_ID}/runs/${RESOURCE_ID}/cancel$`,
);
const WORKSPACE_FILE_READ_PATH = new RegExp(
    ``/api/workspaces/${RESOURCE_ID}/files/(?::changed|diff|preview|tree)$`,
);
const WORKSPACE_FILE_WRITE_PATH = new RegExp(
    ``/api/workspaces/${RESOURCE_ID}/files/content$`,
);
const TERMINAL_COLLECTION_PATH = new RegExp(
    ``/api/workspaces/${RESOURCE_ID}/terminals$`,
);
const TERMINAL_MEMBER_PATH = new RegExp(
    ``/api/workspaces/${RESOURCE_ID}/terminals/${RESOURCE_ID}$`,
);
const TERMINAL_ACTION_PATH = new RegExp(
    ``/api/workspaces/${RESOURCE_ID}/terminals/${RESOURCE_ID}/(?:input|resize|restart)$`,
);
const TERMINAL_EVENTS_PATH = new RegExp(
    ``/api/workspaces/${RESOURCE_ID}/terminals/${RESOURCE_ID}/events/[0-9]{1,10}$`,
);
const PROVIDER_CONNECTION_MEMBER_PATH = new RegExp(
    ``/api/settings/connections/${RESOURCE_ID}$`,
);
const PROVIDER_AUTH_START_PATH = /\^\/auth\/providers\/[a-z0-9-]{1,64}\/start$/;
const PROVIDER_AUTH_CALLBACK_PATH =
    /\^\/auth\/providers\/[a-z0-9-]{1,64}\/callback$/;
const RUNTIME_TRANSPORT_SESSION_BYTES = 16 * 1024 * 1024;

function requiredScope(method: RuntimeMethod, path: string): string {
    const repositoryCollection = path === "/api/repos";
    const repositoryMember = REPOSITORY_MEMBER_PATH.test(path);
    if (method === "GET" && (repositoryCollection || repositoryMember)) {
        return "repository.read";
    }
    if (
        (method === "POST" || method === "PATCH" || method === "DELETE") &&
        (repositoryCollection || repositoryMember)
    ) {
        return "repository.write";
    }
}

const workspaceCollection = WORKSPACE_COLLECTION_PATH.test(path);
const workspaceMember = WORKSPACE_MEMBER_PATH.test(path);

```

```

if (method === "GET" && workspaceCollection) {
    return "workspace.read";
}
if (method === "POST" && workspaceCollection) {
    return "workspace.write";
}
if ((method === "PATCH" || method === "DELETE") && workspaceMember) {
    return "workspace.write";
}

const sessionCollection = SESSION_COLLECTION_PATH.test(path);
const sessionMember = SESSION_MEMBER_PATH.test(path);
const sessionRead = SESSION_READ_PATH.test(path);
if (method === "GET" && (sessionCollection || sessionMember || sessionRead)) {
    return "session.read";
}
if (method === "POST" && sessionCollection) {
    return "session.write";
}
if (method === "PATCH" && sessionMember) {
    return "session.write";
}

const promptRunCollection = PROMPT_RUN_COLLECTION_PATH.test(path);
const promptRunEvents = PROMPT_RUN_EVENTS_PATH.test(path);
const promptRunCancel = PROMPT_RUN_CANCEL_PATH.test(path);
if (method === "POST" && (promptRunCollection || promptRunCancel)) {
    return "session.stream";
}
if (method === "GET" && promptRunEvents) {
    return "session.stream";
}
if (method === "GET" && WORKSPACE_FILE_READ_PATH.test(path)) {
    return "files.read";
}
if (method === "PUT" && WORKSPACE_FILE_WRITE_PATH.test(path)) {
    return "files.write";
}

const terminalCollection = TERMINAL_COLLECTION_PATH.test(path);
const terminalMember = TERMINAL_MEMBER_PATH.test(path);
const terminalAction = TERMINAL_ACTION_PATH.test(path);
const terminalEvents = TERMINAL_EVENTS_PATH.test(path);
if (method === "POST" && (terminalCollection || terminalAction)) {
    return "terminal";
}
if (method === "GET" && terminalEvents) {

```

```

    return "terminal";
}
if (method === "DELETE" && terminalMember) {
    return "terminal";
}
const providerCollection = path === "/api/settings/connections";
const providerMember = PROVIDER_CONNECTION_MEMBER_PATH.test(path);
if ((method === "GET" || method === "POST") && providerCollection) {
    return "provider.configure";
}
if (method === "DELETE" && providerMember) {
    return "provider.configure";
}
if (method === "GET" && path === "/api/catalog") {
    return "provider.configure";
}
if (method === "POST" && PROVIDER_AUTH_START_PATH.test(path)) {
    return "provider.configure";
}
if (
    (method === "GET" || method === "POST") &&
    PROVIDER_AUTH_CALLBACK_PATH.test(path)
) {
    return "provider.configure";
}
if (
    (method === "GET" || method === "DELETE") &&
    path === "/api/settings/pi-config"
) {
    return "pi.configure";
}
if (
    method === "GET" &&
    (path === "/api/settings/pi-config/commands" ||
     path === "/api/settings/pi-release-notes")
) {
    return "pi.configure";
}
if (
    method === "POST" &&
    (path === "/api/settings/pi-config/github" ||
     path === "/api/settings/pi-config/sync")
) {
    return "pi.configure";
}
if (method === "PATCH" && path === "/api/settings/pi-config/categories") {

```

```

        return "pi.configure";
    }
    throw new Error("Remote runtime method or path is not allowed");
}

function parseRemotePath(path: string): {
    readonly pathname: string;
    readonly query: Readonly<Record<string, string>>;
} {
    if (typeof path !== "string" || !path.startsWith("/") || path.includes("#")) {
        throw new Error("Remote runtime path is invalid");
    }
    const [pathname, ...queryParts] = path.split("?");
    if (queryParts.length > 1 || !pathname) {
        throw new Error("Remote runtime path is invalid");
    }
    const query: Record<string, string> = {};
    if (queryParts.length === 1) {
        const rawQuery = queryParts[0];
        if (!rawQuery) {
            throw new Error("Remote runtime query is invalid");
        }
        for (const pair of rawQuery.split("&")) {
            const separatorIndex = pair.indexOf("=");
            if (separatorIndex <= 0) {
                throw new Error("Remote runtime query is invalid");
            }
            let key: string;
            let value: string;
            try {
                key = decodeURIComponent(pair.slice(0, separatorIndex));
                value = decodeURIComponent(pair.slice(separatorIndex + 1));
            } catch {
                throw new Error("Remote runtime query is invalid");
            }
            if (key in query) {
                throw new Error("Remote runtime query keys must be unique");
            }
            query[key] = value;
        }
    }
    return { pathname, query };
}

function isCanonicalRunnerPublicKey(value: string): boolean {
    if (!/^([A-Za-z0-9_-]{43}$|u.test(value))$/.test(value)) {

```

```

    return false;
}
try {
    const binary = atob(`${value.replace(/-/gu, "+").replace(/_/gu, "/")}=`);
    return (
        binary.length === 32 &&
        btoa(binary)
            .replace(/\+/gu, "-")
            .replace(/\//gu, "_")
            .replace(/=+$/u, "") === value
    );
} catch {
    return false;
}
}

function validateRuntime(runtime: RuntimeRef): void {
    if (runtime.location === "local" || runtime.location === "hosted") {
        return;
    }
    if (runtime.location === "managed") {
        if (!isCanonicalRunnerPublicKey(runtime.runnerPublicKey)) {
            throw new Error("Managed runner public key is invalid");
        }
        return;
    }
    if (!runtime.runnerId || runtime.runnerId.length > 256) {
        throw new Error("BYOC runner ID is invalid");
    }
    if (!runtime.runnerPublicKey) {
        throw new Error("BYOC runner public key is required");
    }
}

export function createRuntimeTransport(
    runtime: RuntimeRef,
    dependencies?: RuntimeTransportDependencies,
): RuntimeTransport {
    validateRuntime(runtime);
    const runtimeDependencies = dependencies ?? {
        apiClient:
            runtime.location === "hosted" && window.yinshiDesktop !== undefined
                ? hostedApi
                : api,
        encryptedRequest: requestEncryptedRunner,
        connectEncrypted: connectEncryptedRunner,
    };

```

```

    now: Date.now,
};
if (
    !runtimeDependencies.apiClient ||
    typeof runtimeDependencies.apiClient.get !== "function"
) {
    throw new TypeError("Runtime API client is invalid");
}
if (typeof runtimeDependencies.encryptedRequest !== "function") {
    throw new TypeError("Encrypted runtime request must be callable");
}
if (
    runtimeDependencies.connectEncrypted !== undefined &&
    typeof runtimeDependencies.connectEncrypted !== "function"
) {
    throw new TypeError("Encrypted runtime connector must be callable");
}
interface ManagedConnectionEntry {
    connection: Promise<EncryptedRunnerConnection>;
    tail: Promise<void>;
}

const managedConnections = new Map<string, ManagedConnectionEntry>();
let closed = false;

async function managedRequest<T>(
    scope: string,
    method: RuntimeMethod,
    pathname: string,
    query: Readonly<Record<string, string>>,
    body: unknown,
): Promise<T> {
    if (closed) {
        throw new Error("Runtime transport is closed");
    }
    const connect =
        runtimeDependencies.connectEncrypted ?? connectEncryptedRunner;
    let entry = managedConnections.get(scope);
    if (entry === undefined) {
        const connection = connect({
            expectedRunnerPublicKey:
                runtime.location === "managed" ? runtime.runnerPublicKey : "",
            scopes: [scope],
            maxSessionBytes: RUNTIME_TRANSPORT_SESSION_BYTES,
            capabilityEndpoint: "/api/runtime/capabilities",
        });

```

```

    entry = { connection, tail: Promise.resolve() };
    managedConnections.set(scope, entry);
    void connection.catch(() => {
        if (managedConnections.get(scope) === entry) {
            managedConnections.delete(scope);
        }
    });
}
const activeEntry = entry;
const operation = activeEntry.tail.then(async () => {
    let connection = await activeEntry.connection;
    if (closed) {
        connection.close();
        throw new Error("Runtime transport is closed");
    }
    const now = runtimeDependencies.now ?? Date.now;
    if (
        connection.expiresAtMs !== undefined &&
        connection.expiresAtMs <= now()
    ) {
        connection.close();
        if (closed) {
            throw new Error("Runtime transport is closed");
        }
        const replacement = connect({
            expectedRunnerPublicKey:
                runtime.location === "managed" ? runtime.runnerPublicKey : "",
            scopes: [scope],
            maxSessionBytes: RUNTIME_TRANSPORT_SESSION_BYTES,
            capabilityEndpoint: "/api/runtime/capabilities",
        });
        activeEntry.connection = replacement;
        try {
            connection = await replacement;
        } catch (error) {
            if (managedConnections.get(scope) === activeEntry) {
                managedConnections.delete(scope);
            }
            throw error;
        }
        if (closed) {
            connection.close();
            throw new Error("Runtime transport is closed");
        }
    }
}
try {

```

```

        return await connection.request<T>({
            method,
            path: pathname,
            query,
            body,
        });
    } catch (error) {
        if (managedConnections.get(scope) === activeEntry) {
            managedConnections.delete(scope);
        }
        connection.close();
        throw error;
    }
});
activeEntry.tail = operation.then(
    () => undefined,
    () => undefined,
);
return operation;
}

async function request<T>({
    method: RuntimeMethod,
    path: string,
    body?: unknown,
}): Promise<T> {
    if (runtime.location === "byoc" || runtime.location === "managed") {
        const parsedPath = parseRemotePath(path);
        const scope = requiredScope(method, parsedPath.pathname);
        if (runtime.location === "managed") {
            return managedRequest<T>({
                scope,
                method,
                parsedPath.pathname,
                parsedPath.query,
                body ?? null,
            });
        }
    }
    return runtimeDependencies.encryptedRequest<T>({
        expectedRunnerPublicKey: runtime.runnerPublicKey,
        scopes: [scope],
        method,
        path: parsedPath.pathname,
        query: parsedPath.query,
        body: body ?? null,
        maxSessionBytes: RUNTIME_TRANSPORT_SESSION_BYTES,
    });
}

```



```

    });
  }
  if (method === "GET") {
    return runtimeDependencies.apiClient.get<T>(path);
  }
  if (method === "POST") {
    return runtimeDependencies.apiClient.post<T>(path, body);
  }
  if (method === "PATCH") {
    return runtimeDependencies.apiClient.patch<T>(path, body);
  }
  if (method === "PUT") {
    return runtimeDependencies.apiClient.put<T>(path, body);
  }
  await runtimeDependencies.apiClient.delete(path);
  return undefined as T;
}

return {
  runtime,
  get: <T>(path: string) => request<T>("GET", path),
  post: <T>(path: string, body?: unknown) => request<T>("POST", path, body),
  patch: <T>(path: string, body?: unknown) => request<T>("PATCH", path, body),
  put: <T>(path: string, body?: unknown) => request<T>("PUT", path, body),
  delete: (path: string) => request<void>("DELETE", path),
  upload: <T>(path: string, file: File) => {
    if (runtime.location === "byoc" || runtime.location === "managed") {
      if (path !== "/api/settings/pi-config/upload") {
        return Promise.reject(
          new Error("Remote runtime upload path is not allowed"),
        );
      }
      return uploadEncryptedPiConfig<T>(runtime, file);
    }
    if (runtime.location === "hosted" && window.yinshiDesktop !== undefined) {
      if (path !== "/api/settings/pi-config/upload") {
        return Promise.reject(
          new Error("Hosted runtime upload path is not allowed"),
        );
      }
      return uploadHostedPiConfig<T>(file);
    }
    return runtimeDependencies.apiClient.upload<T>(path, file);
  },
  close: () => {
    if (closed) return;
  }
};

```

```

        closed = true;
        for (const entry of managedConnections.values()) {
            void entry.connection.then(
                (connection) => connection.close(),
                () => undefined,
            );
        }
        managedConnections.clear();
    },
};
}

```

24.10 runtime/terminalChannel.ts

Terminal channels preserve sequence state across reconnects and coordinate input, resize, output, and closure.

```

import type { RuntimeTransport } from "../runtimeTransport";

const RESOURCE_ID_PATTERN = /^[0-9a-f]{32}$/;
const TERMINAL_EVENT_TYPES = new Set([
    "error",
    "terminal_data",
    "terminal_exit",
    "terminal_ready",
]);

export interface RuntimeTerminalOptions {
    readonly cols: number;
    readonly rows: number;
    readonly signal?: AbortSignal;
    readonly pollDelayMs?: number;
}

export interface RuntimeTerminalEvent {
    readonly type: string;
    readonly [key: string]: unknown;
}

export interface RuntimeTerminalChannel {
    readonly id: string;
    events(): AsyncGenerator<RuntimeTerminalEvent>;
    sendInput(data: string): Promise<void>;
    resize(cols: number, rows: number): Promise<void>;
    restart(): Promise<void>;
    close(): Promise<void>;
}

```

```

}

function validateResourceId(value: unknown, field: string): string {
  if (typeof value !== "string" || !RESOURCE_ID_PATTERN.test(value)) {
    throw new Error(`Terminal ${field} is invalid`);
  }
  return value;
}

function validateSize(cols: number, rows: number): void {
  if (!Number.isSafeInteger(cols) || cols < 2 || cols > 500) {
    throw new Error("Terminal cols must be between 2 and 500");
  }
  if (!Number.isSafeInteger(rows) || rows < 2 || rows > 500) {
    throw new Error("Terminal rows must be between 2 and 500");
  }
}

function validateEvent(value: unknown): RuntimeTerminalEvent {
  if (value === null || typeof value !== "object" || Array.isArray(value)) {
    throw new Error("Terminal event is invalid");
  }
  const event = value as Record<string, unknown>;
  if (typeof event.type !== "string" || !TERMINAL_EVENT_TYPES.has(event.type)) {
    throw new Error("Terminal event type is invalid");
  }
  if (event.type === "terminal_data" && typeof event.data !== "string") {
    throw new Error("Terminal output data is invalid");
  }
  return event as RuntimeTerminalEvent;
}

function abortError(): DOMException {
  return new DOMException("Terminal polling aborted", "AbortError");
}

async function delay(
  milliseconds: number,
  signals: readonly (AbortSignal | undefined)[],
): Promise<void> {
  if (signals.some((signal) => signal?.aborted)) throw abortError();
  if (milliseconds === 0) return;
  await new Promise<void>((resolve, reject) => {
    const timer = window.setTimeout(resolve, milliseconds);
    const abort = () => {
      window.clearTimeout(timer);
    }
  })
}

```

```

        reject(abortError());
    };
    for (const signal of signals)
        signal?.addEventListener("abort", abort, { once: true });
    });
}

export async function openRuntimeTerminal(
    transport: RuntimeTransport,
    workspaceIdValue: string,
    options: RuntimeTerminalOptions,
): Promise<RuntimeTerminalChannel> {
    if (
        !transport ||
        typeof transport.post !== "function" ||
        typeof transport.get !== "function"
    ) {
        throw new TypeError("Terminal runtime transport is invalid");
    }
    const workspaceId = validateResourceId(workspaceIdValue, "workspace ID");
    validateSize(options.cols, options.rows);
    const pollDelayMs = options.pollDelayMs ?? 0;
    if (
        !Number.isSafeInteger(pollDelayMs) ||
        pollDelayMs < 0 ||
        pollDelayMs > 5_000
    ) {
        throw new Error("Terminal poll delay is invalid");
    }
    const openedValue = await transport.post<unknown>(
        `~/api/workspaces/${workspaceId}/terminals`,
        { cols: options.cols, rows: options.rows },
    );
    if (
        openedValue === null ||
        typeof openedValue !== "object" ||
        Array.isArray(openedValue)
    ) {
        throw new Error("Terminal start response is invalid");
    }
    const opened = openedValue as Record<string, unknown>;
    if (
        Object.keys(opened).length !== 3 ||
        opened.workspace_id !== workspaceId ||
        opened.status !== "attached"
    ) {

```

```

    throw new Error("Terminal start response did not match the workspace");
}
const terminalId = validateResourceId(opened.id, "channel ID");
const closeController = new AbortController();
let closed = false;
let consumed = false;

function channelPath(suffix = ""): string {
    return `/api/workspaces/${workspaceId}/terminals/${terminalId}${suffix}`;
}

return {
    id: terminalId,
    async *events(): AsyncGenerator<RuntimeTerminalEvent> {
        if (consumed)
            throw new Error("Terminal event stream can only be consumed once");
        consumed = true;
        let nextSequence = 0;
        let retryDelayMs = pollDelayMs;
        while (!closed) {
            if (options.signal?.aborted || closeController.signal.aborted)
                throw abortError();
            let rawBatch: unknown;
            try {
                rawBatch = await transport.get<unknown>(
                    channelPath(`/events/${nextSequence}`),
                );
                retryDelayMs = pollDelayMs;
            } catch (error) {
                if (error instanceof DOMException && error.name === "AbortError") {
                    throw error;
                }
                if (
                    error !== null &&
                    typeof error === "object" &&
                    "status" in error
                ) {
                    const status = (error as { status?: unknown }).status;
                    if (typeof status === "number" && status >= 400 && status < 500)
                        throw error;
                }
                retryDelayMs = Math.min(Math.max(retryDelayMs * 2, 100), 5_000);
                await delay(retryDelayMs, [options.signal, closeController.signal]);
                continue;
            }
            if (

```

```

        rawBatch === null ||
        typeof rawBatch !== "object" ||
        Array.isArray(rawBatch)
    ) {
        throw new Error("Terminal event batch is invalid");
    }
    const batch = rawBatch as Record<string, unknown>;
    if (
        Object.keys(batch).length !== 4 ||
        batch.terminal_id !== terminalId ||
        !Array.isArray(batch.events) ||
        !Number.isSafeInteger(batch.next_sequence) ||
        typeof batch.closed !== "boolean"
    ) {
        throw new Error("Terminal event batch did not match the channel");
    }
    const events = batch.events.map(validateEvent);
    const serverSequence = batch.next_sequence as number;
    if (serverSequence !== nextSequence + events.length) {
        throw new Error("Terminal event sequence is not contiguous");
    }
    nextSequence = serverSequence;
    for (const event of events) yield event;
    if (batch.closed) {
        closed = true;
        closeController.abort();
        await transport.delete(channelPath());
        return;
    }
    await delay(pollDelayMs, [options.signal, closeController.signal]);
},
async sendInput(data: string): Promise<void> {
    if (closed) throw new Error("Terminal channel is closed");
    if (
        typeof data !== "string" ||
        !data ||
        new TextEncoder().encode(data).length > 16_384
    ) {
        throw new Error("Terminal input has an invalid length");
    }
    await transport.post(channelPath("/input"), { data });
},
async resize(cols: number, rows: number): Promise<void> {
    if (closed) throw new Error("Terminal channel is closed");
    validateSize(cols, rows);

```

```

    await transport.post(channelPath("/resize"), { cols, rows });
  },
  async restart(): Promise<void> {
    if (closed) throw new Error("Terminal channel is closed");
    await transport.post(channelPath("/restart"));
  },
  async close(): Promise<void> {
    if (closed) return;
    closed = true;
    closeController.abort();
    await transport.delete(channelPath());
  },
};
}

```

24.11 runtime/useRuntimeResource.ts

This hook cancels stale requests and reloads runtime-backed data when identity or dependencies change.

```

import { useEffect, useState } from "react";

import { parseRuntimeResourceId } from "../runtimeRef";
import { resolveRuntimeRef } from "../resolveRuntime";
import {
  createRuntimeTransport,
  type RuntimeRef,
  type RuntimeTransport,
} from "../runtimeTransport";

export interface ResolvedRuntimeResource {
  readonly resourceId: string;
  readonly runtime: RuntimeRef;
  readonly transport: RuntimeTransport;
}

export interface RuntimeResourceState {
  readonly resource: ResolvedRuntimeResource | null;
  readonly loading: boolean;
  readonly error: string | null;
}

export function useRuntimeResource(encodedId: string | undefined): RuntimeResourceState {
  const [state, setState] = useState<RuntimeResourceState>({
    resource: null,
    loading: true,

```

```

        error: null,
    });

    useEffect(() => {
        let cancelled = false;
        let transport: RuntimeTransport | null = null;
        if (!encodedId) {
            setState({ resource: null, loading: false, error: "Runtime resource ID is missing" });
            return () => {
                cancelled = true;
            };
        }

        const controller = new AbortController();
        setState({ resource: null, loading: true, error: null });
        void (async () => {
            try {
                const parsed = parseRuntimeResourceId(encodedId, {
                    desktop: window.yinshiDesktop !== undefined,
                });
                const runtime = await resolveRuntimeRef(
                    parsed.runtime,
                    {},
                    controller.signal,
                );
                if (!cancelled) {
                    transport = createRuntimeTransport(runtime);
                    setState({
                        resource: {
                            resourceId: parsed.resourceId,
                            runtime,
                            transport,
                        },
                        loading: false,
                        error: null,
                    });
                }
            } catch (error) {
                if (!cancelled) {
                    setState({
                        resource: null,
                        loading: false,
                        error: error instanceof Error ? error.message : "Runtime could not be resolved",
                    });
                }
            }
        })
    });

```



```

    })();

    return () => {
        cancelled = true;
        controller.abort();
        transport?.close();
    };
}, [encodedId]);

return state;
}

```

25 Sidecar Runtime

The sidecar is the execution boundary closest to the agent. It listens on a Unix socket and creates Pi SDK sessions. It manages tools, terminals, catalog data, and runtime Git credentials.

25.1 constants.js

Shared constants keep sidecar timing values consistent across modules. The root chunk below tangles back to `sidecar/src/constants.js`.

```
export const HEALTH_CHECK_INTERVAL = 30000;
```

25.2 index.js

The sidecar process entry point creates a sidecar server and starts listening on the configured Unix socket. The root chunk below tangles back to `sidecar/src/index.js`.

```
import { YinshiSidecar } from "../sidecar.js";

async function main() {
    const sidecar = new YinshiSidecar();
    let shuttingDown = false;

    const shutdown = () => {
        if (shuttingDown) {
            return;
        }
        shuttingDown = true;
        process.removeListener("SIGINT", shutdown);
        process.removeListener("SIGTERM", shutdown);
        sidecar.cleanup();
        process.exit(0);
    };
}

```

```

};

process.once("SIGINT", shutdown);
process.once("SIGTERM", shutdown);

sidecar.initialize();

await sidecar.start();
}

main().catch((err) => {
  console.error("[sidecar] Fatal runtime error");
  process.exit(1);
});

```

25.3 sidecar.js

The sidecar server hosts the Pi SDK bridge, model catalog, runtime settings, prompt execution, and terminal lifecycle. The root chunk below tangles back to `sidecar/src/sidecar.js`.

```

import fs from "node:fs";
import net from "node:net";
import os from "node:os";
import path from "node:path";
import { randomUUID } from "node:crypto";
import { fileURLToPath } from "node:url";
import * as pty from "node-pty";

import {
  createAgentSession,
  DefaultResourceLoader,
  ModelRegistry,
  ModelRuntime,
  SessionManager,
  SettingsManager,
  createEditTool,
  createReadTool,
  createWriteTool,
} from "@earendil-works/pi-coding-agent";
import {
  getSupportedThinkingLevels,
  InMemoryCredentialStore,
} from "@earendil-works/pi-ai";

import { HEALTH_CHECK_INTERVAL } from "../constants.js";

```

```

import { createGitAwareBashTool } from "./git_auth.js";

const __sidecarDir = path.dirname(fileURLToPath(import.meta.url));
const PI_PACKAGE_NAME = "@earendil-works/pi-coding-agent";
const DEFAULT_MODEL_REF = "minimax/MiniMax-M2.7";
const DEFAULT_THINKING_LEVEL = "medium";
const OFF_THINKING_LEVEL = "off";
const STANDARD_THINKING_LEVELS = ["off", "minimal", "low", "medium", "high"];
const XHIGH_THINKING_LEVELS = [...STANDARD_THINKING_LEVELS, "xhigh"];
const THINKING_LEVELS = new Set(XHIGH_THINKING_LEVELS);
const OAUTH_FLOW_COUNT_MAX = 8;
const OAUTH_FLOW_TTL_MS = 30 * 60 * 1000;
const PI_SESSION_IDLE_TTL_MS = 30 * 60 * 1000;
const PI_SESSION_COUNT_MAX = 16;
const LEGACY_MODEL_ALIASES = new Map([
  ["haiku", "anthropic/claude-haiku-4-5-20251001"],
  ["minimax", DEFAULT_MODEL_REF],
  ["minimax-m2.5-highspeed", "minimax/MiniMax-M2.5-highspeed"],
  ["minimax-m2.7", DEFAULT_MODEL_REF],
  ["minimax-m2.7-highspeed", "minimax/MiniMax-M2.7-highspeed"],
  ["opus", "anthropic/claude-opus-4-20250514"],
  ["sonnet", "anthropic/claude-sonnet-4-20250514"],
]);
const TERMINAL_ENVIRONMENT_KEYS = [
  "HOME",
  "LANG",
  "LC_ALL",
  "LOGNAME",
  "NPM_CONFIG_PREFIX",
  "PATH",
  "PIPX_BIN_DIR",
  "PIPX_HOME",
  "TZ",
  "USER",
  "YINSHI_WORKSPACE_ID",
];

function sendToSocket(socket, message) {
  if (socket.destroyed) {
    return;
  }
  socket.write(`${JSON.stringify(message)}\n`);
}

function sendStatusToSocket(
  socket,

```

```

    sessionId,
    status,
    message,
    severity = "info",
    extra = {},
  ) {
    sendToSocket(socket, {
      id: sessionId,
      type: "status",
      status,
      severity,
      message,
      ...extra,
    });
  }

function normalizePositiveInteger(value, fallback, minValue, maxValue) {
  const numericValue = Number(value);
  if (!Number.isFinite(numericValue)) {
    return fallback;
  }
  const integerValue = Math.trunc(numericValue);
  if (integerValue < minValue) {
    return minValue;
  }
  if (integerValue > maxValue) {
    return maxValue;
  }
  return integerValue;
}

function normalizeTerminalId(value) {
  if (typeof value !== "string") {
    throw new Error("terminal workspaceId must be a string");
  }
  const normalized = value.trim();
  if (!/^([0-9a-f]{32})$/i.test(normalized)) {
    throw new Error("terminal workspaceId must be a 32-character hex string");
  }
  return normalized;
}

function normalizeTerminalCwd(value) {
  if (typeof value !== "string") {
    throw new Error("terminal cwd must be a string");
  }
}

```

```

    const normalized = value.trim();
    if (!path.isAbsolute(normalized)) {
      throw new Error("terminal cwd must be absolute");
    }
    if (!fs.existsSync(normalized)) {
      throw new Error("terminal cwd does not exist");
    }
    return normalized;
  }

export function buildTerminalEnvironment(cwd, shell) {
  const terminalEnvironment = {};
  for (const key of TERMINAL_ENVIRONMENT_KEYS) {
    if (process.env[key]) {
      terminalEnvironment[key] = process.env[key];
    }
  }
  terminalEnvironment.HOME = terminalEnvironment.HOME || "/home/yinshi";
  terminalEnvironment.PATH = terminalEnvironment.PATH || "/usr/local/bin:/usr/bin:/bin";
  terminalEnvironment.SHELL = shell;
  terminalEnvironment.TERM = "xterm-256color";
  terminalEnvironment.COLORTERM = "truecolor";
  terminalEnvironment.PWD = cwd;
  return terminalEnvironment;
}

function appendTerminalScrollback(entry, data) {
  if (typeof data !== "string" || data.length === 0) {
    return;
  }
  entry.scrollback += data;
  if (entry.scrollback.length > entry.scrollbackLimitBytes) {
    entry.scrollback = entry.scrollback.slice(-entry.scrollbackLimitBytes);
  }
}

function sendTerminalError(socket, id, err, fallback) {
  sendToSocket(socket, {
    id: id || "terminal",
    type: "error",
    error: err instanceof Error ? err.message : fallback,
  });
}

function normalizeImportedSettings(importedSettings) {
  if (importedSettings === null || importedSettings === undefined) {

```

```

        return null;
    }
    if (typeof importedSettings !== "object" || Array.isArray(importedSettings)) {
        throw new Error("Imported settings must be an object");
    }

    const normalizedSettings = { ...importedSettings };
    if (Object.prototype.hasOwnProperty.call(normalizedSettings, "thinking")) {
        const thinkingOverride = normalizedSettings.thinking;
        if (typeof thinkingOverride !== "boolean") {
            throw new Error("Imported thinking override must be a boolean");
        }
        delete normalizedSettings.thinking;
        if (thinkingOverride) {
            const requestedLevel = normalizedSettings.defaultThinkingLevel;
            if (
                !THINKING_LEVELS.has(requestedLevel)
                || requestedLevel === OFF_THINKING_LEVEL
            ) {
                normalizedSettings.defaultThinkingLevel = DEFAULT_THINKING_LEVEL;
            }
        } else {
            normalizedSettings.defaultThinkingLevel = OFF_THINKING_LEVEL;
        }
    }
    return normalizedSettings;
}

function normalizePiSessionFile(piSessionFile) {
    if (piSessionFile === null || piSessionFile === undefined) {
        return null;
    }
    if (typeof piSessionFile !== "string") {
        throw new Error("piSessionFile must be a string");
    }
    const normalizedPath = piSessionFile.trim();
    if (!normalizedPath) {
        throw new Error("piSessionFile must not be empty");
    }
    if (normalizedPath.includes("\0")) {
        throw new Error("piSessionFile must not contain NUL");
    }
    return normalizedPath;
}

function openSessionManager(cwd, normalizedSessionFile) {

```

```

if (!normalizedSessionFile) {
  return {
    sessionManager: SessionManager.inMemory(),
    resetWarning: null,
    piSessionFile: null,
  };
}

const sessionDir = path.dirname(normalizedSessionFile);
fs.mkdirSync(sessionDir, { recursive: true, mode: 0o700 });
try {
  return {
    sessionManager: SessionManager.open(normalizedSessionFile, sessionDir, cwd),
    resetWarning: null,
    piSessionFile: normalizedSessionFile,
  };
} catch {
  const suffix = new Date().toISOString().replace(/[:.]/g, "-");
  const corruptPath = `${normalizedSessionFile}.corrupt-${suffix}`;
  if (fs.existsSync(normalizedSessionFile)) {
    fs.renameSync(normalizedSessionFile, corruptPath);
  }
  return {
    sessionManager: SessionManager.open(normalizedSessionFile, sessionDir, cwd),
    resetWarning:
      "Pi session context was missing or unreadable, so Yinshi started a fresh model context",
    piSessionFile: normalizedSessionFile,
  };
}
}

function stringifyToolContent(content) {
  if (typeof content === "string") {
    return content;
  }
  if (!content || typeof content !== "object") {
    return String(content ?? "");
  }
  if (content.type === "text" && typeof content.text === "string") {
    return content.text;
  }
  if (content.type === "image") {
    return "[image]";
  }
  return JSON.stringify(content);
}

```

```

function stringifyToolResult(result) {
  if (result === null || result === undefined) {
    return "";
  }
  if (typeof result === "string") {
    return result;
  }
  if (typeof result !== "object") {
    return String(result);
  }
  if (Array.isArray(result.content)) {
    return result.content.map(stringifyToolContent).filter(Boolean).join("\n");
  }
  return JSON.stringify(result, null, 2);
}

function normalizeModelLookup(modelKey) {
  if (typeof modelKey !== "string") {
    return "";
  }
  const trimmedKey = modelKey.trim();
  if (!trimmedKey) {
    return "";
  }
  const normalizedKey = trimmedKey.toLowerCase();
  if (LEGACY_MODEL_ALIASES.has(normalizedKey)) {
    return LEGACY_MODEL_ALIASES.get(normalizedKey) || "";
  }
  return trimmedKey;
}

function buildModelsJsonPath(agentDir) {
  if (!agentDir || typeof agentDir !== "string") {
    return null;
  }
  const modelsJsonPath = path.join(agentDir, "models.json");
  if (!fs.existsSync(modelsJsonPath)) {
    return null;
  }
  return modelsJsonPath;
}

function createYinshiCodingTools(cwd, gitAuth) {
  return [
    createReadTool(cwd),
  ]
}

```



```

        createGitAwareBashTool(cwd, gitAuth),
        createEditTool(cwd),
        createWriteTool(cwd),
    ];
}

// Pi's Theme is a color/styling helper for terminal output. In a web chat
// we have no ANSI support, so every helper returns the text unchanged. The
// shape matches interactive/theme/theme.d.ts so extensions calling
// ctx.ui.theme.fg(...) or ctx.ui.theme.strikethrough(...) don't throw.
function createPassthroughTheme() {
    const passthrough = (_color, text) => (typeof text === "string" ? text : String(text ?? ""));
    const onlyText = (text) => (typeof text === "string" ? text : String(text ?? ""));
    return {
        fg: passthrough,
        bg: passthrough,
        bold: onlyText,
        italic: onlyText,
        underline: onlyText,
        inverse: onlyText,
        strikethrough: onlyText,
        getFgAnsi() {
            return "";
        },
        getBgAnsi() {
            return "";
        },
        getColorMode() {
            return "none";
        },
        getThinkingBorderColor() {
            return onlyText;
        },
        getBashModeBorderColor() {
            return onlyText;
        },
        name: "web",
        path: undefined,
    };
}

```

```

// Extensions (rtk-metrics, plan-mode, etc.) drive their output through the
// same ExtensionUIContext that pi's interactive TUI implements. Without a
// bound context every method throws or no-ops and the command output never
// reaches the user. This adapter fills in the full surface: notify() is
// forwarded as chat text, dialog methods explain the limitation, and

```

```

// text-styling/theme helpers are passthroughs so calls like
// ctx.wi.theme.fg("accent", "...") don't throw inside a handler.
function createWebUIContext(sessionId, socket, model) {
  function emitAssistantText(message) {
    const text = typeof message === "string" ? message : String(message ?? "");
    sendToSocket(socket, {
      id: sessionId,
      type: "message",
      data: {
        type: "assistant",
        message: { content: [{ type: "text", text }] },
      },
    });
  }

  function emitWithLevel(message, level) {
    const prefix = level === "error" ? "Error: " : level === "warning" ? "Warning: " : "";
    emitAssistantText(prefix + (typeof message === "string" ? message : String(message ?? "")));
  }

  const theme = createPassthroughTheme();

  return {
    // notifications
    notify(message, level = "info") {
      console.log(`[sidecar][ui.notify] level=${level}`);
      emitWithLevel(message, level);
    },
    // status/widget/title (TUI-only surfaces)
    // Accept the calls so plan-mode and friends don't throw; the web UI
    // doesn't render these yet, so they're ignored rather than displayed.
    setStatus() {},
    setWorkingMessage() {},
    setWidget() {},
    setHeader() {},
    setFooter() {},
    setTitle() {},
    // interactive dialogs
    // None of these have a web equivalent yet. Emit a brief explanation
    // so the user understands why a command that would prompt in local
    // pi just terminates here, and return sensible defaults.
    async select() {
      emitAssistantText(
        "Interactive selection is not yet supported in the web UI; cancelling the prompt.",
      );
      return undefined;
    }
  }
}

```

```

},
async confirm() {
  emitAssistantText(
    "Interactive confirmation is not yet supported in the web UI; defaulting to no.",
  );
  return false;
},
async input() {
  emitAssistantText(
    "Interactive text input is not yet supported in the web UI; cancelling the prompt.",
  );
  return undefined;
},
async editor() {
  emitAssistantText(
    "The multi-line editor is not yet supported in the web UI; cancelling the prompt.",
  );
  return undefined;
},
async custom() {
  emitAssistantText(
    "Custom overlays are not yet supported in the web UI; cancelling the prompt.",
  );
  return undefined;
},
// editor shims (no-op since there's no pi TUI input line)
pasteToEditor() {},
setEditorText() {},
getEditorText() {
  return "";
},
setEditorComponent() {},
onTerminalInput() {
  return () => {};
},
// theme surface
theme,
getAllThemes() {
  return [{ name: theme.name, path: undefined }];
},
getTheme() {
  return theme;
},
setTheme() {
  return { success: false, error: "Theme switching is not supported in the web UI" };
},

```

```

    // tool output expansion (TUI setting, N/A for chat)
    getToolsExpanded() {
        return true;
    },
    setToolsExpanded() {},
    // Defensive metadata in case extensions read non-standard properties.
    modelName: model?.name,
  };
}

function normalizeApiKeyWithConfigSecret(secret) {
  if (typeof secret === "string") {
    const normalizedSecret = secret.trim();
    if (!normalizedSecret) {
      throw new Error("API key + config auth requires a non-empty apiKey");
    }
    return { apiKey: normalizedSecret };
  }
  if (!secret || typeof secret !== "object" || Array.isArray(secret)) {
    throw new Error("API key + config auth requires an object secret");
  }
  const normalizedSecret = {};
  for (const [key, value] of Object.entries(secret)) {
    if (typeof value !== "string") {
      throw new Error(`API key + config secret field ${key} must be a string`);
    }
    const trimmedValue = value.trim();
    if (!trimmedValue) {
      throw new Error(`API key + config secret field ${key} must not be empty`);
    }
    normalizedSecret[key] = trimmedValue;
  }
  if (typeof normalizedSecret.apiKey !== "string" || !normalizedSecret.apiKey) {
    throw new Error("API key + config auth requires an apiKey field");
  }
  return normalizedSecret;
}

async function writeCredential(credentials, provider, credential) {
  await credentials.modify(
    provider,
    async () => credential,
    { signal: AbortSignal.timeout(5_000) },
  );
}

```

```

async function createCredentialStore(providerAuth) {
  const credentials = new InMemoryCredentialStore();
  if (!providerAuth || typeof providerAuth !== "object") {
    return credentials;
  }
  if (typeof providerAuth.provider !== "string" || !providerAuth.provider) {
    throw new Error("providerAuth.provider must be a non-empty string");
  }
  if (typeof providerAuth.authStrategy !== "string" || !providerAuth.authStrategy) {
    throw new Error("providerAuth.authStrategy must be a non-empty string");
  }
  if (providerAuth.authStrategy === "api_key") {
    if (typeof providerAuth.secret !== "string" || !providerAuth.secret) {
      throw new Error("API key auth requires a non-empty secret");
    }
    await writeCredential(credentials, providerAuth.provider, {
      type: "api_key",
      key: providerAuth.secret,
    });
    return credentials;
  }
  if (providerAuth.authStrategy === "api_key_with_config") {
    const normalizedSecret = normalizeApiKeyWithConfigSecret(providerAuth.secret);
    const { apiKey, ...env } = normalizedSecret;
    await writeCredential(credentials, providerAuth.provider, {
      type: "api_key",
      key: apiKey,
      env,
    });
    return credentials;
  }
  if (providerAuth.authStrategy === "oauth") {
    if (!providerAuth.secret || typeof providerAuth.secret !== "object" || Array.isArray(providerAuth.secret)) {
      throw new Error("OAuth auth requires an object secret");
    }
    await writeCredential(credentials, providerAuth.provider, {
      type: "oauth",
      ...providerAuth.secret,
    });
    return credentials;
  }
  throw new Error(`Unsupported auth strategy: ${providerAuth.authStrategy}`);
}

async function createModelRegistry(providerAuth, agentDir) {
  const credentials = await createCredentialStore(providerAuth);

```

```

const modelsPath = buildModelsJsonPath(agentDir);
// Pass null when there is no imported agentDir so the SDK does not fall back
// to the host machine's ~/.pi/agent/models.json or auth.json.
const modelRuntime = await ModelRuntime.create({
  credentials,
  modelsPath,
  allowModelNetwork: false,
  refreshOnCreate: false,
});
const registry = new ModelRegistry(modelRuntime);
return { credentials, modelRuntime, registry };
}

function getThinkingLevels(model) {
  if (!model.reasoning) {
    return [OFF_THINKING_LEVEL];
  }
  const supportedLevels = new Set(getSupportedThinkingLevels(model));
  return supportedLevels.has("xhigh") ? XHIGH_THINKING_LEVELS : STANDARD_THINKING_LEVELS;
}

function toCatalogModel(model) {
  return {
    ref: `${model.provider}/${model.id}`,
    provider: model.provider,
    id: model.id,
    label: model.name,
    api: model.api,
    reasoning: Boolean(model.reasoning),
    thinking_levels: getThinkingLevels(model),
    inputs: [...model.input],
    context_window: model.contextWindow,
    max_tokens: model.maxTokens,
  };
}

function toCatalogProvider(providerId, models) {
  return {
    id: providerId,
    model_count: models.length,
  };
}

async function getCatalog(agentDir) {
  const { registry } = await createModelRegistry(null, agentDir);
  const models = registry.getAll().map(toCatalogModel);

```

```

const providerIds = new Set(models.map((model) => model.provider));
const providers = [...providerIds]
  .sort()
  .map((providerId) => {
    const providerModels = models.filter((model) => model.provider === providerId);
    return toCatalogProvider(providerId, providerModels);
  });
return {
  default_model: DEFAULT_MODEL_REF,
  providers,
  models,
};
}

function findPackageJson(packageName) {
  if (typeof packageName !== "string" || !packageName) {
    throw new Error("packageName must be a non-empty string");
  }

  let currentPath = __sidecarDir;
  while (true) {
    const packageJsonPath = path.join(currentPath, "node_modules", packageName, "package.json");
    if (fs.existsSync(packageJsonPath)) {
      const packageJson = JSON.parse(fs.readFileSync(packageJsonPath, "utf-8"));
      if (packageJson.name === packageName) {
        return packageJson;
      }
    }
    const parentPath = path.dirname(currentPath);
    if (parentPath === currentPath) {
      break;
    }
    currentPath = parentPath;
  }
  throw new Error(`Unable to locate package.json for ${packageName}`);
}

function getRuntimeVersion() {
  const packageJson = findPackageJson(PI_PACKAGE_NAME);
  if (typeof packageJson.version !== "string" || !packageJson.version) {
    throw new Error(`${PI_PACKAGE_NAME} package.json is missing a version`);
  }
  return {
    package_name: PI_PACKAGE_NAME,
    installed_version: packageJson.version,
    node_version: process.version,
  };
}

```

```

    };
}

// Pi skill/prompt/extension names come from user-uploaded zips. Constrain them
// to a conservative character class so they can't break palette rendering or
// leak control chars into any downstream string interpolation. 64 chars matches
// pi's own skill-name validation.
const NAME_PATTERN = /^[a-zA-Z0-9_.\-]{1,64}$/;
const DESCRIPTION_LENGTH_MAX = 240;
// Hard cap per category so a pathological zip can't balloon the JSON line or
// the browser palette. Real configs are usually dozens; 500 is generous.
const COMMANDS_PER_CATEGORY_MAX = 500;

function safeName(rawName) {
  if (typeof rawName !== "string") {
    return null;
  }
  return NAME_PATTERN.test(rawName) ? rawName : null;
}

function safeDescription(rawDescription) {
  if (typeof rawDescription !== "string") {
    return "";
  }
  return rawDescription.length > DESCRIPTION_LENGTH_MAX
    ? rawDescription.slice(0, DESCRIPTION_LENGTH_MAX)
    : rawDescription;
}

function skillToCommand(skill) {
  const name = safeName(skill?.name);
  if (name === null) {
    return null;
  }
  return {
    kind: "skill",
    name,
    description: safeDescription(skill?.description),
    command_name: `skill:${name}`,
  };
}

function promptToCommand(prompt) {
  const name = safeName(prompt?.name);
  if (name === null) {
    return null;
  }

```



```

    }
    return {
      kind: "prompt",
      name,
      description: safeDescription(prompt?.description),
      command_name: name,
    };
  }

function extensionToCommands(extension) {
  // Pi exposes extension-registered slash commands via the Extension.commands map
  // populated during loader.reload(); keys are the invocation names.
  if (!extension || !(extension.commands instanceof Map)) {
    return [];
  }
  const commands = [];
  for (const [rawCommandName, registered] of extension.commands.entries()) {
    const name = safeName(rawCommandName);
    if (name === null) {
      continue;
    }
    commands.push({
      kind: "extension",
      name,
      description: safeDescription(registered?.description),
      command_name: name,
    });
  }
  return commands;
}

// Cache the list_resources payload per agentDir keyed by the most-recent
// mtime across the agent tree. Without this, every session mount re-evaluates
// every extension module via jiti (moduleCache:false in the pi SDK), which
// DoS-exposes the shared Node event loop.
const _listResourcesCache = new Map();

async function _collectAgentDirMtime(agentDir) {
  // Find the max mtime across the agent tree. A skill file edit, a new
  // extension dropped in, or a category-toggle rename all change this.
  let maxMtimeMs = 0;
  const pending = [agentDir];
  while (pending.length > 0) {
    const current = pending.pop();
    let stats;
    try {

```

```

        stats = await fs.promises.stat(current);
    } catch {
        continue;
    }
    if (stats.mtimeMs > maxMtimeMs) {
        maxMtimeMs = stats.mtimeMs;
    }
    if (!stats.isDirectory()) {
        continue;
    }
    let entries;
    try {
        entries = await fs.promises.readdir(current);
    } catch {
        continue;
    }
    for (const entry of entries) {
        pending.push(path.join(current, entry));
    }
}
return maxMtimeMs;
}

function _capCommands(commands) {
    return commands.length > COMMANDS_PER_CATEGORY_MAX
        ? commands.slice(0, COMMANDS_PER_CATEGORY_MAX)
        : commands;
}

async function listResources(agentDir) {
    // Without an imported agentDir we intentionally return nothing: the SDK would
    // otherwise fall back to the host's ~/.pi/agent and leak host-local resources.
    if (!agentDir || typeof agentDir !== "string") {
        return { commands: [] };
    }

    const mtimeMs = await _collectAgentDirMtime(agentDir);
    const cached = _listResourcesCache.get(agentDir);
    if (cached && cached.mtimeMs === mtimeMs) {
        return cached.payload;
    }

    const loader = new DefaultResourceLoader({
        // Use a neutral cwd so project-local .pi/ directories under the sidecar's
        // working directory don't bleed into this user's resource listing.
        cwd: os.tmpdir(),
    });

```

```

        agentDir,
    });
    await loader.reload();

    const skillsResult = loader.getSkills();
    const promptsResult = loader.getPrompts();
    const extensionsResult = loader.getExtensions();

    const skills = Array.isArray(skillsResult?.skills) ? skillsResult.skills : [];
    const prompts = Array.isArray(promptsResult?.prompts) ? promptsResult.prompts : [];
    const extensions = Array.isArray(extensionsResult?.extensions) ? extensionsResult.extensions : [];

    const skillCommands = _capCommands(skills.map(skillToCommand).filter((c) => c !== null));
    const promptCommands = _capCommands(prompts.map(promptToCommand).filter((c) => c !== null));
    const extensionCommands = _capCommands(extensions.flatMap(extensionToCommands));

    const payload = {
        commands: [...skillCommands, ...promptCommands, ...extensionCommands],
    };
    _listResourcesCache.set(agentDir, { mtimeMs, payload });
    return payload;
}

function _normalizeManualInputPrompt(promptMessage) {
    if (typeof promptMessage === "string") {
        const normalizedPromptMessage = promptMessage.trim();
        if (normalizedPromptMessage) {
            return normalizedPromptMessage;
        }
    }
    return "Paste the final redirect URL or authorization code here.";
}

function _buildHostedCallbackInstructions(baseInstructions) {
    const instructionParts = [];
    if (typeof baseInstructions === "string") {
        const normalizedBaseInstructions = baseInstructions.trim();
        if (normalizedBaseInstructions) {
            instructionParts.push(normalizedBaseInstructions);
        }
    }
    instructionParts.push(
        "If the browser lands on a localhost URL and shows an error, copy the full URL from the error message."
    );
    return instructionParts.join(" ");
}

```

```

function _waitForOAuthManualInput(flow, promptMessage) {
  if (!flow || typeof flow !== "object") {
    throw new Error("OAuth flow is required");
  }
  if (flow.manualInputSubmitted) {
    if (typeof flow.manualInputValue !== "string" || !flow.manualInputValue) {
      throw new Error("Submitted OAuth manual input is missing");
    }
    return Promise.resolve(flow.manualInputValue);
  }
  flow.manualInputRequired = true;
  flow.manualInputPrompt = _normalizeManualInputPrompt(promptMessage);
  if (flow.manualInputPromise) {
    return flow.manualInputPromise;
  }
  flow.manualInputPromise = new Promise((resolve, reject) => {
    flow.manualInputResolve = resolve;
    flow.manualInputReject = reject;
  });
  return flow.manualInputPromise;
}

function _submitOAuthManualInput(flow, authorizationInput) {
  if (!flow || typeof flow !== "object") {
    throw new Error("OAuth flow is required");
  }
  if (typeof authorizationInput !== "string") {
    throw new Error("authorizationInput must be a string");
  }
  const normalizedAuthorizationInput = authorizationInput.trim();
  if (!normalizedAuthorizationInput) {
    throw new Error("authorizationInput must not be empty");
  }
  if (flow.manualInputSubmitted) {
    throw new Error("OAuth manual input was already submitted");
  }
  flow.manualInputRequired = true;
  flow.manualInputSubmitted = true;
  flow.manualInputValue = normalizedAuthorizationInput;
  flow.progress.push("Received manual OAuth callback input.");
  if (flow.manualInputResolve) {
    flow.manualInputResolve(normalizedAuthorizationInput);
    flow.manualInputResolve = null;
    flow.manualInputReject = null;
  } else {

```

```

        flow.manualInputPromise = Promise.resolve(normalizedAuthorizationInput);
    }
}

function resolveModelFromRegistry(registry, modelKey, providerConfig) {
    const normalizedLookup = normalizeModelLookup(modelKey || DEFAULT_MODEL_REF);
    const models = registry.getAll();

    if (normalizedLookup.includes("/")) {
        const slashIndex = normalizedLookup.indexOf("/");
        const provider = normalizedLookup.slice(0, slashIndex);
        const modelId = normalizedLookup.slice(slashIndex + 1);
        const resolved = registry.find(provider, modelId);
        if (!resolved) {
            throw new Error(`Unknown model: ${modelKey}`);
        }
        return applyProviderConfig(resolved, providerConfig);
    }

    const directMatches = models.filter(
        (model) => model.id.toLowerCase() === normalizedLookup.toLowerCase(),
    );
    if (directMatches.length === 1) {
        return applyProviderConfig(directMatches[0], providerConfig);
    }

    const labelMatches = models.filter(
        (model) => model.name.toLowerCase() === normalizedLookup.toLowerCase(),
    );
    if (labelMatches.length === 1) {
        return applyProviderConfig(labelMatches[0], providerConfig);
    }

    throw new Error(`Unknown model: ${modelKey}`);
}

async function resolveModel(modelKey, providerAuth, agentDir, providerConfig) {
    const { registry } = await createModelRegistry(providerAuth, agentDir);
    return resolveModelFromRegistry(registry, modelKey, providerConfig);
}

function applyProviderConfig(model, providerConfig) {
    if (!providerConfig || typeof providerConfig !== "object") {
        return model;
    }
    if (model.provider !== "azure-openai-responses") {

```

```

    return model;
}

const configuredModel = { ...model };
if (typeof providerConfig.baseUrl === "string" && providerConfig.baseUrl.trim()) {
    configuredModel.baseUrl = providerConfig.baseUrl.trim();
}
if (typeof providerConfig.azureDeploymentName === "string" && providerConfig.azureDeploymentName.trim()) {
    configuredModel.azureDeploymentName = providerConfig.azureDeploymentName.trim();
}
return configuredModel;
}

async function resolveProviderRuntimeAuth(provider, modelRef, providerAuth, agentDir, providerConfig) {
    if (!providerAuth || typeof providerAuth !== "object") {
        return {
            provider,
            auth: null,
            model_ref: modelRef,
            runtime_api_key: null,
            model_config: providerConfig || null,
        };
    }

    const { credentials, modelRuntime, registry } = await createModelRegistry(providerAuth, agentDir);
    const runtimeAuth = await modelRuntime.getAuth(provider);
    const runtimeApiKey = runtimeAuth?.auth.apiKey;
    const credential = await credentials.read(provider);
    const resolvedModel = resolveModelFromRegistry(registry, modelRef, providerConfig);
    const modelConfig = {};
    if (resolvedModel.provider === "github-copilot" && typeof resolvedModel.baseUrl === "string") {
        modelConfig.baseUrl = resolvedModel.baseUrl;
    }
    if (resolvedModel.provider === "azure-openai-responses") {
        if (typeof resolvedModel.baseUrl === "string" && resolvedModel.baseUrl) {
            modelConfig.baseUrl = resolvedModel.baseUrl;
        }
        if (typeof resolvedModel.azureDeploymentName === "string" && resolvedModel.azureDeploymentName) {
            modelConfig.azureDeploymentName = resolvedModel.azureDeploymentName;
        }
    }
    let returnedAuth = providerAuth.secret ?? null;
    if (providerAuth.authStrategy === "oauth") {
        returnedAuth = credential || null;
    }
    if (providerAuth.authStrategy === "api_key_with_config") {

```

```

    returnedAuth = normalizeApiKeyWithConfigSecret(providerAuth.secret);
  }
  return {
    provider,
    auth: returnedAuth,
    model_ref: `${resolvedModel.provider}/${resolvedModel.id}`,
    runtime_api_key: runtimeApiKey || null,
    model_config: Object.keys(modelConfig).length > 0 ? modelConfig : null,
  };
}

export class YinshiSidecar {
  constructor() {
    this.activeSessions = new Map();
    this.activePromptSessionsBySocket = new Map();
    this.pendingPiSessionCreations = new Map();
    this.activeOAuthFlows = new Map();
    this.activeTerminals = new Map();
    this.socketPath = process.env.SIDECAR_SOCKET_PATH || "/tmp/yinshi-sidecar.sock";
    this.server = net.createServer((socket) => this.handleConnection(socket));
    this.healthCheckInterval = null;
  }

  initialize() {
    if (process.env.SIDECAR_LOAD_DOTENV === "1") {
      this._loadDotEnv();
    }
  }

  _loadDotEnv() {
    const envPath = path.join(__sidecarDir, "..", "..", ".env");
    if (!fs.existsSync(envPath)) {
      console.log("[sidecar] No .env file found, skipping");
      return;
    }
    const content = fs.readFileSync(envPath, "utf-8");
    for (const line of content.split("\n")) {
      const trimmed = line.trim();
      if (!trimmed || trimmed.startsWith("#")) {
        continue;
      }
      const eqIndex = trimmed.indexOf("=");
      if (eqIndex === -1) {
        continue;
      }
      const key = trimmed.slice(0, eqIndex).trim();

```

```

        const value = trimmed.slice(eqIndex + 1).trim();
        if (!process.env[key]) {
            process.env[key] = value;
            console.log("[sidecar] Loaded one environment entry");
        }
    }
}

createTerminalEntry(id, options) {
    const cwd = normalizeTerminalCwd(options.cwd);
    const cols = normalizePositiveInteger(options.cols, 100, 20, 300);
    const rows = normalizePositiveInteger(options.rows, 30, 5, 120);
    const scrollbarLines = normalizePositiveInteger(options.scrollbackLines, 1000, 100, 500);
    const shell = process.env.SHELL || "/bin/bash";
    const terminalEnvironment = buildTerminalEnvironment(cwd, shell);
    const terminal = pty.spawn(shell, ["-l"], {
        name: "xterm-256color",
        cols,
        rows,
        cwd,
        env: terminalEnvironment,
    });
    const entry = {
        id,
        cwd,
        terminal,
        sockets: new Set(),
        scrollbar: "",
        scrollbarLimitBytes: scrollbarLines * 200,
        suppressExitEvent: false,
    };
    terminal.onData((data) => {
        appendTerminalScrollbar(entry, data);
        for (const subscriber of entry.sockets) {
            sendToSocket(subscriber, { id, type: "terminal_data", data });
        }
    });
    terminal.onExit(({ exitCode, signal }) => {
        if (!entry.suppressExitEvent) {
            for (const subscriber of entry.sockets) {
                sendToSocket(subscriber, {
                    id,
                    type: "terminal_exit",
                    exit_code: exitCode,
                    signal,
                });
            }
        }
    });
}

```



```

    }
  }
  if (this.activeTerminals.get(id) === entry) {
    this.activeTerminals.delete(id);
  }
});
this.activeTerminals.set(id, entry);
return entry;
}

terminalEntry(id, options, restart = false) {
  if (restart) {
    this.killTerminal(id, { suppressExitEvent: true });
  }
  const existing = this.activeTerminals.get(id);
  if (existing) {
    return existing;
  }
  return this.createTerminalEntry(id, options);
}

attachTerminal(id, socket, options, restart = false) {
  const entry = this.terminalEntry(id, options, restart);
  entry.sockets.add(socket);
  sendToSocket(socket, {
    id,
    type: "terminal_ready",
    cwd: entry.cwd,
    pid: entry.terminal.pid,
    replay: entry.scrollback,
  });
}

detachTerminalSocket(socket) {
  for (const entry of this.activeTerminals.values()) {
    entry.sockets.delete(socket);
  }
}

writeTerminal(id, data) {
  const entry = this.activeTerminals.get(id);
  if (!entry) {
    throw new Error("Terminal is not running");
  }
  if (typeof data !== "string") {
    throw new Error("terminal input data must be a string");
  }
}

```

```

    }
    entry.terminal.write(data);
  }

resizeTerminal(id, cols, rows) {
  const entry = this.activeTerminals.get(id);
  if (!entry) {
    throw new Error("Terminal is not running");
  }
  entry.terminal.resize(
    normalizePositiveInteger(cols, 100, 20, 300),
    normalizePositiveInteger(rows, 30, 5, 120),
  );
}

killTerminal(id, { suppressExitEvent = false } = {}) {
  const entry = this.activeTerminals.get(id);
  if (!entry) {
    return;
  }
  entry.suppressExitEvent = Boolean(suppressExitEvent);
  entry.terminal.kill();
  this.activeTerminals.delete(id);
}

async start() {
  this.cleanup();

  return new Promise((resolve, reject) => {
    this.server.listen(this.socketPath, () => {
      try {
        fs.chmodSync(this.socketPath, 0o600);
      } catch (error) {
        this.server.close();
        reject(error);
        return;
      }
    }
    console.log(`SOCKET_PATH=${this.socketPath}`);
    this.healthCheckInterval = setInterval(() => {
      console.log(
        `[sidecar] Health: ${this.activeSessions.size} session(s), ${this.activeOAuthFlows.size} OAuth flows`
      );
    }, HEALTH_CHECK_INTERVAL);
    resolve();
  });
  this.server.on("error", (err) => {

```

```

        console.error("[sidecar] Server error");
        reject(err);
    });
});
}

handleConnection(socket) {
    console.log("[sidecar] New connection");
    sendToSocket(socket, { id: "init", type: "init_status", success: true });

    let buffer = "";
    socket.on("data", (chunk) => {
        buffer += chunk.toString();
        const lines = buffer.split("\n");
        buffer = lines.pop() || "";
        for (const line of lines) {
            const trimmed = line.trim();
            if (!trimmed) {
                continue;
            }
            this.handleData(trimmed, socket);
        }
    });
    socket.on("error", () => {
        console.error("[sidecar] Socket error");
        this._cancelPromptsForSocket(socket);
    });
    socket.on("close", () => {
        this._cancelPromptsForSocket(socket);
        this.detachTerminalSocket(socket);
        console.log("[sidecar] Connection closed");
    });
}

_trackPromptSession(socket, sessionId) {
    let promptsBySession = this.activePromptSessionsBySocket.get(socket);
    if (!promptsBySession) {
        promptsBySession = new Map();
        this.activePromptSessionsBySocket.set(socket, promptsBySession);
    }
    let promptStates = promptsBySession.get(sessionId);
    if (!promptStates) {
        promptStates = new Set();
        promptsBySession.set(sessionId, promptStates);
    }
    const promptState = {

```

```

        sessionId,
        cancelled: false,
        executionStarted: false,
        cancelRequested: false,
    };
    promptStates.add(promptState);
    return promptState;
}

_untrackPromptSession(socket, promptState) {
    const promptsBySession = this.activePromptSessionsBySocket.get(socket);
    const promptStates = promptsBySession?.get(promptState.sessionId);
    if (!promptStates) {
        return;
    }
    promptStates.delete(promptState);
    if (promptStates.size === 0) {
        promptsBySession.delete(promptState.sessionId);
    }
    if (promptsBySession.size === 0) {
        this.activePromptSessionsBySocket.delete(socket);
    }
}

_cancelPromptsForSocket(socket) {
    const promptsBySession = this.activePromptSessionsBySocket.get(socket);
    if (!promptsBySession) {
        return;
    }
    for (const [sessionId, promptStates] of promptsBySession) {
        let cancellationNeeded = false;
        for (const promptState of promptStates) {
            promptState.cancelled = true;
            if (promptState.executionStarted && !promptState.cancelRequested) {
                promptState.cancelRequested = true;
                cancellationNeeded = true;
            }
        }
        if (!cancellationNeeded || this.activeSessions.get(sessionId)?.cancelRequested) {
            continue;
        }
        void this.cancelSession(sessionId).catch(() => {
            console.error("[sidecar] Disconnected prompt cancellation failed");
        });
    }
}

```

```

_sessionHasActivePrompt(sessionId) {
  for (const promptsBySession of this.activePromptSessionsBySocket.values()) {
    if (promptsBySession.has(sessionId)) {
      return true;
    }
  }
  return false;
}

handleData(data, socket) {
  let parsed;
  try {
    parsed = JSON.parse(data);
  } catch (err) {
    sendToSocket(socket, { id: "unknown", type: "error", error: `Parse error: ${err.message}` });
    return;
  }
  this.handleRequest(parsed, socket);
}

_pruneIdlePiSessions(currentTimeMs) {
  if (!Number.isSafeInteger(currentTimeMs) || currentTimeMs < 0) {
    throw new TypeError("Pi session clock must be a non-negative safe integer");
  }
  for (const [sessionId, entry] of this.activeSessions) {
    if (this._sessionHasActivePrompt(sessionId)) {
      continue;
    }
    const lastActivityMs = entry?.lastActivityMs;
    if (!Number.isSafeInteger(lastActivityMs)) {
      // A session from an older code path has no stamp yet. Adopt it rather
      // than destroy work that is still in progress.
      entry.lastActivityMs = currentTimeMs;
      continue;
    }
    if (currentTimeMs - lastActivityMs < PI_SESSION_IDLE_TTL_MS) {
      continue;
    }
    this.activeSessions.delete(sessionId);
    this._disposePiSessionEntry(entry);
  }
}

_evictLeastRecentPiSessions() {
  if (this.activeSessions.size <= PI_SESSION_COUNT_MAX) {

```

```

        return;
    }
    const byOldestFirst = [...this.activeSessions.entries()]
        .filter(([sessionId]) => !this._sessionHasActivePrompt(sessionId))
        .sort((left, right) => left[1].lastActivityMs - right[1].lastActivityMs);
    const excess = this.activeSessions.size - PI_SESSION_COUNT_MAX;
    for (const [sessionId, entry] of byOldestFirst.slice(0, excess)) {
        this.activeSessions.delete(sessionId);
        this._disposePiSessionEntry(entry);
    }
}

_touchPiSession(sessionId, currentTimeMs) {
    const entry = this.activeSessions.get(sessionId);
    if (entry) {
        entry.lastActivityMs = currentTimeMs;
    }
}

handleRequest(request, socket, currentTimeMs = Date.now()) {
    this._pruneIdlePiSessions(currentTimeMs);
    this._evictLeastRecentPiSessions();
    if (request && typeof request === "object") {
        this._touchPiSession(request.id, currentTimeMs);
    }
    if (!request || typeof request !== "object") {
        sendToSocket(socket, { id: "unknown", type: "error", error: "Invalid request format" });
        return;
    }
    const { type, id } = request;
    switch (type) {
        case "auth_resolve":
            void this.handleAuthResolve(id, socket, request);
            break;
        case "cancel":
            void this.handleCancelRequest(id, socket);
            break;
        case "catalog":
            void this.handleCatalog(id, socket, request.options || {});
            break;
        case "version":
            this.handleVersion(id, socket);
            break;
        case "list_resources":
            void this.handleListResources(id, socket, request.options || {});
            break;
    }
}

```

```

case "oauth_clear":
    this.clearOAuthFlow(id, socket, request.flowId);
    break;
case "oauth_start":
    void this.startOAuthFlow(id, socket, request.provider);
    break;
case "oauth_status":
    this.handleOAuthStatus(id, socket, request.flowId);
    break;
case "oauth_submit":
    this.submitOAuthFlowInput(id, socket, request.flowId, request.authorizationInput);
    break;
case "ping":
    sendToSocket(socket, { type: "pong" });
    break;
case "terminal_attach":
    this.handleTerminalAttach(id, socket, request.options || {});
    break;
case "terminal_detach":
    this.detachTerminalSocket(socket);
    sendToSocket(socket, { id: id || "terminal", type: "terminal_detached" });
    break;
case "terminal_input":
    this.handleTerminalInput(id, socket, request.data);
    break;
case "terminal_kill":
    this.handleTerminalKill(id, socket);
    break;
case "terminal_resize":
    this.handleTerminalResize(id, socket, request.cols, request.rows);
    break;
case "terminal_restart":
    this.handleTerminalAttach(id, socket, request.options || {}, true);
    break;
case "query":
    void this.processQuery(id, socket, request.prompt, request.options || {});
    break;
case "session_release":
    sendToSocket(socket, {
        id: id || "unknown",
        type: "session_released",
        released: this.releasePiSession(id),
    });
    break;
case "resolve":
    void this.handleResolve(id, socket, request.model, request.options || {});

```

```

        break;
    case "warmup":
        void this.warmupSession(id, socket, request.options || {});
        break;
    default:
        sendToSocket(socket, { id: id || "unknown", type: "error", error: `Unknown request t
    }
}

handleTerminalAttach(id, socket, options, restart = false) {
    try {
        const terminalId = normalizeTerminalId(options.workspaceId || id);
        this.attachTerminal(terminalId, socket, options, restart);
    } catch (err) {
        sendTerminalError(socket, id, err, "Failed to attach terminal");
    }
}

handleTerminalInput(id, socket, data) {
    try {
        const terminalId = normalizeTerminalId(id);
        this.writeTerminal(terminalId, data);
    } catch (err) {
        sendTerminalError(socket, id, err, "Failed to write terminal input");
    }
}

handleTerminalResize(id, socket, cols, rows) {
    try {
        const terminalId = normalizeTerminalId(id);
        this.resizeTerminal(terminalId, cols, rows);
    } catch (err) {
        sendTerminalError(socket, id, err, "Failed to resize terminal");
    }
}

handleTerminalKill(id, socket) {
    try {
        const terminalId = normalizeTerminalId(id);
        this.killTerminal(terminalId);
        sendToSocket(socket, { id: terminalId, type: "terminal_killed" });
    } catch (err) {
        sendTerminalError(socket, id, err, "Failed to kill terminal");
    }
}

```



```

async handleCatalog(id, socket, options) {
  try {
    const catalog = await getCatalog(options.agentDir || null);
    sendToSocket(socket, {
      id: id || "catalog",
      type: "catalog",
      ...catalog,
    });
  } catch (err) {
    sendToSocket(socket, {
      id: id || "catalog",
      type: "error",
      error: err instanceof Error ? err.message : "Failed to build model catalog",
    });
  }
}

handleVersion(id, socket) {
  try {
    sendToSocket(socket, {
      id: id || "version",
      type: "version",
      ...getRuntimeVersion(),
    });
  } catch (err) {
    sendToSocket(socket, {
      id: id || "version",
      type: "error",
      error: err instanceof Error ? err.message : "Failed to read pi package version",
    });
  }
}

async handleListResources(id, socket, options) {
  try {
    const resources = await listResources(options.agentDir || null);
    sendToSocket(socket, {
      id: id || "list_resources",
      type: "resources",
      ...resources,
    });
  } catch (err) {
    sendToSocket(socket, {
      id: id || "list_resources",
      type: "error",
      error:

```

```

        err instanceof Error
        ? err.message
        : "Failed to list imported pi resources",
    });
}
}

async handleResolve(id, socket, modelKey, options) {
    try {
        const resolved = await resolveModel(
            modelKey,
            options.providerAuth || null,
            options.agentDir || null,
            options.providerConfig || null,
        );
        sendToSocket(socket, {
            id,
            type: "resolved",
            provider: resolved.provider,
            model: `${resolved.provider}/${resolved.id}`,
        });
    } catch (err) {
        sendToSocket(socket, {
            id: id || "unknown",
            type: "error",
            error: err instanceof Error ? err.message : `Unknown model: ${modelKey}`,
        });
    }
}

async handleAuthResolve(id, socket, request) {
    try {
        if (typeof request.provider !== "string" || !request.provider) {
            throw new Error("Provider is required");
        }
        if (typeof request.model !== "string" || !request.model) {
            throw new Error("Model is required");
        }
        const resolved = await resolveProviderRuntimeAuth(
            request.provider,
            request.model,
            request.providerAuth || null,
            request.agentDir || null,
            request.providerConfig || null,
        );
        sendToSocket(socket, {

```

```

        id,
        type: "auth_resolved",
        ...resolved,
    });
} catch (err) {
    sendToSocket(socket, {
        id: id || "auth-resolve",
        type: "error",
        error: err instanceof Error ? err.message : "Failed to resolve provider auth",
    });
}
}

_pruneExpiredOAuthFlows(currentTimeMs = Date.now()) {
    if (!Number.isSafeInteger(currentTimeMs) || currentTimeMs < 0) {
        throw new TypeError("OAuth flow clock must be a non-negative safe integer");
    }
    for (const [flowId, flow] of this.activeOAuthFlows) {
        if (!Number.isSafeInteger(flow.createdAtMs)) {
            this.activeOAuthFlows.delete(flowId);
            continue;
        }
        if (currentTimeMs - flow.createdAtMs < OAUTH_FLOW_TTL_MS) {
            continue;
        }
        if (typeof flow.manualInputReject === "function") {
            flow.manualInputReject(new Error("OAuth flow expired"));
        }
        flow.manualInputResolve = null;
        flow.manualInputReject = null;
        this.activeOAuthFlows.delete(flowId);
    }
}

async startOAuthFlow(id, socket, providerId) {
    try {
        if (typeof providerId !== "string" || !providerId) {
            throw new Error("Provider is required");
        }
        const { modelRuntime } = await createModelRegistry(null, null);
        const provider = modelRuntime.getProvider(providerId);
        if (!provider?.auth.oauth) {
            throw new Error(`OAuth provider is not available: ${providerId}`);
        }
        this._pruneExpiredOAuthFlows();
        if (this.activeOAuthFlows.size >= OAUTH_FLOW_COUNT_MAX) {

```

```

    throw new Error("Too many active OAuth flows");
  }

  const flowId = randomUUID();
  const flow = {
    id: flowId,
    provider: providerId,
    createdAtMs: Date.now(),
    status: "starting",
    authUrl: null,
    instructions: null,
    progress: [],
    credentials: null,
    error: null,
    manualInputRequired: Boolean(provider.usesCallbackServer),
    manualInputPrompt: provider.usesCallbackServer
      ? "Paste the final redirect URL or authorization code here."
      : null,
    manualInputSubmitted: false,
    manualInputValue: null,
    manualInputPromise: null,
    manualInputResolve: null,
    manualInputReject: null,
  };
  this.activeOAuthFlows.set(flowId, flow);

  const loginPromise = modelRuntime.login(providerId, "oauth", {
    prompt: async (prompt) => {
      if (prompt.type === "select" && prompt.options.length > 0) {
        return prompt.options[0].id;
      }
      return _waitForOAuthManualInput(flow, prompt.message);
    },
    notify: (event) => {
      if (event.type === "auth_url") {
        flow.authUrl = event.url;
        flow.instructions = event.instructions || null;
        flow.status = "pending";
        return;
      }
      if (event.type === "device_code") {
        flow.authUrl = event.verificationUri;
        flow.instructions = `Enter code ${event.userCode} in the browser.`;
        flow.status = "pending";
        return;
      }
    }
  });

```

```

        if (event.type === "progress" || event.type === "info") {
            flow.progress.push(event.message);
        }
    },
});

loginPromise
    .then((credentials) => {
        flow.credentials = credentials;
        if (flow.status === "starting") {
            flow.status = "pending";
        }
        flow.status = "complete";
    })
    .catch((err) => {
        flow.error = err instanceof Error ? err.message : String(err);
        flow.status = "error";
    });

const startDeadline = Date.now() + 5_000;
while (!flow.authUrl && flow.status !== "error" && Date.now() < startDeadline) {
    await new Promise((resolve) => setTimeout(resolve, 20));
}
if (!flow.authUrl && flow.status !== "error") {
    throw new Error("OAuth flow did not expose an authorization URL");
}

sendToSocket(socket, {
    id,
    type: "oauth_started",
    flow_id: flowId,
    provider: providerId,
    auth_url: flow.authUrl,
    instructions: flow.instructions,
    manual_input_required: flow.manualInputRequired,
    manual_input_prompt: flow.manualInputPrompt,
    manual_input_submitted: flow.manualInputSubmitted,
});
} catch (err) {
    sendToSocket(socket, {
        id: id || "oauth-start",
        type: "error",
        error: err instanceof Error ? err.message : "Failed to start OAuth flow",
    });
}
}

```

```

handleOAuthStatus(id, socket, flowId) {
  this._pruneExpiredOAuthFlows();
  if (typeof flowId !== "string" || !flowId) {
    sendToSocket(socket, { id: id || "oauth-status", type: "error", error: "flowId is required" });
    return;
  }
  const flow = this.activeOAuthFlows.get(flowId);
  if (!flow) {
    sendToSocket(socket, { id: id || "oauth-status", type: "error", error: "OAuth flow not found" });
    return;
  }
  sendToSocket(socket, {
    id,
    type: "oauth_status",
    flow_id: flow.id,
    provider: flow.provider,
    status: flow.status,
    auth_url: flow.authUrl,
    instructions: flow.instructions,
    progress: flow.progress,
    credentials: flow.status === "complete" ? flow.credentials : null,
    error: flow.error,
    manual_input_required: flow.manualInputRequired,
    manual_input_prompt: flow.manualInputPrompt,
    manual_input_submitted: flow.manualInputSubmitted,
  });
}

submitOAuthFlowInput(id, socket, flowId, authorizationInput) {
  this._pruneExpiredOAuthFlows();
  if (typeof flowId !== "string" || !flowId) {
    sendToSocket(socket, { id: id || "oauth-submit", type: "error", error: "flowId is required" });
    return;
  }
  const flow = this.activeOAuthFlows.get(flowId);
  if (!flow) {
    sendToSocket(socket, { id: id || "oauth-submit", type: "error", error: "OAuth flow not found" });
    return;
  }
  try {
    _submitOAuthManualInput(flow, authorizationInput);
    sendToSocket(socket, {
      id,
      type: "oauth_submitted",
      flow_id: flow.id,
    });
  } catch (error) {
    console.error("Error submitting OAuth flow input:", error);
  }
}

```

```

        provider: flow.provider,
        manual_input_required: flow.manualInputRequired,
        manual_input_prompt: flow.manualInputPrompt,
        manual_input_submitted: flow.manualInputSubmitted,
    });
} catch (err) {
    sendToSocket(socket, {
        id: id || "oauth-submit",
        type: "error",
        error: err instanceof Error ? err.message : "Failed to submit OAuth input",
    });
}
}

clearOAuthFlow(id, socket, flowId) {
    this._pruneExpiredOAuthFlows();
    if (typeof flowId !== "string" || !flowId) {
        sendToSocket(socket, { id: id || "oauth-clear", type: "error", error: "flowId is required" });
        return;
    }
    const flow = this.activeOAuthFlows.get(flowId);
    if (flow?.manualInputReject) {
        flow.manualInputReject(new Error("OAuth flow was cleared before manual input was consumed"));
    }
    this.activeOAuthFlows.delete(flowId);
    sendToSocket(socket, {
        id,
        type: "oauth_cleared",
        flow_id: flowId,
    });
}

async _createPiSession(
    sessionId,
    socket,
    modelRef,
    cwd,
    providerAuth,
    providerConfig,
    gitAuth,
    agentDir,
    importedSettings,
    normalizedPiSessionFile,
) {
    const { modelRuntime, registry } = await createModelRegistry(providerAuth, agentDir);
    const model = resolveModelFromRegistry(registry, modelRef, providerConfig);

```

```

const {
  sessionManager,
  resetWarning,
  piSessionFile: openedPiSessionFile,
} = openSessionManager(cwd, normalizedPiSessionFile);

const settingsManager = SettingsManager.inMemory({
  compaction: { enabled: true },
  retry: { enabled: true, maxRetries: 3 },
});
const normalizedImportedSettings = normalizeImportedSettings(importedSettings);
if (normalizedImportedSettings) {
  settingsManager.applyOverrides(normalizedImportedSettings);
}

const sessionOptions = {
  cwd,
  model,
  // The SDK's `tools` option is an allow-list of tool names. Pass Yinshi's
  // tool implementations as `customTools` so they replace the built-ins
  // while keeping read/bash/edit/write active for the model.
  customTools: createYinshiCodingTools(cwd, gitAuth),
  sessionManager,
  settingsManager,
  modelRuntime,
};
if (agentDir) {
  sessionOptions.agentDir = agentDir;
}

const { session } = await createAgentSession(sessionOptions);
// Bind a web-friendly UI context so extensions (e.g. rtk-metrics) whose
// handlers call ctx.ui.notify() can surface output in the chat. Without
// this binding, notify() calls silently vanish in RPC mode.
const runner = session.extensionRunner;
console.log(`[sidecar] extension runner ${runner ? "available" : "unavailable"}`);
if (runner) {
  runner.setUIContext(createWebUIContext(sessionId, socket, model));
  console.log("[sidecar] UI context bound");
}
if (resetWarning) {
  sendStatusToSocket(
    socket,
    sessionId,
    "context_reset",
    resetWarning,
  )
}

```



```

        "warning",
    );
}
console.log("[sidecar] Pi session created");
return { session, model, piSessionFile: openedPiSessionFile };
}

_disposePiSessionEntry(entry) {
    try {
        if (typeof entry?.unsubscribe === "function") {
            entry.unsubscribe();
        }
    } catch {
        // A failed unsubscribe must not block disposal of the pi session.
    }
    try {
        entry?.piSession?.dispose();
    } catch {
        // Disposal races are expected when a prompt is still unwinding.
    }
}

releasePiSession(sessionId) {
    const entry = this.activeSessions.get(sessionId);
    if (!entry) {
        return false;
    }
    this.activeSessions.delete(sessionId);
    this._disposePiSessionEntry(entry);
    return true;
}

_admitPiSessionCreation(sessionId) {
    if (
        this.activeSessions.has(sessionId)
        || this.pendingPiSessionCreations.has(sessionId)
    ) {
        return;
    }

    const reservedSessionIds = new Set([
        ...this.activeSessions.keys(),
        ...this.pendingPiSessionCreations.keys(),
    ]);
    while (reservedSessionIds.size >= PI_SESSION_COUNT_MAX) {
        const candidate = [...this.activeSessions.entries()]

```

```

        .filter(([activeSessionId]) => (
            !this._sessionHasActivePrompt(activeSessionId)
            && !this.pendingPiSessionCreations.has(activeSessionId)
        ))
        .sort((left, right) => (
            (left[1].lastActivityMs || 0) - (right[1].lastActivityMs || 0)
        ))[0];
    if (!candidate) {
        throw new Error("Pi session capacity reached");
    }
    const [candidateSessionId, candidateEntry] = candidate;
    this.activeSessions.delete(candidateSessionId);
    reservedSessionIds.delete(candidateSessionId);
    this._disposePiSessionEntry(candidateEntry);
}
}

async _withPiSessionCreationLock(sessionId, operation) {
    this._admitPiSessionCreation(sessionId);
    const previous = this.pendingPiSessionCreations.get(sessionId) || Promise.resolve();
    let release;
    const current = new Promise((resolve) => {
        release = resolve;
    });
    this.pendingPiSessionCreations.set(sessionId, current);

    await previous;
    try {
        return await operation();
    } finally {
        release();
        if (this.pendingPiSessionCreations.get(sessionId) === current) {
            this.pendingPiSessionCreations.delete(sessionId);
        }
    }
}

async warmupSession(sessionId, socket, options) {
    const modelRef = options.model || DEFAULT_MODEL_REF;
    const cwd = options.cwd || process.cwd();
    const providerAuth = options.providerAuth || null;
    const providerConfig = options.providerConfig || null;
    const gitAuth = options.gitAuth || null;
    const agentDir = options.agentDir || null;
    const importedSettings = options.settings || null;

```

```

try {
  await this._withPiSessionCreationLock(sessionId, async () => {
    if (this.activeSessions.has(sessionId)) {
      console.log("[sidecar] Pi session already active");
      return;
    }
    const requestedPiSessionFile = normalizePiSessionFile(options.piSessionFile || null);
    const {
      session: piSession,
      model,
      piSessionFile: normalizedPiSessionFile,
    } = await this._createPiSession(
      sessionId,
      socket,
      modelRef,
      cwd,
      providerAuth,
      providerConfig,
      gitAuth,
      agentDir,
      importedSettings,
      requestedPiSessionFile,
    );
    this.activeSessions.set(sessionId, {
      piSession,
      model,
      modelRef,
      cwd,
      providerAuth,
      providerConfig,
      gitAuth,
      importedSettings,
      piSessionFile: normalizedPiSessionFile,
      unsubscribe: null,
      cancelRequested: false,
      lastActivityMs: Date.now(),
    });
    console.log("[sidecar] Pi session warmed");
  });
  sendToSocket(socket, {
    id: sessionId,
    type: "warmup_status",
    success: true,
  });
} catch {
  console.error("[sidecar] Warmup failed");
}

```

```

        sendToSocket(socket, {
            id: sessionId,
            type: "warmup_status",
            success: false,
            error: "Failed to warm up session",
        });
    }
}

async processQuery(sessionId, socket, prompt, options) {
    const promptState = this._trackPromptSession(socket, sessionId);
    const modelRef = options.model || DEFAULT_MODEL_REF;
    const cwd = options.cwd || process.cwd();
    const providerAuth = options.providerAuth || null;
    const providerConfig = options.providerConfig || null;
    const gitAuth = options.gitAuth || null;
    const agentDir = options.agentDir || null;
    const importedSettings = options.settings || null;
    let entry = this.activeSessions.get(sessionId);
    let clearFinalizeTimer = () => {};
    console.log(`[sidecar] Prompt accepted warm=${Boolean(entry)}`);

    try {
        const requestedPiSessionFile = normalizePiSessionFile(options.piSessionFile || null);
        entry = await this._withPiSessionCreationLock(sessionId, async () => {
            const currentEntry = this.activeSessions.get(sessionId);
            const authChanged = JSON.stringify(currentEntry?.providerAuth || null)
                !== JSON.stringify(providerAuth);
            const configChanged = JSON.stringify(currentEntry?.providerConfig || null)
                !== JSON.stringify(providerConfig);
            const gitAuthChanged = JSON.stringify(currentEntry?.gitAuth || null)
                !== JSON.stringify(gitAuth);
            const settingsChanged = JSON.stringify(currentEntry?.importedSettings || null)
                !== JSON.stringify(importedSettings);
            const piSessionFileChanged = (currentEntry?.piSessionFile || null)
                !== requestedPiSessionFile;
            if (
                currentEntry
                && currentEntry.modelRef === modelRef
                && !authChanged
                && !configChanged
                && !gitAuthChanged
                && !settingsChanged
                && !piSessionFileChanged
            ) {
                return currentEntry;
            }
        });
    }
}

```

```

    }

    if (currentEntry) {
      this.activeSessions.delete(sessionId);
      this._disposePiSessionEntry(currentEntry);
    }
    const {
      session: piSession,
      model,
      piSessionFile: normalizedPiSessionFile,
    } = await this._createPiSession(
      sessionId,
      socket,
      modelRef,
      cwd,
      providerAuth,
      providerConfig,
      gitAuth,
      agentDir,
      importedSettings,
      requestedPiSessionFile,
    );
    const createdEntry = {
      piSession,
      model,
      modelRef,
      cwd,
      providerAuth,
      providerConfig,
      gitAuth,
      importedSettings,
      piSessionFile: normalizedPiSessionFile,
      unsubscribe: null,
      cancelRequested: false,
      lastActivityMs: Date.now(),
    };
    this.activeSessions.set(sessionId, createdEntry);
    return createdEntry;
  });

  if (promptState.cancelled) {
    return;
  }

  const { piSession, model } = entry;

```

```

if (entry.unsubscribe) {
  entry.unsubscribe();
}

let usage = null;
// When pi handles a prompt as an extension command (text starting with
// "/" that matches a registered command), it returns from prompt()
// without firing "agent_end". The stream would hang forever waiting
// for a "result" event. Track whether agent_end fired so we can emit
// a synthetic one after prompt() resolves.
let agentEndEmitted = false;
let resultSent = false;
let pendingResult = null;
let compactionActive = false;
let finalizeTimer = null;

const buildResultMessage = (resultUsage) => ({
  id: sessionId,
  type: "message",
  data: {
    type: "result",
    usage: resultUsage || {},
    provider: model.provider,
    model: `${model.provider}/${model.id}`,
  },
});

clearFinalizeTimer = () => {
  if (finalizeTimer) {
    clearTimeout(finalizeTimer);
    finalizeTimer = null;
  }
};

const schedulePendingResult = () => {
  clearFinalizeTimer();
  finalizeTimer = setTimeout(() => {
    finalizeTimer = null;
    if (resultSent || compactionActive || !pendingResult) {
      return;
    }
    sendToSocket(socket, pendingResult);
    pendingResult = null;
    resultSent = true;
  }, 0);
};

```

```

entry.unsubscribe = piSession.subscribe((event) => {
  console.log(`[sidecar][event] type=${event.type}`);
  switch (event.type) {
    case "message_update": {
      const assistantEvent = event.assistantMessageEvent;
      if (assistantEvent.type === "text_delta") {
        sendToSocket(socket, {
          id: sessionId,
          type: "message",
          data: {
            type: "assistant",
            message: {
              content: [{ type: "text", text: assistantEvent.delta }],
            },
          },
        });
      } else if (assistantEvent.type === "thinking_delta") {
        sendToSocket(socket, {
          id: sessionId,
          type: "message",
          data: {
            type: "assistant",
            message: {
              content: [
                { type: "thinking", thinking: assistantEvent.delta },
              ],
            },
          },
        });
      }
      break;
    }
    case "tool_execution_start":
      sendToSocket(socket, {
        id: sessionId,
        type: "message",
        data: {
          type: "tool_use",
          id: event.toolCallId,
          toolName: event.toolName,
          toolInput: event.args,
        },
      });
      break;
    case "tool_execution_update":

```

```

sendToSocket(socket, {
  id: sessionId,
  type: "tool_result",
  tool_use_id: event.toolCallId,
  content: stringifyToolResult(event.partialResult),
  partial: true,
});
break;
case "tool_execution_end":
sendToSocket(socket, {
  id: sessionId,
  type: "tool_result",
  tool_use_id: event.toolCallId,
  content: stringifyToolResult(event.result),
  is_error: event.isError === true,
});
break;
case "turn_end":
if (event.message && event.message.usage) {
  const eventUsage = event.message.usage;
  usage = {
    input_tokens: eventUsage.input || 0,
    output_tokens: eventUsage.output || 0,
    cache_read_input_tokens: eventUsage.cacheRead || 0,
    cache_creation_input_tokens: eventUsage.cacheWrite || 0,
  };
}
break;
case "agent_end":
pendingResult = buildResultMessage(usage);
usage = null;
agentEndEmitted = true;
schedulePendingResult();
break;
case "auto_retry_start":
pendingResult = null;
clearFinalizeTimer();
console.log(
  `[sidecar] Retrying provider request (attempt ${event.attempt}/${event.maxAttempts})`
);
setStatusToSocket(
  socket,
  sessionId,
  "retrying",
  `Retrying after provider error (attempt ${event.attempt}/${event.maxAttempts})`
);

```



```

        break;
    case "auto_retry_end":
        sendStatusToSocket(
            socket,
            sessionId,
            event.success ? "retry_complete" : "retry_failed",
            event.success ? "Retry complete." : "Retry failed.",
            event.success ? "info" : "warning",
        );
        break;
    case "compaction_start":
        compactionActive = true;
        clearFinalizeTimer();
        console.log("[sidecar] Compacting context...");
        sendStatusToSocket(
            socket,
            sessionId,
            "compacting",
            "Compacting context...",
            "info",
            { reason: event.reason },
        );
        break;
    case "compaction_end":
        compactionActive = false;
        if (event.willRetry) {
            pendingResult = null;
        }
        sendStatusToSocket(
            socket,
            sessionId,
            event.errorMessage ? "compaction_failed" : "compacted",
            event.errorMessage || "Context compacted.",
            event.errorMessage ? "warning" : "info",
            {
                reason: event.reason,
                willRetry: event.willRetry === true,
                aborted: event.aborted === true,
            },
        );
        if (!event.willRetry) {
            schedulePendingResult();
        }
        break;
    }
});

```

```

        console.log("[sidecar] Prompt started");
        promptState.executionStarted = true;
        await piSession.prompt(prompt);
        console.log("[sidecar] Prompt ended");
        // Clear cancelRequested after normal completion
        entry.cancelRequested = false;

        // Pi returns from prompt() without firing agent_end when it handles an
        // extension command inline (e.g. `/rtk-stats`). Synthesise the result
        // event so the client stream loop terminates cleanly instead of hanging.
        if (!agentEndEmitted && !resultSent) {
            console.log("[sidecar] Synthesising inline-command result");
            sendToSocket(socket, buildResultMessage(usage));
            resultSent = true;
        }
    } catch (err) {
        clearFinalizeTimer();
        const errorMessage = err instanceof Error ? err.message : String(err);
        if (entry?.cancelRequested) {
            console.log("[sidecar] Prompt cancelled by user");
            sendToSocket(socket, {
                id: sessionId,
                type: "cancelled",
            });
            // Clear cancelRequested after handling cancellation
            entry.cancelRequested = false;
        } else {
            console.error("[sidecar] Prompt failed");
            sendToSocket(socket, {
                id: sessionId,
                type: "error",
                error: errorMessage,
            });
        }
    } finally {
        this._untrackPromptSession(socket, promptState);
    }
}

async handleCancelRequest(sessionId, socket) {
    try {
        await this.cancelSession(sessionId);
        sendToSocket(socket, {
            id: sessionId,
            type: "cancel_status",
        });
    }
}

```

```

        success: true,
    });
} catch {
    console.error("[sidecar] Cancellation failed");
    sendToSocket(socket, {
        id: sessionId,
        type: "cancel_status",
        success: false,
        error: "Failed to cancel session",
    });
}
}

async cancelSession(sessionId) {
    const entry = this.activeSessions.get(sessionId);
    if (!entry) {
        console.log("[sidecar] Cancellation target not found");
        return;
    }
    console.log("[sidecar] Cancelling prompt");
    entry.cancelRequested = true;
    try {
        entry.piSession.abortCompaction();
        entry.piSession.abortRetry();
        await entry.piSession.abort();
    } catch (error) {
        entry.cancelRequested = false;
        throw error;
    }
}

cleanup() {
    try {
        if (fs.existsSync(this.socketPath)) {
            fs.unlinkSync(this.socketPath);
        }
    } catch {
        // ignore cleanup races
    }

    if (this.server) {
        try {
            this.server.close();
        } catch {
            // ignore cleanup races
        }
    }
}

```

```

    }

    for (const [, entry] of this.activeSessions) {
      try {
        if (entry.unsubscribe) {
          entry.unsubscribe();
        }
        entry.piSession.dispose();
      } catch {
        // ignore cleanup races
      }
    }
    this.activeSessions.clear();
    this.activePromptSessionsBySocket.clear();
    this.pendingPiSessionCreations.clear();
    this.activeOAuthFlows.clear();
    for (const [, entry] of this.activeTerminals) {
      try {
        entry.terminal.kill();
      } catch {
        // ignore cleanup races
      }
    }
    this.activeTerminals.clear();

    if (this.healthCheckInterval) {
      clearInterval(this.healthCheckInterval);
      this.healthCheckInterval = null;
    }
  }
}

```

25.4 git_auth.js

Git-auth helpers wrap the sidecar's bash tool so git commands can use runtime credentials without exposing secrets in prompts. The root chunk below tangles back to sidecar/src/git_auth.js.

```

import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import net from "node:net";
import { spawn } from "node:child_process";
import { randomBytes, timingSafeEqual } from "node:crypto";
import { fileURLToPath } from "node:url";

```

```

import { createBashTool, createLocalBashOperations } from "@earendil-works/pi-coding-agent"

const GIT_EXECUTABLE_PATH = "/usr/bin/git";
const GIT_REMOTE_SUBCOMMANDS = new Set(["clone", "fetch", "ls-remote", "pull", "push"]);
const SHELL_AND_OPERATOR = "&&";
const GIT_CREDENTIAL_CAPABILITY_FD = 3;

function assertNonEmptyString(value, name) {
  if (typeof value !== "string") {
    throw new TypeError(`${name} must be a string`);
  }
  const normalizedValue = value.trim();
  if (!normalizedValue) {
    throw new Error(`${name} must be a non-empty string`);
  }
  return normalizedValue;
}

function quoteShellLiteral(value) {
  const normalizedValue = assertNonEmptyString(value, "value");
  return `${normalizedValue.replace(/'/g, `'\\"'`)} `;
}

function buildGitAskpassScript() {
  const clientPath = fileURLToPath(new URL("./git_askpass_client.js", import.meta.url));
  return `#!/bin/sh\n`
    + `exec ${quoteShellLiteral(process.execPath)} ${quoteShellLiteral(clientPath)} \"$1\"`
}

export function normalizeGitAuth(gitAuth) {
  if (gitAuth === null || gitAuth === undefined) {
    return null;
  }
  if (typeof gitAuth !== "object" || Array.isArray(gitAuth)) {
    throw new TypeError("gitAuth must be an object");
  }

  if (gitAuth.strategy !== "github_app_https") {
    throw new Error(`Unsupported git auth strategy: ${gitAuth.strategy}`);
  }
  const normalizedHost = assertNonEmptyString(gitAuth.host, "gitAuth.host");
  if (normalizedHost !== "github.com") {
    throw new Error("gitAuth.host must be github.com");
  }
  const normalizedAccessToken = assertNonEmptyString(
    gitAuth.accessToken,

```

```

        "gitAuth.accessToken",
    );
    return {
        strategy: gitAuth.strategy,
        host: normalizedHost,
        accessToken: normalizedAccessToken,
    };
}

export function tokenizeShellCommand(command) {
    const normalizedCommand = assertNonEmptyString(command, "command");
    const tokens = [];
    let currentToken = "";
    let quoteMode = null;

    const pushCurrentToken = () => {
        if (!currentToken) {
            return;
        }
        tokens.push(currentToken);
        currentToken = "";
    };

    for (let index = 0; index < normalizedCommand.length; index += 1) {
        const character = normalizedCommand[index];

        if (quoteMode === "single") {
            if (character === "'") {
                quoteMode = null;
            } else {
                currentToken += character;
            }
            continue;
        }
        if (quoteMode === "double") {
            if (character === "\"") {
                quoteMode = null;
            }
            continue;
        }
        if (character === "\\") {
            index += 1;
            if (index >= normalizedCommand.length) {
                throw new Error("command ends with an incomplete escape sequence");
            }
            currentToken += normalizedCommand[index];
            continue;
        }
    }
}

```

```

    }
    currentToken += character;
    continue;
}

if (character === "'") {
    quoteMode = "single";
    continue;
}
if (character === "\"") {
    quoteMode = "double";
    continue;
}
if (character === "\\") {
    index += 1;
    if (index >= normalizedCommand.length) {
        throw new Error("command ends with an incomplete escape sequence");
    }
    currentToken += normalizedCommand[index];
    continue;
}
if (/\\s/.test(character)) {
    pushCurrentToken();
    continue;
}
if (character === "&") {
    if (normalizedCommand[index + 1] !== "&") {
        throw new Error("unsupported shell operator: &");
    }
    pushCurrentToken();
    tokens.push(SHELL_AND_OPERATOR);
    index += 1;
    continue;
}
if (character === ";" || character === "|" || character === ">" || character === "<" ||
    throw new Error(`unsupported shell operator: ${character}`);
}
if (character === "$") {
    throw new Error("unsupported shell expansion");
}

currentToken += character;
}

if (quoteMode !== null) {
    throw new Error("command contains an unterminated quote");
}

```

```

    }

    pushCurrentToken();
    return tokens;
}

function resolveGitWorkingDirectory(tokens, defaultCwd) {
    const normalizedDefaultCwd = assertNonEmptyString(defaultCwd, "defaultCwd");
    if (tokens.length === 0) {
        return { gitCwd: normalizedDefaultCwd, gitIndex: 0 };
    }
    if (tokens[0] !== "cd") {
        return { gitCwd: normalizedDefaultCwd, gitIndex: 0 };
    }
    if (tokens.length < 4) {
        return null;
    }
    if (tokens[2] !== SHELL_AND_OPERATOR) {
        return null;
    }
    if (!tokens[1]) {
        return null;
    }

    return {
        gitCwd: path.resolve(normalizedDefaultCwd, tokens[1]),
        gitIndex: 3,
    };
}

export function parseGitCommandForRuntimeAuth(command, defaultCwd) {
    let tokens = null;
    try {
        tokens = tokenizeShellCommand(command);
    } catch {
        return null;
    }
    const resolvedPrefix = resolveGitWorkingDirectory(tokens, defaultCwd);
    if (resolvedPrefix === null) {
        return null;
    }

    const { gitCwd, gitIndex } = resolvedPrefix;
    if (tokens.length <= gitIndex) {
        return null;
    }
}

```



```

    if (tokens[gitIndex] !== "git") {
      return null;
    }

    const gitArguments = tokens.slice(gitIndex + 1);
    if (gitArguments.length === 0) {
      return null;
    }
    if (gitArguments.includes(SHELL_AND_OPERATOR)) {
      return null;
    }
    if (gitArguments[0].startsWith("-")) {
      return null;
    }

    const normalizedSubcommand = gitArguments[0].toLowerCase();
    if (!GIT_REMOTE_SUBCOMMANDS.has(normalizedSubcommand)) {
      return null;
    }

    return {
      command: GIT_EXECUTABLE_PATH,
      cwd: gitCwd,
      gitArguments,
      subcommand: normalizedSubcommand,
    };
  }
}

export function createGitAskpassBundle() {
  const bundleDirPath = fs.mkdtempSync(path.join(os.tmpdir(), "yinshi-git-"));
  fs.chmodSync(bundleDirPath, 0o700);

  const askpassPath = path.join(bundleDirPath, "askpass.sh");
  fs.writeFileSync(
    askpassPath,
    buildGitAskpassScript(),
    { encoding: "utf-8", mode: 0o700 },
  );
  fs.chmodSync(askpassPath, 0o700);

  return {
    askpassPath,
    bundleDirPath,
    cleanup() {
      fs.rmSync(bundleDirPath, { recursive: true, force: true });
    },
  };
}

```

```

    };
}

function createGitExecutionEnvironment(baseEnv, askpassPath, credentialSocketPath) {
    const normalizedAskpassPath = assertNonEmptyString(askpassPath, "askpassPath");
    const normalizedSocketPath = assertNonEmptyString(
        credentialSocketPath,
        "credentialSocketPath",
    );
    const environmentSource = { ...process.env, ...(baseEnv || {}) };
    const executionEnvironment = {};
    for (const name of ["HOME", "LANG", "LC_ALL", "PATH", "TEMP", "TMP", "TMPDIR"]) {
        if (typeof environmentSource[name] === "string") {
            executionEnvironment[name] = environmentSource[name];
        }
    }
    executionEnvironment.GCM_INTERACTIVE = "Never";
    executionEnvironment.GIT_ASKPASS = normalizedAskpassPath;
    executionEnvironment.GIT_CONFIG_NOSYSTEM = "1";
    executionEnvironment.GIT_CONFIG_GLOBAL = os.devNull;
    executionEnvironment.GIT_PAGER = "cat";
    executionEnvironment.GIT_TERMINAL_PROMPT = "0";
    executionEnvironment.YINSHI_GIT_CREDENTIAL_SOCKET = normalizedSocketPath;
    return executionEnvironment;
}

function createUnlinkedCapabilityDescriptor(bundleDirPath, capability) {
    const capabilityPath = path.join(bundleDirPath, "capability");
    fs.writeFileSync(capabilityPath, capability, {
        encoding: "utf-8",
        flag: "wx",
        mode: 0o600,
    });
    const capabilityFd = fs.openSync(capabilityPath, "r");
    fs.unlinkSync(capabilityPath);
    return capabilityFd;
}

function requestHasCapability(request, capability) {
    if (!request || typeof request !== "object") {
        return false;
    }
    if (typeof request.capability !== "string") {
        return false;
    }
    const receivedCapability = Buffer.from(request.capability, "utf-8");

```

```

    const expectedCapability = Buffer.from(capability, "utf-8");
    if (receivedCapability.length !== expectedCapability.length) {
        return false;
    }
    return timingSafeEqual(receivedCapability, expectedCapability);
}

export async function createGitCredentialBroker(accessToken) {
    let credential = assertNonEmptyString(accessToken, "accessToken");
    let capability = randomBytes(32).toString("hex");
    const askpassBundle = createGitAskpassBundle();
    let capabilityFd;
    try {
        capabilityFd = createUnlinkedCapabilityDescriptor(
            askpassBundle.bundleDirPath,
            capability,
        );
    } catch (error) {
        askpassBundle.cleanup();
        throw error;
    }
    const socketPath = path.join(askpassBundle.bundleDirPath, "credential.sock");
    let credentialIssued = false;
    let cleaned = false;
    const server = net.createServer((socket) => {
        socket.setTimeout(5000, () => socket.destroy());
        socket.once("data", (rawRequest) => {
            if (rawRequest.length > 2048) {
                socket.destroy();
                return;
            }
            let request;
            try {
                request = JSON.parse(rawRequest.toString("utf-8"));
            } catch {
                socket.destroy();
                return;
            }
            if (!requestHasCapability(request, capability)) {
                socket.destroy();
                return;
            }
        }
        const prompt = request.prompt;
        if (typeof prompt !== "string" || prompt.length === 0 || prompt.length > 1024) {
            socket.destroy();
            return;
        }
    });
}

```

```

    }
    if (prompt.includes("Username")) {
        socket.end("x-access-token\n");
        return;
    }
    if (credentialIssued) {
        socket.destroy();
        return;
    }
    credentialIssued = true;
    const oneTimeCredential = credential;
    credential = "";
    socket.end(`${oneTimeCredential}\n`);
});
});

try {
    await new Promise((resolve, reject) => {
        server.once("error", reject);
        server.listen(socketPath, () => {
            server.removeListener("error", reject);
            resolve();
        });
    });
    fs.chmodSync(socketPath, 0o600);
} catch (error) {
    fs.closeSync(capabilityFd);
    server.close();
    askpassBundle.cleanup();
    throw error;
}

return {
    askpassPath: askpassBundle.askpassPath,
    capabilityFd,
    socketPath,
    cleanup() {
        if (cleaned) {
            return;
        }
        cleaned = true;
        credential = "";
        capability = "";
        fs.closeSync(capabilityFd);
        server.close();
        askpassBundle.cleanup();
    }
};

```

```

    },
  };
}

function createGitCommandArguments(gitArguments) {
  if (!Array.isArray(gitArguments)) {
    throw new TypeError("gitArguments must be an array");
  }
  if (gitArguments.length === 0) {
    throw new Error("gitArguments must not be empty");
  }
  return [
    "-c",
    "core.hooksPath=/dev/null",
    "-c",
    "credential.helper=",
    ...gitArguments,
  ];
}

async function executeGitCommand(parsedGitCommand, execOptions, gitAuth) {
  if (!parsedGitCommand || typeof parsedGitCommand !== "object") {
    throw new TypeError("parsedGitCommand is required");
  }
  const normalizedGitAuth = normalizeGitAuth(gitAuth);
  if (normalizedGitAuth === null) {
    throw new Error("gitAuth is required for authenticated git execution");
  }
  const credentialBroker = await createGitCredentialBroker(normalizedGitAuth.accessToken);
  const gitCommandArguments = createGitCommandArguments(parsedGitCommand.gitArguments);
  const gitEnvironment = createGitExecutionEnvironment(
    execOptions?.env,
    credentialBroker.askpassPath,
    credentialBroker.socketPath,
  );

  return new Promise((resolve, reject) => {
    const gitChild = spawn(
      parsedGitCommand.command,
      gitCommandArguments,
      {
        cwd: parsedGitCommand.cwd,
        detached: true,
        env: gitEnvironment,
        stdio: ["ignore", "pipe", "pipe", credentialBroker.capabilityFd],
      },
    );
  });
}

```

```

);

let timedOut = false;
const timeoutSeconds = execOptions?.timeout;
const timeoutHandle = timeoutSeconds && timeoutSeconds > 0
  ? setTimeout(() => {
    timedOut = true;
    gitChild.kill();
  }, timeoutSeconds * 1000)
  : null;

const onAbort = () => {
  gitChild.kill();
};

gitChild.stdout?.on("data", execOptions.onData);
gitChild.stderr?.on("data", execOptions.onData);

if (execOptions?.signal) {
  if (execOptions.signal.aborted) {
    onAbort();
  } else {
    execOptions.signal.addEventListener("abort", onAbort, { once: true });
  }
}

const cleanup = () => {
  if (timeoutHandle) {
    clearTimeout(timeoutHandle);
  }
  if (execOptions?.signal) {
    execOptions.signal.removeEventListener("abort", onAbort);
  }
  credentialBroker.cleanup();
};

gitChild.on("error", (error) => {
  cleanup();
  reject(error);
});
gitChild.on("close", (exitCode) => {
  cleanup();
  if (execOptions?.signal?.aborted) {
    reject(new Error("aborted"));
    return;
  }
}

```

```

        if (timedOut) {
            reject(new Error(`timeout:${timeoutSeconds}`));
            return;
        }
        resolve({ exitCode });
    });
});
}

export function createGitRuntimeBashOperations(defaultCwd, gitAuth) {
    const normalizedDefaultCwd = assertNonEmptyString(defaultCwd, "defaultCwd");
    const normalizedGitAuth = normalizeGitAuth(gitAuth);
    const localBashOperations = createLocalBashOperations();

    return {
        exec(command, cwd, execOptions) {
            const effectiveCwd = assertNonEmptyString(cwd || normalizedDefaultCwd, "cwd");
            if (normalizedGitAuth === null) {
                return localBashOperations.exec(command, effectiveCwd, execOptions);
            }

            const parsedGitCommand = parseGitCommandForRuntimeAuth(command, effectiveCwd);
            if (parsedGitCommand === null) {
                return localBashOperations.exec(command, effectiveCwd, execOptions);
            }

            return executeGitCommand(parsedGitCommand, execOptions, normalizedGitAuth);
        },
    };
}

export function createGitAwareBashTool(cwd, gitAuth) {
    const normalizedCwd = assertNonEmptyString(cwd, "cwd");
    return createBashTool(normalizedCwd, {
        operations: createGitRuntimeBashOperations(normalizedCwd, gitAuth),
    });
}

```

25.5 git_askpass_client.js

The askpass client receives no credential in its arguments, environment, or filesystem. It sends Git's prompt to the short-lived Unix-socket broker and writes the one-time response to Git.

```
#!/usr/bin/env node
```

```

import fs from "node:fs";
import net from "node:net";

const socketPath = process.env.YINSHI_GIT_CREDENTIAL_SOCKET;
const prompt = process.argv[2];
if (typeof socketPath !== "string" || socketPath.length === 0) {
  throw new Error("YINSHI_GIT_CREDENTIAL_SOCKET is required");
}
if (typeof prompt !== "string" || prompt.length === 0 || prompt.length > 1024) {
  throw new Error("Git askpass prompt is invalid");
}

const capabilityBuffer = Buffer.alloc(128);
const capabilityLength = fs.readSync(3, capabilityBuffer, 0, capabilityBuffer.length, 0);
if (capabilityLength <= 0) {
  throw new Error("Git credential capability is required");
}
const capability = capabilityBuffer.subarray(0, capabilityLength).toString("utf-8");
const socket = net.createConnection(socketPath);
let response = "";
socket.setEncoding("utf-8");
socket.setTimeout(5000, () => socket.destroy(new Error("Git credential broker timed out")));
socket.on("connect", () => socket.write(JSON.stringify({ capability, prompt })));
socket.on("data", (chunk) => {
  response += chunk;
  if (response.length > 4096) {
    socket.destroy(new Error("Git credential broker response is too large"));
  }
});
socket.on("end", () => {
  if (response.length === 0) {
    process.stderr.write("Git credential broker denied the request\n");
    process.exitCode = 1;
    return;
  }
  process.stdout.write(response);
});
socket.on("error", (error) => {
  process.stderr.write(`${error.message}\n`);
  process.exitCode = 1;
});

```